

Chapter 1 — OOP + Dunder Methods (60 Qs)

Python Interview Prep — 2026 Edition · Learn & Excel · File 04 Core Python Library

This is the OOP foundation chapter. Every Indian Python interview that goes past 20 minutes lands here — Razorpay panels for backend, Zerodha for trading systems, Flipkart for catalogue services, JP Morgan for risk pipelines, Walmart Labs for inventory, Service MNCs (TCS / Infosys / Wipro) for screening, and product cos like Postman / Chargebee / Cred for system design rounds. Writing classes is one thing; explaining *what Python is actually doing under the hood* — `__init__` vs `__new__`, MRO with diamond inheritance, descriptors as the engine behind `@property`, the `__eq__` / `__hash__` contract — is what separates "uses Python" from "knows Python."

Each Q below uses the canonical Step-by-Step Answer format: **Tags** (difficulty + Indian-MNC asked-at + topic), **The question** (interviewer's verbatim phrasing), **30-second answer** (what you say first if the panel is impatient), **Step-by-step thought process** (how to reason out loud), a **code** block (compiles on Python 3.12+), **Edge cases to mention** (where panels watch you trip), **What NOT to say**, **Common follow-up**, and a **Stand-out line** — the single sentence that signals you've actually read the data model.

Section index

- Section A — Class fundamentals (Q1–Q10)
- Section B — Inheritance + MRO (Q11–Q20)
- Section C — Descriptors + properties (Q21–Q30)
- Section D — Dunder method core (Q31–Q40)
- Section E — Dunder method advanced (Q41–Q50)
- Section F — Dataclasses + modern OOP (Q51–Q60)

Section A — Class Fundamentals

Q1 · Class attributes vs instance attributes

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Freshworks · Topic: Class attributes

The question (interviewer's verbatim phrasing):

What's the difference between a class attribute and an instance attribute in Python? Why does mutating a class-level list inside a method affect every instance?

30-second answer: Class attributes live on the class object itself and are shared across every instance; instance attributes live on the instance's `__dict__` and are per-object. Reading falls through instance → class; writing always lands on the instance unless you go through the class explicitly.

Step-by-step thought process: - Anchor it: `Order.tax_rate = 0.18` defines a class attribute — one copy, every instance reads the same value. `self.amount = 500` inside `__init__` defines an instance attribute — one copy per instance. - Read path: `instance.x` first looks in `instance.__dict__`, then walks the class hierarchy (`type(instance).__mro__`). Class attributes are the "default" you fall through to. - Write path is asymmetric: `instance.x = 5` always creates / overwrites `instance.__dict__["x"]` — it does NOT mutate the class attribute. To change the class attribute you write `Cls.x = 5`. - The classic bug: a **mutable** class attribute (a list or dict) gets shared. Append to it via `self.items.append(x)` and every instance sees it because you're mutating the class-level object.

Code:

```
class Order:
    tax_rate = 0.18          # class attribute
    items = []              # BUG: mutable class attribute

    def __init__(self, amount):
        self.amount = amount # instance attribute

a = Order(500)
b = Order(700)
a.items.append("tea")      # mutates the SHARED list
print(b.items)             # ['tea'] — leaked into b

a.tax_rate = 0.05          # creates instance attribute on a only
print(b.tax_rate, Order.tax_rate) # 0.18 0.18 — class untouched
```

Edge cases to mention: - Mutable class attributes (`[]` , `{}` , `set()`) are almost always a bug — same root cause as the mutable-default-argument trap. - `instance.x += 1` on an immutable class attribute (`int`) silently creates an instance attribute on the first write — students often expect it to mutate the class. - Class attributes are visible in `vars(Cls)` and `Cls.__dict__`, NOT in `vars(instance)`. Tab-completion in IDEs shows both.

What NOT to say: - "Class attributes are constants." — wrong. They're just attributes that happen to live on the class object; nothing about them is constant. - "`self.x = 5` modifies the class attribute." —

wrong, and this is the bug. The assignment always creates / overwrites an instance attribute; it never reaches the class.

Common follow-up: "Why is `Order.items.append(x)` a bug if `items` is a class attribute? And how would you fix the class so that each instance gets its own list?" — Class-level `items = []` is shared; switch to `self.items = []` inside `__init__`, or use a `field(default_factory=list)` if it's a dataclass.

Stand-out line: "Class attributes are the fallback layer; instance attributes are the override layer. The bug is always a write that you expected to land on the class but landed on the instance — or vice versa."

Q2 · `__init__` vs `__new__`

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, JP Morgan, Microsoft India · Topic: Constructor vs initialiser

The question (interviewer's verbatim phrasing):

Walk me through the difference between `__new__` and `__init__`. When would you ever override `__new__`?

30-second answer: `__new__` is the constructor — it allocates and returns the new instance.

`__init__` is the initialiser — it runs on the already-created instance to populate state. You override `__new__` only for immutable types, singletons, metaclass tricks, or subclassing built-ins like `tuple` / `str`. For 99% of everyday classes, `__init__` is the only one you touch.

Step-by-step thought process: - Order of operations: `Order(500)` calls `type.__call__(Order, 500)`, which calls `Order.__new__(Order, 500)`, which returns an instance, which then gets passed to `Order.__init__(instance, 500)`. - `__new__` is implicitly a static method (Python special-cases it). It takes `cls` as the first arg, not `self`. It must return the instance — if it returns the wrong type, `__init__` is silently skipped. - The only time `__init__` is too late: immutable types. You can't change a `str` or `tuple` after creation, so subclassing them needs `__new__` to bake in the value. - Singletons / interning / object pools are the other classic `__new__` use case — return an existing instance instead of allocating a new one.

Code:

```
class PaymentID(str):
    """Subclass str to attach a checksum; str is immutable, so __init__ is too late."""
    def __new__(cls, raw: str):
        # normalise + checksum BEFORE the str instance is sealed
        normalised = raw.strip().upper()
        instance = super().__new__(cls, normalised)
        instance.checksum = sum(ord(c) for c in normalised) % 97
        return instance

pid = PaymentID(" razorpay-001 ")
print(pid)                # 'RAZORPAY-001'
print(pid.checksum)        # int
print(type(pid).__mro__)   # (PaymentID, str, object)
```

Edge cases to mention: - Returning an object whose `type(obj)` isn't `cls` (or a subclass) skips `__init__` — silent surprise if you forget. - `__new__` runs even for `cls(*args)` calls from subclasses — it's not "the parent constructor", it's "the new-instance hook for every call". - Calling `super().__init__()` from a subclass is normal; calling `super().__new__()` is only needed when you actually override `__new__`. Most teams never write `__new__` in their lifetime.

What NOT to say: - "`__init__` returns the new object." — wrong. It returns `None`; returning anything else raises `TypeError`. - "`__new__` is just an older `__init__`." — wrong. They run at different stages and do different work. `__new__` allocates; `__init__` populates.

Common follow-up: "If you wanted to implement a singleton in Python, would you override `__new__` or use a different pattern, and why?" — `__new__` returning a cached instance is the textbook way; in practice a module-level instance or `functools.lru_cache` on a factory is usually cleaner.

Stand-out line: "`__new__` builds the instance, `__init__` fills it in — and `__init__` can't help you when the type is immutable, because by the time it runs, the bytes are already frozen."

Q3 · The `__init__` lifecycle in detail

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, JP Morgan, Walmart Labs · Topic: `init` lifecycle

The question (interviewer's verbatim phrasing):

What is the precise contract of `__init__`? What happens if it raises halfway through, and why does that matter for resource handling?

30-second answer: `__init__` runs once per instance, after `__new__` has returned the object. Its job is to populate state, not to return a value — returning anything other than `None` raises

TypeError. Exceptions inside `__init__` leave the partially-initialised object visible to `__del__` and to anything holding a reference, which is the source of half the resource-leak bugs in long-running services.

Step-by-step thought process: - Signature contract: `def __init__(self, ...)`. `self` is the already-allocated instance. Bind every required field here, even if just to `None`, so `__getattr__` doesn't get hit later. - Don't return anything. `return self` is a beginner reflex from other languages; in Python it raises. - Exception during `__init__`: the object IS already created (by `__new__`). If you opened a file, started a thread, or grabbed a DB connection before the exception, that resource leaks unless you use `try/except` or a context manager. - `__init_subclass__` and `__set_name__` run BEFORE the class body finishes — they're class-construction hooks, distinct from `__init__` which is the instance-construction hook.

Code:

```
class DBConnection:
    def __init__(self, url: str, timeout: int = 30):
        self.url = url
        self.timeout = timeout
        self._sock = None
        try:
            self._sock = self._open(url, timeout)
        except OSError:
            self._sock = None          # leave object usable; caller handles
            raise

    def _open(self, url, timeout):
        # placeholder for a real connect
        if not url.startswith("postgres://"):
            raise OSError("bad url")
        return object()               # pretend socket

    def __init__(self, *a, **kw):      # OVERRIDE: would override the above
        pass                          # last def wins – common mistake
```

Edge cases to mention: - Two `__init__` definitions in the same class body: the second one silently replaces the first. Linters catch it; review sometimes doesn't. - `__init__` returning a value (other than `None`) raises `TypeError: __init__() should return None`. - Inheritance bug: subclasses that override `__init__` and forget `super().__init__(...)` end up with parent state never being populated.

What NOT to say: - "`__init__` creates the object." — wrong. `__new__` creates it; `__init__` only populates it. - "If `__init__` raises, the object is destroyed." — wrong. The object exists, the reference may still be held, and `__del__` will run when refs drop.

Common follow-up: "If `__init__` opens a database connection and then fails on a validation a few lines later, where does that connection end up — and how do you avoid leaking it?" — Either use a context manager inside `__init__`, or catch in `__init__` and explicitly close before re-raising.

Stand-out line: "`__init__` populates an instance that already exists — so any exception inside it has to assume there's a half-built object the caller might still touch."

Q4 · `self` mechanics — what's actually being passed

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Freshworks · Topic: Bound methods + descriptors

The question (interviewer's verbatim phrasing):

What is `self` in Python — is it a keyword, a convention, or something else? Walk me through what happens when I call `instance.method(x)`.

30-second answer: `self` is just a convention name for the first parameter of an instance method; Python passes the instance in automatically when you access the method through an instance. The mechanism is the descriptor protocol — `instance.method` returns a bound method object that has `self` pre-filled, so you only pass the remaining arguments at the call site.

Step-by-step thought process: - The compiler doesn't know `self` is special — it's just the first parameter name. You could call it `this` or `me` and Python would work; you'd just confuse every reader. - The magic happens at attribute access: `instance.method` triggers `function.__get__(instance, type(instance))`, which returns a `bound method` object. The bound method has `instance` baked in as the first arg. - `Order.charge(o, 500)` and `o.charge(500)` are equivalent calls — the bound-method form just hides the first argument. - Functions defined in a class body aren't auto-instance-methods at the class level — `Order.charge` is a plain function. The descriptor protocol promotes it to bound-method ONLY when looked up through an instance.

Code:

```
class Order:
    def charge(self, amount):
        return f"charged {amount} for {id(self)}"

o = Order()
print(o.charge(500))           # bound: self filled in
print(Order.charge(o, 500))    # unbound-style: pass self explicitly

bound = o.charge
print(bound(500))              # works — `o` baked in
print(bound.__self__ is o)     # True
print(bound.__func__ is Order.charge) # True — the underlying function
```

Edge cases to mention: - Storing `instance.method` in a variable keeps `instance` alive — common cause of memory leaks in event-handler patterns. - Forgetting `self` in the signature: `def charge(amount)` inside a class causes `TypeError: charge() takes 1 positional argument but 2 were given` on `o.charge(500)`. - `@staticmethod` skips the descriptor magic — no `self` is passed. `@classmethod` swaps `self` for `cls` (the class object).

What NOT to say: - "`self` is a keyword like `this` in Java." — wrong. It's a regular parameter name; the descriptor protocol is what fills it in. - "The interpreter automatically inserts `self` everywhere." — wrong. The descriptor's `__get__` returns a bound method that has `self` pre-filled — that's the mechanism.

Common follow-up: "What's the difference between `o.method` stored as a variable and `Order.method`? Which one keeps `o` alive in memory, and why does that matter for event handlers?" — The bound method holds a strong reference to `o`; the unbound function does not. That's the classic event-handler memory leak.

Stand-out line: "`self` isn't a keyword — it's just a convention. The real magic is that the descriptor protocol pre-binds the instance to the method when you look it up through dot-notation."

Q5 · Class body execution — what runs when

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, JP Morgan · Topic: Class body execution model

The question (interviewer's verbatim phrasing):

When does the body of a class statement actually execute, and how many times? Walk me through what happens to assignments, `def` s, and side effects inside the body.

30-second answer: A class body executes top-to-bottom **once**, at class-definition time, in its own local namespace. Whatever ends up in that namespace becomes the class's `__dict__`. Function `def` s create function objects (not methods yet); plain expressions execute immediately; assignments create class attributes.

Step-by-step thought process: - Class definition is a real Python statement — `class Order:` runs the body like a function body, except the resulting namespace becomes the class dict. - Imports, expensive calls, side effects: all of them run **once at import time**, not per-instance. So `tax_rate = compute_rates()` inside the class body calls `compute_rates` exactly once, when the module is first imported. - `def` statements inside the body create function objects in the class namespace — they become bound methods only when accessed through an instance (descriptor magic). - The body has access to its own locals during execution — you can reference earlier definitions later in the same body. Once execution ends, that namespace is sealed into `__dict__`.

Code:

```

print("module load")

def compute_default_rate():
    print("computing rate")
    return 0.18

class Order:
    print("class body running")      # runs once at import
    tax_rate = compute_default_rate() # called once
    valid_currencies = {"INR", "USD"}

    def __init__(self, amount):
        self.amount = amount

    # can reference earlier class-body names
    default_currency = next(iter(valid_currencies))

print(Order.__dict__["tax_rate"])  # 0.18 – already computed
o1 = Order(100); o2 = Order(200)  # __init__ runs per instance; class body does not

```

Edge cases to mention: - A typo like `tax_rate == 0.18` in the class body executes harmlessly (it's an unused comparison expression) but doesn't define anything. - Nested class definitions: the inner `class Inner:` runs at outer-class-body-execution time, not lazily. - The class body is NOT inside any function scope — `def f(): class C: pass` defines `C` only when `f` is called.

What NOT to say: - "The class body runs every time an instance is created." — wrong. It runs once at import / class-creation time. - "`def` inside a class body creates a method." — almost right but imprecise. It creates a function object that the descriptor protocol promotes to a bound method only on instance access.

Common follow-up: "If I put an expensive `requests.get(...)` call as a class-level constant, when does it actually run — at import or per-instance?" — At import, exactly once. That's both useful (lazy-load defaults) and dangerous (slow imports, networked module load).

Stand-out line: "The class body runs exactly once, at import — every assignment inside it goes onto the class, every `def` becomes a function that the descriptor protocol will later promote to a bound method."

Q6 · `__init_subclass__` — the subclass-registration hook

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Microsoft India, JP Morgan · Topic: Subclass-creation hooks

The question (interviewer's verbatim phrasing):

What does `__init_subclass__` do, when does it run, and why is it considered the modern alternative to metaclasses for plugin registration?

30-second answer: `__init_subclass__` runs every time a class inherits from yours — once per subclass, at subclass-creation time. It's the modern way to register plugins, validate subclass shape, or enforce conventions without writing a metaclass.

Step-by-step thought process: - Mechanism: when Python creates a class `B(A)`, it calls `A.__init_subclass__(cls=B, **kwargs)` after the subclass body has finished. Always declared as a classmethod (Python special-cases it). - The killer use case is plugin registration: a base class collects every subclass into a registry dict, so the rest of the code can look up handlers by name. - It's a metaclass without the metaclass tax. Before Python 3.6, you'd write a metaclass; now you write a classmethod and most readers can follow it. - `**kwargs` flow: `class B(A, plugin_name="razorpay")` passes `plugin_name` straight to `__init_subclass__`. Clean way to annotate subclasses.

Code:

```
class PaymentHandler:
    registry: dict[str, type] = {}

    def __init_subclass__(cls, *, name: str, **kwargs):
        super().__init_subclass__(**kwargs)
        if name in PaymentHandler.registry:
            raise ValueError(f"handler {name!r} already registered")
        PaymentHandler.registry[name] = cls

    def handle(self, amount: int) -> str: ...

class RazorpayHandler(PaymentHandler, name="razorpay"):
    def handle(self, amount): return f"razorpay {amount}"

class StripeHandler(PaymentHandler, name="stripe"):
    def handle(self, amount): return f"stripe {amount}"

print(list(PaymentHandler.registry))    # ['razorpay', 'stripe']
print(PaymentHandler.registry["razorpay"]().handle(500))
```

Edge cases to mention: - Forgetting `super().__init_subclass__(**kwargs)` in a chain of inheritance silently drops kwargs further up the chain. - Runs at subclass-creation time (i.e. at import), not at instance creation — heavy work here slows down module load. - `__init_subclass__` is NOT called on the base class itself, only on classes that inherit from it.

What NOT to say: - "`__init_subclass__` runs on every instance." — wrong. It runs once per subclass at class-creation time. - "It's the same thing as a metaclass." — wrong. It covers most of what metaclasses used to be reached for, but it's a classmethod on the base, not a separate type.

Common follow-up: "How would you use `__init_subclass__` to enforce that every payment-handler subclass declares a `name` keyword argument at class definition?" — Make `name` a required kwarg on `__init_subclass__`; Python raises if a subclass forgets it at `class Subclass(Base, name=...):`.

Stand-out line: "`__init_subclass__` is the 'metaclass for normal humans' — every plugin-registry pattern that used to need a metaclass is now four readable lines."

Q7 · `__class_getitem__` and PEP 585 generic syntax

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft India, Postman, JP Morgan · Topic: Generic type syntax

The question (interviewer's verbatim phrasing):

Explain what `__class_getitem__` does. Why does `list[int]` work at runtime in Python 3.9+ but not in 3.8?

30-second answer: `__class_getitem__` is what makes `list[int]`, `dict[str, Order]`, and `MyClass[T]` work — it's the hook called when you subscript a class object instead of an instance. Builtins implement it for typing; your own generic classes get it for free if you inherit from `typing.Generic`.

Step-by-step thought process: - `Cls[X]` calls `Cls.__class_getitem__(X)` if defined. By default it doesn't exist on plain classes — you'd get `TypeError: ... is not subscriptable`. - PEP 585 (Python 3.9+) added `__class_getitem__` to built-in containers so `list[int]` works at runtime, matching the type-hint syntax. Before that, you needed `typing.List[int]` from the typing module. - The return value is typically a `types.GenericAlias` — a lazy carrier that holds the original class plus the type parameters. It's used by typing tools; it doesn't enforce anything at runtime. - For your own classes, inherit from `typing.Generic[T]` and you get `__class_getitem__` automatically. Don't roll your own unless you have a very specific case.

Code:

```

from typing import Generic, TypeVar

T = TypeVar("T")

class Box(Generic[T]):
    def __init__(self, value: T):
        self.value = value

box_int = Box[int](42)           # the [int] is a type hint; runtime ignores it
print(box_int.value)             # 42
print(Box[int])                  # Box[int] — a GenericAlias

# PEP 585 — builtins are subscriptable too
print(list[int])                 # list[int]
print(dict[str, list[int]])      # dict[str, list[int]]

```

Edge cases to mention: - `list[int]` does NOT type-check at runtime — `list[int]("not an int")` runs fine. Type checkers (mypy, pyright) read these annotations; the interpreter ignores them. - Pre-3.9 code uses `typing.List[int]`. Mixing the two in the same codebase passes mypy but reads inconsistently — pick one and stick to it. - PEP 695 (Python 3.12) added the cleaner `class Box[T]:` syntax — same mechanism, no `Generic` import needed.

What NOT to say: - "`list[int]` enforces that the list contains only ints." — wrong. It's a typing carrier; the interpreter ignores it at runtime. - "PEP 585 changed the typing module." — wrong. PEP 585 added `__class_getitem__` to the built-in containers themselves, so you no longer need `typing.List[int]`.

Common follow-up: "What's the actual object you get back from `list[int]` — is it a class, an instance, or something else? And what does it carry?" — A `types.GenericAlias`. It carries `__origin__` (the class) and `__args__` (the type params); typing tools read these.

Stand-out line: "`__class_getitem__` is purely a runtime carrier for type information — the brackets after a class name look like construction but they're just metadata that mypy reads later."

Q8 · The `__class__` attribute — what it points to

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, Postman · Topic: **class** semantics

The question (interviewer's verbatim phrasing):

What does `instance.__class__` point to, and how does that differ from `type(instance)`? Why does `isinstance` sometimes get fooled but `type()` does not?

30-second answer: `instance.__class__` is the type of the instance — the same as `type(instance)`, but lookup is slightly different. Reading it gives you the class; writing it (rarely) actually re-types the instance, which is a powerful and dangerous capability used in patterns like state machines.

Step-by-step thought process: - For most code: `instance.__class__ is type(instance)` is `True`. Both return the class object. - The subtle difference: `type(instance)` always returns the actual type, ignoring any `__class__` override. `instance.__class__` reads the attribute, which can be overridden — `mock.MagicMock` uses this to lie about its class. - Reassigning `__class__`: `instance.__class__ = OtherClass` literally changes the runtime type of the instance, as long as the layouts (slots / size) are compatible. Some state-machine libraries use this. - Inside a method, `self.__class__` is sometimes used instead of the class name to pick up subclass identity — alternative to using `type(self)` or `cls` from a classmethod.

Code:

```
class Order: pass
class Refund: pass

o = Order()
print(o.__class__)           # <class '__main__.Order'>
print(type(o) is Order)     # True

# Re-typing — works when both classes are plain, no __slots__ collision
o.__class__ = Refund
print(type(o))              # <class '__main__.Refund'> — actually refund now

# Lying without re-typing: override __class__ as a property
class Lier:
    @property
    def __class__(self):
        return int           # type(instance) still shows Lier
print(isinstance(Lier(), int)) # True — isinstance trusts __class__
print(type(Lier()))          # <class '__main__.Lier'>
```

Edge cases to mention: - `isinstance` and `issubclass` use `__class__` and can be fooled by overrides — `type()` cannot. - Re-assigning `__class__` between classes with different `__slots__` or different layouts raises `TypeError`. - `cls = self.__class__` inside an instance method is a common but fragile pattern — `type(self)` is clearer; in a classmethod, use `cls` directly.

What NOT to say: - "`__class__` and `type()` always return the same thing." — wrong. They diverge when `__class__` is overridden by a property or reassigned. - "Reassigning `__class__` is an error." — wrong. Python allows it when the layouts (slots, size) are compatible — used by state-machine libraries.

Common follow-up: "If a mock library overrides `__class__` to return `int`, will `isinstance(mock, int)` be True? Will `type(mock) is int` be True?" — `isinstance` yes (it trusts `__class__`); `type() is` no (it reads the real type).

Stand-out line: "`type(x)` asks the runtime what `x` actually is; `x.__class__` asks `x` what it claims to be — usually the same, occasionally a load-bearing difference."

Q9 · `@classmethod` vs `@staticmethod` vs instance method

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, Freshworks · Topic: Method types

The question (interviewer's verbatim phrasing):

Walk me through the difference between an instance method, a `@classmethod`, and a `@staticmethod`. Which one would you reach for to write an alternate constructor, and why?

30-second answer: An instance method takes `self` and operates on a single instance. A `@classmethod` takes `cls` and operates on the class — usually for alternate constructors or class-level state. A `@staticmethod` takes neither — it's a plain function that lives in the class namespace for organisation, with no implicit first argument.

Step-by-step thought process: - Instance method: `def charge(self, amount)`. Called as `o.charge(500)`. Used for anything that needs per-instance state. - Classmethod: `def from_json(cls, blob)`. Called as `Order.from_json(...)` or `o.from_json(...)` — either way `cls` is the class. The killer use case is alternate constructors that should also work for subclasses. - Staticmethod: `def is_valid_amount(amount)`. Called as `Order.is_valid_amount(500)` or `o.is_valid_amount(500)`. No `self`, no `cls` — it's a function that happens to live on the class. Often used for grouping helpers near the code that uses them. - The classmethod-for-constructors pattern matters because it's polymorphic — `RazorpayOrder.from_json(...)` automatically returns a `RazorpayOrder`, not an `Order`, because `cls` is the actual subclass.

Code:

```

import json

class Order:
    def __init__(self, amount: int, currency: str = "INR"):
        self.amount, self.currency = amount, currency

    @classmethod
    def from_json(cls, blob: str):
        data = json.loads(blob)
        return cls(data["amount"], data.get("currency", "INR"))

    @staticmethod
    def is_valid_amount(amount: int) -> bool:
        return amount > 0 and amount < 10_000_000

    def cost_with_tax(self):
        return self.amount * 1.18

class RazorpayOrder(Order):
    """from_json on this subclass returns a RazorpayOrder, not an Order."""

ro = RazorpayOrder.from_json('{"amount": 500}')
print(type(ro)) # <class '__main__.RazorpayOrder'>
print(Order.is_valid_amount(-5)) # False

```

Edge cases to mention: - Calling a classmethod on an instance still gets you the class as `cls` — `o.from_json(...)` and `Order.from_json(...)` are equivalent. Some readers find that confusing. - A staticmethod that uses `Order.SOMETHING` hard-codes the class — pass `cls` (and make it a classmethod) if you want subclass polymorphism. - Forgetting `@classmethod` and writing `def from_json(cls, ...)` makes Python pass the first positional argument as `cls` — silent failure when called from a real instance.

What NOT to say: - "A `@staticmethod` is just a method without `self` ." — wrong-ish. It's a method with no implicit first argument at all — that's the key, not just "missing `self` ". - "`@classmethod` is for class-level state mutation." — narrow. It's mainly for alternate constructors and operations that should respect subclassing.

Common follow-up: "If I subclass `Order` as `RazorpayOrder` and call `RazorpayOrder.from_json(...)` , what's the return type — and why?" — `RazorpayOrder` , because `cls` is the actual subclass when the classmethod is called via the subclass.

Stand-out line: "Use `@classmethod` for anything that should respect subclassing — alternate constructors, factory methods, registries — and `@staticmethod` only as a namespacing convenience."

Q10 · `__dict__` , `__slots__` , and where attributes actually live

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Walmart Labs, Postman · Topic: Instance layout (**dict** vs **slots**)

The question (interviewer's verbatim phrasing):

How does Python store instance attributes by default, and what changes when you declare `__slots__` ? Why would you ever take the memory win over the flexibility?

30-second answer: Every instance has a `__dict__` by default — a Python dict mapping attribute names to values, which is what makes attribute assignment dynamic. Defining `__slots__` swaps that dict for a fixed array of named slots — smaller memory, faster lookup, but no dynamic attributes and no `__dict__` .

Step-by-step thought process: - Default layout: `instance.__dict__` is a real dict you can poke at. `o.__dict__["amount"] = 500` is the same as `o.amount = 500` . - `__slots__ = ("amount", "currency")` tells Python to allocate exactly two slots in the instance layout. No dict, no surprise attributes — assigning `o.other = 1` raises `AttributeError` . - Memory win: a slotted instance is roughly 5-10x smaller than a dict-based instance. Matters when you have millions of small objects (events, market ticks, parsed records). - Subclassing rules: a subclass that doesn't define `__slots__` re-gains `__dict__` . To keep the slot win all the way down, every class in the chain must declare `__slots__` .

Code:

```

class Loose:
    def __init__(self, amount): self.amount = amount

class Tight:
    __slots__ = ("amount", "currency")
    def __init__(self, amount, currency="INR"):
        self.amount, self.currency = amount, currency

a = Loose(500); b = Tight(500)
print(a.__dict__)          # {'amount': 500}
print(hasattr(b, "__dict__")) # False

a.extra = "fine"           # ok – dict allows it
try:
    b.extra = "blocked"
except AttributeError as e:
    print(e)                # 'Tight' object has no attribute 'extra'

import sys
print(sys.getsizeof(a) + sys.getsizeof(a.__dict__)) # bigger
print(sys.getsizeof(b))                             # smaller

```

Edge cases to mention: - A subclass that omits `__slots__` re-gains a `__dict__` — the memory saving is lost from that subclass down. - `__slots__` + a class attribute with the same name silently masks the slot. Don't write `__slots__ = ("x",); x = 0`. - Slotted classes can't add dynamic attributes — that breaks libraries that monkey-patch (mock, dataclass-style decorators that add fields). Check before retrofitting.

What NOT to say: - "`__slots__` makes attributes faster to access on every class." — partly true and overstated. The real win is memory; access speed is a smaller win. - "All classes should use `__slots__`." — wrong. They break monkey-patching and many ORM / mock libraries.

Common follow-up: "What happens to memory if you have a base class with `__slots__` but a subclass that doesn't declare any?" — The subclass re-gains `__dict__`, and the memory saving stops at that level.

Stand-out line: "`__dict__` is what makes Python objects dynamic; `__slots__` is what makes them small. You give up flexibility for memory, and the trade is only worth it when you have millions of instances."

Section B — Inheritance + MRO

Q11 · Single vs multiple inheritance — when each makes sense

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Cred, Microsoft India · Topic: Inheritance design

The question (interviewer's verbatim phrasing):

When should you reach for multiple inheritance versus composition? What's the warning sign that a hierarchy is getting out of hand?

30-second answer: Single inheritance is the default — one parent, predictable behaviour. Multiple inheritance is for mixing in orthogonal capabilities (logging, serialisation, validation) via small purposeful mixin classes. The moment you find yourself solving the diamond problem in your own design, you've taken multiple inheritance too far — composition is usually the better answer.

Step-by-step thought process: - Single inheritance: `class RazorpayOrder(Order)`. One linear chain. Easy to reason about; method resolution is just "walk up the parents". - Multiple inheritance is fine for **mixins** — small classes that add one capability with no `__init__` of their own. `class AuditedOrder(Order, LoggerMixin, SerialisableMixin)`. - The risk grows fast when two parents have overlapping methods or state. That's where MRO (next Q) earns its keep — and where readers stop being able to predict behaviour. - Composition (holding a reference to a helper object) is usually cleaner than mixing in via inheritance. Reach for inheritance only when you really want "is-a", not "has-a".

Code:

```
class LoggerMixin:
    def log(self, msg): print(f"[{type(self).__name__}] {msg}")

class SerialisableMixin:
    def to_dict(self): return {k: v for k, v in self.__dict__.items()
                              if not k.startswith("__")}

class Order:
    def __init__(self, amount): self.amount = amount

class AuditedOrder(Order, LoggerMixin, SerialisableMixin):
    def charge(self):
        self.log(f"charging {self.amount}")

o = AuditedOrder(500)
o.charge()                # [AuditedOrder] charging 500
print(o.to_dict())        # {'amount': 500}
```

Edge cases to mention: - Mixins with their own `__init__` need careful cooperative `super().__init__(...)` calls — easy to break. - "Diamond" inheritance (`D(B, C)` where both `B` and `C` inherit from `A`) is solved by MRO but still confusing to reviewers. - Multiple inheritance with state is almost always a smell — use composition.

What NOT to say: - "Multiple inheritance is forbidden in Python." — wrong. It's allowed and useful for mixins. - "Composition is always better than inheritance." — too dogmatic. Inheritance is the right tool for genuine "is-a" relationships.

Common follow-up: "Give me a concrete example where a mixin is the right pattern and a concrete example where you'd refuse to use one." — Right: `LoggerMixin` adding a `log` method to handlers. Wrong: a `DatabaseMixin` that holds a connection — that's state masquerading as behaviour.

Stand-out line: "Multiple inheritance is fine for behaviour without state; the moment two parents share state or a constructor, prefer composition."

Q12 · MRO and the C3 linearization algorithm

Tags: Difficulty: Hard · Asked at: Razorpay, Microsoft India, JP Morgan, Postman · Topic: MRO + C3 linearization

The question (interviewer's verbatim phrasing):

Explain Method Resolution Order in Python. What is C3 linearization, and why does Python prefer it over a depth-first walk?

30-second answer: MRO (Method Resolution Order) is the ordered list of classes Python walks when looking up an attribute. Python uses the C3 linearization algorithm — guarantees a consistent order that preserves parent order and respects each class's own order. Read it with `Cls.__mro__` or `Cls.mro()`.

Step-by-step thought process: - C3 in one breath: "leftmost first, parents before grandparents, but never repeat or contradict any class's local order". - For single inheritance, MRO is just the chain — `[Cls, Parent, Grandparent, object]`. Boring. - For multiple inheritance, C3 merges the parent MROs left-to-right, dropping duplicates, ensuring no class appears before its descendant. - Practical use: every attribute lookup walks MRO. `super()` walks MRO from the current class's position. Surprises in MRO produce surprising lookups.

Code:

```

class A:
    def who(self): return "A"

class B(A):
    def who(self): return "B"

class C(A):
    def who(self): return "C"

class D(B, C):
    pass

print(D.__mro__)
# (D, B, C, A, object) — C3 keeps left-first, child-before-parent

d = D()
print(d.who())                # 'B' — first match in MRO

```

Edge cases to mention: - Inconsistent MRO raises `TypeError: Cannot create a consistent method resolution order` at class-creation time. Usually means your bases contradict each other on order. - The MRO is computed once at class-creation, cached, and used for every lookup. - C3 was introduced in Python 2.3 (long ago) but the term still trips juniors — practise reading 3-4-class MROs out loud.

What NOT to say: - "MRO is depth-first left-to-right." — wrong. It used to be in Python 2.2; C3 has been the rule since 2.3. - "MRO is computed at every attribute access." — wrong. It's computed once at class creation and cached.

Common follow-up: "What happens if Python can't compute a consistent MRO — when does the error show up, and what does it mean?" — At class-creation time, with `TypeError: Cannot create a consistent method resolution order`. It means your bases contradict each other on order.

Stand-out line: "MRO is a deterministic merge — given any class hierarchy, C3 picks exactly one ordering that respects every class's local order, or refuses to build the class at all."

Q13 · `super()` — what it actually does

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Microsoft India, Cred · Topic: `super()` semantics

The question (interviewer's verbatim phrasing):

What does `super()` actually return, and why is it wrong to think of it as 'the parent class'?

30-second answer: `super()` returns a proxy object that delegates attribute lookups to the next class in the MRO, starting after the current one. It's not "the parent class" — it's "the next class in the MRO from where I am now", which matters in cooperative multiple inheritance.

Step-by-step thought process: - `super()` in Python 3 is magic: the compiler injects the current class and the first argument (`self`) into `super().__init__` . You don't pass them explicitly. - The delegation target depends on the MRO of the runtime type — not the static class. So `super()` in `B` called on a `D` instance jumps to `C` , not `A` . - This is the foundation of cooperative inheritance: every class in a diamond calls `super().__init__(...)` and the calls thread through every class in MRO order, each exactly once. - The Python 2 form (`super(B, self).method()`) still works and is sometimes needed inside `__init_subclass__` or in code that runs without a frame (unlikely, but exists).

Code:

```
class A:
    def __init__(self): print("A"); super().__init__()

class B(A):
    def __init__(self): print("B"); super().__init__()

class C(A):
    def __init__(self): print("C"); super().__init__()

class D(B, C):
    def __init__(self): print("D"); super().__init__()

D()
# D -> B -> C -> A -> object — each runs once, in MRO order
print(D.__mro__)
```

Edge cases to mention: - `super().__init__()` with the wrong signature (some parents take args, some don't) is the classic cooperative-inheritance pain point. Either every class accepts `*args, **kwargs` , or every class agrees on a signature. - `super()` without arguments only works inside methods of a class body — outside, you need the two-arg form. - `super()` returns a proxy, not the parent class itself — `super().__class__` is `super` , not the parent.

What NOT to say: - "`super()` returns the parent class." — wrong. It returns a proxy that walks MRO from the current position. - "`super()` and `Parent.method(self, ...)` are equivalent." — wrong in multiple-inheritance hierarchies; they pick different next-in-line classes.

Common follow-up: "If I'm in `B.__init__` of a `class D(B, C):` hierarchy, where does `super().__init__()` go — to `A` (B's parent) or to `C` ? Walk me through MRO." — To `C` , because MRO from `D` is `D -> B -> C -> A -> object` , and `super()` from inside `B` jumps to the next class after `B` , which is `C` .

Stand-out line: "`super()` doesn't mean 'my parent' — it means 'the next class in the MRO from where this code is'. The runtime walks MRO; the call site doesn't decide."

Q14 · The diamond problem — how Python resolves it

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Microsoft India · Topic: Diamond inheritance

The question (interviewer's verbatim phrasing):

Explain the diamond problem and how Python resolves it. In a `class D(B, C)` hierarchy where both `B` and `C` inherit from `A`, how many times does `A.__init__` run when `D()` is called, and why?

30-second answer: The diamond problem is when class `D` inherits from `B` and `C`, both of which inherit from `A`. The question is "in what order does Python visit `A` when looking up an attribute via `D`?". Python uses C3 linearization to produce one canonical order — `D -> B -> C -> A -> object` — and `super()` chains follow that.

Step-by-step thought process: - The "problem" exists because some languages (Java) ban multiple inheritance to avoid it; Python embraces it with a deterministic resolution. - C3 ensures `A` is visited **once**, after both `B` and `C` — so `A.__init__` runs once even if both `B` and `C` call `super().__init__()`. - Cooperative super: every class in the diamond uses `super().__init__(...)`. The MRO threads them in C3 order. If one class breaks the chain (e.g. skips `super().__init__()`), the next-up class never runs. - The pattern works only if every constructor in the chain accepts (and ideally forwards) `**kwargs` — otherwise you mismatch signatures and get `TypeError`.

Code:

```

class A:
    def __init__(self, **kw): print("A"); super().__init__()

class B(A):
    def __init__(self, **kw): print("B"); super().__init__(**kw)

class C(A):
    def __init__(self, **kw): print("C"); super().__init__(**kw)

class D(B, C):
    def __init__(self, **kw): print("D"); super().__init__(**kw)

D()
# Output: D, B, C, A — A runs ONCE despite two paths

# Break the chain: C forgets super().__init__()
class CBroken(A):
    def __init__(self, **kw): print("CBroken")

class DBroken(B, CBroken):
    def __init__(self, **kw): print("D"); super().__init__(**kw)

DBroken() # D, B, CBroken — A is silently skipped

```

Edge cases to mention: - Any class in the chain that omits `super().__init__()` truncates the chain — everything above never runs. - Signature drift kills cooperative super — if one class adds a required positional arg, every class below it also needs to handle it. - `object.__init__()` accepts no positional args (since Python 3.3) — the last call must pass no args, or you get `TypeError`.

What NOT to say: - "`A.__init__` runs twice because there are two paths." — wrong. C3 ensures it runs exactly once. - "The diamond problem doesn't exist in Python." — wrong. It exists; Python just resolves it deterministically.

Common follow-up: "What's the single most common way developers break cooperative `super()` chaining — and how does it surface?" — Forgetting `super().__init__()` in one class of the chain; everything above it silently doesn't run, and bugs surface as missing attributes much later.

Stand-out line: "The diamond isn't a problem in Python — it's a contract. C3 picks the order; every class cooperates via `super()` and `**kwargs`; `A` runs exactly once."

Q15 · Mixin patterns — small, stateless, behaviour-only

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Mixin pattern

The question (interviewer's verbatim phrasing):

What makes a class a mixin versus a regular base class? What rules of thumb do you follow when designing one?

30-second answer: A mixin is a class designed to be inherited from but never instantiated alone — it adds one capability (logging, serialisation, comparison ordering) to whatever it's mixed into. The rule of thumb: no state, no `__init__`, name ends in `Mixin`, and put it to the right of the main base class in inheritance lists.

Step-by-step thought process: - Why mixins beat utility functions: methods get `self`, so they can read instance state ("serialise me as a dict") without you passing the instance in. - Convention: name them `XxxMixin` so readers know they're not standalone. Don't give them constructors — they should be no-state contributors. - Place them on the right: `class Order(BaseModel, LoggerMixin)` reads as "an Order is fundamentally a BaseModel, also logged". MRO will visit `Order, BaseModel, LoggerMixin, object`. - `functools.total_ordering` is essentially a programmatic mixin — you define `__eq__` and one ordering method, it fills in the others.

Code:

```
import json

class JSONSerializableMixin:
    def to_json(self) -> str:
        return json.dumps(self.__dict__)

class TimestampedMixin:
    """Adds a created_at field - needs __init_subclass__ rather than __init__."""
    def __init_subclass__(cls, **kw):
        super().__init_subclass__(**kw)
        cls.__has_timestamp__ = True

class Order(JSONSerializableMixin, TimestampedMixin):
    def __init__(self, amount, currency="INR"):
        self.amount, self.currency = amount, currency

o = Order(500)
print(o.to_json())           # {"amount": 500, "currency": "INR"}
print(Order.__has_timestamp__) # True
```

Edge cases to mention: - Mixins with state defeat the point — once you need `__init__`, you've created another diamond risk. - Mixin order matters when two mixins define the same method — leftmost wins via MRO. - Naming the class `XxxMixin` doesn't enforce no-instantiation — convention only. ABCs can enforce it if needed.

What NOT to say: - "A mixin is just a class with `Mixin` in the name." — narrow. The convention is the suffix, but the discipline is no-state, no- `__init__`. - "Mixins go on the left of the inheritance list." — wrong. They go on the right; the main base class is on the left.

Common follow-up: "If a mixin needs to add a `created_at` field to every subclass, where should that field's value come from — `__init__` or `__init_subclass__`?" — Neither in the mixin's `__init__` (that's state); use `__init_subclass__` to attach a class-level marker, or have the host class set the field in its own `__init__`.

Stand-out line: "A mixin's job is one verb — log, serialise, compare. The moment it grows a constructor or holds state, it's no longer a mixin, it's a parent class wearing a costume."

Q16 · Abstract base classes with `abc.ABC`

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Postman, Microsoft India · Topic: Abstract base classes

The question (interviewer's verbatim phrasing):

When should you use `abc.ABC` over a plain base class? How is it different from `typing.Protocol`?

30-second answer: `abc.ABC` is the modern way to declare a class can't be instantiated directly and force subclasses to implement specific methods. Use `@abstractmethod` on the methods that subclasses must override; if a subclass forgets, instantiating it raises `TypeError`.

Step-by-step thought process: - Two pieces: inherit from `abc.ABC` (or use `metaclass=ABCMeta`); decorate required methods with `@abstractmethod`. Instantiation checks both. - The check happens at instantiation, not at class creation — so `class HalfBuilt(Base): pass` is fine until you try `HalfBuilt()`. - Useful for plugin systems where you want to fail loudly if a plugin doesn't implement an expected method, instead of letting `AttributeError` show up at runtime. - `Protocol` (typing module) is the structural-typing alternative — no inheritance required, duck-typed by mypy. Use ABC when you want runtime enforcement; use Protocol for static type checking.

Code:


```

from abc import ABC, abstractmethod

class PaymentHandler(ABC):
    @abstractmethod
    def handle(self, amount: int) -> str: ...

    @abstractmethod
    def name(self) -> str: ...

    def log(self, msg):                # concrete method – shared
        print(f"[{self.name()}] {msg}")

class Razorpay(PaymentHandler):
    def handle(self, amount): return f"razorpay {amount}"
    def name(self): return "razorpay"

try:
    PaymentHandler()                  # TypeError – can't instantiate abstract
except TypeError as e:
    print(e)

r = Razorpay(); r.log("ok")           # works – both abstracts implemented

```

Edge cases to mention: - Forgetting to inherit from `ABC` while using `@abstractmethod`: the decorator is silently ignored, and the abstract check never fires. - `@abstractmethod` must be the innermost decorator if you stack — `@classmethod @abstractmethod` order matters. - A subclass can satisfy `@abstractmethod` by re-declaring the method, even if the implementation is just `...` — Python only checks that the name is overridden.

What NOT to say: - "`abc.ABC` is just a documentation marker." — wrong. The interpreter enforces it at instantiation. - "`Protocol` and `ABC` are interchangeable." — wrong. `ABC` is runtime-enforced inheritance; `Protocol` is structural type-check-time.

Common follow-up: "If a subclass implements `@abstractmethod foo` with a body that just says `pass`, does Python complain?" — No. The check is name-overridden, not behaviour-meaningful. Callers will surface the bug later.

Stand-out line: "`abc.ABC` is runtime enforcement; `typing.Protocol` is static enforcement — pick based on whether the contract should fail at import time or at type-check time."

Q17 · Inheriting from built-ins (`list` , `dict` , `int`)

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Microsoft India, Cred · Topic: Subclassing built-ins

The question (interviewer's verbatim phrasing):

What happens when you subclass `dict` and override `__setitem__` ? Why does `MyDict.update(...)` sometimes bypass your override?

30-second answer: You can subclass `list`, `dict`, `int`, `str`, etc. — but the implementations don't always call your overridden methods internally. `dict.__setitem__` overrides are bypassed by `dict.update`; `list.__getitem__` overrides are bypassed by some slice operations. For dict-like and list-like behaviour, prefer `collections.UserDict` / `UserList` which delegate cleanly.

Step-by-step thought process: - The technical fact: subclassing built-ins works, but the C-implemented internals sometimes skip Python-level overrides for speed. CPython implementation detail; been true forever. - Concrete failure: override `MyDict.__setitem__` to log writes, then call `MyDict.update({"x": 1})`. Your override doesn't fire — `update` writes directly via the C code path. - The fix: `from collections import UserDict, UserList, UserString`. They store data in `self.data` and route every operation through the dunder, so overrides actually run. - For `int` / `str` / `tuple`, subclassing is fine because they're immutable — there's no "set" path to worry about.

Code:

```
from collections import UserDict

class LoggingDict(dict):
    def __setitem__(self, k, v):
        print(f"set {k}={v}")
        super().__setitem__(k, v)

d = LoggingDict()
d["x"] = 1                # set x=1
d.update({"y": 2})        # SILENT — bypassed your override

class LoggingUserDict(UserDict):
    def __setitem__(self, k, v):
        print(f"userdict set {k}={v}")
        super().__setitem__(k, v)

u = LoggingUserDict()
u["x"] = 1                # userdict set x=1
u.update({"y": 2})        # userdict set y=2 — works
```

Edge cases to mention: - `defaultdict` won't honour a `__missing__` override added in a `dict` subclass — same C-path skip. - Overriding `__init__` on a `list` subclass and forgetting to call `super().__init__(iterable)` leaves the list empty. - Pickling subclasses of built-ins works but `copy.copy(LoggingDict({}))` returns a `LoggingDict`, not a plain `dict` — surprise for some serialisers.

What NOT to say: - "Overriding `__setitem__` catches every write." — wrong. The C-implemented methods on `dict` (and `list`) sometimes skip Python-level overrides. - "You can't subclass built-ins." — wrong. You can; the gotcha is the bypass behaviour.

Common follow-up: "If I need every mutation on a dict-like object to fire my logging code, what should I subclass?" — `collections.UserDict`. It stores data in `self.data` and routes every operation through the Python-level dunders.

Stand-out line: "Subclassing `dict` doesn't make every dict operation route through your code — that's a C-level shortcut. Use `UserDict` when you actually need every mutation observed."

Q18 · `isinstance` vs `issubclass` vs `type()`

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman · Topic: Type checks

The question (interviewer's verbatim phrasing):

Walk me through `isinstance`, `issubclass`, and `type()`. When would you ever pick `type()` over `isinstance`?

30-second answer: `isinstance(obj, Cls)` returns True if `obj` is an instance of `Cls` or any subclass. `issubclass(Sub, Cls)` returns True if `Sub` is `Cls` or a subclass. `type(obj) is Cls` is the strict identity check — it returns True only if `obj`'s direct type is exactly `Cls`, ignoring subclassing.

Step-by-step thought process: - 99% of code wants `isinstance` — it respects polymorphism, which is the whole point of inheritance. - `type(x) is SomeClass` is for the rare case where you specifically need to reject subclasses — e.g. validating that something is exactly a `dict`, not a `defaultdict`. - `issubclass` operates on classes, not instances — useful in metaclass code, registries, and `__init_subclass__` checks. - Both `isinstance` and `issubclass` accept a tuple of types: `isinstance(x, (int, float))` — cheaper than two checks.

Code:

```

class Order: pass
class RazorpayOrder(Order): pass

o = RazorpayOrder()
print(isinstance(o, Order))      # True – subclass counts
print(isinstance(o, RazorpayOrder)) # True
print(type(o) is Order)         # False – exact type only
print(type(o) is RazorpayOrder)  # True

print(issubclass(RazorpayOrder, Order)) # True
print(issubclass(Order, RazorpayOrder)) # False

# Tuple form
print(isinstance(o, (Order, dict))) # True – Order matches

# isinstance respects ABC virtual subclasses
from collections.abc import Mapping
print(isinstance({}, Mapping))      # True – dict is a virtual subclass of Mapping

```

Edge cases to mention: - `type(x) == SomeClass` works but `is` is faster and idiomatic. Use `is` - `isinstance(x, list)` returns True for any list subclass — if you need "exactly a list", use `type(x) is list` - `isinstance` checks `__class__`, so a mock that lies about `__class__` can fool `isinstance` but not `type`.

What NOT to say: - "`isinstance` and `type() ==` are equivalent." — wrong. `isinstance` respects subclassing; `type() is` does not. - "You should always use `type()` because it's faster." — wrong. Polymorphism is usually what you want.

Common follow-up: "If I need to confirm something is exactly a `dict` and not a `defaultdict`, which check do I use?" — `type(x) is dict`. `isinstance(x, dict)` would also return True for `defaultdict`.

Stand-out line: "`isinstance` is polymorphism's bouncer; `type() is` is identity-strict. Default to `isinstance` and only reach for `type()` when subclassing is the bug you're guarding against."

Q19 · Method overriding and `super()` chaining

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Cred, Postman · Topic: Method overriding

The question (interviewer's verbatim phrasing):

What's the difference between overriding a method to extend the parent's behaviour and overriding to replace it? How do you signal each pattern in the code?

30-second answer: Method overriding is just defining a method with the same name in a subclass — Python's MRO ensures the subclass version is called. `super().method()` lets the subclass extend (not replace) the parent's behaviour, which is the cleanest way to layer logic.

Step-by-step thought process: - Override semantics: define `def charge(self, ...)` in the subclass, and instance lookup finds it first via MRO. - Two override styles: total replacement (don't call `super`) or extension (call `super` to do the base work, then add your own). Extension is usually what you want unless you have a strong reason to replace. - Order matters: `super().method()` before your extension means "do parent first, then me"; after means "me first, then parent". For things like input validation, validate first then call `super`. - `super()` without arguments is the Python 3 form — the compiler injects the current class and first argument.

Code:

```
class Order:
    def charge(self, amount: int) -> str:
        if amount <= 0: raise ValueError("amount must be positive")
        return f"charged {amount}"

class AuditedOrder(Order):
    def charge(self, amount: int) -> str:
        result = super().charge(amount)           # extend parent
        print(f"audit: {result}")
        return result

class CappedOrder(Order):
    MAX = 100_000
    def charge(self, amount: int) -> str:
        if amount > self.MAX:
            raise ValueError(f"amount > {self.MAX}")
        return super().charge(amount)             # validate first, then super

AuditedOrder().charge(500)
CappedOrder().charge(50_000)
```

Edge cases to mention: - Calling the parent's method via `Order.charge(self, ...)` works but bypasses MRO — `super()` is the right idiom. - Subclasses that change the method signature (different args, different return type) violate Liskov substitution — type checkers and reviewers will flag it. - Overriding `__init__` and forgetting `super().__init__(...)` is the single most common inheritance bug — parent state never initialised.

What NOT to say: - "Always call `super()` in an override." — wrong. Sometimes you genuinely want to replace. - "`Parent.method(self, ...)` is the same as `super().method(...)`." — wrong in multiple inheritance; they pick different next-in-MRO classes.

Common follow-up: "Should validation in an override go before or after the `super()` call?" — Before. Validate the input first, then delegate to the parent so a bad input doesn't waste the parent's work or land it in a half-validated state.

Stand-out line: "Override to specialise, `super()` to layer. The bug is usually the missing `super()` call, not the override itself."

Q20 · Method overloading — Python's stance

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft India, JP Morgan, Postman · Topic: Method overloading

The question (interviewer's verbatim phrasing):

Does Python support method overloading? If not, what's the idiomatic alternative when you want different behaviour based on argument type?

30-second answer: Python doesn't have method overloading in the C++/Java sense — you can't define two methods with the same name but different signatures. Instead, you use default arguments, `*args` / `**kwargs`, type dispatch, or `functools.singledispatch` to fake polymorphism on argument types.

Step-by-step thought process: - Two `def` statements with the same name: second one silently overrides the first. No "best match" dispatch like Java. - The Python way: one method with optional / variadic args, and use `isinstance` inside the body to branch — or use `functools.singledispatch` to register handlers by type. - `@singledispatch` is functional dispatch on the first argument's type — handy for serialisers, formatters, anything that branches on type. - `@singledispatchmethod` is the method version — same idea but for instance methods, dispatching on the second arg (because `self` is first).

Code:

```

from functools import singledispatchmethod

class Order:
    @singledispatchmethod
    def add(self, item):
        raise TypeError(f"can't add {type(item).__name__}")

    @add.register
    def _(self, item: str):
        return f"added string {item!r}"

    @add.register(int)
    def _(self, item: int):
        return f"added int {item}"

    @add.register(list)
    def _(self, items: list):
        return f"added list of {len(items)}"

o = Order()
print(o.add(42))                # added int 42
print(o.add("tea"))             # added string 'tea'
print(o.add([1, 2, 3]))         # added list of 3

```

Edge cases to mention: - `singledispatch` only dispatches on the FIRST argument — it's not full multimethod dispatch. - For complex multi-arg dispatch, `multipledispatch` (third-party) exists, but most codebases use plain `isinstance` chains. - Type hints on overloaded methods need `@overload` from typing — it's a typing-only construct, the implementation still needs a single `def`.

What NOT to say: - "Python supports overloading." — wrong. The second `def` silently overrides the first. - "`@overload` from typing implements overloading at runtime." — wrong. `@overload` is typing-only; the implementation still needs a single `def`.

Common follow-up: "If I need to dispatch on the type of the second argument, can `singledispatch` help?" — Not by itself — it dispatches only on the first arg. Use `multipledispatch` (third-party) or an `isinstance` ladder.

Stand-out line: "Python's answer to overloading is 'one function, branch on type if you must' — for cleaner code, `singledispatch` turns the branching into separate decorated functions."

Section C — Descriptors + Properties

Q21 · `@property` — the simple case

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Infosys, Postman, Freshworks · Topic: `@property` basics

The question (interviewer's verbatim phrasing):

When would you turn a simple attribute into a `@property`, and what's the cost of doing so?

30-second answer: `@property` turns a method into an attribute-style read — callers access `o.amount` instead of `o.amount()`. Pair it with `@amount.setter` and `@amount.deleter` for write and delete. It's the idiomatic way to add computed attributes or validation without changing the call site.

Step-by-step thought process: - Mechanism: `@property` decorates a method, returning a `property` object that's a descriptor. When accessed on an instance, it triggers the getter; assignment triggers the setter. - Use when you want a "computed" attribute (`full_name` from `first_name + last_name`), or to add validation to what used to be a plain attribute, without breaking callers. - The setter pattern: `@<name>.setter` decorates a second method that takes `(self, value)`. The deleter is the same idea for `del o.amount`. - Don't use `@property` for expensive computations — callers expect attribute access to be cheap. Use a regular method or `functools.cached_property` if expensive.

Code:

```
class Order:
    def __init__(self, amount: int):
        self._amount = amount

    @property
    def amount(self) -> int:
        return self._amount

    @amount.setter
    def amount(self, value: int) -> None:
        if value <= 0:
            raise ValueError("amount must be positive")
        self._amount = value

    @property
    def amount_with_tax(self) -> float:
        return self._amount * 1.18

o = Order(500)
print(o.amount, o.amount_with_tax)  # 500 590.0
o.amount = 700                     # setter runs validation
try:
    o.amount = -1                   # raises
except ValueError as e:
    print(e)
```


Edge cases to mention: - Setting `o.amount = 1` when there's only a getter (no setter) raises `AttributeError: can't set attribute`. - Properties don't show up in `vars(instance)` — they live on the class, not the instance. - Forgetting the underscore convention (`self.amount = value` instead of `self._amount = value`) inside a setter causes infinite recursion — `self.amount` triggers the setter again.

What NOT to say: - "`@property` makes the attribute read-only." — wrong by default. Without `@x.setter` it's read-only; add a setter and it's read-write. - "Properties are stored in the instance dict." — wrong. They live on the class.

Common follow-up: "What happens if my setter for `amount` writes `self.amount = value` ?" — Infinite recursion: the assignment triggers the setter again. Use `self._amount = value` and have the getter read `self._amount`.

Stand-out line: "`@property` is your tool to add behaviour to an attribute access without changing the call site — but if the getter is expensive, you've turned a read into a hidden function call, which is a different kind of bug."

Q22 · `@cached_property` — compute once, cache on the instance

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JP Morgan, Walmart Labs · Topic: `cached_property`

The question (interviewer's verbatim phrasing):

When should you reach for `functools.cached_property` over `@property` ? What's the storage trick that makes it zero-overhead after first access?

30-second answer: `functools.cached_property` is `@property` plus memoisation — the value is computed on first access and stored on the instance, so subsequent reads are O(1). Use it for expensive computed attributes that don't change for the lifetime of the instance.

Step-by-step thought process: - Mechanism: it's a descriptor that, on first access, computes the value and writes it directly onto `instance.__dict__[name]`. After that, normal attribute lookup finds the dict entry without calling the descriptor again. - Subtlety: because it writes to `__dict__`, classes with `__slots__` that don't include the cached attribute will raise `AttributeError`. Add it to slots or skip slots. - It's per-instance, not per-class. Two instances cache independently. - To invalidate: `del instance.<name>` — the dict entry goes away, next access re-computes.

Code:

```

from functools import cached_property
import time

class Report:
    def __init__(self, rows: list):
        self.rows = rows

    @cached_property
    def total(self) -> int:
        print("computing")
        time.sleep(0.01)      # pretend expensive
        return sum(self.rows)

r = Report([1, 2, 3, 4])
print(r.total)               # computing \n 10
print(r.total)               # 10 - cached, no print
del r.total                  # invalidate
print(r.total)               # computing \n 10

```

Edge cases to mention: - `__slots__` without `total` in the slots: `cached_property` can't write to `__dict__` because there isn't one. `AttributeError`. - Not thread-safe — concurrent first-access on the same instance can compute twice. Wrap in a lock if you need it. - Storing self-referential or heavy objects in a `cached_property` keeps them alive for the instance's lifetime — memory consideration.

What NOT to say: - "`cached_property` is just `@property` with `@lru_cache`." — wrong. It writes directly to `instance.__dict__`, so subsequent accesses skip the descriptor entirely. - "`cached_property` is thread-safe." — wrong. Concurrent first access can compute twice.

Common follow-up: "If I use `cached_property` on a class with `__slots__`, what happens — and how do I fix it?" — `AttributeError`, because there's no `__dict__` to write to. Add the cached attribute to `__slots__`, or skip slots.

Stand-out line: "`@cached_property` writes the value directly into the instance dict — that's why it has zero overhead after first access, and why it breaks under `__slots__` without an explicit slot."

Q23 · Custom descriptors with `__get__` / `__set__`

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Microsoft India, Postman · Topic: Custom descriptors

The question (interviewer's verbatim phrasing):

Walk me through writing a custom descriptor that adds validation. Where does the per-instance state actually live, and why must the descriptor itself sit at the class level?

30-second answer: A descriptor is any object that defines `__get__`, `__set__`, or `__delete__`. When you set it as a class attribute, Python routes attribute access through the descriptor's methods. `@property` is just a built-in descriptor — writing your own gives you reusable validation, type coercion, or attribute logic across many classes.

Step-by-step thought process: - Descriptor protocol: define a class with `__get__(self, instance, owner)` and `__set__(self, instance, value)`. Set an instance of it as a class attribute. - When you read `o.field`, Python sees a descriptor at the class level and calls `descriptor.__get__(o, type(o))` instead of returning the descriptor itself. - Storage: the descriptor needs somewhere to keep per-instance state. Options: store on `instance.__dict__` under a mangled name, or use a WeakKeyDictionary keyed on the instance. - `__set_name__` (PEP 487, Python 3.6+) is called when the descriptor is assigned in a class body, so it can learn its own attribute name and use it for storage.

Code:

```
class Positive:
    def __set_name__(self, owner, name):
        self.private = f"_{name}"

    def __get__(self, instance, owner):
        if instance is None:
            return self
        return getattr(instance, self.private)

    def __set__(self, instance, value):
        if not isinstance(value, (int, float)) or value <= 0:
            raise ValueError(f"{self.private[1:]} must be positive number")
        setattr(instance, self.private, value)

class Order:
    amount = Positive()
    quantity = Positive()

    def __init__(self, amount, quantity):
        self.amount = amount
        self.quantity = quantity

o = Order(500, 3)
print(o.amount, o.quantity)          # 500 3
try:
    Order(-1, 2)
except ValueError as e:
    print(e)                          # amount must be positive number
```

Edge cases to mention: - Descriptors MUST live on the class, not the instance. Putting one in `__init__` (`self.field = Positive()`) doesn't trigger the protocol. - Without `__set_name__` you have to manually pass the name: `amount = Positive("amount")` — boilerplate. - Forgetting to handle `instance is None` in `__get__` breaks `Cls.field` access from the class itself.

What NOT to say: - "You can put a descriptor on `self` and it'll still trigger." — wrong. The descriptor protocol only fires for class-level descriptors. - "Descriptors store state on themselves." — wrong. There's one descriptor per class attribute, not per instance — store state on the instance.

Common follow-up: "How does `__set_name__` simplify a descriptor's API, and what existed before PEP 487?" — Before, you passed the name in: `amount = Positive('amount')`. With `__set_name__`, you write `amount = Positive()` and the descriptor learns its name from Python at class creation.

Stand-out line: "Descriptors are the engine — `@property`, `@classmethod`, `@staticmethod`, even bound methods are all descriptors. Writing your own pays off when the same validation shows up across five classes."

Q24 · Data descriptors vs non-data descriptors

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Microsoft India · Topic: Data vs non-data descriptors

The question (interviewer's verbatim phrasing):

What's the difference between a data descriptor and a non-data descriptor, and why does the distinction matter at attribute lookup time?

30-second answer: A data descriptor defines both `__get__` and `__set__` (or `__delete__`). A non-data descriptor defines only `__get__`. The distinction matters because data descriptors take precedence over instance `__dict__`; non-data descriptors lose to it.

Step-by-step thought process: - The attribute lookup order: (1) data descriptors on the class, (2) instance `__dict__`, (3) non-data descriptors on the class, (4) regular class attributes, (5) `__getattr__`. - Why it matters: `@property` is a data descriptor, so `instance.x = 5` is intercepted by the setter even if there's an entry in `instance.__dict__`. A plain function (non-data descriptor) loses to an instance attribute — `o.method = something` shadows the bound method. - `@cached_property` is intentionally a non-data descriptor — that's how it lets the cached value in `__dict__` win on subsequent lookups. - Custom descriptor: defining `__set__ = None` doesn't make it non-data; you have to NOT define `__set__` at all.

Code:

```

class DataDesc:
    def __get__(self, instance, owner): return "data-get"
    def __set__(self, instance, value): instance.__dict__["x"] = value

class NonDataDesc:
    def __get__(self, instance, owner): return "non-data-get"

class A:
    data = DataDesc()
    nondata = NonDataDesc()

a = A()
a.__dict__["data"] = "from-dict"
a.__dict__["nondata"] = "from-dict"

print(a.data)           # 'data-get' – data descriptor wins
print(a.nondata)        # 'from-dict' – non-data loses to __dict__

```

Edge cases to mention: - Methods are non-data descriptors — you can shadow them per-instance, which is how monkey-patching individual instances works. - Adding a `__set__` to a previously-non-data descriptor changes lookup behaviour — be careful when retrofitting. - `cached_property` deliberately is non-data so the cached value in `__dict__` shortcuts later lookups.

What NOT to say: - "Data and non-data descriptors have the same priority." — wrong. Data descriptors win over instance dict; non-data lose. - "`__set__ = None` makes a descriptor non-data." — wrong. You have to NOT define `__set__` at all.

Common follow-up: "Why is `@cached_property` intentionally non-data, even though `__set__` would seem to let it invalidate the cache?" — Because being non-data lets the cached value in `__dict__` win on subsequent lookups, which is exactly what gives you O(1) reads after first access.

Stand-out line: "Data descriptors are the gatekeepers of attribute access — they win against the instance dict. Non-data ones step aside the moment the instance has its own value."

Q25 · `__set_name__` — descriptors learn their own name

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft India, Postman, JP Morgan · Topic: `set_name`

The question (interviewer's verbatim phrasing):

What does `__set_name__` do, when does it run, and why was it added in PEP 487?

30-second answer: `__set_name__(self, owner, name)` is called when a descriptor is assigned as a class attribute, with the class and the attribute name. It lets the descriptor know what name it was bound to, so it can use that for per-instance storage without you passing the name as an argument.

Step-by-step thought process: - Mechanism: during class creation, Python walks the class dict and calls `__set_name__` on every descriptor that defines it. Happens once per descriptor, at class-creation time. - Why it matters: before PEP 487, you'd write `amount = Positive("amount")` so the descriptor knew its name. With `__set_name__`, you write `amount = Positive()` and the descriptor learns its name automatically. - Common storage pattern: `self.private = f"_{name}"`, then store / read via `getattr(instance, self.private)`. - Multiple instances of the same descriptor class get separate `name` values — each tracks its own attribute.

Code:

```
class Field:
    def __set_name__(self, owner, name):
        self.name = name
        self.private = f"_{name}"
        print(f"bound {name} on {owner.__name__}")

    def __get__(self, instance, owner):
        if instance is None: return self
        return instance.__dict__.get(self.private)

    def __set__(self, instance, value):
        instance.__dict__[self.private] = value

class Order:
    amount = Field()          # __set_name__ called with name='amount'
    currency = Field()        # __set_name__ called with name='currency'

o = Order()
o.amount = 500
print(o.__dict__)           # {'_field_amount': 500}
```

Edge cases to mention: - `__set_name__` runs at class-creation time, so heavy work here slows imports. - It's only called when the descriptor is at class definition time — assigning a descriptor to a class attribute after creation (`Order.new_field = Field()`) does NOT call `__set_name__` automatically. - The `owner` argument is the class, not an instance — useful for class-level registries.

What NOT to say: - "`__set_name__` runs on every attribute access." — wrong. It runs once at class creation, when the descriptor is set on the class. - "`__set_name__` works for descriptors assigned dynamically after class creation." — wrong. Only descriptors visible in the class body get the hook.

Common follow-up: "What's the storage convention people usually pick for the per-instance value in a descriptor that learned its name via `__set_name__`?" — Stash on `instance.__dict__` under a mangled name like `_field_<name>` so it doesn't collide with the descriptor itself.

Stand-out line: "`__set_name__` is the 'who am I bound to?' hook — it lets a descriptor learn its own attribute name without you wiring it manually."

Q26 · Attribute lookup order — the full ladder

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Microsoft India, Postman · Topic: Attribute lookup order

The question (interviewer's verbatim phrasing):

Walk me through the full attribute lookup order on a read — `instance.x` — including data descriptors, instance dict, non-data descriptors, and `__getattr__`.

30-second answer: When you read `instance.x`, Python walks: (1) data descriptors on the class hierarchy, (2) `instance.__dict__`, (3) non-data descriptors and class attributes on the class hierarchy, (4) `__getattr__` as the fallback. Writing is simpler: `instance.x = y` calls `__setattr__`, which routes through any class-level `__set__` descriptor, otherwise lands in `instance.__dict__`.

Step-by-step thought process: - The full chain for reads (CPython's `object.__getattribute__`):
 1. Walk `type(instance).__mro__` looking for a data descriptor — if found, call its `__get__`.
 2. Look in `instance.__dict__` — return if present.
 3. Walk MRO again for non-data descriptors and plain class attributes — return first match.
 4. If nothing found, fall back to `__getattr__` (if defined) — usually raises `AttributeError`.
 - Memorise the data-descriptor-wins rule — it's why `@property` setters can't be silently shadowed.
 - Writing path: `__setattr__` is the entry point. Default behaviour calls `__set__` on any class-level data descriptor with that name, otherwise writes to `__dict__`.
 - `__getattribute__` is the entry point for EVERY attribute access. `__getattr__` is the fallback called only when the normal path raises `AttributeError`.

Code:

```

class Tracer:
    def __get__(self, instance, owner): return "data-desc"
    def __set__(self, instance, value): instance.__dict__["tracked"] = value

class A:
    tracked = Tracer()          # data descriptor
    classlevel = 99             # plain class attribute

    def __init__(self):
        self.__dict__["tracked"] = "from-dict"
        self.__dict__["classlevel"] = "shadow"

a = A()
print(a.tracked)               # 'data-desc' – descriptor wins
print(a.classlevel)           # 'shadow' – instance dict wins over plain class attr

```

Edge cases to mention: - `__getattr__` is NOT called for every access — only when the normal lookup raises. Define `__getattribute__` if you want every access to route through your code. - Defining `__getattribute__` is dangerous — infinite recursion if you access self attributes inside it. Use `super().__getattribute__(name)` or `object.__getattribute__(self, name)`. - The C-level path is faster than the Python-level path — overriding `__getattribute__` slows every attribute access on that class.

What NOT to say: - "Python checks the instance dict first, then the class." — incomplete. Data descriptors check first. - "`__getattr__` is called for every access." — wrong. Only when the normal lookup raises `AttributeError`.

Common follow-up: "If I define `__getattribute__` to log every access, why does `len(obj)` not appear in the log?" — Because special-method lookup bypasses instance `__getattribute__` — Python looks dunder up directly on the type for speed and correctness.

Stand-out line: "Data descriptors are first in the read order — that's the architectural reason `@property` can enforce invariants even when callers think they're writing a plain attribute."

Q27 · Why `@property` doesn't show up via `vars(instance)`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman · Topic: `vars()` and properties

The question (interviewer's verbatim phrasing):

If I define a `@property` called `amount` on a class and create an instance, why doesn't `vars(instance)` show `amount` ?

30-second answer: Properties live on the class, not the instance — they're descriptors stored in `Cls.__dict__`. `vars(instance)` returns `instance.__dict__`, which is the per-instance dict; properties never go there. To list them, walk the class MRO and look for `property` objects.

Step-by-step thought process: - `vars(o)` is `o.__dict__` — instance-only. Properties never reach there because they intercept access at the class level. - To enumerate properties, use `vars(type(o))` (class-level) or walk MRO with `[a for cls in type(o).__mro__ for a, v in cls.__dict__.items() if isinstance(v, property)]`. - Same applies to `@classmethod`, `@staticmethod`, descriptors, and methods — all live on the class. - `dir(o)` shows everything visible (class + instance), but doesn't tell you what's a property vs what's a plain attribute.

Code:

```
class Order:
    def __init__(self, amount):
        self._amount = amount

    @property
    def amount(self): return self._amount

o = Order(500)
print(vars(o))                # {'_amount': 500} — no 'amount'
print(o.amount)               # 500 — works via class descriptor

# Find properties on the class
props = [name for name, val in vars(Order).items()
          if isinstance(val, property)]
print(props)                  # ['amount']
```

Edge cases to mention: - `hasattr(o, "amount")` returns True even though it's not in `__dict__` — `hasattr` walks the full attribute lookup chain. - `o.__dict__["amount"] = 999` would shadow the property — but only for reads, because the property is a data descriptor and intercepts writes. - Serialisation libraries that walk `__dict__` (some pickle, msgpack, json adapters) MISS properties — they only see the underscore-prefixed backing attribute.

What NOT to say: - "Properties hide themselves from `vars`." — wrong, more precisely. They never go into `instance.__dict__` because they live on the class. - "`vars(instance)` shows everything attached to the instance." — wrong. It shows only the instance dict, not the class-level descriptors.

Common follow-up: "How would you enumerate every `@property` defined across a class and its bases?" — Walk `type(obj).__mro__`; for each class, look in `cls.__dict__` for `isinstance(v, property)`.

Stand-out line: "Anything you decorate at the class level — properties, methods, classmethods — is stored on the class, not the instance. `vars(instance)` only shows what's per-object."

Q28 · Descriptors as the engine behind methods

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Microsoft India · Topic: Methods as descriptors

The question (interviewer's verbatim phrasing):

Methods don't appear in `instance.__dict__`, but `instance.method` returns a callable with `self` already filled in. Walk me through how that works.

30-second answer: Every function defined inside a class is a non-data descriptor — its `__get__` returns a bound method when accessed via an instance. That's why `o.method` gives you a callable with `self` pre-filled even though the underlying `Order.method` is just a function.

Step-by-step thought process: - `Order.charge` is a function object. `instance.charge` calls `function.__get__(instance, type(instance))`, which returns a `bound method`. - The bound method object stores `__self__` (the instance) and `__func__` (the function). Calling it invokes `__func__(__self__, *args)`. - Same descriptor protocol drives `@classmethod` (binds to the class instead of an instance) and `@staticmethod` (returns the function unchanged). - This is also why methods don't show up in `instance.__dict__` — they live on the class as functions, promoted on access.

Code:

```
class Order:
    def charge(self, amount): return f"{id(self)}: {amount}"

o = Order()
print(Order.charge)          # <function Order.charge at ...>
print(o.charge)              # <bound method Order.charge of ...>

bound = o.charge
print(bound.__self__ is o)    # True
print(bound.__func__ is Order.charge) # True

# You can call it the unbound way explicitly
print(Order.charge.__get__(o, Order)(500)) # same as o.charge(500)
```

Edge cases to mention: - Storing `o.charge` in a variable keeps `o` alive — bound methods hold strong references. - A function defined outside the class and then assigned (`Order.charge = some_func`) IS a descriptor — it becomes a bound method on access. That's how monkey-patching works. - `__func__` and `__self__` are the documented attributes — useful for introspection (mypy, sphinx, mock libraries all read them).

What NOT to say: - "Python special-cases methods at attribute lookup." — wrong. Functions are non-data descriptors; `__get__` returns the bound method. - "Bound methods are stored on the instance after first access." — wrong. They're created fresh on each attribute access.

Common follow-up: "Why does storing `o.method` in a local variable keep `o` alive — and where does that bite in production?" — The bound method holds a strong reference to `__self__`. Common cause of leaks in event-handler or callback patterns.

Stand-out line: "Methods are functions in disguise — the descriptor protocol promotes them on access, and `self` is just the first argument injected by `__get__`."

Q29 · `property` with deleter — when you actually need it

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman · Topic: Property deleter

The question (interviewer's verbatim phrasing):

When would you ever define `@<name>.deleter` on a property? What real-world cleanup does it enable?

30-second answer: `@<name>.deleter` defines what happens when someone runs `del obj.attr`. Use it when deletion needs to do cleanup beyond just removing the attribute — close a file, release a lock, mark a record invalid. Most properties don't need a deleter.

Step-by-step thought process: - Same descriptor mechanism as setter — `@<name>.deleter` decorates a method `(self)` that runs on `del`. - Real use: a property backed by a resource (file handle, DB connection) where `del` should release the resource. Rare but real. - More common pattern: use a deleter to invalidate a `cached_property` value or reset to a default state. - If there's no deleter, `del obj.attr` raises `AttributeError`.

Code:

```

class Config:
    def __init__(self):
        self._cache = {"region": "ap-south-1"}

    @property
    def region(self) -> str:
        return self._cache.get("region", "default")

    @region.setter
    def region(self, value: str) -> None:
        self._cache["region"] = value

    @region.deleter
    def region(self) -> None:
        print("resetting region")
        self._cache.pop("region", None)

c = Config()
print(c.region)                # ap-south-1
c.region = "ap-southeast-1"
print(c.region)                # ap-southeast-1
del c.region                   # resetting region
print(c.region)                # 'default' (popped, falls back)

```

Edge cases to mention: - Forgetting the deleter and trying `del o.attr` on a property raises `AttributeError`. - Setting `o.attr = None` is NOT the same as `del o.attr` — only the deleter intentionally cleans up. - Inside the deleter, `del self.attr` would recurse — use the backing store (`self._cache`) directly.

What NOT to say: - "Every property needs a deleter for completeness." — wrong. Most properties never need one. - "`del o.attr` and `o.attr = None` do the same thing." — wrong. The first triggers the deleter; the second goes through the setter.

Common follow-up: "What happens if I `del o.attr` on a property that has no deleter defined?" — `AttributeError: can't delete attribute`. Python doesn't fall back to a default deleter.

Stand-out line: "Properties get a deleter so you can hook `del` — useful when removal is more than just forgetting a value, like releasing a resource or invalidating a cache."

Q30 · `__slots__` vs `__dict__` access patterns

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Walmart Labs · Topic: **slots** access patterns

The question (interviewer's verbatim phrasing):

*Beyond saving memory, what changes about attribute access when you declare `__slots__` ?
When would the trade-off NOT be worth it?*

30-second answer: With `__dict__`, attributes are stored in a Python dict — flexible, supports any attribute name, slower lookup. With `__slots__`, attributes live in a fixed-size C-level array — smaller memory, faster lookup, but no dynamic attributes and the slotted names are baked at class definition time.

Step-by-step thought process: - The default object layout has a `__dict__` per instance. Every attribute write is a dict insert; every read is a dict lookup. - `__slots__ = ("a", "b")` tells Python to allocate exactly two slots in the C-level instance layout — no `__dict__`, no `__weakref__` unless you add them. - Attribute access on slotted objects uses descriptor magic — each slot is implemented as a descriptor on the class. That's why slotted attributes can't be added dynamically. - Adding `__weakref__` to `__slots__` lets weakrefs work; otherwise slotted instances can't be weakly referenced.

Code:

```
import sys

class Loose:
    def __init__(self): self.a = 1; self.b = 2

class Tight:
    __slots__ = ("a", "b")
    def __init__(self): self.a = 1; self.b = 2

l = Loose(); t = Tight()
print(hasattr(l, "__dict__"))           # True
print(hasattr(t, "__dict__"))           # False
print(sys.getsizeof(l) + sys.getsizeof(l.__dict__),
      sys.getsizeof(t))                  # loose much larger

# slot access uses descriptors
print(type(Tight.a).__name__)            # 'member_descriptor'

try:
    t.c = 99
except AttributeError as e:
    print(e)                             # 'Tight' object has no attribute 'c'
```

Edge cases to mention: - Subclassing a slotted class with no `__slots__` re-introduces `__dict__` — the memory saving stops at that subclass. - Multiple inheritance with `__slots__` is hard: at most one base can have non-empty slots, otherwise you get `TypeError: multiple bases have instance lay-out conflict`. - Pickling and copy generally work but some serialisers expect `__dict__` — verify with your serialiser before adopting slots widely.

What NOT to say: - "`__slots__` makes every Python class faster." — overstated. Memory is the main win. - "Slotted classes always block subclassing." — wrong. They allow it, but the subclass regains `__dict__` unless it also declares slots.

Common follow-up: "*What's the failure mode you should worry about when you retrofit `__slots__` onto a class that's already in production?*" — Anything relying on dynamic attributes — mock libraries, ORMs, monkey-patches — silently breaks. Audit the call sites first.

Stand-out line: "*`__slots__` swaps Python's dict-per-instance for a C-level array — memory shrinks by an order of magnitude, but you lose the ability to bolt on attributes you didn't declare.*"

Section D — Dunder Method Core

Q31 · `__repr__` vs `__str__`

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Postman, Infosys, Freshworks · Topic: `repr` vs `str`

The question (interviewer's verbatim phrasing):

What's the difference between `__repr__` and `__str__`, and which one should you implement first?

30-second answer: `__repr__` is the unambiguous, developer-facing string — meant to be informative, ideally something you could paste back into Python to recreate the object. `__str__` is the user-facing string — readable, conversational. If you only define one, define `__repr__`; `str()` falls back to it.

Step-by-step thought process: - `repr(obj)` calls `obj.__repr__()`. `str(obj)` calls `obj.__str__()` if defined, else falls back to `__repr__`. - The convention: `__repr__` should look like a constructor call (`Order(amount=500, currency='INR')`) — explicit, parseable, includes all state. - `__str__` can be free-form ("Order #1234 for ₹500") — for logging, user messages, `print()`. - Always define `__repr__`. The default (`<__main__.Order object at 0x...>`) is useless in debugging.

Code:

```
class Order:
    def __init__(self, amount: int, currency: str = "INR"):
        self.amount, self.currency = amount, currency

    def __repr__(self) -> str:
        return f"Order(amount={self.amount}, currency={self.currency!r})"

    def __str__(self) -> str:
        return f"Order #{id(self) & 0xFFFF} for ₹{self.amount}"

o = Order(500)
print(repr(o))           # Order(amount=500, currency='INR')
print(str(o))           # Order #... for ₹500
print(o)                 # str
print([o])               # [Order(...)] – list uses repr
```

Edge cases to mention: - Containers (`list` , `dict` , `tuple`) call `__repr__` on their elements, not `__str__` . So `print([o])` uses repr. - A `__repr__` that raises is fatal — debuggers and REPLs constantly call repr, and an exception there hides everything. - The `!r` format spec is shorthand for `repr()` — useful in f-strings for quoting strings cleanly.

What NOT to say: - "`__str__` is for printing and `__repr__` is for logs." — partially true and imprecise. `__repr__` is for developers; `__str__` is for users. - "If I don't define either, `print(obj)` raises." — wrong. Default `__repr__` runs (the `<...object at 0x...>` form).

Common follow-up: "Why does `print([obj])` use `__repr__` instead of `__str__` ?" — Containers always call `__repr__` on their elements, because they need an unambiguous representation, not a user-facing one.

Stand-out line: "`__repr__` is for you at 2am with `pdb` open; `__str__` is for the user reading a notification. Always implement `__repr__` ; `__str__` is optional."

Q32 · The `__eq__` / `__hash__` contract

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Microsoft India, Postman · Topic: `eq` / `hash` contract

The question (interviewer's verbatim phrasing):

Walk me through the `__eq__` / `__hash__` contract. What happens if I override `__eq__` and forget `__hash__` ?

30-second answer: If `a == b` then `hash(a) == hash(b)` — that's the contract. Override `__eq__` and Python sets `__hash__ = None` automatically, making the object unhashable. To keep it usable as a dict key or in a set, explicitly define `__hash__` that's consistent with `__eq__`.

Step-by-step thought process: - Default behaviour for `object`: `__eq__` is identity (`a is b`); `__hash__` is `id()`-based. Both consistent. - Define `__eq__` and Python implicitly sets `__hash__ = None` — the rationale is that the default hash (based on id) is no longer consistent with the new equality. Object becomes unhashable. - To make it hashable again, define `__hash__` based on the SAME attributes you compared in `__eq__`. Two equal objects must hash equal; the reverse isn't required (collisions are fine). - Immutable objects (`tuple`, `frozenset`) hash on their contents. Mutable objects (`list`, `dict`) are unhashable — you shouldn't be using them as dict keys.

Code:

```
class Order:
    def __init__(self, order_id: str, amount: int):
        self.order_id, self.amount = order_id, amount

    def __eq__(self, other) -> bool:
        if not isinstance(other, Order): return NotImplemented
        return self.order_id == other.order_id

    def __hash__(self) -> int:
        return hash(self.order_id)           # same attribute as __eq__

a = Order("RZP-1", 500)
b = Order("RZP-1", 700)                     # different amount, same id
print(a == b, hash(a) == hash(b))          # True True

s = {a, b}                                  # set works – they're "equal", so one entry
print(len(s))                              # 1
```

Edge cases to mention: - Defining `__eq__` without `__hash__`: object becomes unhashable. Common silent breakage when adopting equality. - Mutable hash: if you hash on a mutable field and then mutate it, the object's position in any dict / set is wrong. Hash should be based on truly immutable identity. - Returning `NotImplemented` (capital N, not False) for unsupported comparison types lets Python try the reflected operation — best practice.

What NOT to say: - "You can override `__eq__` independently of `__hash__`." — wrong. Python silently sets `__hash__ = None`, making the object unhashable. - "Two objects with the same `__hash__` are equal." — wrong. Hash collisions are legal; equality is the source of truth.

Common follow-up: "If you hash on a mutable field — say `Order.amount` — and then mutate it, what breaks?" — The object's position in any set or dict becomes inconsistent. Lookups and deletions silently fail. Hash should depend on truly immutable identity.

Stand-out line: " `__eq__` and `__hash__` come as a contract — break it, and your objects silently misbehave in sets and dicts. The interpreter even enforces it by zeroing `__hash__` when you override `__eq__`."

Q33 · `__lt__` / `__le__` and `functools.total_ordering`

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Cred · Topic: Ordering dunders

The question (interviewer's verbatim phrasing):

How do you make objects sortable in Python? When does `functools.total_ordering` earn its keep over writing all six comparison dunders?

30-second answer: Define `__lt__` (and `__eq__`) to make objects comparable with `<`. For full ordering (`<`, `<=`, `>`, `>=`), define all four — or use `@functools.total_ordering` to fill in the rest from just two. Comparison operators that aren't implemented return `NotImplemented`, letting Python try the reflected operation.

Step-by-step thought process: - The six comparison dunders: `__lt__`, `__le__`, `__gt__`, `__ge__`, `__eq__`, `__ne__`. Each takes `(self, other)`. - `total_ordering` is the lazy man's friend — define `__eq__` plus one of `<`, `<=`, `>`, `>=`, and the decorator generates the rest. Slight perf cost (calls into others) but worth it for clarity. - Return `NotImplemented` if `other` is the wrong type — Python will try `other.__gt__(self)` etc. and fall back to `TypeError` if nothing works. - Sorting (`sorted`, `min`, `max`, heaps) uses `__lt__`. That's the minimum you need to make objects sortable.

Code:

```

from functools import total_ordering

@total_ordering
class Order:
    def __init__(self, amount: int, priority: int):
        self.amount, self.priority = amount, priority

    def __eq__(self, other):
        if not isinstance(other, Order): return NotImplemented
        return (self.priority, self.amount) == (other.priority, other.amount)

    def __lt__(self, other):
        if not isinstance(other, Order): return NotImplemented
        return (self.priority, self.amount) < (other.priority, other.amount)

orders = [Order(500, 2), Order(700, 1), Order(300, 1)]
for o in sorted(orders):
    print(o.priority, o.amount)
# 1 300 / 1 700 / 2 500

print(Order(500, 1) <= Order(500, 1))    # True — total_ordering filled <= in

```

Edge cases to mention: - `total_ordering` requires `__eq__` AND at least one of `<`, `<=`, `>`, `>=` — forgetting one raises at decoration time. - The decorator-generated methods are slower than handwritten ones — micro-issue, but real in hot loops. - Comparing tuples is the idiomatic Python pattern for multi-key ordering — `(self.a, self.b) < (other.a, other.b)`.

What NOT to say: - "You need all six comparison methods for sorting." — wrong. `sorted()` needs only `__lt__`. - "Return False for incompatible types." — wrong. Return `NotImplemented` to let Python try the reflected operation.

Common follow-up: "What's the idiomatic way to sort by multiple fields with priority?" — Compare tuples: `(self.priority, self.amount) < (other.priority, other.amount)`. Python's tuple comparison is lexicographic.

Stand-out line: "Don't write all six comparison dunder by hand — `@total_ordering` plus `__eq__` and `__lt__` covers every operator, with the clarity of a single source of truth."

Q34 · `__contains__` — the `in` operator

Tags: Difficulty: Easy · Asked at: Razorpay, TCS, Postman · Topic: **contains**

The question (interviewer's verbatim phrasing):

What does `__contains__` do, and when would you implement it on a custom class instead of letting Python fall back to iteration?

30-second answer: `__contains__(self, item)` defines what `x in container` returns. If you don't define it, Python falls back to iterating with `__iter__` and comparing. Implement it explicitly when you have a faster membership test — a set lookup, a hash check, a range test — than walking every element.

Step-by-step thought process: - `x in obj` calls `obj.__contains__(x)`. Return a boolean. - If `__contains__` is missing, Python iterates `obj` and returns True on first match — $O(n)$ by default, even when $O(1)$ would work. - Use it on custom collection classes where you know membership is fast — wrapping a set, a Bloom filter, a sorted list with binary search. - Make sure `__contains__` is consistent with `__iter__` — `x in obj` should be True iff `x` shows up during iteration. Otherwise readers get confused.

Code:

```
class OrderIDs:
    """Wrapper around a set with O(1) membership."""
    def __init__(self, ids):
        self._set = set(ids)

    def __contains__(self, x):
        print("called __contains__")
        return x in self._set

    def __iter__(self):
        return iter(self._set)

ids = OrderIDs(["RZP-1", "RZP-2", "RZP-3"])
print("RZP-2" in ids)           # called __contains__ \n True
print("F00" in ids)            # called __contains__ \n False

# Without __contains__, Python would iterate — O(n) instead of O(1)
```

Edge cases to mention: - `"hello" in "world hello"` works for strings because str defines `__contains__`. The operator's behaviour is type-driven. - For dicts, `key in d` checks the keys — not the values. Common confusion for beginners. - `__contains__` semantics differ in numpy arrays (`x in arr` raises on multi-d arrays in some versions) — be careful with overloads.

What NOT to say: - "`__contains__` is required for `for` loops." — wrong. `__iter__` is. - "`x in obj` always calls `__contains__`." — wrong. If absent, Python iterates and compares.

Common follow-up: "If I wrap a set in a custom class to expose only a few operations, what's the easiest way to make `in` $O(1)$?" — Define `__contains__` that delegates to the underlying set. Without it, Python would iterate — $O(n)$.

Stand-out line: "`__contains__` exists to short-circuit the default iteration — implement it whenever your membership test is faster than walking every element."

Q35 · `__len__` and `__bool__`

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Postman · Topic: `len` and `bool`

The question (interviewer's verbatim phrasing):

How does Python decide whether `if obj:` is True or False when `obj` is a custom class?
What's the precedence between `__bool__` and `__len__` ?

30-second answer: `__len__(self)` returns the size of the container — used by `len()`, by `bool()` as a fallback, and by truth testing. `__bool__(self)` defines truthiness directly. If neither is defined, every instance is truthy. Define `__len__` for collection-like objects; define `__bool__` when truthiness isn't just "non-empty".

Step-by-step thought process: - `len(obj)` calls `obj.__len__()`. Must return a non-negative int.
- `bool(obj)` calls `__bool__()` first; falls back to `__len__()` (with `len > 0` as True); falls back to True if neither exists. - Truth testing in conditionals (`if obj:`) uses `bool()`. So defining `__len__` makes `if my_empty_container:` work as you'd expect — empty is falsy. - `__bool__` matters for non-collection types where truthiness has meaning (`Decimal(0)` is falsy; `Decimal('5')` is truthy).

Code:

```

class Basket:
    def __init__(self, items):
        self._items = list(items)

    def __len__(self):
        return len(self._items)

class FeatureFlag:
    def __init__(self, enabled: bool):
        self._enabled = enabled

    def __bool__(self):
        return self._enabled

b1 = Basket([]); b2 = Basket(["a"])
print(len(b1), bool(b1))           # 0 False - __len__ fallback
print(len(b2), bool(b2))           # 1 True

print(bool(FeatureFlag(False)))    # False - explicit __bool__
print(bool(FeatureFlag(True)))     # True

# No __bool__, no __len__ - always True
class Plain: pass
print(bool(Plain()))               # True

```

Edge cases to mention: - `__len__` returning a negative int raises `ValueError`. - `__len__` returning anything other than an int raises `TypeError` — common when you return a float by accident. - `if obj is None` vs `if obj` — the first is identity check, the second triggers `__bool__`. Mixing them up causes off-by-one bugs.

What NOT to say: - "Every object is truthy unless `__bool__` is defined." — incomplete. `__len__` returning 0 also makes it falsy. - "`__len__` returning -1 is fine." — wrong. Negative length raises `ValueError`.

Common follow-up: "What's the difference between `if obj is None` and `if not obj`?" — `is None` is an identity check that always works. `not obj` triggers `__bool__` / `__len__`, which can be falsy for empty collections, zero numbers, etc. — usually a bug if you meant the first.

Stand-out line: "Collections define `__len__` and get truthiness for free; everything else that needs a non-trivial truthiness defines `__bool__` directly."

Q36 · `__iter__` and `__next__` — the iterator protocol

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Cred, Postman · Topic: Iterator protocol

The question (interviewer's verbatim phrasing):

Explain the iterator protocol. What's the difference between an iterable and an iterator, and why does an iterator's `__iter__` return `self` ?

30-second answer: `__iter__(self)` returns an iterator — anything with a `__next__` method. `__next__(self)` returns the next value, or raises `StopIteration` to signal end. A class can be its own iterator (`__iter__` returns `self` , `__next__` advances), or it can return a separate iterator object on each call to `__iter__` .

Step-by-step thought process: - The two roles: an **iterable** is anything `for` can loop over (has `__iter__`); an **iterator** is what `__iter__` returns (has `__next__`). - Stateful classes that return `self` from `__iter__` can only be iterated once — second `for` loop sees an empty sequence. Use a separate iterator object for re-iteration. - `StopIteration` is the end-of-iteration signal. Inside a generator function, `return` triggers it automatically; in a manual iterator, you raise it. - `for x in obj` : under the hood is roughly `it = iter(obj); while True: try: x = next(it) except StopIteration: break; ...` .

Code:

```
class Countdown:
    """Custom iterable – re-iterable because __iter__ returns a fresh iterator."""
    def __init__(self, start: int):
        self.start = start

    def __iter__(self):
        return CountdownIterator(self.start)

class CountdownIterator:
    def __init__(self, start: int):
        self.n = start

    def __iter__(self): return self

    def __next__(self):
        if self.n <= 0:
            raise StopIteration
        v = self.n; self.n -= 1; return v

c = Countdown(3)
print(list(c))           # [3, 2, 1]
print(list(c))           # [3, 2, 1] – re-iterable

# Compare: generator is its own iterator – one-shot
def countdown_gen(n):
    while n > 0:
        yield n; n -= 1
g = countdown_gen(3)
print(list(g), list(g))  # [3, 2, 1] []
```

Edge cases to mention: - Implementing `__iter__` but not `__next__` : the iterable can be looped, but you can't manually `next(obj)` — needs a separate iterator object or `obj = iter(obj)` first. - Forgetting to raise `StopIteration` : infinite iteration. - Mixing `__iter__` returning `self` (one-shot) with re-iteration expectations is a classic bug — `csv.reader` , `zip` , generators all share this.

What NOT to say: - "`__iter__` and `__next__` are the same protocol." — wrong. `__iter__` defines iterables; `__next__` defines iterators. - "An iterator can be re-iterated." — wrong. Once `StopIteration` fires, it stays exhausted.

Common follow-up: "How do you make a custom class re-iterable so two `for` loops over the same instance both produce the full sequence?" — `__iter__` returns a fresh iterator object each time, instead of returning `self` .

Stand-out line: "Iterables produce iterators; iterators produce values. The class that returns `self` from `__iter__` is its own iterator — and one-shot. The class that returns a new iterator each time is re-iterable."

Q37 · `__getitem__` — indexing, slicing, and the legacy fallback

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Postman, Microsoft India · Topic: `getitem` semantics

The question (interviewer's verbatim phrasing):

What does `__getitem__` handle, and how do you differentiate between integer indexing and slice indexing inside a single implementation?

30-second answer: `__getitem__(self, key)` defines `obj[key]` — what you get when you index. Key can be anything; slices arrive as `slice` objects. If a class defines `__getitem__` but not `__iter__` , Python uses `__getitem__` for iteration as a legacy fallback — calling it with 0, 1, 2, ... until `IndexError` .

Step-by-step thought process: - Two patterns: integer indexing (`obj[0]`) and slice indexing (`obj[1:5]`). Both arrive via the same `__getitem__` , with `key` being either `int` or `slice` . - For dict-like classes, `key` can be anything. Handle types explicitly — check `isinstance(key, slice)` to differentiate. - Slice objects have `start` , `stop` , `step` — `slice(1, 5, None)` for `[1:5]` . The `slice.indices(len(self))` method normalises them against your length. - The legacy iteration: if you define `__getitem__` and someone does `for x in obj` , Python calls `obj[0]` , `obj[1]` , ... until you raise `IndexError` . Avoid relying on this — define `__iter__` explicitly.

Code:

```

class Roster:
    def __init__(self, members):
        self._members = list(members)

    def __getitem__(self, key):
        if isinstance(key, slice):
            return Roster(self._members[key])
        if isinstance(key, int):
            return self._members[key]
        raise TypeError(f"bad key: {type(key).__name__}")

    def __repr__(self):
        return f"Roster({self._members!r})"

r = Roster(["TCS", "Infosys", "Razorpay", "Cred"])
print(r[1])                # 'Infosys'
print(r[1:3])              # Roster(['Infosys', 'Razorpay'])
print(r[::-1])             # Roster(['Cred', 'Razorpay', 'Infosys', 'TCS'])

# Iteration works via the __getitem__ fallback if __iter__ is missing
for x in r:
    print(x)

```

Edge cases to mention: - Negative indices: `obj[-1]` arrives as `key = -1` — handle it yourself or delegate to the underlying list. - `__getitem__` raising the wrong exception (e.g. `KeyError` from an int-indexed thing) breaks the legacy iteration fallback — must be `IndexError`. - `obj[a, b]` arrives as `key = (a, b)` — a tuple. NumPy uses this for multi-d indexing.

What NOT to say: - "`__getitem__` handles only integer indexing." — wrong. Slices, tuples, and any key shape arrive through it. - "`__getitem__` raises `KeyError` on missing items." — depends on collection type. Sequences raise `IndexError`; dicts raise `KeyError`.

Common follow-up: "If I define `__getitem__` but not `__iter__`, can I still loop with `for x in obj`?" — Yes, via the legacy fallback. Python calls `obj[0]`, `obj[1]`, ... until you raise `IndexError`. Avoid relying on it; define `__iter__` explicitly.

Stand-out line: "`__getitem__` is the unified indexing hook — `obj[0]`, `obj[1:5]`, `obj['key']`, and `obj[a, b]` all arrive the same way; you decide how to interpret the key."

Q38 · Arithmetic dunders (`__add__`, `__mul__`) and their reflected siblings

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Microsoft India · Topic: Arithmetic dunders

The question (interviewer's verbatim phrasing):

How does Python dispatch `a + b` when `a` doesn't know how to add `b` ? What's the role of `__radd__` ?

30-second answer: Arithmetic dunder methods implement operators — `__add__` for `+`, `__mul__` for `*`, etc. Reflected versions (`__radd__`, `__rmul__`) are called when the left operand doesn't support the operation. Return `NotImplemented` (not `NotImplementedError`) to let Python try the reflected path.

Step-by-step thought process: - Standard pairs: `__add__` / `__radd__`, `__sub__` / `__rsub__`, `__mul__` / `__rmul__`, `__truediv__` / `__rtruediv__`, etc. - The flow: `a + b` calls `a.__add__(b)`. If it returns `NotImplemented`, Python calls `b.__radd__(a)`. If that also returns `NotImplemented`, `TypeError`. - In-place versions (`__iadd__` for `+=`) fall back to `__add__` if undefined. For mutable types, define `__iadd__` to mutate in place; for immutable, omit it. - Return `NotImplemented` (the singleton) for incompatible types — NOT `NotImplementedError` (the exception). The two are completely different.

Code:

```

class Money:
    def __init__(self, amount: int, currency: str = "INR"):
        self.amount, self.currency = amount, currency

    def __add__(self, other):
        if isinstance(other, Money):
            if self.currency != other.currency:
                raise ValueError("currency mismatch")
            return Money(self.amount + other.amount, self.currency)
        if isinstance(other, int):
            return Money(self.amount + other, self.currency)
        return NotImplemented

    def __radd__(self, other):
        return self.__add__(other)          # support int + Money

    def __mul__(self, n: int):
        if isinstance(n, int):
            return Money(self.amount * n, self.currency)
        return NotImplemented

    __rmul__ = __mul__                      # support int * Money

    def __repr__(self):
        return f"{self.amount} {self.currency}"

print(Money(500) + Money(300))             # 800 INR
print(Money(500) + 100)                    # 600 INR (via __add__)
print(100 + Money(500))                    # 600 INR (via __radd__)
print(Money(500) * 3)                      # 1500 INR
print(3 * Money(500))                      # 1500 INR (via __rmul__)

```

Edge cases to mention: - Returning `NotImplemented` (singleton) lets Python try the reflected operation; returning `False` or raising `TypeError` shuts down the dispatch. - `sum()` starts with `int(0)` as the initial value — without `__radd__` accepting int, `sum([Money(...)])` fails. Pass `sum(iterable, start=Money(0))` instead. - In-place ops on immutable wrappers: `__iadd__` returning a new object is fine but counterintuitive — `a += b` rebinds `a` even if `__iadd__` returns a new instance.

What NOT to say: - "Return `NotImplementedError` for unsupported types." — wrong. Return the singleton `NotImplemented` so Python can try the reflected operation. - "`__iadd__` always returns a new object." — wrong. For mutable types it should mutate self and return self.

Common follow-up: "`sum([Money(...)])` fails with `TypeError`. Why, and how do you fix it without changing the call site?" — `sum` starts with `int(0)` and tries `0 + Money`. Either pass `start=Money(0)` or define `__radd__` to accept `int`.

Stand-out line: "Operator overloading in Python is a two-way negotiation — `__add__` first, `__radd__` as the backup, `NotImplemented` as the signal to keep negotiating."

Q39 · `__ne__` — when to override**Tags:** Difficulty: Easy · Asked at: Razorpay, JP Morgan, Postman · Topic: **ne** derivation**The question (interviewer's verbatim phrasing):**

When should you define `__ne__` separately from `__eq__` ? Why is it almost never needed in modern Python?

30-second answer: Don't, usually. Since Python 3, `__ne__` defaults to the inverse of `__eq__` — define `__eq__` and `!=` works automatically. Override `__ne__` only when you have unusual semantics where "not equal" isn't simply "not equal to".

Step-by-step thought process: - Default: `object.__ne__(self, other)` calls `self.__eq__(other)` and inverts the result. So if `__eq__` returns `NotImplemented`, `__ne__` also returns `NotImplemented`. - Override only when `==` and `!=` aren't symmetric for your domain. Examples: NaN-like values where neither `==` nor `!=` is True; SQL-style NULL semantics where `NULL == NULL` and `NULL != NULL` are both unknown. - For 99% of code, define only `__eq__` and let `__ne__` default. Reviewers expect this. - Pre-Python 3, you HAD to define both — that's why older code still has redundant `__ne__` definitions floating around.

Code:

```
class Order:
    def __init__(self, id):
        self.id = id

    def __eq__(self, other):
        if not isinstance(other, Order): return NotImplemented
        return self.id == other.id

# No __ne__ — Python derives it
print(Order("X") == Order("X"))      # True
print(Order("X") != Order("Y"))      # True
print(Order("X") != Order("X"))      # False — derived

# When override is justified: SQL-NULL semantics
class SQLNull:
    def __eq__(self, other): return False      # NULL = anything → false
    def __ne__(self, other): return False      # NULL != anything → also false
    def __hash__(self): return 0
print(SQLNull() == SQLNull())          # False
print(SQLNull() != SQLNull())          # False — both branches are False
```

Edge cases to mention: - Defining `__ne__` separately from `__eq__` and forgetting to keep them in sync is a real bug — the two diverge silently. - Returning `NotImplemented` from `__eq__` and then defining `__ne__` to return `True` defeats Python's normal dispatch — usually wrong. - `NaN` is the only built-in that follows IEEE-754: `nan == nan` is False, `nan != nan` is True.

What NOT to say: - "`__ne__` is required if you define `__eq__`." — wrong since Python 3. Python derives `__ne__` from `__eq__` automatically. - "`__ne__` always returns the inverse of `__eq__`." — wrong for `NaN`-like or SQL-NULL semantics; both `==` and `!=` can be False.

Common follow-up: "Why does `nan != nan` return True in Python?" — IEEE-754 mandates that NaN is unequal to everything, including itself. Python implements this for `float('nan')` to stay consistent with the spec.

Stand-out line: "`__ne__` is a Python 2 holdover — define `__eq__` and stop. Override `__ne__` only when your equality is non-Boolean, like SQL NULL."

Q40 · `__format__` and f-string customisation

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Postman, Microsoft India · Topic: **format**

The question (interviewer's verbatim phrasing):

What does `__format__` do, and how do you extend it to support a domain-specific format spec?

30-second answer: `__format__(self, spec)` defines how your object responds to `format(obj, spec)` and to the `{:spec}` part of f-strings. The `spec` is the substring after the colon. Use it to add domain-specific format codes — currency formatting, fixed widths, custom number bases.

Step-by-step thought process: - The flow: `f"{obj:spec}"` calls `obj.__format__("spec")`. Plain `f"{obj}"` calls `obj.__format__("")`, which by default delegates to `str(obj)`. - You define the format-spec mini-language for your class. Examples: `f"{money:rupees}"` could format as `"₹500.00"`; `f"{money:words}"` could format as `"five hundred"`. - For numeric types, follow PEP 3101's spec language so callers can use familiar codes: `<`, `>`, `^` for alignment, `,` for thousands separator, `.2f` for fixed decimals. - An empty spec should fall back to `str(self)` — that's what callers expect when they don't specify.

Code:

```

class Money:
    def __init__(self, amount: int):
        self.amount = amount

    def __format__(self, spec: str) -> str:
        if spec == "":
            return str(self.amount)
        if spec == "rupees":
            return f"₹{self.amount:,}"
        if spec == "words":
            return f"{self.amount} rupees"
        if spec.startswith("."):
            # e.g. .2f
            return format(float(self.amount), spec)
        raise ValueError(f"unknown spec {spec!r} for Money")

    def __str__(self): return str(self.amount)

m = Money(150000)
print(f"{m}")           # 150000
print(f"{m:rupees}")    # ₹1,50,000-ish (locale-dependent comma; Python uses Western
print(f"{m:words}")     # 150000 rupees
print(f"{m:.2f}")       # 150000.00

```

Edge cases to mention: - Returning a non-str from `__format__` raises `TypeError: __format__ must return a str`. - Empty spec must work — don't raise on `f"{obj}"` because callers will use this form without thinking. - Numeric formatting locale (the `,` separator) uses Western convention by default. For Indian-style commas (1,50,000), you need the `babel` library or manual formatting.

What NOT to say: - "`__format__` is only called by `format()` ." — wrong. F-strings call it too: `f"{obj:spec}"` translates to `obj.__format__('spec')`. - "`__format__` can return any type." — wrong. It must return a str.

Common follow-up: "How would you make `f'{order:rupees}'` print as `₹1,50,000` while keeping the empty-spec form delegating to `str(obj)` ?" — Branch on `spec` : empty → `str(self)` ; `'rupees'` → format with comma thousands and a rupee glyph; raise on unknown.

Stand-out line: "`__format__` is your class's mini-language for presentation — extend the format-spec rules with domain codes so callers can write `f'{order:rupees}'` instead of `format_as_rupees(order.amount)` ."

Section E — Dunder Method Advanced

Q41 · `__call__` — making instances callable

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Cred, Postman · Topic: **call**

The question (interviewer's verbatim phrasing):

What does it mean for an instance to be callable? What's the use case that's cleaner with `__call__` than with a plain function plus a closure?

30-second answer: Defining `__call__(self, ...)` makes instances callable like functions. The instance becomes a stateful callable — useful for decorators with state, retry-policy objects, accumulators, and anything where you want `obj(x)` syntax but the implementation needs to remember things.

Step-by-step thought process: - Once `__call__` is defined, `obj(*args)` is shorthand for `obj.__call__(*args)`. The instance behaves like a function. - Powerful for stateful callbacks — a `RateLimiter()` instance tracks calls; a `Counter()` instance tallies; a `RetryPolicy()` instance carries config and gets invoked per attempt. - Class-based decorators: define `__init__` to accept the wrapped function, `__call__` to handle the invocation. Cleaner than nested closures for non-trivial decorators. - Instances of `partial`, `methodcaller`, `attrgetter` from `functools` are exactly this pattern.

Code:

```
class RetryPolicy:
    def __init__(self, max_attempts: int = 3, backoff: float = 0.5):
        self.max_attempts = max_attempts
        self.backoff = backoff
        self.attempts_made = 0

    def __call__(self, func, *args, **kwargs):
        import time
        for attempt in range(1, self.max_attempts + 1):
            self.attempts_made = attempt
            try:
                return func(*args, **kwargs)
            except Exception as e:
                if attempt == self.max_attempts: raise
                time.sleep(self.backoff * attempt)
        return None

def flaky():
    import random
    if random.random() < 0.8: raise RuntimeError("transient")
    return "ok"

policy = RetryPolicy(max_attempts=5)
# result = policy(flaky)          # would retry up to 5 times
print(callable(policy))          # True
```

Edge cases to mention: - Callable instances pass `callable(obj) == True`, but `inspect.isfunction(obj) == False`. Sphinx and some tools treat them differently. - Storing state in the callable can cause concurrency bugs — two threads calling the same instance racing on `self.attempts_made`. Use locks or thread-local state. - `__call__` should usually delegate or wrap, not duplicate logic that could live in a regular method.

What NOT to say: - "`__call__` is just for decorators." — narrow. Stateful callbacks, retry policies, accumulators all benefit. - "`callable(obj)` returns True only for functions." — wrong. Instances with `__call__` also pass.

Common follow-up: "If two threads share one `RetryPolicy` instance and call it concurrently, what's the bug?" — Race on `self.attempts_made`. Either keep no per-instance state, use a thread-local, or wrap calls in a lock.

Stand-out line: "`__call__` is how Python turns objects into functions — it's the foundation of every stateful decorator, retry policy, or accumulator that wants function-call syntax."

Q42 · `__getattr__` vs `__getattribute__`

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Microsoft India, Postman · Topic: `getattr` vs `getattribute`

The question (interviewer's verbatim phrasing):

What's the precise difference between `__getattr__` and `__getattribute__`, and when would I override each?

30-second answer: `__getattribute__` is called for **every** attribute access — it's the entry point for all dot-notation lookups. `__getattr__` is called **only** when normal attribute lookup has failed (raised `AttributeError`). Override `__getattr__` for lazy / proxy attributes; override `__getattribute__` only if you want to intercept every access (and accept the perf cost).

Step-by-step thought process: - `__getattribute__` is the regular path — runs the data-descriptor / instance-dict / class-attribute walk. Override only if you really want to wrap the whole thing. - `__getattr__` is the safety net — Python calls it **ONLY** when `__getattribute__` raised `AttributeError`. Cheap to define, used by proxies, ORMs, mock objects. - Inside `__getattribute__`, accessing `self.x` recurses — use `super().__getattribute__(name)` or `object.__getattribute__(self, name)` to avoid infinite loop. - Pattern: define `__getattr__` to lazily load attributes from disk / DB; define `__getattribute__` only for logging or sandboxing every access.

Code:

```

class LazyConfig:
    def __init__(self):
        self._loaded = False
        self._data = {}

    def __getattr__(self, name):
        # called only on missing attrs
        print(f"__getattr__({name})")
        if not self._loaded:
            self._data = {"region": "ap-south-1", "tier": "prod"}
            self._loaded = True
        if name in self._data:
            return self._data[name]
        raise AttributeError(name)

class TracedAccess:
    def __getattribute__(self, name):
        # called for EVERY access
        print(f"__getattribute__({name})")
        return super().__getattribute__(name)

c = LazyConfig()
print(c.region)           # __getattr__(region) -> 'ap-south-1'
print(c._loaded)          # True - no __getattr__ call (regular attr)

t = TracedAccess(); t.x = 1
print(t.x)                # __getattribute__(x) -> 1

```

Edge cases to mention: - `__getattribute__` is on EVERY access — slow if you put non-trivial logic in it. - Forgetting to call `super().__getattribute__` inside an override causes infinite recursion (`self.something` triggers the same method). - Magic method lookup bypasses `__getattribute__` for special methods — `len(obj)` doesn't go through your `__getattribute__("__len__")` override. Python finds dunders on the type directly.

What NOT to say: - "`__getattr__` runs on every attribute access." — wrong. Only on missing ones. - "`__getattribute__` is safe to override anytime." — wrong. Easy to introduce infinite recursion; expensive in hot loops.

Common follow-up: "Why doesn't your `__getattribute__` override intercept `len(obj)` calls?" — Special-method lookup goes directly to the type, bypassing instance `__getattribute__`. Same reason `print(obj)` doesn't log `__str__` lookups.

Stand-out line: "`__getattribute__` runs always; `__getattr__` runs only when the normal lookup gives up. Override the second, leave the first alone."

Q43 · `__setattr__` and `__delattr__`

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Microsoft India · Topic: `setattr` / `delattr`

The question (interviewer's verbatim phrasing):

How would you implement a class where attributes can be set during `__init__` but become read-only afterwards? Walk me through how `__setattr__` interacts with `__init__`.

30-second answer: `__setattr__(self, name, value)` intercepts every attribute write; `__delattr__(self, name)` intercepts every `del`. Use them to validate writes, log changes, enforce immutability, or auto-sync attributes to a backing store. Inside either, always assign via `super().__setattr__(...)` to avoid infinite recursion.

Step-by-step thought process: - The flow: `obj.x = 5` calls `obj.__setattr__("x", 5)`. The default implementation handles the data-descriptor check then writes to `__dict__`. - Common uses: read-only objects (`__setattr__` raises after `__init__`); audit objects (log every write); proxies (forward writes to a remote). - Inside `__setattr__`, doing `self.x = y` recurses back into your override. Use `super().__setattr__("x", y)` or `object.__setattr__(self, "x", y)` for the actual write. - `__delattr__` is parallel — intercepts `del obj.x`. Default writes to `__dict__`; override to forbid or log.

Code:

```

class Frozen:
    """Immutable after __init__."""
    def __init__(self, **fields):
        for k, v in fields.items():
            super().__setattr__(k, v)
        super().__setattr__("_frozen", True)

    def __setattr__(self, name, value):
        if getattr(self, "_frozen", False):
            raise AttributeError(f"can't set {name!r}: frozen")
        super().__setattr__(name, value)

    def __delattr__(self, name):
        raise AttributeError(f"can't del {name!r}: frozen")

class AuditedOrder:
    def __setattr__(self, name, value):
        print(f"set {name} = {value!r}")
        super().__setattr__(name, value)

f = Frozen(amount=500, currency="INR")
print(f.amount, f.currency)
try: f.amount = 700
except AttributeError as e: print(e)

a = AuditedOrder()
a.amount = 500                # set amount = 500
a.currency = "INR"           # set currency = 'INR'

```

Edge cases to mention: - Forgetting `super().__setattr__` and doing `self.__dict__[name] = value` works but bypasses data descriptors — be deliberate. - `__setattr__` runs for `self.x = y` even inside `__init__` — so a frozen-after-init pattern needs a flag like `_frozen` set last. - Auditing performance: logging every write on a hot-path object is noticeable in profilers.

What NOT to say: - "You can call `self.x = y` inside `__setattr__`." — wrong. It recurses. Use `super().__setattr__('x', y)`. - "`__setattr__` only runs for assignments outside `__init__`." — wrong. It runs for every assignment, including the ones in `__init__`.

Common follow-up: "Inside `__setattr__`, why is `self.__dict__[name] = value` sometimes acceptable, and why do reviewers prefer `super().__setattr__`?" — Direct dict write bypasses any data descriptors on the class. `super().__setattr__` goes through the descriptor protocol, which is usually what you want.

Stand-out line: "Override `__setattr__` to put a gatekeeper between callers and your instance dict — but always exit via `super().__setattr__` to avoid recursing into yourself."

Q44 · `__enter__` and `__exit__` — context managers

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Cred, Postman · Topic: Context manager protocol

The question (interviewer's verbatim phrasing):

Walk me through writing a class-based context manager. What does `__exit__` return, and when does returning `True` actually make sense?

30-second answer: `__enter__(self)` runs on `with` entry; `__exit__(self, exc_type, exc_val, exc_tb)` runs on exit (always, even on exception). The pair turns a class into a context manager. Return `True` from `__exit__` to suppress the exception; return `False` / `None` to let it propagate.

Step-by-step thought process: - The contract: `with ctx as x` is sugar for `x = ctx.__enter__()`, run body, then `ctx.__exit__(*sys.exc_info())` whether the body raised or not. - `__exit__` takes three args describing the exception (or `(None, None, None)` if clean exit). Returning `True` swallows the exception; returning `False` / `None` lets it propagate. - The bind-as `x` part is what `__enter__` returns — often `self`, sometimes a different object (a cursor, a transaction handle). - Cleaner alternative: `contextlib.contextmanager` decorator turns a generator function into a context manager. Use the class form when you need to inherit or carry state across multiple enter/exit cycles.

Code:

```

class DBTransaction:
    def __init__(self, conn): self.conn = conn

    def __enter__(self):
        print("begin")
        self.conn.execute("BEGIN")
        return self                # bind as `tx` in `with ... as tx`

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is None:
            print("commit"); self.conn.execute("COMMIT")
        else:
            print(f"rollback ({exc_type.__name__})"); self.conn.execute("ROLLBACK")
            return False            # don't suppress

class FakeConn:
    def execute(self, sql): pass

with DBTransaction(FakeConn()) as tx:
    pass                            # begin / commit

try:
    with DBTransaction(FakeConn()) as tx:
        raise ValueError("bad")    # begin / rollback (ValueError)
except ValueError:
    pass

```

Edge cases to mention: - `__exit__` returning True swallows the exception — sometimes correct (suppressing a specific type), often a bug. - Forgetting that `__exit__` runs on the success path too — make sure cleanup is idempotent. - For async code, use `__aenter__` / `__aexit__` — different dunder methods for `async with`.

What NOT to say: - "`__exit__` runs only if the body raised." — wrong. It runs on both paths. - "Returning True from `__exit__` is always correct." — wrong. It swallows the exception; usually a bug unless you're suppressing a specific type.

Common follow-up: "What's the difference between a class-based context manager and one written with `@contextlib.contextmanager` on a generator?" — The class form is reusable across `with` blocks and supports inheritance; the generator form is more concise for one-shot context. Use the class when you need state or reuse.

Stand-out line: "Context managers are the standard Python pattern for 'always run this cleanup' — the `__exit__` contract guarantees the call even on exception, and the return value decides whether the exception escapes."

Q45 · `__copy__` and `__deepcopy__`**Tags:** Difficulty: Medium · Asked at: Razorpay, JP Morgan, Microsoft India · Topic: copy and deepcopy**The question (interviewer's verbatim phrasing):**

When should you override `__copy__` and `__deepcopy__` ? What's the role of the `memo` dict in `deepcopy`?

30-second answer: `copy.copy(obj)` does a shallow copy — calls `__copy__()` if defined, else `__reduce_ex__`. `copy.deepcopy(obj)` does a deep copy — calls `__deepcopy__(memo)` if defined, else recurses through state. Override them when the default copying produces a wrong shape (shared mutable state, expensive deep walks, custom equivalence).

Step-by-step thought process: - Default behaviour: shallow copy creates a new instance with the same attribute values — but mutable values are shared. Deep copy recursively copies every reachable object. - `__copy__()` returns the shallow copy. `__deepcopy__(self, memo)` takes the memo dict (used to handle cyclic references) and returns the deep copy. Pass memo when you recurse into child deepcopies. - Common overrides: objects with weakrefs, file handles, network connections — you want to copy state but not duplicate the handle. Override `__copy__` to clone state but not the handle. - `copy.deepcopy` with the memo handles cyclic graphs — a self-referencing object survives a deep copy.

Code:

```
import copy

class Cart:
    def __init__(self, items: list):
        self.items = items
        self.metadata = {"created": "2026-05-30"}

    def __copy__(self):
        # shallow: new Cart, same lists / dicts
        c = Cart(self.items)
        c.metadata = self.metadata
        return c

    def __deepcopy__(self, memo):
        # deep: copy items + metadata
        c = Cart(copy.deepcopy(self.items, memo))
        c.metadata = copy.deepcopy(self.metadata, memo)
        return c

a = Cart(["tea", "coffee"])
b = copy.copy(a)
c = copy.deepcopy(a)

a.items.append("juice")
print(b.items)          # ['tea', 'coffee', 'juice'] – shared
print(c.items)          # ['tea', 'coffee'] – independent
```

Edge cases to mention: - Default deepcopy walks every reachable object — heavy on objects holding large data. Override to skip the heavy parts. - Forgetting to pass `memo` to recursive deepcopy calls breaks cycle handling. - `copy.deepcopy` of a class with `__slots__` works but needs care with `__reduce_ex__` — custom `__deepcopy__` is safer.

What NOT to say: - "`copy.copy` and `copy.deepcopy` always give the same result." — wrong. Shallow shares mutable state; deep recurses. - "`memo` is optional." — wrong if you have cycles. Without it, `deepcopy` infinite-loops on self-referential graphs.

Common follow-up: "Your `__deepcopy__` recurses into a child but forgets to pass `memo`. What's the bug?" — Cycle handling breaks: if the child holds a reference back to the parent, the recursion will loop until the stack blows.

Stand-out line: "Shallow copy duplicates references; deep copy duplicates the graph. Override only when the default is wrong — sharing a DB handle versus duplicating it, for instance."

Q46 · `__reduce__` and pickling

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Microsoft India · Topic: Pickling protocol

The question (interviewer's verbatim phrasing):

What hooks does pickle use to serialise an object, and which one should you override for the common case of 'exclude this unpicklable field'?

30-second answer: `__reduce__(self)` defines how an object is pickled — returns either a string (name of a global) or a tuple `(callable, args, state, ...)` that pickle uses to reconstruct the object. Implement it when default pickling doesn't work — instances with unpicklable references (locks, sockets, lambdas), or objects you want to deduplicate across processes.

Step-by-step thought process: - Default pickling: `pickle` walks `__dict__` (or `__getstate__` if defined), serialises class + state. Works for most plain Python objects. - `__getstate__()` returns the state; `__setstate__(state)` restores it. Use these for simple customisation — most "I want to exclude a field" cases. - `__reduce__()` is the heavy hammer — control the full reconstruction. Returns `(callable, args)` (call `callable(*args)` to recreate) or longer tuples with state, iterator, etc. - Don't pickle: file handles, sockets, locks, threads. Exclude them via `__getstate__`.

Code:

```
import pickle
import threading

class Connection:
    def __init__(self, url):
        self.url = url
        self._lock = threading.Lock()           # not picklable

    def __getstate__(self):
        state = self.__dict__.copy()
        del state["_lock"]                       # strip the lock
        return state

    def __setstate__(self, state):
        self.__dict__.update(state)
        self._lock = threading.Lock()           # recreate on the other side

c = Connection("postgres://localhost")
data = pickle.dumps(c)
c2 = pickle.loads(data)
print(c2.url)                                  # postgres://localhost
print(type(c2._lock).__name__)                 # lock — fresh
```

Edge cases to mention: - `__getstate__` returning `None` raises in some Python versions because pickle interprets it as "no state". - Lambdas, nested functions, locks, sockets — none of them pickle. Use `__getstate__` to strip; use `cloudpickle` for the cases where you genuinely need it. -

`__reduce__` returning a non-tuple raises a confusing error — stick to the tuple form unless you really know what you're doing.

What NOT to say: - "`__reduce__` is the simple knob." — wrong. It's the heavy hammer; `__getstate__` / `__setstate__` is the friendly pair. - "Pickle calls `__init__` on the other side." — wrong. It bypasses `__init__` and uses `__setstate__` (or `__reduce__`'s callable) to reconstruct.

Common follow-up: "If your class holds a `threading.Lock`, how do you keep it picklable?" — Strip the lock in `__getstate__`; recreate it in `__setstate__`. Locks themselves don't survive serialisation.

Stand-out line: "`__getstate__` and `__setstate__` are the friendly knobs — exclude what can't pickle, recreate it on the other side. Reach for `__reduce__` only when you need to control the construction call itself."

Q47 · `__slots__` — second-pass details

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Cred, Microsoft India · Topic: **slots** advanced

The question (interviewer's verbatim phrasing):

We covered the memory win earlier. Now walk me through the second-pass details — what happens with `__weakref__`, with `__dict__` in slots, and with multiple inheritance?

30-second answer: `__slots__` is a class-level tuple declaring exactly which attributes the instance can hold. It removes `__dict__` and `__weakref__` by default. Adding `__weakref__` or `__dict__` back to the slots restores those features. Multiple inheritance with non-empty slots is restricted — at most one base can have slot layout.

Step-by-step thought process: - Beyond the memory win covered earlier: slots also speed up attribute access (descriptor-level, no dict lookup). - `__slots__ = ("__dict__",)` gives you slots AND a dict — defeats the memory win but allows dynamic attrs alongside declared slots. - `__slots__ = ("__weakref__", "x", "y")` restores weakref support for slotted instances. - Multiple inheritance: if two parents have non-empty slots, Python raises `TypeError: multiple bases have instance lay-out conflict`. Workaround: empty slots in one parent, or use composition.

Code:


```

import weakref

class A:
    __slots__ = ("x",)

class B:
    __slots__ = ("y",)

# class C(A, B): pass          # TypeError – layout conflict

class WithWeakref:
    __slots__ = ("x", "__weakref__")    # restore weakref
    def __init__(self): self.x = 0

class WithBoth:
    __slots__ = ("x", "__dict__")      # slot + dynamic
    def __init__(self): self.x = 0

w = WithWeakref()
ref = weakref.ref(w)
print(ref() is w)                # True

both = WithBoth()
both.dynamic = "ok"              # allowed
print(both.__dict__)             # {'dynamic': 'ok'}
print(both.x)                    # 0 – from slot

```

Edge cases to mention: - Inheriting from a class without `__slots__` reintroduces `__dict__` in the subclass — even if subclass declares `__slots__`. - Class variables with the same name as a slot are masked (slot wins) — `__slots__ = ("x",); x = 5` raises `ValueError` at class creation. - `__slots__` without `__dict__` blocks monkey-patching — be careful when retrofitting.

What NOT to say: - "`__slots__` always blocks weakref." — wrong. Add `__weakref__` to slots to restore it. - "You can multiply-inherit slotted classes freely." — wrong. At most one base can have non-empty slots.

Common follow-up: "What's the workaround for inheriting from two classes that both want non-empty slots?" — Either redesign to composition or make one base's slots empty. There's no language-level fix.

Stand-out line: "`__slots__` is fine-grained control over your instance layout — add `__weakref__` to restore weak references, add `__dict__` to allow extras, and watch for the multiple-inheritance layout-conflict trap."

Q48 · `__del__` — finalisers and why you usually skip them

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Microsoft India · Topic: `del` finalisers

The question (interviewer's verbatim phrasing):

Why is `__del__` a bad place to put cleanup logic? What's the modern alternative?

30-second answer: `__del__(self)` runs when the object is about to be garbage-collected. It's tempting to use for cleanup, but it has problems — no guarantee on when it runs, can be skipped on interpreter shutdown, breaks ref-cycle GC if it touches other objects. Prefer context managers or `weakref.finalize` for cleanup.

Step-by-step thought process: - The timing is up to the GC — usually immediate when refcount hits zero (CPython), but not in PyPy or under reference cycles. - Exceptions inside `__del__` are silently caught and printed to stderr — they don't propagate. - Reference cycles where one of the objects has `__del__` were unfinalisable before Python 3.4. Python 3.4+ handles them but at the cost of running `__del__` in an unspecified order — risky if order matters. - The modern alternative: `weakref.finalize(obj, callback, *args)` registers a callback to run when `obj` is collected. Cleaner semantics, decoupled from the object's class.

Code:

```
import weakref

class WithDel:
    def __init__(self, name): self.name = name
    def __del__(self):
        print(f"__del__ {self.name}")          # may not run on shutdown

class WithFinalize:
    def __init__(self, name):
        self.name = name
        weakref.finalize(self, print, f"finalised {name}")

a = WithDel("A"); del a                      # __del__ A — usually
b = WithFinalize("B"); del b                  # finalised B — reliable
```

Edge cases to mention: - Code inside `__del__` runs in a fragile state — module globals may already be `None` if it runs at shutdown. `print` itself can fail. - `__del__` blocks reference-cycle GC in old Python versions — modern Python handles it, but performance hit remains. - Subclasses overriding `__del__` need `super().__del__()` to maintain parent cleanup — common omission.

What NOT to say: - "`__del__` runs immediately when the last reference drops." — only in CPython, only outside cycles. PyPy delays it. - "Exceptions in `__del__` propagate." — wrong. They're caught and printed to stderr.

Common follow-up: "What does `weakref.finalize` give you that `__del__` does not?" — Deterministic registration, decoupled from the class, predictable order, no class-level state, and no impact on reference-cycle GC.

Stand-out line: "`__del__` is the finaliser nobody should reach for first — context managers handle 95% of cleanup needs, and `weakref.finalize` handles the rest with sharper semantics."

Q49 · `__init_subclass__` vs metaclasses — the modern replacement

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Microsoft India · Topic: Metaclass vs `__init_subclass__`

The question (interviewer's verbatim phrasing):

When would you ever write a metaclass instead of using `__init_subclass__` ? What's the 80/20 split between the two?

30-second answer: Metaclasses customise class creation — they're "the class of a class". They're powerful and rarely needed. `__init_subclass__` (PEP 487, Python 3.6) covers 80% of metaclass use cases — subclass registration, validation, default behaviour — with normal-looking class syntax. Reach for a metaclass only when you need to customise the class object itself (its `__call__`, its `__getattr__`, etc.).

Step-by-step thought process: - Metaclass: `class Cls(metaclass=Meta)`. `Meta` is itself a class (usually inheriting from `type`). Its `__new__` is called to build `Cls`. - Use cases that metaclasses uniquely solve: classes that should behave differently at the class level (e.g. `MyClass.x` does something custom even before instantiation). - `__init_subclass__` covers: registering subclasses, validating that subclasses define required methods, adding default attributes. Cleaner and more local than a metaclass. - ABCs use metaclasses (`ABCMeta`) to implement abstract method checking. Generic uses metaclasses for type parameter handling. Most application code never needs them.

Code:

```
# Old metaclass way
class RegisterMeta(type):
    registry = {}
    def __new__(mcs, name, bases, ns):
        cls = super().__new__(mcs, name, bases, ns)
        if bases:
            # don't register the abstract base
            RegisterMeta.registry[name] = cls
        return cls

class HandlerOld(metaclass=RegisterMeta): pass
class A1(HandlerOld): pass
class A2(HandlerOld): pass
print(list(RegisterMeta.registry))          # ['A1', 'A2']

# Modern __init_subclass__ way — same effect, no metaclass jargon
class HandlerNew:
    registry: dict = {}
    def __init_subclass__(cls, **kw):
        super().__init_subclass__(**kw)
        HandlerNew.registry[cls.__name__] = cls

class B1(HandlerNew): pass
class B2(HandlerNew): pass
print(list(HandlerNew.registry))           # ['B1', 'B2']
```

Edge cases to mention: - Metaclass conflicts: if you inherit from two classes with different metaclasses, you get `TypeError: metaclass conflict`. Resolve by writing a unifying metaclass that inherits both. - `__init_subclass__` runs ONCE per subclass at class creation, not per instance. - Don't write a metaclass when `__init_subclass__` would do — reviewers will appreciate the simpler tool.

What NOT to say: - "Metaclasses and `__init_subclass__` do the same thing." — wrong. Metaclasses customise class creation itself; `__init_subclass__` runs on subclasses of an existing class. - "You need a metaclass for plugin registration." — wrong post-3.6. `__init_subclass__` handles it cleanly.

Common follow-up: "If you inherit from two classes with different metaclasses, what error do you get and how do you resolve it?" — `TypeError: metaclass conflict`. You write a unifying metaclass that inherits from both, or you refactor so one of the bases doesn't need a metaclass.

Stand-out line: "Metaclasses are 'classes of classes' — necessary for some library internals, almost never needed in application code. `__init_subclass__` covers most of what people used to reach for metaclasses for."

Q50 · `__bytes__`, `__int__`, `__float__` — type coercion dunders

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Microsoft India · Topic: Coercion dunders

The question (interviewer's verbatim phrasing):

What's the difference between `__int__` and `__index__`, and why does Python have both?

30-second answer: `__bytes__(self)` defines what `bytes(obj)` returns. `__int__`, `__float__`, `__complex__` define `int(obj)`, `float(obj)`, `complex(obj)`. Use them to make your domain types coercible to standard Python types — useful for serialisation, math operations, and integration with code that expects ints / floats.

Step-by-step thought process: - Each coercion has a clear contract: `__int__` returns an int, `__float__` returns a float, `__bytes__` returns a bytes object. Wrong return types raise. - `__index__` is the special integer hook for indexing operations (`list[obj]`, `range(obj)`, `bin(obj)`). Implement it instead of (or alongside) `__int__` if your type should be usable as an index. - `__round__` and `__floor__` / `__ceil__` / `__trunc__` are the math-coercion dunder methods for `round()`, `math.floor()`, etc. - A common pattern for `Money`-like types: implement `__int__` to return the rupee count, `__float__` to return the decimal value. Helps when math libraries need a plain number.

Code:

```
import math

class Money:
    def __init__(self, paise: int):
        self.paise = paise

    def __int__(self):
        return self.paise // 100          # rupees as int

    def __float__(self):
        return self.paise / 100           # rupees as float

    def __bytes__(self):
        return self.paise.to_bytes(8, "big")

    def __index__(self):                   # for list[Money(...)] use as index
        return self.paise

    def __round__(self, ndigits=0):
        return round(float(self), ndigits)

m = Money(15050)                          # 150.50 rupees
print(int(m), float(m))                  # 150 150.5
print(bytes(m).hex())                    # 00000000000003ac2
print(round(m, 1))                       # 150.5
nums = [10, 20, 30, 40, 50]
print(nums[Money(2)])                    # uses __index__ -> nums[2] -> 30 (paise=2)
```

Edge cases to mention: - `__int__` and `__index__` aren't the same — `__index__` is for genuine integer-like indexing. Float types have `__int__` but NOT `__index__` (you can't `list[1.5]`). - `bool(obj)` falls back to `__len__()` if `__bool__` undefined, NOT to `__int__()`. - `__bytes__` should produce a serialisation appropriate for round-tripping — undefined format is rarely useful.

What NOT to say: - "`__int__` and `__index__` are aliases." — wrong. `__index__` is specifically for genuine integer-like indexing operations. - "All numeric types implement both." — wrong. `float` implements `__int__` but NOT `__index__` — you can't `list[1.5]`.

Common follow-up: "Why does `bool` implement `__index__`?" — Because `True` and `False` are integers in Python's type hierarchy, and `list[True]` should be valid `list[1]`. Same applies to `range(True)`, `bin(True)`, etc.

Stand-out line: "Coercion dunder let your types speak the dialect of standard Python — implement `__int__` and `__index__` for integer-like values, `__float__` for numeric ones, and your domain class slots into every function that takes a plain number."

Section F — Dataclasses + Modern OOP

Q51 · `@dataclass` basics

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Postman, Freshworks · Topic: `@dataclass` basics

The question (interviewer's verbatim phrasing):

When would you reach for `@dataclass`? What does it generate for you, and what does it not?

30-second answer: `@dataclass` is a decorator that auto-generates `__init__`, `__repr__`, `__eq__`, and (optionally) `__lt__` / `__hash__` from annotated class attributes. Use it for any class that's mostly state with a sensible default behaviour — the alternative is writing 30 lines of boilerplate by hand.

Step-by-step thought process: - Decorate a class with type-annotated attributes; the decorator inspects the annotations and generates dunder. - Default flags: `init=True`, `repr=True`, `eq=True`, `order=False`, `unsafe_hash=False`, `frozen=False`. Override per-class as needed. - The generated `__init__` takes args in declaration order. The generated `__eq__` compares all fields as a tuple. - Type annotations are documentation + (optionally) checked by mypy — Python itself doesn't enforce them at runtime.

Code:

```

from dataclasses import dataclass

@dataclass
class Order:
    order_id: str
    amount: int
    currency: str = "INR"          # default value

o1 = Order("RZP-1", 500)
o2 = Order(order_id="RZP-1", amount=500)
print(o1)                        # Order(order_id='RZP-1', amount=500, currency='INR')
print(o1 == o2)                  # True - generated __eq__

# What @dataclass generated, schematically:
# def __init__(self, order_id, amount, currency="INR"): ...
# def __repr__(self): return f"Order(order_id={self.order_id!r}, ...)"
# def __eq__(self, other): return (self.order_id, self.amount, ...) == ...

```

Edge cases to mention: - Default values for mutable types raise: `items: list = []` raises `ValueError`. Use `field(default_factory=list)` (next Q). - Fields without defaults must come before fields with defaults — same rule as function arguments. - Inheriting from a dataclass with a default into a subclass with a non-default field violates the same rule.

What NOT to say: - "`@dataclass` adds class methods you can call." — wrong. It generates dunders (`__init__`, `__repr__`, `__eq__`), not arbitrary methods. - "Type annotations on dataclass fields are enforced at runtime." — wrong. Python ignores them; mypy reads them.

Common follow-up: "Why does `items: list = []` raise at class-creation time inside a dataclass?" — Mutable default would be shared across instances. Python raises `ValueError` to force you to use `field(default_factory=list)`.

Stand-out line: "`@dataclass` is the lazy man's `__init__` / `__repr__` / `__eq__` — it generates the boilerplate from type annotations, so the class body stays a declarative shape."

Q52 · `frozen=True` — immutable dataclasses

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Cred, Microsoft India · Topic: Frozen dataclasses

The question (interviewer's verbatim phrasing):

What does `@dataclass(frozen=True)` give you over a plain dataclass? Why is it the closest Python gets to value-object semantics?

30-second answer: `@dataclass(frozen=True)` makes the dataclass immutable — `__setattr__` and `__delattr__` raise after construction. Frozen dataclasses are hashable by default (using all fields), so they work as dict keys and set members. Use them for value-object semantics — coordinates, IDs, snapshots.

Step-by-step thought process: - The decorator implements `__setattr__` and `__delattr__` that raise `FrozenInstanceError`. Mutation attempts fail at runtime. - Frozen + `eq=True` → hashable. The generated `__hash__` is based on all fields, consistent with `__eq__`. - Frozen dataclasses are the closest Python gets to records / immutable value objects — common in functional-style code, event sourcing, message types. - To "modify" a frozen dataclass, use `dataclasses.replace(instance, field=new_value)` — returns a new instance with the field changed.

Code:

```
from dataclasses import dataclass, replace, FrozenInstanceError

@dataclass(frozen=True)
class Coord:
    lat: float
    lng: float

c1 = Coord(12.9716, 77.5946)          # Bangalore
try:
    c1.lat = 0.0
except FrozenInstanceError as e:
    print(e)                          # cannot assign to field 'lat'

# Hashable — works as dict key
locations = {c1: "Bangalore"}
print(locations[Coord(12.9716, 77.5946)])  # Bangalore (equal by value)

# "Modify" — replace returns a new instance
c2 = replace(c1, lat=13.0827)
print(c1, c2)
```

Edge cases to mention: - Frozen only blocks the dataclass-managed `__setattr__` — you can still poke `__dict__` directly with `object.__setattr__`. Convention only, not security. - A frozen dataclass holding a mutable field (`items: list`) is shallowly frozen — the list itself can still be mutated. - `unsafe_hash=True` plus `frozen=False` is the rare "I want a custom hash even though the class is mutable" — risky, name reflects it.

What NOT to say: - "Frozen dataclasses are deeply immutable." — wrong. A frozen dataclass holding a list can still see the list mutated. - "Frozen dataclasses can't be hashed." — wrong. Frozen with `eq=True` makes them hashable; that's the value-object win.

Common follow-up: "How would you 'update' one field on a frozen dataclass without mutating the original?" — `dataclasses.replace(instance, field=new_value)` returns a fresh instance with the field changed. Functional style.

Stand-out line: " `frozen=True` gives you value-object semantics — hashable, equality-by-content, dict-key-safe. To 'update' one, you build a new one via `replace` ."

Q53 · `field()` with defaults and factories

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Postman · Topic: `field()`

The question (interviewer's verbatim phrasing):

What does `dataclasses.field()` let you do that bare default values do not? Walk me through `default_factory` , `init=False` , and `compare=False` .

30-second answer: `field()` lets you customise a dataclass field — set a default factory for mutable defaults, exclude a field from `__init__` / `__repr__` / `__eq__` , or attach metadata. The factory pattern is the canonical fix for "mutable default arguments" inside dataclasses.

Step-by-step thought process: - Mutable defaults rule: `items: list = []` raises because Python can't tell whether you meant "every instance shares this list" (almost always wrong) or not. Solution: `items: list = field(default_factory=list)` — factory called per instance. - `field(init=False)` excludes the field from the generated `__init__` — useful for computed or post-init fields. - `field(repr=False)` hides a field from `__repr__` — useful for secrets, large blobs, internal caches. - `field(compare=False)` excludes from `__eq__` and `__hash__` — for fields that are metadata, not identity.

Code:

```

from dataclasses import dataclass, field

@dataclass
class Cart:
    user_id: str
    items: list[str] = field(default_factory=list)      # per-instance fresh list
    metadata: dict = field(default_factory=dict, repr=False, compare=False)
    _cached_total: int | None = field(default=None, init=False, repr=False)

c1 = Cart("user-1")
c2 = Cart("user-1")
c1.items.append("tea")
print(c1.items, c2.items)          # ['tea'] [] — independent
print(c1 == c2)                    # False — items differ

# metadata excluded from compare
c3 = Cart("user-2", items=["coffee"], metadata={"big": 1})
c4 = Cart("user-2", items=["coffee"], metadata={"other": 2})
print(c3 == c4)                    # True — metadata ignored

```

Edge cases to mention: - Calling `field()` without `default` or `default_factory` makes the field required. - `field(default_factory=lambda: [])` works but is less readable than `field(default_factory=list)`. - `field(init=False)` fields must have a default or `default_factory`, or be set in `__post_init__`.

What NOT to say: - "`default_factory` is for any default value." — wrong. Use it specifically for mutable defaults or for values that need fresh computation per instance. - "`init=False` fields don't appear in `__repr__`." — wrong. `init=False` only excludes from `__init__`; use `repr=False` to hide from repr.

Common follow-up: "What's the right pattern for a field that should be excluded from equality comparison — say, a created-at timestamp?" — `field(compare=False)`. The field still appears in `__init__`, `__repr__`, and the instance dict, but `__eq__` ignores it.

Stand-out line: "`field()` is dataclass's escape hatch — mutable defaults via factories, excluded fields, metadata. Most non-trivial dataclasses use it at least once."

Q54 · `__post_init__` — running code after `__init__`

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Postman, Cred · Topic: `post_init`

The question (interviewer's verbatim phrasing):

What does `__post_init__` exist for in a dataclass, and what's an `InitVar`?

30-second answer: `__post_init__(self)` runs at the end of the generated `__init__` — used to validate fields, compute derived values, or normalise inputs after the fields are assigned. It's the dataclass equivalent of "the rest of `__init__`" that the decorator couldn't generate for you.

Step-by-step thought process: - The flow: generated `__init__` runs and assigns every field, then calls `self.__post_init__()`. Define it to add post-construction logic. - Common uses: validate that combinations of fields are coherent (`end_date >= start_date`), compute a derived field (`full_name = first + " " + last`), normalise (`email = email.lower()`). - For `init=False` fields, set them in `__post_init__` — they're not assigned by the generated `__init__`. - `InitVar[X]` declares a field that's an `__init__` argument but NOT stored as an instance attribute — it's passed through to `__post_init__` and then forgotten. Useful for one-shot construction parameters.

Code:

```
from dataclasses import dataclass, field, InitVar

@dataclass
class Order:
    amount: int
    discount_pct: InitVar[int] = 0          # passed to __post_init__, not stored
    final_amount: int = field(init=False)  # computed

    def __post_init__(self, discount_pct):
        if self.amount <= 0:
            raise ValueError("amount must be positive")
        self.final_amount = self.amount - (self.amount * discount_pct // 100)

o = Order(amount=1000, discount_pct=10)
print(o)                                # Order(amount=1000, final_amount=900)
print(hasattr(o, "discount_pct"))        # False — InitVar, not stored

try:
    Order(amount=-5)
except ValueError as e:
    print(e)
```

Edge cases to mention: - `__post_init__` runs after every field has been assigned — you can't use it to alter `__init__`'s argument list. - `InitVar` fields aren't included in `__eq__`, `__repr__`, or the instance dict — they're construction-only. - Inheritance: a subclass that overrides `__post_init__` needs `super().__post_init__()` to keep the parent's logic.

What NOT to say: - "`__post_init__` runs before `__init__`." — wrong. It runs at the end of the generated `__init__`. - "`InitVar` fields are normal attributes." — wrong. They're construction-only — not stored on the instance.

Common follow-up: "If both parent and child dataclasses override `__post_init__`, what's the convention for cooperative chaining?" — Call `super().__post_init__(...)` first, forward whatever `InitVar` args the parent expects. Forgetting it silently skips the parent's logic.

Stand-out line: "`__post_init__` is your hook to run code after the auto-generated `__init__` — validation, derivation, normalisation — without giving up the boilerplate-free field declarations."

Q55 · Dataclass inheritance

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Microsoft India · Topic: Dataclass inheritance

The question (interviewer's verbatim phrasing):

What constraints does dataclass inheritance impose on field order, and what's the Python 3.10+ way out?

30-second answer: A dataclass can inherit from another dataclass — the generated `__init__` includes fields from the parent first, then the child. The default-value rule still applies (no defaulted field can be followed by a non-defaulted one), which makes some inheritance hierarchies awkward.

Step-by-step thought process: - Order: parent fields first in `__init__` signature, child fields after. Reads naturally for additive subclasses. - The constraint: every field with a default must come after every field without. If parent has a defaulted field and child adds a non-defaulted one, you get `TypeError: non-default argument follows default argument`. - Workaround: use `field(default=...)` or `kw_only=True` (Python 3.10+) so order doesn't matter for kwargs. - Equality between parent and child: `Parent(...) == Child(...)` is False by default — `__eq__` checks the class as well. To compare across types, override `__eq__` manually.

Code:

```

from dataclasses import dataclass, field

@dataclass
class Order:
    order_id: str
    amount: int

@dataclass
class RazorpayOrder(Order):
    razorpay_payment_id: str
    is_test: bool = False

ro = RazorpayOrder("RZP-1", 500, "pay_xyz")
print(ro)
# RazorpayOrder(order_id='RZP-1', amount=500, razorpay_payment_id='pay_xyz', is_test=False)

# The defaulted-field problem
try:
    @dataclass
    class Bad(Order):
        amount: int = 0                # parent's amount becomes defaulted
        currency: str                  # ← non-default after default – fails
except TypeError as e:
    print(e)

# Python 3.10+ fix: kw_only
@dataclass(kw_only=True)
class FixedRazorpay(Order):
    razorpay_payment_id: str
    is_test: bool = False
# Order's fields stay positional; new fields are keyword-only.

```

Edge cases to mention: - `Parent(...) == Child(...)` is False even if fields match — `__eq__` checks `type(self) is type(other)`. - Overriding a parent field in the child's class body must repeat the annotation: `amount: int = 0` (not just `amount = 0`). - `kw_only=True` for the whole class or per-field (`field(kw_only=True)`) avoids the default-ordering trap — Python 3.10+ only.

What NOT to say: - "Fields with defaults can come before fields without." — wrong. Same rule as function args; defaulted comes last. - "`Parent(...) == Child(...)` compares by field values." — wrong. The generated `__eq__` also checks `type(self) is type(other)`.

Common follow-up: "Walk me through `kw_only=True` — when do you reach for it, and what does it solve at the dataclass level?" — When a parent has defaulted fields and the child needs to add non-defaulted ones. `kw_only=True` (per-field or whole-class) makes the new fields keyword-only, so positional order no longer applies.

Stand-out line: "Dataclass inheritance is additive at the field level — parent fields come first, child fields after. The default-after-non-default rule still bites; `kw_only=True` is the Python 3.10 way out."

Q56 · Dataclasses vs NamedTuple vs TypedDict vs Protocol

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Microsoft India, Postman · Topic: Shape primitives compared

The question (interviewer's verbatim phrasing):

Compare `@dataclass`, `NamedTuple`, `TypedDict`, and `Protocol`. Which one would you pick for an immutable event type, and why?

30-second answer: All four describe shapes. `dataclass` is a mutable class with methods, fields, inheritance. `NamedTuple` is an immutable tuple with named fields — lightweight and indexable. `TypedDict` is a static-typing-only shape for dicts — exists for mypy, has zero runtime behaviour. `Protocol` is structural-typing for ducks — "anything with these methods" is acceptable.

Step-by-step thought process: - `@dataclass`: full class with `__init__`, `__repr__`, `__eq__`. Mutable by default. Use for typical "object with fields". - `NamedTuple` (`typing.NamedTuple` or `collections.namedtuple`): immutable, tuple-subclass, indexable by position and name. Use for fixed-size records where you want tuple semantics (unpacking, sorting). - `TypedDict`: NOT a runtime type — it's a typing construct. `MyDict = TypedDict("MyDict", {"a": int})` lets mypy check that you pass dicts with the right keys. At runtime it's just `dict`. - `Protocol`: structural typing. `class Drawable(Protocol): def draw(self) -> None: ...` — anything with a `draw` method matches, no inheritance needed. Mypy enforces; runtime checks via `@runtime_checkable`.

Code:

```

from dataclasses import dataclass
from typing import NamedTuple, TypedDict, Protocol, runtime_checkable

@dataclass
class OrderDC:
    id: str
    amount: int

class OrderNT(NamedTuple):
    id: str
    amount: int

class OrderTD(TypedDict):
    id: str
    amount: int

@runtime_checkable
class OrderP(Protocol):
    id: str
    amount: int

# Dataclass — mutable, methods
o1 = OrderDC("X", 500); o1.amount = 600      # ok
# NamedTuple — immutable, tuple
o2 = OrderNT("X", 500); print(o2[0], o2.id) # 'X' 'X' — indexable + named
# TypedDict — just a dict at runtime
o3: OrderTD = {"id": "X", "amount": 500}    # ordinary dict
print(type(o3) is dict)                     # True
# Protocol — structural
print(isinstance(o1, OrderP))                # True via runtime_checkable

```

Edge cases to mention: - `NamedTuple` instances support all tuple operations — `tuple(o2)` works, `o2 == ("X", 500)` is True (gotcha for equality with plain tuples). - `TypedDict` keys can be marked optional with `total=False` (3.8+) or per-key with `NotRequired` (3.11+). - `Protocol` matches by structure — easy to satisfy accidentally, which mypy strictness mitigates.

What NOT to say: - "They're all interchangeable." — wrong. They have very different runtime behaviour. - " `TypedDict` enforces dict keys at runtime." — wrong. It's a typing-only construct; runtime sees a plain dict.

Common follow-up: "Why might you pick `NamedTuple` over a frozen dataclass for a fixed-size record?" — `NamedTuple` is tuple-subclass, so unpacking works, it's lighter in memory, and it's indexable by position. Frozen dataclass is more flexible but heavier.

Stand-out line: "Dataclass for objects with methods, NamedTuple for immutable records, TypedDict for typed dicts, Protocol for structural duck typing — same shape problem, four idiomatic answers depending on what you need at runtime versus type-check time."

Q57 · When NOT to use a dataclass

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Cred, Postman · Topic: When NOT to use dataclasses

The question (interviewer's verbatim phrasing):

When is `@dataclass` the wrong tool? Give me two scenarios where a plain class reads better.

30-second answer: Skip dataclasses when you need full control over `__init__` (complex argument parsing, multiple constructors), heavy validation logic, or non-trivial inheritance with non-additive fields. Also skip them for objects that are mostly behaviour — service classes, controllers, processors — where the fields are an implementation detail.

Step-by-step thought process: - The win of `@dataclass` is replacing boilerplate. Once you need to add custom `__init__` logic, custom `__repr__`, or skip the equality check, the boilerplate-replacement value drops. - Performance: dataclass `__init__` is slightly slower than a hand-written one (generated code, function calls). Matters only for hot loops creating millions of instances. - Inheritance edge cases (changing field order, overlapping defaults, abstract parents) sometimes make dataclasses awkward — a plain class is cleaner. - Service / controller / handler classes — usually 1-3 dependencies in `__init__`, no equality semantics, no `__repr__` beyond default — `@dataclass` adds noise without value.

Code:


```
# Good dataclass candidate – pure shape
from dataclasses import dataclass

@dataclass
class Address:
    line1: str
    city: str
    pincode: str

# Bad dataclass candidate – service with one dep, lots of behaviour
class PaymentService:
    def __init__(self, gateway):
        self.gateway = gateway
    def charge(self, order): ...
    def refund(self, payment_id): ...
    def reconcile(self): ...

# Bad dataclass candidate – complex __init__
class FlexibleOrder:
    def __init__(self, *args, **kwargs):
        if "from_json" in kwargs:
            self._init_from_json(kwargs["from_json"])
        elif "from_csv" in kwargs:
            self._init_from_csv(kwargs["from_csv"])
        else:
            self._init_from_fields(*args)
    # @dataclass can't generate this kind of __init__
```

Edge cases to mention: - Adding `@dataclass` to a class that already has a custom `__init__` silently overrides yours unless you pass `init=False`. - Heavy `__post_init__` logic means the class is no longer "just a shape" — consider whether a plain class would be clearer. - Dataclasses with many fields and default factories slow down construction — measure if it matters.

What NOT to say: - "`@dataclass` works for every class." — wrong. Service classes, controllers, anything with complex `__init__` logic is cleaner as a plain class. - "`@dataclass` is always faster." — wrong. The generated `__init__` has a small overhead vs hand-written.

Common follow-up: "What signal in a class body tells you the dataclass decorator is no longer paying for itself?" — Heavy `__post_init__`, custom `__init__` overrides, multiple constructors, or a class that's mostly behaviour with one or two fields.

Stand-out line: "`@dataclass` is for shapes; once you need a custom constructor or complex behaviour, the boilerplate it saved is no longer the bottleneck — write the class by hand and read it once."

Q58 · `attrs` vs `@dataclass` — when each wins

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JP Morgan, Microsoft India · Topic: `attrs` vs `dataclass`

The question (interviewer's verbatim phrasing):

When would you reach for `attrs` over the stdlib `dataclass` ? What about `pydantic` ?

30-second answer: `dataclasses` are in the stdlib — zero dependency, simple, covers 90% of cases. `attrs` is the third-party library that inspired dataclasses — more features (validators, converters, slots-by-default, more flexible inheritance). Reach for `attrs` when you need its extras; default to `dataclass` .

Step-by-step thought process: - `dataclass` advantages: stdlib, simple syntax (`@dataclass`), no install. - `attrs` advantages: built-in validators (`@attrs.validate` or `validator=`), converters (apply on assignment), `slots=True` by default, easier custom equality, `evolve` for replace-like updates. - The newer `attrs.define` (since `attrs` 21.1) is the modern API — uses `@define` instead of `@attrs.s` . Cleaner. - `pydantic` is yet another option for data validation — heavier, more features (parsing, JSON serialisation, schema generation), used in FastAPI ecosystem.

Code:

```
# Stdlib dataclass
from dataclasses import dataclass

@dataclass(frozen=True, slots=True)           # slots=True since Python 3.10
class Order:
    id: str
    amount: int

# attrs version — more features but adds a dependency
# import attrs
# @attrs.define(frozen=True, slots=True)
# class OrderAttrs:
#     id: str
#     amount: int = attrs.field(validator=attrs.validators.gt(0))

# pydantic version — runtime validation + JSON parsing
# from pydantic import BaseModel, Field
# class OrderPydantic(BaseModel):
#     id: str
#     amount: int = Field(gt=0)
```

Edge cases to mention: - `dataclass(slots=True)` only since Python 3.10 — older versions need a manual `__slots__`. - Mixing `attrs` and `dataclass` in the same codebase — pick one and stick to it. - `pydantic` is great for API boundaries but overkill for internal value objects.

What NOT to say: - "`attrs` and `dataclass` are equivalent." — wrong. `attrs` has more features (validators, converters, slots-by-default). - "`pydantic` is a replacement for dataclasses everywhere." — wrong. It's a parsing-and-validation tool for API boundaries; overkill for internal value objects.

Common follow-up: "Why might a FastAPI codebase use `pydantic` for request models and `dataclass` for internal types?" — Boundary code needs runtime validation, parsing, and schema generation — that's pydantic's home. Internal types just need shape + equality — that's dataclass.

Stand-out line: "`dataclass` is the stdlib baseline; `attrs` is the third-party power tool; `pydantic` is the validation-and-serialisation Swiss army knife. Pick by what you need, not by what's trendy."

Q59 · `match` statement on classes (PEP 634)

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Microsoft India, Postman · Topic: Structural pattern matching

The question (interviewer's verbatim phrasing):

Walk me through `match` with class patterns. How do positional patterns work, and what role does `__match_args__` play?

30-second answer: Python 3.10's `match` statement supports class patterns — `case Order(amount=a)` extracts fields by name (or by position via `__match_args__`). It's structural pattern matching, not just switch — patterns can bind, nest, and use guards. Useful for branching on event types or message shapes.

Step-by-step thought process: - The pattern: `case ClassName(field=binding)`. Python checks `isinstance(subject, ClassName)` and extracts the field. - `__match_args__` (a class attribute) defines positional field order for positional class patterns. Dataclasses set this automatically. - Patterns nest: `case Order(amount=int(a)) if a > 100` matches `Order` with `int` amount > 100 and binds `a`. - Wildcards (`_`) match anything; `|` does alternation; `[a, b, *rest]` is list pattern.

Code:

```

from dataclasses import dataclass

@dataclass
class Charge:
    amount: int
    currency: str = "INR"

@dataclass
class Refund:
    payment_id: str
    amount: int

@dataclass
class Webhook:
    event: str
    data: dict

def handle(event):
    match event:
        case Charge(amount=a, currency="INR") if a > 1000:
            return f"big rupee charge: {a}"
        case Charge(amount=a):
            return f"charge {a}"
        case Refund(payment_id=pid, amount=a):
            return f"refund {pid}: {a}"
        case Webhook(event=name, data={"id": id_}):
            return f"webhook {name} id={id_}"
        case _:
            return "unknown"

print(handle(Charge(2000)))           # big rupee charge: 2000
print(handle(Refund("PAY-1", 500)))  # refund PAY-1: 500
print(handle(Webhook("payment.captured", {"id": 7}))) # webhook payment.captured id=7

```

Edge cases to mention: - Forgetting `__match_args__` for non-dataclass positional patterns: `case Order(500)` raises if the class hasn't declared positional fields. Use `case Order(amount=500)` for keyword-only matching. - Patterns bind by name regardless of how the field is computed — they're not method calls, they're attribute reads. - Guards (the `if` clause) run after binding; failed guard moves to next case.

What NOT to say: - "`match` is Python's `switch`." — wrong. It's structural pattern matching — patterns can destructure, bind, and guard. - "`__match_args__` is required for any class pattern." — wrong. Only for positional patterns. Keyword patterns work without it.

Common follow-up: "What happens when two `case` patterns could both match — does Python pick the most specific?" — No. It picks the first one. Order patterns from most-specific to least-specific yourself.

Stand-out line: "Class patterns turn `match` into a real dispatcher — branch on type, destructure fields, bind names, all in one local syntax. The dataclass-friendly `__match_args__` makes positional patterns work for free."

Q60 · Composition over inheritance — when to apply it

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Cred, Microsoft India · Topic: Composition vs inheritance

The question (interviewer's verbatim phrasing):

What's the rule of thumb for choosing composition over inheritance? Give me a smell that says a hierarchy has gone too deep.

30-second answer: Composition holds another object as a field and delegates to it; inheritance extends a base class. The rule of thumb: use inheritance for "is-a" relationships where the subclass genuinely shares the parent's identity (a `RazorpayOrder` IS an `Order`); use composition for "has-a" or "uses-a" (an `Order` HAS a `PaymentGateway`).

Step-by-step thought process: - Inheritance is a tight coupling — the subclass commits to the parent's API surface, MRO, and any future changes. Powerful but rigid. - Composition is loose — the outer class holds a reference, calls methods on it, can swap the implementation. Much easier to test (inject a fake). - Mixins are a controlled form of inheritance — they add behaviour without state, and you mix in only what you need. - The "Strategy pattern" from OOP literature is just composition — hold a strategy object, delegate to it. Python makes this trivial because functions are first-class.

Code:

```

from abc import ABC, abstractmethod

# Composition: Order HAS a PaymentGateway
class PaymentGateway(ABC):
    @abstractmethod
    def charge(self, amount: int) -> str: ...

class Razorpay(PaymentGateway):
    def charge(self, amount): return f"razorpay {amount}"

class Stripe(PaymentGateway):
    def charge(self, amount): return f"stripe {amount}"

class Order:
    def __init__(self, amount: int, gateway: PaymentGateway):
        self.amount = amount
        self.gateway = gateway          # composed, not inherited

    def pay(self):
        return self.gateway.charge(self.amount)

# Easy to swap, easy to test
o = Order(500, Razorpay())
print(o.pay())                        # razorpay 500
o.gateway = Stripe()
print(o.pay())                        # stripe 500

# Compare with inheritance (less flexible)
class RazorpayOrder(Order):
    def pay(self): return f"razorpay {self.amount}"

class StripeOrder(Order):
    def pay(self): return f"stripe {self.amount}"

# Now you have a class hierarchy to maintain – and can't swap gateway at runtime

```

Edge cases to mention: - Overusing inheritance leads to deep hierarchies that nobody can read —

`EmployeeBase -> Employee -> SalariedEmployee -> SalariedManagerEmployee` is a smell. - Composition makes testing easier — inject a mock gateway, no subclass needed. - Some patterns (Decorator, Strategy, Adapter) are inherently composition. Python's first-class functions often let you skip even the helper class.

What NOT to say: - "Inheritance is a code smell." — too dogmatic. It's the right tool for genuine 'is-a' relationships. - "Composition always means dependency injection." — narrow. Composition is just 'hold a reference and delegate'; DI is one delivery mechanism.

Common follow-up: *"How does Python's first-class functions change the composition vs inheritance trade-off compared to Java?"* — In Python, you can compose with a plain function (`self.charge_fn = charge_fn`) instead of a class. Many patterns that require boilerplate classes in Java reduce to one closure here.

Stand-out line: *"Inheritance binds; composition delegates. The default should be composition — reach for inheritance only when the relationship is truly 'is-a' and you'd be writing the same boilerplate three times without it."*

Chapter 2 — Decorators

Core Python Library · 50 Questions · Python Interview Prep 2026 Edition · Learn & Excel

Decorators are the single most-asked Python topic at the senior end of Indian interviews. A backend role at Razorpay, a data-engineering screen at Swiggy, a platform round at PhonePe — all three will ask you to explain `@decorator`, write a `@retry`, and predict the order of `@a` stacked over `@b`. The reason is unkind: every Indian Python codebase uses decorators heavily (Flask routes, FastAPI dependencies, Django views, pytest fixtures, pandas method chains), and the candidates who cannot reason about `f = decorator(f)` ship subtle bugs around `functools.wraps`, mutable state, and parameterized decorators.

This chapter has fifty questions across seven sections. Section A is the mental model — what a decorator actually is, how the `@` sugar desugars, how `f = decorator(f)` reads in your head. Section B is `functools.wraps` and metadata preservation. Section C is decorator factories — the three-level nesting that every senior round asks you to write on a whiteboard. Section D is class decorators. Section E is decorators with state. Section F is the `functools.cache` family. Section G is real-world patterns — `@retry`, `@deprecated`, `@require_auth`, Flask routes, pytest fixtures. Each question follows the same Step-by-Step Answer format: tags, the question, the 30-second answer, step-by-step thought process, edge cases, what NOT to say, common follow-up, and a runnable code block.

Table of Sections

- Section A — Decorator basics (Q1 to Q10)
 - Section B — `functools.wraps` and metadata (Q11 to Q15)
 - Section C — Decorator factories and parameterized decorators (Q16 to Q25)
 - Section D — Class decorators (Q26 to Q30)
 - Section E — Decorators with state (Q31 to Q35)
 - Section F — `functools.cache`, `lru_cache`, `cached_property` (Q36 to Q40)
 - Section G — Real-world decorator patterns (Q41 to Q50)
-

Section A — Decorator basics

The mental model section. If you cannot recite `f = decorator(f)` in your sleep, the rest of this chapter will feel like magic. Decorators are not a language feature in the C++ sense — they are a syntactic shortcut for "call this function on this function and rebind the name." Every senior-level question in the next six sections reduces to that one mechanic.

Q1 · What is a decorator in Python

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant, Razorpay, Freshworks · Topic: Decorators · Fundamentals

The question (interviewer's verbatim phrasing):

What is a decorator in Python? Explain it like you would to a junior who has used Flask routes but never written one.

The 30-second answer: A decorator is a callable that takes a function and returns a function. The `@decorator` line above a `def` is syntactic sugar for `func = decorator(func)`. The returned function usually wraps the original to add behaviour — timing, logging, auth, caching — without modifying the original code. Decorators exist because Python functions are first-class objects, so passing one function into another and returning a third is normal Python.

Step-by-step thought process: 1. Start with the definition — "a callable that takes a function and returns a function." That one line covers every decorator you will ever write. 2. Show the sugar-vs-desugared equivalence. `@deco` above `def f` is exactly `f = deco(f)`. The interviewer is checking whether you know decorators are not special syntax — they are rebinding. 3. State the use-case bucket: cross-cutting concerns. Timing, logging, retry, caching, auth — anything that wraps the call but is not the core logic. 4. Mention that decorators work because Python functions are first-class: you can pass them, return them, store them in lists, attach attributes. A language that did not have first-class functions could not have decorators in this form.

Edge cases to mention: - The decorator does not have to return a function. It can return anything — a class instance, a string, even `None`. Returning `None` is almost always a bug, but it is legal. - Decorators can be applied to async functions, generators, classes, and methods — not only plain `def f`. - The decorator runs at function-definition time, not at call time. By the time you call the wrapped function, the decoration is done.

What NOT to say: - "A decorator is a way to extend a function." This sounds correct but is vague — extends how, with what mechanism? Be precise about the rebinding. - "It's like a Java annotation." Java annotations are metadata read by frameworks at runtime; Python decorators actually replace the function with a new one. Different mechanism.

Common follow-up: "If a decorator is just `f = decorator(f)`, why bother with the `@` syntax?" — Readability. The `@deco` sits visually next to the `def` so a reader scanning the file sees "this function is wrapped." The desugared form would push the wrap call below the body and away from the signature.

```
# The @ sugar
def timer(f):
    def wrapper(*args, **kwargs):
        import time
        start = time.perf_counter()
        result = f(*args, **kwargs)
        print(f"{f.__name__} took {time.perf_counter() - start:.4f}s")
        return result
    return wrapper

@timer
def slow_sum(n):
    return sum(range(n))

# Exactly equivalent to:
def slow_sum_v2(n):
    return sum(range(n))
slow_sum_v2 = timer(slow_sum_v2)

slow_sum(1_000_000)    # prints "slow_sum took 0.02s"
slow_sum_v2(1_000_000) # same behavior
```

Q2 · Desugar `@decorator` — what does the `@` line actually do

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Zoho · Topic: Decorators · Sugar Semantics

The question (interviewer's verbatim phrasing):

Take this code: `@log_calls` above `def add(a, b): return a + b`. Show me the exact equivalent code without the `@` sugar.

The 30-second answer: `@log_calls\ndef add(a, b): return a + b` desugars to `def add(a, b): return a + b` followed by `add = log_calls(add)`. The `@` sits above a `def`, the decorator is called with the function as its only argument, and the result is bound to the original name in the enclosing scope. That is the entire mechanic.

Step-by-step thought process: 1. Write the desugared form side by side. Most candidates have only seen the sugar; the interviewer is checking that you can mechanically translate. 2. Emphasise the rebinding. The original `add` function object is not modified — it is replaced. If something else has already captured a reference to the un-decorated `add`, that reference is unaffected. 3. Mention the

order in which things run: the `def add` creates the function, then the decorator is called immediately, then the returned object is bound to the name. 4. Note that stacked decorators apply bottom-up — `@a\n@b\ndef f` becomes `f = a(b(f))`. We will cover this in Q9.

Edge cases to mention: - The decorator runs at module import time if the decorated function is defined at module scope. So any side effects inside the decorator body — registering routes in Flask, for instance — fire at import. - If the decorator raises, the `def` statement fails — the name does not get bound. This is occasionally used for "register-this-function" patterns where missing registrations should crash early. - The expression after `@` is fully evaluated. `@module.deco` works; `@deco_factory(x=1)` also works because `deco_factory(x=1)` returns a decorator.

What NOT to say: - "It's like a function call but with extra magic." There is no magic. It is one function call and one rebinding. Saying "magic" loses you the point. - Confusing decorator application with decorator definition. Application is a single function call at def-time; definition is the body that creates the wrapper.

Common follow-up: "What if the `@` line has a name lookup that fails — like a typo? When does the error happen?" — At the `def` itself, because the name after `@` is resolved at def-time. So `@notarealthing` blows up immediately, not when you later call the function.

```
# Sugared form
@log_calls
def add(a, b):
    return a + b

# Mechanically equivalent desugared form
def add(a, b):
    return a + b
add = log_calls(add)

# The expression after @ is fully evaluated.
@some_module.utils.deco      # attribute lookup, all fine
def f1(): ...

@deco_factory(retries=3)     # call the factory, get back a decorator
def f2(): ...
```

Q3 · Why decorators exist — the cross-cutting concerns argument

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Postman, Freshworks · Topic: Decorators · Why

The question (interviewer's verbatim phrasing):

Why does Python have decorators? What problem do they solve that you could not solve with a plain function call?

The 30-second answer: Decorators solve cross-cutting concerns — behaviour you want around many functions but do not want to inline into each one. Timing, logging, auth checks, retries, caching, route registration. Without decorators you would either repeat the wrapper code inside every function or pollute every call site with a helper call. The decorator pattern lets the wrapping live in one place and apply itself by name.

Step-by-step thought process: 1. Define the term — cross-cutting concern is something that touches many functions but is not their primary purpose. Auth is a cross-cutting concern for a Django view; the view's primary job is to render a response. 2. Show what life without decorators looks like — copy-pasted try/except, copy-pasted timing, copy-pasted auth checks. The boilerplate dwarfs the actual function. 3. Compare to AOP in Java/C# — the same problem (cross-cutting) was solved with AspectJ or Spring AOP using bytecode weaving. Python solved it with first-class functions and the `@` sugar. 4. Note the registration pattern — Flask, FastAPI, pytest all use decorators not just to wrap but to register. `@app.route("/users")` does not modify the function much; it sticks the function in a routing table.

Edge cases to mention: - Not every wrapping job is a decorator job. If you only need the wrap once, a context manager or a plain helper is cleaner. - Over-decorating is a real cost — stacking five decorators on every view makes stack traces unreadable. - Some "decorators" do almost no wrapping. `@dataclass` for instance generates `__init__`, `__repr__`, `__eq__` and returns the class largely unchanged; it is a code-generator that happens to use decorator syntax.

What NOT to say: - "Decorators make code DRY." True but vague. Be specific — they centralise the cross-cutting concern. - "Decorators are an alternative to inheritance." They are not — they wrap callables, not types. You can write a class decorator (Q26) but that is still wrapping, not subclassing.

Common follow-up: "Give me three real-world examples of cross-cutting concerns you'd solve with decorators." — Auth checks on web views, retry-with-backoff on flaky API calls, memoisation on expensive pure functions. Each is orthogonal to the function's main job.

```
# Without decorators – boilerplate everywhere
def get_user(user_id):
    import time
    start = time.perf_counter()
    try:
        result = db.fetch(user_id)
    except Exception:
        print(f"get_user failed")
        raise
    print(f"get_user took {time.perf_counter() - start:.4f}s")
    return result

# With decorators – boilerplate lives once
@log_calls
@timer
def get_user(user_id):
    return db.fetch(user_id)
```

Q4 · `f = decorator(f)` — the mental model

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro · Topic: Decorators · Mental Model

The question (interviewer's verbatim phrasing):

When I see `@deco` above a function, what mental picture should I keep in my head about what's happening?

The 30-second answer: Read `@deco` as "rebind this name to the result of calling `deco` on this function." The original function object goes into `deco`, a new function object comes out, the original name now points to the new one. That is it. Once you read every `@` line that way, parameterised decorators, stacked decorators, and class decorators all stop being mysterious.

Step-by-step thought process: 1. State the mental rule: "see `@deco`, think `name = deco(name)`." Repeat it for stacked: "see `@a` above `@b`, think `name = a(b(name))`." 2. Walk through what `decorator(f)` returns. Usually a new wrapper function that closes over `f` and calls it inside. Sometimes a `partial`. Sometimes the same `f` after a side effect (route registration). Sometimes a class instance whose `__call__` calls `f`. 3. Tie it to first-class functions — Python functions are objects, so passing them into other functions is unremarkable. The `@` is just an ergonomic shorthand for that pattern. 4. Note the reassignment. The original `f` is not "decorated in place" — there is no such operation. The name `f` is rebound. Any other variable that captured the un-decorated `f` before the `@` ran still points at the original.

Edge cases to mention: - The wrapper does not have to call the original. You can write a decorator that replaces the function entirely. Rare and usually a bug, but legal. - The wrapper can call the original multiple times — `@retry` does this. Each call is a fresh invocation of the original function. - If the decorator is a class with `__call__`, `f` becomes an instance of that class. `f(args)` calls `__call__(args)`. Same mental model, different object type.

What NOT to say: - "The decorator modifies the function." It does not — it produces a new function and rebinds the name. The distinction matters for closures and for thinking about object identity. - "Decorators are evaluated lazily." They are not. They run immediately at the `def`.

Common follow-up: "If I import a module that defines `@app.route('/x')` on a function `view`, what runs at import time?" — `view` gets defined, then `app.route('/x')` is called to produce a decorator, then that decorator is called with `view` to produce the wrapped function (or registers `view` and returns it unchanged), then the result is bound to `view`. All at import.

```
def deco(f):
    def wrapper(*args, **kwargs):
        return f(*args, **kwargs) * 2
    return wrapper

@deco
def square(x):
    return x * x

# square is now the wrapper. The original square-function lives only
# inside the closure of wrapper. There's no way to call the un-decorated
# square from outside unless deco kept a reference somewhere.
print(square(3))  # 18 — calls wrapper, which calls original, then *2
```

Q5 · Decorator vs context manager — when to use which

Tags: Difficulty: Medium · Asked at: Razorpay, Zerodha, JP Morgan, Walmart Labs · Topic: Decorators · Design

The question (interviewer's verbatim phrasing):

Decorators and context managers both wrap "setup-and-teardown" logic around code. When would you use a decorator and when a context manager?

The 30-second answer: Decorators wrap a callable — they apply to every call of a named function. Context managers wrap a code block — they apply once, at the `with` line. If the same setup/teardown applies every time a function is called, decorate the function. If it applies to a specific section of code

inside one function, use `with`. They are not mutually exclusive — `contextlib.contextmanager` produces an object that works as both via `@contextmanager` and `@contextdecorator`.

Step-by-step thought process: 1. State the granularity rule: decorator is per-function, context manager is per-block. That is the easiest decision lens. 2. Examples — `@timer` on every call of `get_user` vs `with timer():` around just the SQL block inside `get_user`. Different scope, different tool. 3. Mention `contextlib.contextmanager` and `ContextDecorator`. The first turns a generator into a context manager; the second is a base class so a context manager doubles as a decorator. Useful when you want the same setup/teardown to be reusable in both modes. 4. Note that decorators can be more invasive — they replace the function. Context managers leave the function alone and only wrap the indented block. For library code that you do not want to surprise callers with, `with` is gentler.

Edge cases to mention: - `@contextlib.contextmanager` is a decorator that builds a context manager. The `@` is on the generator-factory, not on the consumer. - Some libraries expose the same thing both ways — `pytest.raises` works as a context manager (`with pytest.raises(ValueError):`) and as a parametrize-style argument. - Async — `@asynccontextmanager` and `async with` are the asyncio equivalents.

What NOT to say: - "They're interchangeable." They are not — granularity and target differ. - "Context managers are for files, decorators are for everything else." File handling is a tiny slice of context manager use; transactions, locks, timing blocks, mocking patches all use `with`.

Common follow-up: "Can you write a `@timer` and a `with timer():` that share implementation?" — Yes, using `contextlib.ContextDecorator`. Subclass it, implement `__enter__` and `__exit__`, and the same object works as both a decorator and a `with`-block.

```
from contextlib import ContextDecorator
import time

class Timer(ContextDecorator):
    def __init__(self, label="block"):
        self.label = label
    def __enter__(self):
        self.start = time.perf_counter()
        return self
    def __exit__(self, *exc):
        print(f"{self.label}: {time.perf_counter() - self.start:.4f}s")
        return False

# Use as decorator
@Timer("get_user")
def get_user(uid):
    return {"id": uid}

# Use as context manager
def get_user_v2(uid):
    with Timer("sql block"):
        return {"id": uid}
```

Q6 · Decorating regular functions vs methods — the `self` trap

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Microsoft India · Topic: Decorators · Methods

The question (interviewer's verbatim phrasing):

Does your decorator work the same way on a regular function and on an instance method? Is there a `self` gotcha?

The 30-second answer: A decorator written with `def wrapper(*args, **kwargs)` works on both because `self` arrives as the first positional argument and gets passed through. The gotcha is if you wrote `def wrapper(arg)` assuming one explicit positional — that breaks on a method because `self` plus the real arg are two positionals. Always use `*args, **kwargs` in the wrapper unless you have a specific reason not to.

Step-by-step thought process: 1. State the mechanic — methods are functions; the descriptor protocol injects `self` at lookup time. So when a wrapper sees `*args`, `args[0]` is `self`. 2. Show the breakage — a decorator written to log "called with (x)" using `args[0]` will print the `self` instance instead of the intended argument when used on a method. 3. Generalise — always write wrappers with `*args, **kwargs` and pass them through. The decorator should be argument-agnostic. 4. Edge — class methods and static methods. `@classmethod` and `@staticmethod` must usually be outermost so your decorator sees the underlying function before the descriptor magic. Stacking order matters here (Q9).

Edge cases to mention: - If your decorator caches by argument tuple (memoisation), `self` will be in the tuple and will need to be hashable. `lru_cache` on a method memoises per-instance — see Q38. - A bound method's `__self__` attribute is the instance. If your decorator inspects `f.__self__`, that only exists after the method is bound, not on the unbound function the decorator receives. - Decorating `__init__` works but the return value is ignored by Python — `__init__` is expected to return `None`. A wrapper that returns the result of `__init__` (which is `None`) is fine; one that returns something else is silently dropped.

What NOT to say: - "You need a special method-decorator." You do not. The same decorator works on both if written generically. - "You can't decorate methods." False — `@property`, `@classmethod`, `@cached_property` are all method decorators in the standard library.

Common follow-up: "What happens if you decorate a method but the decorator returns a plain function, not a descriptor?" — Most decorators return a plain function. That is fine for instance methods because Python's descriptor protocol kicks in when the method is looked up on an instance. It is not fine if you return a non-function (a class instance), because non-functions are not descriptors unless they implement `__get__`.

```
import functools

def log_call(f):
    @functools.wraps(f)
    def wrapper(*args, **kwargs):
        print(f"calling {f.__name__} with args={args[1:] if args else ()}")
        return f(*args, **kwargs)
    return wrapper

class UserService:
    @log_call
    def get(self, user_id):
        return {"id": user_id}

UserService().get(42)
# prints: calling get with args=(42,)
# self is args[0]; we slice it out for the log.
```

Q7 · `@staticmethod` , `@classmethod` , `@property` — what they actually do

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Zoho · Topic: Decorators · Library

The question (interviewer's verbatim phrasing):

Walk me through `@staticmethod` , `@classmethod` and `@property` . What does each one return, and why are they decorators rather than keywords?

The 30-second answer: `@staticmethod` returns a `staticmethod` object that ignores both `self` and `cls` . `@classmethod` returns a `classmethod` object that gets the class as first argument. `@property` returns a `property` object that turns attribute access into a method call. All three are built-in decorators that return descriptor objects — that descriptor protocol is what makes them special when looked up on an instance.

Step-by-step thought process: 1. For each, state the return type and the descriptor behaviour. `staticmethod.__get__` returns the raw function; `classmethod.__get__` returns a bound method with the class; `property.__get__` calls the getter. 2. Explain why they are decorators not keywords — Python's metaclass and descriptor machinery are flexible enough that built-in behaviour like this can be implemented as ordinary classes that happen to use decorator syntax. Adding a keyword would freeze the design. 3. Mention version notes — `@staticmethod` and `@classmethod` became callable directly in Python 3.10. Before that, calling them outside a class raised `TypeError`. `@property` has had setter and deleter forms (`@x.setter`) forever. 4. Use case quick map — `@staticmethod` for "this belongs in the class namespace but does not use the instance"; `@classmethod` for

alternative constructors (`from_json` , `from_dict`); `@property` for computed attributes that should look like fields.

Edge cases to mention: - `@property` without a setter is read-only. Trying to assign raises `AttributeError`. Add `@x.setter` to make it writable. - `@classmethod` does not give you the class of the instance — it gives you the class the lookup was made on. For polymorphism, `cls` is what subclasses will receive. - In dataclasses, mixing `@property` with a declared field of the same name is a common bug — the field shadows the property in `__init__`.

What NOT to say: - "`@staticmethod` is a function that just lives inside a class." Mechanically not quite — it is a descriptor that returns the function on lookup. The distinction matters for `__func__` and for super-calls. - "Use `@property` to make a method look like a Java getter." That is the surface effect, but it misses the point: properties enable later refactors (computed values, validation in setter) without changing callers.

Common follow-up: "When would you choose `@staticmethod` over a module-level function?" — When the function logically belongs to the class (it operates on the class's data shapes, or is found via the class in code search) but does not need `self` or `cls`. Otherwise prefer module-level.

```
class Order:
    def __init__(self, amount):
        self.amount = amount

    @staticmethod
    def fee_rate():                # no self, no cls
        return 0.02

    @classmethod
    def from_paise(cls, paise):    # cls is Order (or subclass)
        return cls(amount=paise / 100)

    @property
    def fee(self):                # accessed as order.fee, not order.fee()
        return self.amount * self.fee_rate()

o = Order.from_paise(49900)
print(o.amount, o.fee)          # 499.0 9.98
```

Q8 · Python decorators vs Java/C# attributes — same idea, different mechanism

Tags: Difficulty: Medium · Asked at: Infosys, JP Morgan, Walmart Labs · Topic: Decorators · Comparison

The question (interviewer's verbatim phrasing):

You have a Java background and used `@Transactional` and `@Override` annotations. How do Python decorators differ from Java annotations?

The 30-second answer: Java annotations are metadata — they sit on a method but do not change it. Frameworks read them at runtime via reflection (or compile-time via APT) and use the metadata to wrap calls, generate code, or alter behaviour. Python decorators are not metadata — they are function calls that immediately replace the function with a new one. The decorator runs at definition time and returns the function object that gets bound to the name.

Step-by-step thought process: 1. State the core difference — annotations are passive metadata; decorators are active transformations. The Java framework has to read the annotation and act; the Python decorator has already acted by the time you see the wrapped function. 2. Example — `@Transactional` in Spring is bytecode-woven by Spring AOP at proxy creation, not by the annotation itself. Python's equivalent `@transaction` would directly return a wrapper that opens a transaction, calls the function, and commits. 3. Mention compile-time vs runtime — Java has compile-time annotation processors (Lombok, Dagger) and runtime annotations (reflection-based). Python has only runtime decoration. There is no compile-time decorator that emits new code without the decorator being called. 4. C# is closer to Java — attributes are metadata, read by reflection or by source generators. Same passive model.

Edge cases to mention: - Type hints in Python (`def f(x: int) -> str`) are closer to Java annotations — they are passive metadata stored in `__annotations__`. Decorators that consume those annotations exist (`@validate_args` , `@dataclass`) and bridge the two worlds. - Python's `typing.dataclass_transform` is a hint to type-checkers that a decorator behaves like `@dataclass`. This is metadata about the decorator, not about the function. - Some Python decorators do nothing at runtime and exist only as markers — `@override` in Python 3.12 — which is closer to a Java annotation than a transformer.

What NOT to say: - "Java doesn't have decorators." Not literally true — annotations plus a framework approximate them. Be precise about the mechanism difference. - "Python decorators are faster because they don't use reflection." Performance comparison is not the point; the semantic difference (active vs passive) is.

Common follow-up: "What's the closest Java equivalent to `@functools.cache` ?" — Spring's `@Cacheable`. Both wrap a method so calls with the same arguments return cached results. Spring's needs a CacheManager config; Python's needs nothing — it is in the standard library.

```
# Python: decorator actively replaces the function
def cache(f):
    store = {}
    def wrapper(*args):
        if args not in store:
            store[args] = f(*args)
        return store[args]
    return wrapper

@cache
def compute(x):
    return x * x

# Java-equivalent in spirit:
# @Cacheable("compute") int compute(int x) { return x * x; }
# but Spring has to wire it via @EnableCaching and a CacheManager bean.
```

Q9 · Stacked decorators — order of application and order of execution

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Microsoft India · Topic: Decorators · Stacking

The question (interviewer's verbatim phrasing):

If I write `@auth` on top of `@cache` on top of a function, in what order do they apply, and in what order do they run when I call the function?

The 30-second answer: Decorators apply bottom-up — `@auth` over `@cache` over `def f` desugars to `f = auth(cache(f))`. So `cache` wraps `f` first, then `auth` wraps the cache wrapper. At call time, the outermost wrapper runs first — `auth` checks permissions, then if it passes, calls the cache wrapper, which checks the cache, and on miss calls the original `f`. Bottom-up to apply, top-down to execute.

Step-by-step thought process: 1. Show the desugar — `@a\n@b\n@c\ndef f` becomes `f = a(b(c(f)))`. Innermost decorator (`c`) sees the original function; outermost (`a`) sees the most-wrapped result. 2. Now execution order. When you call the resulting `f(args)`, you are calling `a`'s wrapper. `a`'s wrapper does its setup, calls `b`'s wrapper, which does its setup, calls `c`'s wrapper, which does its setup, calls the original. Then unwind in reverse. 3. Mental model — decorators stack like an onion. Application builds the onion bottom-up. Execution peels from the outside in. 4. Practical example — `@retry` on the outside of `@cache` makes sense (retry the cached fetch). `@cache` on the outside of `@retry` does not (you would cache successful retries, which is fine, but you cache the entire retry loop including any retry-aware behaviour).

Edge cases to mention: - Some decorators must be in a specific position. `@staticmethod` and `@classmethod` are usually outermost — they need to wrap the final callable, not a wrapper. - `@functools.wraps` does not change the order — it only copies metadata. It still wraps innermost-first. - If two decorators both register the function somewhere (Flask route + Click command), order matters because each registration sees a different function object.

What NOT to say: - "Decorators run top-down." Application is bottom-up; execution of the call is top-down. Mixing them up is a classic interview slip. - "Stacking is just sugar — order doesn't matter." It does. Different orderings produce different behaviour.

Common follow-up: "You have `@retry(3)`, `@log`, `@timer`. In what order should they stack so the log includes the total time including retries?" — `@retry(3)` innermost, then `@timer`, then `@log` outermost. At call time, `log` runs, calls `timer`, which calls `retry`, which calls the function up to 3 times. `timer` measures the whole retry loop; `log` records the timing.

```
def deco_a(f):
    def w(*a, **k):
        print("a-enter")
        r = f(*a, **k)
        print("a-exit")
        return r
    return w

def deco_b(f):
    def w(*a, **k):
        print("b-enter")
        r = f(*a, **k)
        print("b-exit")
        return r
    return w

@deco_a
@deco_b
def hello():
    print("inside")

hello()
# a-enter
#   b-enter
#     inside
#   b-exit
# a-exit
```

Q10 · The five things to check before adding a decorator to a function

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, BrowserStack · Topic: Decorators · Design

The question (interviewer's verbatim phrasing):

I have a function and I'm tempted to add a third decorator on top of it. What are the questions I should ask myself before doing that?

The 30-second answer: Five: does this concern truly apply to every call (or only sometimes)? Does the stacking order conflict with existing decorators? Will `functools.wraps` preserve enough metadata for callers (signature, docstring, type hints)? Is the decorator side-effecting at import time, and if yes, do I want that? And does this decorator hide errors that the function should propagate?

Step-by-step thought process: 1. "Every call or sometimes" — if the concern is conditional, use a context manager or an explicit helper, not a decorator. 2. "Stacking conflicts" — does this decorator need to be outermost? Innermost? Where does it sit relative to `@staticmethod`, `@property`, `@cache`? 3. "Metadata preservation" — does the wrapper use `functools.wraps`? Without it, your function loses its `__name__`, `__doc__`, signature. Frameworks that introspect (FastAPI, Click) break. 4. "Import-time side effects" — if the decorator registers the function or opens a resource, that happens at import. Big surprise for callers. 5. "Error hiding" — `@retry` and `@suppress_exceptions` can hide real bugs. Be explicit about which exceptions are retryable.

Edge cases to mention: - Decorators with mutable state (counters, caches) are global per-process by default. In threaded servers this is fine; in multi-process gunicorn workers each worker has its own copy. - Type-checkers (mypy, pyright) sometimes lose track of decorated signatures. `ParamSpec` (PEP 612) and `typing.ParamSpec` exist to help — use them on decorator definitions if mypy compliance matters. - Async-aware decorators must be written explicitly with `async def wrapper` and `await f(...)`. A sync decorator on an async function returns a coroutine without awaiting it, which the caller will then have to await — sometimes a bug.

What NOT to say: - "Decorators are always a good abstraction." No — they have real costs around debuggability, type-checking, and stack-trace readability. - "Just add `functools.wraps` and everything works." Wraps copies metadata but does not fix signature preservation for tools that need actual `inspect.signature` accuracy. See Q14.

Common follow-up: "You're adding `@retry` to a Razorpay payment-capture function. What's the risk?" — Retrying a non-idempotent operation. If the first call succeeded but the response was lost, retrying captures the payment twice. The decorator must check idempotency keys or only retry on retryable status codes.

```
# A "before I decorate" checklist as actual code
import functools, inspect

def is_safe_to_decorate(deco, target):
    """Sanity checks before stacking a decorator."""
    wrapped = deco(target)
    # 1. Metadata preserved?
    assert wrapped.__name__ == target.__name__, "use functools.wraps"
    # 2. Signature preserved?
    assert inspect.signature(wrapped) == inspect.signature(target), \
        "wrapper signature drift"
    # 3. Sync/async match?
    assert inspect.iscoroutinefunction(wrapped) == \
        inspect.iscoroutinefunction(target), "sync/async mismatch"
    return True
```

Section B — `functools.wraps` and metadata

`functools.wraps` is the second thing every interviewer asks after "what is a decorator." The candidates who answer it well say "without `wraps`, the wrapped function loses its name, docstring, and signature — and any tool that introspects breaks." The ones who fumble say "it's just good practice." Be the first group.

Q11 · Why does `functools.wraps` matter

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Swiggy, Postman · Topic: Decorators · `functools`

The question (interviewer's verbatim phrasing):

Why do I need `@functools.wraps(f)` inside my decorator? What breaks if I leave it out?

The 30-second answer: Without `wraps`, the wrapper function's `__name__` becomes "wrapper", its `__doc__` becomes the wrapper's docstring (or `None`), and `inspect.signature` reports the wrapper's signature (usually `(*args, **kwargs)`). Anything that introspects — Sphinx docs, FastAPI route docs, pytest fixture detection, `help(f)` — breaks. `functools.wraps` copies the metadata from the wrapped function onto the wrapper, fixing all of this in one line.

Step-by-step thought process: 1. Explain what `wraps` copies. `WRAPPER_ASSIGNMENTS` is the tuple of attributes copied — `__module__`, `__name__`, `__qualname__`, `__annotations__`, `__doc__`. `WRAPPER_UPDATES` is `__dict__`, which gets `update`-d. 2. Show the breakage without `wraps` — `help(decorated)` shows the wrapper's empty docstring, traceback shows

"wrapper" as the function name, FastAPI generates docs for `wrapper` not for your real function. 3. Show the fix — one line, `@functools.wraps(f)` above the `def wrapper`. Costs nothing, prevents debugging headaches. 4. Note `__wrapped__` — `wraps` also sets `wrapper.__wrapped__ = f`, so `inspect.unwrap` can find the original. Useful for tests that want to call the un-decorated function.

Edge cases to mention: - `wraps` does not preserve the actual signature for `inspect.signature` to use — `inspect.signature` falls back to `__wrapped__` and reads its signature. If your wrapper takes different args than the wrapped, the reported signature is misleading. - Class-based decorators need `wraps` on `__init__` or on `__call__` depending on which method does the work. The pattern is the same: `@wraps(f)` decorates the call site. - `wraps` is itself a decorator factory — `functools.wraps(f)` returns a decorator that you apply to `wrapper`. This is the parameterised-decorator pattern (Q16) in the standard library.

What NOT to say: - "wraps is for performance." It is not — it is for metadata preservation. - "You only need wraps if you publish the library." You need it any time someone might introspect — that includes you, debugging, six months from now.

Common follow-up: "Show me what `functools.WRAPPER_ASSIGNMENTS` is and why it's a tuple." — It's a tuple so that users can extend or replace it via the `assigned=` parameter of `wraps`. In 3.12 it is `('__module__', '__name__', '__qualname__', '__annotations__', '__doc__')`.

```
import functools

def bad_deco(f):
    def wrapper(*a, **k):
        return f(*a, **k)
    return wrapper

def good_deco(f):
    @functools.wraps(f)
    def wrapper(*a, **k):
        return f(*a, **k)
    return wrapper

@bad_deco
def bad_hello():
    """Says hello."""
    pass

@good_deco
def good_hello():
    """Says hello."""
    pass

print(bad_hello.__name__, bad_hello.__doc__) # wrapper None
print(good_hello.__name__, good_hello.__doc__) # good_hello Says hello.
```

Q12 · `__name__` and `__doc__` preservation — what `help()` shows**Tags:** Difficulty: Easy · Asked at: TCS, Infosys, Zoho · Topic: Decorators · Metadata**The question (interviewer's verbatim phrasing):**

If I decorate a function and then call `help()` on it, what do I see, and how do I make `help()` show the original docstring?

The 30-second answer: Without `wraps`, `help()` shows the wrapper's signature (`wrapper(*args, **kwargs)`) and the wrapper's docstring (often empty). With `wraps`, `help()` looks at `__wrapped__` and shows the original function's signature and docstring. So `@functools.wraps(f)` on the wrapper is the fix. Same applies to `pydoc`, IDE tooltips, and Jupyter's `?` introspection.

Step-by-step thought process: 1. Trace what `help()` does. `pydoc.help` looks at `__doc__` and `__signature__` (or `inspect.signature(obj)`). Both end up reading the wrapper unless `wraps` fixed them up. 2. Show the before/after. Decorated without `wraps`: `help(f)` is useless. With `wraps`: identical to the un-decorated help. 3. Mention that `wraps` does more than docstring — it also copies `__module__` and `__qualname__`, which matters for tracebacks and for serialisation libraries like pickle. 4. Note `inspect.unwrap(f)` — peels off layers of `__wrapped__` until it finds a function without that attribute. Useful when you have multiple decorators and want the bottom one.

Edge cases to mention: - IDE tooltips often use `inspect.signature` which reads `__wrapped__` chains. Some IDEs cache aggressively and miss recently changed decorators — reload the kernel/server. - `pydoc` (the CLI tool) has the same behaviour as the `help()` builtin. They share code. - Jupyter's `?` is essentially `help()` with prettier output. `??` shows source — that one calls `inspect.getsource` which also follows `__wrapped__` in most cases.

What NOT to say: - "`help()` shows the original function automatically." Not without `wraps` it does not. - "Just use `__doc__` directly." Sphinx and IDEs use richer introspection than just reading `__doc__`.

Common follow-up: "What if I want `help()` to show both the original docstring and a note saying 'wrapped by @retry'?" — Set `wrapper.__doc__` manually after `wraps`: `wrapper.__doc__ = (f.__doc__ or "") + "\n\n[Wrapped by @retry]"`.


```
import functools

def cache(f):
    @functools.wraps(f)
    def wrapper(*a, **k):
        return f(*a, **k)
    return wrapper

@cache
def fetch_user(uid: int) -> dict:
    """Fetch a user from the database by id."""
    return {"id": uid}

help(fetch_user)
# fetch_user(uid: int) -> dict
#     Fetch a user from the database by id.
```

Q13 · `functools.WRAPPER_ASSIGNMENTS` and `WRAPPER_UPDATES`

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India · Topic: Decorators · functools

The question (interviewer's verbatim phrasing):

What's the difference between `WRAPPER_ASSIGNMENTS` and `WRAPPER_UPDATES`, and when would you customise them?

The 30-second answer: `WRAPPER_ASSIGNMENTS` is the tuple of attributes that `wraps` overwrites on the wrapper from the wrapped function — `__module__`, `__name__`, `__qualname__`, `__annotations__`, `__doc__`. `WRAPPER_UPDATES` is attributes that get `update`-d (dict-merge), default just `__dict__`. You customise via the `assigned=` and `updated=` parameters of `wraps`. The use case is when your wrapper sets its own attribute (like `__cache_info__`) and you want to keep it while still copying others.

Step-by-step thought process: 1. Explain assign vs update — assign is "copy from source overwriting destination"; update is "merge from source into destination dict-style". Different semantics for different attribute kinds. 2. Why two tuples — most attributes (`__name__`, `__doc__`) are simple values you want overwritten. `__dict__` is a mapping you want to merge, not overwrite, because the wrapper might have its own custom attributes set. 3. Customisation example — if your wrapper has set `wrapper.cache_clear = clear_func` (like `lru_cache` does), you do not want `wraps` to overwrite that. Pass `assigned=` excluding the problematic key, or set the custom attribute after `wraps` runs. 4. Note `functools.update_wrapper` — the underlying function that `wraps` calls. `wraps(f)` returns `partial(update_wrapper, wrapped=f)`. So you can call `update_wrapper(wrapper, f, assigned=...)` directly if you do not want decorator syntax.

Edge cases to mention: - In Python 3.12 the tuple is `('__module__', '__name__', '__qualname__', '__annotations__', '__doc__')`. In 3.14 `__annotate__` was added; check the docs for the exact version you target. - Missing attributes are tolerated — if the wrapped function does not have `__annotations__`, `wraps` skips it. No error. - Frozen classes and slots — `__dict__` may not exist on the wrapper, in which case the `update` is a no-op.

What NOT to say: - "`assigned` is what `wraps` copies and `updated` is what `wraps` does not copy." Wrong — both get propagated, just with different merge semantics. - "You should never customise these." You sometimes should — `lru_cache` and `cache` both customise to preserve `cache_info`, `cache_clear`, `cache_parameters`.

Common follow-up: "Why is `__dict__` in `WRAPPER_UPDATES` and not `WRAPPER_ASSIGNMENTS`?" — Because the wrapper might already have entries in `__dict__` that you want to preserve. Assignment would replace the entire dict; update merges, keeping existing wrapper keys unless overwritten by the same key on the wrapped function.

```
import functools

# WRAPPER_ASSIGNMENTS — replaced wholesale
# WRAPPER_UPDATES — merged
print(functools.WRAPPER_ASSIGNMENTS)
# ('__module__', '__name__', '__qualname__', '__annotations__', '__doc__')
print(functools.WRAPPER_UPDATES)
# ('__dict__',)

def deco(f):
    @functools.wraps(f, assigned=('__name__', '__doc__')) # narrow it down
    def wrapper(*a, **k):
        return f(*a, **k)
    wrapper.extra = "custom marker"
    return wrapper
```

Q14 · Signature preservation gotchas

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, FastAPI users · Topic: Decorators · Signatures

The question (interviewer's verbatim phrasing):

Does `@functools.wraps` make `inspect.signature` return the original function's signature? Are there cases where it lies?

The 30-second answer: `wraps` sets `wrapper.__wrapped__ = f`. `inspect.signature` by default follows `__wrapped__` and reports the wrapped function's signature. So yes, it usually works. It

lies when your wrapper takes genuinely different arguments (e.g., adds a `timeout=` param the original does not have) — `inspect.signature` shows the original's signature, hiding the new one. Pass `follow_wrapped=False` to see the wrapper's real signature, or use `__signature__` explicitly.

Step-by-step thought process: 1. State the mechanism — `inspect.signature` follows the `__wrapped__` chain unless `follow_wrapped=False`. This is what makes wraps "work" for introspection. 2. Show the lie — if your wrapper genuinely accepts new kwargs, the signature still reports the original. Frameworks that bind arguments by signature (FastAPI dependency injection, Click) will reject the new kwarg as "unknown." 3. Fix — set `wrapper.__signature__ = inspect.Signature(...)` explicitly. Combine the original signature with the new params using `inspect.Signature.replace` and `inspect.Parameter`. 4. Note `typing.ParamSpec` and `Concatenate` — PEP 612 — let you express "this decorator adds a leading argument" in type annotations. mypy and pyright then check call sites correctly.

Edge cases to mention: - `inspect.signature(f, follow_wrapped=False)` gives the wrapper's literal signature. Useful for debugging signature drift. - The `__signature__` attribute, if set, takes precedence over `__wrapped__` chasing. So you can pin a signature explicitly. - Generators and async functions need extra care — `inspect.iscoroutinefunction` checks the wrapper, not the wrapped, by default. If you `@wraps(f)` a sync wrapper around an async function, `iscoroutinefunction` reports false. The right pattern: write an async wrapper for async functions.

What NOT to say: - "wraps preserves the signature perfectly." It preserves it via `__wrapped__`; if the wrapper has a different real signature, the report is misleading. - "Sphinx always shows the right signature." Sphinx uses `inspect.signature` too — it inherits the same lie.

Common follow-up: "How does FastAPI know what query params your route accepts when the route is decorated?" — FastAPI calls `inspect.signature` on the un-decorated function during route registration. If you wrapped the function with a decorator that changes its real signature without updating `__signature__`, FastAPI's dependency injection misbehaves.

```
import functools, inspect

def add_timeout(f):
    @functools.wraps(f)
    def wrapper(*args, timeout=10, **kwargs):
        # ... ignore timeout for demo
        return f(*args, **kwargs)
    return wrapper

@add_timeout
def call_api(url):
    return f"called {url}"

print(inspect.signature(call_api))           # (url)           - lies
print(inspect.signature(call_api, follow_wrapped=False)) # (*args, timeout=10, **kwargs)
```

Q15 · `inspect.signature` interaction with decorators

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, FastAPI users · Topic: Decorators · inspect

The question (interviewer's verbatim phrasing):

Your FastAPI dependency injection uses `inspect.signature` on your handler. If you stack two decorators on the handler, will FastAPI still see the right params?

The 30-second answer: If both decorators use `functools.wraps` and do not add or remove parameters, yes — `inspect.signature` follows the `__wrapped__` chain back to the original. If any decorator does not use wraps, the chain breaks at that layer and FastAPI sees `(*args, **kwargs)`. If a decorator adds a parameter, FastAPI does not see it unless you explicitly set `__signature__`.

Step-by-step thought process: 1. Trace the `__wrapped__` chain. `inspect.signature(outermost)` follows `__wrapped__` to the next layer, repeats until no more `__wrapped__`. So the original signature surfaces if every layer used wraps. 2. Break the chain — if `@deco_b` does not use wraps, `inspect.signature` stops at `deco_b`'s wrapper and reports its raw `(*args, **kwargs)`. 3. Add a parameter — if `@deco_a` truly takes a new arg, the chain still leads back to the original, hiding the new arg. Frameworks like FastAPI that bind kwargs by signature will not pass the new arg. 4. Solve — every decorator in the stack should either (a) use wraps and preserve the signature, or (b) explicitly set `__signature__` to reflect the new shape.

Edge cases to mention: - `pydantic.validate_call` (or earlier `validate_arguments`) introspects the signature at decoration time. If your custom decorator wraps `validate_call`-decorated functions, the order matters — `validate_call` innermost so it sees the real types. - `dataclasses.dataclass` uses `inspect.signature` to detect field shapes. Class decorators that wrap a class can break field detection. - For very fancy decorators (like ones that rewrite call signatures), the `decorator` PyPI library exists — it preserves the literal signature using `exec`. Heavy machinery, but it solves the "FastAPI sees the wrong shape" problem.

What NOT to say: - "FastAPI is fine if you use wraps." Usually yes, but new parameters are not automatically reflected. Be precise. - "Use `__signature__` always." Only when you genuinely change the signature; otherwise wraps alone is enough.

Common follow-up: "Show me how to write a decorator that adds a `request_id` kwarg and have FastAPI know about it." — Set `wrapper.__signature__` to a Signature with `request_id` appended as a `KEYWORD_ONLY` parameter, after copying the original's signature.

```
import functools, inspect

def with_request_id(f):
    @functools.wraps(f)
    def wrapper(*args, request_id=None, **kwargs):
        return f(*args, **kwargs)
    # Make inspect.signature see the new kwarg
    orig = inspect.signature(f)
    new_param = inspect.Parameter(
        "request_id",
        kind=inspect.Parameter.KEYWORD_ONLY,
        default=None,
    )
    wrapper.__signature__ = orig.replace(
        parameters=[*orig.parameters.values(), new_param]
    )
    return wrapper

@with_request_id
def handler(path: str):
    return path

print(inspect.signature(handler))  # (path: str, *, request_id=None)
```

Section C — Decorator factories and parameterised decorators

The whiteboard section. Every senior Python interview in Bengaluru, Hyderabad, and Pune asks you to write a parameterised decorator on the spot. `@retry(times=3)` . `@cache(maxsize=128)` . `@require_role("admin")` . The three-level nesting trips most candidates — outer factory, middle decorator, inner wrapper. If you can write it from memory in under a minute, you have signalled senior-level Python.

Q16 · The three-level nesting — outer factory, decorator, wrapper

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, JP Morgan, Microsoft India · Topic: Decorators · Factories

The question (interviewer's verbatim phrasing):

Write a decorator `@retry(times=3, delay=1)` from scratch. Explain the three levels of nesting.

The 30-second answer: A parameterised decorator has three levels. The outermost function (`retry`) takes the parameters (`times` , `delay`) and returns the actual decorator. The middle function takes the target function and returns the wrapper. The innermost function is the wrapper that runs at call time and implements the retry loop. `@retry(times=3, delay=1)` calls `retry(times=3, delay=1)` to get the decorator, which then decorates the function.

Step-by-step thought process: 1. Identify the three levels — factory, decorator, wrapper. Each closes over the layer above's variables. 2. Outer: `def retry(times, delay): ... return decorator` . Takes the parameters, returns the actual decorator. 3. Middle: `def decorator(f): ... return wrapper` . Takes the function, returns the wrapper. Uses `times` and `delay` from the enclosing factory's closure. 4. Inner: `def wrapper(*args, **kwargs): ...` . Loops up to `times` , calls `f` , sleeps `delay` on failure, raises if exhausted.

Edge cases to mention: - `wraps(f)` goes on the wrapper, not on the decorator or factory. The thing being passed to and called by the outside world is the wrapper. - Make `times` and `delay` keyword-only with `*` if you want to force `@retry(times=3)` over the positional `@retry(3)` . Positional is fine; keyword is self-documenting. - The factory itself can validate inputs — raise if `times < 1` at decoration time, not at call time. Fail fast.

What NOT to say: - "Just use `partial` ." `partial` does not work cleanly for this pattern because you need to wrap the function, not partial-apply the decorator. - "Three levels is too much nesting." It is what Python requires. Embrace it.

Common follow-up: "Can you make `@retry` work both as `@retry` (no args) and `@retry(times=5)` ?" — Yes, but it requires detecting whether the single argument is a function or not. See Q19.

```
import functools, time

def retry(times=3, delay=1):
    def decorator(f):
        @functools.wraps(f)
        def wrapper(*args, **kwargs):
            last_exc = None
            for attempt in range(times):
                try:
                    return f(*args, **kwargs)
                except Exception as e:
                    last_exc = e
                    if attempt + 1 < times:
                        time.sleep(delay)
            raise last_exc
        return wrapper
    return decorator

@retry(times=3, delay=0.1)
def flaky():
    import random
    if random.random() < 0.7:
        raise RuntimeError("nope")
    return "ok"

print(flaky())
```

Q17 · Reading `@decorator(args)` — what each parenthesis means

Tags: Difficulty: Medium · Asked at: TCS, Razorpay, Zoho · Topic: Decorators · Sugar

The question (interviewer's verbatim phrasing):

When you write `@retry(times=3)` above a function, what gets called and in what order?

The 30-second answer: First, `retry(times=3)` is called — that returns the actual decorator. Then the decorator is applied to the function below `@`. So `@retry(times=3)\ndef f` becomes `f = retry(times=3)(f)`. Two function calls, in order. The factory call (`retry(times=3)`) builds and returns a decorator that closes over `times=3`; the decorator call (`decorator(f)`) wraps the function.

Step-by-step thought process: 1. Distinguish `@deco` (one call: `deco(f)`) from `@deco_factory(args)` (two calls: `deco_factory(args)(f)`). 2. Show the desugar. `@retry(3)\ndef f` becomes `f = retry(3)(f)`. The expression after `@` is fully evaluated; in this case it is a call expression, so it executes. 3. Note when each call happens. Both happen at

definition time — the factory call, then the decorator call. By the time you reach the next line, `f` is wrapped. 4. Compare to bare `@deco`. There is no inner call; the decorator is directly applied. The two-call pattern only kicks in when you use parentheses.

Edge cases to mention: - Forgetting the parens — `@retry` (no args) calls `retry(f)`, treating the function as the factory's argument. Either crashes or, worse, silently produces an unwrapped function. Q19 covers the "maybe factory" pattern that handles both. - Multiple args — `@cache(maxsize=128, typed=True)` is the same shape, just more factory args. - The factory's return must be callable. If you return a non-function, the next call `(f)` will fail.

What NOT to say: - "The decorator runs at call time." It runs at def time. Both the factory call and the resulting decorator call happen when the `def` is executed. - "It's like passing the function and args together." It is two separate calls — clarify.

Common follow-up: "What if I forget the parens — `@retry` instead of `@retry()`?" — `retry` is called with `f` as its first positional. If `retry` expected `times=` only, it receives a function instead of an integer. Either crashes or produces bizarre behaviour. Q19 covers the safer pattern.

```
def deco_factory(label):
    def decorator(f):
        def wrapper(*a, **k):
            print(f"[{label}] before")
            r = f(*a, **k)
            print(f"[{label}] after")
            return r
        return wrapper
    return decorator

@deco_factory("api")
def call_api():
    print("calling")

# Equivalent to:
# tmp_deco = deco_factory("api") # factory call
# call_api = tmp_deco(call_api) # decorator call

call_api()
# [api] before
# calling
# [api] after
```

Q18 · Default arguments in a decorator factory

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman · Topic: Decorators · Factories

The question (interviewer's verbatim phrasing):

If my decorator factory has defaults for all its params — `@cache(maxsize=128, typed=False)` — can I call it as `@cache()` with no args?

The 30-second answer: Yes, as long as you write `@cache()` with parens. The factory `cache()` is called with zero args, picks up the defaults, returns the decorator, which then decorates the function. The trap is forgetting the parens — `@cache` (no parens) calls `cache(f)` which usually crashes because `maxsize` is now a function object instead of an int.

Step-by-step thought process: 1. Confirm — defaults make all params optional, so `@cache()` is valid. The empty parens are still a call expression. 2. Show the trap — without parens, `cache` is the decorator, so `cache(f)` is the call. If `cache` is written as a factory expecting `maxsize`, it tries to use `f` as `maxsize`. 3. Pattern — to allow both `@cache` and `@cache()`, you need the "maybe factory" trick (Q19). Otherwise pick one convention and document it. 4. Note that `functools.lru_cache` allows both — `@lru_cache` and `@lru_cache(maxsize=128)`. This was changed in Python 3.8; before then, only the parenthesised form worked.

Edge cases to mention: - The defaults are evaluated once at factory-definition time. Mutable defaults (like `default=[]`) suffer the same trap as regular functions (Q21 in Chapter 7). - Type-checkers can be picky — `@cache()` is fine, but `@cache` may flag a type error because the decorator expects a function, not an int. - Documentation matters — if your decorator allows both forms, say so in the docstring.

What NOT to say: - "Defaults make the decorator easier to use." Sometimes — they also expand the "forgot the parens" trap surface. - "Always force parens." Reasonable convention but inconsistent with `stdlib` (`lru_cache` works either way).

Common follow-up: *"How does `lru_cache` handle both `@lru_cache` and `@lru_cache(maxsize=128)`?"* — It detects the first call: if the single positional arg is callable and no other args, treat it as the function to wrap. Otherwise treat as a factory call. The detection is the "maybe factory" pattern.

```
def cache(maxsize=128, typed=False):
    def decorator(f):
        store = {}
        def wrapper(*args, **kwargs):
            key = (args, tuple(sorted(kwargs.items()))) if typed else args
            if key not in store:
                if len(store) >= maxsize:
                    store.pop(next(iter(store)))
                store[key] = f(*args, **kwargs)
            return store[key]
        return wrapper
    return decorator

@cache() # works — defaults to maxsize=128
def square(x): return x * x

@cache(maxsize=256) # works — overrides maxsize
def cube(x): return x * x * x
```

Q19 · The "always decorate or call as factory" pattern (`@maybe_factory`)

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, JP Morgan · Topic: Decorators · Factories

The question (interviewer's verbatim phrasing):

How would you write a decorator that works both as `@deco` (no args, no parens) and `@deco(times=3)` (with args)?

The 30-second answer: Detect whether the decorator is being called with a function as its single positional arg. If yes, treat the call as `@deco` -style and wrap directly. If no, treat it as a factory call and return a decorator. The detection is `callable(args[0]) and len(args) == 1 and not kwargs`. Wrap that detection at the top of your decorator and branch.

Step-by-step thought process: 1. State the ambiguity. `@deco` calls `deco(f)`. `@deco()` calls `deco()()`. `@deco(x=1)` calls `deco(x=1)(f)`. You need to tell them apart at runtime. 2. Detection rule — if the first positional is callable and there are no kwargs and no other positionals, you are in the `@deco` -as-decorator path. Otherwise you are in the factory path. 3. Implement with a wrapping function. The outer `def deco(*args, **kwargs)` does the detection. Inside, build the actual decorator. If we are in the no-args path, apply it directly; otherwise return it. 4. Caveat — this pattern fails if a legitimate first argument is callable (rare but possible, like `@retry(callback_fn)`). Force keyword-only args to avoid this.

Edge cases to mention: - Force keyword-only with `*` in the signature: `def retry(*, times=3, delay=1)`. Then positional callable is unambiguous. - `functools.lru_cache` uses this pattern.

Read its source for the canonical implementation. - The pattern adds runtime overhead at decoration time but zero at call time — the detection runs once.

What NOT to say: - "Just pick one." Reasonable for new code; not an answer to "how would you implement it." - "It's impossible." It is not — see `lru_cache`.

Common follow-up: "Why does `functools.lru_cache` support both forms even though it's confusing?" — Backwards compatibility plus ergonomics. The parens-less form is shorter for the common no-config case; the parens form is for tuning. Both forms cover real use cases.

```
import functools

def retry(*args, times=3, delay=1):
    # Detect: @retry (no args) vs @retry(times=N)
    if len(args) == 1 and callable(args[0]) and not (times != 3 or delay != 1):
        f = args[0]
        return _build_retry(f, times=3, delay=1)
    # Factory mode
    def decorator(f):
        return _build_retry(f, times=times, delay=delay)
    return decorator

def _build_retry(f, times, delay):
    import time
    @functools.wraps(f)
    def wrapper(*a, **k):
        last = None
        for i in range(times):
            try: return f(*a, **k)
            except Exception as e:
                last = e
                if i + 1 < times: time.sleep(delay)
        raise last
    return wrapper

@retry                                # no parens
def f1(): return "ok"

@retry(times=5, delay=0.1)           # with args
def f2(): return "ok"
```

Q20 · `functools.partial` as an alternative to parameterised decorators

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Walmart Labs · Topic: Decorators · functools

The question (interviewer's verbatim phrasing):

Is `functools.partial` a way to build parameterised decorators? Compare it with the three-level nesting.

The 30-second answer: `partial` does not replace the three-level nesting — it complements it. You can use `partial` to build the inner decorator from the outer factory in one line, saving a `def`. But you still need three logical levels: the parameters, the decorator, the wrapper.

`partial(decorator_template, times=3)` returns a callable that, when applied to a function, calls `decorator_template(f, times=3)`. Saves boilerplate; same semantics.

Step-by-step thought process: 1. Show the standard three-level shape (Q16). The factory returns the decorator. 2. Show the partial version. Define `decorator_template(f, times)` directly, then `retry = lambda times: partial(decorator_template, times=times)` — or use `partial` at the factory level. 3. State the trade-off — `partial` is shorter but less readable for unfamiliar reviewers. The three-level `def` is verbose but unambiguous. 4. Where `partial` really helps — when you have many parameter combinations and want to expose them as named constants, like `retry_fast = partial(retry, times=3, delay=0.1)` and `retry_slow = partial(retry, times=10, delay=2)`.

Edge cases to mention: - `partial` does not preserve `inspect.signature` perfectly — it shows the partial-ed function's signature minus the bound args. Usually fine. - `partial` objects are not regular functions — they do not have `__name__` by default. Use `functools.partialmethod` for class methods. - `partialmethod` is the descriptor-aware version for use inside class bodies.

What NOT to say: - "partial replaces the decorator factory." It does not; it shortens it. - "partial is faster than the three-level nesting." Both have the same call-time cost. The decoration-time cost is negligible.

Common follow-up: "Show me using `partial` to create named pre-configured decorators." — `cache_small = functools.partial(cache, maxsize=8)` and `cache_large = functools.partial(cache, maxsize=2048)`. Now `@cache_small\ndef f(): ...` is shorter than `@cache(maxsize=8)`.

```
import functools, time

def retry_template(f, *, times, delay):
    @functools.wraps(f)
    def wrapper(*a, **k):
        for i in range(times):
            try: return f(*a, **k)
            except Exception:
                if i + 1 < times: time.sleep(delay)
        raise
    return wrapper

# partial gives us preconfigured decorators
retry_fast = functools.partial(retry_template, times=3, delay=0.1)
retry_slow = functools.partial(retry_template, times=10, delay=2)

@retry_fast
def quick_api(): return "ok"

@retry_slow
def stubborn_api(): return "ok"
```

Q21 · Decorator factories that take a function instead of plain args

Tags: Difficulty: Hard · Asked at: Razorpay, Postman · Topic: Decorators · Factories

The question (interviewer's verbatim phrasing):

Write `@retry(on_exception=ValueError)` and `@retry(should_retry=lambda exc: isinstance(exc, IOError))`. How do you accept either a class or a callable predicate?

The 30-second answer: The factory takes a parameter that can be either an exception class or a callable. At decoration time, normalise it — if it is a class (subclass of `BaseException`), wrap it in a predicate that does `isinstance`. If it is callable already, use it directly. The wrapper then calls the predicate on each caught exception to decide whether to retry.

Step-by-step thought process: 1. Show the API. `@retry(on_exception=ValueError)` and `@retry(should_retry=callable)`. Two arg names, only one used at a time. Or one arg name that accepts both shapes. 2. Normalise inside the factory. Detect `inspect.isclass(arg)` and `issubclass(arg, BaseException)` — if true, build a predicate `lambda e: isinstance(e, arg)`. Otherwise assume it is the predicate. 3. Pass the normalised predicate into the wrapper closure. The wrapper catches exceptions and tests them via the predicate. 4. Defensive — if neither a class nor a callable, raise `TypeError` at decoration time. Fail fast.

Edge cases to mention: - Multiple exception types — accept a tuple of classes. Most callers expect this from `except (A, B)` familiarity. - Predicates that look at the exception's attributes (status codes, error messages) are a common need — that is why the callable form is worth supporting. - Async — the same pattern works for async wrappers, but use `async def wrapper` and `await f(...)`.

What NOT to say: - "Just take a class." Restrictive — predicates are a real use case. - "Use `try/except ExceptionType` ." That hard-codes the type at decorator-definition; we want runtime configurability.

Common follow-up: *"What if I want to retry only on HTTP 5xx but not 4xx?"* — Pass a predicate: `should_retry=lambda e: e.response.status_code >= 500` . The class-only API cannot express this.

```
import functools, inspect, time

def retry(*, on_exception=Exception, times=3, delay=0.5):
    if inspect.isclass(on_exception) and issubclass(on_exception, BaseException):
        predicate = lambda e, _t=on_exception: isinstance(e, _t)
    elif callable(on_exception):
        predicate = on_exception
    else:
        raise TypeError("on_exception must be an exception class or a callable")

    def decorator(f):
        @functools.wraps(f)
        def wrapper(*a, **k):
            for i in range(times):
                try:
                    return f(*a, **k)
                except BaseException as e:
                    if not predicate(e) or i + 1 == times:
                        raise
                    time.sleep(delay)
            return wrapper
        return decorator

@retry(on_exception=lambda e: isinstance(e, IOError))
def read_file(): return open("/tmp/x").read()
```

Q22 · Parameterised decorator with state shared across calls

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Microsoft India · Topic: Decorators · State

The question (interviewer's verbatim phrasing):

Write `@rate_limit(calls=10, per=60)` that limits a function to 10 calls per 60 seconds. Where does the state live?

The 30-second answer: The state — a list of recent timestamps — lives in the closure of the wrapper, created when the decorator is applied. Each call appends the current timestamp, removes timestamps older than the window, and checks the count. If the count exceeds the limit, sleep or raise. The state is per-decorated-function, not per-call.

Step-by-step thought process: 1. State the design. The window is sliding — track the timestamps of the last N calls. If the oldest is within the window and we are at the limit, deny or wait. 2. Where state lives. The factory creates the decorator; the decorator creates the wrapper and a `timestamps` list inside the wrapper's closure. One list per decorated function. 3. Implement. On each call: filter `timestamps` to keep only `t > now - per`. If `len(timestamps) >= calls`, sleep until the oldest expires (or raise `RateLimitExceeded`). Append `now`. Call `f`. 4. Thread safety — naive lists are not thread-safe. Wrap modifications in a `threading.Lock`. For asyncio, use `asyncio.Lock`.

Edge cases to mention: - Per-process. In multi-process gunicorn, each worker has its own state. For true global rate-limiting, use Redis-backed counters. - Per-decoration. Two different functions with `@rate_limit(10, 60)` have independent counters. Wanted, usually. - Behaviour on limit — sleeping is one option, raising is another. Make it configurable.

What NOT to say: - "Use a global counter." Globals are fine but make the decorator harder to apply to many functions. - "Just use Redis." For an interview demo, in-process is fine — mention Redis as the production path.

Common follow-up: "How would you make this work across multiple worker processes?" — Move the state to Redis: each call does `INCR key EXPIRE per` and reads the count. Slightly more complex but correct under multi-process and multi-host load.

```
import functools, time, threading

def rate_limit(*, calls, per):
    def decorator(f):
        timestamps = []
        lock = threading.Lock()
        @functools.wraps(f)
        def wrapper(*a, **k):
            with lock:
                now = time.time()
                # drop expired timestamps
                cutoff = now - per
                while timestamps and timestamps[0] < cutoff:
                    timestamps.pop(0)
                if len(timestamps) >= calls:
                    raise RuntimeError("rate limit exceeded")
                timestamps.append(now)
            return f(*a, **k)
        return wrapper
    return decorator

@rate_limit(calls=2, per=1)
def ping(): return "pong"

print(ping(), ping())    # ok
# print(ping())          # raises RuntimeError
```

Q23 · Why three levels and not four — can you skip one

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Zoho · Topic: Decorators · Factories

The question (interviewer's verbatim phrasing):

Why does a parameterised decorator need three levels of nesting? Can I write it with two functions instead of three?

The 30-second answer: Three levels because each level binds different things. The factory binds the decorator parameters. The decorator binds the wrapped function. The wrapper binds the call arguments. You can collapse them with `functools.partial` (Q20) or with class-based decorators (Q24), but conceptually all three bindings happen. There is no two-function shape that handles all three without some trick.

Step-by-step thought process: 1. Walk through what each level binds. Factory binds `times` and `delay`. Decorator binds `f`. Wrapper binds the runtime `args` and `kwargs`. 2. Show why two levels is not enough. `def deco(f, times): ...` cannot be applied as `@deco(times=3)`

because `@deco(times=3)` calls `deco(times=3)` first, which crashes since `f` is missing. 3. Collapse via partial. `deco = functools.partial(template, times=3)` replaces the factory level with a partial application. Now `@deco` works because partial-deco-f-times yields the decorator behaviour. But the three logical bindings are still happening — partial just hides the explicit `def`. 4. Collapse via class. A class with `__init__(times)` and `__call__(f)` and `__call__(args)` on the wrapper instance has the same three bindings expressed as init, call, and inner-call. Q26 covers this.

Edge cases to mention: - `functools.lru_cache` -style overloading (Q19) effectively merges the factory and decorator levels into one function. That function inspects its args and branches. - For decorators that take zero params, you genuinely only need two levels — the decorator and the wrapper. Three levels is the cost of parameters. - Curry-style — `decorator(times)(f)(*args)` is exactly three calls, one per level. Worth thinking of decorators as curried functions.

What NOT to say: - "It's just convention." Three levels is structural — each binds a separate scope. - "Closures are slow, that's why three levels." Closure access is fast in CPython — single-pointer lookup. Not a performance concern.

Common follow-up: "Could I write the entire decorator as a single lambda?" — In theory, with enough currying — `retry = lambda times: lambda f: lambda *a, **k: f(*a, **k)`. Legal but unreadable; never do this in production.

```
import functools

# Three-level (canonical)
def deco(times):
    def decorator(f):
        def wrapper(*a, **k):
            for _ in range(times):
                try: return f(*a, **k)
                except: pass
            return wrapper
        return decorator
    return decorator

# Two-level via partial
def template(f, *, times):
    def wrapper(*a, **k):
        for _ in range(times):
            try: return f(*a, **k)
            except: pass
        return wrapper
    return wrapper

deco2 = lambda times: functools.partial(template, times=times)
# @deco2(3) works exactly like @deco(3)
```

Q24 · Class-based decorators — `__init__` and `__call__`**Tags:** Difficulty: Medium · Asked at: Razorpay, Swiggy, Microsoft India · Topic: Decorators · Class**The question (interviewer's verbatim phrasing):**

Write a parameterised decorator as a class instead of as nested functions. When would you prefer the class form?

The 30-second answer: A class-based decorator stores parameters in `__init__` and wraps the function in `__call__`. `@MyDeco(times=3)` calls `MyDeco(times=3)` to create an instance; that instance is then called with the function, so `__call__` becomes the decorator step. Use class form when the decorator has substantial state, needs methods for introspection (like `cache_info`), or you want to subclass and override behaviour.

Step-by-step thought process: 1. Show the structure. `class Retry: __init__(self, times), __call__(self, f)` returns the wrapped function. So `@Retry(3)` becomes `f = Retry(3)(f)`. 2. Two-call shape. `Retry(3)` is `__init__`. `Retry(3)(f)` is `__call__`. So instances of the class are the decorator, not the class itself. 3. When to prefer. Class form lets you expose methods on the decorated function — `f.cache_info()`, `f.reset()`. It also makes state explicit (in `self`) rather than implicit (in closure). 4. Watch out — class-based decorators on methods need extra care because the instance does not auto-bind to `self`. You may need `__get__` to make it a descriptor.

Edge cases to mention: - `__call__` should call `functools.wraps(f)` on the returned wrapper. Without it, the instance has no `__name__`, `__doc__`. - For decorating methods, the class needs to implement `__get__` to return a bound method, or you need to use a function-based decorator with closure. - Sharing one instance across many decorations (one `Retry` instance reused for multiple functions) requires `__call__` to take the function — yes; the instance state is initial config.

What NOT to say: - "Class decorators are obsolete." They are not — `dataclass` and `cached_property` are class-style. - "Use closures, classes are slower." Slightly slower attribute access, but irrelevant for any real decorator.

Common follow-up: "Write a class-based decorator with a `reset_count` method exposed on the wrapped function." — Set `wrapper.reset_count = lambda: self._count_reset()` inside `__call__`, before returning wrapper. The caller can then `f.reset_count()`.

```
import functools

class Retry:
    def __init__(self, times=3):
        self.times = times

    def __call__(self, f):
        @functools.wraps(f)
        def wrapper(*args, **kwargs):
            for i in range(self.times):
                try:
                    return f(*args, **kwargs)
                except Exception:
                    if i + 1 == self.times:
                        raise
            return wrapper

@Retry(times=5)
def flaky():
    import random
    if random.random() < 0.8:
        raise IOError("nope")
    return "ok"
```

Q25 · Parameterised decorator that preserves the original function

Tags: Difficulty: Medium · Asked at: Razorpay, Postman · Topic: Decorators · Factories

The question (interviewer's verbatim phrasing):

I want `@trace(level="INFO")` to wrap the function for logging but also expose the undecorated original. How?

The 30-second answer: After wrapping, attach the original as an attribute — `wrapper.__wrapped__ = f`. `functools.wraps` does this automatically, so if you use wraps you already have it. Callers can reach the original via `wrapper.__wrapped__` or `inspect.unwrap(wrapper)`. For tests, this lets you call the un-instrumented version directly.

Step-by-step thought process: 1. State that wraps sets `__wrapped__` to the original function. So the original is reachable via attribute access. 2. Show the test pattern — `test_get_user.py` calls `get_user.__wrapped__(uid)` to bypass auth/cache/etc. 3. `inspect.unwrap(f)` peels off all decorator layers — useful when stacked. By default it walks the chain to the bottom. 4. Note that decorators that do not use wraps break this chain. So this pattern relies on every layer using wraps.

Edge cases to mention: - `inspect.unwrap(f, stop=callable)` lets you peel partially — useful when you want the layer one below your custom decorator. - Setting `wrapper.original_func = f` (custom attribute) is an alternative if you want a non-conflicting name. But `__wrapped__` is the convention. - Some decorators intentionally do not set `__wrapped__` to prevent test code from bypassing them (security decorators, for instance). Rare.

What NOT to say: - "You can't access the original." You can, via `__wrapped__`. - "Tests should always go through the decorator." Sometimes yes — the wrapper might be the bug. Bypassing is a useful debugging tool.

Common follow-up: "What does `inspect.unwrap` do differently from chaining `__wrapped__` manually?" — `unwrap` walks the chain in a loop and detects cycles (returning to a function already seen). It is safer than `while f.__wrapped__: f = f.__wrapped__`.

```
import functools, inspect

def trace(level="INFO"):
    def decorator(f):
        @functools.wraps(f)
        def wrapper(*a, **k):
            print(f"[{level}] calling {f.__name__}")
            return f(*a, **k)
        return wrapper
    return decorator

@trace(level="DEBUG")
def compute(x, y):
    return x + y

print(compute.__wrapped__)           # <function compute at ...>
print(compute.__wrapped__(1, 2))     # 3, no trace printed
print(inspect.unwrap(compute)(1, 2)) # 3, same thing
```

Section D — Class decorators

Class decorators are not "decorators that produce classes" — they are decorators that take a class as input. The classic examples are `@dataclass` (generates `__init__`, `__repr__`, `__eq__`) and `@total_ordering` (generates the missing comparison methods). The Indian product-co interviews ask about them after the function-decorator section, usually as "have you written one?" — the answer should be yes.

Q26 · Decorating a class — what changes vs decorating a function

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, JP Morgan · Topic: Decorators · Class

The question (interviewer's verbatim phrasing):

Can decorators be applied to classes, not just functions? Show me an example and explain what `@decorator` above `class Foo` actually does.

The 30-second answer: Yes — `@decorator` above `class Foo` desugars to `Foo = decorator(Foo)`, exactly like the function case. The decorator receives the class object and returns something — usually the same class with modifications (new methods, new attributes) or a different class entirely. `@dataclass`, `@total_ordering`, and `@unique` (enum) are stdlib examples.

Step-by-step thought process: 1. State the symmetry — class decorators work identically to function decorators. The `@` line desugars to assignment. Only the type of the input changes. 2. Show the canonical pattern — receive the class, attach methods or override attributes, return the class. The class is mutable; you can monkey-patch it in place. 3. Distinguish from metaclasses. A metaclass controls class creation; a class decorator runs after creation. Use class decorator when you want simple post-creation modification; use metaclass when you want to influence creation itself. 4. Example — `@register` decorator that adds the class to a registry. Or `@final` that prevents subclassing (with `__init_subclass__` hook).

Edge cases to mention: - Subclassing — if you decorate a base class, subclasses inherit. If your decorator added methods, those methods are on the base class, so subclasses get them. - If the decorator returns a different class, the name is bound to the new class — subclasses defined after the decoration inherit from the new one. `class Foo(Original)` is hard to do if `Original` is already replaced. - `@dataclass` is technically `dataclasses.dataclass`. It modifies the class in place: adds methods, sets `__dataclass_fields__`, and returns the same class.

What NOT to say: - "Class decorators are metaclasses." They are not — completely different mechanisms. - "Class decorators replace inheritance." Compose, not replace. Sometimes inheritance is still the right tool.

Common follow-up: "Write a `@register_handler('event_name')` decorator that adds the class to a registry under the given name." — Factory returns a decorator that puts `cls` into a dict keyed by name and returns `cls` unchanged.

```
def add_repr(cls):
    """Class decorator that adds a simple __repr__."""
    def __repr__(self):
        attrs = ", ".join(f"{k}={v!r}" for k, v in vars(self).items())
        return f"{cls.__name__}({attrs})"
    cls.__repr__ = __repr__
    return cls

@add_repr
class Order:
    def __init__(self, id, amount):
        self.id = id
        self.amount = amount

print(Order(1, 499))    # Order(id=1, amount=499)
```

Q27 · Class decorator vs metaclass — when to pick which

Tags: Difficulty: Hard · Asked at: Razorpay, Microsoft India, Goldman Sachs · Topic: Decorators · Class

The question (interviewer's verbatim phrasing):

When would you reach for a class decorator versus a metaclass? They both customise classes — what's the trade-off?

The 30-second answer: Class decorator runs after the class is created — it modifies the finished product. Metaclass runs during creation — it intercepts the `class` statement itself. Class decorators are simpler, do not propagate to subclasses unless explicitly applied, and compose with other decorators. Metaclasses are more powerful (control `__init__`, `__new__`, validate at creation) but harder to reason about, infect subclasses, and conflict with other metaclasses. Pick decorator first; metaclass only when decorator is not enough.

Step-by-step thought process: 1. Mechanics. `class Foo(metaclass=Meta)` causes `Meta.__call__(name, bases, dict)` to run, returning the class. Decorator runs after that on the resulting class. 2. Inheritance. Metaclass propagates — subclasses share the metaclass. Decorator does not — each subclass needs its own decoration. 3. Composition. You can stack decorators. You cannot stack metaclasses easily; conflicts must be resolved by a combined metaclass. 4. Use cases — registry, simple validation, adding methods all suit decorator. Enforcing abstract-method discipline (like `abc.ABC`) and validating at creation time suit metaclass.

Edge cases to mention: - `__init_subclass__` is a third option — it runs when a subclass is created, no metaclass needed. Often the cleanest middle ground. - `dataclass_transform` (PEP 681) hint lets type-checkers know your decorator behaves like `@dataclass`. Useful for libraries. -

Performance — metaclasses add a tiny overhead at class-creation time, but creation is rare so this rarely matters.

What NOT to say: - "Metaclasses are evil." They are not, just heavy. Used right (sparingly), they solve problems decorators cannot. - "Class decorators can do everything metaclasses can." Mostly true, but not for influencing bases or for affecting subclass creation transparently.

Common follow-up: "What about `__init_subclass__` — is it always better than a metaclass?" — Often yes. It hooks subclass creation without the metaclass complexity. Use it when you only need post-creation behaviour for subclasses.

```
# Class decorator: explicit, per-class, simple
def autorepr(cls):
    cls.__repr__ = lambda self: f"{cls.__name__}({vars(self)})"
    return cls

# Metaclass: implicit, propagates to subclasses
class AutoRepr(type):
    def __new__(mcs, name, bases, ns):
        ns["__repr__"] = lambda self: f"{name}({vars(self)})"
        return super().__new__(mcs, name, bases, ns)

@autorepr
class A:
    def __init__(self): self.x = 1

class B(metaclass=AutoRepr):
    def __init__(self): self.x = 1

# A and B both get __repr__, but B's subclasses also get it automatically;
# A's subclasses would need @autorepr again.
```

Q28 · `@dataclass` -style decorators — what they generate

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman · Topic: Decorators · Class

The question (interviewer's verbatim phrasing):

Walk me through what `@dataclass` actually does to a class. Could you write a stripped-down version?

The 30-second answer: `@dataclass` reads the class's annotations (`__annotations__`), generates `__init__`, `__repr__`, and `__eq__` based on them, sets `__dataclass_fields__` for introspection, and returns the same class. It uses `exec` internally to build the generated methods

so signatures are correct. A stripped-down version reads `__annotations__`, builds an `__init__` that assigns each annotated name from a parameter, and sticks it onto the class.

Step-by-step thought process: 1. State what it generates: `__init__`, `__repr__`, `__eq__`, and optionally `__hash__`, `__lt__` /etc. via `order=True`. Plus `__match_args__` for pattern matching. 2. Mechanism. Reads `cls.__annotations__` to find fields. For each, picks up default value from class body (or `field(default=...)`). Generates source code via string formatting and `exec` s it into a function object. 3. Why `exec`. The generated `__init__` needs the right signature so `inspect.signature` and type-checkers see correct types. A wrapper with `*args, **kwargs` would not. 4. Variants — `@dataclass(frozen=True)` makes instances immutable via `__setattr__` rejection. `@dataclass(slots=True)` (3.10+) generates `__slots__`. `@dataclass(kw_only=True)` makes fields keyword-only.

Edge cases to mention: - Field defaults that are mutable (`x: list = []`) raise `ValueError` at decoration time — dataclasses caught the mutable-default trap and force `field(default_factory=list)`. - `__post_init__` is called by the generated `__init__` after field assignment — useful for validation. - Inheritance and field ordering — fields from base classes come first in `__init__`; fields with defaults must follow fields without defaults across the whole MRO.

What NOT to say: - "`@dataclass` is just for data classes." It is a code generator for any class that fits the pattern. You can mix methods with dataclass fields freely. - "It's slow because of `exec`." `exec` runs once at decoration time, not at call. Performance at call time is identical to hand-written.

Common follow-up: "Write a 20-line mini-dataclass decorator." — Use `__annotations__`, build an `__init__` from them, attach it. Skip `__repr__` / `__eq__` for brevity.

```
def mini_dataclass(cls):
    fields = list(cls.__annotations__.keys())
    def __init__(self, *args, **kwargs):
        for name, value in zip(fields, args):
            setattr(self, name, value)
        for name, value in kwargs.items():
            setattr(self, name, value)
    cls.__init__ = __init__
    cls.__repr__ = lambda self: (
        f"{cls.__name__}(" +
        ", ".join(f"{f}={getattr(self, f, '?')}!r)" for f in fields) +
        ")"
    )
    return cls

@mini_dataclass
class Order:
    id: int
    amount: float

print(Order(1, 499.0))    # Order(id=1, amount=499.0)
```


Q29 · Modifying class attributes from a decorator

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Swiggy · Topic: Decorators · Class

The question (interviewer's verbatim phrasing):

If I write a class decorator that adds attributes or methods to the class, when do those additions take effect and are they visible to existing instances?

The 30-second answer: The decorator runs after `class Foo` finishes defining itself, before the name is bound to `Foo`. So any attribute or method you add is visible on the class. Instances created later see the changes (Python looks up attributes via the class). Instances created before — there is none, because the decorator runs before any instance can be created — so this question is moot for fresh classes. For monkey-patching at runtime after instances exist, those instances also see the change because attribute lookup is dynamic.

Step-by-step thought process: 1. Trace timing. `class Foo: ...` body executes, producing a class. Decorator runs on the class. The name `Foo` is bound to the (possibly different) returned class. 2. Visibility. Since the decorator runs before `Foo` is exposed, instances created from `Foo` see whatever the decorator left. The order is class-definition → decorator → name binding → instances. 3. Live class. Even after instances exist, modifying the class adds attributes that instances see — Python attribute lookup walks `instance.__dict__`, then `type(instance).__dict__`, then MRO. So `Foo.new_method = lambda self: ...` makes all instances of `Foo` (existing or future) suddenly have `new_method`. 4. Slots break this. If `Foo` defines `__slots__`, you cannot add new instance attributes. Class attributes are still addable.

Edge cases to mention: - Returning a different class — if your decorator returns a subclass or a new class, existing instances of the original class (if any pre-exist somehow) are not affected. Future `Foo(...)` calls create instances of the returned class. - Bound vs unbound — if you add a function as a class attribute, it becomes a method automatically via the descriptor protocol. Adding a `staticmethod` or `classmethod` instance attribute is also legal. - Frozen dataclasses — decorator-added attributes work at class level but instance-level `setattr` is blocked.

What NOT to say: - "The decorator can't modify the class." It absolutely can — that is the whole point. - "Existing instances won't see decorator changes." For changes made at decoration time, all instances are future instances. For runtime monkey-patches, existing instances do see them.

Common follow-up: "Write a class decorator `@inject_log_method` that adds a `log(msg)` method to any class it decorates." — Set `cls.log = lambda self, msg: print(...)` inside the decorator, return `cls`.

```
def inject(name, func):
    def decorator(cls):
        setattr(cls, name, func)
        return cls
    return decorator

@inject("log", lambda self, msg: print(f"[{type(self).__name__}] {msg}"))
class Worker:
    def __init__(self, wid): self.wid = wid

w = Worker(1)
w.log("starting")    # [Worker] starting
```

Q30 · When a class decorator is better than a metaclass

Tags: Difficulty: Hard · Asked at: Razorpay, Microsoft India · Topic: Decorators · Class

The question (interviewer's verbatim phrasing):

Give me three concrete cases where a class decorator is strictly better than a metaclass.

The 30-second answer: One — adding methods or attributes to a single class without affecting subclasses. Two — applying multiple independent class-level transformations (decorators stack; metaclasses fight). Three — when you cannot modify the metaclass because the class already inherits from a third-party base whose metaclass you do not control. In each case, decorator composes cleanly where metaclass either propagates unwanted or conflicts.

Step-by-step thought process: 1. Case one — explicit per-class application. `@register_endpoint` should register exactly the decorated class, not its subclasses. A metaclass would re-register every subclass. 2. Case two — stacking. `@validate_fields\n@register\n@autorepr\nclass Foo` stacks three orthogonal transformations. Combining three metaclasses requires a fourth combined metaclass. 3. Case three — third-party base classes. Django's `models.Model` has its own metaclass; you cannot add another. But you can apply a class decorator on top freely. 4. Bonus — easier to reason about. A reader sees `@deco` at the class definition; they know exactly when and what runs. A metaclass effect is silent — you only learn about it by reading the metaclass.

Edge cases to mention: - Pydantic v2 and SQLAlchemy use metaclasses for their core. Custom additions on those classes should be decorators, not new metaclasses. - `attrs` (the library) uses class decorators (`@attr.s`); they chose decorator over metaclass precisely for these reasons. - The pattern of "decorator does the work, metaclass would just be overkill" applies in most real codebases.

What NOT to say: - "Metaclasses are bad." They are not, just heavier. Pick the right tool. - "Decorators always win." They lose when you need to control class creation, when subclasses must inherit the behaviour automatically, or when you need to validate before the class exists.

Common follow-up: *"Show me a case where a metaclass beats a decorator."* — Enforcing abstract methods: `ABCMeta` checks at instance creation that all abstract methods are implemented. A decorator could not intercept instantiation of subclasses easily.

```
# Three independent decorators stacked – easy
@register_handler("payments")
@validate_required_methods("process", "rollback")
@auto_repr
class PaymentService:
    def process(self): ...
    def rollback(self): ...

# Three metaclasses would require a combined metaclass:
# class CombinedMeta(MetaA, MetaB, MetaC): pass
# class Foo(metaclass=CombinedMeta): ...
```

Section E — Decorators with state

Stateful decorators are the most interview-tested variety after basics. The classic state-holding patterns are counters, caches, rate-limiters, and circuit-breakers. The trap is concurrency — stateful decorators almost always need a lock if used in a threaded context. The interviewer wants to hear "this needs a lock" without prompting.

Q31 · Closure-based call counter

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay · Topic: Decorators · State

The question (interviewer's verbatim phrasing):

Write a decorator `@count_calls` that records how many times the function has been called and exposes the count via an attribute.

The 30-second answer: The wrapper closes over a counter — either a mutable container (list, dict) or use `nonlocal` to rebind an int. Each call increments. Expose `wrapper.count` by setting it as an attribute or by exposing the closure variable through a helper attribute.

Step-by-step thought process: 1. Closure stores the count. Use `nonlocal count; count += 1` inside wrapper, or use `wrapper.count` as an attribute (simpler, more discoverable). 2. Expose count externally. Setting `wrapper.count = 0` initially and updating `wrapper.count += 1` inside is the cleanest pattern — readable, attribute-accessible. 3. Thread safety. Naive `+= 1` is not atomic if `count` involves complex state, but for a single integer in CPython the GIL provides atomic increment. For dicts/lists, use a lock. 4. Reset. Add `wrapper.reset = lambda: setattr(wrapper, "count", 0)` to give callers a way to reset.

Edge cases to mention: - For asyncio code, increments on `wrapper.count` are atomic because asyncio is single-threaded. No lock needed. - For multi-process workloads, each process has its own counter. Use Redis for global counts. - `functools.wraps` does not interfere — it copies metadata but does not touch wrapper's custom attributes.

What NOT to say: - "Use a global variable." Discouraged — globals make the decorator non-reusable. - "Use threading.local." Local is per-thread, the opposite of what you want for a global call count.

Common follow-up: "What if I want a per-instance call count on a method?" — Store the count on `self`, not on the wrapper. The wrapper accesses `args[0]` (which is `self`) and does `setattr(self, "_count", getattr(self, "_count", 0) + 1)`.

```
import functools

def count_calls(f):
    @functools.wraps(f)
    def wrapper(*args, **kwargs):
        wrapper.count += 1
        return f(*args, **kwargs)
    wrapper.count = 0
    wrapper.reset = lambda: setattr(wrapper, "count", 0)
    return wrapper

@count_calls
def greet():
    return "hi"

greet(); greet(); greet()
print(greet.count)    # 3
greet.reset()
print(greet.count)    # 0
```

Q32 · Class-as-decorator with instance-level state

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, JP Morgan · Topic: Decorators · State

The question (interviewer's verbatim phrasing):

Rewrite the call-counter as a class. Where does the state live now?

The 30-second answer: The state lives on the instance — `self.count`. The class implements `__init__(self, f)` (storing the wrapped function and initialising count) and `__call__(self, *args, **kwargs)` (incrementing count and calling f). Each decoration creates one instance, so each decorated function has its own counter on its own instance.

Step-by-step thought process: 1. State the instance pattern. `CountCalls(f)` creates an instance; the instance is bound to the function name; calling the wrapped function calls `__call__`. 2. State lives on `self`. `self.count`, `self.f`. Methods like `reset` are also on the class for free. 3. `functools.wraps` adapter for instances. Use `functools.wraps(f)(self.__call__)` is tricky; the cleaner pattern is `functools.update_wrapper(self, f)` in `__init__`. 4. Method decoration caveat. A class-based decorator does not auto-bind as a method. You need to implement `__get__` on the class to return a bound method, or convert to function form.

Edge cases to mention: - Pickling — class instances may not pickle if the wrapped function is not picklable. Function-based decorators have the same issue. - Inspection — `inspect.signature` on a class instance looks at `__call__`. `update_wrapper` propagates the wrapped's signature. - Subclassing — you can subclass `CountCalls` to add per-instance behaviour. Cannot do this cleanly with closure-based form.

What NOT to say: - "Classes are always slower than closures for decorators." Marginally, but irrelevant for any wrapper that does real work. - "Instance state is harder than closure state." Equivalent — just different syntax.

Common follow-up: "Why do I need `update_wrapper(self, f)` in `__init__`?" — Without it, the instance has no `__name__`, `__doc__`, etc. — `help()` and tracebacks are useless.

```
import functools

class CountCalls:
    def __init__(self, f):
        functools.update_wrapper(self, f)
        self.f = f
        self.count = 0

    def __call__(self, *args, **kwargs):
        self.count += 1
        return self.f(*args, **kwargs)

    def reset(self):
        self.count = 0

@CountCalls
def greet():
    return "hi"

greet(); greet()
print(greet.count, greet.__name__)  # 2 greet
```

Q33 · Thread safety in stateful decorators

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Microsoft India · Topic: Decorators · Concurrency

The question (interviewer's verbatim phrasing):

Your `@count_calls` is being used in a Flask app with threaded workers. Is the counter accurate? What about a `@cache` with a dict?

The 30-second answer: For a single integer counter, `count += 1` is essentially atomic under CPython's GIL — increments are serialised so the count is accurate. For a dict-based cache, simple `dict[key] = value` is atomic at the C level, but check-then-set (`if key not in store: store[key] = compute()`) is not — two threads can both check, both compute, and one's value gets overwritten. Wrap multi-step operations in a `threading.Lock`.

Step-by-step thought process: 1. Differentiate single-op vs multi-op. Single bytecode-level operations (like `dict[k] = v`) are atomic under GIL. Multi-op sequences (`if k not in d: d[k] = compute()`) are not. 2. Show the race. Thread A checks `key not in store` — true. Thread B checks — also true. Both compute. Both store. The expensive `compute()` ran twice. For pure functions, the result is correct but wasted work. For functions with side effects, it might be wrong. 3. Fix with `threading.Lock`. Wrap the check-and-set in `with lock`. Adds minor overhead but guarantees one-compute-per-key. 4. For higher concurrency, use `threading.local` (per-thread

cache) or `concurrent.futures` with `as_completed` to coordinate. Or use `functools.lru_cache`, which has internal locking.

Edge cases to mention: - `lru_cache` is thread-safe — it has a lock internally. You do not need to wrap it. - For asyncio, no GIL issues — but if your function awaits, multiple coroutines can interleave. Use `asyncio.Lock`. - For multi-process, `threading.Lock` does not help. Use `multiprocessing.Lock` or Redis SETNX.

What NOT to say: - "Python is single-threaded because of the GIL." Not quite — multiple threads run; they just do not run Python bytecode in parallel. I/O and many C extensions release the GIL. - "Just lock everything." Excessive locking kills concurrency. Lock only the critical section.

Common follow-up: "What's the difference between `threading.Lock` and `threading.RLock` for decorators?" — `RLock` (re-entrant) allows the same thread to acquire multiple times. Useful if your wrapped function recursively calls back into the decorated path; `Lock` would deadlock.

```
import functools, threading

def thread_safe_cache(f):
    store = {}
    lock = threading.Lock()
    @functools.wraps(f)
    def wrapper(*args):
        with lock:
            if args not in store:
                store[args] = f(*args)
            return store[args]
    return wrapper

@thread_safe_cache
def slow(x):
    import time; time.sleep(0.1)
    return x * 2
```

Q34 · Memoisation as a decorator — the classic LRU

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs · Topic: Decorators · Caching

The question (interviewer's verbatim phrasing):

Write a basic memoise decorator without `functools.lru_cache`. Then compare to `lru_cache`.

The 30-second answer: A basic memoise: a dict keyed by the argument tuple, populated on first call, hit on repeat. Plus a lock for thread safety. The differences from `functools.lru_cache`: `lru_cache` has a max size with LRU eviction, separates positional vs keyword by `typed=`, exposes `cache_info()` for hits/misses, and is implemented in C for speed. Use `lru_cache` unless you have a specific reason to roll your own.

Step-by-step thought process: 1. Implement the basic. Dict, check, return-or-compute. Use args tuple as key — assumes all args are hashable. 2. Add a lock for thread safety (Q33). 3. Compare. `lru_cache`: bounded size, LRU eviction, `cache_info` / `cache_clear`, `typed=True` separates `f(1)` from `f(1.0)`. Yours: unbounded, no eviction, no info. 4. When roll your own — when you need cache-warming, custom eviction (LFU, TTL), or per-instance cache on methods (Q38).

Edge cases to mention: - Unhashable args (list, dict) raise `TypeError` from dict lookup. Wrap with `tuple()` conversion, or document that args must be hashable. - Keyword arguments — `kwargs` is a dict, not hashable. Convert to `tuple(sorted(kwargs.items()))` to use as key, or `frozenset(kwargs.items())`. - Memory — unbounded caches grow forever. Use `WeakValueDictionary` or `lru_cache(maxsize=N)` to bound.

What NOT to say: - "Just use a global dict." Globals make the decorator non-composable. - "`lru_cache` is for backwards compatibility." It is the recommended caching decorator in modern Python.

Common follow-up: "How does `lru_cache` evict?" — Linked list ordered by recent access. On a hit, move the node to the head. On a miss when full, drop the tail. $O(1)$ for both operations.

```
import functools, threading

def memoise(f):
    store = {}
    lock = threading.Lock()
    @functools.wraps(f)
    def wrapper(*args, **kwargs):
        key = (args, frozenset(kwargs.items()))
        with lock:
            if key not in store:
                store[key] = f(*args, **kwargs)
            return store[key]
    return wrapper

@memoise
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)

print(fib(50))    # blazing fast despite the recursive definition
```


Q35 · Rate-limiter decorator with token bucket

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Postman · Topic: Decorators · State

The question (interviewer's verbatim phrasing):

Implement a token-bucket rate limiter as a decorator. Bucket holds 10 tokens, refills 1 per second. Calls take 1 token. If empty, block until a token is available.

The 30-second answer: Token bucket has a capacity (10) and a refill rate (1/sec). Track the current token count and the timestamp of last refill. On each call, compute elapsed time, add `elapsed * rate` tokens (capped at capacity), then if at least 1 token take it and proceed; otherwise sleep until one is available. State (token count, last refill time) lives in the wrapper's closure with a lock.

Step-by-step thought process: 1. State the algorithm. Bucket starts full. Refills continuously at `rate` tokens/sec. Each call consumes one. If empty, wait. 2. Implement refill. On each call, `now - last_refill` gives elapsed seconds. Add `elapsed * rate` to current tokens, cap at capacity, update `last_refill`. 3. Consume or wait. If `tokens >= 1`, decrement and proceed. Else, compute `time_to_one_token = (1 - tokens) / rate`, sleep that long, then take the token. 4. Lock. The bucket is shared state — modifying it from multiple threads needs a `threading.Lock`. Sleeping inside the lock blocks other callers fully, so sleep outside the lock if you want concurrent waiters.

Edge cases to mention: - Multi-token consumption — extend to `take(n)` for "this call costs 3 tokens." Useful for variable-weight requests. - Asyncio variant — use `asyncio.Lock` and `await asyncio.sleep`. Same algorithm. - Multi-process — bucket must be in shared memory (`multiprocessing.Value`) or in Redis.

What NOT to say: - "Token bucket is the same as sliding window." Not quite — token bucket allows bursty traffic (up to capacity instantly); sliding window caps strictly N per window. - "Just use a sleep loop." Token bucket is the algorithm; the sleep is one implementation detail.

Common follow-up: "What's the difference between leaky bucket and token bucket as rate-limiters?" — Token bucket allows burst up to capacity; leaky bucket smooths to a constant output rate regardless of input bursts.

```

import functools, time, threading

def token_bucket(capacity=10, rate=1.0):
    def decorator(f):
        tokens = capacity
        last_refill = time.monotonic()
        lock = threading.Lock()
        @functools.wraps(f)
        def wrapper(*a, **k):
            nonlocal tokens, last_refill
            with lock:
                now = time.monotonic()
                tokens = min(capacity, tokens + (now - last_refill) * rate)
                last_refill = now
                if tokens < 1:
                    sleep_for = (1 - tokens) / rate
                else:
                    sleep_for = 0
                    tokens -= 1
                if sleep_for:
                    time.sleep(sleep_for)
                    with lock: tokens = max(0, tokens - 1)
            return f(*a, **k)
        return wrapper
    return decorator

@token_bucket(capacity=2, rate=1.0)
def call_api(): return "ok"

```

Section F — `functools.cache`, `lru_cache`, `cached_property`

The stdlib caching decorators are a small set with sharp differences. The interviewer wants to know — when do you reach for `cache`, when for `lru_cache(maxsize=N)`, when for `cached_property`, and when for nothing at all? Sharp answers here separate junior from mid-level.

Q36 · `functools.cache` vs `functools.lru_cache` — what changed and why

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs · Topic: Decorators · functools

The question (interviewer's verbatim phrasing):

What's the difference between `functools.cache` and `functools.lru_cache` ? When would you use one over the other?

The 30-second answer: `functools.cache` (added 3.9) is unbounded — it stores every distinct call forever. `functools.lru_cache(maxsize=N)` (since 3.2) is bounded — it evicts the least-recently-used entry when full. `cache` is faster (no LRU bookkeeping) and shorter to write, but unsafe for memory if the argument space is large. Use `cache` for pure functions with small argument spaces (like fibonacci, factorial); use `lru_cache(maxsize=128)` for caches you do not want growing without bound.

Step-by-step thought process: 1. State the bounded vs unbounded distinction. `cache` = unbounded dict; `lru_cache(maxsize=128)` = LRU-evicting bounded dict. 2. Speed. `cache` is slightly faster because no LRU tracking. For most workloads the difference is invisible. 3. When to pick. Small argument space (factorial of 20 numbers) → `cache`. Large but skewed (last 1000 unique inputs) → `lru_cache(maxsize=1000)`. Unbounded by accident → `cache` accidentally becomes a memory leak. 4. `cache` is literally `lru_cache(maxsize=None)`. So the decision is just whether you want eviction.

Edge cases to mention: - Both support `cache_info()` and `cache_clear()`. - Both fail on unhashable args (list, dict, set). Convert to tuple or frozenset before calling. - Bound methods — caching a method memoises per-instance because `self` is part of the key. See Q38.

What NOT to say: - "`cache` is for new code, `lru_cache` is legacy." Not true — both are current. Pick based on memory bounds. - "`lru_cache` is slow." Trivial overhead for sane maxsize.

Common follow-up: "What does `cache_info()` return?" — A named tuple with `hits`, `misses`, `maxsize`, `currsize`. Useful for tuning cache effectiveness.

```
import functools

@functools.cache          # unbounded
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)

@functools.lru_cache(maxsize=128) # bounded
def fetch_user(user_id):
    return {"id": user_id, "name": "..."}

print(fib(100))
print(fetch_user.cache_info())
# CacheInfo(hits=0, misses=0, maxsize=128, currsize=0)
```

Q37 · Choosing `maxsize` for `lru_cache`

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy · Topic: Decorators · Caching

The question (interviewer's verbatim phrasing):

How would you pick `maxsize` for `lru_cache` ? What happens at the boundary?

The 30-second answer: Pick `maxsize` based on the realistic hot-set size — the number of distinct inputs you expect to see in a typical window. Too small and you thrash (evict before re-use); too large and you waste memory on cold entries. Default of 128 works for many use cases. Profile with `cache_info()` : high miss-to-hit ratio after warmup means `maxsize` is too small.

Step-by-step thought process: 1. Hot-set analysis. If you serve 10,000 users but 95% of requests are for the top 200, `maxsize=256` captures the hot set. Above 256 is wasted RAM. 2. Power-of-two values. 128, 256, 512 — convention, not requirement. Any positive integer works. 3. `maxsize=None` — unbounded; equivalent to `cache` . Risky for large input domains. 4. Profile. Run for an hour, call `cache_info()` . `hits / (hits + misses)` is hit rate. If <50% after warmup, raise `maxsize`.

Edge cases to mention: - `maxsize=0` is legal — every call is a miss, no caching at all. Use case: keep the decorator in place but disable caching for benchmarking. - Per-process — each gunicorn worker has its own cache. Effective `maxsize` across the fleet is `maxsize * worker_count` . - For functions called concurrently on many threads, the LRU eviction is locked — contention shows up as latency spikes under high concurrency.

What NOT to say: - "Bigger is always better." More memory, slower scan on cold caches, possibly worse locality. - "Use a fixed 128." Reasonable default but not always right.

Common follow-up: "How does `cache_info()` help you tune `maxsize` ?" — Watch `currsz` plateau and miss rate stabilise. If `currsz` hits `maxsize` quickly and miss rate stays high, your hot set is bigger than `maxsize`. Raise it.

```
import functools

@functools.lru_cache(maxsize=4)
def f(x):
    return x * 2

for x in [1, 2, 3, 4, 5, 1, 2]:
    f(x)

info = f.cache_info()
print(info)
# CacheInfo(hits=2, misses=5, maxsize=4, currsz=4)
# Hit on f(1) and f(2) at the end because they got evicted then refetched.
# Actually no — they ARE evicted by f(5). Let me re-check at runtime.
```

Q38 · `lru_cache` on methods — the per-instance trap

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Microsoft India · Topic: Decorators · Caching

The question (interviewer's verbatim phrasing):

If I put `@lru_cache` on an instance method, what happens to the cache when I create multiple instances? Is there a memory leak risk?

The 30-second answer: The cache is per-method (shared across all instances) but keyed by `self` as the first argument. So each instance contributes its own entries to the shared cache. Two issues — `self` must be hashable (instances are by default, by id), and instances are kept alive by the cache, preventing garbage collection. Use `functools.cached_property` for per-instance lazy values, or a `WeakValueDictionary` for the cache to avoid leaks.

Step-by-step thought process: 1. The cache lives on the function/method object, not on instances. So all instances share the cache dict. 2. `self` is part of the key. Two instances of the same class with the same other args get separate cache entries. 3. The leak. `self` is in the key — the cache holds a strong reference to `self`. The instance cannot be collected until the cache evicts it (LRU) or the cache is cleared. For long-lived caches and short-lived instances, this is a memory leak. 4. Fix. For per-instance caching, use `cached_property` (Q39). For LRU with weak references to instances, use a `weakref` pattern explicitly.

Edge cases to mention: - `functools.cached_property` (3.8+) stores the cached value in the instance's `__dict__` — once per instance, no LRU. No leak. See Q39. - For class methods, `cls` is in the key — same issue applies but classes are usually long-lived anyway. - Custom `__hash__` on the instance can change cache behaviour. If your `__hash__` is mutable, the cache lookups become wrong.

What NOT to say: - "`lru_cache` on methods is broken." It works; it has trade-offs. Be precise about them. - "Just `cache_clear()` regularly." Hack — the right fix is the right data structure.

Common follow-up: "Show me a memory-leak case where `lru_cache` keeps instances alive." — Cache a method on a short-lived request handler instance. The cache (long-lived, on the class) holds a reference to each handler instance. Instances are never collected.

```
import functools, gc

class Worker:
    def __init__(self, wid):
        self.wid = wid

    @functools.lru_cache(maxsize=128)
    def compute(self, x):
        return self.wid * x

w1 = Worker(1)
w2 = Worker(2)
w1.compute(10); w2.compute(10)
print(Worker.compute.cache_info()) # currsiz=2
# Both w1 and w2 are pinned in the cache. del w1 wouldn't release until cache evicts.
```

Q39 · `cached_property` — once-per-instance, not LRU

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman · Topic: Decorators · Caching

The question (interviewer's verbatim phrasing):

What is `functools.cached_property` and how does it differ from `@property` plus `lru_cache` ?

The 30-second answer: `@cached_property` (3.8+) computes a property's value once per instance on first access and stores the result in the instance's `__dict__`. Future accesses hit `__dict__` directly, skipping the descriptor. Unlike `@property` (which recomputes every access) or `@property + @lru_cache` (which has all the `lru_cache` method gotchas), `cached_property` is the cleanest pattern for "compute once per instance, then it never changes."

Step-by-step thought process: 1. Mechanism. `cached_property` is a descriptor that stores its computed value in `instance.__dict__[name]` on first access. The descriptor only intercepts when `__dict__` does not have the key. 2. Compare to `@property`. Property is read-only computation every time; `cached_property` is once-then-cached. 3. Compare to `@property + @lru_cache`. The cache is on the function, shared across instances — memory leak (Q38), wrong granularity. `cached_property` is per-instance, automatic cleanup with instance. 4. Caveat — the instance must have a writable `__dict__`. `__slots__` classes do not, and `cached_property` raises. Workaround: include the cache slot in `__slots__`.

Edge cases to mention: - Not thread-safe by default — two threads accessing for the first time can both compute. Use a per-instance lock if needed. - Invalidate by `del self.attr` — removes the

entry from `__dict__`, next access recomputes. - For Python 3.8+. Earlier versions used third-party (`cached_property` package).

What NOT to say: - " `cached_property` is the same as `@property + lru_cache` ." It is not — different storage, different lifecycle, different concurrency semantics. - "Always use `cached_property` ." When you want recomputation, plain `@property` is right.

Common follow-up: "How do I use `cached_property` with `__slots__` ?" — Add the property name to `__slots__`. The descriptor uses `setattr(instance, name, value)` which goes through the slot.

```
import functools, time

class Report:
    def __init__(self, rows):
        self.rows = rows

    @functools.cached_property
    def summary(self):
        print("computing...")
        time.sleep(0.5)
        return {"total": sum(self.rows), "count": len(self.rows)}

r = Report([1, 2, 3])
print(r.summary)    # computing... {'total': 6, 'count': 3}
print(r.summary)    # {'total': 6, 'count': 3}    no recompute
del r.summary
print(r.summary)    # computing... again
```

Q40 · `typed=True`, hashable args, and unhashable-arg workarounds

Tags: Difficulty: Medium · Asked at: Razorpay, Postman · Topic: Decorators · Caching

The question (interviewer's verbatim phrasing):

What does `typed=True` do in `lru_cache`? And what happens if I cache a function that takes a list as an argument?

The 30-second answer: `typed=True` makes the cache treat arguments of different types as separate entries — `f(1)` and `f(1.0)` are cached separately. Default `typed=False` treats them as the same key because `1 == 1.0` and they hash the same. For a function taking a list — `TypeError: unhashable type: 'list'`. Workaround: convert the list to a tuple inside the wrapper, or accept a tuple in the public signature.

Step-by-step thought process: 1. `typed=False` (default). The cache key includes only values via `hash()`. Different types that compare equal share an entry. Usually what you want. 2. `typed=True`. Key also includes the type. `f(1)` and `f(1.0)` get separate entries. Use when type-specific behaviour matters (rare). 3. Unhashable args. `lru_cache` keys are dicts under the hood, so args must be hashable. Lists, dicts, sets raise `TypeError`. 4. Workarounds — convert in the caller (`f(tuple(my_list))`), wrap with a converter decorator, or use `frozenset` / `tuple` in the public signature.

Edge cases to mention: - Custom classes are hashable by default (by id). If you override `__eq__` without `__hash__`, the class becomes unhashable. - Numpy arrays are unhashable. Convert to tuple of bytes if you must cache. - For complex hash needs, write a wrapper that computes a hashable key (`json.dumps(sort_keys=True)` is one trick).

What NOT to say: - "Convert to JSON for the key." Sometimes works but slow and brittle for arbitrary types. - "`typed=True` makes lookups slower." Marginal.

Common follow-up: "Write a memoise decorator that handles list args by converting them to tuples internally." — Pre-process args inside the wrapper. Trade-off: convert always, even for hashable args; or convert only the unhashable ones.

```
import functools

@functools.lru_cache(typed=False)
def f1(x): return type(x).__name__

print(f1(1), f1(1.0), f1.cache_info())
# int int CacheInfo(hits=1, misses=1, ...) - second call hit, treated same

@functools.lru_cache(typed=True)
def f2(x): return type(x).__name__

print(f2(1), f2(1.0), f2.cache_info())
# int float CacheInfo(hits=0, misses=2, ...) - separate entries
```

Section G — Real-world decorator patterns

The final section. What decorators actually look like in real Indian Python codebases — Flask, FastAPI, Django, pytest, retry libraries. The interviewer wants to know you have written these, not just understood them.

Q41 · `@deprecated` — warn callers without breaking them

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Microsoft India · Topic: Decorators · Patterns

The question (interviewer's verbatim phrasing):

Write a `@deprecated(reason="use X instead")` decorator that emits a warning when the function is called, then runs the function normally.

The 30-second answer: The decorator uses `warnings.warn` with `DeprecationWarning` category. Inside the wrapper, emit the warning once per call site or once per function (use `stacklevel=2` so the warning points to the caller, not to the wrapper). Then call the wrapped function normally. The decorator does not break behaviour — it just adds the warning.

Step-by-step thought process: 1. Use `warnings.warn(msg, DeprecationWarning, stacklevel=2)`. `stacklevel=2` makes Python show the warning at the caller's line, not at the wrapper's `warn` call. 2. Format the message with the function name and the reason. `f"{f.__name__} is deprecated: {reason}"`. 3. Optional — warn only once per call site by tracking which (file, line) tuples have been warned. Or rely on Python's default once-per-location. 4. Return `f(*args, **kwargs)` after warning. Do not break the function.

Edge cases to mention: - `DeprecationWarning` is silent by default in user code (only shown in `__main__` and tests). Use `PendingDeprecationWarning` for less urgent, `FutureWarning` for end-user visibility. - `since=` parameter — `@deprecated(reason="x", since="2.5")` documents when deprecation started. Useful for changelogs. - Python 3.13 adds `warnings.deprecated` as a built-in decorator. Until you can target 3.13, roll your own.

What NOT to say: - "Use `print` to warn." Warnings have configurable filters; print does not. - "Just remove the function." Breaks all callers immediately. Deprecate first, remove later.

Common follow-up: "How does `warnings.warn` differ from `logging.warning`?" — Warnings have categories (`DeprecationWarning`, `UserWarning`, etc.) and can be filtered/promoted to errors via `-W` flags. Logging is for runtime telemetry, not API evolution.

```
import functools, warnings

def deprecated(reason="", since=""):
    def decorator(f):
        @functools.wraps(f)
        def wrapper(*a, **k):
            msg = f"{f.__name__} is deprecated"
            if since: msg += f" since {since}"
            if reason: msg += f": {reason}"
            warnings.warn(msg, DeprecationWarning, stacklevel=2)
            return f(*a, **k)
        return wrapper
    return decorator

@deprecated(reason="use new_api() instead", since="2.5")
def old_api():
    return "ok"

old_api()
# DeprecationWarning: old_api is deprecated since 2.5: use new_api() instead
```

Q42 · @retry with backoff and exception filtering

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, JP Morgan · Topic: Decorators · Patterns

The question (interviewer's verbatim phrasing):

Write a production-grade `@retry` decorator. It should support max attempts, exponential backoff with jitter, and only retry on a specified exception class.

The 30-second answer: Parameters — `attempts`, `base_delay`, `max_delay`, `exceptions` (tuple of retryable types), `jitter` (random factor). Each attempt: try the call, catch the listed exceptions, sleep `min(base_delay * 2**i + random_jitter, max_delay)`, repeat up to `attempts`. After last attempt, re-raise the last exception. For Razorpay payment APIs, only retry on `ConnectionError` and 5xx — never on 4xx or successful idempotency-key conflicts.

Step-by-step thought process: 1. Parameters. `attempts=3`, `base_delay=1.0`, `max_delay=30.0`, `exceptions=(Exception,)`, `jitter=0.1`. 2. Loop. `for i in range(attempts):` — try, on exception in `exceptions` sleep and retry, on other exception re-raise immediately. 3. Backoff. Wait time = `min(base_delay * 2**i + uniform(0, jitter * delay), max_delay)`. Exponential with jitter prevents thundering-herd retries. 4. Re-raise. After `attempts` failures, raise the last exception (or wrap in a `RetryExhausted` custom exception).

Edge cases to mention: - Idempotency. Only retry idempotent operations. POST `/payments/capture` is not safely retryable unless the API supports idempotency keys. - Async — write `async def wrapper` with `await asyncio.sleep`. Pattern identical otherwise. - Logging — log each retry attempt with the attempt number and the exception. Production retries should be observable.

What NOT to say: - "Retry all exceptions." Catches `KeyboardInterrupt`, `SystemExit`, programming bugs. Be narrow. - "Skip the jitter." Without jitter, N replicas retry in lockstep and overload the downstream service.

Common follow-up: "How does `tenacity` (the library) differ from a hand-rolled `@retry`?" — Tenacity adds composable `retry_if_exception_type`, `wait_random_exponential`, `stop_after_attempt`, `before_sleep` hooks. Worth using over hand-rolled for anything non-trivial.

```
import functools, time, random

def retry(*, attempts=3, base_delay=1.0, max_delay=30.0,
         exceptions=(Exception,), jitter=0.1):
    def decorator(f):
        @functools.wraps(f)
        def wrapper(*a, **k):
            last = None
            for i in range(attempts):
                try:
                    return f(*a, **k)
                except exceptions as e:
                    last = e
                    if i + 1 == attempts:
                        break
                    delay = min(base_delay * (2 ** i), max_delay)
                    delay += random.uniform(0, jitter * delay)
                    time.sleep(delay)
            raise last
        return wrapper
    return decorator

@retry(attempts=4, exceptions=(ConnectionError, TimeoutError))
def fetch_payment_status(order_id): ...
```

Q43 · `@validate_args` — type-checking decorator

Tags: Difficulty: Medium · Asked at: Razorpay, Postman · Topic: Decorators · Patterns

The question (interviewer's verbatim phrasing):

Write a `@validate_args` decorator that reads the function's type annotations and raises `TypeError` if arguments do not match the annotated types.

The 30-second answer: Inside the decorator, read `f.__annotations__` to get the expected types. In the wrapper, use `inspect.signature(f).bind(*args, **kwargs)` to map runtime args to parameter names. For each argument, `isinstance(value, expected_type)`. Raise `TypeError` with a clear message on mismatch. For real production use, `pydantic.validate_call` does this better.

Step-by-step thought process: 1. Read annotations once at decoration time — `sig = inspect.signature(f)`. Pre-compute the type for each parameter from `param.annotation`. 2. In the wrapper, `bound = sig.bind(*args, **kwargs)` maps the call to parameter names. `bound.arguments` is an ordered dict. 3. For each `(name, value)`, look up the expected type and run `isinstance(value, expected)`. Skip parameters without annotations. 4. Raise. `raise TypeError(f"{name} expected {expected.__name__}, got {type(value).__name__}")`.

Edge cases to mention: - `Optional[X]` is `Union[X, None]` — use `typing.get_origin` and `typing.get_args` to handle generics. Plain `isinstance` does not work for `List[int]`. - For real type checking with generics, use `typeguard` or `pydantic.validate_call`. Hand-rolled is for simple cases. - Annotations can be strings (PEP 563 future annotations) — use `typing.get_type_hints(f)` to resolve them.

What NOT to say: - "Just use type hints alone." Type hints are not enforced at runtime; you need explicit validation. - "Pydantic is too heavy." For non-trivial validation, pydantic is fast and well-tested. Hand-rolling is a learning exercise.

Common follow-up: "How would you handle `List[int]` — would `isinstance(value, List[int])` work?" — No. Use `typing.get_origin(annotation)` to get `list`, then check `isinstance(value, list)` and validate each element separately.

```
import functools, inspect

def validate_args(f):
    sig = inspect.signature(f)
    hints = inspect.get_annotations(f, eval_str=True)
    @functools.wraps(f)
    def wrapper(*args, **kwargs):
        bound = sig.bind(*args, **kwargs)
        for name, value in bound.arguments.items():
            expected = hints.get(name)
            if expected is None or expected is type(None):
                continue
            if not isinstance(value, expected):
                raise TypeError(
                    f"{name} expected {expected.__name__}, "
                    f"got {type(value).__name__}"
                )
        return f(*args, **kwargs)
    return wrapper

@validate_args
def transfer(amount: int, currency: str):
    return f"{amount} {currency}"

transfer(100, "INR")          # ok
# transfer(100.0, "INR")     # TypeError: amount expected int, got float
```

Q44 · @timer — measure function execution time

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Swiggy · Topic: Decorators · Patterns

The question (interviewer's verbatim phrasing):

Write a `@timer` decorator that records and reports how long each function call takes. Use it on a database fetch.

The 30-second answer: Wrapper records start time with `time.perf_counter()`, calls the function, computes elapsed, logs it (or sends to a metrics system like StatsD/Prometheus). Use `perf_counter`, not `time.time()` — `perf_counter` is monotonic and high-resolution; `time.time` is wall-clock and can jump backward on NTP adjustments. Production version pushes the metric to your observability backend instead of printing.

Step-by-step thought process: 1. Use `time.perf_counter()` for monotonic high-res timing. `time.time()` is wall-clock and can drift. 2. Compute `elapsed = perf_counter() - start` after the call. Use `try/finally` to record even on exceptions. 3. Output. For a demo, `print`. For

production, `logger.info` or push to a metrics backend with the function name as a tag. 4. Async — write `async def wrapper`, time around `await f(*args, **kwargs)`.

Edge cases to mention: - Async — `await` releases the loop, so the timer measures wall-clock including time the coroutine was paused. Often what you want; sometimes not. - Exceptions — record timing even on failure, so you do not lose data for failed paths. Use `try/finally`. - High-frequency calls — push to a histogram (Prometheus) rather than logging each call. Logging every microsecond floods.

What NOT to say: - "Use `datetime.now()` for timing." Wall-clock, not monotonic. Wrong tool. - "Just print elapsed." Production observability needs structured metrics, not printlines.

Common follow-up: "What's the difference between `perf_counter` and `process_time` for timing?" — `perf_counter` is monotonic wall-clock including sleeps. `process_time` excludes sleeps; useful for CPU profiling, not for "how long did the call take from the user's view."

```
import functools, time, logging

log = logging.getLogger(__name__)

def timer(f):
    @functools.wraps(f)
    def wrapper(*a, **k):
        start = time.perf_counter()
        try:
            return f(*a, **k)
        finally:
            elapsed = time.perf_counter() - start
            log.info("%s took %.4fs", f.__name__, elapsed)
    return wrapper

@timer
def fetch_user(uid):
    time.sleep(0.05)
    return {"id": uid}

fetch_user(42)
# INFO:__main__:fetch_user took 0.0501s
```

Q45 · `@log_calls` — structured call logging

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Postman · Topic: Decorators · Patterns

The question (interviewer's verbatim phrasing):

Write a `@log_calls` decorator that logs each call with its arguments and return value. What should you redact?

The 30-second answer: Wrapper logs `f.__name__`, the args, and the result. For PII or secret args (passwords, tokens, card numbers), redact based on a list of sensitive parameter names. Logging at `INFO` for normal calls, `ERROR` for exceptions. Production version uses structured logging (JSON) so log aggregators can index by function name and outcome.

Step-by-step thought process: 1. Log the call. Format `f"{f.__name__}(args, kwargs)"`. Use `repr` on args/kwargs to escape strings. 2. Redact. Maintain a set of sensitive names — `{"password", "token", "card_number", "cvv"}`. If a kwarg key matches, log `"**"` instead of the value. 3. Log the result on success, the exception on failure. Use `logger.exception()` in the failure path to capture the traceback. 4. Structured logging — emit a dict with `function`, `args`, `outcome`, `latency_ms` rather than a flat string. Modern log pipelines parse JSON natively.

Edge cases to mention: - Large args (10MB strings) flood logs. Truncate or summarise. `repr(arg)[:200]` is a quick guard. - Async — log start and end separately if you want to see in-flight calls. Synchronous decorator logs only after completion. - Recursive calls — log each invocation; can produce a lot of noise for recursive algorithms. Consider a depth limit.

What NOT to say: - "Log everything." Floods. Pick info, debug, or trace based on production volume. - "Don't worry about redaction." PII leakage in logs is a compliance failure (especially under DPDP Act in India).

Common follow-up: "What redaction pattern works for nested dicts?" — Walk the dict recursively; for any key matching the sensitive set, replace value with `"**"`. Or use a structured logger with field-level masking.

```

import functools, logging

log = logging.getLogger(__name__)

SENSITIVE = {"password", "token", "card_number", "cvv", "secret"}

def _redact(kwarg):
    return {k: ("***" if k in SENSITIVE else v) for k, v in kwarg.items()}

def log_calls(f):
    @functools.wraps(f)
    def wrapper(*args, **kwarg):
        log.info("call %s args=%s kwarg=%s",
                 f.__name__, args, _redact(kwarg))
        try:
            result = f(*args, **kwarg)
            log.info("done %s result=%r", f.__name__, result)
            return result
        except Exception:
            log.exception("error %s", f.__name__)
            raise
    return wrapper

@log_calls
def login(user, password=None):
    return {"user": user}

login("abhi", password="hunter2")
# call login args=('abhi',) kwarg={'password': '***'}
# done login result={'user': 'abhi'}

```

Q46 · `@require_auth` — role-based access control

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India · Topic: Decorators · Patterns

The question (interviewer's verbatim phrasing):

Write `@require_auth(role="admin")` for Flask-style request handlers. How does the decorator access the current user?

The 30-second answer: The decorator factory takes a role. The wrapper inspects the request context — `flask.g.user`, `request.user`, or whatever your framework exposes — and checks the user's role. If unauthorised, raise a framework exception (`Forbidden`, `HTTPException(403)`) or return an error response. If authorised, call the handler normally. The "where the user comes from" is framework-specific; the wrapper just reads it.

Step-by-step thought process: 1. Factory takes `role`. Returns a decorator closed over the role. 2. Wrapper reads the current user from context. Flask: `flask.g.user`. FastAPI: dependency-injected. Django: `request.user` is the first kwarg. 3. Check authorisation. If `user is None`, raise 401 Unauthorised. If user exists but role does not match, raise 403 Forbidden. 4. On success, call the handler. The wrapper does not modify args.

Edge cases to mention: - Multi-role — accept `roles=("admin", "manager")` and check membership. More flexible than single role. - Hierarchical roles — admin implies manager implies user. Encode in a lookup or in the user model. - FastAPI uses dependencies instead of decorators — `Depends(check_admin)` is more idiomatic there. Mention as alternative.

What NOT to say: - "Decorators can't do auth." They can, and they are the standard pattern in Flask/Django. - "Just inline the check." That is what decorators avoid — DRY across every protected route.

Common follow-up: "What's the FastAPI equivalent?" — `Depends(verify_admin_role)` — a function that raises `HTTPException(403)` if unauthorised. FastAPI injects it as a parameter and the dependency runs automatically.

```
import functools
from flask import g, abort

def require_auth(role=None):
    def decorator(f):
        @functools.wraps(f)
        def wrapper(*args, **kwargs):
            user = getattr(g, "user", None)
            if user is None:
                abort(401)
            if role and user.role != role:
                abort(403)
            return f(*args, **kwargs)
        return wrapper
    return decorator

@require_auth(role="admin")
def delete_user(user_id):
    return f"deleted {user_id}"
```

Q47 · `@cached_with_ttl` — cache with expiration

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman · Topic: Decorators · Patterns

The question (interviewer's verbatim phrasing):

`lru_cache` does not expire entries. Write a TTL cache decorator — entries expire after N seconds.

The 30-second answer: Store each cache entry as `(value, expiry_timestamp)`. On lookup, check if `now > expiry`; if expired, treat as miss and recompute. On insertion, set `expiry = now + ttl`. The decorator factory takes a `ttl` parameter; the wrapper does the check-or-compute under a lock. Memory is bounded by either LRU eviction or a max-size cap.

Step-by-step thought process: 1. Cache stores `{key: (value, expiry_time)}`. Use `time.monotonic()` for expiry to avoid wall-clock issues. 2. Lookup. `if key in cache and cache[key][1] > now: return cache[key][0]`. Else compute and store. 3. Cleanup. Periodically remove expired entries to keep memory bounded, or just lazily evict on lookup. 4. Lock for thread safety. Same pattern as Q33.

Edge cases to mention: - Lazy vs eager eviction. Lazy is simpler but lets stale entries linger. Eager (scheduled cleanup) frees memory but adds a thread. - LRU + TTL — pre-built libraries like `cachetools.TTLCache` combine both. Use them in production. - TTL of 0 — always expired, effectively no cache. Useful for disabling.

What NOT to say: - "Just use `lru_cache` with `cache_clear()` on a schedule." Coarse — clears the whole cache, not just expired entries. - "TTL is the same as `maxsize`." Different — TTL is time-based, `maxsize` is count-based.

Common follow-up: "How would you combine TTL with LRU?" — `cachetools.TTLCache(maxsize=128, ttl=60)` does this. Each entry has a TTL and the cache evicts both expired entries and the LRU when at capacity.

```
import functools, time, threading

def cached_with_ttl(ttl=60):
    def decorator(f):
        cache = {}
        lock = threading.Lock()
        @functools.wraps(f)
        def wrapper(*args):
            now = time.monotonic()
            with lock:
                if args in cache:
                    value, expiry = cache[args]
                    if expiry > now:
                        return value
                value = f(*args)
                cache[args] = (value, now + ttl)
                return value
        wrapper.cache = cache
        wrapper.cache_clear = lambda: cache.clear()
        return wrapper
    return decorator

@cached_with_ttl(ttl=5)
def fetch_rate(currency):
    return 83.50    # mock
```

Q48 · Flask `@app.route` and FastAPI `@app.get` — how they work

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Freshworks · Topic: Decorators · Patterns

The question (interviewer's verbatim phrasing):

What does `@app.route("/users")` in Flask actually do under the hood? Compare to `@app.get("/users")` in FastAPI.

The 30-second answer: `@app.route("/users")` calls `app.route("/users")`, returning a decorator that registers the function in Flask's URL map under the path and returns the function unchanged. So the function name still references the original — no wrapping. FastAPI's `@app.get("/users")` does the same conceptually, but it also introspects the function's signature (annotations, defaults) to build the request-parsing and response-serialisation logic, then registers the route. Both run at import time.

Step-by-step thought process: 1. Both are registration decorators. They register the handler with the framework's router and return the function (or a wrapped version). 2. Flask's `route` is thin — registers a rule in `app.url_map` keyed by path, returns the function as-is. 3. FastAPI is heavier. It uses

`inspect.signature` to detect path params, query params, body schema, dependencies. Builds an internal route object with all that pre-computed. 4. Both run at import. So defining your routes in a module triggers all the registration when the module is imported.

Edge cases to mention: - Flask `@app.route` accepts `methods=["POST"]` — defaults to GET. FastAPI has `@app.post`, `@app.get`, etc. as separate decorators. - FastAPI's signature inspection means you should always use type hints — without them, the framework cannot infer the request schema. - Multiple routes per function. Flask allows `@app.route("/a"); @app.route("/b")` stacked. FastAPI uses `@app.api_route(path, methods=...)` or stacked decorators.

What NOT to say: - "They wrap the function." Flask doesn't really wrap; it registers and returns the original. FastAPI wraps minimally. - "Routes are evaluated lazily." They register at import time, eagerly.

Common follow-up: *"What happens if I define the same route on two different functions?"* — Flask raises `AssertionError`. FastAPI accepts and the second registration wins (last one called registered).

```
# Flask-style minimal route decorator
class App:
    def __init__(self):
        self.routes = {}

    def route(self, path, methods=("GET",)):
        def decorator(f):
            for m in methods:
                self.routes[(m, path)] = f
            return f          # returns unchanged
        return decorator

app = App()

@app.route("/users", methods=("GET", "POST"))
def users():
    return "list of users"

print(app.routes)
# {('GET', '/users'): <function users>, ('POST', '/users'): <function users>}
```

Q49 · `@pytest.fixture` and `@pytest.mark` — pytest's decorator landscape

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Freshworks · Topic: Decorators · Patterns

The question (interviewer's verbatim phrasing):

Walk me through `@pytest.fixture` and `@pytest.mark.parametrize`. How do they wire fixtures into tests?

The 30-second answer: `@pytest.fixture` marks a function as a fixture — registers it with pytest so other test functions that have the fixture name as a parameter receive the fixture's return value. `@pytest.mark.parametrize("x", [1, 2, 3])` registers the test for multiple runs with `x` taking each value. Both work via introspection: pytest reads test function signatures and matches parameter names to fixtures or to parametrize values.

Step-by-step thought process: 1. `@pytest.fixture`. Returns a marked function. pytest collects fixtures at session start. Test functions whose parameter names match a fixture name receive the fixture's return value. 2. `@pytest.mark.parametrize`. Adds parametrization metadata to the function. pytest re-runs the test for each value. 3. Fixture scopes — `@pytest.fixture(scope="session")` creates the fixture once per test session; `scope="function"` (default) is per test. Scope is a fixture-config parameter. 4. Discovery — pytest discovers tests by name (`test_*`) and fixtures by decorator marker. The decorator is the registration mechanism.

Edge cases to mention: - Fixture with `yield` — setup before yield, teardown after. `@pytest.fixture` plus generator function. - `autouse=True` — fixture applies to every test without being named in parameters. Use sparingly; surprises debuggers. - Parametrize stacking — `@pytest.mark.parametrize("x", ...)\n@pytest.mark.parametrize("y", ...)` produces the cartesian product.

What NOT to say: - "`@pytest.fixture` is just `@property`." It is not — completely different mechanism. - "Fixtures are global." They are scoped, with explicit lifetimes.

Common follow-up: "How does pytest know which fixture to inject?" — By parameter name match. If your test takes `def test_thing(db, user)`, pytest looks up fixtures named `db` and `user` and injects their values.

```
# pseudo-pytest, illustrating the registration mechanic

_FIXTURES = {}

def fixture(scope="function"):
    def decorator(f):
        _FIXTURES[f.__name__] = (f, scope)
        return f
    return decorator

@fixture()
def db():
    return {"connection": "open"}

# A test runner would do:
# for test in tests:
#     sig = inspect.signature(test)
#     kwargs = {name: _FIXTURES[name][0]() for name in sig.parameters}
#     test(**kwargs)
```

Q50 · Django's `@login_required` and the `@method_decorator` adapter

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Freshworks · Topic: Decorators · Patterns

The question (interviewer's verbatim phrasing):

Django's `@login_required` is a function decorator. How do you apply it to a class-based view, which has `dispatch` rather than a single function?

The 30-second answer: Use `@method_decorator(login_required, name='dispatch')` on the class. `method_decorator` adapts a function-style decorator (designed to wrap a view function) so it can wrap a class-based view's method. It handles the descriptor protocol so `self` is correctly bound when the decorator's wrapper is called. Alternative: apply to `dispatch` directly via `@method_decorator(login_required)` above the `dispatch` method.

Step-by-step thought process: 1. Problem. `login_required` expects a request as its first argument, but a class-based view's method has `self` as first. 2. `method_decorator(decorator, name='dispatch')` wraps the named method on the class. It handles binding so the decorator sees a function whose first arg is request, not self. 3. Apply on the class (`@method_decorator(login_required, name='dispatch')`) or on the method directly. 4. Cleaner alternative — Django's `LoginRequiredMixin`. Subclass it instead of using the decorator. Pythonic, less indirection.

Edge cases to mention: - `@method_decorator` accepts a list of decorators —

`@method_decorator([login_required, csrf_protect], name='dispatch')` stacks them.

- For DRF (Django REST Framework), `permission_classes = [IsAuthenticated]` is the idiomatic equivalent — declarative, not decorator-based. - `LoginRequiredMixin` propagates to subclasses via inheritance; decorator does not.

What NOT to say: - "You can't use function decorators on classes." You can, with `method_decorator`. -

"Use mixins instead." Often right, but the question is about how to use the decorator. Answer both.

Common follow-up: "What's the difference between `LoginRequiredMixin` and

`@login_required` on `dispatch`?" — `LoginRequiredMixin` is inheritance-based —

subclasses automatically check login. Decorator is per-class — needs to be applied to every view. Mixin is more Pythonic; decorator is more explicit.

```
# Pseudo-Django to illustrate the adapter
import functools

def login_required(view_func):
    @functools.wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not getattr(request, "user_authenticated", False):
            return "redirect to /login"
        return view_func(request, *args, **kwargs)
    return wrapper

def method_decorator(decorator, name):
    def cls_decorator(cls):
        original = getattr(cls, name)
        @functools.wraps(original)
        def bound(self, request, *args, **kwargs):
            # decorator expects a function; build one that includes self
            inner = lambda req, *a, **k: original(self, req, *a, **k)
            return decorator(inner)(request, *args, **kwargs)
        setattr(cls, name, bound)
        return cls
    return cls_decorator

@method_decorator(login_required, name="dispatch")
class DashboardView:
    def dispatch(self, request):
        return "dashboard for " + request.user

class _Req:
    user_authenticated = True
    user = "abhi"

print(DashboardView().dispatch(_Req())) # dashboard for abhi
```

End of Chapter 2.

Fifty questions across decorator basics, `functools.wraps`, decorator factories, class decorators, stateful decorators, caching decorators, and real-world patterns. The senior-level signal in this chapter is the parameterised decorator (Q16, Q22), the wraps gotchas (Q14, Q15), and the framework decorators (Q48, Q49, Q50). If you can write a three-level parameterised decorator on the whiteboard, defend the choice of `cache` vs `lru_cache` vs `cached_property` for a specific use case, and explain why `@login_required` needs `method_decorator` on a class-based view — you have signalled mid-to-senior Python.

The next chapter is generators and iterators — equally heavy in interviews, equally rich in traps.

Chapter 3 — Generators + Iterators · Sub-chunk 1 of 6 · Iterator Protocol (Q1–Q10)

Python Interview Prep — 2026 Edition · Learn & Excel · File 04 Core Python Library

This sub-chunk covers the iterator protocol — the contract under every `for` loop in Python. Razorpay panels for backend roles, Swiggy for order-pipeline questions, Flipkart for catalogue streaming, Cred for transaction ledgers, Postman for parsing engines, and Service MNCs (TCS / Infosys / Wipro) for screening rounds all probe whether you understand the `__iter__` / `__next__` contract — not just that `for x in xs:` works. Most candidates can write a generator; far fewer can explain what the `for` loop calls under the hood and why `iter(obj)` returning `obj` is the rule that separates an iterable from an iterator.

Each Q uses the same shape: **30-second answer** (what you say first if the panel is impatient), **thought process** (how to reason out loud), a **code** block (compiles on Python 3.12+), the **gotchas** (where panels watch you trip), **what NOT to say**, **common follow-up**, and a **stand-out line** — the single sentence that signals you've read the data model.

Q1 · `__iter__` and `__next__` — the iterator protocol contract

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, TCS, Infosys, Flipkart · Topic: Iterator protocol fundamentals

The question: "What two methods define Python's iterator protocol, and what does each return?"

The 30-second answer: An object is an **iterator** if it implements `__next__` (returns the next item or raises `StopIteration`) AND `__iter__` (returns `self`). An object is an **iterable** if it implements

`__iter__` (returns a fresh iterator). The `for` loop calls `iter(obj)` once to get an iterator, then `next(iterator)` until `StopIteration` fires.

Step-by-step thought process: 1. "There are two protocols, not one — iterable and iterator. People conflate them." 2. "`__iter__` on an iterable returns a brand new iterator. `__iter__` on an iterator returns `self` — that's the rule." 3. "`__next__` is the work method. It either returns the next value or raises `StopIteration` to signal the end." 4. "`StopIteration` is not an error in the bug sense — it's the agreed-upon signal. The `for` loop catches it silently." 5. "Once `StopIteration` fires, the iterator is exhausted. Calling `next()` again must keep raising `StopIteration` — you don't reset."

Code:

```
class Countdown:
    """An iterator – implements both __iter__ and __next__."""
    def __init__(self, start: int):
        self.current = start

    def __iter__(self):
        return self                # iterator returns SELF

    def __next__(self):
        if self.current <= 0:
            raise StopIteration    # signal: nothing more
        value = self.current
        self.current -= 1
        return value

c = Countdown(3)
print(next(c), next(c), next(c))    # 3 2 1
try:
    next(c)
except StopIteration:
    print("exhausted")              # exhausted

# In a for loop – iter() is called once, then next() until StopIteration
for n in Countdown(3):
    print(n)                        # 3, 2, 1
```

Gotchas: - Forgetting to raise `StopIteration` → infinite loop. The `for` loop will run forever. - `StopIteration` must be raised, not returned. `return None` from `__next__` returns `None` as a value (the loop keeps going). - An iterator that yields data AFTER `StopIteration` was raised once is broken — the protocol says once exhausted, stay exhausted.

What NOT to say: - "Just use `yield` ." — you're being asked the protocol underneath generators. Sidestepping signals you don't know what `yield` compiles to. - "`__iter__` returns a list of items." — wrong; it returns an iterator object, not a materialised collection.

Common follow-up: "What's the difference between an iterable and an iterator — and why does an iterator's `__iter__` return `self` ?"

Stand-out line: "`__iter__` says 'I can be iterated'; `__next__` says 'I am the iterator'. The `for` loop calls one then loops on the other — and `StopIteration` is the only legal way to say done."

Q2 · `iter()` and `next()` built-ins

Tags: Difficulty: Easy · Asked at: Cred, Postman, Walmart Labs, Infosys, Razorpay · Topic: Built-in iteration helpers

The question: "What do the `iter()` and `next()` built-ins actually do, and when would you use them directly instead of a `for` loop?"

The 30-second answer: `iter(obj)` calls `obj.__iter__()` and returns the iterator. `next(iterator)` calls `iterator.__next__()` and returns the next value (or raises `StopIteration`). You use them directly when you want a fresh iterator handle for manual stepping — peeking at the first item, pairing two iterables in lockstep, or processing a stream one chunk at a time.

Step-by-step thought process: 1. "These two built-ins are how the `for` loop talks to the protocol — they're thin wrappers." 2. "`iter(obj)` is one call per loop. After that, the loop only calls `next()` on the returned iterator." 3. "`next(it)` raises `StopIteration` when exhausted. `next(it, default)` returns the default instead — no exception. This two-arg form is gold for peek-style code." 4. "Direct use is rare in everyday code; it shows up in parsers, stream readers, and tests that need fine-grained control." 5. "For interview purposes, knowing the two-arg `next(it, default)` form alone signals practitioner-level fluency."

Code:

```
# Manual stepping – peek at the header row of a CSV stream
rows = iter([
    ("name", "amount"),
    ("alice", 500),
    ("bob", 700),
])
header = next(rows)                # ('name', 'amount')
for row in rows:                  # remaining rows
    print(dict(zip(header, row)))

# Two-arg next() – no try/except needed
it = iter([1, 2])
print(next(it, "done"))           # 1
print(next(it, "done"))           # 2
print(next(it, "done"))           # done (default kicks in)
print(next(it, "done"))           # done (still default; stays exhausted)
```

Gotchas: - `iter(obj)` raises `TypeError` if `obj` doesn't implement the protocol — not a generic "iteration failed" error. - The two-arg `next(it, default)` is the cleanest way to test for exhaustion without wrapping in `try/except StopIteration`. - Calling `iter()` on something that's already

an iterator is fine — it returns the same iterator (because iterator's `__iter__` returns self). Don't expect a reset.

What NOT to say: - "`iter()` resets the iterator." — wrong; calling `iter()` on an already-exhausted iterator returns the same exhausted iterator. - "`next()` is like `++`." — misleading C-ism; `next()` calls a method that can raise. It's not a pointer increment.

Common follow-up: "If you call `iter()` twice on the same list, do you get the same iterator or two independent ones?"

Stand-out line: "`iter()` and `next()` are the only two functions the `for` loop uses — once you know that, every `for` loop in Python is a four-line desugaring you can write by hand."

Q3 · `StopIteration` — when raised, the for-loop dance

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, JP Morgan, TCS, Postman · Topic: StopIteration semantics

The question: "What is `StopIteration` — how does the `for` loop handle it, and what happens if `StopIteration` leaks out of a generator?"

The 30-second answer: `StopIteration` is the sentinel exception that signals iterator exhaustion. The `for` loop wraps every `next()` call in an implicit `try/except StopIteration: break`. Inside a generator (PEP 479, default since Python 3.7), an unexpected `StopIteration` is converted to `RuntimeError` to prevent silent loop termination.

Step-by-step thought process: 1. "It's a control-flow exception, not an error exception. Catching it is normal; letting it propagate from where it shouldn't is the bug." 2. "The `for` loop's pseudocode: `while True: try: x = next(it) except StopIteration: break; <body>`." 3. "PEP 479 closed a footgun — pre-3.7, a `StopIteration` raised inside a generator (e.g. from `next(other_iter)`) would silently end the generator. That caused real bugs in pipelines." 4. "Now, a stray `StopIteration` inside a generator becomes `RuntimeError: generator raised StopIteration` — much louder." 5. "`StopIteration` has a `.value` attribute — used by generators to communicate a final value via `return` statement. Most code ignores it."

Code:

```

# What the for loop actually does
def manual_for(iterable, body):
    it = iter(iterable)
    while True:
        try:
            x = next(it)
        except StopIteration:
            break
        body(x)

manual_for([1, 2, 3], print)           # 1 2 3 – same as a real for loop

# PEP 479 – StopIteration inside a generator becomes RuntimeError
def generator_with_leak():
    yield 1
    next(iter([]))                     # raises StopIteration – leaks

g = generator_with_leak()
print(next(g))                        # 1
try:
    next(g)
except RuntimeError as e:
    print(type(e).__name__, str(e)[:50]) # RuntimeError generator raised StopIteration

# Generator return value rides on StopIteration.value
def with_return():
    yield 1
    yield 2
    return "final"

g = with_return()
try:
    while True:
        print(next(g))                # 1, 2
except StopIteration as e:
    print("return value:", e.value)    # return value: final

```

Gotchas: - Pre-PEP 479 code that relied on `StopIteration` to break a generator silently is now broken on Python 3.7+. Audit before upgrading old codebases. - `StopIteration.value` carries the generator's `return` value — `async/await` machinery and pipelines use this; ordinary `for` loops discard it. - Catching `Exception` instead of `StopIteration` in custom iteration code swallows real bugs. Catch `StopIteration` specifically.

What NOT to say: - "`StopIteration` is an error." — it's a control-flow signal. Calling it an error suggests you'd log it or treat it as a bug. - "I always catch `Exception` to be safe." — catches `StopIteration` too, which breaks iteration protocols.

Common follow-up: "What changed in PEP 479, and why was the old behaviour considered a bug?"

Stand-out line: " `StopIteration` is exhaustion-as-an-exception — clever because it lets `next()` return any value (including `None`) without ambiguity, but the trade-off is that you have to keep the exception contained inside the iterator boundary. "

Q4 · Iterable vs iterator — the "iter on iter returns self" rule

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Flipkart, Walmart Labs, Infosys · Topic: Protocol distinction

The question: "What's the difference between an iterable and an iterator — and why does calling `iter()` on an iterator return the iterator itself?"

The 30-second answer: An iterable is anything you can call `iter()` on (lists, strings, dicts) — its `__iter__` returns a **fresh iterator** each call. An iterator is the stateful, single-pass object that produces values — its `__iter__` returns `self`. The "returns self" rule means you can put an iterator into a `for` loop directly; the loop calls `iter()` and gets back the same iterator, then steps via `next()`.

Step-by-step thought process: 1. "Iterable = re-iterable (most of the time). Iterator = one-shot, stateful." 2. "A list is an iterable, not an iterator. `iter(list)` returns a `list_iterator` — that's the actual iterator with `__next__`." 3. "Why must iterator's `__iter__` return self? So the `for` loop can accept either an iterable or an iterator transparently. Same call site, same protocol." 4. "Concrete consequence: a generator function returns a generator object that IS an iterator — `for x in gen()` works once, then the generator is exhausted." 5. "You can usually iterate a list twice (two `for` loops, two separate iterators). You can NEVER iterate a generator twice — it's the same exhausted iterator."

Code:

```

xs = [1, 2, 3]                                # iterable, not iterator
it1 = iter(xs)                                # fresh iterator
it2 = iter(xs)                                # ANOTHER fresh iterator
print(it1 is it2)                             # False – independent

# Iterator's __iter__ returns self
print(iter(it1) is it1)                       # True – same object

# Two passes over a list – works, each loop calls iter() fresh
for x in xs: print(x, end=" ")                # 1 2 3
print()
for x in xs: print(x, end=" ")                # 1 2 3 – independent pass
print()

# Two passes over a generator – second pass is empty
def gen():
    yield 1; yield 2; yield 3

g = gen()
for x in g: print(x, end=" ")                  # 1 2 3
print()
for x in g: print(x, end=" ")                  # (nothing – exhausted)
print("done")

# Why the rule matters – for-loop accepts both
for x in [1, 2, 3]: pass                       # iter() returns a fresh iterator
for x in iter([1, 2, 3]): pass                 # iter() returns the iterator itself

```

Gotchas: - Passing a generator to a function that iterates it twice gives empty results on the second pass — common bug in data pipelines. - `file` objects are iterators (not iterables) — once read to EOF, you can't re-read without `seek(0)`. - `zip()`, `map()`, `filter()`, `enumerate()` all return iterators — one-shot. Wrap in `list()` if you need multiple passes.

What NOT to say: - "They're the same thing." — they're not; the distinction is the entire point of the protocol. - "Lists are iterators." — they're iterables. `iter(list)` returns a `list_iterator`, which is the actual iterator.

Common follow-up: "What happens if you iterate over a `zip()` object twice — and how would you make it re-iterable?"

Stand-out line: "The rule 'iterator's `__iter__` returns self' is what lets the `for` loop be polymorphic — it accepts iterables and iterators with one call, because every iterator is also (trivially) an iterable."

Q5 · Custom iterator class — full example

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Swiggy, TCS, Cred · Topic: Implementing the protocol

The question: *"Write a custom iterator class — say, one that yields prime numbers up to N — and explain how it differs from a generator function."*

The 30-second answer: A custom iterator class needs `__iter__` returning `self` and `__next__` returning the next value or raising `StopIteration`. The class keeps its state in instance attributes — `self.current`, `self.limit`, etc. The same logic as a generator function but spelled out — you trade `yield`'s implicit state machine for explicit instance state.

Step-by-step thought process: 1. "Two methods only. `__iter__` returns `self`, `__next__` does the work." 2. "State lives on the instance — `self.current`, `self.limit`. Every `next()` call uses and updates this state." 3. "Compare to a generator: a generator function with `yield` compiles to the same protocol — the interpreter creates the instance state automatically from local variables paused at each `yield`." 4. "Class form wins when you need extra methods, state inspection, or multiple separate iterators backed by shared resources." 5. "Generator form wins for 90% of cases — less ceremony, easier to read. But interviewers ask the class form because it forces you to articulate the protocol."

Code:

```

class PrimesUpTo:
    """Iterator that yields prime numbers up to (and including) limit."""
    def __init__(self, limit: int):
        self.limit = limit
        self.current = 1          # last checked

    def __iter__(self):
        return self

    def __next__(self):
        while True:
            self.current += 1
            if self.current > self.limit:
                raise StopIteration
            if self._is_prime(self.current):
                return self.current

    @staticmethod
    def _is_prime(n: int) -> bool:
        if n < 2: return False
        for d in range(2, int(n ** 0.5) + 1):
            if n % d == 0:
                return False
        return True

print(list(PrimesUpTo(20)))          # [2, 3, 5, 7, 11, 13, 17, 19]

# Equivalent generator function – much shorter
def primes_up_to(limit: int):
    for n in range(2, limit + 1):
        if all(n % d != 0 for d in range(2, int(n ** 0.5) + 1)):
            yield n

print(list(primes_up_to(20)))        # same output

# Class form lets you do things generators can't easily – inspect state
p = PrimesUpTo(20)
next(p); next(p); next(p)
print(p.current)                    # 5 – generator equivalent is opaque

```

Gotchas: - Forgetting `__iter__` returning `self` makes the object usable with `next()` but NOT with `for` (because `for` calls `iter()` first and expects something with `__next__`). - Storing the iteration state on the iterable (instead of a separate iterator) means two parallel `for` loops share state — the second corrupts the first. - Subclassing `collections.abc.Iterator` gives you a free `__iter__` and forces you to implement `__next__` — preferred for production code.

What NOT to say: - "Just use a generator." — true in real code, but the question is testing protocol knowledge. Answer the question first, then mention generators. - "I'll store the state on the class, not the instance." — class-level state shared across instances is a memory-bug pattern.

Common follow-up: "Convert this class to a generator function — what state disappears, and what state stays on the underlying frame?"

Stand-out line: "A generator function is just sugar — the bytecode the interpreter generates is identical to writing the class form by hand, with `yield` becoming a pause-and-return point and local variables becoming the implicit instance state."

Q6 · Infinite iterators — `itertools.count`, `cycle`, `repeat`

Tags: Difficulty: Medium · Asked at: Swiggy, Flipkart, Postman, Walmart Labs, JP Morgan · Topic: `itertools` infinities

The question: "What are `itertools.count`, `cycle`, and `repeat` — and how do you safely use them when they never raise `StopIteration`?"

The 30-second answer: They're three infinite iterators. `count(start, step)` yields an arithmetic sequence forever. `cycle(iterable)` loops over an iterable forever. `repeat(value, times)` yields the same value either forever (if `times` is omitted) or N times. Safe usage requires pairing them with something that bounds the loop — `zip`, `islice`, or a `break` condition.

Step-by-step thought process: 1. "All three live in `itertools`. They're C-implemented, memory-O(1) — they don't materialise the sequence." 2. "`count(10, 2)` → 10, 12, 14, 16, ... forever. Useful as an index source with `zip`." 3. "`cycle([1, 2, 3])` → 1, 2, 3, 1, 2, 3, 1, ... forever. Useful for round-robin assignment." 4. "`repeat('x', 5)` → 'x', 'x', 'x', 'x', 'x'. Without the 5, it's forever." 5. "The safety rule: never put an infinite iterator in a bare `for x in inf: ...` — pair it with `zip(inf, finite)`, `islice(inf, N)`, or a `break`."

Code:

```

from itertools import count, cycle, repeat, islice

# count – useful for indexed enumeration with a custom start
for i, name in zip(count(1), ["alice", "bob", "carol"]):
    print(f"{i}. {name}")
# 1. alice / 2. bob / 3. carol

# cycle – round-robin shard assignment for an order pipeline
shards = cycle(["shard-A", "shard-B", "shard-C"])
orders = ["order-1", "order-2", "order-3", "order-4", "order-5"]
for order, shard in zip(orders, shards):
    print(order, "→", shard)
# order-1 → shard-A / order-2 → shard-B / order-3 → shard-C / order-4 → shard-A / order-5 → shard-B

# repeat – supply a constant to map/zip
totals = list(zip(["alice", "bob", "carol"], repeat(0)))
print(totals)
# [('alice', 0), ('bob', 0), ('carol', 0)]

# islice – bound an infinite iterator
first_10_evens = list(islice(count(0, 2), 10))
print(first_10_evens)
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

```

Gotchas: - `list(count(0))` hangs forever and eventually OOMs the process. Same for `list(cycle(...))` and unbounded `repeat()`. - `cycle()` internally caches the input — feeding it a huge list keeps that list in memory forever. - `count()` defaults to `count(0, 1)` — same as `itertools.count()` with no args. The step can be a float.

What NOT to say: - "I'd use a `while True` loop instead." — works but misses the point; the question is testing knowledge of `itertools`, which is heavily used in Indian backend codebases. - "Just put it in a `for` loop." — without bounding, you've written an infinite loop.

Common follow-up: "How would you assign N orders to a fixed pool of K workers in round-robin fashion without materialising the sequence?"

Stand-out line: "Infinite iterators turn unbounded structure into a constant-memory generator — `count` for indices, `cycle` for round-robin, `repeat` for filler. The trick is always bounding them at the call site, not inside the iterator."

Q7 · `iter(callable, sentinel)` — the two-arg form

Tags: Difficulty: Hard · Asked at: Postman, Razorpay, JP Morgan, Cred, Infosys · Topic: `iter()` advanced form

The question: "What does the two-argument form of `iter()` do, and where would you actually use it?"

The 30-second answer: `iter(callable, sentinel)` returns an iterator that calls `callable()` with no args and yields each return value until the return equals `sentinel`, at which point `StopIteration` fires. The classic use is reading a stream in fixed-size chunks until EOF — `iter(lambda: f.read(4096), b'')` reads 4KB chunks until the read returns empty bytes.

Step-by-step thought process: 1. "Most people only know `iter(obj)` with one argument. The two-arg form is one of Python's most underused gems." 2. "Signature: `iter(callable, sentinel)`. The callable must take zero arguments — that's why you wrap in `lambda` or `functools.partial`." 3. "On each `next()` call, the iterator calls `callable()`. If the result equals `sentinel`, `StopIteration` is raised. Otherwise, the result is yielded." 4. "Killer use case: chunk-reading a binary file. `for chunk in iter(lambda: f.read(4096), b'')` — terminates when read returns empty bytes." 5. "Other use: poll-until-done patterns — `for msg in iter(queue.get, None):` keeps consuming until a `None` sentinel is posted."

Code:

```
# Read a binary file in 4KB chunks — clean, no while-True loop
with open("/etc/hosts", "rb") as f:
    chunks = []
    for chunk in iter(lambda: f.read(4096), b''):
        chunks.append(chunk)
print(f"read {sum(len(c) for c in chunks)} bytes")

# Poll a queue until a sentinel is posted
import queue
q = queue.Queue()
for item in ["order-1", "order-2", "order-3"]:
    q.put(item)
q.put(None)                                # sentinel

for msg in iter(q.get, None):
    print("processed", msg)
# processed order-1 / processed order-2 / processed order-3

# Roll a die until you get a 6 — every other value is yielded
import random
random.seed(42)
rolls = list(iter(lambda: random.randint(1, 6), 6))
print(rolls)                                # whatever non-6 rolls came before the first 6
```

Gotchas: - The sentinel is compared with `==`, not `is`. Returning a fresh object that *equals* sentinel still terminates. - The callable must take no arguments. `iter(f.read, b'')` does NOT work because `f.read` requires no arg by default — but `iter(lambda: f.read(4096), b'')` is needed if you want sized reads. - If `callable()` raises, the exception propagates out of `next()` — not converted to `StopIteration`.

What NOT to say: - "I'd write a `while True` with a break." — works but verbose; the two-arg `iter()` is the idiomatic answer. - "It's deprecated." — it's not; it's been stable since Python 2 and is heavily used in standard library code.

Common follow-up: "How would you read a socket in chunks until the peer closes the connection — using the two-arg `iter()`?"

Stand-out line: "`iter(callable, sentinel)` is the cleanest 'call this until done' pattern in Python — it turns a `while True` with a break into a `for` loop you can compose with `map`, `filter`, and the rest of `itertools`."

Q8 · `reversed()` — when supported, `__reversed__`

Tags: Difficulty: Medium · Asked at: TCS, Wipro, Flipkart, Walmart Labs, Infosys · Topic: Reverse iteration

The question: "How does `reversed()` work — what objects support it, and how do you make a custom class reversible?"

The 30-second answer: `reversed(obj)` returns an iterator that walks `obj` from last to first. It works on any object that implements `__reversed__` (preferred) OR has both `__len__` and `__getitem__` with integer keys. Lists, tuples, strings, and ranges support it; generic dicts only from Python 3.8+; plain generators do NOT.

Step-by-step thought process: 1. "Reverse iteration is NOT free — it requires random access (via `__getitem__`) or an explicit `__reversed__` method." 2. "That's why `reversed(some_generator)` raises `TypeError` — generators are forward-only, no random access." 3. "The lookup order: Python calls `obj.__reversed__()` if defined; else falls back to using `__len__` + `__getitem__` with indices $N-1$ down to 0." 4. "Lists, tuples, strings, ranges, bytes — all have C-level `__reversed__`. Sets do NOT — sets are unordered." 5. "For a custom class, define `__reversed__` if you can produce a reverse iterator efficiently; otherwise rely on `__len__` + `__getitem__`."

Code:

```

# Built-ins — all work
print(list(reversed([1, 2, 3])))          # [3, 2, 1]
print("".join(reversed("razorpay")))      # yaprozar
print(list(reversed(range(5))))           # [4, 3, 2, 1, 0]

# Generators DON'T support reversed()
def gen():
    yield 1; yield 2; yield 3
try:
    list(reversed(gen()))
except TypeError as e:
    print(e)                             # argument to reversed() must be a sequence

# Custom class with __reversed__
class OrderHistory:
    def __init__(self, orders):
        self.orders = orders

    def __iter__(self):
        return iter(self.orders)

    def __reversed__(self):
        return reversed(self.orders)      # delegate to list's reversed

    def __len__(self):
        return len(self.orders)

h = OrderHistory(["order-1", "order-2", "order-3"])
print(list(reversed(h)))                  # ['order-3', 'order-2', 'order-1']

# Fallback path — no __reversed__, but __len__ + __getitem__ work
class Indexed:
    def __init__(self, data):
        self.data = data
    def __len__(self):
        return len(self.data)
    def __getitem__(self, i):
        return self.data[i]

print(list(reversed(Indexed([10, 20, 30])))) # [30, 20, 10] — uses fallback

```

Gotchas: - `reversed()` on a `dict` works only from Python 3.8+ (when dicts gained insertion-order guarantees). On older Python, raises `TypeError`. - Sets, frozensets, plain generators, and `map` / `filter` / `zip` results don't support `reversed()` — wrap in `list()` first. - A custom `__reversed__` should return an iterator, not a list. Returning a list works but defeats the streaming benefit.

What NOT to say: - "Just use `list(...)[::-1]` ." — works for lists but materialises in memory. `reversed()` is iterator-based, so it's lazy. - "All iterables support `reversed()` ." — only sequences do (or those with explicit `__reversed__`).

Common follow-up: "Why can't you call `reversed()` on a generator — what fundamental property is missing?"

Stand-out line: " `reversed()` needs random access or an explicit reverse hook — that's why it works on lists and ranges but not on generators or sets. Forward-only iteration is one-directional by design."

Q9 · For-loop mechanics under the hood

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Cred, Walmart Labs, JP Morgan · Topic: For-loop desugaring

The question: "Walk me through exactly what Python does when it executes `for x in obj: body`. What's the equivalent code if you wrote it without using `for`?"

The 30-second answer: Python calls `iter(obj)` once to get an iterator, then in a loop calls `next(iterator)`. Each return value is bound to the loop variable `x` and the body runs. When `next()` raises `StopIteration`, the loop catches it and exits. `break` exits early without exception; `continue` jumps back to the next `next()` call; `else` runs only if the loop exits naturally (no `break`).

Step-by-step thought process: 1. "Step 1: `iter(obj)` is called exactly once at loop entry. This is why mutating `obj` mid-loop has the well-known surprises." 2. "Step 2: `next(iterator)` is called at each iteration. The return value is bound to `x`." 3. "Step 3: the body runs. `break` exits, `continue` jumps back to step 2." 4. "Step 4: when `next()` raises `StopIteration`, the loop catches it silently — that's the implicit `try/except`." 5. "The `else` clause runs only if the loop completes without `break` — handy for 'searched and didn't find' patterns."

Code:

```

# What `for x in obj: body` actually compiles to (conceptually)
def for_loop_equivalent(obj, body):
    it = iter(obj)                # called ONCE
    while True:
        try:
            x = next(it)          # called PER ITERATION
        except StopIteration:
            break                 # natural exit
        body(x)
    else:
        pass                     # for-else lands here on natural exit
    # break would skip the else

# Real for vs manual – both print the same
for x in [1, 2, 3]:
    print(x, end=" ")            # 1 2 3
print()
for_loop_equivalent([1, 2, 3], lambda x: print(x, end=" "))
print()

# for-else – search-with-fallback pattern
def find_handler(handlers, name):
    for h in handlers:
        if h.name == name:
            return h
        # break skips the else
    else:
        raise ValueError(f"no handler {name}") # ran because no break

# Mutating mid-loop – surprising behaviour
xs = [1, 2, 3, 4]
for x in xs:                    # iter() returns a list_iterator that indexes positional
    print(x, end=" ")
    if x == 2: xs.remove(x)
# 1 2 4 - '3' was skipped because indices shifted
print()

```

Gotchas: - Mutating a list while iterating it produces undefined-feeling results — the iterator tracks indices, not items. - The `else` clause of a `for` loop is one of Python's least-known features and is often a clearer expression of search-with-fallback than a `found` flag. - A `break` inside the loop body skips the `else`; a return / raise also skips it.

What NOT to say: - "The `for` loop is a special construct in C." — it IS implemented in C, but it's nothing more than the desugared `iter` + `next` + `try/except StopIteration` pattern. Pretending it's magic signals you haven't read the data model. - "`for-else` runs on errors." — wrong; `else` runs only on natural (no-break) exit. Errors propagate and skip both body and else.

Common follow-up: "What happens if the body of the loop raises an exception — does the iterator's cleanup run?"

Stand-out line: "A `for` loop is four lines of desugaring: one `iter()`, one `while True`, one `next()` inside `try`, one `except StopIteration: break`. Everything else — `break`, `continue`, `else` — is structured around those four lines."

Q10 · Iterating a dict — what you get, `.items()` / `.values()` / `.keys()` views

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, TCS, Flipkart, Cred · Topic: Dict iteration

The question: "When you iterate over a dict directly, what do you get? And what's the difference between `.keys()`, `.values()`, and `.items()`?"

The 30-second answer: Iterating a dict directly yields its **keys**, in insertion order (guaranteed since Python 3.7). `.keys()`, `.values()`, `.items()` return **view objects** — live, lazy windows into the dict that reflect later mutations and support set-like operations for `keys()` and `items()`. Views are NOT iterators themselves; calling `iter()` on a view returns a fresh iterator over the current state.

Step-by-step thought process: 1. "`for k in d:` is the same as `for k in d.keys():`. Keys-by-default is a Python convention worth remembering." 2. "Insertion order was an implementation detail in Python 3.6, then promoted to a language guarantee in 3.7. You can rely on it now." 3. "`d.keys()` returns a `dict_keys` view. Same for `.values()` (`dict_values`) and `.items()` (`dict_items`). All three are live — mutating the dict reflects immediately." 4. "Views are not iterators — they're iterable. Calling `iter(view)` gives an iterator over the current state at that moment." 5. "`keys()` and `items()` views support set operations (`|`, `&`, `-`, `^`) — handy for diff'ing two dicts. `values()` does not, because values aren't necessarily hashable."

Code:


```

d = {"alice": 500, "bob": 700, "carol": 300}

# Direct iteration – gives keys
for x in d:
    print(x, end=" ")          # alice bob carol
print()

# Views – three flavours
print(list(d.keys()))         # ['alice', 'bob', 'carol']
print(list(d.values()))       # [500, 700, 300]
print(list(d.items()))        # [('alice', 500), ('bob', 700), ('carol', 300)]

# Views are LIVE – they see mutations
view = d.keys()
print(view)                   # dict_keys(['alice', 'bob', 'carol'])
d["dave"] = 100
print(view)                    # dict_keys(['alice', 'bob', 'carol', 'dave'])

# Set operations on key views
a = {"alice": 1, "bob": 2, "carol": 3}
b = {"bob": 20, "carol": 30, "dave": 40}
print(a.keys() & b.keys())     # {'bob', 'carol'} – intersection
print(a.keys() - b.keys())     # {'alice'} – difference
print(a.keys() | b.keys())     # {'alice', 'bob', 'carol', 'dave'} – union

# Items view too – works because (key, value) tuples ARE hashable (if values are)
print(a.items() & b.items())   # set() – no matching (k,v) pairs

# Mutating mid-iteration raises RuntimeError
try:
    for k in d:
        if k == "bob": d["new"] = 999
except RuntimeError as e:
    print(e)                   # dictionary changed size during iteration

```

Gotchas: - Mutating a dict mid-iteration raises `RuntimeError: dictionary changed size during iteration`. Snapshot via `list(d)` first if you need to add/remove keys during iteration. - `dict_values` does NOT support set operations — values can be unhashable. `dict_items` works only if all values are hashable. - Views are cheap (no copy) but tied to the dict's lifetime — pickling a view is awkward; pickle the dict instead.

What NOT to say: - "Iterating a dict gives you items by default." — wrong; it gives keys. Common mistake from JavaScript / Java background. - "`.keys()` returns a list." — it returned a list in Python 2; in Python 3 it returns a view. The difference matters for memory and live-reflection semantics.

Common follow-up: "How would you find keys present in dict A but missing from dict B — using view set operations?"

Stand-out line: "Dict views are the unsung MVP of Python 3 — live, lazy, set-capable. Once you start using `a.keys() - b.keys()` instead of comprehensions, dict-diff code drops to one line and reads like math."

End of sub-chunk 1 of 6 — Iterator protocol (Q1–Q10). Next sub-chunk: Generator functions + yield (Q11–Q20).

Chapter 3 — Generators + Iterators · Sub-chunk 2 of 6 · Generator Functions (Q11–Q20)

Python Interview Prep — 2026 Edition · Learn & Excel · File 04 Core Python Library

This sub-chunk covers generator functions — the `yield` keyword, the suspended-frame model, and the patterns that ride on top. Razorpay panels for payment-pipeline roles, Swiggy for order-stream processing, Flipkart for catalogue ETL, Cred for ledger replay, Postman for response streaming, Zerodha for tick-by-tick market data, and Service MNCs (TCS / Infosys / Wipro) for screening rounds all ask whether you can write a generator AND explain what the interpreter does with `yield`. Most candidates know `yield` produces values lazily; far fewer can articulate the frame-suspension model, the `return value` trick, or why `gen.close()` raises `GeneratorExit` inside the function body.

Each Q below uses the same shape: **30-second answer** (what you say first if the panel is impatient), **thought process** (how to reason out loud), a **code** block (compiles on Python 3.12+), the **gotchas** (where panels watch you trip), **what NOT to say**, **common follow-up**, and a **stand-out line** — the single sentence that signals you've read the data model.

Q11 · `yield` mechanics — what makes a function a generator function

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, TCS, Infosys, Flipkart · Topic: yield fundamentals

The question: "What turns an ordinary function into a generator function — and what does Python do differently when it compiles one?"

The 30-second answer: The presence of `yield` (or `yield from`) anywhere in the function body — even unreachable — turns the function into a generator function at compile time. Calling it does not run the body; it returns a generator object (an iterator). The body runs only when you call `next()` on that object, pausing at each `yield` and resuming on the next `next()`.

Step-by-step thought process: 1. "The trigger is lexical, not runtime. The compiler scans the body for `yield` once. If found, the function gets the `CO_GENERATOR` flag and becomes a factory." 2. "Calling a generator function does zero work in the body — it just wraps the code object in a generator frame and hands it back." 3. " `yield` is two things at once: it produces a value AND it pauses the frame. The local state (variables, instruction pointer, exception state) is stored on the frame." 4. "On the next `next()` call, the frame is resumed at the instruction after the `yield`. Locals are exactly where they were." 5. "When the body falls off the end or hits `return`, `StopIteration` is raised and the frame is discarded."

Code:

```
def gen_with_yield():
    print("body starts")
    yield 1
    print("after first yield")
    yield 2
    print("body ends")

g = gen_with_yield()           # NO output – body has not run
print(type(g).__name__)       # generator

print(next(g))                # body starts / 1
print(next(g))                # after first yield / 2
try:
    next(g)
except StopIteration:
    print("done")              # body ends / done

# yield doesn't have to be reachable – its presence is enough
def conditional_gen(flag):
    if False:
        yield                  # never executed, but still flags the function
    return

print(type(conditional_gen(True)).__name__) # generator – even though yield is dead code

# Inspect the flag directly via the code object
import inspect
print(inspect.isgeneratorfunction(gen_with_yield)) # True
print(gen_with_yield.__code__.co_flags & 0x20)    # 32 – CO_GENERATOR bit set
```

Gotchas: - A `return` statement with a value inside a generator does NOT return that value to the caller — it raises `StopIteration(value)` (covered in Q15). - `yield` inside a `try/finally` does run the `finally` block when the generator is closed or garbage-collected — relevant for resource cleanup. - Mixing `yield` and a plain `return value` in the same function used to be a `SyntaxError` pre-Python 3.3; now allowed because of `StopIteration.value`.

What NOT to say: - " `yield` is like `return` ." — misleading; `return` ends the function, `yield` pauses it. Different control-flow primitive entirely. - "Generator functions run lazily because Python is

lazy." — Python is not lazy; generator functions are the explicit lazy primitive. Confusing the language with the construct signals fuzzy understanding.

Common follow-up: "If `yield` appears inside a `try/finally`, when does the `finally` block run — and what triggers it?"

Stand-out line: "A generator function is a syntactic switch — the compiler sees `yield` and emits a different code object, one that produces a frame-pausing iterator instead of running to completion. The function never runs when you call it; it runs when you iterate it."

Q12 · Generator function vs regular function — return values + when called

Tags: Difficulty: Easy · Asked at: Cred, Postman, Walmart Labs, Infosys, Razorpay · Topic: Generator vs regular function

The question: "What's the difference between calling a regular function and calling a generator function — what comes back, and when does the body run?"

The 30-second answer: A regular function runs its body immediately on call and returns whatever `return` produces (or `None`). A generator function does NOT run the body on call — it returns a generator object (an iterator). The body runs incrementally, one chunk per `next()` call, pausing at each `yield`. Same call syntax, completely different execution model.

Step-by-step thought process: 1. "Calling looks identical: `f(x)` for both. The difference is what comes back and when work happens." 2. "Regular function: arguments bound, body runs top-to-bottom, return value handed back. One invocation, one execution." 3. "Generator function: arguments bound, generator object created with a paused frame, body NOT executed. Returned object is an iterator — you have to iterate it to do the work." 4. "This is why a print at the top of a generator function doesn't fire until the first `next()` — the body hasn't started yet." 5. "Concrete consequence: errors raised by the body don't surface on the call — they surface on the first `next()`. Validate arguments before the first `yield` if you want eager errors."

Code:

```

def regular(amount):
    print("regular: body running")
    return amount * 2

def generator(amount):
    print("generator: body running")
    yield amount
    yield amount * 2

# Regular function – work happens at call time
r = regular(100)                # prints 'regular: body running'
print(r)                        # 200

# Generator function – call does nothing visible
g = generator(100)              # NO print – body has not started
print(g)                        # <generator object generator at 0x...>
print(next(g))                  # generator: body running / 100
print(next(g))                  # 200

# Argument validation pitfall – error surfaces on first next(), not on call
def parse_orders(rows):
    if not rows:
        raise ValueError("empty input")
    for row in rows:
        yield row.upper()

g = parse_orders([])             # NO error – body hasn't run
try:
    next(g)                      # ValueError fires HERE
except ValueError as e:
    print("caught:", e)

# Eager-validate fix: wrap the generator in a thin function that validates first
def parse_orders_eager(rows):
    if not rows:
        raise ValueError("empty input")    # raised at call time
    return _parse(rows)

def _parse(rows):
    for row in rows:
        yield row.upper()

```

Gotchas: - Argument-validation errors inside the generator body don't fire on the call site — they fire on the first `next()`. Confusing to debug. - Generator objects are not reusable. Calling the function again gives a fresh generator; the old one stays exhausted. - `inspect.isgeneratorfunction(f)` checks the function; `inspect.isgenerator(g)` checks the object. Mix them up and the test fails.

What NOT to say: - "They're basically the same except for `yield` ." — they have completely different execution models. Glossing over this is a junior-tier answer. - "The function returns the first yielded value." — wrong; it returns a generator object. The first yielded value comes from the first `next()`.

Common follow-up: *"If a generator function validates its arguments at the top, when does that validation actually run?"*

Stand-out line: *"Calling a generator function is a no-op for the body — it's frame construction, not execution. The function literally never runs when you call it; it runs when you iterate the object it returned."*

Q13 · Lazy evaluation — the suspended-execution model, frame state

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, JP Morgan, Postman, Walmart Labs · Topic: Lazy evaluation + frame model

The question: *"What does it mean that a generator is 'lazy' — and what gets stored on the generator object between `yield`s?"*

The 30-second answer: A generator is lazy because the body runs only when consumed — one `yield` per `next()`. Between yields, the entire execution frame is suspended on the generator object: local variables, the instruction pointer, the exception state, and the `try/finally` stack. The frame is real Python memory — you can inspect it via `gen.gi_frame` (covered in Q16) — and it lives until the generator is exhausted or garbage-collected.

Step-by-step thought process: 1. *"Lazy = nothing happens until asked. Each `next()` does just enough work to produce one value, then stops."* 2. *"The pause is implemented by saving the frame — same frame that a normal function would discard on return. The interpreter keeps it alive on the generator object."* 3. *"What's on the frame: local variables, the bytecode instruction pointer (where to resume), the value stack, the exception block stack, and (if applicable) the `with` / `try` cleanup contexts."* 4. *"Memory cost: $O(\text{local-variable footprint})$ per generator. Cheap for most uses; matters if you spawn millions."* 5. *"Practical consequence: a generator holds references to everything in its local scope. A generator that captures a 10MB list keeps that list alive until exhaustion."*

Code:

```

def stream_orders(source):
    pending = []                                # local – survives between yields
    for order in source:
        pending.append(order)
        if len(pending) >= 3:
            yield pending[:]                    # pause here; pending stays on the frame
            pending.clear()
    if pending:
        yield pending                          # last batch

orders = ["o1", "o2", "o3", "o4", "o5", "o6", "o7"]
g = stream_orders(orders)
print(next(g))                                # ['o1', 'o2', 'o3'] – frame paused inside loop
print(next(g))                                # ['o4', 'o5', 'o6'] – resumed from saved instruction
print(next(g))                                # ['o7'] – final batch
# next call would raise StopIteration

# Frame holds references – generator captures the world it was started with
def hold_resource():
    big = [0] * 1_000_000                      # 8 MB of ints
    for i in range(3):
        yield i                                # `big` stays alive between yields
    # only freed when generator is exhausted or GC'd

g = hold_resource()
next(g); next(g); next(g)
try: next(g)
except StopIteration: pass                    # NOW `big` is releasable

# Lazy evaluation in pipeline form – no intermediate lists
def squared(xs):
    for x in xs:
        yield x * x

def even(xs):
    for x in xs:
        if x % 2 == 0:
            yield x

# Source → square → filter → consume – each value flows through end-to-end
total = sum(even(squared(range(10))))          # 0 + 4 + 16 + 36 + 64 = 120
print(total)

```

Gotchas: - Local references on a generator frame keep the referenced objects alive. A generator holding a database connection keeps the connection open until exhaustion. - The frame state includes `try` block context — if you `yield` inside a `try`, the `except` arms are still wired up when you resume. - Two generators from the same function call have independent frames. `g1` and `g2` from `gen()` and `gen()` share zero state.

What NOT to say: - "Lazy means it computes only when needed." — half-right but vague; the panel wants you to mention frame suspension, not just "deferred work." - "Generators don't use memory." —

wrong; they use frame-sized memory, which is small but not zero. They avoid the $O(N)$ cost of materialising the sequence.

Common follow-up: "If a generator yields from inside a `with` block, when does the context manager exit — at exhaustion, at close, or at GC?"

Stand-out line: "A generator is a function whose frame outlived the call — the same frame a regular function discards on return is parked on the generator object, ready to resume at the next instruction pointer."

Q14 · When a generator is faster than building a list

Tags: Difficulty: Medium · Asked at: Swiggy, Flipkart, Razorpay, Postman, Cred · Topic: Performance + memory characteristics

The question: "In what situations is a generator measurably faster than building a list first — and when is the list actually faster?"

The 30-second answer: Generators win on memory ($O(1)$ vs $O(N)$), on early-exit consumers (`any` , `all` , `next` , breaking out of a loop), and on pipelines where values flow end-to-end without intermediate materialisation. Lists win when you need multiple passes, random access, `len()` , or when the producer is so cheap that frame-suspension overhead dominates. The rule of thumb: stream when input is large or the consumer might stop early.

Step-by-step thought process: 1. "Three wins for generators: memory, early-exit, pipeline composition. Memorise these — every interview answer maps to one of them." 2. "Memory: a generator over a 100M-row file uses ~kilobytes; a list materialises the whole thing into RAM. For ETL on big files, this is the entire reason to use a generator." 3. "Early-exit: `any(is_fraud(o) for o in orders)` stops at the first match. The list version computes every check, then `any` short-circuits — but you already paid for the comprehension." 4. "Pipeline composition: `sum(price(o) for o in active(orders))` flows one order at a time through three stages. The list version builds three intermediate lists." 5. "Lists win when: you iterate multiple times (generator exhausts), need `len()` (generator has no length), or the per-value work is so tiny that yield overhead is more expensive than just building the list."

Code:


```

import sys

# Memory comparison – generator vs list comprehension
gen = (x * x for x in range(1_000_000))    # generator expression
lst = [x * x for x in range(1_000_000)]    # list comprehension

print(sys.getsizeof(gen))                  # ~200 bytes
print(sys.getsizeof(lst))                  # ~8 MB+

# Early-exit win – `any` stops at first True
def is_fraud(order):
    return order.get("amount", 0) > 100_000

orders = [{"id": i, "amount": i * 100} for i in range(10_000)]

# Generator – short-circuits at first match (around order ~1001)
print(any(is_fraud(o) for o in orders))    # True – exits after ~1001 checks

# List – comprehension runs all 10K, THEN any short-circuits
print(any([is_fraud(o) for o in orders]))  # True – but 10K checks done first

# Pipeline composition – three stages, no intermediates
def parse(lines):
    for ln in lines: yield ln.strip().split(",")

def to_dict(rows):
    for r in rows: yield {"id": r[0], "amount": int(r[1])}

def large_only(records):
    for rec in records:
        if rec["amount"] > 1000:
            yield rec

source = ["o1,500", "o2,1500", "o3,800", "o4,2500"]
result = list(large_only(to_dict(parse(source))))
# Each line flows through parse → to_dict → large_only one at a time

# When lists win – need random access / multiple passes / len
data = [1, 2, 3, 4, 5]
print(data[2])                          # 0(1) random access – generators can't do this
print(len(data))                        # 0(1) length – generators have no len()
for _ in range(3):                      # multiple passes – generators exhaust on the first
    print(sum(data))

```

Gotchas: - `any([gen_expr])` materialises before short-circuiting — wrap in parens, not brackets, to keep laziness. - A generator passed to a function that iterates twice will be empty on the second pass — debugging this is a 30-minute footgun. - Per-yield overhead is real (~50–100 ns). For tight inner loops with cheap work, a list comprehension can be 2–3x faster than a generator expression.

What NOT to say: - "Generators are always faster." — false; for small data with multiple passes, lists win on both speed and clarity. - "Lazy is always good." — laziness has costs: per-yield overhead, no random access, can't be reused. Picking the right tool requires knowing both sides.

Common follow-up: *"You have a 10GB log file and need to count error lines — do you stream or materialise, and why?"*

Stand-out line: *"Generators win on three axes: bounded memory, short-circuit consumers, and pipeline composition. Lists win on random access and multi-pass — and the decision is made by which side dominates your workload, not by which feels more 'Pythonic'."*

Q15 · `return value` inside a generator (raises `StopIteration` with `.value`)

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, JP Morgan, Cred, Walmart Labs · Topic: Generator return semantics

The question: *"What happens when you write `return some_value` inside a generator function — where does that value end up?"*

The 30-second answer: A `return value` statement inside a generator does NOT yield the value. Instead, it raises `StopIteration(value)` — the value rides on the exception's `.value` attribute. Ordinary `for` loops discard it; the `yield from` construct and async coroutines use it to pass a final result up the chain. It's the mechanism that makes generator-based coroutines (pre-asyncio) workable.

Step-by-step thought process: 1. *"In a regular function, `return value` hands the value to the caller. In a generator, the caller is `next()`, and `next()` only returns yielded values."* 2. *"So Python re-uses the exhaustion exception: `return value` becomes `raise StopIteration(value)`. The exception carries the value on `.value`."* 3. *"`for` loops catch `StopIteration` and immediately `break` — the `.value` is thrown away. So in a plain loop, `return value` is invisible."* 4. *"`yield from gen()` IS the mechanism that captures the value. The expression `result = yield from sub_gen()` binds `result` to the inner generator's return value."* 5. *"This is foundational for old-school coroutines (pre-3.5 `@asyncio.coroutine` decorator). Modern `async def` uses real `return`, but the underlying mechanism is the same."*

Code:

```

def parse_payload(rows):
    """Yields parsed rows, returns a summary at the end."""
    count = 0
    for row in rows:
        yield row.upper()
        count += 1
    return {"parsed": count}                # rides on StopIteration.value

# Manual capture
g = parse_payload(["a", "b", "c"])
try:
    while True:
        print(next(g))                    # A / B / C
except StopIteration as e:
    print("summary:", e.value)            # summary: {'parsed': 3}

# For-loop discards the value – never see it
for row in parse_payload(["x", "y"]):
    print(row)                            # X / Y – but no summary

# yield from CAPTURES the value
def driver():
    summary = yield from parse_payload(["a", "b", "c"])
    yield ("DONE", summary)                # emit summary as the last value

g = driver()
print(next(g))                            # A
print(next(g))                            # B
print(next(g))                            # C
print(next(g))                            # ('DONE', {'parsed': 3})

# Bare `return` is equivalent to `return None`
def bare_return():
    yield 1
    return                                # raises StopIteration(None)

g = bare_return()
next(g)
try: next(g)
except StopIteration as e: print(e.value) # None

```

Gotchas: - Pre-Python 3.3, mixing `yield` and `return value` in the same function was a `SyntaxError`. Allowed now via PEP 380. - The `for` loop will never see the return value — it's stripped during the `StopIteration` catch. Use `yield from` or manual `try/except` to see it. - PEP 479 (Python 3.7+): a stray `StopIteration` from inner code becomes `RuntimeError`, but explicit `return value` is exempt — it's still legal.

What NOT to say: - "`return value` in a generator yields the value." — wrong; it raises `StopIteration(value)`. The value is invisible to plain `for` loops. - "You can't return from a

generator." — wrong since Python 3.3. Returning a value is a deliberate language feature for the `yield from` protocol.

Common follow-up: "How would you write a generator that yields chunks of data AND reports a checksum at the end — using only `yield` and `return`?"

Stand-out line: "`return value` in a generator is the back-channel — the value rides out on `StopIteration.value`, invisible to `for` loops but caught by `yield from`. It's how the old `@coroutine` decorator passed final results before `async` / `await` existed."

Q16 · `gen.gi_frame` and inspecting generator state

Tags: Difficulty: Hard · Asked at: Cred, JP Morgan, Postman, Razorpay, Walmart Labs · Topic: Generator introspection

The question: "How do you inspect the live state of a paused generator — what attributes does the generator object expose, and what can you do with them?"

The 30-second answer: A generator object exposes four `gi_*` attributes: `gi_frame` (the live execution frame — `None` when exhausted), `gi_code` (the compiled code object), `gi_running` (True if currently executing), and `gi_yieldfrom` (the inner iterator if paused inside `yield from`). The frame's `f_locals` gives you the local variables at the pause point, and `f_lineno` gives the line about to execute next. Useful for debugging stuck pipelines and writing introspective test harnesses.

Step-by-step thought process: 1. "Four attributes, all prefixed `gi_`. Memorise them: `gi_frame`, `gi_code`, `gi_running`, `gi_yieldfrom`." 2. "`gi_frame` is the live frame object. After exhaustion, it becomes `None` — useful flag for 'is this generator done?'." 3. "`gi_frame.f_locals` is a dict of local variables at the pause point. `f_lineno` is the line number about to execute next." 4. "`gi_running` is True only while a `next()` call is actively executing the body — usually False from outside the generator." 5. "`gi_yieldfrom` is non-None if the generator is paused inside a `yield from sub_gen` — it points at `sub_gen`. Lets you trace nested generators."

Code:

```

import inspect

def stream(orders):
    seen = 0
    for o in orders:
        seen += 1
        yield o
    # exhausted here

g = stream(["o1", "o2", "o3"])
print(g.gi_frame is None)           # False – not started, but frame exists
print(g.gi_running)                 # False
print(g.gi_yieldfrom)               # None

next(g)
print(g.gi_frame.f_locals)          # {'orders': [...], 'seen': 1, 'o': 'o1'}
print(g.gi_frame.f_lineno)          # line of the yield statement

next(g)
print(g.gi_frame.f_locals["seen"])  # 2

next(g)
try: next(g)
except StopIteration: pass
print(g.gi_frame is None)           # True – exhausted, frame gone

# Tracing yield-from chains via gi_yieldfrom
def inner():
    yield 1; yield 2; yield 3

def outer():
    yield from inner()               # paused HERE
    yield "end"

g = outer()
next(g)                             # consume from inner
print(g.gi_yieldfrom)                # <generator object inner at 0x...>
print(inspect.getgeneratorstate(g))  # GEN_SUSPENDED

# inspect.getgeneratorstate – four states
def s(): yield 1; yield 2

g = s()
print(inspect.getgeneratorstate(g))  # GEN_CREATED – not started
next(g)
print(inspect.getgeneratorstate(g))  # GEN_SUSPENDED – between yields
list(g)                              # exhaust
print(inspect.getgeneratorstate(g))  # GEN_CLOSED – done

# Practical: peek at a stuck pipeline in a long-running job
def pipeline(source):
    for item in source:
        yield item * 2

```

```
g = pipeline(iter([1, 2, 3, 4, 5]))
next(g); next(g)
print(f"paused at line {g.gi_frame.f_lineno}, locals: {dict(g.gi_frame.f_locals)}")
```

Gotchas: - `gi_frame.f_locals` is a snapshot — mutating it does NOT change the actual locals (CPython caches them in fast slots, sync is one-way). - `gi_frame` is `None` after exhaustion or `gen.close()`. Test `is None` before reading frame attributes. - `gi_running` is for cooperative generators inspecting themselves — it's almost always `False` from outside code because you can't observe a generator while it's running another `next()`.

What NOT to say: - "You can't inspect a generator." — false; introspection has been available since the early Python 3 days. Saying this signals you've never debugged a stuck pipeline. - "Use `dir(gen)` to find the attributes." — works but lazy; the panel wants you to name `gi_frame`, `gi_code`, `gi_running`, `gi_yieldfrom`.

Common follow-up: "A long-running ETL has stalled — how would you check from outside which line of which generator it's paused at?"

Stand-out line: "The `gi_*` attributes turn a generator from a black box into a debuggable object — `gi_frame.f_lineno` plus `f_locals` is enough to attach a debugger to a stalled pipeline without restarting it."

Q17 · Double-iteration of a generator function vs of a generator object (factory pattern)

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Flipkart, Postman, Cred · Topic: Re-iteration + factory pattern

The question: "You want to iterate the same data twice. What's the difference between calling the generator function twice and iterating the generator object twice?"

The 30-second answer: A generator object is single-shot — iterating it twice gives data the first time, nothing the second. The generator function, however, is a factory — each call produces a fresh generator with a fresh frame. So `for x in gen(): ...` twice works fine because each `gen()` call makes a new iterator. For re-iterable data, wrap a generator in a class with `__iter__` that returns a fresh generator each call.

Step-by-step thought process: 1. "Generator object = iterator. Iterating an iterator twice gives data once, then nothing. That's the protocol." 2. "Generator function = factory. Each call creates a new generator object with a fresh frame. So `gen()` twice = two independent iterators." 3. "The fix when you need re-iterability: don't store the generator object — store something you can call to produce one (the function itself, or a class with `__iter__`)." 4. "Class form: implement `__iter__` to return a fresh generator on each call. Now the class is iterable and re-iterable." 5. "This pattern shows up in API design

— if you ship an SDK function that returns a generator, downstream code that iterates it twice silently produces wrong results."

Code:

```
def order_stream():
    yield "o1"; yield "o2"; yield "o3"

# Iterating the same OBJECT twice – second pass empty
g = order_stream()
print(list(g))          # ['o1', 'o2', 'o3']
print(list(g))          # [] – exhausted

# Iterating the FUNCTION (factory) twice – both passes work
print(list(order_stream())) # ['o1', 'o2', 'o3']
print(list(order_stream())) # ['o1', 'o2', 'o3']

# The trap – function that returns a generator
def get_orders():
    return order_stream()          # returns ONE generator object

g = get_orders()
print(list(g))          # ['o1', 'o2', 'o3']
print(list(g))          # [] – same object, exhausted

# The fix – class with __iter__ that produces a fresh generator each time
class OrderStream:
    def __init__(self, source):
        self.source = source

    def __iter__(self):
        # Called fresh each time someone iterates – returns NEW generator
        for o in self.source:
            yield o

orders = OrderStream(["o1", "o2", "o3"])
print(list(orders))      # ['o1', 'o2', 'o3']
print(list(orders))      # ['o1', 'o2', 'o3'] – re-iterable

# Alternative – store the callable, call it per iteration
class Reloadable:
    def __init__(self, gen_factory):
        self.gen_factory = gen_factory          # store the function, not the result

    def __iter__(self):
        return self.gen_factory()

r = Reloadable(order_stream)
print(list(r))           # ['o1', 'o2', 'o3']
print(list(r))           # ['o1', 'o2', 'o3']
```

Gotchas: - Passing a generator to a library function (e.g. `pd.DataFrame`) that iterates twice silently drops data — common pandas / numpy bug. - A class with `__iter__` returning a generator is the standard "re-iterable wrapper" pattern — preferred over manually tracking state. -

`itertools.tee(gen, 2)` gives two iterators sharing the underlying generator — buffers values internally, so it can use unbounded memory if the consumers fall out of sync.

What NOT to say: - "Generators reset automatically." — false; they don't. This belief is the source of most "why is my second loop empty?" bugs. - "Just convert to a list." — works but defeats the memory benefit. The factory pattern preserves laziness AND re-iterability.

Common follow-up: "How would you build a class that exposes a remote API as a re-iterable data source without re-fetching on every iteration?"

Stand-out line: "A generator object is an iterator — single-shot by protocol. A generator function is a factory — call it again and you get a fresh iterator. The class-with- `__iter__` pattern is how you ship re-iterable lazy data without surprising your callers."

Q18 · Generator as a coroutine-flavored pattern (pre-asyncio history)

Tags: Difficulty: Hard · Asked at: Postman, JP Morgan, Cred, Razorpay, Walmart Labs · Topic: Generators as coroutines / PEP history

The question: "Before `async` / `await`, generators were used to write coroutines. How did that work — and what made `yield` a two-way street with `.send()`?"

The 30-second answer: PEP 342 (Python 2.5) made `yield` an expression as well as a statement: `value = yield x` sends `x` out AND receives a value back when the caller does `gen.send(received_value)`. This turned generators into bidirectional coroutines — the caller pushes data in, the generator pushes data out. PEP 380 (`yield from`, Python 3.3) added delegation. PEP 492 (`async def`, Python 3.5) replaced this whole machinery with cleaner syntax — but the original coroutine model is the engine.

Step-by-step thought process: 1. "Pre-asyncio history: PEP 342 (2005) → PEP 380 (3.3) → PEP 492 (3.5). Three steps, one journey." 2. "PEP 342: `yield` becomes an expression. `received = yield emitted` sends `emitted` out and receives via `.send()`." 3. "PEP 380: `yield from sub_gen` delegates iteration AND passes `.send()` / `.throw()` calls down to the sub-generator. Solves the 'how do I compose coroutines' problem." 4. "PEP 492: `async def` + `await` replace the whole pattern. `await sub_coro()` is the new `yield from sub_gen()`. The plumbing moves into the event loop." 5. "Why interviews still ask: legacy codebases use `@asyncio.coroutine` decorator, and understanding the generator-coroutine model explains why `async` works the way it does."

Code:


```

# Generator-as-coroutine — value flowing IN via .send()
def averager():
    total = 0
    count = 0
    average = None
    while True:
        value = yield average          # send out average, receive next value
        if value is None:
            break
        total += value
        count += 1
        average = total / count
    return average                    # final return on StopIteration.value

g = averager()
next(g)                             # prime: advance to first yield (average=None)
print(g.send(10))                    # 10.0
print(g.send(20))                    # 15.0
print(g.send(30))                    # 20.0
try:
    g.send(None)                     # trigger break + return
except StopIteration as e:
    print("final:", e.value)          # final: 20.0

# yield from — delegating to a sub-coroutine
def sub_coroutine():
    x = yield "request input"
    yield from inner_sub(x)
    yield "done"

def inner_sub(x):
    y = yield f"received {x}, send another"
    yield f"combined: {x} + {y} = {x + y}"

g = sub_coroutine()
print(next(g))                       # 'request input'
print(g.send(10))                    # 'received 10, send another'
print(g.send(20))                    # 'combined: 10 + 20 = 30'
print(next(g))                       # 'done'

# Modern equivalent — async/await
import asyncio

async def averager_modern():
    total = 0
    count = 0
    async for v in source():
        total += v; count += 1
    return total / count

async def source():
    for v in [10, 20, 30]:
        yield v

```

```
print(asyncio.run(averager_modern())) # 20.0 - same result, cleaner syntax
```

Gotchas: - A generator-coroutine must be "primed" with `next(g)` or `g.send(None)` before the first real `send()` — you can't send into a not-yet-started generator (raises `TypeError`). - `gen.throw(ExcType)` injects an exception INTO the generator at the pause point — used for cancellation in old asyncio code. - Mixing `yield` and `await` in the same function is illegal; `async def` is a separate function flavor.

What NOT to say: - "Generators are coroutines." — they CAN be used as coroutines, but real coroutines are now `async def`. Treating the two as identical confuses legacy and modern. - "`async` is just generator sugar." — partial truth; `async` / `await` is a separate protocol (`__await__`) with an event loop, not just yield-with-different-keyword.

Common follow-up: "What does `gen.send(value)` actually do — how does the value get into the function body, and what happens before the first `send`?"

Stand-out line: "PEP 342 turned `yield` from a one-way pipe into a two-way street — values flow out via the expression, values flow in via `.send()`. That bidirectionality is the foundation everything coroutine-shaped in Python is built on, including modern `async` / `await`."

Q19 · Garbage collection of generators — `GeneratorExit` and resource cleanup

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Postman, Cred, Walmart Labs · Topic: Generator lifecycle + cleanup

The question: "What happens when a generator is garbage-collected before being fully consumed — and how do you make sure resources opened inside the generator get released?"

The 30-second answer: When a generator is GC'd (or you call `gen.close()`), Python injects a `GeneratorExit` exception at the pause point. The body wakes up, the exception propagates, and any `finally` / `with` blocks on the way out execute their cleanup. If the body catches `GeneratorExit` and yields again, Python raises `RuntimeError`. The pattern: wrap resources in `try/finally` or `with` so cleanup runs deterministically when the generator is closed.

Step-by-step thought process: 1. "Three exit paths: natural exhaustion, explicit `close()`, garbage collection. All three trigger cleanup, but at different times." 2. "On exhaustion: body falls off the end naturally, `try/finally` runs as usual." 3. "On `close()` or GC: Python raises `GeneratorExit` at the pause point. `try/finally` and `with` blocks run their cleanup. The exception then propagates out silently — caller doesn't see it." 4. "Footgun: catching `GeneratorExit` and continuing to yield raises `RuntimeError: generator ignored GeneratorExit`." 5. "Best practice: use `with` for resources. The context manager's `__exit__` runs deterministically, regardless of how the generator ends."

Code:

```

def stream_file(path):
    f = open(path)
    try:
        for line in f:
            yield line
    finally:
        print(f"closing {path}")
        f.close()                                # runs on exhaustion, close(), or GC

# Path 1 – natural exhaustion
for line in stream_file("/etc/hosts"):
    pass
# prints 'closing /etc/hosts' at the end

# Path 2 – explicit close()
g = stream_file("/etc/hosts")
next(g); next(g)                                # consume two lines
g.close()                                        # injects GeneratorExit
# prints 'closing /etc/hosts'

# Path 3 – garbage collection (no explicit close)
def consume_some():
    g = stream_file("/etc/hosts")
    next(g); next(g)
    # g goes out of scope here – GC eventually closes it

consume_some()
# 'closing' fires whenever GC runs – non-deterministic timing

# The footgun – catching GeneratorExit and yielding again
def bad():
    try:
        yield 1
    except GeneratorExit:
        yield 2                                # ILLEGAL – caller has closed us

g = bad()
next(g)
try:
    g.close()
except RuntimeError as e:
    print(e)                                    # generator ignored GeneratorExit

# `with` block – cleanest pattern
from contextlib import contextmanager

@contextmanager
def transaction(db):
    print(f"BEGIN on {db}")
    try:
        yield                                # control returns to caller body
    except Exception:
        print(f"ROLLBACK on {db}")

```

```

        raise
    else:
        print(f"COMMIT on {db}")

def replay_orders(db, orders):
    with transaction(db):                # __exit__ runs on any exit path
        for o in orders:
            yield process(o)

def process(o): return f"processed-{o}"

# If the consumer stops early, GeneratorExit triggers __exit__ → COMMIT (no exception)
g = replay_orders("primary", ["o1", "o2", "o3"])
next(g); next(g)
g.close()                                # BEGIN / COMMIT on primary

```

Gotchas: - GC timing in CPython is mostly reference-counted (immediate when refcount hits 0), but cycles need the gen-1/2 collector — cleanup can be delayed minutes in cyclic graphs. - Catching `GeneratorExit` (e.g. with bare `except:`) and yielding raises `RuntimeError`. Use `except Exception:` (which excludes `GeneratorExit`, `KeyboardInterrupt`, `SystemExit`). - `async def` generators (asynccgen) use `aclose()` for the same purpose — the cleanup contract is identical but async-flavored.

What NOT to say: - "Generators don't need cleanup." — wrong; if they hold files, sockets, locks, or transactions, they need deterministic cleanup. `finally` / `with` is mandatory. - "GC handles everything." — eventually, but timing is unspecified. For resources with tight lifetimes (DB connections, locks), explicit `close()` is required.

Common follow-up: "You have a generator that opens a database transaction. The consumer breaks out of the loop after one iteration. When does the transaction commit or roll back — and how do you make sure?"

Stand-out line: " `GeneratorExit` is Python's way of asking 'are you done?' — it wakes the generator at its pause point so `finally` and `with` can run. Catch it without cleaning up, and you've leaked the resource and earned a `RuntimeError` when the next caller tries to close you."

Q20 · Generator inside `any()` / `all()` — short-circuit behavior

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Postman, Flipkart, Cred · Topic: Short-circuit consumers

The question: "What's the practical difference between `any(is_fraud(o) for o in orders)` and `any([is_fraud(o) for o in orders])` — and which one would a panel expect you to write?"

The 30-second answer: The generator-expression form (parentheses) lets `any()` short-circuit on the first True — only as many predicates run as needed. The list-comprehension form (brackets) builds the entire list first, runs every predicate, THEN passes the materialised list to `any()`. The generator form is strictly better unless you need the materialised values for something else. Same applies to `all()`, which short-circuits on the first False.

Step-by-step thought process: 1. "`any()` and `all()` are short-circuit functions — they stop the moment the outcome is decided." 2. "With a generator expression, each value is produced on demand. `any()` consumes one, checks, exits if True. So work scales with how long until the first hit." 3. "With a list comprehension, the comprehension runs to completion first — all N predicates evaluated — then `any()` walks the materialised list." 4. "For an expensive predicate (DB lookup, hash compute, regex), generator form can be orders of magnitude faster on early-hit data." 5. "Practical interview answer: 'I default to generator form for `any` / `all` / `next` / `sum` over a filter — anything that can short-circuit or stream'."

Code:

```

import time

def slow_check(order):
    """Simulates an expensive predicate – DB lookup, regex, hash, etc."""
    time.sleep(0.01)          # 10 ms each
    return order.get("amount", 0) > 100_000

orders = [{"id": i, "amount": i * 100} for i in range(2000)]
# orders[1001] is the first one above 100_000

# Generator form – short-circuits at ~order 1001
start = time.perf_counter()
result = any(slow_check(o) for o in orders)
print(f"gen: {result} in {time.perf_counter() - start:.2f}s") # ~10 sec

# List form – runs ALL 2000 predicates first
start = time.perf_counter()
result = any([slow_check(o) for o in orders])
print(f"list: {result} in {time.perf_counter() - start:.2f}s") # ~20 sec

# `all()` – short-circuits on first False
def is_valid(order):
    return order["id"] >= 0 and order["amount"] >= 0

print(all(is_valid(o) for o in orders))    # True – runs all 2000 (no False)

# Add a bad order at index 5
orders[5]["amount"] = -1
print(all(is_valid(o) for o in orders))    # False – short-circuits at index 5

# next() with generator + default – find first match
first_large = next((o for o in orders if o["amount"] > 100_000), None)
print(first_large["id"])                   # 1001

# sum() does NOT short-circuit, but still benefits from no materialisation
total = sum(o["amount"] for o in orders if o["amount"] > 0)
print(total)

# Common interview gotcha – extra brackets kill the optimisation
# WRONG:
# any([slow_check(o) for o in orders])    # materialises first
# RIGHT:
# any(slow_check(o) for o in orders)      # streams

```

Gotchas: - Wrapping a generator expression in `[]` inside `any()` / `all()` is silent — the result is correct but performance is 2x worse on early-hit data. - Generator expressions need parentheses ONLY when they're the sole argument to a function — `any(g for g in xs)` is legal without extra parens. - `any()` on an empty iterable returns `False`; `all()` on an empty iterable returns `True`. Trips candidates who think both default the same way.

What NOT to say: - "The list version is faster because it's eager." — backwards; eager evaluation does MORE work when the consumer can short-circuit. - "`any`" always loops through everything." — false; that's the entire point of short-circuit. Saying this signals confusion about the predicate semantics.

Common follow-up: "You need to check if any of 10,000 records match a slow predicate, and you also need to keep the matching records. Generator or list — and why?"

Stand-out line: "`any()` and `all()` with a generator expression is the cleanest short-circuit pattern in Python — predicate cost scales with time-to-first-hit, not collection size. Wrap the generator in brackets and you've quietly turned an $O(k)$ check into an $O(N)$ one."

End of sub-chunk 2 of 6 — Generator functions (Q11–Q20). Next sub-chunk: Generator expressions + comprehensions (Q21–Q30).

Chapter 3 — Generators + Iterators · Sub-chunk 3 of 6 · Generator Expressions + `yield from` (Q21–Q30)

Python Interview Prep — 2026 Edition · Learn & Excel · File 04 Core Python Library

This sub-chunk covers generator expressions — the lazy cousin of list comprehensions — and `yield from`, the delegation primitive that turned generator pipelines from manual loops into one-liners. Razorpay panels for data-platform roles, Swiggy for order-stream processing, Postman for parser engines, Cred for ledger pipelines, Flipkart for catalogue feeds, and Service MNCs (TCS / Infosys / Wipro) for screening rounds all probe whether you reach for a list comp when a genexp would do — and whether you know `yield from` is more than a syntactic shortcut over a `for x in sub: yield x` loop.

Each Q uses the same shape: **30-second answer** (what you say first if the panel is impatient), **thought process** (how to reason out loud), a **code** block (compiles on Python 3.12+), the **gotchas** (where panels watch you trip), **what NOT to say**, **common follow-up**, and a **stand-out line** — the single sentence that signals you've internalised the pause-and-resume model.

Q21 · Generator expression vs list comprehension — memory and laziness

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, TCS, Flipkart, Infosys · Topic: Genexp fundamentals

The question: "What's the difference between `(x*2 for x in nums)` and `[x*2 for x in nums]` — beyond the brackets?"

The 30-second answer: The list comp `[...]` builds the entire list in memory at once — $O(N)$ memory, eager evaluation. The generator expression `(...)` returns a generator object that produces values one at a time on demand — $O(1)$ memory, lazy. For a million-row CSV stream, the genexp keeps memory flat; the list comp loads the whole transformed dataset into RAM.

Step-by-step thought process: 1. "Brackets pick the data structure: `[]` materialises a list, `()` returns a generator." 2. "List comp runs the loop to completion before the next line executes. Genexp doesn't run anything until something iterates it." 3. "Memory profile is the headline difference — a list comp over 10M rows is 10M items in RAM; a genexp is one item at a time plus the frame state." 4. "Second difference: a list is re-iterable; a genexp is single-pass. Once exhausted, it's done — no rewind." 5. "Default choice in modern code: genexp for one-pass pipelines, list comp when you need indexing, `len()`, multiple passes, or random access."

Code:

```
import sys

nums = range(1_000_000)

# List comp — materialises all 1M items
lc = [x * 2 for x in nums]
print(sys.getsizeof(lc) // 1024, "KB list")      # ~8000+ KB

# Genexp — constant size, no items materialised yet
gx = (x * 2 for x in nums)
print(sys.getsizeof(gx), "bytes genexp")        # ~200 bytes (frame only)

# Lazy proof — genexp body runs only when consumed
def shout(x):
    print("computing", x)
    return x * 2

g = (shout(x) for x in [1, 2, 3])                # nothing prints
print("after build")                            # after build
next(g)                                         # computing 1
next(g)                                         # computing 2

# Single-pass — second iteration gets nothing
g = (x for x in [10, 20, 30])
print(list(g))                                  # [10, 20, 30]
print(list(g))                                  # [] — exhausted
```

Gotchas: - A genexp is a generator — it's exhausted after one pass. Code that passes the same genexp to two functions sees the second function get an empty iterator. - `sum([x for x in big])` builds the list first, then sums; `sum(x for x in big)` streams. Same result, very different memory. - A genexp captures the surrounding scope — closing over a loop variable that changes later is the classic late-binding trap (see Q25).

What NOT to say: - "They're the same, just different syntax." — wrong; they have fundamentally different memory and re-iteration semantics. - "Use list comp always — it's faster." — list comp is faster *to fully materialise*, but slower per-item-needed when you only consume a prefix.

Common follow-up: "You have a 10GB CSV. Walk me through summing the second column without loading the file into RAM."

Stand-out line: "List comp is a snapshot; genexp is a stream. The brackets aren't cosmetic — they decide whether the data exists in memory or only conceptually until someone asks for the next item."

Q22 · Generator expression inside a function call — no extra parens

Tags: Difficulty: Easy · Asked at: Cred, Postman, Walmart Labs, Razorpay, Infosys · Topic: Genexp call syntax

The question: "Why can you write `sum(x*2 for x in lst)` without the outer parens, but `print(x for x in lst)` prints a generator object?"

The 30-second answer: When a generator expression is the **sole argument** to a function call, Python lets you drop the surrounding parens — `sum(x for x in lst)` is the same as `sum((x for x in lst))`. `print` showing a generator object isn't a syntax issue; it's because `print()` calls `repr()` on its args instead of iterating them. The genexp is the argument, not the thing being printed.

Step-by-step thought process: 1. "Rule one: paren-dropping works only when the genexp is the single argument. `func(x for x in lst, extra)` is a syntax error." 2. "Rule two: the receiving function must iterate the argument for the laziness to mean anything — `sum`, `max`, `min`, `any`, `all`, `''.join`, `sorted` all do." 3. "`print(g)` doesn't iterate `g` — it calls `repr(g)`, which returns the `<generator object ...>` string. That's the bug pattern beginners hit." 4. "`print(*g)` does work — unpacking iterates the generator before passing the items as args." 5. "The idiom for the reducer family: `sum(x*x for x in nums)`, `max(score for score in scores if score > 0)`, `''.join(str(x) for x in xs)`. All single-arg, all stream."

Code:

```

nums = [1, 2, 3, 4, 5]

# Paren-drop – single argument, no extra parens needed
print(sum(x * 2 for x in nums))          # 30
print(max(x ** 2 for x in nums))        # 25
print(any(x > 3 for x in nums))         # True
print("".join(str(x) for x in nums))    # 12345

# Multi-arg – parens required
from functools import reduce
total = reduce(lambda a, b: a + b, (x * 2 for x in nums), 0) # parens MUST be there
print(total)                                # 30

# The print trap – print does NOT iterate
g = (x for x in [1, 2, 3])
print(g)                                    # <generator object <genexpr> at 0x...>
print(*g)                                # 1 2 3 – unpacking iterates

# Razorpay-style – sum amounts that pass a fraud filter, streaming
def safe_score(txn):
    return txn["amount"] if txn["score"] > 0.7 else 0

txns = [{"amount": 500, "score": 0.2}, {"amount": 9000, "score": 0.95}]
clean_total = sum(safe_score(t) for t in txns) # paren-drop, no list built
print(clean_total)                            # 500

```

Gotchas: - Adding redundant parens (`sum((x for x in lst))`) is legal but reads as a tuple in code review — strip the inner pair. - `print(g)` mystifies juniors. The fix is `print(*g)` if you want the items, or `print(list(g))` if you want a printable representation. - Two-arg `reduce(func, genexp, initial)` requires the genexp parens — paren-drop only works when the genexp is the *only* arg.

What NOT to say: - "Python has a special syntax for `sum` ." — wrong; it's general grammar for any single-arg call. - "Always wrap in `list()` first." — defeats the streaming purpose; only do it when you genuinely need a list.

Common follow-up: "Why does `print(x for x in [1,2,3])` not give a syntax error even though it doesn't print the values?"

Stand-out line: "Paren-drop is grammatical sugar for the common reducer pattern — `sum` , `max` , `any` , `join` were the use cases the syntax was designed around, and once you see that, every genexp becomes the natural argument shape."

Q23 · Composing generator expressions — chained pipelines

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Postman, Cred, JP Morgan · Topic: Genexp composition

The question: "Show me how you'd build a streaming pipeline that filters, transforms, and aggregates a million records — using only generator expressions."

The 30-second answer: Chain genexps by feeding the output of one as the input to the next —

`parsed = (parse(line) for line in lines); valid = (p for p in parsed if p.ok); amounts = (p.amount for p in valid); total = sum(amounts)` . Each stage is lazy; values flow one at a time through the whole pipe. Memory stays O(1) regardless of input size, and you can swap stages without touching the rest.

Step-by-step thought process: 1. "Each genexp consumes one upstream and produces one downstream — that's a pipe stage." 2. "Composition is just naming:

`stage_b = (transform(x) for x in stage_a)` . No materialisation happens between stages." 3. "The whole pipeline doesn't execute until a terminal consumer pulls — `sum(...)` , `list(...)` , `next(...)` , a `for` loop." 4. "This is functionally equivalent to `map` / `filter` chains but reads more like the data shape — `(amount for txn in txns if txn.status == 'paid' for amount in [txn.amount])` is one fused stage." 5. "Production pattern: define each stage as a separate genexp so logs / metrics can be attached, and the pipeline reads top-to-bottom like a SQL CTE chain."

Code:

```

# Razorpay-style: parse → filter → transform → aggregate
import io

raw = io.StringIO(
    "txn_001,500,paid\n"
    "txn_002,9000,failed\n"
    "txn_003,1200,paid\n"
    "txn_004,300,refunded\n"
    "txn_005,2500,paid\n"
)

# Stage 1 – parse each line into a dict
parsed = (
    {"id": parts[0], "amount": int(parts[1]), "status": parts[2].strip()}
    for parts in (line.split(",") for line in raw)
)

# Stage 2 – filter only paid
paid = (txn for txn in parsed if txn["status"] == "paid")

# Stage 3 – extract amount with 18% GST
with_gst = (txn["amount"] * 1.18 for txn in paid)

# Terminal – sum
total = sum(with_gst)
print(round(total, 2))                                # 4956.0

# Nothing is materialised – try the same on 10M rows, RAM stays flat

# Inline fused stage – same idea, one genexp with two `for` and one `if`
def fee(amount):
    return amount * 0.02

txns = [
    {"id": "t1", "amount": 500, "status": "paid"},
    {"id": "t2", "amount": 9000, "status": "failed"},
    {"id": "t3", "amount": 1200, "status": "paid"},
]
fees = sum(
    fee(t["amount"])
    for t in txns
    if t["status"] == "paid"
)
print(round(fees, 2))                                # 34.0

```

Gotchas: - A pipeline is exhausted on first consumption — calling `list(paid)` after `sum(with_gst)` gives `[]` because the upstream is drained. - Exceptions raised mid-pipeline propagate up through every stage to the terminal consumer — debugging means tracing through each `(x for x in ...)`. - Lazy pipelines hide side effects — if `parse(line)` writes a log, you won't see the log until the terminal consumer pulls.

What NOT to say: - "Use Spark for streaming." — overkill; genexps handle millions of rows on a single laptop with no dependencies. - "Wrap each stage in `list()` for clarity." — kills the memory advantage; the whole point is staying streaming.

Common follow-up: *"How would you add a 'log every 1000 rows' side-effect stage without breaking laziness?"*

Stand-out line: *"A chained genexp pipeline is the Pythonic answer to `map` / `filter` / `reduce` — each line reads like a SQL stage, but nothing executes until the terminal consumer pulls, and memory stays flat no matter how big the input."*

Q24 · When to choose a list comp anyway

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Walmart Labs, Cred, Flipkart · Topic: When NOT to use genexp

The question: *"You said genexps are memory-efficient. When would you reach for a list comp instead?"*

The 30-second answer: List comp when you need any of: multiple passes over the data, random access by index, `len()` of the result, membership testing more than once, or when the downstream API requires a sequence (e.g. `random.choice`, `bisect`, JSON serialisation). Also when the dataset is small enough that the materialisation cost is irrelevant — readability wins.

Step-by-step thought process: 1. "Genexp is single-pass. If I need to iterate twice, list comp wins — otherwise I'd wrap in `list()` anyway and lose the benefit." 2. "Indexing: `result[0]`, `result[-1]`, `result[5:10]` need a sequence. Genexps don't support `__getitem__`." 3. "`len()` is $O(1)$ on a list, but a genexp has no length until exhausted — `len(gx)` raises `TypeError`." 4. "APIs that expect a sequence: `random.choice`, `bisect.insort`, `json.dump`, `sorted` (well, `sorted` accepts genexps but materialises internally anyway)." 5. "Small data + readability: a 50-item list comp is fine to materialise. The genexp advantage shows up at thousands+ items or unbounded streams."

Code:

```

import random
import json

# Need len() – genexp can't give it
gx = (x * 2 for x in range(10))
try:
    print(len(gx))                                # TypeError
except TypeError as e:
    print("genexp:", e)                            # object of type 'generator' has no len()

lc = [x * 2 for x in range(10)]
print("list len:", len(lc))                        # 10

# Need indexing – genexp doesn't support it
try:
    print(gx[0])                                  # TypeError
except TypeError as e:
    print("genexp:", e)                            # 'generator' object is not subscriptable

print("list[0]:", lc[0])                          # 0

# random.choice – needs a sequence
choices = ["upi", "card", "netbanking", "wallet"]
print(random.choice(choices))                     # OK – list works
# random.choice(x for x in choices)               # IndexError under the hood

# Multiple passes – list comp obviously
amounts = [500, 1200, 300, 9000, 2500]
filtered = [a for a in amounts if a < 5000]        # need it twice
total = sum(filtered)
avg = total / len(filtered)
print(f"total={total} avg={avg:.0f}")              # total=4500 avg=1125

# Small data + readability – list comp is the friendlier choice
states = ["KA", "MH", "TN", "DL"]
upper = [s.lower() for s in states]               # 4 items – genexp is overkill
print(upper)

```

Gotchas: - `len(gx)` raising `TypeError` confuses people because `len(list(gx))` works — but it exhausts the generator. - `sorted(genexp)` works but quietly materialises into a list internally — the laziness vanishes the moment `sorted` is called. - JSON serialisation of a generator raises `TypeError: Object of type generator is not JSON serializable` — convert to list first if you're dumping.

What NOT to say: - "Genexp is always better." — wrong; the right choice depends on access pattern. - "List comp is for beginners, genexp is pro." — both are tools; reaching for the wrong one is unprofessional.

Common follow-up: "You wrote `sum(g) + max(g)` and got `max() arg is an empty sequence`. Why?"

Stand-out line: "Genexp is a stream; list comp is a snapshot. Pick the stream when data is large or one-pass; pick the snapshot when downstream needs indexing, length, or repeated access."

Q25 · Generator expression late-binding gotcha

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, JP Morgan, Cred, Walmart Labs · Topic: Closure semantics in genexps

The question: "What does this print: `gs = [(x for x in [i]) for i in range(3)]; print([list(g) for g in gs])` — and why?"

The 30-second answer: It prints `[[0], [1], [2]]` — but the closely-related `gs = [lambda: i for i in range(3)]` prints `[2, 2, 2]`. The difference: the **outermost** `for` clause of a genexp is evaluated *eagerly* when the genexp is created, so `[i]` is captured immediately. But variables in the *body* and in *inner* `for` / `if` clauses are looked up lazily — that's where late-binding bites.

Step-by-step thought process: 1. "Late binding = the name is resolved when the closure runs, not when it was defined. By that time, the loop variable has moved on." 2. "Critical detail per the language reference: the iterable in the leftmost `for` clause of a genexp is evaluated eagerly. Everything else is lazy." 3. "So `(x for x in some_iterable)` evaluates `some_iterable` at creation; `x` is rebound per `next()` call. Variables outside the genexp captured by name are late-bound." 4. "Classic bug: `funcs = [(x + i for x in [10]) for i in range(3)]`. The `i` is looked up when each genexp runs — by then the comprehension's loop var `i` is `2`. All three give `12`." 5. "The fix: capture by default-argument trick `(x + i for x in [10] for i in [i])` — or just use a list comp / materialise eagerly."

Code:


```

# Surprise 1 – leftmost iterable IS eagerly evaluated
gs = [(x for x in [i]) for i in range(3)]
print([list(g) for g in gs])           # [[0], [1], [2]] – looks fine

# Surprise 2 – variables referenced by name in the body are LATE-bound
funcs = [(x + i for x in [10]) for i in range(3)]
print([list(f) for f in funcs])       # [[12], [12], [12]] – all use i=2

# Why: the genexp closes over the cell `i`, not the value at creation
# By the time any genexp is consumed, the outer comprehension's i has finished at 2

# The standard fix – bind via the leftmost for clause (evaluated eagerly)
funcs = [(x + i for i in [i] for x in [10]) for i in range(3)]
print([list(f) for f in funcs])       # [[10], [11], [12]] – fixed

# Lambda equivalent of the same bug – shows it's a closure issue, not genexp-specific
fns = [lambda x: x + i for i in range(3)]
print([f(10) for f in fns])          # [12, 12, 12] – same trap

# The "default argument" capture trick – works for lambdas
fns = [lambda x, i=i: x + i for i in range(3)]
print([f(10) for f in fns])          # [10, 11, 12]

# Razorpay-style trap – building per-shard reducers
shard_ids = ["A", "B", "C"]
reducers = [(f"{sid}: {amt}" for amt in [100]) for sid in shard_ids]
print([list(r) for r in reducers])   # ['A: 100'], ['B: 100'], ['C: 100'] – safe (sid)

```

Gotchas: - The leftmost `for` clause is evaluated eagerly — this is a documented Python guarantee, not an implementation accident. - Variables from the enclosing scope referenced inside the genexp body are looked up when the genexp is consumed, not when it's created. - The "default argument" trick (`lambda x, i=i: ...`) doesn't translate cleanly to genexps; the idiom there is to push the binding into a leftmost `for` clause.

What NOT to say: - "Just use a list comp." — works, but doesn't show you understand the closure semantics; interviewers want the cause. - "Python is buggy here." — it's deliberate; same rule as every other closure in Python. Saying "buggy" downgrades you.

Common follow-up: "Refactor the broken loop using `functools.partial` instead — does that fix it?"

Stand-out line: "The leftmost `for` is eager, the rest is lazy — same closure rule as every other Python scope, but it bites harder in genexps because people expect each generator to remember the value of `i` at creation time, when really it's keeping a reference to the cell."

Q26 · `yield from` basics — delegation to a sub-iterable

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, Flipkart, Cred · Topic: `yield from` fundamentals

The question: "What does `yield from` do, and how is it different from `for x in sub: yield x`?"

The 30-second answer: `yield from <iterable>` delegates the entire iteration to a sub-iterable — every value the sub-iterable produces flows through to whoever's consuming the outer generator. It's syntactic sugar for the manual `for` loop **for plain iteration** — but for full generator delegation, it also routes `send()`, `throw()`, and the sub-generator's `return` value, which the manual loop does not.

Step-by-step thought process: 1. "Surface use: collapse a nested generator pipeline into one line. Instead of `for x in inner(): yield x`, write `yield from inner()`." 2. "PEP 380 added it in 3.3. Before that, every delegation needed the manual loop, which couldn't carry the bidirectional channel." 3. "For consumer-side iteration only (`for x in outer():`), the manual loop and `yield from` are observationally identical." 4. "For coroutine-style generators using `send()` / `throw()` / sub-`return`, `yield from` is the only correct delegation primitive." 5. "Mental model: `yield from inner` makes the outer generator a transparent pipe — values, exceptions, and the return value flow through."

Code:

```

# Surface-level – flatten a tree of sub-generators
def odd_squares():
    yield 1
    yield 9
    yield 25

def even_squares():
    yield 4
    yield 16
    yield 36

def all_squares():
    yield from odd_squares()
    yield from even_squares()

print(list(all_squares()))          # [1, 9, 25, 4, 16, 36]

# Manual equivalent – works for plain iteration
def all_squares_manual():
    for x in odd_squares():
        yield x
    for x in even_squares():
        yield x

print(list(all_squares_manual()))   # same output

# Practical use – flatten nested order batches
def batches():
    yield ["order-1", "order-2"]
    yield ["order-3"]
    yield ["order-4", "order-5", "order-6"]

def flat_orders():
    for batch in batches():
        yield from batch            # delegate to the list (iterable)

print(list(flat_orders()))          # ['order-1', 'order-2', ..., 'order-6']

# Recursive flattening with yield from – clean and idiomatic
def flatten(nested):
    for item in nested:
        if isinstance(item, list):
            yield from flatten(item) # recurse via delegation
        else:
            yield item

print(list(flatten([1, [2, [3, [4, 5]], 6], 7]))) # [1, 2, 3, 4, 5, 6, 7]

```

Gotchas: - `yield from <not-an-iterable>` raises `TypeError` – the right operand must be iterable. - `yield from gen()` exhausts `gen()` – calling `next()` on the outer generator after delegation finishes raises `StopIteration` normally. - For a chain of `yield from` calls, exceptions

raised deep inside the chain propagate up through every level transparently — easy to debug, hard to mask.

What NOT to say: - "It's just shorter syntax for a for loop." — true for plain iteration only; the bidirectional channel is the real reason it was added. - "You only need it for async code." — wrong; `yield from` predates `async/await` and works on plain generators.

Common follow-up: "Write a recursive directory walker using `yield from` that produces every file path."

Stand-out line: "`yield from` is the transparent-pipe operator — values, exceptions, `send()`, `throw()`, and the sub-generator's return value all flow through it untouched. The manual `for` loop carries values only; everything else gets dropped."

Q27 · `yield from` exception propagation

Tags: Difficulty: Hard · Asked at: Cred, Postman, Razorpay, JP Morgan, Walmart Labs · Topic: Exception flow through delegation

The question: "If a sub-generator raises an exception, what happens to the outer generator using `yield from`? Walk through both directions — exceptions from the sub-gen, and exceptions thrown into the outer via `gen.throw()`."

The 30-second answer: Exceptions raised *inside* the sub-generator propagate up through `yield from` to the outer generator's caller — same as if the outer raised it directly. Exceptions thrown *into* the outer via `outer.throw(...)` are routed down to the sub-generator at its current `yield`. If the sub-gen doesn't handle it, the exception bubbles back up through the `yield from` boundary. Clean, two-way exception transparency.

Step-by-step thought process: 1. "Upward direction: sub-gen raises → `yield from` lets it through → outer's caller sees the exception. No catching at the `yield from` line unless you wrap it in `try/except`." 2. "Downward direction: `outer.throw(E)` → if outer is currently paused at a `yield from`, the exception is sent INTO the sub-generator at its current yield point." 3. "Sub-gen can handle it with a `try/except` around its `yield`. If it doesn't, the exception bubbles back through `yield from` to the outer's caller." 4. "PEP 380 specified this routing — it's what makes `yield from` correct for coroutine delegation, not just iteration." 5. "In practice: wrap the `yield from` in `try/except` if the outer needs to recover; otherwise let it propagate. Cleaner than the manual loop, which has to re-route exceptions manually."

Code:

```

# Upward propagation – sub-gen raises, outer's caller sees it
def faulty():
    yield 1
    raise ValueError("bad row")

def outer_pass_through():
    yield "start"
    yield from faulty()
    yield "end"                                # never reached

g = outer_pass_through()
print(next(g))                                # 'start'
print(next(g))                                # 1
try:
    next(g)
except ValueError as e:
    print("caught upward:", e)                 # caught upward: bad row

# Outer can catch and recover
def outer_recovers():
    yield "start"
    try:
        yield from faulty()
    except ValueError:
        yield "recovered"
    yield "end"

print(list(outer_recovers()))                 # ['start', 1, 'recovered', 'end']

# Downward propagation – throw INTO the outer, gets routed to sub-gen
def sub_handles_throw():
    try:
        yield "sub-1"
        yield "sub-2"
    except RuntimeError:
        yield "sub-caught"

def outer_throws_down():
    yield "outer-1"
    yield from sub_handles_throw()
    yield "outer-end"

g = outer_throws_down()
print(next(g))                                # 'outer-1'
print(next(g))                                # 'sub-1' – paused inside sub-gen
print(g.throw(RuntimeError))                  # 'sub-caught' – sub-gen handled it
print(next(g))                                # 'outer-end'

# Sub-gen doesn't handle – bubbles back up
def sub_doesnt_handle():
    yield "sub-a"
    yield "sub-b"

```

```
def outer_no_catch():
    yield "outer-a"
    yield from sub_doesnt_handle()
    yield "outer-b"

g = outer_no_catch()
next(g); next(g)
try:
    g.throw(RuntimeError("unhandled"))
except RuntimeError as e:
    print("bubbled all the way up:", e)          # bubbled all the way up: unhandled
```

Gotchas: - A bare `except:` around `yield from` swallows `GeneratorExit` — bad practice; it stops the generator from cleaning up properly. - `throw()` requires the generator to be at a `yield` (started, not exhausted) — calling `throw()` on a fresh generator raises immediately at the first line. - If the sub-gen handles the exception by yielding, the value flows back out — `g.throw(E)` returns the next yielded value, not `None`.

What NOT to say: - "Exceptions stop everything." — wrong; the outer can catch and continue, or the sub-gen can handle the thrown exception. - "I'd just try/except around every `yield`." — over-engineered; `yield from`'s transparency is the whole point.

Common follow-up: "What if the sub-generator runs cleanup code in a `finally` block — does it execute when the outer is garbage-collected mid-iteration?"

Stand-out line: "`yield from` is exception-transparent in both directions — raises bubble up, `throw()` is routed down to the current sub-gen yield point. That bidirectional symmetry is what lets coroutine libraries (the old `asyncio.coroutine` era) treat delegation as a primitive."

Q28 · Equivalence to manual loop — and when it differs

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Postman, Flipkart, Cred · Topic: yield from vs manual loop

The question: "Is `yield from gen()` exactly the same as `for x in gen(): yield x`? Where do they actually diverge?"

The 30-second answer: For one-way iteration with no `send`, `throw`, or sub-return, they're observationally identical. The divergence shows up for: (1) `send()` values are routed to the sub-gen with `yield from`, dropped with the manual loop; (2) `throw()` is routed into the sub-gen, raised at the manual `yield` in the loop case; (3) the sub-gen's `return value` is captured by `yield from` as the expression value, lost entirely with the manual loop.

Step-by-step thought process: 1. "PEP 380 spelled out the full equivalence — it's about 30 lines of `try / except / finally` to replicate `yield from` manually." 2. "Forward-only iteration: identical. Both pull from `gen()` and yield each value." 3. "Bidirectional: `outer.send(v)` — `yield from`

routes `v` into the sub-gen's `yield`; manual `for x in gen(): yield x` discards it (the sent value goes into the outer's `yield x`, then nowhere)." 4. "Throw: `outer.throw(E)` — `yield from` calls `sub.throw(E)`; manual loop raises at `yield x`, which terminates the outer with the exception, never entering the sub." 5. "Return value: `result = yield from sub()` captures `sub`'s return; the manual loop has no place to put it."

Code:

```

# Sub-generator that uses send() AND returns a value
def echoing_sub():
    received = yield "give me input"
    received2 = yield f"got {received}"
    return f"final-{received2}"

# Outer using yield from – bidirectional, captures return
def outer_with_yield_from():
    result = yield from echoing_sub()
    yield f"sub returned: {result}"

g = outer_with_yield_from()
print(next(g))                # 'give me input'
print(g.send("hello"))        # 'got hello' – send routed into sub
print(g.send("world"))        # 'sub returned: final-world'

# Outer using manual loop – send is lost, return is lost
def outer_with_manual_loop():
    for x in echoing_sub():
        yield x

g = outer_with_manual_loop()
print(next(g))                # 'give me input'
print(g.send("hello"))        # 'got None' – send went to outer's yield, sub got None
# The 'final-...' return value is silently dropped

# Throw routing difference
def sub_with_catch():
    try:
        yield "a"
        yield "b"
    except ValueError:
        yield "caught"
    yield "c"

def outer_yf():
    yield from sub_with_catch()

def outer_manual():
    for x in sub_with_catch():
        yield x

g1 = outer_yf()
print(next(g1))                # 'a'
print(g1.throw(ValueError))    # 'caught' – sub handled it

g2 = outer_manual()
print(next(g2))                # 'a'
try:
    g2.throw(ValueError)        # raises at outer's `yield x` – sub never sees it
except ValueError:
    print("manual loop: sub never got the throw")

```


Gotchas: - The PEP 380 expansion is non-trivial — about 30 lines including `try/except/finally` to handle close, throw, and return-value capture. - A common bug pattern when refactoring old code from the manual loop to `yield from`: code that relied on the outer catching `throw()` breaks because `yield from` now routes the throw to the sub. - `yield from` calls `sub.close()` when the outer is closed — manual loop does not, so resources held inside the sub-gen leak.

What NOT to say: - "They're the same." — wrong for any coroutine-style use; this answer fails the senior bar. - "PEP 380 just added syntactic sugar." — minimises the bidirectional channel that made coroutines (and later `async/await`) possible.

Common follow-up: "Show me the full PEP 380 expansion of `RESULT = yield from EXPR` — including the close, throw, and return-value handling."

Stand-out line: "For dumb iteration they're twins; for coroutine delegation they're not even cousins. `yield from` is bidirectional — send, throw, close, and return all flow correctly. The manual loop is one-way, which is fine for data pipelines but breaks coroutine semantics."

Q29 · Sub-generator return value capture

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Cred, JP Morgan, Walmart Labs · Topic: Generator return values

The question: "A generator can have a `return value` statement. How do you capture that return value with `yield from`?"

The 30-second answer: Write `result = yield from sub_gen()`. The expression value of `yield from` is whatever the sub-generator returned. Internally, the sub-gen's `return value` raises `StopIteration(value)`; `yield from` catches that, extracts `.value`, and provides it as the result of the expression. The manual `for x in sub_gen(): yield x` loop has no way to access this — `StopIteration.value` is silently discarded by `for`.

Step-by-step thought process: 1. "In Python 3.3+, a generator can have `return some_value` — it raises `StopIteration(some_value)` to signal both 'done' and 'here's a final result'." 2. "Plain `for` loops discard `StopIteration.value` — that's the data-loss point if you don't use `yield from`." 3. "`yield from` is the only legitimate way to consume the return value without manually wrapping the sub-gen call in `try/except` `StopIteration as e: result = e.value`." 4. "Use case: pipeline stages that report a final summary — number of rows processed, total dropped, last seen ID." 5. "This was the killer feature that made `yield from` more than sugar — coroutine libraries built on top of it routed final return values back to `await`-style consumers."

Code:

```

# Sub-gen yields rows AND returns a summary
def parse_rows(lines):
    total = 0
    skipped = 0
    for line in lines:
        try:
            parts = line.split(",")
            yield {"id": parts[0], "amount": int(parts[1])}
            total += 1
        except (ValueError, IndexError):
            skipped += 1
    return {"processed": total, "skipped": skipped}

# Outer captures the return value via yield from
def pipeline(lines):
    summary = yield from parse_rows(lines)
    print(f"summary captured: {summary}")
    yield summary                                # surface to consumer too

lines = ["t1,500", "t2,bad", "t3,1200", "broken"]
result = list(pipeline(lines))
# summary captured: {'processed': 2, 'skipped': 2}
print(result[-1])                             # {'processed': 2, 'skipped': 2}

# Manual loop – return value is silently lost
def pipeline_manual(lines):
    for row in parse_rows(lines):
        yield row
    # summary is gone – no way to access it here

# To capture without yield from, you'd write:
def pipeline_verbose(lines):
    gen = parse_rows(lines)
    while True:
        try:
            row = next(gen)
        except StopIteration as e:
            summary = e.value                    # the only way without yield from
            yield {"_summary": summary}
            return
    yield row

print(list(pipeline_verbose(["a,1", "b,2"])))  # rows + summary

```

Gotchas: - Bare `return` (no value) in a generator yields `StopIteration(None)` — `result` will be `None`, not unset. - `yield from` only captures the return value of the IMMEDIATE sub-gen — if there's a chain, only the directly-delegated one's return surfaces. - A `for` loop over a generator that returns a value silently throws away the return value — common data-loss point for pipelines.

What NOT to say: - "Generators can't return values." — wrong since Python 3.3; this answer dates you to Python 2. - "Use a closure-over-list to capture the result." — works but ugly; `yield from` is the language-level answer.

Common follow-up: *"Build a streaming JSON-array parser that yields each element and returns the byte offset where parsing finished."*

Stand-out line: *"`return value` inside a generator is the only clean way to ship a side-channel final result — and `yield from` is the only clean way to receive it. Everything else is `try/except StopIteration` boilerplate."*

Q30 · Transparent-pipe pattern for chaining generators

Tags: Difficulty: Hard · Asked at: Swiggy, Razorpay, Postman, Cred, JP Morgan · Topic: yield from architecture

The question: *"Design a multi-stage data pipeline where each stage is a generator. Use `yield from` so the consumer sees one flat stream — and explain why this beats nested loops."*

The 30-second answer: Each stage is a generator function that takes an upstream iterable and yields transformed values. `yield from upstream` (with optional inline filter / map) makes the stage a transparent pipe. The consumer sees one flat stream regardless of how many stages there are. Beats nested loops because: stages are independently testable, composable, and `send` / `throw` route correctly through the whole chain.

Step-by-step thought process: 1. "Stage anatomy: `def stage(upstream): for item in upstream: yield transformed(item)` — but if a stage delegates entirely, write `yield from upstream` and add inline logic." 2. "Composition is just function calls: `stage3(stage2(stage1(source)))`. Each stage returns a generator; chaining is plumbing." 3. "`yield from` makes branches transparent — a stage that splits an input into multiple sub-streams and re-merges them stays correct even with backpressure." 4. "Testing: each stage is a generator taking an iterable. Unit-test by feeding a list and consuming the output — no mocking needed." 5. "Why beats nested loops: linear visual layout (top-to-bottom = data flow), independent stages, no manual exception routing, single-pass and memory-flat."

Code:

```

# Razorpay-style: parse raw lines → enrich with shard → filter by amount → tag with priority

# Source – a generator over raw bytes-style input
def source():
    yield "txn_001,500,user_a"
    yield "txn_002,9000,user_b"
    yield "txn_003,300,user_c"
    yield "txn_004,1200,user_d"

# Stage 1 – parse
def parse(upstream):
    for line in upstream:
        parts = line.split(",")
        yield {"id": parts[0], "amount": int(parts[1]), "user": parts[2]}

# Stage 2 – enrich (delegates plain pass-through then adds field)
def enrich(upstream):
    shard_map = {"user_a": "shard-1", "user_b": "shard-2",
                 "user_c": "shard-1", "user_d": "shard-3"}
    for txn in upstream:
        txn["shard"] = shard_map.get(txn["user"], "shard-default")
        yield txn

# Stage 3 – filter (small enough to inline)
def filter_high_value(upstream, threshold=1000):
    yield from (t for t in upstream if t["amount"] >= threshold)

# Stage 4 – tag (delegates via yield from with sub-gen helper)
def tag_priority(upstream):
    def with_tag(t):
        t["priority"] = "P0" if t["amount"] >= 5000 else "P1"
        return t
    yield from (with_tag(t) for t in upstream)

# Wire the pipeline
pipeline = tag_priority(filter_high_value(enrich(parse(source()))))

for txn in pipeline:
    print(txn)
# {'id': 'txn_002', 'amount': 9000, 'user': 'user_b', 'shard': 'shard-2', 'priority': 'P0'}
# {'id': 'txn_004', 'amount': 1200, 'user': 'user_d', 'shard': 'shard-3', 'priority': 'P1'}

# Branching + merging with yield from – a fan-out / fan-in stage
def split_by_shard(upstream):
    """Split into per-shard sub-streams, then re-merge in order."""
    buckets = {}
    for txn in upstream:
        buckets.setdefault(txn["shard"], []).append(txn)
    for shard, txns in sorted(buckets.items()):
        yield from txns # transparent re-merge

batch = split_by_shard(enrich(parse(source())))
print([t["id"] for t in batch]) # ordered by shard

```

Gotchas: - A stage that materialises (calls `list()` or builds a dict) breaks streaming for everything downstream — memory grows. - Stages that depend on previous-row state need explicit accumulators inside the stage; `yield from` doesn't carry across stages. - Exception in stage N propagates all the way to the consumer — log at the consumer, not at intermediate stages, or you'll log the same error N times.

What NOT to say: - "Use a class-based pipeline framework." — over-engineering for most cases; functional genexp pipelines are the Pythonic baseline. - "I'd materialise between stages for safety." — defeats the streaming model that's the entire point.

Common follow-up: "How would you add a 'commit every 1000 rows' side-effect stage that doesn't break streaming for downstream?"

Stand-out line: " `yield from` is what lets a generator pipeline read like a Unix shell pipe — each stage is a transparent transformer, composition is function call, and the consumer sees one flat stream regardless of how many stages deep the data flowed."

End of sub-chunk 3 of 6 — Generator expressions + `yield from` (Q21–Q30). Next sub-chunk: `itertools` core (Q31–Q40).

Chapter 3 — Generators + Iterators · Sub-chunk 4 of 6 · Bidirectional Generators + itertools Part A (Q31–Q40)

Python Interview Prep — 2026 Edition · Learn & Excel · File 04 Core Python Library

This sub-chunk covers two clusters that turn a generator from a one-way data tap into a two-way coroutine, and the `itertools` recipes that show up most often in Indian backend interviews. Bidirectional generators (`send` , `throw` , `close`) are pre- `async/await` machinery — but Razorpay payment-state-machine rounds, Postman parser-engine panels, Cred ledger-replay questions, and JP Morgan event-sourcing rounds still ask them because the protocol underneath `async def` is still the generator protocol. Then we move into `itertools` proper — `chain` , `groupby` , `tee` , `accumulate` — the standard-library tools every Swiggy / Flipkart / Walmart Labs backend engineer is expected to reach for instead of writing list-comprehension acrobatics.

Each Q uses the same shape: **30-second answer**, **thought process**, a **code** block (compiles on Python 3.12+), the **gotchas**, **what NOT to say**, **common follow-up**, and a **stand-out line**.

Q31 · `gen.send(value)` — sending into a generator

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Cred, JP Morgan, Walmart Labs · Topic: Bidirectional generators

The question: "What does `gen.send(value)` do, and what's the role of `value = yield` inside the generator?"

The 30-second answer: `gen.send(value)` resumes a paused generator and makes the paused `yield` expression evaluate to `value`. Inside the generator, `value = yield something` does two things at the same `yield` — it yields `something` out to the caller, then on the next `send(x)` it binds `x` to `value`. This turns the generator into a two-way pipe, not just a data stream.

Step-by-step thought process: 1. "A plain generator is one-way — values flow OUT via `yield`. `send()` adds a second channel — values flow IN at the same `yield`." 2. "Mechanically: the generator pauses at `yield`. When the caller resumes it via `send(x)`, the paused `yield` expression evaluates to `x` instead of `None`." 3. "`next(g)` is equivalent to `g.send(None)`. Both resume the generator; `next` just sends nothing in." 4. "This is the protocol that `await` builds on. Every `async def` you write is a generator with `send` as the resume mechanism — the event loop is the caller." 5. "Production use today is rare — `asyncio` covers most cases. But for stateful parsers and stream protocols, hand-written `send`-driven generators are still the cleanest design."

Code:

```
def running_average():
    total = 0
    count = 0
    average = None
    while True:
        value = yield average          # send-in AND yield-out at the same line
        if value is None:
            break
        total += value
        count += 1
        average = total / count

g = running_average()
next(g)                               # prime — see Q32
print(g.send(100))                    # 100.0
print(g.send(200))                    # 150.0
print(g.send(300))                    # 200.0

# A second example — a configurable filter pipeline
def thresholded():
    threshold = yield                 # first yield receives the config
    while True:
        x = yield
        if x is not None and x >= threshold:
            print(f"alert: {x} >= {threshold}")

p = thresholded()
next(p)
p.send(500)                           # set threshold
p.send(300)                           # below — silent
p.send(700)                           # alert: 700 >= 500
```

Gotchas: - `value = yield X` is a single expression — the `=` binds the *incoming* `send` value, not `X`. Reading it as assignment-then-yield is the wrong mental model. - A bare `yield` (no value after it) yields `None` to the caller — perfectly legal when you only want the send-in channel. - `send()` on a generator that hasn't been primed (no `next()` yet) raises `TypeError: can't send non-None value to a just-started generator`.

What NOT to say: - "`send` is just like `next` but with an argument." — true mechanically, but misses the point that `value = yield` is doing two things at once `yield`. Interviewer wants to hear that nuance. - "I'd use `async/await` instead." — fair in production but sidesteps the protocol question. Answer `send` first, then mention `async` is built on top.

Common follow-up: "What is the protocol difference between `next(g)` and `g.send(None)` — when would you prefer one over the other?"

Stand-out line: "`send` turns a generator into a coroutine — the `yield` becomes a bidirectional gate where data flows out as a value and back in as a binding. That's the same mechanic the `asyncio` event loop uses to drive every coroutine you've ever awaited."

Q32 · Priming with `next()` before first `send()`

Tags: Difficulty: Hard · Asked at: Postman, Razorpay, Cred, Swiggy, JP Morgan · Topic: Generator priming

The question: "Why must you call `next(g)` (or `g.send(None)`) before the first `g.send(value)` — what happens if you don't?"

The 30-second answer: A fresh generator hasn't started executing yet — it's paused *before* the first line, not at the first `yield`. There's no `yield` expression waiting to receive a value. Calling `send(value)` on an unprimed generator raises `TypeError: can't send non-None value to a just-started generator`. Priming with `next(g)` advances execution to the first `yield`, which is where `send` has somewhere to land.

Step-by-step thought process: 1. "Generator creation is lazy — `g = my_gen()` builds the generator object but runs zero lines of the body." 2. "The body only starts running on the first resume. Before that, there is no paused `yield` for a `send` to feed into." 3. "The runtime enforces this: `send(non_None)` on an un-started generator raises `TypeError`. `send(None)` is allowed because it's equivalent to `next()`." 4. "Idiomatic priming: `next(g)` or `g.send(None)`. Both advance to the first `yield`. Pick `next(g)` for clarity." 5. "You can write a decorator `@coroutine` that auto-primers — historically common pre-asyncio. PEP 342 and the Beazley generator-coroutine talks made this the standard pattern."

Code:


```

def echo():
    while True:
        msg = yield
        print("got:", msg)

g = echo()
try:
    g.send("hello")                # FAILS — not primed
except TypeError as e:
    print(e)                       # can't send non-None value to a just-started generator

g = echo()
next(g)                           # PRIMED — now paused at the first yield
g.send("hello")                   # got: hello
g.send("world")                   # got: world

# Auto-priming decorator — the classic pre-asyncio pattern
from functools import wraps

def coroutine(fn):
    @wraps(fn)
    def primed(*args, **kwargs):
        g = fn(*args, **kwargs)
        next(g)
        return g
    return primed

@coroutine
def auto_echo():
    while True:
        msg = yield
        print("auto:", msg)

g = auto_echo()                   # already primed by decorator
g.send("first")                   # auto: first
g.send("second")                  # auto: second

```

Gotchas: - Forgetting to prime is the #1 bug in hand-written coroutines. Symptom: cryptic `TypeError` on the first `send`. - `g.send(None)` is sometimes preferred over `next(g)` in long pipelines because it makes the "no input yet" semantics explicit in code review. - An auto-priming decorator can hide bugs — if the generator raises before its first `yield`, the decorator surfaces the error at construction time, not at use time.

What NOT to say: - "You don't need to prime; it works without." — only works if your first call is `g.send(None)` or `next(g)`. Otherwise raises. - "Priming runs the generator to completion." — wrong; it advances exactly to the first `yield`, then pauses.

Common follow-up: "Write a decorator that auto-priming a coroutine — and explain when you'd choose not to use it."

Stand-out line: "Priming is the cost of using `yield` as a two-way gate — the body has to reach a `yield` before it has somewhere to receive a value. `@coroutine` hides the ceremony, but the protocol underneath stays the same."

Q33 · `gen.throw(exc)` — raising an exception inside the generator

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Postman, Cred, Walmart Labs · Topic: Generator exception injection

The question: "What does `gen.throw(exc)` do — and how would you use it in a real pipeline?"

The 30-second answer: `gen.throw(exc)` resumes the generator and raises `exc` at the paused `yield` expression — as if the `yield` itself threw. The generator can catch it with a `try/except` around the `yield`, or let it propagate back out to the caller. The classic use is signalling "abort the current step but keep the pipeline alive" — for example, telling a parser coroutine to discard the partial state and resync.

Step-by-step thought process: 1. "`throw` is the third resume verb — `next`, `send`, `throw`. All three resume a paused generator; `throw` resumes it by raising at the `yield`." 2. "If the generator wraps `yield` in `try/except`, it catches the exception and continues — and the next thing it `yield`s comes back to `throw`'s caller as the return value." 3. "If the generator doesn't catch, the exception propagates up through `throw()` and the generator is finalised (no more values)." 4. "Real use: parser/state-machine coroutines that need to be told 'invalid frame, reset.' Cleaner than passing a magic sentinel via `send`." 5. "Python 3.12 simplified the signature — `throw(exc_instance)` is now standard; the older 3-arg `throw(type, value, traceback)` form still works but is deprecated."

Code:

```

class ParseError(Exception):
    pass

def parser():
    buffer = []
    while True:
        try:
            chunk = yield buffer
            buffer.append(chunk)
        except ParseError:
            print("parser: reset on error")
            buffer = []          # discard partial state, resume clean

p = parser()
next(p)
p.send("hello")
p.send("world")
print(p.send("!"))             # ['hello', 'world', '!']
p.throw(ParseError("bad frame")) # parser: reset on error
print(p.send("fresh"))         # ['fresh'] - buffer was cleared

# If generator doesn't catch - exception propagates
def naive():
    while True:
        x = yield
        print("got", x)

g = naive()
next(g)
try:
    g.throw(RuntimeError("kaboom"))
except RuntimeError as e:
    print("caller caught:", e)    # caller caught: kaboom

```

Gotchas: - `throw` on an un-started generator runs the body up to the first `yield` and raises there. Subtle — feels like priming. - After `throw` propagates without being caught, the generator is closed. Further `send/next/throw` raises `StopIteration`. - The exception you throw is raised AT the `yield` line, not at the next line. `try/except` must wrap the `yield` itself, not the code after.

What NOT to say: - "It's just `raise` inside the generator." — wrong; the caller initiates it. The generator only sees the raise *at the paused yield*. - "Use it for normal error handling." — overkill. It's for protocol-level signalling, not for "this number was negative."

Common follow-up: "What's the difference between `gen.throw(SomeError)` and just raising `SomeError` somewhere else — when is the throw idiom the right tool?"

Stand-out line: " `throw` is how you inject a control-flow message into a paused coroutine — the same way an event loop tells a hung task to cancel itself. The generator chooses whether to absorb it (clear state, resume) or let it kill the coroutine."

Q34 · `gen.close()` and `GeneratorExit` — the cleanup protocol

Tags: Difficulty: Hard · Asked at: Cred, Postman, Razorpay, Walmart Labs, JP Morgan · Topic: Generator finalisation

The question: "What does `gen.close()` do, what's `GeneratorExit`, and how do you use them to release resources held by a generator?"

The 30-second answer: `gen.close()` raises `GeneratorExit` at the paused `yield`. `GeneratorExit` is a special exception that *should* let the generator finalise — `try/finally` blocks run, cleanup happens, and the generator must NOT yield another value (doing so is an error). It's the protocol that makes `with` blocks inside generators safe to abandon mid-iteration.

Step-by-step thought process: 1. "Generators are not garbage-collected immediately when you drop the reference — but when they are, the runtime calls `close()` on them automatically." 2. "`close()` raises `GeneratorExit` at the current `yield`. This is your one chance to clean up files, sockets, DB cursors." 3. "`GeneratorExit` inherits from `BaseException`, not `Exception` — so a blanket `except Exception:` does NOT catch it. That's intentional." 4. "Inside the generator: wrap allocations in `try/finally`. The `finally` runs during `close()` and your file handle / connection closes cleanly." 5. "Yielding another value during `GeneratorExit` handling raises `RuntimeError: generator ignored GeneratorExit`. The exception means 'we're shutting down; finalise, don't produce.'"

Code:

```

def stream_rows(path):
    f = open(path, "r")
    try:
        for line in f:
            yield line.rstrip()
    finally:
        print("closing file")
        f.close()

g = stream_rows("/etc/hosts")
print(next(g)[:30])          # first line
g.close()                    # closing file - finally ran

# What happens if you try to yield during GeneratorExit
def bad():
    try:
        yield 1
    except GeneratorExit:
        yield 99              # FORBIDDEN

g = bad()
next(g)
try:
    g.close()
except RuntimeError as e:
    print(e)                  # generator ignored GeneratorExit

# GeneratorExit is NOT caught by except Exception
def safe():
    try:
        yield 1
    except Exception:
        print("won't run on close")    # never prints
    finally:
        print("finally on close")

g = safe()
next(g)
g.close()                    # finally on close

```

Gotchas: - `GeneratorExit` does not inherit from `Exception` — it inherits from `BaseException`. `except Exception:` does NOT swallow it. This is by design. - If you catch `GeneratorExit` and re-raise something else, the runtime will let your exception through — but you've effectively suppressed the close signal. - In CPython, generators are closed deterministically when refcount drops to zero. PyPy and other runtimes don't make this guarantee — don't rely on GC-driven cleanup.

What NOT to say: - "It's just a regular exception." — it's not; it inherits from `BaseException` and has special protocol meaning. - "I always wrap `yield` in `try/except Exception`." — that won't catch `GeneratorExit`. Use `try/finally` for cleanup, not `except`.

Common follow-up: *"What happens if a generator holds an open DB connection and the consumer breaks out of the loop mid-iteration — does the connection close?"*

Stand-out line: *"`GeneratorExit` is the protocol's way of saying 'we're done — finalise.' That's why it sits outside the `Exception` hierarchy — a blanket exception handler shouldn't be able to suppress a shutdown signal."*

Q35 · Coroutine-flavored generators — the pre- `async/await` pattern

Tags: Difficulty: Hard · Asked at: JP Morgan, Postman, Razorpay, Goldman Sachs, Walmart Labs · Topic: Historical coroutines

The question: *"Before `async/await`, how did Python handle coroutines using generators? What does that code look like, and where do you still see it today?"*

The 30-second answer: PEP 342 (2005) added `send`, `throw`, and `close` to generators, turning them into bidirectional coroutines. PEP 380 (2014) added `yield from` for delegation. Pre-asyncio frameworks like Tornado and early Twisted used generator-coroutines with a decorator like `@coroutine` that auto-primed and ran them through a custom event loop. Modern code uses `async def` — but the underlying protocol is still generators.

Step-by-step thought process: 1. "PEP 342 gave generators the three resume verbs — `send`, `throw`, `close`. That's the moment a generator stopped being a one-way stream and became a coroutine." 2. "PEP 380 added `yield from <gen>` — delegation. The outer generator transparently passes `send/throw/close` to the inner one and re-yields its values." 3. "Pre-asyncio frameworks used this. Tornado's `gen.coroutine` and asyncio's old `@asyncio.coroutine` decorator both worked this way." 4. "Python 3.5 added `async def` and `await` as proper syntax. PEP 492. Under the hood: still a generator, but the runtime now distinguishes 'coroutine objects' from generators with a `CO_COROUTINE` flag — and using them with bare `yield` is a `TypeError`." 5. "Why this matters in interviews: when a panel asks 'how does `await` work', the answer is 'it's `yield from` under a different name, with type-checking on top.'"

Code:

```

# PEP 380 – yield from for delegation
def reader():
    while True:
        line = yield
        if line == "STOP":
            return "done"
        print("read:", line)

def driver():
    result = yield from reader()    # delegates send/throw/close
    print("driver got:", result)

g = driver()
next(g)
g.send("hello")                    # read: hello
g.send("world")                    # read: world
try:
    g.send("STOP")                  # driver got: done
except StopIteration:
    pass

# Pre-asyncio style – a custom event loop running generator coroutines
import collections

def task1():
    print("task1: step 1")
    yield                           # yield to scheduler
    print("task1: step 2")
    yield

def task2():
    print("task2: step 1")
    yield
    print("task2: step 2")

def run(tasks):
    ready = collections.deque(tasks)
    while ready:
        t = ready.popleft()
        try:
            next(t)
            ready.append(t)          # not done – requeue
        except StopIteration:
            pass

run([task1(), task2()])
# task1: step 1 / task2: step 1 / task1: step 2 / task2: step 2

# Modern equivalent – async/await (Python 3.5+)
# async def task(): ...; asyncio.run(...)

```

Gotchas: - `@asyncio.coroutine` decorator was removed in Python 3.11. Old codebases using it must migrate to `async def`. - `yield from gen` and `await coro` are similar but not interchangeable — `await` requires a coroutine object (or awaitable), not a plain generator. - Tornado's `gen.coroutine` style still exists in legacy code at large Indian banks and trading firms — knowing the pattern helps when reading old code.

What NOT to say: - "Generators and coroutines are the same." — they share machinery but `async def` produces objects with a `CO_COROUTINE` flag and stricter type checks. - "Use `asyncio` for everything." — true going forward, but the question is asking about pre-asyncio history, which still informs reading legacy code.

Common follow-up: "What's the difference between `yield from gen` and `await coro` — and what changed in Python 3.5?"

Stand-out line: " `async def` is the cosmetic surface. Underneath, it's still PEP 342 + 380 — a generator with `send`, `throw`, `close`, and `yield from`. Knowing the history is what lets you reason about why a hung coroutine looks the way it does in a stack trace. "

Q36 · `itertools.chain` — flattening multiple iterables

Tags: Difficulty: Easy · Asked at: Swiggy, Flipkart, Walmart Labs, Razorpay, TCS · Topic: itertools chain

The question: "What does `itertools.chain` do, and when is it better than concatenating lists with `+`?"

The 30-second answer: `itertools.chain(a, b, c, ...)` returns an iterator that yields from `a`, then `b`, then `c`, in sequence. It does NOT materialise anything — memory is $O(1)$ regardless of how many iterables or how big they are. Use it when you'd otherwise write `list(a) + list(b) + list(c)` and don't actually need the combined list in memory.

Step-by-step thought process: 1. " `chain` is the lazy concat. `[1,2] + [3,4]` builds a new list; `chain([1,2], [3,4])` builds an iterator that walks them in order." 2. "Signature: `chain(*iterables)`. Takes any number of iterables — lists, tuples, generators, any mix." 3. "Memory: $O(1)$. Time: $O(N)$ where N is total items, same as `+`. The win is no copy step — useful when iterables are big, or when you only iterate once." 4. "Real use: combining multiple data sources into one stream — e.g. yesterday's orders + today's orders, processed once for a report." 5. "For Indian backend codebases: chain shows up a lot in pagination — chaining page iterators into one continuous stream so the downstream code doesn't have to know about page boundaries."

Code:


```

from itertools import chain

# Multiple lists into one stream
a = [1, 2, 3]
b = [4, 5, 6]
c = [7, 8]
print(list(chain(a, b, c)))          # [1, 2, 3, 4, 5, 6, 7, 8]

# Mix lists, tuples, generators
def gen():
    yield 100
    yield 200

print(list(chain([1, 2], (3, 4), gen()))) # [1, 2, 3, 4, 100, 200]

# Real-world: process pages from a paginated API as one stream
def page(n):
    return [f"order-{n}-{i}" for i in range(3)]

pages = (page(n) for n in range(1, 4)) # generator of pages
for order in chain.from_iterable(pages): # see Q37 – handles unknown count
    print(order)
# order-1-0 / order-1-1 / order-1-2 / order-2-0 / order-2-1 / ... order-3-2

# Memory comparison
import sys
big = list(range(1_000_000))
concat = big + big                    # allocates a 2M list
chained = chain(big, big)             # allocates a tiny iterator
print(sys.getsizeof(concat))          # ~8 MB
print(sys.getsizeof(chained))         # ~100 bytes (just iterator state)

```

Gotchas: - `chain` exhausts iterables in order — once you've started consuming, you can't restart any of them. Wrap in `itertools.tee` or rebuild generators if you need multiple passes. - Passing a single iterable-of-iterables doesn't unpack — `chain([[1,2], [3,4]])` yields the two inner lists, not their contents. Use `chain.from_iterable` for that (see Q37). - `chain` does NOT do interleaving. It's sequential concat. For interleave, use `itertools.zip_longest` + flatten or `roundrobin` recipe from the docs.

What NOT to say: - "Just use `+`." — works for lists but copies. Hidden cost in big-data pipelines. - "It's slower than `+`." — chain has slightly more per-item overhead, but on million-item lists the copy cost dominates and chain wins.

Common follow-up: "What's the difference between `chain(iterables)` and `chain.from_iterable(iterable_of_iterables)` — when do you need each?"

Stand-out line: "`chain` is the operator-style concat for iterators — same semantics as `+` on lists, but with $O(1)$ memory because no copy ever happens. The right answer when downstream is going to iterate once anyway."

Q37 · `itertools.chain.from_iterable` — when to prefer

Tags: Difficulty: Medium · Asked at: Postman, Razorpay, Walmart Labs, Flipkart, JP Morgan · Topic: `itertools chain.from_iterable`

The question: "What does `chain.from_iterable` do that plain `chain(*iterable)` does not — and when must you use it?"

The 30-second answer: `chain.from_iterable(iter_of_iters)` takes a single argument — an iterable that itself yields iterables — and flattens it lazily. `chain(*iter_of_iters)` unpacks the argument *eagerly* into positional args, which materialises the outer iterable in memory. Use `from_iterable` when the outer iterable is itself a generator, very large, or infinite — because unpacking would defeat the purpose.

Step-by-step thought process: 1. "`chain(*xs)` forces `xs` to be fully consumed at call time so Python knows how many args to pass. If `xs` is a generator yielding 10 million pages, you've just materialised them all." 2. "`chain.from_iterable(xs)` takes `xs` directly and walks it one outer-item at a time. Both outer and inner stay lazy." 3. "Signature: `chain.from_iterable(iterable)` — exactly one arg. The arg is the iterable-of-iterables." 4. "Use it for: streaming paginated API results, flattening a generator of database batches, walking a tree-shaped iterable lazily." 5. "This is the version you reach for in production. `chain(*...)` is fine for fixed argument lists; `from_iterable` is the streaming version."

Code:

```

from itertools import chain

# chain(*xs) — eager unpacking
def page_gen():
    for n in range(3):
        print(f" generating page {n}")
        yield [f"item-{n}-{i}" for i in range(2)]

print("chain(*page_gen()):")
result = list(chain(*page_gen()))          # all pages generated BEFORE chain runs
# generating page 0 / generating page 1 / generating page 2 — all upfront
print(result)

print("\nchain.from_iterable(page_gen()):")
for item in chain.from_iterable(page_gen()):
    print(" got:", item)
# generates page 0, yields its items, THEN generates page 1, etc — streaming

# The killer case — an infinite outer
from itertools import count

def infinite_pages():
    for n in count(0):
        yield [f"item-{n}-{i}" for i in range(3)]

# chain.from_iterable handles this; chain(*...) would hang forever
from itertools import islice
first_10 = list(islice(chain.from_iterable(infinite_pages()), 10))
print(first_10)                          # 10 items across pages 0-3

# Flattening a list of lists (also works fine)
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(list(chain.from_iterable(matrix)))  # [1, 2, 3, 4, 5, 6, 7, 8, 9]

```

Gotchas: - `chain(*iter_of_iters)` on a generator silently materialises the outer iterable — no error, but you've broken laziness. Easy to miss in code review. - `chain.from_iterable` only flattens ONE level. For deeply nested structures you need recursive flattening or `more-itertools.collapse`. - The inner iterables can be any iterable type — lists, tuples, generators — they don't all have to match.

What NOT to say: - "They're the same thing." — they're not. The argument-passing mechanism is fundamentally different (unpacking vs single iterable). - "I'd use a list comprehension." — `[x for sub in xs for x in sub]` works but is eager. `from_iterable` is the lazy form, which is what the question is really asking about.

Common follow-up: "What would happen if you wrote `chain(*infinite_generator())` — and why does `chain.from_iterable` handle it?"

Stand-out line: " `chain.from_iterable` is the streaming flatten — the outer iterable stays lazy because there's no `*` unpacking to force it. That's what makes it the right tool for paginated APIs and any generator-of-generators pipeline."

Q38 · `itertools.groupby` — must pre-sort, common surprises

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Cred, JP Morgan · Topic: itertools groupby

The question: "How does `itertools.groupby` work — what's the most common mistake people make with it?"

The 30-second answer: `groupby(iterable, key=fn)` walks the iterable and yields `(key_value, group_iterator)` pairs whenever the key changes from one item to the next. It does NOT sort first — it only groups *consecutive runs* of the same key. The most common mistake is forgetting to sort by the same key first; without sorting, you get fragmented groups instead of one group per key.

Step-by-step thought process: 1. " `groupby` is the SQL `GROUP BY` only if your data is pre-sorted by the key. Otherwise it's the SQL equivalent of grouping by sequence position." 2. "It's a single-pass operator — walks the iterable once, emits a new `(key, group)` pair whenever the key value changes." 3. "The group is an iterator, not a list. It's tied to the underlying iterator, so consuming the NEXT group invalidates the previous one. Materialise via `list(group)` before moving on." 4. "Pre-sort with the same key: `groupby(sorted(items, key=fn), key=fn)`. This is the canonical `GROUP BY` pattern." 5. "Real use: rolling up sorted log lines by user, rolling up sorted orders by city, computing run-length encoding."

Code:

```

from itertools import groupby

# WRONG – unsorted input gives fragmented groups
orders = [
    ("alice", 100), ("bob", 200), ("alice", 150),
    ("carol", 50), ("bob", 300),
]
for user, group in groupby(orders, key=lambda x: x[0]):
    print(user, list(group))
# alice [('alice', 100)]
# bob [('bob', 200)]
# alice [('alice', 150)]      – alice split into two groups
# carol [('carol', 50)]
# bob [('bob', 300)]         – bob also split

# RIGHT – sort first
orders_sorted = sorted(orders, key=lambda x: x[0])
for user, group in groupby(orders_sorted, key=lambda x: x[0]):
    print(user, list(group))
# alice [('alice', 100), ('alice', 150)]
# bob [('bob', 200), ('bob', 300)]
# carol [('carol', 50)]

# Group iterator gotcha – must materialise before moving on
groups = []
for k, g in groupby([1, 1, 2, 2, 3], key=lambda x: x):
    groups.append((k, g))      # storing the LIVE iterator

print([(k, list(g)) for k, g in groups])
# [(1, []), (2, []), (3, [])] – all empty! Iterators were exhausted by groupby's advance.

# Right way – materialise inside the loop
groups = []
for k, g in groupby([1, 1, 2, 2, 3], key=lambda x: x):
    groups.append((k, list(g))) # materialise now
print(groups)
# [(1, [1, 1]), (2, [2, 2]), (3, [3])]

# Real use – run-length encoding
def rle(s):
    return [(ch, sum(1 for _ in g)) for ch, g in groupby(s)]

print(rle("aaabbbccdaa"))      # [('a', 3), ('b', 3), ('c', 2), ('d', 1), ('a', 2)]

```

Gotchas: - The biggest one: forgetting to sort. If your data isn't already sorted by the group key, you get one entry per consecutive run, not one per key. - The group iterator is invalidated when you advance to the next group. Always materialise (`list(g)`) before storing or moving on. - `sorted` is $O(N \log N)$; if your data source is already sorted (e.g. from a SQL `ORDER BY`), skip the sort and save the cost.

What NOT to say: - "It groups by key like SQL GROUP BY." — only after sorting. Without sorting, it's groupby-consecutive-runs, which is different. - "I'd use a `defaultdict` instead." — defensible for

unsorted data, but skirts the question. Answer `groupby` first, then mention `defaultdict` as the unsorted-data alternative.

Common follow-up: "You need to group 100 million pre-sorted log lines by user — should you use `groupby` or `defaultdict`, and why?"

Stand-out line: " `groupby` is groupby-consecutive-runs, not groupby-key. That single fact explains 90% of the bugs people hit with it — and `sorted(..., key=fn) + groupby(..., key=fn)` is the SQL-style pattern you almost always actually want."

Q39 · `itertools.tee` — memory cost, the 2-consumer pattern

Tags: Difficulty: Hard · Asked at: Postman, Razorpay, JP Morgan, Walmart Labs, Cred · Topic: `itertools` `tee`

The question: "What does `itertools.tee` do, and what's the memory cost? When would you actually use it?"

The 30-second answer: `tee(iterable, n=2)` returns `n` independent iterators that each yield the items of the original iterable. Internally, `tee` buffers items the slower iterator hasn't seen yet — so if one consumer races ahead, the buffer grows to hold the gap. Use it when you genuinely need two passes over a one-shot source (generator, file) and you're consuming both in lockstep; if one consumer runs to completion before the other starts, `list()` is cheaper.

Step-by-step thought process: 1. "Signature: `tee(iter, n=2)` returns a tuple of `n` iterators. They share an underlying buffer." 2. "Internally: a deque per iterator. When one calls `next()`, the result is appended to every OTHER iterator's deque, then popped from its own." 3. "Memory cost: $O(\text{gap})$, where $\text{gap} = \text{max difference in position between the fastest and slowest consumer}$. Lockstep consumers = $O(1)$; race-ahead patterns = $O(N)$." 4. "After calling `tee`, do NOT use the original iterator. The behaviour is undefined — `tee` assumes ownership." 5. "Real use: pair-up windows over a generator (one iterator one step ahead), branching a stream into multiple parallel processors."

Code:

```

from itertools import tee

# Basic: two independent passes over one generator
def gen():
    for i in range(5):
        yield i * 10

a, b = tee(gen(), 2)
print(list(a))           # [0, 10, 20, 30, 40]
print(list(b))           # [0, 10, 20, 30, 40] – independent

# Lockstep pairwise – the canonical use (Python 3.10+ has itertools.pairwise)
def pairwise(iterable):
    a, b = tee(iterable)
    next(b, None)         # advance b by one
    return zip(a, b)

print(list(pairwise([1, 2, 3, 4, 5]))) # [(1, 2), (2, 3), (3, 4), (4, 5)]

# Memory blow-up – race-ahead consumer
import sys
from itertools import tee

a, b = tee(range(1_000_000), 2)
list(a)                  # consume a FULLY first
print("buffer roughly", sys.getsizeof(b)) # buffer is now huge – holds all 1M items for b

# Better pattern for sequential use – just convert to list once
data = list(range(1_000_000))
# two passes over the SAME list – no buffering, no tee
for x in data: pass
for x in data: pass

# Forbidden – using the source after tee
src = iter([1, 2, 3])
a, b = tee(src, 2)
next(src)                # DON'T – undefined behaviour

```

Gotchas: - If one tee consumer races ahead by N items, the buffer for the slower consumer holds all N. On a 100M-item generator with skewed consumers, this can OOM the process. - After `tee(src)`, the original iterator must not be used. tee takes ownership. - For two passes over a list (already in memory), tee is pure overhead. Just iterate the list twice. tee earns its keep only when the source is a one-shot iterator.

What NOT to say: - "It's free." — it's not. Buffer cost can be enormous with race-ahead consumers. - "Use it to iterate a list twice." — wasteful. tee is for one-shot sources (generators, files), not lists.

Common follow-up: "What's the memory cost of `tee` if one consumer runs to completion before the other starts — and what's the alternative?"

Stand-out line: " `tee` is the only way to fork a one-shot iterator into multiple independent passes — but the cost is a buffer sized to the gap between the fastest and slowest consumer. If your consumers aren't roughly in lockstep, just `list()` the source and iterate twice."

Q40 · `itertools.accumulate` — running totals, custom binary op

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Walmart Labs, Postman, JP Morgan · Topic: `itertools.accumulate`

The question: "What does `itertools.accumulate` do, and how do you use it with a custom binary operator?"

The 30-second answer: `accumulate(iterable)` yields the running cumulative sum: `a, a+b, a+b+c, ...`. With `func=` you can supply any 2-arg function — `operator.mul` for running products, `max` for running maximums, a custom lambda for any fold. With `initial=`, you can supply a starting value (Python 3.8+). It's the lazy `reduce` that emits every intermediate result.

Step-by-step thought process: 1. "Signature: `accumulate(iterable, func=operator.add, *, initial=None)`. Returns an iterator the same length as input (or +1 if `initial` is given)." 2. "Default `func` is `operator.add` — running sum. Swap for `operator.mul` for running product, `max` for running max, etc." 3. "`func(prev_accum, current_item)` — same signature as `reduce`. The output of one call becomes the first arg of the next." 4. "`initial=0` (Python 3.8+) seeds the accumulator and adds one more item to the output. Useful when you want to start from a known baseline." 5. "Real use: running balance in a ledger, cumulative max in a stock price stream, running OR / AND in a permissions chain."

Code:


```

from itertools import accumulate
import operator

# Default – running sum
print(list(accumulate([1, 2, 3, 4, 5])))          # [1, 3, 6, 10, 15]

# Running product
print(list(accumulate([1, 2, 3, 4, 5], operator.mul))) # [1, 2, 6, 24, 120]

# Running max – useful for stock-price tracking
prices = [100, 95, 110, 105, 120, 115]
print(list(accumulate(prices, max)))              # [100, 100, 110, 110, 120, 120]

# Custom – running balance on a ledger
transactions = [500, -200, 100, -50, 300]
print(list(accumulate(transactions)))              # [500, 300, 400, 350, 650]

# With initial – start from an opening balance
print(list(accumulate(transactions, initial=1000))) # [1000, 1500, 1300, 1400, 1350, 1650]

# Custom lambda – apply a discount cap each step
def cap_at(prev, curr):
    return min(prev + curr, 1000)

bonuses = [200, 400, 500, 300]
print(list(accumulate(bonuses, cap_at)))           # [200, 600, 1000, 1000] – capped

# Real-world: cumulative customer LTV from a stream of order amounts
orders = [500, 700, 300, 1200, 400]
ltv = list(accumulate(orders))
print(ltv)                                         # [500, 1200, 1500, 2700, 3100]

```

Gotchas: - `accumulate` emits the same number of items as the input. `reduce` returns only the final value. Pick `accumulate` when you want every intermediate; pick `reduce` when you only want the end. - `initial=` requires Python 3.8+. On older versions, prepend the value manually: `accumulate(chain([init], items), func)`. - The function must take exactly 2 args and return the new accumulator value. Functions with extra optional args may need a lambda wrapper.

What NOT to say: - "It's the same as `reduce`." — `reduce` returns one value; `accumulate` returns the full running sequence. Different use cases. - "I'd write a manual loop." — works, but `accumulate` is the standard-library idiom that an interviewer specifically wants to hear named.

Common follow-up: "How would you compute a running 'high water mark' over a stream of values — and what does `initial=` add?"

Stand-out line: " `accumulate` is the lazy fold that yields every intermediate state — `reduce` collapses to a single value, `accumulate` keeps the whole running curve. Swap the binary op and you get running sum, product, max, or any custom rollout."

End of sub-chunk 4 of 6 — Bidirectional generators + itertools Part A (Q31–Q40). Next sub-chunk: *itertools Part B (Q41–Q50)*.

Chapter 3 — Generators + Iterators · Sub-chunk 5 of 6 · Itertools Deep (Part B) + Iterator Pitfalls (Part A) (Q41–Q50)

Python Interview Prep — 2026 Edition · Learn & Excel · File 04 Core Python Library

This sub-chunk goes deeper into `itertools` — `pairwise`, the `takewhile` / `dropwhile` pair, `zip_longest`, the combinatorial trio (`product` / `permutations` / `combinations`), and `islice` — and then opens the iterator-pitfalls section with the five exhaustion-and-one-shot bugs that ship to production every quarter. Razorpay panels for backend, Swiggy for pipeline reasoning, Cred for ledger replays, Postman for stream parsers, Walmart Labs for catalogue jobs, and Service MNCs (TCS / Infosys / Wipro) for screening all probe these. Most candidates can name `itertools.product`; far fewer can explain why `for x in z; for x in z` gives an empty second pass when `z = zip(a, b)`. That gap — knowing the protocol *enough* to predict the bug before it hits prod — is what these ten Qs train for.

Each Q uses the same shape: **30-second answer**, **thought process**, a **code** block (compiles on Python 3.12+), **gotchas**, **what NOT to say**, **common follow-up**, and a **stand-out line**.

Q41 · `itertools.pairwise` — sliding window of two

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Cred, Postman, Walmart Labs · Topic: `itertools.pairwise`

The question: "What does `itertools.pairwise` do, when was it added, and what would you write before Python 3.10 to achieve the same thing?"

The 30-second answer: `pairwise(iterable)` yields successive overlapping pairs — `(a, b)`, `(b, c)`, `(c, d)`, ... — one item shorter than the input. It was added in Python 3.10 (no backport in `itertools` for earlier versions). Pre-3.10 you'd hand-roll it with `tee` and `next` or use a `zip(iterable, iterable[1:])` shortcut on sequences.

Step-by-step thought process: 1. "Sliding-window-of-2 is the single most common windowing need — diffs between consecutive samples, change detection, edge-pair iteration." 2. "`pairwise([1, 2, 3, 4])` produces `(1, 2)`, `(2, 3)`, `(3, 4)`. Always one fewer pair than input length." 3. "Empty or single-element input produces zero pairs — no error, just an empty iterator." 4. "Pre-3.10 hand-roll: `a, b = tee(it); next(b, None); zip(a, b)`. Cleaner: `zip(seq, seq[1:])` on a list." 5. "For an

N-window (not just 2), there's no built-in — use `collections.deque(maxlen=N)` or stitch with `tee` + multiple `next` calls.

Code:

```
from itertools import pairwise

# Detect order-amount swings between consecutive payments
amounts = [500, 700, 700, 1200, 900]
swings = [b - a for a, b in pairwise(amounts)]
print(swings)                                # [200, 0, 500, -300]

# Edge cases — empty and single-element
print(list(pairwise([])))                    # []
print(list(pairwise([99])))                 # []
print(list(pairwise([1, 2])))               # [(1, 2)]

# Pre-3.10 equivalent using tee
from itertools import tee
def pairwise_old(iterable):
    a, b = tee(iterable)
    next(b, None)                            # advance b by one
    return zip(a, b)

print(list(pairwise_old([1, 2, 3, 4])))      # [(1, 2), (2, 3), (3, 4)]

# Sequence-only shortcut (works on lists/tuples, not generators)
seq = [1, 2, 3, 4]
print(list(zip(seq, seq[1:])))               # [(1, 2), (2, 3), (3, 4)]
```

Gotchas: - The `zip(seq, seq[1:])` shortcut requires a sequence that supports slicing — fails for generators, file objects, `map` / `filter` results. - `pairwise` returns an iterator, not a list — wrap in `list()` if you need to traverse twice. - One-shot: once consumed, calling `next()` again raises `StopIteration`. Re-create from the source iterable for another pass.

What NOT to say: - "I'd write a `for` loop with `i in range(len(xs)-1)` ." — works but ignores the idiomatic answer and signals you've not opened `itertools` in a while. - "It's the same as `zip(xs, xs[1:])` ." — only true for sequences. `pairwise` works on any iterable, including generators.

Common follow-up: "How would you extend `pairwise` to a sliding window of size `N` — what's the cleanest implementation?"

Stand-out line: " `pairwise` is the small-but-correct primitive — once you stop hand-rolling `zip(seq, seq[1:])` and reach for `pairwise` , your diff-and-delta code works on generators and file streams without copying anything."

Q42 · `itertools takewhile` / `dropwhile` — when each, lazy semantics

Tags: Difficulty: Medium · Asked at: Cred, Swiggy, Razorpay, JP Morgan, Postman · Topic: itertools predicates

The question: "What's the difference between `takewhile` and `dropwhile`, and why do they short-circuit instead of running the predicate on every element?"

The 30-second answer: `takewhile(pred, it)` yields items from `it` as long as `pred(item)` is true and stops at the first false — even if later items would satisfy the predicate. `dropwhile(pred, it)` is the mirror — it skips items while `pred(item)` is true and then yields the rest, **including items where `pred` would have been false again**. Both run the predicate once per element until the boundary is found, then stop calling it.

Step-by-step thought process: 1. "Both take a predicate and an iterable. They split the iterable at the first boundary where the predicate flips." 2. " `takewhile` is 'eat until full' — yields a prefix. `dropwhile` is 'skip the warm-up' — yields a suffix." 3. "The key bit: once the boundary is found, the predicate is never called again. `dropwhile` doesn't re-check — it just keeps yielding." 4. "Lazy semantics: both are iterators. `dropwhile(lambda x: x < 100, count())` doesn't materialise the dropped prefix — it streams." 5. "Filter vs `takewhile` distinction: `filter(pred, ...)` runs predicate on every element forever; `takewhile` stops at the first false. Different shapes, different use cases."

Code:

```
from itertools import takewhile, dropwhile, count

# takewhile — read failed payments from a sorted-by-status log
events = [("FAILED", 1), ("FAILED", 2), ("SUCCESS", 3), ("FAILED", 4)]
failed_prefix = list(takewhile(lambda e: e[0] == "FAILED", events))
print(failed_prefix)                # [('FAILED', 1), ('FAILED', 2)] — '4' NOT included

# dropwhile — skip leading comments in a config stream
lines = ["# comment", "# header", "key=value", "# trailing", "other=v"]
config = list(dropwhile(lambda l: l.startswith("#"), lines))
print(config)                       # ['key=value', '# trailing', 'other=v']
                                    # NOTE: trailing '# trailing' is NOT re-filtered

# Compare to filter — runs predicate on every element
filtered = list(filter(lambda l: not l.startswith("#"), lines))
print(filtered)                     # ['key=value', 'other=v'] — different result

# Lazy — works on infinite iterators
first_under_100 = list(takewhile(lambda n: n < 100, count(95)))
print(first_under_100)              # [95, 96, 97, 98, 99] — stops cleanly
```

Gotchas: - `dropwhile` does NOT re-check the predicate after the first false — items matching the predicate AFTER that point still get yielded. - `takewhile` consumes (and discards) the boundary item from the underlying iterator — if you `tee` first and then `takewhile` one branch, the other branch is

unaffected. - Both eat the iterator once. Wrap in `list()` if you need to inspect both `taken` and `dropped` halves separately.

What NOT to say: - "`takewhile` is the same as `filter`." — wrong; `filter` checks every element, `takewhile` stops at the first false. - "`dropwhile` filters out everything matching the predicate." — wrong; it only drops the prefix, then yields everything including items that match the predicate.

Common follow-up: "How would you use `dropwhile` and `takewhile` together to extract a middle slice of an iterator where the boundaries are predicate-defined?"

Stand-out line: "`takewhile` and `dropwhile` are the prefix-and-suffix splitter — they cut the stream at one boundary and don't look back. That one-shot semantics is what makes them composable on infinite iterators where `filter` would never terminate."

Q43 · `itertools.zip_longest` — vs `zip`, `fillvalue`

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Postman, TCS, Swiggy · Topic: `zip_longest` semantics

The question: "How is `itertools.zip_longest` different from the built-in `zip`, and when would you reach for it in a real pipeline?"

The 30-second answer: Plain `zip(a, b, c)` stops at the shortest input — extra items in longer iterables are silently dropped. `zip_longest(a, b, c, fillvalue=X)` continues until the longest input is exhausted, filling missing positions with `fillvalue` (default `None`). You reach for it when "padding to align" is the correct semantics — pairing two ledgers of different lengths, joining a header row with a longer data file, or zipping a finite list against an infinite generator that you want to truncate explicitly.

Step-by-step thought process: 1. "`zip` is shortest-input. That's its biggest footgun — extra items vanish without warning." 2. "`zip_longest` is longest-input. Missing positions get `fillvalue` (default `None`)." 3. "`fillvalue` is a single value — applied to every missing slot across all iterables. There's no per-iterable fill — if you need that, use a sentinel and post-process." 4. "Use it when you want every item from every iterable visible. Drop to `zip` only when you genuinely want shortest-wins semantics." 5. "In pandas/numpy land you'd reach for `pd.concat` or `np.pad`; in pure stdlib, `zip_longest` is the answer."

Code:

```

from itertools import zip_longest

# Two ledgers of different lengths – zip silently drops the extras
payments = [500, 700, 1200, 900]
refunds = [0, 100]                                # only 2 entries

# Plain zip – silent loss
print(list(zip(payments, refunds)))                # [(500, 0), (700, 100)] – 1200 and 900 vanished

# zip_longest – pads with default None
print(list(zip_longest(payments, refunds)))
# [(500, 0), (700, 100), (1200, None), (900, None)]

# Custom fillvalue – useful for numeric maths
totals = [a - b for a, b in zip_longest(payments, refunds, fillvalue=0)]
print(totals)                                     # [500, 600, 1200, 900]

# Three-way zip_longest
a, b, c = [1, 2, 3], ["x"], [10, 20]
print(list(zip_longest(a, b, c, fillvalue="-")))
# [(1, 'x', 10), (2, '-', 20), (3, '-', '-')]

# Chunking pattern – group an iterable into fixed-size tuples
def chunked(iterable, n, fill=None):
    iters = [iter(iterable)] * n                    # SAME iterator referenced n times
    return zip_longest(*iters, fillvalue=fill)

print(list(chunked("abcdefg", 3, fill="_")))
# [('a', 'b', 'c'), ('d', 'e', 'f'), ('g', '_', '_')]

```

Gotchas: - The `[iter(iterable)] * n` trick is a famous idiom — it creates N references to the SAME iterator, so each `zip_longest` round advances it n times. Misreading this as "n separate iterators" is a classic bug. - `fillvalue` must be a single value — there's no per-iterable fill. Wrap in a `(value, source)` tuple if you need to distinguish padded from real. - `zip_longest` returns an iterator. Wrap in `list()` for multi-pass needs.

What NOT to say: - "`zip` and `zip_longest` are the same." — they have opposite termination rules; using the wrong one is a silent data-loss bug. - "I'll just use `zip` and pad manually." — works but error-prone; the `stdlib` already did this for you.

Common follow-up: "What's wrong with `[iter(xs)] * n` — and why does it work for `zip_longest` chunking?"

Stand-out line: "`zip` is silent truncation, `zip_longest` is loud padding. The one you choose is a declaration about which iterable's length is the source of truth — pick wrong and your nightly batch silently drops rows."

Q44 · `itertools.product`, `permutations`, `combinations` — the combinatorial trio

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy, Razorpay, JP Morgan, Walmart Labs · Topic: Combinatorics in itertools

The question: "Explain `product`, `permutations`, and `combinations` — when do you use each, and how do their output sizes differ?"

The 30-second answer: `product(*iters, repeat=N)` is the Cartesian product — every combination of one item from each iterable, with `repeat=N` simulating N nested loops over the same iterable. `permutations(it, r)` is ordered r-tuples without replacement — order matters. `combinations(it, r)` is unordered r-tuples without replacement — order doesn't matter. Sizes: product is the product of input lengths; permutations is $n! / (n-r)!$; combinations is $n! / (r! (n-r)!)$.

Step-by-step thought process: 1. "Think 'three nested for loops' versus 'pick r ordered' versus 'pick r unordered'." 2. "`product([1,2], ['a','b'])` → `(1,'a'), (1,'b'), (2,'a'), (2,'b')`. Cartesian — every cross." 3. "`product(range(3), repeat=2)` is the same as two nested `for i in range(3): for j in range(3):` — 9 pairs." 4. "`permutations('ABC', 2)` → `('A','B'), ('A','C'), ('B','A'), ('B','C'), ('C','A'), ('C','B')`. Order matters; no repeats." 5. "`combinations('ABC', 2)` → `('A','B'), ('A','C'), ('B','C')`. Order doesn't matter; lexicographic by input order." 6. "There's also `combinations_with_replacement` for unordered with-repeats — less common but real."

Code:

```

from itertools import product, permutations, combinations, combinations_with_replacement

# product – try every shipping option × every payment method
shipping = ["std", "express"]
payment = ["upi", "card", "cod"]
print(list(product(shipping, payment)))
# [('std', 'upi'), ('std', 'card'), ('std', 'cod'), ('express', 'upi'), ('express', 'card'), ('express', 'cod')]

# product with repeat – generate all 4-digit OTPs
otps = list(product("0123456789", repeat=4))
print(len(otps)) # 10_000

# permutations – first 3 finishers in a race (order matters)
runners = ["alice", "bob", "carol", "dave"]
print(list(permutations(runners, 3))[:3])
# [('alice', 'bob', 'carol'), ('alice', 'bob', 'dave'), ('alice', 'carol', 'bob')]
print(len(list(permutations(runners, 3)))) # 24 = 4! / 1!

# combinations – pick a team of 2 (order doesn't matter)
print(list(combinations(runners, 2)))
# [('alice', 'bob'), ('alice', 'carol'), ('alice', 'dave'), ('bob', 'carol'), ('bob', 'dave'), ('carol', 'dave')]
print(len(list(combinations(runners, 2)))) # 6 = 4! / (2! 2!)

# combinations_with_replacement – pick 2 with repeats allowed
print(list(combinations_with_replacement("AB", 2)))
# [('A', 'A'), ('A', 'B'), ('B', 'B')]

```

Gotchas: - All three are iterators — `len(combinations(...))` requires materialising via `list()` or using `math.comb(n, r)` for the count. - Output sizes grow combinatorially — `permutations(range(15))` is ~1.3 trillion tuples; never `list()` a large one. - Input is consumed once (it's the iterable, not the iterator) — internally these C-implemented helpers buffer the input to walk it multiple times.

What NOT to say: - "Permutations and combinations are the same." — they aren't; order distinguishes them and the counts differ. - "I'll write nested loops." — works for `product` with small repeats; for arbitrary `r` from variable input length, `permutations` / `combinations` is the only clean answer.

Common follow-up: "What's the difference between `combinations` and `combinations_with_replacement`, and when does the with-replacement version model reality?"

Stand-out line: "The trio splits combinatorial space three ways — `product` for Cartesian, `permutations` for ordered picks, `combinations` for unordered picks. Naming the right one in the interview signals you've used them on a real problem, not just read the docs."

Q45 · `itertools.islice` — slice an iterator without materialising

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Swiggy, Walmart Labs, Cred · Topic: islice semantics

The question: "What does `itertools.islice` do, and why can't you just write `iterator[start:stop]` like you would for a list?"

The 30-second answer: `islice(iterable, start, stop, step)` is the slice operation that works on any iterable, including generators. It advances the iterator from position 0, discarding items until `start`, then yields items until `stop`, stepping by `step`. You can't use `iterator[start:stop]` because iterators don't support `__getitem__` — they're one-shot streams with no random access. `islice` walks the iterator linearly to simulate slicing.

Step-by-step thought process: 1. "Generators and other iterators don't support indexing. `gen[3:10]` raises `TypeError`." 2. "`islice(it, 3, 10)` advances `it` past the first 3, then yields the next 7. After it returns, `it` is at position 10 — it has been consumed up to there." 3. "Signatures: `islice(it, stop)`, `islice(it, start, stop)`, `islice(it, start, stop, step)`. Negative values are NOT allowed — no `islice(it, -3, None)` for 'last 3'." 4. "To get the last N, use `collections.deque(it, maxlen=N)` — that's the streaming-aware idiom." 5. "`islice` is $O(\text{stop})$ time and $O(1)$ memory. It does NOT skip ahead — it walks every item up to `start`."

Code:

```

from itertools import islice, count

# Slice an infinite iterator – fully safe
first_5_evens = list(islice(count(0, 2), 5))
print(first_5_evens)                # [0, 2, 4, 6, 8]

# Middle slice – start, stop
items = (n * 10 for n in range(10))    # generator
middle = list(islice(items, 3, 7))
print(middle)                        # [30, 40, 50, 60]

# Step argument – every 3rd
print(list(islice(range(20), 0, 20, 3))) # [0, 3, 6, 9, 12, 15, 18]

# islice consumes the underlying iterator – items "before stop" are GONE
g = (n for n in range(10))
first_3 = list(islice(g, 3))
print(first_3)                      # [0, 1, 2]
print(list(g))                      # [3, 4, 5, 6, 7, 8, 9] – g advanced

# Negative slice does NOT work
try:
    list(islice(range(10), -3, None))
except ValueError as e:
    print(e)                        # Stop argument for islice() ... must be ... 0 <= ...

# "Last N" of a stream – use deque, not islice
from collections import deque
last_3 = list(deque(range(10), maxlen=3))
print(last_3)                       # [7, 8, 9]

```

Gotchas: - `islice` mutates the underlying iterator — items consumed by `islice` are gone from the source. This trips people coming from list-slicing where the source is untouched. - No negative indices — `islice(it, -3, None)` raises `ValueError`. The "last N" pattern needs `deque(it, maxlen=N)`. - `islice(it, None, None, step)` is the way to skip-stride a whole iterator — both bounds open.

What NOT to say: - "I'd just do `list(gen)[3:7]` ." — defeats the purpose; that materialises the entire generator, blowing memory on large streams. - "It's the same as Python slice syntax." — semantically yes, but `islice` consumes the iterator, doesn't support negative indices, and doesn't allow `:` shorthand.

Common follow-up: "How would you get the LAST 100 lines of a multi-GB log file without materialising the whole thing in memory?"

Stand-out line: " `islice` is the slice operation that doesn't materialise — it streams. Once you grasp that lists support random-access slicing and iterators don't, `islice` becomes the only correct answer for skipping a header or capping a stream."

Q46 · Iterator exhaustion — generator/zip/enumerate consumed once

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, Flipkart, Cred · Topic: Iterator exhaustion (pitfall)

The question: "You wrote `z = zip(a, b)`, iterated `z` once in a function, then tried to iterate it again — and got nothing. Walk me through why, and how you'd structure the code to avoid this."

The 30-second answer: `zip`, `map`, `filter`, `enumerate`, and generator objects are all **iterators** — single-pass, stateful, and `StopIteration` is permanent once raised. The first `for` loop consumes the entire iterator; the second sees an exhausted iterator and exits immediately. Fix it by either materialising once (`z = list(zip(a, b))`) or re-constructing fresh (`zip(a, b)` inline at each call site).

Step-by-step thought process: 1. "This is the #1 production bug from people who came from Python 2 or from JavaScript — they assume `zip` returns a list." 2. "In Python 3, all the 'lazy' wrappers return iterators, not lists. One pass and they're done." 3. "`zip(a, b)` returns a `zip` object — iterable AND iterator (its `__iter__` returns self). Stepping through it advances internal state on `a` and `b`." 4. "Once you iterate it fully, `next(z)` raises `StopIteration`. A second `for z in ...` sees `iter(z)` returning the same exhausted iterator." 5. "Fix: `list(zip(a, b))` if you need multi-pass and the data fits; `zip(a, b)` inline at each call site if you don't want to materialise."

Code:

```

# The bug – two passes over the same zip
a = [1, 2, 3]
b = ['x', 'y', 'z']
z = zip(a, b)

for pair in z:
    print(pair, end=" ")          # (1,'x') (2,'y') (3,'z')
print()

for pair in z:                   # SECOND pass – empty
    print(pair, end=" ")
print("done")                   # done – nothing printed before this

# Fix 1 – materialise once
pairs = list(zip(a, b))
for p in pairs: print(p, end=" ") # works
print()
for p in pairs: print(p, end=" ") # works again – list is multi-pass
print()

# Fix 2 – re-construct at each call site
def show():
    for pair in zip(a, b):        # fresh zip every call
        print(pair, end=" ")
    print()

show(); show()                  # two clean passes

# Same trap with generators
def amounts():
    yield 100; yield 200; yield 300

g = amounts()
print(sum(g))                   # 600
print(sum(g))                   # 0 – exhausted

# Filter, map, enumerate – all one-shot
e = enumerate(["a", "b"])
print(list(e))                  # [(0, 'a'), (1, 'b')]
print(list(e))                  # [] – exhausted

f = filter(lambda x: x > 0, [1, -1, 2])
print(list(f))                  # [1, 2]
print(list(f))                  # [] – exhausted

```

Gotchas: - The exhausted iterator is silent — no exception, just zero iterations. Linters and tests catch it; runtime won't. - `list(zip(...))` is the correct fix for "small data, multi-pass needed". For large data, prefer the inline-reconstruct pattern. - Passing a generator to a function that "might iterate twice" is a documentation gap — annotate the function or convert to list at the boundary.

What NOT to say: - "I'll reset it with `iter(z)` ." — calling `iter()` on an exhausted iterator returns the same exhausted iterator. No reset possible. - "It's a bug in Python." — it's the documented protocol. Iterators are stateful by design.

Common follow-up: *"How would you write a defensive helper that detects whether the argument is single-pass and either copies or warns?"*

Stand-out line: *"Python 3 made everything lazy by default, which is a memory win — but the cost is that every `zip`, `map`, `filter`, `enumerate`, and generator is one-shot. Either materialise at the boundary or document that the caller owns the freshness."*

Q47 · `zip` short-circuits on shortest input — silently drops extra

Tags: Difficulty: Medium · Asked at: Walmart Labs, Razorpay, Cred, TCS, Postman · Topic: zip silent truncation (pitfall)

The question: *"You zipped two lists, one with 1000 items and one with 999, and an order silently disappeared from your output. Why, and what's the right fix?"*

The 30-second answer: `zip(a, b)` stops at the shortest input. The 1000th item in the longer list is never yielded — it's dropped without warning. The fix is `zip_longest` for "keep everything, pad the gap" semantics, or `zip(a, b, strict=True)` (Python 3.10+) which raises `ValueError` if the lengths differ — turning a silent bug into a loud failure.

Step-by-step thought process: 1. "`zip` is shortest-wins by design — that's the documented behaviour." 2. "It's a silent failure mode. No exception, no log line. Your row count is just wrong by one." 3. "Python 3.10 added `strict=True` — `zip(a, b, strict=True)` raises `ValueError` if any iterable is shorter than the others at exhaustion. This is the modern fix." 4. "Pre-3.10, you'd assert `len(a) == len(b)` before zipping. Now `strict=True` is the idiom." 5. "When you genuinely want one-sided truncation, document it: `# zip stops at orders; refunds tail intentionally dropped` . Otherwise default to `strict=True` ."

Code:

```

# Silent truncation bug
orders = [1, 2, 3, 4]
refunds = [0, 0, 0] # one short
pairs = list(zip(orders, refunds))
print(pairs) # [(1,0), (2,0), (3,0)] – order 4 vanished
print(f"orders processed: {len(pairs)}") # 3 – silently wrong

# Fix 1 – strict=True (Python 3.10+)
try:
    list(zip(orders, refunds, strict=True))
except ValueError as e:
    print("loud failure:", e) # zip() argument 2 is shorter than argument 1

# Fix 2 – zip_longest, pad with 0
from itertools import zip_longest
pairs_padded = list(zip_longest(orders, refunds, fillvalue=0))
print(pairs_padded) # [(1,0), (2,0), (3,0), (4,0)] – order 4 kept

# Fix 3 – assert before zip (pre-3.10 idiom)
assert len(orders) == len(refunds), "length mismatch"

# Three-iterable case – strict checks ALL of them
a, b, c = [1, 2], [10, 20], [100]
try:
    list(zip(a, b, c, strict=True))
except ValueError as e:
    print(e) # zip() argument 3 is shorter than arguments 1-2

```

Gotchas: - `strict=True` only raises when an iterable is shorter at the END — not at intermediate length mismatches with infinite iterables. - `zip(a, b, strict=True)` with one iterable infinite (e.g. `count()`) raises only when the other ends, then `count()` is detected as longer — correct loud failure. - Existing code that relied on the silent truncation needs auditing before flipping `strict=True` on globally — some pipelines depend on the truncation.

What NOT to say: - "`zip` always pads." — wrong; the default is truncation, not padding. - "I'd just slice the longer one." — masks the bug; you still don't know that you had unequal lengths.

Common follow-up: "What's the difference between `zip(..., strict=True)` and `zip_longest(..., fillvalue=...)` — when do you reach for each?"

Stand-out line: "Silent truncation is the worst kind of bug — your output looks right, just shorter. Python 3.10's `strict=True` turns this into a loud `ValueError`, and it's the one-line change every team should make to legacy zip-heavy pipelines."

Q48 · `enumerate` is a generator (one-shot) — common confusion with list

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman, Swiggy · Topic: enumerate one-shot (pitfall)

The question: "You wrote `e = enumerate(items)` and passed `e` to two different functions that both iterated it. The second function got an empty result. What's happening?"

The 30-second answer: `enumerate` returns an iterator, not a list. The first function consumed it fully; the second saw an exhausted iterator and iterated zero times. The fix is either materialising once (`e = list(enumerate(items))`) or constructing a fresh `enumerate` at each call site.

Step-by-step thought process: 1. "`enumerate(items)` returns an `enumerate` object — iterable AND iterator. Single-pass." 2. "Same family as `zip`, `map`, `filter` — all of them return iterators in Python 3." 3. "In Python 2, `enumerate` returned a list — that's where the confusion comes from, especially in legacy-trained teams." 4. "The bug shape: pass `e` to two functions, second one sees empty. Or `for i, x in e:` followed by another loop over `e` — second loop is empty." 5. "Fix: `list(enumerate(items))` once, or `enumerate(items)` inline at the point of use. Don't bind to a name and pass around."

Code:

```

# The bug
items = ["alice", "bob", "carol"]
e = enumerate(items)

def first_pass(it):
    print("first:", list(it))           # consumes all

def second_pass(it):
    print("second:", list(it))         # gets nothing

first_pass(e)                          # first: [(0,'alice'), (1,'bob'), (2,'carol')]
second_pass(e)                         # second: []

# Fix 1 – materialise once
enum_list = list(enumerate(items))
first_pass(iter(enum_list))            # works
second_pass(iter(enum_list))           # iter() on a list is always fresh

# Fix 2 – inline at each call site
first_pass(enumerate(items))           # fresh
second_pass(enumerate(items))          # fresh

# A subtler bug – partial consumption
e = enumerate(items)
print(next(e))                         # (0, 'alice')
for i, x in e: print(i, x)              # 1 bob / 2 carol – picks up from 1
# enumerate doesn't reset to 0 on re-iteration

# enumerate with start
for i, x in enumerate(items, start=1):
    print(f"#{i}: {x}")                 # #1: alice / #2: bob / #3: carol

```

Gotchas: - `enumerate(items, start=N)` sets the starting index — useful for one-based output. - Calling `iter(e)` on an enumerate object returns the same object — no reset. You'd need a fresh `enumerate(items)`. - A teammate's quick "let me just enumerate this" inside a function that's called multiple times with the same generator is a recurring source of silent bugs.

What NOT to say: - "`enumerate` returns a list." — true in Python 2, false in Python 3. Citing this dates you. - "I'd index manually with `for i in range(len(items)): items[i]` ." — works only for sequences; fails on generators and is unidiomatic Python.

Common follow-up: "Why is `enumerate` an iterator and not a list — what's the design rationale?"

Stand-out line: "`enumerate` is a one-shot wrapper, just like `zip` and `map` — it's lazy on purpose, so it works on streams. The cost is the exhaustion footgun, and the fix is always 'materialise at the boundary or re-construct at the call site'."

Q49 · Two for-loops over the same iterator — second loop empty

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, JP Morgan, Flipkart · Topic: Iterator state (pitfall)

The question: "You have a generator `g`, and you wrote `for x in g: ...` followed by `for x in g: ...`. The second loop is empty. Why, and what are your three options to make both loops run?"

The 30-second answer: The first `for` loop exhausted the generator — `StopIteration` was raised, and the iterator is permanently done. The second `for` loop calls `iter(g)`, gets back the same exhausted iterator (because iterator's `__iter__` returns self), and exits immediately. Your three options: (1) materialise with `list(g)` so each loop iterates the list, (2) re-construct the generator (`g = gen_fn()` between loops), (3) use `itertools.tee(g, 2)` to fork into two independent iterators (with memory cost).

Step-by-step thought process: 1. "The protocol is unambiguous: iterator's `__iter__` returns self; once exhausted, stays exhausted." 2. "The bug shape — two adjacent `for` loops on the same iterator — is the canonical demonstration of this rule." 3. "Option 1: `xs = list(g)` then `for x in xs: ...; for x in xs: ...`. Simple, memory cost = full materialisation." 4. "Option 2: regenerate `g = gen_fn()` between loops. Memory cost = zero, but only works if the generator is cheaply re-runnable." 5. "Option 3: `from itertools import tee; g1, g2 = tee(g, 2)`. Forks. Both branches are independent iterators. Memory cost = whatever distance the slower branch is behind the faster one."

Code:

```

from itertools import tee

def amounts():
    for x in [100, 200, 300]: yield x

# The bug
g = amounts()
print("first:", [x for x in g])          # first: [100, 200, 300]
print("second:", [x for x in g])        # second: [] – exhausted

# Option 1 – materialise
xs = list(amounts())
print("a:", [x for x in xs])            # a: [100, 200, 300]
print("b:", [x for x in xs])            # b: [100, 200, 300] – list is multi-pass

# Option 2 – re-construct
print("a:", [x for x in amounts()])     # a: [100, 200, 300]
print("b:", [x for x in amounts()])     # b: [100, 200, 300] – fresh generator each time

# Option 3 – tee
g1, g2 = tee(amounts(), 2)
print("a:", [x for x in g1])            # a: [100, 200, 300]
print("b:", [x for x in g2])            # b: [100, 200, 300] – independent fork

# tee's hidden cost – buffer between branches
def big_stream():
    for n in range(1_000_000): yield n

a, b = tee(big_stream(), 2)
list(a)                                # consume a fully
# b now has 1M items buffered in tee's internal queue
# Memory cost: O(N) until b also consumes

```

Gotchas: - `tee` is NOT free — it buffers items not yet consumed by the slower branch. If branches diverge widely, memory blows up. - After calling `tee(g, n)`, you must NOT continue using `g` directly — `tee` consumes from it under the hood. Using both `g` and a `tee` branch causes data races. - For genuinely huge streams that you need to walk twice, the right answer is "make the source replayable" (e.g. a file path you can re-open), not `tee`.

What NOT to say: - "Call `g.reset()` ." — generators have no reset method. There's no rewind. - "Convert to list — always works." — fine for small data; for a 100GB log it's a disaster.

Common follow-up: "When is `itertools.tee` the right choice, and when is it actually the wrong choice compared to just regenerating the source?"

Stand-out line: "There are three escape hatches from the one-shot-iterator trap — materialise, regenerate, or tee — and choosing among them is a memory-vs-CPU-vs-source-replayability call you make per pipeline, not a default."

Q50 · Materialising with `list(g)` — when needed, when wasteful

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, Walmart Labs, Cred · Topic: `list()` materialisation tradeoff (pitfall)

The question: "When should you wrap a generator in `list()`, and when is doing so a mistake that defeats the whole point of using a generator?"

The 30-second answer: Materialise with `list(g)` when you need multi-pass iteration, random access by index, `len()`, slicing, or to pin the data before it changes. Don't materialise when the generator is the whole reason for using lazy semantics — large or infinite streams, pipelines where peak memory matters, or when only the first N items are needed. The rule: `list()` at the boundary where laziness stops being useful, not by reflex.

Step-by-step thought process: 1. "Generators are lazy for a reason — constant-memory streaming. `list()` collapses that to $O(N)$ memory." 2. "Reasons to materialise: need to iterate twice; need `len()`; need random access (`xs[i]`); need slicing (`xs[3:7]`); need to inspect total before processing." 3. "Reasons NOT to materialise: data is huge or infinite; you only need the first N (use `islice`); you're going to sum/count/any/all which work fine on iterators." 4. "Pipeline rule: keep things lazy as long as possible, materialise once at the consumption point. Don't `list()` between every stage — defeats the streaming win." 5. "Diagnostic: if you `list()` a generator and use only `sum(xs)` afterward, you wasted memory. If you `list()` and use `xs[0]`, `len(xs)`, and a second iteration — you needed it."

Code:

```

from itertools import islice

# When you NEED list() – multi-pass, len, indexing
events = (line.strip() for line in open("/etc/hosts"))
events_list = list(events)
print(f"count: {len(events_list)}")      # needs len
print(f"first: {events_list[0]}")       # needs indexing
for e in events_list: pass              # pass 1
for e in events_list: pass              # pass 2

# When list() is WASTEFUL – sum on a generator works
total = sum(n * n for n in range(1_000_000)) # O(1) extra memory
# Don't do: total = sum(list(n * n for n in range(1_000_000))) # O(N) memory waste

# When list() is BAD – infinite or huge
from itertools import count
# list(count()) # would OOM the process

# Correct: bound first, then materialise the slice
first_100 = list(islice(count(), 100))
print(first_100[:5])                    # [0, 1, 2, 3, 4]

# Pipeline – keep lazy, materialise at the END
def pipeline(rows):
    cleaned = (r.strip() for r in rows)
    parsed  = (r.split(",") for r in cleaned if r)
    typed   = ((int(a), b) for a, b in parsed)
    return typed                          # still a generator

# Caller decides materialisation
result = list(pipeline(open("data.csv"))) # OK – final consumer
# OR
top_10 = list(islice(pipeline(open("data.csv")), 10)) # better – only what's needed

```

Gotchas: - `sum`, `min`, `max`, `any`, `all`, `count`, and `for` loops all accept iterators — no need to `list()` first. - `list(iter)` materialises eagerly — for an exception-raising generator, the exception fires inside `list()`, not at the iteration site. Sometimes that hides where the bug is. - `tuple(g)` is the immutable cousin — same memory cost, but signals "won't mutate" to readers and works as a dict key.

What NOT to say: - "Always `list()` generators, just in case." — defeats laziness. Memory bug factory. - "Never `list()` — always stream." — wrong; sometimes you genuinely need indexing or multi-pass and materialisation is correct.

Common follow-up: "In a multi-stage pipeline, where should you put the single `list()` call — at the source, between stages, or at the consumer?"

Stand-out line: " `list()` is a boundary marker, not a habit — it says 'past this point, I need random access or multiple passes'. Sprinkling `list()` mid-pipeline turns a constant-memory stream into an N-memory copy chain, which is the opposite of what generators were meant to give you."

End of sub-chunk 5 of 6 — Itertools deep (Part B) + Iterator pitfalls (Part A) (Q41–Q50). Next sub-chunk: Iterator pitfalls (Part B) + advanced generator patterns (Q51–Q60).

Chapter 3 — Generators + Iterators · Sub-chunk 6 of 6 · Iterator Pitfalls (Part B) + Async Iterators (Q51–Q60)

Python Interview Prep — 2026 Edition · Learn & Excel · File 04 Core Python Library

This sub-chunk closes the iterator chapter with two halves. The first five Qs cover the remaining iterator pitfalls — generators inside `any()` / `all()`, `GeneratorExit` and cleanup, the PEP 479 `StopIteration` swallow, mutating-source surprises, and the file-handle iterator pattern. The last five Qs introduce the **async iterator protocol** — `__aiter__` / `__anext__`, `async for` desugaring, the 3.10+ `aiter()` / `anext()` built-ins, async generator functions, and the semantic distinction between an async generator and an async function returning an iterator. Razorpay panels for backend roles, Cred for streaming payment ledgers, Swiggy for order-pipeline questions, Postman for parsing engines, and Walmart Labs / JP Morgan for high-throughput async services all probe whether you can switch between sync and async iteration without losing the protocol underneath.

Each Q uses the same shape: **30-second answer**, **thought process**, a **code** block (compiles on Python 3.12+), **gotchas**, **what NOT to say**, **common follow-up**, **stand-out line**.

Q51 · Generator in `any()` / `all()` — short-circuit benefit + the no-side-effect rule

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Postman, Walmart Labs · Topic: Short-circuit iteration

The question: "Why is passing a generator expression to `any()` or `all()` better than a list comprehension — and what's the rule about side effects inside the generator?"

The 30-second answer: `any()` and `all()` short-circuit — `any()` stops at the first truthy value, `all()` stops at the first falsy. Passing a generator (not a list) means only the values up to the short-circuit point are produced — saves both work and memory. The rule: the generator's body must have NO observable side effects, because callers can't tell how many items were consumed before `any` / `all` bailed out.

Step-by-step thought process: 1. "`any()` and `all()` are built-in short-circuiters. They stop the moment the answer is decided." 2. "`all(expensive(x) for x in xs)` only calls `expensive()`

until the first falsy result — vs `all([expensive(x) for x in xs])` which materialises the whole list first." 3. "Memory-wise: the comprehension allocates a full list; the generator allocates a single frame. For a million-element check, the difference is huge." 4. "The trap: if `expensive(x)` has a side effect — logs, increments a counter, writes a row — your caller can't tell how many ran. Reproducibility goes out the window." 5. "Rule of thumb: generator inside `any` / `all` should be pure predicate evaluation only. Side effects belong in an explicit `for` loop."

Code:

```
# Generator — short-circuits, O(1) memory
orders = range(1_000_000)
has_big_order = any(o > 999_995 for o in orders)
print(has_big_order)                                # True — stopped early

# List comprehension — materialises all 1M booleans first
# any([o > 999_995 for o in orders])                # wasteful: builds full list then checks

# Counting calls to prove short-circuit
calls = 0
def check(x):
    global calls
    calls += 1
    return x > 5

result = any(check(x) for x in range(100))
print(result, "calls:", calls)                       # True calls: 7 — stopped at x=6

# Same with all() — stops at first False
calls = 0
def positive(x):
    global calls
    calls += 1
    return x > 0

result = all(positive(x) for x in [1, 2, 3, -1, 4, 5])
print(result, "calls:", calls)                       # False calls: 4 — stopped at -1

# Side-effect anti-pattern — DON'T do this
log = []
def log_and_check(x):
    log.append(x)                                    # SIDE EFFECT
    return x > 100

any(log_and_check(x) for x in [50, 200, 300])
print(log)                                           # [50, 200] — caller can't predict log size
```

Gotchas: - `any([])` returns `False` ; `all([])` returns `True` (vacuous truth). Trips up engineers expecting symmetric behaviour. - `all(gen)` where `gen` raises mid-stream — the exception propagates, not silently treated as `False` . Wrap in a try if you want best-effort. - Putting `print()`

inside the generator for "debugging" counts as a side effect — output will be truncated at the short-circuit point.

What NOT to say: - "List comprehension is more readable." — readable yes, but you pay full materialisation cost. The interviewer is testing whether you know the trade-off. - "It doesn't matter, they're equivalent." — they're equivalent in result, not in cost. Saying this signals you haven't profiled either.

Common follow-up: "What's the result of `any(x > 5 for x in [])` — and why does that matter for input-validation code?"

Stand-out line: " `any` and `all` with a generator are short-circuit by construction — the generator only advances as far as the boolean decision needs. Put a side effect in the generator and you've made your code's behaviour depend on the short-circuit point, which is the wrong direction for a predicate."

Q52 · Generator GC + `GeneratorExit` — closing on cleanup, holding resources

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, JP Morgan, Flipkart · Topic: Generator lifecycle

The question: "What happens when a generator is garbage-collected before it's exhausted — and how do you write a generator that cleans up an open file or connection correctly?"

The 30-second answer: When a generator is garbage-collected mid-stream, Python calls its `close()` method, which injects a `GeneratorExit` exception at the current `yield` point. You catch it (or let it propagate) using `try/finally` around the `yield` — the `finally` block is your cleanup hook. This is why `contextlib.contextmanager` and `with open(...)` inside generators work — the `finally` runs on GC, not just on natural exhaustion.

Step-by-step thought process: 1. "Generators are stateful. A half-iterated generator holds onto resources — open files, sockets, DB cursors — until something cleans up." 2. "Python's GC closes the generator automatically when its refcount hits zero. `close()` injects `GeneratorExit` at the suspended `yield`." 3. " `GeneratorExit` is NOT an Exception — it inherits from `BaseException` . Bare `except:` catches it; `except Exception:` does NOT. The right pattern is `try / yield / finally` ." 4. "Inside the `finally` , do the cleanup — close the file, release the lock, return the connection to the pool." 5. "Catching `GeneratorExit` and yielding again is a protocol violation — Python raises `RuntimeError: generator ignored GeneratorExit` ."

Code:

```

# Resource cleanup via try/finally inside generator
def read_lines(path):
    f = open(path, "r")
    try:
        for line in f:
            yield line.rstrip()
    finally:
        print(f"closing {path}")          # runs on GC, break, or exhaustion
        f.close()

# Natural exhaustion – finally runs
for line in read_lines("/etc/hosts"):
    pass                                # finally fires after last yield

# Break mid-stream – generator is GC'd, finally runs
gen = read_lines("/etc/hosts")
for i, line in enumerate(gen):
    if i >= 2:
        break                            # generator falls out of scope
del gen                                # forces GC; finally fires now
print("after del")

# Manual close() – explicit cleanup
gen = read_lines("/etc/hosts")
next(gen)
gen.close()                            # injects GeneratorExit; finally runs

# Anti-pattern – swallowing GeneratorExit
def broken_gen():
    try:
        yield 1
    except GeneratorExit:
        yield 99                        # NO – protocol violation

g = broken_gen()
next(g)
try:
    g.close()
except RuntimeError as e:
    print(e)                            # generator ignored GeneratorExit

```

Gotchas: - `GeneratorExit` inherits from `BaseException`, not `Exception` — `except Exception:` does NOT catch it. Use `try/finally` for cleanup. - Yielding inside a `finally` block of a generator is technically allowed but messy — better to do the cleanup as the last statement. - CPython's reference counting closes generators promptly; PyPy / Jython use tracing GC and may delay cleanup. Don't rely on GC timing for must-close resources — use `with contextlib.closing(gen):` if you need it deterministic.

What NOT to say: - "Generators auto-close when the function returns." — they auto-close on GC, which is not the same as natural function return. The distinction matters for `break` / `exception` / `del`. - "I'd put

cleanup after the for-`yield` loop." — wrong; if the caller breaks early, you never reach the line after the loop. The `finally` is the only safe place.

Common follow-up: "What's the difference between catching `GeneratorExit` and not catching it — and why does Python raise `RuntimeError` if you yield again after catching it?"

Stand-out line: "A generator with `try / yield / finally` is the cleanest streaming-resource pattern in Python — the `finally` runs on every exit path including GC, break, exception, and explicit `close()`. Anything you allocated before the yield gets released, no matter how the consumer leaves the loop."

Q53 · `StopIteration` swallowed in comprehension (PEP 479 — now `RuntimeError`)

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Cred, Walmart Labs, JP Morgan · Topic: PEP 479

The question: "What did PEP 479 change about `StopIteration` inside generators, and what was the silent bug it fixed?"

The 30-second answer: PEP 479 (default behaviour since Python 3.7) converts any `StopIteration` that leaks out of a generator's body into a `RuntimeError: generator raised StopIteration`. Pre-3.7, a stray `StopIteration` from an inner `next()` call would silently terminate the outer generator — producing pipelines that quietly stopped mid-stream with no error. Now you get a loud `RuntimeError` instead.

Step-by-step thought process: 1. "`StopIteration` is the iterator's sentinel. Inside a `for` loop, it's caught silently. Inside a generator body, it used to be too — that was the bug." 2. "Concrete failure: a generator calls `next(some_iter)` directly. If that iter is exhausted, `StopIteration` raised — and pre-3.7, the outer generator just stopped. No `RuntimeError`, no log, nothing." 3. "PEP 479 made this an error: any `StopIteration` propagating out of the generator's frame is now wrapped as `RuntimeError`." 4. "This also affects comprehensions inside generators — `(x for x in xs if next(it_empty))` raises `RuntimeError`, not silent termination." 5. "The fix: use `next(it, default)` with a default, or wrap in `try/except StopIteration` and convert to a clean `return` (which ends the generator naturally)."

Code:

```

# Pre-3.7 silent-stop, now RuntimeError
def filter_orders(orders, allowed):
    allowed_iter = iter(allowed)
    for order in orders:
        if next(allowed_iter) == order["id"]: # raises StopIteration when exhausted
            yield order

orders = [{"id": 1}, {"id": 2}, {"id": 3}]
allowed = [1, 2] # only 2 items
try:
    list(filter_orders(orders, allowed))
except RuntimeError as e:
    print(type(e).__name__, str(e)[:50]) # RuntimeError generator raised StopIteration

# Fix 1 – use the two-arg next()
def filter_orders_v2(orders, allowed):
    allowed_iter = iter(allowed)
    for order in orders:
        if next(allowed_iter, None) == order["id"]:
            yield order

print(list(filter_orders_v2(orders, allowed))) # [{'id': 1}, {'id': 2}]

# Fix 2 – catch and convert to return (clean exhaustion)
def filter_orders_v3(orders, allowed):
    allowed_iter = iter(allowed)
    for order in orders:
        try:
            target = next(allowed_iter)
        except StopIteration:
            return # clean generator end
        if target == order["id"]:
            yield order

print(list(filter_orders_v3(orders, allowed))) # [{'id': 1}, {'id': 2}]

# Same trap inside a generator expression
def parse_pairs(stream):
    return (x for x in stream if next(iter([]))) # StopIteration mid-comprehension

try:
    list(parse_pairs([1, 2, 3]))
except RuntimeError as e:
    print(type(e).__name__) # RuntimeError

```

Gotchas: - The wrapping happens at the generator boundary only. `StopIteration` from inside a regular function still propagates as `StopIteration`. - Async generators got the same treatment via PEP 525 — a stray `StopAsyncIteration` becomes `RuntimeError`. - Old codebases that explicitly used `raise StopIteration` to exit a generator early break under PEP 479 — replace with `return`.

What NOT to say: - "Just catch `Exception` around `next()` ." — catches the wrong things (real bugs masked). Catch `StopIteration` specifically or use the two-arg `next(it, default)` . - "PEP 479 is opt-in." — it WAS in 3.5/3.6 via `__future__` . From 3.7 it's the default. Saying opt-in dates you.

Common follow-up: "How would you rewrite a generator that uses `raise StopIteration` to terminate early, so it works under PEP 479?"

Stand-out line: "PEP 479 promoted a silent footgun to a loud `RuntimeError` — generators no longer terminate quietly when an inner iterator runs dry. If your code relied on that silent exit, you've now got an honest error pointing at where to add `next(it, default)` or a clean `return` ."

Q54 · Generator iterating over a mutating source — sees later mutations

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Postman, Walmart Labs · Topic: Lazy evaluation + mutation

The question: "If a generator is iterating over a list and the list is mutated before the generator finishes, what does the generator see — and why does this surprise people?"

The 30-second answer: Generators are lazy — they read the source each time `next()` is called, not at construction. So a generator over `xs` sees later appends to `xs` (as long as they happen before the corresponding `next()` call). For lists, the iterator tracks a position index, so appends are visible but deletions / inserts shift the index in surprising ways. This is the same trap as `for x in xs: xs.remove(x)` .

Step-by-step thought process: 1. "Generator construction does NOT snapshot the source. The generator is a frame holding a reference to the iterable." 2. "Every `next()` call reads from the live source via the underlying list iterator's index pointer." 3. "Append after construction → visible. The list grew, the index hasn't reached the end yet, new items get yielded." 4. "Delete in the middle → shifts indices. The generator skips the item that took the deleted slot's position." 5. "Same hazard with dict mutation during iteration — but dicts raise `RuntimeError` explicitly; lists fail silently."

Code:

```

# Append during iteration – generator sees the new items
xs = [1, 2, 3]
gen = (x * 10 for x in xs)
print(next(gen))           # 10
xs.append(4)                # mutate source mid-stream
xs.append(5)
print(list(gen))            # [20, 30, 40, 50] – appends visible

# Same with a generator function
def stream_orders(orders):
    for o in orders:
        yield o

queue = ["order-1", "order-2"]
gen = stream_orders(queue)
print(next(gen))            # order-1
queue.append("order-3")     # added while we're iterating
print(list(gen))            # ['order-2', 'order-3']

# Delete during iteration – index shifts, item skipped
xs = [1, 2, 3, 4, 5]
out = []
for x in xs:
    out.append(x)
    if x == 2:
        xs.remove(x)        # removes index-1; 3 shifts to index 1
print(out)                  # [1, 2, 4, 5] – 3 skipped

# Dict mutation during iteration – RuntimeError
d = {"a": 1, "b": 2, "c": 3}
try:
    for k in d:
        if k == "b":
            d["new"] = 99
except RuntimeError as e:
    print(e)                 # dictionary changed size during iteration

# Safe pattern – snapshot via list() or tuple()
xs = [1, 2, 3]
for x in list(xs):          # iterate over a COPY
    if x == 2:
        xs.append(99)       # safe – original snapshot is fixed
print(xs)                   # [1, 2, 3, 99]

```

Gotchas: - `enumerate(xs)` over a mutating list has the same shift problem — the underlying index drives the iteration. - A generator over a dict view (`d.keys()`) is live — mutating the dict mid-stream raises `RuntimeError`. View semantics differ from list-iterator semantics. - `collections.deque` raises `RuntimeError` on mutation during iteration — safer than lists for queue patterns.

What NOT to say: - "Generators snapshot the input." — they don't. They hold a reference and read lazily. Saying snapshot is the #1 misconception here. - "Just use `for x in xs[:]:` ." — works (it's the same snapshot trick) but obscures intent. Prefer `for x in list(xs):` for readability.

Common follow-up: *"If you need to filter a list and remove the filtered items, what's the safe pattern that doesn't trigger this bug?"*

Stand-out line: *"Generators are lazy windows into the source — they don't copy, they don't freeze. Mutating the source while iterating means the generator yields whatever the source has when `next()` runs, which is rarely what the author intended."*

Q55 · File handle as iterator — for-loop iteration, the line generator pattern

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Cred, JP Morgan, Swiggy · Topic: File-as-iterator pattern

The question: *"A file object in Python is itself an iterator — what does iterating over it yield, and how do you use this for memory-efficient log processing?"*

The 30-second answer: A text-mode file opened with `open(path)` is an iterator whose `__next__` returns the next line (including the trailing `\n`) and raises `StopIteration` at EOF. `for line in f:` streams line by line in constant memory — never loading the whole file. This is the canonical pattern for processing multi-GB log files, JSONL streams, and CSV exports on machines that can't hold them in RAM.

Step-by-step thought process: 1. "`open(path)` returns a file object that IS an iterator — its `__iter__` returns self, its `__next__` reads the next line." 2. "`for line in f:` is the same as `for line in iter(f):` — and the iterator yields strings up to and including the next `\n`." 3. "This is memory-O(1) regardless of file size. A 10GB log file uses the buffer-size chunk in RAM, not the full file." 4. "`line.rstrip()` (or `line.rstrip('\n')`) removes the trailing newline. Forgetting this is the #1 newbie bug with file iteration." 5. "For binary files, iteration yields fixed-size chunks (not lines) — you usually want `iter(lambda: f.read(N), b'')` instead, the two-arg iter pattern."

Code:

```

# Canonical line-streaming pattern – constant memory
with open("/etc/hosts") as f:
    for line in f:
        print(repr(line[:30]))
        # break – uncomment to see one line

# Strip newlines and parse JSONL
import json
with open("/tmp/orders.jsonl", "w") as f:
    f.write('{ "id": 1, "amount": 500}\n')
    f.write('{ "id": 2, "amount": 700}\n')

with open("/tmp/orders.jsonl") as f:
    for line in f:
        order = json.loads(line)
        print(order["id"], order["amount"])
        # rstrip not needed – json handles trailing whitespace

# Generator pipeline – compose for big logs
def parse_orders(path):
    with open(path) as f:
        for line in f:
            line = line.rstrip()
            if not line:
                continue
            yield json.loads(line)
        # skip blank lines

def big_orders(orders, threshold):
    for o in orders:
        if o["amount"] > threshold:
            yield o

# Composed pipeline – O(1) memory, end-to-end
big = big_orders(parse_orders("/tmp/orders.jsonl"), 600)
for o in big:
    print(o)
    # {'id': 2, 'amount': 700}

# File is an iterator, not iterable – you can't restart it
f = open("/etc/hosts")
print(sum(1 for _ in f))
print(sum(1 for _ in f))
f.seek(0)
print(sum(1 for _ in f))
f.close()
# count lines
# 0 – exhausted; need f.seek(0)
# back to original count

```

Gotchas: - The yielded line INCLUDES the trailing `\n` (the last line may or may not, depending on file). Always `.rstrip()` or `.rstrip('\n')` before parsing. - The file is its own iterator (not just iterable) — once exhausted, you can't `for x in f:` again without `f.seek(0)`. Reading the same file twice without seek is the #2 newbie bug. - Reading a binary file iterates bytes (well, chunks) — usually wrong; use `iter(lambda: f.read(4096), b'')` for binary streaming.

What NOT to say: - "I'll use `f.readlines()` ." — loads the entire file into memory as a list. Defeats the streaming benefit; OOMs on big files. - "Files are not iterators." — they are. Saying this signals you've never iterated a file.

Common follow-up: *"How would you stream a 10GB compressed log file (`.gz`) line by line without decompressing it to disk first?"*

Stand-out line: *"A file is its own iterator — `for line in f:` is the lowest-ceremony, lowest-memory streaming pattern in Python. Compose it with generator pipelines and you can process arbitrarily large files in constant RAM."*

Q56 · `__aiter__` and `__anext__` — the async iterator protocol

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Postman, Walmart Labs · Topic: Async iterator protocol

The question: *"What's the async iterator protocol — what two methods does it require, and how do they differ from `__iter__` / `__next__` ?"*

The 30-second answer: An async iterator implements `__aiter__` (returns `self` , synchronous) and `__anext__` (an `async def` method that returns the next value or raises `StopAsyncIteration`). The shape mirrors the sync protocol exactly, but `__anext__` is awaitable — meaning each step can suspend on I/O. `async for x in obj:` calls `__aiter__` once, then awaits `__anext__()` until `StopAsyncIteration` .

Step-by-step thought process: 1. "Two methods: `__aiter__` and `__anext__` . The 'a' prefix everywhere signals async." 2. "`__aiter__` is synchronous — it returns the iterator (`self`). It does NOT yield values. People mix this up." 3. "`__anext__` is the worker. It's `async def` , returns a coroutine, awaited each step. Returns the next value or raises `StopAsyncIteration` ." 4. "`StopAsyncIteration` is the async cousin of `StopIteration` — the sentinel signaling exhaustion." 5. "Use cases: streaming results from an async DB cursor, paginated API responses, an async message queue consumer."

Code:

```

import asyncio

# Async iterator – custom class implementing the protocol
class AsyncOrderStream:
    def __init__(self, orders):
        self.orders = orders
        self.index = 0

    def __aiter__(self):
        # sync – returns self
        return self

    async def __anext__(self):
        # async – returns next or raises
        if self.index >= len(self.orders):
            raise StopAsyncIteration
        await asyncio.sleep(0.01)
        # simulate I/O wait
        order = self.orders[self.index]
        self.index += 1
        return order

async def main():
    stream = AsyncOrderStream(["order-1", "order-2", "order-3"])
    async for order in stream:
        print("processed", order)

asyncio.run(main())
# processed order-1 / processed order-2 / processed order-3

# Common slip – making __aiter__ async instead of sync
class WrongStream:
    async def __aiter__(self):
        # WRONG – must be sync
        return self
    async def __anext__(self):
        raise StopAsyncIteration

# async for raises TypeError because __aiter__ returned a coroutine, not an iterator
async def buggy():
    try:
        async for x in WrongStream():
            pass
    except TypeError as e:
        print("error:", str(e)[:60])

asyncio.run(buggy())

```

Gotchas: - `__aiter__` is synchronous. Marking it `async def` makes it return a coroutine, breaking the `async for` protocol — `TypeError`. - `__anext__` returns a coroutine you await — the coroutine itself yields nothing; the awaited result is the next value. - `StopAsyncIteration` (not `StopIteration`) signals exhaustion. Raising `StopIteration` from inside `__anext__` triggers PEP 525's wrap to `RuntimeError`.

What NOT to say: - "It's the same as the sync protocol with `async` prefixed." — close, but `__aiter__` is NOT async. The asymmetry trips up most candidates. - "I'd just use `__iter__` and `__next__`." — `async for` does not call those; it specifically looks up `__aiter__` and `__anext__`.

Common follow-up: "Why is `__aiter__` synchronous while `__anext__` is async — what would break if both were async?"

Stand-out line: "The async iterator protocol mirrors the sync one with one asymmetry: `__aiter__` stays sync because it's just identity-return; `__anext__` is async because that's where the I/O suspension happens. `StopAsyncIteration` closes the loop."

Q57 · `async for` — what it desugars to under `__anext__`

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Postman, JP Morgan · Topic: Async-for desugaring

The question: "What does `async for x in obj: body` actually compile to? Walk me through the desugared form."

The 30-second answer: `async for` calls `obj.__aiter__()` once (synchronously) to get an async iterator, then in a loop awaits `iterator.__anext__()`, binds the result to `x`, and runs the body. When `__anext__()` raises `StopAsyncIteration`, the loop catches it silently and exits. The pseudocode is identical to sync `for` except the inner `next()` is replaced by `await __anext__()` and the sentinel is `StopAsyncIteration`.

Step-by-step thought process: 1. "Same shape as sync `for`: one `__aiter__` call up front, then a `__anext__` loop until exhaustion." 2. "The key difference: `__anext__` is awaited. That's the suspension point — the event loop can run other coroutines while this one waits." 3. "Pseudocode: `it = obj.__aiter__(); while True: try: x = await it.__anext__() except StopAsyncIteration: break; body`." 4. "`break`, `continue`, `else` clauses all work the same as sync `for`." 5. "`async for` is ONLY legal inside `async def` — using it in a regular function is a `SyntaxError`."

Code:

```

import asyncio

# What `async for x in obj: body` compiles to
async def async_for_equivalent(obj, body):
    it = obj.__aiter__()                # sync call
    while True:
        try:
            x = await it.__anext__()    # await the next value
        except StopAsyncIteration:
            break
        await body(x)
    # else clause would run here on natural exit

# Compare side by side
class Stream:
    def __init__(self, items):
        self.items = list(items)
        self.i = 0
    def __aiter__(self):
        return self
    async def __anext__(self):
        if self.i >= len(self.items):
            raise StopAsyncIteration
        await asyncio.sleep(0.005)
        v = self.items[self.i]
        self.i += 1
        return v

async def main():
    # Real async for
    s = Stream([1, 2, 3])
    async for x in s:
        print("real:", x)

    # Manually desugared
    s = Stream([1, 2, 3])
    async def body(x):
        print("manual:", x)
    await async_for_equivalent(s, body)

asyncio.run(main())

# async for with else – runs on natural exit
async def find_async(stream, target):
    async for x in stream:
        if x == target:
            print("found", x)
            return
    else:
        print("not found")                # runs because no return

asyncio.run(find_async(Stream([1, 2, 3]), 99)) # not found

```

```
# SyntaxError outside async def
# def bad():
#     async for x in something: pass          # SyntaxError
```

Gotchas: - `async for` outside `async def` is a `SyntaxError`. Use `asyncio.run(...)` to enter the async context. - Each `await __anext__()` is a suspension point — other coroutines can run, exceptions in concurrent code can affect this loop. - The `else` clause on `async for` runs only on natural exhaustion, exactly like sync `for-else`. Break / return / raise skip it.

What NOT to say: - "`async for` is just `for` wrapped in `asyncio.run`." — not at all; it's a different protocol that requires `__aiter__` / `__anext__`, not `__iter__` / `__next__`. - "I'd use `for x in await obj.fetch_all():` instead." — works but defeats the streaming benefit. You've materialised the full result before iterating; that's not async iteration, that's eager.

Common follow-up: "What's the difference between `async for x in stream:` and `for x in await stream.fetch_all():` — when would you pick each?"

Stand-out line: "`async for` is four lines of desugaring: one sync `__aiter__()`, one `while True`, one `await __anext__()` inside `try`, one `except StopAsyncIteration: break`. The `await` is the suspension point — that's the entire async win."

Q58 · `aiter()` / `anext()` (3.10+) — built-in helpers

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, Cred, Walmart Labs · Topic: `aiter/anext` built-ins

The question: "Python 3.10 added `aiter()` and `anext()` built-ins. What do they do, and when would you use them directly instead of `async for`?"

The 30-second answer: `aiter(obj)` calls `obj.__aiter__()` and returns the async iterator (sync call). `anext(async_iter)` returns an awaitable that yields the next value when awaited, or raises `StopAsyncIteration`. They're the async cousins of `iter()` / `next()` and let you step an async iterator manually — peeking at the first item, lockstep pairing, processing one chunk at a time without committing to a full `async for` loop.

Step-by-step thought process: 1. "These showed up in Python 3.10. Before that, you had to call `obj.__aiter__()` / `obj.__anext__()` directly — clunky." 2. "`aiter(obj)` is synchronous — returns the async iterator. Same one-shot rule as sync: calling it on something already-an-iterator returns the same object." 3. "`anext(it)` returns a coroutine. You MUST await it to get the value. Calling `anext(it)` and forgetting to await is a common bug." 4. "Two-arg form: `anext(it, default)` returns the default if the iterator is exhausted — no `StopAsyncIteration`. Saves a try/except." 5. "Use case: reading a header chunk from an async stream, then handing the rest off to a different consumer."

Code:

```

import asyncio

class Pages:
    """Async pagination – yields page numbers 0..n."""
    def __init__(self, n):
        self.n = n
        self.i = 0
    def __aiter__(self):
        return self
    async def __anext__(self):
        if self.i >= self.n:
            raise StopAsyncIteration
        await asyncio.sleep(0.005)
        v = self.i
        self.i += 1
        return v

async def main():
    # Manual stepping – peek at the first page (the "header")
    pages = Pages(5)
    it = aiter(pages)                                # returns the iterator
    first = await anext(it)                          # await – get the value
    print("header:", first)                          # header: 0

    # Remaining pages via async for
    async for page in it:                            # continues from where we left off
        print("page:", page)

    # Two-arg anext – no StopAsyncIteration when done
    pages = Pages(2)
    it = aiter(pages)
    print(await anext(it, -1))                       # 0
    print(await anext(it, -1))                       # 1
    print(await anext(it, -1))                       # -1 (default kicks in)
    print(await anext(it, -1))                       # -1 (stays exhausted)

asyncio.run(main())

# Common bug – forgetting to await anext()
async def buggy():
    pages = Pages(3)
    it = aiter(pages)
    val = anext(it)                                  # returns a coroutine, not the value
    print(type(val).__name__)                       # coroutine – not what was wanted
    val.close()                                     # close the unawaited coroutine to silence the warning

asyncio.run(buggy())

```

Gotchas: - `anext()` returns a coroutine — you MUST await it. Forgetting the await gives you a coroutine object and a "never-awaited" `RuntimeWarning`. - `aiter()` is sync. Awaiting it (`await aiter(x)`) is a `TypeError` — `aiter()` doesn't return an awaitable. - The two-arg `anext(it,`

`default)` returns an awaitable that yields the default if exhausted. The default itself is returned (not the value's type-coerced version).

What NOT to say: - "I'd call `__anext__()` directly." — works but the built-in is the idiomatic 3.10+ form. Using the dunder directly signals dated code. - "`anext()` is the same as `await` ." — confused; `anext()` returns an awaitable. You still need `await` .

Common follow-up: "What's the cleanest way to peek at the first value of an async iterator without consuming the rest?"

Stand-out line: " `aiter()` and `anext()` made manual async-iteration stepping a first-class operation in 3.10. The two-arg `anext(it, default)` form is the cleanest way to handle exhaustion without wrapping every step in `try/except StopAsyncIteration` ."

Q59 · Async generator functions (`yield` inside `async def`) — vs async iterator class

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Cred, Postman, JP Morgan · Topic: Async generators

The question: "What's an async generator function — how does putting `yield` inside an `async def` differ from a regular generator, and when would you write the iterator class form instead?"

The 30-second answer: An async generator is an `async def` function that contains `yield` . Calling it returns an **async generator object** that implements `__aiter__` / `__anext__` automatically — you get the async iterator protocol for free, with the same pause-and-resume semantics as a regular generator, but each step can `await` I/O. The class form is preferred when you need extra methods, state inspection, or the generator's lifecycle to outlive the function frame.

Step-by-step thought process: 1. " `yield` inside `async def` → async generator. Calling it returns an async generator object — NOT a coroutine." 2. "The object implements `__aiter__` and `__anext__` automatically. The protocol is filled in by the interpreter, not you." 3. "Inside the body, you can both `await` and `yield` . That's the whole point — pause for I/O AND pause for the consumer." 4. " `return value` is illegal in an async generator (raises `SyntaxError` on attempt to return a non-None value). PEP 525 forbids it because the return value has nowhere to ride — there's no `StopAsyncIteration.value` semantic." 5. "Class form wins when you need to expose `.close()` , `.athrow()` , custom buffering, or pluggable backpressure logic."

Code:

```

import asyncio

# Async generator function – the cleanest form
async def stream_orders(n):
    for i in range(n):
        await asyncio.sleep(0.005)          # simulate I/O wait per item
        yield f"order-{i}"

async def main_gen():
    async for order in stream_orders(3):
        print("from gen:", order)           # order-0 / order-1 / order-2

asyncio.run(main_gen())

# Equivalent class form – more verbose, more flexible
class OrderStream:
    def __init__(self, n):
        self.n = n
        self.i = 0

    def __aiter__(self):
        return self

    async def __anext__(self):
        if self.i >= self.n:
            raise StopAsyncIteration
        await asyncio.sleep(0.005)
        v = f"order-{self.i}"
        self.i += 1
        return v

async def main_class():
    async for order in OrderStream(3):
        print("from class:", order)

asyncio.run(main_class())

# return with value is illegal in async generator
async def bad():
    yield 1
    return "no good"                       # SyntaxError on parse – return value in async gen

# Inspect generator type
async def gen():
    yield 1

g = gen()
import inspect
print(inspect.isasyncgen(g))               # True
print(type(g).__name__)                   # async_generator

# Async generator with try/finally for cleanup
async def streamed_query(conn, sql):

```

```

cursor = await conn.execute(sql)
try:
    async for row in cursor:
        yield row
finally:
    await cursor.close() # runs on GC, break, exhaustion

```

Gotchas: - `return value` (with a value) inside an async generator is a `SyntaxError`. Bare `return` is fine — it ends the generator. - `aclose()` is the async cousin of `close()`. You await it; it injects `GeneratorExit` and runs your `try/finally`. - Async generators inherit PEP 479-style handling — `StopIteration` / `StopAsyncIteration` leaking from inside becomes `RuntimeError`.

What NOT to say: - "An async generator is just a coroutine that yields." — coroutines and generators are distinct objects; an async generator is a third category. Calling it a coroutine is wrong. - "I'd use a list and `await`." — defeats the streaming benefit. The point of an async generator is constant memory + I/O suspension per item.

Common follow-up: "Why is `return value` (with a value) illegal in an async generator — what's the design reason behind the restriction?"

Stand-out line: "An async generator is the cleanest expression of 'stream items while doing I/O' — `yield` for the consumer, `await` for the network. The class form exists for cases where you need extra methods or external lifecycle control; otherwise the `async def` + `yield` form wins on readability."

Q60 · Async generator vs async function returning an iterator — semantic difference

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Swiggy, Walmart Labs · Topic: Async iteration semantics

The question: "What's the difference between an async generator and an `async def` function that returns an iterator — and why does the distinction matter for streaming workloads?"

The 30-second answer: An **async generator** (`async def` + `yield`) returns an async iterator immediately and streams items one at a time, each step possibly awaiting I/O — constant memory, interleaved with the event loop. An **async function that returns an iterator** (`async def` + `return iter(...)`) runs to completion first, materialising or building the iterator eagerly, then returns it — you wait for the whole computation before getting any items. The async generator streams; the async-returns-iterator batches.

Step-by-step thought process: 1. "Surface looks similar — both produce something iterable from an async context. Semantics are opposite." 2. "Async generator: lazy, per-item, suspends on `await` between yields. Consumer sees the first item as soon as it's ready." 3. "`async def` returning iterator:

eager, runs the entire body to completion before returning. By the time you get the iterator, the work is done." 4. "Concrete consequence: if upstream is paginated (page 1 → row 1..50, page 2 → row 51..100), async generator returns row 1 after page 1's network call; the function form waits for ALL pages before returning anything." 5. "The hybrid trap: `async def fetch_all(): return [item async for item in stream]` — that's a list comprehension over an async stream, eager. Async generator gives you the streaming version."

Code:


```

import asyncio
import time

# Helper – simulates a paginated upstream
async def fetch_page(n):
    await asyncio.sleep(0.05)          # network wait
    return [f"row-{n}-{i}" for i in range(3)]

# Form 1 – async generator (streams)
async def stream_rows(pages):
    for p in range(pages):
        rows = await fetch_page(p)
        for r in rows:
            yield r                    # consumer sees each row as it arrives

# Form 2 – async function returning iterator (batches)
async def all_rows(pages):
    out = []
    for p in range(pages):
        rows = await fetch_page(p)
        out.extend(rows)
    return iter(out)                  # all work done; iter is over a full list

async def main():
    # Streaming form – first row visible after ~50ms (page 1 done)
    t0 = time.perf_counter()
    async for row in stream_rows(3):
        elapsed_ms = (time.perf_counter() - t0) * 1000
        print(f"stream: {row} at {elapsed_ms:.0f}ms")

    print()

    # Batched form – nothing visible until ~150ms (ALL pages done)
    t0 = time.perf_counter()
    rows = await all_rows(3)
    for row in rows:
        elapsed_ms = (time.perf_counter() - t0) * 1000
        print(f"batch: {row} at {elapsed_ms:.0f}ms")

asyncio.run(main())

# stream rows arrive every ~50ms in groups
# batch rows all arrive at ~150ms, then iterate instantly in memory

# Trap – list comprehension over async iter is eager
async def collect_eager():
    return [r async for r in stream_rows(3)] # materialises the whole stream

# This is fine if you NEED a list; not fine if you want streaming semantics

# Async generator can be paused / resumed; batched can't be cancelled mid-stream
async def consumer():
    gen = stream_rows(100)

```

```
await anext(gen)          # one row
await gen.aclose()        # done — upstream pages 2..99 never fetched
```

Gotchas: - An `async def` with `yield` is an async generator. An `async def` with `return` is a coroutine. You can't have both in the same function — `yield` flips it permanently to generator. - `await fn()` of an async generator function returns the generator object (you get a `TypeError: object async_generator can't be used in 'await' expression`). You don't await it; you `async for` over it. - `async def` returning an iterator wastes async-ness if all the awaiting happens upfront — the function is effectively sync after the awaits.

What NOT to say: - "They're the same thing." — they are not; one streams, the other batches. Saying same signals you haven't thought about backpressure. - "I'd always pick the function form for simplicity." — wrong default; for I/O-heavy paginated streams, the generator form is constant memory and gets first results sooner.

Common follow-up: "If your downstream consumer can only process 100 rows / second and upstream produces 10K / second, which form gives you backpressure for free?"

Stand-out line: "An async generator streams item-by-item with I/O suspension between yields — first result fast, constant memory, free backpressure. An `async def` that returns an iterator runs the whole thing eagerly first — last result is the first result you see. Pick streaming for paginated APIs, batched for small results you'll re-iterate anyway."

End of sub-chunk 6 of 6 — Iterator pitfalls (Part B) + Async iterators (Q51–Q60). End of Chapter 3 — Generators + Iterators.

Chapter 4 — The GIL + Concurrency (50 Qs)

Python Interview Prep — 2026 Edition · Learn & Excel · File 04 Core Python Library

This is the chapter every Indian Python interview routes through eventually — sometimes in the first ten minutes (the "explain the GIL in 60 seconds" warm-up), sometimes in round two (the "your scraper is slow, why and what would you change?" debug). Backend panels at Razorpay, Swiggy, Flipkart, Cred and PhonePe push it hard; GCC panels at JPMC, Goldman, Walmart Labs and Microsoft India use it to filter the candidate who has *used* threading from the one who has *understood* the GIL. Even data roles get a softer version — "why does multiprocessing show up in your pandas workflows but threading doesn't?"

The 50 questions below are sequenced from mechanics outward. Section A walks the GIL itself — what it is, why it exists, when it releases, how the switch interval works. Section B is the implication layer —

the CPU-bound versus I/O-bound divide, the one-core ceiling, contention in C extensions. Section C is the `threading` module surface — `Thread`, locks, primitives, queues. Section D is `multiprocessing` — `Process`, pools, `Manager`, pickling, fork vs spawn. Section E is `asyncio` end-to-end — event loop, `TaskGroup`, cancellation, the cooperative model. Section F looks at 3.13 free-threaded builds, PEP 703, per-interpreter GIL (PEP 684), and what you say in an interview *today* about all of it.

Each Q follows the standard format: tags, the question phrased the way an interviewer would say it, the 30-second answer, the step-by-step thought process you say out loud, edge cases the panel is waiting to hear, what NOT to say, and the common follow-up. Code compiles on Python 3.12+ unless noted.

Table of Sections

- Section A — The GIL · core mechanics (Q1–Q10)
- Section B — GIL implications · what it means in practice (Q11–Q15)
- Section C — The `threading` module (Q16–Q25)
- Section D — The `multiprocessing` module (Q26–Q35)
- Section E — `asyncio` end-to-end (Q36–Q45)
- Section F — Python 3.13 + the future (Q46–Q50)

Section A — The GIL · Core Mechanics

This section is the mechanism layer. If you can explain what the GIL physically is, why CPython has it, when it gets released, and how the switch interval works, you have already cleared the GIL bar at every Indian product company round. The trap is candidates who memorise a one-liner ("Python has a global lock") and stall the moment the interviewer asks "released when?" or "why was it added in the first place?" Drill the mechanics; the rest of the chapter falls out of it.

Q1 · What is the GIL — Defined From First Principles

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, Flipkart, JPMC · Topic: GIL Mechanics

The question (interviewer's verbatim phrasing):

Explain the Global Interpreter Lock to me in 60 seconds. What is it, where does it live, and what does it protect?

The 30-second answer: The GIL is a mutex inside the CPython interpreter that allows only one OS thread to execute Python bytecode at any given moment. It lives in the CPython runtime — `ceval.c` — not in the language spec. It protects the interpreter's internal state, mainly the per-object reference counts and other C-level structures that are not thread-safe. It is a CPython implementation detail; Jython and IronPython have no GIL, PyPy has one but with a different switch model, and CPython 3.13 ships an experimental free-threaded build that removes it.

Step-by-step thought process: 1. Lead with "it is a mutex, not a language feature" — that one sentence tells the interviewer you understand the GIL is a CPython implementation choice, not a Python design choice. 2. Locate it physically: it lives in `Python/ceval.c` inside the CPython source. Threads acquire it before running any bytecode and release it on certain transitions. 3. State what it protects — reference counts. Every Python object has a `Py_ssize_t ob_refcnt` field that increments on every name binding and decrements on every name release. Without the GIL, two threads incrementing the same refcount could lose updates, and the object would either leak or be freed prematurely. 4. Close with the scope clarification: it is a *CPython* lock. The interviewer knows the bar candidate stops at "Python has a GIL" — clearing the bar means saying "CPython has a GIL; Jython, IronPython, and the 3.13 free-threaded build do not."

Edge cases to mention: - The GIL guards bytecode execution; C extensions that release it (NumPy, hashlib, file I/O syscalls) run in parallel across threads. - The GIL is reentrant per thread in the sense that a thread holding it can recursively call back into the interpreter — it does not deadlock against itself. - The GIL is NOT a single lock per process for the OS — it is one mutex per CPython interpreter; PEP 684 introduces per-interpreter GIL so multiple interpreters in one process each get their own.

What NOT to say: - "Python has a GIL." — Sloppy. CPython has a GIL. The language does not. - "The GIL protects against race conditions in user code." — Wrong. The GIL protects the *interpreter's* internal state. User-level data races still exist; you still need `threading.Lock` for your own shared variables. - "The GIL prevents threading." — Wrong. Threading works fine; what the GIL prevents is parallel *bytecode* execution.

Common follow-up: "*Then why does threading exist in Python at all?*" — Because most workloads are I/O-bound. The GIL releases on I/O syscalls, so threads waiting on disk or network run concurrently even if their Python code does not run in parallel. Threading is the right primitive for I/O concurrency; it is the wrong one for CPU-bound parallelism.

```
# The GIL is invisible from user code – you cannot import it or unlock it.
# You can see its effect: this loop will NOT speed up across threads.
import threading, time

def cpu_burn(n):
    while n > 0:
        n -= 1

start = time.perf_counter()
threads = [threading.Thread(target=cpu_burn, args=(10_000_000,)) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
print(f"4 threads: {time.perf_counter() - start:.2f}s") # ~ same as 1 thread
```

Q2 · Why the GIL Exists — The Reference-Counting Justification

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, PhonePe, Goldman, Walmart Labs · Topic: GIL Mechanics

The question (interviewer's verbatim phrasing):

Why does CPython have the GIL at all? Other languages don't have it — what specifically forced it into CPython?

The 30-second answer: CPython manages memory by reference counting: every object carries a refcount that bumps up when a name binds to it and bumps down when a name releases it; when the count hits zero, the object frees. Refcount increments and decrements are not atomic in C without explicit synchronisation. Without a global lock, two threads modifying the same refcount could lose updates — leading to memory corruption, double-frees, or leaks. The GIL is the single coarse-grained lock that made the rest of the C codebase safe to write without fine-grained per-object locks; removing it without a redesign of memory management was historically considered worse than the cost.

Step-by-step thought process: 1. Start with the mechanism, not the politics: CPython uses refcounting. Java uses tracing GC, which is why it does not have this problem. 2. Refcount operations are tiny — `Py_INCREF` is a single `++` on a C field; `Py_DECREF` is `--` plus a zero-check. Making each one atomic costs ~25% performance in single-threaded code due to memory barriers and cache invalidation. 3. The historical attempt: Larry Hastings's Gilectomy (2015–2017) replaced the GIL with fine-grained per-object locks. Single-threaded performance dropped by roughly 30%, and the project stalled — the cost was unacceptable for the 99% of code that doesn't benefit from removal. 4. The 2024 redemption: PEP 703 (Sam Gross) ships in 3.13 as the free-threaded build. It uses biased reference counting + deferred reference counting, which keeps single-thread cost low while making refcount updates safe across threads. Different design, viable answer, ten years later.

Edge cases to mention: - The GIL is not *required* for thread safety of objects — it is required for thread safety of CPython's *internal* bookkeeping. If you redesign the bookkeeping (PEP 703), the lock can go. - Other implementations chose differently: PyPy has a GIL but with software transactional memory experiments; Jython rides the JVM's GC and has no GIL; GraalPy is in flux. - The trade-off the GIL enforces is "easy to write C extensions" vs "easy to scale to many cores" — CPython picked the first; that choice gave us NumPy, SciPy, scikit-learn, PyTorch.

What NOT to say: - "Guido added the GIL because he doesn't like threading." — Pop-tech-blog answer. The GIL exists for refcount safety, not preference. - "Removing the GIL is easy." — Hostile to the interviewer's intelligence. Gilectomy proved otherwise; PEP 703 took a near-complete redesign. - "The GIL is a Python design flaw." — It is a CPython implementation trade-off. The language spec says nothing about it.

Common follow-up: "*Could CPython have used tracing GC instead?*" — Yes, and it would not have needed the GIL for refcount safety. But tracing GC introduces unpredictable pauses and breaks the deterministic destruction semantics that Python users rely on (the "object freed when last reference dies, so file is closed, so socket is released" pattern). Switching would have broken the language contract more than it improved it.

```
# Refcount visibility from user code – sys.getrefcount
import sys
x = object()
print(sys.getrefcount(x))    # 2 – one for x, one for the argument frame
y = x
print(sys.getrefcount(x))    # 3 – y holds a reference too
del y
print(sys.getrefcount(x))    # 2 – back down
```

Q3 · When Does the GIL Release — The Three Triggers

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, JPMC, Goldman · Topic: GIL Mechanics

The question (interviewer's verbatim phrasing):

When does the GIL actually release? Walk me through the situations where a CPython thread gives up the lock.

The 30-second answer: Three triggers: first, the interpreter periodically yields based on the switch interval (default 5 milliseconds) — any thread can be preempted between bytecode boundaries. Second, blocking I/O syscalls release the GIL — `read`, `write`, `recv`, `accept`, `time.sleep` and similar all wrap their underlying C call in `Py_BEGIN_ALLOW_THREADS` and `Py_END_ALLOW_THREADS` macros. Third, well-behaved C extensions release the GIL around long-

running computations — NumPy releases it during large matrix ops, hashlib releases it during big digests, file I/O releases it during the read.

Step-by-step thought process: 1. The three triggers are the bar — name them in order: (a) timer-based preemption via the switch interval, (b) I/O syscall transitions, (c) explicit C-extension release. 2. The switch interval is the proactive one — every ~5ms, the running thread checks if another thread wants the GIL and yields if so. Before 3.2 this was tick-based ("100 bytecode ticks"); modern CPython is time-based via `sys.setswitchinterval`. 3. The I/O-bound case is the killer feature — `socket.recv(4096)` releases the GIL the moment it makes the kernel call and reacquires it when data arrives. While one thread is waiting on the network, three other threads can run Python bytecode. That is *why* threading helps web scrapers. 4. The C-extension release is the multi-core escape hatch — pure-Python code stays single-core, but a `numpy.dot(a, b)` on a large array uses BLAS, releases the GIL, and saturates all cores. That is *why* the data stack works at scale on Python.

Edge cases to mention: - Bytecode that does *no* I/O and calls *no* GIL-releasing extension (a tight Python `while` loop) holds the GIL until the switch interval times out — and only then yields. - The switch interval is best-effort — the running thread checks at safe points (loop tops, function returns); it cannot be preempted in the middle of a single bytecode op. - A misbehaved C extension that calls back into the interpreter without holding the GIL will crash the process — that is why `Py_BEGIN_ALLOW_THREADS` discipline matters.

What NOT to say: - "The GIL releases every 100 ticks." — Pre-3.2 behavior. Modern CPython is time-based, not tick-based. - "Threads are preempted whenever." — Sloppy. They are preempted at bytecode boundaries, not within a single op. - "I/O does not release the GIL." — Wrong. The GIL releases on every I/O syscall — that is the entire reason threading is useful for I/O.

Common follow-up: "How would you confirm the GIL is being released around a NumPy call?" — Run a CPU-bound NumPy op (`np.linalg.svd` on a big matrix) in multiple threads and measure wall-clock time. If the GIL is being released, total time scales with cores; if not, it stays single-threaded. NumPy releases the GIL, so you should see speedup; pure-Python `sum(...)` will not.

```
# I/O releases the GIL — this scales across threads
import threading, time, urllib.request

def fetch(url):
    urllib.request.urlopen(url).read()

urls = ["https://httpbin.org/delay/1"] * 4
start = time.perf_counter()
threads = [threading.Thread(target=fetch, args=(u,)) for u in urls]
for t in threads: t.start()
for t in threads: t.join()
print(f"Threaded fetch: {time.perf_counter() - start:.2f}s") # ~1s, not ~4s
```

Q4 · The Switch Interval — `sys.setswitchinterval`

Tags: Difficulty: Medium · Asked at: Razorpay, Goldman, Walmart Labs, JPMC · Topic: GIL Mechanics

The question (interviewer's verbatim phrasing):

What is the GIL switch interval, what is its default, and when would you change it?

The 30-second answer: The switch interval is the duration after which the currently-executing thread is asked to release the GIL so another waiting thread can acquire it. Default is 5 milliseconds (0.005 seconds) since Python 3.2. You read it with `sys.getswitchinterval()` and set it with `sys.setswitchinterval(seconds)`. You almost never change it in production — the only realistic reason is a latency-sensitive workload where you want shorter intervals to reduce thread-switching jitter, or a throughput-sensitive workload where you want longer intervals to amortise the cost of acquiring and releasing the lock.

Step-by-step thought process: 1. State the default — 5ms — and the API:

`sys.getswitchinterval()` returns seconds as a float, `sys.setswitchinterval(0.001)` would make it 1ms. 2. Explain what the interval *does*: the running thread checks an internal flag at safe points; when the interval elapses, the flag is set, the thread releases the GIL at the next safe point, the OS chooses who runs next. 3. Pre-3.2 the granularity was bytecode ticks — the `sys.setcheckinterval(100)` API. It was deprecated because tick-based intervals had wildly variable wall-clock durations: a tick that hit a dict lookup ran in nanoseconds, a tick that hit a `re.search` ran in milliseconds. 4. Be honest about tuning: changing it is rarely worth it. The OS scheduler also has latency; a smaller switch interval means more context switches, more cache pressure, and often slower overall throughput. Tune your code first, change the interval only if profiling proves it matters.

Edge cases to mention: - The interval is per-process and global — it affects every thread in the interpreter. There is no per-thread fairness knob. - Setting it to zero does NOT make it preemptive at every bytecode — minimum granularity is one safe-point check. - Setting it absurdly high (`sys.setswitchinterval(1.0)`) effectively makes long-running CPU-bound code never yield until done — bad for latency, occasionally good for cache locality on a single tight loop.

What NOT to say: - "The default is 100 ticks." — Pre-3.2. Modern default is 0.005 seconds. - "Always tune the switch interval for performance." — Cargo-cult advice. It is rarely the bottleneck. - "The interval is enforced by the OS." — No, it is enforced by the CPython interpreter at safe points. The OS only sees the thread giving up the lock.

Common follow-up: "How would you measure GIL contention?" — Use `py-spy` (`py-spy top --pid PID` shows what each thread is doing); `perf` on Linux with `--call-graph dwarf`; or in code, count GIL drops via the `gil_drop_request` flag exposed in newer CPython instrumentation. For most apps, a simple comparison of single-thread vs multi-thread wall-clock time on the same workload tells you whether the GIL is the bottleneck.


```
import sys
print(sys.getswitchinterval())           # 0.005 – default 5ms

sys.setswitchinterval(0.001)           # 1ms – more responsive, more overhead
print(sys.getswitchinterval())         # 0.001

sys.setswitchinterval(0.005)           # restore default
```

Q5 · Bytecode Boundaries — Why You Can't Preempt Mid-Op

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman · Topic: GIL Mechanics

The question (interviewer's verbatim phrasing):

What is a "bytecode boundary" and why does it matter for the GIL? Can a thread be preempted in the middle of a `list.append(x)` ?

The 30-second answer: A bytecode boundary is the point between two interpreter operations — specifically, after the current op has fully completed and before the next op begins. The GIL release check happens at these boundaries, not mid-op. So a `LOAD_NAME` followed by a `BINARY_ADD` can be preempted between the two, but the `BINARY_ADD` itself cannot. For `list.append(x)`, the entire C-level append call runs without interruption because the CPython implementation of `list.append` releases nothing — it is atomic from the perspective of other Python threads.

Step-by-step thought process: 1. Define the bytecode boundary first — the gap between two CPython VM operations, where the eval loop checks the `gil_drop_request` flag. 2. Show what is NOT atomic: a Python-level statement like `x += 1` compiles to `LOAD_FAST`, `LOAD_CONST`, `BINARY_OP`, `STORE_FAST` — four bytecodes. A switch can happen between any pair. Two threads doing `x += 1` can lose updates. 3. Show what IS atomic — operations whose entire implementation is a single bytecode op handled by C-level code that does not release the GIL. `list.append`, `dict.__setitem__` with hashable keys, `set.add` — these are commonly cited as thread-safe in CPython. 4. Caveat the atomicity claim: it is a CPython implementation detail, not a language guarantee. PyPy may differ; the free-threaded build may differ. If you want thread safety, use a `Lock` or `queue.Queue` rather than relying on bytecode atomicity.

Edge cases to mention: - `dis.dis(func)` shows you the bytecodes — looking at a function with `dis` is how you check what is atomic and what is not. - `list.append` is atomic, but `lst[i] = lst[i] + 1` is not — it is a load, add, store sequence. - The free-threaded build (3.13+) changes some of this — refcount updates use a different scheme, but per-bytecode atomicity is still preserved at the user level.

What NOT to say: - "Every Python statement is atomic." — False. Most are not. Multi-byte statements (`+=` , attribute lookup with method call, chained calls) compile to multiple ops. - "Bytecode boundaries are an implementation detail and don't matter." — They matter when reasoning about race conditions in threaded code. - "The GIL makes my code thread-safe." — Refcount-safe yes; user-level race-condition-safe no.

Common follow-up: "Show me a race condition that the GIL does NOT prevent." — A counter that two threads increment. `counter += 1` is load-add-store, not atomic. Run two threads each incrementing a million times and the final value will be less than two million — exactly the lost-update race a `Lock` is meant to fix.

```
import dis
def increment(x):
    x += 1
    return x

dis.dis(increment)
# LOAD_FAST x
# LOAD_CONST 1
# BINARY_OP +=
# STORE_FAST x
# LOAD_FAST x
# RETURN_VALUE
# Four ops before the store — preemption window between any pair.
```

Q6 · The GIL is CPython-Specific — Jython, PyPy, GraalPy

Tags: Difficulty: Easy · Asked at: Razorpay, Walmart Labs, JPMC · Topic: GIL Implementations

The question (interviewer's verbatim phrasing):

Is the GIL part of the Python language specification or is it a CPython thing? Name implementations that don't have it.

The 30-second answer: The GIL is a CPython implementation detail, not part of the language spec. Jython (Python on the JVM) has no GIL — it uses Java's monitor locks for thread safety and rides Java's GC. IronPython (Python on .NET) has no GIL — same reason, hosted runtime handles synchronisation. GraalPy (Python on GraalVM) has no GIL in its native runtime. PyPy has a GIL but with a different switch model and active research on stm and software transactional memory. CPython 3.13 ships an experimental free-threaded build with the GIL removed.

Step-by-step thought process: 1. Hit the spec-vs-implementation distinction first. The Python Language Reference does not mention the GIL — it is a CPython artefact. 2. Categorise the

implementations: hosted (Jython, IronPython, GraalPy) ride their host runtime's threading model and naturally avoid the GIL; native (CPython, PyPy) implement their own and choose how to lock. 3. Note adoption reality: Jython tops out at Python 2.7 and is rarely used in new code; IronPython has spotty 3.x support; PyPy is excellent for pure-Python performance but lags CPython on C-extension coverage; GraalPy is promising but new. 4. Land on the practical answer: in 2026, "Python without a GIL" in production means either the 3.13 free-threaded build (early adopter, scientific computing) or accepting that you are not on CPython mainline.

Edge cases to mention: - Jython's lack of GIL is real but Jython itself stopped active 3.x development; you cannot use it for modern Python code. - PyPy has GIL but its JIT eliminates some refcount overhead; for CPU-bound pure Python, PyPy is often faster than CPython with threading. - The 3.13 free-threaded build is opt-in — you need a separate interpreter (`python3.13t`) and many C extensions need patches to be safe.

What NOT to say: - "Python is single-threaded." — Wrong on multiple counts. Threads exist; only Python bytecode execution is serialised in CPython. - "All Python implementations have the GIL." — False. Jython, IronPython, GraalPy, and the 3.13t build do not. - "PyPy has no GIL." — False. PyPy has a GIL with different mechanics, but it is not gone.

Common follow-up: *"If Jython has no GIL, why don't people just use Jython for parallel Python?"* — Three reasons: Jython only supports Python 2.7; the ecosystem (NumPy, pandas, requests) targets CPython C ABI; and JVM cold-start latency makes Jython unsuitable for short-lived scripts. The cost of switching ecosystems exceeds the parallelism benefit for most teams.

```
import platform
print(platform.python_implementation()) # 'CPython' on mainline
# 'PyPy' on PyPy
# 'Jython' on Jython
# 'IronPython' on IronPython
```

Q7 · GIL Acquire/Release on Bytecode — The Eval Loop View

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Microsoft India · Topic: GIL Internals

The question (interviewer's verbatim phrasing):

Walk me through what happens in the CPython eval loop when the GIL switch is requested. From bytecode execution to thread handoff.

The 30-second answer: The eval loop runs each bytecode op in a switch dispatch — at safe points (top of the loop, after certain ops), it checks a flag called `gil_drop_request`. When set, the running thread saves its frame state, releases the GIL via `PyEval_ReleaseLock`, posts a condition signal so

other waiting threads see it, and then immediately tries to reacquire — the OS scheduler decides who wins. The reacquiring thread restores its frame and resumes its eval loop. The whole handoff costs a few microseconds on a healthy system.

Step-by-step thought process: 1. Anchor in the loop: `ceval.c`'s `_PyEval_EvalFrameDefault` is the heart — it is a giant switch over opcodes inside a `for(;;)` loop. 2. At safe points (top of loop, function return, exception handling), the loop checks `eval_breaker` — a bitfield that includes the GIL drop request, pending calls, signals, async exception. Any flag triggers exit from the fast path. 3. On drop: thread state is preserved (`tstate`), the GIL mutex is unlocked, `pthread_cond_signal` (or Windows equivalent) wakes waiters. The dropping thread then calls `take_gil` again — it goes to the back of the queue. 4. On acquire: the winning thread restores its `tstate` pointer in thread-local storage, the eval loop resumes at the next bytecode. The other waiters go back to sleep until the next signal.

Edge cases to mention: - `eval_breaker` is checked, not on every bytecode, but at "safe points" — chosen to balance preemption latency against per-op overhead. - `take_gil` uses a condition variable, not a spinlock — sleeping threads do not burn CPU waiting. - The GIL itself is a `PyMutex` in modern CPython — a custom struct on top of `pthread_mutex_t` on POSIX, `SRWLock` on Windows.

What NOT to say: - "The GIL is checked every instruction." — Too frequent; would crush perf. It is checked at safe points. - "Thread switching is free." — Microseconds per switch, plus cache effects. Adds up if your interval is too short. - "The OS picks who runs next." — Half-right; OS picks who runs but only among threads the runtime allows to compete, and the runtime's gil-handoff logic biases against starvation.

Common follow-up: "What changed in PEP 583's GIL fairness fix?" — Pre-3.2, the GIL had a starvation problem: a CPU-bound thread would reacquire the GIL immediately after releasing it because the I/O-bound thread was still waking up. Antoine Pitrou's redesign (Python 3.2) added a forced wait via a condition variable so the releasing thread cannot immediately reacquire, giving waiters a fair shot.

```
# You cannot see the eval loop from user code, but py-spy or perf reveals it.
# This snippet shows the cost is real:
import threading
def worker(): pass
threads = [threading.Thread(target=worker) for _ in range(100)]
import time
start = time.perf_counter()
for t in threads: t.start()
for t in threads: t.join()
print(f"100 thread create+join: {time.perf_counter() - start:.3f}s")
```

Q8 · The "Tick" Concept — Why `setcheckinterval` Is Deprecated

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wipro, Cognizant, Walmart Labs · Topic: GIL History

The question (interviewer's verbatim phrasing):

What was the difference between `sys.setcheckinterval` and `sys.setswitchinterval`, and why was the first deprecated?

The 30-second answer: `sys.setcheckinterval(n)` controlled tick-based switching — every `n` bytecodes, the interpreter would check whether to release the GIL. It was deprecated in 3.2 and removed in 3.9, replaced by `sys.setswitchinterval(seconds)` which is time-based. The change happened because tick-based granularity had wildly variable wall-clock cost — one tick on a dict lookup is nanoseconds; one tick on a regex match is milliseconds — making GIL behavior unpredictable and starvation common.

Step-by-step thought process: 1. Old API: ticks were CPython bytecode operations, defaulting to 100. After 100 ops the running thread checked the GIL drop flag. 2. The starvation problem: a CPU thread tight-looping through cheap ops would hit 100 ticks in microseconds and re-acquire the lock before an I/O thread had a chance to wake up. The result — I/O threads on a busy CPU thread would essentially stall. 3. Antoine Pitrou's fix in 3.2: switch to wall-clock time (5ms default), and add a forced "wait for the other thread to wake up" handshake so the releasing thread cannot reacquire instantly. 4. Mention the deprecation timeline cleanly — deprecated 3.2, removed 3.9. Modern code uses `setswitchinterval` exclusively; production code rarely needs to touch either.

Edge cases to mention: - The old behavior is still occasionally referenced in older tutorials and SO answers — flag it if a candidate cites a 5+ year old source. - Some embedded uses of Python in C/C++ apps pin to old CPython versions where `setcheckinterval` still exists — but new code does not. - The change is one of the few user-visible improvements from PEP 583 / Pitrou's GIL redesign.

What NOT to say: - "They are the same thing." — They are not. Ticks vs time. - "The GIL still uses ticks." — It does not, since 3.2. - "`setcheckinterval` still works in 3.11." — False, removed in 3.9.

Common follow-up: "What is the equivalent of `setcheckinterval(100)` in modern Python?" — There isn't one. `setswitchinterval` cannot be expressed in ticks; the units differ. The closest analog is `sys.setswitchinterval(0.005)` (the default), which gives roughly the same fairness behavior without the per-op variability.

```
import sys
# Modern API
sys.setswitchinterval(0.005) # 5ms
print(sys.getswitchinterval())

# Old API — gone in 3.9+
try:
    sys.setcheckinterval(100)
except AttributeError as e:
    print("Removed since 3.9:", e)
```

Q9 · GIL vs `threading.Lock` — Different Locks, Different Jobs

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy · Topic: GIL vs User Locks

The question (interviewer's verbatim phrasing):

If the GIL already serialises Python execution, why do I still need `threading.Lock` in my code?

The 30-second answer: The GIL protects the *interpreter's* internal state — refcounts and C-level structures. It does not protect *your* user-level invariants. A Python statement like `counter += 1` compiles to multiple bytecodes; between any two, the GIL can be dropped and another thread can run. Your "increment" can interleave with another thread's "increment" and lose an update.

`threading.Lock` is the user-level mutex that protects your invariants across multiple bytecode operations.

Step-by-step thought process: 1. Name the two layers: GIL is the runtime's lock; `Lock` is the user's lock. Different jobs, both necessary. 2. Show the race: thread A does `LOAD x`, gets preempted, thread B does `LOAD x`, `INC`, `STORE x` (x is now 1), thread A resumes with its old value, does `INC` to 1, `STORE x` (x is still 1). One increment lost. 3. The fix: wrap the read-modify-write in `with lock:`. Now between the threads, only one can be inside the critical section at a time. 4. Reinforce: the GIL gives you Python-VM safety; you give you Python-program safety. The first does not imply the second.

Edge cases to mention: - Some single-bytecode operations *are* atomic without a lock — `list.append`, `dict.__setitem__` with hashable keys, `set.add`. These are CPython-specific guarantees and may differ in PyPy or the free-threaded build. - `counter += 1` on a plain `int` is NOT atomic. Use `threading.Lock`, `multiprocessing.Value`, or `queue.Queue` instead. - `collections.Counter()` is also NOT thread-safe for in-place updates — same race as plain int counters.

What NOT to say: - "I don't need locks because of the GIL." — Wrong. Lost updates happen even with the GIL. - "The GIL is a `Lock`." — Different layer. The GIL is in C; `Lock` is in Python. - "All Python operations are atomic." — Most are not. Only single-bytecode ops are.

Common follow-up: "How would you make `counter += 1` thread-safe?" — Easiest: `with self.lock: self.counter += 1`. Or use `itertools.count()` — its `__next__` is atomic in CPython. Or use `multiprocessing.Value` with an explicit lock. Or use `queue.Queue` to serialise increments through a single consumer.

```
import threading

counter = 0
lock = threading.Lock()

def worker():
    global counter
    for _ in range(100_000):
        with lock:
            # protects the read-modify-write
            counter += 1

threads = [threading.Thread(target=worker) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
print(counter) # 400_000 – exact with lock; would be less without
```

Q10 · The GIL on `import` — Why Imports Are Special

Tags: Difficulty: Hard · Asked at: Cred, Goldman, Microsoft India · Topic: GIL Edge Cases

The question (interviewer's verbatim phrasing):

Are there cases where Python protects something with a separate lock instead of the GIL? Walk me through one.

The 30-second answer: Yes — the most famous is the import lock. CPython holds a per-module import lock (since 3.3, more granular than the old single import lock) during `import` to prevent two threads from initialising the same module simultaneously. The GIL alone is insufficient because the import process makes multiple bytecode calls (resolving the spec, executing the module body, registering into `sys.modules`) — the GIL is dropped between them. A second lock specifically serialises imports of the same module. The history is messy: pre-3.3 it was a single global import lock causing deadlocks; 3.3+ used per-module locks via `importlib._bootstrap.ModuleLock`.

Step-by-step thought process: 1. Name the lock — the import lock, used during `__import__`. State that it is separate from the GIL. 2. Explain why GIL is not enough — import is a multi-step process (find spec, load source, compile, execute module body, register). The GIL releases between bytecode ops in the body, so a second thread importing the same module concurrently would see a partially-initialised module. 3. The pre-3.3 design — a single global import lock — caused deadlocks when two threads imported modules that imported each other; circular import + threading = hang. 4. The 3.3 fix — per-module locks managed by `importlib._bootstrap` with deadlock detection; if thread A waits on module X locked by thread B who is waiting on module Y locked by thread A, the lock detects the cycle and breaks out with a partial module.

Edge cases to mention: - This is *the* reason `import` in a thread can hang in some legacy codebases — circular imports plus threads triggered the old single import lock to deadlock. - The `_thread.allocate_lock` primitive used by both the GIL machinery and the import lock is the same low-level lock type, but they are distinct lock instances. - `imp.acquire_lock()` and `imp.release_lock()` were the old user-facing handles; in 3.x they are deprecated and the import machinery manages locking internally.

What NOT to say: - "The GIL handles everything." — It does not handle import races; the import lock does. - "Imports are thread-safe automatically." — They are, but via a separate mechanism, not via the GIL. - "The import lock was removed in 3.x." — False. It became per-module; it did not go away.

Common follow-up: "Why does the import lock matter for asyncio?" — Lazy imports inside an async function can block the event loop if the import contends with another thread's import. The standard practice: do all imports at module top, not inside coroutines, to avoid blocking the event loop on import-lock contention.

```
import sys, importlib
# Per-module locks live on the module spec — visible via importlib internals
spec = importlib.util.find_spec("json")
print(spec.name)           # 'json'
print(spec.loader)         # SourceFileLoader for stdlib modules
# Import locks are managed by importlib._bootstrap.ModuleLock — not user-visible
```

Section B — GIL Implications · What It Means In Practice

This section is where candidates trip when they have the mechanism but not the implications. Round 2 interviewers do not test "what is the GIL"; they test "your scraper hits a wall at 4 threads, why, what would you change." The right answers come from the CPU-bound vs I/O-bound split, the one-core ceiling for pure Python, and an honest understanding of what C extensions buy you.

Q11 · CPU-Bound vs I/O-Bound — The Defining Split

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Flipkart, Cred, PhonePe · Topic: GIL Implications

The question (interviewer's verbatim phrasing):

I want to speed up a slow Python script. The first question I ask is whether it's CPU-bound or I/O-bound. Why does that distinction matter and how do I tell?

The 30-second answer: CPU-bound means the bottleneck is computation — the program is busy doing arithmetic, parsing, hashing, image transforms, ML inference in pure Python. I/O-bound means the bottleneck is waiting — waiting on disk, network, database, subprocess. The distinction matters because of the GIL: pure-Python CPU-bound work does NOT speed up with threads (the GIL prevents parallel bytecode); I/O-bound work DOES speed up with threads (the GIL releases on syscalls). For CPU-bound, you reach for `multiprocessing`, native extensions, or async-with-process-pool patterns. For I/O-bound, you reach for `threading` or `asyncio`. How to tell: profile. If `top` shows your Python process at 100% of one core, it is CPU-bound; if it shows 5% CPU and a long wall-clock time, it is I/O-bound.

Step-by-step thought process: 1. Lead with the definitions — what each "bound" means in operational terms. CPU = busy. I/O = waiting. 2. Connect to the GIL — pure Python CPU work serialises on the GIL, so threads do not help. I/O work releases the GIL on the syscall, so threads do help. 3. Map to the right primitive: CPU-bound → `multiprocessing.Pool`, `ProcessPoolExecutor`, or a native extension (numpy, numba, cython). I/O-bound → `threading`, `ThreadPoolExecutor`, or `asyncio`. 4. How to tell — profile. `cProfile` and `py-spy` show where time is spent; `top` and `htop` show whether CPU is the limit. If your `requests.get()` calls dominate the wall-clock time, you are I/O-bound.

Edge cases to mention: - Hybrid workloads exist — a scraper that fetches HTML (I/O) and then parses it with BeautifulSoup (CPU). Use `ThreadPoolExecutor` for fetch, `ProcessPoolExecutor` for parse, or `async + run_in_executor` for the CPU part. - "CPU-bound" in pandas can be misleading — pandas calls into NumPy which releases the GIL, so some pandas ops do benefit from threading. Vectorisation usually beats threading regardless. - Memory-bound is a third category — operations limited by memory bandwidth (large data movements). Threading does not help; multi-processing only helps if data partitions cleanly.

What NOT to say: - "Use threads for everything." — Wrong for CPU-bound; will not scale. - "Use multiprocessing for everything." — Wrong for I/O-bound; pickling overhead and process startup cost dominate. - "Async is always faster." — Async is faster for many-small-I/O patterns; for a few-large-I/O it is comparable to threading.

Common follow-up: "What does `time` show on the command line and how does it relate to this?" — `time my_script.py` prints `real`, `user`, `sys`. `real` is wall-clock. `user` is CPU time spent in user code; `sys` is in kernel. If `user` ≈ `real`, CPU-bound. If `real` >> `user + sys`, you are waiting on I/O — I/O-bound.

```
# CPU-bound: threads don't help
import time, threading
def fib(n):
    if n < 2: return n
    return fib(n-1) + fib(n-2)

start = time.perf_counter()
threads = [threading.Thread(target=fib, args=(30,)) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
print(f"CPU-bound across 4 threads: {time.perf_counter() - start:.2f}s")
# ~ same as running 4x sequentially
```

Q12 · Why Threads Do Not Speed Up CPU-Bound Code

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Cred · Topic: GIL Implications

The question (interviewer's verbatim phrasing):

Walk me through, mechanically, why running my CPU-heavy function in four threads gives me roughly the same wall-clock time as running it in one.

The 30-second answer: Because of the GIL. Each of those four threads has bytecode to execute, but only one thread holds the GIL at a time. The interpreter time-slices them — each gets a few milliseconds, then yields. Total work done equals one-thread's worth, divided among four threads who each pay an additional context-switch cost on every handoff. Net result: same wall-clock, sometimes slightly slower due to switching overhead. The only way to truly run four cores of Python bytecode in parallel is to run four processes (multiprocessing) or to use the 3.13 free-threaded build.

Step-by-step thought process: 1. The mechanism in one sentence — the GIL serialises bytecode execution; threads share the CPU one-at-a-time even on a 16-core machine. 2. Show the timing: thread A runs for 5ms, releases the GIL, thread B runs for 5ms, etc. The OS schedules them round-robin, but they never actually overlap on Python ops. 3. Add the overhead: each handoff costs microseconds for the GIL acquire/release, plus CPU cache thrash because a different thread now runs on a different core. So four threads doing CPU work can be measurably slower than one thread doing the same total work. 4. Land on the fix: multiprocessing for parallel CPU; or use a native extension (NumPy, scikit-learn) that releases the GIL during its computation; or migrate to the free-threaded 3.13t build for true parallelism in pure Python.

Edge cases to mention: - "Roughly the same" is not "exactly the same" — sometimes four threads is slightly faster (better cache pre-fetching) or slightly slower (GIL contention overhead). Both signals indicate "not parallel." - A workload that mixes Python and a GIL-releasing native call (`numpy.dot` , `hashlib.sha256` on big input) DOES scale across threads because the native portion runs in parallel.

- The asymmetry: cooperative releasing in the native code means N threads on N cores can run, but the moment they return to Python, they serialise on the GIL again.

What NOT to say: - "Threads are parallel." — They are concurrent; CPython threads are not parallel for pure-Python CPU work. - "It's about thread overhead." — The overhead is real but small; the GIL is the dominant cause. - "You need to use C extensions to fix it." — Multiprocessing also fixes it. C extensions are one of several escape hatches.

Common follow-up: *"Then why does Java's thread pool scale to all cores?"* — Java has no GIL. The JVM's GC handles thread-safety differently (tracing GC, fine-grained locks per object header), so multiple threads can execute Java bytecode in parallel on multiple cores. The GIL is a CPython-specific design choice; the JVM picked the other path.

```
import threading, time
def burn():
    n = 50_000_000
    while n > 0: n -= 1

# Single thread baseline
start = time.perf_counter()
burn()
print(f"1 thread: {time.perf_counter() - start:.2f}s")

# 4 threads – GIL serialises, no speedup
start = time.perf_counter()
threads = [threading.Thread(target=burn) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
print(f"4 threads: {time.perf_counter() - start:.2f}s") # ~4x, not 1x
```

Q13 · Why I/O-Bound Code Benefits From Threading

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Flipkart, Cred, Goldman · Topic: GIL Implications

The question (interviewer's verbatim phrasing):

Walk me through why a script that downloads 100 URLs runs 10x faster with 10 threads, even though the GIL exists.

The 30-second answer: Because every blocking I/O syscall releases the GIL. When `requests.get(url)` reaches the network read, the underlying `socket.recv` call wraps itself in `Py_BEGIN_ALLOW_THREADS`, releases the GIL, blocks on the kernel until data arrives, then reacquires the GIL. While that thread is blocked, other threads holding the GIL run their own Python code —

including making their own `requests.get` calls. With 10 threads, you have 10 concurrent network waits; the GIL only matters for the brief CPU work between waits.

Step-by-step thought process: 1. The release pattern: `socket.recv`, `socket.send`, `file.read`, `file.write`, `time.sleep`, `subprocess.communicate` — every one drops the GIL before the syscall, picks it up after. 2. The wait is on the kernel — the OS suspends the thread until the network response arrives. The CPU is idle for that thread; other threads use the CPU. 3. The cost model: 100 URLs at ~100ms each is 10 seconds sequential. With 10 concurrent threads, you compress those waits — the bottleneck is the slowest single fetch, not the sum. 4. Where it caps: too many threads incur OS context-switch overhead, file descriptor limits, and target-server rate limits. Typical sweet spot is 10–50 threads for HTTP fetching; beyond that, switch to async.

Edge cases to mention: - The speedup is bounded by the slowest individual I/O. If one URL is slow, threads do not help that one; they help the *aggregate*. - Memory grows with thread count — each thread has its own stack (typically 8MB on Linux); 1000 threads = 8GB of stack memory. - For massive concurrency (1000+ connections), threading hits OS limits — that is where asyncio wins, by handling concurrency without per-task OS threads.

What NOT to say: - "Threading is single-threaded." — Threading is multi-threaded; bytecode is single-threaded. Different layers. - "Threading is async." — Threading is OS-thread-based; async is event-loop-based with one thread by default. Different mechanism. - "Threads are free." — Each thread has a stack and bookkeeping overhead. Not free, but cheap enough for hundreds, not for tens of thousands.

Common follow-up: *"At what concurrency level should I switch from threading to asyncio?"* — Rule of thumb: under 100 concurrent I/O ops, threading is fine and simpler. From 100 to 10,000, asyncio scales much better — no per-task OS thread, lower memory, lower context-switch cost. Above 10,000 you usually need async plus tuning.

```
import threading, time, urllib.request

urls = ["https://httpbin.org/delay/1"] * 10 # each sleeps 1s server-side

# Sequential
start = time.perf_counter()
for url in urls: urllib.request.urlopen(url).read()
print(f"Sequential: {time.perf_counter() - start:.2f}s") # ~10s

# Threaded
start = time.perf_counter()
threads = [threading.Thread(target=lambda u=u: urllib.request.urlopen(u).read()) for u in urls]
for t in threads: t.start()
for t in threads: t.join()
print(f"10 threads: {time.perf_counter() - start:.2f}s") # ~1s
```

Q14 · The One-Core Ceiling — A Practitioner's Mental Model

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Cred · Topic: GIL Implications

The question (interviewer's verbatim phrasing):

What does "one core for pure Python" actually mean in practical engineering terms?

The 30-second answer: It means the maximum CPU usage of a single CPython process running pure Python code is one core's worth — typically shown as 100% in `top` on a multi-core machine. No matter how many threads you start, pure Python code shares that one core. To use a second core, you must spawn a second process (multiprocessing), or call into a native extension that releases the GIL, or run in the experimental free-threaded build. The ceiling is a CPython runtime property; it has nothing to do with how many physical cores the box has.

Step-by-step thought process: 1. State the ceiling clearly: one CPython process running pure Python = one core, regardless of physical cores. 2. The visual signal: `top` showing 100% CPU on a 16-core box means the process is fully using one core; another 1500% remain idle from that process's perspective. 3. The escape hatches: (a) spawn more processes — each is its own GIL, each can use a core; (b) call into NumPy / scikit-learn / hashlib whose native code releases the GIL; (c) use the 3.13t free-threaded build. 4. The economic implication: an EC2 instance with 16 cores running a single CPython web app on one process is paying for 15 idle cores. That is why production deployments use gunicorn with N workers, or uvicorn with N processes — to use the cores you are paying for.

Edge cases to mention: - The "one core" is not pinned to a specific physical core; the OS scheduler can move the thread between cores. From `top`'s perspective, you see one core's worth of work distributed across cores. - Native extensions partially break the ceiling — a NumPy call on a large array can use all cores for its duration; pure-Python code between calls returns to the ceiling. - On a single-core machine, the ceiling is meaningless; on a 1000-core machine, it is the whole story.

What NOT to say: - "Python uses all cores." — Pure Python in CPython does not. Multiprocessing does; the free-threaded build does. - "Threading uses multiple cores." — In CPython for pure Python: no. For I/O: yes, because waiting does not need a core. - "The OS limits you to one core." — The runtime limits you, not the OS. The OS would happily give you all cores.

Common follow-up: *"How do you decide between threads and processes for a web server?"* — gunicorn / uvicorn defaults: spawn N worker processes (one per core) for CPU parallelism; each worker may use a thread pool for blocking I/O within a request. Async frameworks (FastAPI on uvicorn) handle I/O concurrency cooperatively within each process. Combine processes for CPU + async for I/O = production setup.

```
import os, multiprocessing as mp
print(f"Cores available: {os.cpu_count()}")
print(f"This CPython process can only fully use 1 of them in pure Python.")
print(f"To use all {os.cpu_count()}, spawn {os.cpu_count()} processes.")
```

Q15 · GIL Contention In C Extensions — When It Goes Wrong

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Microsoft India, Walmart Labs · Topic: GIL Internals

The question (interviewer's verbatim phrasing):

What does "GIL contention in a C extension" actually look like, and how would you diagnose it?

The 30-second answer: GIL contention in a C extension means the extension holds the GIL longer than necessary — typically by failing to wrap its computational core with `Py_BEGIN_ALLOW_THREADS` / `Py_END_ALLOW_THREADS`. The symptom is that workloads using that extension do not scale with threading even though they look like they should. Diagnosis: profile under multi-thread load with `py-spy --threads` or `perf top`, look for the C function holding the GIL during long computations. Fix: patch the extension to release the GIL around the computation, or wrap calls in `concurrent.futures.ProcessPoolExecutor` to bypass the GIL entirely.

Step-by-step thought process: 1. The mechanism: a C extension function inherits the GIL from the caller. If it does a 200ms hash computation without releasing, no other Python thread runs for those 200ms. 2. The protocol — `Py_BEGIN_ALLOW_THREADS` releases the GIL; the C code does its work without touching any Python objects; `Py_END_ALLOW_THREADS` reacquires. Forgetting either side either holds the GIL too long or crashes the interpreter. 3. The discovery: thread-saturating workload but only one core lit up on `htop`. Run `py-spy top --pid PID --threads`; the dominant function is your offender. 4. The fix path — file an issue / patch upstream; if not possible, isolate the extension call to a subprocess via `ProcessPoolExecutor` so its GIL hold does not block the parent.

Edge cases to mention: - NumPy, hashlib, lzma, bz2, ssl, sqlite3 — all release the GIL around their heavy work. Good citizens. - Older or less-maintained extensions (some image libraries, some XML parsers) often forget to release. They are why your image-resize workload is single-threaded. - The release is asymmetric: the C extension can release during a long computation but **MUST** reacquire before touching any `PyObject*`. Forgetting this is the source of segfaults.

What NOT to say: - "C extensions are always faster." — They are faster per-op but can be GIL-contention bottlenecks for multi-threaded workloads. - "The GIL never releases in C code." — It does, when the extension cooperates. - "Profilers cannot see this." — They can. `py-spy` and `perf` will both show the function holding the GIL.

Common follow-up: "How would you find out if PIL's `Image.resize` releases the GIL?" — Quickest: run two threads each doing 100 resizes; measure wall-clock time. If it is roughly the same as one thread doing 200, GIL is not released; if it is half, GIL is released. Confirm by reading the C source — look for `Py_BEGIN_ALLOW_THREADS` around the inner loop.

```
# Demonstrate: NumPy releases the GIL, pure-Python does not
import threading, time
import numpy as np
def numpy_work():
    a = np.random.rand(2000, 2000)
    np.linalg.svd(a) # releases the GIL during BLAS call

start = time.perf_counter()
threads = [threading.Thread(target=numpy_work) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
print(f"4 NumPy threads: {time.perf_counter() - start:.2f}s")
# Substantially less than 4x because GIL is released during SVD
```

Section C — The `threading` Module

This section is the API surface — `Thread`, `Lock`, `RLock`, `Semaphore`, `Event`, `Condition`, `Barrier`, `Queue`, and the patterns that hold them together. The interviews here are concrete: "implement a producer-consumer with two threads," "explain when you would use an `RLock`," "show me a deadlock and how to detect it." The mechanics matter more than the philosophy.

Q16 · `Thread` Basics — Construct, Start, Join

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Accenture, Cognizant · Topic: threading

The question (interviewer's verbatim phrasing):

Walk me through the lifecycle of a `threading.Thread`. What does `start` do, what does `join` do, and what happens if I forget `join`?

The 30-second answer: `threading.Thread(target=fn, args=(...))` constructs a thread object — no OS thread is created yet. `t.start()` creates the OS thread and begins running `fn` in it; the call returns immediately. `t.join()` blocks the caller until the thread finishes. If you forget `join`, the parent thread continues without waiting; the child runs to completion in the background, and

if it is a non-daemon thread, it keeps the process alive even after `main` returns. Forgetting `join` is fine for fire-and-forget, dangerous when you need the child's result or want clean shutdown.

Step-by-step thought process: 1. The constructor is cheap — just bookkeeping; no OS thread until `start()`. 2. `start()` is one-shot — calling it twice raises `RuntimeError`. After it finishes, the thread is "done"; you cannot restart. 3. `join()` is the synchronisation point — the caller waits. With a `timeout=` arg, you can give up after N seconds. Use `t.is_alive()` after a timed join to check whether the child finished. 4. Daemon vs non-daemon decides shutdown behavior: a daemon thread is killed when the main thread exits; a non-daemon thread keeps the process alive. Default is non-daemon; set `t.daemon = True` before `start()` if you want fire-and-forget.

Edge cases to mention: - `join()` does not return a value — the target function's return value is lost. To capture it, use `concurrent.futures.ThreadPoolExecutor` and `future.result()`. - A thread that raises an unhandled exception logs it and dies — the exception is NOT propagated to the parent. You need to catch it inside the target. - `t.run()` exists but you almost never call it directly — it bypasses the OS thread and runs the body in the calling thread.

What NOT to say: - "`Thread.start` runs the function." — It schedules the function to run in a new OS thread; the calling thread continues immediately. - "`join` returns the result." — It does not; the target's return value is dropped. Use `ThreadPoolExecutor` for return values. - "I always need to join." — Daemon threads are fire-and-forget and require no join.

Common follow-up: "How would you capture the return value of a threaded function?" — Either use `ThreadPoolExecutor.submit` and call `.result()` on the returned future, or pass a mutable shared object (`list` or `Queue`) into the target and have it write its result there. Avoid global state for return values — it does not scale past one thread.

```
import threading, time

def worker(name, secs):
    print(f"{name} starting")
    time.sleep(secs)
    print(f"{name} done")

t = threading.Thread(target=worker, args=("scraper", 2))
t.start()           # OS thread starts; returns immediately
print("main continues")
t.join()           # blocks until worker done
print("worker joined")
```

Q17 · Daemon Threads — When and Why

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, JPMC · Topic: threading

The question (interviewer's verbatim phrasing):

What is a daemon thread and when would you use one?

The 30-second answer: A daemon thread is one that runs in the background and is automatically killed when all non-daemon threads (including the main thread) exit. Use it for fire-and-forget work that should not prevent process shutdown — log shippers, metric reporters, heartbeat senders, background cleanup. Set `t.daemon = True` before `t.start()`. Once started, you cannot change daemon status. The trade-off: daemon threads are killed abruptly with no chance to finish or run cleanup — never use daemon for work whose interruption would corrupt state.

Step-by-step thought process: 1. Default behavior: non-daemon. The process waits for them. Set `daemon=True` to opt out. 2. The use case: background work that supports the main task — flush logs every N seconds, send health pings, prune in-memory caches. If the app exits, you do not care about that work continuing. 3. The danger: a daemon writing to disk can be killed mid-write, leaving a partial file. A daemon holding a database connection is force-disconnected. If interruption matters, do not use daemon. 4. The alternative: a non-daemon worker with an explicit shutdown signal (`threading.Event`) and a graceful exit. More code but safer for state.

Edge cases to mention: - Daemon status must be set before `start()` — after, it raises `RuntimeError`. - Sub-threads inherit daemon status from their parent unless explicitly set. - The main thread is always non-daemon — that is what keeps the process alive while user code runs.

What NOT to say: - "Daemon threads are faster." — No performance difference; only shutdown behavior differs. - "Daemon threads survive Ctrl-C." — They do not; SIGINT kills the main thread which kills daemons. - "Daemon means safe to ignore." — It means abruptly-killable; not the same as safe.

Common follow-up: *"What is the difference between a daemon thread and a daemon process?"* — Daemon thread: thread inside a process that auto-terminates on process exit. Daemon process: long-running background process (Unix daemonised), entirely separate from the terminal — handled by `daemon` tools, `systemd`, or `os.fork`. Different scopes, different mechanisms.

```
import threading, time

def background_pinger():
    while True:
        print("ping", time.time())
        time.sleep(0.5)

t = threading.Thread(target=background_pinger, daemon=True)
t.start()
time.sleep(2)
print("main exiting — daemon pinger dies with us")
# Pinger printed ~4 times, then the process exits
```

Q18 · `threading.local` — Per-Thread State

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, JPMC, Goldman · Topic: threading

The question (interviewer's verbatim phrasing):

What is `threading.local` and when would you actually use it in a Python application?

The 30-second answer: `threading.local()` returns an object whose attributes are per-thread — each thread sees its own values, isolated from other threads. Use it for state that is logically global but should not be shared across threads: a per-thread database connection, a per-thread request context, a per-thread logging buffer. The classic use case is web frameworks — Django's request object is thread-local so each request handler sees its own request without passing it through every function. You assign to it once per thread (typically on first access) and read from it elsewhere.

Step-by-step thought process: 1. The semantic: `local = threading.local(); local.x = "A"` in thread 1 does NOT make `local.x` visible in thread 2. Each thread has its own attribute namespace on the same object. 2. The implementation: under the hood, `threading.local` uses thread identity (the OS thread ID) as a key into a per-instance dict. Reads and writes hit the right slot transparently. 3. The use case: database connections in a thread pool — each thread maintains one connection, and you do not want to thread it through every function signature. `local.db = create_connection()` once per thread; read `local.db` everywhere. 4. The trap: `threading.local` does NOT work with asyncio — coroutines run on the same OS thread, so they all see the same value. For async per-task state, use `contextvars.ContextVar`.

Edge cases to mention: - A thread that exits leaves its thread-local data eligible for cleanup, but only the next time the local object is accessed or the interpreter cleans up. - Subclassing `threading.local` and overriding `__init__` runs `__init__` once per thread on first access — handy for lazy initialisation. - `contextvars.ContextVar` is the async-safe replacement and also works in threaded code; in 2026 codebases prefer `ContextVar` for new code.

What NOT to say: - "`threading.local` is a regular dict." — It is not; per-thread isolation is the whole point. - "Use `threading.local` for asyncio." — Do not; use `contextvars.ContextVar`. - "`threading.local` is automatically cleaned up." — Depends on access pattern and interpreter shutdown; do not rely on it for resource lifecycle.

Common follow-up: "Why does Django use `threading.local` for the request object?" — Because the per-request state (current user, query parameters, transaction state) needs to be globally accessible without being passed through every function call. Pre-async Django runs each request in a thread; `threading.local` gives each thread its own slot. In async Django (3.0+), this moved to `contextvars`.

```
import threading

local = threading.local()

def worker(name):
    local.value = name          # per-thread attribute
    print(f"{name} sees {local.value}")

t1 = threading.Thread(target=worker, args=("A",))
t2 = threading.Thread(target=worker, args=("B",))
t1.start(); t2.start(); t1.join(); t2.join()
# A sees A, B sees B — never crosses over
```

Q19 · `threading.Lock` — The Basic Mutex

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Cred, Walmart Labs · Topic: threading

The question (interviewer's verbatim phrasing):

Walk me through how `threading.Lock` works. What are the three ways to use it?

The 30-second answer: `threading.Lock()` creates a non-reentrant mutex. Three ways to use it: (1) `lock.acquire()` then `lock.release()` — must always release, usually in a `try/finally`; (2) `with lock:` — context manager handles acquire and release including on exception; (3) `lock.acquire(timeout=N)` returns True if acquired in time, False otherwise — for "give up after N seconds." The lock has two states, locked and unlocked. A thread that tries to acquire an already-locked Lock blocks until released. A thread that calls `release` on a Lock it did not acquire raises `RuntimeError`. A thread that acquires the same Lock twice deadlocks against itself — use `RLock` if you need reentrancy.

Step-by-step thought process: 1. Idiomatic use: `with lock:` everywhere. The context manager guarantees release on exception. 2. Manual use: `try: lock.acquire(); ...; finally: lock.release()`. Same effect, more code. Use when you need the acquire/release at different scopes. 3. Timed use: `if lock.acquire(timeout=5.0): try: ... finally: lock.release()`. Useful for "do not wait forever" patterns. 4. The reentrancy trap: a function that holds a lock and calls another function that tries to acquire the same lock deadlocks. Either restructure the code or use `RLock`.

Edge cases to mention: - `lock.locked()` tells you whether it is currently held — useful in tests but never as a "check then acquire" pattern (race condition). - `Lock` is not associated with any specific thread — any thread can call `release`. That makes it cheap but unforgiving. - A deadlock between

two locks is the classic threading bug: thread A holds L1 wants L2, thread B holds L2 wants L1. Avoid by always acquiring multiple locks in a globally consistent order.

What NOT to say: - " `Lock` tracks which thread acquired it." — It does not; that is `RLock`. - " `with lock:` is slower than `acquire/release` ." — Same performance; just safer. - "Locks are free." — Cheap but not free; acquiring an uncontended lock is ~50ns, contended can be microseconds.

Common follow-up: "Show me a deadlock and how to detect it." — Two threads, two locks, acquired in opposite order. Detection in Python: `py-spy dump` shows each thread's stack; if both are stuck on `acquire`, you have a deadlock. Prevention: always acquire locks in a single canonical order, or use `try_acquire` with timeouts.

```
import threading

balance = {"alice": 100, "bob": 50}
lock = threading.Lock()

def transfer(src, dst, amount):
    with lock:                # safe across threads
        balance[src] -= amount
        balance[dst] += amount

threads = [threading.Thread(target=transfer, args=("alice", "bob", 10))
           for _ in range(5)]
for t in threads: t.start()
for t in threads: t.join()
print(balance)  # alice 50, bob 100 – consistent
```

Q20 · `RLock` — When Reentrancy Is Needed

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JPMC, Goldman · Topic: threading

The question (interviewer's verbatim phrasing):

What is the difference between `Lock` and `RLock` ? When would you use `RLock` ?

The 30-second answer: `Lock` is non-reentrant — a thread that tries to acquire it twice deadlocks. `RLock` (reentrant lock) tracks which thread holds it and how many times; the same thread can acquire it any number of times without blocking, but must call `release` the same number of times before it actually unlocks. Use `RLock` when a method protected by a lock calls another method that also wants the same lock — common in OO code where multiple class methods share invariants but call each other. The trade-off: `RLock` is slightly slower due to the bookkeeping, and a thread holding an `RLock` reentrantly hides bugs (you cannot tell from inspection how many releases are needed).

Step-by-step thought process: 1. The reentrancy semantic: `RLock.acquire()` called by the same thread increments an internal counter; `release` decrements. Lock is only free when counter hits zero. 2. The use case: a class with `def deposit(self):` and `def transfer(self):` where `transfer` internally calls `deposit`. Both are guarded by `self.lock`. With `Lock`, `transfer` deadlocks the moment it tries to enter `deposit`. With `RLock`, it works because the same thread already holds the lock. 3. The alternative: refactor the code so the inner method does not take the lock — call into an unlocked helper that assumes the caller holds the lock. Cleaner but more code. 4. The bug-hiding trap: `RLock` makes lock-re-entry silent. Code that "happens to" recurse through the lock works but is fragile. Prefer explicit locking discipline.

Edge cases to mention: - `RLock` is per-thread — only the holding thread can re-acquire. Another thread still blocks normally. - `RLock` cannot be released by a different thread than the one that acquired it (unlike `Lock`). - The `Condition` primitive uses `RLock` internally because the `wait()` mechanism needs to re-enter the lock around the callback.

What NOT to say: - "`RLock` is always safer than `Lock`." — It hides reentrant code paths, which can mask bugs. - "`RLock` works for any thread." — Reentrancy is per-thread; other threads still block. - "`Lock` is deprecated in favour of `RLock`." — Both have their uses; `Lock` is fine for single-method protection.

Common follow-up: "Show me a case where `Lock` fails but `RLock` works." — A class method `withdraw` that calls `log_event` which also takes the same lock. With `Lock`, the recursive acquire deadlocks; with `RLock`, the same thread sees its own counter and proceeds.

```
import threading

class Account:
    def __init__(self):
        self.balance = 0
        self.lock = threading.RLock()    # reentrant

    def deposit(self, amount):
        with self.lock:
            self.balance += amount
            self.log(f"deposit {amount}") # also takes self.lock

    def log(self, msg):
        with self.lock:                  # safe with RLock
            print(msg)

a = Account()
a.deposit(100)    # works; would deadlock with plain Lock
```

Q21 · Semaphore and BoundedSemaphore

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Goldman, JPMC · Topic: threading

The question (interviewer's verbatim phrasing):

When would you reach for `Semaphore` instead of `Lock` ?

The 30-second answer: A `Semaphore` allows N threads to hold it simultaneously, not just one. Use it to cap concurrency — limit to N concurrent database connections, N concurrent HTTP fetches, N concurrent file uploads. `Semaphore(5)` starts with 5 permits; each `acquire` decrements, each `release` increments. `BoundedSemaphore(5)` is the same but raises `ValueError` if you over-release — useful for catching the "I released too many times" bug. `Lock` is the special case `Semaphore(1)` but cheaper and more explicit about intent.

Step-by-step thought process: 1. Semantic: a counter that allows up to N concurrent holders. Acquire blocks when the count is zero; release wakes one waiter. 2. The canonical use case: rate limiting. "No more than 10 concurrent HTTP requests" — wrap each request in a Semaphore acquire/release; the 11th waits until one finishes. 3. `BoundedSemaphore` adds a check that you never release more than you acquired; helps catch the "release-without-acquire" bug. Prefer it in new code. 4. The pattern: `with sem:` works because Semaphore implements the context manager protocol. Cleaner than manual acquire/release.

Edge cases to mention: - Semaphore is not thread-aware — any thread can release. `BoundedSemaphore` does not check identity, only the count cap. - `Semaphore(0)` is a degenerate case — no thread can acquire until something releases. Sometimes used as a signal. - For coroutine-level concurrency limits, use `asyncio.Semaphore` — same idea, async-compatible.

What NOT to say: - "Semaphore is just a fancy Lock." — Lock is the N=1 case; Semaphore is the general form. - "Use BoundedSemaphore everywhere — it's strictly better." — Slightly more overhead; in tight loops, plain Semaphore may matter. - "Semaphores are for synchronisation." — They are for *limiting concurrency*. Synchronisation is more general — that is what Condition is for.

Common follow-up: "How would you cap your HTTP scraper to 10 concurrent connections?" — Wrap each fetch in a `Semaphore(10)`. The 11th thread blocks on acquire until one of the 10 returns and releases. Same pattern in asyncio with `asyncio.Semaphore`.

```
import threading, time, random

sem = threading.BoundedSemaphore(3) # max 3 concurrent

def task(i):
    with sem:
        print(f"task {i} running")
        time.sleep(random.uniform(0.5, 1.0))
        print(f"task {i} done")

threads = [threading.Thread(target=task, args=(i,)) for i in range(10)]
for t in threads: t.start()
for t in threads: t.join()
# Never more than 3 concurrent prints of "running"
```

Q22 · Event — A One-Way Signal

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, JPMC · Topic: threading

The question (interviewer's verbatim phrasing):

What is `threading.Event` and when would you use it instead of a `Lock` ?

The 30-second answer: `Event` is a boolean flag that threads wait on. `event.set()` flips it to True; `event.clear()` flips back to False. `event.wait()` blocks until the flag is True; if it is already True, it returns immediately. Use Event for one-way signals — "the system is ready," "shutdown requested," "config reloaded." It is not a lock — it does not protect data; it coordinates timing. Multiple waiters all wake up when `set()` is called; there is no "wake one" mode like with `Condition`.

Step-by-step thought process: 1. Semantic: Event is a sticky boolean. Once set, it stays set until cleared. Multiple waiters all release when set. 2. The use case: graceful shutdown. Main thread holds an Event; worker threads check `event.is_set()` periodically and exit when True. Main thread does `event.set()` to signal all workers at once. 3. The `wait(timeout=N)` variant returns True if event got set within N seconds, False on timeout. Useful for "wait for ready, but give up if it takes too long." 4. The non-pattern: do not use Event to protect data. It does not guarantee mutual exclusion. Use `Lock` for protection, Event for signalling.

Edge cases to mention: - A late waiter on an already-set Event returns immediately — that is feature, not bug. Lets you "missed the signal but still know it happened." - `clear()` plus `set()` is racy — a waiter may see the cleared state and block. Use `Condition` if you need clear-then-resignal semantics. - Event is not auto-resetting like Win32's auto-reset events; once set, it stays set until you clear.

What NOT to say: - "Event is a lock." — It is not. It is a flag. - "Event waits for one thread at a time." — All waiters release on set. - "Event protects data." — It does not. It signals.

Common follow-up: "How would you implement graceful shutdown of a pool of worker threads?" —

Create a `threading.Event` named `shutdown_flag`. Workers loop checking `if shutdown_flag.is_set(): return`. Main thread, on SIGTERM, calls `shutdown_flag.set()` and then joins each worker.

```
import threading, time

shutdown = threading.Event()

def worker(name):
    while not shutdown.is_set():
        print(f"{name} working")
        time.sleep(0.5)
    print(f"{name} stopping cleanly")

t = threading.Thread(target=worker, args=("A",))
t.start()
time.sleep(2)
shutdown.set()      # signal all workers
t.join()
```

Q23 · `Condition` — Predicate-Based Waiting

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Microsoft India · Topic: threading

The question (interviewer's verbatim phrasing):

What is `threading.Condition` and how is it different from `Event` ?

The 30-second answer: `Condition` is a lock plus a wait queue, used for the "wait until predicate is true" pattern. `Event` is binary (set or clear); `Condition` lets you wait on arbitrary predicates against shared state. The protocol: acquire the lock, check the predicate; if false, call `cond.wait()` which atomically releases the lock and blocks; another thread that updates the state calls `cond.notify()` (one waiter) or `cond.notify_all()` (all waiters) while holding the lock. The waiter wakes up, reacquires the lock, rechecks the predicate (always in a `while` loop, not `if`), and proceeds when true.

Step-by-step thought process: 1. The use case: producer-consumer where the consumer waits for "queue is non-empty." A simple Event cannot do this cleanly — the consumer needs to recheck after waking because another consumer might have already taken the item. 2. The protocol: always `with`

`cond: while not predicate: cond.wait()` . The `while` (not `if`) handles spurious wakeups and the multi-consumer race. 3. The producer side: `with cond: state.append(x); cond.notify()` . Must hold the lock while modifying state and while calling notify. 4. The `notify_all` variant: wake every waiter. Use when the state change might satisfy multiple predicates. The default `notify()` wakes just one — efficient for single-consumer-per-item patterns.

Edge cases to mention: - Spurious wakeups are possible — `cond.wait()` can return even without a `notify` . Always re-check the predicate in a `while` loop. - `Condition()` defaults to using its own internal `RLock` ; pass one in if you want to share a lock with other code. - `wait(timeout=N)` returns True if a notify happened within N seconds, False on timeout.

What NOT to say: - "Use `if` to check the predicate." — Use `while` ; spurious wakeups will bite. - "`Condition` is just a fancier Event." — Condition supports predicates and per-state waiters; Event does not. - "`notify` wakes all waiters." — `notify` wakes one; `notify_all` wakes all.

Common follow-up: "Show me a bounded buffer using `Condition` ." — Two conditions on the same lock — `not_full` for producers, `not_empty` for consumers. Producer waits on `not_full` , notifies `not_empty` after append. Consumer waits on `not_empty` , notifies `not_full` after pop. Standard textbook pattern.

```
import threading, time, collections

buf = collections.deque()
cond = threading.Condition()

def producer():
    for i in range(5):
        with cond:
            buf.append(i)
            cond.notify()          # wake one consumer
            time.sleep(0.1)

def consumer(name):
    while True:
        with cond:
            while not buf:         # while, not if – handles spurious wakeup
                cond.wait()
            item = buf.popleft()
            print(f'{name} got {item}')

c = threading.Thread(target=consumer, args=("C1",), daemon=True)
c.start()
producer()
time.sleep(0.5)
```

Q24 · Barrier — N-Way Rendezvous

Tags: Difficulty: Medium · Asked at: Goldman, JPMC, Microsoft India · Topic: threading

The question (interviewer's verbatim phrasing):

What is `threading.Barrier` and when would you use it?

The 30-second answer: A `Barrier(n)` is a synchronisation point where N threads wait for each other; once all N have called `wait()`, they all release together. Use it for "phase-based" parallel algorithms — all threads complete phase 1, then all start phase 2 together. Common in scientific computing, simulations, and tests that want to maximise contention by lining up all threads at the same moment. After `n_waiters == n`, the barrier resets and can be used for another round. Set `parties` to N at construction; pass an optional `action` callable to run once on the final waiter.

Step-by-step thought process: 1. The semantic: a counted rendezvous. `Barrier(3).wait()` blocks until 3 threads have called it; then all 3 proceed. 2. The use case: phase-based computation. Threads compute phase 1, hit barrier, then phase 2 begins atomically — guarantees that no thread starts phase 2 until phase 1 globally completes. 3. The action arg: `Barrier(n, action=fn)` runs `fn()` exactly once when the barrier releases — useful for "the last thread does the aggregation before all unblock." 4. The error path: if a waiting thread is interrupted or `Barrier.abort()` is called, all waiters get `BrokenBarrierError`. Reusability requires `reset()`.

Edge cases to mention: - `Barrier` is reusable — after release, it resets to zero waiters. Same instance for many phases. - `wait(timeout=N)` returns an integer (which waiter you were) or raises `BrokenBarrierError` on timeout/abort. - For "wait for K out of N" semantics, use Semaphore arithmetic; Barrier is strictly all-N-or-bust.

What NOT to say: - "Barrier is for one-time use." — It is reusable. - "Barrier replaces Event." — Different semantic; Barrier is N-way rendezvous. - "Barrier protects data." — It does not. It coordinates timing.

Common follow-up: "How would you use Barrier in a benchmark to make threads start at the same moment?" — Construct `Barrier(N)` for the N worker threads plus the main thread (so N+1 total). Each worker does setup, calls `barrier.wait()`, then runs its measurement. Main thread calls `barrier.wait()` last, which releases all workers simultaneously — eliminates startup-time skew.

```
import threading, time

start_barrier = threading.Barrier(4)

def runner(i):
    print(f"{i} preparing")
    start_barrier.wait()          # wait for all to be ready
    print(f"{i} go!")

threads = [threading.Thread(target=runner, args=(i,)) for i in range(4)]
for t in threads: t.start()
for t in threads: t.join()
# All 4 "go!" prints happen near-simultaneously
```

Q25 · `queue.Queue` — Thread-Safe Producer/Consumer

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Flipkart, Cred, JPMC · Topic: threading

The question (interviewer's verbatim phrasing):

What is `queue.Queue` and why is it preferred over a shared list with a lock?

The 30-second answer: `queue.Queue` is a thread-safe FIFO queue with built-in blocking semantics. `q.put(item)` adds; `q.get()` removes and blocks if empty; `q.put_nowait` and `q.get_nowait` raise `Empty` / `Full` instead of blocking. It uses an internal lock plus two conditions (not-empty for getters, not-full for putters) and handles all the edge cases — multiple producers, multiple consumers, max size, shutdown. Prefer it over a `list + Lock` because it has the blocking semantics built in and avoids the busy-wait patterns of "check the lock, sleep, check again."

Step-by-step thought process: 1. Operations: `put`, `get`, `put_nowait`, `get_nowait`, `task_done`, `join`. The first four are the data path; `task_done` / `join` are for "wait for all queued work to finish." 2. The `maxsize` parameter caps the queue — `Queue(maxsize=10)` blocks producers when 10 items are buffered. Backpressure baked in. 3. Three variants: `Queue` (FIFO), `LifoQueue` (stack), `PriorityQueue` (min-heap with `put((priority, item))`). Same threading semantics, different ordering. 4. The `task_done` / `join` pattern: each consumer calls `task_done()` after processing; the producer (or main thread) calls `q.join()` to wait until all enqueued items are marked done. Useful for "shut down only after all work is processed."

Edge cases to mention: - `Queue` is for *thread* communication, not process communication — `multiprocessing.Queue` is the process-safe analog with different implementation. - A consumer that never calls `task_done` causes `join` to hang forever — easy to forget. - For high-throughput message passing, `collections.deque` with a `Lock` can be faster than `Queue` for some workloads — but you give up the blocking and `task_done` semantics.

What NOT to say: - "Use `list` instead of `Queue` for performance." — In most cases, `Queue` is fast enough and dramatically safer. - "Queue is the same as `asyncio.Queue`." — Different module, different semantics. `Queue` is sync-thread-safe; `asyncio.Queue` is coroutine-safe. - "Queue protects against deadlock." — It avoids one class of deadlock (producer-consumer signalling), but you can still deadlock if you mismanage locks elsewhere.

Common follow-up: "What is the difference between `Queue.put` blocking vs `put_nowait`?" — `put` blocks if the queue is at maxsize; useful for backpressure. `put_nowait` raises `queue.Full` immediately; useful when you have a fallback (drop, log, retry).

```
import threading, queue, time

q = queue.Queue(maxsize=10)

def producer():
    for i in range(20):
        q.put(i)                # blocks if queue full
        q.put(None)            # sentinel for end

def consumer():
    while True:
        item = q.get()
        if item is None:
            q.task_done()
            return
        print(f"got {item}")
        q.task_done()

t1 = threading.Thread(target=producer)
t2 = threading.Thread(target=consumer)
t1.start(); t2.start(); t1.join(); t2.join()
```

Section D — The `multiprocessing` Module

This section is the "I actually want multiple cores" answer. `multiprocessing` spawns OS processes, each with its own Python interpreter and its own GIL — so the GIL ceases to be a parallelism barrier. The trade-off is process startup cost, IPC overhead, and the pickling restriction on what can be sent between processes. Indian backend panels at Razorpay, Cred and Swiggy ask the fork-vs-spawn question often; data panels at GCCs use it to test understanding of `Pool` and `Manager`.

Q26 · `Process` Basics — When and Why

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Goldman, JPMC · Topic: multiprocessing

The question (interviewer's verbatim phrasing):

When would you reach for `multiprocessing.Process` over `threading.Thread` ?

The 30-second answer: When the work is CPU-bound in pure Python. Each `Process` runs in a separate OS process, each with its own Python interpreter, each with its own GIL — so N processes can saturate N cores doing pure-Python compute. The trade-offs: startup is slow (tens to hundreds of milliseconds per process vs microseconds for a thread); memory is duplicated (each process has its own heap, no shared mutable state); IPC overhead via pipes or queues taxes data movement (objects must pickle on the way through). Use `Process` for compute, `Thread` for I/O.

Step-by-step thought process: 1. Mental model: a process is a full OS-level isolated runtime. Threads share memory and the GIL; processes share neither. 2. The cost calculus: process startup is slow because it must boot an interpreter, import modules, optionally fork or spawn. Threads start in microseconds. For short tasks, the startup cost dominates. 3. The API: `Process(target=fn, args=...)`, `p.start()`, `p.join()`. Identical to `Thread`; semantics differ in isolation. 4. The communication path: `multiprocessing.Queue`, `Pipe`, `Manager` for shared state. Whatever you send must pickle.

Edge cases to mention: - On Linux, processes default to `fork` start method — child inherits a copy-on-write copy of parent memory. Fast but inherits open file descriptors and signal handlers. - On macOS (default since 3.8) and Windows, the default is `spawn` — child starts fresh, re-imports the module. Slower but cleaner. - A child process that crashes does not crash the parent; check `p.exitcode` after join.

What NOT to say: - "Multiprocessing is always slower than threading." — For CPU work, multiprocessing is dramatically faster. - "Processes share memory." — They do not by default. Use `Manager` or `shared_memory` for shared state. - "Process is just like Thread but bigger." — Different isolation model, different startup cost, different IPC needs.

Common follow-up: *"What is the overhead of starting a process in 2026 Python?"* — Tens of ms on Linux with `fork`; hundreds of ms on macOS/Windows with `spawn` (interpreter boot + re-import). For workloads under ~1 second per task, that overhead dominates; for longer tasks, it amortises to nothing.

```
import multiprocessing as mp, time

def cpu_burn(n):
    while n > 0: n -= 1

if __name__ == "__main__":
    # required on spawn platforms
    start = time.perf_counter()
    procs = [mp.Process(target=cpu_burn, args=(50_000_000,)) for _ in range(4)]
    for p in procs: p.start()
    for p in procs: p.join()
    print(f"4 processes: {time.perf_counter() - start:.2f}s")
    # ~ 1/4 the time of running 4x sequentially – true parallelism
```

Q27 · `Pool.map` and `Pool.apply_async`

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Goldman, JPMC, Walmart Labs · Topic: multiprocessing

The question (interviewer's verbatim phrasing):

Walk me through `multiprocessing.Pool`. What is the difference between `map`, `apply_async`, and `imap`?

The 30-second answer: `Pool(processes=N)` creates a fixed pool of N worker processes. `pool.map(fn, iterable)` is the simplest: applies `fn` to each item, returns a list of results in order, blocks until all done. `pool.apply_async(fn, args)` submits one task, returns an `AsyncResult` immediately — call `.get()` later to fetch the result; useful for non-uniform work or when you want to dispatch and continue. `pool.imap(fn, iterable)` is a lazy iterator version of `map` — yields results as they finish, lower memory for large iterables. `imap_unordered` yields whichever finishes first regardless of input order — best for max throughput when order does not matter.

Step-by-step thought process: 1. Mental model: a thread pool, but with processes. Pre-spawned workers wait for work; you dispatch tasks; results come back via the pool's internal queues. 2. `map(fn, iterable)` is the workhorse — clean, ordered, blocks. Use when you have a fixed list of inputs and want the full result. 3. `apply_async(fn, args)` is the granular form — submit, get a future, fetch later. Use when tasks vary or you want to interleave dispatch with other work. 4. `imap` / `imap_unordered` are streaming variants — yield results as they arrive. `imap_unordered` is the fastest for "process each result as it lands, don't care about order."

Edge cases to mention: - `Pool` should be used as a context manager (`with mp.Pool() as p:`) — guarantees workers are cleaned up even on exception. - The `chunksize` parameter on `map` / `imap` controls how many items each worker gets per dispatch — too small wastes IPC overhead, too

large unbalances load. - A pool worker that crashes is auto-replaced — but the input task is lost; design idempotently.

What NOT to say: - "`map` and `apply_async` are the same." — `map` blocks until all done; `apply_async` returns immediately with a future. - "Pool always speeds up code." — IPC overhead can dominate for tiny tasks. - "Pool uses threads internally." — It uses processes; that is the whole point.

Common follow-up: "What is the right `processes` count for a Pool?" — Default is `os.cpu_count()`. For pure CPU work, `cpu_count()` is optimal. For mixed CPU+I/O, `cpu_count() + 4` or so to overlap waits. Empirical: benchmark on your workload.

```
import multiprocessing as mp

def square(x): return x * x

if __name__ == "__main__":
    with mp.Pool(processes=4) as pool:
        # map – blocks, ordered
        print(pool.map(square, range(10)))    # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

        # imap_unordered – lazy, fastest-first
        for r in pool.imap_unordered(square, range(10)):
            print(r, end=" ")

        # apply_async – future-based
        fut = pool.apply_async(square, (100,))
        print(fut.get())                    # 10000
```

Q28 · `Manager` — Shared Proxy Objects

Tags: Difficulty: Medium · Asked at: Razorpay, Goldman, Microsoft India · Topic: multiprocessing

The question (interviewer's verbatim phrasing):

Walk me through `multiprocessing.Manager`. When would you actually use it?

The 30-second answer: `Manager()` starts a server process that hosts shared Python objects — `dict`, `list`, `Namespace`, `Event`, `Queue`, etc. Other processes get proxy objects that forward operations to the manager over a socket. Use it when you need shared mutable state across processes — a shared counter, a shared dict of results, a shared list of in-flight tasks. The cost: every operation on a proxy is an IPC roundtrip — `manager.dict()["k"] = "v"` is hundreds of microseconds, not nanoseconds. Use it for low-frequency coordination, not high-throughput data movement.

Step-by-step thought process: 1. The architecture: Manager starts an additional process. Your workers communicate with that process via sockets; the Manager mediates all reads and writes to the shared object. 2. The use case: tasks need to update a shared `done_tasks` dict, or collect results into a shared list. Without Manager, you would have to design explicit IPC; with Manager, the data structure is conceptually shared. 3. The cost: each `proxy["k"] = "v"` is a full message round-trip through the Manager. Hundreds of microseconds at minimum. Don't do high-frequency updates in a hot loop. 4. The alternative for raw data: `multiprocessing.shared_memory` (3.8+) gives you a shared `bytes` buffer with no IPC overhead — perfect for NumPy arrays. Manager is for richer data; `shared_memory` is for bulk bytes.

Edge cases to mention: - Manager proxies are transparent only at the top level — `manager.dict()["nested"].append(x)` does NOT propagate; you need to reassign: `d = manager.dict(); d["k"] = []; tmp = d["k"]; tmp.append(x); d["k"] = tmp` (or use a `manager.list()` for the nested part). - Manager processes inherit the parent's environment but not its imports — be aware of side effects on start. - For typed shared values, use `multiprocessing.Value` and `multiprocessing.Array` — cheaper than Manager for primitives.

What NOT to say: - "Manager is free." — Every proxy op is an IPC round-trip. - "`manager.dict()` is like `dict()` ." — It looks like one but operations have IPC overhead. - "Use Manager for high-throughput data." — Use `Queue` or `shared_memory` for that.

Common follow-up: "What is the difference between `Manager.dict()` and `multiprocessing.Value` ?" — `Manager.dict()` is rich (any Python object as values) but slow (IPC). `Value('i', 0)` is a typed C-level shared int — fast (~hundreds of nanoseconds) but only one primitive value. Pick by need.

```
import multiprocessing as mp

def worker(d, i):
    d[i] = i * i

if __name__ == "__main__":
    with mp.Manager() as mgr:
        shared = mgr.dict()
        procs = [mp.Process(target=worker, args=(shared, i)) for i in range(5)]
        for p in procs: p.start()
        for p in procs: p.join()
        print(dict(shared))    # {0:0, 1:1, 2:4, 3:9, 4:16}
```

Q29 · `multiprocessing.Queue` and `Pipe`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, JPMC · Topic: multiprocessing

The question (interviewer's verbatim phrasing):

What are `multiprocessing.Queue` and `Pipe` and when would you pick one over the other?

The 30-second answer: Both are IPC primitives. `Queue` is a multi-producer multi-consumer FIFO backed by a pipe plus a background feeder thread that pickles items; thread-safe and process-safe. `Pipe` returns two `Connection` objects representing the two ends of an OS pipe — one producer, one consumer (or `duplex=True` for bidirectional). Pick `Queue` when multiple processes are reading or writing; pick `Pipe` for point-to-point communication between exactly two processes (lower overhead, simpler). Both pickle objects on the way through; both have a size cap on the underlying OS pipe buffer.

Step-by-step thought process: 1. `Queue` is the higher-level — built for fan-in / fan-out, multi-producer/multi-consumer, with the same API as `queue.Queue` (put, get, etc.). 2. `Pipe` is the lower-level — a direct connection between two processes via OS-level pipe. Faster per-message but not safe for many writers. 3. Both pickle — objects you put in must be picklable; sockets, lambdas, local functions cannot be sent. 4. The capacity gotcha: the underlying OS pipe buffer is typically 64KB on Linux. Sending more data than that without a reader draining it will block the writer — easy way to deadlock.

Edge cases to mention: - `Queue` has an internal feeder thread per process — that thread pickles items asynchronously. Sometimes `queue.qsize()` is approximate as a result. - `Pipe` is faster but requires you to handle synchronization yourself when multiple writers — generally don't. - For sharing NumPy arrays, prefer `shared_memory` over `Queue` or `Pipe` — pickling a 1GB array kills throughput.

What NOT to say: - "Queue and Pipe are the same." — Different abstraction; Queue is multi-end, Pipe is two-end. - "Pipe is dangerous." — It is fine when used correctly; just don't share one end with multiple writers. - "Pipes bypass pickling." — They don't; both pickle.

Common follow-up: "How would you send 1GB of data between processes efficiently?" — Don't send it via `Queue` or `Pipe` (pickling cost is prohibitive). Use `multiprocessing.shared_memory.SharedMemory` (3.8+) — both processes attach to the same memory region, no copy needed. Coordinate access via a `Queue` carrying offsets and metadata.

```
import multiprocessing as mp

def child(conn):
    conn.send([1, 2, 3])    # send list to parent
    conn.close()

if __name__ == "__main__":
    parent_conn, child_conn = mp.Pipe()
    p = mp.Process(target=child, args=(child_conn,))
    p.start()
    print(parent_conn.recv())    # [1, 2, 3]
    p.join()
```

Q30 · Pickling Restrictions — What Can Cross Process Boundaries

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Goldman, Microsoft India · Topic: multiprocessing

The question (interviewer's verbatim phrasing):

What can and cannot be passed to a `multiprocessing.Pool` worker as a target or as an argument?

The 30-second answer: Anything sent to a worker process must be picklable. Picklable: top-level functions, classes defined at module level, primitive types, lists/dicts/tuples of picklable types, dataclasses, NumPy arrays, most stdlib objects. NOT picklable: lambdas, nested functions (defined inside another function), local classes, generators, open file handles, sockets, database connections, weakrefs, tracebacks, threading primitives. The fix when a lambda or local function is needed: move it to module scope. When a non-picklable resource is needed in the worker, create it INSIDE the worker (per-process), not in the parent.

Step-by-step thought process: 1. The rule: cross-process communication serialises through `pickle`. If `pickle.dumps(obj)` raises, the worker call fails. 2. The most common offender: a lambda passed as `target`. `pool.map(lambda x: x*2, range(10))` errors on `spawn` platforms. Fix: define `def double(x): return x*2` at module level. 3. The second offender: nested functions. A `closure` captured by an inner function may not pickle. Refactor or use `functools.partial` (which is picklable for top-level targets). 4. The right pattern for resources (DB connections, sockets): initialise per-worker with `Pool(initializer=init_fn)` — `init_fn` runs once per worker and stores the resource in a global.

Edge cases to mention: - `cloudpickle` is a third-party library that can pickle lambdas and closures; `dask` and `joblib` use it. Not stdlib, but in production data pipelines it shows up. - The Pool's pickling is per-call — every `map` call pickles the target and args anew, even though the worker process is reused. Big args, repeated, are an anti-pattern. - On Linux with `fork` start method, the child inherits

parent memory — you can sometimes "smuggle" non-picklable resources via globals. Fragile; not portable to macOS/Windows.

What NOT to say: - "Anything can be sent to a worker." — Lambdas and locals cannot. - "Pickling is fast." — It is fast for small objects, slow for large NumPy arrays, glacial for nested dicts. - "Use lambdas everywhere." — Will break on spawn platforms.

Common follow-up: "How would you initialise a database connection once per worker process?" — Use `Pool(initializer=init_db, initargs=(dsn,))`. `init_db` runs once per worker on startup, stores the connection in a module-level global. Worker tasks read that global.

```
import multiprocessing as mp

def top_level(x):
    # picklable — defined at module scope
    return x * 2

if __name__ == "__main__":
    with mp.Pool(4) as pool:
        # works
        print(pool.map(top_level, range(5)))
        # fails on spawn platforms — lambda not picklable
        # print(pool.map(lambda x: x * 2, range(5)))
```

Q31 · `fork` vs `spawn` vs `forkserver` Start Methods

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Microsoft India, JPMC · Topic: multiprocessing

The question (interviewer's verbatim phrasing):

Explain the three start methods for `multiprocessing` — `fork`, `spawn`, `forkserver`. Which is the default on macOS and why did it change?

The 30-second answer: `fork` (Linux default) clones the parent process via `os.fork()` — child inherits all memory, file descriptors, and threading state, as a copy-on-write copy. Fast (microseconds) but inherits open connections, signal handlers, and locks held by other threads — leading to deadlocks if the parent had any live threads. `spawn` (Windows default, macOS default since 3.8) starts a fresh Python interpreter, re-imports the entry module, and only ships the target function + args via pickle. Slow (hundreds of ms) but clean. `forkserver` (Linux only) is a hybrid — a "server" process is fork-spawned once with no threads; subsequent workers fork from that clean server. Fast like fork, clean like spawn. macOS switched to `spawn` in 3.8 because Apple's frameworks (NSURLSession, AVFoundation) crash under `fork` post the Mojave runtime changes.

Step-by-step thought process: 1. `fork` : traditional Unix. Cheap and inherits everything. Dangerous if the parent has multiple threads, holds locks, or owns OS resources that the child should not touch. 2. `spawn` : fresh start. Re-imports the module — so `if __name__ == "__main__":` guard is mandatory or you get infinite recursion. Slow but safe. 3. `forkserver` : a one-time fork of a small "template" process, from which workers spawn cheaply. Cleaner than fork (no thread inheritance) but Linux-only. 4. The 3.8 change: macOS switched from `fork` to `spawn` because Apple's Objective-C runtime is not safe to fork — many GUI apps crashed silently. Explicit set via `mp.set_start_method('fork')` if you really need fork (and accept the risk).

Edge cases to mention: - The `if __name__ == "__main__":` guard is mandatory under `spawn` and `forkserver` — the child re-imports the script, and without the guard, it tries to re-spawn workers recursively. - `fork` on Linux is COW — memory pages are shared until a write happens, then duplicated. Big inherited memory does not cost RAM until the child writes. - `forkserver` is great in long-running services because the server warm-cache is reused across many worker spawns.

What NOT to say: - "fork is always fastest." — True on Linux for startup, but the inherited state can cause subtle bugs. - "spawn is the default everywhere." — Default is per-platform; Linux is still fork. - "Start methods are interchangeable." — They have different behavior with imports, signal handlers, and inherited locks.

Common follow-up: "Why does my `multiprocessing` Pool sometimes deadlock on macOS but not Linux?" — Likely a `fork` vs `spawn` issue. Pre-3.8 macOS used fork; if a thread (logging, Tornado, etc.) was running in the parent at fork time, the child inherits the lock state without the thread that would release it — instant deadlock. The 3.8 switch to spawn fixed this; but if you forced `fork`, the bug returns.

```
import multiprocessing as mp

if __name__ == "__main__":
    # Show available + default
    print(mp.get_all_start_methods())    # ['fork', 'spawn', 'forkserver'] on Linux
    print(mp.get_start_method())        # 'fork' on Linux, 'spawn' on macOS/Windows

    # Force spawn explicitly
    ctx = mp.get_context('spawn')
    p = ctx.Process(target=print, args=("hello from spawn",))
    p.start()
    p.join()
```

Q32 · Why `if __name__ == "__main__":` Is Mandatory

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Cred, JPMC · Topic: multiprocessing

The question (interviewer's verbatim phrasing):

Why does multiprocessing on Windows or macOS demand the `if __name__ == "__main__":` guard?

The 30-second answer: On `spawn` (Windows default, macOS default since 3.8), the child process starts a fresh Python interpreter and re-imports the module that contains the target function. Without the guard, the module-level code that creates the `Process` or `Pool` runs again in the child — which spawns more children, which run the module again, recursively. You get a fork bomb. The guard ensures that the `Process(...)` / `Pool(...)` calls only run when the script is the entry point, not when it is being imported by a spawned child.

Step-by-step thought process: 1. The mechanism: under `spawn`, the child needs the parent's code. It re-imports the module containing `target=fn`. Module-level code runs as a side effect of import. 2. Without the guard: `mp.Process(target=fn).start()` at module top runs in the child too. The child spawns more children, ad infinitum. Process explosion in seconds. 3. With the guard: the spawn code (`Process(...)`, `Pool(...)`, etc.) lives inside the guarded block. On the entry-point run, `__name__ == "__main__"` is True and the code runs. On import (by the child), `__name__` is the module name, the block is skipped. 4. Under `fork` (Linux default), the child inherits memory rather than re-importing — so the guard is not strictly required. But best-practice is to use it anyway; portability is cheap insurance.

Edge cases to mention: - The guard requirement applies to anything spawned: `Process`, `Pool`, `concurrent.futures.ProcessPoolExecutor`. `ThreadPoolExecutor` does not need it. - A library that wraps multiprocessing and runs at import time will explode on Windows / macOS — common bug in older data pipelines. - Jupyter notebooks need extra care — `__name__` in a notebook cell is `__main__`, but the spawn mechanism may not find the target function reliably.

What NOT to say: - "The guard is a style preference." — It is a correctness requirement on spawn platforms. - "Linux doesn't need the guard." — Strictly true under `fork`, but portable code should include it. - "Fork bombs aren't possible in Python." — Without the guard, they very much are.

Common follow-up: "What happens if you forget the guard on Windows?" — Either an immediate crash with `RuntimeError: An attempt has been made to start a new process before the current process has finished its bootstrapping phase`, or, in some configurations, exponential process growth until the OS refuses to fork.

```
import multiprocessing as mp

def worker(x): return x * 2

# CORRECT — guarded
if __name__ == "__main__":
    with mp.Pool(4) as pool:
        print(pool.map(worker, range(10)))

# WRONG — would fork-bomb on spawn platforms
# with mp.Pool(4) as pool:
#     print(pool.map(worker, range(10)))
```

Q33 · `Pool.apply_async` Callbacks and Error Handling

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JPMC, Goldman · Topic: multiprocessing

The question (interviewer's verbatim phrasing):

How do exceptions in a `Pool.apply_async` task propagate to the parent?

The 30-second answer: The exception is captured in the worker, pickled, sent back to the parent, and re-raised when you call `.get()` on the `AsyncResult`. If you never call `.get()`, the exception is silently swallowed — that is the most common error-handling bug in `multiprocessing` code. The `callback` and `error_callback` parameters give you a non-blocking way to handle results and errors as they arrive. For `Pool.map`, exceptions in any worker abort the call and re-raise at the parent.

Step-by-step thought process: 1. The capture: when the target raises, the worker pickles the exception (including stack trace via `tblib` if installed) and sends it back through the pool's result queue. 2. The propagation point: `.get()` raises the exception. `apply_async(...)` returns immediately and gives you an `AsyncResult`; the exception lives in that future until you call `.get()`. 3. The silent-failure trap: code that does `pool.apply_async(work)` and never calls `get` loses the error. Workers crash silently; only the missing result tips you off. 4. The callback path: `pool.apply_async(work, callback=on_success, error_callback=on_error)`. Both run in a separate thread in the parent process; design them to be fast and not raise.

Edge cases to mention: - `Pool.map` re-raises at the parent on any worker failure — the first failed task aborts the entire map. - `imap_unordered` raises on the iterator's `next()` for the failed item — you see other results first, then the exception. - Pickling restrictions apply to exceptions too — custom exception classes must be picklable.

What NOT to say: - "Exceptions kill the pool." — They do not; the worker is replaced. - "Workers crash silently." — Only if you fail to call `.get()`. - "Errors propagate immediately." — They propagate at `.get()`, not at submission.

Common follow-up: "How would you log every worker error without blocking the dispatch?" — Use `error_callback`. Submit each task with `apply_async(work, error_callback=lambda e: logger.error(e))`. The callback runs in a parent-side thread when the worker fails, asynchronously.

```
import multiprocessing as mp

def maybe_fail(x):
    if x == 5: raise ValueError(f"bad {x}")
    return x * 2

if __name__ == "__main__":
    with mp.Pool(4) as pool:
        results = [pool.apply_async(maybe_fail, (i,)) for i in range(10)]
        for r in results:
            try:
                print(r.get())
            except ValueError as e:
                print(f"caught: {e}")
```

Q34 · Measuring IPC Overhead — When Is It Worth It

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Microsoft India · Topic: multiprocessing

The question (interviewer's verbatim phrasing):

How do you decide if `multiprocessing` is actually faster than single-process for a workload?

The 30-second answer: Measure. The model: $\text{total_time} = \max(\text{per_task_time} \times \text{tasks} / \text{processes}) + \text{IPC overhead} + \text{startup overhead}$. For workloads where per-task time is large compared to IPC + startup, multiprocessing wins (often by $N\times$). For small tasks, IPC dominates and multiprocessing loses.

Benchmark: time the single-process version, time the multi-process version at $N=2,4,8$ cores, plot speedup. If you see sub-linear scaling or a flat curve, IPC or startup is the bottleneck. Mitigations: larger `chunksize`, fewer larger tasks, or shared memory for big args.

Step-by-step thought process: 1. Build a rule of thumb: if per-task work is under $\sim 10\text{ms}$, multiprocessing is rarely worth it for that task. Over $\sim 100\text{ms}$, it usually is. 2. The startup tax: spawning N processes costs $N \times \sim 100\text{ms}$ on macOS / Windows. For a workload that runs once and exits, this is a fixed cost. 3. The IPC tax: pickling each input/output adds microseconds per small object, milliseconds

per medium dict, seconds per big NumPy array. 4. The optimization: `chunksize` increases the granularity of work per IPC round trip — fewer trips, larger batches, less per-task overhead.

Edge cases to mention: - If your inputs are read-only and large, use `shared_memory` rather than sending them through pickling — order of magnitude faster for arrays. - `Pool(initializer=...)` lets you preload heavy state per worker once, amortising startup. - For very small tasks, batch them yourself: pass `[i for i in range(1000)]` as one argument, do all 1000 in the worker, return the result.

What NOT to say: - "Multiprocessing is always faster." — Not for small tasks. - "Benchmarking is unnecessary; you know it scales." — You don't; measure. - "Pickling is negligible." — Often dominates for small-task workloads.

Common follow-up: *"You see 2× speedup on 4 cores. What's likely the bottleneck?"* — Either IPC overhead (large args being pickled), or the workload has serial portions (Amdahl's law), or the OS has contention from other processes. Profile with `cProfile` inside the worker and compare against the IPC time.

```
import multiprocessing as mp, time

def work(n):
    # cheap task – IPC dominates
    return n + 1

if __name__ == "__main__":
    # Single process
    start = time.perf_counter()
    result = [work(i) for i in range(100_000)]
    print(f"Single: {time.perf_counter() - start:.3f}s")

    # 4-process pool – IPC overhead may make this SLOWER
    start = time.perf_counter()
    with mp.Pool(4) as pool:
        result = pool.map(work, range(100_000), chunksize=10_000)
    print(f"Pool: {time.perf_counter() - start:.3f}s")
```

Q35 · `concurrent.futures` — The Higher-Level API

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Swiggy, JPMC · Topic: multiprocessing

The question (interviewer's verbatim phrasing):

What is `concurrent.futures` and when would you use it over `multiprocessing.Pool` ?

The 30-second answer: `concurrent.futures` is a higher-level abstraction over both threading and multiprocessing — `ThreadPoolExecutor` and `ProcessPoolExecutor` have the same API. You submit work and get back `Future` objects; you can wait on them with `as_completed`, `wait`, or call `.result()` directly. The advantage over `multiprocessing.Pool`: uniform API across threads and processes, easy to switch between them based on workload, idiomatic Future-based composition, integrates cleanly with asyncio via `loop.run_in_executor`. Use it as the default for new code; reach for `multiprocessing.Pool` directly only when you need its specific features (`imap_unordered` ordering, `initializer`).

Step-by-step thought process: 1. The API: `executor.submit(fn, args)` returns a `Future`; `executor.map(fn, iterable)` is equivalent to `Pool.map`. `as_completed(futures)` yields them in completion order. 2. The interchangeability: write your code against `Executor`, swap `ThreadPoolExecutor` for `ProcessPoolExecutor` when you discover the workload is CPU-bound. Zero other changes needed. 3. The async integration: `await loop.run_in_executor(executor, fn, *args)` runs `fn` in the executor and lets asyncio await the result. Clean way to combine async I/O with CPU work in processes. 4. The trade-off vs `multiprocessing.Pool`: less direct access to advanced features (`imap_unordered`, `chunksize` -with-iterators), and `ProcessPoolExecutor`'s pickling has been historically buggier on edge cases.

Edge cases to mention: - Always use `with ... as executor:` to guarantee shutdown — the context manager calls `executor.shutdown(wait=True)` on exit. - `as_completed(futures, timeout=N)` raises after N seconds — useful for global deadlines. - `executor.map(fn, iterable, timeout=N)` enforces a per-result deadline.

What NOT to say: - "`concurrent.futures` is a wrapper that does the same thing as `Pool`." — Same idea, but different defaults, future-based composition, and asyncio integration. - "`ThreadPoolExecutor` runs in parallel." — It runs concurrently; GIL still applies for CPU-bound. - "`Future.result()` returns immediately." — It blocks until the result is ready (unless timeout is given).

Common follow-up: "Convert your `Pool.map` code to use `concurrent.futures`." — `with ProcessPoolExecutor() as ex: results = list(ex.map(fn, items))`. One line; same semantics; portable to `ThreadPoolExecutor` if I/O-bound.

```
from concurrent.futures import ProcessPoolExecutor, as_completed

def work(n): return n * n

if __name__ == "__main__":
    with ProcessPoolExecutor(max_workers=4) as ex:
        futures = [ex.submit(work, i) for i in range(10)]
        for fut in as_completed(futures):
            # yields as they finish
            print(fut.result())
```

Section E — `asyncio` End-to-End

This is the section where Python's concurrency story gets interesting for I/O-heavy backends — FastAPI, aiohttp, async DB drivers, telegram-bot frameworks. The interviews here are concrete: "explain the event loop," "what is the difference between `asyncio.gather` and `asyncio.TaskGroup`," "how do you handle cancellation." Modern code (3.11+) leans on `TaskGroup`, structured concurrency, and `asyncio.timeout`; older code uses `gather` and `wait_for`. Both show up — own both.

Q36 · The Event Loop — Mental Model

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, FastAPI shops, Cred · Topic: asyncio

The question (interviewer's verbatim phrasing):

Walk me through what the asyncio event loop is and how it differs from threading.

The 30-second answer: The event loop is a single-threaded scheduler that runs coroutines cooperatively — each coroutine voluntarily yields control at `await` points; the loop picks the next ready coroutine to run. There is one event loop per thread; by default, your main thread has one. Different from threading: no preemption, no GIL contention between coroutines (they share one thread), no per-task OS thread cost. Concurrency comes from interleaving — while one coroutine awaits I/O, others run. Parallelism does not — it is single-threaded by design. To use multiple cores, combine asyncio with `ProcessPoolExecutor` for CPU work.

Step-by-step thought process: 1. Mental model: a `while True` loop that polls a set of "tasks ready to run" and runs each until it hits an `await` that blocks. Then poll again. 2. Cooperative scheduling: a coroutine that does pure CPU work for 10 seconds without `await`-ing blocks the entire loop. All other coroutines wait. That is the cooperative model's fragility. 3. Why it's fast for I/O: thousands of concurrent network connections can coexist as coroutines because each is just a small state machine plus a few KB of stack. Compared to threading, the memory and context-switch cost is an order of magnitude lower. 4. The integration: `asyncio.run(main())` starts a loop, runs `main` to completion, closes the loop. Inside `main`, you `await` other coroutines or schedule them as tasks.

Edge cases to mention: - `uvloop` is a libuv-backed event loop drop-in replacement that is 2-4x faster than the stdlib loop — used by gunicorn-uvicorn in production. - Each thread can have its own loop (`asyncio.new_event_loop`) — useful when integrating asyncio with non-asyncio code. - The loop is closed and the program ends when `asyncio.run`'s main coroutine returns — pending tasks are cancelled.

What NOT to say: - "asyncio is multithreaded." — It is single-threaded by default. - "asyncio is faster than threading for CPU work." — No; it's the same single-thread ceiling, often slower because of the loop overhead. - "Coroutines are processes." — They are state-machine objects scheduled by a loop.

Common follow-up: "What happens if a coroutine does `time.sleep(1)` instead of `await asyncio.sleep(1)`?" — `time.sleep` is synchronous — it blocks the OS thread for one second. Since asyncio runs on one thread, the entire event loop freezes for that second. All other coroutines wait. Always use `await asyncio.sleep` in async code; reach for blocking sleep only inside `run_in_executor`.

```
import asyncio, time

async def task(name, secs):
    print(f"{name} start")
    await asyncio.sleep(secs)           # cooperative – releases the loop
    print(f"{name} done")

async def main():
    start = time.perf_counter()
    await asyncio.gather(
        task("A", 1),
        task("B", 1),
        task("C", 1),
    )
    print(f"Total: {time.perf_counter() - start:.2f}s") # ~1s, not 3s

asyncio.run(main())
```

Q37 · `asyncio.run` — The Right Entry Point

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, FastAPI shops · Topic: asyncio

The question (interviewer's verbatim phrasing):

What does `asyncio.run` do under the hood and when would you use it?

The 30-second answer: `asyncio.run(coro)` is the canonical entry point for an asyncio program. It creates a new event loop, runs the coroutine to completion, cancels any remaining tasks, runs their cleanup, and closes the loop. It is intended to be called exactly once per program, typically as `asyncio.run(main())` at the bottom. Do not use it inside an already-running loop (will raise `RuntimeError`); inside a running loop, await the coroutine directly. Pre-3.7 you had to do `loop = asyncio.get_event_loop(); loop.run_until_complete(coro)` manually; `asyncio.run` collapses that into one line and ensures cleanup.

Step-by-step thought process: 1. The semantic: one call. Sets up, runs, tears down. Designed to be used as the program's entry point. 2. The cleanup: when the main coroutine finishes, asyncio cancels any pending tasks (they raise `CancelledError`), awaits their cleanup, and then closes the loop. 3. The "already running" trap: inside a coroutine, you cannot call `asyncio.run` again — the current loop

is already running. Use `await` for inner coroutines. 4. The `debug=True` arg enables slow-callback warnings — if a coroutine blocks the loop for more than 100ms, asyncio logs it. Great for finding accidentally-blocking code.

Edge cases to mention: - Jupyter notebooks have a running event loop already — `asyncio.run` raises. Use `await coro()` directly in cells; some setups need `nest_asyncio` for legacy code. - The cleanup phase respects task cancellation — your coroutines should handle `CancelledError` gracefully (often by re-raising after cleanup). - `asyncio.run(coro, debug=True)` is the single best debugging knob — turn it on in development.

What NOT to say: - "Call `asyncio.run` multiple times." — Designed for one call per program. - "asyncio.run is the same as `loop.run_until_complete` ." — It includes setup + teardown; `run_until_complete` does not. - "asyncio.run is the only way to start a loop." — `run_until_complete` and `run_forever` exist for explicit loop management.

Common follow-up: "What changes when you set `debug=True` in `asyncio.run` ?" — Enables slow-callback warnings (default threshold 100ms), task-tracking by where they were created, and stricter error reporting. Catches the "accidentally synchronous code blocking the loop" bug.

```
import asyncio

async def main():
    print("hello")
    await asyncio.sleep(0.1)
    print("world")

asyncio.run(main(), debug=True)
```

Q38 · Task vs Future vs Coroutine

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JPMC, Goldman · Topic: asyncio

The question (interviewer's verbatim phrasing):

What is the difference between a coroutine, a `Task` , and a `Future` in asyncio?

The 30-second answer: A coroutine is an `async def` function call — an object that has a `.send()` method and yields awaitable values. By itself, it does nothing — it must be awaited or scheduled. A `Future` is a lower-level object that represents a value-yet-to-arrive; it has `.set_result` , `.set_exception` , `.done()` , `.add_done_callback` . A `Task` is a subclass of `Future` that wraps a coroutine and schedules it on the event loop —

`asyncio.create_task(coro)` returns a Task. Mental model: coroutines are the *function*; Tasks are the *running instance*; Futures are the *value*.

Step-by-step thought process: 1. Coroutine: `async def foo(): ...; coro = foo()` — `coro` is a coroutine object, not yet running. You must await it or schedule it. 2. Task: `task = asyncio.create_task(foo())` — schedules the coroutine on the loop, returns a Task you can await, cancel, or query. 3. Future: a primitive container for "value-soon" — used inside the asyncio internals and for bridging with callback-based code. Most user code uses Task; Future is for library authors. 4. The pattern: spawn tasks with `create_task`, gather them with `gather` or a TaskGroup, await their results.

Edge cases to mention: - A coroutine that is never awaited or scheduled raises a `RuntimeWarning` ("coroutine was never awaited") when garbage-collected. - A Task running in the background must be held onto — if the only reference is lost, garbage collection can cancel it. Bind tasks to a variable or container. - `asyncio.ensure_future(coro_or_future)` is older API for "schedule if coroutine, else return as-is." Prefer `create_task` for clarity.

What NOT to say: - "Task and Future are the same." — Task is a Future subclass that wraps a coroutine; Future is the lower-level primitive. - "Coroutines run automatically." — They do not. They must be scheduled or awaited. - "asyncio.create_task awaits the coroutine." — It schedules it; awaiting is a separate step.

Common follow-up: "What happens to a Task whose only reference is lost?" — Asyncio holds a weak reference, so it can be garbage-collected. To prevent that, keep a strong reference — add it to a set, then remove on completion via `add_done_callback`. This is the famous "fire and forget" task pattern.

```
import asyncio

async def worker(n):
    await asyncio.sleep(0.1)
    return n * 2

async def main():
    # coroutine — not scheduled yet
    coro = worker(5)

    # task — scheduled, running
    task = asyncio.create_task(worker(10))

    # await both
    print(await coro)    # 10 — awaiting runs it
    print(await task)    # 20 — awaiting task waits for completion

asyncio.run(main())
```

Q39 · `gather` vs `TaskGroup` — The 3.11 Upgrade**Tags:** Difficulty: Medium · Asked at: Razorpay, Swiggy, FastAPI shops, Cred · Topic: asyncio**The question (interviewer's verbatim phrasing):**

What is the difference between `asyncio.gather` and `asyncio.TaskGroup` and which would you use today?

The 30-second answer: `asyncio.gather(*coros)` runs multiple coroutines concurrently and returns their results as a tuple. It is the old API. `asyncio.TaskGroup` (3.11+) is the structured concurrency replacement: a context manager that owns its tasks, waits for all of them on exit, and cancels all of them if any one fails. The key difference: `gather` with one failure does NOT automatically cancel the others — you must handle that manually with `return_exceptions=True` and explicit cleanup. `TaskGroup` enforces "all succeed or all are cancelled" — that is the structured concurrency contract. Use `TaskGroup` in 3.11+ code; reach for `gather` only when you need its `return_exceptions=True` semantics or compatibility with older Python.

Step-by-step thought process: 1. `gather(*coros)` — the classic API. Concurrent execution; returns results in order; on first exception, raises it but other tasks continue running in the background. 2. The `return_exceptions=True` flag changes `gather` to return exceptions as values rather than raising — useful for "I want to know which succeeded and which failed." 3. `TaskGroup` (PEP 654 era, shipped 3.11) — structured concurrency. Spawn tasks with `tg.create_task(coro)`; on exit from the `with` block, awaits all; on any failure, cancels all and raises an `ExceptionGroup`. 4. Migration in 2026: new code uses `TaskGroup`. Old code uses `gather`. Both are correct; `TaskGroup` is the modern idiomatic choice.

Edge cases to mention: - `TaskGroup` exceptions are `ExceptionGroup` (PEP 654) — wrapping multiple errors. Use `except* SomeError` syntax (PEP 654) to catch by type. - `gather` with `return_exceptions=False` (default) raises the first exception but leaves other tasks running — common source of "phantom tasks" continuing to log after main thread thinks it's done. - `TaskGroup` blocks on `__aexit__` until all tasks complete or are cancelled — there is no "fire and forget" semantically.

What NOT to say: - "`gather` and `TaskGroup` are the same." — Different cancellation contracts. - "`gather` always cancels other tasks on failure." — It does not by default. - "`TaskGroup` is just sugar for `gather`." — Different semantics and a stronger correctness contract.

Common follow-up: "How do you handle `ExceptionGroup` from a `TaskGroup`?" — Use the `except*` syntax:

```
try: ... except* ValueError as eg: for e in eg.exceptions: ...
```

 The `*` says "match all exceptions of this type inside the group." Old-style `except` catches the whole `ExceptionGroup` as one.

```
import asyncio

async def good(): await asyncio.sleep(0.1); return "ok"
async def bad(): await asyncio.sleep(0.05); raise ValueError("boom")

async def with_taskgroup():
    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(good())
            tg.create_task(bad())          # fails – cancels the other
            tg.create_task(good())
    except* ValueError as eg:
        print(f"caught {len(eg.exceptions)} ValueErrors")

async def with_gather():
    results = await asyncio.gather(good(), bad(), good(), return_exceptions=True)
    print(results)  # ['ok', ValueError(...), 'ok']

asyncio.run(with_taskgroup())
asyncio.run(with_gather())
```

Q40 · `asyncio.timeout` — The Modern Timeout

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred · Topic: asyncio

The question (interviewer's verbatim phrasing):

How do you put a timeout around an async operation in modern Python?

The 30-second answer: `async with asyncio.timeout(seconds):` is the 3.11+ context-manager way. The block raises `TimeoutError` if it does not complete in time; everything inside is cancelled gracefully. The old API is `await asyncio.wait_for(coro, timeout=N)` — wraps a single coroutine, returns its value or raises `TimeoutError`. `asyncio.timeout` is the more composable and modern choice — works around arbitrary blocks of code, integrates with `TaskGroup` cleanly. Both produce a `TimeoutError` (since 3.11, `asyncio.TimeoutError` is an alias for the builtin `TimeoutError`).

Step-by-step thought process: 1. The modern way: `async with asyncio.timeout(5):` wraps a block. If it does not exit within 5 seconds, all inflight awaits are cancelled and `TimeoutError` is raised at the `async with` boundary. 2. The old way: `await asyncio.wait_for(coro, timeout=5)`. Single-coroutine wrapper. Still supported, slightly less ergonomic. 3. The advantage of `timeout`: composes with `TaskGroup` — `async with asyncio.timeout(5), asyncio.TaskGroup() as tg:`. Nested timeouts (`asyncio.timeout(2)` inside `asyncio.timeout(5)`) work as expected;

inner expires first. 4. The cancellation contract: timeout uses task cancellation under the hood. Your code can catch `CancelledError` and do cleanup; re-raise to honour the cancellation.

Edge cases to mention: - `asyncio.timeout(None)` disables the timeout — useful for dynamic timeouts where None means "no limit." - `asyncio.timeout_at(loop.time() + 5)` is the absolute-time variant — handy for deadline-driven code. - A `CancelledError` raised inside the block can shadow the `TimeoutError` if not re-raised properly — common subtle bug.

What NOT to say: - "`asyncio.wait_for` is deprecated." — It is not; still fine to use. - "Timeouts are hard cancellation." — They are cooperative — the inner code must reach an `await` point for cancellation to take effect. - "`asyncio.timeout` kills the task immediately." — It requests cancellation; the task gets `CancelledError` at its next `await`.

Common follow-up: "How do timeouts compose when nested?" — Inner timeouts fire first. `asyncio.timeout(5)` outer + `asyncio.timeout(2)` inner means: if inner block takes >2s, inner expires (`TimeoutError` raised at the inner `async with`). If outer block takes >5s overall, outer fires. No conflict; nesting works.

```
import asyncio

async def slow_op():
    await asyncio.sleep(10)
    return "done"

async def main():
    try:
        async with asyncio.timeout(1):
            await slow_op()
    except TimeoutError:
        print("timed out cleanly")

asyncio.run(main())
```

Q41 · `asyncio.shield` — Protecting From Cancellation

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, Microsoft India · Topic: asyncio

The question (interviewer's verbatim phrasing):

What is `asyncio.shield` and when would you use it?

The 30-second answer: `asyncio.shield(coro)` wraps a coroutine so that cancellation of the *outer* awaiter does NOT cancel the wrapped coroutine. The shielded coro continues running in the background; the outer awaiter sees `CancelledError`. Use it to protect critical cleanup or state-

mutating operations from being cancelled mid-way — for example, a payment confirmation that must complete or your books are inconsistent. Without `shield`, a parent cancellation propagates into the child; with `shield`, the child runs to completion and the parent gets the cancellation. The child's result is lost if the parent does not catch the cancellation and re-await the shield.

Step-by-step thought process: 1. The cancellation propagation problem: `asyncio.timeout(5)` cancels everything inside, including any `await` currently in flight. If that `await` is "commit transaction," cancelling it leaves the system inconsistent. 2. `shield` is the fence: `await asyncio.shield(commit_transaction())`. If the outer scope is cancelled, the commit still runs; the parent awaiter raises `CancelledError` immediately. 3. The result: the shielded task continues but you no longer await its return. To get the result, the parent must re-await the shielded task in a `finally` or catch the cancellation specifically. 4. The use case: idempotent cleanup, distributed-transaction commits, half-committed file writes. Anything where "abort halfway" is worse than "wait for completion."

Edge cases to mention: - `shield` does not prevent the wrapped task itself from being cancelled if you call `.cancel()` on it directly. It protects only against cancellation propagated through the outer awaiter. - The shielded task continues running even after the outer raises — orphaned tasks consume resources. Keep a reference if you need to clean up. - `shield` and `wait_for / timeout` interact subtly — read the docstrings carefully when combining them.

What NOT to say: - "`shield` cancels the task." — Opposite. It *prevents* propagated cancellation. - "`shield` is always safe." — Misuse creates orphaned background tasks. - "`shield` is for performance." — It is for cancellation correctness, not speed.

Common follow-up: "Give me a realistic example where you'd use `shield`." — Acquiring a distributed lock with a TTL and releasing it on completion. If the release coroutine is cancelled mid-way, the lock stays held until TTL expires — extending the critical section. Shield the release to ensure it always completes.

```
import asyncio

async def must_complete():
    print("starting critical work")
    await asyncio.sleep(1)
    print("finished critical work")
    return "result"

async def main():
    try:
        async with asyncio.timeout(0.5):
            # without shield, must_complete would be cancelled
            result = await asyncio.shield(must_complete())
    except TimeoutError:
        print("outer timed out – but must_complete still finishes in background")
        await asyncio.sleep(1) # give it time to print "finished"

asyncio.run(main())
```

Q42 · `asyncio.wait` Patterns — `FIRST_COMPLETED`, `ALL_COMPLETED`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, FastAPI shops · Topic: asyncio

The question (interviewer's verbatim phrasing):

What is `asyncio.wait` and when would you use it over `gather` ?

The 30-second answer: `asyncio.wait(tasks)` waits for multiple tasks and returns two sets — `done` and `pending`. You control the return condition with `return_when=` (defaults to `ALL_COMPLETED`; `FIRST_COMPLETED` returns as soon as any one finishes; `FIRST_EXCEPTION` returns on first failure). Use it for race-style patterns — "race three queries, take the first result, cancel the rest." `gather` always waits for all and returns ordered results; `wait` is the lower-level "give me back done/pending so I can choose what to do." Modern equivalent: `TaskGroup` for structured concurrency; `wait` for first-completed patterns; `gather` for "concurrent and ordered."

Step-by-step thought process: 1. The semantic: `asyncio.wait` returns two sets, `done` and `pending`. Unlike `gather`, it does not return results in order or even results at all — you fetch them from the done tasks. 2. The `return_when` options: `ALL_COMPLETED` (default), `FIRST_COMPLETED` (race), `FIRST_EXCEPTION` (early-exit on failure). 3. The race pattern: `done, pending = await asyncio.wait(tasks, return_when=FIRST_COMPLETED); for t in pending: t.cancel()`. Standard "fan out, take first" idiom. 4. Don't pass raw coroutines — pass `Tasks`. `wait` requires `Tasks` or `Futures`; passing coroutines emits a deprecation warning in 3.8+ and errors in 3.12+.

Edge cases to mention: - `wait` with a `timeout=N` returns whatever is done within N seconds, with the rest still in `pending`. Useful for "do as much as you can in 5s." - Cancelling tasks in `pending` requires awaiting them after `.cancel()` to ensure they actually stop. The cancellation is requested, not immediate. - TaskGroup is generally cleaner for the "all-must-complete" case; `wait` is the right tool for "first-completed" semantics.

What NOT to say: - "`wait` is the same as `gather`." — Different semantics (sets vs ordered results, configurable completion condition). - "`wait` cancels pending tasks." — It does not; you must explicitly cancel them. - "Pass coroutines to `wait`." — Pass Tasks; coroutine support is deprecated.

Common follow-up: "Implement a 'race' between three DB queries." — Wrap each query in `asyncio.create_task`, then `done, pending = await asyncio.wait(tasks, return_when=FIRST_COMPLETED)`. Take the winner from `done`, cancel each task in `pending`, await them to ensure cleanup.

```
import asyncio, random

async def query(name):
    await asyncio.sleep(random.uniform(0.1, 1.0))
    return f"{name} result"

async def main():
    tasks = [asyncio.create_task(query(n)) for n in ("A", "B", "C")]
    done, pending = await asyncio.wait(tasks, return_when=asyncio.FIRST_COMPLETED)

    for t in pending:
        t.cancel()

    for t in done:
        print(t.result())

asyncio.run(main())
```

Q43 · Cancellation Semantics — What `CancelledError` Means

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Microsoft India · Topic: asyncio

The question (interviewer's verbatim phrasing):

Walk me through how cancellation works in asyncio. What is `CancelledError` and how should code handle it?

The 30-second answer: Cancellation in asyncio is cooperative. Calling `task.cancel()` does not immediately stop the task; it schedules a `CancelledError` to be raised at the task's next `await`

point. The coroutine can catch it, do cleanup, and re-raise — that propagates the cancellation up. If you swallow `CancelledError` without re-raising, the cancellation is suppressed and the task continues. Best practice: catch `CancelledError` only for cleanup (closing connections, releasing locks), then re-raise. Since 3.8, `CancelledError` inherits from `BaseException`, not `Exception` — bare `except:` clauses catch it; `except Exception:` does not. This means a sloppy `try/except Exception:` no longer accidentally swallows cancellation.

Step-by-step thought process: 1. The mechanism: `task.cancel()` sets a flag; at the next `await` boundary inside the coroutine, asyncio raises `CancelledError`. It is delivered as an exception, not as a signal. 2. The handle-and-rethrow idiom: `try: await something() except asyncio.CancelledError: cleanup(); raise`. Always re-raise; failing to do so swallows cancellation. 3. The inheritance change in 3.8 — `CancelledError` moved from `Exception` to `BaseException`. Code that did `except Exception:` no longer accidentally catches it; you need bare `except:` or `except BaseException:` to do so. 4. The structured concurrency interplay: `TaskGroup` and `asyncio.timeout` use cancellation under the hood. Your code must cooperate by re-raising for those primitives to work correctly.

Edge cases to mention: - A task that catches and ignores `CancelledError` becomes "uncancellable." Use this rarely and only with `asyncio.shield` for principled reasons. - `task.cancel(msg="reason")` (3.9+) lets you attach a reason to the cancellation, visible via the `CancelledError`'s args. - Multiple cancellation requests stack — the `cancelling()` count increments; the task is cancellable when count > 0.

What NOT to say: - "Cancellation is immediate." — It is delivered at the next `await`; until then, the coroutine continues. - "Swallow `CancelledError` to keep the task running." — Anti-pattern; breaks structured concurrency. - "`except Exception:` catches `CancelledError`." — Not since 3.8.

Common follow-up: *"Show me how to clean up a database connection on cancellation."* — `try: ...; await query() except asyncio.CancelledError: await conn.close(); raise`. The cleanup runs before the cancellation propagates up; the re-raise ensures the parent sees it.

```
import asyncio

async def worker():
    try:
        print("working")
        await asyncio.sleep(10)
        print("never reached")
    except asyncio.CancelledError:
        print("got cancellation – cleaning up")
        raise

async def main():
    t = asyncio.create_task(worker())
    await asyncio.sleep(0.5)
    t.cancel()
    try:
        await t
    except asyncio.CancelledError:
        print("task cleanly cancelled")

asyncio.run(main())
```

Q44 · The Single-Threaded Cooperative Model — Implications

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, FastAPI shops · Topic: asyncio

The question (interviewer's verbatim phrasing):

Explain "single-threaded cooperative" in asyncio. What can go wrong?

The 30-second answer: asyncio runs all coroutines on one thread, taking turns at `await` points. Cooperative means each coroutine must voluntarily yield — there is no preemption. What goes wrong: any coroutine that does CPU-bound work or calls a blocking sync function (regular `time.sleep`, sync HTTP libraries, `pandas.read_csv`) freezes the entire loop for that duration. All other coroutines wait. The symptoms are mysterious latency spikes, missed timeouts, and "my async server hangs sometimes." The fix: never block in async code — use async-native libraries, or push blocking work to `loop.run_in_executor`.

Step-by-step thought process: 1. The model: one thread, one loop, many coroutines taking turns. Concurrency comes from the awaits, not from threads. 2. The cooperative invariant: every coroutine must hit an `await` within ~10s of milliseconds to keep the loop responsive. 3. The failure modes: (a) accidentally calling a sync API (`requests.get` instead of `httpx.AsyncClient.get`); (b) a CPU-heavy loop with no awaits (`while True: pure_python_compute(...)`); (c) a sync library used inside async code (sqlalchemy's sync API, pandas operations, scipy calls). 4. The fix: identify the blocker

via `asyncio.run(main(), debug=True)` which logs slow callbacks; either replace with an async equivalent, or wrap in `await loop.run_in_executor(executor, blocking_fn)`.

Edge cases to mention: - A coroutine that does heavy CPU work but yields with `await asyncio.sleep(0)` periodically lets the loop schedule others — a workaround but not a fix. - `run_in_executor` with `ThreadPoolExecutor` runs the blocking work in a thread, which is fine for I/O-bound sync libs; use `ProcessPoolExecutor` for CPU-bound. - The `debug=True` mode is the killer feature for finding accidental blocking — turn it on always in dev.

What NOT to say: - "asyncio is multi-threaded." — It is not. - "Blocking is fine as long as it's fast." — "Fast" is microseconds in async terms; sync calls of milliseconds visibly degrade throughput. - "I'll use threading inside async code." — Mixing threading and asyncio without `run_in_executor` is a source of subtle bugs.

Common follow-up: "How would you call a sync library from inside an async function?" — `result = await loop.run_in_executor(None, sync_fn, *args)`. The `None` executor uses the default thread pool. For CPU-bound, pass a `ProcessPoolExecutor` instead. The await returns the function's result.

```
import asyncio, time

async def good():
    await asyncio.sleep(1)           # cooperative – yields the loop

async def bad():
    time.sleep(1)                   # blocking – freezes the loop

async def main_good():
    start = time.perf_counter()
    await asyncio.gather(good(), good(), good())
    print(f"good: {time.perf_counter() - start:.2f}s")  # ~1s

async def main_bad():
    start = time.perf_counter()
    await asyncio.gather(bad(), bad(), bad())
    print(f"bad: {time.perf_counter() - start:.2f}s")   # ~3s – sequential

asyncio.run(main_good())
asyncio.run(main_bad())
```

Q45 · `run_in_executor` — Bridging Sync and Async

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, FastAPI shops, JPMC · Topic: asyncio

The question (interviewer's verbatim phrasing):

How do you call a blocking sync function from an async coroutine without freezing the event loop?

The 30-second answer: `await loop.run_in_executor(executor, sync_fn, *args)`. This schedules the sync function to run in the given executor (a `ThreadPoolExecutor` by default if you pass `None`), and the coroutine awaits the result. The event loop continues running other coroutines while the sync work runs in the background thread. For CPU-bound sync work, pass a `ProcessPoolExecutor` to bypass the GIL. Modern equivalent: `asyncio.to_thread(fn, *args)` (3.9+) is a shorthand for the common `ThreadPoolExecutor` case. Use `asyncio.to_thread` when in doubt; `run_in_executor` when you need to specify a particular executor.

Step-by-step thought process: 1. The bridge: `run_in_executor` takes a sync function, runs it in another thread (or process), and returns a Future that the coroutine awaits. 2. The shortcut: `await asyncio.to_thread(sync_fn, *args)` since 3.9 — equivalent to `run_in_executor(None, fn, *args)`. Cleaner for the common case. 3. The CPU-vs-I/O choice: for sync I/O (database, file, HTTP), use `ThreadPoolExecutor` — the GIL releases on syscalls. For sync CPU (image processing, heavy parsing), use `ProcessPoolExecutor` — bypasses the GIL. 4. The thread pool size: defaults to `min(32, cpu_count + 4)`. Tune via `loop.set_default_executor(ThreadPoolExecutor(max_workers=N))` for high-concurrency apps.

Edge cases to mention: - `run_in_executor` does NOT support kwargs directly — use `functools.partial(fn, **kwargs)` to bind them. - The thread or process running the sync function does not have access to the event loop — you cannot await inside it without `asyncio.run_coroutine_threadsafe`. - For CPU-bound work in production, prefer `ProcessPoolExecutor` and design the function to be pickle-friendly.

What NOT to say: - "`run_in_executor` runs the function in the event loop thread." — It does not; it runs in the executor thread. - "`asyncio.to_thread` is slow." — It is the right tool for sync I/O bridging. - "Use threading directly instead." — `run_in_executor` is the correct, awaitable way to integrate with asyncio.

Common follow-up: "How do you call a sync function with kwargs from async code?" — `await asyncio.to_thread(functools.partial(sync_fn, a=1, b=2))`. `to_thread` only accepts positional args directly; `partial` binds the kwargs.

```
import asyncio, time

def sync_blocking(n):
    time.sleep(1)      # represents sync I/O or CPU work
    return n * 2

async def main():
    start = time.perf_counter()
    # 3 sync calls concurrently – would be 3s sequential, ~1s here
    results = await asyncio.gather(
        asyncio.to_thread(sync_blocking, 1),
        asyncio.to_thread(sync_blocking, 2),
        asyncio.to_thread(sync_blocking, 3),
    )
    print(f"{results} in {time.perf_counter() - start:.2f}s")

asyncio.run(main())
```

Section F — Python 3.13 + The Future

This is the section that lets a candidate sound current rather than vintage. The free-threaded build (PEP 703) is the headline change of 2024 — CPython without a GIL, opt-in, experimental, real. PEP 684's per-interpreter GIL (groundwork in 3.12) is the other half — multiple Python interpreters in one process, each with its own GIL, true parallelism without sub-interpreter pain. Interview-today: know the names, know the timelines, know what to say.

Q46 · PEP 703 — The Free-Threaded Build

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Microsoft India, Walmart Labs · Topic: 3.13 Free-Threaded

The question (interviewer's verbatim phrasing):

What is PEP 703 and what changes in the free-threaded build of Python 3.13?

The 30-second answer: PEP 703 (Sam Gross, accepted 2023) removes the GIL from CPython in an opt-in build called the "free-threaded" build, available as `python3.13t`. The change required replacing reference counting with biased reference counting (single owner thread for most objects, atomic refcount only when needed) plus deferred reference counting for short-lived objects. Single-threaded performance is roughly 5-10% slower than the GIL build initially, with the gap closing in subsequent releases. Multi-threaded pure-Python CPU code can now scale across cores — true parallelism without

`multiprocessing`. The 3.13 build is experimental; the GIL build remains the default. Production migration depends on C extension support; many extensions need patches.

Step-by-step thought process: 1. The PEP — accepted by the Steering Council in July 2023 after years of work and a successful gilectomy reboot. 2. The mechanism — biased refcount (one "owner" thread per object, others use atomic increments only when borrowing) + deferred refcount for short-lived objects. Single-thread cost stays low; multi-thread cost stays manageable. 3. The build: `python3.13t` (the `t` suffix) is the free-threaded interpreter. It is a separate build that ships alongside the GIL build. Imports are slightly slower; refcount-heavy code may regress; thread-parallel pure-Python code accelerates. 4. The ecosystem: C extensions (NumPy, PyTorch, lxml) need to be patched for thread-safety in the free-threaded environment. Major ones are landing patches in 2024-2025; expect another year or two before mainstream production use.

Edge cases to mention: - The free-threaded build still has a GIL for some operations (interpreter shutdown, signal handling) — but not for bytecode execution. - The `--disable-gil` configure flag at build time produces the free-threaded build; binary distributions ship both. - Single-threaded performance: ~5-10% slower in 3.13t initially. Improvements expected in 3.14 and beyond.

What NOT to say: - "PEP 703 removes the GIL in default Python." — It is opt-in via a separate build. - "PEP 703 is production-ready." — Experimental in 3.13; production migration is multi-year. - "PEP 703 makes Python 'like Java.'" — Different memory model; pure-Python parallel scaling without re-architecting your code, but C extensions need updates.

Common follow-up: *"What should you tell your manager about migrating to free-threaded Python?"* — Today: not yet. The 3.13t build is for early experimentation, not production. The right answer is to track ecosystem readiness (NumPy, your DB driver, your async framework) and revisit in 12-18 months. In the meantime, multiprocessing remains the answer for parallelism.

```
# Visible difference: the free-threaded build advertises itself
import sys
print(sys.version)           # includes 'free-threading' if 3.13t
print(sys._is_gil_enabled()) # 3.13+ API — True on GIL build, False on 3.13t
```

Q47 · PEP 684 — Per-Interpreter GIL

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Microsoft India · Topic: 3.12 Per-Interpreter GIL

The question (interviewer's verbatim phrasing):

What is PEP 684 and how does per-interpreter GIL differ from PEP 703?

The 30-second answer: PEP 684 (Eric Snow) splits the GIL so that each sub-interpreter has its own GIL — multiple interpreters in one process can run pure-Python code on multiple cores without sharing the lock. Groundwork landed in 3.12; the user-facing API (`interpreters` module) is in PEP 734, shipping in 3.13. Different from PEP 703: 703 removes the GIL entirely from a single interpreter; 684 keeps one GIL per interpreter and runs multiple interpreters concurrently. The trade-off: 684 requires data to be marshalled between interpreters (no shared mutable state); 703 lets threads share state but requires C extensions to be thread-safe. Both are paths to parallelism; both will coexist.

Step-by-step thought process: 1. The PEP — accepted in March 2023, groundwork shipped in 3.12, user-facing API in 3.13 via PEP 734's `interpreters` module. 2. The mechanism: each sub-interpreter has its own GIL, its own modules table, its own interned strings, its own thread state. From the OS perspective, they share a process but are nearly fully isolated. 3. The difference from PEP 703: 703 removes the GIL within one interpreter — threads share memory. 684 keeps the GIL but allows multiple interpreters per process — no shared mutable state, communication via channels. 4. The use case: web servers (gunicorn-style) and embedded Python (running multiple Python "modules" in one C++ app) get parallelism without the cost of separate processes for fork/spawn or the C-extension safety burden of PEP 703.

Edge cases to mention: - Sub-interpreters communicate via channels (`interpreters.create_channel()`) — not via shared variables. Lower throughput than shared memory but cheaper than full multiprocessing IPC. - C extensions must be "subinterpreter-safe" — most stdlib modules are; many third-party ones are not (NumPy was historically not; recent work fixes this). - The two PEPs are not competing — they target different audiences: 703 for true pure-Python parallelism, 684 for safe isolation.

What NOT to say: - "PEP 684 removes the GIL." — It does not; it gives each interpreter its own. - "PEP 684 and 703 are the same thing." — Different mechanisms, different trade-offs. - "Sub-interpreters can share Python objects." — They cannot directly; channels marshal data.

Common follow-up: *"When would you pick per-interpreter GIL over the free-threaded build?"* — Pick per-interpreter when you want strong isolation (multi-tenant code, embedded Python) and don't need shared mutable state. Pick free-threaded when you need pure-Python parallelism with shared data structures (scientific computing, data pipelines) and are willing to accept the C-extension migration cost.

```
# 3.13+ — interpreters module (PEP 734)
# import interpreters
#
# main = interpreters.get_main()
# new = interpreters.create()
# new.exec("print('hello from sub-interpreter')")
#
# # Each interpreter has its own GIL — parallel execution possible
```

Q48 · What Changes For Libraries — The Migration Cost

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Microsoft India, Walmart Labs · Topic: 3.13 Migration

The question (interviewer's verbatim phrasing):

What changes for library authors when the GIL is removed in PEP 703?

The 30-second answer: C extensions can no longer assume the GIL protects their shared state. Any module-level variable, any cached value, any singleton must be made thread-safe by the library itself. Three migration steps: (1) audit module-level state for thread safety — global dicts, caches, free lists; (2) wrap necessary operations in fine-grained locks or use atomic primitives; (3) test under contention — many bugs only show up with N threads. NumPy, PyTorch, lxml, scikit-learn are working through this. The ecosystem migration is a multi-year effort. For pure-Python libraries: very little changes — Python-level code that worked under GIL works under no-GIL, just now in actual parallel.

Step-by-step thought process: 1. The interface contract change: under PEP 703, the GIL no longer serialises C-extension calls. Two threads can run inside the same extension function simultaneously. 2. The audit: every `static PyObject *cached = NULL;` or equivalent module-level state is now a race-condition candidate. Every cache, every singleton, every refcount-tweaking trick. 3. The fix: per-object locks for shared state, immutable design for caches that can be thread-local, atomic refcount discipline (the new `Py_AtomicLoad` family of macros). 4. The pure-Python author's view: almost nothing changes for you. Your `dict` is now actually parallel-safe per CPython spec; your code that already used `Lock` continues working. The ecosystem just gained an actual parallelism story.

Edge cases to mention: - Some libraries (asyncio, threading itself) are already thread-aware and need only minor changes. - The "subinterpreter-safe" effort (PEP 684) covers a lot of ground that PEP 703 also benefits from — shared work. - Production migration depends on the slowest C extension in your stack — practically, the entire data ecosystem.

What NOT to say: - "Nothing changes for users." — Library compat is the gating factor. - "Migration is easy." — It is multi-year for major C extensions. - "Pure Python needs to be rewritten." — It does not.

Common follow-up: "What's the migration plan for NumPy in the free-threaded build?" — NumPy has been adding GIL-release annotations and per-array locking; the team's roadmap targets full free-threaded compatibility through 2024-2025. Track the NumPy release notes for milestone announcements.

```
# Detection at runtime – useful for libs that ship both code paths
import sys
if hasattr(sys, "_is_gil_enabled"):
    if not sys._is_gil_enabled():
        # take the no-GIL code path
    ...
```

Q49 · What to Say In An Interview Today About All This

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JPMC, Goldman · Topic: 3.13 Interview Framing

The question (interviewer's verbatim phrasing):

What's your view on the GIL changes coming in 3.13? How do you talk about it in an interview?

The 30-second answer: Three points. First — know the names: PEP 703 (free-threaded build), PEP 684 (per-interpreter GIL), 3.13's `python3.13t` opt-in build. Second — frame it as multi-year migration: the path is real and exciting but production code in 2026 still runs on the GIL build because the C-extension ecosystem (NumPy, pandas, PyTorch, your DB driver) is still being made safe. Third — say what you would do today: for CPU parallelism, multiprocessing; for I/O concurrency, asyncio; track the free-threaded build as an experimental path you'll evaluate in 12-18 months when the ecosystem catches up. Confidence comes from being current without overpromising.

Step-by-step thought process: 1. Name the things — "PEP 703," "PEP 684," "free-threaded build," "per-interpreter GIL." Saying the names signals you read the change docs, not a tutorial from 2022. 2. Bound the claim — "it's experimental in 3.13" and "the ecosystem is still catching up." Don't overclaim; you'll get caught. 3. Map it to today's decisions — "Today I'd still use multiprocessing for parallel CPU and asyncio for I/O; I'd track NumPy/PyTorch milestones and revisit in 12-18 months." 4. End with a forward view — "PEP 703 is the path; PEP 684 is the safer alternative for multi-tenant; both will coexist long-term."

Edge cases to mention: - Different teams will adopt at different rates — scientific computing has incentive to migrate (CPU-heavy, NumPy-centric); web backends have less, since they already scale via processes. - Watch the talk by Sam Gross at PyCon 2024 — the canonical reference for the design. - The interviewer may not know more than you — be confident and specific without inventing details.

What NOT to say: - "The GIL is removed in Python 3.13." — Not in default; opt-in build. - "I'd switch to PEP 703 today." — Premature for production. - "The GIL never mattered." — It mattered enormously; the change is the bigger story.

Common follow-up: "Why is the free-threaded build not the default in 3.13?" — Two reasons: single-threaded performance is slightly slower (~5-10%), and the C-extension ecosystem is not yet ready.

Making it default would regress single-threaded workloads and break unmigrated libraries. The community plan is to migrate over several releases as both improve.

```
# In 2026, the right "GIL story" for production:  
# - asyncio or threading for I/O concurrency  
# - multiprocessing for CPU parallelism  
# - track 3.13t as an experimental path for 2027 production  
# - per-interpreter GIL (3.13+) for embedded / multi-tenant cases
```

Q50 · Performance Expectations — What 3.13t Actually Delivers

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Microsoft India, Walmart Labs · Topic: 3.13 Performance

The question (interviewer's verbatim phrasing):

If you ran your CPU-bound Python code on 3.13t today, what speedup would you actually see?

The 30-second answer: For pure-Python CPU-bound work in N threads on N cores, you would see close-to-N× speedup in the ideal case, dropping toward 2-4× in real code due to refcount contention on shared objects. Single-threaded performance is roughly 5-10% slower than the GIL build. Apps with heavy NumPy or other GIL-releasing C code see modest changes — they already scaled, and now C-extension contention on shared state may dominate. Real benchmarks (Sam Gross's PyCon 2024 talk, the Python team's published numbers) show 8-12× speedup on parallel-friendly pure-Python loops at 16 threads, with diminishing returns at higher counts. The "your mileage will vary" caveat is honest: workload structure matters more than the build switch.

Step-by-step thought process: 1. The best case: workloads where each thread operates on its own data (numerical loops, parsing pipelines) approach linear scaling. 8 threads on 8 cores → 6-8× speedup. 2. The worst case: workloads with heavy shared-object refcount churn (creating and destroying many small Python objects, shared dicts, shared lists) suffer from atomic refcount contention. Speedup can drop to 2-3× even on 8 cores. 3. The single-threaded tax: ~5-10% slower than the GIL build at the moment. Improvements are landing release-by-release. 4. The honest answer: measure your workload. Some apps will see dramatic wins; some will be flat or marginal. The headline numbers come from benchmarks; production numbers come from your code.

Edge cases to mention: - Refcount contention can be partially mitigated with "immortalisation" of long-lived objects (PEP 683) — common globals get a special refcount that does not change, dodging the contention. - For mixed CPU + I/O workloads, the win may come more from removing the cost of process-based parallelism (no IPC, no startup) than from raw threading speedup. - The 3.13t numbers will improve through 3.14, 3.15 as the team optimises the atomic refcount paths.

What NOT to say: - "You'll see 16x speedup on 16 cores." — Optimistic; refcount contention degrades real workloads. - "There's no performance change." — Pure-Python parallel CPU work definitely scales. - "It's just like Java." — Different memory model; expect different scaling characteristics.

Common follow-up: *"How would you measure GIL contention in 3.13t vs the GIL build today?"* — Run the same workload on `python3.13` (GIL build) and `python3.13t` (no-GIL build). Measure wall-clock time as a function of thread count. The GIL build flatlines beyond ~1 thread for pure Python; the no-GIL build scales near-linearly until refcount contention saturates. The shape of the curve is the diagnostic.

```
# Conceptual benchmark — run this on both builds and compare
import threading, time
def burn(n):
    while n > 0: n -= 1

for n_threads in [1, 2, 4, 8]:
    start = time.perf_counter()
    threads = [threading.Thread(target=burn, args=(10_000_000,))
               for _ in range(n_threads)]
    for t in threads: t.start()
    for t in threads: t.join()
    print(f"{n_threads} threads: {time.perf_counter() - start:.2f}s")

# GIL build: ~ linear in n_threads (4 threads ~= 4x time)
# 3.13t build: ~ constant in n_threads up to N cores
```

Chapter 4 Recap

Fifty questions, six sections, one underlying story — CPython has a GIL because of refcounts; the GIL serialises bytecode but releases on I/O and in well-behaved C extensions; threading helps I/O, multiprocessing helps CPU, asyncio helps concurrent I/O at scale, and the 3.13 free-threaded build will eventually offer pure-Python parallelism without GIL constraints.

Interview today, the bar is: explain the mechanism, pick the right primitive for the workload, name the modern APIs (`TaskGroup` , `asyncio.timeout` , `concurrent.futures`), and have a credible view on PEP 703 and PEP 684 without overclaiming. The candidate who treats the GIL as a punchline ("Python is slow because GIL") loses; the candidate who explains *exactly* when the GIL releases, why it exists, and what the migration path looks like wins. The questions in this chapter are sequenced so that by the end, you can do both.

Next chapter — **Memory + Mutability** — picks up where this leaves off: refcounts as a window into Python's memory model, the `is` vs `==` distinction, shallow vs deep copy, garbage collection, the `__slots__` discipline. Many of the same primitives (`sys.getrefcount` , the reference-counting backbone) recur there.

Chapter 5 — Memory & Mutability (50 Qs)

Python Interview Prep — 2026 Edition · Learn & Excel · File 04 Core Library

This chapter is the one that decides whether the panel marks you as "writes Python" or "understands Python." Every other chapter assumes you know what `is` does, why `dict` keys must be immutable, what the small-int cache is, when `copy.deepcopy` is the wrong tool, and how reference counting interacts with cyclic GC. Indian product cos (Razorpay, Swiggy, Zerodha, Cred, PhonePe, Flipkart) lean on this hard in round 1. Service MNCs (TCS, Infosys, Wipro) ask the canonical `is` vs `==` opener as a tone-setter. GCC panels (Walmart Labs, JP Morgan, Goldman, Microsoft, Target) probe deeper — they want to hear you talk about `__slots__` memory savings in numbers, name a real cyclic-GC scenario, and reach for `weakref` without being prompted.

The chapter runs 50 questions across six sections, each one designed to harden a different muscle of memory reasoning:

- **Section A — Identity vs equality** (10 Qs): `id()` semantics, the `is` operator, the small-int cache, string interning, when `is` is provably safe.
- **Section B — Mutable vs immutable types** (10 Qs): which built-ins are which, why immutability buys hashability, `frozen=True` dataclasses, the `__hash__ = None` pattern.
- **Section C — Reference semantics** (10 Qs): pass-by-object-reference, rebinding vs mutation, the mutable default trap, list aliasing on multiplication, slice as shallow copy.
- **Section D — The `copy` module** (5 Qs): shallow vs deep, `__copy__` / `__deepcopy__` hooks, pandas 2.x copy-on-write, when `deepcopy` is too expensive.
- **Section E — Garbage collection and reference counting** (10 Qs): CPython refcounts, the generational cyclic collector, `gc.collect()`, `weakref` for caches that don't pin objects, `tracemalloc` for leak hunting.
- **Section F — `__slots__` and memory savings** (5 Qs): what `__slots__` does to `__dict__`, the concrete memory savings, inheritance rules, when slots are wrong.

Format per Q: tags (difficulty / asked at / topic) · the interviewer's verbatim phrasing · 30-second answer · numbered step-by-step thought process (what you say out loud) · edge cases · what NOT to say · common follow-up · compiling code on Python 3.12+.

Section A — Identity vs Equality (10 Qs)

Q1 · `is` vs `==` — The Distinction Every Interview Opens With

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Swiggy, Cred · Topic: Identity

The question (interviewer's verbatim phrasing):

What's the difference between `is` and `==` in Python? When would you use one over the other?

The 30-second answer: `==` asks "are these two values equal?" — it calls the object's `__eq__` method. `is` asks "are these two names bound to the same object in memory?" — it compares `id(a)` and `id(b)`. Use `==` for value comparisons (numbers, strings, lists, custom objects). Use `is` only for singletons — `None`, `True`, `False`, and `NotImplemented` — because those are the only objects the language guarantees are unique at runtime.

Step-by-step thought process: 1. Open with the semantic split — value equality versus identity. That single sentence covers 80% of the question. 2. Make the singleton rule explicit: `if x is None` is the right idiom, not `if x == None`. The reason: any class can override `__eq__` to return `True` against `None` (NumPy arrays famously raise on element-wise comparison instead), so `==` can mislead. `is` cannot be overridden — it's an operator built into the interpreter. 3. Mention linter enforcement — `ruff`, `pylint`, `flake8` all flag `if x == None`. Treat it as a hard rule in code review. 4. Be ready for the small-int trap: `256 is 256` evaluates to `True` because of the CPython integer cache, but `257 is 257` is `False` in many contexts. That's an implementation detail of CPython, not a language guarantee — never rely on it.

Edge cases to mention: - `nan == nan` is `False` (IEEE-754 specifies this), but `nan is nan` is `True` if both names point to the same nan object. Both behaviours have caused production bugs. - Comparing custom objects with `==` walks the MRO looking for `__eq__`; comparing with `is` does not — so two objects can be `==` and not `is`. - For boolean checks against `True` / `False`, the linter prefers `if flag:` over `if flag is True:` — the second form is correct but signals "I don't trust Python's truthiness."

What NOT to say: - Do not say `is` is "faster than `==`" as a reason to use it. It is faster, but that's not the reason — correctness is. - Do not write `if x is "tag"` for string comparisons. String interning makes it work for short literals but breaks for dynamically constructed strings — a classic landmine.

Common follow-up: "Why is `if x is None` the recommended idiom for None checks?" — Because `None` is a singleton (only one `NoneType` instance exists per interpreter), identity comparison is provably correct, and `__eq__` could be overridden to lie about None on any custom class.


```
x = None
print(x is None)      # True – correct
print(x == None)      # True but linter-banned

class Liar:
    def __eq__(self, other): return True

print(Liar() == None)  # True – __eq__ lied
print(Liar() is None)  # False – identity cannot lie
```

Q2 · `id()` and What It Actually Returns in CPython

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Walmart Labs, Razorpay · Topic: Identity

The question (interviewer's verbatim phrasing):

What does `id()` return in CPython? Is it the memory address?

The 30-second answer: In CPython, `id(obj)` returns the memory address of the object as an integer — that's an implementation detail of the reference interpreter, not a language guarantee. The language spec only promises that `id()` is unique among objects alive at the same time. Two objects can share the same `id()` if one was garbage-collected before the other was created; that's why holding `id()` values across time is a bug.

Step-by-step thought process: 1. State the implementation reality first — CPython uses the address, PyPy and Jython use a different scheme. The language guarantee is weaker than the CPython behaviour. 2. The uniqueness rule: `id()` is unique only among **simultaneously alive** objects. Once an object is destroyed, its id can be recycled. That's why caches keyed on `id()` need `weakref` to prevent recycling collisions. 3. Mention the `id() % cache_size` hashing pattern from older Python code — it works in CPython because the address has good distribution, but it's a layering violation. 4. The `is` operator is just `id(a) == id(b)` with no Python-level function call — that's why `is` is faster than `==`.

Edge cases to mention: - `id()` on a string literal in two different modules may return the same value because of interning — that's the interpreter being smart, not a bug. - `id(int_literal)` between `-5` and `256` returns the same id every time you ask, because those ints are cached at interpreter startup. - In threaded code, an object's `id()` does not change during its lifetime, but between processes (multiprocessing) the same Python object has different ids in each process because they live in different address spaces.

What NOT to say: - Do not claim `id()` is portable across Python implementations. It is not. - Do not store `id()` values in a long-lived database or cache as object identifiers — the next interpreter run will use different ids.

Common follow-up: "Can two different objects have the same `id()`?" — Not at the same time, but yes across time: if object A is destroyed, its memory can be reused for object B, and `id(B)` could equal what `id(A)` was. That's why a cache that holds `id()` keys without holding the object alive is a use-after-free in disguise.

```
a = [1, 2, 3]
print(id(a))           # e.g., 140234567890

b = a
print(id(a) == id(b))  # True — same object
print(a is b)          # True — equivalent to the above

del a
c = [4, 5, 6]
# id(c) MAY equal the old id(a) — CPython reused the memory
```

Q3 · The Small-Integer Cache (-5 to 256)

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Goldman, Microsoft · Topic: Identity

The question (interviewer's verbatim phrasing):

Why does `a = 100; b = 100; a is b` evaluate to `True`, but the same pattern with `1000` evaluates to `False`?

The 30-second answer: CPython pre-allocates a pool of integer objects for the range `-5` to `256` at interpreter startup and reuses them for every reference to those values. Any integer literal in that range points to the cached object, so `a is b` is `True`. Integers outside the cache get a fresh object allocation each time, so two `1000` literals usually point to different objects — `a is b` is `False`. This is a CPython optimisation, not a language guarantee.

Step-by-step thought process: 1. State the range exactly: `-5` to `256` inclusive. Memorise the boundaries — interviewers will test "is 256 cached?" (yes) and "is 257 cached?" (no). 2. Explain why CPython does it: those values cover 99% of integer literals in real code (loop indices, small constants, status codes, list sizes), and the cache trades a few KB of memory for skipping millions of allocations. 3. Mention the exception: within a single compile unit (a function body, a module), the compiler may also fold identical literals to the same object — that's a separate optimisation called "literal folding," not the small-int cache. So `a = 1000; b = 1000` inside one function can return `True` even though 1000

is outside the cache. 4. The whole reason to know this: it's the trap that breaks careless `is` usage with integers. Never use `is` on integers in production — use `==`.

Edge cases to mention: - The range was `-5` to `256` for many releases; check `sys.maxsize` and `sys.int_info` if you ever need to verify in a specific build. - The cache covers `bool` too — `True is True` and `False is False` are trivially `True` because they are singletons of `bool`. - Strings have their own interning rules (Q5) — they're not part of the integer cache.

What NOT to say: - Do not say "all small numbers are cached" without naming the range — that's the lazy answer panels reject. - Do not say "this is a language feature" — it's a CPython implementation detail. PyPy may cache different ranges or none at all.

Common follow-up: "Can you force CPython to cache other integers?" — There's no public API for it; `sys.intern()` works on strings only. The cache range is hard-coded in the interpreter source. The right answer to "I want to compare integers cheaply" is "use `==` and stop reaching for `is`."

```
a = 256
b = 256
print(a is b)          # True — cached

a = 257
b = 257
print(a is b)          # False (usually) — fresh allocation each time

# Inside one compile unit, peephole folding can lie:
def f():
    a = 1000
    b = 1000
    return a is b      # May return True due to literal folding
```

Q4 · Why `is` Is Safe Against `None`, `True`, `False` (And Almost Nothing Else)

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Cred, JP Morgan · Topic: Identity

The question (interviewer's verbatim phrasing):

The PEP 8 style guide says always use `is` for `None` checks. Why? And what other values is it safe to compare with `is` ?

The 30-second answer: PEP 8 mandates `is None` because `None` is a singleton — there is exactly one `NoneType` instance per Python interpreter, guaranteed by the language. Identity comparison is therefore provably correct and cannot be subverted by a custom `__eq__`. The same logic applies to

`True`, `False`, `NotImplemented`, and `Ellipsis` (`...`) — all singletons. Everything else (numbers, strings, lists, custom objects) should use `==`.

Step-by-step thought process: 1. Anchor on the singleton guarantee. `None` is the only `NoneType` value. `True` and `False` are the only `bool` values. The interpreter intern-ifies them at startup and never makes another. 2. The reason this matters: NumPy arrays and pandas Series override `__eq__` to return an array of comparisons element-wise, which raises in boolean context. `if arr == None` raises a `DeprecationWarning` or returns a confused array; `if arr is None` does the right thing. 3. List the five singletons explicitly — `None`, `True`, `False`, `NotImplemented`, `Ellipsis`. The last two are rarely tested but it shows you've read the data model docs. 4. Generalise: anything you can write as a literal that produces the same object every time across the program is `is`-safe. That includes the singletons above and (by accident) cached small ints and interned strings — but the latter two are not language guarantees, so don't lean on them.

Edge cases to mention: - `is True` and `is False` are technically correct but stylistically discouraged — `if flag:` and `if not flag:` are preferred because Python's truthiness already covers the intent. - `is NotImplemented` is the correct check inside `__eq__` and friends — `return NotImplemented` is how you tell the interpreter to try the reflected operation. You compare with `is`, not `==`. - For sentinel values (private singletons used to distinguish "no value passed" from `None`), define `_MISSING = object()` and compare with `is _MISSING`. Idiomatic, fast, type-correct.

What NOT to say: - Do not say "you can use `is` whenever you want — the interpreter handles it" — that's the answer that loses you the round. - Do not suggest `if x is True` for general boolean checks — it's a code smell that signals you don't trust truthiness.

Common follow-up: "How do you make your own singleton-style sentinel?" — Create a module-level `_MISSING = object()` and use `is _MISSING` to test. Or use `enum.Enum` for a richer pattern. Never use `None` for "no value" if `None` is a valid user value — that's when sentinels earn their keep.

```
import numpy as np

arr = np.array([1, 2, 3])
# print(arr == None)    # DeprecationWarning / element-wise weirdness
print(arr is None)      # False — clean

# Custom sentinel for "no value passed"
_MISSING = object()

def get(d, key, default=_MISSING):
    if key in d:
        return d[key]
    if default is _MISSING:
        raise KeyError(key)
    return default
```

Q5 · String Interning — When `"foo" is "foo"` Returns True (And When It Doesn't)

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Goldman · Topic: Identity

The question (interviewer's verbatim phrasing):

Does `"hello" is "hello"` always return `True` ? What about `"hel" + "lo" is "hello"` ?

The 30-second answer: String interning means the interpreter stores one canonical copy of certain strings and reuses it everywhere. CPython auto-interns short, all-ASCII identifier-shaped strings (`"hello"` , `"name"` , `"_x"`) at compile time, so `"hello" is "hello"` is `True` for those. Dynamically constructed strings (`"hel" + "lo"`) are not auto-interned and usually return `False` . You can force interning with `sys.intern(s)` , but you should still never use `is` for string equality in production — use `==` .

Step-by-step thought process: 1. State the auto-interning rule: short, identifier-shaped string literals get interned at compile time by the peephole optimiser. That's why `"hello" is "hello"` is `True` inside a module, but `"hello world" is "hello world"` (with a space) may not be. 2. Mention the contrast: anything produced by string concatenation, slicing, formatting, file I/O, network read, JSON parsing is **not** auto-interned. Two strings with the same value will usually have different ids. 3. `sys.intern(s)` returns the canonical interned version. Pattern: when you have millions of duplicated string keys (log tags, column names, status codes), interning saves memory because every duplicate becomes the same object. Used heavily by parsers, SQL drivers, and pandas Categorical columns. 4. The bottom line: never use `is` to compare strings in production. Even if it works for literals, it breaks for everything else. The performance win is marginal and the bug surface is large.

Edge cases to mention: - The Python source code uses `sys.intern` for module names, function names, and dict keys — most attribute lookups internally hit interned strings, which is one reason attribute access is fast. - f-strings produce fresh string objects every time they evaluate — `f"hello" is f"hello"` is implementation-defined and can vary across CPython versions. - For Python 3.12+, the compiler is more aggressive about constant folding, so some expressions you'd expect to allocate (e.g., `"a" * 5`) can produce interned strings if the result is short.

What NOT to say: - Do not claim "string interning is always on for short strings" — the exact rules are implementation details and have changed across CPython versions. - Do not use `is` to test if two parsed JSON keys are the same — they may have been read by different decoder paths and never share identity.

Common follow-up: "When would `sys.intern` save real memory?" — When you have a large number of duplicated string keys (e.g., 10M dicts each with the same 30 keys). Interning collapses 300M string objects into 30. The CSV/JSON parsers in `pandas` do this internally for `Categorical` and `string[pyarrow]` dtypes.

```
import sys

print("hello" is "hello")      # True – auto-interned literal
print("hello world" is "hello world") # Often True (compile-time)

a = "hel"
b = a + "lo"
print(b == "hello", b is "hello") # True, often False

s1 = sys.intern(b)
s2 = sys.intern("hello")
print(s1 is s2)                # True – forced interning
```

Q6 · `id()` Recycling After Object Destruction

Tags: Difficulty: Medium · Asked at: Microsoft, Walmart Labs, Goldman, Razorpay · Topic: Identity

The question (interviewer's verbatim phrasing):

If I do `x = []`; `old_id = id(x)`; `del x`; `y = []`, can `id(y)` equal `old_id` ?

The 30-second answer: Yes. Once `x` is destroyed, its memory is released back to the allocator. The next allocation of a similar-sized object can land at the same address, so `id(y)` can equal the old `id(x)`. That's why `id()` is only unique among simultaneously alive objects — not across object lifetimes. Caches that key on `id()` without holding the object alive risk silently mixing up unrelated objects.

Step-by-step thought process: 1. State the language guarantee precisely: `id()` is unique only among objects alive at the same time. Once an object is dead, its id is available for reuse. 2. Explain the practical implication: a cache like `{id(obj): expensive_result}` is unsafe if `obj` can be garbage-collected. After `obj` dies, a new object with the same id can collide with the cached entry. 3. The fix: use `weakref.WeakKeyDictionary` (or `WeakValueDictionary`) which automatically removes entries when the referenced object is collected. The dict's keys are weakrefs, not the objects themselves, so they don't pin objects in memory but also don't suffer id-recycling bugs. 4. The pattern shows up in: memoisation caches, identity-keyed maps, observer registrations. If you've ever built a "remember this object I saw" cache, you've hit this.

Edge cases to mention: - CPython's pymalloc allocator pools small objects by size, so id recycling is very likely for short-lived small objects. For large objects allocated via `malloc`, the OS may also recycle addresses. - The recycling is not deterministic — it depends on allocator state, GC timing, and other allocations. Bugs may pass tests and fail in production. - The CPython `is` operator is safe across this gap because it's evaluated at the moment of comparison — you can never compare an old destroyed object with `is` because the name pointing to it is also gone.

What NOT to say: - Do not claim "objects keep their id forever" — they don't. Lifetime matters. - Do not build production code that stores `id()` values in long-lived data structures without keeping references to the objects.

Common follow-up: "What's the right pattern for an identity-keyed cache?" —

`weakref.WeakKeyDictionary` if the cached value should die with the key;

`weakref.WeakValueDictionary` if the cached key is permanent but the value should die when no one else holds it; for objects that don't support weakrefs (like `int`, `tuple`), you can't do it — use

`==` on a hashable representation instead.

```
import weakref

cache = weakref.WeakKeyDictionary()

class User:
    def __init__(self, name):
        self.name = name

u = User("Aanya")
cache[u] = "expensive_computation"
print(cache[u])          # 'expensive_computation'

del u                    # u is collected; cache entry auto-removes
print(len(cache))        # 0
```

Q7 · `is not` vs `not is` — Why Python Has a Dedicated Operator

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay · Topic: Identity

The question (interviewer's verbatim phrasing):

What's the difference between `x is not None` and `not x is None`? Are they ever different?

The 30-second answer: They're equivalent in behaviour because `is not` is a single operator with the same precedence as `is`, so `not x is None` is parsed as `not (x is None)` — the result is the same. Stylistically, PEP 8 mandates `x is not None`: it reads as natural English ("x is not None") and avoids any confusion about precedence. The variant `not x is None` is correct but looks wrong, like a precedence bug waiting to bite a future reader.

Step-by-step thought process: 1. Lead with the precedence answer — `is` binds tighter than `not`, so `not x is None` evaluates as `not (x is None)`. Both forms produce the same result. 2. State the PEP 8 rule explicitly: prefer `is not`. The standard library, every linter, every code review tool flag

`not ... is` as a style violation. 3. Mention readability: `if x is not None:` reads as "if x is not None"; `if not x is None:` reads as "if not x is None" which is harder to parse mentally and could be mistaken for `if (not x) is None`. 4. The same pattern exists for `in / not in`. `x not in lst` is the right form, not `not x in lst`.

Edge cases to mention: - `is not` is a single token in the Python grammar, lexed as one operator. You cannot put a comment or other code between `is` and `not`. - The `not in` operator is similarly atomic — `x not in lst` is one operator, not the negation of `in`. - This is one of the few places in Python grammar where two whitespace-separated words form a single operator. The others are `not in` and `and not` / `or not` (which is just precedence).

What NOT to say: - Do not claim `not x is None` is buggy — it's not, it's just stylistically wrong. - Do not suggest `x != None` as the modern alternative — that's the `== / !=` variant and PEP 8 prohibits both.

Common follow-up: "What's the correct way to check 'list is not empty'?" — `if lst:` (uses truthiness — empty list is falsy) is the most Pythonic. `if len(lst) > 0:` is correct but verbose. `if lst is not None and lst:` is the right form when `None` and `[]` need different handling.

```
x = None

if x is not None:
    print("happens when x has a value")

if not x is None:    # same behaviour, ugly style
    print("happens when x has a value")

# PEP 8 also flags this:
if x != None:       # use 'is not None' instead
    pass
```

Q8 · `sys.intern()` — When Manual Interning Pays Off

Tags: Difficulty: Hard · Asked at: Walmart Labs, Goldman, Razorpay (perf rounds) · Topic: Identity

The question (interviewer's verbatim phrasing):

When would you reach for `sys.intern()` in a real codebase? Give an example where it materially helps.

The 30-second answer: `sys.intern(s)` returns the canonical interned version of a string, so two equal interned strings share identity. The classic use case is processing large volumes of repeated string keys — log parsing, JSON ingestion, SQL column names, dict-key normalisation. If you have 10 million

records each carrying the string `"status"` as a key, interning collapses them into one object, saving memory (~50 bytes per duplicate on CPython 3.12). It also speeds up subsequent dict lookups marginally because hash + identity check beats hash + equality check.

Step-by-step thought process: 1. Anchor the use case in scale — manual interning matters when you have millions of duplicates, not when you have a few hundred. 2. Show the concrete savings: a CPython string object is about 50 bytes plus the character data. Ten million duplicated 6-char strings is roughly 500 MB; interning brings that to one object plus 10 million pointers (80 MB on a 64-bit system). That's a 6x memory win. 3. Mention the dict-lookup speedup: CPython's dict implementation does pointer-equality as a fast path before falling back to full string equality. Interned keys hit that fast path every time. 4. The libraries that do this internally: `pandas` for `Categorical` and `string[pyarrow]` dtypes; `lxml` for XML element tags; CPython itself for module names and function attribute names.

Edge cases to mention: - `sys.intern` works only on `str`; for `bytes` there's no equivalent — `bytes` objects are not interned at the language level. - Interned strings are held by an internal table and never garbage-collected as long as the interpreter runs. For long-running services, interning user-controlled input is a slow leak — don't intern arbitrary user strings without limits. - The interning table is global to the interpreter. In multi-interpreter Python (PEP 684, 3.12+), each subinterpreter has its own table.

What NOT to say: - Do not intern every string you see — the table grows without bound and never reclaims memory. - Do not claim `sys.intern` makes string equality "always O(1)" — it's still O(n) in the worst case; the fast path is pointer equality, but a hash collision still falls back to full compare.

Common follow-up: *"What if you intern user-controlled input?"* — Memory leak vector. Each unique input adds a permanent entry to the intern table. For an API receiving arbitrary strings, this is unbounded growth. The pattern in safe code is: intern only known, bounded vocabularies (column names, log levels, enum-like values), never raw user input.

```
import sys

# Without interning: each record allocates a new "status" string
records = [{"status": "ok"} for _ in range(10_000_000)]

# With interning: every "status" key is the same object
key = sys.intern("status")
records = [{key: "ok"} for _ in range(10_000_000)]
# Memory: ~80MB lower, dict lookups hit the pointer-equality fast path
```

Q9 · Identity Caching of `bool` and `NoneType`

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Infosys · Topic: Identity

The question (interviewer's verbatim phrasing):

Why are `True`, `False`, and `None` always safe to compare with `is`?

The 30-second answer: Because they are singletons — the language guarantees that `NoneType` has exactly one instance, and `bool` has exactly two (`True` and `False`). The interpreter creates them at startup and never creates more, even when you write `bool(1)` or `type(None)()`. So `x is True`, `x is False`, `x is None` are provably correct, cannot be defeated by overriding `__eq__`, and are the canonical idiom in Python.

Step-by-step thought process: 1. Make the singleton claim precise: there is exactly one `None`, one `True`, one `False` per interpreter. The language spec backs this. 2. Show the constructor trap doesn't break it: `bool(1)` returns the same `True` object every time. `type(None)()` returns the same `None`. The constructors are written to return the cached singletons. 3. Mention `NotImplemented` (returned from `__eq__` etc.) and `Ellipsis` (the `...` literal) are also singletons. `return NotImplemented` is the pattern in dunder methods. 4. The corollary: comparing `bool` with `==` against `int` is dangerous because `True == 1` returns `True` (`bool` inherits from `int`). `True is 1` returns `False`. If you want strict boolean comparison, `is True` is unambiguous.

Edge cases to mention: - `bool` is a subclass of `int`. `True + 1 == 2`. `isinstance(True, int)` is `True`. That's why `True == 1` is `True` and only `True is 1` distinguishes them. - `None` is not a keyword in Python 2 but is in Python 3 — you cannot assign to it (`None = 5` is a `SyntaxError`). Same for `True` and `False`. - In multi-interpreter Python, each interpreter has its own `None`, `True`, `False`. Cross-interpreter `is` comparisons may surprise you, but that's an edge case in 3.12+ subinterpreters.

What NOT to say: - Do not say "True equals 1 so they're the same" — they are equal but not identical. `True == 1` is `True`; `True is 1` is `False`. - Do not use `if x == True` — Python's truthiness already covers it as `if x`, and `is True` is the strict form.

Common follow-up: "What does `isinstance(True, int)` return and why?" — `True`, because `bool` inherits from `int`. `True` and `False` are integer instances at the language level. That's why `sum([True, True, False])` is `2`. Surprising at first; idiomatic once you've seen it.

```
print(True is True)           # True – singleton
print(False is False)        # True – singleton
print(None is None)          # True – singleton

print(True == 1)              # True – value equality (bool < int)
print(True is 1)              # False – identity
print(isinstance(True, int))  # True – bool inherits from int

print(bool(1) is True)        # True – bool(1) returns the cached True
```

Q10 · Why You Should Never Use `is` for Integer or String Equality in Production

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Infosys · Topic: Identity

The question (interviewer's verbatim phrasing):

Even though `100 is 100` is `True` and `"hello" is "hello"` is often `True`, you said never use `is` for these. Why?

The 30-second answer: Because both behaviours rely on CPython implementation details — the small-int cache for integers (-5 to 256) and the peephole interning of short string literals. As soon as values escape those narrow conditions (a number above 256, a string built from concatenation or read from a file, a value from a different interpreter), `is` returns `False` even when the values are equal. The result is bugs that pass unit tests with literals and fail in production with computed values. `==` is correct in both cases and only marginally slower.

Step-by-step thought process: 1. Restate the implementation-detail framing — the question is testing whether you treat `is` as a feature or as the leaky abstraction it is. 2. Walk a concrete failure mode: a user lookup keyed by user id. Tested with `assert user_id is 42` because that worked in the REPL. In production, ids come from the database as freshly allocated `int` objects in the millions range. The `is` check silently returns `False` for every real user. 3. Same for strings: a status filter `status is "active"` works in tests where `status` is a literal. In production, `status` is read from a database row and is a fresh string. The comparison silently fails — and worse, `is "active"` doesn't raise; it just doesn't match. 4. The fix is `==` everywhere except the four documented singletons.

Edge cases to mention: - The peephole optimiser may fold `"act" + "ive"` to `"active"` at compile time inside a single function, making `is "active"` work there but fail when the same logic moves to a multi-statement form. - Python 3.12 issues a `SyntaxWarning` for `is` with a literal — `if x is 42` triggers it. That warning was added precisely because the trap had become common. - Some legacy code uses `is "frozen"` or `is 0` as a "fast check" — it's incorrect; the speed difference is microscopic; the bug surface is large.

What NOT to say: - Do not say `is` "works for short strings because they're interned" as if it were a guarantee — it isn't, and your code reviewer will reject it. - Do not optimise from `==` to `is` for "performance" — you're trading nanoseconds for correctness.

Common follow-up: "What did Python 3.12 add to help catch this?" — `SyntaxWarning` on `is` against literals. The interpreter now warns at compile time when it sees `x is 0`, `x is "foo"`, `x is 1.5`. Treat the warning as a hard error in CI.

```
# Python 3.12+ raises SyntaxWarning here:
# x is 42
# x is "active"

# Correct forms:
user_id = 42
print(user_id == 42)          # True – value equality

status = "active"
print(status == "active")     # True – value equality

# Only these are 'is'-safe:
x = None
print(x is None)              # True – singleton
```

Section B — Mutable vs Immutable Types (10 Qs)

Q11 · The Built-in Mutable vs Immutable Catalogue

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Swiggy, JP Morgan · Topic: Mutability

The question (interviewer's verbatim phrasing):

List the mutable and immutable built-in types in Python. Why does the distinction matter?

The 30-second answer: Immutable built-ins: `int`, `float`, `complex`, `bool`, `str`, `bytes`, `tuple`, `frozenset`, `range`, `NoneType`. Mutable built-ins: `list`, `dict`, `set`, `bytearray`, most user classes. The distinction matters because: (1) only immutable objects are hashable by default, so only immutable types can be dict keys or set elements; (2) mutable objects passed as function arguments can be modified in place, which affects callers; (3) default arguments are evaluated once, so a mutable default leaks state across calls.

Step-by-step thought process: 1. Lead with the explicit catalogue — interviewers want to hear "int, float, str, bytes, tuple, frozenset, range, NoneType" rattled off without hesitation, then the mutable counterparts. 2. State the hashability rule: `hash(obj)` requires the object's value to be stable for the object's lifetime. Mutable types can't promise that, so they have `__hash__ = None` by default. 3. The three downstream consequences: dict-key constraint, function-argument behaviour, default-argument trap. Each one shows up in interview questions for the rest of this chapter. 4. The grey areas: `tuple` of immutables is hashable, but `tuple` containing a list is not — Python checks recursively. `frozenset` exists specifically to be a hashable set.

Edge cases to mention: - `bytes` is immutable; `bytearray` is the mutable sibling. Same data model, different mutability. - `frozenset` is immutable; `set` is mutable. Same data model. - Custom classes default to mutable unless you opt in with `__slots__`, `frozen=True` dataclasses, or careful `__setattr__` overrides.

What NOT to say: - Do not call `tuple` "always immutable" without the recursive caveat — a tuple of lists is unhashable. - Do not say `str` is mutable "because you can do `s += 'x'` " — that's rebinding, not mutation; the original string object never changes.

Common follow-up: "If `str` is immutable, how does `s += 'x'` work in a loop?" — Each iteration allocates a new string and rebinds `s`. The complexity is $O(n^2)$ in the total length. The right pattern is `"".join(parts)` or building a list and joining at the end.

```
# Immutable types – hashable by default
hash(42)           # works
hash("hello")      # works
hash((1, 2, 3))    # works
hash(frozenset({1, 2})) # works

# Mutable types – unhashable
try:
    hash([1, 2, 3])
except TypeError as e:
    print(e)          # unhashable type: 'list'

try:
    hash({1: 2})
except TypeError as e:
    print(e)          # unhashable type: 'dict'
```

Q12 · Why Immutable Types Are Hashable

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, Microsoft, Swiggy · Topic: Mutability

The question (interviewer's verbatim phrasing):

Why does Python tie hashability to immutability? Why can't a list be a dict key?

The 30-second answer: A dict (or set) places each key in a bucket based on the key's `hash()` value, and finds it later by recomputing the hash and walking the bucket. For this to work, the key's hash must never change while it's stored — otherwise the dict can't find it in the right bucket. Immutable objects can promise a stable hash because their value never changes. Mutable objects can't, so allowing them as keys would silently corrupt the dict on mutation.

Step-by-step thought process: 1. Frame the contract: `hash(x)` must equal `hash(y)` whenever `x == y`. And `hash(x)` must not change for the lifetime of `x` while it's a dict key. 2. Walk the failure mode for a mutable key: store `lst = [1, 2]` as a key with `hash([1, 2])`. Mutate it: `lst.append(3)`. The dict's internal slot was chosen by `hash([1, 2])` but now `hash(lst)` (if it existed) would be `hash([1, 2, 3])` — they'd land in different buckets. The next lookup misses the key even though it's there. 3. Python's solution: mutable types simply don't define `__hash__` (it's set to `None`), so attempting `hash([1, 2])` raises `TypeError: unhashable type: 'list'`. The check is at insertion time, not at corruption time. 4. The `frozenset` exists because sets are mutable; `tuple` works as a key because tuples are immutable. Same pattern.

Edge cases to mention: - A `tuple` containing a list is technically immutable at the tuple level but unhashable because hashing recurses into elements: `hash((1, [2, 3]))` raises `TypeError` because list is unhashable. - If you override `__eq__` on a custom class, you must also override `__hash__` or set `__hash__ = None`; otherwise the default hash (based on id) won't match equal-but-not-identical instances. - A `dataclass(frozen=True)` synthesises `__hash__` automatically because the equality contract is now stable.

What NOT to say: - Do not say "mutable types can't be dict keys because of how memory is allocated" — that misses the point. The reason is the contract between `__eq__` and `__hash__`. - Do not propose "freeze the list before using it as a key" — there's no built-in freeze; use `tuple(lst)` to convert to a hashable representation.

Common follow-up: "If I override `__eq__` on my custom class, what happens to `__hash__`?" — Python automatically sets `__hash__` to `None`, making the class unhashable. The rule: if you redefine equality, you must redefine hashing too — either implement `__hash__` returning a value consistent with `__eq__`, or accept the unhashable default.

```
# Tuple of lists – unhashable because recursive
try:
    hash((1, [2, 3]))
except TypeError as e:
    print(e)          # unhashable type: 'list'

# Convert list to tuple for keying
lst = [1, 2, 3]
key = tuple(lst)
d = {key: "stored"}
print(d[(1, 2, 3)])  # 'stored'

# Custom class: override __eq__ kills __hash__
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __eq__(self, other):
        return (self.x, self.y) == (other.x, other.y)

try:
    hash(Point(1, 2))
except TypeError as e:
    print(e)          # unhashable type: 'Point'
```

Q13 · `__hash__ = None` — The Explicit Opt-Out

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Microsoft · Topic: Mutability

The question (interviewer's verbatim phrasing):

What does setting `__hash__ = None` on a class do, and when would you do it?

The 30-second answer: Setting `__hash__ = None` explicitly marks the class as unhashable, so `hash(instance)` raises `TypeError` and instances cannot be used as dict keys or set elements. You'd do this when the class is mutable and you want to prevent surprising bugs from someone accidentally using an instance as a key. It's also the value Python automatically assigns when you override `__eq__` without overriding `__hash__` — making the unhashability explicit.

Step-by-step thought process: 1. State the mechanism — `__hash__ = None` at class scope makes the class unhashable. It's the same machinery the interpreter uses internally when you override `__eq__`. 2. Explain the philosophy: hashing is part of the equality contract. If you redefine equality, the old id-based hash is wrong. Better to fail loudly at `hash()` than silently corrupt a dict. 3. The pattern shows up in: classes representing mutable state (a `Connection`, a `RequestContext`), classes you want to prevent from being cached by identity, and abstract base classes that don't have a meaningful

hash. 4. The contrast: if you want a hashable mutable class, you must implement `__hash__` returning something stable and consistent with `__eq__` — usually based on a unique id field, not mutable state.

Edge cases to mention: - `__hash__ = None` is the value, not `__hash__ = lambda self: None` — the latter would make `hash(instance)` return 0, which is wrong. - A subclass can reset `__hash__` to a method if needed; it's not permanent at the class hierarchy level. - `dict.fromkeys([obj1, obj2])` and `set([obj1, obj2])` both call `hash()` on each element, so unhashable classes fail there too — useful when you want a class that can never be silently deduplicated.

What NOT to say: - Do not claim `__hash__ = None` "speeds up the class" — it doesn't change runtime cost; it just prevents one operation. - Do not implement `__hash__` returning `id(self)` for a mutable class — that defeats the equality contract because `__eq__` should compare values, not identity.

Common follow-up: *"How do you make a mutable class hashable on purpose?"* — Override both `__eq__` and `__hash__`. The hash must be consistent with equality (equal objects → equal hashes) and stable for the object's lifetime as a key. Usually this means hashing a unique immutable id field, never the mutable state.

```
class MutableState:
    def __init__(self, value):
        self.value = value

    def __eq__(self, other):
        return self.value == other.value

    __hash__ = None    # explicit: don't use me as a key

try:
    hash(MutableState(42))
except TypeError as e:
    print(e)           # unhashable type: 'MutableState'

try:
    d = {MutableState(1): "x"}
except TypeError as e:
    print(e)           # same - set/dict construction fails
```

Q14 · `frozen=True` Dataclasses and Hashability

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, JP Morgan · Topic: Mutability

The question (interviewer's verbatim phrasing):

What does `@dataclass(frozen=True)` do, and why would I want it instead of a regular dataclass?

The 30-second answer: `@dataclass(frozen=True)` produces an immutable dataclass: setting attributes after construction raises `FrozenInstanceError`, and the synthesised `__hash__` makes instances hashable so they can be used as dict keys, set elements, or `lru_cache` arguments. Use it for value objects — money amounts, geographic coordinates, configuration snapshots, anything where equality should be by value and the instance shouldn't change after creation.

Step-by-step thought process: 1. State the two effects: immutability (no assignment after `__init__`) and hashability (auto-generated `__hash__` based on the field tuple). 2. Walk the use cases — value objects are the canonical pattern: a `Money(amount, currency)` should never have its amount or currency changed in place; if you need a different amount, build a new `Money`. That maps cleanly to `frozen=True`. 3. Mention the `__post_init__` exception: inside `__post_init__`, you cannot do `self.x = y` directly because the instance is already frozen. The workaround is `object.__setattr__(self, 'x', y)` — ugly but documented. 4. Contrast with a regular dataclass: by default, dataclasses are mutable and unhashable (because `__eq__` is auto-generated and that nukes the inherited `__hash__`). `frozen=True` re-enables hashing in the right way.

Edge cases to mention: - `frozen=True` only blocks attribute assignment via `instance.x = y`. If a field holds a mutable object (a list), you can still mutate that list. The freeze is shallow. - For deep immutability, use immutable field types: `tuple` not `list`, `frozenset` not `set`, `MappingProxyType` not `dict`. - `eq=True` is the default; `frozen=True` requires `eq=True` because freezing implies a stable equality contract.

What NOT to say: - Do not say `frozen=True` "makes the instance fully immutable" without the shallow-freeze caveat — a frozen instance with a list field is not deeply immutable. - Do not claim `frozen=True` is "always slower" — the synthesised `__setattr__` adds one check, which is negligible compared to anything else you'd do with the instance.

Common follow-up: "How does `frozen=True` interact with `slots=True` (Python 3.10+)?" — They compose cleanly: `@dataclass(frozen=True, slots=True)` gives you an immutable, slotted, hashable class. The combination is the closest Python has to a C-struct-flavoured value object. Memory savings are real (no `__dict__`); the instance is genuinely fixed at construction.

```

from dataclasses import dataclass

@dataclass(frozen=True)
class Money:
    amount: int
    currency: str

m = Money(100, "INR")
print(hash(m))           # works — frozen → hashable
basket = {m: 1}          # works — usable as dict key

try:
    m.amount = 200
except Exception as e:
    print(type(e).__name__, e)
# FrozenInstanceError: cannot assign to field 'amount'

```

Q15 · Tuples Are Immutable — Until They Contain a List

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Cognizant · Topic: Mutability

The question (interviewer's verbatim phrasing):

Is `t = (1, [2, 3])` immutable? Can I mutate it?

The 30-second answer: The tuple itself is immutable — you cannot replace its elements: `t[1] = [4, 5]` raises `TypeError`. But the list inside the tuple is still mutable: `t[1].append(4)` works and changes what `t[1]` shows. The tuple holds references; the references are fixed, but what they point to can change. That's why `hash(t)` raises — the tuple is unhashable because it contains an unhashable element.

Step-by-step thought process: 1. Separate the two layers: the tuple's slots (immutable — cannot swap one element for another) and the contents of each slot (mutable if the element is mutable). 2. Walk the failure case: assigning `t[1] = [4, 5]` fails because tuples don't support item assignment. But `t[1].append(4)` succeeds because you're calling a method on the list object, not on the tuple. 3. Hashing is the practical consequence: `hash((1, [2, 3]))` raises `TypeError: unhashable type: 'list'`. Python checks hashability of each element — if any element is unhashable, the tuple is unhashable. 4. The fix for "I need a truly immutable container" is `tuple(map(tuple, nested))` or use `frozenset` for set semantics. Or wrap with `MappingProxyType` for a read-only dict view.

Edge cases to mention: - `tuple` is the only built-in container that has both an immutable structure and possibly-mutable contents. `frozenset` requires all elements to be hashable, and dicts have similar key requirements. - A tuple of all-immutable elements is hashable and safe as a dict key: `(1,`

"a", (2, 3)) . Mix in any list, dict, or set and it becomes unhashable. - `tuple` comparison is element-wise — `(1, [2, 3]) == (1, [2, 3])` is `True` because the lists are compared by value. But the equality might give different results before and after mutation.

What NOT to say: - Do not say "tuples are fully immutable" without the contents caveat — the panel is testing whether you understand the distinction. - Do not propose using a mutable-containing tuple as a dict key with a workaround — there is no workaround; convert to a fully immutable representation.

Common follow-up: "What's the fully immutable equivalent of a nested list?" —

`tuple(tuple(inner) for inner in outer)` — recursively converts every level to tuples. For dicts, use `frozenset(d.items())` . For sets, use `frozenset(s)` . For mixed structures, write a recursive freeze helper.

```
t = (1, [2, 3])

# Tuple structure is immutable
try:
    t[1] = [4, 5]
except TypeError as e:
    print(e)          # 'tuple' object does not support item assignment

# But the list inside is mutable
t[1].append(4)
print(t)              # (1, [2, 3, 4])

# Hashing fails – recursive check
try:
    hash(t)
except TypeError as e:
    print(e)          # unhashable type: 'list'
```

Q16 · `bytes` vs `bytearray` — The Mutability Twin

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, JP Morgan, Cognizant · Topic: Mutability

The question (interviewer's verbatim phrasing):

What's the difference between `bytes` and `bytearray` ? When would you reach for `bytearray` ?

The 30-second answer: `bytes` is immutable — you cannot change its contents after construction. `bytearray` is the mutable sibling — same API otherwise, but you can do `ba[0] = 65` , `ba.append(66)` , `ba.extend(b"more")` . Reach for `bytearray` when you need to build up binary data incrementally (constructing a network packet, accumulating bytes from a stream, in-place

editing of binary buffers). Use `bytes` for everything else — keys in a dict, function arguments where you don't want callers mutating, anything that should be stable.

Step-by-step thought process: 1. State the mutability contrast — same data model, mutability differs. `bytes` is to `bytearray` what `tuple` is to `list`, what `frozenset` is to `set`. 2. The performance angle: building a 1 MB `bytes` by repeated concatenation is $O(n^2)$ — each `+=` allocates a new bytes. Building the same 1 MB with `bytearray.extend()` is $O(n)$ — single buffer with amortised growth. For streaming binary I/O, this matters. 3. Hashability follows the pattern: `bytes` is hashable, `bytearray` is not. You can use `bytes` as a dict key (common for caching binary blobs by content); `bytearray` requires conversion via `bytes(ba)` first. 4. Conversion is cheap and explicit: `bytes(ba)` and `bytearray(b)` are $O(n)$ copies. Most APIs accept both via the buffer protocol, so you rarely need to convert.

Edge cases to mention: - `memoryview` over either gives you a zero-copy view; mutating through a memoryview of a `bytes` raises, of a `bytearray` works. - `bytearray.fromhex("ab cd")` and `bytes.fromhex(...)` both exist and skip whitespace — useful for parsing hex-encoded protocol data. - The repr of `bytearray` looks like `bytearray(b'...')` — distinctive enough that you can spot it in debug output even without a type check.

What NOT to say: - Do not use `bytes += chunk` in a hot loop for building output — that's $O(n^2)$; use `bytearray` and convert at the end. - Do not claim `bytearray` is "always faster than bytes" — it's faster for incremental construction; for read-only access they're the same.

Common follow-up: "How do you efficiently build a large `bytes` object from many small chunks?" — `b"".join(chunks)` if you have them in a list; `bytearray` accumulator if you're streaming. Both are $O(\text{total length})$. Never use `+=` in a loop on `bytes`.

```
# Immutable: every += allocates a new bytes object
data = b""
for chunk in [b"abc", b"def", b"ghi"]:
    data += chunk
print(data)                # b'abcdefghi'

# Mutable: bytearray accumulates in place
buf = bytearray()
for chunk in [b"abc", b"def", b"ghi"]:
    buf.extend(chunk)
print(bytes(buf))          # b'abcdefghi' — convert at the end

# Item mutation
ba = bytearray(b"hello")
ba[0] = ord("H")
print(ba)                  # bytearray(b'Hello')
```

Q17 · Custom Class Mutability — The Default and How to Restrict It

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Microsoft, JP Morgan · Topic: Mutability

The question (interviewer's verbatim phrasing):

Is a regular custom class mutable by default? How would you make one immutable?

The 30-second answer: Yes — by default, custom classes are fully mutable. You can set any attribute (`instance.x = 42`) and the existing ones can be reassigned. To make a class immutable, you have four options ranked by readability: (1) `@dataclass(frozen=True)` — cleanest for value objects; (2) override `__setattr__` to raise after init — manual control; (3) `__slots__` plus careful initialisation — restricts which attributes exist but doesn't prevent assignment; (4) use a `NamedTuple` — fully immutable but tuple-like indexing.

Step-by-step thought process: 1. Anchor the default: custom classes inherit from `object`, which allows attribute assignment. There's no "immutable by default" mode. 2. Rank the options by use case. `frozen=True` dataclass is the right answer for value objects. `__setattr__` override is for cases where the dataclass synthesis is too restrictive. `NamedTuple` is when you want positional and named access on a small struct. 3. Mention the `__slots__` confusion: slots restrict the set of attributes but don't make them read-only. You can still do `instance.x = 5` after init — slots don't help immutability. 4. The pattern for partial immutability (some fields fixed, some mutable): set the fixed ones in `__init__` and override `__setattr__` to reject only those names. Common for "id is fixed but state can change" classes.

Edge cases to mention: - A class with `__slots__` and no `__dict__` cannot have new attributes added at runtime — that's a different restriction from immutability of existing attribute values. - `types.MappingProxyType` wraps a dict in a read-only view — useful when you want to expose an immutable view of mutable internals. - Class-level attributes (defined on the class, not the instance) can also be mutated unless you use `__init_subclass__` or metaclass tricks to lock them.

What NOT to say: - Do not claim `__slots__` makes a class immutable — it only restricts which attribute names are allowed, not whether their values change. - Do not override `__setattr__` to silently swallow assignments — that's the worst pattern; the assignment looks like it worked but didn't.

Common follow-up: "What if I want only some attributes immutable?" — Override `__setattr__` to allow assignment only if the name is not in a "fixed" set, or only during `__init__`. Pattern: `if hasattr(self, name) and name in self._fixed: raise AttributeError(...)`. Verbose but explicit.

```

from dataclasses import dataclass

# Option 1: dataclass frozen
@dataclass(frozen=True)
class Coord:
    lat: float
    lon: float

# Option 2: manual __setattr__ override
class Locked:
    def __init__(self, x):
        object.__setattr__(self, "x", x)
        object.__setattr__(self, "_locked", True)

    def __setattr__(self, name, value):
        if getattr(self, "_locked", False):
            raise AttributeError(f"{name} is read-only")
        object.__setattr__(self, name, value)

l = Locked(42)
try:
    l.x = 100
except AttributeError as e:
    print(e)          # x is read-only

```

Q18 · `NamedTuple` vs `frozen=True` Dataclass — Which One to Pick

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred · Topic: Mutability

The question (interviewer's verbatim phrasing):

When would you choose `typing.NamedTuple` over `@dataclass(frozen=True)` ?

The 30-second answer: Both produce immutable, hashable value objects with auto-generated `__eq__`, `__hash__`, and `__repr__`. `NamedTuple` adds tuple semantics — positional access (`p[0]`), tuple unpacking (`x, y = p`), and slicing. Use `NamedTuple` when callers benefit from treating the instance as a small tuple (return values from a function, coordinate-style data). Use `frozen=True` dataclass when you want richer features — default values, `__post_init__` validation, inheritance, `slots=True`, custom methods — without tuple semantics.

Step-by-step thought process: 1. State the common ground: both are immutable, hashable, eq-by-value. Both produce small struct-like instances with named fields. 2. The tuple semantics of `NamedTuple` are sometimes a feature and sometimes a footgun. Feature: returning `Result(value, error)` and unpacking at the call site. Footgun: someone iterates over the instance accidentally, or compares it as a tuple to a regular tuple — `Point(1, 2) == (1, 2)` is `True`, which can mask

type bugs. 3. Dataclass features that `NamedTuple` lacks: `__post_init__`, default values with mutable handling, inheritance hierarchy, `frozen=True, slots=True` combo, easy customisation per field via `field()`. 4. Performance: `NamedTuple` is a `tuple` subclass with slots-backed storage — very compact. `dataclass(frozen=True, slots=True)` is similarly compact. Functional parity for memory; pick on semantics.

Edge cases to mention: - `NamedTuple` cannot have a method named `_fields` or any name starting with `_` (those are reserved by the namedtuple machinery). - Multiple inheritance is awkward with `NamedTuple` (it's already a tuple subclass); dataclasses chain inheritance more cleanly. - Pre-3.6 `NamedTuple` used the `collections.namedtuple` API which is functional, not class-based — modern code uses the class-based `typing.NamedTuple`.

What NOT to say: - Do not say `NamedTuple` is "the lighter version" without qualification — they're similar in cost; choose on semantics. - Do not propose `NamedTuple` for a class with methods or complex validation — those are dataclass strengths.

Common follow-up: "What does `Point(1, 2) == (1, 2)` return?" — `True`, because `NamedTuple` instances are real tuples and compare as tuples. That's why `frozen=True` dataclass is safer when you don't want accidental equality with plain tuples — dataclass equality requires the types to match.

```
from typing import NamedTuple
from dataclasses import dataclass

class Point(NamedTuple):
    x: float
    y: float

@dataclass(frozen=True)
class Coord:
    x: float
    y: float

p = Point(1, 2)
c = Coord(1, 2)

print(p == (1, 2))      # True — tuple comparison
print(c == (1, 2))      # False — dataclass requires type match
print(p[0], p.x)        # 1 1 — both work
# print(c[0])           # TypeError — no item access on dataclass
```

Q19 · `MappingProxyType` — A Read-Only View Over a Dict

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Microsoft, Walmart Labs · Topic: Mutability

The question (interviewer's verbatim phrasing):

How do you expose a dict as a read-only attribute on a class without copying it?

The 30-second answer: Wrap it in `types.MappingProxyType(d)`. The proxy supports all read operations (`__getitem__`, `__contains__`, `keys()`, `values()`, `items()`, iteration) but raises `TypeError` on any write operation. It's a view, not a copy, so changes to the underlying dict are visible through the proxy — meaning you keep one source of truth without giving callers the ability to mutate. The class itself is the same machinery the interpreter uses for `__dict__` of a class object: that's why `MyClass.__dict__` is a `MappingProxyType`.

Step-by-step thought process: 1. State the use case: you have a `_config` dict on a class and you want to expose it as `Config.options` without giving callers write access. Copying on every read is wasteful; the proxy is O(1). 2. The proxy is a view, not a snapshot. If the underlying dict changes, the proxy reflects the change. That's usually what you want — the class controls writes via its own methods, and external readers see live state without being able to corrupt it. 3. Mention that `type.__dict__` returns a `MappingProxyType` for the same reason: classes need to expose their attribute dict for introspection while preventing external mutation that would corrupt the type machinery. 4. The proxy is shallow — values can still be mutable. If `proxy["users"]` is a list, callers can `proxy["users"].append(...)`. Combine with deep-immutable values (`tuple`, `frozenset`) for true immutability.

Edge cases to mention: - `MappingProxyType` lives in the `types` module, not `typing` — easy to mistype during the interview. - The proxy supports `==` against another dict if the contents match; it's not strictly type-equal to dict, but value-equal. - For sequences (not mappings), there's no built-in `ReadOnlySequence` — you can use `tuple(lst)` for a snapshot or build a custom class.

What NOT to say: - Do not say "I'll just `dict.copy()` and return it" without qualification — that's a snapshot; mutations to the original after the copy are invisible, which may or may not be what you want. - Do not claim the proxy is "fully immutable" — values inside it can still be mutated if they're mutable types.

Common follow-up: "How does `MyClass.__dict__` use this?" — Classes expose their attribute dict as a `MappingProxyType` so introspection works (`MyClass.__dict__['method_name']`) but you can't corrupt the class by writing through it. Setting an attribute goes through the class object's `__setattr__`, which manages the underlying dict safely.


```

from types import MappingProxyType

class Config:
    def __init__(self):
        self._data = {"region": "asia-south1", "max_retries": 3}
        self.view = MappingProxyType(self._data)

    def update(self, key, value):
        self._data[key] = value

c = Config()
print(c.view["region"])      # asia-south1

try:
    c.view["region"] = "us-east1"
except TypeError as e:
    print(e)                  # 'mappingproxy' object does not support item assignment

c.update("region", "us-east1")
print(c.view["region"])      # us-east1 – view reflects underlying change

```

Q20 · Hashability vs Mutability — Where Custom Classes Go Wrong

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Microsoft, JP Morgan · Topic: Mutability

The question (interviewer's verbatim phrasing):

What happens if I override `__eq__` on a class but forget to override `__hash__` ? What's the right way to handle hashing on a mutable class?

The 30-second answer: If you override `__eq__` without `__hash__` , Python automatically sets `__hash__` to `None` and instances become unhashable. The reason is the contract: equal objects must have equal hashes; redefining equality invalidates the inherited identity-based hash. For a mutable class that needs to be hashable, you must either (a) hash a stable, immutable identifier (a uuid, a database id, a creation timestamp) that doesn't change as the rest of the state changes, or (b) accept that the class isn't hashable and accept the consequences for dict/set usage.

Step-by-step thought process: 1. Walk the auto-behaviour: Python's data model says overriding `__eq__` invalidates the default `__hash__` . The interpreter sets `__hash__ = None` for you. That's safe by default — the class can't be used as a key, so no silent corruption. 2. State the contract: `a == b` must imply `hash(a) == hash(b)` . The reverse isn't required (hash collisions are allowed). But equal-but-different-hash is a bug — the dict will fail to find the entry. 3. Walk the hashable-mutable pattern: hash a field that's stable for the object's lifetime. Common choices: a UUID set at construction, a primary key from a database, a content hash for value-like immutable parts. Never hash mutable state.

4. The corollary: if the class is genuinely mutable in ways that change its identity, it shouldn't be hashable. Use `__hash__ = None` explicitly to make that intent clear, and use an external `id` map if you need lookups.

Edge cases to mention: - Inheritance: if a parent class defines `__hash__` and the child overrides `__eq__`, the child's `__hash__` is set to `None` unless explicitly re-declared. The auto-disabling cascades down the inheritance. - `__hash__ = None` is the actual sentinel; `__hash__ = lambda self: 0` (always returning 0) is a legal but terrible hash that puts every instance in the same bucket — O(n) lookups. - Dataclasses follow the same rule: by default they get auto-generated `__eq__` and no `__hash__`. `frozen=True` re-enables `__hash__` because the value is now stable.

What NOT to say: - Do not implement `__hash__` returning `id(self)` for a class with custom `__eq__` — equal-by-value objects will have different hashes, and dicts will fail. - Do not implement `__hash__` over mutable fields — mutation after insertion corrupts the dict.

Common follow-up: "How do dataclasses handle this?" — A plain `@dataclass` gets `__eq__` synthesised (value equality on fields) and `__hash__` set to `None` — unhashable by default. `@dataclass(frozen=True)` flips that: the instance is immutable, so equality is stable, so `__hash__` is auto-generated over the field tuple. `@dataclass(eq=False)` keeps the inherited identity-based `__hash__` and identity-based equality — rarely useful.

```
class WithEqNoHash:
    def __init__(self, x):
        self.x = x
    def __eq__(self, other):
        return self.x == other.x
    # No __hash__ defined

try:
    hash(WithEqNoHash(1))
except TypeError as e:
    print(e)          # unhashable type: 'WithEqNoHash'

# Right pattern: stable id field
import uuid

class Hashable:
    def __init__(self, x):
        self.x = x
        self._id = uuid.uuid4()
    def __eq__(self, other):
        return isinstance(other, Hashable) and self._id == other._id
    def __hash__(self):
        return hash(self._id)

h = Hashable(1)
{h: "stored"}          # works — stable hash
h.x = 999              # state changes, but hash is stable
{h: "still works"}     # lookup still finds the entry
```

Section C — Reference Semantics (10 Qs)

Q21 · Pass-by-Object-Reference — What Python Actually Does

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, Swiggy, Cognizant, JP Morgan · Topic: References

The question (interviewer's verbatim phrasing):

Is Python pass-by-value or pass-by-reference?

The 30-second answer: Neither, strictly. Python is pass-by-object-reference (sometimes called "call by sharing"). When you call `f(x)`, the parameter inside `f` becomes a new local name bound to the same object that `x` points to. Rebinding the parameter inside `f` (`x = something_else`) does not affect the caller's `x` — it just rebinds the local name. Mutating the object that the parameter points to (`x.append(1)`) does affect the caller, because both names point to the same underlying object.

Step-by-step thought process: 1. State the model cleanly: parameter names are local references to the same objects the caller has. There's no copy of the value (so it's not pass-by-value) and there's no aliasing the variable slot (so it's not pass-by-reference in the C++ sense). 2. The "rebinding doesn't escape, mutation does" rule is the whole insight. Rebinding (`x = ...`) makes the local name point somewhere else; the caller's name is untouched. Mutation (`x.append(...)`, `x[0] = ...`, `x.attr = ...`) operates on the shared object; the caller sees it. 3. Walk a worked example: `def f(lst): lst.append(1)` does affect the caller; `def f(lst): lst = [1]` does not. 4. Frame it as a discipline rule: if a function takes a mutable argument and shouldn't mutate it, pass a copy or document the contract explicitly. If a function should mutate, return `None` (like `list.sort()` does) to signal the in-place semantics.

Edge cases to mention: - Augmented assignment is mutation for mutable types and rebinding for immutable types: `x += 1` for an int is rebinding (new int object); `lst += [1]` for a list is mutation (calls `__iadd__` in place). - Default arguments are evaluated once at `def` time and shared — that's the Q23 mutable default trap. - Closures capture by reference, not by value — that's the late-binding closure trap (Q28).

What NOT to say: - Do not say "Python is pass-by-reference" — that's the Java/C# answer. Reassigning the parameter would change the caller, which doesn't happen. - Do not say "Python is pass-by-value" — that implies a copy, which doesn't happen for mutable objects.

Common follow-up: "Inside a function, if I do `x = [1, 2]` where `x` was a parameter, does the caller see it?" — No. That's rebinding, not mutation. The caller's `x` is unchanged. If you want the caller to see a new list, you must return it.

```
def append_one(lst):
    lst.append(1)          # mutation – caller sees this

def rebind(lst):
    lst = [1, 2, 3]        # rebinding – caller does NOT see this

a = []
append_one(a)
print(a)                  # [1]

b = [99]
rebind(b)
print(b)                  # [99] – unchanged
```

Q22 · Rebinding vs Mutation — The Local vs Caller View

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Microsoft, Walmart Labs · Topic: References

The question (interviewer's verbatim phrasing):

Why does `lst.append(1)` inside a function affect the caller, but `lst += [1]` sometimes doesn't?

The 30-second answer: `lst.append(1)` is always mutation — it calls a method on the list object and modifies it in place. `lst += [1]` is augmented assignment, which calls `__iadd__` if defined (in-place mutation), otherwise falls back to `lst = lst + [1]` (rebinding to a new list). For lists, `__iadd__` is defined and does in-place extension, so the caller does see it. For tuples, `__iadd__` isn't defined, so `t += (1,)` rebinds the local `t` to a new tuple — caller's tuple is unchanged.

Step-by-step thought process: 1. Walk the two operations side by side. Method call (`.append`) is unambiguous mutation. Augmented assignment is a dual-mode operator that depends on whether the type supports in-place mutation. 2. The decision tree for `lst += [1]`: Python looks for `__iadd__` on the type. Found and returns `self` (or any object): rebind the local name to that object, but for `list` it's the same list object so the caller's view is identical. Not found: fall back to `lst = lst + [1]` which is rebinding to a new object. 3. The trap: for a tuple parameter, `t += (1,)` rebinds the local name to a new tuple. The caller's tuple is unaffected. But for a list parameter, `lst += [1]` mutates the caller's list. Same syntax, different semantics, because of the type's `__iadd__` definition. 4. The defensive pattern: if you need to "extend" a parameter without ambiguity, write `lst.extend([1])` (clearly mutation) or `lst = lst + [1]` (clearly rebinding). Avoid `+=` when the type's mutability is uncertain.

Edge cases to mention: - For numpy arrays and pandas DataFrames, `+=` is in-place mutation when shapes are compatible — surprisingly, more like list than tuple semantics. - For strings, `s += "x"` is rebinding because strings are immutable; the caller's string never changes from `+=` regardless. - The trap is most painful inside a class: `self.lst += [1]` mutates `self.lst` in place, but if `self.lst` was the class-level attribute, you've now silently created an instance attribute that shadows it (next question).

What NOT to say: - Do not claim `+=` "is always mutation" or "is always rebinding" — it depends on the type. - Do not use `+=` on a tuple parameter expecting the caller to see the change.

Common follow-up: "What's the difference between `x += y` and `x = x + y` for a list?" — For a list, `x += y` calls `list.__iadd__` which mutates `x` in place and returns `self`. `x = x + y` allocates a new list (calls `list.__add__`) and rebinds. The caller's `x` sees the mutation in case 1, doesn't see case 2.

```
def add_via_iadd(coll):
    coll += [1]      # list: in-place. tuple: rebind.

def add_via_method(coll):
    coll.append(1)   # always mutation (lists/etc only)

# List: both affect caller
a = [10]
add_via_iadd(a)
print(a)            # [10, 1]

# Tuple: iadd rebinds local; caller untouched
t = (10,)
def add_to_tuple(t):
    t += (1,)
    print("inside:", t)  # (10, 1)
add_to_tuple(t)
print("outside:", t)    # (10,) — unchanged
```

Q23 · The Mutable Default Argument Trap

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, Cred, Wipro · Topic: References

The question (interviewer's verbatim phrasing):

What does this code print, and why? `def f(x=[]): x.append(1); return x` — called three times.

The 30-second answer: Default arguments are evaluated **once**, at function-definition time, and stored on the function object (`f.__defaults__`). Every call that doesn't override the default uses the same mutable object. So `f()` called three times returns `[1]` , `[1, 1]` , `[1, 1, 1]` — the list grows across calls because every call mutates the shared default. The fix is

`def f(x=None): x = [] if x is None else x` — default to a sentinel, allocate inside the body.

Step-by-step thought process: 1. State the mechanism precisely: `def` is a statement that runs once, creating the function object. Default values are evaluated as part of that statement and stored.

Subsequent calls reuse the stored objects. 2. Walk the failure: `f()` no argument → uses default `[]` → appends 1 → returns `[1]` . Second call → no argument → uses the **same** `[]` , now `[1]` → appends → returns `[1, 1]` . The default never resets. 3. The fix: `def f(x=None): x = [] if x is None else x` . Use `None` as a sentinel meaning "no value provided" and allocate the mutable inside the function body where it runs fresh every call. 4. Why panels love this question: it's the single most-asked Python gotcha in Indian interview screens. Service MNCs hit it in round 1, product cos in screening calls. The candidate who explains the mechanism and the fix in one breath is the senior signal.

Edge cases to mention: - Same trap with `dict` (`def f(d={}):`) and `set` (`def f(s=set()):`). Any mutable default is wrong. - Immutable defaults are safe — `def f(x=0)` , `def f(s="")` , `def f(t=())` all work as expected because you can't mutate them. - Linters catch this: `ruff B006` , `pylint dangerous-default-value` . Enable the rule in CI; the false-positive rate is near zero.

What NOT to say: - Do not say "Python should evaluate defaults on each call" — that's a language-design opinion; the question is about what it actually does. - Do not propose `def f(x=copy.copy([]))` — same trap, the copy is also evaluated once.

Common follow-up: "When would you ever want a shared mutable default?" — Almost never, but the pattern is sometimes used for memoisation: `def fib(n, cache={0: 0, 1: 1}):` uses the shared dict as a persistent cache across calls. It's a clever idiom but discouraged in modern code because it conflates argument default with state.

```
def buggy(x=[]):
    x.append(1)
    return x

print(buggy())      # [1]
print(buggy())      # [1, 1] — shared default
print(buggy())      # [1, 1, 1]
print(buggy.__defaults__) # ([1, 1, 1],) — the shared list

def fixed(x=None):
    x = [] if x is None else x
    x.append(1)
    return x

print(fixed())      # [1]
print(fixed())      # [1] — fresh
print(fixed())      # [1]
```

Q24 · List Aliasing on Multiplication — `[a] * n`

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Cognizant, JP Morgan · Topic: References

The question (interviewer's verbatim phrasing):

What does `[[[]] * 3]` produce, and why does mutating one inner list affect the others?

The 30-second answer: `[[[]] * 3]` produces `[[[], [], []]]` — a list of three references to the **same** inner list, not three separate lists. The `*` operator on a list shallow-copies the references; for mutable elements that means aliasing. Calling `outer[0].append(1)` changes all three "rows" to `[1]` because they're the same object. The fix is a comprehension: `[[[] for _ in range(3)]]`, which evaluates `[]` three times and creates three distinct lists.

Step-by-step thought process: 1. State the aliasing: `[a] * n` repeats the references to `a` `n` times. If `a` is immutable, you can't tell; if `a` is mutable, you'll see all rows change in sync. 2. The 2D-array trap is the classic case: `grid = [[0] * cols] * rows` produces `rows` references to one row, so `grid[0][0] = 1` sets every row's first element. The fix is `grid = [[0] * cols for _ in range(rows)]`. 3. The same applies to `*=` on a list — `lst *= 3` extends in place but the new entries are references to the same objects that were there before. 4. Mention numpy: `np.zeros((rows, cols))` does the right thing — a real 2D array with no aliasing because numpy allocates a single contiguous buffer.

Edge cases to mention: - `[0] * 3` produces `[0, 0, 0]` — three references to the same int 0. Aliasing exists but is invisible because 0 is immutable; you can never demonstrate the shared identity

through mutation. - `tuple` multiplication has the same aliasing semantics: `((,),) * 3` is three references to the same empty tuple. Again invisible because immutable. - For string repetition (`"a" * 3`), there's no aliasing risk because the result is a new immutable string with the characters concatenated.

What NOT to say: - Do not claim `[[] ] * 3` is "three empty lists" — it's three references to one empty list. - Do not propose `list(range(3))` as the 2D-init fix — that creates ints, not lists.

Common follow-up: "Why does `[[0] * 3 for _ in range(3)]` work but `[[0] * 3] * 3` doesn't?" — The comprehension runs `[0] * 3` three times, producing three distinct lists. The multiplication runs it once and aliases. Anything inside a comprehension is fresh per iteration; anything inside the multiplication is reused.

```
# Aliasing trap
bad = [[]] * 3
print(bad)           # [[], [], []]
bad[0].append(1)
print(bad)           # [[1], [1], [1]] — all three changed

# The fix
good = [[] for _ in range(3)]
good[0].append(1)
print(good)          # [[1], [], []]

# 2D init
grid = [[0] * 3 for _ in range(3)]
grid[0][0] = 9
print(grid)          # [[9, 0, 0], [0, 0, 0], [0, 0, 0]]
```

Q25 · Slice as a Shallow Copy

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cognizant, Walmart Labs · Topic: References

The question (interviewer's verbatim phrasing):

What does `b = a[:]` do, and how is it different from `b = a` ?

The 30-second answer: `b = a` creates a new name pointing to the same list — `a is b` is `True`, and mutations through either name affect both. `b = a[:]` creates a new list with the same elements (a slice from start to end), so `a is b` is `False` and `b.append(1)` doesn't affect `a`. But the slice is shallow — if the elements are mutable objects, both lists hold references to the same inner objects, and mutating an element through one is visible through the other.

Step-by-step thought process: 1. State the two cases. Assignment shares the object; slicing creates a new list and copies the references. 2. Demonstrate shallow vs deep: `a = [[1, 2]]; b = a[:]` — `b` is a new list, but `b[0]` is the same `[1, 2]` as `a[0]`. So `b[0].append(3)` shows up in `a` too. The slice copied the outer list but not the inner lists. 3. The fix for deep independence: `import copy; b = copy.deepcopy(a)`. Or for lists of lists, a comprehension: `b = [inner[:] for inner in a]`. 4. Performance: `a[:]` is $O(n)$ — it allocates a new list and walks the source. For large lists, `list(a)` is equivalent and slightly more explicit. For dict/set, the analogues are `dict(a)` and `set(a)`.

Edge cases to mention: - `a[:]` returns a list even if `a` is a tuple — wait, no — for a tuple, `a[:]` returns the same tuple (CPython optimisation: slice of an immutable container that covers the whole range returns the original). So `t is t[:]` is `True` for tuples but `False` for lists. - For a string, `s[:]` returns the original string for the same reason. Strings are immutable; there's nothing to copy. - `numpy` array slicing returns a **view**, not a copy. `arr[:]` shares memory. Use `arr.copy()` for an independent array. This is the canonical pandas/numpy gotcha.

What NOT to say: - Do not claim `a[:]` is a deep copy — it's shallow. Inner mutable elements are still shared. - Do not say `b = list(a)` and `b = a[:]` "do the same thing always" — they do for lists, but the latter behaves differently for tuples and strings.

Common follow-up: "What's the right way to copy a list of dicts so mutations to one don't affect the other?" — `copy.deepcopy(lst)`. Shallow copies (slice, `list()`, `copy.copy()`) all share the inner dicts. Deep copy walks the whole tree and copies everything.

```
import copy

# Assignment: aliasing
a = [1, 2, 3]
b = a
b.append(4)
print(a)           # [1, 2, 3, 4] — same object

# Slice: shallow copy
c = a[:]
c.append(5)
print(a)           # [1, 2, 3, 4] — unchanged
print(c)           # [1, 2, 3, 4, 5]

# Shallow vs deep with nested
nested = [[1], [2]]
shallow = nested[:]
shallow[0].append(99)
print(nested)       # [[1, 99], [2]] — inner shared

deep = copy.deepcopy(nested)
deep[1].append(99)
print(nested)       # [[1, 99], [2]] — still unchanged
```

Q26 · Closures Capture Names, Not Values

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, Microsoft · Topic: References

The question (interviewer's verbatim phrasing):

What does this print? `funcs = [lambda: i for i in range(3)]; print([f() for f in funcs])`.

The 30-second answer: It prints `[2, 2, 2]`, not `[0, 1, 2]`. The lambdas in the comprehension capture the **name** `i`, not its value at definition time. By the time you call any of the lambdas, the comprehension has finished and `i` is bound to its final value, 2. This is called late binding. The fix is to bind `i` early via a default argument: `lambda i=i: i` — defaults are evaluated at lambda-definition time and stored, so each lambda gets its own snapshot.

Step-by-step thought process: 1. Explain late binding: a closure captures the enclosing scope by reference. The lambda body says "look up `i` in the enclosing scope when called." The enclosing scope is the comprehension; `i` lives there and is updated each iteration. 2. The lambda doesn't snapshot `i` at the moment of its creation — it just stores a reference to "wherever `i` lives." When you call any lambda after the loop, all of them look up the same `i` and see the final value. 3. The fix is to force early binding via a default argument: `lambda i=i: i` — Python evaluates the default `i=i` at definition time (per Q23), so each lambda gets its own captured `i`. Now `[f() for f in funcs]` returns `[0, 1, 2]`. 4. Same trap appears with `for` loops outside comprehensions, in any callback registration, and most painfully in event handlers. The cure is always: default-argument capture, or `functools.partial`, or a factory function.

Edge cases to mention: - The same trap appears with `def` not just `lambda`: `def make_f(): return [(lambda: i) for i in range(3)]` — all three return 2. - `functools.partial(lambda i: i, i)` is the verbose explicit fix — it binds `i` at construction. - In Python 3, comprehensions have their own scope, so `i` doesn't leak to the outer scope after the comprehension. But the lambdas defined inside still see the comp's `i` by reference.

What NOT to say: - Do not claim "the lambda freezes `i` when defined" — it doesn't; that's the bug. - Do not propose `lambda: copy.copy(i)` — copying happens at call time too; it doesn't help.

Common follow-up: "How does the default-argument trick fix it?" — `lambda i=i: i` evaluates the default `i` at definition time and stores it on the lambda. The body's `i` then refers to the parameter (which has the stored default), not the enclosing scope. Each lambda gets its own captured value.

```
# Late binding bug
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])    # [2, 2, 2]

# Default-arg fix
funcs = [lambda i=i: i for i in range(3)]
print([f() for f in funcs])    # [0, 1, 2]

# Factory function fix
def make_returner(value):
    return lambda: value

funcs = [make_returner(i) for i in range(3)]
print([f() for f in funcs])    # [0, 1, 2]
```

Q27 · Aliasing After Assignment — `a = b = []`

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay · Topic: References

The question (interviewer's verbatim phrasing):

What does `a = b = []` do? After `a.append(1)`, what does `b` look like?

The 30-second answer: `a = b = []` evaluates `[]` once and binds both `a` and `b` to that same list. So `a is b` is `True`, and `a.append(1)` mutates the shared list — `b` is now `[1]` too. This is chained assignment with aliasing, not "create two empty lists." For independent lists, write `a, b = [], []` (which evaluates two separate `[]`).

Step-by-step thought process: 1. State the parse: `a = b = []` is `temp = []; a = temp; b = temp`. One allocation, two names. 2. The intent vs reality gap: many people write this expecting two empty lists; they get one shared list. The bug surfaces the first time either name is mutated and the other "magically" sees it. 3. The fix is explicit: `a, b = [], []` — this is tuple unpacking with two distinct values. Each `[]` is evaluated separately. 4. The pattern generalises: any chained assignment shares the rightmost value. `x = y = z = SomeClass()` constructs one instance and binds three names to it. For three instances, write three lines or three tuple elements.

Edge cases to mention: - `a = b = 0` is fine because `0` is immutable; the aliasing exists but you can never observe it. - `a, b = [[]] * 2` is the same trap repeated — one list, aliased twice. - Class-level default for instance attributes hits this too: `class Foo: items = []` — every instance shares the class-level list unless `__init__` rebinds it.

What NOT to say: - Do not assume chained assignment creates independent objects — it doesn't. - Do not say `a = b = []` is "a Python idiom for two empty lists" — it's an idiom for two aliased names.

Common follow-up: "What's the difference between `a = b = []` and `a, b = [], []`?" — Chained assignment shares one object across both names. Tuple unpacking with explicit construction creates two independent objects. The first form aliases; the second doesn't.

```
# Aliasing trap
a = b = []
print(a is b)          # True
a.append(1)
print(b)               # [1] - aliased

# Independent
a, b = [], []
print(a is b)          # False
a.append(1)
print(b)               # [] - independent
```

Q28 · The Class-Level Mutable Default

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Cognizant · Topic: References

The question (interviewer's verbatim phrasing):

What's wrong with `class Foo: items = []` if every instance is supposed to have its own list?

The 30-second answer: `items = []` at class scope creates a single list shared by all instances via the class object. The first `foo.items.append(x)` mutates the shared class-level list; every other instance sees the change. The fix is to assign the mutable in `__init__`: `def __init__(self): self.items = []`. That creates a fresh list per instance, stored as an instance attribute that shadows the class-level one.

Step-by-step thought process: 1. State the attribute lookup model: `foo.items` looks first in the instance's `__dict__`, then walks the class hierarchy. A class-level `items = []` is found in step two, so every instance returns the same object. 2. Walk the bug: `foo1 = Foo(); foo1.items.append("a"); foo2 = Foo(); print(foo2.items)` prints `["a"]`. The two instances share state via the class object. 3. The fix: assign in `__init__`. Now `foo.items` finds it in step one (instance dict) and never falls through to the class-level. Each instance has its own list. 4. The subtle case: `foo.items = []` (assignment, not mutation) creates an instance attribute that shadows the class-level one. So mutations to the class-level list are no longer visible to that instance, but other instances still share the original. Mixing assignment and mutation here is the recipe for confusion.

Edge cases to mention: - Same trap with `dict`, `set`, and any mutable type at class scope. Anything that you'd want fresh per instance must be initialised in `__init__`. - Dataclasses handle this

correctly via `field(default_factory=list)`: the factory runs per instance, producing a fresh list. Writing `items: list = []` directly in a dataclass raises a `ValueError` at class definition because of exactly this trap. - For class-level constants that are immutable (`MAX_RETRIES = 3`), it's fine — no mutation is possible, no shared state issue.

What NOT to say: - Do not say "Python is buggy" — the language is consistent; class-level attributes are shared by design (they're how methods work). - Do not propose `class Foo: items = list()` as a fix — same trap, `list()` is evaluated once at class definition.

Common follow-up: "How do dataclasses prevent this?" — They refuse to accept a mutable default value at class scope. `@dataclass class Foo: items: list = []` raises `ValueError: mutable default <class 'list'> for field items is not allowed: use default_factory`. The required pattern is `items: list = field(default_factory=list)`.

```
class Bad:
    items = []          # shared across all instances

a = Bad(); b = Bad()
a.items.append("x")
print(b.items)         # ['x'] — shared!

class Good:
    def __init__(self):
        self.items = [] # fresh per instance

a = Good(); b = Good()
a.items.append("x")
print(b.items)         # [] — independent

from dataclasses import dataclass, field

@dataclass
class Cleanest:
    items: list = field(default_factory=list)

a = Cleanest(); b = Cleanest()
a.items.append("x")
print(b.items)         # []
```

Q29 · `global` and `nonlocal` — Rebinding Across Scopes

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JP Morgan, Microsoft · Topic: References

The question (interviewer's verbatim phrasing):

Inside a nested function, when do I need `nonlocal` versus `global` ?

The 30-second answer: `global x` tells Python "when I assign to `x` in this function, rebind the module-level `x`, not create a local." `nonlocal x` tells Python "when I assign to `x` in this nested function, rebind the enclosing function's `x`, not create a local." You don't need either to **read** a variable from an outer scope — Python finds it via name lookup. You need them only to **rebind**. Mutating an object the outer scope holds (`outer_list.append(1)`) doesn't need either declaration.

Step-by-step thought process: 1. State the rule: any assignment inside a function creates a local by default, unless declared otherwise. `global` and `nonlocal` change that default. 2. Reads always work without declaration — the LEGB rule (Local, Enclosing, Global, Built-in) finds the name. Writes default to Local; you need the declaration to redirect. 3. Mutation vs rebinding again: `outer_list.append(1)` is mutation through a name lookup, never an assignment to the name itself, so no `global` / `nonlocal` needed. `outer_list = []` is rebinding and needs the declaration. 4. The pragmatic advice: `global` is usually a code smell — it suggests you're using a module-level variable for something that should be a class attribute or a return value. `nonlocal` is more legitimate (closures, counters, state in decorators).

Edge cases to mention: - `global x` declared before any use, even reads, is good practice for clarity, though only required for writes. - `nonlocal` requires the name to already exist in an enclosing non-global scope. If there's no matching binding, you get `SyntaxError`. - You cannot use `nonlocal` to rebind a module-level variable — that's what `global` is for. They're disjoint.

What NOT to say: - Do not say "use `global` whenever you need to read a module variable" — reads don't need it. - Do not propose mutating a class attribute from inside a method via `self.x = ...` and expect it to update the class-level value — that creates an instance attribute that shadows it.

Common follow-up: "Why does `count += 1` inside a function need `global count` but `count.append(1)` doesn't?" — `count += 1` for an integer is `count = count + 1` — that's an assignment, which by default creates a local. `count.append(1)` for a list is a method call on the object found via name lookup — no assignment to the name itself, no declaration needed.

```

counter = 0
log = []

def bump():
    global counter
    counter += 1      # rebinding the module variable

def append_log(item):
    log.append(item)  # mutation — no global needed

bump(); bump()
print(counter)        # 2

append_log("hello")
print(log)            # ['hello']

def outer():
    x = 0
    def inner():
        nonlocal x
        x += 1
    inner()
    return x

print(outer())        # 1

```

Q30 · Why Returning `None` After In-Place Mutation Is a Convention

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, JP Morgan · Topic: References

The question (interviewer's verbatim phrasing):

Why does `lst.sort()` return `None` instead of the sorted list?

The 30-second answer: Python's design convention is: methods that mutate in place return `None` to signal "I changed the object you gave me; don't pretend I returned a new one." `lst.sort()`, `lst.append()`, `lst.reverse()`, `dict.update()`, `set.add()` all follow this. The contrast is functional methods that return a new object: `sorted(lst)` returns a new sorted list and leaves the original alone. The `None` return prevents the bug `chained = lst.sort().filter(...)` where someone expects `sort` to return the list.

Step-by-step thought process: 1. State the convention: in-place mutation → returns `None`. Functional version → returns new object. `lst.sort()` vs `sorted(lst)`; `lst.reverse()` vs `reversed(lst)`. 2. The bug it prevents: chained calls that assume a return value. Without the convention, you'd see `a = b.sort()` everywhere, and `a` would be the sorted list. With the

convention, `a is None`, and the bug is caught at the very next line. 3. The library API design:

`list.sort` mutates and returns `None`; `sorted` is a function that takes any iterable and returns a new sorted list. Both exist precisely so you can pick which semantics you want. 4. The corollary: when you write your own mutation methods, follow the convention. Return `None`. If you want functional semantics, write a separate function or method that constructs a new object.

Edge cases to mention: - `dict.update` and `set.update` return `None` for the same reason. The fluent dict-merge in 3.9+ is the `|=` operator, which mutates in place but is also an expression — it returns the dict, breaking the convention slightly. - numpy and pandas have a different convention: most methods take an `inplace=True` argument; without it they return a new object, with it they return `None`. Mixed and noisy; the modern advice is to avoid `inplace=True` because it interacts badly with copy-on-write. - `collections.OrderedDict.move_to_end` returns `None` — same convention.

What NOT to say: - Do not call the convention "Python's mistake" — it's a deliberate design choice and a stable contract. - Do not propose `lst.sort()` returning the list for "fluent chaining" — that would mask the mutation-vs-return distinction the convention exists to highlight.

Common follow-up: "How do you sort a list and use the result in the same expression?" — Use `sorted(lst)` — it returns a new sorted list and is chainable. Use `lst.sort()` when you want the original mutated and don't need the return value. Don't try to combine them with `lst.sort()` then use `lst` — that works but the chaining temptation will bite a teammate.

```
lst = [3, 1, 2]

# Mutation: returns None
result = lst.sort()
print(result)          # None
print(lst)             # [1, 2, 3]

# Functional: returns new list
lst2 = [3, 1, 2]
result = sorted(lst2)
print(result)          # [1, 2, 3]
print(lst2)            # [3, 1, 2] — original unchanged

# The bug the convention prevents
try:
    chained = [3, 1, 2].sort()
    print(chained[0])   # TypeError: 'NoneType' object is not subscriptable
except TypeError as e:
    print(e)
```


Section D — The `copy` Module (5 Qs)

Q31 · `copy.copy()` vs `copy.deepcopy()` — Shallow vs Deep

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, JP Morgan · Topic: Copy

The question (interviewer's verbatim phrasing):

What's the difference between `copy.copy` and `copy.deepcopy` ?

The 30-second answer: `copy.copy(obj)` creates a new top-level object with references to the same nested objects — a shallow copy. `copy.deepcopy(obj)` recursively walks the object graph and copies every level, producing a fully independent tree. Use shallow when the top-level container needs to be distinct but inner objects can be shared; use deep when any mutation through one tree shouldn't be visible through the other. Deep copy is expensive — proportional to the size of the entire graph — and can fail on unpicklable objects.

Step-by-step thought process: 1. Show the contrast on a worked example: `a = [[1, 2], [3, 4]]`; `b = copy.copy(a)` — `a is not b` (top-level distinct), but `a[0] is b[0]` (inner shared). Mutating `b[0].append(5)` affects `a[0]` too. 2. With deepcopy: `c = copy.deepcopy(a)` — `a is not c` and `a[0] is not c[0]`. Independent at every level. 3. Performance and gotchas: `deepcopy` uses `pickle`-like machinery to walk arbitrary object graphs. It handles cycles via a memo dict (an object seen once is reused on subsequent visits, breaking cycles). But it's slow — $O(\text{graph size})$ — and fails on certain unpicklable objects (open files, sockets, threading locks). 4. Custom types can hook into both: define `__copy__(self)` for shallow and `__deepcopy__(self, memo)` for deep. Most classes don't need to; the default machinery works for plain attribute-holding classes.

Edge cases to mention: - `copy.copy(obj)` for immutable types (`int` , `str` , `tuple`) returns the same object — there's nothing to copy. - Cycles are handled correctly by `deepcopy` via the `memo` dict; without it, a circular reference would recurse forever. You can pass a `memo` explicitly in `__deepcopy__` overrides to integrate with the cycle handling. - Numpy arrays have their own `.copy()` method which is faster than `copy.deepcopy` for numerical data. Same for pandas DataFrames.

What NOT to say: - Do not call `a[:]` a deep copy — it's shallow. - Do not use `copy.deepcopy` "to be safe" without thinking about cost — for a 1 GB pandas DataFrame, deepcopy can be tens of seconds. Use the right tool.

Common follow-up: "What does `copy.deepcopy` do with a cyclic reference?" — It tracks visited objects in a memo dict keyed by `id()`. The first visit copies and stores the new object; subsequent visits return the stored copy. That handles cycles correctly and preserves shared-substructure identity. Without the memo, deepcopy would infinite-recurse.

```
import copy

a = [[1, 2], [3, 4]]

# Shallow
b = copy.copy(a)
print(a is b)           # False
print(a[0] is b[0])     # True – inner shared
b[0].append(99)
print(a[0])             # [1, 2, 99] – leaked!

# Deep
c = copy.deepcopy(a)
print(c[0] is a[0])     # False – independent
c[0].append(88)
print(a[0])             # [1, 2, 99] – unchanged

# Cycle handling
lst = [1]
lst.append(lst)         # lst contains itself
clone = copy.deepcopy(lst)
print(clone[1] is clone) # True – memo preserved the cycle
```

Q32 · `__copy__` and `__deepcopy__` — Hooking into the Copy Protocol

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Microsoft, Goldman · Topic: Copy

The question (interviewer's verbatim phrasing):

How does the `copy` module decide how to clone an object? Can you customise it?

The 30-second answer: `copy.copy(obj)` first checks if the type defines `__copy__`; if so, it calls that. Otherwise it falls back to the pickle protocol (`__reduce_ex__`, `__reduce__`) or constructs a fresh instance via `__class__.__new__` and copies the dict. For deep copy, it checks `__deepcopy__(self, memo)` first, then falls back to the same chain. You'd override these when the default doesn't capture what "copy" means for your class — for example, a database connection where copying should re-open a new connection, not duplicate the file descriptor.

Step-by-step thought process: 1. State the lookup order: `__copy__` / `__deepcopy__` first, then the pickle protocol, then a default that copies `__dict__`. 2. The signature: `def __copy__(self): return SomeNewInstance(...)`. For deep copy: `def __deepcopy__(self, memo): return ...` where `memo` is the dict used to handle cycles — you should pass it through to any nested `deepcopy` calls and register `self` in it before recursing. 3. Worked example: a class wrapping a file handle should override `__copy__` to either open a new handle or refuse to be copied.

Without the override, the default would copy the handle reference, and both copies would race on the same descriptor. 4. The pattern for "I want a fresh instance with these attributes deep-copied": `def __deepcopy__(self, memo): new = self.__class__(); memo[id(self)] = new; for k, v in self.__dict__.items(): setattr(new, k, copy.deepcopy(v, memo)); return new`. Register `new` in `memo` **before** recursing so cycles back to `self` see the partial copy.

Edge cases to mention: - Forgetting to register in `memo` before recursing is the classic `__deepcopy__` bug for cyclic structures — infinite recursion. - For classes inheriting from `dict`, `list`, or `set`, the default `__copy__` produces a base-class instance, not the subclass. You usually need to override. - `__reduce_ex__` is the pickle protocol that copy falls back to when no copy hooks exist. Implementing it is more work but gives you both pickling and copying for free.

What NOT to say: - Do not call `copy.deepcopy(self)` inside your own `__deepcopy__` — that's infinite recursion. Use the building blocks. - Do not silently return `self` from `__copy__` for a mutable class to "avoid the copy" — that's a lie that defeats the contract.

Common follow-up: "What goes wrong if `__deepcopy__` doesn't register the new object in memo before recursing?" — Infinite recursion when the object graph has a cycle. The cycle traversal hits `self`, calls `deepcopy(self, memo)`, which calls your `__deepcopy__`, which starts over without remembering that you're already in the middle of copying this object.

```
import copy

class Node:
    def __init__(self, label):
        self.label = label
        self.children = []

    def __deepcopy__(self, memo):
        new = self.__class__(self.label)
        memo[id(self)] = new          # register BEFORE recursing
        new.children = [copy.deepcopy(c, memo) for c in self.children]
        return new

root = Node("root")
child = Node("child")
root.children.append(child)
child.children.append(root)         # cycle

clone = copy.deepcopy(root)
print(clone.label, clone.children[0].label)  # root child
print(clone.children[0].children[0] is clone) # True — cycle preserved
```

Q33 · Copy-on-Write in Pandas 2.x

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Cred (data roles), Walmart Labs · Topic: Copy

The question (interviewer's verbatim phrasing):

What is copy-on-write in pandas 2.x and why was it introduced?

The 30-second answer: Pandas 2.x introduces copy-on-write (CoW) semantics where every operation that returns a DataFrame returns a logical copy — the underlying data may be shared until either copy is modified, at which point the modified copy gets its own buffer. This eliminates the long-standing `SettingWithCopyWarning` ambiguity where it was unclear whether `df[cond]['col'] = x` modified the original or a temporary copy. CoW makes the answer unambiguous: it modifies neither (because the chained access returns a copy) — and pandas raises a clear error so you know to chain differently.

Step-by-step thought process: 1. Anchor in the old pain: `df[df.amount > 100]['status'] = 'large'` would silently modify either the original or a temporary, depending on whether the boolean filter returned a view or a copy. Pandas raised `SettingWithCopyWarning` which was famous for being ignored. 2. CoW (PEP-equivalent — see PDEP-7) makes every DataFrame logically independent. Operations that previously returned views now return copies semantically; the actual buffer is shared until mutation forces a separation. 3. The performance angle: shared buffers mean operations like `df2 = df.copy()` are O(1) until you write — only when you actually mutate does pandas allocate a new buffer. For workflows that copy-and-then-don't-modify (defensive copying for thread safety), this is a free speedup. 4. Enable CoW with `pd.options.mode.copy_on_write = True` (in 2.0–2.2; default in 3.0). The migration path involves checking that your code never relied on chained assignment to mutate the original — if it did, CoW silently breaks it.

Edge cases to mention: - CoW changes the behaviour of `df['col'] = ...` from "modify in place" to "modify the buffer, possibly triggering a copy" — usually transparent but observable for code that aliased columns. - Numpy arrays underneath have their own view-vs-copy semantics (`.view()` vs `.copy()`); pandas CoW operates above that layer. - Inplace methods like `df.fillna(inplace=True)` are deprecated under CoW — the recommended pattern is `df = df.fillna(...)` because the perf cost of the copy is now negligible.

What NOT to say: - Do not claim CoW "makes everything faster" — it can be slower for code that depended on view semantics for in-place updates. - Do not call CoW "a workaround for `SettingWithCopyWarning`" — it's a semantic redesign, not a warning patch.

Common follow-up: "How does CoW interact with `inplace=True` methods?" — `inplace=True` is being deprecated under CoW because its semantics conflict with the "every return is independent" model. The recommended pattern is reassignment: `df = df.fillna(0)`. Under CoW, this is no slower than `inplace=True` because the buffer isn't actually copied until you mutate it.

```
import pandas as pd

pd.options.mode.copy_on_write = True

df = pd.DataFrame({"amount": [100, 200, 300], "status": ["a", "b", "c"]})

# Under CoW: this raises ChainedAssignmentError
try:
    df[df.amount > 150]["status"] = "large"
except Exception as e:
    print(type(e).__name__)  # ChainedAssignmentError or similar

# Correct pattern: use .loc on a single chained selector
df.loc[df.amount > 150, "status"] = "large"
print(df)
#    amount  status
# 0     100      a
# 1     200   large
# 2     300   large
```

Q34 · When Deep Copy Is Too Expensive

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Microsoft, JP Morgan · Topic: Copy

The question (interviewer's verbatim phrasing):

When would you NOT use `copy.deepcopy`, and what would you use instead?

The 30-second answer: Avoid `deepcopy` when the data structure is large (DataFrames, numpy arrays, deep object graphs) — its overhead is proportional to the entire graph, and it adds memo-dict overhead per object. Use type-specific methods instead: `arr.copy()` for numpy, `df.copy()` for pandas, `dict()` or `dict.copy()` for dicts, `list(lst)` for lists when shallow is enough, `pickle.loads(pickle.dumps(obj))` for surprising-fast deep copy via the pickle protocol (often faster than `deepcopy` for picklable types because it avoids the memo overhead).

Step-by-step thought process: 1. State the cost model: `copy.deepcopy` walks every object reachable from the root, recursively calling itself, maintaining a memo dict for cycle handling. For a graph of N objects, it's O(N) time and O(N) memory for the memo. For a 1 GB DataFrame, that's tens of seconds. 2. Alternatives: type-specific copy methods are tuned for their layout. `np.array.copy()` is a single memcopy of the underlying buffer. `pd.DataFrame.copy()` (or CoW under 2.x) is similar. These run in O(bytes) with high cache locality. 3. The pickle trick: `pickle.loads(pickle.dumps(obj))` serialises and deserialises. For deeply nested picklable objects, it's often 2-5x faster than `copy.deepcopy` because the pickle path is more optimised in CPython. It fails on unpicklable types (open files, locks). 4. The "don't copy at all" option: many cases

that reach for deepcopy are really cases for immutability. A `dataclass(frozen=True)` or a `tuple`-based representation eliminates the need to defensive-copy because nothing can mutate the shared instance.

Edge cases to mention: - For deeply recursive structures (trees with thousands of levels), `deepcopy` can hit Python's recursion limit and fail. Workaround: increase the limit with `sys.setrecursionlimit` or rewrite the type's `__deepcopy__` iteratively. - For objects with C-extension state (open sockets, file handles), `deepcopy` and `pickle` both fail. The answer there is to reopen the resource explicitly on the new instance. - For very large dicts, `dict.copy()` is O(n) but with single-pass memcopy of the hash table — much faster than `deepcopy` of the same dict.

What NOT to say: - Do not blanket-recommend `pickle.loads(pickle.dumps(obj))` — it's faster for some types and broken for others; check that your types are picklable first. - Do not use `json.loads(json.dumps(obj))` as a "poor person's deepcopy" — it silently drops non-JSON types (datetimes, Decimal, sets, custom classes).

Common follow-up: "When is `pickle.loads(pickle.dumps(obj))` actually faster than `copy.deepcopy`?" — For graphs of standard picklable types (dicts of strings/ints/lists) with no cycles, pickle's serialisation path is more optimised and avoids the memo dict overhead per object. Benchmarks for moderate-sized structures show 2-5x speedups. For graphs with cycles, deepcopy handles them; pickle does too but with overhead.

```
import copy
import pickle
import numpy as np

big = list(range(100_000))

# Slow path
import time
t = time.perf_counter()
a = copy.deepcopy(big)
print(f"deepcopy: {(time.perf_counter() - t)*1000:.2f}ms")

# Type-specific (much faster for known types)
t = time.perf_counter()
b = list(big)
print(f"list(): {(time.perf_counter() - t)*1000:.2f}ms")

# pickle round-trip (often faster than deepcopy)
t = time.perf_counter()
c = pickle.loads(pickle.dumps(big))
print(f"pickle: {(time.perf_counter() - t)*1000:.2f}ms")

# numpy: type-specific is single memcopy
arr = np.arange(1_000_000)
t = time.perf_counter()
d = arr.copy()
print(f"arr.copy(): {(time.perf_counter() - t)*1000:.2f}ms")
```

Q35 · Why `copy.copy(immutable)` Returns the Same Object

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Cognizant · Topic: Copy

The question (interviewer's verbatim phrasing):

What does `copy.copy(42)` return? What about `copy.copy("hello")` and `copy.copy((1, 2, 3))` ?

The 30-second answer: For immutable atomic types (int, float, str, bytes, frozenset) and certain immutable containers (tuple of immutables), `copy.copy(obj)` returns the same object — `copy.copy(42) is 42` is `True`. There's no point allocating a duplicate when the original can never change. `copy.deepcopy(obj)` for the same types behaves the same way. The optimisation cascades through tuples: `copy.copy((1, 2, 3))` returns the same tuple because all elements are immutable.

Step-by-step thought process: 1. State the principle: immutability makes copying pointless. The copy contract is "I want an object I can mutate without affecting the original" — for immutables, there is no mutation to worry about. 2. The `copy` module's dispatch table knows the common immutable types and returns the original directly. For custom classes, it falls back to creating an instance. 3. The tuple case is interesting: `copy.copy((1, 2, 3))` checks elements; if all are immutable, returns the same tuple. If a tuple contains a list, the tuple is still returned as-is by `copy.copy` (shallow), but `copy.deepcopy` would walk in and copy the list. 4. Practical implication: you can copy a `tuple` of `frozenset`s freely without performance worry — the copy is O(1). Defensive copying of immutable data is free.

Edge cases to mention: - For a tuple containing a mutable element, `copy.copy(t)` returns the same tuple (shallow); `copy.deepcopy(t)` returns a new tuple with deep-copied elements. - `copy.copy("hello")` returns the same string. Useful to know when you're profiling copy-heavy code paths. - The dispatch table is in `copy._copy_dispatch` (private) — not for production reliance, but useful to know it exists.

What NOT to say: - Do not claim `copy.copy(42)` "creates a new int" — it doesn't, and you can verify with `is`. - Do not use `copy.copy` to "force a fresh instance" of an immutable — there's no such thing as a fresh instance.

Common follow-up: "What's the point of even calling `copy.copy` on something you don't know the type of?" — Generic code that copies opaque arguments uses `copy.copy` because it handles both mutable (where copying is needed) and immutable (where copying is a no-op) correctly. The shape of the call site doesn't need to special-case immutables.


```
import copy

print(copy.copy(42) is 42)           # True
print(copy.copy("hello") is "hello") # True (also interned)
print(copy.copy((1, 2, 3)) is (1, 2, 3)) # True (same tuple)

# Tuple of mutables: shallow returns same tuple
t = (1, [2, 3])
print(copy.copy(t) is t)             # True

# Deep copy walks in
print(copy.deepcopy(t) is t)         # False
print(copy.deepcopy(t)[1] is t[1])   # False
```

Section E — GC and Reference Counting (10 Qs)

Q36 · Reference Counting in CPython — The Basics

Tags: Difficulty: Medium · Asked at: TCS, Razorpay, Swiggy, JP Morgan, Microsoft · Topic: GC

The question (interviewer's verbatim phrasing):

How does CPython manage memory? Explain reference counting.

The 30-second answer: Every Python object in CPython has a refcount — a counter of how many references point to it. The refcount goes up when a new name binds to the object (`x = obj`), passes to a function, gets added to a container. It goes down when a reference is dropped (`del x` , a function returns, a container item is replaced). When the refcount hits zero, the object is destroyed immediately and its memory returned to the allocator. That's deterministic destruction — far simpler than mark-and-sweep — but it cannot reclaim cycles, which is why CPython also has a separate cyclic collector.

Step-by-step thought process: 1. State the core mechanism: every PyObject has a `ob_refcnt` field. Every name binding, container insertion, function argument, attribute lookup either increments or decrements it. Hitting zero triggers immediate destruction. 2. The strength of refcounting: destruction is deterministic. The moment the last reference is dropped, the object is gone. Files are closed promptly, locks are released, memory is freed. That's why `with open(...) as f:` works robustly in CPython. 3. The weakness: refcounting cannot detect cycles. If `a.ref = b` and `b.ref = a` and no external reference exists, both refcounts are 1 (each pointing at the other), and they're never freed by refcounting alone. 4. The fix: a separate cyclic garbage collector runs periodically to find unreachable cycles. Together, refcounting + cyclic GC handles all cases. Other Python implementations (PyPy, Jython) use mark-and-sweep instead of refcounting; semantically equivalent, very different performance profile.

Edge cases to mention: - `sys.getrefcount(obj)` returns one more than the actual refcount because the argument itself is one reference. So a freshly created object whose only reference is the function argument shows refcount 2. - Some objects (None, True, False, small ints) have very high refcounts because they're referenced everywhere. You can see this with `sys.getrefcount(None)` — typically in the thousands. - Refcount operations are not free — they're a write to the object's header on every reference change. For multithreaded code, this requires the GIL or atomic increments. PEP 703 (free-threaded Python) makes refcounts atomic per-object.

What NOT to say: - Do not say "Python uses garbage collection" without qualifying — CPython uses refcounting primarily and cyclic GC as a fallback. Saying "garbage collection" alone misses the refcount layer. - Do not claim refcounting "is faster than mark-and-sweep" — it's faster for short-lived objects, slower for write-heavy workloads where every reference change is a memory write.

Common follow-up: *"Why can't refcounting handle cycles?"* — Because every object in a cycle has refcount ≥ 1 (the other cycle members reference it), so the count never hits zero. The cyclic collector finds these by walking the object graph from known reachable roots and reclaiming what's not reached. Q39 covers it.

```
import sys

x = [1, 2, 3]
print(sys.getrefcount(x))      # 2 — x itself + the argument

y = x
print(sys.getrefcount(x))      # 3 — added one binding

del y
print(sys.getrefcount(x))      # 2

# The cycle that refcounting can't free
class Node:
    pass

a = Node(); b = Node()
a.ref = b
b.ref = a
del a, b    # external refs gone; cycle keeps refcount > 0
# Only the cyclic GC will free this
```

Q37 · The Cyclic Garbage Collector — Generational Mechanics

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, Microsoft, Walmart Labs · Topic: GC

The question (interviewer's verbatim phrasing):

Walk me through CPython's cyclic garbage collector. Why is it generational?

The 30-second answer: CPython's cyclic collector runs periodically to find unreachable reference cycles that refcounting alone can't free. It's generational — objects are partitioned into three generations (0, 1, 2). New objects start in gen 0; survivors of a gen-0 collection get promoted to gen 1; survivors of gen 1 to gen 2. Younger generations are collected more often. The generational design is based on the observation that most objects die young — collecting frequently among newly allocated objects catches the common case cheaply; older objects are collected less often because they're likely to stay alive.

Step-by-step thought process: 1. State the generations: 0 (newest), 1, 2 (oldest). Collection thresholds (`gc.get_threshold()` returns `(700, 10, 10)` by default in CPython 3.12) decide when each runs. 2. The algorithm: for the chosen generation, walk all tracked objects and look for ones unreachable from outside the cycle. Cycle detection works by simulating "if we removed external references, which objects would still have `refcount > 0` due to internal cycle references?" — those are the cycle members. 3. Why generational matters for performance: most allocations are temporary (loop variables, function locals, intermediate computations). Collecting gen 0 frequently catches these without scanning the whole heap. Older objects (long-lived caches, module globals) are scanned rarely. 4. You can manually trigger with `gc.collect()` — returns the number of objects collected. Useful in tests to verify no cycles leak; not usually needed in production because the automatic schedule works.

Edge cases to mention: - Only "container" objects (those that can hold references — list, dict, custom classes) are tracked by the cyclic GC. Atomic objects (int, float, str) cannot participate in cycles by construction, so they're skipped. - `gc.disable()` turns off the cyclic GC entirely. Useful for short-lived programs (scripts that complete in seconds) where the cycles will be cleaned up by process exit. Long-running services should leave it on. - The collector pauses execution while scanning (stop-the-world). Large heaps can see pauses of tens of milliseconds in gen 2 collections — visible in p99 latency.

What NOT to say: - Do not claim the cyclic GC "is mark-and-sweep" — it's a different algorithm that uses refcount-difference logic, not graph colouring. - Do not say "Python's GC frees memory back to the OS" — it returns memory to CPython's allocator, which may or may not return it to the OS depending on size and arena state.

Common follow-up: "When should you manually call `gc.collect()`?" — Almost never in production. Two legitimate cases: (1) after a known cycle-creating operation in a test, to verify no leak; (2) before a memory-sensitive operation (forking a worker, taking a snapshot), to reduce footprint. In normal code, trust the automatic schedule.

```
import gc

print(gc.get_threshold())      # e.g., (700, 10, 10)
print(gc.get_count())         # objects since last collection per gen

# Create a cycle
class Node:
    pass

def make_cycle():
    a = Node(); b = Node()
    a.ref = b
    b.ref = a
    # function returns; a and b's external refs gone
    # but cycle keeps them alive until GC runs

for _ in range(1000):
    make_cycle()

collected = gc.collect()
print(f"collected {collected} cycle members")
```

Q38 · `gc.collect()` — When to Call It Manually

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft, Walmart Labs · Topic: GC

The question (interviewer's verbatim phrasing):

In what situations would you call `gc.collect()` explicitly in production code?

The 30-second answer: Almost never in normal code — the automatic schedule handles cycles efficiently. The legitimate cases are: (1) just before a memory-sensitive operation like `os.fork()` to reduce the child's copy-on-write footprint; (2) after a known cycle-heavy phase (e.g., closing a request that built up a graph) when you want to ensure cleanup before measuring memory; (3) in tests, to verify no cycles leak by asserting on collection counts. Forcing GC frequently in a hot loop hurts performance because the collector scans the whole tracked-object set.

Step-by-step thought process: 1. State the default position: don't call `gc.collect()`. The automatic schedule is tuned for typical workloads. 2. The fork case: child processes inherit the parent's memory via copy-on-write. Reducing the parent's footprint before fork means less COW divergence; the child uses less RAM. `gc.collect()` before `os.fork()` is a common pattern in worker pools. 3. The phase case: a long-running web worker handles a request, builds a graph (parser AST, dependency graph), and finishes. The cycles in that graph may not be collected until enough gen-0 allocations trigger a sweep — meanwhile the memory is held. Manually collecting at the end of each request bounds

memory growth. 4. The testing case:

```
before = gc.get_count(); do_thing(); gc.collect(); assert no cycles created
```

is a valid leak-detection assertion.

Edge cases to mention: - `gc.collect()` returns the number of unreachable objects found. Zero means no cycles to clean up; non-zero means cycles existed. - You can pass a generation:

`gc.collect(0)` only scans gen 0 — faster but misses cycles that crossed generation boundaries. - For latency-sensitive servers, the automatic collection can cause periodic pauses. Some teams disable GC and call `gc.collect()` at quiet moments (between requests) instead.

What NOT to say: - Do not call `gc.collect()` in a tight loop "to free memory" — it adds overhead and rarely helps; the automatic schedule already triggers when needed. - Do not claim `gc.collect()` frees memory back to the OS — it returns memory to CPython's allocator pools, which may keep it for future allocations.

Common follow-up: "What's `gc.freeze()` and when would you use it?" — Added in 3.7, it moves all current tracked objects to a permanent generation that's not scanned again. The use case is pre-fork: freeze long-lived parent objects so the child's gen-0 and gen-1 collections are faster and the COW pages don't get dirtied by refcount writes during GC. Standard pattern in some web servers and ML workers.

```
import gc, os

# Pattern 1: pre-fork
gc.collect()
# os.fork() # child inherits compact heap

# Pattern 2: gc.freeze before fork
gc.freeze() # parent objects go to permanent generation
# os.fork() # child won't dirty COW pages by scanning them

# Pattern 3: per-request cleanup in a worker
def handle_request(req):
    # ... build graph, process ...
    pass

# After handling a heavy request:
collected = gc.collect()
if collected > 100:
    print(f"warning: {collected} cycle objects per request")
```

Q39 · Reference Cycles in Practice — Where They Sneak In

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Microsoft, JP Morgan · Topic: GC

The question (interviewer's verbatim phrasing):

Give me three real-world scenarios where reference cycles cause memory leaks in Python code.

The 30-second answer: (1) Parent-child object graphs where the child stores a back-reference to the parent — common in tree structures, GUI widget hierarchies, AST node trees; (2) callback registrations where an object registers itself as a callback on another object and never deregisters — the registry holds the object, the object holds the registry; (3) closures that capture `self` while being stored on `self` — `self.handler = lambda: self.process(...)` creates a cycle. All three are caught eventually by the cyclic GC but cost real memory between collections.

Step-by-step thought process: 1. The parent-back-ref case: a Tree node holds `self.children: list[Node]` and each child has `self.parent: Node`. The root, the children, and the back-references form cycles. When you `del tree`, refcounting can't free it; the cyclic GC eventually does. 2. The callback case: an object registers `bus.subscribe(self.on_event)` — the method holds `self`, the bus stores the method. If `self` is never removed from the bus, the cycle persists. Fix: deregister in `__del__` or use `weakref.WeakMethod`. 3. The closure case: `self.handler = lambda: self.value` — the lambda captures `self`, the lambda is stored on `self`. Cycle. Fix: use `weakref.ref(self)` inside the lambda, or store the closure outside `self`. 4. The diagnostic tools: `gc.get_referrers(obj)` shows who's pointing at `obj`. `objgraph.show_backrefs(obj)` (third-party) visualises the cycle. For leak hunting, run `gc.collect()` then check what's left.

Edge cases to mention: - Custom `__del__` methods used to prevent the cyclic GC from collecting their cycle — fixed in Python 3.4+ (PEP 442). Now `__del__` runs even on cyclic objects. - Frame objects in tracebacks can hold references to local variables. A traceback stored in an exception's `__traceback__` may keep large objects alive long after the exception was caught. Pattern: explicitly clear with `exc.__traceback__ = None` if you store the exception. - ThreadLocal storage and module-level caches are common cycle hidings — they're alive for the process lifetime, so any cycle through them never gets collected even by the cyclic GC.

What NOT to say: - Do not claim "Python has no memory leaks because of GC" — cycles cost memory between collections; module-level state can leak forever; C extensions can leak independently of Python's GC. - Do not propose "just call `gc.collect()` everywhere" — it's a workaround, not a fix. The leak is the design; collecting more often just papers over it.

Common follow-up: "How would you diagnose a slow memory leak in a long-running service?" — Use `tracemalloc` to track allocation sites. Take snapshots at intervals; compare to see which sites grew. Cross-reference with `gc.get_objects()` to see what's living. `objgraph` for visualising back-references. The combination usually points at the cycle within an hour.

```
import gc, weakref

# Cycle 1: parent / child back-reference
class TreeNode:
    def __init__(self):
        self.children = []
        self.parent = None
    def add_child(self, c):
        c.parent = self
        self.children.append(c)

root = TreeNode()
root.add_child(TreeNode())
del root      # cycle holds memory until GC

# Cycle 2: closure captures self
class Worker:
    def __init__(self):
        self.handler = lambda: self.process()
    def process(self):
        pass

w = Worker(); del w    # cycle

# Cycle-free with weakref
class WorkerSafe:
    def __init__(self):
        self_ref = weakref.ref(self)
        self.handler = lambda: self_ref().process()
    def process(self):
        pass
```

Q40 · **weakref** — Caching Without Pinning

Tags: Difficulty: Hard · Asked at: Razorpay, Microsoft, Walmart Labs, Goldman · Topic: GC

The question (interviewer's verbatim phrasing):

What is **weakref** and when would you use it?

The 30-second answer: **weakref** creates a reference to an object that does not increment the refcount, so the object can still be garbage-collected even while the weakref exists. When the object dies, the weakref auto-clears (calling it returns **None**). The classic use case is caches that shouldn't prevent their keys from being collected — **weakref.WeakKeyDictionary** and **WeakValueDictionary**. Also used for observer patterns, parent back-references in trees (to break cycles), and any case where you want "remember this object if it's still alive."

Step-by-step thought process: 1. State the core: a regular reference (a name binding, a list slot, a dict entry) bumps refcount. A weakref doesn't. The object can still die; the weakref becomes a dangling pointer that auto-clears. 2. The standard collections: `WeakKeyDictionary` (keys are weak, so when a key is collected the entry vanishes), `WeakValueDictionary` (values are weak), `WeakSet`. Pick based on which side should be allowed to die. 3. The parent back-reference pattern: tree nodes hold strong refs to children, weak refs to parent. The tree can be cleanly freed when the root is dropped; no cycle. 4. The observer pattern: a publisher holds weakrefs to subscribers. When a subscriber dies, the publisher's set auto-cleans. No need for explicit deregistration. (`weakref.WeakMethod` for bound methods specifically, since those have their own lifetime quirks.)

Edge cases to mention: - Not all types support weakrefs. Built-in atomic types (int, str, tuple, dict, list) don't — they have no `__weakref__` slot. Custom classes do by default (you can opt out with `__slots__` that doesn't include `__weakref__`). - Calling a weakref returns the object or `None`. Always check: `obj = ref(); if obj is None: ...`. - `weakref.proxy(obj)` returns a transparent proxy that forwards attribute access to the underlying object, but raises `ReferenceError` if accessed after the object is collected. Use when you want the weakref to look like the original.

What NOT to say: - Do not store a weakref and use it like the original without the `ref()` call — `weakref.ref(obj).method()` raises because the weakref isn't the object. - Do not propose weakrefs for primitive types — they don't support it.

Common follow-up: "How is `WeakValueDictionary` different from a regular dict with manual cleanup?" — Automatic. The moment a value is collected (because no strong references remain), the entry is removed from the dict by the weakref callback machinery. No manual scan, no stale entries, no cleanup thread needed.

```
import weakref

class Resource:
    def __init__(self, name):
        self.name = name

r = Resource("connection")
ref = weakref.ref(r)
print(ref())          # <Resource ...> – still alive
del r
print(ref())          # None – collected

# WeakValueDictionary – auto-cleanup
cache = weakref.WeakValueDictionary()
r1 = Resource("r1")
cache["r1"] = r1
print(len(cache))     # 1
del r1
print(len(cache))     # 0 – entry auto-removed
```

Q41 · `tracemalloc` — Finding Where Memory Goes

Tags: Difficulty: Hard · Asked at: Razorpay, Microsoft, Walmart Labs, JP Morgan · Topic: GC

The question (interviewer's verbatim phrasing):

Your Django service grows from 200 MB to 2 GB over six hours and then crashes. How do you find the leak?

The 30-second answer: Reach for `tracemalloc` — it's the stdlib tool for tracking allocation sites. Start it at process start, take snapshots at intervals, then compare snapshots to see which source lines accumulated the most memory between them. The top growth line is almost always the leak site. For complex cases, combine with `gc.get_objects()` to inspect what's living, and `objgraph` for visualising the reference chains keeping things alive.

Step-by-step thought process: 1. Set up: `import tracemalloc; tracemalloc.start(25)` (the argument is how many stack frames to capture per allocation — 25 is plenty). 2. Take snapshots periodically: `snap1 = tracemalloc.take_snapshot()`, do work, `snap2 = tracemalloc.take_snapshot()`. The library tracks all live allocations with their stack traces. 3. Compare: `top = snap2.compare_to(snap1, 'lineno')`. This returns a list of source lines sorted by memory growth. The top entry is your prime suspect. 4. Cross-reference: for each suspect, look at what objects exist at that site (`gc.get_objects()` filtered by `id`), and use `gc.get_referrers(obj)` to see who's holding them alive. The reference chain back to a long-lived root (a module global, a thread-local, a cache) is the leak.

Edge cases to mention: - `tracemalloc` has runtime overhead — typically 10-30% in heavy allocators. Don't leave it on in production; turn it on under a feature flag for diagnosis. - The "growth" line in `compare_to` shows source line, not the variable name. You may need to look at the surrounding code to figure out which object is leaking. `top_stats[0].traceback.format()` shows the full call chain. - For C-extension leaks, `tracemalloc` only sees Python-side allocations. Native leaks (numpy, lxml, custom C ext) need different tools (`valgrind`, `mtrace`).

What NOT to say: - Do not start with `gc.collect()` as the "fix" — that's not a fix, it's hiding the leak. - Do not propose restarting the worker every N requests as the "fix" — same thing; the leak is still there; you're just bounding the damage.

Common follow-up: "What if `tracemalloc` shows the leak is in a third-party library?" — File an issue with a minimal reproducer extracted from the trace. In the meantime, find the user-side trigger (which API call accumulates memory) and either avoid it, work around it (recycle the offending object explicitly), or restart workers periodically until the upstream fix lands.


```
import tracemalloc

tracemalloc.start(25)

snap1 = tracemalloc.take_snapshot()

# ... do work, possibly leak ...
big = []
for _ in range(1000):
    big.append([i for i in range(1000)])

snap2 = tracemalloc.take_snapshot()

top = snap2.compare_to(snap1, 'lineno')
for stat in top[:5]:
    print(stat)
# .../app.py:N: size=8.0 MiB (+8.0 MiB), count=1000 (+1000)
```

Q42 · `gc.get_referrers` — Who's Holding This Object?

Tags: Difficulty: Hard · Asked at: Razorpay, Microsoft, Goldman · Topic: GC

The question (interviewer's verbatim phrasing):

You suspect an object is leaking. How do you find out what's keeping it alive?

The 30-second answer: `gc.get_referrers(obj)` returns a list of objects that currently hold a reference to `obj` — the dicts, lists, frames, classes that are keeping it from being collected. Walk that list, then call `get_referrers` on the suspects, and you trace the chain back to a long-lived root (a module global, a thread-local, a class attribute). The leak fix is usually at the root — clear that reference, and the whole chain becomes collectable.

Step-by-step thought process: 1. State the API: `gc.get_referrers(obj)` returns objects with direct references to `obj`. The list itself is not a referrer — `gc` excludes the list it returns. 2. Walk strategy: the immediate referrers are usually local frame dicts or container objects holding `obj`. To find the real root, recursively call `get_referrers` on the referrers, but with care — the call tree explodes quickly. 3. The complementary API: `gc.get_referents(obj)` is the reverse direction — what does `obj` point at? Less useful for leak hunting; more useful for graph traversal. 4. The `objgraph` library is the practical workhorse: `objgraph.show_backrefs([obj], max_depth=10, filename='/tmp/leak.png')` produces a graphviz of the back-reference chain. Visualising is faster than chasing through `get_referrers` manually.

Edge cases to mention: - `gc.get_referrers` skips frames that are currently executing (the function calling `get_referrers` itself) — otherwise it would always include itself. - For objects not tracked by the cyclic GC (atomic types — int, str), `get_referrers` returns an empty list. Use `gc.get_referrers` for container/custom-class instances. - The list returned can be large and stale by the time you walk it — references may have changed. Diagnose under a stable state (e.g., just after a request handler finishes).

What NOT to say: - Do not propose `del obj` as the fix for a leak — if `obj` is leaking, deleting your local reference doesn't help; something else is holding it. - Do not call `get_referrers` on `None` or other singletons — you'll get a giant list of nothing useful.

Common follow-up: "What's the difference between `get_referrers` and `get_referents`?" — Referrers = who points at `obj` (incoming edges). Referents = what `obj` points at (outgoing edges). For leak hunting, you walk referrers back to find the leak source. For dependency analysis, you walk referents forward.

```
import gc

class Leaked:
    pass

obj = Leaked()
container = {"key": obj}
also_holds = [obj]

# Who's holding obj?
referrers = gc.get_referrers(obj)
print(len(referrers))      # at least 2 (the dict and the list)
for r in referrers:
    print(type(r), id(r))

# To break the chain
del container, also_holds
referrers = gc.get_referrers(obj)
print(len(referrers))      # 1 (just our local 'obj' name)
```

Q43 · `__del__` and Why It's Discouraged

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Microsoft · Topic: GC

The question (interviewer's verbatim phrasing):

What is `__del__` and why do most style guides recommend against using it?

The 30-second answer: `__del__` is the destructor — called when the object's refcount hits zero or when the cyclic GC collects it. Style guides discourage it because: (1) execution order is not guaranteed in cycles; (2) exceptions raised in `__del__` are silently logged to stderr, not propagated; (3) on interpreter shutdown, module globals may already be cleared, making attribute access in `__del__` fail in confusing ways; (4) most cleanup is better expressed via `with` (context managers) which gives deterministic, exception-safe execution.

Step-by-step thought process: 1. State the mechanism: `__del__(self)` is invoked when the object is about to be freed. CPython calls it during refcount-zero destruction; the cyclic GC also calls it for cycle members. 2. Walk the four problems. Order: in a cycle, no `__del__` runs before another — the GC picks an arbitrary order, which can break "child cleans up before parent" invariants. Exceptions: silently logged. Shutdown: module globals (including imports `__del__` depends on) may be gone. Context manager alternative: `with` runs `__exit__` deterministically. 3. Pre-3.4, the cyclic GC refused to collect objects with `__del__` because of order ambiguity, causing slow leaks. PEP 442 (3.4+) changed this: `__del__` runs even on cycle members, but the order problem remains. 4. The recommended pattern: use `with` and `__enter__` / `__exit__` for cleanup. Use `weakref.finalize(obj, cleanup_func)` if you need a cleanup callback without `__del__` — it's safer because it doesn't depend on the object's lifecycle hooks.

Edge cases to mention: - `__del__` is not called on `del obj` if other references exist — `del` decrements one refcount; destructor runs only when count hits zero. - For C-extension objects (open files, sockets), the underlying resource has its own cleanup separate from `__del__`. Closing the resource in `__del__` is a safety net; closing it explicitly in `with` is the contract. - `weakref.finalize(obj, callback, *args)` registers a callback that runs when `obj` is collected. It's the modern replacement for `__del__` in most cases — safer because it doesn't keep `obj` alive and runs predictably.

What NOT to say: - Do not propose `__del__` for "guaranteed cleanup" — there are no guarantees about when or whether it runs. - Do not let exceptions propagate from `__del__` — they're silently logged; you'll never see them.

Common follow-up: "What's `weakref.finalize` and how is it better?" — Registers a callback function (plus its arguments) that runs when the target object is collected. The finalizer is held externally, so it doesn't create a cycle through `__del__`. You can also call `finalizer.detach()` to cancel it, or `finalizer()` to run it explicitly — useful for "close this resource now, don't wait for GC."

```
import weakref

class Resource:
    def __init__(self, name):
        self.name = name
        self._fin = weakref.finalize(self, self._cleanup, name)

    @staticmethod
    def _cleanup(name):
        print(f"cleaning up {name}")

    def close(self):
        self._fin() # run finalizer now, detach

r = Resource("db-conn")
del r # cleanup runs

# Compare to __del__ (discouraged):
class BadResource:
    def __init__(self, name):
        self.name = name
    def __del__(self):
        print(f"__del__ on {self.name}") # may not run reliably
```

Q44 · The GIL and Refcount Operations

Tags: Difficulty: Hard · Asked at: Razorpay, Microsoft, Goldman, Walmart Labs · Topic: GC

The question (interviewer's verbatim phrasing):

How does the GIL interact with reference counting? What changes in free-threaded Python?

The 30-second answer: With the GIL, only one thread executes Python bytecode at a time, so refcount increments and decrements don't need atomic operations — the GIL serialises them. That's a major reason the GIL is so cheap to keep around for single-threaded code; removing it would require atomic refcount operations on every reference change, which would slow down all Python code. PEP 703 (free-threaded Python, experimental in 3.13) makes refcounts atomic per-object using a biased reference counting scheme that's fast in the single-threaded case and falls back to atomic ops only on cross-thread sharing.

Step-by-step thought process: 1. Anchor the GIL model: every refcount change is a non-atomic memory write protected by the global lock. Single-threaded perf is great; multi-threaded CPU-bound code is bottlenecked by serialised execution. 2. Why free-threading is hard: every refcount operation in pure-atomic form is a **LOCK XADD** on x86, roughly 10x slower than a plain increment. Across billions of

refcount changes per second, that adds up. 3. PEP 703's bias: each object has an "owning thread." The owner does cheap non-atomic refcount changes. Non-owners use atomic operations and a slower path. For objects that stay on one thread (most objects), the overhead is small. 4. Practical implication: free-threaded Python won't be a silver bullet — code that does shared-memory threading still needs careful synchronisation around the data, and certain workloads see throughput improvements while others (heavy refcount churn across threads) see regressions.

Edge cases to mention: - The GIL is released around long-running C calls (numpy, I/O), so threads can parallelise on those. That's why threading helps numerical or I/O-bound code today, despite the GIL. - C extensions that don't follow the GIL protocol can corrupt refcounts. The free-threaded build will require C ext recompilation and audit; expect ecosystem churn. - `_Py_DECREF` is the C-level macro for decrementing refcounts. Looking at the CPython source shows how minimal the operation is under the GIL — that's the cost everyone wants to keep.

What NOT to say: - Do not claim "the GIL makes Python slow" — it makes multi-threaded CPU code slow; single-threaded code is fine. - Do not say "free-threaded Python will make all code faster" — it removes a bottleneck for some workloads and adds overhead for others.

Common follow-up: *"When does threading actually help in current CPython?"* — When the work is I/O-bound (network calls, file reads) or executes inside a C extension that releases the GIL (numpy linear algebra, image processing). Anything that spends most time in pure Python bytecode hits the GIL and won't parallelise on threads — use multiprocessing instead, until 3.13's free-threading matures.

```
import threading
import time

counter = 0

def increment(n):
    global counter
    for _ in range(n):
        counter += 1

# Under the GIL, this isn't really racy in the C sense
# (every += is one bytecode and the GIL serialises them at that granularity?
# Actually no — += is LOAD/ADD/STORE, race window exists)
t1 = threading.Thread(target=increment, args=(100_000,))
t2 = threading.Thread(target=increment, args=(100_000,))
t1.start(); t2.start()
t1.join(); t2.join()

print(counter) # Often < 200_000 — race between LOAD and STORE
```

Q45 · How Many Objects Are Tracked by GC vs Not?

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft, Walmart Labs · Topic: GC

The question (interviewer's verbatim phrasing):

Are all Python objects tracked by the cyclic garbage collector?

The 30-second answer: No. Only "container" objects — those that can hold references to other objects (list, dict, set, tuple, custom classes, modules) — are tracked. Atomic objects (int, float, str, bytes, complex) cannot participate in cycles by construction, so they're not tracked. This saves significant overhead: most allocations in real code are atomic, and the GC only scans the tracked subset.

`gc.is_tracked(obj)` tells you whether a specific object is in the tracked set.

Step-by-step thought process: 1. State the principle: cycles require references; only objects that can hold references can participate in cycles. Atomic types hold values, not references, so they're excluded. 2. The tracking decision happens at allocation. The GC maintains an internal list of tracked objects; when a new container is created, it's appended; when destroyed (via refcount or cycle collection), it's removed. 3. Tuples and dicts have a clever optimisation: a tuple whose elements are all untracked atomic types is itself untracked. The check happens at tuple creation. Same for "small" dicts under some conditions in modern CPython. 4. Practical implication: code with mostly atomic data (numerical processing) has very few tracked objects, so the cyclic GC is essentially free. Code with object graphs (web frameworks, ORMs, AST walkers) has many tracked objects and may see GC pauses.

Edge cases to mention: - A `tuple` of (int, int) is not tracked. A `tuple` containing a list is tracked. The check is recursive on creation. - `gc.get_objects()` returns only tracked objects — you won't see ints, strings, or basic types. Useful for "what's in my heap" inventory. - The `untrack` happens on demotion too: a tuple that becomes "no longer containing trackable" via mutation? Wait — tuples are immutable, this can't happen. But for mutable containers (list, dict), the GC re-checks periodically and untracks if all contents become atomic.

What NOT to say: - Do not claim "every Python object is GC-managed" — atomic types are managed only by refcounting; cyclic GC ignores them. - Do not assume `gc.get_objects()` gives a full heap inventory — it gives tracked objects only.

Common follow-up: "Why are `int` and `str` not tracked?" — They cannot hold references. An `int`'s value is itself; it cannot point at another object. Same for `str` (a sequence of code points, not a sequence of object references). Without the ability to participate in cycles, there's no reason for GC to scan them.

```
import gc

print(gc.is_tracked(42))          # False – atomic
print(gc.is_tracked("hello"))    # False – atomic
print(gc.is_tracked([1, 2, 3]))  # True – container
print(gc.is_tracked((1, 2, 3)))  # False – tuple of atomics
print(gc.is_tracked((1, [2])))  # True – tuple holds list

class A: pass
print(gc.is_tracked(A()))        # True – custom class is container

# Inventory of tracked objects
all_tracked = gc.get_objects()
print(f"tracked objects: {len(all_tracked)}")
```

Section F — `__slots__` and Memory Savings (5 Qs)

Q46 · What `__slots__` Does and Why It Saves Memory

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Microsoft, JP Morgan, Walmart Labs ·

Topic: `slots`

The question (interviewer's verbatim phrasing):

What is `__slots__` and how much memory does it actually save?

The 30-second answer: `__slots__` is a class-level declaration that lists the allowed attribute names. When set, instances don't get the usual `__dict__` — instead, they use fixed slots in the instance's C-level struct. The savings come from skipping the per-instance `__dict__` (typically 100-200 bytes) and storing attributes in a compact array of pointers. For small instances (3-5 attributes), savings are 30-50% per instance — meaningful when you have millions. Bonus side effect: attribute access is slightly faster, and accidentally adding new attribute names raises `AttributeError`.

Step-by-step thought process: 1. State the mechanism: `class Foo: __slots__ = ('x', 'y')` tells Python to allocate exactly two slots for instances. No `__dict__`, no `__weakref__` unless explicitly requested. 2. The memory breakdown: a regular instance has a header (~16 bytes) plus a `__dict__` (a hash table of at least 100-200 bytes even when empty). A slotted instance has the header plus a packed pointer array — for 3 slots, about 40 bytes total. That's a 4-5x reduction for small classes. 3. The bonus: slot access uses a descriptor that goes through a fixed C-level offset, avoiding the dict hash lookup. Microbenchmark wins of 20-30% on attribute access. 4. The constraint: instances can have only the declared attributes. `foo.new_attr = 5` raises `AttributeError` — usually a feature (catches typos), occasionally a problem (mixin patterns that add attributes).

Edge cases to mention: - For very small classes with two ints, slots can save 50% of memory — measurable on 10M instances. - For classes with many attributes (>20), the savings shrink because the dict overhead matters less per attribute. - `dataclass(slots=True)` (Python 3.10+) synthesises slots from the field declarations. Modern idiom: `@dataclass(slots=True, frozen=True)` gives you compact, immutable, hashable value objects in one line.

What NOT to say: - Do not claim slots are "always faster" — for dynamic-attribute workloads (frameworks that add attributes at runtime), slots break the pattern entirely. - Do not propose slots for "every class" — they have real tradeoffs (no `__dict__`, restricted attributes, inheritance complications).

Common follow-up: *"How do I measure the memory savings concretely?"* — `sys.getsizeof(instance)` for one, multiplied by instance count. Or use `pyimpler.asizeof` for deep size. Common result on a typical small class: ~56 bytes with slots vs ~152 bytes without — about 2.5x reduction. For 1M instances, that's 100 MB saved.

```
import sys

class Regular:
    def __init__(self, x, y):
        self.x = x
        self.y = y

class Slotted:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x = x
        self.y = y

r = Regular(1, 2)
s = Slotted(1, 2)

print(sys.getsizeof(r), sys.getsizeof(r.__dict__))
print(sys.getsizeof(s))    # smaller; no __dict__

# Slot constraint
try:
    s.z = 99
except AttributeError as e:
    print(e)                # 'Slotted' object has no attribute 'z'
```

Q47 · `__slots__` and Inheritance

Tags: Difficulty: Hard · Asked at: Razorpay, Microsoft, Goldman, Walmart Labs · Topic: **slots**

The question (interviewer's verbatim phrasing):

If I define `__slots__` on a class and inherit from it, what happens? Do the savings still apply?

The 30-second answer: For the savings to apply, every class in the inheritance chain must define `__slots__`. If any base class lacks slots (including the default `object`'s normal subclass behaviour with `__dict__`), instances get a `__dict__` and the savings disappear. The child should declare its own `__slots__` listing only its new attributes (not the parent's). Multiple inheritance with slots is fragile — only one path through the MRO can contribute slots; others must have empty `__slots__ = ()`.

Step-by-step thought process: 1. State the inheritance rule: slots restriction applies only if the whole chain is slotted. One unslotted class anywhere in the bases means instances get `__dict__`. 2. The child's slots list its **own** new attributes. Don't re-declare the parent's slots — that's a different slot for the same name and causes confusing behaviour. 3. For multiple inheritance, only one base can have non-empty slots. If both have slots and they overlap or even differ, you get `TypeError: multiple bases have instance lay-out conflict`. The workaround is empty slots on all but one: `class Mix: __slots__ = ()`. 4. Practical advice: combine slots with single inheritance, or use mixins with empty slots. For complex hierarchies, slots add more friction than they're worth.

Edge cases to mention: - A subclass with `__slots__ = ('z',)` of a parent with `__slots__ = ('x', 'y')` has all three slots available, no `__dict__`. - A subclass that omits `__slots__` declaration entirely gets a `__dict__` automatically — the savings are gone. Always declare empty slots if you want to inherit the slots benefit without adding new ones. - A subclass with `__slots__ = ('x',)` of a parent with `__slots__ = ('x',)` creates a second slot for `x` — the child's hides the parent's, and the parent's is wasted memory. Don't redeclare.

What NOT to say: - Do not claim "inheriting from a slotted class is the same as not inheriting" — you must declare your own `__slots__` (even empty) to keep the savings. - Do not propose multiple inheritance with two slotted parents — it usually fails with a layout conflict.

Common follow-up: "What's the trick to make mixins work with slots?" — Empty `__slots__ = ()` on the mixin. That declares it as slot-compatible (no new attributes, no `__dict__`) so it can be combined with a slotted concrete class. Pattern: mixins are pure-behaviour (methods only), so empty slots are natural.

```

class Base:
    __slots__ = ('x',)
    def __init__(self, x):
        self.x = x

class Mixin:
    __slots__ = ()
    def describe(self):
        return f"x={self.x}"

class Child(Mixin, Base):
    __slots__ = ('y',)
    def __init__(self, x, y):
        super().__init__(x)
        self.y = y

c = Child(1, 2)
print(c.describe())          # x=1
print(hasattr(c, '__dict__')) # False - no dict, slots intact

try:
    c.z = 99
except AttributeError as e:
    print(e)

```

Q48 · `__weakref__` Slot — When You Need to Add It

Tags: Difficulty: Hard · Asked at: Razorpay, Microsoft, Walmart Labs · Topic: **slots**

The question (interviewer's verbatim phrasing):

I added `__slots__` to my class and now `weakref.ref(instance)` raises `TypeError`. Why?

The 30-second answer: A normal Python object has a hidden `__weakref__` slot that holds the weakrefs pointing to it. When you declare `__slots__`, you opt out of that default — only the slots you list are allocated. To re-enable weakrefs, include `__weakref__` in your slots tuple: `__slots__ = ('x', 'y', '__weakref__')`. Without it, `weakref.ref(instance)` raises `TypeError: cannot create weak reference to 'Foo' object`.

Step-by-step thought process: 1. State the cause: slots replace the default instance layout. The default layout included `__dict__` and `__weakref__`; slots cut both unless explicitly included. 2. The fix is one line: add `__weakref__` to the slots tuple. It takes one pointer of memory (8 bytes on 64-bit) — usually negligible, but worth knowing about when minimising every byte. 3. Same trick for `__dict__`: if you want slots-style fixed attributes for the common case plus a `__dict__` for occasional extras,

include `'__dict__'` in slots. You get the worst of both — verbose declaration, dict still allocated — but it's a valid pattern for migration. 4. Practical case: any class that might be used as a key in a `WeakKeyDictionary` or value in a `WeakValueDictionary` needs `__weakref__`. If you're not sure, include it; the cost is one pointer.

Edge cases to mention: - Inherited `__weakref__` from a non-slotted parent class works without explicit declaration in the child. The slot is inherited. - `__weakref__` cannot be listed in slots of a class whose bases already provide it — `TypeError: __weakref__ slot disallowed`. Just inherit silently. - Modern dataclasses with `slots=True` automatically include `__weakref__` if any base class needs it; otherwise you need `dataclass(slots=True, weakref_slot=True)` (Python 3.11+).

What NOT to say: - Do not propose dropping slots "because weakref doesn't work" — add `__weakref__` to slots instead. - Do not claim "weakrefs are incompatible with slots" — they're compatible if you declare the slot.

Common follow-up: "What's `dataclass(slots=True, weakref_slot=True)`?" — In 3.11+, dataclasses gained explicit control over the weakref slot. `weakref_slot=True` includes `__weakref__` in the generated `__slots__`. Necessary if instances will be used as weakref targets and the dataclass has no base class providing it.

```
import weakref

class WithoutWeakref:
    __slots__ = ('x',)
    def __init__(self, x):
        self.x = x

obj = WithoutWeakref(1)
try:
    ref = weakref.ref(obj)
except TypeError as e:
    print(e)  # cannot create weak reference

class WithWeakref:
    __slots__ = ('x', '__weakref__')
    def __init__(self, x):
        self.x = x

obj = WithWeakref(1)
ref = weakref.ref(obj)  # works now
print(ref())           # <WithWeakref ...>
```

Q49 · When NOT to Use `__slots__`

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, JP Morgan, Cognizant · Topic: **slots**

The question (interviewer's verbatim phrasing):

Are there cases where you'd specifically avoid `__slots__` ?

The 30-second answer: Yes. Avoid `__slots__` when: (1) the class is part of a deeply mixin-based hierarchy where the slot conflict rules cause more pain than memory saved; (2) the class is meant to be dynamically extended at runtime (frameworks like Django models that add attributes via metaclasses); (3) you only have a few hundred instances — the savings don't matter; (4) you need pickling with arbitrary state and the slot constraints fight the pickle protocol; (5) the class uses `__getattr__` / `__setattr__` overrides that depend on `__dict__` being present.

Step-by-step thought process: 1. Anchor on the savings calculus: slots help when you have many instances of a small class. For 10 instances of a 5-field class, saving 100 bytes per instance is 1 KB total — irrelevant. For 10M instances, it's 1 GB. 2. The framework conflict: Django models, SQLAlchemy declarative bases, Pydantic v1 — all build classes via metaclasses that expect to dynamically add attributes during class creation. Slots fight this; the framework either errors out or silently falls back to non-slotted. 3. The pickling case: by default, pickle uses `__dict__` to capture state. Slotted classes need `__getstate__` / `__setstate__` to serialise correctly. Pickle does handle slots automatically in modern Python, but custom pickle paths can break. 4. The dynamic attribute case: if your class is meant to support `obj.anything = value` (a generic record type, a config bag), slots are wrong by design — they constrain attribute names.

Edge cases to mention: - `__slots__ = ('__dict__',)` re-enables `__dict__` on a slotted class — opting into both forms of storage. Rarely useful; usually a sign that slots aren't the right tool. - Slots interfere with `cached_property` from `functools` — the cache stores its value in `__dict__`, which doesn't exist on slotted classes. Workaround: include `__dict__` in slots, or use `lru_cache` instead. - Modern dataclasses with `slots=True` produce a new class internally — the decorator returns a different class than the original, which can break code that compared class identity. Test before adopting on existing dataclasses.

What NOT to say: - Do not recommend slots "by default for all classes" — they're an optimisation with real costs. - Do not claim slots break pickling — they don't, but they may need extra plumbing for custom pickle paths.

Common follow-up: "Does `functools.cached_property` work with slots?" — Not by default — `cached_property` stores its computed value in the instance `__dict__`, which slotted classes don't have. Workaround: add `'__dict__'` to slots (re-enabling the dict), or use `functools.lru_cache` on a method, which uses a separate cache structure.

```

from functools import cached_property

class WithSlots:
    __slots__ = ('x',)
    def __init__(self, x):
        self.x = x

    @cached_property
    def doubled(self):
        return self.x * 2

w = WithSlots(5)
try:
    print(w.doubled)
except TypeError as e:
    print(e)    # 'WithSlots' object has no attribute '__dict__'

# Fix: include __dict__ in slots (defeats purpose somewhat)
class Hybrid:
    __slots__ = ('x', '__dict__')
    def __init__(self, x):
        self.x = x

    @cached_property
    def doubled(self):
        return self.x * 2

h = Hybrid(5)
print(h.doubled)    # 10

```

Q50 · The Modern Idiom — `@dataclass(slots=True, frozen=True)`

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Microsoft, JP Morgan · Topic: **slots**

The question (interviewer's verbatim phrasing):

What's the modern way to define a value class that's compact, immutable, and hashable?

The 30-second answer: `@dataclass(slots=True, frozen=True)` is the modern idiom. `slots=True` (added in 3.10) generates `__slots__` from the field declarations, giving you the memory compactness. `frozen=True` makes instances immutable and auto-generates `__hash__`. The combination produces a class that behaves like a C struct — small, fast attribute access, hashable for use in dicts/sets, no accidental mutation. For the common case of value objects (Money, Coord, Snapshot, Configuration), this is the right default in 2026.

Step-by-step thought process: 1. State the combination: `slots=True` for memory, `frozen=True` for immutability and hashability. Together: a compact, immutable, hashable struct. 2. The synthesised behaviour: `__init__` sets all fields once. `__eq__` compares value-wise. `__hash__` is generated based on the field tuple. `__setattr__` is overridden to raise `FrozenInstanceError` after init. `__slots__` is populated from the field names. 3. The performance angle: attribute access is slot-based (faster than dict lookup). Memory per instance is ~40-50% of a non-slotted dataclass for small classes. Hashing is fast because the field tuple is itself fast. 4. The migration story: existing `@dataclass` code can adopt `slots=True` by changing one argument. Test for: subclasses that depend on `__dict__`, framework integration that doesn't expect slots, and any code using `weakref` (add `weakref_slot=True` in 3.11+).

Edge cases to mention: - `slots=True` produces a **new** class object (replacing the original) because slots must be known at class creation. Code that compared class identity may break. - Combine with `kw_only=True` (3.10+) to require keyword arguments at construction — useful for value classes with many fields where positional construction is unclear. - The pattern `@dataclass(slots=True, frozen=True, kw_only=True)` is becoming the modern Python value-object idiom — equivalent to records in Java 16+ or structs in Rust.

What NOT to say: - Do not adopt `slots=True` on every dataclass without checking — frameworks and certain inheritance patterns conflict. - Do not claim this combination is "free" — there are tradeoffs (no dynamic attributes, slight subclass complexity).

Common follow-up: "What other dataclass options compose well with `slots=True`, `frozen=True`?" — `kw_only=True` (require keyword arguments), `eq=True` (the default, gives value equality), `weakref_slot=True` (3.11+, include `__weakref__` if you'll use weakrefs), `order=True` (synthesise comparison ops). For an enum-like immutable value with rich behaviour: `@dataclass(slots=True, frozen=True, kw_only=True)` plus methods.

```

from dataclasses import dataclass, field
import sys

@dataclass(slots=True, frozen=True, kw_only=True)
class Money:
    amount_paise: int
    currency: str = "INR"

m1 = Money(amount_paise=49900)
m2 = Money(amount_paise=49900)

# Immutable
try:
    m1.amount_paise = 0
except Exception as e:
    print(type(e).__name__)    # FrozenInstanceError

# Hashable
print(hash(m1) == hash(m2))    # True
print({m1: "first"}[m2])      # 'first' – equality works

# Compact
print(sys.getsizeof(m1))       # smaller than non-slotted equivalent

# Equality
print(m1 == m2)                # True

# Construct only with kw args
try:
    Money(49900, "USD")
except TypeError as e:
    print(e)                    # positional argument disallowed

```

Chapter close

Memory and mutability are the chapter where Python's design choices — refcounting, the GIL, the data model, the value/reference split — all interact in ways that only become visible when you've debugged a leak, optimised a hot loop, or had a stranger explain why your cache "doesn't work in production." The fifty questions above cover what panels actually probe: identity rules they can verify in one line, the mutable-default trap that catches every confident dev once, the difference between a slice and a deepcopy, the small handful of `weakref` patterns that prevent the cycles, and the slots idiom that takes a class from 152 bytes to 56 with one decorator argument.

If you only retain three things from this chapter: (1) `is` is for singletons, `==` is for value comparisons — never invert this; (2) mutation propagates through aliases, rebinding doesn't — the mutable-default trap and the late-binding closure trap are both this rule in disguise; (3) `@dataclass(slots=True,`

`frozen=True`) is the modern default for value objects, and it's three flags long because each one does exactly one thing.

Next chapter (Ch 6 — Exceptions) tests how you handle the things that go wrong. Read it after this one — the EAFP idiom (Easier to Ask Forgiveness than Permission) is built on top of Python's reference semantics.

Chapter 6 — Exceptions (40 Questions)

Core Python Library · Python Interview Prep — 2026 Edition · Learn & Excel

Forty questions on how Python signals, catches, translates, groups, suppresses and logs failure. Exceptions are the chapter most candidates "know" — and the chapter where the most experienced devs lose marks, because the modern surface area has grown faster than most people's habits. EAFP versus LBYL is older than your Python interpreter, but exception groups, `except*`, structured chaining with `from`, `contextlib.ExitStack`, and the `logger.exception` pattern are 2026 staples. Indian product-co interviews now reach for at least one chaining question and at least one `ExceptionGroup` question in any backend round above SDE-2, and data-platform interviews use the context-manager + logging questions as a quick read on whether you've shipped code or only written notebooks.

Each question follows the pack's standard shape: tags · the question · 30-second answer · step-by-step thought process · edge cases · what NOT to say · common follow-up · code where it sharpens the answer.

Sections in this chapter

- **Section A — Fundamentals (Q1–Q10):** EAFP/LBYL, the hierarchy, `try / except / else / finally`, ordering, bare-except.
- **Section B — Custom Exceptions (Q11–Q15):** when worth defining, naming, hierarchy, extra attributes, `__str__`.
- **Section C — Chaining (Q16–Q25):** `raise X from Y`, `__cause__` vs `__context__`, `from None`, implicit chaining, debugging long chains.
- **Section D — Exception Groups (Q26–Q30):** PEP 654, `ExceptionGroup`, `except*`, `asyncio.TaskGroup`, `.split()` / `.subgroup()`.
- **Section E — Context Managers + Exceptions (Q31–Q35):** `__enter__` / `__exit__`, suppressing from `__exit__`, `@contextmanager`, `suppress`, `ExitStack`.
- **Section F — Real-World Patterns (Q36–Q40):** `logger.exception`, re-raise vs swallow, retry-with-backoff, structured exceptions for APIs, `__del__` raising.

Section A — Fundamentals

Q1 · EAFP vs LBYL — When Each Is the Right Default

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Freshworks, Zoho, TCS · Topic: Exceptions Fundamentals

The question (interviewer's verbatim phrasing):

What does EAFP mean? Why is it the default Python style and when do you still reach for LBYL?

The 30-second answer: EAFP is "easier to ask forgiveness than permission" — attempt the operation inside a `try` and catch the failure. LBYL is "look before you leap" — check a precondition first with an `if`. EAFP is preferred in Python because it's atomic (no time-of-check / time-of-use races), faster on the happy path (one lookup, not two), and reads cleanly. LBYL still wins when the check is genuinely cheap and the operation is heavy, or when the operation has side effects you don't want to roll back.

Step-by-step thought process: 1. "EAFP says do the thing inside a `try`. LBYL says check first with an `if`. Python's standard library and idioms lean EAFP because the language already raises clean, specific exceptions for almost every failure mode you'd want to test for." 2. "The dict example is the canonical one. LBYL — `if 'k' in d: v = d['k']` — does two lookups. EAFP — `try: v = d['k'] except KeyError: ...` — does one and is atomic. With concurrent mutation by another thread, LBYL has a race window. EAFP doesn't." 3. "The file example shows the TOCTOU race more sharply. `if os.path.exists(p): open(p)` — another process can `rm` the file between those two lines. `try: open(p) except FileNotFoundError:` — one syscall, atomic at the kernel level." 4. "Performance: raising an exception is expensive (maybe 5–10× a normal function call), but a `try` block that doesn't raise is essentially free in CPython 3.11+ thanks to zero-cost exceptions. So EAFP on the happy path is cheap; EAFP when you expect to raise every iteration is not." 5. "LBYL still has a place. If the operation has unwanted side effects on the failure path (writes a partial file, sends a network call, takes a lock), the cost of just-trying-and-catching can be higher than the cheap precondition check. Also: if the precondition is a domain rule — `if order.is_paid:` — that's not exception territory at all, it's normal control flow."

Edge cases to mention: - `dict.get(k, default)` is a third path that sidesteps EAFP vs LBYL for dict access — use it whenever a default value is meaningful. - For `list.pop()` on a possibly-empty list, EAFP catches `IndexError`; LBYL checks `if lst:`. Either works because both are cheap and there's no concurrency concern in single-threaded code. - `try / except` over an iterator that yields a million items is fine — the cost is in `raise`, not in the suspended `try`.

What NOT to say: - "EAFP is always better." It's the default style, not a rule. Saying "always" signals you parrot blog posts. - "Exceptions in Python are slow so I write LBYL everywhere." Half-right and half-wrong. Raising is expensive; the entered `try` block is essentially free in modern CPython.

Common follow-up: "Give me a real bug LBYL caused that EAFP would have prevented." — File-existence check then open, with the file deleted between. Or a dict-key check followed by a read on a dict another thread is mutating.

```
# LBYL – two lookups, race-prone for shared dicts
if "user_id" in payload:
    user = payload["user_id"]
else:
    user = None

# EAFP – one lookup, atomic, Pythonic
try:
    user = payload["user_id"]
except KeyError:
    user = None

# Best for this specific case – the dict API has a built-in default
user = payload.get("user_id")
```

Q2 · The Exception Hierarchy — Walk Me Through The Tree

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro, Freshworks · Topic: Exceptions Fundamentals

The question (interviewer's verbatim phrasing):

Draw or describe Python's exception class hierarchy. Where does Exception sit, what's above it, what are the major branches below it?

The 30-second answer: `BaseException` is the root. Directly below it sit four siblings: `SystemExit`, `KeyboardInterrupt`, `GeneratorExit`, and `Exception`. Everything user code normally raises lives under `Exception`. Major branches under `Exception` include `ArithmeticError` (ZeroDivisionError, OverflowError), `LookupError` (KeyError, IndexError), `OSError` (FileNotFoundError, PermissionError, ConnectionError), `ValueError`, `TypeError`, `RuntimeError`, and 3.11's `ExceptionGroup`. Always raise and catch under `Exception`, not `BaseException`.

Step-by-step thought process: 1. "`BaseException` is at the top. It exists so a handler can be written for absolutely anything, but it's not what user code targets." 2. "Four children of `BaseException`. Three are system-level — `SystemExit` (raised by `sys.exit()`), `KeyboardInterrupt` (Ctrl-C), `GeneratorExit` (`close()` on a generator). The fourth is `Exception`, which is what everyone else inherits from." 3. "Under `Exception` the most-asked branches are `LookupError` (parent of `KeyError`, `IndexError`), `OSError` (parent of file and network errors —

`FileNotFoundError`, `PermissionError`, `ConnectionRefusedError`, `TimeoutError`, `ArithmeticError` (parent of `ZeroDivisionError`, `OverflowError`, `FloatingPointError`), plus the standalones `ValueError`, `TypeError`, `RuntimeError`, `StopIteration`." 4. "Catching at the right level matters. `except OSError` catches every file and network failure in one line. `except FileNotFoundError` only catches the specific missing-file case. Catch as narrowly as you can act on, as widely as you can handle." 5. "Python 3.11 added `BaseExceptionGroup` and `ExceptionGroup` — same split as `BaseException` versus `Exception`. `ExceptionGroup` can only hold `Exception` subclasses; `BaseExceptionGroup` is needed when a group contains a `KeyboardInterrupt` or `SystemExit`."

Edge cases to mention: - `StopIteration` is under `Exception`, not `BaseException`, and is what `next()` raises when an iterator is exhausted. Don't catch generic `Exception` inside generator code or you may silently swallow it. - `OSError` collapsed many previously-distinct exceptions (`IOError`, `EnvironmentError`, `WindowsError`) in 3.3 — they're aliases now. - `Warning` is also under `Exception` — `DeprecationWarning`, `UserWarning`, etc. Usually displayed, not raised, but technically catchable.

What NOT to say: - "Everything inherits from `Exception`." Not true — `KeyboardInterrupt` and `SystemExit` inherit from `BaseException` directly, which is exactly why they exist. - "I catch `BaseException` to be safe." That's the opposite of safe — you'll eat `Ctrl-C` and `sys.exit()`, both of which the user/developer wants to propagate.

Common follow-up: "What's the difference between `except Exception` and bare `except:`?" — Bare `except:` catches `BaseException`, so it also eats `KeyboardInterrupt` and `SystemExit`. `except Exception` catches everything you should catch and leaves system-level interruptions alone.

```

BaseException
├── SystemExit
├── KeyboardInterrupt
├── GeneratorExit
├── BaseExceptionGroup      # 3.11+
│   └── ExceptionGroup      # 3.11+
├── Exception
│   ├── StopIteration
│   ├── ArithmeticError
│   │   ├── ZeroDivisionError
│   │   └── OverflowError
│   ├── LookupError
│   │   ├── KeyError
│   │   └── IndexError
│   ├── OSError
│   │   ├── FileNotFoundError
│   │   ├── PermissionError
│   │   ├── ConnectionError
│   │   │   ├── ConnectionRefusedError
│   │   │   └── ConnectionResetError
│   │   └── TimeoutError
│   ├── ValueError
│   │   └── UnicodeError
│   ├── TypeError
│   ├── RuntimeError
│   │   └── RecursionError
│   └── ...

```

Q3 · BaseException vs Exception — Why Custom Exceptions Inherit From Exception

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Postman, Goldman, JP Morgan · Topic: Exceptions Fundamentals

The question (interviewer's verbatim phrasing):

Why should custom exceptions inherit from `Exception` and not `BaseException` ?

The 30-second answer: Because callers writing `except Exception:` expect to catch every application-level failure. If your exception inherits from `BaseException`, that catch-all misses it, which means user code that's already correctly designed will silently let your failures escape.

`BaseException` is the namespace for "things user code should not normally catch" — `KeyboardInterrupt`, `SystemExit`, `GeneratorExit`. Your domain error is not one of those.

Step-by-step thought process: 1. "`BaseException` has three subclasses that exist precisely because they shouldn't be caught by generic handlers: `KeyboardInterrupt` (Ctrl-C), `SystemExit` (`sys.exit()`), and `GeneratorExit` (generator cleanup). Putting your domain error in this category means it gets the same 'do not catch lightly' treatment." 2. "The convention everyone codes to: `try: ... except Exception: log_and_recover()`. That's the standard guard at request boundaries in Django, FastAPI, Flask, Celery. If your `PaymentError` inherits from `BaseException`, that guard misses it and the worker crashes instead of returning 500." 3. "A second reason: `pytest` and most test frameworks rely on `Exception` being the universal base of expected failure. `pytest.raises(Exception)` will not match `BaseException`-derived classes, so your test setup quietly stops working." 4. "There is one legitimate use of `BaseException` subclassing — building a framework that needs an interruption mechanism that should not be caught by user `except Exception` (the way `pytest`'s internal `_pytest.outcomes.Exit` does it). For application code, never." 5. "Same logic at the group level: inherit `ExceptionGroup`, not `BaseExceptionGroup`. Application code that catches `except* Exception:` won't match a `BaseExceptionGroup`."

Edge cases to mention: - Older code sometimes inherits from `StandardError` — that was Python 2 and was removed in Python 3. Anything inheriting from it won't even import on modern Python. - An exception inheriting from a specific built-in like `ValueError` is fine — `ValueError` is itself under `Exception`. That's a useful technique when your error genuinely is a value error. - A class can multiply-inherit from `Exception` and a mixin, but don't multi-inherit from two different exception classes — MRO gets confusing and `isinstance` checks become harder to reason about.

What NOT to say: - "BaseException is more secure." Backwards — it makes your exception harder to catch, which means accidentally-uncaught failures crash the worker. - "It doesn't matter, callers can catch BaseException." They can but they shouldn't, because the framework code around them assumes `Exception` is the catch-all.

Common follow-up: "What's a legitimate reason to ever subclass `BaseException`?" — Building an interruption signal in a framework or test runner that should propagate past application-level `except Exception` handlers (e.g., `pytest`'s internal `exit`). Not for ordinary application errors.

```
# Correct — inherit from Exception
class PaymentError(Exception):
    """Base for all payment-module failures."""

class PaymentDeclined(PaymentError):
    pass

# Wrong — application code that does `except Exception:` won't catch this
class PaymentDeclinedBroken(BaseException):
    pass

# At a framework boundary (e.g., Django middleware), this guard is standard:
try:
    response = view(request)
except Exception: # catches PaymentDeclined; misses PaymentDeclinedBroken
    response = error_500()
```

Q4 · try / except / else / finally — What Runs When

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant, Accenture · Topic: Exceptions Fundamentals

The question (interviewer's verbatim phrasing):

Walk me through the four clauses — `try`, `except`, `else`, `finally`. What runs when, and why does `else` exist?

The 30-second answer: `try` runs the protected code. `except` runs only if a matching exception was raised in `try`. `else` runs only if `try` finished without raising — it's the "happy path continuation." `finally` runs always — exception or not, return or not, even if `except` itself raises. `else` exists so the lines that depend on the try succeeding can sit outside the try block, which keeps `except` from accidentally catching exceptions from the success path.

Step-by-step thought process: 1. "`try` runs first. If nothing is raised, control flows to `else` (if present), then `finally`." 2. "If an exception is raised in `try`, Python walks each `except` clause in order. The first one whose class matches handles it. After the handler, control goes to `finally`. `else` is skipped." 3. "If an exception is raised and no `except` matches, `finally` still runs and the exception then propagates out of the try." 4. "`else` is for the success-only logic. Classic example — `open()` is the risky line, `json.load()` is the success-path line. Putting `json.load()` in `try` means `JSONDecodeError` gets caught by `except FileNotFoundError` (well, not literally that one, but you risk catching unrelated exceptions). Putting it in `else` keeps the catch tight." 5. "`finally` always runs. Even if `try` has a `return`, even if `except` has a `return`, even if the exception is propagating because nothing caught it. That's why `finally` is for resource cleanup —

close files, release locks, roll back transactions. Modern Python prefers a context manager (`with`) for this; `try/finally` is for cases where no context manager fits."

Edge cases to mention: - A `return` in `finally` overrides any return or exception from the `try/except` blocks. That's almost always a bug — don't put control flow in `finally`. - An exception raised inside `except` or `else` is also handled by `finally`. So `finally` can run while a second exception is propagating up. - If `try` raises an exception and `finally` raises a different one, the original is set as `__context__` on the new one. Both show in the traceback.

What NOT to say: - "`else` runs at the end of `try`." Wrong — `else` runs only if no exception was raised. Confusing it with `finally` is the classic mistake. - "`finally` doesn't run if you `return`." Wrong — `finally` runs even on `return`. That's its whole point.

Common follow-up: "Why put a line in `else` instead of at the bottom of `try`?" — Because lines at the bottom of `try` can themselves raise, and you don't want `except` to silently catch errors from the success-path logic.

```
def load_config(path: str) -> dict:
    try:
        f = open(path, encoding="utf-8")
    except FileNotFoundError:
        return {}          # missing config is fine, use defaults
    else:
        # runs only if open succeeded; JSON errors propagate without
        # being caught by the FileNotFoundError handler
        data = json.load(f)
        return data
    finally:
        # runs whether open succeeded or not — guard for the failed case
        if "f" in locals():
            f.close()
```

Q5 · Multiple except Clauses — Ordering Rules

Tags: Difficulty: Easy · Asked at: TCS, Cognizant, Wipro, Freshworks · Topic: Exceptions Fundamentals

The question (interviewer's verbatim phrasing):

If I have multiple `except` clauses, what order do they run in? What's wrong with catching `Exception` before `KeyError`?

The 30-second answer: Python walks the `except` clauses top to bottom and runs the first one whose class matches the raised exception via `isinstance`. So broader catches must come after narrower ones. `except Exception` before `except KeyError` is dead code — `Exception` matches

`KeyError` so the `KeyError` branch never runs. Some linters flag this as `unreachable except` (Pyflakes W0705, Ruff B025).

Step-by-step thought process: 1. "The matching rule is `isinstance`. If the raised exception's class is or subclasses the except's class, the except matches. The first match wins." 2. "That means ordering matters whenever your except classes form a hierarchy. `KeyError` is under `LookupError` is under `Exception`. So `except LookupError` after `except KeyError` only catches `IndexError` and other non-`KeyError` lookup errors — which may be what you want." 3. " `except Exception` should be last if it's there at all. Anything before it that's narrower will be reached for that specific class; anything after it will never be reached." 4. "You can also catch multiple types in one clause with a tuple: `except (KeyError, IndexError) as e:`. Use this when the handler is the same for both. Don't write the same handler twice." 5. "If you want one piece of code on the success path and a fallback on any failure, you don't actually need multiple except — one `except Exception:` is enough. Multiple excepts exist for different recovery logic per failure mode."

Edge cases to mention: - A tuple in an `except` clause is the only way to match multiple unrelated types in one branch. The variable `as e` binds to whichever was caught. - Subclassing relationships make order subtle — if you catch `OSError` first, `FileNotFoundError`, `PermissionError`, `ConnectionError` are all swallowed by it. Order narrow-to-wide if the handlers differ. - Ruff's lint rule `B025` flags duplicate exception types in a try block — a real bug class because Python silently picks the first.

What NOT to say: - "Order doesn't matter, Python figures it out." Wrong. First match wins, so narrow-then-wide is the only safe ordering when handlers differ. - "Use one giant `except Exception` and dispatch inside." Loses the dispatch-by-type efficiency the language gives you for free.

Common follow-up: "What if I have two excepts that catch the same exact class?" — Python uses the first one; the second is dead code. Tools like Pyflakes and Ruff flag this.

```
# Correct – narrow before wide
try:
    do_database_work()
except KeyError as e:
    log.warning("missing key %s", e)
except LookupError as e:
    log.warning("other lookup failure: %s", e)
except Exception as e:
    log.exception("unexpected database error")
    raise

# Wrong – first branch swallows everything; the others are dead code
try:
    do_database_work()
except Exception as e:    # this matches every Exception subclass
    log.exception("uh oh")
except KeyError:          # unreachable
    pass
```


Q6 · Catching Multiple Exception Types In One Clause

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Razorpay · Topic: Exceptions Fundamentals

The question (interviewer's verbatim phrasing):

How do you catch two different exception types in the same handler?

The 30-second answer: Pass a tuple to `except` : `except (KeyError, IndexError) as e:` . Any exception whose class matches one of the tuple's entries is caught and bound to `e` . The tuple form is the only correct way — `except KeyError, IndexError:` (no parens) was Python 2 syntax meaning something else and is a syntax error in Python 3.

Step-by-step thought process: 1. "Tuple syntax — `except (TypeA, TypeB) as e:` . The match is `isinstance(raised, (TypeA, TypeB))` . The `as` binding is whatever was raised, typed as the common ancestor or as the union of those types." 2. "Useful when the recovery logic is identical for both. Catching `ConnectionError` and `TimeoutError` together with the same retry — same response, no point writing two handlers." 3. "You can also chain with the parent class — `except (FileNotFoundError, PermissionError) as e:` is equivalent to `except OSError as e:` if those two are the only `OSError` subclasses you care about. The tuple is more explicit; the parent is shorter. Pick what reads better in context." 4. "From 3.11+, this also works with `except*` for groups — `except* (ValueError, TypeError) as eg:` . Same tuple semantics, applied across all leaves of the group." 5. "What you cannot do — mix types and module references. `except (KeyError, json.JSONDecodeError)` is fine because both are types. `except (KeyError, json):` is a `TypeError` at runtime because `json` is a module, not an exception class."

Edge cases to mention: - A single class doesn't need parens — `except KeyError as e:` is fine. A tuple does — Python 2 allowed `except KeyError, e:` for naming; Python 3 requires `as` . - `as e` binds `e` only inside the `except` block — after the block, `e` is deleted from the namespace. Different from a regular variable assignment. - If you want different handlers for the same exception based on attributes (e.g., HTTP status code), catch the type once and dispatch on `e.status_code` inside.

What NOT to say: - "I write the same handler twice for clarity." Increases maintenance and risks the two versions drifting. Use the tuple. - "`except KeyError, IndexError:` works in Python 3." Wrong — that's Python 2 syntax. Python 3 requires the tuple.

Common follow-up: "How do you know which exception was caught when two types share a handler?" — Use `isinstance(e, KeyError)` inside the block, or branch on `type(e)` . Usually the handler doesn't need to know, which is the whole reason for grouping them.

```
try:
    response = http.get(url, timeout=5)
    payload = response.json()
except (ConnectionError, TimeoutError) as e:
    # one handler — both mean "try again later"
    log.warning("transient http failure: %s", e)
    schedule_retry()
except (KeyError, json.JSONDecodeError) as e:
    # different recovery, different handler
    log.error("malformed response: %s", e)
    return None
```

Q7 · The Bare except — Why It's Almost Always Wrong

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Postman, Cred · Topic: Exceptions Fundamentals

The question (interviewer's verbatim phrasing):

What's wrong with writing `except:` with no class? When is it ever okay?

The 30-second answer: Bare `except:` catches everything under `BaseException`, including `KeyboardInterrupt` and `SystemExit` — so Ctrl-C stops working and `sys.exit()` becomes a no-op. It also makes debugging painful because you've discarded all class information. The fix is `except Exception:`, which catches every normal failure and leaves the three system-level interrupts alone. Bare except is acceptable in exactly one place: a top-level crash logger that genuinely needs to log absolutely anything before re-raising.

Step-by-step thought process: 1. "Bare `except:` is short for `except BaseException:` — and `BaseException` includes `KeyboardInterrupt`, `SystemExit`, `GeneratorExit`. Catching those breaks the basic contract that Ctrl-C interrupts and `sys.exit()` exits." 2. "It also hides bugs. A `NameError` from a typo gets swallowed by the same line meant to handle `ConnectionError`. You then debug for an hour wondering why the recovery branch keeps running. Tight excepts catch only what you can act on." 3. "Linters universally flag this. `Ruff E722`, `Pyflakes W0702`, `Flake8 E722` all complain. Most production codebases treat bare except as a CI failure." 4. "The 'okay' use case — a top-level crash boundary that needs to log every possible failure including `SystemExit` before re-raising. Even there, the better pattern is `except BaseException` (explicit) followed by `raise` — and even better is to let the framework's signal handlers do the catching." 5. "If you genuinely need 'every Exception', write `except Exception:`. It's six characters longer and tells the reader exactly what's caught and what isn't."

Edge cases to mention: - A bare except inside a generator catches `GeneratorExit`, which breaks `close()` semantics. The generator never finalizes cleanly and the calling code may leak resources. - The bare-except pattern in older libraries is a maintenance hazard — when you upgrade Python, new `BaseException` subclasses (none yet, but conceivable) start being swallowed silently. - `try: ... except: pass` is the worst form — silently eats everything including programmer errors. Reviewers should reject this on sight.

What NOT to say: - "Bare except is fine if I add `raise` at the end." Then write `except Exception:` and the same `raise`. Bare except adds nothing and risks the `BaseException` subclasses. - "I use bare except for safety." Backwards — it removes the safety net Python provides between `Exception` and `BaseException`.

Common follow-up: "What's the cleanest way to log every uncaught exception in a script?" — `sys.excepthook = my_handler` or, in modern code, a `try: main() except Exception: log.exception('crash')` at the top with `KeyboardInterrupt` allowed to propagate.

```
# Wrong – eats Ctrl-C and sys.exit
try:
    do_work()
except:
    log.error("something failed")

# Wrong – silently passes on everything including programmer errors
try:
    do_work()
except:
    pass

# Right – explicit, catches application failures, lets system signals propagate
try:
    do_work()
except Exception:
    log.exception("work failed")
    raise
```

Q8 · Re-raising an Exception — bare raise vs raise X

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Postman, Goldman · Topic: Exceptions Fundamentals

The question (interviewer's verbatim phrasing):

Inside an `except` block, what's the difference between bare `raise` and `raise SomeException(...)` ?

The 30-second answer: Bare `raise` re-raises the current exception with its original traceback intact — it's "let this go up the stack untouched." `raise X(...)` raises a new exception and sets the current exception as the new one's implicit context (`__context__`). Use bare `raise` when you want to log or do cleanup but not change the error. Use `raise X(...) from e` when you're translating into a domain exception.

Step-by-step thought process: 1. "Inside an `except` block, `raise` with no argument re-raises whatever `except` caught. Traceback is preserved exactly — you'll see the original raise site, not the line of the bare `raise` ." 2. " `raise NewError(...)` creates a new exception object and raises it. Python automatically sets the just-handled exception as `NewError.__context__` . Traceback will show both — 'during handling of the above exception, another exception occurred.' " 3. " `raise NewError(...) from e` is explicit chaining. Sets `NewError.__cause__ = e` and suppresses the context. Traceback shows 'the above exception was the direct cause.' This is what you want at API boundaries — your domain error references the underlying technical cause, but only once." 4. "Sometimes you want to re-raise the same exception with extra info on it — modify `e.args` , set an attribute, then bare `raise` . The traceback stays clean and the additional info travels up." 5. "Bare `raise` outside an `except` block is a `RuntimeError` — there's nothing to re-raise."

Edge cases to mention: - `raise e` (with the variable, not bare) is allowed but mildly worse — it sets a fresh traceback starting at the `raise e` line in 3.11+ behavior is correct, but the older Python semantic was to truncate the original traceback. Use bare `raise` to be safe. - After 3.11, the traceback of bare `raise` shows the same lines whether you write `raise` or `raise e` , but bare is the canonical form. - Inside a `finally` block, a bare `raise` re-raises the original `try` -block exception (if any). If you raise a new exception in `finally` , it shadows the original (which lives on as `__context__`).

What NOT to say: - " `raise e` and `raise` are identical." Functionally close in modern Python but not historically. Bare `raise` is the canonical idiom. - "I always `raise SomeException(str(e))` so I have control." Loses the original traceback frames and chaining. Use `from e` if you must wrap.

Common follow-up: "How do I add information to an exception without losing the traceback?" — Modify `e.args` or set an attribute on `e` , then bare `raise` . Or use `raise type(e)(extra_info, *e.args).with_traceback(e.__traceback__) from None` for a heavier rewrap.

```

try:
    response = http.get(url)
except ConnectionError as e:
    # Option 1 – log and let it propagate untouched
    log.warning("connection failed: %s", e)
    raise

try:
    response = http.get(url)
except ConnectionError as e:
    # Option 2 – translate at the API boundary
    raise PaymentTimeout("gateway unreachable") from e

try:
    parse(data)
except ValueError as e:
    # Option 3 – annotate and re-raise (3.11+ has add_note for this)
    e.add_note(f"while parsing field={field}")
    raise

```

Q9 · Exception Object Attributes — args, traceback, cause, context, notes

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Exceptions Fundamentals

The question (interviewer's verbatim phrasing):

What attributes does an exception object carry, and what's `add_note` from 3.11?

The 30-second answer: Every exception carries `args` (the tuple passed to `__init__`), `__traceback__` (the linked traceback chain), `__cause__` (set by `raise X from Y`), `__context__` (set automatically when raising during except), `__suppress_context__` (set by `raise X from None`), and from 3.11+ `__notes__` (a list of strings added via `add_note(msg)` for richer diagnostic context without subclassing).

Step-by-step thought process: 1. "`args` holds whatever you passed to the constructor — `Exception('boom', 42)` gives `args = ('boom', 42)`. `str(e)` joins them; for single-arg exceptions it's just that arg." 2. "`__traceback__` is the linked list of frames from where the exception was raised. Accessible programmatically — `traceback.format_tb(e.__traceback__)` formats it the way the interpreter would." 3. "`__cause__` is set by `raise X from Y`. Read it to see the deliberate chain. `__context__` is set automatically when an exception is raised during the handling of another — the implicit chain. `__suppress_context__` is True when you used `from None`." 4. "`__notes__` (3.11+) is a list of strings. Call `e.add_note('while processing order 42')` and the note shows in the traceback below the exception line. Much cleaner than wrapping in a new

exception just to add a string." 5. "Two patterns you'll see in real code —

`traceback.TracebackException.from_exception(e)` for structured serialization, and `logging.Logger.exception(msg)` which already grabs `sys.exc_info()` and formats it for you."

Edge cases to mention: - `args` must be a tuple — if you set `self.args = "string"`, Python silently stores it as a 1-tuple but iteration over `e.args` then iterates characters. Use `super().__init__(message)` to set it properly. - `__traceback__` can be set explicitly via `with_traceback(tb)` — useful when re-raising after capturing. - `add_note` accepts only strings. Don't pass exception objects or dicts; format them yourself.

What NOT to say: - "Exceptions are just strings." Misses that they're full objects with their own attributes, methods, and state. - "`__notes__` replaces chaining." It doesn't — chaining links exceptions; notes add prose to a single exception.

Common follow-up: "How do you preserve the original traceback when re-raising after some processing?" — Bare `raise` does it automatically. If you've left the except block, use `raise OriginalException(...).with_traceback(saved_tb)`.

```
import traceback

try:
    process(order)
except ValidationError as e:
    e.add_note(f"order_id={order.id}")          # 3.11+
    e.add_note(f"received_at={order.created_at}")
    raise

# Reading the chain
def explain(exc: BaseException) -> None:
    print(f"caught: {type(exc).__name__}: {exc.args}")
    print(f"notes: {getattr(exc, '__notes__', [])}")
    if exc.__cause__:
        print(f"caused by: {exc.__cause__!r}")
    if exc.__context__ and not exc.__suppress_context__:
        print(f"during handling of: {exc.__context__!r}")
    print("traceback:", "".join(traceback.format_tb(exc.__traceback__)))
```

Q10 · Exception in finally — Which Exception Wins

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, Microsoft India · Topic: Exceptions Fundamentals

The question (interviewer's verbatim phrasing):

If `try` raises one exception and `finally` raises another, which one propagates? What happens to the first one?

The 30-second answer: The `finally` exception wins and propagates up. The original (from `try`) becomes its `__context__` and shows in the traceback under "during handling of the above exception, another exception occurred." Both are still on the exception object — the finally exception is the live one, the try exception is reachable via `__context__`. So information isn't lost, but the caller sees the finally one first.

Step-by-step thought process: 1. "When `try` raises and `finally` runs cleanly, the try exception propagates. Easy case." 2. "When `try` raises and `finally` also raises, Python attaches the original to the new one's `__context__`. The new one is what `except` upstream sees; the original is reachable via `caught.__context__`." 3. "This is exactly why `finally` should be cleanup-only. If your cleanup fails (file close errored, lock release timed out), it shouldn't shadow the real bug from `try`. Wrap cleanup logic in its own try/except inside `finally` and log the cleanup failure rather than let it raise." 4. "`return` in `finally` has even worse semantics — it suppresses the try exception entirely (Python interprets the `return` as the function's actual result). Almost always a bug. Linters like Ruff catch this with `B012`." 5. "Context managers — the modern alternative — solve this neatly. `__exit__` can decide whether to suppress; if `__exit__` itself raises, you get the same context-stacking behavior, but the contract is explicit."

Edge cases to mention: - A `break`, `continue`, or `return` inside `try` is also subject to `finally` — if `finally` then raises, the break/continue/return is canceled and the new exception propagates. - If `finally` raises and there's no `try` exception, it just propagates normally — no context shenanigans. - Generators have a similar pattern with `GeneratorExit` — if `finally` in a generator raises, the cleanup of `.close()` raises that instead.

What NOT to say: - "The try exception always wins." Wrong — finally's exception takes precedence and the try exception becomes context. - "Putting `return` in finally is a clean way to override." Almost always a bug; the linter will flag it and your team will hate you.

Common follow-up: "How do you make sure cleanup failures don't hide the original error?" — Wrap the cleanup in its own try/except inside finally and log the cleanup failure; let the original propagate. Or use a context manager that internally guards `__exit__`.

```
# Bad – close failure shadows the real bug
def write_report(path):
    try:
        f = open(path, "w")
        f.write(make_report())          # imagine this raises ValueError
    finally:
        f.close()                      # close raises OSError; ValueError lost as __context__

# Better – protect cleanup, let the real error propagate
def write_report_v2(path):
    f = None
    try:
        f = open(path, "w")
        f.write(make_report())
    finally:
        if f is not None:
            try:
                f.close()
            except OSError:
                log.warning("close failed", exc_info=True)

# Best – context manager
def write_report_v3(path):
    with open(path, "w") as f:
        f.write(make_report())
```

Section B — Custom Exceptions

Q11 · When Is It Worth Defining A Custom Exception

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Postman, Freshworks · Topic: Custom Exceptions

The question (interviewer's verbatim phrasing):

When do you write a custom exception class versus just raising `ValueError` or `RuntimeError`?

The 30-second answer: Define a custom exception when callers need to distinguish your specific failure mode from the dozens of generic ones the language and standard library already throw. If your code raises `ValueError("payment declined")`, no caller can catch "just payment declined" without parsing the message. A `PaymentDeclined` class makes that catch one line. The threshold is "would a reasonable caller want to write `except YourError:` separately from other errors?" — if yes, define one.

Step-by-step thought process: 1. "The test is — does the caller want different recovery for your failure than for a generic one? If the answer is yes, you need a class. If the caller would just log-and-continue regardless, a `stdlib` exception is fine." 2. "Custom exceptions are also valuable for API contracts. A REST handler that catches `PaymentError` and converts to HTTP 402 needs to distinguish payment failures from validation failures from infrastructure failures. Generic exceptions don't carry that semantic." 3. "Don't over-engineer. Single-use scripts, internal helpers, and one-shot CLI tools rarely justify their own exception class. Catching `ValueError` is fine for code with one caller — yourself." 4. "Use `stdlib` classes when they fit. `ValueError` for bad input, `TypeError` for wrong type, `KeyError` for missing dict key, `FileNotFoundError` for missing file, `TimeoutError` for any timeout. These read naturally and integrate with existing handlers." 5. "Custom exceptions earn their keep at module/package boundaries. A `requests` library raises `requests.exceptions.ConnectionError` so users can catch it without importing the lower-level `urllib3` exceptions. Same pattern — your module owns its failure namespace."

Edge cases to mention: - Even for one-off scripts, a custom exception can be useful if the script uses retry logic — `except RetryableError:` and `except FatalError:` separate the two paths cleanly. - Subclassing a `stdlib` class is a half-step — `class InvalidPincode(ValueError)` reads naturally and existing `except ValueError` handlers still catch it. - Counter-pattern: defining 20 custom exceptions for a 50-line module is over-engineering. One base + 2-3 specific subclasses is usually the sweet spot.

What NOT to say: - "Always define custom exceptions for everything." Bloats the codebase and makes simple tools feel like enterprise software. - "Never define custom exceptions, `stdlib` is enough." Loses API expressiveness at module boundaries.

Common follow-up: "What's the smallest possible custom exception?" — `class PaymentError(Exception): pass`. No methods, no `init` — Python's default `init` from `Exception` is enough. The class itself is the contract.

```
# Minimal — pass-only class with a docstring
class PaymentError(Exception):
    """Base for all payment-module failures."""

# Slightly richer — extra attributes for the caller to use
class PaymentDeclined(PaymentError):
    def __init__(self, reason_code: str, gateway_message: str):
        super().__init__(f"declined: {reason_code} — {gateway_message}")
        self.reason_code = reason_code
        self.gateway_message = gateway_message

# Subclassing a stdlib type — fine when the relationship is real
class InvalidPincode(ValueError):
    """Pincode failed format or registry check."""
```

Q12 · Naming Convention — Why Always Use The Error Suffix

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman · Topic: Custom Exceptions

The question (interviewer's verbatim phrasing):

Why do Python exception classes end with `Error` and not `Exception` ?

The 30-second answer: PEP 8 convention. Python's own hierarchy uses `Error` as the suffix — `ValueError` , `TypeError` , `KeyError` , `FileNotFoundError` . The `Exception` suffix is reserved for the root `Exception` class itself and for direct subclasses of `BaseException` that aren't errors per se (e.g., `StopIteration`). Following this convention means your `PaymentError` reads consistently with `ValueError` next to it in code; `PaymentException` looks visually wrong inside `except (ValueError, PaymentException):` .

Step-by-step thought process: 1. "PEP 8 says: 'Because exceptions should be classes, the class naming convention applies here. However, you should use the suffix `Error` on your exception names (if the exception actually is an error).'

2. "Look at the `stdlib` — `ValueError` , `TypeError` , `KeyError` , `IndexError` , `OSError` , `RuntimeError` , `LookupError` , `AttributeError` , `ImportError` . Hundreds of `*Error` names, almost no `*Exception` names. Match the convention and your code looks native."

3. "The few exceptions to the rule — `Exception` itself, `StopIteration` , `KeyboardInterrupt` , `GeneratorExit` . These aren't errors in the 'something went wrong' sense; they're control-flow signals. So `Exception` is reserved for the root, and signal classes use other names."

4. "Why not `PaymentException` ? It works, Python doesn't enforce naming. But it visually clashes inside catch clauses — `except (ValueError, PaymentException)` looks awkward. Code review at any serious team will flag it."

5. "Frameworks reinforce the convention. Django uses `ObjectDoesNotExist` , `MultipleObjectsReturned` , `ValidationError` . SQLAlchemy uses `IntegrityError` , `OperationalError` , `DataError` . Pydantic uses `ValidationError` . They all follow the `*Error` rule."

Edge cases to mention: - Some legacy libraries (especially Python 2-era code) use `*Exception` as a suffix — Twisted has `ConnectionRefusedException` historically. Don't follow the legacy; PEP 8 settled this around 2001. - For warning classes (not errors), the convention is `*Warning` — `DeprecationWarning` , `UserWarning` . Different namespace, same idea. - Don't put your team name or company in the class name (`RazorpayPaymentError`) — keep the module prefix where it belongs, in the import path.

What NOT to say: - " `Exception` is more descriptive." It's not, and the convention is settled. Use `Error` . - "It doesn't matter, Python doesn't enforce it." Style isn't enforced by the interpreter but is enforced by every team that does code review.

Common follow-up: "What about classes that aren't errors but signal something?" — Different suffix or no suffix. `StopIteration`, `GeneratorExit`. For warnings, `DeprecationWarning`. Only "this is a failure" classes get `Error`.

```
# Right – convention-compliant
class PaymentError(Exception): pass
class PaymentDeclinedError(PaymentError): pass
class GatewayTimeoutError(PaymentError): pass

# Wrong – works but reads off
class PaymentException(Exception): pass

# Library example following the convention
class RazorpayError(Exception):
    """Base for Razorpay SDK errors."""
class BadRequestError(RazorpayError): pass
class AuthenticationError(RazorpayError): pass
class ServerError(RazorpayError): pass
```

Q13 · Designing An Exception Hierarchy For A Module

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Postman, Cred · Topic: Custom Exceptions

The question (interviewer's verbatim phrasing):

I'm writing a payments module. Design the exception hierarchy.

The 30-second answer: One module-level base — `PaymentError(Exception)`. Then specific subclasses for each failure mode the caller can reasonably distinguish: `PaymentDeclined`, `PaymentTimedOut`, `InvalidCardNumber`, `InsufficientFunds`. Optionally one more layer to separate transient from permanent — `TransientPaymentError(PaymentError)` and `PermanentPaymentError(PaymentError)` — so retry logic can catch the family without enumerating individual classes.

Step-by-step thought process: 1. "The base — `class PaymentError(Exception):` — is the catch-all for anything this module raises. Callers that want a wide net write `except PaymentError`. Callers that want specifics catch the subclasses." 2. "Specific failures — one class per distinguishable mode. `PaymentDeclined` for gateway refusals, `InvalidCardNumber` for Luhn failures, `InsufficientFunds` for balance issues, `PaymentTimedOut` for network timeouts. Each catchable separately." 3. "A second layer of taxonomy is useful when callers' branching cuts across the specific classes. Retry-vs-fatal is the classic split — a connection timeout is retryable, an invalid card number isn't. So `class TransientPaymentError(PaymentError):` and `class PermanentPaymentError(PaymentError):` as intermediate bases.

`PaymentTimedOut(TransientPaymentError)`,
`InvalidCardNumber(PermanentPaymentError)`." 4. "Document the hierarchy in the module docstring. Callers shouldn't have to read source to know what they can catch. The hierarchy IS the API contract." 5. "Don't go four levels deep. `PaymentError` → `CardError` → `VisaError` → `Visa3DSError` is over-engineered. Keep it 2-3 levels: module base, optional category, specific failure."

Edge cases to mention: - If your module wraps a third-party library (e.g., `requests`), translate at the boundary so callers don't have to import the library's exception classes. `except requests.exceptions.ConnectionError as e: raise PaymentTimedOut from e`. - Inherit from a stdlib class only when the inheritance reads true. `class InvalidCardNumber(ValueError, PaymentError):` is fine — invalid input genuinely is a `ValueError`. Multi-inheritance from two unrelated classes is harder to reason about. - Sometimes the right "subclass" is a flag/attribute rather than a class. `PaymentError(reason_code='declined')` works if `reason_code` is small and rarely catchable individually.

What NOT to say: - "One generic `PaymentError` is enough." Loses the specific-catch ability. Callers can't distinguish "declined" from "timed out", which they need for retry decisions. - "Make a class for every possible reason code." Bloats the namespace. Use attributes for fine-grained reasons; classes for actionably-different cases.

Common follow-up: "How do you handle a new failure mode without breaking existing callers?" — Add it as a new subclass of an existing intermediate base. Existing `except PaymentError` still catches it; specific handlers ignore it; callers can opt in.

```
# Layered hierarchy
class PaymentError(Exception):
    """Base for all payment-module failures."""

class TransientPaymentError(PaymentError):
    """Network / gateway issues; safe to retry."""

class PermanentPaymentError(PaymentError):
    """Business-rule failures; do not retry without changes."""

class PaymentTimedOut(TransientPaymentError): pass
class GatewayUnavailable(TransientPaymentError): pass

class PaymentDeclined(PermanentPaymentError):
    def __init__(self, reason_code: str):
        super().__init__(f"declined: {reason_code}")
        self.reason_code = reason_code

class InvalidCardNumber(PermanentPaymentError): pass
class InsufficientFunds(PermanentPaymentError): pass

# Caller – retry logic catches the family
for attempt in range(3):
    try:
        charge_card(order)
        break
    except TransientPaymentError as e:
        log.info("retry %d: %s", attempt, e)
    except PermanentPaymentError:
        log.error("fatal – not retrying")
        raise
```

Q14 · Custom Exception init — Carrying Extra Attributes

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, JP Morgan · Topic: Custom Exceptions

The question (interviewer's verbatim phrasing):

How do you put extra structured data on a custom exception — say, the HTTP status code and response body from a failed API call?

The 30-second answer: Override `__init__` to accept the extra args, call `super().__init__(message)` to set `args` and the human-readable message, then assign attributes on `self`. Callers can read them off the caught exception. For consistency, also override `__str__` if the message should change based on the attributes, and `__repr__` if you want clean debugging output. The attributes are normal Python — anything pickleable works.

Step-by-step thought process: 1. "Subclass the right base, override `__init__` to take the structured args, call `super().__init__(some_human_message)` so `args`, `str(e)`, and pickling work." 2. "After `super().__init__()`, assign the extra attributes — `self.status_code = status_code`, `self.body = body`. These are normal instance attributes; the caller reads them with `e.status_code`." 3. "Make sure the human-readable message is informative without dumping the whole body. `f'API failed with status {status_code}'` is fine; including a 1MB response body in `str(e)` floods logs." 4. "Override `__str__` if you want the printed form to vary. Default `Exception.__str__` just stringifies `args[0]`, which is usually fine. Override `__repr__` for `repr(e)` if you want something like `APIError(status=502, message='gateway')`." 5. "Make extra attributes optional with defaults so existing callers that only know the base form don't break. `def __init__(self, message, status_code=None, body=None):` is friendlier than requiring all three."

Edge cases to mention: - Extra attributes don't show in `args` unless you pass them to `super().__init__`. If you want them in `args` for pickling — `super().__init__(message, status_code, body)` — but then `str(e)` may include them. Tradeoff. - If you forget `super().__init__`, your exception will have an empty `args` tuple and `str(e)` returns `''`. Always call super. - For very simple "tag" attributes, consider using `dataclasses.dataclass` on a subclass — but it adds a tiny amount of magic; many teams prefer explicit `__init__`.

What NOT to say: - "Just put everything in `args`." `args` is for the human message and pickle support; for structured data, attributes are clearer. - "I'd extend the base class with new methods." Methods are fine; the question is data. Attributes carry data; methods carry behavior.

Common follow-up: "How does the caller catch this and access the attributes?" — `except APIError as e: log.error('failed: %s', e.status_code)`. The variable bound by `as e` exposes everything on the instance.

```

class APIError(Exception):
    """A call to the gateway failed at the HTTP layer."""

    def __init__(
        self,
        message: str,
        *,
        status_code: int | None = None,
        body: str | None = None,
        request_id: str | None = None,
    ):
        super().__init__(message)
        self.status_code = status_code
        self.body = body
        self.request_id = request_id

    def __str__(self) -> str:
        # Override only when extra info improves the printed form
        if self.status_code is not None:
            return f"{self.args[0]} (status={self.status_code}, req={self.request_id})"
        return self.args[0]

# Raising
raise APIError(
    "razorpay capture failed",
    status_code=502,
    body=response.text[:500],
    request_id=response.headers.get("X-Request-Id"),
)

# Catching
try:
    capture_payment(order)
except APIError as e:
    log.error("status=%s req=%s body=%s", e.status_code, e.request_id, e.body)
    if e.status_code and e.status_code >= 500:
        schedule_retry(order)

```

Q15 · str vs repr On An Exception

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Goldman · Topic: Custom Exceptions

The question (interviewer's verbatim phrasing):

What's the difference between `__str__` and `__repr__` on an exception class? Which does the traceback use?

The 30-second answer: The traceback shows `f"{type(e).__name__}: {e}"` — so it's `__str__`. `__repr__` is what you see when you print the exception object in a REPL or log it with `%r`. Override `__str__` for the human-readable message; override `__repr__` for the debugging-readable form. Default `Exception.__str__` returns `str(args[0])` for single-arg or `str(args)` for multiple. Default `Exception.__repr__` returns `f"{type(e).__name__}({args!r})"`.

Step-by-step thought process: 1. "`__str__` controls `str(e)` and is what appears in the traceback after the colon. `__repr__` controls `repr(e)` and is what appears in REPL output, `%r` formatting, and debugger displays." 2. "Most exceptions just need a good `__str__`. The default — print `args[0]` — is fine for `raise ValueError('bad pincode')`. Override when you want richer formatting from structured fields." 3. "Override `__repr__` when the default `ClassName(arg1, arg2)` is unhelpful — e.g., when your `args` contain large bodies. A clean `APIError(status=502, request='req_abc123')` reads better than `APIError('failed', 502, 'huge body...')`." 4. "Implication for logging — `log.error('caught %s', e)` calls `__str__`; `log.error('caught %r', e)` calls `__repr__`. Pick the format string deliberately." 5. "Implication for tracebacks — Python prints `ClassName: <str(e)>`. If `str(e)` is empty (you forgot to call `super().__init__(message)`), the traceback just shows the class name. Common bug; check that your custom exceptions have non-empty `str()`."

Edge cases to mention: - `__str__` returning `''` is allowed and produces a traceback line that ends at the colon with nothing after. Visually broken. - `__repr__` is what `pytest` uses in assertion failure messages. A clear repr helps test diagnostics. - If you override `__str__` to do expensive work (e.g., format a 10k-row dataframe), logging the exception becomes slow. Keep `__str__` cheap.

What NOT to say: - "`__str__` and `__repr__` are the same." They're different protocols with different defaults and different consumers. - "Override one and ignore the other." Reasonable for `__repr__` if the default suffices; rarely reasonable to skip `__str__` for a custom exception with structured data.

Common follow-up: "What does the default `Exception.__repr__` look like?" — `f"{type(e).__name__}{e.args!r}"` — e.g., `PaymentError('declined', 502)`. Useful but loud when args are big.


```

class APIError(Exception):
    def __init__(self, message: str, *, status_code: int, request_id: str):
        super().__init__(message)
        self.status_code = status_code
        self.request_id = request_id

    def __str__(self) -> str:
        return f"{self.args[0]} (status={self.status_code})"

    def __repr__(self) -> str:
        return (
            f"APIError(message={self.args[0]!r}, "
            f"status_code={self.status_code}, request_id={self.request_id!r})"
        )

e = APIError("capture failed", status_code=502, request_id="req_42")
print(str(e)) # capture failed (status=502)
print(repr(e)) # APIError(message='capture failed', status_code=502, request_id='req_42')
# Traceback would show:
# APIError: capture failed (status=502)

```

Section C — Exception Chaining

Q16 · raise X from Y — What It Does

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Postman, Goldman, Microsoft India · Topic: Exception Chaining

The question (interviewer's verbatim phrasing):

What does `raise X from Y` do, and when do you use it?

The 30-second answer: `raise X from Y` declares that exception Y caused exception X. Python sets `X.__cause__ = Y` and `X.__suppress_context__ = True`. The traceback shows both with the message "The above exception was the direct cause of the following exception." Use it at boundaries where you're translating a low-level technical failure into a domain-level one — your module raises `PaymentError`, the underlying `requests.ConnectionError` becomes the `__cause__`, so debuggers still see the original.

Step-by-step thought process: 1. "`from Y` sets two things — `X.__cause__ = Y` and `X.__suppress_context__ = True`. The first is the explicit cause; the second tells the traceback printer to skip the implicit `__context__` chain that would otherwise show." 2. "The pattern this exists for — translating exceptions at boundaries. Your module exposes its own exception types, but internally it

uses libraries that raise their own. Translating with `from` preserves the chain so debugging is possible."

3. "Without `from`, an exception raised during except handling still chains automatically via `__context__` — Python sets `X.__context__ = Y` even without the `from` keyword. The difference is intent — `from` says 'I deliberately translated'; bare context says 'this happened during handling'." 4. "Traceback formatting reflects the difference. `from` produces 'The above exception was the direct cause'. Implicit context produces 'During handling of the above exception, another exception occurred'. Both are useful, but `from` reads as intentional." 5. "Practical rule — whenever you write `raise NewError(...)` inside an `except SomeOldError as e:` block, you almost certainly want `raise NewError(...) from e`. The exceptions to this are when the original is noise the caller doesn't need to see (covered in the `from None` question)."

Edge cases to mention: - `from` can chain to any exception object, not just one you caught.

`raise A from cached_error` is legal. - `from` outside an except block also works — `raise A from B` sets `A.__cause__ = B` regardless of where you are. - The cause itself can have its own cause, leading to chains 3-5 deep. Tracebacks walk the chain and show every layer.

What NOT to say: - "`from` just sets the traceback string." Misses that `__cause__` is a real attribute you can inspect programmatically. - "I just log the underlying and raise a new one." Loses the link. Use `from e` and the original travels automatically.

Common follow-up: "What's the difference between `from e` and `from None`?" — `from e` chains the original as `__cause__`; `from None` suppresses the implicit context that would otherwise show. Use the second when the original is internal noise.

```

import requests

class PaymentTimedOut(Exception):
    """Gateway didn't respond in time."""

def charge(order):
    try:
        response = requests.post(
            "https://api.razorpay.com/v1/payments",
            json=order.to_dict(),
            timeout=5,
        )
    except requests.exceptions.Timeout as e:
        # Translate at the boundary; original preserved via __cause__
        raise PaymentTimedOut(f"gateway didn't respond in 5s for order {order.id}") from e

# Traceback when this fires upstream:
#
# Traceback (most recent call last):
#   File "module.py", line ..., in charge
#     response = requests.post(...)
# requests.exceptions.Timeout: HTTPSConnectionPool(...): Read timed out.
#
# The above exception was the direct cause of the following exception:
#
# Traceback (most recent call last):
#   File "caller.py", line ..., in run
#     charge(order)
# module.PaymentTimedOut: gateway didn't respond in 5s for order 1234

```

Q17 · cause vs context — The Difference

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Goldman · Topic: Exception Chaining

The question (interviewer's verbatim phrasing):

What's the difference between `__cause__` and `__context__` on an exception?

The 30-second answer: `__cause__` is set deliberately by `raise X from Y`. `__context__` is set automatically whenever you `raise X` while another exception is already being handled. Both produce chains in the traceback but with different prose — `__cause__` shows "the direct cause"; `__context__` shows "during handling of the above exception, another exception occurred." `__cause__` is intent; `__context__` is history.

Step-by-step thought process: 1. "`__context__` is automatic. The moment you write `raise B` inside an `except A` block (without `from`), Python sets `B.__context__ = A`. You don't have to

do anything." 2. " `__cause__` is explicit. Only set when you write `raise B from A` . It also sets `__suppress_context__ = True` to keep the traceback from showing `__context__` as well (which would be redundant since `__cause__` is the same exception)." 3. "Why two? Because not every nested raise is a deliberate chain. Sometimes you raise a brand-new exception during handling — say, your error handler itself fails. That's implicit context, not a deliberate translation. `__context__` captures it for debugging without forcing intent." 4. "When debugging, check both. If `__cause__` is `None` and `__context__` is set, the original developer didn't use `from` but you still get to see what was happening when the new exception fired." 5. " `raise X from None` clears the traceback display by setting `__suppress_context__ = True` but does not actually unset `__context__` — the original is still on the object. Use it when the context is internal noise."

Edge cases to mention: - An exception raised outside any except block has both `__cause__` and `__context__` as `None` — there's nothing to chain. - `__suppress_context__` is just a boolean; you can set it manually if you really need to: `e.__suppress_context__ = True` . Almost always cleaner to use `from None` . - An exception with a non-`None` `__cause__` also has `__suppress_context__ = True` automatically. The two attributes are coordinated.

What NOT to say: - " `__cause__` and `__context__` are the same thing." They're closely related but distinct — one is intent, the other is history. - "Just use whichever shows in the traceback." Both show by default; the difference is the prose between them and whether you wrote `from` .

Common follow-up: "How do you suppress `__context__` from the traceback without changing `__cause__` ?" — Set `e.__suppress_context__ = True` directly, or `raise e from None` (the second also sets `__cause__` to `None`, so only do that if you want to suppress both).

```

try:
    try:
        d = {}
        d["missing"]
    except KeyError:
        # No `from`. The KeyError is __context__ of ZeroDivisionError.
        # Traceback shows: "During handling of the above exception, another
        # exception occurred:" between them.
        1 / 0
except ZeroDivisionError as e:
    print("cause: ", e.__cause__)          # None
    print("context: ", e.__context__)      # KeyError('missing')
    print("suppress:", e.__suppress_context__) # False

try:
    try:
        d = {}
        d["missing"]
    except KeyError as ke:
        # Explicit chain. Now __cause__ is set and __suppress_context__ True.
        # Traceback shows: "The above exception was the direct cause..."
        raise RuntimeError("recovery failed") from ke
except RuntimeError as e:
    print("cause: ", e.__cause__)          # KeyError('missing')
    print("context: ", e.__context__)      # also KeyError('missing')
    print("suppress:", e.__suppress_context__) # True

```

Q18 · Implicit Chaining — What Happens When You raise During except

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, JP Morgan · Topic: Exception Chaining

The question (interviewer's verbatim phrasing):

Inside an `except` block, if I `raise NewError(...)` without `from`, what does Python do automatically?

The 30-second answer: Python sets the new exception's `__context__` to the exception currently being handled, then propagates the new one. The traceback will display "During handling of the above exception, another exception occurred" and show both. It's a safety net — even if you forget `from`, the original exception is not lost; it's chained automatically.

Step-by-step thought process: 1. "The rule — every exception raised while another is being handled has `__context__` set automatically to the one being handled. The handler being 'active' is what matters; you don't have to do anything explicit." 2. "This is on by default to make sure no exception ever vanishes silently. Even a forgotten `from` doesn't lose the original — it still appears in the traceback as

context." 3. "`__context__` is a real attribute. You can read it programmatically — `caught.__context__` returns the original exception object, not just a string." 4. "Why not just always make this `__cause__` ? Because sometimes the raise inside except is not a translation — it's a brand-new failure (your error handler crashed). 'Context' is the neutral term; 'cause' is the deliberate version." 5. "The trade-off — implicit chaining can leak internals. If your library catches an internal `binascii.Error` and raises `ConfigInvalid`, the `binascii` is still in `__context__` and shows in the user's traceback. `from None` is how you suppress this."

Edge cases to mention: - The "currently being handled" exception is per-thread. Different threads don't see each other's contexts. - Inside an `async` function, the context is captured per-task — `asyncio` preserves the chain correctly across await boundaries. - A new exception raised in `finally` (not just `except`) also chains to whatever exception is propagating from `try`. Same mechanism.

What NOT to say: - "If I don't use `from`, the original exception is lost." Not true — it's still in `__context__`. - "Implicit chaining means I never need `from`." Wrong — `from` signals intent and produces cleaner tracebacks for translations.

Common follow-up: "If I raise during except and the new one has `__context__` set, can I still access the original later?" — Yes. `caught_exception.__context__` is the original. It survives all the way up the stack.

```
class ConfigInvalid(Exception):
    pass

def parse_config(text: str) -> dict:
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        # No `from` — context still set automatically
        raise ConfigInvalid("config is not valid JSON")

try:
    parse_config("not json")
except ConfigInvalid as e:
    print("cause: ", e.__cause__)      # None
    print("context:", e.__context__)  # JSONDecodeError(...)
    # Both are visible in the printed traceback even though
    # we didn't write `from`. Python's safety net.
```

Q19 · raise X from None — Suppressing The Chain

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Exception Chaining

The question (interviewer's verbatim phrasing):

When would you ever use `raise X from None` ?

The 30-second answer: When the underlying exception is internal noise the caller doesn't need to see. Your library wraps a low-level encoding error into a clean `ConfigInvalid`. The encoding error reveals implementation details (`binascii.Error: Incorrect padding`) and the user can't act on them. `from None` sets `__suppress_context__ = True` so the traceback shows only your clean exception, not the internal mess. Use sparingly — usually `from e` is friendlier because it preserves the debuggable chain.

Step-by-step thought process: 1. " `from None` is the way to say 'do not show the chain in the traceback'. It sets `__cause__ = None` and `__suppress_context__ = True` ." 2. "The pattern — at module boundaries, when the internal exception is implementation noise. Your config-loading library uses `binascii.b2a_base64` internally; if it fails with `binascii.Error`, the caller doesn't want to know `binascii` exists. `raise ConfigInvalid(...) from None` cleans the slate." 3. "Don't reach for `from None` reflexively. Most of the time `from e` is better — it preserves the chain for the developer debugging. Suppress only when the chain is genuinely confusing or revealing of internals." 4. "The original exception is still on the new one's `__context__` attribute — `from None` only suppresses display, not the attribute. Debuggers and structured logging can still find it if needed." 5. "Rare alternative use case — when you've already logged the original with full context elsewhere and don't want it appearing twice in the user-visible traceback. `from None` keeps the traceback short for the user, while the structured log has the full story."

Edge cases to mention: - `from None` outside an except block is legal but pointless — there's no context to suppress. - Setting `__suppress_context__ = True` manually has the same effect on display, but `from None` is the idiomatic form. - The new exception's `__context__` attribute is still set after `from None` if you raised it during except handling — only the traceback formatter respects `__suppress_context__`. Programmatic access still works.

What NOT to say: - " `from None` is more secure." Not a security tool. It's a UX tool for tracebacks. - "Always use `from None` to keep tracebacks short." Loses debuggability. Default to `from e`; use `from None` deliberately.

Common follow-up: "If I use `from None`, is the original exception completely gone?" — No. `__context__` still has it. `from None` only affects the traceback printer; programmatic inspection still finds the original.

```
import binascii
import json

class ConfigInvalid(Exception):
    pass

def decode_config(blob: str) -> dict:
    try:
        raw = binascii.a2b_base64(blob)
    except binascii.Error:
        # Internal detail – caller doesn't need to know we use base64
        raise ConfigInvalid("config is not valid base64") from None

    try:
        return json.loads(raw)
    except json.JSONDecodeError as e:
        # JSON error IS something the caller might care about – chain it
        raise ConfigInvalid("config payload is not valid JSON") from e

# When decode_config fails with "not valid base64", the traceback shows:
#   ConfigInvalid: config is not valid base64
# (no mention of binascii)

# When it fails with "not valid JSON", the traceback shows:
#   json.decoder.JSONDecodeError: ...
#
#   The above exception was the direct cause of the following exception:
#
#   ConfigInvalid: config payload is not valid JSON
```

Q20 · Debugging A Long Exception Chain

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Microsoft India, Walmart Labs · Topic: Exception Chaining

The question (interviewer's verbatim phrasing):

You hit a production traceback that's six layers deep — exception A caused B caused C caused D. How do you debug it?

The 30-second answer: Walk the chain from the bottom up, because the bottom is the live exception (what your code actually sees). The top of the chain is the root cause. In code: `e.__cause__` until None — that walks the deliberate chain. `e.__context__` walks the implicit chain. Use `traceback.format_exception(e)` to get the standard printed form, or

`traceback.TracebackException.from_exception(e)` for a structured object you can iterate programmatically.

Step-by-step thought process: 1. "Live exception is the one Python is currently propagating — bottom of the printed traceback, top of the call stack. That's what your except block catches. Its `__cause__` or `__context__` is the next layer down." 2. "Walk the chain — `while e: print(e); e = e.__cause__ or e.__context__`. Each step takes you one layer deeper toward the root cause." 3. "In a debugger — `pdb.post_mortem()` drops you at the live exception. To walk down, set `e = e.__cause__` and inspect each layer." 4. "In production logs — `logger.exception(msg)` already grabs the full chain via `sys.exc_info()` and formats it. Look for the topmost stanza in the printed traceback — that's the root." 5. "For structured analysis — `traceback.TracebackException` is iterable. You can filter by class, count layers, extract just the inner traceback frames. Useful for incident dashboards that need 'root cause exception class' as a tag."

Edge cases to mention: - A cycle in the chain (X causes Y causes X) is technically possible but never happens in practice — Python doesn't prevent it, but writing it deliberately would take work. - If a chain mixes `__cause__` and `__context__` at different levels, your walk needs to handle both. `e.__cause__ or e.__context__` covers most cases. - Stack frames in `__traceback__` are per-exception, not chain-wide. To see all frames you need to walk each exception's traceback separately.

What NOT to say: - "Just look at the top of the traceback." The top of the printed traceback is the root cause; the bottom is the live exception. Knowing the direction matters. - "Chain depth doesn't matter." It does — six-layer chains often indicate that a low-level error is being re-wrapped at every framework boundary, which is a smell.

Common follow-up: "How do you get a structured representation of the chain for logging?" — `traceback.TracebackException.from_exception(e)`. It exposes `.exceptions`, `.__cause__`, `.__context__` as iterable objects you can serialize.

```
import traceback

def walk_chain(exc: BaseException) -> list[BaseException]:
    """Return the chain from live → root (live first)."""
    chain = []
    cur = exc
    while cur is not None:
        chain.append(cur)
        # Prefer cause (deliberate) over context (implicit)
        cur = cur.__cause__ or (
            cur.__context__ if not cur.__suppress_context__ else None
        )
    return chain

try:
    do_thing_that_chains()
except Exception as e:
    chain = walk_chain(e)
    for i, layer in enumerate(chain):
        log.error("layer %d: %s: %s", i, type(layer).__name__, layer)
    log.error("root cause: %s", type(chain[-1]).__name__)
    # Or just dump the standard formatted traceback
    log.error("full traceback:\n%s", "".join(traceback.format_exception(e)))
```

Q21 · When To Chain vs When To Swallow

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Cred, Postman · Topic: Exception Chaining

The question (interviewer's verbatim phrasing):

Decision tree — when do you chain an exception, when do you swallow it silently, and when do you let it propagate as-is?

The 30-second answer: Chain (`raise X from e`) when you're translating across an API boundary and the caller benefits from seeing both. Let it propagate (no try/except) when the caller is already in the right namespace to handle it. Swallow (catch and return a default) only when the failure is expected and recoverable in this exact spot — and even then, log it. The default mode for a handler should be "re-raise"; swallowing is the exceptional case.

Step-by-step thought process: 1. "Default — let it propagate. Most code shouldn't catch exceptions at all. The caller decides what to do; your function just does its job and raises on failure." 2. "Catch and translate — at API boundaries where your module raises its own types. Your `payments` module catches `requests.ConnectionError` and raises `PaymentTimeout from e`. Caller benefits from the domain language, debugger still sees the underlying cause." 3. "Catch and recover — when the

failure is genuinely expected and recoverable. Reading a missing optional config file → return `{}`.
 Cache miss → fetch from source. These are not exceptional; they're routine." 4. "Catch and swallow — almost never. The smell is `except Exception: pass`. The legitimate cases are vanishingly rare — a non-critical telemetry call in a shipping path, with a log of the failure. And it should still be `except Exception: log.warning('...', exc_info=True)`, not silent pass." 5. "Decision shortcut — if you swallow, can you defend the decision to a reviewer? 'I logged it and continued because the failure is non-critical and recovery is impossible here.' If you can't articulate it that crisply, you're hiding bugs."

Edge cases to mention: - Re-raising after side effects (logging, metric increment, alert) is propagation, not swallowing. `try: ... except: log; raise` is fine. - Swallowing in tests is a smell too — `pytest.raises` is the idiom, not `try: ... except: pass`. The exception is the test signal. - A `contextlib.suppress(SpecificException)` block is a sanctioned form of swallowing when the suppression is the whole point of that small block.

What NOT to say: - "Always catch exceptions to keep the program running." That's how silent corruption ships to production. - "Never catch exceptions, always let them propagate." Loses recovery opportunities at the right boundaries.

Common follow-up: "How do you tell at code review whether a `try/except` is swallowing too much?" — If the except block has no `raise`, no `return` of a meaningful value, and no logging — it's swallowing. Flag it.

```
# Propagate — default
def parse_pincode(s: str) -> int:
    return int(s)    # caller handles ValueError if malformed

# Translate at boundary
def fetch_payment(order_id: str) -> Payment:
    try:
        response = http.get(f"/payments/{order_id}", timeout=5)
    except requests.ConnectionError as e:
        raise PaymentTimedOut from e    # chain — caller speaks our language

# Recover — failure is routine
def load_config(path: str) -> dict:
    try:
        with open(path) as f:
            return json.load(f)
    except FileNotFoundError:
        return {}    # legit: no config means use defaults

# Anti-pattern — swallow with no signal
def update_cache(key, value):
    try:
        cache.set(key, value)
    except Exception:
        pass    # bug: cache misses silently, no metric, no log
```

Q22 · Exception Chaining In Generators And Async — How It Behaves

Tags: Difficulty: Hard · Asked at: Razorpay, Microsoft India, Goldman · Topic: Exception Chaining

The question (interviewer's verbatim phrasing):

What happens to the exception chain when an async function awaits another and the inner one raises?

The 30-second answer: Chains propagate normally across `await` — `await inner()` re-raises the inner's exception in the outer's frame, with the chain intact. If the outer catches and re-raises with `from e`, the chain extends. In generators, `yield` is the equivalent boundary — exceptions raised in the consumer via `throw()` enter the generator's `yield` line and can be caught with try/except as usual. The chain attribute model is the same as synchronous code.

Step-by-step thought process: 1. "`await` is transparent for exceptions. The inner coroutine raising `KeyError` causes `await inner()` to raise `KeyError` in the outer. The traceback shows both frames stitched together via the `await`." 2. "In `asyncio.TaskGroup` (3.11+), failures from tasks are bundled into an `ExceptionGroup`. The chain still works per leaf — each leaf exception has its own `__cause__` and `__context__` if anything translated it before being added to the group." 3. "`gather(return_exceptions=True)` returns exceptions as values, not raises them. The chain is still on each exception object, but you have to inspect each one — no automatic propagation." 4. "In generators — `gen.throw(SomeError)` raises `SomeError` at the current `yield` line of the generator. Inside the generator you can `except SomeError:` and even `raise NewError from e`. The chain travels back to the consumer normally." 5. "`yield from` propagates exceptions in both directions — exceptions raised inside the delegated-to generator surface in the consumer; exceptions thrown into the consumer reach the delegated-to generator. Chain attributes are preserved across `yield from` boundaries."

Edge cases to mention: - `asyncio.shield(coro)` lets the outer be cancelled without cancelling the inner. The `CancelledError` gets a `__context__` from the inner state. - Inside a coroutine, `raise` inside an `except` after `await` chains normally. `await` doesn't reset the exception-handling state. - Coroutines that never `await` and only `raise` behave like normal functions — no async-specific chain behavior.

What NOT to say: - "`await` swallows exception chains." It doesn't — chains pass through normally. - "Generators reset `__context__`." They don't — chains are on the exception objects, which travel with the exception.

Common follow-up: "What happens to `__context__` when an exception travels through multiple `awaits`?" — Stays intact. Each exception object carries its chain; the `await` mechanism doesn't strip or modify it.

```
import asyncio

class APIError(Exception): pass
class TimeoutError_API(APIError): pass

async def fetch_one(url):
    try:
        return await http_get(url, timeout=5)
    except asyncio.TimeoutError as e:
        raise TimeoutError_API(f"timed out: {url}") from e

async def fetch_many():
    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(fetch_one("https://api.razorpay.com/a"))
            tg.create_task(fetch_one("https://api.razorpay.com/b"))
    except* TimeoutError_API as eg:
        for err in eg.exceptions:
            # err.__cause__ is the original asyncio.TimeoutError
            log.warning("api timeout: %s (caused by %r)", err, err.__cause__)
```

Q23 · Wrapping Library Exceptions At Your Module Boundary

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, PhonePe, Postman · Topic: Exception Chaining

The question (interviewer's verbatim phrasing):

You're writing a payments module that internally uses `requests` and `psycopg`. Should you let those libraries' exceptions propagate to your callers, or translate them?

The 30-second answer: Translate them at the boundary. Callers of your `payments` module shouldn't have to import `requests.exceptions` or `psycopg.errors` to handle failures. Catch the library's exceptions in your module, translate to your own `PaymentError` family, and use `raise NewError from e` to preserve the original for debugging. Keeps your API self-contained and lets you swap libraries later without breaking callers.

Step-by-step thought process: 1. "The principle — your module's exception classes are part of its public API. Callers should never need to know what libraries you use internally to catch failures." 2. "At each catch point, translate. `except requests.ConnectionError as e: raise PaymentTimedOut(...) from e`. The caller now writes `except PaymentTimedOut` without importing requests." 3. "The `from e` part matters. It preserves the original for debugging — developers see the underlying `ConnectionError` in tracebacks and can reason about network conditions. Lose this and debugging becomes detective work." 4. "Library swap becomes trivial. If you switch from `requests` to `httpx`, you only need to catch the new library's exceptions and translate to the same

domain exceptions. Callers see no change." 5. "There's a cost — every library call needs a translation block. Mitigate with helper functions or a context manager that catches and re-raises in one place: `with translate_errors():`. Pattern works well for thin modules around messy libraries."

Edge cases to mention: - Don't translate at every level — translate at the module boundary that's the public API. Internal helpers can let library exceptions flow up to the boundary's catch. - Some library exceptions are already useful domain types — `FileNotFoundError` from `pathlib` is fine to let through if your API is also about files. Translation isn't always necessary. - Make sure your translations don't lose information. Always `from e`, and consider attaching key context (URL, request ID) via `add_note` or a custom attribute.

What NOT to say: - "Let library exceptions propagate to keep things simple." Couples your callers to your dependencies. Refactoring becomes painful. - "Translate and lose the original — callers don't need it." Lose-original makes production debugging fight you. Always `from e`.

Common follow-up: "How do you avoid wrapping every single library call?" — A context manager that catches and translates in one place, plus a decorator for the common cases.

```
import requests
import psycopg
from contextlib import contextmanager

class PaymentError(Exception): pass
class PaymentTimeout(PaymentError): pass
class PaymentDB(PaymentError): pass

@contextmanager
def translate_errors(context: str):
    try:
        yield
    except requests.Timeout as e:
        raise PaymentTimeout(f"{context}: gateway timeout") from e
    except requests.ConnectionError as e:
        raise PaymentTimeout(f"{context}: gateway unreachable") from e
    except psycopg.errors.UniqueViolation as e:
        raise PaymentError(f"{context}: duplicate idempotency key") from e
    except psycopg.OperationalError as e:
        raise PaymentDB(f"{context}: database unreachable") from e

def charge(order):
    with translate_errors(f"charge order={order.id}"):
        response = requests.post(GATEWAY_URL, json=order.dict(), timeout=5)
        with psycopg.connect(DSN) as conn:
            conn.execute("INSERT INTO charges ...", (order.id, response.json()["id"]))
```

Q24 · The from Keyword Is Not Just For Tracebacks

Tags: Difficulty: Hard · Asked at: Razorpay, Microsoft India, Goldman · Topic: Exception Chaining

The question (interviewer's verbatim phrasing):

Apart from the traceback message, does `from` change anything programmatically?

The 30-second answer: Yes. `raise X from Y` sets `X.__cause__ = Y` and `X.__suppress_context__ = True` as real attributes. Code that inspects exceptions can read these. Structured logging, observability tools, and incident-management dashboards all use `__cause__` to identify root cause classes. So `from` is both a presentation directive and a data attribute on the exception object that downstream code consumes.

Step-by-step thought process: 1. "`__cause__` is a normal attribute. After `raise NewError from OldError`, `caught.__cause__` is the `OldError` object, and you can read its class, args, and traceback." 2. "Structured logging libraries (structlog, the structured-logging integrations of Sentry / Datadog / New Relic) walk `__cause__` to extract a chain summary. They use it to populate the 'root cause' field in incident dashboards." 3. "This is also why `from None` is more than cosmetic. Code that walks the chain via `e.__cause__` sees nothing — same as if there was no chain. Useful when you want to make the boundary clean both for humans and for observability." 4. "`__suppress_context__ = True` is also a real attribute. Custom traceback formatters or incident parsers can read it to decide whether to display context." 5. "You can construct chains manually for testing — `e = NewError(...); e.__cause__ = OldError(...)`. Useful when reproducing complex production scenarios in a test."

Edge cases to mention: - `__cause__` and `__context__` survive pickling, so chained exceptions can cross process boundaries (e.g., multiprocessing) with the chain intact. - Some observability tools (older Sentry SDK versions) preferred `__context__` over `__cause__`. Modern versions support both; check your SDK docs. - Setting `__cause__` manually after the fact also sets `__suppress_context__`? — no, that coordination only happens via the `from` syntax. Setting `__cause__` alone doesn't suppress display of `__context__`.

What NOT to say: - "`from` is just a traceback printer directive." It's also a data attribute consumed by observability tooling. - "Programmatic chain inspection is rare." It's standard in production logging stacks — almost every error tracker uses it.

Common follow-up: "How would you tag exceptions in Sentry with their root cause class?" — Walk `e.__cause__` until `None` and use the class name of the last one as a tag. Most SDKs do this for you, but the algorithm is the same.


```
def root_cause(e: BaseException) -> BaseException:
    """Walk the deliberate chain to its root."""
    while e.__cause__ is not None:
        e = e.__cause__
    return e

# Practical use – tagging incidents by root cause class
try:
    charge_card(order)
except PaymentError as e:
    root = root_cause(e)
    metrics.increment(
        "payment.failure",
        tags={
            "domain_class": type(e).__name__,
            "root_class": type(root).__name__,
        },
    )
    raise
```

Q25 · Chained Exceptions Across Subprocess And Multiprocessing Boundaries

Tags: Difficulty: Hard · Asked at: Razorpay, Microsoft India, Goldman · Topic: Exception Chaining

The question (interviewer's verbatim phrasing):

If a worker process raises an exception with a chain, does the chain survive when the parent process retrieves it via a queue or `concurrent.futures` ?

The 30-second answer: Yes — exception chains survive pickling in `multiprocessing` and `concurrent.futures.ProcessPoolExecutor`. The exception object, its `__cause__`, `__context__`, and `__notes__` are all pickled and unpickled. The `__traceback__` does not pickle cleanly — instead it's serialized as a formatted string in the unpickled exception's `__traceback__`-related metadata. So you see the chain and a stringified traceback per frame.

Step-by-step thought process: 1. "`multiprocessing` and `ProcessPoolExecutor` use pickling to ship objects across process boundaries. Exceptions are picklable if their args are picklable. The chain attributes (`__cause__`, `__context__`, `__suppress_context__`, `__notes__`) are all attributes on the exception object and pickle along with it." 2. "`__traceback__` is special — traceback objects don't pickle. Python's multiprocessing handles this by stringifying the traceback before pickling, and the unpickled exception has the formatted string accessible via a fake `__traceback__`-like attribute or via the worker's traceback module." 3.

"`concurrent.futures.ProcessPoolExecutor` does the same dance — the `Future.exception()` method returns the unpickled exception with chain intact and a

`tb_frame` -less traceback that prints correctly even though it can't be walked." 4. "In practice — for debugging, you see the worker's traceback in the parent log when you call `future.result()`, which re-raises. The chain prints. The frames are from the worker process, which is exactly what you want." 5. "Caveats — your custom exception class must be picklable. If `__init__` requires positional args you don't pass to `super().__init__`, unpickling fails. Make sure `super().__init__(*reconstruct_args)` is called consistently."

Edge cases to mention: - `multiprocessing.Queue` -based passing has the same pickling rules. If you put an exception on a queue, you get it back with chain intact. - A custom exception that holds an un-picklable object (open file, database connection, threading lock) will fail to ship. Carry IDs, not handles. - For long chains, the pickled representation grows linearly — six layers means six exception objects pickled. Usually negligible.

What NOT to say: - "Multiprocessing loses exception chains." Not true — chains survive intact via pickle. - "Just log inside the worker and don't propagate." Loses the parent's chance to handle. Use queues and `future.result()` to get the exception back.

Common follow-up: "What if my custom exception isn't picklable?" — Make it picklable. Set `__reduce__` if you need custom serialization, or simplify `__init__` to only take picklable args. Or convert to a serializable structure in the worker.

```
from concurrent.futures import ProcessPoolExecutor

class WorkerError(Exception):
    def __init__(self, worker_id: int, reason: str):
        super().__init__(f"worker {worker_id}: {reason}")
        self.worker_id = worker_id
        self.reason = reason

def task(worker_id):
    try:
        risky_thing()
    except ValueError as e:
        raise WorkerError(worker_id, "validation failed") from e

with ProcessPoolExecutor() as pool:
    futures = [pool.submit(task, i) for i in range(4)]
    for f in futures:
        try:
            f.result()
        except WorkerError as e:
            # e is the unpickled exception, with __cause__ pointing at
            # the original ValueError from the worker process
            print(e.worker_id, e.__cause__)
```

Section D — Exception Groups (3.11+)

Q26 · ExceptionGroup And BaseExceptionGroup — What They Are

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Exception Groups

The question (interviewer's verbatim phrasing):

What's `ExceptionGroup`, what's `BaseExceptionGroup`, and when did they show up?

The 30-second answer: PEP 654, Python 3.11+. `BaseExceptionGroup` and `ExceptionGroup` are exception classes that hold multiple exceptions inside one. `BaseExceptionGroup` can hold any `BaseException` subclass; `ExceptionGroup` is restricted to `Exception` subclasses (so it cannot contain `KeyboardInterrupt`). They exist for cases where multiple operations fail concurrently and you want to report all the failures — `asyncio.TaskGroup`, parallel HTTP fan-out, batch validation — instead of just the first.

Step-by-step thought process: 1. "Before 3.11, parallel failures were lossy. Run ten tasks, three fail — `asyncio.gather` would surface one exception and silently lose the others. `ExceptionGroup` fixes that." 2. " `ExceptionGroup(message, exceptions)` takes a message and a non-empty list of exceptions. The whole group is itself an exception — raisable, catchable, chainable. Iterate `eg.exceptions` to see each contained exception." 3. "The class split mirrors `BaseException` / `Exception`. `ExceptionGroup` rejects any non-Exception subclass at construction. `BaseExceptionGroup` accepts both. If you might mix `KeyboardInterrupt` with regular errors, you need `BaseExceptionGroup` — but most user code wants `ExceptionGroup`." 4. "Python auto-narrows the type at construction. `BaseExceptionGroup('errs', [ValueError(), TypeError()])` actually returns an `ExceptionGroup` because all members are Exceptions. Convenience, but worth knowing for type-checking." 5. "Catching uses the new `except*` syntax (one star). `except* ValueError` catches all `ValueError` leaves of the group; remaining leaves form a new group that propagates. Plain `except` matches against the group type itself, not against its contents — so `except ExceptionGroup` catches the wrapper but doesn't unpack."

Edge cases to mention: - The exceptions list must be non-empty. `ExceptionGroup('msg', [])` raises `ValueError`. Either you have failures (group them) or you don't (raise nothing). - `ExceptionGroup` is iterable indirectly via `.exceptions`; the group's own `.exceptions` is a tuple, not a list. - Groups can nest — an `ExceptionGroup` can contain another `ExceptionGroup` as one of its exceptions. Nested groups are how `except*` produces leftover groups.

What NOT to say: - "ExceptionGroup is just a list of exceptions." It's a real exception class with `raise` / `except` semantics, nesting, and `except*` integration. - "I just use `asyncio.gather(return_exceptions=True)`." Pre-3.11 workaround. `TaskGroup` + `ExceptionGroup` is the modern path interviewers want to hear about.

Common follow-up: "What's the type of the exception bound by `except* ValueError as eg`?"

— `ExceptionGroup[ValueError]` — always a group, even if only one exception matched. You always iterate `eg.exceptions` to see the actual `ValueErrors`.

```
# Constructing manually
errs = [
    ValueError("bad pincode"),
    KeyError("missing email"),
    ConnectionError("gateway down"),
]
raise ExceptionGroup("validation + network failed", errs)

# Catching with except* — one syntax style per try
try:
    raise ExceptionGroup("multi", [ValueError("a"), TypeError("b")])
except* ValueError as eg:
    # eg is ExceptionGroup[ValueError] containing just ValueError('a')
    # TypeError('b') is in a separate residual group that propagates
    for v in eg.exceptions:
        log.warning("value: %s", v)
except* TypeError as eg:
    for t in eg.exceptions:
        log.warning("type: %s", t)
```

Q27 · The `except*` Syntax — How It Differs From `except`

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Microsoft India, Goldman · Topic: Exception Groups

The question (interviewer's verbatim phrasing):

Walk me through `except*`. How is it different from `except`, and can I mix them in one `try`?

The 30-second answer: `except*` matches against the leaves of an `ExceptionGroup`, not against the group itself. Each `except*` clause catches matching leaves into a new group; remaining leaves form a residual group that propagates after the try. All `except*` clauses run (not just the first matching one). You cannot mix `except` and `except*` in the same try — one syntax style per try block. Also, the handler always receives a group, even for a single matching exception.

Step-by-step thought process: 1. "`except* T` matches if any leaf of the raised group is a `T`. Python splits the group, gives you `ExceptionGroup[T]` containing the matching leaves, and continues to the next `except*` clause with the residual." 2. "This is different from plain `except`, which goes top-to-bottom and runs only the first matching clause. With `except*`, every clause runs against the residual

group. So you can write `except* ValueError` and `except* TypeError` in the same try and both run if both kinds were raised." 3. "You always get a group, never a single exception. `except* ValueError as eg` binds `eg` to an `ExceptionGroup[ValueError]`, even if only one `ValueError` was in the group. Iterate `eg.exceptions` to get the individual ones." 4. "Mixing `except` and `except*` is a syntax error. Choose one style per try. If you're catching from `asyncio.TaskGroup` or parallel code, use `except*`. If you're catching a single exception, use `except`." 5. "If the residual group is empty after all `except*` clauses ran, nothing further propagates. If non-empty, it propagates as a group — possibly a nested one — to the next handler up."

Edge cases to mention: - A bare exception (not a group) raised inside a try with `except*` clauses — Python wraps it in a group automatically so the `except*` matching still works. - `except* (A, B)` with a tuple matches leaves of either type. Both go into the same matched group. - The order of `except*` clauses doesn't affect matching (unlike `except`), since each clause runs against the residual. But it affects which handler runs first.

What NOT to say: - "`except*` is like `except` with extra syntax." It has different semantics — multiple clauses run, residuals propagate, bound variable is always a group. - "I can mix `except` and `except*`." Wrong — syntax error. Pick one style per try block.

Common follow-up: "Show me what `except* ValueError` gives me when the group has one `ValueError` and one `TypeError`." — `eg` is `ExceptionGroup[ValueError]` containing just that one `ValueError`. The `TypeError` stays in a separate residual group that propagates after the try (unless a later `except* TypeError` catches it).

```
def run_validators():
    raise ExceptionGroup(
        "validation",
        [
            ValueError("pincode"),
            ValueError("phone"),
            TypeError("amount"),
        ],
    )

try:
    run_validators()
except* ValueError as eg:
    # eg = ExceptionGroup[ValueError] containing two ValueErrors
    print("values:", [str(v) for v in eg.exceptions])
except* TypeError as eg:
    # eg = ExceptionGroup[TypeError] containing one TypeError
    print("types:", [str(t) for t in eg.exceptions])
# Output:
# values: ['pincode', 'phone']
# types: ['amount']
```

Q28 · `asyncio.TaskGroup` — Why It Raises `ExceptionGroup`

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Microsoft India, Goldman · Topic: Exception Groups

The question (interviewer's verbatim phrasing):

Why does `asyncio.TaskGroup` raise `ExceptionGroup` and not just the first exception?
How is this different from `asyncio.gather` ?

The 30-second answer: `TaskGroup` (3.11+) implements structured concurrency — when any task fails, the remaining tasks are cancelled and an `ExceptionGroup` containing all the failures is raised on exit. This preserves every failure for inspection and recovery. `asyncio.gather` (the older API) raises the first exception by default (losing others) or returns exceptions as values with `return_exceptions=True`. `TaskGroup` is the structured-concurrency model: all child tasks complete before the with-block exits, all failures are reported.

Step-by-step thought process: 1. "`TaskGroup` is structured concurrency. The `async with` block waits for every task spawned inside it. If any task raises, the group cancels the remaining tasks and collects every failure into an `ExceptionGroup` raised at the end of the `with`." 2. "`gather(*tasks)` is older. Default behavior — first failure cancels nothing, but raises that one exception and silently abandons others. `gather(return_exceptions=True)` returns a list where exceptions are values, not raised — caller has to sort them out. Both are lossy or awkward compared to `TaskGroup`." 3. "`except*` is the matched-pair for `TaskGroup`. You write `async with asyncio.TaskGroup() as tg:` and `except* SpecificError:` outside. The catching syntax is designed for the exact shape `TaskGroup` produces." 4. "Cancellation interaction — when one task raises, `TaskGroup` sends `CancelledError` to the others. Those cancellations may also surface in the group. So an `ExceptionGroup` from `TaskGroup` often contains one 'real' error plus several `CancelledErrors`. Filter with `except* CancelledError: pass` if you only want the real ones." 5. "For pre-3.11 code that wants similar semantics, the workaround was `asyncio.gather(*tasks, return_exceptions=True)` plus a manual loop to collect non-None exceptions and raise something. `TaskGroup` is cleaner and standard."

Edge cases to mention: - A task spawned inside the `TaskGroup` that itself uses `asyncio.gather` produces nested `ExceptionGroups` if both layers fail — flatten with `.split()` or iterate carefully. - `KeyboardInterrupt` during `TaskGroup` execution wraps into a `BaseExceptionGroup` (not `ExceptionGroup`) — handle with `except* KeyboardInterrupt:` carefully or let it propagate. - `TaskGroup`'s `__aexit__` waits for all tasks. If you have a long-running task you don't want to wait on, don't put it in a `TaskGroup` — use a detached task with `asyncio.create_task` outside the group.

What NOT to say: - "`TaskGroup` is just nicer-syntax `gather`." It's structured concurrency — cancellation semantics, completeness guarantees, and `ExceptionGroup` output are all different. - "I'd use `gather` because it's simpler." Pre-3.11 mindset. In modern code, `TaskGroup` is the default for parallel awaitable work.

Common follow-up: *"What happens if every task in a TaskGroup succeeds?"* — Nothing raises. The `async with` exits normally and execution continues after.

```
import asyncio

class FetchFailed(Exception): pass

async def fetch(url: str) -> dict:
    if url.endswith("/fail"):
        raise FetchFailed(url)
    return {"url": url, "data": "..."}

async def main():
    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(fetch("https://razorpay.com/api/a"))
            tg.create_task(fetch("https://razorpay.com/api/b/fail"))
            tg.create_task(fetch("https://razorpay.com/api/c/fail"))
    except* FetchFailed as eg:
        for err in eg.exceptions:
            log.warning("fetch failed: %s", err)
    except* asyncio.CancelledError:
        pass # cancellations from the failure are noise; ignore

asyncio.run(main())
```

Q29 · Iterating And Searching Through An ExceptionGroup

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Microsoft India · Topic: Exception Groups

The question (interviewer's verbatim phrasing):

I caught an `ExceptionGroup` with deep nesting. How do I get every leaf exception in a flat list?

The 30-second answer: Walk recursively — for each item in `eg.exceptions`, if it's itself a group, recurse; otherwise it's a leaf. Or use the `.subgroup(predicate)` / `.split(predicate)` methods, which return a new group of just the matching exceptions. For a flat list, a small generator works in three lines. For complex filtering, `.split` is cleaner because it preserves the group structure on both sides.

Step-by-step thought process: 1. "Recursive iteration — for each item in `eg.exceptions`, check if it's a group. If yes, recurse. If no, yield it. A short generator does it cleanly." 2.

"`.subgroup(predicate)` returns a new `ExceptionGroup` containing only the leaves where `predicate(leaf)` is True. Useful for 'all the `ValueErrors`' or 'all errors with a specific status code'." 3.

"`.split(predicate)` returns a tuple `(matched_group, rest_group)`. Either side can be `None` if empty. Use when you want both matched and unmatched separately." 4. "Predicate can be a class — `.split(ValueError)` matches any `ValueError` leaf. Or a callable — `.split(lambda e: getattr(e, 'status', 0) >= 500)` for richer logic." 5. "For analysis — count by class, group by attribute, or build summary metrics — flatten first, then use normal Python. The recursive walk is reusable enough to live in a small util module."

Edge cases to mention: - An `ExceptionGroup` with one nested `ExceptionGroup` with one exception is a 3-level nest. `.subgroup(...)` preserves nesting structure; flattening loses it. Pick based on what you need. - Predicate that matches no exception returns an empty result from `.subgroup` — actually `None`, not an empty group. Same for the second value of `.split`. - `BaseExceptionGroup.split` exists too and behaves the same way for `BaseException` subclasses.

What NOT to say: - "Iterate `eg.exceptions` and you're done." Misses nesting — nested groups are exceptions too, not leaves. - "ExceptionGroup is a list." It's a tree. Flat iteration loses structure; intentional walking respects it.

Common follow-up: "How do I check if any leaf is a specific exception type without iterating?" — `eg.subgroup(MyError) is not None` returns `True` iff any leaf matches. Cleanest one-liner.

```
def iter_leaves(eg: BaseExceptionGroup) -> "Iterator[BaseException]":
    for exc in eg.exceptions:
        if isinstance(exc, BaseExceptionGroup):
            yield from iter_leaves(exc)
        else:
            yield exc

# Flat list of every leaf exception
leaves = list(iter_leaves(eg))

# Match by class – returns a new group of just ValueError (None if none)
value_errors = eg.subgroup(ValueError)

# Split – both sides as groups
http_errors, rest = eg.split(lambda e: isinstance(e, HTTPError))

# Predicate examples
server_errors = eg.subgroup(
    lambda e: isinstance(e, HTTPError) and e.status_code >= 500
)
```

Q30 · Constructing And Raising Your Own ExceptionGroup

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman · Topic: Exception Groups

The question (interviewer's verbatim phrasing):

Show me a use case for constructing your own `ExceptionGroup` outside of `asyncio.TaskGroup`.

The 30-second answer: Batch validation. You're checking a payload with N fields; each can fail independently; the user wants to see every error at once, not fix them one at a time. Collect each `ValidationError` into a list, and if the list is non-empty after the loop, `raise ExceptionGroup("validation failed", errors)`. Same pattern works for batch processing — read 100 rows, collect every row-level failure, raise one group at the end.

Step-by-step thought process: 1. "The pattern — fail-collecting loop. For each item, try the operation; on failure, append to a list; at the end, if the list is non-empty, raise an `ExceptionGroup`." 2. "This beats raising on the first failure because users can fix multiple problems in one round-trip. A form with five errors should report all five, not just the first." 3. "Construct with `ExceptionGroup(message, errors_list)`. Message is a human-readable summary; the list is the actual exceptions. The list must be non-empty — guard the raise with `if errors:`." 4. "Type-narrowing is automatic. If all errors are `ValueError`, the constructed object is `ExceptionGroup[ValueError]`. Useful for static checking — callers know what to expect." 5. "Catchers use `except* ValidationError` (or whatever your specific class is). The group's flat structure means callers get every error in one place."

Edge cases to mention: - Don't construct a group with one exception just to satisfy the API — raise the single exception directly. `ExceptionGroup` is for genuine multi-failure scenarios. - The exceptions in the list must all be Exception subclasses for `ExceptionGroup`; use `BaseExceptionGroup` for mixed. - You can attach `__cause__` to the group itself: `raise ExceptionGroup(...) from original`. Cleaner than chaining each leaf.

What NOT to say: - "ExceptionGroup is only for asyncio." Plenty of non-async use cases — batch validation, batch processing, parallel work without asyncio. - "Why not just return a list of errors?" Because callers wouldn't know to check; raising forces handling at the boundary.

Common follow-up: "How does the caller know which field each error refers to?" — Custom exception class with a `.field` attribute. The group contains those custom exceptions; callers iterate and read `.field`.


```

class ValidationError(Exception):
    def __init__(self, field: str, message: str):
        super().__init__(f"{field}: {message}")
        self.field = field

def validate(payload: dict) -> None:
    errors: list[ValidationError] = []

    if "pincode" not in payload:
        errors.append(ValidationError("pincode", "is required"))
    elif not str(payload["pincode"]).isdigit() or len(str(payload["pincode"])) != 6:
        errors.append(ValidationError("pincode", "must be 6 digits"))

    if "phone" not in payload:
        errors.append(ValidationError("phone", "is required"))
    elif len(str(payload["phone"])) != 10:
        errors.append(ValidationError("phone", "must be 10 digits"))

    if not isinstance(payload.get("amount"), (int, float)) or payload.get("amount", 0) <= 0:
        errors.append(ValidationError("amount", "must be positive number"))

    if errors:
        raise ExceptionGroup("validation failed", errors)

# Caller
try:
    validate({"pincode": "abc", "phone": "12345"})
except* ValidationError as eg:
    for err in eg.exceptions:
        print(err.field, err.args[0])

# Output:
# pincode pincode: must be 6 digits
# phone   phone: must be 10 digits
# amount  amount: must be positive number

```

Section E — Context Managers + Exception Handling

Q31 · The Context Manager Protocol — enter And exit

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Postman, TCS · Topic: Context Managers

The question (interviewer's verbatim phrasing):

What's the context manager protocol? What does `__exit__` actually receive?

The 30-second answer: A context manager is any object with `__enter__` and `__exit__` .

`__enter__` runs at the start of the `with` block; its return value is bound by `as` .

`__exit__(exc_type, exc_val, exc_tb)` runs at the end, receiving information about whether an exception was raised in the block — `(None, None, None)` for clean exit, or the exception's class, instance, and traceback if it raised. Returning a truthy value from `__exit__` suppresses the exception; returning falsy lets it propagate.

Step-by-step thought process: 1. "`__enter__(self)` is called at the start of `with` . Whatever it returns is bound by `as name:` . If you don't write `as` , the return value is discarded but `__enter__` still runs." 2. "`__exit__(self, exc_type, exc_val, exc_tb)` is called when leaving the `with` block — normally, via exception, or via `return / break` . The three args are `None, None, None` for clean exit; otherwise they're the exception class, instance, and traceback." 3. "`__exit__` return value matters. Return a truthy value to suppress the exception (Python treats the block as having completed normally). Return falsy (or no return, which is `None`) to let the exception propagate. Suppressing is rare and should be deliberate." 4. "The classic file example — `__enter__` opens the file and returns it; `__exit__` closes it regardless of whether an exception was raised. This is why `with open(p) as f:` is the safe way to read files — close happens even on error." 5. "For thread-safety and locks — `__enter__` acquires the lock, returns the lock object; `__exit__` releases it. Even if the `with` body raises, the lock releases. Critical for not deadlocking on partial failures."

Edge cases to mention: - `__enter__` can raise — in which case `__exit__` is not called, and the `with` block doesn't execute. This is by design: if setup fails, there's no resource to clean up. - If `__exit__` itself raises, that new exception replaces the original; the original becomes `__context__` . Same rules as raising inside `finally` . - Modern Python: `@contextlib.contextmanager` lets you write context managers as generators, much shorter for one-off cases.

What NOT to say: - "`__exit__` is just like `finally` ." Similar but more powerful — it gets the exception info as args and can suppress. - "I always return `True` from `__exit__` to be safe." That suppresses every exception, which is the opposite of safe — bugs get silently swallowed.

Common follow-up: "What's the difference between `__exit__` returning `False` and not returning anything?" — None of the difference at runtime. `return None` and `return False` are both falsy; both let the exception propagate. Explicit `return False` is sometimes preferred for clarity.

```
class Timer:
    """Time a block and log the duration, even on failure."""

    def __init__(self, label: str):
        self.label = label

    def __enter__(self):
        self.start = time.monotonic()
        return self    # `as t:` would bind to this instance

    def __exit__(self, exc_type, exc_val, exc_tb):
        elapsed = time.monotonic() - self.start
        if exc_type is None:
            log.info("%s ok in %.2fs", self.label, elapsed)
        else:
            log.warning("%s failed in %.2fs: %s", self.label, elapsed, exc_val)
        # Returning None (falsy) – exceptions propagate normally
        return None

with Timer("charge_card"):
    charge_card(order)    # success or failure, duration is logged
```

Q32 · Suppressing An Exception From exit

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Context Managers

The question (interviewer's verbatim phrasing):

Walk me through how `__exit__` can suppress an exception. When would you do that?

The 30-second answer: Return a truthy value from `__exit__` and Python treats the `with` block as if no exception was raised — control continues after the `with` normally. Use this only when the context manager's purpose is to swallow a specific exception type — for example, a retry wrapper that catches the inner failure and decides not to surface it, or a "silence this specific error" helper like `contextlib.suppress`. Don't return True blindly; check `exc_type` and only suppress what you mean to.

Step-by-step thought process: 1. "`__exit__` returning truthy is Python's signal that 'this exception is handled, continue normally'. The exception object is discarded as far as the propagation is concerned, though it's still accessible inside `__exit__` while it's running." 2. "The pattern — check `exc_type` and only suppress what you intend. `if exc_type is None: return False` then `if isinstance(exc_type, SomeError): handle_it(); return True; else: return False`." 3. "`contextlib.suppress(SomeError)` is the standard library version. `with suppress(KeyError):` lets `KeyError` vanish quietly and lets everything else propagate. Very useful for

one-line cleanup." 4. "Don't return True unconditionally. That swallows every exception including programmer bugs (TypeError, NameError). Future-you debugging at 2 a.m. will not be amused." 5. "Common legitimate uses — retry helpers that signal 'retry exhausted, give up' by returning normally instead of raising; test-doubles that need to verify an exception was raised; instrumentation that adds tags before letting the exception propagate (which means returning falsy, but doing work first)."

Edge cases to mention: - Suppressing an exception inside `__exit__` doesn't reset the local state of the `with` block — execution continues after the `with`, not back into the block. - If `__exit__` raises a new exception (whether it suppresses the original or not), the new one takes over and the original becomes `__context__`. - `contextlib.suppress(A, B, C)` accepts a tuple of types — useful for catching several types in one line.

What NOT to say: - "Always return True from `__exit__` for safety." Backwards — that hides bugs. - "Suppressing exceptions is bad." Sometimes it's exactly right (e.g., `contextlib.suppress`). The question is whether the suppression is deliberate.

Common follow-up: "What's the difference between `contextlib.suppress(KeyError)` and `try: ... except KeyError: pass`?" — Semantically equivalent for one block. `suppress` reads better when the goal is "ignore this specific error" and the suppression is the whole point.

```
from contextlib import contextmanager, suppress

# Standard library — clean one-liner
with suppress(FileNotFoundError):
    os.remove("/tmp/maybe-not-there.lock")

# Custom — explicit class
class IgnoreSpecific:
    """A demo context manager that suppresses a specific exception type."""

    def __init__(self, *types):
        self.types = types

    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is None:
            return False
        if isinstance(exc_type, self.types):
            log.info("suppressed %s", exc_type.__name__)
            return True      # tell Python to swallow it
        return False        # let everything else propagate

with IgnoreSpecific(KeyError, IndexError):
    d = {}
    print(d["missing"])    # KeyError — suppressed
```

Q33 · contextlib.contextmanager — Writing One As A Generator

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Postman, Freshworks · Topic: Context Managers

The question (interviewer's verbatim phrasing):

Show me `@contextmanager` . How does it handle exceptions raised inside the with block?

The 30-second answer: `@contextlib.contextmanager` lets you write a context manager as a generator function with one `yield` . Code before `yield` runs as `__enter__` ; the yielded value is bound by `as` ; code after `yield` runs as `__exit__` . To handle an exception raised inside the with block, wrap the `yield` in `try/except` — the exception is re-raised at the `yield` line. To suppress, catch and swallow inside the generator (the `with` block won't see it after the catch). It's the cleaner form for one-off context managers.

Step-by-step thought process: 1. "The decorator turns a generator into a context manager. The function must yield exactly once. Before yield = enter; yield expression = the value bound by `as` ; after yield = exit." 2. "Exception handling — wrap the `yield` in a try block. If the with block raises, Python throws that exception into the generator at the `yield` line. Your `except` catches it; control reaches the cleanup code; the generator either re-raises (by letting the except finish) or suppresses (by handling and finishing normally)." 3. "To suppress, catch and don't re-raise. To not suppress, let the exception propagate out of the generator. To translate, catch and `raise NewError from e` . Same patterns as ordinary try/except." 4. "The decorator is for short, one-off context managers — usually 5-15 lines. For complex stateful ones, write a class with explicit `__enter__` / `__exit__` . The decorator hides the protocol slightly; a class is more explicit." 5. "Common uses — temporary directory cleanup, transactional wrappers around a database session, instrumentation (start/end metric), feature-flag overrides for tests. All cases where the resource has obvious setup and teardown around a single yield point."

Edge cases to mention: - A generator that doesn't yield at all, or yields twice, raises `RuntimeError` when used as a context manager. The decorator enforces one yield. - If the cleanup code after `yield` raises, that exception replaces any from the body, with the body's as `__context__` . - `@contextmanager` async equivalent is `@asynccontextmanager` — same pattern with `async def` and `yield` .

What NOT to say: - "`@contextmanager` is always cleaner than a class." For simple cases, yes; for complex stateful resources, a class is clearer. - "I can't suppress exceptions with `@contextmanager` ." You can — catch in the generator and don't re-raise.

Common follow-up: "What happens if the generator forgets to put cleanup in a `finally` ?" — If the body raises and your generator doesn't re-raise from the except, cleanup that's after the except runs only on the exception path. Put cleanup in `finally` for guaranteed execution.

```

from contextlib import contextmanager

@contextmanager
def db_transaction(session):
    """Commit on success, rollback on exception, always close."""
    session.begin()
    try:
        yield session
    except Exception:
        session.rollback()
        raise          # let it propagate after cleanup
    else:
        session.commit()
    finally:
        session.close()

# Usage
with db_transaction(session) as s:
    s.execute("INSERT INTO orders ...")
    s.execute("INSERT INTO line_items ...")
# Either both inserts commit, or both roll back. Session always closes.

# Suppressing example
@contextmanager
def ignore(*types):
    try:
        yield
    except types:
        pass          # suppressed by not re-raising

with ignore(FileNotFoundError):
    os.remove("/tmp/maybe-not.lock")    # no error if missing

```

Q34 · contextlib.suppress — The Standard Library Helper

Tags: Difficulty: Easy · Asked at: Razorpay, PhonePe, Postman, Freshworks · Topic: Context Managers

The question (interviewer's verbatim phrasing):

What is `contextlib.suppress` and when do you use it instead of a try/except?

The 30-second answer: `contextlib.suppress(*exceptions)` is a context manager that catches and silently swallows the specified exception types from inside the `with` block. `with suppress(FileNotFoundError): os.remove(p)` is the same as `try: os.remove(p) except FileNotFoundError: pass` but reads cleaner for one-line ops.

Use it when suppression is the whole point and the body is short. For anything more than a couple of lines, write the try/except explicitly.

Step-by-step thought process: 1. " `from contextlib import suppress` . Pass one or more exception classes. Anything raised in the with block that matches gets swallowed; anything else propagates." 2. "The win is readability. `with suppress(KeyError): del d['k']` is two clear words versus `try/except/pass` which is five lines. For idempotent cleanup operations, the explicitness of suppress is nice." 3. "Anti-pattern — using suppress to hide bugs. `with suppress(Exception): risky_thing()` is just `try/except: pass` with extra syntax. The whole point is intentional, narrow suppression." 4. "It's a real context manager — supports `as` (but `as` binds nothing useful since suppress doesn't return a value)." 5. "Performance-wise, the overhead is the same as a try/except. Use it for clarity, not speed."

Edge cases to mention: - `suppress(KeyError, IndexError, AttributeError)` accepts multiple classes — useful for one-line cleanup chains. - It only suppresses; it can't translate. For translation use a custom context manager or a try/except. - Suppressing inside a loop body skips just that iteration's failure; doesn't break the loop.

What NOT to say: - " `suppress(Exception)` is a clean alternative to bare except." Not clean — same problem of hiding bugs. Use narrow types. - "It's just a shortcut, no behavioral difference." That's right, but the readability difference matters for code review.

Common follow-up: "What about suppressing in a loop that processes many items?" — Same syntax; `with suppress(SomeError):` around just the failure-prone op inside the loop body. Don't put the loop itself inside the with — that suppresses the wrong scope.

```
from contextlib import suppress
import os

# Idempotent cleanup — remove if exists, no error if not
with suppress(FileNotFoundError):
    os.remove("/tmp/lockfile")

# Multiple types in one call
with suppress(KeyError, AttributeError):
    value = config.cache.get("k")

# Loop pattern — suppress per-item, not per-loop
for path in paths:
    with suppress(FileNotFoundError):
        os.remove(path)

# Each missing file is silently skipped; other errors still propagate

# Bad — too wide, hides bugs
with suppress(Exception):      # also catches NameError, TypeError, etc.
    risky_thing()
```

Q35 · contextlib.ExitStack — Dynamic Context Managers

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Microsoft India, Goldman · Topic: Context Managers

The question (interviewer's verbatim phrasing):

What's `ExitStack` and when do you use it?

The 30-second answer: `contextlib.ExitStack` lets you open an unknown number of context managers at runtime — when you don't know how many until you start iterating. Push each context manager onto the stack with `stack.enter_context(cm)` ; when the `with ExitStack()` block exits, every pushed manager is exited in reverse order. Use it for opening N files based on input, acquiring N locks, or building a transaction across an arbitrary number of resources.

Step-by-step thought process: 1. "You can't write `with a, b, c, d:` if you don't know how many at compile time. `ExitStack` solves that — open the stack, push each context manager as you discover it, the stack guarantees clean exit on all of them." 2. "`stack.enter_context(cm)` calls `cm.__enter__()` and registers `cm.__exit__` for later. Return value is whatever `cm.__enter__` returned — same as `as` binding in a normal `with`." 3. "`stack.callback(func, *args)` registers a plain function to call on exit — useful for non-context-manager cleanup (close a socket, delete a temp file)." 4. "On `with ExitStack()` exit, every registered callable runs in reverse-push order, regardless of whether the body raised. Exceptions in cleanup chain via `__context__` ." 5. "Classic pattern — open every file in a directory and process them together. Or acquire N database connections for a multi-shard query. Or build a transaction with conditional steps — push a step into the stack only if it was actually started."

Edge cases to mention: - `stack.pop_all()` transfers the registered cleanups to a new `ExitStack` — useful for handing ownership off to the caller. - `stack.close()` exits everything early (without exiting the `with` block). Lets you finalize cleanup mid-flow. - `ExitStack` has an async cousin: `AsyncExitStack` for `enter_async_context` and async cleanup callbacks.

What NOT to say: - "Just use nested `with` statements." Doesn't work when N is dynamic. `ExitStack` is the answer. - "`ExitStack` is overkill, I'd use a list and a `try/finally`." Reinvents what `ExitStack` does correctly, including reverse-order cleanup and exception chaining.

Common follow-up: "What does the reverse-order cleanup guarantee?" — If you opened A, then B, then C, exit runs C first, then B, then A. Same as nested `with A: with B: with C:` . Mirrors LIFO stack behavior.


```

from contextlib import ExitStack

def read_all_files(paths: list[str]) -> list[str]:
    """Open N files, read each, close all even on failure."""
    contents = []
    with ExitStack() as stack:
        for path in paths:
            f = stack.enter_context(open(path, encoding="utf-8"))
            contents.append(f.read())
        # All files are open here. The stack will close them all on exit.
    return contents

# Conditional steps – only register cleanup for steps that ran
def complex_setup():
    with ExitStack() as stack:
        db = stack.enter_context(open_db())
        if needs_cache:
            cache = stack.enter_context(open_cache())
        if needs_queue:
            queue = stack.enter_context(open_queue())
        # Only the openings that ran are registered for cleanup.
        do_work(db, cache=cache, queue=queue)

```

Section F — Real-World Exception Patterns

Q36 · `logger.exception` vs `logger.error` With `exc_info`

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Postman, Goldman, JP Morgan · Topic: Real-World Patterns

The question (interviewer's verbatim phrasing):

What's `logger.exception` and how is it different from `logger.error(msg, exc_info=True)` ?

The 30-second answer: `logger.exception(msg)` is shorthand for `logger.error(msg, exc_info=True)` — both log at ERROR level and include the current exception's traceback. The shortcut form is the idiomatic call from inside an `except` block. Use either to capture full chain plus traceback; don't try to manually format the exception via `f-string` because you'll lose chain info, traceback frames, and structured fields your log aggregator expects.

Step-by-step thought process: 1. " `logger.exception(msg, *args)` is the standard pattern from inside an `except` block. Logs at ERROR level and grabs the current exception from

`sys.exc_info()`, attaching it as `exc_info` to the log record." 2. "The log handler formats the exception into a full traceback, including chained exceptions (`__cause__` and `__context__`). Aggregators like Sentry, Datadog, and Splunk parse the structured `exc_info` field." 3. "`logger.error(msg, exc_info=True)` is the same thing, just more verbose. Use whichever your team's style prefers; both work. `logger.exception` is shorter and signals 'I'm in an except block' to readers." 4. "`exc_info=ex` (with a specific exception object) lets you log an exception you saved earlier — useful when you logged at one boundary and want to log again at another. Pass the exception, not just `True`." 5. "Anti-pattern — `logger.error(f'failed: {e}')`. Loses traceback. Loses chain. Loses structured fields. Always pass `exc_info` or use `logger.exception`."

Edge cases to mention: - `logger.exception` outside an except block logs `None` for the exception — Python emits a "NoneType: None" line. Always call from inside an except. - Use `logger.warning(..., exc_info=True)` for expected, non-fatal errors that you still want full traceback for — same mechanism, different level. - For one-off debugging, `traceback.print_exc()` writes to stderr without the logging infrastructure. Useful in scripts; not for production.

What NOT to say: - "`logger.exception` and `logger.error` are the same." `exception` is `error + exc_info=True`. Different in what they log. - "I just `print(e)` and move on." Loses traceback, loses chain, loses correlation IDs your logging infra adds.

Common follow-up: "How do you correlate the logged exception with a specific request?" — Use a `LoggerAdapter` or contextvars-based log filter to inject `request_id` into every log record. Then your exception log has the trace plus the `request_id`.

```
import logging
log = logging.getLogger(__name__)

# Idiomatic – captures traceback and chain
try:
    charge_card(order)
except PaymentError:
    log.exception("failed to charge order %s", order.id)
    raise

# Equivalent
try:
    charge_card(order)
except PaymentError:
    log.error("failed to charge order %s", order.id, exc_info=True)
    raise

# Anti-pattern – loses traceback and chain
try:
    charge_card(order)
except PaymentError as e:
    log.error(f"failed: {e}")    # no traceback, no chain, harder to debug

# Logging a specific exception later (outside except)
saved_exc = None
try:
    charge_card(order)
except PaymentError as e:
    saved_exc = e

# ... later, after some processing ...
if saved_exc is not None:
    log.error("delayed report", exc_info=saved_exc)
```

Q37 · Retry With Exponential Backoff — The Standard Pattern

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Postman, Cred, Goldman · Topic: Real-World Patterns

The question (interviewer's verbatim phrasing):

Write a retry-with-backoff loop for a flaky HTTP call. What gets caught, what doesn't, and how do you give up?

The 30-second answer: Loop with a bounded number of attempts. Catch only the transient exception types you genuinely want to retry — `TimeoutError`, `ConnectionError`, specific HTTP 5xx status codes. After each catch, sleep for an exponentially-growing duration with jitter, then loop. After the last

attempt, let the exception propagate or wrap it. Don't catch base `Exception` — that retries programmer bugs and validation errors too, which never recover.

Step-by-step thought process: 1. "Define what's retryable. Network timeouts, connection failures, 5xx responses — yes. `ValueError`, `TypeError`, 4xx responses — no. The catch in the retry loop should match exactly the retryable set." 2. "Bound the attempts. A fixed N (say 3-5) prevents runaway retries. A timeout budget instead of attempts is sometimes nicer — keep retrying until total elapsed exceeds X seconds." 3. "Exponential backoff with jitter — `delay = base * 2 ** attempt + random.uniform(0, jitter)`. Doubles each time; jitter prevents thundering-herd when multiple clients retry simultaneously." 4. "After the last attempt, decide. Either let the underlying exception propagate (caller handles it), or wrap in `RetryExhausted` with `from e` to signal explicitly. The wrapped form is friendlier for callers who want to distinguish 'this failed' from 'this kept failing'." 5. "For production code, use `tenacity` (PyPI). Decorator-style retry with stop conditions, wait strategies, and retry-on conditions. Hand-rolling is fine for learning; tenacity is what teams ship."

Edge cases to mention: - Cap the max delay — exponential growth without cap reaches absurd values. `min(base * 2 ** attempt, max_delay)` keeps it sane. - For idempotency, retry only operations that are safe to repeat. POST to a payments endpoint is not idempotent unless you send an idempotency key. - Async version uses `await asyncio.sleep(delay)` instead of `time.sleep`. Otherwise the loop is identical.

What NOT to say: - "Retry on `Exception` so we recover from everything." Retries bugs forever. Use specific types. - "Constant delay between retries." Causes thundering herd. Use backoff with jitter.

Common follow-up: "What happens to the chain when retries exhaust?" — If you wrap in `RetryExhausted from last_exception`, the chain is preserved and debugging shows both. If you let propagate, only the last attempt's exception is visible.

```
import time
import random
from typing import Callable, TypeVar

T = TypeVar("T")

class RetryExhausted(Exception):
    pass

def retry(
    func: Callable[[], T],
    *,
    retryable: tuple[type[Exception], ...] = (TimeoutError, ConnectionError),
    attempts: int = 4,
    base_delay: float = 0.5,
    max_delay: float = 8.0,
) -> T:
    last: Exception | None = None
    for attempt in range(attempts):
        try:
            return func()
        except retryable as e:
            last = e
            if attempt == attempts - 1:
                break
            delay = min(base_delay * 2 ** attempt, max_delay)
            delay += random.uniform(0, delay / 2)  # jitter
            log.warning("attempt %d failed: %s; retry in %.2fs", attempt + 1, e, delay)
            time.sleep(delay)
    assert last is not None
    raise RetryExhausted(f"gave up after {attempts} attempts") from last

# Usage
result = retry(lambda: http.get("https://api.razorpay.com/orders/123"))
```

Q38 · Structured Exceptions For API Error Responses

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Postman, Freshworks · Topic: Real-World Patterns

The question (interviewer's verbatim phrasing):

Design an exception class that carries enough structured data to be serialized into a JSON error response with status code, error code, and a user-message.

The 30-second answer: Define a base class with `status_code`, `error_code`, `user_message`, and `details` fields. Subclasses specialize for specific cases (`PaymentDeclined`, `ValidationFailed`). The framework's exception handler catches the base, reads the fields, and renders JSON. The pattern is what FastAPI's `HTTPException`, Django REST Framework's `APIException`, and Flask's custom error handlers all use — your job is to define the class and the handler.

Step-by-step thought process: 1. "Base class — `APIError(Exception)` with `__init__(self, status_code, error_code, user_message, details=None)`. Store each as an instance attribute. The status code maps to HTTP; the error code is a stable string for clients to switch on; `user_message` is what the UI shows; `details` is a dict of additional context." 2. "Subclasses specialize. `PaymentDeclined` sets `status_code=402`, `error_code='payment_declined'`. `ValidationFailed` sets `status_code=422`, `error_code='validation_error'`. Each is a one-line class definition that captures the API contract." 3. "Framework-level handler catches the base. In FastAPI — `@app.exception_handler(APIError)`. In Django — middleware or DRF `exception_handler`. In Flask — `@app.errorhandler(APIError)`. The handler reads attributes and builds the JSON response." 4. "Don't leak internal messages. The `user_message` is what the customer sees; the original exception's `str(e)` may have stack-trace-y details. Two fields — user-facing message and developer-facing logging context — keep these separate." 5. "Pair with logging. The handler logs the full chain (`log.exception`) so SRE has the technical detail, and returns the user-safe JSON to the client. Two audiences, two channels."

Edge cases to mention: - Don't put PII (user emails, card numbers) in the error message — gets logged with PII risk. Pass IDs. - Validation errors often need a list of field-specific errors; either use a `details` list or use Pydantic's `ValidationError` which has structured field info. - The `error_code` should be stable across versions — clients code against it. Don't change the string casually.

What NOT to say: - "Just return a string message." Loses the structure clients need to display useful UI. - "Wrap everything in 500." Loses semantic distinction; clients can't tell user errors from server errors.

Common follow-up: "How do you make sure every framework exception gets caught by your handler?" — Register handlers for the base `APIError` plus the framework's exception type at the top of the middleware chain. Anything not subclassing `APIError` falls through to a generic 500 handler that logs and returns a generic message.

```

class APIError(Exception):
    """Base for any error that should become a structured JSON response."""

    status_code: int = 500
    error_code: str = "internal_error"
    user_message: str = "Something went wrong. Please try again."

    def __init__(
        self,
        user_message: str | None = None,
        *,
        details: dict | None = None,
    ):
        super().__init__(user_message or self.user_message)
        if user_message:
            self.user_message = user_message
        self.details = details or {}

    def to_response(self) -> dict:
        return {
            "error": {
                "code": self.error_code,
                "message": self.user_message,
                "details": self.details,
            }
        }

class PaymentDeclined(APIError):
    status_code = 402
    error_code = "payment_declined"
    user_message = "Your bank declined the transaction."

class ValidationFailed(APIError):
    status_code = 422
    error_code = "validation_error"
    user_message = "Some fields are not valid."

# FastAPI-style handler
@app.exception_handler(APIError)
async def handle_api_error(request, exc: APIError):
    log.exception("api error %s on %s", exc.error_code, request.url.path)
    return JSONResponse(exc.to_response(), status_code=exc.status_code)

```

Q39 · Exceptions In del — What Happens

Tags: Difficulty: Hard · Asked at: Razorpay, Microsoft India, Goldman, Cred · Topic: Real-World Patterns

The question (interviewer's verbatim phrasing):

What happens if `__del__` raises an exception? Why does it matter?

The 30-second answer: The exception is captured and reported as an "ignored" message to stderr by the interpreter — it does not propagate to the code that triggered the destruction. This makes `__del__` a dangerous place to do any work that can fail. Production resources should be cleaned up via context managers or explicit `close()` methods, not via `__del__`. If you must use `__del__`, wrap the body in a broad try/except that logs and swallows — never let it raise.

Step-by-step thought process: 1. "`__del__` is called when an object's refcount drops to zero, or at GC time for cycle-broken objects. The caller of `__del__` is the interpreter, not user code — so exceptions can't be caught by user except clauses." 2. "The interpreter catches any exception from `__del__` and writes 'Exception ignored in: ...' to stderr. Your error is logged in an ugly way that may not even appear in your structured logging pipeline." 3. "Worse, you can't predict when `__del__` runs. In CPython it's usually deterministic at refcount-zero, but cycles delay it until GC. So a **del** failure may appear hours after the failure-inducing event, in a different request context." 4. "The standard answer for resource cleanup — context managers (`with` statement) — runs deterministically at the end of the with block. If you need cleanup at object destruction, prefer making the caller use a context manager." 5. "If you absolutely must put cleanup in `__del__` (e.g., for legacy compatibility), wrap the body in `try: ... except Exception: log.warning(..., exc_info=True)`. Never raise out."

Edge cases to mention: - During interpreter shutdown, `__del__` may run when modules are already cleared. Accessing `os` or `logging` may raise `AttributeError` because the module is gone. Defensive code is required. - A `__del__` that tries to close a database connection during interpreter shutdown may block forever — the network library it depends on may be partially torn down. Cleanup at shutdown should be opportunistic only. - `weakref.finalize(obj, func)` is the modern replacement for `__del__` — runs at object death, doesn't interfere with refcount, and exceptions still get the "ignored" treatment but the API is cleaner.

What NOT to say: - "`__del__` is reliable for resource cleanup." Not in the presence of cycles, exceptions, or shutdown. - "I'll raise from `__del__` to signal that close failed." The caller can't catch it. Use a `close()` method and let callers handle the error.

Common follow-up: "What's the modern alternative to `__del__` for cleanup?" — Context managers via `__enter__` / `__exit__` for explicit-scope resources. `weakref.finalize` for "do this when the object dies" without coupling to GC details.


```
import weakref
import logging
log = logging.getLogger(__name__)

# Bad – exceptions from __del__ are ignored, not raisable
class FileWrapperBad:
    def __init__(self, path):
        self.f = open(path)

    def __del__(self):
        self.f.close() # if close raises, "Exception ignored" on stderr

# Better – context manager for deterministic cleanup
class FileWrapperGood:
    def __init__(self, path):
        self.f = open(path)

    def __enter__(self):
        return self.f

    def __exit__(self, *args):
        self.f.close()

# Cleanup-on-death without __del__
class ResourceWithFinalizer:
    def __init__(self, path):
        self.f = open(path)
        # Registers cleanup to run when self is collected
        self._finalizer = weakref.finalize(self, self._safe_close, self.f)

    @staticmethod
    def _safe_close(f):
        try:
            f.close()
        except Exception:
            log.warning("close failed during finalization", exc_info=True)

    def close(self):
        # Explicit close – the modern caller would prefer this
        self._finalizer()
```

Q40 · Re-Raising After Side Effects — The Standard Production Pattern

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Postman, Goldman, JP Morgan · Topic: Real-World Patterns

The question (interviewer's verbatim phrasing):

Show me the canonical production pattern for handling an exception that you can't recover from but want to instrument before letting it propagate.

The 30-second answer: Catch the specific exception type, do the instrumentation (log, metric increment, alert), then bare `raise`. The bare raise preserves the original traceback exactly. Don't transform the exception unless you have a real reason to translate; don't swallow it; don't re-raise as a new exception that hides the original. The pattern is: catch narrowly, instrument, re-raise. That's the entire shape of a clean failure-instrumentation block.

Step-by-step thought process: 1. "Catch narrowly. `except PaymentError` not `except Exception`. The narrower the catch, the more confident you are about what to do." 2. "Instrument. `log.exception` for the structured traceback. `metrics.increment` for the failure count with tags (operation, error class). Maybe `alert(...)` for high-severity failures. All before re-raising." 3. "Bare `raise`. Re-raises the current exception with the original traceback. No transformation, no chain reset. The caller upstream sees the exception exactly as it would have without your block." 4. "Optionally, `e.add_note(f'while processing order={order.id}')` (3.11+) before raising. Adds the order context to the traceback for the upstream observer." 5. "This is the boundary pattern at every framework layer — request handlers, middleware, background workers. Every layer instruments and re-raises until something at the top renders an error response. Each layer adds its own context."

Edge cases to mention: - Don't call `log.exception` at every layer — you'll log the same exception N times. Pick one boundary (usually the outermost) to log; inner layers can log at INFO/DEBUG if useful for context. - `metrics.increment` should happen once per failure, not at every layer. Use a metric tag for the layer if you need granularity. - For retry boundaries, the pattern is different — catch, log, retry; only re-raise after the last attempt.

What NOT to say: - "I'd transform the exception to add my context." That breaks the chain. Use `add_note` or `from e` for context. - "Catch `Exception` and re-raise to be safe." Too wide; obscures whether you can actually act on the exception.

Common follow-up: "What's the difference between bare `raise` and `raise e` at the end of an except block?" — Bare `raise` is canonical and preserves traceback exactly. `raise e` works in 3.11+ but is mildly off-style; older Python truncates the traceback. Always prefer bare raise.

```
import logging
import metrics

log = logging.getLogger(__name__)

def process_order(order):
    try:
        validate(order)
        charge_card(order)
        ship(order)
    except ValidationError as e:
        log.exception("validation failed for order %s", order.id)
        metrics.increment("order.failure", tags={"stage": "validation", "type": type(e).__name__})
        e.add_note(f"order_id={order.id}")
        raise          # bare – original traceback preserved
    except PaymentError as e:
        log.exception("payment failed for order %s", order.id)
        metrics.increment("order.failure", tags={"stage": "payment", "type": type(e).__name__})
        e.add_note(f"order_id={order.id}")
        raise
    except ShippingError as e:
        log.exception("shipping failed for order %s", order.id)
        metrics.increment("order.failure", tags={"stage": "shipping", "type": type(e).__name__})
        e.add_note(f"order_id={order.id}")
        raise

# Upstream caller sees the original exception with the traceback intact
# plus the order_id note attached. Log line and metric have fired exactly once
# at this boundary.
```

End of Chapter 6

Forty questions covering Python's exception system from EAFP through structured concurrency. The chapters in File 04 that follow — comprehensions and slicing, functions and closures, typing and protocols — assume you can hold these mental models comfortably. If any of the exception-group, context-manager, or chaining patterns felt fresh, mark them and run them through interview-style explanation out loud once before you sit in a real round.

Chapter 7 — Comprehensions & Slicing (40 Qs)

Python Interview Prep — 2026 Edition · Learn & Excel · File 04 Core Python Library

Comprehensions and slicing are the two pieces of Python syntax interviewers reach for when they want a quick read on whether you write Python the way a Python programmer writes it, or the way a Java programmer writes it. There is nothing intellectually heavy here — every line in this chapter would survive a Python 3.12 interpreter — but the questions are sharp because the gotchas are quiet: a comprehension that looks like a one-liner but secretly leaks a name in Python 2 (and doesn't in 3.x); a `[::-1]` that reverses but doesn't reverse a numpy array the same way it reverses a list; a generator expression that looks identical to a list comprehension but uses constant memory; a walrus inside a comprehension that captures an intermediate so cleanly that two-line code suddenly looks clumsy. The candidates who clear this chapter are the ones who can read a comprehension out loud as a sentence — "for every order in the list, if the status is paid, take the amount" — and who treat slicing as a first-class API (the `slice` object, the `__getitem__` protocol, view-vs-copy semantics across list / str / numpy / pandas) rather than bracket-magic.

The 40 questions below follow the same Step-by-Step Answer shape as Chapters 1 through 6 — interviewer's verbatim phrasing, 30-second answer, the out-loud thought process, edge cases, what NOT to say, and the likely follow-up. The Indian product-co / GCC / Service-MNC mix is intentional: TCS / Infosys / Wipro / Accenture lean on the "what does this print" reading-comprehension form; Razorpay / Swiggy / Zerodha / Flipkart / Cred / PhonePe / Postman push you to the perf and gotcha layer; Walmart Labs / JP Morgan / Goldman / Microsoft / Target panels mix both. Treat the code blocks as the interviewer's whiteboard: every one runs on CPython 3.12+ and the output comments are what the interpreter actually prints.

Sections in this chapter

- Section A — List comprehensions (Q1–Q10)
- Section B — Dict and set comprehensions (Q11–Q15)
- Section C — Generator expressions (Q16–Q20)
- Section D — The walrus operator inside comprehensions (Q21–Q25)
- Section E — Slicing fundamentals (Q26–Q35)
- Section F — Slicing gotchas (Q36–Q40)

Section A — List Comprehensions

Q1 · The Three-Part Form — `[expr for x in iterable if cond]`

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Accenture, Cognizant · Topic: List Comprehensions

The question (interviewer's verbatim phrasing):

Walk me through the syntax of a list comprehension. What are the three parts and in what order do they execute?

The 30-second answer: A list comprehension is `[expr for x in iterable if cond]`. Three parts: the projection expression on the left, one or more `for` clauses in the middle that drive iteration, and an optional `if` filter at the end. Execution order matches a nested for-loop — the `for` runs the iterator, the `if` filters the iterated value, the expression on the left runs last and contributes one item to the output list.

Step-by-step thought process: 1. Open with the equivalence. `[expr for x in it if cond]` is the same as `out = []; for x in it: if cond: out.append(expr)`. Saying that out loud anchors everyone in the room. 2. Stress the left-to-right reading order: iterate first, filter second, project last. The fact that the projection is written first is a quirk of the syntax — it doesn't run first. 3. Mention that the `if` is optional, and that you can have multiple `for` clauses (nested), and you can have multiple `if` clauses (chained AND). All three knobs are independent. 4. Land on what makes it idiomatic: the comprehension is a single expression, so it can be passed directly to `sum`, `max`, `any`, used as a dict value, returned inline — for-loops can't.

Edge cases to mention: - The `if` belongs to the closest `for` to its left when there are multiple `for` clauses — important for nested comprehensions. - A comprehension always returns a brand new list; it does not append to an existing one. - An empty source iterable returns an empty list — no exception, just `[]`.

What NOT to say: - "List comprehensions are always faster than for-loops" — they're usually 5-20% faster, but the gap closes when the body is complex. Faster is a side-effect, not the goal. - "The if runs before the for" — wrong; the `for` produces values, the `if` filters them.

Common follow-up: "When would you write the equivalent for-loop instead of a list comprehension?" — When the body needs more than one statement, when you mutate external state, or when the comprehension is so dense it stops reading as a sentence.

```
nums = [1, 2, 3, 4, 5]

# expr      for-clause      if-clause
squares = [n * n for n in nums if n % 2 == 0]
print(squares)                # [4, 16]

# Equivalent for-loop
out = []
for n in nums:
    if n % 2 == 0:
        out.append(n * n)
print(out)                    # [4, 16]
```

Q2 · Nested Comprehensions — Flatten a List of Lists

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Flipkart, Walmart Labs, JP Morgan · Topic: List Comprehensions

The question (interviewer's verbatim phrasing):

Given `matrix = [[1, 2], [3, 4], [5, 6]]`, write a comprehension that flattens it to `[1, 2, 3, 4, 5, 6]`.

The 30-second answer: `[x for sub in matrix for x in sub]`. The outer `for sub in matrix` walks the rows, the inner `for x in sub` walks the items inside each row, and `x` on the left is what lands in the output list. The clauses read left-to-right exactly like a nested for-loop with the same variable order.

Step-by-step thought process: 1. State the rule: when you have multiple `for` clauses, they read in the same order as a nested for-loop. Outer first, inner second. Many candidates get this backwards on the whiteboard. 2. Show the equivalent expansion so the reader can verify the order: `for sub in matrix: for x in sub: out.append(x)`. 3. Mention the standard-library alternative: `itertools.chain.from_iterable(matrix)` returns an iterator over the flattened items, which is cheaper if you don't need a list. 4. Call out the depth limit: a comprehension flattens one level. For arbitrary nesting depth, write a recursive function — don't try to nest more comprehensions inside.

Edge cases to mention: - Sub-lists of different lengths work fine — comprehensions don't require a rectangular structure. - Empty sub-lists silently contribute zero items, no special handling. - A nested comprehension is not the same as a comprehension that builds a nested list — `[[x for x in sub] for sub in matrix]` rebuilds the structure; `[x for sub in matrix for x in sub]` flattens it.

What NOT to say: - "I'd use `sum(matrix, [])`" — works for shallow flatten but is $O(n^2)$ because each `+` builds a new list. Linters and reviewers will catch you. - "I'd write `[matrix[i][j] for i in range(len(matrix)) for j in range(len(matrix[i]))]`" — correct but un-Pythonic; you've turned a comprehension into Java-with-square-brackets.

Common follow-up: "How would you flatten a list nested three levels deep?" — Either two nested comprehensions or a recursive helper. The comprehension stops being readable past two levels.

```
matrix = [[1, 2], [3, 4], [5, 6]]
flat = [x for sub in matrix for x in sub]
print(flat)                                # [1, 2, 3, 4, 5, 6]

# Build a new nested list of the same shape (NOT a flatten)
shape = [[x * 10 for x in sub] for sub in matrix]
print(shape)                              # [[10, 20], [30, 40], [50, 60]]

from itertools import chain
print(list(chain.from_iterable(matrix)))   # [1, 2, 3, 4, 5, 6]
```

Q3 · Conditional Expression in the Projection — `[x if cond else y for x in it]`

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Cred, Postman · Topic: List Comprehensions

The question (interviewer's verbatim phrasing):

What is the difference between `[x for x in nums if x > 0]` and `[x if x > 0 else 0 for x in nums]` ?

The 30-second answer: The first is a filter — it drops negative numbers entirely, so the output length can be smaller than the input. The second is a transform — it keeps every input value but replaces negatives with `0`, so the output length always equals the input length. Filter `if` goes to the right of the `for`; ternary `if/else` goes to the left, inside the projection.

Step-by-step thought process: 1. Anchor on intent. "Filter" means drop some inputs; "transform" means change some inputs. The syntactic position of the `if` tells you which one you're doing. 2. Filter form: `[x for x in nums if x > 0]`. The `if` is a clause after the `for`. There is no `else` because filtering is binary — keep or drop. 3. Transform form: `[x if x > 0 else 0 for x in nums]`. The `if/else` is a conditional expression inside the projection. You must have an `else` because every input must produce an output. 4. Combine when you need to filter AND transform: `[x * 2 if x > 0 else 0 for x in nums if x is not None]`. Read it as "for each x in nums (skip None), double it if positive, else zero."

Edge cases to mention: - Forgetting the `else` in the transform form gives a `SyntaxError` — the conditional expression always needs both branches. - The order of filter and transform matters: filter happens during iteration; transform happens during projection. So `if x is not None` as a filter and `x.value if x else 0` as a transform protect different things. - Some style guides ban combining both forms in one comprehension because it reads slowly — split into two lines when the reader has to pause.

What NOT to say: - "Both forms do the same thing" — wrong; one changes the length of the output, the other preserves it. - "Put the `else` after the `for` clause" — that's a syntax error; the conditional expression lives in the projection, the filter clause does not.

Common follow-up: "How would you replace negatives with zero AND drop None values in one pass?" — `[x if x > 0 else 0 for x in nums if x is not None]` — filter clause for the None drop, ternary for the negative replacement.

```
nums = [-2, -1, 0, 1, 2]

# Filter — drops negatives, output shorter
filtered = [x for x in nums if x > 0]
print(filtered)                # [1, 2]

# Transform — replaces negatives with 0, output same length
clipped = [x if x > 0 else 0 for x in nums]
print(clipped)                 # [0, 0, 0, 1, 2]

# Combined — filter and transform
mixed = [x * 2 if x > 0 else 0 for x in nums if x != 0]
print(mixed)                   # [0, 0, 2, 4]
```

Q4 · When a For-Loop Reads Better Than a Comprehension

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Cred, Postman, Microsoft India · Topic: List Comprehensions

The question (interviewer's verbatim phrasing):

You've written this list comprehension and your reviewer says "rewrite as a for-loop." When would you agree with them?

The 30-second answer: Three cases. One, when the body needs more than a single expression — you can't put a `try/except`, a multi-line transformation, or a logging call inside a comprehension. Two, when the comprehension has more than two `for` clauses or more than two `if` clauses — it stops reading as a sentence. Three, when you need to mutate external state — comprehensions exist to build a list, not to side-effect.

Step-by-step thought process: 1. Open with the principle: a comprehension is good when it reads as one English sentence. The moment a reviewer has to slow down and parse, you've lost the readability win. 2. The multi-statement rule: any time you need `try/except`, multiple assignments, a `print` for debugging, or a logger call, the for-loop wins. Comprehensions are expressions, not statements. 3. The complexity rule: two `for` clauses is fine, three is a code-smell. Same for `if` clauses chained with

AND. 4. The side-effect rule: `[do_thing(x) for x in xs]` to invoke `do_thing` for its side-effects is a misuse — write the for-loop. The comprehension allocates a list of return values you immediately throw away.

Edge cases to mention: - Performance is rarely the right reason to pick one over the other — both are roughly the same CPython speed unless you're in a hot loop with a billion items. - A comprehension inside a comprehension (`[ [...] for ...]`) is fine when you're building a 2D structure deliberately. - Some teams enforce "no comprehension longer than 80 chars" via ruff/pylint — a hard line cuts the argument short.

What NOT to say: - "Comprehensions are always more Pythonic" — they're more Pythonic when they're readable. A six-line comprehension is less Pythonic than a clear for-loop. - "For-loops are slower" — the difference is negligible unless you're optimising a hot path.

Common follow-up: *"Rewrite this comprehension as a for-loop and tell me which version your reviewer accepts."* — Show the reviewer your decision criterion: number of statements per item, depth, side-effects, and 80-char width.

```
orders = [{"id": 1, "amt": 100}, {"id": 2, "amt": None}, {"id": 3, "amt": 200}]

# Tempting comprehension — but it needs a try/except, so:
totals = []
for o in orders:
    try:
        totals.append(o["amt"] * 1.18)
    except TypeError:
        # log and skip
        continue
print(totals)                                # [118.0, 236.0]

# A comprehension version would have to swallow None silently:
totals_comp = [o["amt"] * 1.18 for o in orders if o["amt"] is not None]
# Same output, but loses the "logged the bad row" behaviour
```

Q5 · Comprehension vs For-Loop — Performance

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Zerodha, Walmart Labs, JP Morgan · Topic: List Comprehensions

The question (interviewer's verbatim phrasing):

Is a list comprehension faster than the equivalent for-loop with `.append` ? Why?

The 30-second answer: Yes, usually by 10-30 percent on CPython. The comprehension uses a dedicated bytecode (`LIST_APPEND`) instead of a method lookup and call on every iteration — no `LOAD_ATTR` for `.append` , no `CALL` bytecode. The gap is biggest when the projection is cheap; it shrinks to nothing when the body is expensive because the bottleneck moves elsewhere.

Step-by-step thought process: 1. State the mechanism: in a for-loop with `out.append(x)` , every iteration does two extra bytecode operations — `LOAD_METHOD` for `.append` and `CALL_METHOD` . In a comprehension, the `LIST_APPEND` opcode appends to the result list directly without going through the attribute machinery. 2. Quantify: for a tight loop over a million items where the body is a simple expression, you'll see something like 80ms for the comprehension vs 110ms for the for-loop. That gap is real but small. 3. Stress what doesn't matter: if the body calls a function that takes 10ms, the comprehension vs for-loop overhead is invisible. 4. Mention `map` and `filter` : in modern CPython, `list(map(f, xs))` is roughly comparable to the comprehension when `f` is a built-in, but slower for Python functions because of call overhead.

Edge cases to mention: - The `dis` module shows the bytecode difference: `dis.dis` a comprehension and a for-loop and you'll see `LIST_APPEND` vs `LOAD_METHOD/CALL` . - PyPy compiles both to similar code, so the gap disappears there. - For numerical work over arrays, numpy vectorisation beats both by 100-1000x — neither comprehension nor for-loop is the right tool.

What NOT to say: - "Comprehensions are always faster" — only marginally, and only for simple bodies. - "The speed difference is huge" — it's measurable, not dramatic.

Common follow-up: "When would you reach for `map` instead of a comprehension?" — When you already have the function (don't lambda just to use map), and when you want a lazy iterator instead of a materialised list.

```
import dis

def comp():
    return [x * x for x in range(10)]

def loop():
    out = []
    for x in range(10):
        out.append(x * x)
    return out

# dis.dis(comp) shows LIST_APPEND
# dis.dis(loop) shows LOAD_METHOD ('append'), CALL_METHOD — slower per iter

# Quick timing intuition (don't paste into interview, just narrate):
# import timeit
# timeit.timeit(comp, number=100000)    # ~0.45s
# timeit.timeit(loop, number=100000)    # ~0.65s
```

Q6 · Comprehension Scoping in Python 3 — The Sealed Scope

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Microsoft India, Goldman · Topic: List Comprehensions

The question (interviewer's verbatim phrasing):

In Python 2, this code printed `4` at the end. In Python 3, it raises `NameError`. Why? `[x for x in range(5)]; print(x)`.

The 30-second answer: Python 3 gave comprehensions their own scope. The loop variable `x` lives inside the comprehension's implicit function and never leaks to the enclosing namespace. Python 2 leaked it — running `[x for x in range(5)]` at module level left `x = 4` behind, which was a footgun for anyone who happened to have an outer `x`. The fix in 3.0 was deliberate: comprehensions, dict comps, set comps, and generator expressions all have a sealed scope.

Step-by-step thought process: 1. Anchor on the version change: this was a Python 3.0 design decision, not a recent change. Many candidates still write code that assumes the leak. 2. Explain the implementation: each comprehension compiles to an implicit nested function that the interpreter calls with the iterable as its argument. The loop variable is a local of that function, so it's invisible outside. 3. Show the consequence: an outer `x` is not overwritten by a comprehension that uses `x` as its loop variable — different scopes. 4. Mention the for-loop contrast: a normal for-loop does leak — `for x in range(5): pass; print(x)` prints `4`. Comprehensions and for-loops differ on this exact point in Python 3.

Edge cases to mention: - The sealed scope applies to all four comprehension forms: list, set, dict, and generator. They all desugar to the same implicit-function pattern. - The implicit function still closes over outer names — you can read from the enclosing scope, you just can't write back to it. - The walrus operator is the one exception: `[(y := x*2) for x in nums]` does write `y` into the enclosing scope (covered in Q21).

What NOT to say: - "Comprehensions in Python 3 are syntactic sugar for a for-loop" — true at the algorithm level, false at the scoping level. The scope is the whole point of the change. - "It's a bug they should fix" — no; the leak was the bug, the fix shipped in 3.0.

Common follow-up: "Does a for-loop also have its own scope in Python 3?" — No, only comprehensions do. A for-loop variable leaks to the enclosing function scope.

```
# For-loop variable leaks
for x in range(5):
    pass
print(x)                                # 4

# Comprehension variable does not leak
[y for y in range(5)]
# print(y)                             # NameError: name 'y' is not defined

# Outer x is safe from a comprehension using x as loop var
x = 99
result = [x * 2 for x in range(3)]
print(x)                               # 99 — untouched
print(result)                           # [0, 2, 4]
```

Q7 · The `[x for x in [1,2,3]]` Lift — When This Is Genuinely Useful

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Postman, Freshworks · Topic: List Comprehensions

The question (interviewer's verbatim phrasing):

Your colleague wrote `[x for x in some_list]`. What does it do, and is there ever a reason to write it that way?

The 30-second answer: It produces a shallow copy of the input list — same items in the same order. Most of the time it's a code smell: `list(some_list)` or `some_list[:]` is shorter and clearer. The one defensible case is when you need to materialise an iterator into a list and you want to filter or transform in the same step — but if you're not filtering or transforming, the comprehension adds no value.

Step-by-step thought process: 1. Confirm the behaviour: with no `if` and an identity projection, every input item lands in the output unchanged, in the same order. It is a copy. 2. Compare alternatives: `list(some_list)` is one function call, `some_list[:]` is two characters, `some_list.copy()` is explicit. All three beat the comprehension for clarity. 3. Concede the rare win: `[x for x in iterator if cond]` is fine — you're filtering. `[x for x in iterator]` without an `if` is just `list(iterator)` with extra steps. 4. Mention the readability rule: if the comprehension looks like an identity, your reviewer will ask why you wrote it as a comprehension. Have a real answer or replace it.

Edge cases to mention: - For a generator, `[x for x in gen]` and `list(gen)` both exhaust it — the comprehension form lets you add an `if` later without restructuring. - For a dict, `[x for x in d]` gives you a list of keys — same as `list(d)`. Same code smell, same fix. - For a custom iterable, the comprehension calls `__iter__` once, same as `list()`. No difference at the protocol level.

What NOT to say: - "It's a deep copy" — it isn't. The list is new, the items inside are still the same references. - "It's faster than `list()`" — it isn't; `list()` is usually a hair faster because there's no Python-level loop body.

Common follow-up: "What's the difference between `[x for x in lst]`, `list(lst)`, and `lst[:]`?" — All three produce shallow copies. The comprehension runs a Python loop, the others use C-level copy paths. For readability and speed, prefer `list(lst)` or `lst[:]`.

```
src = [1, 2, 3]

# All three are shallow copies
a = [x for x in src]
b = list(src)
c = src[:]
print(a == b == c)           # True
print(a is src, b is src, c is src)  # False False False

# Defensible use of the comprehension form: filter while copying
adults = [u for u in users if u.age >= 18]
```

Q8 · Multi-Iter Comprehensions — Cartesian Product Inline

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Walmart Labs, Goldman · Topic: List Comprehensions

The question (interviewer's verbatim phrasing):

Write a comprehension that builds every `(suit, rank)` pair for a deck of cards.

The 30-second answer: `[(suit, rank) for suit in suits for rank in ranks]`. Two `for` clauses, outer-to-inner left-to-right, just like a nested for-loop. Produces 52 tuples for a standard deck. For three or more dimensions, prefer `itertools.product(suits, ranks, ...)` — it stays flat in the head.

Step-by-step thought process: 1. State the form: a comprehension can have any number of `for` clauses. Each one nests inside the previous one. Outer drives the slow axis; inner drives the fast axis. 2. Show the output structure: 4 suits times 13 ranks equals 52 items. The first 13 share the same suit, then the next 13, and so on. 3. Mention the inner-references-outer pattern: `[(i, j) for i in range(3) for j in range(i)]` lets the inner `for` read the outer loop variable, useful for triangular iteration. 4. Pivot to `itertools.product`: once you have three or more axes, `product` reads better than a comprehension and returns an iterator instead of a materialised list.

Edge cases to mention: - The inner `for` is re-evaluated for every outer iteration — if the inner iterable is a generator, it will be exhausted after the first outer iteration. Wrap it in `list()` if it's reusable. - The order matters for output: `[(i, j) for i in a for j in b]` is not the same as `[(i, j) for j in b for i in a]` — the second one varies `i` slowest. - An `if` between two `for` clauses filters using the variables available at that point — useful for pruning the cartesian product early.

What NOT to say: - "Three `for` clauses are fine" — they're legal but rarely readable. Pivot to `product`. - "Comprehensions are not for cartesian products" — they are, when you have two axes.

Common follow-up: "How would you generate every triangular pair `(i, j)` with `i < j` from `range(5)`?" — `[(i, j) for i in range(5) for j in range(i+1, 5)]` — the inner range depends on the outer variable.

```
suits = ["S", "H", "D", "C"]
ranks = ["A", "K", "Q", "J", "10", "9", "8", "7", "6", "5", "4", "3", "2"]
deck = [(s, r) for s in suits for r in ranks]
print(len(deck))           # 52
print(deck[:3])            # [('S', 'A'), ('S', 'K'), ('S', 'Q')]

# Triangular pairs (i < j)
pairs = [(i, j) for i in range(4) for j in range(i+1, 4)]
print(pairs)                # [(0,1), (0,2), (0,3), (1,2), (1,3), (2,3)]

# Beyond 2 axes — prefer itertools.product
from itertools import product
for s, r, c in product("SH", "AK", "rb"):
    pass    # 8 combinations, flat to read
```

Q9 · Comprehension with Multiple `if` Clauses — Chained AND

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Postman, JP Morgan · Topic: List Comprehensions

The question (interviewer's verbatim phrasing):

Can a comprehension have more than one `if` ? What is the difference between `if a if b` and `if a and b` ?

The 30-second answer: Yes. Multiple `if` clauses are chained with logical AND — every condition must be true for the item to pass. `[x for x in lst if a if b]` is exactly equivalent to `[x for x in lst if a and b]`. They produce the same output; the chained form sometimes reads better when each predicate fits on its own line.

Step-by-step thought process: 1. Confirm the equivalence first — interviewers want to hear "they're the same." `if a if b` ANDs the two predicates. 2. Mention the short-circuit semantics: like `and`, the second predicate is only evaluated when the first is true. Useful when the second predicate is expensive or could raise. 3. Argue the readability angle: when predicates are short, `and` reads better. When predicates are long, splitting them into two `if` clauses on separate lines helps the reviewer. 4. Note that there is no analogous OR form — you write `if a or b` explicitly. The chained `if` is AND-only.

Edge cases to mention: - Short-circuiting matters: `[x for x in lst if x is not None if x.amount > 100]` is safe even when `x` is `None`, because the second clause is skipped. Combining with `and` would behave the same way. - The order of the predicates affects performance — put the cheap, high-rejection-rate one first. - More than two `if` clauses gets visually heavy — refactor to a helper function `def ok(x): ...` and write `if ok(x)`.

What NOT to say: - "They're different — `if a if b` is OR" — wrong; both forms are AND. - "The second predicate always runs" — wrong; like `and`, the second is short-circuited.

Common follow-up: "How do you express OR between predicates in a comprehension?" — Use a single `if a or b` clause. The chained `if` form is AND-only.

```
orders = [
    {"id": 1, "amount": 500, "status": "paid"},
    {"id": 2, "amount": 50, "status": "paid"},
    {"id": 3, "amount": 600, "status": "failed"},
]

# Three equivalent ways to filter for paid orders > 100
a = [o for o in orders if o["status"] == "paid" if o["amount"] > 100]
b = [o for o in orders if o["status"] == "paid" and o["amount"] > 100]
c = [o for o in orders if (lambda x: x["status"] == "paid" and x["amount"] > 100)(o)]
print(a == b == c)                                # True

# Short-circuit safety
data = [None, {"amount": 200}, {"amount": 50}, None]
big = [d for d in data if d is not None if d["amount"] > 100]
print(big)                                         # [{'amount': 200}]
```

Q10 · The Big Comprehension That Should Be a Function

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India, Walmart Labs · Topic: List Comprehensions

The question (interviewer's verbatim phrasing):

Refactor this comprehension and explain why your reviewer asked for it: `[u.name.title() for u in users if u.country in {"IN", "US", "GB"} and u.age >= 18 and u.email_verified and u.consents.get("marketing", False)]`.

The 30-second answer: The comprehension does too much in one expression — five predicates and a transformation. Extract the predicate into a named function: `def is_eligible(u): ...` returns the boolean; the comprehension becomes `[u.name.title() for u in users if is_eligible(u)]`. The named function gets a docstring, a unit test, and a place for the predicate to evolve. The comprehension stays one English sentence.

Step-by-step thought process: 1. Diagnose the smell. Five predicates in one `if` clause means the comprehension is doing two jobs — predicate logic AND projection. Split them. 2. Show the refactor. `is_eligible(u)` carries the country-and-age-and-verified-and-marketing logic. The comprehension carries only the projection (`u.name.title()`). 3. State the wins. Testability — `is_eligible` is now a one-arg function you can call from a unit test. Reuse — three places in your codebase probably already need this predicate. Readability — the comprehension fits on one line again. 4. Mention the side benefit: when the predicate grows (sixth condition next quarter), you add it to the function, not to the comprehension. Diff stays small.

Edge cases to mention: - If the predicate is genuinely one-off and won't be reused, naming it adds noise — leave it inline. - If the predicate is short (two conditions) and reads as a sentence, leave it inline — extraction is a tool, not a rule. - When the projection is also complex, consider extracting both: `def display_row(u): return u.name.title()`.

What NOT to say: - "Comprehensions should always be one-liners" — they can wrap across lines; that's not the problem. - "I'd write a for-loop instead" — the for-loop has the same readability problem with a five-condition `if`. The fix is extracting the predicate, not changing the construct.

Common follow-up: "How do you test the comprehension once it depends on `is_eligible`?" — Unit test `is_eligible` directly with synthetic users. The comprehension itself is then trivially a `map+filter` and doesn't need its own test.


```
def is_eligible(u):
    return (
        u.country in {"IN", "US", "GB"}
        and u.age >= 18
        and u.email_verified
        and u.consents.get("marketing", False)
    )

eligible_names = [u.name.title() for u in users if is_eligible(u)]

# Unit-testable
class FakeUser:
    def __init__(self, **kw): self.__dict__.update(kw)

u1 = FakeUser(country="IN", age=25, email_verified=True,
               consents={"marketing": True}, name="aisha")
assert is_eligible(u1)
```

Section B — Dict and Set Comprehensions

Q11 · Dict Comprehension Syntax — `{k: v for ...}`

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Postman · Topic: Dict Comprehensions

The question (interviewer's verbatim phrasing):

Write a dict comprehension that maps each word in a list to its length.

The 30-second answer: `{w: len(w) for w in words}`. Dict comprehensions look like list comprehensions with a key-colon-value pair instead of a single expression. The braces signal dict, the colon distinguishes it from a set comprehension. Same `for` and `if` clauses, same scoping rules.

Step-by-step thought process: 1. State the syntax: braces, colon between key and value, then the same `for ... in ... if ...` clauses you'd write in a list comprehension. 2. Stress duplicate-key behaviour: if two iterations produce the same key, the later value wins (last write wins). This is dict semantics, not a comprehension special case. 3. Mention the read order: same as list comp — iterate, filter, project. The projection here is the `k: v` pair. 4. Bring up the `dict()` constructor alternative: `dict((w, len(w)) for w in words)` is equivalent but slower because of the tuple allocation per item; the comprehension is the idiom.

Edge cases to mention: - Duplicate keys silently overwrite — `{i % 3: i for i in range(10)}` gives you only three keys (0, 1, 2), each holding the last `i` that mapped to it. - The key expression is

evaluated before the value expression on each iteration — order matters if either side has side-effects. - Just like list comps, the loop variable does not leak into the enclosing scope.

What NOT to say: - "Use `{}` for an empty dict comprehension" — `{}` is an empty dict literal, not a comprehension. A comprehension always has at least one `for`. - "Dict comps are slower than `dict()` of tuples" — they're faster, because no tuple allocation.

Common follow-up: "What happens with duplicate keys in a dict comprehension?" — Last write wins. The dict ends up with one entry per unique key; the iteration order of the source determines which value survives.

```
words = ["razorpay", "swiggy", "cred", "swiggy"]

lengths = {w: len(w) for w in words}
print(lengths)                # {'razorpay': 8, 'swiggy': 6, 'cred': 4}
# 'swiggy' appears twice in input but once in dict — last write wins

# Build a lookup from a list of tuples
pairs = [("INR", 83.2), ("USD", 1.0), ("EUR", 0.92)]
rates = {code: rate for code, rate in pairs}
print(rates["INR"])           # 83.2

# Filter + transform
upper_lengths = {w.upper(): len(w) for w in words if len(w) > 4}
print(upper_lengths)          # {'RAZORPAY': 8, 'SWIGGY': 6}
```

Q12 · Set Comprehension Syntax — `{x for ...}`

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Cred · Topic: Set Comprehensions

The question (interviewer's verbatim phrasing):

Given a list with duplicates, write a set comprehension that returns the unique values squared.

The 30-second answer: `{x*x for x in lst}`. Braces with a single expression (no colon) is a set comprehension. Duplicate outputs are automatically deduplicated — that's the whole point of a set. The same `for` and `if` clauses work, and items must be hashable.

Step-by-step thought process: 1. State the syntax distinction: braces with `k: v` is a dict comprehension; braces with a single expression is a set comprehension; the colon is what tells the parser which one you wrote. 2. Stress the dedup property: if the projection produces the same value twice (e.g., 2 and -2 both square to 4), the set keeps one. Order is not preserved. 3. Call out hashability: set elements must be hashable, so a set comprehension producing lists fails at runtime with

TypeError . Tuples work; lists don't. 4. Mention the `set()` alternative: `set(x*x for x in lst)` works but allocates a generator first. The comprehension is the idiom.

Edge cases to mention: - A set comprehension producing duplicates silently dedupes — if you needed multiplicity, use a list comprehension. - A set comprehension over an empty source returns an empty set (`set()`), not `{}` — that braces literal is an empty dict. - Set comprehensions inherit set's lack of ordering — don't rely on iteration order matching insertion order.

What NOT to say: - "{} creates an empty set" — wrong; that's an empty dict. `set()` is the empty-set constructor. - "Sets preserve order" — they don't. Dict comps preserve insertion order in Python 3.7+; sets do not.

Common follow-up: "How do you build an empty set vs an empty dict?" — Empty dict is `{}` or `dict()` . Empty set is `set()` only — `{}` is unambiguously a dict.

```
nums = [1, 2, 2, -2, 3, -3, 3]

squares = {n * n for n in nums}
print(squares)                # {1, 4, 9} – dedup'd, order may vary

# Set comprehension with filter
positives = {n for n in nums if n > 0}
print(positives)              # {1, 2, 3}

# Hashability – this fails:
# bad = {[n, n] for n in nums}    # TypeError: unhashable type: 'list'
# Use tuples instead:
good = {(n, n) for n in nums}
print(good)                   # {(1,1), (2,2), (-2,-2), (3,3), (-3,-3)}
```

Q13 · Dict Comp from an Iterable of Tuples — Building a Lookup

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Flipkart, Postman, JP Morgan · Topic: Dict Comprehensions

The question (interviewer's verbatim phrasing):

You have `pairs = [("a", 1), ("b", 2), ("c", 3)]` . Build a dict with the comprehension form and with the `dict()` constructor. Which one would you ship?

The 30-second answer: Comprehension: `{k: v for k, v in pairs}` . Constructor: `dict(pairs)` . Both produce `{"a": 1, "b": 2, "c": 3}` . Ship `dict(pairs)` when you're literally converting a list of two-tuples — it's shorter and clearer. Ship the comprehension when you want to transform during the build — `{k.upper(): v * 10 for k, v in pairs}` .

Step-by-step thought process: 1. Confirm both forms work. The `dict()` constructor accepts any iterable of two-element iterables and treats them as (key, value) pairs. 2. State the readability rule: when there's no transformation, `dict(pairs)` reads better. When there's any transformation (case-changing keys, scaling values, filtering pairs), the comprehension is the idiom. 3. Mention the unpacking form: `{k: v for k, v in pairs}` unpacks each tuple into two names. If you need both the original tuple and its parts, capture: `{p[0]: p[1] for p in pairs}`. 4. Pivot to `zip`: a common dict-build is `dict(zip(keys, values))` — same idea, different source. If you also need to filter, the comprehension wins.

Edge cases to mention: - If the pairs iterable yields three-element tuples by accident, `dict()` raises `ValueError`; the comprehension with `for k, v in pairs` also raises but at unpack time — both fail loudly. - `dict(pairs)` is faster than the comprehension for the no-transform case because it can specialise on the iterable type. - If you want last-write-wins on duplicates, both forms have it. If you want first-write-wins, you need `{}.setdefault` in a for-loop.

What NOT to say: - "Always use comprehension" — for the trivial conversion, `dict()` is shorter. - "`dict()` doesn't handle filtering" — correct, which is exactly when you reach for the comprehension.

Common follow-up: "What if your pairs come from a generator and you want to drop duplicate keys, keeping the first one?" — Build a dict the for-loop way: `out = {}; for k, v in gen: out.setdefault(k, v)`. Comprehensions can't express first-wins.

```

pairs = [("a", 1), ("b", 2), ("c", 3)]

# Trivial conversion — prefer dict()
d1 = dict(pairs)
print(d1)                                # {'a': 1, 'b': 2, 'c': 3}

# With transformation — prefer comprehension
d2 = {k.upper(): v * 100 for k, v in pairs}
print(d2)                                # {'A': 100, 'B': 200, 'C': 300}

# zip-based build
codes = ["INR", "USD", "EUR"]
rates = [83.2, 1.0, 0.92]
fx = dict(zip(codes, rates))
print(fx["INR"])                          # 83.2

```

Q14 · Set Comprehension for Dedup — Idiomatic Distinct

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Swiggy, Cred, Walmart Labs · Topic: Set Comprehensions

The question (interviewer's verbatim phrasing):

Given a list of emails with mixed case, write the shortest expression that gives you the distinct lowercased emails.

The 30-second answer: `{e.lower() for e in emails}`. The set comprehension does the lowercase transform and the dedup in one pass. If you need the result as a list, wrap in `list(...)`; the comprehension itself returns a set. `set(e.lower() for e in emails)` is identical but goes through a generator first — same result, marginally slower.

Step-by-step thought process: 1. State the idiom: set comprehension is the right tool for "transform-and-dedup in one expression." 2. Compare with the for-loop alternative: `seen = set(); for e in emails: seen.add(e.lower())`. Same outcome, three lines. 3. Mention the order-stability angle: a set comp doesn't preserve insertion order. If you need order-preserved dedup, use `dict.fromkeys(e.lower() for e in emails)` and take the keys — dict insertion order is preserved. 4. Note that the transformation can be more than `lower()` — it could be `e.strip().lower()` or any normalisation. The set still dedupes on the final value.

Edge cases to mention: - Empty input gives `set()`, not `{}` — that braces literal is a dict. - If the transform is expensive, the comprehension still calls it for every input — even duplicates. Pre-filter duplicates first if cost matters. - `set(map(str.lower, emails))` is another idiom — equivalent perf for built-ins, slightly slower for lambdas.

What NOT to say: - "Use a list comprehension and then convert to set" — that wastes a list allocation; set comp does both in one pass. - "Sets keep insertion order" — they don't. Reach for `dict.fromkeys` if you need order-preserved dedup.

Common follow-up: "How do you preserve insertion order while deduping?" — `list(dict.fromkeys(emails))`. Dicts keep insertion order in Python 3.7+ and `fromkeys` ignores duplicate keys.

```
emails = ["A@x.com", "b@x.com", "a@x.com", "B@X.COM", "c@x.com"]

# Distinct lowercased — set comp
unique = {e.lower() for e in emails}
print(unique)                                # {'a@x.com', 'b@x.com', 'c@x.com'}

# Order-preserved dedup (with case-normalised key)
ordered = list(dict.fromkeys(e.lower() for e in emails))
print(ordered)                              # ['a@x.com', 'b@x.com', 'c@x.com']

# Normalise more aggressively before dedup
import re
def norm(e): return re.sub(r"\s+", "", e).lower()
print({norm(e) for e in emails})             # same 3 entries
```

Q15 · The `{}` Trap — Empty Dict vs Empty Set

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant, Postman · Topic: Dict / Set Comprehensions

The question (interviewer's verbatim phrasing):

What's the type of `{}` and how do you write an empty set?

The 30-second answer: `{}` is an empty **dict**, not an empty set — this surprises everyone the first time. The empty set is `set()`, full stop. The reason is historical: the dict literal predates set literals by a decade, so when sets got literal syntax in Python 3, the parser had to keep `{}` meaning dict to avoid breaking existing code.

Step-by-step thought process: 1. State the rule and the cause. Dict literal predates set literal — `type({})` is `dict`, and you cannot change that. 2. Show the workarounds: `set()` to build an empty set; `{x}` with at least one element to use set literal syntax. 3. Tie it back to comprehensions: an empty source iterable in a set comprehension still returns an empty set (`set()`), not `{}`. The parser determines dict-vs-set from the presence of a colon, not from emptiness. 4. Mention the frozenset analogue: `frozenset()` for empty, no literal syntax at all. Frozensets always need the constructor.

Edge cases to mention: - `type({})` is `dict`. `type({1})` is `set`. `type({1: 2})` is `dict`. The colon switches the parse. - `len({})` is 0, but it's still a dict — checking `if x == set()` for an empty container is wrong if `x` might be `{}`. - The empty set has no literal even in Python 3.12 — the language never carved out a syntax for it.

What NOT to say: - "`{}` is an empty set" — no; it's a dict. This is the most common Python literal mistake. - "I'd use `{*[]}` for empty set" — works, but `set()` is the idiom and reviewers will rewrite it.

Common follow-up: "How would a dict comprehension and a set comprehension differ in syntax for the same expression?" — Add the colon: `{x: x*2 for x in lst}` is a dict comp; `{x for x in lst}` is a set comp. Colon is the discriminator.

```
print(type({}))                # <class 'dict'>
print(type({1, 2, 3}))         # <class 'set'>
print(type({1: "a"}))         # <class 'dict'>
print(type(set()))            # <class 'set'>

# Set comprehension result type – set, even when empty
empty_set = {x for x in [] if x > 0}
print(empty_set, type(empty_set))    # set() <class 'set'>

# A common bug: this is NOT how you build a set of one item from a list
# wrong = {[ ]}                # TypeError: unhashable type: 'list'
right = set()
right.add("ok")
print(right)                   # {'ok'}
```

Section C — Generator Expressions

Q16 · Generator Expression Syntax — `(x for x in ...)`

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Cred, JP Morgan, Walmart Labs · Topic: Generator Expressions

The question (interviewer's verbatim phrasing):

What is the difference between `[x*x for x in range(10)]` and `(x*x for x in range(10))` ?

The 30-second answer: The first is a list comprehension — it materialises a list of ten integers in memory. The second is a generator expression — it returns a generator object that computes each value lazily, one at a time, using constant memory. They have the same shape, the same iteration semantics, and the same scoping rules. The only differences are the brackets and the laziness.

Step-by-step thought process: 1. State the syntactic difference: square brackets give you a list comp, round brackets give you a generator expression. Same body, same `for` and `if` clauses. 2. State the runtime difference: list comp eagerly builds the entire list (memory cost equals the output size); generator expression yields values one at a time (memory cost is constant per active iteration). 3. Stress the consumption rule: a generator can be iterated exactly once. After exhaustion, it's empty. List comps don't have this property — you can iterate the list as many times as you want. 4. Anchor the choice rule: if the result fits comfortably in memory AND you want random access or multiple iterations, list comp. If the input is huge OR you only iterate once, generator expression.

Edge cases to mention: - A generator expression captures the source iterable lazily — the iteration starts when you actually iterate, not when you write the gen exp. - Calling `len()` on a generator raises `TypeError` — it has no length until exhausted. - Generators are not subscriptable — `gen[3]` raises `TypeError`. Wrap in a list if you need indexing.

What NOT to say: - "Generator expressions are slower" — wrong; they're often faster for one-pass consumption because no list allocation. - "You can iterate a generator twice" — you cannot; it's exhausted after the first pass.

Common follow-up: "When would you choose a generator expression over a list comprehension?" — Large data, one-pass consumption, or when feeding into `sum` / `any` / `all` / `max` / `min` directly.

```
# List comprehension — materialised
lc = [x * x for x in range(5)]
print(lc)                # [0, 1, 4, 9, 16]
print(len(lc))            # 5

# Generator expression — lazy
ge = (x * x for x in range(5))
print(ge)                # <generator object <genexpr> at 0x...>
# print(len(ge))          # TypeError: object of type 'generator' has no len()
print(list(ge))           # [0, 1, 4, 9, 16] — materialised on demand
print(list(ge))           # [] — already exhausted

# Memory contrast — never paste this if your interviewer is impatient
# total = sum([x*x for x in range(10**8)]) # allocates 100M-element list
# total = sum(x*x for x in range(10**8))  # constant memory
```

Q17 · Generator Inside a Function Call — Drop the Parentheses

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Generator Expressions

The question (interviewer's verbatim phrasing):

Why is `sum(x*x for x in range(10))` legal — there are no parentheses around the generator?

The 30-second answer: When a generator expression is the only argument to a function call, the parser lets you skip the inner parentheses — the function-call parens serve as the gen exp's parens. `sum(x*x for x in range(10))` is exactly the same as `sum((x*x for x in range(10)))`. The shortcut only works for single-argument calls; with multiple args, you need the inner parens to disambiguate.

Step-by-step thought process: 1. State the syntactic rule clearly: single-argument function call with a gen exp → inner parens optional. Multi-argument call → inner parens required. 2. Walk through why: with one argument, the parser can tell the `for` keyword starts a gen exp, not a kwarg. With multiple arguments separated by commas, the comma would be ambiguous between "next arg" and "tuple element." 3. Show the multi-arg case that breaks: `f(x for x in nums, 10)` is a `SyntaxError`. You must write `f((x for x in nums), 10)`. 4. Mention the idiom: `sum(x*x for x in nums)`, `any(u.is_active for u in users)`, `max(o.amount for o in orders)`, `min(p.price for p in items)`. These are all canonical Pythonic forms — call them out by name.

Edge cases to mention: - The omitted-parens form applies only to function calls. In all other contexts (assignments, returns, expressions), a gen exp needs its own parens. - Method calls behave the same way: `some_obj.process(x for x in items)` works without inner parens. - The shortcut does not apply to dict/set/list comp inside function calls — they always need their own brackets.

What NOT to say: - "It's a special case for `sum`" — it isn't; the parser rule applies to any single-arg function call. - "Always include the inner parens for clarity" — most teams skip them; reviewers expect the lean form.

Common follow-up: "Write `min` over a list of tuples picking the lowest second element." — `min((t[1] for t in pairs))` or, since `min` accepts a `key=` arg, `min(pairs, key=lambda t: t[1])`. The second form is preferred.

```
nums = [3, 1, 4, 1, 5, 9, 2, 6]

# Single-arg call – inner parens optional
print(sum(x * x for x in nums))      # 173
print(max(x for x in nums if x < 5)) # 4
print(any(x > 8 for x in nums))      # True
print(all(x > 0 for x in nums))      # True

# Multi-arg call – inner parens required
# print(sorted(x for x in nums, reverse=True)) # SyntaxError
print(sorted((x for x in nums), reverse=True)) # works
# Better: use the key argument
print(sorted(nums, reverse=True))      # [9, 6, 5, 4, 3, 2, 1, 1]
```

Q18 · Generator vs List Comp — Memory Math

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Walmart Labs, JP Morgan · Topic: Generator Expressions

The question (interviewer's verbatim phrasing):

You're summing the squares of 100 million integers. List comp or generator expression? Walk through the memory cost.

The 30-second answer: Generator expression, no contest. `sum(x*x for x in range(10**8))` uses constant memory — the generator yields one int at a time, `sum` keeps a single running total. `sum([x*x for x in range(10**8)])` allocates a 100-million-element list — roughly 800 MB on a 64-bit Python because each int object plus pointer eats about 28 bytes. Same answer, vastly different memory footprint.

Step-by-step thought process: 1. State the principle: laziness wins when you're aggregating, filtering or consuming in one pass. Materialisation is only worth its memory cost when you need random access or multiple iterations. 2. Run the actual numbers: 100M Python ints at ~28 bytes each plus list overhead is roughly 800 MB. The gen exp uses ~200 bytes total — the gen object itself plus one int. 3. Mention the time side: the gen exp is also faster for this case because the list allocator doesn't run, and the GC doesn't have to clean up 100M small objects. 4. Land on the rule: every `sum`, `any`, `all`, `max`, `min`, `next`, or `for` loop over a gen exp is a memory win. Only switch to list comp when you need the list itself.

Edge cases to mention: - The `range` object is itself lazy — it doesn't materialise 100M numbers. So the list comp's memory is on the output, not the input. - For numerical work, `numpy.arange(10**8)**2 .sum()` uses ~800 MB of contiguous C ints (4 or 8 bytes each) and is 50-100x faster than either Python form. - Generators don't help if you call `list()` on them — you've materialised the same data, just one level deferred.

What NOT to say: - "List comps are always faster" — not when you're paying for allocation you don't need. - "Generators are slower because they yield one at a time" — the per-iteration overhead is small; the allocation savings dominate.

Common follow-up: "What if you need both the sum and the count of those squared values?" — Iterate once into two accumulators:

`total, n = 0, 0; for x in (i*i for i in range(N)): total += x; n += 1`. Or use `more_itertools.tee` if you must consume twice.

```

N = 10**6    # bounded for the interview whiteboard

# Generator — constant memory
import sys
gen = (x * x for x in range(N))
print(sys.getsizeof(gen))          # ~104 bytes — the gen object itself
total = sum(gen)
print(total)                       # 333332833333500000

# List comp — output size scales with N
lst = [x * x for x in range(N)]
print(sys.getsizeof(lst))          # millions of bytes for 1M ints
print(sum(lst))                   # same total

```

Q19 · Composing Generator Expressions — A Pipeline

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India, Goldman · Topic: Generator Expressions

The question (interviewer's verbatim phrasing):

You have a generator of order dicts from a CSV. Write a pipeline that drops failed orders, scales amounts to paise, and sums the result — without building any intermediate list.

The 30-second answer: Chain gen exps. Each stage is a generator that consumes the previous one lazily. `paid = (o for o in orders if o["status"] == "paid")`. Then `paise = (int(o["amount"] * 100) for o in paid)`. Then `sum(paise)`. Three references, zero intermediate lists, constant memory throughout the pipeline.

Step-by-step thought process: 1. State the principle: gen exps compose cleanly because each one is itself an iterable. The downstream consumer pulls one item at a time through the whole chain. 2. Walk through the three stages. Filter (drop bad rows). Transform (scale amount). Aggregate (sum). Each stage adds about one line and zero memory. 3. Compare with the list comp version: `paid = [o for o in orders if ...]; paise = [int(o['amount']*100) for o in paid]; sum(paise)` — two intermediate lists, both proportional to input size. 4. Mention the inline form when readability wins: `sum(int(o["amount"] * 100) for o in orders if o["status"] == "paid")`. Single line, same semantics, no intermediates. Use this when the pipeline stages don't need names.

Edge cases to mention: - Each generator is exhaustible — if you need to iterate the same pipeline twice, refactor into a function that returns a fresh gen each call. - `itertools` is the natural companion — `chain`, `islice`, `takewhile`, `dropwhile`, `filterfalse` all compose with gen exps. - Debugging is harder than with list comps because you can't print the intermediate without exhausting it. Insert `list(gen)` temporarily during dev.

What NOT to say: - "Pipelines need a framework" — no, plain Python composes gen exps fine. - "Naming each stage is overkill" — it is for two stages; it pays off at four or more.

Common follow-up: "Rewrite the pipeline using `itertools.compress` or `filter` and tell me which form your team would merge." — Both work; gen exps stay readable longest. `filter` becomes cleaner when the predicate is already a named function.

```
orders = [
    {"id": 1, "amount": 499.99, "status": "paid"},
    {"id": 2, "amount": 199.50, "status": "failed"},
    {"id": 3, "amount": 1299.00, "status": "paid"},
    {"id": 4, "amount": 50.00, "status": "refunded"},
]

# Named-stage pipeline — easier to debug
paid      = (o for o in orders if o["status"] == "paid")
paise     = (int(o["amount"] * 100) for o in paid)
total_paise = sum(paise)
print(total_paise)                # 179899

# Inline form — same semantics
total = sum(int(o["amount"] * 100) for o in orders if o["status"] == "paid")
print(total)                      # 179899
```

Q20 · When to Choose Generator Expression Over List Comp — The Decision

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Cred, JP Morgan, Microsoft India · Topic: Generator Expressions

The question (interviewer's verbatim phrasing):

Give me your decision rule: list comp or generator expression?

The 30-second answer: List comp when you need the list — repeated iteration, indexing, length, slicing, JSON-serialising. Generator expression when you don't — one-pass aggregation (sum/any/all/max/min), feeding into another generator, large data where allocation costs matter, or a for-loop that iterates exactly once. If you're ever unsure, default to gen exp inside function calls and list comp when assigning to a variable that survives.

Step-by-step thought process: 1. Identify the consumption pattern first. One-pass consume → gen exp. Multi-pass or random-access → list comp. 2. Identify the size pattern second. Bounded small data → either works. Large/unbounded → gen exp. 3. Identify the receiver. Functions like `sum`, `any`, `all`, `max`, `min`, `next` accept iterables — gen exp wins. Functions like `json.dumps`, `sorted` (with reverse), `.join` need a materialised collection or iterate multiple times — list usually

wins. 4. Last tiebreak: readability. If naming intermediate variables helps a reviewer understand the pipeline, name them as gen exps. If a single line tells the whole story, the inline gen exp inside a function call is the idiom.

Edge cases to mention: - `sorted` and `min / max` both accept gen exps, but `sorted` materialises internally anyway — no memory win, but no loss either. - `json.dumps(gen_exp)` does NOT work — JSON serialisers can't iterate arbitrary iterables; they want lists/dicts. - `"".join(gen_exp)` works and is the canonical fast way to build a long string from many parts.

What NOT to say: - "Always use generator expressions" — wrong when you need length or random access. - "List comps are always more Pythonic" — both are; pick by consumption pattern.

Common follow-up: " `"".join([str(x) for x in lst])` vs `"".join(str(x) for x in lst)` — which is faster?" — The list-comp form is slightly faster in CPython because `join` reads the length up front and pre-allocates. For small inputs the difference is invisible; for large ones, the list comp wins by a hair.

```
# Use gen exp — one-pass aggregation
total = sum(x * x for x in range(1000))
has_neg = any(x < 0 for x in numbers)
big = max(o.amount for o in orders)

# Use list comp — repeated iteration / indexing
names = [u.name for u in users]
for n in names: log(n)                # iterate 1
for n in names: send_email(n)         # iterate 2 — gen exp would be exhausted
print(names[0])                       # index
print(len(names))                     # length

# join — list comp slightly faster but both work
big_str = "".join(str(x) for x in range(1000))
big_str = "".join([str(x) for x in range(1000)]) # marginally faster
```

Section D — The Walrus Operator Inside Comprehensions

Q21 · Walrus `:=` Basics — Assignment Inside an Expression

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Microsoft India, JP Morgan · Topic: Walrus Operator

The question (interviewer's verbatim phrasing):

What does the walrus operator do, and when did it land in Python?

The 30-second answer: The walrus `:=` is the assignment expression — it binds a value to a name AND returns the value, in one expression. Landed in Python 3.8 via PEP 572. Lets you assign inside an `if`, a `while`, or a comprehension where a regular `=` would be a syntax error. The classic use is reading from a file: `while (chunk := f.read(8192)): process(chunk)`.

Step-by-step thought process: 1. State the mechanism. `x := expr` evaluates `expr`, binds the result to `x` in the enclosing scope, and the whole `x := expr` expression evaluates to that result. Regular `=` is a statement and doesn't return a value, so it can't appear inside expressions. 2. State the canonical use. The "read a chunk and check if it's non-empty" pattern in 3.7 needed a `while True: chunk = f.read(...); if not chunk: break`. With walrus: `while chunk := f.read(...): ...` — one line, no break. 3. Mention version: 3.8. Older code can't use it; if you're targeting 3.7 you'll need to refactor. 4. Scope subtlety: the walrus writes into the enclosing function scope, not the comprehension scope (covered in Q24). This is what makes it useful inside comprehensions specifically.

Edge cases to mention: - The walrus is an expression, so it needs parens in many contexts: `x := 5` alone is a syntax error; `(x := 5)` is the expression. - You cannot use walrus at module top-level as a bare statement — `(x := 5)` works but `x := 5` doesn't. - Inside an `if`, walrus is heaven: `if (match := re.match(pat, line)): group = match.group(1)`.

What NOT to say: - "Walrus is just shorthand for `=`" — it isn't; it returns a value, regular assignment doesn't. - "Walrus works in Python 2" — Python 2 is long retired; this is a 3.8+ feature.

Common follow-up: "Show me one place walrus genuinely improves readability." — Read-and-test: `while line := f.readline(): process(line)`. Without walrus you'd write a `while True` with a break — three lines vs one.

```
# Read-and-test pattern
import io
data = io.StringIO("line1\nline2\nline3\n")
while line := data.readline():
    print(line.rstrip())           # line1, line2, line3 – one line of logic

# If with capture
import re
log_line = "ERROR 2026-05-30 connection refused"
if (m := re.match(r"(ERROR|WARN) (\S+)", log_line)):
    print(m.group(1), m.group(2))  # ERROR 2026-05-30

# Bare walrus needs parens
# x := 5                          # SyntaxError
(x := 5)                          # OK
print(x)                          # 5
```

Q22 · Walrus Inside a List Comp — Capturing an Intermediate

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Walrus Operator

The question (interviewer's verbatim phrasing):

You're calling an expensive `compute(x)` and you want both its result and its square in the output. Show me the walrus form.

The 30-second answer: `[(y := compute(x), y * y) for x in nums]`. The walrus captures `compute(x)` once into `y`, then the projection uses `y` twice — once as the first tuple element, once squared. Without walrus you'd call `compute(x)` twice per item or write a helper function. The walrus form saves the duplicate call.

Step-by-step thought process: 1. State the problem the walrus solves: in a comprehension, you cannot create a local intermediate name without a helper function — until walrus. `[(compute(x), compute(x)**2) for x in nums]` runs `compute` twice per item. 2. Walk through the walrus form: `y := compute(x)` runs `compute` once and binds the result to `y`; then `y` is reused freely in the rest of the projection. 3. Stress when this matters: when `compute` is expensive (network call, DB query, heavy math). When it's cheap (a one-line transform), the duplication isn't worth a walrus. 4. Mention readability: some teams find walrus inside comprehensions hard to read at first. The two-line equivalent is `[(lambda y: (y, y*y))(compute(x)) for x in nums]` — uglier, or a helper function — clearer.

Edge cases to mention: - The walrus inside a comprehension leaks `y` into the enclosing scope — see Q24 for the exact semantics. - If the walrus is in the filter clause (`if (y := compute(x)) > 0`), it's only evaluated when the iteration reaches that item — short-circuiting still works. - You can use the same walrus name in both the filter and the projection: `[y for x in nums if (y := compute(x)) > 0]` filters AND uses the result.

What NOT to say: - "Walrus inside comp is always clearer" — depends; a helper function often beats it for non-trivial bodies. - "The walrus name is local to the comprehension" — it isn't; it leaks. Q24 covers this.

Common follow-up: "What does this print:

`[y for x in [1, 2, 3] if (y := x * 10) > 15]; print(y)` ?" — Prints the list `[20, 30]`, and `y` is `30` in the outer scope because the walrus leaks.

```
def compute(x):
    print(f"computing {x}")          # side-effect to count calls
    return x * 2

nums = [1, 2, 3]

# Without walrus – compute runs twice per item
without = [(compute(x), compute(x) ** 2) for x in nums]
# computing 1, computing 1, computing 2, computing 2, computing 3, computing 3

# With walrus – compute runs once per item
with_walrus = [(y := compute(x), y ** 2) for x in nums]
# computing 1, computing 2, computing 3
print(with_walrus)                  # [(2, 4), (4, 16), (6, 36)]

# Filter-and-project
results = [y for x in [1, 5, 10] if (y := x * 10) > 30]
print(results)                      # [50, 100]
```

Q23 · The Two-Line Equivalent — When Walrus Wins, When It Doesn't

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Walrus Operator

The question (interviewer's verbatim phrasing):

Rewrite this walrus comprehension as a two-line for-loop. Which version does your reviewer prefer?

The 30-second answer: Two-line form: `out = []; for x in nums: y = compute(x); out.append((y, y*y))`. Same output, more lines, clearer to a reviewer who hasn't seen walrus before. Reviewer choice depends on team norms: a 3.8+ codebase that uses walrus elsewhere accepts the comprehension; a mixed-experience team often prefers the for-loop. The walrus wins when the rest of the body is genuinely one expression; loses when you'd want more than one statement per item anyway.

Step-by-step thought process: 1. State the equivalence: walrus inside a comp is mechanically equivalent to a for-loop that assigns then projects. The compiler bytecode is broadly similar. 2. Identify the win condition for walrus: when the comprehension was already on one line, walrus saves the duplicate computation without expanding to three lines. 3. Identify the loss condition: when the body has logging, branching, or any reason to be multi-statement, the for-loop wins — you can't squeeze those into a comprehension anyway. 4. Team norms matter. Some 3.12 codebases ban walrus by style guide; some use it freely. Match the surrounding code.

Edge cases to mention: - The walrus name leaks; the for-loop variable also leaks. So the scope difference between the two forms is small — the comprehension's loop var doesn't leak but the walrus name does. - For multi-stage transforms (capture A, then B from A), walrus chains work: `[(b := f(a := compute(x)), b*2) for x in nums]`. Readability drops fast. - If the team standard is "no walrus in comprehensions," reviewers will ask for the for-loop regardless of merits.

What NOT to say: - "Walrus is always cleaner" — depends entirely on team and complexity. - "For-loop is always slower" — the bytecode difference is minimal; not a perf consideration here.

Common follow-up: "Show me a real production case where walrus in a comp earns its keep." — Reading until end-of-stream while accumulating: `chunks = [c for c in iter(lambda: (c := stream.read(8192)), b"")]` — captures and tests in one shot. Most uses outside this pattern can be rewritten.

```
def compute(x):
    return x * x + 1

nums = [3, 4, 5]

# Walrus form — one line of logic
walrus = [(y := compute(x), y - x) for x in nums]
print(walrus)                                # [(10, 7), (17, 13), (26, 21)]

# Two-line for-loop — what the reviewer often asks for
out = []
for x in nums:
    y = compute(x)
    out.append((y, y - x))
print(out)                                    # [(10, 7), (17, 13), (26, 21)]
# Both readable; pick by team norm
```

Q24 · Scope of the Walrus Name Inside a Comp — It Leaks

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Microsoft India, Goldman, Walmart Labs · Topic: Walrus Operator

The question (interviewer's verbatim phrasing):

The walrus name is supposed to be available in the comprehension. But after the comprehension finishes, can you still read it from the enclosing scope?

The 30-second answer: Yes. The walrus deliberately writes into the enclosing scope, not the comprehension's sealed scope. This is the one exception to Python 3's comprehension-has-its-own-scope rule. The walrus was designed this way precisely because the typical use is capturing an

intermediate that the outer code wants to see. After `[(y := f(x)) for x in lst]`, the variable `y` exists outside and holds the value from the last iteration.

Step-by-step thought process: 1. State the rule and the contradiction. Comprehension loop variables don't leak (Q6). Walrus names DO leak. The walrus was carved out as the explicit exception. 2. Show what value leaks: it's the last value assigned during the iteration. If the comprehension iterated zero times (empty source), the walrus name is never bound and reading it raises `NameError`. 3. Mention the PEP 572 design intent: the leak is deliberate because the most common use is "use the captured value later." If it were sealed, walrus inside comp would be useless for that. 4. Edge case: nested comprehensions and walrus interact in ways that surprised early adopters — the walrus writes to the enclosing function scope, not to any intermediate comprehension scope.

Edge cases to mention: - Empty source iteration → walrus name unbound → `NameError` on read. - Multiple walruses in one comprehension all leak, each to the last value assigned. - The walrus cannot be the comprehension's loop variable — `[x := i for i in range(5)]` works but `[x for (x := i) in ...]` doesn't.

What NOT to say: - "Walrus follows the same scoping as the loop var" — wrong; the loop var stays inside, the walrus leaks out. - "You can't read the walrus name outside" — you can, except when iteration produced zero items.

Common follow-up: "What's the value of `last` after `[last := x * 2 for x in range(3)]`?" — `4` (the value at `x=2`, the last iteration). If you'd written `range(0)`, `last` would be undefined and reading it raises `NameError`.

```
# Walrus name leaks into enclosing scope
result = [(y := x * 2) for x in [1, 2, 3]]
print(result)           # [2, 4, 6]
print(y)                # 6 – the LAST iteration's value

# Comprehension loop var does NOT leak
result = [x * 3 for x in [1, 2, 3]]
print(result)           # [3, 6, 9]
# print(x)              # NameError

# Empty source – walrus never binds
result = [(z := i) for i in []]
print(result)           # []
# print(z)              # NameError: name 'z' is not defined
```

Q25 · Walrus in the Filter Clause — Capture Once, Test Once, Project Once

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Microsoft India, Goldman · Topic: Walrus Operator

The question (interviewer's verbatim phrasing):

You're parsing log lines with a regex. You want every line where the regex matches, and you want the matched group, not the whole line. Write it.

The 30-second answer: `[m.group(1) for line in lines if (m := re.match(pattern, line))]`. The walrus inside the `if` clause runs the regex match once per line and binds the result to `m`. If the match is `None` (falsy), the line is dropped. If it matches, the projection uses `m.group(1)`. One pass over the lines, one regex call per line, no helper function.

Step-by-step thought process: 1. Open with the canonical use case: regex match in a comprehension. Without walrus, you'd either call `re.match` twice (once in the filter, once in the projection) or write a for-loop. Walrus collapses it. 2. Walk through the semantics. `if (m := re.match(...))` evaluates the match. A `Match` object is truthy, `None` is falsy. The `if` clause filters on that truthiness. The walrus name `m` is bound when truthy and available in the projection. 3. Stress that the walrus binds even on the falsy iterations — but the filter drops them, so the projection never runs for those. The leaked-after-loop value of `m` is from the last iteration that ran, which might be a falsy `None`. 4. Mention the regex idiom is the most-cited "walrus genuinely earns its place" case. Other uses are debatable; this one is hard to beat.

Edge cases to mention: - `re.match` returns `None` on no-match, which is falsy. Some other APIs return empty objects that look truthy — read the docs before relying on truthiness. - The same pattern works with `dict.get`: `[d.get("amount") for d in orders if (amount := d.get("amount")) is not None]` — capture once, test against `None` explicitly because zero is also falsy. - If you need both the match object AND the captured group, use it in the projection: `[m for line in lines if (m := re.match(pattern, line))]`.

What NOT to say: - "Walrus is just sugar" — for this specific use it's a real readability win, not just sugar. - "I'd call `re.match` twice" — that's the anti-pattern walrus exists to fix.

Common follow-up: "What if you want the *LINE NUMBER* too?" — `enumerate` the source: `[(i, m.group(1)) for i, line in enumerate(lines) if (m := re.match(pattern, line))]`.

```
import re

lines = [
    "ERROR 503 service unavailable",
    "INFO startup complete",
    "ERROR 401 unauthorized",
    "DEBUG cache hit",
    "ERROR 500 internal error",
]

# Capture-and-filter in one pass
errors = [
    m.group(1)
    for line in lines
    if (m := re.match(r"ERROR (\d{3})", line))
]
print(errors)                                # ['503', '401', '500']

# With dict.get – testing against None explicitly
orders = [{"amount": 100}, {"amount": 0}, {}, {"amount": 50}]
amounts = [
    amt
    for o in orders
    if (amt := o.get("amount")) is not None
]
print(amounts)                              # [100, 0, 50] – zero kept, missing dropped
```

Section E — Slicing Fundamentals

Q26 · [start:stop:step] — The Three-Argument Form

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Postman · Topic: Slicing

The question (interviewer's verbatim phrasing):

Walk me through Python's slice syntax. What do start, stop, and step each control?

The 30-second answer: `sequence[start:stop:step]` . `start` is the index of the first element (inclusive), `stop` is the index where slicing stops (exclusive), and `step` is the stride between selected elements. All three are optional and default to None, which the slicing logic translates to "from the beginning," "to the end," and "step of 1." Negative values count from the end for `start` and `stop` , and reverse direction for `step` .

Step-by-step thought process: 1. State the form. Three colon-separated integers inside square brackets. Order is start, stop, step. 2. Anchor on inclusivity. `start` is included; `stop` is excluded. This is the same convention as `range` — you're picking elements with index `i` where `start <= i < stop`. 3. Walk through defaults. `lst[:5]` is start=0, stop=5. `lst[2:]` is start=2, stop=len. `lst[::2]` is every other element. `lst[:]` is the full copy. 4. Cover negatives. `lst[-3:]` is the last three. `lst[:-2]` is everything except the last two. `lst[::-1]` reverses.

Edge cases to mention: - Out-of-range indices are silently clamped: `lst[10:]` on a 5-element list returns `[]`, not an error. This is how slice differs from index access. - `start >= stop` with positive step gives empty: `lst[5:2]` is `[]`. - `step` cannot be zero — `lst[::0]` raises `ValueError`.

What NOT to say: - "Stop is inclusive" — wrong; stop is exclusive. - "Negative step doesn't work" — it does; `[::-1]` is the canonical reverse.

Common follow-up: "What's the difference between `lst[2]` and `lst[2:3]`?" — `lst[2]` returns the element at index 2; `lst[2:3]` returns a one-element list containing that element. Index access returns the item; slicing returns a sub-sequence.

```
lst = [10, 20, 30, 40, 50, 60, 70, 80, 90]

# start, stop, step
print(lst[2:5])           # [30, 40, 50]
print(lst[::2])           # [10, 30, 50, 70, 90]
print(lst[1::2])          # [20, 40, 60, 80]
print(lst[::-1])          # [90, 80, 70, 60, 50, 40, 30, 20, 10]

# Negative indices
print(lst[-3:])           # [70, 80, 90]
print(lst[:-3])           # [10, 20, 30, 40, 50, 60]

# Out-of-range — silently clamped
print(lst[100:])          # []
print(lst[-100:3])        # [10, 20, 30]
```

Q27 · Negative Indices — Counting From the End

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Cred · Topic: Slicing

The question (interviewer's verbatim phrasing):

What does `s[-1]` return? What does `s[-3:]` return? Explain the rule.

The 30-second answer: `s[-1]` is the last element. `s[-3:]` is the last three elements. Negative indices count from the end: -1 is the last, -2 is second-to-last, and so on. The rule applies to both index access and the start/stop of a slice — negative stride values reverse the iteration direction.

Step-by-step thought process: 1. State the indexing rule. `-i` is equivalent to `len(s) - i`. So `s[-1] == s[len(s) - 1]`. The negative form is shorter and doesn't require computing the length. 2. Walk through slicing. `s[-3:]` is the last three. `s[:-3]` is everything except the last three. `s[-5:-2]` is from the fifth-to-last up to but not including the second-to-last. 3. Step rule. A negative step reverses iteration. `s[::-1]` is the full reverse. `s[5:2:-1]` walks indices 5, 4, 3. 4. The `len`-or-negative idiom: when you want to drop the trailing N items, `s[:-n]` is cleaner than `s[:len(s)-n]`.

Edge cases to mention: - Index `-1` on an empty list raises `IndexError`. Negative indices don't make empty containers safe. - Slicing tolerates out-of-range negatives: `s[-100:]` on a 5-element list returns the whole list, not an error. - Mixing positive and negative: `s[0:-1]` is everything except the last item. `s[-3:5]` works if `-3` and `5` resolve to a valid forward range.

What NOT to say: - "Negative indices error" — they don't on a non-empty sequence; they're a first-class Python feature. - "I'd convert to positive first" — unnecessary; the language handles it.

Common follow-up: "How do you get the last 3 elements of a list whose length you don't know?" — `lst[-3:]`. The negative-index slice handles short lists safely too: on a 2-element list it returns both elements, no error.

```
s = "razorpay"
lst = [10, 20, 30, 40, 50]

# Index access
print(s[-1])           # 'y'
print(s[-3])           # 'p'
print(lst[-2])         # 40

# Slicing
print(s[-3:])           # 'pay'
print(s[:-3])           # 'razor'
print(lst[-3:-1])       # [30, 40] — half-open from end

# Safe with short input
short = [1, 2]
print(short[-3:])       # [1, 2] — not an error
```

Q28 · Omitted Bounds — The Default Means "End"

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman, Walmart Labs · Topic: Slicing

The question (interviewer's verbatim phrasing):

What does `lst[:]` produce? What about `lst[2:]` and `lst[:2]` ?

The 30-second answer: `lst[:]` is a shallow copy of the whole list — start defaults to 0, stop defaults to length. `lst[2:]` is from index 2 to the end. `lst[:2]` is from start to index 2 exclusive — so the first two elements. Omitted means "default": 0 on the left, length on the right, 1 for step.

Step-by-step thought process: 1. State the rule. Each of start, stop, step has a default that the slice machinery substitutes when you omit it. None on the left becomes 0 (or len for negative steps), None on the right becomes len (or -len-1 for negative steps), step None becomes 1. 2. Walk through the three common forms. `lst[:]` is the copy idiom. `lst[n:]` is the "drop first n" idiom. `lst[:n]` is the "take first n" idiom. 3. Mention what these are NOT. `lst[:]` is not the same object as `lst` — it's a new list. `lst[2:]` doesn't include index 1. 4. Pivot to the negative-step case. `lst[::-1]` is start=len-1, stop=-len-1, step=-1 — the full reverse. The defaults swap meaning based on step direction.

Edge cases to mention: - `lst[:]` on a list is shallow — items are shared. Use `copy.deepcopy(lst)` if you need a deep copy. - `lst[::]` is identical to `lst[:]` — extra colons with no values are legal and harmless. - For strings and tuples, `s[:]` returns the same object — strings and tuples are immutable so the optimisation is safe.

What NOT to say: - "`lst[:]` returns the same list" — for mutable types (list), it returns a new object. For immutable (str, tuple), it can return the same object. - "Omitting both bounds is illegal" — it's the canonical copy idiom.

Common follow-up: "What's the shortest way to copy a list?" — `lst[:]`, `list(lst)`, `lst.copy()`. All three are equally idiomatic; pick by team style.

```
lst = [10, 20, 30, 40, 50]

print(lst[:])           # [10, 20, 30, 40, 50] — full shallow copy
print(lst[2:])          # [30, 40, 50]
print(lst[:2])          # [10, 20]
print(lst[::])          # [10, 20, 30, 40, 50] — same as [:]

# Is it the same object?
print(lst[:] is lst)    # False — new list
s = "hello"
print(s[:] is s)        # True — strings are immutable, optimised

# Reverse
print(lst[::-1])        # [50, 40, 30, 20, 10]
```

Q29 · Step Semantics — Every Nth, Reversal, and the Zero-Step Trap

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Swiggy, Cred, Microsoft India · Topic: Slicing

The question (interviewer's verbatim phrasing):

What does `lst[::-3]` return? What about `lst[::-2]` ? And what happens with `lst[:,0]` ?

The 30-second answer: `lst[::-3]` returns every third element starting from index 0. `lst[::-2]` walks the list in reverse, taking every second element from the end. `lst[:,0]` raises `ValueError("slice step cannot be zero")` — zero stride is meaningless. Positive step iterates left-to-right; negative step iterates right-to-left.

Step-by-step thought process: 1. State the rule. Step is the stride. A step of `k` skips `k-1` elements between each selected one. Positive `k` goes forward, negative `k` goes backward. 2. Walk through examples. `lst[:,2]` is every other (even indices). `lst[1::2]` is every other starting from index 1 (odd indices). `lst[:,3]` is every third. 3. Negative step reverses direction. `lst[::-1]` is reverse. `lst[::-2]` is reverse-and-every-other. The defaults for start/stop also flip — with negative step, omitted start means "from the end" and omitted stop means "to the start." 4. Step of zero is illegal. Python raises `ValueError` because there's no sensible interpretation — you'd walk in place forever.

Edge cases to mention: - `lst[5:1:-1]` walks indices 5, 4, 3, 2 — start is greater than stop in the negative-step direction. - `lst[5:1:1]` (positive step with start > stop) returns `[]` — empty, not an error. - Mixing negative step with negative indices works as expected: `lst[-1:-5:-1]` walks the last four elements in reverse.

What NOT to say: - "Step is the number of elements to take" — wrong; it's the stride between selected elements. - "Negative step means error" — wrong; it's the standard reverse idiom.

Common follow-up: "Reverse a string." — `s[::-1]`. The canonical Python idiom. `"".join(reversed(s))` also works but is longer.


```

lst = [10, 20, 30, 40, 50, 60, 70, 80, 90]

# Every nth
print(lst[::2])           # [10, 30, 50, 70, 90]
print(lst[::3])           # [10, 40, 70]
print(lst[1::3])          # [20, 50, 80]

# Reverse stride
print(lst[::-1])          # [90, 80, ..., 10]
print(lst[::-2])          # [90, 70, 50, 30, 10]
print(lst[5:1:-1])        # [60, 50, 40, 30]

# Zero step
try:
    bad = lst[::0]
except ValueError as e:
    print(e)               # slice step cannot be zero

```

Q30 · Reversing With `[::-1]` — The Idiom

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman, JP Morgan · Topic: Slicing

The question (interviewer's verbatim phrasing):

Three ways to reverse a list. Which one ships?

The 30-second answer: `lst[::-1]` returns a reversed copy. `list(reversed(lst))` does the same via the `reversed()` built-in. `lst.reverse()` mutates the original in place and returns `None`. Ship `lst[::-1]` when you want a reversed copy (the common case); ship `lst.reverse()` when you genuinely want to mutate and have no need for the unreversed list afterwards.

Step-by-step thought process: 1. List the three forms and their differences. Slice returns new list. `reversed()` returns an iterator. `.reverse()` mutates and returns `None`. 2. State the choice rule. For "I want a reversed copy," `lst[::-1]` is the idiom — short, clear, returns a list. 3. For "I want to iterate in reverse without building a list," `reversed(lst)` is the memory-cheap choice — it doesn't allocate, it walks the original in reverse. 4. For "I want to mutate in place," `lst.reverse()` is the right call — but the moment a colleague reads `lst.reverse()` and wonders if the original is preserved, you've lost a small readability battle.

Edge cases to mention: - `reversed()` works on any sequence with `__len__` and `__getitem__`, OR with `__reversed__`. Generators don't support it. - For strings, `s[::-1]` is

the only short option; `s.reverse()` doesn't exist (strings are immutable). - For numpy arrays, `arr[::-1]` returns a view, not a copy — covered in Q37.

What NOT to say: - "`lst.reverse()` returns the reversed list" — it returns None; the reversal is in place. - "Slicing is slower" — for in-memory lists, all three are O(n); slicing is fast and is the idiom.

Common follow-up: "Reverse a string." — `s[::-1]`. There's no `.reverse()` for strings because strings are immutable.

```
lst = [1, 2, 3, 4, 5]

# 1. Slice — returns new list
rev1 = lst[::-1]
print(rev1)                # [5, 4, 3, 2, 1]
print(lst)                 # [1, 2, 3, 4, 5] — unchanged

# 2. reversed() — returns iterator
rev2 = list(reversed(lst))
print(rev2)                # [5, 4, 3, 2, 1]
for x in reversed(lst): pass # no list allocated, walk in reverse

# 3. .reverse() — in place
lst.reverse()
print(lst)                 # [5, 4, 3, 2, 1] — mutated

# Strings
s = "razorpay"
print(s[::-1])             # 'yaprozar'
# s.reverse()              # AttributeError — str has no reverse
```

Q31 · Slice of Slice — Composition Works Without Pre-Allocation

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Walmart Labs, Goldman · Topic: Slicing

The question (interviewer's verbatim phrasing):

What does `lst[1:8][::2]` produce? Walk me through each step.

The 30-second answer: First slice: `lst[1:8]` takes indices 1 through 7 — that's seven elements. Second slice: `[::2]` walks every other of those seven — indices 0, 2, 4, 6 of the intermediate, which gives four elements. Slices compose left-to-right; each slice operates on the result of the previous one. The intermediate is a new list allocation — so `lst[1:8][::2]` allocates twice.

Step-by-step thought process: 1. Walk through left-to-right. The first slice `[1:8]` creates an intermediate list. The second slice `[::2]` operates on that intermediate, not on the original. 2.

Compute the result step by step. With `lst = list(range(10))`, `lst[1:8]` is `[1,2,3,4,5,6,7]`. `[1,2,3,4,5,6,7][::2]` is `[1,3,5,7]`. 3. Mention the cost. Each slice allocates a new list. `lst[a:b][c:d]` allocates twice — fine for small data, wasteful for large data. 4. The single-slice equivalent saves the allocation: `lst[1:8:2]` produces `[1,3,5,7]` directly, allocating once. Always reach for the single slice when you can.

Edge cases to mention: - The intermediate list is GC-collected once the outer expression finishes, so memory pressure is brief — but allocation is still a real cost in hot loops. - For numpy arrays, `arr[1:8][::2]` returns views, not copies — both slices are O(1) memory, and the composition is free. - For strings, `s[a:b][c:d]` allocates two new strings; same waste pattern as lists.

What NOT to say: - "Slices share memory by default" — Python lists don't; numpy arrays do. - "Composition is always faster than a single slice" — almost never; the single slice is cleaner and cheaper.

Common follow-up: "Rewrite `lst[1:8][::2]` as a single slice." — `lst[1:8:2]`. Same result, one allocation.

```
lst = list(range(10))      # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Composition — two allocations
step1 = lst[1:8]           # [1, 2, 3, 4, 5, 6, 7]
step2 = step1[::2]         # [1, 3, 5, 7]
print(lst[1:8][::2])       # [1, 3, 5, 7]

# Single slice — one allocation, same result
print(lst[1:8:2])          # [1, 3, 5, 7]

# Numpy contrast — both forms share memory
import numpy as np
a = np.arange(10)
print(a[1:8][::2])         # [1 3 5 7] — both slices are views, no copy
```

Q32 · Slicing Returns a New Object — Shallow Copy

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Microsoft India, JP Morgan · Topic: Slicing

The question (interviewer's verbatim phrasing):

If I do `a = [1, 2, 3]; b = a[:]`, is `b is a` ? What about the items inside?

The 30-second answer: `b is a` is `False` — slicing always allocates a new list object. But the items inside the new list are the same references as in the original — it's a shallow copy. So `b[0] is a[0]` is `True`. If the items themselves are mutable (a list of lists), modifying an inner item is visible through both `a` and `b`. For independent inner items, use `copy.deepcopy`.

Step-by-step thought process: 1. State the rule. Slicing produces a new container. The items are references, shared with the original. Outer object: independent. Inner objects: shared. 2. Walk through the consequence. `b.append(4)` doesn't affect `a` — the outer lists are independent. But `b[0].append("x")` on a list-of-lists DOES affect `a` — the inner list is shared. 3. Mention `copy.copy()` and `copy.deepcopy()`. Slicing equals `copy.copy()` for lists — same shallow semantics. `deepcopy` walks the tree and clones every object. 4. State when shallow is fine. When the items are immutable (ints, strings, tuples of immutables), shallow copy is functionally equivalent to deep copy because the items can't be mutated.

Edge cases to mention: - For tuples, slicing of a slice can return the same tuple (CPython optimisation) since tuples are immutable. Don't rely on it. - For strings, `s[:]` returns the same string (immutable optimisation). The "is it a copy?" question is irrelevant. - For a `list` of immutable items (`[1, 2, 3]`), shallow is enough — you can never mutate an int through aliasing.

What NOT to say: - "Slicing is a deep copy" — it isn't, and the gotcha bites when items are mutable. - "Slicing returns the same object" — it doesn't for mutable containers; it does for some immutables as an optimisation.

Common follow-up: *"Demonstrate the shallow-copy gotcha with a list of lists."* — `a = [[1], [2]]; b = a[:]; b[0].append(99); print(a)` prints `[[1, 99], [2]]` — the shared inner list mutated through both.

```
import copy

# Shallow case – items are immutable, no gotcha
a = [1, 2, 3]
b = a[:]
print(b is a)                # False – new list
print(b[0] is a[0])          # True – int is shared (also small-int cache)
b.append(4)
print(a, b)                  # [1, 2, 3] [1, 2, 3, 4]

# Shallow gotcha – items are mutable
a = [[1, 2], [3, 4]]
b = a[:]
print(b is a)                # False
print(b[0] is a[0])          # True – inner list shared
b[0].append(99)
print(a)                     # [[1, 2, 99], [3, 4]] – surprise

# Deep copy fixes
a = [[1, 2], [3, 4]]
b = copy.deepcopy(a)
b[0].append(99)
print(a)                     # [[1, 2], [3, 4]] – unaffected
```

Q33 · The `slice` Object — Slicing as a First-Class Citizen

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India, Goldman · Topic: Slicing

The question (interviewer's verbatim phrasing):

What is the relationship between `lst[2:5]` and `slice(2, 5)` ?

The 30-second answer: `lst[2:5]` is syntactic sugar for `lst[slice(2, 5)]`. The `slice` built-in creates a slice object that holds start, stop, step. Sequences pass it to `__getitem__`. Making the slice object first-class lets you store it in a variable, pass it around, and reuse it — useful when you have multiple parallel sequences you want to slice the same way.

Step-by-step thought process: 1. State the equivalence. `lst[a:b:c]` and `lst[slice(a, b, c)]` produce identical results. The colon syntax is just sugar over a `slice(...)` call. 2. Show why this matters. You can save a slice as `window = slice(10, 20)` and use it across multiple arrays. Common in numerical code where the same slice applies to many parallel arrays. 3. Mention `slice.indices(n)`. Given the length `n` of the sequence, this method returns concrete (start, stop, step) integers — useful when implementing `__getitem__` on your own class. 4. State the type. `slice` is a built-in; `slice` objects have `.start`, `.stop`, `.step` attributes. They are NOT iterable themselves — you use them by passing to `__getitem__`.

Edge cases to mention: - `slice` objects are hashable, so they can be dict keys — uncommon but legal. - `slice` has no truthiness override — even `slice(0, 0)` is truthy. - Custom classes accepting slices implement `__getitem__(self, key)` and check `isinstance(key, slice)` to detect the slice-vs-index case.

What NOT to say: - "The colon syntax is special; `slice()` can't replace it" — wrong; they're equivalent. - "Slice objects iterate" — they don't; they describe a slice, not produce elements.

Common follow-up: "Implement `__getitem__` on a class so it handles both `obj[5]` and `obj[2:8]`." — Detect `isinstance(key, slice)` and call `key.indices(len(self))` to normalize, then loop.

```

lst = [10, 20, 30, 40, 50, 60, 70, 80, 90]

# Sugar equivalence
print(lst[2:5])           # [30, 40, 50]
print(lst[slice(2, 5)])   # [30, 40, 50]
print(lst[2:8:2])         # [30, 50, 70]
print(lst[slice(2, 8, 2)]) # [30, 50, 70]

# Save and reuse
window = slice(1, 4)
a = ["a", "b", "c", "d", "e"]
b = [10, 20, 30, 40, 50]
print(a[window], b[window]) # ['b', 'c', 'd'] [20, 30, 40]

# slice.indices normalises against a length
s = slice(None, None, -1)
print(s.indices(5))        # (4, -1, -1) — concrete bounds for length 5

# Custom __getitem__
class Vec:
    def __init__(self, data): self.data = data
    def __getitem__(self, key):
        if isinstance(key, slice):
            return Vec(self.data[key])
        return self.data[key]

v = Vec([1, 2, 3, 4, 5])
print(v[1:4].data)         # [2, 3, 4]

```

Q34 · Slice Assignment — Mutating Via Slice

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Walmart Labs, Goldman · Topic: Slicing

The question (interviewer's verbatim phrasing):

What does `lst[2:5] = [99, 100]` do? How does the list length change?

The 30-second answer: Slice assignment replaces the items at indices 2 through 4 with the right-hand-side iterable. Original `lst[2:5]` was three items; replaced with `[99, 100]` (two items). So the list shrinks by one. The replacement length doesn't have to match the slice length — you can replace 3 items with 5 items, with zero items (delete), or with the same count.

Step-by-step thought process: 1. State the mechanism. Slice assignment is `lst.__setitem__(slice(2, 5), [99, 100])`. It removes the old items and inserts the new ones. The list's length changes by `len(new) - len(old)`. 2. Walk through the four cases. Same

length → in-place replace. New shorter → list shrinks. New longer → list grows. New empty (`lst[2:5] = []`) → delete the slice, equivalent to `del lst[2:5]`. 3. Mention the step-restricted form. With a step (`lst[::2] = ...`), the replacement length MUST match the slice length — otherwise `ValueError`. The interpreter enforces it because step-skipping has no obvious way to grow or shrink. 4. Pivot to use cases. Slice assignment is the bulk-replace idiom. Avoids loops, faster than repeated insert/remove, and is the right tool when "I want indices 5 through 10 to become these values."

Edge cases to mention: - `lst[2:5] = "abc"` works because strings are iterable — assigns `['a', 'b', 'c']` to the slice. Possibly surprising. - `lst[2:2] = [99, 100]` inserts at index 2 without removing anything — pure insertion. - `del lst[2:5]` is equivalent to `lst[2:5] = []`.

What NOT to say: - "Slice assignment needs matching length" — only with a step. Default step allows arbitrary length. - "It mutates in place but doesn't change length" — it does change length when the new and old slice lengths differ.

Common follow-up: "Delete every other element of a list." — `del lst[::2]`. The step-restricted form deletes the selected positions without growing.

```
lst = [10, 20, 30, 40, 50]

# Same length – in-place replace
lst[1:3] = [200, 300]
print(lst)                    # [10, 200, 300, 40, 50]

# Shrink
lst = [10, 20, 30, 40, 50]
lst[1:4] = [99]
print(lst)                    # [10, 99, 50]

# Grow
lst = [10, 20, 30, 40, 50]
lst[1:2] = [21, 22, 23, 24]
print(lst)                    # [10, 21, 22, 23, 24, 30, 40, 50]

# Insert at position
lst = [1, 2, 3]
lst[1:1] = [10, 11]
print(lst)                    # [1, 10, 11, 2, 3]

# Step form must match length
lst = [1, 2, 3, 4, 5, 6]
lst[::2] = [10, 20, 30]
print(lst)                    # [10, 2, 20, 4, 30, 6]
try:
    lst[::2] = [99]
except ValueError as e:
    print(e)                  # attempt to assign sequence of size 1 to extended slice of
```

Q35 · Slicing on Strings, Tuples, and Bytes — Same Syntax, Same Rules

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Swiggy, Postman · Topic: Slicing

The question (interviewer's verbatim phrasing):

Does slicing work on strings and tuples the same way as on lists? What about bytes?

The 30-second answer: Yes — the `[start:stop:step]` syntax is part of the sequence protocol and works identically on strings, tuples, and bytes. The result type matches the input type — slicing a string gives a string, slicing a tuple gives a tuple, slicing bytes gives bytes. They're all immutable, so slice assignment doesn't work — `s[2:5] = "xyz"` raises `TypeError`.

Step-by-step thought process: 1. State the unified rule. All built-in sequence types — `list`, `str`, `tuple`, `bytes`, `bytearray`, `range` — support the same slice syntax via the sequence protocol. 2. State the difference. The result type matches the input. Slicing a `str` returns a `str`. Slicing a `tuple` returns a `tuple`. Slicing `bytes` returns `bytes`. Slicing `range` returns another `range` (lazy). 3. State the mutability divide. List and bytearray are mutable — slice assignment works. Str, tuple, bytes are immutable — slice assignment raises `TypeError`. 4. Mention the perf note. Slicing a string is $O(n)$ in the size of the slice because Python copies the underlying buffer. There's no string-view type built in — covered in Q39.

Edge cases to mention: - `range[1:5]` returns another `range` object — `range(1, 5)` — without materialising integers. Lazy slicing. - `bytes` and `bytearray` slicing returns the same type, so `b"hello"[1:4]` is `bytes`, while `bytearray(b"hello")[1:4]` is `bytearray`. - Custom subclasses of `list` / `str` — slicing returns the BASE type unless you override `__getitem__`. Subclassing `list` and slicing returns a plain `list`.

What NOT to say: - "Slicing works only on lists" — wrong; it works on every sequence type. - "Slicing a tuple returns a list" — wrong; the result type matches the input.

Common follow-up: "Why doesn't `s[1:3] = 'xx'` work?" — Strings are immutable. The only way to "change" a string is to build a new one: `s = s[:1] + 'xx' + s[3:]`.


```
# Strings
s = "razorpay"
print(s[2:5])           # 'zor' – str
print(type(s[2:5]))     # <class 'str'>
# s[2:5] = "xxx"        # TypeError – str doesn't support item assignment

# Tuples
t = (10, 20, 30, 40, 50)
print(t[1:4])           # (20, 30, 40) – tuple
print(type(t[1:4]))     # <class 'tuple'>

# Bytes vs bytearray
b = b"razorpay"
print(b[2:5])           # b'zor' – bytes (immutable)
ba = bytearray(b"razorpay")
ba[2:5] = b"XX"
print(ba)               # bytearray(b'raXXpay') – mutable

# range is lazy
r = range(10)
print(r[2:8])           # range(2, 8) – not materialised
print(list(r[2:8:2]))    # [2, 4, 6]
```

Section F — Slicing Gotchas

Q36 · List Slice Creates a Copy — The Memory Cost

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Walmart Labs, JP Morgan · Topic: Slicing Gotchas

The question (interviewer's verbatim phrasing):

You have a list of 100 million ints and you write `chunk = big[1_000_000:1_001_000]` .
How much memory did you just allocate?

The 30-second answer: You allocated a new list with 1,000 PyObject pointers — about 8 KB of pointers plus list overhead. The ints themselves are NOT copied; both lists hold references to the same int objects. So the allocation is small in absolute terms, but it's a real allocation — slicing a Python list never returns a view. If you slice a million times in a hot loop, the GC pressure adds up.

Step-by-step thought process: 1. State the rule. Python list slicing is `O(k)` in the slice size and always allocates a new list object. No view, no shared storage at the list level. 2. Quantify. Each element of a Python list is a PyObject pointer — 8 bytes on 64-bit Python. A 1,000-element slice is 8 KB of

pointer storage. The int objects themselves stay shared. 3. Compare with numpy.

`numpy_arr[start:stop]` returns a view — zero allocation, the slice shares the underlying buffer with the original. That's why numpy is preferred for numerical slicing. 4. Mention `memoryview` for bytes. `memoryview(b)[start:stop]` is a view over the bytes, not a copy. The closest Python comes to a built-in view type.

Edge cases to mention: - The shared int objects don't really cost extra memory because of small-int caching for -5 through 256, but for floats and big ints, the items are reference-shared, not duplicated. - `bytearray` slicing also copies (returns a new bytearray). `memoryview(bytearray_obj)[a:b]` views without copying. - `array.array` slicing copies too — no view semantics in the standard library outside numpy and memoryview.

What NOT to say: - "Slicing is free" — it allocates `O(k)` and writes pointers. - "The items are duplicated" — they're reference-shared; only the pointer storage is new.

Common follow-up: "How would you iterate a window of a list without copying?" — Use indices (`for i in range(start, stop): use(lst[i])`) or use `itertools.islice` which doesn't materialise. Or move to numpy.

```
import sys

big = list(range(1_000_000))

# Slice allocates a new list
chunk = big[100_000:101_000]
print(sys.getsizeof(chunk))          # ~8,056 bytes – 1000 pointers + overhead
print(chunk[0] is big[100_000])      # True – same int object referenced

# itertools.islice – iterator, no materialisation
from itertools import islice
total = sum(islice(big, 100_000, 101_000))
print(total)                        # 100499500 – never built a list

# Numpy – slice is a view
import numpy as np
arr = np.arange(1_000_000)
view = arr[100_000:101_000]
print(view.base is arr)              # True – shares memory
print(view.nbytes)                   # 8000 bytes BUT 0 new allocation
```

Q37 · Numpy Slicing Returns a View — Not a Copy

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Flipkart, Walmart Labs, Goldman · Topic: Slicing Gotchas (Numpy)

The question (interviewer's verbatim phrasing):

```
a = np.arange(10); b = a[2:5]; b[0] = 999; print(a)
```

 . What does it print and why?

The 30-second answer: Prints `[0 1 999 3 4 5 6 7 8 9]` . The slice `a[2:5]` returns a VIEW into `a` , not a copy — `b` is a window over the same underlying buffer. Writing through `b` writes through to `a` . Standard numpy behaviour, and it's the single biggest gotcha for Python developers coming from list semantics. To force a copy, write `b = a[2:5].copy()` .

Step-by-step thought process: 1. State the core difference. Python list slicing copies; numpy slicing views. This is a deliberate design choice — numpy operates on large arrays where copying every slice would kill performance. 2. Show the consequence. Modifying the view modifies the original. The view has `.base` set to the original array; the Python list slice has no such concept. 3. State the copy idioms. `arr.copy()` , `np.copy(arr)` , `arr[2:5].copy()` , `arr[mask]` (fancy indexing returns a copy not a view). Fancy indexing breaks the view rule and always copies. 4. Mention pandas inheritance. Pandas Series and DataFrames built on numpy can return views or copies depending on the operation — the source of the famous `SettingWithCopyWarning`. Covered in Q38.

Edge cases to mention: - `.copy()` makes a fully independent copy, breaking the link. - Basic indexing (slices) returns views; advanced indexing (boolean mask, integer array) returns copies. - You can check with `view.base is arr` — if True, it's a view; if None, it's a copy.

What NOT to say: - "Numpy slicing is the same as list slicing" — fundamentally different on the view vs copy axis. - "Fancy indexing is faster because it's a view" — wrong; fancy indexing is a copy, basic slicing is a view.

Common follow-up: "How do you check whether a numpy array is a view or a copy?" — Check `.base` . If it's not None, the array is a view into that base. If it's None, the array owns its data.

```

import numpy as np

a = np.arange(10)
b = a[2:5]
print(b)                                # [2 3 4]
b[0] = 999
print(a)                                # [ 0  1 999  3  4  5  6  7  8  9]
print(b.base is a)                      # True — view

# Copy to break the link
c = a[2:5].copy()
c[0] = -1
print(a)                                # unchanged (still has 999 at index 2)
print(c.base)                           # None — owns its data

# Fancy indexing always copies
mask = a > 5
filt = a[mask]
filt[0] = -100
print(a)                                # unchanged — fancy indexing copied
print(filt.base)                        # None

```

Q38 · Pandas `.iloc` vs `.loc` Slicing — Half-Open vs Closed

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Flipkart, Walmart Labs, Goldman · Topic: Slicing Gotchas (Pandas)

The question (interviewer's verbatim phrasing):

What's the difference between `df.iloc[2:5]` and `df.loc[2:5]` on a DataFrame whose index is the default 0, 1, 2, 3, ...?

The 30-second answer: `df.iloc[2:5]` uses positional slicing with Python's half-open convention — rows at positions 2, 3, 4. `df.loc[2:5]` uses label-based slicing with a CLOSED interval — rows whose label is 2, 3, 4, AND 5. Even when the labels happen to be the default integers, the slicing rule differs by one row. This is the single most common pandas trip-up in interviews.

Step-by-step thought process: 1. State the rule. `.iloc` follows Python: stop is exclusive. `.loc` follows label semantics: stop is INCLUSIVE. Pandas designed it this way because label slicing without inclusivity is rarely what you want — "from June to December" should include December. 2. Show the off-by-one. On a default-integer-index DataFrame, `df.iloc[2:5]` returns 3 rows; `df.loc[2:5]` returns 4 rows. Same numbers, different counts. 3. Discuss when each is right. `.iloc` for "rows at these positions, regardless of label." `.loc` for "rows with these labels, regardless of position." If your index is dates, `.loc` is the natural reach because you think in date ranges. 4. Mention the chained-

indexing trap: `df[df.x > 0]['y'] = 99` triggers `SettingWithCopyWarning` because the boolean mask returns a copy, and the assignment lands on the copy. Use `df.loc[df.x > 0, 'y'] = 99` instead.

Edge cases to mention: - For a DataFrame with non-monotonic labels, `.loc[2:5]` can raise — sorted-or-unique requirement applies for slicing. - `.loc[:, ['a', 'b']]` selects columns by label. Mixing row and column slices is the standard 2D form. - Boolean masking with `.loc` returns a writable view-or-copy depending on pandas' copy-on-write mode (pandas 2.x changed this; see File 03 for the 2026 update).

What NOT to say: - "`.iloc` and `.loc` are the same when labels equal positions" — they're not; the stop-inclusive rule still differs. - "Always use `.iloc`" — when your index is meaningful (dates, IDs), `.loc` is the right tool.

Common follow-up: "What does `df.iloc[2:5, 1:3]` return?" — Three rows by two columns at positions [2,3,4] and [1,2]. The 2D slice applies the same half-open rule to both axes.

```
import pandas as pd

df = pd.DataFrame({"x": [10, 20, 30, 40, 50, 60, 70]})
# Index is default 0..6

# .iloc — positional, half-open
print(df.iloc[2:5])
#    x
# 2 30
# 3 40
# 4 50      # 3 rows

# .loc — label-based, CLOSED
print(df.loc[2:5])
#    x
# 2 30
# 3 40
# 4 50
# 5 60      # 4 rows — includes stop label

# Date index makes the closed semantics natural
df2 = pd.DataFrame({"sales": [100, 200, 300]},
                    index=pd.to_datetime(["2026-01", "2026-02", "2026-03"]))
print(df2.loc["2026-01":"2026-02"]) # includes Feb
```

Q39 · String Slicing Is O(n) — No Built-In String View

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India, Goldman · Topic: Slicing Gotchas (Strings)

The question (interviewer's verbatim phrasing):

What's the time complexity of `s[1000:2000]` on a long string? Why?

The 30-second answer: `O(k)` in the slice length — Python copies the underlying character buffer into a new string object. There's no built-in string-view type, so every slice is a real copy. For a string of millions of characters, slicing repeatedly in a loop dominates runtime. The workaround for byte-level processing is `memoryview(b'...')`, which is a view over bytes — but no analogous type exists for `str`.

Step-by-step thought process: 1. State the rule. CPython strings are immutable and stored as compact character buffers. Slicing creates a new string with a fresh buffer of the right size and copies the relevant chars. 2. Quantify. For a 1MB string, slicing a 100KB chunk allocates 100KB and copies 100KB. That's fast in absolute terms but expensive in a tight loop over many chunks. 3. State the workaround. For text, there's no built-in view. For bytes, `memoryview` gives you a zero-copy view: `mv = memoryview(big_bytes); chunk = mv[1000:2000]` is `O(1)`. 4. Mention third-party tools. Libraries like `pyahocorasick` or sliding-window text processors avoid slicing entirely by using indices into the original buffer. For Python-only code, prefer iterating with start/end indices and calling `.startswith`, `.find` directly on the source string.

Edge cases to mention: - A slice of length zero (`s[5:5]`) still allocates a new (empty) string. Tiny cost, but it's not free. - Unicode strings in CPython 3.12 use compact storage (PEP 393) — ASCII strings are 1 byte per char, etc. Slice cost still scales linearly. - For repeated substring extraction from a long string, consider `str.find(pat, start)` to locate the position and `s[start:end]` only once with the final bounds.

What NOT to say: - "String slicing is `O(1)`" — wrong; it's `O(k)` because of the copy. - "Use `memoryview` on a string" — `memoryview` works on bytes, not `str`; you'd need to encode first.

Common follow-up: "How would you efficiently extract many overlapping windows of a long string?" — Iterate with positional indices, calling APIs that take start/end (`re.search`, `str.find`) instead of slicing into temporaries.

```
import sys

s = "x" * 1_000_000

# Each slice allocates and copies
import time
t0 = time.perf_counter()
for _ in range(1000):
    chunk = s[100_000:200_000]      # allocates 100KB each time
    print(f"slice: {time.perf_counter() - t0:.3f}s")

# memoryview — only works on bytes
b = s.encode()
mv = memoryview(b)
t0 = time.perf_counter()
for _ in range(1000):
    chunk = mv[100_000:200_000]      # 0(1), no copy
    print(f"memoryview: {time.perf_counter() - t0:.3f}s")
# memoryview is dramatically faster for byte-level processing
```

Q40 · Dict and Set Don't Support Slicing — And Why

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Microsoft India · Topic: Slicing Gotchas (Containers)

The question (interviewer's verbatim phrasing):

`d = {"a": 1, "b": 2, "c": 3}`. What does `d["a":"c"]` give you?

The 30-second answer: `TypeError: unhashable type: 'slice'`. Dicts and sets don't support slicing because they're keyed by hash, not position — there is no "between A and C" order semantics in a dict. The error is that the dict's `__getitem__` tries to use the `slice(...)` object as a hash key, and slice objects aren't hashable. If you need ordered slicing, convert to a list of (key, value) pairs first.

Step-by-step thought process: 1. State the rule. `dict` and `set` are hash tables. Slicing assumes a defined order between positions — dicts/sets don't expose one as a slicing dimension. 2. State the runtime behaviour. `d["a":"c"]` tries to look up the slice object as a key — slice isn't hashable, so `TypeError`. Different error message than "unsupported operation," but same root cause. 3. State the workaround. For dicts, `list(d.items())[2:5]` slices the ordered list of items (Python 3.7+ preserves insertion order). For sets, you'd convert to a sorted list first — and at that point you've left set semantics. 4. Mention the design alternative. If you need order-AND-fast-lookup, use `collections.OrderedDict` (Python 3.7+ regular dict is already ordered) or sort the keys and pair with a separate dict for lookups.

Edge cases to mention: - Python 3.7+ regular dicts preserve insertion order — so `list(d.items())` `[start:stop]` gives you a meaningful slice. - `collections.OrderedDict` has `.popitem(last=...)` for ordered removal but no slicing — still need to convert. - For sorted dicts, `sortedcontainers.SortedDict` (third-party) supports indexed access, but it's not in the standard library.

What NOT to say: - "Dicts are ordered so they should support slicing" — order is preserved at the dict level, but slicing requires positional indexing too. - "Sets are slowly indexable" — they aren't indexable at all; they don't expose a position.

Common follow-up: "How would you get the first 3 entries of a dict?" — `dict(list(d.items())[:3])` or `{k: d[k] for k in list(d)[:3]}` — both convert keys to a list first to fall back on list slicing.

```
d = {"a": 1, "b": 2, "c": 3, "d": 4, "e": 5}

# Direct slicing fails
try:
    _ = d["a":"c"]
except TypeError as e:
    print(e)                                # unhashable type: 'slice'

# Convert to ordered list first
items = list(d.items())
print(items[1:4])                           # [('b', 2), ('c', 3), ('d', 4)]
print(dict(items[1:4]))                     # {'b': 2, 'c': 3, 'd': 4}

# Common idiom — first N keys
first_three = {k: d[k] for k in list(d)[:3]}
print(first_three)                         # {'a': 1, 'b': 2, 'c': 3}

# Sets — no slicing, no order
s = {3, 1, 4, 1, 5, 9, 2, 6}
try:
    _ = s[0:2]
except TypeError as e:
    print(e)                                # 'set' object is not subscriptable

# If you need order, sort first
print(sorted(s)[:3])                       # [1, 2, 3]
```

Wrap-up — How to Carry This Into the Interview

Comprehensions and slicing reward candidates who read them out loud. When you're whiteboarding, narrate the comprehension as a sentence ("for every order in the list, if it's paid, take the amount") and the panel can hear you write Python the way Python is written. When the question is slicing, say the

convention explicitly — "half-open on the right" for lists and numpy, "closed on the right" for pandas `.loc` — and the interviewer will hear that you've been bitten by the off-by-one before.

Three things that separate the prepared from the confident in this chapter. One, the view-vs-copy axis: list slice copies, numpy slice views, string slice copies, memoryview views, pandas depends on the operation. Two, the scoping carve-out for the walrus operator: it's the one expression that writes through a comprehension's sealed scope, and PEP 572 designed it that way for exactly the use case where you want the captured value. Three, the empty-set literal: `{}` is a dict, `set()` is the empty set, and a candidate who writes `if x == {}` to test for an empty set has just told the panel they've never run this code.

Carry the four idioms into the room: `lst[::-1]` to reverse, `lst[:]` to shallow-copy, `dict(zip(keys, values))` to build a lookup, `{e.lower() for e in lst}` to transform-and-dedup. If a panel asks for any of these, the one-liner is the answer; the for-loop is the consolation prize. And if they push back on the comprehension you wrote, the prepared response is "happy to expand it to a for-loop — here's where the readability turns" rather than defending the one-liner reflexively. Reviewers like candidates who can argue both sides.

Chapter 8 — Functions + Closures (50 Qs)

Python Interview Prep — 2026 Edition · Learn & Excel · File 04 Core Library

Functions are the first-class objects of Python — they carry attributes, get passed around, get returned, and capture state via closures. This chapter is the one where interviewers tell who has read the data model and who has only written `def` statements. Indian product cos (Razorpay, Swiggy, Cred, Zerodha, PhonePe, Flipkart) lean on closures and late-binding traps in round 1 phone screens. Service MNCs (TCS, Infosys, Wipro, Accenture, Cognizant) hammer `*args` / `**kwargs`, default arguments, and the mutable-default trap. GCC panels (Walmart Labs, JP Morgan, Goldman Sachs, Target India, Microsoft India) prefer `functools` mastery and introspection — `inspect.signature`, `partial`, `singledispatch`, `wraps`-preserving decorators.

Every Q below uses the Step-by-Step Answer format: tags, the interviewer's verbatim phrasing, a 30-second answer, the layered reasoning, edge cases to mention, what NOT to say, and the common follow-up. Code compiles on Python 3.12+ unless explicitly marked.

Table of Sections

- Section A — First-class functions (Q1–Q10)
- Section B — Closures (Q11–Q20)
- Section C — Function arguments (Q21–Q30)

- Section D — Function introspection (Q31–Q35)
- Section E — `functools` deep (Q36–Q40)
- Section F — Higher-order functions (Q41–Q50)

Section A — First-class functions (10 Qs)

Q1 · What "functions are first-class objects" actually means

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant, Accenture · Topic: First-class functions

The question (interviewer's verbatim phrasing):

What do people mean when they say Python functions are first-class objects? Give me three things you can do with a function that you can do with an int.

The 30-second answer: First-class means a function is an ordinary value in the language. You can bind it to a name, store it in a list or dict, pass it as an argument, return it from another function, and read and write attributes on it. The function object lives in memory just like an int or a list — `def f(): ...` is mostly sugar for "create a function object, bind the name `f` to it."

Step-by-step thought process: 1. Open with the value-not-keyword framing: `def` is an executable statement, not a compile-time directive. It creates an object of type `function` and binds a name to it. 2. List the three concrete operations interviewers want to hear — pass, return, store — and add a fourth they don't ask but love: attach attributes (`f.retry_count = 3`). 3. Mention `type(f)` is `<class 'function'>`, and `callable(f)` is `True`. Both are how you'd verify it at runtime. 4. Close on the implication: this is what makes decorators, callbacks, and higher-order functions possible without any special syntax — they're just normal value flow.

Edge cases to mention: - Built-in functions (`len`, `print`) are technically `builtin_function_or_method`, not `function`, but they're equally first-class — `callable(len)` is `True`. - Methods are also first-class: `obj.method` is a bound-method object you can store and call later. - `lambda` expressions create the exact same `function` type as `def` — only the `__name__` differs (`<lambda>` vs the bound name).

What NOT to say: - "Functions in Python work like in C" — C has function pointers, but they aren't first-class objects with attributes and dunder. - "First-class means the function runs first" — wrong axis; "first-class" is a values-vs-syntax distinction, nothing to do with order of execution.

Common follow-up: "If functions are objects, what's the difference between `f` and `f()`?" — `f` is the function object itself; `f()` is the result of invoking it. Tip them once: forgetting the parens is the

single most common mistake when passing a callback (`thread.start(target=my_handler)`) versus the wrong `thread.start(target=my_handler())`).

```
def greet(name):
    return f"hello, {name}"

# Bind to other names
g = greet
print(g("Razorpay"))           # hello, Razorpay

# Store in a dict
handlers = {"greet": greet, "shout": str.upper}
print(handlers["greet"]("Swigg")) # hello, Swigg

# Attach attributes
greet.retries = 3
print(greet.retries)           # 3
```

Q2 · Passing a function as an argument

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swigg · Topic: First-class functions

The question (interviewer's verbatim phrasing):

Write a function `apply(func, value)` that takes any callable and a value, and returns `func(value)`. Then call it three different ways — with a `def`, a `lambda`, and a built-in.

The 30-second answer: Declare the parameter as you would any other name, then call it with `()`. Python does not require any "function pointer" syntax — if the object is callable, the parens invoke it. Type hints are optional but make intent explicit: `Callable[[int], int]` from `typing`.

Step-by-step thought process: 1. Show the minimal definition — no special declaration, just call the parameter. This is the bedrock of the callback pattern. 2. Demonstrate three call sites: a named function (`square`), a lambda (`lambda x: x + 1`), and a built-in (`abs`). All three pass through identically. 3. Mention the type hint path: `from typing import Callable; def apply(func: Callable[[int], int], value: int) -> int`. Helps editors and `mypy`. 4. Mention the duck-typing reality: `apply` doesn't care if `func` is a function, a method, a `functools.partial`, or any object with `__call__`. If it's callable, it works.

Edge cases to mention: - A bound method works too: `apply(str.upper, "tcs") -> "TCS"`. Methods are first-class. - Forgetting the `()` is the classic mistake — `apply(square(5), 3)` calls `square(5)` first and tries to call `25(3)`, raising `TypeError: 'int' object is not callable`. - Passing a generator-builder versus the generator itself is a common confusion —

`apply(my_gen, value)` calls `my_gen(value)`, which gives back a generator (and you usually wanted the values).

What NOT to say: - "I need a special decorator to pass a function" — no, that's a different pattern; passing a function is just normal value passing. - "Python doesn't have function pointers" — technically right but misleading; the equivalent (and more) is first-class function objects.

Common follow-up: "What if `func` raises an exception — should `apply` catch it?" — Usually no. A general-purpose passer should let exceptions propagate; the caller knows their semantics. Catch only if you're building a retry or a specific error-handling utility.

```
def apply(func, value):
    return func(value)

def square(x):
    return x * x

print(apply(square, 5))           # 25
print(apply(lambda x: x + 1, 10)) # 11
print(apply(abs, -7))             # 7
print(apply(str.upper, "razorpay")) # RAZORPAY
```

Q3 · Returning a function from a function

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, PhonePe, Postman · Topic: First-class functions

The question (interviewer's verbatim phrasing):

Write a `make_multiplier(n)` that returns a function which multiplies its argument by `n`. Then explain how `multiplier3 = make_multiplier(3)` works internally.

The 30-second answer: Define an inner function inside `make_multiplier`, then `return` it (without parens — you're returning the object, not its result). The inner function captures `n` from the enclosing scope; the returned object remembers `n` even after `make_multiplier` has exited. That's a closure, and it's the cornerstone of decorators, factories, and currying.

Step-by-step thought process: 1. Show the canonical shape: an outer function with a parameter, an inner function that uses the parameter, and `return inner` at the end of the outer. 2. Walk through what happens at `make_multiplier(3)`: outer runs, binds `n=3` in its local frame, creates the inner function object, attaches that frame's `n` to the inner's `__closure__`, returns inner. 3. Mention the lifetime: the outer frame would normally get garbage-collected when outer returns. The closure keeps a

reference, so `n=3` survives. 4. Tip the listener that every decorator you've written is this exact pattern — the "function returning a function" sentence is the foundation.

Edge cases to mention: - `make_multiplier(3)` and `make_multiplier(3)` create two distinct function objects, each with their own captured `n` — even if the values are equal, identity is not. - The inner function can be a `def` or a `lambda` — `return lambda x: x * n` works identically. - If the outer parameter is mutable (a list, say) and the inner mutates it, every call sees the mutations — closures share the reference, not the value.

What NOT to say: - "I need to use `nonlocal` for this to work" — no, reading `n` from the enclosing scope works directly. `nonlocal` is only required when you want to rebind it. - "The function copies `n`" — it doesn't; it captures the binding (cell). Reads see the current value of that cell.

Common follow-up: "What does `multiplier3.__closure__` contain?" — A tuple of one cell holding the value `3`. Read it with `multiplier3.__closure__[0].cell_contents`. That's the inspection lever you reach for when a closure misbehaves.

```
def make_multiplier(n):
    def multiplier(x):
        return x * n
    return multiplier

mult3 = make_multiplier(3)
mult5 = make_multiplier(5)
print(mult3(10))           # 30
print(mult5(10))           # 50
print(mult3.__closure__[0].cell_contents) # 3
```

Q4 · `def` vs `lambda` — when to pick each

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, JP Morgan, Walmart Labs · Topic: First-class functions

The question (interviewer's verbatim phrasing):

When would you use a `lambda` instead of a `def`, and when would you not? Give me a clear rule of thumb.

The 30-second answer: Use `lambda` for tiny one-expression callables passed inline as arguments — `sorted(items, key=lambda x: x.price)` or `max(orders, key=lambda o: o.amount)`. Use `def` for anything that needs more than one expression, a docstring, a `__name__` you'll see in logs, a `return` statement that isn't trivial, or any reuse. Lambdas are throwaway; `def` is named.

Step-by-step thought process: 1. Open with the syntactic rule: a `lambda` body is a single expression — no statements, no `return`, no docstrings, no annotations on the body. If you need any of those, you need `def`. 2. State the readability rule: if the lambda is more than ~40 characters or has nested ternaries, switch to `def`. Lambdas in long form are write-only code. 3. Mention the debugging cost: `lambda` objects have `__name__ == '<lambda>'`, which is useless in tracebacks. `def` gives you the function name in the stack trace. 4. Style references — PEP 8 doesn't ban lambdas but says don't assign one to a name. `square = lambda x: x*x` should be `def square(x): return x*x` — same length, better introspection.

Edge cases to mention: - `lambda` cannot have annotations on its parameters (no `lambda x: int = 5`). Use `def` for typed callables. - `lambda` captures variables the same way as `def` — both have late-binding loops (see Q15). - Inside a `key=` argument, lambdas are idiomatic and good — `key=lambda r: (r.priority, r.created_at)` reads cleanly.

What NOT to say: - "Lambdas are faster than `def`" — false; they generate identical bytecode for equivalent bodies. - "Lambdas are functional, `def` is imperative" — they're the same thing. The functional flavour comes from how you use them, not which keyword you wrote.

Common follow-up: "Why does PEP 8 say not to assign a lambda to a name?" — Because you lose the function name in tracebacks and the only marginal saving (one line) isn't worth that loss. The official linter rule (`E731`) flags it.

```
# Good — lambda inline as a key
orders = [{"id": 1, "amount": 500}, {"id": 2, "amount": 100}]
top = max(orders, key=lambda o: o["amount"])
print(top)                                # {'id': 1, 'amount': 500}

# Bad — lambda assigned to a name (linter E731)
# square = lambda x: x * x                # write this as def instead

# Good — def for anything beyond one expression
def order_score(order):
    """Higher priority + older orders score higher."""
    return order["priority"] * 10 + order["age_days"]
```

Q5 · `functools.partial` — what it actually does

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Goldman Sachs, JP Morgan · Topic: First-class functions

The question (interviewer's verbatim phrasing):

What does `functools.partial` do? Show me how you would use it to turn `requests.get(url, timeout=2, headers=...)` into a `safe_get(url)` you can reuse.

The 30-second answer: `partial(func, *args, **kwargs)` returns a new callable with some arguments pre-bound. Call the result with the remaining arguments and you get the final function call. It's how you adapt a function with too many parameters into something shorter that fits a callback slot — without writing a wrapper `def` or a `lambda`.

Step-by-step thought process: 1. Open with the mental model: `partial(f, a, b)` is the same as `lambda *args, **kwargs: f(a, b, *args, **kwargs)`, but with introspection and pickling benefits. 2. Show the canonical use — pre-bind `timeout=2` and a `headers` dict to `requests.get`, so the rest of your code calls `safe_get(url)` without re-specifying. 3. Mention the introspection edge: `partial(f, 1).func` is `f`, `.args` is `(1,)`, `.keywords` is `{}`. Useful when debugging "what is this callable actually wrapping?" 4. Tip the listener: `partial` objects are not technically functions — `isinstance(p, types.FunctionType)` is `False`. They are callable, which is all that matters.

Edge cases to mention: - Keyword arguments bound via `partial` can be overridden at call time: `p = partial(f, x=1); p(x=2)` calls `f(x=2)`. - Positional arguments cannot be overridden — they're locked into the head of the arg list. - `partial` does not preserve `__name__` or `__doc__`. Use `functools.update_wrapper` if you need them.

What NOT to say: - "Partial is the same as curry" — close, not identical. Currying is one-arg-at-a-time; `partial` lets you bind any subset, in any order, at once. - "Partial is just a lambda" — semantically similar, but `partial` is pickle-friendly (lambdas aren't) and has the `.func` / `.args` / `.keywords` triple for inspection.

Common follow-up: "How would you partially apply a keyword argument that's positional in the original signature?" — Use the keyword form: `partial(func, key=value)`. Python is happy to route it as a keyword as long as the parameter accepts both.

```
from functools import partial

def make_call(url, timeout, headers, retries):
    print(f"GET {url} timeout={timeout} retries={retries}")

safe_get = partial(make_call, timeout=2, headers={"X-API": "v1"}, retries=3)
safe_get("https://api.razorpay.com/v1/payments")
# GET https://api.razorpay.com/v1/payments timeout=2 retries=3

print(safe_get.func)          # <function make_call at 0x...>
print(safe_get.keywords)     # {'timeout': 2, 'headers': {'X-API': 'v1'}, 'retries': 3}
```

Q6 · The `operator` module — `itemgetter`, `attrgetter`, `methodcaller`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, PhonePe, Goldman, Walmart Labs · Topic: First-class functions

The question (interviewer's verbatim phrasing):

Sort a list of dicts by the `price` key. Show me at least two ways, and tell me which is faster.

The 30-second answer: The two idioms are `sorted(items, key=lambda d: d["price"])` and `sorted(items, key=itemgetter("price"))`. The `itemgetter` version is faster — it's implemented in C — and reads cleaner once you know it. `operator.attrgetter` is the same for object attributes, and `operator.methodcaller` calls a named method. Reach for these when the key extraction is simple; reach for a lambda when it isn't.

Step-by-step thought process: 1. Open with the lambda version (every Python writer knows it) and the `operator` version (every Python reader should know it). 2. State the speed claim and why — `itemgetter` is a C-implemented callable in the standard library; the lambda goes through Python's eval loop. The gap is small per call but real over a million-item sort. 3. Show the multi-key form: `itemgetter("price", "name")` returns a callable that returns a tuple — the natural way to sort by price then name without writing a lambda that returns a tuple. 4. Mention `attrgetter` for dataclasses or any object — `attrgetter("user.email")` even drills through dotted attributes — and `methodcaller("strip")` for calling a no-arg method as a key.

Edge cases to mention: - `itemgetter("price")` raises `KeyError` if any item lacks `"price"` — same behaviour as `d["price"]`. No `.get()` fallback. - `attrgetter("a.b.c")` walks three dots — handy for nested records, no need for a lambda. - `methodcaller("split", ",")` is a callable that calls `.split(",")` on each input — pass extra args and kwargs to the method.

What NOT to say: - "Lambdas are always fine, `operator` is just a style preference" — they are not always fine; in a hot sort loop, the C version is meaningfully faster. - "`itemgetter` only works on dicts" — it works on anything that supports `__getitem__`, which includes lists (by index), tuples, and any custom indexer.

Common follow-up: "How would you sort by descending price?" — Either `sorted(items, key=itemgetter("price"), reverse=True)`, or pre-negate for numeric keys with `key=lambda d: -d["price"]`. `reverse=True` is the cleaner idiom.


```

from operator import itemgetter, attrgetter, methodcaller
from dataclasses import dataclass

products = [
    {"name": "tea", "price": 20},
    {"name": "coffee", "price": 80},
    {"name": "lassi", "price": 50},
]
print(sorted(products, key=itemgetter("price")))
# [{'name': 'tea', 'price': 20}, {'name': 'lassi', 'price': 50}, {'name': 'coffee', 'price': 80}]

@dataclass
class Order:
    id: int
    amount: float

orders = [Order(1, 500), Order(2, 100)]
print(sorted(orders, key=attrgetter("amount"))) # by .amount

names = [" tcs ", " razorpay ", "swiggy"]
print(sorted(names, key=methodcaller("strip")))

```

Q7 · Function attributes — `__name__`, `__doc__`, custom

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Goldman · Topic: First-class functions

The question (interviewer's verbatim phrasing):

Functions have attributes. Name three you'd actually use in practice, and tell me what happens to `__name__` when a function is wrapped by a decorator.

The 30-second answer: The three you actually use are `__name__` (the function name, shown in tracebacks and logs), `__doc__` (the docstring, used by `help(f)` and Sphinx), and `__qualname__` (the dotted path including class — `Order.refund` vs just `refund`). When a decorator wraps a function, the wrapper's `__name__` becomes the wrapper's name (usually `'wrapper'`), which breaks logs and introspection. Fix it with `functools.wraps` on the wrapper.

Step-by-step thought process: 1. Open with the three names every Python author should know — and mention that they're not magic. They're regular attributes you can read and write. 2. Mention `__module__` (the module the function was defined in), `__defaults__` (the tuple of default values), and `__annotations__` (the type hints dict). These come up in framework code. 3. Demonstrate the decorator clobbering: define `decorator` without `@functools.wraps`, wrap a function, print `wrapped.__name__` — see `'wrapper'`. Add `@functools.wraps(func)` to the wrapper, run

again — see the original name. 4. Mention custom attributes — `f.retries = 3` works because function objects have a `__dict__`. Frameworks like Flask and Django use this for route metadata and admin hooks.

Edge cases to mention: - C-implemented built-ins (`len`, `print`) don't have a writable `__dict__` — `len.x = 1` raises `AttributeError`. Pure-Python functions do. - `lambda` has `__name__ == '<lambda>'` — useless for tracebacks. That alone is a reason to prefer `def` when the function will appear in stack traces. - `__qualname__` and `__name__` differ for methods and nested functions — `Order.refund.__qualname__` is `'Order.refund'`, but `__name__` is `'refund'`.

What NOT to say: - "Decorators preserve `__name__` automatically" — they don't. You have to apply `functools.wraps` explicitly. - "Function attributes are just like class attributes" — they live on the function object, separate from any class. They're rarely the right place for shared state.

Common follow-up: "How does Flask use function attributes?" — `@app.route("/path")` attaches the URL rule to the function before registering it. The decorator pattern leans heavily on function-level metadata.

```
def greet(name):
    """Return a greeting string."""
    return f"hi, {name}"

print(greet.__name__)          # greet
print(greet.__doc__)          # Return a greeting string.
print(greet.__qualname__)      # greet
print(greet.__defaults__)     # None (no defaults here)

# Custom attribute
greet.calls = 0
greet.calls += 1
print(greet.calls)            # 1

# Decorator clobbering
from functools import wraps

def trace(func):
    @wraps(func)                # preserves __name__, __doc__
    def wrapper(*a, **kw):
        return func(*a, **kw)
    return wrapper

@trace
def pay(amount): return amount
print(pay.__name__)            # pay (not 'wrapper')
```

Q8 · `__call__` — making any object callable

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft, Goldman · Topic: First-class functions

The question (interviewer's verbatim phrasing):

If `f = MyClass()` and I write `f(10)`, what method is invoked, and what's the difference between a callable class and a regular function?

The 30-second answer: `f(10)` invokes `MyClass.__call__(f, 10)`. Any object whose class defines `__call__` is callable — `callable(f)` returns `True`. The difference vs a regular function is that the callable can carry state on `self`. Use it when your "function" needs initialisation, configuration, or accumulators that would otherwise live in module-level globals or closures.

Step-by-step thought process: 1. Open with the dispatch rule: `obj(...)` is exactly `type(obj).__call__(obj, ...)`. The `__call__` lives on the class, not the instance — same as every other dunder. 2. Show the canonical example: a `Counter` class with `__init__` to set the start, `__call__` to increment and return. Compare with the closure version (`def make_counter(): n = 0; def step(): nonlocal n; n += 1; return n; return step`) — the class is often cleaner for non-trivial state. 3. Mention the framework pattern: scikit-learn estimators (`model.fit()`), PyTorch modules (`net(x)`), validation classes (`is_valid(value)`) all use `__call__` so the instance feels like a function. 4. State the trade-off: classes are more verbose; closures are tighter. Pick the class when you have multiple operations on the same state (a `Counter` that also has `.reset()`), or want pickling / inspection / typing benefits.

Edge cases to mention: - A function is also a callable object — `function.__call__` exists. That's how `callable()` works uniformly. - Defining `__call__` on a class does not make the class itself callable in a new way — calling the class still goes through `__init__` to construct an instance. The instance is what's callable. - `functools.partial` instances are callable objects with their own `__call__` — that's why they slot into "anywhere a function works."

What NOT to say: - "`__call__` makes the class a function" — no, it makes instances callable. Different distinction. - "I'd never use a class for a callback" — sometimes the right choice; PyTorch's entire `nn.Module` pattern is callable classes.

Common follow-up: "Closure vs callable class for a rate limiter — which would you pick?" — Class. A rate limiter has reset, configuration, multiple methods (`.allow()`, `.reset()`, `.stats()`). Closures shine for single-purpose, single-state callables; classes win once you have multiple operations.

```
class Counter:
    def __init__(self, start=0):
        self.n = start

    def __call__(self, step=1):
        self.n += step
        return self.n

c = Counter()
print(c())           # 1
print(c(10))         # 11
print(callable(c))   # True
```

Q9 · Storing functions in a dispatch table

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Postman · Topic: First-class functions

The question (interviewer's verbatim phrasing):

Instead of a long `if event == "...": ... elif ...:` chain, how would you build a dispatch table using a dict of functions?

The 30-second answer: Map names to callables in a dict, then look up and call: `handlers = {"paid": handle_paid, "refund": handle_refund}; handlers[event](payload)`. The lookup is `0(1)`, adding a new event is a one-line dict entry, and unit tests can replace handlers cleanly. Use `dict.get(event, default_handler)` to handle unknown events without `KeyError`.

Step-by-step thought process: 1. Open with the anti-pattern — a 30-line `if/elif` chain — and the fix: a dict that maps event names to functions. 2. State the three operational wins: `0(1)` dispatch, easy extension (add a key), and easy testing (monkey-patch the dict in a fixture). 3. Mention `dict.get(event, default)` for unknown events — common interview tweak: "what if the event isn't in the table?" The right answer is a default handler, usually one that logs and ignores. 4. Mention the registration-decorator variant: decorate each handler with `@register("paid")` and let the decorator populate the dict. That's the pattern Flask routes and Django signals use.

Edge cases to mention: - Keys must be hashable — strings, ints, tuples. Don't try to use a list of args as the key. - The dict values are function references, not calls — `{"paid": handle_paid}`, not `{"paid": handle_paid()}`. - Pre-binding arguments via `partial` lets you put parameterized handlers in the table — `{"paid": partial(handle, event_type="paid")}`.

What NOT to say: - "Use a long `match/case` statement instead" — `match/case` (PEP 634, 3.10+) is fine for structural patterns, but a dict is faster and more extensible for plain name-to-function dispatch. - "Dicts aren't safe for hot paths" — they are. Python's dict lookup is one of its most-tuned operations.

Common follow-up: "How would you make the table extensible from other modules?" — Use a registration decorator. Other modules import the decorator and register their handlers, no need to edit the main file. That's the open-closed principle in Python form.

```
def handle_paid(payload):
    print(f"paid: {payload}")

def handle_refund(payload):
    print(f"refund: {payload}")

def handle_unknown(payload):
    print(f"unknown event: {payload}")

handlers = {
    "paid": handle_paid,
    "refund": handle_refund,
}

def dispatch(event, payload):
    handlers.get(event, handle_unknown)(payload)

dispatch("paid", {"order": 7})      # paid: {'order': 7}
dispatch("voided", {"order": 8})   # unknown event: {'order': 8}
```

Q10 · Function composition — chaining callables

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman · Topic: First-class functions

The question (interviewer's verbatim phrasing):

Write a `compose(f, g)` such that `compose(f, g)(x)` is `f(g(x))`. Then extend it to compose any number of functions left to right.

The 30-second answer: Two-arg form: `def compose(f, g): return lambda x: f(g(x))`.

Variadic form using `functools.reduce`: `compose(*funcs)` returns `lambda x: reduce(lambda v, f: f(v), reversed(funcs), x)`. For left-to-right ("pipe") semantics, drop the `reversed`. The pattern is most useful for data pipelines — clean a string, then validate it, then format it.

Step-by-step thought process: 1. Open with the two-function form. State the mathematical convention — right-to-left, so `compose(f, g)(x)` is `f(g(x))`. 2. Generalize using `reduce` for arbitrary arity. State the choice of direction (right-to-left vs left-to-right) and which is "pipe" semantics — left-to-right is what bash, JS, and most data engineers expect. 3. Mention the readability ceiling: composing 6 functions is usually less readable than a `for` loop or three named intermediate variables. Composition

shines for two or three steps that conceptually belong together. 4. Mention `toolz.compose` and `fancy.compose` as third-party choices — and point out the standard library deliberately doesn't ship one because the right answer is "it's a one-liner if you really need it."

Edge cases to mention: - `compose()` with no arguments is ill-defined — usually `identity` (return the input unchanged). Document this if you build it. - The functions must be unary (one argument). For multi-arg pipelines, you usually pass a tuple or dict, or use `partial` to pre-bind. - Composition does not preserve `__name__` — debug output will say `<lambda>`. Use `functools.wraps` on the outer lambda or write a proper `def`.

What NOT to say: - "Python has a built-in pipe operator" — it doesn't. Some libraries fake it with `|`, but the language doesn't ship one. - "Composition is always cleaner than a `for` loop" — past three or four steps, it isn't. Readability beats elegance.

Common follow-up: *"When would you choose composition over a method chain?"* — When the steps aren't methods of the same type, or when you want to swap steps independently. Method chains require all methods on the same object; composition treats steps as separate values.

```
from functools import reduce

def compose2(f, g):
    return lambda x: f(g(x))

def pipe(*funcs):
    """Left-to-right composition: pipe(f, g, h)(x) == h(g(f(x)))."""
    return lambda x: reduce(lambda v, f: f(v), funcs, x)

strip = str.strip
upper = str.upper
prefix = lambda s: f"USER:{s}"

normalize = pipe(strip, upper, prefix)
print(normalize(" razorpay "))      # USER:RAZORPAY
```

Section B — Closures (10 Qs)

Q11 · What a closure actually is

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, PhonePe, Postman · Topic: Closures

The question (interviewer's verbatim phrasing):

Define a closure in your own words and give me a real example you would use in production code.

The 30-second answer: A closure is a function plus the variables it captured from its enclosing scope. When an inner function references a name from an outer function, Python wraps that name in a cell and stores it on the inner function's `__closure__`. The captured names survive after the outer function returns. Real example: a configured logger — `make_logger(prefix)` returns a function that always prefixes log lines with the captured value.

Step-by-step thought process: 1. Open with the definition — function plus environment. Avoid the math jargon ("lambda calculus"); the interviewer wants Python intuition. 2. Mention `__closure__` (the tuple of cells) and `__code__.co_freevars` (the captured names). These are the two introspection levers. 3. Give the production example: pre-configured loggers, validators with thresholds, decorators that take parameters. All three are closures. 4. State the alternative — a class with `__call__` — and the trade-off: closures are tighter and don't need a name. Classes are easier to introspect and extend.

Edge cases to mention: - A function with no captured names has `__closure__ == None`, not an empty tuple. Be careful when inspecting. - The cells are shared, not copied — if the outer function mutates a captured mutable, the closure sees the mutation. - Lambdas form closures the same way as `def` — there's nothing special about either keyword.

What NOT to say: - "A closure is a function" — incomplete; it's a function plus the captured environment. - "Closures are slow" — they aren't. The dispatch through cells adds nanoseconds, not milliseconds.

Common follow-up: "What's the difference between a closure and a lambda?" — Orthogonal. A lambda is a syntax for an anonymous function. A closure is a function (lambda or `def`) that captures variables from an enclosing scope. A lambda can be a closure; a `def` can also be a closure; not all functions are closures.

```
def make_logger(prefix):
    def log(msg):
        print(f"[{prefix}] {msg}")
    return log

razorpay_log = make_logger("razorpay")
swiggy_log = make_logger("swiggy")
razorpay_log("payment OK")           # [razorpay] payment OK
swiggy_log("order received")         # [swiggy] order received

# Introspection
print(razorpay_log.__closure__)      # (<cell at 0x...: str object at ...>,)
print(razorpay_log.__closure__[0].cell_contents)  # razorpay
print(razorpay_log.__code__.co_freevars)          # ('prefix',)
```

Q12 · LEGB scoping — name resolution rules

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, Swiggy, JP Morgan · Topic: Closures

The question (interviewer's verbatim phrasing):

When Python looks up a name, what's the order it searches in? Walk me through it on an example with a global, a nested function, and a local.

The 30-second answer: LEGB: **L**ocal, **E**nclosing, **G**lobal, **B**uilt-in. When a name is read, Python checks the current function's local scope first, then any enclosing functions' scopes, then the module's global scope, then the built-ins. The first match wins. Writes by default land in Local — use `global` to rebind a module-level name and `nonlocal` to rebind an enclosing-function name.

Step-by-step thought process: 1. Spell out the acronym and what each scope is. Mention "Built-in" is the namespace from the `builtins` module — `len`, `print`, `range`, etc. 2. Walk through an example: a `nested()` inside `outer()` referencing `x`. If `x` isn't bound locally in `nested`, Python looks in `outer`, then the module, then `builtins`. 3. State the write rule: any assignment inside a function makes that name local to the function — even if the name exists in an outer scope. That's why `count = count + 1` raises `UnboundLocalError` without a `global` or `nonlocal` declaration. 4. Mention the class trap: class-body scope is NOT searched by methods defined inside the class. Methods see the module globals, not the class body locals.

Edge cases to mention: - `del x` inside a function also makes `x` local — it's an assignment statement in spirit. - Comprehensions have their own scope in Python 3 (PEP 3104) — `[x for x in range(3)]` does not leak `x` into the enclosing scope. - `exec` and `eval` can break LEGB; avoid them.

What NOT to say: - "Python searches all scopes simultaneously" — no, it's strictly ordered $L \rightarrow E \rightarrow G \rightarrow B$. - "Class body counts as an enclosing scope for methods" — it doesn't. That's a famous gotcha for newcomers.

Common follow-up: "What does `UnboundLocalError` mean and how do you fix it?" — It means Python saw an assignment to that name inside the function (making it local), but the read happened before the assignment. Fix with `global` / `nonlocal`, or restructure to not shadow.


```

x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)          # local
    inner()
    print(x)              # enclosing

outer()
print(x)                  # global

# Common mistake – shadowing without nonlocal
def counter():
    n = 0
    def step():
        # n += 1          # UnboundLocalError: n referenced before assignment
        nonlocal n
        n += 1
        return n
    return step

```

Q13 · `nonlocal` — what it does and when you need it

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India, Goldman · Topic: Closures

The question (interviewer's verbatim phrasing):

What does `nonlocal` do? Show me a counter closure that uses it and explain why it's required.

The 30-second answer: `nonlocal` declares that the named variable refers to the nearest enclosing function's binding, not a new local. Without it, an assignment inside an inner function creates a new local, shadowing the outer name. Use it whenever you need to rebind a captured variable — most commonly in counter or accumulator closures.

Step-by-step thought process: 1. State the mechanics: `nonlocal n` says "all reads and writes to `n` in this function bind to the `n` in the nearest enclosing function scope." Python 3 added it (PEP 3104); Python 2 didn't have it. 2. Show why it's needed: without `nonlocal`, `n = n + 1` inside `step` raises `UnboundLocalError` because Python sees an assignment and decides `n` is local — then the read on the right-hand side fails. 3. State the boundary — `nonlocal` only reaches functions, not class bodies or modules. For modules use `global`. There's no built-in for module-and-also-

enclosing. 4. Mention the alternative — wrap the state in a list (`n = [0]; n[0] += 1`). That works in Python 2 and is sometimes seen in old code; in Python 3, `nonlocal` is cleaner.

Edge cases to mention: - Mutating a mutable doesn't need `nonlocal` — `lst.append(x)` is a method call, not an assignment, so the name stays bound to the outer object. - `nonlocal` must reference a name that already exists in an enclosing scope — declaring `nonlocal n` when no outer `n` exists raises `SyntaxError`. - You cannot use `nonlocal` at the module level — `SyntaxError`. `global` is for module-level rebinding.

What NOT to say: - "`nonlocal` is the same as `global`" — they're parallel keywords but reach different scopes. - "You always need `nonlocal` in closures" — not true. Reads work without it. Only writes (rebinding) need it.

Common follow-up: "Why does `n += 1` need `nonlocal` but `lst.append(x)` doesn't?" — Because `+=` is rebinding (`n = n + 1`), which by default creates a local. `append` is a method call that mutates the existing object; the name still points to the same list.

```
def make_counter():
    n = 0
    def step():
        nonlocal n
        n += 1
        return n
    return step

c = make_counter()
print(c(), c(), c())          # 1 2 3

# Without nonlocal — fails
def broken_counter():
    n = 0
    def step():
        n += 1                # UnboundLocalError: local 'n' referenced before assignment
        return n
    return step

# Mutable state — no nonlocal needed
def make_collector():
    items = []
    def add(x):
        items.append(x)        # method call, not rebinding
        return items
    return add
```

Q14 · Closures vs classes — when to pick each

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Goldman, Microsoft · Topic: Closures

The question (interviewer's verbatim phrasing):

If I want to track state across calls, I could use a closure or a class. When would you pick a closure over a class?

The 30-second answer: Closures for single-purpose callables with one operation — a counter, a callback with config, a memoizer. Classes when you need multiple methods on the same state, when you want serialization or inheritance, or when the state's shape becomes non-trivial. Closures are tighter and don't pollute the namespace; classes are more explicit and easier to debug.

Step-by-step thought process: 1. Open with the one-line rule: closure for one operation, class for many. A closure can do exactly one thing (call it); a class can have `.start()`, `.stop()`, `.reset()`, `.stats()`, all sharing state. 2. State the secondary criteria: pickling (closures with free variables are unpicklable; classes are pickle-friendly), introspection (`dir(obj)` on a class instance is richer), and inheritance (classes can subclass; closures can't). 3. State the closure wins: less code, no need to invent a class name, the state isn't exposed (encapsulation by lexical scoping, no `__private` convention). 4. Mention `__slots__`, `dataclass`, `attrs` as the class-side ergonomic helpers — they bring class boilerplate down to closure levels for many cases.

Edge cases to mention: - A class with only `__init__` and `__call__` is essentially a closure with extra ceremony — the class form is preferred when you'll add methods later. - Closures over mutable state share that state across all calls — same as instance attributes, just less visible. - For unit tests, classes are easier to reset between runs because you can construct a fresh instance. Closures need to be re-created from the factory.

What NOT to say: - "Closures are functional, classes are OOP — they don't mix" — they mix all the time. Python is multi-paradigm; pick the right tool per problem. - "Closures are always faster" — they're not consistently faster. Modern CPython optimizes both well.

Common follow-up: "How would you decide between a closure-based decorator and a class-based decorator?" — Closure for a simple wrapper without state; class with `__call__` when the decorator has configuration or methods like `.disable()` or `.cache_clear()`. `functools.lru_cache` itself is a callable object, not a closure, for exactly this reason.

```
# Closure — one operation, one state
def make_counter():
    n = 0
    def step():
        nonlocal n
        n += 1
        return n
    return step

# Class — multiple operations, shared state
class Counter:
    def __init__(self):
        self.n = 0
    def step(self):
        self.n += 1
        return self.n
    def reset(self):
        self.n = 0
    def stats(self):
        return {"current": self.n}

c1 = make_counter()
c2 = Counter()
print(c1(), c1())           # 1 2
print(c2.step(), c2.step(), c2.stats()) # 1 2 {'current': 2}
```

Q15 · The late-binding closure trap (the classic for-loop lambda)

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Swiggy, PhonePe, Goldman · Topic: Closures

The question (interviewer's verbatim phrasing):

Predict the output:

`funcs = [lambda: i for i in range(3)]; print([f() for f in funcs])`. Then tell me why.

The 30-second answer: Output is `[2, 2, 2]`, not `[0, 1, 2]`. Each lambda captures the *variable* `i` (not its value at lambda-creation time), and by the time the lambdas are called, the loop has finished with `i` set to `2`. Fix by binding the value at definition time: `[lambda i=i: i for i in range(3)]` uses default-argument capture, which evaluates at `def` time.

Step-by-step thought process: 1. State the output upfront — interviewers want you to predict it without running it. 2. State the cause: closures capture names, not values. The lambdas share a single `i` cell; all three reads happen after the loop has exited. 3. Show the two fixes — default-argument capture (`lambda i=i: i`) and `functools.partial` (`partial(lambda i: i, i)`). The default-

argument trick is the idiomatic one. 4. Connect this to a Pythonic surprise: this is the same family of bug as the mutable-default-argument trap. Both involve the difference between definition time and call time.

Edge cases to mention: - Generator expressions also late-bind in the same way — `[(lambda: i)() for i in range(3)]` is `[0, 1, 2]` (called inside the comp) but `funcs = [lambda: i for i in range(3)]; [f() for f in funcs]` is `[2, 2, 2]`. - Class-body loops have the same bug if you build method-like lambdas inside the class. - `enumerate` doesn't fix it — the issue is the closure, not the loop variable source.

What NOT to say: - "Python is broken" — it isn't. This is the consequence of lexical scoping plus first-class closures; languages like JavaScript had the same issue before `let` was introduced (3.13's free-threading work won't change this). - "I'd write a list comprehension to avoid it" — comprehensions don't fix the closure problem; only binding at definition does.

Common follow-up: "Why does `for i in range(3): funcs.append(lambda: i)` have the same bug?" — Same reason; the lambdas capture the loop variable `i`, not its successive values. The for-loop body runs three times, but the closures all point to the same cell.

```
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])           # [2, 2, 2] - late-binding trap

# Fix 1 - default-argument capture
funcs_fixed = [lambda i=i: i for i in range(3)]
print([f() for f in funcs_fixed])     # [0, 1, 2]

# Fix 2 - functools.partial
from functools import partial
funcs_partial = [partial(lambda i: i, i) for i in range(3)]
print([f() for f in funcs_partial])   # [0, 1, 2]
```

Q16 · Inspecting closure variables

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft, Goldman · Topic: Closures

The question (interviewer's verbatim phrasing):

Given a closure `f`, how would you find out what variables it captured and what their current values are?

The 30-second answer: Read `f.__code__.co_freevars` for the captured names (a tuple) and `f.__closure__` for the cells (a parallel tuple). Each cell has a `.cell_contents` attribute holding the current value. Zip them: `dict(zip(f.__code__.co_freevars, (c.cell_contents for c in f.__closure__)))`.

Step-by-step thought process: 1. Open with the two attributes — `__code__.co_freevars` (names) and `__closure__` (cells). They're parallel — index `i` in one matches index `i` in the other. 2. State that `__closure__` is `None` for functions with no captured names, not an empty tuple. Defensive code should check. 3. Show the zip idiom for a debug dictionary. 4. Mention this is what debuggers (`pdb`, `ipdb`) and IDEs use under the hood to display "captured variables" in their watch panels.

Edge cases to mention: - `co_freevars` is empty `()` if no names are captured; `__closure__` is `None` (asymmetry). - Cells can be modified — `cell.cell_contents = new_value` works and rebinds the closure variable from outside. Powerful and ugly; rarely used. - `inspect.getclosurevars(f)` returns a structured object with `nonlocals`, `globals`, `builtins`, `unbound` — friendlier than reaching into `__closure__` manually.

What NOT to say: - "Closures are private and can't be inspected" — they can; the cells are visible through `__closure__`. - "`__closure__` returns a list" — it's a tuple, and `None` if nothing was captured.

Common follow-up: "How would you use this to debug a misbehaving closure?" — Print the freevars-cells dict, see what each captured value actually is. Most "closure bugs" are late-binding (Q15) or accidental sharing (mutable captured by multiple closures).

```
from inspect import getclosurevars

def make_adder(n, suffix="!"):
    def add(x):
        return f"{n + x}{suffix}"
    return add

a = make_adder(10, suffix="?")
print(a.__code__.co_freevars)      # ('n', 'suffix')
print(a.__closure__)               # (<cell at ...: int 10>, <cell at ...: str '?'>)
print(a.__closure__[0].cell_contents) # 10
print(a.__closure__[1].cell_contents) # ?

print(getclosurevars(a).nonlocals)  # {'n': 10, 'suffix': '?'}
```

Q17 · Closures and mutable captured state

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Goldman · Topic: Closures

The question (interviewer's verbatim phrasing):

If two closures from the same factory share a captured list, what happens if one of them mutates it?

The 30-second answer: Both see the mutation. Closures capture references to cells, not copies. If the outer function binds a list and two inner functions both reference it, they share the same list — any `.append`, `.remove`, or rebind via `nonlocal` is visible to both. To isolate them, give each call its own mutable inside the factory or pass a fresh list as an argument.

Step-by-step thought process: 1. State the underlying rule — Python passes references, and closures share references. The captured list is shared, not deep-copied. 2. Show the example: `make_pair()` returns `(append, read)`. `append("x")` then `read()` shows `["x"]`. That's a feature for some patterns, a bug for others. 3. State the isolation patterns — fresh allocation per factory call (each closure pair gets its own list), copy-on-capture (`list(items)` inside the inner), or class-based state with an explicit `__init__`. 4. Mention the multi-thread implication: shared mutable state in closures is not thread-safe. Use `threading.Lock` or per-thread storage.

Edge cases to mention: - A closure capturing `nonlocal x` where `x` is a list — rebinding `x = []` only rebinds the closure's cell, not anything outside. - `lst = lst.copy()` inside an inner does NOT need `nonlocal` because copy is a method call followed by rebinding — but the rebinding is local unless you declare `nonlocal lst`. - Sharing state intentionally between closures is the entire point of patterns like factory + reader/writer pairs.

What NOT to say: - "Each closure gets its own copy of captured variables" — wrong; they share references. - "Mutating captured state is always a bug" — sometimes it's deliberate (caching, accumulators, event buses).

Common follow-up: "How would you build a `make_appender` that returns isolated closures?" — Create a fresh list inside the factory and capture it. Each call to the factory gives a new list, and the returned closure owns it.

```
def make_pair():
    items = []
    def append(x):
        items.append(x)
    def read():
        return list(items)
    return append, read

add, get = make_pair()
add("razorpay"); add("swiggy")
print(get())                # ['razorpay', 'swiggy']

# Same factory, separate state per call
add2, get2 = make_pair()
add2("cred")
print(get())                # ['razorpay', 'swiggy'] - unaffected
print(get2())               # ['cred']
```

Q18 · Closures vs partial application

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, JP Morgan · Topic: Closures

The question (interviewer's verbatim phrasing):

What's the difference between a closure and a partially-applied function via `functools.partial` ?

The 30-second answer: A closure is a function with captured variables from a lexical scope. A `partial` is a function with pre-bound arguments. They overlap in effect — both produce a callable with fewer required arguments — but a closure is a Python language feature, while `partial` is a stdlib utility. Closures are more flexible (can capture any variable, can rebind via `nonlocal`); `partials` are more introspectable (have `.func`, `.args`, `.keywords`) and pickle-friendly.

Step-by-step thought process: 1. State the conceptual distinction: closure captures *scope*, partial captures *arguments*. A closure captures `i` because it's in an enclosing function; a `partial` captures `5` because you told it to. 2. Show the same effect achieved both ways: `add5_closure = (lambda x: lambda y: x + y)(5)` vs `add5_partial = partial(lambda x, y: x + y, 5)`. Both work; same result. 3. State the trade-offs — closures are more general (mutation, multiple values, complex setup) but harder to inspect. Partials are simpler but limited to argument binding. 4. Tip for the listener: in idiomatic Python, use `partial` for "fix the first N arguments" and closures for "build a function that needs setup beyond argument binding."

Edge cases to mention: - `partial` instances are picklable (you can `pickle.dumps(partial(f, 1))`). Closures over local functions usually aren't (lambdas can't be pickled). - `partial(f, x=1)(x=2)` is allowed — keyword args can be overridden at call time. With a closure, you'd have to wire it in. - `inspect.signature` works on `partial` (shows the remaining parameters); on a closure it shows the closure's own parameters.

What NOT to say: - "They're the same thing" — they overlap but aren't equivalent. Closures have more capabilities. - "`partial` is functional, closures are OOP" — both are functional patterns.

Common follow-up: "Why would you use `partial` instead of a `lambda` ?" — Three reasons: introspection (`.func`, `.args`), pickling, and avoiding the late-binding trap from Q15 (since `partial` evaluates arguments immediately).


```

from functools import partial

# Closure
def make_add(x):
    return lambda y: x + y

add5_closure = make_add(5)
print(add5_closure(3))           # 8

# Partial
def add(x, y): return x + y
add5_partial = partial(add, 5)
print(add5_partial(3))          # 8

# Partial is introspectable
print(add5_partial.func, add5_partial.args, add5_partial.keywords)
# <function add at 0x...> (5,) {}

```

Q19 · Closures and reference cycles

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Microsoft India · Topic: Closures

The question (interviewer's verbatim phrasing):

Can a closure cause a memory leak? If yes, give me an example and tell me how the garbage collector handles it.

The 30-second answer: Yes — most commonly when a closure captures `self` of an object that also holds a reference to the closure (typical in event handlers or callbacks attached to long-lived objects). The cycle is `obj → closure → obj`. CPython's reference counting can't break this on its own; the cyclic garbage collector eventually collects it, but until then the object is alive. The fix is `weakref.ref` to the captured object, or `weakref.WeakMethod` for bound methods.

Step-by-step thought process: 1. State the canonical cycle — a UI widget that holds a callback, where the callback closes over the widget itself. Widget refcount stays positive even after the user "closes" it. 2. Explain how CPython handles it: refcount alone can't free a cycle, so the generational GC (the `gc` module) is what reclaims it. That happens periodically, not immediately. 3. State the production cost — until GC runs, the widget's resources (memory, file handles, network connections) are held. In long-running services, leaked widgets accumulate. 4. State the fix patterns: `weakref.ref(self)` for the captured object, or restructure the closure to not capture `self` (pass values it needs directly).

Edge cases to mention: - Disabling the cyclic GC (`gc.disable()`) — sometimes done for latency — turns these leaks into true leaks. Never disable GC unless you've audited cycles. - `__del__` on objects in a cycle was unreliable before Python 3.4 (PEP 442). Since 3.4 the GC can collect cycles even

when `__del__` is defined. - Closures captured by `asyncio` tasks and signal handlers are common leak sources — they live as long as the loop or the process.

What NOT to say: - "Python has no memory leaks because it's garbage collected" — it has cycle-induced leaks until GC runs, plus C-extension leaks. - "Cycles always cause leaks" — they don't, but they delay collection from "immediately" to "next GC cycle."

Common follow-up: "How would you debug a suspected closure cycle?" — `gc.garbage` after `gc.collect()` shows objects involved in cycles that couldn't be collected. `objgraph` is the third-party library most teams use for visualising cycles.

```
import weakref

class Widget:
    def __init__(self, name):
        self.name = name
        self.handlers = []

    def on_click(self):
        # Closure captures self — cycle!
        def handler():
            print(f"clicked {self.name}")
        self.handlers.append(handler)

    def on_click_safe(self):
        # weakref breaks the cycle
        weak_self = weakref.ref(self)
        def handler():
            real = weak_self()
            if real is not None:
                print(f"clicked {real.name}")
        self.handlers.append(handler)

w = Widget("checkout")
w.on_click_safe()
w.handlers[0]()                # clicked checkout
```

Q20 · Decorator without `functools.wraps` — closure problem

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft · Topic: Closures

The question (interviewer's verbatim phrasing):

If a decorator returns a wrapper without `functools.wraps`, what breaks? Give three concrete consequences.

The 30-second answer: The wrapper takes the original function's place, but its `__name__` is `'wrapper'`, its `__doc__` is `None` (or the wrapper's docstring), its `__module__` is the decorator's module, and its `__wrapped__` doesn't exist. Three concrete consequences: (1) logs and tracebacks show `wrapper` instead of the real function name, (2) `help(f)` shows the wrong docstring, (3) tools like Sphinx, pytest, and Flask that introspect functions misidentify or skip them.

Step-by-step thought process: 1. State the three consequences upfront — interviewer wants the list. 2. State the fix — `@functools.wraps(func)` on the wrapper. It copies `__name__`, `__doc__`, `__module__`, `__qualname__`, `__annotations__`, and `__dict__` from the wrapped function. It also sets `__wrapped__` so you can reach the original. 3. State the pytest/Flask consequence concretely: pytest discovers tests by `__name__` starting with `test_`. If your decorator strips that, pytest skips the test silently. Flask uses `__name__` for route registration uniqueness — a clobbered name causes "View function mapping is overwriting an existing endpoint" errors. 4. Mention `inspect.unwrap(f)` as the helper that follows `__wrapped__` chains — useful when you have stacked decorators and want the original.

Edge cases to mention: - `functools.wraps` doesn't preserve the function's signature for `inspect.signature` unless the wrapper accepts `*args, **kwargs`. Use `functools.wraps` plus `inspect.signature(func)` if you need precise signature handling. - Decorators that change the signature deliberately (add a parameter) should NOT use `wraps` for the signature, because that lies about what the wrapper accepts. - `wraps` is itself a decorator — `@wraps(func)` is sugar for `wrapper = wraps(func)(wrapper)`.

What NOT to say: - "Without `wraps` the function doesn't work" — it works fine; only introspection breaks. - "`wraps` is for performance" — it's for metadata preservation.

Common follow-up: "How would you unwrap to get the original function back?" — `f.__wrapped__` (if `wraps` was used) or `inspect.unwrap(f)` for nested chains.

```
from functools import wraps

def trace(func):
    @wraps(func)                # preserves metadata
    def wrapper(*args, **kwargs):
        print(f"calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@trace
def pay(amount):
    """Process a payment in INR."""
    return amount * 1.18        # add GST

print(pay.__name__)             # pay    (not 'wrapper')
print(pay.__doc__)              # Process a payment in INR.
print(pay.__wrapped__)          # <function pay at 0x...> (original)
```

Section C — Function arguments (10 Qs)

Q21 · Positional vs keyword arguments — what each is and when to use

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay · Topic: Function arguments

The question (interviewer's verbatim phrasing):

Explain positional and keyword arguments. When would you force a caller to use one over the other?

The 30-second answer: A positional argument is bound to a parameter by its order — `f(1, 2)` binds 1 to the first parameter, 2 to the second. A keyword argument is bound by name — `f(a=1, b=2)`. By default Python accepts both. Force keyword-only with a bare `*` (`def f(a, *, b)`) when the parameter name conveys meaning the position doesn't. Force positional-only with `/` (`def f(a, /, b)`) when the parameter name is meaningless or you want freedom to rename it.

Step-by-step thought process: 1. State the binding rule clearly — order versus name. Both can coexist in a single call: `f(1, b=2)`. 2. State the rule from PEP 3102 — bare `*` in the signature begins keyword-only parameters. After `*`, every parameter must be passed by name. Same idea: `def transfer(amount, *, currency="INR")` forces `transfer(500, currency="USD")`, never `transfer(500, "USD")`. 3. State the positional-only rule from PEP 570 (Python 3.8+) — bare `/` ends positional-only parameters. Before `/`, the parameter name cannot be used as a keyword. Useful for built-ins like `pow(x, y, z, /)`. 4. State the rule of thumb: anything truthy/falsy or boolean-like should be keyword-only — `def open(path, *, append=False)` is much clearer than `open(path, False)`.

Edge cases to mention: - Keyword arguments can come in any order; positionals must be in declared order. - You cannot pass a parameter both positionally and by keyword in the same call — Python raises `TypeError: got multiple values for argument`. - Positional-only is rare in user code but common in C-implemented stdlib functions.

What NOT to say: - "Keyword arguments are slower" — at function-call time the difference is negligible. - "Positional-only is just a hint" — it's enforced; calling `pow(base=2, exp=3, mod=5)` raises `TypeError` because they're positional-only in 3.8+.

Common follow-up: "Give me a real signature where you'd want both `/` and `*`." — `def replace(string, old, new, /, *, count=-1)` — the three positionals are by-order, `count` is keyword-only for clarity. The stdlib `str.replace` has this exact shape.

```
def transfer(amount, currency="INR"):
    return f"{amount} {currency}"

print(transfer(500))           # 500 INR
print(transfer(500, "USD"))    # 500 USD (positional)
print(transfer(500, currency="EUR")) # 500 EUR (keyword)

# Force keyword-only
def transfer_safe(amount, *, currency="INR"):
    return f"{amount} {currency}"

# transfer_safe(500, "USD")      # TypeError – currency is keyword-only
print(transfer_safe(500, currency="USD")) # 500 USD
```

Q22 · Default values — how they're evaluated

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, Swiggy, JP Morgan · Topic: Function arguments

The question (interviewer's verbatim phrasing):

When is a default value evaluated — at function definition time, or at every call?

The 30-second answer: At function definition time, once. The resulting object is stored on `f.__defaults__` and reused for every call that doesn't provide that argument. This is fine for immutable defaults (`0` , `""` , `None` , `()`) and the source of the mutable-default trap — every call without an override gets the same list/dict/set.

Step-by-step thought process: 1. State the answer first — definition time, once. The `def` statement is executable; it evaluates the default expression then. 2. Show the storage: `f.__defaults__` (for positional / pos-or-kw) and `f.__kwdefaults__` (for keyword-only) hold the cached objects. 3. Lead into the mutable-default trap (Q23) — that's the consequence of "once at def time" for mutable types. 4. Mention the dynamic-default idiom: `def f(x=None): x = [] if x is None else x` — the fix that puts evaluation back at call time for each call.

Edge cases to mention: - Defaults can reference module-level names; those are resolved at call time. `def f(x=CURRENT_USER)` evaluates `CURRENT_USER` at definition time, so it captures whatever the value was then. - Mutable defaults are flagged by linters (ruff `B006` , pylint `dangerous-default-value`). - `inspect.signature(f).parameters` exposes defaults; useful for introspection-based libraries.

What NOT to say: - "Defaults are evaluated at every call" — wrong; that's the mistaken mental model that produces the trap. - "Mutable defaults are forbidden" — they're allowed but almost always a bug.

Common follow-up: "Show me how to fix `def append_to(item, target=[])`." — Use `None` as the sentinel: `def append_to(item, target=None): if target is None: target = []; target.append(item); return target`.

```
def f(x=[]):
    x.append(1)
    return x

print(f())           # [1]
print(f())           # [1, 1] — the SAME list, reused
print(f.__defaults__) # ([1, 1],)

# Fix — sentinel pattern
def g(x=None):
    if x is None:
        x = []
    x.append(1)
    return x

print(g())           # [1]
print(g())           # [1] — fresh each call
```

Q23 · The mutable-default-argument trap

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, Swiggy, Cred, PhonePe · Topic: Function arguments

The question (interviewer's verbatim phrasing):

What does `def f(x=[]): x.append(1); return x` return on the second call, and why?

The 30-second answer: On the first call, returns `[1]`. On the second call, returns `[1, 1]` — the same list is reused because the default `[]` was created once at `def` time and stored on the function. Every call without an override mutates that shared list. Fix by using `None` as the sentinel and creating the list inside the body.

Step-by-step thought process: 1. State the output of the first three calls — `[1]`, `[1, 1]`, `[1, 1, 1]`. Drive home that the default is a single shared object. 2. Show `f.__defaults__` — it's a tuple containing the same list object that the function mutated. 3. State the fix idiom and why it works: `None` is immutable, so the sentinel is safe. Inside the body, `[] if x is None else x` creates a fresh list per call. 4. Mention why this is the single most-asked Python interview gotcha — it's the trap that catches confident devs in interviews because their code "works" in their daily Jupyter cell where they only call once.

Edge cases to mention: - Same trap with `dict`, `set`, `bytearray`, and any custom mutable. Tuples and frozensets are safe because they're immutable. - The "fix in place" idiom — `if x is None: x = []` — uses `is`, not `==`. `None` comparison should always use `is`. - Linters flag this — `ruff` rule `B006`, `pylint` `dangerous-default-value`, `flake8-bugbear` same family.

What NOT to say: - "I'd never write this in production" — the question is whether you understand why it fails. Saying "I'd never do it" deflects without showing knowledge. - "Use a list comprehension instead" — that doesn't apply here; the trap is about the default, not the body.

Common follow-up: "Why doesn't the trap apply to `def f(x=0): x += 1; return x`?" — Because `int` is immutable. `x += 1` rebinds the local `x` to a new int; it doesn't mutate the default. Mutable types share; immutable types rebind.

```
def append_to(item, target=[]):
    target.append(item)
    return target

print(append_to(1))      # [1]
print(append_to(2))      # [1, 2] — leaked!
print(append_to(3))      # [1, 2, 3]

# The sentinel fix
def append_to_safe(item, target=None):
    if target is None:
        target = []
    target.append(item)
    return target

print(append_to_safe(1))  # [1]
print(append_to_safe(2))  # [2] — fresh
```

Q24 · `*args` — variadic positional arguments

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay · Topic: Function arguments

The question (interviewer's verbatim phrasing):

What does `*args` mean inside `def f(*args)`, and what type does `args` have inside the function?

The 30-second answer: `*args` collects any extra positional arguments into a tuple. Inside the function, `args` is a `tuple` — immutable, ordered, indexable. The name `args` is convention; you

can call it anything (`*items` , `*values`). At the call site, the same `*` unpacks: `f(*[1, 2, 3])` is equivalent to `f(1, 2, 3)` .

Step-by-step thought process: 1. State the asymmetry: in `def` , `*args` collects; at the call site, `*lst` unpacks. Same symbol, opposite directions. 2. State the type — `tuple` . Not a list. Some candidates expect a list and get confused when `.append()` fails. 3. Show the mix — `def f(a, b, *rest)` binds `a` and `b` normally, collects everything else into `rest` . `f(1)` raises `TypeError: missing 1 required positional argument: 'b'` . 4. Mention idiomatic uses — wrappers, decorators, and forwarders use `*args, **kwargs` to pass arguments through without knowing them.

Edge cases to mention: - `args` is empty `()` if no extra positionals — never `None` . - Order matters: `def f(*args, x)` is legal in Python 3 — `x` becomes keyword-only after `*args` . `def f(x, *args)` is the more common shape. - You can have only one `*args` parameter per function.

What NOT to say: - "`args` is a list" — it's a tuple. - "`*args` makes the function accept any number of arguments" — only extra positionals; required ones still need to be passed.

Common follow-up: "*Why a tuple and not a list?*" — Tuples are slightly cheaper to allocate and signal "fixed bundle of values for this call" rather than "growable collection." Argument bundles are conceptually immutable.

```
def total(*args):
    return sum(args)

print(total())           # 0
print(total(1, 2, 3))    # 6
print(type(total.__code__)) # <class 'code'>

# Unpack at the call site
nums = [1, 2, 3]
print(total(*nums))      # 6

# Mix required and variadic
def log(level, *messages):
    for msg in messages:
        print(f"[{level}] {msg}")

log("info", "Razorpay OK", "Swiggy OK")
```

Q25 · `**kwargs` — variadic keyword arguments

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, Cred · Topic: Function arguments

The question (interviewer's verbatim phrasing):

What does `**kwargs` collect, and what's the type of `kwargs` inside the function? Walk me through unpacking a dict into a function call.

The 30-second answer: `**kwargs` collects extra keyword arguments into a `dict`. The dict is ordered (insertion-order since Python 3.7 / CPython 3.6). At the call site, `**d` unpacks a dict into keyword arguments: `f(**{"a": 1, "b": 2})` is `f(a=1, b=2)`. The keys of the dict must be valid Python identifiers (strings) when unpacking.

Step-by-step thought process: 1. State the collecting behaviour at `def` time and the unpacking behaviour at the call site. 2. State the type and order — `dict`, insertion-order preserving since 3.7. 3. Show the unpacking constraint — keys must be valid identifiers. `f(**{"1bad": 2})` raises `TypeError: keywords must be strings` (or syntactic). 4. Mention duplicate keys: `f(a=1, **{"a": 2})` raises `TypeError: got multiple values for keyword argument 'a'`. Python won't silently overwrite.

Edge cases to mention: - You can mix `*args` and `**kwargs`: `def f(*args, **kwargs)`. Common in decorators. - A bare `**` after parameters with no name is not allowed in `def` — you need a name like `**kwargs`. - The unpacked dict's keys are case-sensitive; `**{"A": 1}` does not match a parameter named `a`.

What NOT to say: - "`kwargs` is a tuple" — it's a dict. - "Keys can be anything" — they must be strings when unpacked.

Common follow-up: "How do you forward both `*args` and `**kwargs` to another function?" — `wrapper(*args, **kwargs) → target(*args, **kwargs)`. This is the decorator pattern.

```
def make_user(**kwargs):
    return kwargs

print(make_user(name="Razorpay", role="admin"))
# {'name': 'Razorpay', 'role': 'admin'}

# Unpack a dict at the call site
config = {"name": "Swiggy", "role": "user"}
print(make_user(**config))
# {'name': 'Swiggy', 'role': 'user'}

# Mix
def transfer(amount, *, currency="INR", **metadata):
    return f"{amount} {currency} {metadata}"

print(transfer(500, currency="USD", tag="Q3", source="UPI"))
# 500 USD {'tag': 'Q3', 'source': 'UPI'}
```

Q26 · Positional-only / — what PEP 570 changed

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Microsoft, Goldman · Topic: Function arguments

The question (interviewer's verbatim phrasing):

What does the / mean in `def divmod(a, b, /):` ? When would you put it in your own function?

The 30-second answer: The / marks the end of positional-only parameters. Anything to the left of / cannot be passed as a keyword — only by position. Built-ins have used this forever (`len(obj, /)` , `pow(x, y, z, /)`); PEP 570 added syntax to express it in pure Python (3.8+). Use it when the parameter name is meaningless to callers, or when you want to freely rename the parameter without breaking callers' keyword usage.

Step-by-step thought process: 1. State the rule — / is a separator; positional-only parameters are to its left. The name of those parameters can be anything; callers can never write `divmod(a=10, b=3)` . 2. State why it matters: it lets the implementer rename `a` and `b` without an API break. `len(obj=lst)` would couple every caller to the parameter name `obj` ; positional-only frees that. 3. State the second use — it prevents `**kwargs` from accidentally consuming a name. `def f(name, /, **kwargs)` lets you have a `name` parameter and also a `name` key in `kwargs` without conflict. 4. Mention the version — Python 3.8+. In older code, the convention was a leading underscore (`_internal`) — informal, not enforced.

Edge cases to mention: - You can combine / and * : `def f(pos_only, /, pos_or_kw, *, kw_only)` . Both separators in one signature. - Default values work with positional-only params: `def f(a=1, b=2, /)` . The defaults still apply if the caller omits them. - Type checkers (`mypy` , `pyright`) respect / since 3.8.

What NOT to say: - "It's the same as the leading underscore convention" — it's not; `_internal` is a convention, / is enforced by the runtime. - "It's only for built-ins" — it's available in user code since 3.8.

Common follow-up: "Why would you want a `name` parameter AND a `name` key in `**kwargs` ?" — When you're building a function that wraps another and needs to pass through arbitrary `**kwargs` but also needs its own `name` -shaped configuration. Positional-only lets both exist without collision.

```
def divmod_local(a, b, /):
    return a // b, a % b

print(divmod_local(10, 3))          # (3, 1)
# divmod_local(a=10, b=3)          # TypeError: divmod_local() got some positional-only arguments

# Real-world: avoid name clash with **kwargs
def render(template, /, **context):
    return template.format(**context)

print(render("Hi {name}", name="Razorpay")) # Hi Razorpay
# template can have a 'template' key in context if needed
```

Q27 · Keyword-only * — the safer signature

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Postman, Goldman · Topic: Function arguments

The question (interviewer's verbatim phrasing):

What does the bare * in `def transfer(amount, *, currency):` mean, and why would you use it?

The 30-second answer: The bare * marks the start of keyword-only parameters — anything after it must be passed by name, never by position. Use it when the parameter is a boolean, a configuration option, or anything whose meaning isn't obvious from position. `def open(path, *, append=False)` forces `open("x.txt", append=True)`, which reads cleanly. `open("x.txt", True)` would be a mystery.

Step-by-step thought process: 1. State the rule — * marks the boundary. Parameters after * are keyword-only. 2. State the design reason: keyword-only makes call sites self-documenting. `requests.get(url, timeout=2)` is clear; `requests.get(url, 2)` would be guessable but ugly. 3. State the rule of thumb: any boolean parameter, any optional config, any parameter that's not the primary subject of the call — make it keyword-only. 4. Mention `def f(*args, x)` — when you have *args, anything after it is automatically keyword-only. The * is implicit.

Edge cases to mention: - * without a name is allowed (`def f(a, *, b)`); *args with a name is also allowed (`def f(a, *args, b)`). Both make b keyword-only. - You can give keyword-only parameters defaults: `def f(a, *, b=10)`. Without a default, omitting it raises `TypeError`. - The PEP is 3102, accepted for Python 3.0 — this has been available for ages.

What NOT to say: - "`*`" is just for `*args`" — it serves two purposes: the bare form for keyword-only boundaries, the named form for variadic collection. - "Keyword-only is slower" — same dispatch cost as positional.

Common follow-up: "When would you NOT make something keyword-only?" — When it's the primary subject of the call and its meaning is obvious from position. `len(lst)` doesn't need `len(obj=lst)`; positional is fine.

```
def transfer(amount, *, currency="INR", recipient):
    return f"{amount} {currency} -> {recipient}"

# transfer(500, "USD", "merchant")          # TypeError
print(transfer(500, currency="USD", recipient="merchant"))
# 500 USD -> merchant

# *args also makes following params keyword-only
def log(*messages, level="info"):
    for m in messages:
        print(f"[{level}] {m}")

log("paid", "OK", level="audit")
# [audit] paid
# [audit] OK
```

Q28 · Unpacking at call sites — `f(*lst, **dct)`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, JP Morgan · Topic: Function arguments

The question (interviewer's verbatim phrasing):

If I have `args = [1, 2]` and `kwargs = {"x": 3}`, how do I call `f` with those? And what happens if I unpack twice — `f(*a, *b)`?

The 30-second answer: `f(*args, **kwargs)` unpacks the list as positionals and the dict as keywords. PEP 448 (Python 3.5+) allows multiple unpackings: `f(*a, *b, *c)` chains positionals from three lists, and `f(**d1, **d2)` merges two dicts as keyword args (with conflict detection — duplicate keys raise `TypeError`).

Step-by-step thought process: 1. State the basic unpack: `*` for iterables to positionals, `**` for mappings to keywords. At the call site, these are unpack operators. 2. State the PEP 448 extension: you can have multiple `*` and `**` in one call. Useful for combining a default kwargs dict with override kwargs: `f(**defaults, **overrides)`. 3. State the conflict behaviour: if `**d1` and `**d2` share a key, Python raises `TypeError: got multiple values for keyword argument`.

There's no silent overwrite. 4. State the iterable-vs-iterator subtlety: `*` works on anything iterable.

`f(*gen)` consumes the generator. `f(*set_of_kwargs.keys())` works because keys is iterable.

Edge cases to mention: - `f(*a, b, *c)` — you can mix unpacked and direct positionals. Order matters; arguments resolve left-to-right. - Strings are iterable, so `f(*"abc")` unpacks to `f('a', 'b', 'c')`. Sometimes surprising. - `**` requires a mapping, not just any iterable — `f(**[(1, 2)])` raises `TypeError`, even though `dict([(1, 2)])` would work.

What NOT to say: - "`*` and `**` only work in function definitions" — they work at both definition and call sites, with opposite directions. - "Duplicate keys silently override" — they don't. `TypeError`.

Common follow-up: "How would you merge `defaults` and `overrides` to call `f`?" — `f(**defaults, **overrides)`. If keys overlap, Python raises. If you want override semantics, use `f(**{**defaults, **overrides})` (build the merged dict first; later wins).

```
def transfer(amount, *, currency, recipient):
    return f"{amount} {currency} -> {recipient}"

args = [500]
kwargs = {"currency": "USD", "recipient": "merchant"}
print(transfer(*args, **kwargs))          # 500 USD -> merchant

# PEP 448 multi-unpack
defaults = {"currency": "INR", "recipient": "self"}
overrides = {"recipient": "vendor"}
# Direct merge would raise duplicate-key TypeError:
# print(transfer(500, **defaults, **overrides)) # TypeError

# Build merged dict first
print(transfer(500, **{**defaults, **overrides})) # 500 INR -> vendor
```

Q29 · The full parameter order in a function definition

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Microsoft, Goldman · Topic: Function arguments

The question (interviewer's verbatim phrasing):

What's the full ordering of parameter kinds in a function signature? Construct an example with all five.

The 30-second answer: Left to right: positional-only, positional-or-keyword, `*args` (variadic positional), keyword-only, `**kwargs` (variadic keyword). The separators `/` and `*` define the boundaries. Full example: `def f(p_only, /, p_or_kw, *args, kw_only, **kwargs)`. Each kind is optional, but if they appear, they appear in this order.

Step-by-step thought process: 1. State the order — five kinds, this sequence, all optional. 2. Show the canonical maximal signature and walk through what each parameter accepts. 3. State the rule of thumb on when to use each — most user code uses positional-or-keyword (the default). Positional-only and keyword-only are tools for clarity and API stability. 4. Mention `inspect.Parameter.kind` — it returns `POSITIONAL_ONLY`, `POSITIONAL_OR_KEYWORD`, `VAR_POSITIONAL`, `KEYWORD_ONLY`, or `VAR_KEYWORD`. That's the runtime classification.

Edge cases to mention: - Defaults on positional-or-keyword must all come after non-default ones: `def f(a, b=2, c=3)`, not `def f(a=1, b, c=3)`. - Keyword-only parameters can have defaults independently — `def f(a, *, b=10, c)` is legal (`c` has no default; caller must pass). - `**kwargs` must be last. Nothing can follow it.

What NOT to say: - "Order doesn't matter" — it strictly does. - "You can have multiple `*args`" — no, only one.

Common follow-up: "Construct a signature that requires the caller to pass exactly one positional and exactly one keyword." — `def f(x, /, *, y): ...`. `x` is positional-only, `y` is keyword-only.

```
def signature_demo(
    pos_only,
    /,
    pos_or_kw,
    *args,
    kw_only,
    **kwargs,
):
    return {
        "pos_only": pos_only,
        "pos_or_kw": pos_or_kw,
        "args": args,
        "kw_only": kw_only,
        "kwargs": kwargs,
    }

print(signature_demo(
    1,          # pos_only
    2,          # pos_or_kw (could also be pos_or_kw=2)
    3, 4, 5,    # *args
    kw_only=10,
    extra="hi", # **kwargs
))
# {'pos_only': 1, 'pos_or_kw': 2, 'args': (3, 4, 5),
#  'kw_only': 10, 'kwargs': {'extra': 'hi'}}
```

Q30 · The argument-binding algorithm — order of operations

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, JP Morgan · Topic: Function arguments

The question (interviewer's verbatim phrasing):

When Python binds arguments to parameters, what's the order? Walk me through what happens when you call `f(1, 2, c=3)` against `def f(a, b, c, d=4)`.

The 30-second answer: Python binds positionals left-to-right first, then keyword arguments by name, then fills in defaults for unbound parameters, then collects leftover positionals into `*args` and leftover keywords into `**kwargs`. If any required parameter is unbound after that, `TypeError`. For `f(1, 2, c=3)`: `a=1`, `b=2` (positionals), `c=3` (keyword), `d=4` (default).

Step-by-step thought process: 1. Walk through the steps in order — that's the question. Positionals first, keywords next, defaults third, then variadics, then check. 2. State the conflict rules: passing the same parameter positionally and by keyword raises `TypeError: got multiple values`. 3. State the omission rule: a required parameter (no default) that ends up unbound raises `TypeError: missing 1 required positional argument`. 4. Mention `inspect.Signature.bind(*args, **kwargs)` — it runs this exact algorithm explicitly. Great for validating arguments before calling.

Edge cases to mention: - A keyword for a positional-only parameter is rejected — `f(a=1, b=2)` against `def f(a, b, /)` raises `TypeError`. - A keyword that doesn't match any parameter goes into `**kwargs` if present; otherwise `TypeError: unexpected keyword argument`. - Defaults are checked after positional + keyword binding, so providing a value (even `None`) defeats the default.

What NOT to say: - "Keywords are bound first, then positionals" — wrong order. - "Defaults are bound at call time" — defaults are filled in at call time but evaluated at definition time (Q22).

Common follow-up: "How would you use `inspect.Signature.bind` to validate?" — `sig = inspect.signature(f); ba = sig.bind(*args, **kwargs); ba.apply_defaults(); print(ba.arguments)`. That gives you the full bound argument dict, with defaults applied, ready to log or audit.

```
import inspect

def transfer(amount, currency, recipient, retries=3):
    return (amount, currency, recipient, retries)

sig = inspect.signature(transfer)
bound = sig.bind(500, "INR", recipient="merchant")
bound.apply_defaults()
print(bound.arguments)
# {'amount': 500, 'currency': 'INR', 'recipient': 'merchant', 'retries': 3}

# Conflict
try:
    transfer(500, "INR", "x", amount=600)
except TypeError as e:
    print(e) # transfer() got multiple values for argument 'amount'
```

Section D — Function introspection (5 Qs)

Q31 · `inspect.signature` — the modern way to read a signature

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Goldman · Topic: Introspection

The question (interviewer's verbatim phrasing):

How do you read a function's parameters at runtime? What does `inspect.signature` give you that `f.__defaults__` doesn't?

The 30-second answer: `inspect.signature(f)` returns a `Signature` object that exposes parameters as a mapping of name → `Parameter`. Each `Parameter` has `.kind` (positional-only / positional-or-keyword / VAR_POSITIONAL / keyword-only / VAR_KEYWORD), `.default`, and `.annotation`. It's the structured, post-PEP-570 replacement for the old `inspect.getfullargspec`. Use it for validation, documentation generators, and decorators that need to know what they're wrapping.

Step-by-step thought process: 1. State the API — `inspect.signature(f)` returns a `Signature`, `.parameters` is an `OrderedDict`-like mapping of name to `Parameter`. 2. State what's available per parameter — `.kind`, `.default`, `.annotation`, `.name`. Use `Parameter.empty` to detect missing defaults or annotations. 3. State why prefer over `getfullargspec` — `signature` handles all five parameter kinds correctly, follows `__wrapped__` for decorated functions, and works on built-ins where `getfullargspec` doesn't. 4.

Mention `signature.bind(*args, **kwargs)` for argument validation and `signature.parameters['x'].default` for default extraction.

Edge cases to mention: - For built-ins, `inspect.signature` works in 3.4+ thanks to signature info bundled with stdlib. Third-party C extensions may lack it. - `Parameter.empty` is a sentinel — comparison must be `p.default is Parameter.empty`, not `== None`. - Annotations come back as the Python objects themselves (`int`, `str`, `list[int]`), not strings — unless the module uses `from __future__ import annotations`, then they're strings.

What NOT to say: - "`getfullargspec` is the standard" — it was, but it's been superseded by `signature` since Python 3.3. - "`signature` doesn't work on lambdas" — it does; lambdas have parameters and `signature` reads them.

Common follow-up: "How would you list only the required parameters of a function?" — Iterate `sig.parameters.values()` and filter where `p.default is Parameter.empty` and `p.kind` is positional-or-keyword or keyword-only. Variadic params don't count as required.

```
import inspect

def transfer(amount: int, *, currency: str = "INR", recipient: str):
    pass

sig = inspect.signature(transfer)
print(sig)
# (amount: int, *, currency: str = 'INR', recipient: str)

for name, param in sig.parameters.items():
    print(name, param.kind.name, param.default, param.annotation)
# amount POSITIONAL_OR_KEYWORD <Parameter.empty> <class 'int'>
# currency KEYWORD_ONLY INR <class 'str'>
# recipient KEYWORD_ONLY <Parameter.empty> <class 'str'>
```

Q32 · `inspect.getfullargspec` vs `inspect.signature` — the older API

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Goldman · Topic: Introspection

The question (interviewer's verbatim phrasing):

Why does Python have two introspection APIs — `getfullargspec` and `signature` ?
Which should you reach for in 2026?

The 30-second answer: `getfullargspec` is the legacy API — flat tuples of `args`, `varargs`, `varkw`, `defaults`, `kwoonlyargs`, `kwoonlydefaults`, `annotations`. It pre-dates positional-only `/` and doesn't fully express modern signatures. `signature` is the recommended modern

replacement: structured, follows `__wrapped__`, handles `/`. Use `signature` in new code; `getfullargspec` only when you're maintaining old code or need the flat-tuple shape for some legacy reason.

Step-by-step thought process: 1. State the API divide — `getfullargspec` came in 3.0 to replace `getargspec`; `signature` came in 3.3 to replace both with a richer object model. 2. State the concrete limitations of `getfullargspec` — doesn't report positional-only parameters distinctly, doesn't follow `__wrapped__` chains, doesn't validate calls. 3. State the modern recommendation — `signature` is the documented "use this" API since 3.3. The PEP that introduced it (PEP 362) explicitly says so. 4. Mention the migration pattern — third-party libraries (Flask, Django, pydantic) all moved to `signature` years ago.

Edge cases to mention: - `getfullargspec` does not work on some built-in functions that `signature` can handle. - For decorators that want to expose the wrapped function's true signature, only `signature` follows `__wrapped__` automatically. - Both fail on C-extension functions that don't expose introspection metadata.

What NOT to say: - "They're the same" — they're not; `signature` is strictly more capable for modern Python. - "`getfullargspec` is deprecated" — it's not formally deprecated, just superseded. Old code still uses it.

Common follow-up: "What does `getfullargspec` return on `def f(a, /, b)`?" — It puts `a` and `b` both into the `args` list with no flag indicating `a` is positional-only. That's why it's unsuitable for 3.8+ signatures.

```
import inspect

def f(a, b=2, *args, c, **kwargs):
    pass

print(inspect.getfullargspec(f))
# FullArgSpec(args=['a', 'b'], varargs='args', varkw='kwargs',
#             defaults=(2,), kwoonlyargs=['c'], kwoonlydefaults=None, annotations={})

print(inspect.signature(f))
# (a, b=2, *args, c, **kwargs)

# Modern, structured access
for name, param in inspect.signature(f).parameters.items():
    print(name, param.kind.name)
```

Q33 · `inspect.getsource` — reading a function's source code

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft, Goldman · Topic: Introspection

The question (interviewer's verbatim phrasing):

How would you print out the source code of a function from inside another function? When does it fail?

The 30-second answer: `inspect.getsource(f)` returns the function's source as a string by reading the file from `f.__code__.co_filename`. It fails for functions defined in the REPL or `exec -d` strings (no source file), for built-ins (no Python source), and for compiled-only modules (`.pyc` without `.py`). Use `inspect.getsourcelines(f)` if you need the source and the starting line number.

Step-by-step thought process: 1. State what it returns and what it reads from. It opens the file, walks the AST, and slices out the function's lines. 2. State the failure modes — REPL definitions, lambdas built via `exec`, built-ins, and `.pyc`-only modules all raise `OSError: source code not available`. 3. State the related helpers — `inspect.getsourcefile(f)` for the path, `inspect.getsourcelines(f)` for `(lines, start_line)`, `inspect.getfile(f)` for the file the function lives in. 4. Mention the canonical use — building docs generators, error messages that show the offending function, and `pytest --tb=long` style tracebacks.

Edge cases to mention: - A function compiled with `dis` or written to a file by another process might have outdated source on disk; `getsource` reads the file, not the compiled bytecode. - `inspect.getsource(lambda: 1)` returns the line the lambda was defined on, but only if it could be identified — multiple lambdas on one line make this ambiguous. - For methods, `getsource(Cls.method)` works the same way.

What NOT to say: - "It reads the bytecode" — it reads the source file. Bytecode is `dis.dis(f)`. - "It works everywhere" — not in the REPL or for built-ins.

Common follow-up: "What does `inspect.getsource` return if the function was redefined in the REPL?" — `OSError`. The REPL doesn't persist source to a file, so there's nothing to read.

```
import inspect

def square(x):
    """Return x squared."""
    return x * x

print(inspect.getsource(square))
# def square(x):
#     """Return x squared."""
#     return x * x

print(inspect.getsourcefile(square))    # /path/to/this_file.py
lines, start = inspect.getsourcelines(square)
print(start, len(lines))                # 3 3
```

Q34 · Building a runtime argument validator

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Goldman · Topic: Introspection

The question (interviewer's verbatim phrasing):

Without using pydantic or any framework, write a decorator `@validate_types` that checks the types of arguments against the function's annotations at call time.

The 30-second answer: Read the signature with `inspect.signature`, bind the call's arguments with `sig.bind(*args, **kwargs)`, walk the bound arguments, and check each value against the parameter's `.annotation`. Raise `TypeError` on mismatch. Skip parameters whose annotation is `Parameter.empty`. Apply defaults with `bound.apply_defaults()` before checking if you want defaults validated too.

Step-by-step thought process: 1. Outline the algorithm — signature, bind, walk, check, raise. 2. State the annotation check — `isinstance(value, annotation)` for simple types. For generics like `list[int]`, `isinstance` doesn't work on the parameterized form; use `typing.get_origin` and `typing.get_args` for that. 3. State the practical scope — this works for basic types; for complex types (Unions, generics, Pydantic models), pydantic does this for you with much more care. 4. Mention the `functools.wraps` part — preserve the wrapped function's metadata so the decorator doesn't break introspection downstream.

Edge cases to mention: - Parameter without annotation should be skipped, not rejected. - `Optional[int]` is `int | None` — handle `None` as a separate case before `isinstance`. - The bind step itself raises if the call is malformed (wrong arity, unknown keyword), which is often what you want anyway.

What NOT to say: - "Use `isinstance(value, list[int])`" — Python 3.9+ accepts this for the unparameterized form (`isinstance(x, list)`) but `isinstance(x, list[int])` raises `TypeError: isinstance() argument 2 cannot be a parameterized generic`. - "Just use pydantic" — fine in production; this question is about understanding the underlying mechanism.

Common follow-up: "How would you extend this to validate return types?" — Wrap the call: `result = func(*args, **kwargs)`, then check `result` against `sig.return_annotation`. Same `isinstance` mechanics.

```

import inspect
from functools import wraps

def validate_types(func):
    sig = inspect.signature(func)
    @wraps(func)
    def wrapper(*args, **kwargs):
        bound = sig.bind(*args, **kwargs)
        bound.apply_defaults()
        for name, value in bound.arguments.items():
            ann = sig.parameters[name].annotation
            if ann is inspect.Parameter.empty:
                continue
            if not isinstance(value, ann):
                raise TypeError(
                    f"{name}={value!r} is not a {ann.__name__}"
                )
        return func(*args, **kwargs)
    return wrapper

@validate_types
def transfer(amount: int, currency: str):
    return f"{amount} {currency}"

print(transfer(500, "INR"))           # 500 INR
try:
    transfer("oops", "INR")
except TypeError as e:
    print(e)                          # amount='oops' is not a int

```

Q35 · Preserving signature inside a decorator

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Goldman, Microsoft · Topic: Introspection

The question (interviewer's verbatim phrasing):

If I wrap a function with `@decorator`, `inspect.signature(decorated)` should show the wrapped function's parameters. How do you make that work?

The 30-second answer: Apply `@functools.wraps(func)` on the wrapper. It sets `wrapper.__wrapped__ = func`, and `inspect.signature` follows that link by default. Without `wraps`, `signature(wrapper)` shows the wrapper's literal parameters (usually `*args, **kwargs`), which is useless for documentation and validation.

Step-by-step thought process: 1. State the rule — `wraps` sets `__wrapped__`. `signature` walks the chain via `follow_wrapped=True` (the default). 2. Show the contrast: a wrapper accepting

`*args, **kwargs` without `wraps` has `signature(wrapper) == (*args, **kwargs)`. With `wraps`, you get the wrapped function's signature. 3. State a counterexample: if your decorator deliberately changes the signature (adds a parameter), don't use `wraps` for signature copying — wrap manually and set `__wrapped__` thoughtfully. 4. Mention the override —

`inspect.signature(wrapper, follow_wrapped=False)` gives you the wrapper's literal signature when you need it for debugging.

Edge cases to mention: - Stacked decorators chain `__wrapped__` — `signature` walks until it finds a function without `__wrapped__`. - `wraps` updates several attributes — `__name__`, `__module__`, `__qualname__`, `__doc__`, `__dict__`, `__wrapped__`. Forgetting one is the cause of many mysterious framework bugs. - For class-based decorators, use `functools.update_wrapper(self, func)` inside `__init__` to get the same behaviour.

What NOT to say: - "`wraps` is just for the name" — it's for the whole bundle of metadata. - "You can't have a wrapper with a different signature" — you can; just don't use `wraps` for the signature copy, or apply `update_wrapper` selectively.

Common follow-up: "How does `inspect.signature` decide what to follow?" — It checks the `__wrapped__` attribute on the input. If present, it recurses. You can disable with `follow_wrapped=False`.

```
from functools import wraps
import inspect

def trace(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@trace
def transfer(amount: int, *, currency: str = "INR"):
    return amount, currency

print(inspect.signature(transfer))
# (amount: int, *, currency: str = 'INR') # the wrapped function's signature

print(inspect.signature(transfer, follow_wrapped=False))
# (*args, **kwargs) # the wrapper's literal signature

print(transfer.__wrapped__)
# <function transfer at 0x...> (original)
```

Section E — `functools` deep (5 Qs)

Q36 · `functools.reduce` — when it's the right call

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JP Morgan, Goldman · Topic: `functools`

The question (interviewer's verbatim phrasing):

What does `functools.reduce` do, and when would you actually use it over a `for` loop or `sum` ?

The 30-second answer: `reduce(func, iterable[, initial])` applies a two-argument function cumulatively left-to-right, collapsing the iterable to a single value. `reduce(lambda x, y: x + y, [1, 2, 3], 0)` is `((0 + 1) + 2) + 3`. Reach for it when there's no specialized aggregator (no `sum`, `min`, `max`, `any`, `all`), and the reduction isn't trivially expressible as a loop. In practice that's narrow — dict-merging, function-chaining, and a few math reductions.

Step-by-step thought process: 1. State the mechanics — two-arg function, left-fold, optional initial value. Without initial, the first element of the iterable seeds the accumulator (and an empty iterable raises `TypeError`). 2. State when to prefer something else: `sum` for addition, `math.prod` for multiplication, `max` / `min` for extremes, `any` / `all` for boolean reductions, `operator.add` etc. for chaining. 3. State the legitimate uses: merging dicts (`reduce(operator.or_, dicts)`), composing functions, computing cumulative state where no built-in exists. 4. Mention Python's history — Guido moved `reduce` out of built-ins in Python 3 because most uses are clearer as loops or specialized functions. It still lives in `functools` for the cases where it really is the cleanest.

Edge cases to mention: - No initial value plus empty iterable raises `TypeError: reduce() of empty iterable with no initial value`. - The function is applied left-to-right; if order matters (e.g., function composition), state it clearly in the function. - For commutative operations (`+`, `*`, `|`), the order doesn't matter; for non-commutative ones (string concat with separator, function composition), it does.

What NOT to say: - "`reduce` is always faster than a loop" — usually not; readability matters more for most code. - "Python doesn't have fold" — it does; `reduce` is left-fold.

Common follow-up: "How would you compute the product of a list?" — `math.prod(lst)` (Python 3.8+). Before 3.8: `reduce(operator.mul, lst, 1)`. `math.prod` is the modern idiom.

```

from functools import reduce
from operator import mul, or_

print(reduce(mul, [1, 2, 3, 4], 1))          # 24
print(reduce(lambda a, b: a + b, [1, 2, 3])) # 6

# Merging dicts
dicts = [{"a": 1}, {"b": 2}, {"c": 3}]
print(reduce(or_, dicts))                   # {'a': 1, 'b': 2, 'c': 3} (3.9+)

# When something else is clearer
print(sum([1, 2, 3]))                      # 6 — prefer sum
import math
print(math.prod([1, 2, 3, 4]))              # 24 — prefer math.prod

```

Q37 · `functools.cache` vs `lru_cache` — which to use in 3.9+

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Goldman, Microsoft · Topic: functools

The question (interviewer's verbatim phrasing):

What's the difference between `@cache` and `@lru_cache` ? When do you want one over the other?

The 30-second answer: `@cache` (Python 3.9+) is an unbounded memoizer — every input/output pair stays in memory forever. `@lru_cache(maxsize=N)` is bounded — when the cache hits `N` entries, the least recently used one is evicted. Use `@cache` when the input domain is small and bounded (you'll never have more than a few hundred unique inputs). Use `@lru_cache` for everything else, because unbounded caches leak in long-running services.

Step-by-step thought process: 1. State the bounded vs unbounded distinction. `cache` was added in 3.9 as sugar for `lru_cache(maxsize=None)`. Same machinery, different default. 2. State the requirements — both require arguments to be hashable. Lists and dicts as arguments will raise `TypeError`. Use tuples and frozensets if you must. 3. State the operational ops — `.cache_info()` returns hits, misses, maxsize, cursize. `.cache_clear()` empties it. Useful for debugging and tests. 4. Mention the threading note — `lru_cache` is thread-safe for cache reads. Cache misses (which call the function) might be called concurrently.

Edge cases to mention: - Default `maxsize=128` for `lru_cache` without args is a historical default; tune for your workload. - Methods get tricky — `lru_cache` on a method keeps every instance alive (cache key includes `self`). Use `cached_property` for instance-level caching, or `functools.cache` on a regular function called with the instance's identifier. - `typed=True` separates int and float — `f(1)` and `f(1.0)` would otherwise share a cache entry.

What NOT to say: - "`@cache` is faster" — same dispatch; the difference is the eviction policy. - "Use `@cache` everywhere" — unbounded is dangerous in long-running services. Memory leak waiting to happen.

Common follow-up: "How would you cap memory on `@cache`?" — Switch to `@lru_cache(maxsize=10_000)`. Measure your hit-rate vs memory; tune accordingly.

```
from functools import cache, lru_cache

@cache
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print(fib(30))                # 832040
print(fib.cache_info())       # CacheInfo(hits=28, misses=31, maxsize=None, currsize=31)

@lru_cache(maxsize=128)
def expensive(x):
    return x * x * x

expensive(10); expensive(20); expensive(10)
print(expensive.cache_info()) # CacheInfo(hits=1, misses=2, maxsize=128, currsize=2)
expensive.cache_clear()
```

Q38 · `functools.singledispatch` — Pythonic overloading

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Goldman · Topic: functools

The question (interviewer's verbatim phrasing):

Python doesn't have method overloading by signature. How do you dispatch on the type of the first argument idiomatically?

The 30-second answer: `@functools.singledispatch` turns a function into a single-dispatch generic — register type-specific implementations with `.register(type)`. `format_for(123)` calls the int version; `format_for("abc")` calls the str version. It's the standard-library answer to type-based polymorphism without writing `if isinstance(x, int)` chains.

Step-by-step thought process: 1. State the API — `@singledispatch` on the base function, then `@func.register(SomeType)` on each specialized version. 2. State the dispatch rule — by the type of the *first* argument. Single dispatch, not multiple. Multi-dispatch needs a third-party library (e.g., `multipledispatch`). 3. State the resolution — subclass lookups use MRO. `register(int)` matches `bool` (since `bool` subclasses `int`) unless you register `bool` first. 4. Mention `singledispatchmethod` (3.8+) for the same pattern on instance methods, and the typing-hint-based

variant: `@func.register` followed by a function annotated `def _(arg: int): ...` (Python 3.7+).

Edge cases to mention: - Dispatch only on the first arg; subsequent args are passed through unchanged. - `None` doesn't have a useful type for dispatch; register `type(None)` if you need it. - ABCs (e.g., `Mapping`, `Sequence`) work as dispatch targets — `register(Mapping)` matches dicts, ordered dicts, etc.

What NOT to say: - "Python doesn't support overloading" — it does, via `singledispatch` and `singledispatchmethod`. Not signature-based, but type-based. - "Use `if isinstance` chains" — that's what `singledispatch` exists to replace.

Common follow-up: "How would you dispatch on the second argument?" — Not with `singledispatch`. Either restructure to put the dispatch-type first, or use `multipledispatch` from PyPI.

```
from functools import singledispatch

@singledispatch
def serialize(value):
    return str(value)

@serialize.register
def _(value: int):
    return f"int:{value}"

@serialize.register
def _(value: list):
    return "[" + ",".join(serialize(v) for v in value) + "]"

print(serialize("hi"))           # hi
print(serialize(42))             # int:42
print(serialize([1, 2, "x"]))    # [int:1,int:2,x]
```

Q39 · `functools.cached_property` — caching at the instance level

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Postman · Topic: functools

The question (interviewer's verbatim phrasing):

What's the difference between `@property` and `@functools.cached_property`? When would you reach for the cached form?

The 30-second answer: `@property` recomputes the value on every access. `@cached_property` (3.8+) computes once per instance and stores the result on the instance's `__dict__`. Use `cached_property` when the computation is expensive and the underlying state doesn't change for the lifetime of the instance. The cache lives until the instance is garbage-collected; there's no explicit invalidation API.

Step-by-step thought process: 1. State the dispatch difference — `property` is always a descriptor that delegates to the getter on every access. `cached_property` writes the computed value to `instance.__dict__[attr_name]` on first access, so subsequent accesses hit the dict directly (no descriptor dispatch). 2. State the requirement — the instance must have a writable `__dict__`. Classes with `__slots__` and no `__dict__` slot will raise `TypeError`. 3. State the invalidation pattern — there's no built-in invalidate; delete the attribute (`del instance.attr_name`) to force recomputation on next access. 4. Mention the threading note — first-access initialization is not synchronized. Two threads can compute the value simultaneously; the second write wins. For thread-safe caching, add a lock.

Edge cases to mention: - `cached_property` requires the class to have a writable `__dict__`. `__slots__` without `'__dict__'` breaks it. - It works on instance methods only — it's an instance-level cache, not class-level. For class-level, use `@lru_cache` on a regular method or a `classmethod`. - The cache key is implicit (`self`); no support for parameterized caching like `lru_cache`.

What NOT to say: - "`cached_property` is thread-safe" — first-access is not synchronized; race conditions on construction. - "Use it as a drop-in for `@property`" — only if the property is genuinely constant after first access. Otherwise you get stale data.

Common follow-up: "How would you invalidate a `cached_property`?" — `del obj.attr_name`. Or, more robustly, set the attribute manually with `obj.attr_name = new_value`. Both bypass the descriptor on next access until the instance dict is cleared.

```

from functools import cached_property

class Report:
    def __init__(self, raw):
        self.raw = raw

    @cached_property
    def parsed(self):
        print("parsing...")
        return [int(x) for x in self.raw.split(",")]

r = Report("1,2,3,4")
print(r.parsed)           # parsing... \n [1, 2, 3, 4]
print(r.parsed)           # [1, 2, 3, 4] — no re-parse

# Invalidate
del r.parsed
print(r.parsed)           # parsing... \n [1, 2, 3, 4] — recomputed

```

Q40 · `functools.total_ordering` — generating the comparison dunder

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Goldman, JP Morgan · Topic: functools

The question (interviewer's verbatim phrasing):

If I define `__eq__` and `__lt__` on a class, can `@functools.total_ordering` generate the other three for me? What's the trade-off?

The 30-second answer: Yes — `@total_ordering` on a class that defines `__eq__` plus any one of `__lt__`, `__le__`, `__gt__`, `__ge__` generates the missing three using derivations from the one you provide. Trade-off: the generated methods are slower than hand-written ones (extra dispatch through the helper), and the generation only fires once at class definition. For performance-critical comparisons, write all four by hand; for ordinary code, `total_ordering` is the right call.

Step-by-step thought process: 1. State the contract — define `__eq__` and any one ordering dunder, decorate, get the other three for free. 2. State the implementation hint — the helper derives `__gt__` from `not __lt__ and not __eq__`, and similarly for `__ge__` and `__le__`. Three short comparisons per call, instead of one. 3. State the perf trade-off — a generated `__gt__` is ~2x slower than a hand-written one because of the chained calls. For tight loops, hand-write. 4. Mention the alternative — `@dataclass(order=True)` generates all four ordering dunder from the field order. Better for plain data containers.

Edge cases to mention: - `total_ordering` does not generate `__hash__` — if you override `__eq__`, you also need `__hash__` (or `__hash__ = None`). - If your `__eq__` returns

`NotImplemented`, the generated methods propagate that correctly. - `@dataclass(order=True)` generates the four ordering methods plus `__eq__` for you in one go — usually the better choice when you control the class.

What NOT to say: - "`total_ordering` generates `__eq__` too" — it doesn't; you must provide `__eq__`. - "Generated methods are as fast as hand-written" — they're noticeably slower.

Common follow-up: "*Why doesn't Python generate the comparisons automatically?*" — Because comparison semantics differ per type, and the language designers chose explicit over implicit. `total_ordering` exists for the common case where you want them all derived from one rule.

```
from functools import total_ordering

@total_ordering
class Money:
    def __init__(self, paise):
        self.paise = paise
    def __eq__(self, other):
        return isinstance(other, Money) and self.paise == other.paise
    def __lt__(self, other):
        return self.paise < other.paise
    def __hash__(self):
        return hash(self.paise)

a, b = Money(100), Money(200)
print(a < b, a <= b, a > b, a >= b)      # True True False False
print(a == b, a != b)                   # False True
```

Section F — Higher-order functions (10 Qs)

Q41 · `map` and `filter` — when to prefer a comprehension

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy · Topic: HOFs

The question (interviewer's verbatim phrasing):

`map(str, nums)` vs `[str(n) for n in nums]` — which would you write in production?

The 30-second answer: The list comprehension. PEP 8 and most Python style guides prefer comprehensions over `map` / `filter` for readability — `[str(n) for n in nums]` reads top-to-bottom, while `map(str, nums)` requires the reader to know that `map` returns an iterator (not a list) in Python 3. Reach for `map` when the function is already named and the result feeds straight into another iterable consumer (`list(map(int, lines))` is fine).

Step-by-step thought process: 1. State the readability rule — comprehensions are the Python-canonical idiom. 2. State the type difference — `map` returns a `map` object (lazy iterator), not a list. `list(map(...))` materializes; without `list`, you have a one-shot stream. 3. State when `map` is acceptable — when the function is already named, the comprehension would be `[f(x) for x in xs]` (no real transformation beyond the call), and the result feeds a consumer like `list`, `set`, `dict`. 4. Mention the historical context — Guido considered removing `map` and `filter` from Python 3. They survived, but comprehensions are the official preferred style.

Edge cases to mention: - `map(func, iter1, iter2)` calls `func(x, y)` for parallel iteration — useful but less obvious. A comprehension for the same is `[func(x, y) for x, y in zip(iter1, iter2)]`. - `map` returns a one-shot iterator. Iterating it twice yields the second time empty. - `filter(None, iterable)` keeps truthy elements — equivalent to `[x for x in iterable if x]`.

What NOT to say: - "`map` is functional, comprehensions are imperative" — both are equivalent in effect; the style choice is about readability. - "`map` is faster" — in CPython, comprehensions are often the same speed or faster because they avoid Python-level function-call overhead unless the function itself is C-implemented.

Common follow-up: "When would you use a generator expression instead of `map` or a comprehension?" — When the result is consumed once and the iterable is large. `sum(x*x for x in nums)` is constant memory; `sum([x*x for x in nums])` allocates the whole list first.

```
nums = [1, 2, 3]

print(list(map(str, nums)))          # ['1', '2', '3']
print([str(n) for n in nums])        # ['1', '2', '3']

# When map is fine — function is named, consumed by list/set/dict
ids = list(map(int, ["1", "2", "3"]))

# When comprehension wins — any transformation beyond a single call
prices = [{"p": 20}, {"p": 80}, {"p": 50}]
labels = [f"{x['p']}rs" for x in prices]
```

Q42 · `sorted` with `key=` — the workhorse pattern

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Cred, Swiggy, JP Morgan · Topic: HOFs

The question (interviewer's verbatim phrasing):

How do you sort a list of dicts by two keys — `priority` desc and `created_at` asc?

The 30-second answer: Pass a `key=` callable that returns a tuple. Sort is stable, so the simplest pattern is `sorted(items, key=lambda d: (-d["priority"], d["created_at"]))` — negate `priority` to flip its direction. For non-numeric desc sorts, do the sort in two passes (sort by secondary first, then by primary), relying on stability.

Step-by-step thought process: 1. State the `key=` rule — Python sorts by the value of `key(item)` for each item, in ascending order. Tuple keys sort lexicographically: first element first, then second, etc. 2. State the negation trick for numeric desc — multiply by `-1` to reverse direction within a tuple key. 3. State the stable-sort fallback for non-numeric desc — sort by the secondary key first, then by the primary key. Python's sort is stable (Timsort), so the secondary order is preserved. 4. Mention `operator.itemgetter` and `attrgetter` as the C-implemented alternatives — faster for hot loops.

Edge cases to mention: - `reverse=True` reverses the whole sort, not just the primary key. Useful only when both keys should be desc. - Mixing types (`int` and `str`) in the key tuple raises `TypeError` in Python 3. - `None` values cannot be compared to other types in Python 3; either filter them out first or wrap them in a default.

What NOT to say: - "`sort` modifies in place; `sorted` returns a new list" — true and worth saying, but doesn't answer the question. - "Use `cmp` parameter" — `cmp` is Python 2; Python 3 only has `key`. Use `functools.cmp_to_key` if you genuinely need cmp-style.

Common follow-up: "How is `sorted` implemented?" — Timsort, a hybrid of merge sort and insertion sort. Stable. $O(n \log n)$ worst case, often near $O(n)$ on partially-sorted input.

```
tasks = [
    {"id": 1, "priority": 3, "created_at": "2026-01-10"},
    {"id": 2, "priority": 5, "created_at": "2026-01-08"},
    {"id": 3, "priority": 3, "created_at": "2026-01-05"},
]

# Priority desc, created_at asc
print(sorted(tasks, key=lambda t: (-t["priority"], t["created_at"])))
# [{'id': 2, 'priority': 5, ...},
#  {'id': 3, 'priority': 3, '2026-01-05'},
#  {'id': 1, 'priority': 3, '2026-01-10'}]

# Faster: itemgetter when keys are simple
from operator import itemgetter
print(sorted(tasks, key=itemgetter("priority", "created_at")))
```

Q43 · `min` / `max` with `key=` — picking the right element

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Swiggy, JP Morgan · Topic: HOFs

The question (interviewer's verbatim phrasing):

Given a list of orders, return the one with the highest amount. Two ways — `max(key=...)` and a `for` loop. Which would you write?

The 30-second answer: `max(orders, key=lambda o: o["amount"])`. One line, intent-clear, runs in C-speed for the comparison. Reach for the `for` loop only if you also need running state (running total, count of ties) or if `max` doesn't handle the empty-input case the way you want.

Step-by-step thought process: 1. State the idiom — `max(iter, key=callable)` returns the element with the largest key. `min` is symmetric. 2. State the empty-input behaviour — `max([])` raises `ValueError`. Pass `default=` (Python 3.4+) to get a fallback: `max([], default=None)`. 3. State the tie-breaking — `max` returns the first element with the maximum key; `min` returns the first with the minimum. Sort order on ties matches input order. 4. Mention `heapq.nlargest(n, iter, key=...)` for the top-N case — fewer comparisons than a full sort when `n` is small.

Edge cases to mention: - Mixing comparable and non-comparable types raises `TypeError`. - `max` over a one-shot generator consumes it. You can't call `len(gen)` afterwards. - `max(*args, key=...)` over individual arguments works too: `max(a, b, c, key=abs)`.

What NOT to say: - "Use a `for` loop for clarity" — `max(items, key=...)` is the Python-canonical clarity for this. - "`max` is slower than a loop" — generally false in CPython; `max` is implemented in C.

Common follow-up: "What if you need the top 3, not just the max?" — `heapq.nlargest(3, items, key=lambda x: x["amount"])`. Faster than `sorted(...)[:3]` when the list is large and `n` is small.

```
orders = [{"id": 1, "amount": 500}, {"id": 2, "amount": 1500}, {"id": 3, "amount": 200}]

print(max(orders, key=lambda o: o["amount"]))      # {'id': 2, 'amount': 1500}
print(min(orders, key=lambda o: o["amount"]))      # {'id': 3, 'amount': 200}

# Empty handling
print(max([], default=None))                       # None

# Top-N
import heapq
top2 = heapq.nlargest(2, orders, key=lambda o: o["amount"])
print(top2)                                         # [{'id': 2, ...}, {'id': 1, ...}]
```

Q44 · `any` and `all` — short-circuit semantics

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Cred, Postman, Goldman · Topic: HOFs

The question (interviewer's verbatim phrasing):

When `any(gen)` returns `True`, how many elements of `gen` were consumed? What about `all`?

The 30-second answer: `any` stops at the first truthy element — only that many elements consumed. `all` stops at the first falsy element. Both short-circuit and never iterate past the decisive value. That makes them safe to call on infinite generators when you know the answer comes early: `any(is_root(n) for n in nums)` returns as soon as one matches.

Step-by-step thought process: 1. State the short-circuit rule — `any` is OR, stops on first `True`; `all` is AND, stops on first `False`. 2. State the empty-input behaviour — `any([]) == False`, `all([]) == True`. The latter is "vacuous truth" and surprises people regularly. 3. State the generator-friendly property — both are streaming. `any(huge_generator)` doesn't materialize anything; it consumes until the decision is made. 4. Mention the truthiness rule — Python's notion of truthy applies. `any([0, "", None, [], "x"])` is `True` because `"x"` is truthy.

Edge cases to mention: - Don't write `any([f(x) for x in xs])` — the list comp evaluates `f` on every element before `any` even runs. Use `any(f(x) for x in xs)` (generator) for short-circuit. - `all` on an empty iterable returning `True` is the source of the "no exceptions = no problems" pattern, but also a common bug source ("why is my validator passing an empty list?"). - `any(...)` and `all(...)` over dicts iterate the keys by default.

What NOT to say: - "`any` consumes the whole generator" — it short-circuits. - "`all([])` is `False`" — it's `True`. Vacuous truth.

Common follow-up: "How is `any(gen)` different from `next(filter(None, gen), False)`?" — `any` returns a `bool` (always `True` / `False`); `filter`-based version returns the matching element or the default. Use `any` when you need the boolean; `next(filter(...))` when you need the actual matching element.

```
def gen():
    for n in [0, 0, 1, 1/0, 5]:          # 1/0 would raise ZeroDivisionError if reached
        yield n

print(any(gen()))                      # True — stops at the first 1, never reaches 1/0
# All empties
print(all([]))                        # True — vacuous truth
print(any([]))                        # False

# Short-circuit pattern
def is_valid(x): return x > 0
print(any(is_valid(x) for x in [-1, -2, 5, -3])) # True
```

Q45 · Currying via `partial`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, JP Morgan · Topic: HOFs

The question (interviewer's verbatim phrasing):

Show me how to "curry" a function like `add(x, y, z)` so I can write `add(1)(2)(3)`. Then tell me if `functools.partial` is the same thing.

The 30-second answer: True curry would be `def curry(f): return lambda x: lambda y: lambda z: f(x, y, z)` — one argument per layer. `functools.partial` is partial application: bind any subset of arguments, get back a callable expecting the rest. They overlap but aren't identical: curry forces one-at-a-time, partial doesn't. In Python, you almost always want `partial`; true curry is a haskellism that rarely fits.

Step-by-step thought process: 1. State the conceptual distinction — currying transforms an n-ary function into n nested unary functions; partial application binds some args, leaves the rest. 2. Show a curried `add` and a partialized `add`. Both reduce arity, but the curried version mandates the call shape. 3. State the Python idiom — `partial` is in the stdlib and is the canonical way to "bind some args." True curry is usually overkill and ugly. 4. Mention the third-party `toolz.curry` if a candidate insists on auto-currying — there's a library, but the language doesn't push you toward it.

Edge cases to mention: - Curry over `*args` is meaningless — you don't know when to stop. Curry is for fixed-arity functions. - Decorator-based curry exists in libraries; reinventing it is rarely worth it. - `partial` can bind keyword args; curry typically only handles positionals.

What NOT to say: - "Python has built-in curry" — it doesn't; `partial` is the closest. - "Curry and partial are interchangeable" — they overlap but differ in the call shape they accept.

Common follow-up: "When would you want true curry over `partial`?" — Almost never in idiomatic Python. The only use case is when interoperating with code (e.g., functional pipelines, parser combinators) that expects single-arg callables at every step.

```
from functools import partial

def add(x, y, z):
    return x + y + z

# True curry
def curry3(f):
    return lambda x: lambda y: lambda z: f(x, y, z)

cadd = curry3(add)
print(cadd(1)(2)(3))          # 6

# partial
print(partial(add, 1, 2)(3))   # 6
print(partial(partial(add, 1), 2)(3)) # 6 – same effect, ergonomically worse
```

Q46 · Function composition patterns

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman · Topic: HOFs

The question (interviewer's verbatim phrasing):

Beyond a single `compose(f, g)`, how would you build a multi-step data pipeline using composed functions?

The 30-second answer: Either compose via `reduce`-based composition (`pipe(strip, lower, validate, format)`) or accept a list and `for`-loop through it (`for step in steps: value = step(value)`). The pipe pattern is idiomatic when the steps are first-class callables; the loop is clearer when you also need logging, retries, or per-step error handling. Don't over-engineer — three explicit lines beats a clever six-deep composition.

Step-by-step thought process: 1. State the two implementations — `reduce`-based pipe vs explicit loop. Show both. 2. State when each wins — pipe is cleaner for short, side-effect-free chains. Loop is better when you need logging, error handling, or step-by-step state. 3. State the readability ceiling — past 4-5 steps, the pipe form becomes hard to debug because tracebacks show generic lambda frames. Switch to named functions. 4. Mention `toolz.compose`, `functools.compose`, and the `pipe` operator emulation libraries — there are options, but the stdlib doesn't ship one because the right answer is "write it inline if you need it."

Edge cases to mention: - All steps must be unary (one argument). For multi-arg pipelines, wrap the state in a tuple or dict. - Errors raised mid-pipeline propagate; the chain stops. Wrap in `try/except` per step if you need partial-success semantics. - The pipe pattern doesn't preserve `__name__` — debugger output is `<lambda>`. Wrap with `wraps` if you'll grep logs.

What NOT to say: - "Use a class for everything" — overkill for a 3-step transform. - "Python has `|` for pipelines" — only in some third-party libraries; not in the stdlib.

Common follow-up: "How would you add logging between each step?" — Wrap each step in a `log_wrap(name, func)` decorator that prints before and after, then compose the wrapped versions. That's the right place to use `functools.wraps`.

```
from functools import reduce

def pipe(*funcs):
    return lambda x: reduce(lambda v, f: f(v), funcs, x)

normalize = pipe(str.strip, str.lower, lambda s: f"USER:{s}")
print(normalize(" RAZORPAY "))      # USER:razorpay

# Loop form — easier to add per-step logging
def run_pipeline(value, steps):
    for step in steps:
        value = step(value)
    return value

print(run_pipeline(" RAZORPAY ", [str.strip, str.lower, lambda s: f"USER:{s}"]))
```

Q47 · When HOFs are clearer than `for` loops

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JP Morgan, Goldman · Topic: HOFs

The question (interviewer's verbatim phrasing):

Give me three cases where a higher-order function is more readable than a `for` loop, and one where the loop wins.

The 30-second answer: HOFs win when (1) you're transforming each element via a named function (`map(parse, lines)`), (2) you're aggregating with a named reducer (`sum` , `max` , `any`), and (3) you're sorting by a clear key (`sorted(orders, key=...)`). Loops win when you need running state, multiple things happening per iteration, or per-iteration error handling — anything where the body would be more than one expression.

Step-by-step thought process: 1. State the three HOF wins — single-purpose transforms, aggregations, sorts. All have a Python-stdlib idiom. 2. State the loop win — anything with multi-statement bodies, accumulators, side effects, or branching. 3. State the readability rule of thumb — if you'd need parentheses around the expression to make it fit, the HOF is too complex; switch to a `for` loop. 4. Mention the comprehension as a middle ground — `[x.upper() for x in xs if len(x) > 3]` reads better than `filter(lambda x: len(x) > 3, map(str.upper, xs))`.

Edge cases to mention: - "Functional purity" isn't a Python virtue — clarity is. Don't compose for the sake of composing. - HOFs are fine even in performance-sensitive code; `sum`, `max`, `min` are C-implemented. - The line between HOF and comprehension is blurry — both are functional in spirit. Pick whichever reads cleaner per case.

What NOT to say: - "Python isn't a functional language, avoid HOFs" — Python embraces functional patterns where they fit; HOFs are first-class citizens. - "Use HOFs everywhere for purity" — purity isn't the Python goal; readability is.

Common follow-up: "Why is `sum(x*x for x in nums)` better than `total = 0; for x in nums: total += x*x; return total`?" — Three lines compressed into one, intent is `sum`, no risk of forgetting to reset `total`. The loop form only beats it when the body has more going on.

```
# HOF win — aggregation
print(sum(x * x for x in [1, 2, 3]))    # 14

# HOF win — sort by key
orders = [{"a": 5}, {"a": 1}]
print(sorted(orders, key=lambda o: o["a"]))

# HOF win — transformation
print(list(map(int, ["1", "2", "3"])))

# Loop win — multi-statement body with logging
running = 0
for x in [1, 2, 3]:
    if x % 2:
        print(f"odd: {x}")
        running += x
print(running)                        # 4
```

Q48 · Lazy evaluation with `itertools`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Goldman · Topic: HOFs

The question (interviewer's verbatim phrasing):

Walk me through three `itertools` functions you actually reach for, and what each replaces in naive code.

The 30-second answer: `chain` flattens any number of iterables into one stream (replaces nested `for` loops over `for x in [a, b, c]: for y in x: ...`). `islice` slices any iterable without materializing it (replaces `list(gen)[:10]`, which loads everything). `groupby` walks an

iterable and yields runs of consecutive equal-keyed elements (replaces hand-written run-length encoding). All three are lazy — they compose with each other without intermediate lists.

Step-by-step thought process: 1. State the three picks and what each replaces. Use names every Python writer should know. 2. State the laziness rule — `itertools` returns iterators, not lists. `list(islice(gen, 10))` materializes; `islice` alone doesn't. 3. State the `groupby` warning — it requires the input to be pre-sorted by the same key for "all duplicates grouped" semantics. Otherwise you get runs, not groups. 4. Mention other regulars — `chain.from_iterable`, `accumulate`, `tee`, `product`, `combinations`, `permutations`. Most have well-defined use cases; learning them pays off.

Edge cases to mention: - `tee(iter, n)` splits an iterator into `n` parallel iterators — useful when you need to consume the same source twice. - `groupby` is one of the most-mistaken stdlib functions; almost every bug is "I forgot to sort first." - `islice` doesn't support negative indices; for that, you'd materialize and slice the list.

What NOT to say: - "`itertools` is slower" — generally faster than equivalent Python loops because the building blocks are C-implemented. - "Use `itertools` for everything" — for one-off two-line problems, a comprehension is cleaner.

Common follow-up: "How would you `groupby` a list of dicts by a key?" — Sort first, then group: `sorted(items, key=lambda d: d["category"])`, then `itertools.groupby(sorted_items, key=lambda d: d["category"])`.

```
from itertools import chain, islice, groupby

# chain — flatten without nested for
print(list(chain([1, 2], [3, 4], [5])))      # [1, 2, 3, 4, 5]

# islice — slice a generator without materializing
def naturals():
    n = 1
    while True:
        yield n; n += 1

print(list(islice(naturals(), 5)))          # [1, 2, 3, 4, 5]

# groupby — only useful on pre-sorted input
items = sorted([{"cat": "A"}, {"cat": "B"}, {"cat": "A"}], key=lambda d: d["cat"])
for k, g in groupby(items, key=lambda d: d["cat"]):
    print(k, list(g))
# A [{"cat": 'A'}, {"cat": 'A'}]
# B [{"cat": 'B'}]
```

Q49 · Avoiding the late-binding trap in generator expressions

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman · Topic: HOFs

The question (interviewer's verbatim phrasing):

Predict the output: `funcs = [(lambda: i) for i in range(3)]; print([f() for f in funcs])`. Then fix it.

The 30-second answer: Output is `[2, 2, 2]` — same late-binding bug as Q15, just in a comprehension form. Each lambda captures the variable `i`, not the value at construction time. Fix with default-argument capture: `[(lambda i=i: i) for i in range(3)]` → `[0, 1, 2]`. The default `i=i` evaluates at lambda construction (right then, inside the comp's iteration), binding the current value.

Step-by-step thought process: 1. Recall the late-binding rule from Q15 — closures capture names, not values. Inside a comprehension, the iteration variable still has one binding. 2. State the fix — default-argument capture. It looks weird (`i=i`) but it's the standard idiom and every Python writer should recognize it. 3. State the alternative — `functools.partial(lambda i: i, i)`. Equally correct, slightly less idiomatic. 4. State the broader lesson — any time you build closures in a loop or comprehension, decide whether you want late or early binding. If early (almost always), bind with default args or `partial`.

Edge cases to mention: - Generator expressions have the same bug — `gen = (lambda: i for i in range(3))`; consuming `gen` later and calling the lambdas all see the final `i`. - Inside a regular comprehension, the iteration variable is local to the comp scope (since Python 3) — but the closures still share the cell within that scope. - Class-body closures have an even weirder interaction; see Q12 (class-body scope isn't searched by methods).

What NOT to say: - "Use a regular `for` loop instead" — same bug; the issue is the closure, not the comprehension. - "Python is broken here" — it's deliberate lexical-scoping behaviour. The fix is well-known.

Common follow-up: "What's the difference between `lambda i=i: i` and `lambda: i`?" — The first has a parameter `i` with a default value evaluated immediately (early-binding). The second is a no-arg lambda that reads `i` from the closure when called (late-binding). For closures over loop variables, default-arg capture is the standard fix.

```
# The bug
funcs = [(lambda: i) for i in range(3)]
print([f() for f in funcs])           # [2, 2, 2]

# Fix 1 – default argument capture
funcs_fixed = [(lambda i=i: i) for i in range(3)]
print([f() for f in funcs_fixed])    # [0, 1, 2]

# Fix 2 – functools.partial
from functools import partial
funcs_partial = [partial(lambda i: i, i) for i in range(3)]
print([f() for f in funcs_partial])  # [0, 1, 2]
```

Q50 · Decorator factories — closures all the way down

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Microsoft, Goldman · Topic: HOFs + closures

The question (interviewer's verbatim phrasing):

Write a `@retry(times=3, delay=1)` decorator that retries the wrapped function on exception, delaying between attempts. Walk through what each layer of nesting is for.

The 30-second answer: A parameterized decorator is three nested functions: the outer captures the decorator arguments (`times` , `delay`), the middle (the actual decorator) captures the wrapped function, the innermost (the wrapper) runs the retry logic and is what replaces the original. Each layer is a closure over its enclosing scope's variables. `functools.wraps` on the innermost preserves the wrapped function's metadata.

Step-by-step thought process: 1. State the three layers and what each captures. Outer: parameters. Middle: the function being decorated. Inner: per-call retry logic. 2. Walk through `@retry(times=3, delay=1)` : `retry(times=3, delay=1)` runs first and returns the actual decorator; that decorator is then applied to `func` , returning the wrapper. 3. State the implementation — `for attempt in range(times)` , `try: return func(*args, **kwargs)` , `except: time.sleep(delay)` (with the last attempt re-raising). 4. Mention `functools.wraps` on the innermost wrapper — preserves `__name__` , `__doc__` , `__wrapped__` . Without it, you lose introspection.

Edge cases to mention: - Use `exception_class` (defaulting to `Exception`) as a parameter to scope the retry to specific exceptions. - Add jitter (`delay * random.uniform(0.5, 1.5)`) to avoid thundering-herd retries in production. - For async functions, the wrapper needs `await func(*args, **kwargs)` and `asyncio.sleep` — write a separate `aretry` or detect `inspect.iscoroutinefunction(func)` .

What NOT to say: - "Decorators are magic" — they're functions returning functions. The "magic" is just function-as-value semantics. - "You can do this without nesting" — three-layer nesting is the canonical shape for parameterized decorators.

Common follow-up: *"What if I want `@retry` (no parens, default config) to also work?"* — Detect whether the first call passed a function (no parens) or arguments (with parens). Use `callable(arg)` and `not args[1:]` and `not kwargs` to distinguish. It's a common library pattern (Click, tenacity, retry).

```
import time
from functools import wraps

def retry(times=3, delay=1, exc=Exception):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            last = None
            for attempt in range(times):
                try:
                    return func(*args, **kwargs)
                except exc as e:
                    last = e
                    if attempt < times - 1:
                        time.sleep(delay)
            raise last
        return wrapper
    return decorator

@retry(times=2, delay=0)
def flaky(n):
    if n < 5:
        raise ValueError("too low")
    return n

print(flaky(10))                # 10
try:
    print(flaky(1))
except ValueError as e:
    print(e)                    # too low
```

End of Chapter 8

That's 50 questions on functions and closures — first-class function values, the LEGB scoping rules and how closures capture cells, the parameter kinds and their separators, the introspection levers, the `functools` toolkit, and the higher-order functions that turn loops into one-liners. The next chapter (Ch 9) moves to typing and protocols — the type system as it stands in Python 3.12+, where the modern interview questions about `Protocol`, `TypedDict`, `Self`, and `override` live.

When you walk into the panel and they hand you a snippet with a `for` loop building lambdas — say the answer out loud, name the late-binding rule, and offer the default-argument fix in one breath. That's the signal that you've read the data model, not just written Python.

Chapter 9 — Typing + Protocols (40 Questions)

Python Interview Prep — 2026 Edition · Learn & Excel · File 04 Core Python Library

This chapter sits in an awkward seam in the Indian interview circuit. Five years ago, type hints were a Pydantic/FastAPI curiosity and most candidates could skip them; today an interviewer at Razorpay, Cred, Postman, or any Microsoft GCC will ask three to five questions on typing inside a single round — and freshers at the Service-MNC pool are increasingly asked them too because internal codebases at TCS Digital, Infosys Topaz, and the GCC contracts are all on mypy or pyright now. The Indian shift was sudden: 2024 was "do you know what `Optional` means", 2026 is "explain why `Self` is better than `T = TypeVar('T', bound='Order')` for a fluent builder."

The 40 questions below assume Python 3.12+ as the baseline (PEP 695 native generics, `type` statement, `Self` type, `Required` / `NotRequired` in TypedDict). Each one is in the standard Step-by-Step Answer format: tags · question · 30-second answer · step-by-step thought process · edge cases · what NOT to say · common follow-up. Code blocks compile against 3.12; where 3.13 is needed it's flagged inline.

The five sections, each tuned to a different kind of Indian interview round:

- **Section A — Type-hint basics (Q1–Q10)** — Service-MNC + GCC fresher-pool typing fundamentals. The "annotation is a runtime no-op" question lives here.
- **Section B — Generic functions and classes (Q11–Q20)** — TypeVar legacy syntax, PEP 695 native syntax, bounded/constrained vars, ParamSpec, Concatenate, TypeVarTuple, the `Self` type. Razorpay/Postman lateral rounds and Microsoft GCC panels lean hard here.
- **Section C — `typing.Protocol` (Q21–Q30)** — Structural subtyping, `@runtime_checkable`, when Protocol beats ABC, generic Protocols, intersections. The 2026 differentiator question.
- **Section D — TypedDict + NamedTuple (Q31–Q35)** — Shape typing for dicts, total/partial, Required/NotRequired (3.11+), NamedTuple vs dataclass vs typing.NamedTuple.
- **Section E — Static checkers + runtime gap (Q36–Q40)** — Mypy vs pyright, `cast`, `assert_type`, `reveal_type`, `TYPE_CHECKING`, `from __future__ import annotations`.

Section A — Type-hint basics

The questions every annotated codebase exposes. If you fumble these you don't get to talk about Protocol — the interviewer moves on.

Q1 · What Type Hints Actually Do at Runtime

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant, Accenture · Topic: Typing fundamentals

The question (interviewer's verbatim phrasing):

If I write `def add(a: int, b: int) -> int: return a + b` and then call `add("hi", "there")`, what happens?

The 30-second answer: It returns the string `"hithere"`. Type hints are annotation-only at runtime — the interpreter records them in `__annotations__` but performs zero enforcement. `+` works on two strings, so the call succeeds. The hint is a signal to static type checkers and editors, nothing more. Runtime checking only happens if you wire up a library like Pydantic or `beartype` to read those annotations.

Step-by-step thought process: 1. "First, what the annotation does at parse time — it stores a dictionary on the function: `add.__annotations__` becomes `{'a': int, 'b': int, 'return': int}`. That's literally it." 2. "The interpreter does not insert any `isinstance` check, does not coerce, does not raise. The body runs whatever values you passed. Two strings concatenate, you get `'hithere'` back." 3. "This is by design — Python's typing PEPs (484, 526) made it explicit that types are hints for tools, not runtime contracts. The cost is zero. The trade-off is that you need a separate process — `mypy`, `pyright` — to actually catch the mismatch." 4. "What does enforce types — `Pydantic` models, `dataclasses` with `__post_init__` validation, the `beartype` decorator, the `typeguard` library. All of these read `__annotations__` and add their own checks on top. The hint itself stays a no-op." 5. "For interview punch — say the line out loud: 'Python's typing system is gradual and opt-in. At runtime the hints are documentation; at check time they're contracts; at refactor time they're how my IDE knows what to autocomplete.'"

Edge cases to mention: - Variable annotations like `x: int = 5` also go into the module-level or class-level `__annotations__` dict, again with zero runtime enforcement. - Functions defined in a `from __future__ import annotations` module get string annotations — `__annotations__['a']` is the string `'int'`, not the type `int`. Runtime introspection then needs `typing.get_type_hints()` to resolve. - Decorators that wrap a function can lose `__annotations__` unless they use `functools.wraps`. Worth pointing out — interview gotcha.

What NOT to say: - "Python will raise `TypeError` because the types don't match." That's the textbook trap — Python does not raise. If you say this, you've signalled you've never run `mypy`. - "Annotations are

enforced when you use `-O` mode." Wrong — `-O` strips `assert` statements and docstrings on `assert`, but annotations are kept.

Common follow-up: "How does Pydantic make annotations behave like real types?" — At class definition, Pydantic reads `__annotations__`, generates a per-field validator, and rewrites `__init__` to call those validators on each kwarg. The annotation triggers code generation; the annotation itself still does nothing.

```
def add(a: int, b: int) -> int:
    return a + b

print(add.__annotations__)      # {'a': <class 'int'>, 'b': <class 'int'>, 'return': <class 'int'>}
print(add("hi", "there"))      # 'hithere' – runs fine, no error
print(add([1, 2], [3, 4]))      # [1, 2, 3, 4] – list concatenation, also fine

# What enforces? A separate validator, not the annotation.
from dataclasses import dataclass
@dataclass
class Order:
    qty: int
o = Order(qty="five")           # still works – dataclass doesn't validate either
```

Q2 · Variable Annotations and the `__annotations__` Attribute

Tags: Difficulty: Easy · Asked at: Postman, Freshworks, Zoho, JP Morgan · Topic: Typing fundamentals

The question (interviewer's verbatim phrasing):

I write `count: int = 0` at module level. What's the difference between that and `count = 0`? Where does the annotation go?

The 30-second answer: The annotated form sets `count` to `0` and also adds `'count': int` to the module's `__annotations__` dict. The unannotated form just binds the variable. There's no runtime difference in behavior — assigning a string to `count` later is fine in both — but tools and `typing.get_type_hints` can now see the declared type.

Step-by-step thought process: 1. "Module-level annotation — `count: int = 0` — does two things: creates the binding `count = 0`, and updates `__annotations__` on the module. Class-level annotations work the same way against the class object." 2. "You can annotate without assigning — `count: int` declares a type without a value. The name does not get bound. So `count: int` followed by `print(count)` raises `NameError`. This trips people up — declaration in Python is not the same as initialization." 3. "Inside a function body, annotations on local variables are stored only if there's an assignment alongside — but they don't go into a runtime-accessible dict for the function."

They're parsed and discarded at runtime. Static checkers still see them through AST analysis." 4.

"Practical use — for class attributes you want the type on. `class Order: qty: int; total: Decimal` declares two typed attributes without setting defaults. mypy sees them; the runtime sees an empty class until you set them in `__init__`." 5. "The interview-grade nuance — `__annotations__` at the module level is automatically created the first time you write any module-level annotation. Before that it doesn't exist; afterwards it persists. Same for class bodies."

Edge cases to mention: - A class with `qty: int` and no `__init__` raises `AttributeError` if you read `Order().qty` — the annotation does not provide a default. To declare a default, write `qty: int = 0`. - For dataclasses, the presence of an annotation is what marks a name as a field. `qty = 0` without annotation in a dataclass body becomes a class attribute, not a field — common subtle bug. - `from __future__ import annotations` turns even the value side into strings: `__annotations__['count']` is `'int'` not `int`. Use `typing.get_type_hints(module)` to evaluate.

What NOT to say: - "Variable annotations create a typed variable that can't change type." Wrong — Python ignores the annotation; the variable can be reassigned to any value. - "You always need a default to make the annotation work." Wrong — `count: int` alone is a valid type-only declaration, often used in class bodies and dataclasses.

Common follow-up: "What's the difference between `qty: int = 0` and `qty = 0` inside a dataclass?" — The first becomes a field (in `__init__`, in `__repr__`, in equality). The second becomes a class attribute, ignored by the dataclass decorator. The annotation is the trigger.

```
# Module level
count: int = 0
print(__annotations__)           # {'count': <class 'int'>}}

# Class level
class Order:
    qty: int                      # declared, not bound
    total: float = 0.0           # declared and bound
    note = "manual"              # bound, no annotation

print(Order.__annotations__)     # {'qty': <class 'int'>, 'total': <class 'float'>}}
o = Order()
# print(o.qty)                  # AttributeError – never bound
print(o.total)                  # 0.0
```

Q3 · `Optional[X]` , `X | None` , and the Default-Value Mistake

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Microsoft India · Topic: Typing fundamentals

The question (interviewer's verbatim phrasing):

Walk me through every way to express "this can be an int or None" in a 3.12 codebase. Where do candidates trip up?

The 30-second answer: Four forms — all equivalent to the type checker: `Optional[int]`, `Union[int, None]`, `int | None` (PEP 604, 3.10+), and the implicit form `def f(x: int = None)` which static checkers explicitly flag as a mistake. The first three are interchangeable; in new 3.10+ code prefer `int | None` for the reading. The default-value gotcha — `x: int = None` is wrong; the correct form is `x: int | None = None`.

Step-by-step thought process: 1. "All three explicit forms — `Optional[int]`, `Union[int, None]`, `int | None` — mean the same thing. Mypy and pyright lower all of them to the same internal representation. Pick the one your codebase prefers." 2. "`int | None` won because of three things: no import needed, fewer characters, reads like English. PEP 604 added it in 3.10. PEP 563 lets you use it as a string annotation even on older Pythons via `from __future__ import annotations`." 3. "The classic mistake — `def fetch(user_id: int = None)`. The default is `None`, but the declared type is `int`. Mypy in strict mode flags this; PEP 484 originally allowed it implicitly as `Optional[int]`, but that 'implicit Optional' was removed from defaults in mypy 0.990 and pyright. The correct form is `def fetch(user_id: int | None = None)`." 4. "Independence — `Optional` types the value (it can be None), the default `= None` is about callability (the argument can be omitted). They're separate decisions. A parameter can be Optional without defaulting; it can default without being Optional, if the default is a non-None value." 5. "For dataclass fields you cannot do `value: list[int] = []` — mutable default, you'll get an error. The pattern is `value: list[int] | None = None` plus a `__post_init__` that fills in `[]` if None, or `value: list[int] = field(default_factory=list)`."

Edge cases to mention: - The runtime representation differs — `Optional[int]` evaluates to `typing.Union[int, None]` (instance of `typing.GenericAlias`); `int | None` evaluates to `types.UnionType`. For most runtime introspection it doesn't matter; for libraries that case-split on `type(annotation)`, it does. - For dictionary value types, `dict[str, int | None]` is common; for "the key may not exist OR may be None" you'd combine with `dict.get(key) or default`, but be careful — `or` short-circuits on 0 and empty string too. - Pyright distinguishes `None` from `Optional` for return types — a function annotated `-> None` that has a `return` statement with a value is an error; a function annotated `-> int | None` may return either.

What NOT to say: - "Optional means the argument can be omitted." Classic confusion — Optional is about the *value* being possibly None, not about the *argument* being possibly missing. The default value is what controls omission. - "Implicit Optional still works — `def f(x: int = None)`." Used to in old mypy; modern strict mode flags it. Don't fight the checker; write the explicit `| None`.

Common follow-up: "What's the runtime difference between `int | None` and `Optional[int]`?" — `int | None` is `types.UnionType`, `Optional[int]` is `typing.Union[int, None]` which is `typing._UnionGenericAlias`. Same semantic meaning, different concrete instances.

`isinstance(value, int | None)` works in 3.10+; with the older form you need `isinstance(value, (int, type(None)))`.

```
# All three forms, all equivalent to the checker
from typing import Optional, Union

def a(x: Optional[int]) -> int: ...
def b(x: Union[int, None]) -> int: ...
def c(x: int | None) -> int: ...    # preferred in 3.10+

# The wrong way – implicit Optional via default
def fetch(user_id: int = None):    # flagged in strict mode
    ...

# Right way – explicit
def fetch(user_id: int | None = None):
    if user_id is None:
        return None
    return db.get(user_id)

# Dataclass field with mutable optional
from dataclasses import dataclass, field
@dataclass
class Req:
    tags: list[str] | None = None
    items: list[str] = field(default_factory=list)
```

Q4 · `Union[X, Y]`, the `|` Operator, and Why `isinstance` Now Accepts Them

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Walmart Labs · Topic: Typing fundamentals

The question (interviewer's verbatim phrasing):

In 3.12, what works at runtime — `isinstance(x, int | str)` or `isinstance(x, (int, str))` ? Are they the same?

The 30-second answer: Both work in 3.10+. `isinstance(x, int | str)` is the modern form, accepted by the interpreter because `int | str` produces a `types.UnionType` object that `isinstance` knows how to walk. The tuple form `(int, str)` has worked since Python's earliest versions. Pick the pipe form in new code — it matches your annotations, no need to remember two syntaxes.

Step-by-step thought process: 1. "PEP 604 (3.10) didn't just give us the `|` annotation — it also made `int | str` produce a real runtime object usable by `isinstance` and `issubclass`. Before 3.10,

`isinstance(x, int | str)` would raise `TypeError` because `int | str` itself raised `TypeError` at runtime." 2. "The runtime type — `type(int | str)` is `types.UnionType`. It supports `__instancecheck__`. Internally `isinstance(x, int | str)` walks each member and returns `True` if any match. Same semantics as the tuple form." 3. "What this means for your code — you can have one source of truth. Annotate `x: int | str`, then `if isinstance(x, int | str)` for runtime branching uses the same expression. Less drift between hints and runtime checks." 4. "What still doesn't work — generic types inside the union. `isinstance(x, list[int] | dict)` raises because `list[int]` isn't a runtime-checkable type — generics aren't reified. You'd need `isinstance(x, (list, dict))` instead." 5. "Practical gotcha — `int | None` for `isinstance`. `type(None)` is `NoneType`; `None | int` includes it. So `isinstance(x, int | None)` works and matches both ints and the literal `None`. Replaces the older `isinstance(x, (int, type(None)))`."

Edge cases to mention: - `match` statement and pattern matching naturally pair with union types. `match v: case int(): ... case str(): ...` covers `v: int | str` exhaustively; pyright in strict mode warns if you miss a case. - `isinstance` with a union that includes a Protocol raises unless the Protocol is `@runtime_checkable`. Static checks pass, runtime fails. Worth knowing for the Protocol section. - The `|` form chains — `int | str | bytes` is just `int | str | bytes`. Mypy normalizes; runtime preserves order, useful only if you introspect.

What NOT to say: - "`isinstance(x, int | str)` doesn't work — you must use the tuple form." That was true pre-3.10, but in 2026-era Python it's not. Don't carry old habits. - "Union and pipe behave differently." They don't — for type-check purposes they're identical, and for `isinstance` they both work in modern Python.

Common follow-up: "What happens if I do `isinstance(x, list[int])` in 3.12?" — `TypeError`. Parameterized generics aren't valid for runtime checks. Use `isinstance(x, list)` and then iterate to check element types.


```
def parse(v: int | str) -> str:
    if isinstance(v, int | str):           # modern form
        return str(v)
    raise TypeError

# Equivalent tuple form — also valid
def parse_legacy(v: int | str) -> str:
    if isinstance(v, (int, str)):
        return str(v)

# Pattern matching with union types
def handle(event: int | str | dict):
    match event:
        case int():
            return event * 2
        case str():
            return event.upper()
        case {"type": t, "payload": p}:
            return (t, p)

# What does NOT work
# isinstance(x, list[int])           # TypeError — generics not runtime-checkable
```

Q5 · Any versus object — Why They're Opposites

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, JP Morgan · Topic: Typing fundamentals

The question (interviewer's verbatim phrasing):

What's the difference between `Any` and `object` in a type annotation? Aren't they both 'anything'?

The 30-second answer: They're opposites in spirit. `object` is the top of the type hierarchy — every value is an `object` — so a parameter typed `object` says "I accept anything, but I won't do anything with it that requires a specific type." `Any` is an escape hatch — it disables type checking for that value. A value of type `Any` can be passed anywhere, used in any operation; a value of type `object` can be passed only where `object` is accepted and supports only `object`'s methods (basically `__repr__`).

Step-by-step thought process: 1. "`object` is the literal root class — every value in Python inherits from it. A parameter `x: object` is a real constraint: the function may only call `repr(x)`, `str(x)`, `x is something`, things defined on `object`. Calling `x.upper()` is a type error because `object` doesn't have it." 2. "`Any` is a typing concept, not a class. It's a sentinel that tells the checker

'turn off rules for this value.' You can assign Any to anything, anything to Any, and call any method. It's the equivalent of an `unchecked` block." 3. "Where each one belongs — `object` for genuinely opaque inputs your function won't introspect (debugging printers, registry keys, identity checks). `Any` for boundaries where you're talking to untyped code — `json.loads` returns Any, ORM rows are often Any until you cast, raw user input you'll validate yourself." 4. "The cost of `Any` — it silently spreads. If `db.query()` returns `Any` and you do `result.name`, mypy doesn't complain even if `result` is actually a `dict` without a `.name`. `object` would have flagged the access; `Any` doesn't. So overusing `Any` defeats the whole point of typing." 5. "Practical pattern — annotate boundaries with the real type using `cast`. `result = cast(User, db.query(...))`. Now everything downstream is typed; the cast is the one trust boundary."

Edge cases to mention: - `Any` is special because it's both a subtype and a supertype of every type. `object` is only a supertype. So `def f() -> Any` can be assigned to `int`, but `def f() -> object` cannot — assigning the result requires an explicit cast. - A function with parameter `x: object` that calls `len(x)` is a static error even if you know `x` will always be a list. Either narrow with `isinstance` first, or annotate as `Sized` (a Protocol), or use `Any` as escape. - `Any` in a generic — `list[Any]` is "a list whose elements we won't check." `list[object]` is "a list where we promise nothing element-specific." Pyright treats both as effectively-equivalent for most cases, but `Any` silently propagates.

What NOT to say: - "`Any` and `object` are the same — both accept anything." Conceptually wrong — `Any` disables checking; `object` constrains to base-class operations. - "`Any` is more permissive, so it's always safer." Backwards — `Any` is permissive in a way that hides bugs. `object` is restrictive in a way that catches them.

Common follow-up: "When would you ever use `object` instead of `Any`?" — When you accept arbitrary values but only forward them, store them, or call `repr` on them. Logging utilities, identity-keyed caches, debug printers. The restriction is the point.

```

from typing import Any, cast

def log_value(x: object) -> None:
    # All you can do with object – repr, str, identity
    print(repr(x))
    # x.upper()                                # type error – object has no .upper

def parse_json(s: str) -> Any:
    import json
    return json.loads(s)                        # Any – we trust the caller to narrow

# Spread of Any
data = parse_json('{"name": "abhi"}')
print(data.name)                              # mypy passes – Any swallows errors

# Use cast to seal the boundary
from typing import TypedDict
class User(TypedDict): name: str
user = cast(User, parse_json('{"name": "abhi"}'))
print(user["name"])                           # typed from here on

```

Q6 · **Final** and Why It's Different from **const**

Tags: Difficulty: Medium · Asked at: Postman, Cred, Microsoft India, Goldman Sachs · Topic: Typing fundamentals

The question (interviewer's verbatim phrasing):

What does **Final** mean in a Python annotation? Does it make the value immutable?

The 30-second answer: **Final** is a static-only declaration that says "this name will not be reassigned after this point." It does not freeze the value. `x: Final = []` lets you mutate the list with `.append`, but reassigning `x = something_else` is a type error. It's about *bindings*, not *values*. Use it for constants, for class attributes that subclasses shouldn't override, and to mark module-level configuration.

Step-by-step thought process: 1. "**Final** from **typing** is purely advisory at runtime. The checker enforces it; the interpreter ignores it. `MAX_RETRIES: Final = 3` followed by `MAX_RETRIES = 4` later is a static error but runs fine." 2. "What it controls — the name binding. The object the name points to can still be mutated if it's mutable. `tags: Final = []`; `tags.append('new')` is fine; `tags = ['fresh']` is the error." 3. "On a class attribute — `class Settings: TIMEOUT: Final = 30`. A subclass writing `TIMEOUT = 60` is a static error. Equivalent to **final** in Java for fields, but only for the binding, not the value." 4. "With explicit type — `MAX_RETRIES: Final[int] = 3`. With inferred type — `MAX_RETRIES: Final = 3` and the checker infers `int`. Both work; the latter is shorter for

simple constants." 5. "Methods — `@typing.final` is the decorator counterpart. It marks a method as not overridable by subclasses. Same enforcement model — checker yes, runtime no. Useful for marking the public API of a base class."

Edge cases to mention: - `Final` does not propagate. A list inside a `Final` variable is still a mutable list; if you want immutability use `tuple` or `frozenset` or call `MappingProxyType` on a dict. - Module-level `Final` is the canonical use — pyright/mypy treat the name as a true constant. Editors will refuse autocomplete on assignment. - `Final` cannot be combined with `ClassVar` in the same annotation; you pick one. For a class constant use `Final` alone — `ClassVar` is for non-final class-shared state.

What NOT to say: - "`Final` makes the object immutable." Wrong — only the binding is fixed; the object can be mutated. - "`Final` is enforced at runtime." It isn't — reassign the name and Python doesn't complain; only the type checker does.

Common follow-up: "What's the difference between `Final` and `ClassVar`?" — `Final` marks the binding as unchangeable. `ClassVar` marks the attribute as belonging to the class, not to instances — so `Settings.TIMEOUT = 60` reassigns the class attribute (and would be flagged if also `Final`). Different concerns.

```
from typing import Final, final

# Module-level constant
MAX_RETRIES: Final = 3
# MAX_RETRIES = 5                                # type error: cannot reassign

tags: Final[list[str]] = []
tags.append("fresh")                               # fine — mutation of value is allowed
# tags = ["new"]                                   # type error: reassignment forbidden

# Class attribute
class Settings:
    TIMEOUT: Final[int] = 30

class FastSettings(Settings):
    pass
    # TIMEOUT = 60                                # type error: override forbidden

# Method that subclasses cannot override
class Base:
    @final
    def fingerprint(self) -> str:
        return "base"

class Child(Base):
    pass
    # def fingerprint(self) -> str:                # type error: cannot override
    #     return "child"
```

Q7 · `Callable[[...], R]` — Typing Functions, Methods, and Lambdas

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Microsoft India · Topic: Typing fundamentals

The question (interviewer's verbatim phrasing):

How do you type a parameter that takes another function? Say a callback that accepts two ints and returns an int.

The 30-second answer: `Callable[[int, int], int]` from `typing` (or `collections.abc.Callable` since 3.9 — they're interchangeable for typing purposes). First bracket holds the argument types as a list; second holds the return type. For a callable that takes no arguments and returns nothing — `Callable[[], None]`. For "any callable" — use bare `Callable` (or `...` as the param list).

Step-by-step thought process: 1. "Base form — `Callable[[ArgType1, ArgType2], ReturnType]`. The inner list is positional. So `Callable[[int, str], bool]` accepts callables taking an int and a str positionally, returning bool." 2. "For any callable that returns X — `Callable[..., X]`. The ellipsis says 'I don't care about the parameter signature.' Useful for decorators and registries where each registered function can have a different signature." 3. "For keyword-only arguments, default values, and `*args/**kwargs` — basic `Callable` can't express any of those. You need `ParamSpec` (3.10+) or `Protocol` with a `__call__` method. We'll cover those in the generics section." 4. "In 3.9+ prefer `from collections.abc import Callable`. The runtime cost is the same and you sidestep the `typing` import. The `typing.Callable` form still works and is identical to the checker." 5. "Practical lambda typing — `transform: Callable[[int], int] = lambda x: x * 2`. The annotation flows to the lambda. Pyright will narrow the lambda's body using the parameter type, so refactoring is safer."

Edge cases to mention: - `Callable[[int], int]` does *not* accept `Callable[[int, int], int]`. Argument counts must match, despite tuples being structurally similar. This catches people from JavaScript backgrounds where extra arguments are ignored. - Variance — `Callable` is contravariant in arguments, covariant in return. `Callable[[Animal], int]` is a subtype of `Callable[[Dog], int]`, not the other way. Indian senior-role interviews ask about variance occasionally; the rule of thumb is "more permissive inputs OK, more specific outputs OK." - An async function — `Callable[[int], Coroutine[Any, Any, int]]` or, more readably with the `Awaitable` Protocol: `Callable[[int], Awaitable[int]]`. The async/sync distinction is in the return type.

What NOT to say: - "`Callable[int, int, int]` works." Wrong syntax — args go in a *list* (inner bracket), return is a separate slot. The flat form is invalid. - "Lambdas don't have types." They do — same as the function the annotation flows from. Pyright/mypy infer the body using the parameter type.

Common follow-up: "How do you type a callback that takes keyword arguments and `**kwargs`?" — Basic `Callable` can't express it. Use `ParamSpec` for decorators that forward exactly, or a `Protocol` class with a `__call__` method that spells out the signature.

```
from collections.abc import Callable
from typing import Awaitable

# Two ints in, int out
def apply(op: Callable[[int, int], int], a: int, b: int) -> int:
    return op(a, b)

print(apply(lambda x, y: x + y, 2, 3))    # 5
print(apply(int.__add__, 2, 3))          # 5

# Any callable returning bool — common for predicates
def first_match(items: list[int], pred: Callable[..., bool]) -> int | None:
    for it in items:
        if pred(it):
            return it
    return None

# Async callback
async def gather_one(fn: Callable[[int], Awaitable[int]], x: int) -> int:
    return await fn(x)
```

Q8 · Generic Container Annotations — `list[int]` vs `List[int]`

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Freshworks, Zoho, JP Morgan · Topic: Typing fundamentals

The question (interviewer's verbatim phrasing):

In modern Python should I write `list[int]` or `List[int]` ? Why?

The 30-second answer: `list[int]` since 3.9 — PEP 585 made all the builtin containers (`list` , `dict` , `tuple` , `set` , `frozenset` , `type`) and the `collections.abc` types subscriptable directly. No import needed. The capitalized `List` , `Dict` , `Tuple` from `typing` still work for backwards compatibility but are deprecated for new code. Same for `Sequence` , `Mapping` , `Iterable` — import them from `collections.abc` , not `typing` .

Step-by-step thought process: 1. "Before 3.9, `list` wasn't subscriptable — `list[int]` at runtime raised `TypeError`, so you used `typing.List[int]` which had `__class_getitem__` defined. PEP 585 added subscriptability to the builtins, removing the need for the `typing` aliases." 2. "In 3.9+ the recommended forms — `list[int]` , `dict[str, int]` , `tuple[int, str]` ,

`set[str]`, `type[Order]`. From `collections.abc` — `Sequence[int]`, `Mapping[str, int]`, `Iterable[int]`, `Callable[[int], int]`. All work at runtime without `typing`." 3. "For tuples there's a quirk worth knowing — `tuple[int, str]` means 'exactly two elements, an int and a str.' `tuple[int, ...]` means 'any length tuple of ints.' The ellipsis is the variadic marker." 4. "Migration — `typing.List[int]` still works in 3.12 and won't break, but mypy and pyright emit deprecation warnings in strict mode. New code should use the lowercase forms; old code can migrate gradually." 5. "For backward compat with 3.8 — `from __future__ import annotations`. Now `list[int]` at top level is a string at runtime, so it doesn't blow up even on 3.8. Resolves at type-check time. The 3.8 train left in 2024 for production, but plenty of older codebases use this."

Edge cases to mention: - `type[X]` is the type of a class — `type[Order]` means "the class `Order` itself," not an instance. Useful for factories and class-method registries. Replaces older `Type[X]` from `typing`. - `dict[str, int]` covers exact dict; `Mapping[str, int]` is the read-only protocol (no `.update`, no `del`). Use `Mapping` in function parameters when you don't intend to mutate — gives callers more flexibility. - `list[int]` requires `int`; passing a `list[bool]` is technically a subtype (`bool` is a subclass of `int`), but pyright in strict mode often warns because of variance — lists are invariant.

What NOT to say: - "`List[int]` and `list[int]` mean different things." They don't — purely a syntax preference for the same idea. - "Lowercase forms only work in 3.10+." Wrong — they've worked since 3.9 for runtime annotations, and from any version with `from __future__ import annotations`.

Common follow-up: "What's the difference between `list[int]` and `Sequence[int]` as a parameter type?" — `list[int]` requires exactly a list. `Sequence[int]` accepts list, tuple, str (it's a sequence of one-char strings!), any class that implements `__getitem__` + `__len__`. Use the broader `Sequence` when you only need to iterate or index.

```
# 3.9+ preferred forms – no typing import needed
def total(items: list[int]) -> int:
    return sum(items)

def by_key(rows: dict[str, int]) -> int:
    return rows["key"]

# Variadic tuple – any length
def average(values: tuple[float, ...]) -> float:
    return sum(values) / len(values)

# Fixed-length tuple – exactly three slots
def colour(rgb: tuple[int, int, int]) -> str:
    return f"#{rgb[0]:02x}{rgb[1]:02x}{rgb[2]:02x}"

# Abstract collections
from collections.abc import Sequence, Mapping, Iterable

def first(seq: Sequence[int]) -> int:
    return seq[0] # accepts list, tuple, range

def lookup(m: Mapping[str, int], key: str) -> int:
    return m[key] # read-only contract; mutation not allowed
```

Q9 · `Literal`, `LiteralString`, and Why They Show Up in 2026 Codebases

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Microsoft India · Topic: Typing fundamentals

The question (interviewer's verbatim phrasing):

What's `Literal['admin', 'user']` and when would you use it instead of an Enum?

The 30-second answer: `Literal` (PEP 586, 3.8+) is "this value is one of these exact constants." Type checkers track which literal matched and narrow accordingly. Useful for tagged unions, mode flags, and APIs where introducing an Enum feels heavyweight. `LiteralString` (PEP 675, 3.11+) is different — it means "any string literal known at compile time, not a string built at runtime" — used to mark SQL or shell commands as safe from injection.

Step-by-step thought process: 1. "Basic *Literal* —

`def open(mode: Literal['r', 'w', 'a'])`. Now `open('z')` is a type error; `open('r')` is fine. The checker enforces the value, not just the type." 2. "Combined with `match` — pyright in strict mode does exhaustiveness on *Literal* unions.

`match mode: case 'r': ... case 'w': ... case 'a': ...` — leave out `'a'` and the

checker flags it. This catches missing cases at refactor time." 3. "Tagged union pattern — `class Read: kind: Literal['read']` and `class Write: kind: Literal['write']`. Now `Event = Read | Write` lets the checker narrow on `if event.kind == 'read'` without `isinstance`. Powerful for protocol handlers." 4. " `LiteralString` is the 3.11+ addition for security. A function `def execute(sql: LiteralString)` accepts string literals and concatenations of literals, but not strings built from user input. Means `execute(f'SELECT * FROM {user_input}')` is a type error — the f-string isn't a literal because it interpolates." 5. "When to prefer Enum — when the values are part of your domain (statuses, priorities) and you want methods on them, iteration, or `.value` / `.name` semantics. Literal is for plumbing — flags, modes, internal sentinels. Enum is for first-class concepts."

Edge cases to mention: - `Literal[None]` is valid and means "the value is exactly None." Same as `None` in most type positions; sometimes useful for explicit disambiguation. - `Literal[True]` and `Literal[False]` exist — narrow a function on a bool flag. `def get(strict: Literal[True]) -> User` versus `def get(strict: Literal[False]) -> User | None`. Overload-based behavior change. - `LiteralString` does not propagate through `+=`. Concatenation of two `LiteralString` stays `LiteralString`; concatenation of `LiteralString` with `str` becomes `str`. Catches the case where you mix in a runtime value.

What NOT to say: - "Literal is the same as Enum." Different surface — Enum is a class with iteration and methods; Literal is a value-narrowing type expression. Different use cases. - "LiteralString stops SQL injection at runtime." Wrong — it's a static check, runtime does nothing. It tells you at code-review time whether the argument is a static string or a dynamic one.

Common follow-up: "How would you type a function whose return type depends on a Literal argument?" — Use `@overload`. `@overload def get(id: int, strict: Literal[True]) -> User:` and `@overload def get(id: int, strict: Literal[False]) -> User | None:`. The checker picks the right return based on the literal passed.

```

from typing import Literal, LiteralString, overload

# Mode flag with Literal
def open_file(path: str, mode: Literal['r', 'w', 'a']) -> str:
    with open(path, mode) as f:
        return f.read() if mode == 'r' else ""

# open_file('x', 'z')          # type error – 'z' not in Literal

# Tagged union for events
from dataclasses import dataclass
@dataclass
class Login: kind: Literal['login']; user_id: int
@dataclass
class Logout: kind: Literal['logout']; user_id: int

def handle(event: Login | Logout) -> None:
    match event:
        case Login(kind='login', user_id=u): print(f"in: {u}")
        case Logout(kind='logout', user_id=u): print(f"out: {u}")

# LiteralString for query safety
def run_query(sql: LiteralString) -> list:
    ...

run_query("SELECT * FROM orders WHERE status = 'paid'") # OK – static
user = "abhi"; DROP TABLE orders;--"
# run_query(f"SELECT * FROM users WHERE name = '{user}'") # type error – not literal

# Return-type narrowing via overload
@overload
def get(id: int, strict: Literal[True]) -> dict: ...
@overload
def get(id: int, strict: Literal[False]) -> dict | None: ...
def get(id: int, strict: bool) -> dict | None:
    row = {"id": id} # placeholder
    if strict and not row:
        raise KeyError(id)
    return row

```

Q10 · **Annotated[X, metadata]** — Carrying Validators Alongside Types

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India · Topic: Typing fundamentals

The question (interviewer's verbatim phrasing):

What is **Annotated[int, "max=100"]** and why does FastAPI use it everywhere?

The 30-second answer: `Annotated` (PEP 593, 3.9+) lets you attach arbitrary metadata to a type — `Annotated[int, Range(0, 100)]` means "an int, with this attached marker." Type checkers ignore the metadata; libraries that read it (FastAPI, Pydantic, attrs, msgspec) use it for validation, dependency injection, documentation. It's how you keep the type and its constraints in one place without polluting the type itself.

Step-by-step thought process: 1. "`Annotated[X, m1, m2, ...]` is type `X` plus any metadata you want to ride along. To the type checker it's transparent — `Annotated[int, 'anything']` is just `int`. To `typing.get_type_hints(..., include_extras=True)` it preserves the metadata." 2. "FastAPI is the canonical user — `def get_user(user_id: Annotated[int, Path(gt=0)])`. The `int` is for the type checker; `Path(gt=0)` tells FastAPI 'this comes from the URL path, must be a positive integer.' Both pieces of information in one annotation." 3. "Pydantic v2 uses it heavily — `Annotated[int, Field(ge=0, le=100)]` declares an int constrained to 0-100. The type is `int`, the constraint runs at validation. Replaces the older `Field(...)` default-value trick." 4. "For your own libraries — define metadata classes (often dataclasses), have your library read `typing.get_type_hints(..., include_extras=True)` and pick up the metadata. The user gets normal typing AND your validation in one annotation. No leaky abstractions." 5. "Layering — `Annotated[Annotated[int, Range(0, 10)], 'comment']` collapses to `Annotated[int, Range(0, 10), 'comment']`. Order is preserved. Libraries can pick the metadata they care about by `isinstance` checks on the metadata items."

Edge cases to mention: - The metadata can be any value — strings, instances of dataclasses, custom marker classes. Convention is to define a small metadata class per library so different libraries don't conflict. - Stripping metadata — `typing.get_type_hints(f)` without `include_extras=True` returns the base type. So a `Annotated[int, X]` becomes just `int`. Most introspecting code uses this default. - `Annotated` in a class field works the same as anywhere — `class User: age: Annotated[int, Range(0, 150)]`. Pydantic, attrs, and dataclasses (via metadata) all pick it up.

What NOT to say: - "`Annotated` is just a documentation comment." Wrong — it's structured metadata, machine-readable, used by validators and serializers. - "Type checkers enforce the constraints in `Annotated`." They don't — only the base type is checked. The constraints run when a library opts in via `include_extras`.

Common follow-up: "What's the difference between using `Annotated` and a custom subclass like `class PositiveInt(int)`?" — Subclass introduces a new type that propagates through your code — every function touching it must declare `PositiveInt`. `Annotated` is a marker — the type stays `int`, the metadata is opt-in. `Annotated` is lighter; subclass is more strict.

```

from typing import Annotated, get_type_hints
from dataclasses import dataclass

@dataclass
class Range:
    lo: int
    hi: int

def quantity(q: Annotated[int, Range(1, 100)]) -> int:
    return q

# To the type checker, q is just int
# To a library, the metadata is reachable
hints = get_type_hints(quantity, include_extras=True)
print(hints)          # {'q': Annotated[int, Range(lo=1, hi=100)], 'return': <class 'int'>}

# FastAPI-style — pseudo code
# def get(user_id: Annotated[int, Path(ge=1)]): ...
# Pydantic-style
# class Order(BaseModel):
#     qty: Annotated[int, Field(ge=1, le=100)]

```

Section B — Generics: TypeVars, PEP 695, ParamSpec, Self

This is where 2026 Python diverges sharply from 2022 Python. PEP 695 (3.12) introduced native generic syntax — `def first[T](items: list[T]) -> T` — without `TypeVar`. Interviews now ask whether you know both forms and when each is appropriate. Razorpay-, Postman-, Cred-, Microsoft GCC-level rounds lean heavily here.

Q11 · `TypeVar` — The Legacy Generic Syntax

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Walmart Labs · Topic: Generics

The question (interviewer's verbatim phrasing):

Walk me through `TypeVar` — what it is, where you put it, and what it does.

The 30-second answer: `TypeVar` is the legacy way to declare a generic type parameter — `T = TypeVar('T')` at module level, then use `T` in function or class annotations. The checker tracks which concrete type fills it on each call, ensuring consistency. For PEP 695 (3.12+) native syntax — `def`

`first[T](items: list[T]) -> T` — you don't need TypeVar at all; the bracketed name does the same thing.

Step-by-step thought process: 1. "Pre-3.12 — `from typing import TypeVar; T = TypeVar('T')` . Then `def first(items: list[T]) -> T:` . The checker sees the same `T` in input and output, so calling with `list[int]` returns `int` , calling with `list[str]` returns `str` . Without TypeVar, you'd lose that link." 2. "The string name in `TypeVar('T')` must match the variable name. Convention is single uppercase letters: `T` , `U` , `K` , `V` . The string is used only for repr; the binding is the variable." 3. "Scope — TypeVar is module-level. Multiple functions sharing the same `T` does NOT mean they share the same concrete type — each function's call site binds its own. Two separate `def first(...) -> T` calls with different element types are fine." 4. "On a class — `class Container(Generic[T])` . Now the class is parameterized; `Container[int]` and `Container[str]` are distinct types to the checker, even though at runtime they're the same class." 5. "3.12+ supersedes — `def first[T](items: list[T]) -> T` and `class Container[T]` . No TypeVar import, no Generic base. Internally the runtime creates a real TypeVar; you just don't have to spell it out."

Edge cases to mention: - A function with two unrelated type parameters —

`T = TypeVar('T'); U = TypeVar('U'); def pair(a: T, b: U) -> tuple[T, U]` . Each binds independently. Same idea with PEP 695: `def pair[T, U](a: T, b: U) -> tuple[T, U]` . - TypeVar appearing only in the return position — `def make(x: int) -> T` — is a type error. The checker can't infer what T should be. Type vars must appear at least once in a parameter (or be bound via a default in 3.12+). - `TypeVar` has a `default` parameter (3.13+ via PEP 696) — `T = TypeVar('T', default=int)` . Lets generic types have a default parameterization. Niche but increasingly common in library code.

What NOT to say: - "TypeVar makes the function generic at runtime." Wrong — runtime sees a regular function. The TypeVar only matters for static analysis. - "You can use any string as the TypeVar's name argument." You can, but if it doesn't match the variable name, error messages confuse you. Conventionally they match.

Common follow-up: "What does it mean if I have two functions both using `T` from the same module-level TypeVar?" — Each call binds T independently for that function's signature. The shared module-level object is just a convenience; concrete types don't leak across calls.

```
# Legacy pre-3.12 syntax
from typing import TypeVar, Generic

T = TypeVar('T')

def first(items: list[T]) -> T:
    return items[0]

x: int = first([1, 2, 3])          # T = int
y: str = first(["a", "b"])        # T = str

# Generic class
class Box(Generic[T]):
    def __init__(self, value: T) -> None:
        self.value = value
    def get(self) -> T:
        return self.value

b: Box[int] = Box(42)
b.get()                          # int

# Two unrelated parameters
U = TypeVar('U')
def pair(a: T, b: U) -> tuple[T, U]:
    return (a, b)
```

Q12 · PEP 695 Native Generic Syntax (3.12+)

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India · Topic: Generics

The question (interviewer's verbatim phrasing):

What's the 3.12 way to write generics? Why is it different from TypeVar?

The 30-second answer: PEP 695 added bracket syntax — `def first[T](items: list[T]) -> T` and `class Container[T]`. No `TypeVar` import, no `Generic` base. The type parameters are scoped to the function or class body, not module-level. The runtime creates real `TypeVar` objects under the hood (accessible via `__type_params__`), so existing tooling keeps working. It's purely cleaner ergonomics for the most common case.

Step-by-step thought process: 1. "The form — `def name[T, U](...)` for functions, `class Name[T, U](...)` for classes. The brackets sit between the name and the parameter list. Inside, `T` and `U` are usable as types." 2. "What changed conceptually — scope. Pre-3.12, `T = TypeVar('T')` was module-level and could leak across function signatures. Post-3.12, `def first[T]` confines `T` to that function's signature. Cleaner mental model." 3. "Bound and constraints work — `def first[T:`

`Comparable]` for bounded, `def first[T: (int, str)]` for constrained. The syntax mirrors `Annotated`-style metadata in the brackets." 4. "Type aliases got the same treatment — PEP 695's `type` statement. `type Vector[T] = list[T]` creates a generic alias. Lazy-evaluated, so forward references work without `from __future__`." 5. "Runtime — `def first[T](items: list[T])` creates a real `TypeVar` named 'T' as a function attribute. `first.__type_params__` returns `(~T,)`. Existing introspection (`typing.get_type_hints`, libraries that walk `TypeVar`) keep working."

Edge cases to mention: - Type parameters declared with PEP 695 syntax are *not* re-usable across functions. Each `def first[T]` has its own T. To share, use a module-level `TypeVar` with the legacy syntax. - The `type` statement is lazy — `type Tree[T] = T | list[Tree[T]]` works for recursive aliases because the right-hand side is not evaluated until accessed. Pre-3.12 you needed forward-ref strings. - Mixing styles — a class can use PEP 695 for its own parameters while methods inside accept module-level `TypeVars`. Legal but messy; pick one style per project.

What NOT to say: - "PEP 695 replaced `TypeVar` entirely." It didn't — `TypeVar` still exists and is still needed for shared parameters across multiple definitions, for runtime-introspecting libraries, and for older Python versions. - "PEP 695 changes runtime behavior." It changes the syntax and adds `__type_params__`. The runtime semantics — generics aren't reified, no checking — are unchanged.

Common follow-up: "What does `class Container[T](Generic[T])` do in 3.12?" — Double-declares T. Pyright flags it as a redundant generic base. With PEP 695 syntax, the `Generic[T]` base is automatic and shouldn't be added explicitly.

```
# 3.12+ PEP 695 – cleaner, no imports
def first[T](items: list[T]) -> T:
    return items[0]

class Container[T]:
    def __init__(self, value: T) -> None:
        self.value = value
    def get(self) -> T:
        return self.value

# Two parameters
def pair[T, U](a: T, b: U) -> tuple[T, U]:
    return (a, b)

# Type alias with `type` statement
type Vector[T] = list[T]
type Tree[T] = T | list['Tree[T]']      # recursive – works because lazy

v: Vector[int] = [1, 2, 3]

# Runtime introspection
print(first.__type_params__)           # (~T,)
print(Container.__type_params__)       # (~T,)
```

Q13 · Bounded `TypeVar` — `bound=` Parameter**Tags:** Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Generics**The question (interviewer's verbatim phrasing):**

What does `TypeVar("T", bound=Comparable)` mean and when would I use it?

The 30-second answer: A bounded `TypeVar` restricts `T` to subclasses of the bound. `TypeVar("T", bound=Comparable)` says "T must implement the `Comparable` protocol." Inside the function body, the checker treats `T` as having all of `Comparable`'s methods, so `t1 < t2` is valid. In 3.12 syntax — `def sort[T: Comparable](items: list[T])`. The bound is an upper bound; `T` can be any subtype but no looser.

Step-by-step thought process: 1. "Without a bound, `T` could be anything — so inside the function you can't call any methods on a `T` value beyond what `object` supports. Bounded `TypeVar` widens what you can do." 2. "Example — a max function: `def max_of[T: Comparable](items: list[T]) -> T`. Inside, you can do `if a < b: ...` because the bound guarantees `<` exists. Without the bound, `<` is a type error." 3. "The bound can be a Protocol, ABC, or concrete class. Common bounds — `bound=numbers.Number` for numeric, `bound=Hashable` for keys, `bound=SupportsLessThan` (a custom Protocol) for sortable." 4. "The bound is an upper limit — `T` can be a subclass of the bound. So `T: Animal` accepts `Dog` and `Cat` but not `int`. Internal code can rely on `Animal`'s interface; callers pass any subtype." 5. "3.12 syntax — `def sort[T: Comparable](items: list[T])`. The colon-and-type after `T` sets the bound. Equivalent to the legacy `TypeVar` form but scoped locally."

Edge cases to mention: - Bound is *single* — `bound=X`. For "T must be one of these specific types," use constrained `TypeVar` (next question) with positional args. - Bound can be a forward reference (string) for circular or self-referential cases — `T = TypeVar('T', bound='Node')`. PEP 695's `type` statement handles this more cleanly. - Bound interacts with `Self` — for methods that return "an instance of my class or subclass," `Self` is usually clearer than a bounded `TypeVar`.

What NOT to say: - "Bound and constrained are the same thing." They're not — bound is an upper limit (subclasses OK), constrained is "exactly one of these specific types, no subtypes." - "You can have multiple bounds." You can't; `bound=` takes a single type. Use an intersection-like Protocol if you need multiple capabilities.

Common follow-up: "What's the difference between `T: Animal` and `T: int | str`?" — The first is a bound — `T` is `Animal` or any subclass. The second is a constrained `TypeVar` (treated specially); `T` must be *exactly* `int` or *exactly* `str`, not a subclass. Two different mental models.


```

from typing import TypeVar, Protocol

class Comparable(Protocol):
    def __lt__(self, other: 'Comparable', /) -> bool: ...

# Pre-3.12 form
T = TypeVar('T', bound=Comparable)
def smallest(items: list[T]) -> T:
    best = items[0]
    for x in items[1:]:
        if x < best:
            best = x
    return best

print(smallest([3, 1, 4, 1, 5]))      # 1 - T = int
print(smallest(["b", "a", "c"]))     # 'a' - T = str

# 3.12+ form - cleaner
def smallest_new[T: Comparable](items: list[T]) -> T:
    best = items[0]
    for x in items[1:]:
        if x < best:
            best = x
    return best

```

Q14 · Constrained `TypeVar` — Multiple Specific Types

Tags: Difficulty: Medium · Asked at: Cred, Postman, Microsoft India · Topic: Generics

The question (interviewer's verbatim phrasing):

When would I use `TypeVar("T", int, str)` instead of `TypeVar("T", bound=...)` ?

The 30-second answer: A constrained `TypeVar` — `TypeVar("T", int, str)` — restricts `T` to exactly the listed types, not subclasses. Inside the function, `T` is treated as a union of those types; on each call, `T` binds to one of them. Use it when the function is meaningful for a small fixed set (e.g., "only `int` or `str`") and you don't want subclasses sneaking in. Bounded `TypeVar` is for capability ("anything `Comparable`"); constrained is for finite choice.

Step-by-step thought process: 1. "Syntax — `TypeVar('T', int, str)` lists the allowed types as positional arguments. Two or more types — one type would just be that type, not a `TypeVar`. In 3.12 syntax — `def f[T: (int, str)]`." 2. "Behavior — on call, `T` binds to one of the listed types based on the argument. `f(5)` binds `T = int`; `f('x')` binds `T = str`; `f(3.0)` is a type error because `float` isn't listed." 3. "Subclasses don't count — `f(True)` is treated as binding `T = int`, but specifically not subclasses-of-`int` as an open set. The checker treats the constraint as 'one of these literal types.' `Bool`

happens to be one of int's listed bindings only when bool is explicitly listed." 4. "Inside the body, the checker treats *T* as a union — `int | str`. So you can do `t.upper()` only if you narrow with `isinstance` first. Methods common to both types (`__add__`, `__eq__`) are usable directly." 5. "Use case — `def double(x: T) -> T: return x + x` where you genuinely want to support `int` and `str` (both have `+`) but not floats. Or pandas-style functions that take `ints` or `strings` as index keys. Most code is better served by Protocol bounds, but constraint is the right tool for finite, non-hierarchical sets."

Edge cases to mention: - A `bool` argument binds `T = int` because the checker resolves to the listed type (`bool` is a subtype of `int` and `int` is listed). Sometimes catches people; rarely a real problem. - Constrained TypeVars can also have a `bound` in 3.12+ via the `default` parameter, but that's rare. Usually pick one. - Constraints don't compose well with generics — `list[T]` where `T` is constrained means each list element must be `int` OR `str` (uniformly per call), not mixed.

What NOT to say: - "Constrained TypeVar means 'either `int` or `str` at the same time.'" It means 'one of `int` or `str` per call.' Each call site binds `T` to one type. - "Constrained and Union are the same." They're different — `Union[int, str]` means "any value of type `int` or `str`"; constrained TypeVar links the same `T` across multiple positions in the signature.

Common follow-up: "What's the difference between `def f(x: T) -> T` with `T = TypeVar('T', int, str)` and `def f(x: int | str) -> int | str`?" — TypeVar links input to output type — `f(5)` returns `int`, `f('x')` returns `str`. Union doesn't link them — the return could be either regardless of input. Strong tie versus independent values.

```
from typing import TypeVar

# Constrained — T is either int or str, per call
T = TypeVar('T', int, str)

def double(x: T) -> T:
    return x + x

double(5)          # int -> int
double("hi")       # str -> str
# double(3.0)      # type error — float not in constraints

# 3.12+ form
def double_new[T: (int, str)](x: T) -> T:
    return x + x

# Contrast with Union
def double_union(x: int | str) -> int | str:
    return x + x

# Now the return type is the broad union, not linked to input
result = double_union(5)  # type is int | str, even though we know it's int
```

Q15 · Generic[T] — Subclassing for Custom Generic Classes**Tags:** Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Goldman Sachs · Topic: Generics**The question (interviewer's verbatim phrasing):**

If I want my class `Repository` to be generic over the entity type, how do I write it pre-3.12 versus post-3.12?

The 30-second answer: Pre-3.12 — declare a `TypeVar`, inherit from `Generic[T]`, use `T` in your methods. Post-3.12 — bracket syntax on the class, no `Generic` base needed. `class Repository[T]:` does the same thing as `class Repository(Generic[T]):` with `T = TypeVar('T')`. Inside, `T` is usable in method signatures.

Step-by-step thought process: 1. "Pre-3.12 — `from typing import TypeVar, Generic; T = TypeVar('T'); class Repository(Generic[T]): def get(self, id: int) -> T: ...`. `Generic[T]` base does two things: makes the class subscriptable (`Repository[User]`) and tells the checker the class has a generic parameter `T`." 2. "Post-3.12 — `class Repository[T]: def get(self, id: int) -> T: ...`. The brackets after the name declare `T` scoped to this class. No `Generic[T]` base; no `TypeVar` import. Identical semantics." 3. "Subscripting — `Repository[User]` creates a generic alias. At runtime it's a `types.GenericAlias` (or `typing._GenericAlias` for legacy). Calling it — `Repository[User]()` — returns a regular `Repository` instance; the type parameter is for the checker only." 4. "Storing state — type variables are NOT runtime values. So you can't do `class Repository[T]: def __init__(self): self.t = T`. `T` is meaningless inside the body except in annotations. If you need the runtime type, pass it explicitly: `def __init__(self, t_class: type[T])`." 5. "Combining — generic class with generic method. `class Repository[T]:` and inside `def find_one[K](self, key: K) -> T | None`. Each scope has its own type parameters."

Edge cases to mention: - Generic class with default for the parameter — PEP 696 (3.13+) allows `class Repository[T = User]`. Then `Repository()` (no brackets) defaults to `Repository[User]`. Useful for library APIs that have a sensible default. - Generic base classes — `class UserRepo(Repository[User])`. Now `UserRepo` isn't generic itself; `T` is fixed. Methods inherit the right specialized signatures. - Multiple type parameters — `class Pair[K, V]`. Subscript with both: `Pair[int, str]`. Same idea, just more brackets.

What NOT to say: - "`Generic[T]` is required in 3.12 too." It isn't; PEP 695 syntax replaces it. - "Type parameters can be accessed at runtime via `self`." Not directly — they're erased like Java generics. You can recover them via `__orig_class__` (instance) or `__class_getitem__` mechanics, but that's an advanced topic.

Common follow-up: "How do I access the actual type at runtime if I need to?" — Pass the type explicitly: `def __init__(self, t_class: type[T])`. Then `t_class()` constructs a `T`,

`isinstance(x, t_class)` checks against the real type. Erasure means you can't get T from the generic; you have to be given a reference.

```
# Pre-3.12 form
from typing import TypeVar, Generic
T = TypeVar('T')

class Repository(Generic[T]):
    def __init__(self, t_class: type[T]) -> None:
        self._cls = t_class
        self._cache: dict[int, T] = {}
    def get(self, id: int) -> T | None:
        return self._cache.get(id)
    def add(self, id: int, value: T) -> None:
        if not isinstance(value, self._cls):
            raise TypeError
        self._cache[id] = value

# 3.12+ form – cleaner
class RepositoryNew[T]:
    def __init__(self, t_class: type[T]) -> None:
        self._cls = t_class
        self._cache: dict[int, T] = {}
    def get(self, id: int) -> T | None:
        return self._cache.get(id)

# Subclassing – fixes T
class User: ...
class UserRepo(Repository[User]):
    def by_email(self, email: str) -> User | None:
        return None # placeholder
```

Q16 · ParamSpec — Typing Decorators That Forward Arguments

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Microsoft India, Goldman Sachs · Topic: Generics

The question (interviewer's verbatim phrasing):

I want to write a decorator that preserves the exact signature of the wrapped function. How do I type it?

The 30-second answer: Use `ParamSpec` (PEP 612, 3.10+). `P = ParamSpec('P')` captures the parameters of the wrapped function; `Callable[P, R]` accepts a function with those parameters, and your wrapper accepts `*args: P.args, **kwargs: P.kwargs` to forward them. The checker

tracks the original signature through the decorator, so type errors on the wrapped function point at the original call site.

Step-by-step thought process: 1. "Before ParamSpec — typing a decorator was lossy. The best you could do was `Callable[..., R]` which meant 'any signature returning R', losing the parameter names and types. The IDE would show you the wrapped function's signature, not the original." 2. "With ParamSpec — declare `P = ParamSpec('P')` and `R = TypeVar('R')`. The decorator takes `Callable[P, R]` and returns `Callable[P, R]` — exactly the same signature. Inside the wrapper, `*args: P.args, **kwargs: P.kwargs` capture and forward." 3. "3.12 syntax — `def cache(**P, R)(fn: Callable[P, R]) -> Callable[P, R]`. The `**P` declares P as a ParamSpec; the `R` is a regular TypeVar. The bracket form scopes them to the function." 4. "Practical wins — autocomplete on the decorated function works. Calling `@cache def get_user(uid: int) -> User`, then `get_user(uid='abhi')` is correctly flagged because uid expects int. Without ParamSpec, the kwarg name would be lost." 5. "Limitations — ParamSpec captures positional and keyword args together. To express 'add an argument', use `Concatenate` (next question). Bare ParamSpec only handles 'same signature, in and out.'"

Edge cases to mention: - `P.args` and `P.kwargs` are special — they're not real attributes you can index; they're symbolic markers the checker understands. Trying to inspect them at runtime returns regular `args / kwargs` tuples and dicts. - ParamSpec inherits from TypeVar conceptually but lives in `typing.ParamSpec`. The 3.12 `[**P]` syntax is purely sugar. - For class-based decorators (a class with `__call__`), ParamSpec still works; the class is `Generic[P, R]` and `__call__` takes `*args: P.args, **kwargs: P.kwargs`.

What NOT to say: - "I'll use `*args, **kwargs` typed as `Any`." That works but defeats type checking — the decorator becomes a typing black hole. - "ParamSpec is the same as TypeVar." Different — TypeVar is one type, ParamSpec is a whole signature shape (multiple parameters with names and types).

Common follow-up: "What if my decorator adds extra logic that requires an extra argument?" — Use `Concatenate[NewArg, P]` to say "prepend a new argument to the captured signature." Next question covers this.

```

from typing import Callable, ParamSpec, TypeVar
from functools import wraps
import time

P = ParamSpec('P')
R = TypeVar('R')

def timed(fn: Callable[P, R]) -> Callable[P, R]:
    @wraps(fn)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        start = time.perf_counter()
        try:
            return fn(*args, **kwargs)
        finally:
            print(f"{fn.__name__}: {time.perf_counter() - start:.3f}s")
    return wrapper

@timed
def fetch_user(user_id: int, *, strict: bool = False) -> dict:
    return {"id": user_id}

fetch_user(42)           # type checker sees real signature
# fetch_user("abhi")     # type error – int expected
# fetch_user(42, strict="x") # type error – bool expected

# 3.12 form
def timed_new[P, R](fn: Callable[P, R]) -> Callable[P, R]:
    @wraps(fn)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        return fn(*args, **kwargs)
    return wrapper

```

Q17 · **Concatenate** — Adding an Argument to a Captured Signature

Tags: Difficulty: Hard · Asked at: Cred, Postman, Microsoft India · Topic: Generics

The question (interviewer's verbatim phrasing):

My decorator passes a context object as the first arg to the wrapped function. How do I type that?

The 30-second answer: **Concatenate** (PEP 612) lets you prepend new parameters to a ParamSpec.

`Callable[Concatenate[Ctx, P], R]` says "this callable takes a Ctx first, then whatever P captures." Useful for decorators that inject a database session, request context, or user object as the first argument while preserving the rest of the original signature.

Step-by-step thought process: 1. "Setup — `P = ParamSpec('P'); R = TypeVar('R')` .

Without Concatenate, a decorator that wants to inject a context can capture `Callable[P, R]` only if `P` already includes the context. We want to STRIP one parameter (the context) from the captured signature." 2. "Concatenate prepends — `Callable[Concatenate[Ctx, P], R]` is 'takes Ctx and then everything in P.' If P captures `(int, str)` , then the full signature is `(Ctx, int, str) -> R` ." 3. "Use case — `@inject_db def fetch(db: Database, user_id: int) -> User` . The decorator takes the signature `(Database, int) -> User` and returns `(int) -> User` — caller doesn't pass db, decorator supplies it. Concatenate models this." 4. "Decorator type — `def inject_db(fn: Callable[Concatenate[Database, P], R]) -> Callable[P, R]` . Takes a function-with-Database-first, returns a function-without-Database (Database is supplied internally)." 5. "3.12 syntax — `def inject_db(**P, R)(fn: Callable[Concatenate[Database, P], R]) -> Callable[P, R]` . Same shape, scoped parameters. Concatenate itself doesn't change between syntaxes."

Edge cases to mention: - Concatenate only prepends, not appends. There's no Append type — for adding a trailing argument you'd need to spell out a full Protocol or restructure. Most decorators add leading context, so this is rarely a constraint. - Concatenate's last element must be a ParamSpec — `Concatenate[A, B, P]` is OK, `Concatenate[A, B]` is not. The ParamSpec is the variadic tail. - Combined with default values — Concatenate doesn't carry default values; if the original function had defaults, the wrapped version preserves them via call-site behavior, but the type-checker tracks just the types.

What NOT to say: - "Concatenate can append." It can't — only prepend. Library design typically puts injected context first; this matches Concatenate's shape. - "Concatenate is for joining signatures." Not really — it's for prepending fixed parameters to a captured variable signature.

Common follow-up: "What if I want to strip multiple leading arguments?" — `Callable[Concatenate[A, B, P], R]` to the decorator becomes `Callable[P, R]` returned. The wrapped function appears to lose both A and B from the outside.

```

from typing import Callable, Concatenate, ParamSpec, TypeVar
from functools import wraps

P = ParamSpec('P')
R = TypeVar('R')

class Database:
    def query(self, sql: str) -> list:
        return []

_global_db = Database()

def inject_db(fn: Callable[Concatenate[Database, P], R]) -> Callable[P, R]:
    @wraps(fn)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        return fn(_global_db, *args, **kwargs)
    return wrapper

@inject_db
def fetch_user(db: Database, user_id: int) -> dict:
    rows = db.query(f"SELECT * FROM users WHERE id = {user_id}")
    return rows[0] if rows else {}

# Callers don't pass db
fetch_user(42)                                # type checker sees signature (int) -> dict
# fetch_user(_global_db, 42)                  # type error – too many args

# 3.12 form
def inject_db_new[**P, R](
    fn: Callable[Concatenate[Database, P], R]
) -> Callable[P, R]:
    @wraps(fn)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        return fn(_global_db, *args, **kwargs)
    return wrapper

```

Q18 · **TypeVarTuple** — Variadic Generics (PEP 646, 3.11+)

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman Sachs, ML-role panels · Topic: Generics

The question (interviewer's verbatim phrasing):

What's a variadic generic? When would I use **TypeVarTuple** ?

The 30-second answer: **TypeVarTuple** (PEP 646, 3.11+) lets a generic type take an arbitrary number of type parameters — useful for tensor shapes, tuple types, and any "variable number of parallel

types" situation. `Ts = TypeVarTuple('Ts')` then `class Tensor(Generic[Unpack[Ts]])` captures the shape as a tuple. 3.12 syntax — `class Tensor[*Ts]`. Mostly used by numerical libraries (NumPy stubs, JAX) and rarely by application code.

Step-by-step thought process: 1. "Motivating problem — NumPy tensors have shapes. A 2D matrix is `Tensor[Height, Width]`, a 3D batch is `Tensor[Batch, Height, Width]`. With regular `TypeVar`, the number of parameters is fixed. `TypeVarTuple` lets the count vary." 2. "Declaration — `from typing import TypeVarTuple, Unpack; Ts = TypeVarTuple('Ts')`. In a class — `class Tensor(Generic[Unpack[Ts]])`. The `Unpack` is needed pre-3.12 to indicate `Ts` is a variadic position. 3.12 — `class Tensor[*Ts]` uses star syntax." 3. "Use sites — `Tensor[int, int]`, `Tensor[int, int, int]`. The checker tracks how many dimensions and their types. Functions can operate on them with shape-preserving signatures: `def transpose[*Ts](t: Tensor[*Ts]) -> Tensor[*Ts]`." 4. "Combinable — you can mix regular `TypeVars` with `TypeVarTuple`. `class Model[T, *Ts]` has one normal param plus a variadic. Powerful for things like 'a function that takes `T` followed by arbitrary positional args.'" 5. "Real-world — torch stubs, jax types, and increasingly NumPy's array-typing PEP — all use `TypeVarTuple` to model shape. Application code rarely needs it; it shows up in library boundaries."

Edge cases to mention: - `TypeVarTuple` has no `bound` parameter in 3.11; PEP 696 (3.13+) is adding defaults but no bounds. The variadic part is for shape, not capability constraints. - `Unpack` of `TypeVarTuple` in function signatures — `def cat(*args: Unpack[Ts])` — was experimental in 3.11 and improved in 3.12. Pyright supports it better than mypy historically. - In 3.12, `*Ts` inside brackets is the preferred syntax. `Unpack[Ts]` still works for explicit cases (e.g., inside a tuple).

What NOT to say: - "TypeVarTuple is for tuples." Sort of, but it's really for *parallel sequences of types* — tuples are one example, tensor shapes are the main motivation. - "Most code should use it." Not really — variadic generics are a niche tool. Application code rarely needs them; library boundaries do.

Common follow-up: "How is `TypeVarTuple` different from `tuple[int, ...]`?" — `tuple[int, ...]` is "any number of ints, all the same type." `TypeVarTuple` is "any number of types, each preserved independently." Different abstractions for different problems.

```

from typing import TypeVarTuple, Unpack, Generic

Ts = TypeVarTuple('Ts')

# Pre-3.12 form — Unpack needed
class Tensor(Generic[Unpack[Ts]]):
    def __init__(self, shape: tuple[int, ...]) -> None:
        self.shape = shape

# Shape-preserving transpose
def transpose(t: 'Tensor[Unpack[Ts]]' -> 'Tensor[Unpack[Ts]]':
    return t

# 3.12 form — star syntax
class TensorNew[*Ts]:
    def __init__(self, shape: tuple[int, ...]) -> None:
        self.shape = shape

def transpose_new[*Ts](t: TensorNew[*Ts]) -> TensorNew[*Ts]:
    return t

# Use
import builtins
t2d: TensorNew[int, int] = TensorNew((3, 4))
t3d: TensorNew[int, int, int] = TensorNew((2, 3, 4))

```

Q19 · Self Type (PEP 673, 3.11+)

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Microsoft India · Topic: Generics

The question (interviewer's verbatim phrasing):

When would I use `Self` instead of declaring a `TypeVar` bound to my class?

The 30-second answer: `Self` is the type "the current class, or any subclass of it." Use it as the return type for fluent-builder methods, factory class methods, and `__enter__`. Without `Self`, you'd write `TypeVar('T', bound='MyClass')` and decorate every method with it. `Self` cleans this up — one keyword, no `TypeVar`, and subclass calls return the correct subclass type.

Step-by-step thought process: 1. "Setup — a builder pattern. `class QueryBuilder: def where(self, cond): self._where.append(cond); return self`. Annotating `-> 'QueryBuilder'` works but breaks for subclasses — `class UserQuery(QueryBuilder): pass; UserQuery().where(...)` returns a `QueryBuilder`, not a `UserQuery`. Type checker sees the base." 2. "Pre-3.11 fix — `TypeVar` bound to the class. `T = TypeVar('T', bound='QueryBuilder');` `class QueryBuilder: def where(self: T,`

`cond) -> T: ...` . Now subclass calls return the subclass type. Verbose." 3. "3.11+ fix — `Self` .
`from typing import Self; class QueryBuilder: def where(self, cond) ->`
`Self: ...` . Same behavior, no TypeVar." 4. "Use cases — fluent builders (chained
`.where().limit().order_by()`), classmethods that construct an instance (`@classmethod def`
`from_dict(cls, d) -> Self`), context managers (`__enter__` returning `Self`), copy methods
(`def clone(self) -> Self`)." 5. "Inside a method, `Self` is the actual concrete type. So a
method that returns `Self` and is called on a `UserQuery` returns a `UserQuery` . The checker
preserves the precise type through subclassing."

Edge cases to mention: - `Self` only makes sense in instance methods or classmethods of a class. In
a module-level function, `Self` is meaningless and rejected. - For factories — `@classmethod def`
`create(cls, ...) -> Self` . Calling `UserQuery.create(...)` returns `UserQuery` , even
though the method is on the base. - `Self` does not handle "an instance of any related class." For "a
method that returns its own type OR a transformed type," you still need explicit annotations.

What NOT to say: - " `Self` is the same as `'MyClass'` ." Different — `'MyClass'` is the literal base
class; `Self` adapts to subclass. - " `Self` requires generics setup." It doesn't — it's a builtin sentinel
from `typing` , no TypeVar machinery.

Common follow-up: "How does `Self` work with copy/clone methods on a generic class?" — `class`
`Container[T]: def clone(self) -> Self: return type(self)(self.value)` . The `Self`
preserves both the concrete subclass AND the generic parameter `T`. Pre-`Self` this needed a doubled
TypeVar dance.

```

from typing import Self

class QueryBuilder:
    def __init__(self) -> None:
        self._where: list[str] = []
        self._limit: int | None = None

    def where(self, cond: str) -> Self:
        self._where.append(cond)
        return self

    def limit(self, n: int) -> Self:
        self._limit = n
        return self

    @classmethod
    def empty(cls) -> Self:
        return cls()

class UserQuery(QueryBuilder):
    def by_email(self, email: str) -> Self:
        return self.where(f'email = '{email}''')

# Self preserves the subclass type through chaining
q = UserQuery().by_email("abhi@x.in").where("active = 1").limit(10)
# q is UserQuery, not QueryBuilder – by_email is accessible on the chain

# Classmethod construction
default = UserQuery.empty()           # type is UserQuery, not QueryBuilder

```

Q20 · Variance — Covariance, Contravariance, Invariance in TypeVars

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman Sachs, ML-role panels · Topic: Generics

The question (interviewer's verbatim phrasing):

What's the difference between `TypeVar('T', covariant=True)` and the default? When does it matter?

The 30-second answer: Variance controls subtyping for generic containers. A covariant `TypeVar` (`covariant=True`) means `Container[Dog]` is a subtype of `Container[Animal]` — read-only positions. Contravariant means the opposite — `Handler[Animal]` is a subtype of `Handler[Dog]` — write-only positions. Invariant (the default) means neither — `list[Dog]` is *not* a subtype of `list[Animal]` because lists are mutable. Mostly relevant when designing generic Protocols and immutable containers.

Step-by-step thought process: 1. "Setup — `Animal` and `Dog(Animal)`. A `list[Dog]` — is it usable wherever `list[Animal]` is expected? No, because someone with a `list[Animal]` could append a Cat, breaking the Dog list. So `list` is invariant — the parameter must match exactly." 2. "For read-only — `Sequence[Dog]` is a subtype of `Sequence[Animal]` because you can only read; appending isn't possible. Sequence is covariant in its parameter." 3. "For write-only — a `Callable[[Animal], None]` is a subtype of `Callable[[Dog], None]` because if you can handle any Animal, you can certainly handle a Dog. Callable is contravariant in its parameters." 4. "Declaring — `T = TypeVar('T')` (invariant), `T_co = TypeVar('T_co', covariant=True)`, `T_contra = TypeVar('T_contra', contravariant=True)`. The `_co` and `_contra` suffixes are stdlib convention for the variant TypeVars." 5. "When you care — designing Protocols and generic immutable types. `class ReadOnlyContainer(Protocol[T_co])` makes the container covariant; users get more flexible substitution. Mutable containers stay invariant for safety."

Edge cases to mention: - A type variable used in both input and output positions of a method usually must be invariant. Mypy will refuse to declare it co/contravariant if the use is mixed. - For Self type (3.11+), variance is automatic — Self behaves correctly without explicit annotation. - 3.12 PEP 695 syntax does *not* yet support variance annotations on bracket-form TypeVars. Use the legacy TypeVar with `covariant=True` if you need variance.

What NOT to say: - "Variance only matters for advanced library design." Sort of true, but it shows up surprisingly often when Protocols intersect with subclass hierarchies. Worth knowing the concept. - "Lists should be covariant." Wrong — that's the classic Java/C# mistake. Mutable containers must be invariant for type safety.

Common follow-up: "What's a real-world example of contravariance you'd run into?" — Event handlers and callbacks. `Callable[[BaseEvent], None]` accepts a function that handles BaseEvent or any subclass position; substituting a function that handles `LoginEvent` would fail because the handler isn't equipped to deal with other BaseEvent subclasses. Contravariance gets this right.

```

from typing import TypeVar, Protocol, Generic

T = TypeVar('T')                                # invariant
T_co = TypeVar('T_co', covariant=True)          # covariant
T_contra = TypeVar('T_contra', contravariant=True) # contravariant

class Animal: ...
class Dog(Animal): ...

# Covariant read-only container
class ReadOnly(Protocol[T_co]):
    def get(self) -> T_co: ...

def read_animal(r: ReadOnly[Animal]) -> None: ...
def read_dog(r: ReadOnly[Dog]) -> None: ...

dog_box: ReadOnly[Dog]
# read_animal(dog_box)          # OK – Dog is Animal, covariance allows it

# Contravariant handler
class Handler(Protocol[T_contra]):
    def handle(self, x: T_contra) -> None: ...

def use_dog_handler(h: Handler[Dog]) -> None: ...

animal_handler: Handler[Animal]
# use_dog_handler(animal_handler) # OK – Animal-handler can do Dog, contravariance

# Invariant – must match exactly
def receives_dog_list(items: list[Dog]) -> None: ...
animals: list[Animal] = [Dog()]
# receives_dog_list(animals)      # type error – list is invariant

```

Section C — `typing.Protocol`

The 2026 differentiator. Protocol is what makes Python typing fundamentally different from Java's typing: structural subtyping at the language-tooling level. Razorpay, Cred, Postman senior rounds and Microsoft GCC panels will ask three to four Protocol questions back-to-back to separate "knows the syntax" from "thinks in structural types."

Q21 · Structural vs Nominal Subtyping — The Mental Model

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Microsoft India, Goldman · Topic: Protocols

The question (interviewer's verbatim phrasing):

What's the difference between structural and nominal subtyping? Where does Python sit?

The 30-second answer: Nominal subtyping is "X is a subclass of Y because the declaration says so" — Java's `class Dog extends Animal`. Structural subtyping is "X is a subtype of Y because X has all the methods and attributes Y requires" — Go interfaces, TypeScript types, Python's `Protocol`. Python defaults to nominal for ABCs (`isinstance(x, MyABC)` checks registration) but adds structural typing via `Protocol` (PEP 544, 3.8+) for the duck-typing tradition.

Step-by-step thought process: 1. "Nominal — relationships are by name. To be a `Reader`, you must `class MyReader(Reader)` or `Reader.register(MyReader)`. The relationship is declared explicitly." 2. "Structural — relationships are by shape. To satisfy `class Reader(Protocol): def read(self) -> bytes: ...`, any class with a method called `read` returning bytes counts, regardless of inheritance. No declaration needed." 3. "Python's duck typing has always been structural at runtime — 'if it walks like a duck...' But before Protocol, the type system couldn't express this. You had to use `Any` (no checking) or ABC (forced inheritance). Protocol bridges that gap." 4. "When nominal wins — when the relationship is part of the design contract. `IOError` should subclass `OSError` to participate in the exception hierarchy; that's by-name. Domain types — `class PaidOrder(Order)` — are nominal: you want the relationship to be intentional." 5. "When structural wins — for plumbing and adapters. 'I take any cache-like object with `.get` and `.set`.' Forcing every cache implementer to import your Protocol breaks the dependency direction. Protocol lets you describe the shape without owning the types."

Edge cases to mention: - A class can satisfy a Protocol by accident. `class Logger: def read(self): return b'log'` satisfies the Reader Protocol unintentionally because of name overlap. Naming Protocols thoughtfully (`SupportsRead`) reduces false positives. - Protocol checks happen at static analysis. At runtime there's no implicit relationship — `isinstance(obj, MyProtocol)` raises unless `@runtime_checkable` is applied. - Nominal and structural can coexist — a class can subclass ABC AND structurally match an unrelated Protocol. Type checker handles both.

What NOT to say: - "Python only does structural typing." Wrong — it does both. ABC and concrete classes are nominal; Protocol is structural. - "Duck typing and structural typing are the same." Closely related but not identical — duck typing is the runtime behavior (no checking at all); structural typing is the static-analysis discipline.

Common follow-up: "Give me a case where nominal subtyping is wrong but Protocol gets it right." — A function that takes "anything writable." The list of types is open-ended — file, BytesIO, socket, custom buffer. Forcing them all to subclass a `Writable` ABC is impossible (third-party types you don't own). Protocol describes the shape; anyone with the right methods qualifies.

```

from typing import Protocol

# Nominal – must inherit from ABC
from abc import ABC, abstractmethod

class ReaderABC(ABC):
    @abstractmethod
    def read(self) -> bytes: ...

class FileReader(ReaderABC):
    def read(self) -> bytes:
        return b"file"

# Some other reader I don't own
class CacheReader:
    def read(self) -> bytes:
        return b"cache"

# isinstance(CacheReader(), ReaderABC) # False – not registered

# Structural – Protocol matches by shape
class ReaderProto(Protocol):
    def read(self) -> bytes: ...

def use(r: ReaderProto) -> bytes:
    return r.read()

use(FileReader())      # OK – nominal subclass also satisfies
use(CacheReader())     # OK – structural match, no inheritance needed

```

Q22 · Defining a Protocol — Methods, Attributes, Properties

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Microsoft India · Topic: Protocols

The question (interviewer's verbatim phrasing):

Show me how to define a Protocol with both methods and attributes. What about properties?

The 30-second answer: Inherit from `typing.Protocol`. Methods are declared with `...` as the body (no implementation). Attributes are declared with type annotations and no value — `name: str`. Properties use the `@property` decorator with `...` body. The Protocol class itself doesn't implement anything; it just describes the shape that satisfying classes must have.

Step-by-step thought process: 1. "Class declaration — `class Drawable(Protocol):`. From `typing` import `Protocol`. The body lists method signatures with `...` (or `pass`) — the body is a placeholder." 2. "Method — `def draw(self) -> None: ...`. The checker verifies any class

passed where *Drawable* is expected has a `draw` method with this signature." 3. "Attribute — `width: int` declared inside the Protocol body without a value. Satisfying classes must have an instance attribute named `width` of type `int`. Static checkers verify this on `__init__` and at access sites." 4. "Property — `@property def area(self) -> float: ...` declares a read-only property. Satisfying classes can implement `area` as a property OR as a regular attribute — both work for the Protocol." 5. "Default implementations — Protocols can have method bodies that are real implementations. Then satisfying classes inherit them if they explicitly subclass. But typically Protocols are pure shape descriptions — leave the bodies as `...`."

Edge cases to mention: - An attribute on a Protocol can be marked `ClassVar` — `code: ClassVar[int] = 0` — to require it as a class attribute, not instance. - Protocols don't enforce method *names* at runtime; if a class is missing a method, the error surfaces only when you try to call it. - Private attributes (`_internal: int`) in a Protocol are still part of the structural check — the satisfying class must have `_internal` accessible. Usually you wouldn't put private attributes in a Protocol.

What NOT to say: - "Protocols use `pass` instead of `...`." Either works; `...` is convention. - "Protocols can have only methods, not attributes." Wrong — attributes are first-class in Protocols.

Common follow-up: "What happens if I add a method to a Protocol after a class is already 'satisfying' it?" — Type checker re-evaluates at the next check. The previously-satisfying class is now missing the new method; type errors surface at all use sites. Real refactor pain at scale — version Protocols carefully.

```

from typing import Protocol, ClassVar

class Drawable(Protocol):
    # Attributes
    width: int
    height: int

    # Property
    @property
    def area(self) -> float: ...

    # Method
    def draw(self, x: int, y: int) -> None: ...

    # Class-level
    SHAPE_NAME: ClassVar[str]

# A satisfier – implements everything structurally
class Rectangle:
    SHAPE_NAME = "rect"

    def __init__(self, w: int, h: int) -> None:
        self.width = w
        self.height = h

    @property
    def area(self) -> float:
        return self.width * self.height

    def draw(self, x: int, y: int) -> None:
        print(f"draw rect at {x},{y}")

def render(d: Drawable) -> None:
    d.draw(0, 0)

render(Rectangle(10, 20))    # OK – structurally matches

```

Q23 · @runtime_checkable — When isinstance Works on a Protocol

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Protocols

The question (interviewer's verbatim phrasing):

If I have `class Reader(Protocol): def read(self) -> bytes: ...`, does `isinstance(obj, Reader)` work?

The 30-second answer: Not by default. `isinstance(obj, Reader)` raises `TypeError: only @runtime_checkable protocols can be used with isinstance()`. Apply the `@runtime_checkable` decorator — `@runtime_checkable class Reader(Protocol)` — and now `isinstance` does a name-only check (does the object have a `read` attribute?). It does *not* check method signatures, so a class with `read` taking different arguments still passes.

Step-by-step thought process: 1. "Default behavior — Protocols are static-analysis only. At runtime, `isinstance(x, Reader)` raises `TypeError` because `Protocol` is not a runtime check by default. The static checker covers it." 2. "`@runtime_checkable` flips this — the protocol gains a `__instancecheck__` that walks the protocol's declared attributes and methods, verifying each is present on the object via `hasattr`." 3. "What it checks — only that the names exist, not that the signatures match. A class with `def read(self, n: int) -> str: ...` will pass `isinstance(obj, Reader)` even though the signature differs. The runtime check is name-based; the static checker is signature-based." 4. "Cost — `isinstance` on a `@runtime_checkable` protocol is slower than a regular `isinstance` because it walks every protocol attribute via `getattr` on the instance. For hot paths, prefer static checking and skip the runtime check." 5. "When to use — adapters, dispatch tables, framework code that accepts duck-typed inputs at runtime. For example, `if isinstance(obj, Iterable): for x in obj` lets you accept anything iterable without forcing inheritance."

Edge cases to mention: - Attribute-only Protocols (no methods, just `name: str`) still work with `@runtime_checkable` but the check only verifies the attribute exists, not that it's a string. Type erasure applies to attribute types too. - Subclasses inherit `@runtime_checkable` — if you extend a runtime-checkable Protocol, the child is also runtime-checkable. - 3.12 changed the failure mode — using `isinstance` on a non-runtime-checkable Protocol used to be silently False in older versions; now it raises `TypeError` explicitly to catch the mistake.

What NOT to say: - "`@runtime_checkable` makes the check rigorous." It doesn't — runtime checks signature-less, name-only. Static checks remain the source of truth. - "All Protocols should be runtime-checkable." No — only when you need `isinstance` at runtime. Otherwise the decorator adds nothing.

Common follow-up: "Why doesn't runtime check the signatures?" — Performance and reflection limitations. Walking signatures via `inspect` is slow and can't always introspect C-implemented or dynamically-created methods. Name-only is the pragmatic compromise; static checkers catch signature mismatches.

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class Reader(Protocol):
    def read(self) -> bytes: ...

class GoodReader:
    def read(self) -> bytes:
        return b"good"

class WeirdReader:
    def read(self, n: int) -> str:    # different signature
        return "weird"

print(isinstance(GoodReader(), Reader))    # True
print(isinstance(WeirdReader(), Reader))    # True — name match, sig not checked!

# Static checker catches the WeirdReader mismatch; runtime doesn't.

# Without @runtime_checkable
class Writer(Protocol):
    def write(self, data: bytes) -> int: ...

class FileWriter:
    def write(self, data: bytes) -> int:
        return len(data)

try:
    isinstance(FileWriter(), Writer)
except TypeError as e:
    print(e)    # only @runtime_checkable protocols can be used with isinstance()
```

Q24 · Protocol vs ABC — When Each Wins

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Goldman, Microsoft India · Topic: Protocols

The question (interviewer's verbatim phrasing):

When would you reach for Protocol versus ABC? Don't they do the same thing?

The 30-second answer: ABC for *nominal* relationships you own — your domain hierarchy where being a member is meaningful and intentional (Payment, Refund, Order types). Protocol for *structural* relationships across boundaries — describing the shape of inputs from third-party or pluggable code without forcing them to import your interface. ABC enforces inheritance and gets `isinstance` for free; Protocol works on any matching class and stays static-only by default.

Step-by-step thought process: 1. "Decision rule — who owns the implementers? If you do, ABC is fine. If callers do, Protocol is the only reasonable choice; you can't force them to inherit from your class." 2. "ABC strengths — `isinstance` works without decoration. ABC can have default method implementations that subclasses inherit. Abstract methods raise at instantiation if not overridden. Good for hierarchies inside your codebase." 3. "Protocol strengths — no import dependency in the implementer's direction. Third-party types satisfy automatically. Better matches Python's duck-typing soul. The 2026-era go-to for interface design." 4. "Hybrid pattern — declare the shape as a Protocol (for static checking on inputs) AND register concrete implementations with an ABC (for runtime dispatch). Less common but legal." 5. "Performance — at runtime, ABCs and runtime-checkable Protocols both add cost to `isinstance`. ABCs are slightly faster (one mro walk vs attribute checks). For hot loops, avoid both — use static checking only."

Edge cases to mention: - ABCs in `collections.abc` (`Iterable`, `Sized`, `Container`) are partly structural — they use `__subclasshook__` to recognize classes with the right methods even without registration. So `collections.abc.Iterable` is somewhere between ABC and Protocol. - Protocols can subclass other Protocols — `class WritableReader(Reader, Writer)` describes a class with both shapes. ABCs can do the same. - ABCs can have non-abstract methods that subclasses inherit; Protocols *can* but conventionally don't. If you find yourself wanting shared logic, ABC is the better fit.

What NOT to say: - "Protocols are always better." Not always — for domain hierarchies inside your codebase, ABCs are clearer because the relationship is the design. - "ABCs and Protocols are mutually exclusive." A class can implement an ABC AND match a Protocol; both can describe it.

Common follow-up: "Can a Protocol have default method implementations?" — Yes, technically. But the convention is to keep Protocols as pure shape descriptions; default implementations land in ABCs or mixins.

```

from typing import Protocol, runtime_checkable
from abc import ABC, abstractmethod

# Domain hierarchy – ABC fits
class PaymentMethod(ABC):
    @abstractmethod
    def charge(self, amount: int) -> bool: ...

    def receipt(self) -> str:                # default implementation
        return "Thank you for your payment"

class UPI(PaymentMethod):
    def charge(self, amount: int) -> bool:
        return True

# Boundary interface – Protocol fits
@runtime_checkable
class Logger(Protocol):
    def info(self, msg: str) -> None: ...
    def error(self, msg: str) -> None: ...

# Third-party loggers – none of them know about us
import logging
def use_logger(log: Logger) -> None:
    log.info("starting")
    log.error("failed")

use_logger(logging.getLogger("app"))        # works – stdlib logger structurally matches

class MyJsonLogger:
    def info(self, msg): print({"lvl": "info", "msg": msg})
    def error(self, msg): print({"lvl": "err", "msg": msg})

use_logger(MyJsonLogger())                 # works – no inheritance needed

```

Q25 · Generic Protocols — Parameterizing Shape Descriptions

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Microsoft India, Goldman · Topic: Protocols

The question (interviewer's verbatim phrasing):

How would I write a Protocol for "a container with a get-by-key method that returns the value type"?

The 30-second answer: A generic Protocol — parameterize it with a TypeVar. `class`
`Lookup(Protocol[T]): def get(self, key: str) -> T: ...`. Now `Lookup[int]` is a

container of ints, `Lookup[User]` is a container of Users. The TypeVar can be covariant for read-only access (`T_co`) and the Protocol describes any matching shape, generic-correctly.

Step-by-step thought process: 1. "Setup — `T = TypeVar('T');` `class Lookup(Protocol[T]):` . Note `Protocol[T]` not `Generic[T]` plus Protocol — Protocol itself takes the type parameter, no need to inherit Generic separately." 2. "Inside the body, `T` can appear in any method signature. `def get(self, key: str) -> T: ...` ties the lookup to a specific element type at the use site." 3. "3.12 syntax — `class Lookup[T](Protocol):` . The bracket form for the type parameter, Protocol as the base. Equivalent to the legacy form." 4. "Variance for read-only Protocols — declare `T` as covariant. `T_co = TypeVar('T_co', covariant=True);` `class Lookup(Protocol[T_co])` . Now `Lookup[Dog]` is a subtype of `Lookup[Animal]` — flexibility for callers." 5. "Use site — `def render(items: Lookup[str]) -> None` . Any class with a matching `.get(key) -> str` method satisfies, including custom dict-like classes you don't own."

Edge cases to mention: - A generic Protocol's structural check is parameterized too. A class with `def get(self, key: str) -> int` satisfies `Lookup[int]` but not `Lookup[str]` — element type matters. - Mixing generic and concrete in the same Protocol — `class Cache(Protocol[T]):` `name: str; def get(self, k: str) -> T: ...` . The `name` attribute is fixed; `T` parameterizes the storage type. - Multiple type parameters — `class Mapping(Protocol[K, V]):` `def __getitem__(self, k: K) -> V: ...` . Common for dict-like protocols.

What NOT to say: - "Generic Protocols need both `Generic[T]` and `Protocol` ." Wrong — `Protocol[T]` covers both roles. - "Protocols can't be parameterized." They can, since Protocol was introduced in 3.8.

Common follow-up: "What's the runtime cost of a generic Protocol?" — Same as a non-generic Protocol — zero at runtime unless `@runtime_checkable` , in which case the element type is *not* checked at runtime (only the method names). Static checker carries the parameterized check.

```

from typing import Protocol, TypeVar

T = TypeVar('T')
T_co = TypeVar('T_co', covariant=True)

# Generic Protocol – Protocol takes T directly
class Lookup(Protocol[T_co]):
    def get(self, key: str) -> T_co: ...

# 3.12 form
class LookupNew[T](Protocol):
    def get(self, key: str) -> T: ...

# Concrete implementer – no inheritance
class StringStore:
    def __init__(self) -> None:
        self._d: dict[str, str] = {}
    def get(self, key: str) -> str:
        return self._d.get(key, "")

class IntStore:
    def __init__(self) -> None:
        self._d: dict[str, int] = {}
    def get(self, key: str) -> int:
        return self._d.get(key, 0)

# Use sites with parameterization
def render_str(items: Lookup[str]) -> None: ...
def render_int(items: Lookup[int]) -> None: ...

render_str(StringStore())      # OK
render_int(IntStore())         # OK
# render_str(IntStore())       # type error – int doesn't match str

# Multi-parameter mapping-like Protocol
K = TypeVar('K')
V = TypeVar('V')
class TypedMap(Protocol[K, V]):
    def __getitem__(self, k: K) -> V: ...
    def __contains__(self, k: object) -> bool: ...

```

Q26 · Protocol Intersection — Multiple Shape Requirements

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman, Cred · Topic: Protocols

The question (interviewer's verbatim phrasing):

I want to accept an object that is both Readable AND Closeable. How do I express that?

The 30-second answer: Python's typing has no built-in `Intersection` (yet — there's draft PEP 750 territory). The canonical workaround is a new Protocol that inherits from both — `class ReadableCloseable(Reader, Closeable, Protocol): ...`. The child Protocol structurally requires all methods from both parents. Use this combined Protocol as the parameter type.

Step-by-step thought process: 1. "Wishlist — `def use(x: Reader & Closeable)` to express 'must have read AND close methods.' Python doesn't have `&` for type intersection at the type level (yet)." 2. "Workaround — `class ReadableCloseable(Reader, Closeable, Protocol): pass`. By inheriting `Reader` and `Closeable` (both Protocols), the new Protocol's structural requirement is the union of methods from both. Pure shape composition." 3. "Note the `Protocol` base must be in the inheritance — otherwise it becomes a regular class that requires nominal subclassing. The Protocol mark is what keeps it structural." 4. "For one-off cases — `TypeVar` with multiple bounds doesn't exist either (`bound=` is single). Sometimes a quick Protocol inline does the job: `class _RC(Reader, Closeable, Protocol): pass` and use `_RC`." 5. "In practice — most real intersections are 'Reader + Closeable' style (IO interfaces). `Stdlib` already has `IO[bytes]` / `IO[str]` Protocols that combine these. For most cases the `stdlib` intersections are pre-built."

Edge cases to mention: - Multiple inheritance of Protocols only works cleanly when methods don't conflict. If `Reader` and `Writer` both define `def close(self) -> None`, the merged Protocol still has one `close` with matching signature. If signatures conflict, you get a type error at Protocol definition time. - The intersection Protocol works at runtime with `@runtime_checkable` — `isinstance(x, ReadableCloseable)` walks all inherited Protocol's attributes. - PEP 750 / Intersection types are in active discussion as of 2026; expect `Intersection[A, B]` or `A & B` in future Python. For now, intersection via inheritance is the pattern.

What NOT to say: - "Use `Union[Reader, Closeable]`." Union means "Reader OR Closeable" — the opposite. Intersection means "Reader AND Closeable" both at once. - "Python has `&` for type intersection." Not yet — that's the proposed future syntax.

Common follow-up: "What if I want one of `Reader` / `Writer` plus `Closeable`?" — Compose: `class ReadOrWriteCloseable(Closeable, Protocol): ...` paired with a parameter type `(Reader | Writer) & Closeable` — but the intersection part still needs the explicit combined Protocol. Express what you can, leave the rest to docstrings.

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class Reader(Protocol):
    def read(self) -> bytes: ...

@runtime_checkable
class Closeable(Protocol):
    def close(self) -> None: ...

# Intersection via inheritance – must include Protocol in bases
@runtime_checkable
class ReadableCloseable(Reader, Closeable, Protocol):
    pass    # no new methods; just composes the requirement

def use(stream: ReadableCloseable) -> bytes:
    try:
        return stream.read()
    finally:
        stream.close()

# Any class with BOTH methods structurally matches
class ByteFile:
    def read(self) -> bytes:
        return b"hello"
    def close(self) -> None:
        print("closed")

class ReadOnly:
    def read(self) -> bytes:
        return b"data"
    # no close – won't satisfy

use(ByteFile())           # OK
# use(ReadOnly())         # type error – missing close
```

Q27 · Protocol as Duck Typing With Types — The "Cache Argument" Idiom

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred · Topic: Protocols

The question (interviewer's verbatim phrasing):

I accept a `cache` argument in my function and call `.get()` and `.set()` on it. How do I type that without forcing the caller to use a specific class?

The 30-second answer: Define a Protocol with just `.get` and `.set` . `class`
`CacheLike(Protocol): def get(self, key: str) -> bytes | None: ...; def`
`set(self, key: str, value: bytes) -> None: ...` . Any class with matching methods
 satisfies — Redis client, dict wrapper, in-memory cache, test double. Use `cache: CacheLike` as the
 parameter type. The caller never has to import or inherit anything.

Step-by-step thought process: 1. "The duck-typing impulse — 'I need a cache, I don't care what kind, as long as it has get and set.' Pre-Protocol you'd type it `Any` and lose checking. Protocol gives you the same flexibility with static guarantees." 2. "The Protocol describes the contract — get returns bytes-or-None on miss, set returns None. Methods you actually call. Don't include methods you don't use; over-specifying makes the Protocol harder to satisfy." 3. "Naming convention — `SupportsX` from `stdlib` (`SupportsRead`, `SupportsInt`), or `XLike` (`DictLike`, `CacheLike`), or descriptive (`KeyValueStore`). Pick something that signals 'this is a shape, not a class.'" 4. "Test doubles — easy.

`class FakeCache: def get(self, key): return self._d.get(key); def set(self, key, value): self._d[key] = value` . No `MagicMock` , no `unittest.mock` heroics. Static checker verifies the shape matches the real Protocol." 5. "Real-world traceback — `def fetch(url: str, cache: CacheLike) -> bytes` . Pass `redis.Redis()` , `dict()` wrapped, or your `FakeCache` . All work, all type-check. Vendor-neutral signatures."

Edge cases to mention: - Match `dict` directly — `dict` has `.get` but its `.set` is `__setitem__` , not `.set` . So `dict` doesn't satisfy a Protocol requiring `.set` method. Adjust the Protocol to use `__getitem__` / `__setitem__` if you want dict to fit, or wrap dict in an adapter. - Mutability — Protocols are descriptive, not enforced. A "read-only cache" Protocol with just `.get` doesn't prevent the caller from passing a mutable cache that happens to satisfy. - Asynchronous variant — `class AsyncCache(Protocol): async def get(self, key: str) -> bytes | None: ...` . Protocols handle async methods natively.

What NOT to say: - "Use `Any` for the cache argument." Loses all type safety; defeats the point of adding types in the first place. - "Force the caller to use my `CacheBase` class." Inverts the dependency — Protocol gets it right with structural matching.

Common follow-up: "How would you mock a Protocol-typed dependency in tests?" — Just implement a fake class with the right methods. No mock library needed. The Protocol guarantees structural fit; the static checker verifies. Fakes become first-class.

```

from typing import Protocol

class CacheLike(Protocol):
    def get(self, key: str) -> bytes | None: ...
    def set(self, key: str, value: bytes) -> None: ...

def fetch(url: str, cache: CacheLike) -> bytes:
    cached = cache.get(url)
    if cached is not None:
        return cached
    # ... network fetch ...
    data = b"fetched from network"
    cache.set(url, data)
    return data

# In-memory cache – satisfies structurally
class DictCache:
    def __init__(self) -> None:
        self._d: dict[str, bytes] = {}
    def get(self, key: str) -> bytes | None:
        return self._d.get(key)
    def set(self, key: str, value: bytes) -> None:
        self._d[key] = value

# Test double – no inheritance, no mock
class FakeCache:
    def __init__(self) -> None:
        self.calls: list[tuple] = []
    def get(self, key: str) -> bytes | None:
        self.calls.append(("get", key))
        return None
    def set(self, key: str, value: bytes) -> None:
        self.calls.append(("set", key, value))

# Both work, both type-check
fetch("/page", DictCache())
fetch("/page", FakeCache())

```

Q28 · `__protocol_attrs__` and Internal Protocol Mechanics

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman, ML-role panels · Topic: Protocols

The question (interviewer's verbatim phrasing):

How does Python's runtime know what attributes a Protocol declares, for runtime-checkable Protocols?

The 30-second answer: Internally each Protocol class stores its declared method and attribute names on `__protocol_attrs__` (3.12+) — a frozenset of strings. When `isinstance(x, MyProto)` runs on a runtime-checkable Protocol, the dispatcher walks `__protocol_attrs__` and calls `hasattr(x, name)` for each. If all pass, the instance check returns True. Earlier versions used `_get_protocol_attrs` (private helper); 3.12 promoted it to a class attribute.

Step-by-step thought process: 1. "Behind the scenes — when you define `class Reader(Protocol): def read(self): ...`, Python walks the class body, picks out method definitions and annotated attributes, and stores their names in `__protocol_attrs__`. Special names like `__init__`, `__new__`, `__class__` are excluded automatically." 2. "Why this exists — runtime introspection for `isinstance`. Without storing the list, the check would have to walk the MRO every call. Storing it once at class-definition time makes the check $O(attrs)$ per call." 3. "3.12 change — promoted `__protocol_attrs__` to a public(ish) class attribute. Pre-3.12 it was `_proto_hook` machinery, internal and not stable. 3.12 makes inspection reliable." 4. "Edge — methods inherited from non-Protocol bases are excluded. So if a Protocol extends a regular class for default impls, only the Protocol-side methods count toward `__protocol_attrs__`. Keeps the check focused on the shape." 5. "Practical — library authors writing Protocol-aware code (Pydantic v2 plugins, attrs hooks) can read `__protocol_attrs__` to generate validators or adapters. Application code shouldn't touch it."

Edge cases to mention: - Empty Protocols — `class Empty(Protocol): pass` has empty `__protocol_attrs__`. `isinstance(anything, Empty)` returns True. Sometimes used as a marker, but mostly an error condition. - Protocols inheriting Protocols — attrs are merged. `class WriteableReader(Reader, Writer, Protocol)` has attrs from both, with no duplicates. - Names starting with `_` (single underscore) are included; names starting with `__` and *not* ending in `__` are name-mangled and rarely useful in Protocols.

What NOT to say: - "`__protocol_attrs__` is documented public API." It's quasi-public — 3.12 promoted it but the typing module's surface around Protocol is still evolving. Treat as internal but stable enough for library use. - "Runtime checks Protocol signatures via `__protocol_attrs__`." Names only, not signatures.

Common follow-up: "What's the performance of `isinstance` on a runtime-checkable Protocol with 20 attributes?" — Linear in the attribute count — 20 `hasattr` calls. For most use cases negligible; for tight inner loops, prefer static checking.

```

from typing import Protocol, runtime_checkable

@runtime_checkable
class Vehicle(Protocol):
    wheels: int
    def drive(self) -> None: ...
    def stop(self) -> None: ...

# 3.12+ – inspect the runtime metadata
print(Vehicle.__protocol_attrs__)
# frozenset({'wheels', 'drive', 'stop'})

class Bike:
    wheels = 2
    def drive(self) -> None: print("pedal")
    def stop(self) -> None: print("brake")

class Skateboard:
    wheels = 4
    def drive(self) -> None: print("push")
    # missing stop

print(isinstance(Bike(), Vehicle))      # True – all attrs present
print(isinstance(Skateboard(), Vehicle)) # False – missing stop

```

Q29 · Protocols for `__call__` — Typing Callables With Keyword Arguments

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Microsoft India · Topic: Protocols

The question (interviewer's verbatim phrasing):

`Callable[[int], int]` can't express keyword-only arguments. How do I type "any callable with this exact signature including kwargs"?

The 30-second answer: Define a Protocol with a `__call__` method. `class Transformer(Protocol): def __call__(self, value: int, *, scale: float = 1.0) -> int: ...`. Any callable — function, lambda, callable class — with matching `__call__` signature satisfies the Protocol. This is how you express keyword-only args, defaults, and other shape details that `Callable[[...], R]` can't capture.

Step-by-step thought process: 1. "`Callable[[int, float], int]` is positional-only. You can't say 'one positional, one keyword-only.' Protocol with `__call__` fixes this — `def __call__(self, value: int, *, scale: float)` declares the signature naturally." 2. "At the use site — `def apply(transform: Transformer, x: int) -> int: return transform(x, scale=2.0)`. The caller passes any function with the right signature; the checker verifies." 3. "For default values —

defaults in the `__call__` signature are part of the Protocol. A function without the default doesn't satisfy: `def f(value: int, *, scale: float)` is a type error against `Transformer` because the default differs." 4. "Generic — Protocol with `__call__` can be generic. `class Reducer(Protocol[T]): def __call__(self, items: list[T], init: T) -> T: ...`. Now `Reducer[int]` is reducers over `int`." 5. "Overloaded — Protocol's `__call__` can be `@overload`-decorated. So a Protocol can describe a callable with multiple acceptable signatures, like the stdlib `sorted` Protocol."

Edge cases to mention: - A regular function with the right signature satisfies the Protocol — no need for the function to wrap in a class. `def double(value: int, *, scale: float = 1.0) -> int: return int(value * scale)` is a valid Transformer. - Lambdas satisfy too, but lambdas can't have keyword-only arguments syntactically. So Protocols with keyword-only requirements rule out lambdas as implementers — a feature, not a bug. - Bound methods satisfy — `obj.method` is a callable with the implicit self stripped. The Protocol's `call` doesn't include self either; alignment is correct.

What NOT to say: - " `Callable` can do keyword arguments with `Callable[[int, str], int, kwargs={'foo': int}]` ." No such syntax — `Callable` is positional-only. - "Protocol `call` requires the class to inherit from Protocol." Wrong — only the Protocol class inherits Protocol; the implementer is any callable with matching shape.

Common follow-up: "What if the callable has variadic kwargs?" — Protocol `__call__` with `**kwargs: int` works — declares kwarg type. For untyped kwargs use `**kwargs: Any` . Variadic positional similarly with `*args: int` .

```

from typing import Protocol, TypeVar

T = TypeVar('T')

class Transformer(Protocol):
    def __call__(self, value: int, *, scale: float = 1.0) -> int: ...

def apply(transform: Transformer, x: int) -> int:
    return transform(x, scale=2.0)

# Regular function – satisfies structurally
def double(value: int, *, scale: float = 1.0) -> int:
    return int(value * scale * 2)

# Callable class with __call__
class Multiplier:
    def __init__(self, factor: int) -> None:
        self.factor = factor
    def __call__(self, value: int, *, scale: float = 1.0) -> int:
        return int(value * self.factor * scale)

apply(double, 5)           # OK
apply(Multiplier(3), 5)    # OK
# apply(lambda v: v * 2, 5) # type error – wrong signature

# Generic callable Protocol
class Reducer(Protocol[T]):
    def __call__(self, items: list[T], init: T) -> T: ...

def reduce_one(r: Reducer[int], xs: list[int]) -> int:
    return r(xs, 0)

def sum_reduce(items: list[int], init: int) -> int:
    return sum(items) + init

reduce_one(sum_reduce, [1, 2, 3])    # 6

```

Q30 · Protocols in the Standard Library — Iterable, Sized, SupportsInt

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Microsoft India · Topic: Protocols

The question (interviewer's verbatim phrasing):

What stdlib Protocols should I know? Where do I find them?

The 30-second answer: The big ones live in `collections.abc` and `typing`. From `collections.abc` — `Iterable`, `Iterator`, `Sized`, `Container`, `Hashable`,

`Reversible`, `Collection`, `Sequence`, `Mapping`, `Set`. From `typing` — `SupportsInt`, `SupportsFloat`, `SupportsAbs`, `SupportsRound`, `SupportsBytes`, `SupportsComplex`. These are the official "duck typing shapes" — any class implementing the right dunder methods satisfies them, with `isinstance` checks built in.

Step-by-step thought process: 1. "`collections.abc` Protocols — `Iterable` (has `__iter__`), `Sized` (has `__len__`), `Container` (has `__contains__`), `Hashable` (has `__hash__`). These are ABCs but use `__subclasshook__` to recognize classes structurally — half ABC, half Protocol." 2. "`Sequence` / `Mapping` / `Set` are composite — `Sequence` requires `__getitem__` + `__len__` + `__iter__` + `__contains__` + `__reversed__` + `index` + `count`. `MutableSequence` adds `__setitem__`, `__delitem__`, `insert`, `append`, etc." 3. "`typing.Supports*` family — `SupportsInt` says 'has `__int__`', so any class convertible to `int` via `int(obj)`. Use as parameter types for utility functions: `def to_int(x: SupportsInt) -> int: return int(x)`." 4. "3.9+ — these are subscriptable: `Iterable[int]`, `Sequence[str]`, `Mapping[str, int]`. The parameterized form is the standard way to type collection inputs." 5. "When to prefer stdlib Protocols — for collection shapes, always. Writing a custom `class Iterable(Protocol)` would shadow the stdlib version and miss the runtime-check integration. Use the stdlib version."

Edge cases to mention: - `collections.abc.Iterable` is also a real ABC that classes can inherit from — but they don't need to. The `__subclasshook__` recognizes any class with `__iter__`. - `Mapping[K, V]` is the read-only protocol; `MutableMapping[K, V]` adds `__setitem__`, `__delitem__`, etc. Pick the narrower one as parameter type when you're not mutating. - `typing.Mapping` and `collections.abc.Mapping` are aliases in 3.9+; both subscriptable, both run-checkable. Pick consistently within your codebase.

What NOT to say: - "Stdlib doesn't have Protocols." It has many — `collections.abc` is the cabinet, `typing.Supports*` is the rest. - "`Iterable[int]` does runtime element-type checking." Wrong — the parameterization is for static checking only. Runtime `isinstance` checks just whether `__iter__` exists.

Common follow-up: "What's the difference between `list[int]` and `Sequence[int]` as a parameter type?" — `list[int]` requires an actual list. `Sequence[int]` accepts list, tuple, deque, custom sequence — anything with the Sequence Protocol. Use `Sequence` for read-only iteration; use `list` when you specifically need list operations like `.append`.

```

from collections.abc import Iterable, Sequence, Mapping, Sized
from typing import SupportsInt

def total(values: Iterable[int]) -> int:
    return sum(values)

total([1, 2, 3])           # OK – list is Iterable
total((1, 2, 3))           # OK – tuple is Iterable
total(range(4))            # OK – range is Iterable
total({1, 2, 3})           # OK – set is Iterable

def first_or_none(seq: Sequence[int]) -> int | None:
    return seq[0] if seq else None

# Set is NOT a Sequence – no indexing
# first_or_none({1, 2, 3})    # type error

def length_of(s: Sized) -> int:
    return len(s)

length_of([1, 2])          # 2
length_of("hello")         # 5
length_of({1, 2, 3})       # 3

# Supports* – for builtin conversions
def to_int_safely(x: SupportsInt) -> int:
    return int(x)

class Counter:
    def __init__(self, n: int) -> None: self.n = n
    def __int__(self) -> int: return self.n

to_int_safely(Counter(42))  # 42
to_int_safely(3.7)         # 3 – float has __int__

```

Section D — TypedDict + NamedTuple

For JSON shapes and lightweight records. TypedDict for dicts, NamedTuple for tuples-with-names. Both have evolved in 3.11/3.12 — `Required`, `NotRequired`, `total=False`, generic NamedTuple. Indian Backend rounds at FastAPI/Pydantic-heavy shops (Razorpay, Cred, Postman) ask about these.

Q31 · TypedDict — Typing Dict Shapes

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Microsoft India · Topic: TypedDict

The question (interviewer's verbatim phrasing):

When would I use TypedDict instead of a dataclass?

The 30-second answer: TypedDict types a *dict* with specific keys and value types — `class User(TypedDict): id: int; name: str`. The runtime object is a plain dict; the typing is static. Use it when you must work with dict-shaped data (JSON from APIs, kwargs forwarded as dict, legacy dict structures) and want type safety on the keys. Dataclass when you control the object representation and want methods, defaults, equality, immutability options.

Step-by-step thought process: 1. "TypedDict declaration — `class User(TypedDict): id: int; name: str`. Instantiate with dict syntax: `u: User = {'id': 1, 'name': 'abhi'}`. The runtime sees a regular dict; the checker sees a User." 2. "Versus dataclass — dataclass is a real class with `__init__`, `__repr__`, `__eq__`. Methods, properties, all the OOP machinery. TypedDict is just a dict with type information. No methods, no `__init__`, just shape." 3. "When TypedDict wins — JSON shapes (`json.loads` returns dicts), API requests (FastAPI takes dicts), kwargs forwarding (`def f(**kwargs: Unpack[User])` in 3.11+). Anywhere a dict is the natural representation." 4. "When dataclass wins — domain objects with behavior: `class Order: total: int; status: str; def is_paid(self) -> bool`. Methods belong on dataclasses; TypedDict's missing them is a feature for pure data shapes." 5. "Migration — JSON code often starts as TypedDict for typed access to incoming data, then converts to dataclass after validation. Two layers, two abstractions."

Edge cases to mention: - Functional syntax — `User = TypedDict('User', {'id': int, 'name': str})` works for keys that aren't valid Python identifiers (e.g., 'user-id' with hyphen). Less common but useful for APIs with kebab-case keys. - TypedDict inheritance — `class AdminUser(User): role: str`. The new class requires id, name, AND role. No diamond hierarchy concerns. - `total=False` on the class — makes all keys optional. `class PartialUser(TypedDict, total=False)` means id and name are not required. Half measures — usually you want Required/NotRequired (next Q) for finer control.

What NOT to say: - "TypedDict creates a special dict subclass." It doesn't — the runtime object is `dict`, not a subclass. The typing is purely static. - "TypedDict supports methods." It can't — methods on a TypedDict declaration are class-level and don't apply to the dict instances.

Common follow-up: "What happens at runtime if I assign a dict that doesn't match the TypedDict shape?" — Nothing. The runtime sees a regular dict. The mismatch surfaces only when the static checker runs. For runtime validation, use Pydantic or msgspec.

```

from typing import TypedDict

# Class-based syntax – preferred
class User(TypedDict):
    id: int
    name: str
    email: str

# Instantiate as dict
u: User = {"id": 1, "name": "abhi", "email": "abhi@x.in"}

# Inheritance
class AdminUser(User):
    role: str

admin: AdminUser = {"id": 1, "name": "abhi", "email": "a@x.in", "role": "admin"}

# Functional syntax for special keys
ApiResponse = TypedDict('ApiResponse', {
    'status-code': int,
    'body': str,
})
resp: ApiResponse = {"status-code": 200, "body": "ok"}

# Versus dataclass
from dataclasses import dataclass
@dataclass
class UserClass:
    id: int
    name: str
    email: str
    def initials(self) -> str:
        return "".join(part[0] for part in self.name.split())

uc = UserClass(1, "abhi mittal", "a@x.in")
uc.initials() # 'am' – methods only on dataclass

```

Q32 · Required and NotRequired (PEP 655, 3.11+)

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred · Topic: TypedDict

The question (interviewer's verbatim phrasing):

How do I make some keys of a TypedDict optional but others required?

The 30-second answer: PEP 655 (3.11+) added `Required[X]` and `NotRequired[X]` for per-field control. `class User(TypedDict): id: Required[int]; nickname: NotRequired[str]`.

Pre-3.11 you had `total=True` (all keys required, the default) or `total=False` (all optional) — coarse-grained. Now you can mix freely.

Step-by-step thought process: 1. "Default — `class User(TypedDict)` makes every key required. Pass a dict missing 'id' and the checker flags it." 2. " `total=False` — every key becomes optional. Useful when you have a `partial` form of a record. But mixing required and optional fields needed inheritance gymnastics pre-3.11." 3. "3.11+ Required/NotRequired — per-key control. `class User(TypedDict): id: Required[int]; nick: NotRequired[str]`. The default (`total=True`) plus `NotRequired` on nick makes nick optional while keeping id required." 4. "Combined with `total=False` — `class PartialUser(TypedDict, total=False): id: Required[int]; name: str`. All keys optional EXCEPT id, which is forced required. Inverts the default per key." 5. "Runtime — these markers don't affect the runtime dict. They're advisory to checkers. So `User(id=1)` works even though `nick` is missing — and the checker is happy because nick is `NotRequired`."

Edge cases to mention: - Use `from typing import Required, NotRequired` (3.11+) or `from typing_extensions import Required, NotRequired` for older Python. Both import paths resolve to the same symbols. - `Required[X]` applied to an already-required key is a no-op but still valid. Useful in code generators that emit `Required` uniformly. - Nested TypedDicts can have their own required/optional structure. `class A(TypedDict): b: NotRequired[B]` where `class B(TypedDict): val: int`.

What NOT to say: - "Required and NotRequired affect runtime behavior." They don't — the runtime dict is unaffected. - "Pre-3.11 you couldn't mix required and optional keys." You could, via inheritance — `class Base(TypedDict, total=True): id: int` and `class Full(Base, total=False): nick: str`. Required/NotRequired just makes it cleaner.

Common follow-up: "How do you check at runtime if a TypedDict key is set?" — Regular dict operations: `'nick' in u` or `u.get('nick')`. TypedDict doesn't change dict semantics, only types.

```

from typing import TypedDict, Required, NotRequired

# Default — all required
class User(TypedDict):
    id: Required[int]
    name: Required[str]
    nickname: NotRequired[str] # optional

u1: User = {"id": 1, "name": "abhi"} # OK — nickname missing fine
u2: User = {"id": 1, "name": "abhi", "nickname": "a"} # OK
# u3: User = {"name": "abhi"} # type error — id missing

# total=False with per-key Required
class PartialUser(TypedDict, total=False):
    id: Required[int] # required despite total=False
    name: str # optional now
    nickname: str # optional

p1: PartialUser = {"id": 1} # OK
p2: PartialUser = {"id": 1, "name": "abhi"} # OK
# p3: PartialUser = {"name": "abhi"} # type error — id missing

# Nested
class Address(TypedDict):
    city: str
    pincode: int

class Profile(TypedDict):
    user: User
    address: NotRequired[Address] # whole address is optional

```

Q33 · `typing.NamedTuple` vs `collections.namedtuple`

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, TCS, Infosys, JP Morgan · Topic: NamedTuple

The question (interviewer's verbatim phrasing):

What's the difference between `typing.NamedTuple` and `collections.namedtuple` ?
Which should I use?

The 30-second answer: `typing.NamedTuple` is the class-based, typed form — `class Point(NamedTuple): x: int; y: int`. `collections.namedtuple` is the older factory function — `Point = namedtuple('Point', ['x', 'y'])` with no types. Use `typing.NamedTuple` always in new code; same runtime behavior, plus type hints and IDE completion. The `collections` form is legacy stdlib that pre-dates typing.

Step-by-step thought process: 1. "Both produce the same kind of object — an immutable tuple subclass with named fields. Equality, hashability, iteration all work like a tuple. `Point(1, 2) == (1, 2)` is True." 2. " `typing.NamedTuple` form — `class Point(NamedTuple): x: int; y: int` . Class syntax means type hints, defaults, and docstrings all work naturally. mypy and pyright pick up the types." 3. " `collections.namedtuple` form — `Point = namedtuple('Point', ['x', 'y'])` . No types. You can add defaults via the `defaults` parameter, but it's positional and awkward." 4. "When to prefer `typing.NamedTuple` — every time. It's the typed, modern way. The collections form survives only for backward compat and quick one-liners in scripts." 5. "When to prefer dataclass over either — when you want mutability (`frozen=False`), methods beyond what `NamedTuple` offers (`NamedTuple` supports methods but it's idiomatic to keep them small), or non-tuple semantics (no positional unpacking). Dataclasses are the general-purpose record type; `NamedTuple` is for tuple-shaped immutable records."

Edge cases to mention: - `NamedTuples` can have methods — `class Point(NamedTuple): x: int; y: int; def magnitude(self): return (self.x**2 + self.y**2)**0.5` . Common for small geometric or value types. - Defaults — `class Point(NamedTuple): x: int = 0; y: int = 0` works in both forms (`typing.NamedTuple` supports it cleanly). - Generic `NamedTuple` — `class Pair[T](NamedTuple): first: T; second: T` works in 3.11+ with PEP 695 sugar; pre-3.11 you needed dance to combine `NamedTuple` with Generic.

What NOT to say: - " `collections.namedtuple` is deprecated." It's not — still works, still in stdlib. Just not what you'd reach for in new code. - "NamedTuples are slow." They're tuples — fast indexing, immutable, hashable. Often faster than dataclasses for read-heavy workloads.

Common follow-up: "When would you use `NamedTuple` over a frozen dataclass?" — When the value needs to be a tuple — for `dict` keys, set membership where hash matters, positional unpacking (`x, y = point`), comparison-as-tuple semantics. Frozen dataclass for value-objects where you don't need tuple-ness.

```

from typing import NamedTuple
from collections import namedtuple

# typing.NamedTuple – class syntax, typed
class Point(NamedTuple):
    x: int
    y: int
    label: str = ""          # default

    def magnitude(self) -> float:
        return (self.x ** 2 + self.y ** 2) ** 0.5

p = Point(3, 4)
print(p.x, p.y)              # 3 4
print(p.magnitude())         # 5.0
print(p == (3, 4, ""))      # True – tuple equality

# collections.namedtuple – older factory form, no types
PointLegacy = namedtuple('PointLegacy', ['x', 'y'], defaults=[0])
pl = PointLegacy(3, 4)
print(pl.x, pl.y)            # 3 4
# No type hints – IDE doesn't know p.x is int

# Generic NamedTuple – 3.11+ with PEP 695 syntax
class Pair[T](NamedTuple):
    first: T
    second: T

ints: Pair[int] = Pair(1, 2)
strs: Pair[str] = Pair("a", "b")

```

Q34 · NamedTuple vs Dataclass vs TypedDict — The Right Tool

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Records

The question (interviewer's verbatim phrasing):

When would you reach for NamedTuple, dataclass, or TypedDict? Walk me through the tradeoffs.

The 30-second answer: NamedTuple for immutable tuple-shaped value records — small, hashable, supports unpacking. Dataclass for general-purpose objects with methods, mutability options, and rich behavior — the default record type. TypedDict for dict-shaped data crossing API boundaries — JSON, kwargs forwarding, legacy dict structures. Three different problems; pick the one whose representation matches your runtime needs.

Step-by-step thought process: 1. "Runtime shape — the deciding factor. `NamedTuple` is a tuple subclass; you can index it, unpack it, hash it. `Dataclass` is a regular class; methods, properties, mutability optional. `TypedDict` is literally a dict; serializable as JSON without conversion." 2. "`NamedTuple` wins when — small (2-5 fields), immutable, used as dict keys or set members, positional unpacking matters. Geometric types, RGB triples, return-type bundling." 3. "`Dataclass` wins when — methods needed, mutability sometimes useful, the object is a domain entity with behavior. Most application classes. The default choice for 'a record with structure.'" 4. "`TypedDict` wins when — the data crosses API or storage boundaries as dicts. JSON in, JSON out, no conversion. `Pydantic` generates one-way validators from these; `FastAPI` accepts them as request bodies." 5. "Performance ordering — `NamedTuple` (smallest, fastest), then `__slots__`-decorated `dataclass`, then `dataclass`, then `TypedDict` (just a dict). For most code this doesn't matter; for hot paths it does."

Edge cases to mention: - A frozen `dataclass` with `__slots__` (`@dataclass(frozen=True, slots=True)`) gives you a hashable, immutable, fast record — competitive with `NamedTuple` for many uses. Lacks the tuple-ness. - `TypedDict` can be combined with `Required` / `NotRequired` for partial shapes that match many API patterns; `dataclass` with defaults is more rigid because the field always exists. - For function return types of multiple values — `NamedTuple` is conventional (`return Result(success, errors)`). Tuple unpacking works at the call site; readable code.

What NOT to say: - "Use whichever you like; they're interchangeable." They're not — runtime semantics differ significantly. - "Dataclasses are always slower than `NamedTuples`." Often true but not relevant for non-hot paths. Decision should be readability and semantics first.

Common follow-up: "For a JSON API response shape, which would you pick?" — `TypedDict` on the way in (JSON → typed dict for static checking) and either `dataclass` or `Pydantic` model on the way out (your service layer's typed result). Different concerns at different layers.

```

from typing import NamedTuple, TypedDict
from dataclasses import dataclass

# NamedTuple – small immutable, hashable, indexable
class Point(NamedTuple):
    x: int
    y: int

p1 = Point(1, 2)
x, y = p1                # unpack
{p1: "origin"}           # hashable

# Dataclass – full-featured record with methods
@dataclass
class Order:
    id: int
    items: list[str]
    total: float = 0.0
    def add_item(self, name: str) -> None:
        self.items.append(name)

o = Order(1, [])
o.add_item("book")        # mutable, methods

# Frozen + slots – immutable record, fast
@dataclass(frozen=True, slots=True)
class Money:
    amount: int
    currency: str

m = Money(100, "INR")
# m.amount = 200          # FrozenInstanceError

# TypedDict – dict shape
class UserDict(TypedDict):
    id: int
    name: str

u: UserDict = {"id": 1, "name": "abhi"}
import json
print(json.dumps(u))      # serializes directly – it IS a dict

```

Q35 · TypedDict with **Unpack** for ****kwargs** (3.12+)

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Microsoft India · Topic: TypedDict

The question (interviewer's verbatim phrasing):

How do I type a function that takes `**kwargs` where the kwargs have specific names and types?

The 30-second answer: PEP 692 (3.12+) introduced `Unpack[SomeTypedDict]` for `**kwargs`. Declare a TypedDict for the kwargs shape — `class UpdateKwargs(TypedDict, total=False): name: str; email: str` — then annotate the function parameter as `def update(**kwargs: Unpack[UpdateKwargs])`. The checker treats each kwarg as the corresponding TypedDict field, validating presence and types.

Step-by-step thought process: 1. "Pre-3.12 — `**kwargs: int` meant 'all keyword values are ints' (uniform). You couldn't say 'name is str, email is str, age is int' as kwargs." 2. "3.12 with PEP 692 — declare a TypedDict for the kwargs shape, then `def f(**kwargs: Unpack[Kwargs])`. The checker now treats kwargs as having those specific keys with those specific types." 3. "TypedDict matters — use `total=False` for all-optional kwargs (typical), `Required/NotRequired` for mixed. The TypedDict structure controls which kwargs are required." 4. "At the call site — `f(name='abhi', email='a@x.in')`. The checker verifies each kwarg matches an entry in the TypedDict. Unknown kwargs are errors; missing required kwargs are errors." 5. "Forwarding to another function — perfect with `ParamSpec` or with another `Unpack[SameKwargs]`. Especially useful for thin wrapper functions and decorators that transform the call."

Edge cases to mention: - `Unpack` is from `typing` in 3.12+; from `typing_extensions` for earlier versions. Same symbol. - Mixing positional and `Unpack` kwargs — `def f(user_id: int, **kwargs: Unpack[Kwargs])`. The positional comes first; kwargs has the TypedDict shape. - Kwargs with the same name as a positional parameter — the checker treats this as an error in 3.12. The TypedDict can't include the positional names.

What NOT to say: - "`**kwargs: dict[str, int]`." Wrong syntax — kwargs typing is for the value type, and TypedDict shapes are the 3.12 way. - "Unpack only works with NamedTuple." No — it's for TypedDict in kwargs, and `TypeVarTuple` in generics. Different shapes.

Common follow-up: "What if my kwargs include a field whose Python name is a reserved word, like `from`?" — Use the functional TypedDict form — `Kwargs = TypedDict('Kwargs', {'from': str, 'to': str})`. Then `f(**{'from': 'a@x.in', 'to': 'b@x.in'})`. Static checker is happy.

```

from typing import TypedDict, Unpack, NotRequired

class UpdateKwargs(TypedDict, total=False):
    name: str
    email: str
    age: int

def update_user(user_id: int, **kwargs: Unpack[UpdateKwargs]) -> None:
    print(f"updating {user_id} with {kwargs}")

update_user(1, name="abhi", email="a@x.in")    # OK
update_user(1, age=30)                        # OK – all kwargs optional
# update_user(1, address="...")                # type error – not in TypedDict
# update_user(1, name=42)                      # type error – wrong type

# Required + optional mix
class CreateKwargs(TypedDict):
    name: str                                # required
    email: str                              # required
    nickname: NotRequired[str]              # optional

def create_user(**kwargs: Unpack[CreateKwargs]) -> int:
    return 0                                # placeholder

create_user(name="abhi", email="a@x.in")      # OK
create_user(name="abhi", email="a@x.in", nickname="a") # OK
# create_user(name="abhi")                    # type error – email missing

# Forwarding
def wrapper(user_id: int, **kwargs: Unpack[UpdateKwargs]) -> None:
    update_user(user_id, **kwargs)           # signature carries through

```

Section E — Static checkers + runtime gap

The five questions that separate "I added type hints" from "I run mypy in CI". The Indian senior interviews — Razorpay, Microsoft GCC, Goldman, Postman — will ask at least one of these.

Q36 · Mypy vs Pyright — How They Differ in Practice

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Static checkers

The question (interviewer's verbatim phrasing):

What's the difference between mypy and pyright? Why would I pick one over the other?

The 30-second answer: Both are static type checkers. Mypy (PSF, written in Python) is the original and the reference for PEP semantics — slower, conservative, plugin-friendly. Pyright (Microsoft, written in TypeScript) is faster, has stricter defaults, and pairs with Pylance for VS Code IDE integration. Most Indian shops in 2026 use pyright for IDE feedback and either mypy or pyright in CI — pick based on speed (pyright wins) and ecosystem (some libraries ship mypy plugins).

Step-by-step thought process: 1. "Mypy — the original PEP-484 reference checker. Written in Python; runs as `mypy src/`. Has a plugin system that libraries (SQLAlchemy 2, Django via `django-stubs`, Pydantic) use to teach mypy about their dynamic patterns." 2. "Pyright — Microsoft's faster check. Written in TypeScript, runs as `pyright src/`. Strict mode (`'strict'`) is more aggressive than mypy's strict mode; many shops use pyright strict for new code and mypy normal for legacy." 3. "Speed — pyright is 5-10x faster on large codebases due to incremental analysis and parallelism. For a million-line codebase, mypy can take minutes; pyright finishes in seconds." 4. "Plugin gap — mypy has plugins (`django-stubs`, `pydantic mypy plugin`, `sqlalchemy mypy plugin`). Pyright relies on stub files and inline type comments. For Pydantic v2, both work well; for SQLAlchemy ORMs, mypy's plugin is sometimes needed." 5. "IDE story — pyright IS Pylance underneath. Anyone using VS Code with Pylance is already running pyright continuously. Mypy in the IDE works but is slower."

Edge cases to mention: - Mypy and pyright disagree on edge cases occasionally. Pyright's strict mode rejects more code than mypy strict. Standardize on one checker for CI; let the other run in the IDE for second-opinion. - Type comments (`# type: int`) are legacy mypy syntax for Python 2 / early Python 3. Both checkers support them; neither needs them in modern code with PEP 526 annotation syntax. - mypy's `--strict` is a bundle of strict flags; pyright's strict mode is also a bundle. The exact flag sets differ; check each tool's docs for what strict includes.

What NOT to say: - "mypy is the official one, pyright is unofficial." Both are widely adopted; pyright IS Microsoft's official typing tooling. - "You can run both in CI for double-coverage." Possible but slow and produces conflicting messages. Pick one.

Common follow-up: "What's the migration path for adding mypy/pyright to an existing untyped codebase?" — Run with `--ignore-missing-imports` first, then `--check-untyped-defs`, then `--disallow-untyped-defs` module by module. Both checkers support gradual mode where new code is strict but old code is grandfathered.

```
# Typical mypy.ini
"""
[mypy]
python_version = 3.12
strict = True
plugins = pydantic.mypy
ignore_missing_imports = True
"""

# Typical pyrightconfig.json
"""
{
    "pythonVersion": "3.12",
    "typeCheckingMode": "strict",
    "useLibraryCodeForTypes": true,
    "reportMissingTypeStubs": false
}
"""

# Same code, both checkers see types
def total(items: list[int]) -> int:
    return sum(items)

# Both flag this
# total(["a", "b"]) # error: List item 0 has incompatible type "str"

# Pyright strict catches: unused imports, implicit Any in lambdas,
# missing return statements; mypy strict catches: untyped definitions,
# implicit Optional in defaults.
```

Q37 · `cast()` — Telling the Checker You Know Better

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Static checkers

The question (interviewer's verbatim phrasing):

What does `typing.cast` do? When should I use it?

The 30-second answer: `cast(T, value)` tells the static checker to treat `value` as type `T` without any runtime check. The runtime returns `value` unchanged — `cast` is identity at runtime. Use it at boundaries where you have more knowledge than the checker — JSON deserialization, ORM query results, dynamic dispatch — and you're sealing in a typed view. Use sparingly; every cast is a trust assertion the checker can't verify.

Step-by-step thought process: 1. "Signature — `cast(T, value: Any) -> T`. Two arguments; returns the value typed as `T`. At runtime it's literally `def cast(t, v): return v`. The first argument

is a hint only." 2. "Use cases — `result = cast(User, db.query(...))` when the query returns `Any` but you know it's a `User`. `data = cast(UserDict, json.loads(s))` when JSON gives `Any`. Anywhere the checker has too little information and you have more." 3. "What it does NOT do — runtime validation. `cast(int, 'not an int')` happily returns the string. The checker is satisfied; the runtime is broken. If you need runtime validation, use Pydantic or `isinstance` checks." 4. "Where it's a code smell — inside business logic. `cast` should live at boundaries. If you're casting in the middle of your function, the upstream types are probably wrong; fix those instead." 5. "Alternatives — `typing.assert_type(value, T)` (3.11+) is for asserting the checker already infers `T`, no `cast`; `isinstance` with narrowing teaches the checker without casting. Reach for `cast` only when neither alternative applies."

Edge cases to mention: - `cast` doesn't narrow type variables in generic contexts. `cast(T, value)` where `T` is unbound is a no-op for the checker. - `cast` works with Generics — `cast(list[User], rows)` is fine. The runtime returns `rows` unchanged. - Combined with `Union` — `cast(int, x)` where `x` is `int | str` tells the checker "trust me, it's the int side." Use `isinstance` narrowing first if you can.

What NOT to say: - "`cast` converts the value to type `T`." Wrong — it's a static hint, no runtime conversion happens. - "Use `cast` liberally to silence type errors." Defeats the point — `cast` should be a deliberate boundary act, not error suppression.

Common follow-up: "What's the difference between `cast(int, x)` and `int(x)`?" — `int(x)` is a real runtime conversion: it parses or converts the value to `int`. `cast(int, x)` just tells the type checker "treat this as `int`" — no parsing, no conversion. Different tools for different problems.

```

from typing import cast, Any
import json

# Common boundary – JSON deserialization
def load_user(s: str) -> dict:
    raw: Any = json.loads(s)          # json.loads returns Any
    return cast(dict, raw)            # we know it's a dict

# Database row – ORM returns generic Row
def fetch_user(user_id: int) -> dict:
    row = db_query(f"SELECT * FROM users WHERE id = {user_id}")  # Any
    if not row:
        raise ValueError
    return cast(dict, row)

# Anti-pattern – cast in business logic
def process(x: int | str) -> int:
    # Wrong – cast hides the failure
    y = cast(int, x)
    return y * 2

# Right – narrow with isinstance
def process_right(x: int | str) -> int:
    if not isinstance(x, int):
        raise TypeError
    return x * 2                                # checker knows x is int here

# Cast with generics
from typing import TypeVar
T = TypeVar('T')
def fetch_any() -> Any: return []
rows = cast(list[dict], fetch_any())

def db_query(q): return None

```

Q38 · `assert_type` and `reveal_type` — Debugging the Checker

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Microsoft India · Topic: Static checkers

The question (interviewer's verbatim phrasing):

What does `reveal_type(x)` do and how is it different from `assert_type` ?

The 30-second answer: `reveal_type(x)` is a debugging tool — when the checker runs, it prints what type it thinks `x` is. It's not a real function; at runtime it does nothing (mypy used to make it error if you tried to import). `assert_type(x, T)` (3.11+) is an *assertion* that the checker's inferred type for `x`

matches `T`; if it doesn't, the check fails. Use `reveal_type` to explore; `assert_type` in tests to lock in what the checker should see.

Step-by-step thought process: 1. "`reveal_type(x)` — debugging only. Run `mypy` or `pyright`; you'll see `Revealed type is 'int | None'` printed. At runtime, the checker pretends the function doesn't exist; in older `mypy` you literally couldn't import it from anywhere." 2.

"`assert_type(x, int | None)` — 3.11+. Tells the checker 'I claim `x` is `int | None`; verify.' If the checker disagrees, type error. At runtime it's a no-op — returns `x` unchanged." 3. "Difference in intent — `reveal_type` is exploratory ('what does the checker think?'); `assert_type` is regression-protective ('the checker should think this'). Use `assert_type` in unit tests for typing-critical code to catch checker regressions." 4. "Practical — write a tricky function with overloads or narrowing. After writing, add `assert_type` at expected sites. Now refactoring that breaks the type signatures fails the type check, not just the runtime tests." 5. "Pre-3.11 alternative — `cast(int, x)` plus `reveal_type` was the dance. `assert_type` makes the intent declarative."

Edge cases to mention: - `reveal_type` can't be imported — it's a magic builtin recognized by the checker. `Mypy` and `pyright` both handle it without import. - `assert_type` does NOT enforce subtypes — `assert_type(my_dog, Animal)` fails if the checker infers the more specific `Dog`. For "the type is at least `Animal`," use `cast` or `isinstance`. - Both functions are imported from `typing`: `from typing import assert_type, reveal_type` works in 3.11+. Pre-3.11 you'd use `typing_extensions.assert_type`.

What NOT to say: - "reveal_type is a runtime tool." No — it's a static-analysis-time mechanism; the runtime does nothing meaningful. - "assert_type does runtime checks." No — it's static-only. For runtime, use `assert isinstance(x, T)`.

Common follow-up: "How would you write a test that ensures my function signature stays generic?" — Use `assert_type` in a test file. `def test_signature(): assert_type(my_func([1, 2, 3]), list[int])`. If a refactor breaks the genericity, the type checker catches it before the test runs.

```

from typing import assert_type, reveal_type

def find(items: list[int], target: int) -> int | None:
    for i, x in enumerate(items):
        if x == target:
            return i
    return None

# Debugging – what does the checker see?
result = find([1, 2, 3], 2)
reveal_type(result)           # mypy prints: Revealed type is "int | None"

# Pin the inference – fails if the checker disagrees
assert_type(result, int | None) # OK
# assert_type(result, int)      # type error – int | None is what the checker sees

# Test-style signature pinning
from typing import TypeVar
T = TypeVar('T')

def first(items: list[T]) -> T:
    return items[0]

assert_type(first([1, 2]), int)    # OK – generic instantiated to int
assert_type(first(["a", "b"]), str) # OK
# assert_type(first([1, 2]), str)  # type error

```

Q39 · TYPE_CHECKING — Handling Circular Imports for Types

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: Static checkers

The question (interviewer's verbatim phrasing):

I have a circular import because module A imports module B for a type hint, and B imports A for the same reason. How do I fix it?

The 30-second answer: Use `if TYPE_CHECKING:` from `typing`. The block is `True` at type-check time, `False` at runtime. Put the circular-causing imports inside the block; use string forward references (or `from __future__ import annotations`) for the annotations. Now the runtime doesn't execute the imports — no circular error — and the checker still sees them.

Step-by-step thought process: 1. "The problem — module A has `def f(b: B)` where B is in module B; module B has `def g(a: A)` where A is in module A. Both modules try to import the other at module load — circular import error or one of them is half-initialized." 2. "`TYPE_CHECKING` is a constant from `typing`. At runtime it's `False`. At static-analysis time the checker treats it as `True`."

So `if TYPE_CHECKING: from b import B` runs the import only for the checker." 3. "Annotations must then be strings (forward refs) or use `from __future__ import annotations` (PEP 563). The runtime never resolves the type; the checker resolves via the conditional import." 4. "Practical — almost every Django, FastAPI, or SQLAlchemy app has this pattern. Service-layer modules need types from the model layer; model modules need types from the service layer. `TYPE_CHECKING` is the standard escape." 5. " `from __future__ import annotations` is the bigger hammer — makes ALL annotations strings at runtime. Pairs naturally with `TYPE_CHECKING`: imports stay conditional, annotations stay strings, runtime is clean."

Edge cases to mention: - Without `from __future__ import annotations`, you must write annotations as strings yourself: `def f(b: 'B')`. Easy to forget; future annotations is cleaner. - `typing.get_type_hints()` will fail at runtime if the type isn't importable — by design, since `TYPE_CHECKING`-gated imports aren't available. For runtime introspection you need to either eagerly import or pass `globals` / `locals` dicts. - Dataclass with `from __future__ import annotations` — fields are still typed correctly, but `dataclasses.fields()` returns string types, not real types. Mostly fine; matters for libraries that introspect.

What NOT to say: - "TYPE_CHECKING is True at runtime sometimes." It's never True at runtime. It's `False` always at runtime; the checker pretends it's True. - "Circular imports are always a code-smell." Sometimes they're inherent to the design (mutual recursion between modules). `TYPE_CHECKING` is the modern fix.

Common follow-up: "How does pydantic v2 handle `TYPE_CHECKING`-only imports for forward references?" — Pydantic eagerly resolves forward references when you call `model.model_rebuild()`. So you keep `TYPE_CHECKING` for static, then call `rebuild` after both modules have loaded.

```
# Module: orders.py
from __future__ import annotations
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from customers import Customer    # circular at runtime, OK at check time

class Order:
    def __init__(self, customer: Customer, total: float) -> None:
        self.customer = customer
        self.total = total

    def buyer_label(self) -> str:
        return self.customer.name    # checker knows Customer

# Module: customers.py
from __future__ import annotations
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from orders import Order

class Customer:
    def __init__(self, name: str) -> None:
        self.name = name
        self.orders: list[Order] = [] # forward ref via TYPE_CHECKING + future

    def total_spent(self) -> float:
        return sum(o.total for o in self.orders)
```

Q40 · `from __future__ import annotations` — Why It Helps

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India, JP Morgan · Topic: Static checkers

The question (interviewer's verbatim phrasing):

What does `from __future__ import annotations` do, and why does mypy documentation recommend it?

The 30-second answer: PEP 563 (3.7+) — `from __future__ import annotations` makes all annotations in the module evaluate as strings at runtime, not actual type objects. So `def f(x: int) -> int` stores `__annotations__['x'] = 'int'` not `int`. Three wins: no runtime import cost for type-only imports, no `NameError` on forward references, modern syntax (`list[int]` , `int | None`) works on older Python versions where the runtime would otherwise reject it.

Step-by-step thought process: 1. "Default Python behavior — annotations are evaluated at function definition time. `def f(x: List[int])` requires `List` to be importable at that moment. This costs imports, breaks on forward references, and blocks modern syntax on older Python." 2. "With `from __future__ import annotations` — all annotations become strings, stored as strings in `__annotations__`. The actual import only matters when something calls `typing.get_type_hints()` to resolve." 3. "Three concrete wins. First — `list[int]` works on 3.7+ even though pre-3.9 the runtime couldn't subscript `list`. With future-annotations, the syntax is a string, never evaluated. Second — forward references — `def f(x: 'OtherClass')` no longer needs the quotes; the whole annotation is a string. Third — `TYPE_CHECKING` imports become safe everywhere." 4. "Runtime introspection — `typing.get_type_hints(f)` evaluates the strings and returns real types. Libraries that introspect (Pydantic, dataclasses) call this when needed. The cost moves from import-time to introspection-time." 5. "3.13 status — PEP 649 (deferred evaluation of annotations) was approved as the long-term direction, lazy-evaluating annotations natively. `from __future__ import annotations` remains the explicit opt-in until PEP 649's behavior becomes default (currently slated for 3.14)."

Edge cases to mention: - `from __future__ import annotations` doesn't help dataclasses or Pydantic with `frozen` defaults — they still need to resolve types to set up `__init__`. They call `get_type_hints` internally. - Module-level annotations (`x: int = 5`) — same effect; `__annotations__['x']` becomes the string `'int'`. - runtime-checkable Protocols still work — the `isinstance` check uses `__protocol_attrs__` which doesn't depend on annotation evaluation.

What NOT to say: - "It speeds up runtime by 10x." Marginal in practice; the main wins are import-cost reduction and forward-reference handling. - "It's deprecated in 3.13." Not deprecated — it's the explicit opt-in for PEP 563 semantics. PEP 649 introduces lazy evaluation as the new default; 563 stays as one option.

Common follow-up: "What's the difference between PEP 563 and PEP 649?" — PEP 563 makes annotations into strings. PEP 649 makes annotations lazy-evaluated (real types, but constructed on demand). 649 is better for tools that introspect; both achieve the import-cost reduction. PEP 649 will become default in 3.14.

```

# Without future annotations – pre-3.9 issue
# from typing import List
# def f(x: List[int]) -> List[int]: # works
#     return x
# def g(x: list[int]) -> list[int]: # TypeError on 3.7/3.8 at runtime
#     return x

# With future annotations – works on 3.7+
from __future__ import annotations

def g(x: list[int]) -> list[int]:    # string at runtime, parsed by checker
    return x

# Forward refs work without quotes
class Node:
    def add_child(self, child: Node) -> None: # was 'Node' before; now plain
        ...

# Runtime introspection
import typing
def h(x: int, y: str) -> bool:
    return True

print(h.__annotations__)
# {'x': 'int', 'y': 'str', 'return': 'bool'} – strings
print(typing.get_type_hints(h))
# {'x': <class 'int'>, 'y': <class 'str'>, 'return': <class 'bool'>} – resolved

# Pairs naturally with TYPE_CHECKING
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from expensive_module import HeavyType

def process(item: HeavyType) -> None:    # never imported at runtime
    ...

```

End of Chapter 9

Forty questions through the typing system as it stands in 2026: basics (annotations, runtime no-op, Optional / Union / Any / Final / Callable / generic containers / Literal / Annotated), generics (TypeVar legacy, PEP 695 bracket syntax, bounded and constrained vars, ParamSpec, Concatenate, TypeVarTuple, Self, variance), Protocols (structural vs nominal, definition, runtime_checkable, vs ABC, generic, intersection, duck-typing-with-types, **protocol_attrs**, **call**, stdlib Protocols), TypedDict and NamedTuple (basic TypedDict, Required / NotRequired, NamedTuple comparison, dataclass/NamedTuple/TypedDict tradeoff, Unpack with kwargs), and the static-runtime gap (mypy vs pyright, cast, assert_type, reveal_type, TYPE_CHECKING, from **future** import annotations).

The connecting line through all forty — type hints don't run, but they shape every other part of how 2026 Python codebases stay correct as they grow. The Indian senior interviews this year — Razorpay, Postman, Cred, Microsoft GCC, Goldman — will probe at least four of these questions back-to-back. Knowing which subset of typing to reach for, and why each form exists, is what separates the candidates who "use FastAPI" from the ones who design libraries with it.

Next chapter (Chapter 10) covers modules and packaging — `__init__.py`, namespace packages, `sys.path`, circular imports.

Chapter 10 — Modules & Packaging (40 Questions)

Core Python Library · Python Interview Prep — 2026 Edition · Learn & Excel

This is the chapter most experienced devs skip and then fumble on. Writing Python every day, you `pip install`, you `import`, you ship — but the interview tests whether you actually understand what happens when `import foo` runs, why `__pycache__` exists, why the same `from .a import b` works in one file and explodes in another, and what `pyproject.toml` is doing under the hood. Forty questions split across the five sub-areas: the import system itself, packages and `__init__.py`, the circular-import surgery, `pyproject.toml` and modern PEP-517/518/660 packaging, and the `venv` / `pip` / `pipx` / `uv` landscape that has changed sharply in the 2024-2026 window. Each q follows the standard step-by-step answer format — tags, the interviewer's phrasing, the 30-second answer, the step-by-step you say out loud, edge cases, what NOT to say, and the common follow-up. Default version assumption is Python 3.12 unless a question explicitly tags a 3.11 or 3.13 difference.

Section A — Import System Fundamentals (Q1–Q10)

Ten questions on what `import` actually does. The interview tells, almost without exception, by whether the candidate can explain `sys.modules`, `sys.path`, and the difference between binding a name versus executing a module. If you can say "import runs the module top-level code exactly once and caches the resulting module object in `sys.modules`," the rest of this section answers itself.

Q1 · `import x` versus `from x import y`

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Postman · Topic: Import System

The question (interviewer's verbatim phrasing):

What's the actual difference between `import os` and `from os import path` ? Both work — why have two?

The 30-second answer: Both forms execute the target module exactly once and store the resulting module object in `sys.modules`. The only difference is what name lands in the current namespace. `import os` binds the name `os` to the module. `from os import path` runs the same module, then binds the name `path` to the attribute `os.path` in the current namespace. The module is cached either way; nothing gets re-executed.

Step-by-step thought process: 1. "At runtime both statements do the same loading work — they find the module, run its top-level code if not already in `sys.modules`, and put the result in the cache. The behaviour difference is purely a local-namespace decision." 2. "`import os` lets me write `os.path.join(...)`. The `os` name is a reference to the module object. If something later rebinds `os.path` — `os.path = MyFakePath()` — my code picks up the change because each access goes through the module." 3. "`from os import path` copies the current value of `os.path` into a local name `path`. If someone later rebinds `os.path`, my local `path` still points to the original. That makes the form slightly faster on tight loops (one less attribute lookup) but also slightly less monkeypatch-friendly." 4. "The cosmetic difference matters more in practice. `from x import y` is direct and reads well — `from collections import Counter` is how it's done. `import x` keeps a namespace boundary that helps when names would clash — `from logging import config` would shadow the stdlib `config` you might also want, so people write `import logging` and then `logging.config`." 5. "For test mocking, `from x import y` is the gotcha. `mock.patch('x.y')` does not patch the local `y` in the consuming module — you have to patch `consuming_module.y`. This trips up roughly every Python developer once."

Edge cases to mention: - `from x import *` runs `x` and copies in either everything in `__all__` if defined, otherwise every name not starting with an underscore. It does NOT chain — `from x import *` inside a function is a `SyntaxError` in 3.x. - `import x.y.z` runs all three modules (`x`, `x.y`, `x.y.z`) in order, but only `x` is bound locally. You write `x.y.z.something`. - `import x.y as alias` is equivalent to `from x import y as alias` for purposes of the local binding — both bind `alias` to the submodule.

What NOT to say: - "`from x import y` is faster because it doesn't load the whole module." Wrong on both counts — it loads the whole module, and the perf difference is one attribute lookup per use, not at import time. - "They're interchangeable, use whichever you prefer." Misses the `mock.patch` trap and the namespace-collision argument.

Common follow-up: "What happens if I do `from x import y` twice in the same file?" — Second one is a no-op. Module is already in `sys.modules`; the second `from x import y` just re-reads the attribute and re-binds it. No re-execution.


```
# Import the module, namespace-qualify usage
import os
print(os.path.join("/tmp", "x"))

# Import a specific name — direct, but mock.patch needs to target the caller
from os import path
print(path.join("/tmp", "x"))

# Star import — discouraged in app code, common in __init__.py and REPL
from collections import * # pulls in OrderedDict, Counter, etc.
```

Q2 · What runs at import time

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Swiggy, JP Morgan · Topic: Import System

The question (interviewer's verbatim phrasing):

If I `import some_module`, what actually executes? What's the difference between a function definition and a function call inside that module?

The 30-second answer: Import executes the module's top-level code from top to bottom, exactly once.

`def f(): ...` runs the def statement, which creates a function object and binds the name — the body does not run. `x = expensive_call()` runs the expensive call right now, at import time. So anything you don't want to happen on import has to be hidden behind a function, a class, or an `if __name__ == "__main__":` guard.

Step-by-step thought process: 1. "Top level of a module is just regular Python that runs once. Every statement at column zero executes in order, from line 1 down to the last line." 2. "`def` and `class` are executable statements — they create the function or class object and bind a name. The body of `def` is compiled but not executed. The body of `class` IS executed at import time, because a class body is essentially a function that builds a namespace." 3. "Expressions at top level run immediately. If I write `CONFIG = json.loads(open('config.json').read())` at the top of a module, importing that module reads and parses the JSON file. That's why config-loading and resource-grabbing belong inside functions, not at module level." 4. "Side effects on import are the classic newbie trap. If `db.py` runs `connect()` at top level, the first `import db` opens a connection — and your test suite that imports it 50 times opens 50 connections. Wrap it in `def get_connection():` or a class with lazy init." 5. "The module's top-level code runs in a fresh namespace which becomes `module.__dict__`. After import finishes, that dict is what you see when you write `module.name`. So `module.name` is identical to looking up `name` in the namespace where the top-level code ran."

Edge cases to mention: - Decorators at top level execute at import time. `@app.route("/x")` runs `app.route("/x")(view_func)` immediately, which is why Flask routes are registered on import. -

Class body code runs at import time including method definitions, decorators, and metaclass machinery. Heavy logic in a class body delays import. - An exception raised during the module's top-level code causes the import to fail and leaves no entry in `sys.modules`. The next import attempt re-runs the code.

What NOT to say: - "Nothing runs on import — it just loads the file." Wrong — top-level code runs. This answer signals you've only used Python at a script level. - "Only function definitions run." Half-right — the `def` runs, the body doesn't, and any non-def statement DOES run.

Common follow-up: "How would you make a module that registers handlers on import without doing it at top level?" — Use a registration decorator that callers apply, or expose a `register(app)` function called by the consumer. Pure-import side effects are a code smell.

```
# bad_module.py – runs at import time
print("loading bad_module")           # prints on import
CONNECTION = open_db()                # opens DB on import
CONFIG = json.load(open("cfg.json"))  # reads file on import

def query(sql):                       # def runs; body waits for a call
    return CONNECTION.execute(sql)

# good_module.py – lazy, no import side effects
_connection = None

def get_connection():
    global _connection
    if _connection is None:
        _connection = open_db()
    return _connection

def query(sql):
    return get_connection().execute(sql)
```

Q3 · `__pycache__` and `.pyc` files

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Postman · Topic: Import System

The question (interviewer's verbatim phrasing):

What is the `__pycache__` directory? Should I commit it to git? When does Python regenerate the `.pyc` files inside it?

The 30-second answer: `__pycache__` holds the compiled bytecode for each `.py` file in the same directory, one `.pyc` per source file per Python version — `foo.cpython-312.pyc` is the bytecode

for `foo.py` under Python 3.12. The compile is automatic and lazy — Python regenerates the `.pyc` when the source's mtime or size changes. Never commit it to git; add `__pycache__` to `.gitignore`.

Step-by-step thought process: 1. "Python is compiled to bytecode before execution. The compile is fast but not free. Caching the bytecode means the second import of the same module skips the parse-and-compile step." 2. "The cache location used to be alongside the source as `foo.pyc`. PEP 3147 (Python 3.2) moved it to `__pycache__/foo.cpython-XY.pyc` so that multiple Python versions don't fight over the same file. The `cpython-312` tag is the magic that identifies which interpreter built it." 3. "Regeneration triggers — Python compares the source file's mtime and size against the metadata embedded in the `.pyc` header. Mismatch means recompile. So edit a `.py` file and the next import rebuilds the corresponding `.pyc` automatically." 4. "In production deployments people sometimes use the `hash` invalidation mode introduced in 3.7 — the `.pyc` stores a SipHash of the source, not its mtime, so containers built at different times with identical source still match. Set `PYTHONPYCACHEPREFIX` or `--check-hash-based-pycs` if you need it." 5. "Never commit `__pycache__`. It's machine-generated, interpreter-specific, and bloats diffs. Standard `.gitignore` entries: `__pycache__` and `*.pyc`."

Edge cases to mention: - `python -B` or `PYTHONDONTWRITEBYTECODE=1` disables `.pyc` writing — useful in read-only containers or sandboxes. - Stale `.pyc` with no matching `.py` is loaded as if the source still existed in Python 2; in 3 it's ignored unless you used `compileall` with the right flags. - The `compileall` module pre-compiles a tree of `.py` files — common in Docker builds to pay the compile cost during the build, not on the first request.

What NOT to say: - "`.pyc` files are an obfuscation tool." No — they're a perf optimization, and `dis` plus `uncompile6` can recover code easily. - "Delete `__pycache__` before every run." Pointless busy-work; Python invalidates automatically.

Common follow-up: "Where does Python put `.pyc` files if it can't write to `__pycache__`?" — It silently skips caching. Read-only deployments still run fine, just paying the compile cost on every cold start. Setting `PYTHONPYCACHEPREFIX=/tmp/cache` redirects all caches to a writable tree.

```
# Force a clean cache rebuild
find . -name "__pycache__" -type d -exec rm -rf {} +
python -c "import myapp" # rebuilds __pycache__ on import

# Pre-compile a tree at build time (typical Dockerfile line)
python -m compileall -q -j 0 /app

# Disable bytecode caching for a single run
PYTHONDONTWRITEBYTECODE=1 python script.py
```

Q4 · `sys.path` — how import finds modules

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Goldman, JP Morgan · Topic: Import System

The question (interviewer's verbatim phrasing):

When I write `import foo`, where does Python look? What controls the search order?

The 30-second answer: Python walks `sys.path` in order, asking each entry to find the module.

`sys.path` is a list of strings — by default it's seeded with the directory of the running script (or current directory in the REPL), then `PYTHONPATH` entries, then site-packages from the active interpreter. The first finder that can produce a loader wins. Built-in modules and frozen modules are handled before `sys.path` is consulted at all.

Step-by-step thought process: 1. "The import machinery is layered. `sys.meta_path` is the top-level list of finders — by default `BuiltinImporter`, `FrozenImporter`, and `PathFinder`. They're tried in order. `BuiltinImporter` handles built-ins like `sys` and `gc`. `FrozenImporter` handles modules frozen into the interpreter. Anything else falls through to `PathFinder`, which walks `sys.path`." 2. "`sys.path` is constructed at interpreter startup. For `python script.py`, the script's directory is index 0. For `python -m pkg`, the current working directory is index 0. For `python -c '...'` or the REPL, again the cwd is index 0. Then comes whatever `PYTHONPATH` contained, then the installation's standard library, then site-packages." 3. "Inside a venv, `site-packages` points to the venv's own directory — not the system Python's. That's how venvs achieve isolation. The interpreter checks `sys.prefix` at startup to decide which site-packages to add." 4. "For each entry on `sys.path`, `PathFinder` consults the path-importer cache. If the entry is a directory, the finder looks for `foo/__init__.py` (regular package) or `foo.py` (module) or `foo/` with no init as a namespace package. If the entry is a zip file or wheel, the zipimport finder takes over." 5. "You can mutate `sys.path` at runtime — append a path, then your next import will look there. Common in test setup. But it's nicer to install the package properly (`pip install -e .`) so `sys.path` is correct without runtime hacks."

Edge cases to mention: - `sys.path[0]` being the script's directory is the source of the "I named my file `random.py` and imports broke" beginner pitfall — your file shadows the stdlib. - `.pth` files inside site-packages get processed at startup and can add more directories to `sys.path` — that's how `setuptools` editable installs used to work pre-PEP 660. - `sys.path_hooks` and `sys.path_importer_cache` are the lower layers that let zip imports and other non-filesystem sources work transparently.

What NOT to say: - "Python searches the current directory first always." Half-right — only true for the REPL and `-c`. For `python script.py` it searches the script's directory, not cwd. - "I just edit `sys.path` in production to fix imports." Code smell that survives until the next deployment; install the package or fix `PYTHONPATH`.

Common follow-up: "What's the difference between `python -m mypkg` and `python mypkg/__main__.py`?" — `-m` adds the current directory to `sys.path`, so relative imports inside the package work. Running the file directly adds `mypkg/`'s directory to `sys.path`, which usually breaks relative imports because the package isn't on the path as a package.

```
import sys
print(sys.path)
# ['/path/to/script-dir',
#  '/usr/lib/python312.zip',
#  '/usr/lib/python3.12',
#  '/usr/lib/python3.12/lib-dynload',
#  '/path/to/venv/lib/python3.12/site-packages']

# Add a directory at runtime (test setup, not production)
sys.path.insert(0, "/opt/internal-libs")
import internal_thing
```

Q5 · `sys.modules` — the module cache

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft GCC · Topic: Import System

The question (interviewer's verbatim phrasing):

What is `sys.modules`? What happens if I delete an entry from it and re-import?

The 30-second answer: `sys.modules` is the cache of every module that has been imported in this interpreter. Keys are dotted module names, values are module objects. When you `import foo`, Python first checks `sys.modules['foo']` — if it's there, the cached module is bound to the local name and no code runs. Deleting the entry and re-importing forces a full reload, but stale references in other modules still point to the old module object.

Step-by-step thought process: 1. "The cache is what makes import idempotent. Without it, every import would re-execute the module's top-level code, which would be both slow and semantically wrong — module-level state would reset on every use." 2. "The check is the very first thing `__import__` does. `sys.modules.get('foo')` is consulted before any finder is invoked. So once a module is in there, the loading machinery is skipped entirely." 3. "The entry is added BEFORE the module's top-level code finishes executing — this is the key fact behind circular imports working at all. Python creates an empty module object, registers it in `sys.modules`, then starts running the body. If during that body another module imports back to this one, it gets the half-finished module rather than a fresh second copy." 4. "`del sys.modules['foo']` followed by `import foo` does force re-execution, but it doesn't update existing references. If module A did `from foo import bar`, A still holds the old `bar`. Reload semantics are subtle — `importlib.reload(foo)` is the proper tool, and even it has

the same reference-staleness issue." 5. "Practical use cases for poking `sys.modules` — test isolation (delete a module so the next test re-imports fresh), debugging (inspect what's loaded), plugin systems that hot-reload. In application code, leave `sys.modules` alone."

Edge cases to mention: - `sys.modules` can contain `None` for a module name to mark it as deliberately not importable — older Python uses this internally; modern code uses `ModuleNotFoundError`. - A module that raises during its top-level code does NOT leave an entry in `sys.modules` (it's removed on the way out), so the next import attempts a fresh load. - For packages, `sys.modules` holds entries for `pkg`, `pkg.sub`, `pkg.sub.subsub` independently. Deleting only `pkg.sub` leaves `pkg` cached but breaks future `pkg.sub` access.

What NOT to say: - "`sys.modules` is just for debugging." Misses that it's the actual mechanism import uses, not a side notebook. - "Reload always works." It works, but stale references in other modules don't update. Reload is a fragile tool.

Common follow-up: "How would you reload a module so that all callers see the new version?" — You don't, cleanly. Either restart the process, or have callers go through the module dictionary every time (`mod.func()` rather than `from mod import func`). Hot-reload frameworks like uvicorn's `--reload` restart the whole process, not selected modules.

```
import sys

# Inspect what's loaded
print(len(sys.modules))          # ~200 immediately after startup
print("collections" in sys.modules) # True after stdlib uses it

# Force a re-import (rarely a good idea in real code)
import mymodule
del sys.modules["mymodule"]
import mymodule                  # re-runs top-level code

# Proper way to reload — still does not update existing references
import importlib
importlib.reload(mymodule)
```

Q6 · `importlib.import_module` — dynamic imports

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft GCC, JP Morgan · Topic: Import System

The question (interviewer's verbatim phrasing):

How do you import a module when you only know its name as a string at runtime?

The 30-second answer: Use `importlib.import_module("dotted.name")`. It returns the module object — same as `import dotted.name` would put in `sys.modules`, but with the module name as a string parameter. For dynamic plugin loading, factory patterns, and CLI commands routed by name, this is the clean approach. Avoid the older `__import__` builtin for the same job — its return semantics for dotted names are awkward.

Step-by-step thought process: 1. "`importlib.import_module(name, package=None)` is the public API. Returns the module corresponding to the dotted name. The `package` argument is only used when `name` starts with a dot (relative imports inside a known parent package)." 2. "Use case one — plugin systems. Config says `handlers = ['myapp.handlers.email', 'myapp.handlers.sms']`. Iterate and `import_module` each one, then call a known function on the resulting module." 3. "Use case two — factory registry. A class-name string from a database row needs to be turned into a class. Split into `module.path` and `ClassName`, `import_module` the first, `getattr` the second." 4. "Underneath, `import_module` goes through the same `_bootstrap._find_and_load` path that the `import` statement uses — same finders, same `sys.modules` cache. So calling it twice is cheap; second call is a dict lookup." 5. "The older `__import__` builtin is what the `import` statement compiles down to. For dotted names it returns the TOP-level package, not the leaf — `__import__('a.b.c')` returns `a`, not `c`. That's surprising and almost never what you want. `import_module` returns the leaf. Prefer it."

Edge cases to mention: - `import_module(".sibling", package="myapp.subpkg")` resolves the relative dot against `myapp.subpkg` — useful inside packages that load plugins by relative name. - The function raises `ModuleNotFoundError` (subclass of `ImportError`) if the name doesn't resolve. Catch `ImportError` to be tolerant of both missing modules and broken imports. - For lazy / optional dependencies, the idiomatic pattern is `try: pkg = importlib.import_module("optional_dep") except ImportError: pkg = None` rather than top-level imports that crash on missing packages.

What NOT to say: - "Just use `exec('import ' + name)`." Slow, unsafe with untrusted strings, and bypasses the proper APIs. This is the answer that signals a Python hobbyist. - "`__import__` is what you should use because it's a builtin." Returns the top-level package for dotted names, which is rarely what you mean.

Common follow-up: "What about importing a specific class or function from a module whose name you have as a string?" — Two steps. `mod = importlib.import_module("dotted.path")` then `cls = getattr(mod, "ClassName")`. Some codebases write a helper that takes `"dotted.path:ClassName"` and does both.


```
import importlib

# Plugin-style – config has module names as strings
for handler_path in config["handlers"]:
    mod = importlib.import_module(handler_path)
    mod.handle(event)

# Factory registry – load a class by string name
def resolve(spec: str):
    """spec like 'myapp.payments.gateways:Razorpay'."""
    mod_path, class_name = spec.split(":")
    mod = importlib.import_module(mod_path)
    return getattr(mod, class_name)

GatewayClass = resolve("myapp.payments.gateways:Razorpay")
gateway = GatewayClass(api_key="...")
```

Q7 · `__import__` – the low-level builtin

Tags: Difficulty: Hard · Asked at: Microsoft GCC, Goldman, JP Morgan · Topic: Import System

The question (interviewer's verbatim phrasing):

What is `__import__` and when would you actually call it directly?

The 30-second answer: `__import__(name, globals, locals, fromlist, level)` is the builtin function that the `import` statement compiles to. You almost never call it directly — `importlib.import_module` is the supported public API. The one legitimate reason to touch `__import__` is to override it (replace `builtins.__import__`) for instrumentation, sandboxing, or audit hooks. Otherwise, treat it as a private implementation detail of the import machinery.

Step-by-step thought process: 1. "`import a.b.c` desugars roughly to `a = __import__('a.b.c', globals(), locals(), [], 0)`. `from a.b import c` desugars to `__import__('a.b', globals(), locals(), ['c'], 0)` and then a `c = a.b.c`." 2. "The surprising part — when `fromlist` is empty, `__import__('a.b.c')` returns the top-level package `a`, not `c`. This is so that the import statement can do `a = result` and then expressions like `a.b.c` work via attribute access. When `fromlist` is non-empty, it returns the leaf module — so `from`-style imports work." 3. "This is exactly why you should not call `__import__` directly for dynamic imports. The asymmetric return semantics confuse callers. `importlib.import_module` smooths over this — it always returns what you asked for." 4. "Legitimate reason to override — `builtins.__import__ = my_import` lets you intercept every import in the program. Used by security sandboxes that disallow certain modules, audit logging tools that track which packages got loaded when, and lazy-import frameworks that defer execution." 5. "Auditing hooks are a cleaner

mechanism for the security use case as of 3.8 — `sys.audit('import', ...)` is fired for every import, and `sys.addaudithook(callback)` lets you observe without overriding `__import__`. Prefer audit hooks for read-only inspection."

Edge cases to mention: - The `level` argument controls relative vs absolute — 0 is absolute, positive integers are relative (one dot = 1, two dots = 2). The implicit-relative behaviour of Python 2 used -1, removed in Python 3. - The `globals` and `locals` arguments are mostly informational for the finder — relative imports use `globals['__package__']` to figure out the anchor. - Overriding `__import__` is fragile — any subtle behaviour change breaks the entire ecosystem of `from x import y` statements that depend on the exact semantics.

What NOT to say: - "It's deprecated, use `importlib`." Not deprecated — it's still the engine the `import` statement runs on. Just not recommended as a public API. - "I use it for dynamic imports because it's a builtin." The return-value asymmetry will bite you with dotted names.

Common follow-up: "Show me how `from a.b import c` actually expands." — Roughly: `_temp = __import__('a.b', globals(), locals(), ['c'], 0); c = _temp.c`. The `['c']` fromlist tells `__import__` to ensure `a.b` is loaded AND that the leaf `a.b` is returned so the trailing attribute access finds `c`.

```
# What the import statement compiles to (conceptually)
import a.b.c
# becomes roughly:
a = __import__('a.b.c', globals(), locals(), [], 0)
# Note: returns 'a', the top-level package. You access a.b.c later.

from a.b import c
# becomes roughly:
_tmp = __import__('a.b', globals(), locals(), ['c'], 0)
c = _tmp.c
# Note: returns 'a.b', the leaf, so we can do .c on it.

# Override for instrumentation (rarely warranted)
import builtins
_real_import = builtins.__import__

def audited_import(name, *args, **kwargs):
    print(f"import: {name}")
    return _real_import(name, *args, **kwargs)

builtins.__import__ = audited_import
```

Q8 · Absolute imports — the default in 3.x

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman · Topic: Import System

The question (interviewer's verbatim phrasing):

What does an absolute import look like and why are absolute imports the default in Python 3?

The 30-second answer: An absolute import names the full module path from the top of `sys.path` — `from myproject.payments import charge_card`. It's the default because relative-implicit imports (where `import foo` inside `mypkg/` could mean either `mypkg.foo` or top-level `foo`) was ambiguous and bug-prone in Python 2. PEP 328 removed the implicit relative import and made every dotted name an absolute lookup unless you use the explicit `from .` syntax.

Step-by-step thought process: 1. "In Python 2, `import foo` inside `mypkg/bar.py` first looked for `mypkg/foo.py`, then fell back to top-level `foo`. That two-stage lookup made code surprising — adding a new top-level package named the same as a sibling module could silently change which module `bar.py` saw." 2. "PEP 328 (Python 2.5, default in Python 3) made `import foo` always absolute. The implicit-relative meaning is gone — to import a sibling you must say `from . import foo` explicitly." 3. "My default for application code is absolute imports — `from myapp.utils import format_phone`. They survive moving files around, they're unambiguous in stack traces, and a reader doesn't have to know the file's current location to understand them." 4. "Absolute imports also play well with mypy and IDEs — both can resolve the path without needing to know the current package context. Relative imports inside a deeply nested package work, but tooling occasionally trips." 5. "The classic counter-argument is rename-portability. If I rename `myapp` to `myapp_v2`, every absolute import inside the package has to update. A relative `from . import sibling` survives the rename. So relative imports earn their keep inside packages that may be relocated."

Edge cases to mention: - `from __future__ import absolute_import` was the Python 2 escape hatch to opt into PEP 328 early. In Python 3 it's a no-op and can be removed. - A top-level module (not inside a package) cannot use relative imports — there's no package context to resolve `.` against. Trying it gets `ImportError: attempted relative import with no known parent package`. - `python script.py` versus `python -m mypkg.script` matters here — the former has no package context, the latter does. Relative imports require the `-m` form.

What NOT to say: - "Absolute imports are slower." False — there's no perf difference once the cache is warm. The choice is about clarity, not speed. - "Relative imports are deprecated." Not true — `from . import x` is fully supported. Only implicit-relative is gone.

Common follow-up: "What's the difference between `import myapp.utils` and `from myapp import utils`?" — Both run the same modules. `import myapp.utils` binds `myapp` locally; you access via `myapp.utils.thing`. `from myapp import utils` binds `utils` locally; you access via `utils.thing`. Pure namespacing preference.

```
# Absolute — full path from sys.path root
from myapp.payments.gateways import RazorpayGateway
import myapp.utils

# Inside myapp/payments/gateways.py:
# Explicit-relative — only inside a package
from . import logging_utils          # sibling module
from .helpers import format_amount   # sibling, specific name
from ..config import API_KEY         # parent package's config

# Top-level script — relative imports forbidden
# # script.py
# from . import helper                # ImportError: no known parent package
```

Q9 · Relative imports — `from .` and `from ..`

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft GCC, Goldman · Topic: Import System

The question (interviewer's verbatim phrasing):

What does `from .` mean? How deep can the dots go and when does Python error?

The 30-second answer: The dots in `from . import x` and `from ..pkg import y` count package levels relative to the current module's package. One dot is the current package, two dots is the parent package, three dots is the grandparent. Python resolves the dots against the module's `__package__` attribute. If the current module isn't part of a package (i.e., `__package__` is `None` or empty), any relative import fails immediately with `ImportError: attempted relative import with no known parent package`.

Step-by-step thought process: 1. "Each dot peels off one level of the dotted package path. If my current module is `myapp.payments.gateways.razorpay`, then `from . import foo` looks in `myapp.payments.gateways`, `from .. import foo` looks in `myapp.payments`, and `from ... import foo` looks in `myapp`." 2. "The resolution uses `__package__`, which Python sets at import time. For a module inside a package, `__package__` is the dotted name of the containing package — for `myapp.payments.gateways.razorpay`, `__package__` is `myapp.payments.gateways`. Add the number of leading dots to count back." 3. "`from . import x` resolves to importing `x` from `__package__`. The `x` can be either a submodule (`myapp.payments.gateways.x`) or a name defined in `myapp/payments/gateways/__init__.py`. Python prefers the submodule if both exist." 4. "Why use them — inside a package, relative imports survive renames of the top-level package. If `myapp` ships as `myapp` in dev but `vendored_myapp` in a customer's install, absolute imports break and relative ones don't." 5. "Why not

use them — readers without IDE assistance have to mentally count dots and match against the file location. Stack traces that say `from .. import config` are less searchable than `from myapp.config import config`. My rule — relative imports inside libraries that get vendored or renamed; absolute everywhere else."

Edge cases to mention: - You can mix them — `from ..siblings import foo` is two-level-up-and-then-down. Same dot-count math. - Running a file directly with `python myapp/payments/gateways/razorpay.py` sets `__package__` to `None`, so any relative import in that file fails. Use `python -m myapp.payments.gateways.razorpay` instead. - Too many dots — `from ....x import y` from a 3-level-deep module fails because there's no 4th level up. `ImportError: attempted relative import beyond top-level package`.

What NOT to say: - "Use one dot for the current module." Wrong — one dot is the current PACKAGE, not the current module. The current module has no name in dot syntax. - "Relative imports work in the REPL." They don't — the REPL has no package context.

Common follow-up: "If I'm at `myapp/payments/__init__.py` and I want a sibling submodule, what's the syntax?" — `from . import gateways` if the sibling is `myapp/payments/gateways.py` or `myapp/payments/gateways/__init__.py`. The single dot from inside `__init__.py` refers to that package itself.

```
# Directory layout
# myapp/
#   __init__.py
#   config.py
#   payments/
#     __init__.py
#     gateways/
#       __init__.py
#       razorpay.py
#       paytm.py

# Inside myapp/payments/gateways/razorpay.py:
from . import paytm           # sibling: myapp.payments.gateways.paytm
from .helpers import normalize # current pkg submod
from .. import payments_logger # parent pkg: myapp.payments.payments_logger
from ..helpers import format_inr # parent pkg submod
from ...config import API_KEY  # grandparent: myapp.config

# Running this file:
# python myapp/payments/gateways/razorpay.py      → ImportError
# python -m myapp.payments.gateways.razorpay      → works
```

Q10 · Lazy imports — when and how

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft GCC, Goldman, Postman · Topic: Import System

The question (interviewer's verbatim phrasing):

What's a lazy import? Why would you defer importing a module until inside a function?

The 30-second answer: A lazy import is one done inside a function or method instead of at module top level — the import only runs the first time the function is called, and stays cached in `sys.modules` for later calls. You defer when the dependency is heavy (slow import), optional (may not be installed), or part of a circular chain you want to break. The cost — repeated attribute lookups per call after the first, and slightly harder static analysis.

Step-by-step thought process: 1. "Top-level imports run during module load, contributing to import-time cost and circular-import risk. Moving an `import` statement inside a function postpones both." 2. "Use case one — heavy optional dependency. My `report` module needs `pandas` to render Excel, but most calls just print to stdout. A top-level `import pandas` makes `pip install report` pull in 50 MB of NumPy and pandas wheels even when those features aren't used. `def to_excel(df): import pandas as pd; ...` keeps the dependency lazy." 3. "Use case two — breaking circular imports. Module A imports B at top level; B's top-level needs something from A. The cycle deadlocks. Move B's import of A inside the function that needs it. By call time, both modules are fully loaded." 4. "Use case three — startup speed. CLI tools live or die by cold-start latency. `python mytool.py --help` shouldn't load TensorFlow. Standard pattern — top-level imports are cheap (`argparse`, `sys`, `os`), heavy imports go inside the subcommand functions." 5. "Cost — each call repays the `sys.modules` lookup (fast) and the attribute access for any names you bind locally. For a function called millions of times in a hot loop, hoist the import back to top level. For one-shot CLI command handlers, lazy is fine."

Edge cases to mention: - The `importlib.util.LazyLoader` mechanism creates a module that defers its execution until first attribute access — used by some frameworks for true lazy modules without function-scope imports. Subtle to set up; rarely worth it in app code. - PEP 690 (lazy imports) was proposed and rejected in 2022 — would have made all imports lazy by default in CPython. Killed because of debugger and observability concerns. - Tools like `mypy` and `pyright` still analyse function-scope imports correctly. IDE jump-to-definition works the same way.

What NOT to say: - "Lazy imports are a bad practice." Blanket statement — they're idiomatic for optional deps and circular fixes. - "It saves memory." Not really — once imported, the module sits in `sys.modules` for the process lifetime. Lazy imports save startup time, not steady-state RAM.

Common follow-up: "What's the perf cost of a function-scope import in a hot path?" — One `sys.modules` dict lookup per call (sub-microsecond). For most code, immeasurable. For a function called in a tight inner loop millions of times, the cumulative cost is meaningful and you'd hoist it.

```
# Top-level – runs on import, slows every consumer
import tensorflow as tf
import pandas as pd

def maybe_train(data):
    if some_condition:
        model = tf.keras.Sequential(...)

# Lazy – runs only inside the function call
def maybe_train(data):
    if some_condition:
        import tensorflow as tf
        model = tf.keras.Sequential(...)

# Optional dependency pattern
def to_excel(df, path):
    try:
        import openpyxl # noqa: F401 – required by pandas
    except ImportError as e:
        raise RuntimeError("install with: pip install myapp[excel]") from e
    import pandas as pd
    df.to_excel(path)

# Circular-import fix – lazy at point of use
# myapp/orders.py
def mark_order_paid(order_id):
    from .payments import refund_handler # was: top-level, caused cycle
    refund_handler.notify(order_id)
```

Section B — `__init__.py` and Packages (Q11–Q20)

Ten questions on what a package actually is, how `__init__.py` shapes the public API, what `__all__` controls and what it doesn't, and the modern namespace-package machinery (PEP 420) that lets `__init__.py` be optional. Interviewers at Razorpay, Postman, and the GCCs lean on these — getting the package-vs-module distinction crisp is what separates Python developers from Python-as-a-second-language candidates.

Q11 · Module versus package

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Postman, Razorpay · Topic: Packages

The question (interviewer's verbatim phrasing):

What's the difference between a module and a package?

The 30-second answer: A module is a single `.py` file Python can import. A package is a directory that can be imported — historically requiring an `__init__.py` file inside it, modernly allowed to be empty (namespace package, PEP 420). Both end up as module objects in `sys.modules`; the difference is whether the importable thing is one file or a tree.

Step-by-step thought process: 1. "Module is the basic unit — any `.py` file Python can find via `sys.path` is a module. `import math`, `import json`, `import myutils` — all modules." 2. "Package is a module that contains other modules. The directory structure `mypkg/foo.py` lets you write `import mypkg.foo` — `mypkg` is a package, `foo` is a submodule. Inside `sys.modules` you see both `mypkg` and `mypkg.foo` as separate entries." 3. "A regular package has an `__init__.py` (possibly empty) — that file gets executed when the package is first imported and becomes the package's module object. Attributes you bind in `__init__.py` are accessible as `mypkg.thing`." 4. "A namespace package (PEP 420, Python 3.3+) has NO `__init__.py`. It's still importable, but it has no module-level code and no `__file__`. Multiple directories on `sys.path` can contribute pieces to the same namespace package — useful for plugin systems." 5. "From the consumer's point of view, the package-vs-module distinction is invisible. `import json` and `import urllib` look the same; you only notice when you start adding submodules or looking at the file system."

Edge cases to mention: - A package can have a `__main__.py` to be executable via `python -m pkg`. That submodule runs when the package is invoked as a script. - A single-file package is sometimes shipped as a package directory with one submodule that re-exports things — `mypkg/__init__.py` doing `from .core import *` — to leave room for future expansion. - Compiled extension modules (`.so` on Linux, `.pyd` on Windows) are also modules — they're just not source `.py` files. `numpy` is a package; `numpy.core._multiarray_umath` is a compiled module inside it.

What NOT to say: - "A package is a module that has multiple files." Closer — but a single-file package is also a real thing (one `__init__.py` and nothing else). - "Packages need `__init__.py`." Not since Python 3.3 — namespace packages are valid without one.

Common follow-up: "What's the difference between `import mypkg.sub` and `from mypkg import sub`?" — Both load `mypkg` then `mypkg.sub` and cache them. `import mypkg.sub` binds `mypkg` to the local name (you access submod via `mypkg.sub`). `from mypkg import sub` binds `sub` directly.


```
# Module – single file
# utils.py
def format_inr(amount): ...

import utils
utils.format_inr(1500)

# Regular package – directory with __init__.py
# mypkg/
#   __init__.py      ← makes it a regular package
#   core.py
#   helpers.py

import mypkg
import mypkg.core
from mypkg.helpers import normalize

# Namespace package – no __init__.py (PEP 420)
# myns/
#   submod.py        ← no __init__.py at myns/ level

# Both `from myns import submod` and `import myns.submod` still work.
```

Q12 · Regular vs namespace packages (PEP 420)

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft GCC, Postman, Goldman · Topic: Packages

The question (interviewer's verbatim phrasing):

What's a namespace package? How does it differ from a regular one and when would you use it?

The 30-second answer: A namespace package is a package without an `__init__.py`. Multiple directories on `sys.path` can contribute portions of the same namespace, and Python merges them into one virtual package at import. Regular packages have an `__init__.py` (even if empty) and own exactly one filesystem directory. Use namespace packages for plugin-style distributions where multiple vendors each ship a sub-package under a shared namespace — the classic example is `zope.*` and modern company namespaces like `mycompany.*`.

Step-by-step thought process: 1. "PEP 420 (Python 3.3) introduced namespace packages. The key change — if Python finds a directory matching a package name but no `__init__.py`, it doesn't fail. It marks the directory as a portion and keeps searching `sys.path` for more portions." 2. "Multi-directory merging — say `sys.path` contains `/a` and `/b`. `/a/mycompany/billing/` exists with no `__init__.py`. `/b/mycompany/payments/` exists with no `__init__.py`. After `import mycompany.billing` and `import mycompany.payments`, both work, and `mycompany` is a

single namespace package combining contributions from both directories." 3. "Regular packages cannot be split. If `/a/mypkg/__init__.py` exists, that's the one `mypkg` — adding `/b/mypkg/` would be ignored because the first finder won." 4. "Use case — plugin distribution. `pip install mycompany-billing` and `pip install mycompany-payments` install separate distributions but both contribute to `mycompany.*`. No need to coordinate `__init__.py` content across maintainers." 5. "When NOT to use — single-team monolithic apps. If you control everything, a regular package with an `__init__.py` is clearer and lets you put package-level code (logging setup, version constants) in one place. Namespace packages confuse newcomers and have subtle gotchas around `__path__`."

Edge cases to mention: - A namespace package has a special `__path__` (a `_NamespacePath`) that lists all contributing directories. Regular packages have a list with one entry. - Mixing — putting `__init__.py` in one directory and not the other for the same logical package gets ambiguous. The directory with `__init__.py` "wins" and the other is ignored. Stick to one mode per package. - Tools like `setuptools.find_namespace_packages()` (vs `find_packages()`) tell setuptools to include namespace-package directories during a build. Missing this is a common packaging bug.

What NOT to say: - "Namespace packages are slower." Not measurably — the path-walking has minor overhead at first import, negligible at runtime. - "Namespace packages need a special declaration." Not in PEP 420 — they're implicit. The older PEP 382 namespace packages needed a `pkgutil.extend_path()` call in `__init__.py`; PEP 420 dropped this.

Common follow-up: "What happens if you put an `__init__.py` in only one of the namespace-package contributing directories?" — That directory becomes a regular package and stops being a portion of a namespace. The other directories get ignored. Either all in or none in.

```
# Two distributions both contributing to `mycompany`:
#
# pip install mycompany-billing → installs:
#   site-packages/mycompany/billing/api.py
#   (no site-packages/mycompany/__init__.py)
#
# pip install mycompany-payments → installs:
#   site-packages/mycompany/payments/gateway.py
#   (still no site-packages/mycompany/__init__.py)

# Both work after install:
import mycompany.billing.api
import mycompany.payments.gateway

# Inspecting the namespace
import mycompany
print(mycompany.__path__)
# _NamespacePath(['site-packages/mycompany'])
# (may contain multiple entries if portions live in multiple sys.path roots)
print(hasattr(mycompany, "__file__")) # False — namespace packages have no __file__
```

Q13 · `__init__.py` — when it runs and why it matters

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Swiggy, Goldman · Topic: Packages

The question (interviewer's verbatim phrasing):

What does `__init__.py` do? Does it run every time I import anything from the package?

The 30-second answer: `__init__.py` is the package's own module body. It runs exactly once — the first time anything in the package is imported, OR the first time the package itself is imported — whichever comes first. After that it sits in `sys.modules` like any other module. Importing a submodule (`from mypkg.sub import x`) triggers `__init__.py` to run before `sub.py` runs.

Step-by-step thought process: 1. "When Python resolves an import inside a package, the package itself must be loaded first. `from mypkg.sub import x` triggers: load `mypkg` (run `__init__.py`), then load `mypkg.sub` (run `sub.py`), then look up `x` ." 2. "The `__init__.py` runs in a namespace that becomes the package's module object. Anything you bind there — variables, imported names, decorated functions — becomes a `mypkg.something` ." 3. "Common content for `__init__.py` — version constants (`__version__ = '1.2.3'`), re-exports (`from .core import MainClass`), public-API gating (`__all__ = [...]`), and lazy submodule loading via `__getattr__` . Heavy content is discouraged — it slows down every consumer of the package." 4. "Empty `__init__.py` is fine and common — it just marks the directory as a regular package. Modern code increasingly skips `__init__.py` entirely to use namespace-package semantics, but explicit packages are still the safer default." 5. "Side effects in `__init__.py` are the classic packaging foot-gun. If `mypkg/__init__.py` opens a database connection or reads a config file, every `import mypkg.anything` triggers it. Wrap in a function or move to a `bootstrap()` the caller explicitly invokes."

Edge cases to mention: - Failure during `__init__.py` execution removes the package from `sys.modules` AND causes the importing line to raise. Recovery — fix the bug and re-import. - `__init__.py` can define a `__getattr__` (PEP 562, Python 3.7+) for lazy attribute loading — `import mypkg; mypkg.heavy_thing` defers loading `heavy_thing` until the access. - The order of imports inside `__init__.py` matters when those imports cross-reference. Putting `from .a import x` before `from .b import y` exposes `x` to `b` if `b` imports from `mypkg` .

What NOT to say: - " `__init__.py` runs on every import from the package." Wrong — runs once, cached after. - "It can be skipped if I don't need it." Skipping makes the directory a namespace package, not a regular one — that's a different thing with different semantics.

Common follow-up: "What's the rule of thumb for what goes in `__init__.py` ?" — Public-API re-exports (`from .core import MainClass`), `__all__` if you want to control star imports, package metadata (`__version__` , `__author__`), and nothing heavy. If you find yourself doing real work in `__init__.py` , move it to a function callers invoke.

```
# mypkg/__init__.py – typical content
"""mypkg – short package description."""

__version__ = "1.4.2"

# Re-export public API
from .core import MainProcessor, SubProcessor
from .helpers import normalize, validate
from .exceptions import MyPkgError, ValidationError

# Control what `from mypkg import *` exposes
__all__ = [
    "MainProcessor",
    "SubProcessor",
    "normalize",
    "validate",
    "MyPkgError",
]

# PEP 562 lazy attribute – module-level __getattr__
def __getattr__(name):
    if name == "heavy_thing":
        from .heavy import HeavyThing
        return HeavyThing
    raise AttributeError(f"module 'mypkg' has no attribute {name!r}")
```

Q14 · `__all__` and what `from pkg import *` actually does

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft GCC, Cred · Topic: Packages

The question (interviewer's verbatim phrasing):

What is `__all__` for? What happens to `from mypkg import *` if I define it versus if I don't?

The 30-second answer: `__all__` is a list of strings naming the public API of a module. When you `from x import *`, Python copies into your namespace either everything in `__all__` (if defined) OR every name that doesn't start with an underscore (if `__all__` is missing). `__all__` is also used by tooling — IDE auto-complete, mypy, documentation generators — as the canonical "what's public." It does NOT prevent explicit `from x import _private` or restrict attribute access in any other way.

Step-by-step thought process: 1. "`__all__` is a list of strings at module level. When `from mymod import *` runs, Python looks for `mymod.__all__`. If present, only those names are imported. If absent, every name not starting with underscore is imported." 2. "This is the only place `__all__` directly affects behaviour. `from mymod import explicitly_named_thing` always works, even if

`explicitly_named_thing` is not in `__all__` or starts with underscore. `mymod.thing` also always works. `__all__` is a convention, not enforcement." 3. "Why bother — `from pkg import *` is real in REPL workflows and inside `__init__.py` for re-exports. Without `__all__`, your private helpers leak into the consumer's namespace. With `__all__`, you control exactly what wildcard import means." 4. "Secondary value — tooling. IDEs use `__all__` to decide auto-complete suggestions; documentation generators (Sphinx) use it for the "public API" pages; type checkers warn if you import something not in `__all__`. Treat it as the public contract." 5. "Practical pattern — define `__all__` at the bottom of the module after all the public names exist. Use a list of strings, alphabetised, one per line. Easy to diff in code review. Re-export in `__init__.py` and the package's public API is crystal clear."

Edge cases to mention: - `__all__` can contain names of submodules — `__all__ = ["sub1", "sub2"]` in `__init__.py` means `from mypkg import *` imports the submodules. This requires those submodules to already be loaded or importable as attributes. - A typo in `__all__` is a runtime error only at the moment of `from x import *` — Python raises `AttributeError` when copying. Easy to miss until someone wildcard-imports. - Star imports are a `SyntaxError` inside function bodies in Python 3 — only allowed at module top level.

What NOT to say: - "`__all__` makes names private." It doesn't — anyone can still `from mod import _private` if they know the name. - "Star imports are evil and never used." They're discouraged in app code but standard in test fixtures, REPL, and inside `__init__.py` re-exports.

Common follow-up: "What's the alternative to `__all__` for hiding implementation details?" — Underscore-prefix the implementation names (`_helper`, `_internal`). Star imports won't pick them up, and PEP 8 readers know `_x` means private. Use `__all__` when you want to be explicit AND positive (these are the names) rather than negative (everything but underscores).

```
# mymod.py — without __all__
def public_one(): ...
def public_two(): ...
def _helper(): ...
INTERNAL_CONSTANT = 42

# from mymod import * — imports public_one, public_two, INTERNAL_CONSTANT
# (_helper is skipped because of the underscore)

# mymod.py — with __all__
__all__ = ["public_one", "public_two"]

def public_one(): ...
def public_two(): ...
def _helper(): ...
INTERNAL_CONSTANT = 42

# from mymod import * — imports only public_one and public_two
# INTERNAL_CONSTANT no longer leaks even without underscore prefix
```

Q15 · Exposing public API at the top level via `__init__.py`**Tags:** Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft GCC, Swiggy · Topic: Packages**The question (interviewer's verbatim phrasing):**

How do you make `from mypkg import MainThing` work when `MainThing` is defined in `mypkg/core.py` ?

The 30-second answer: Inside `mypkg/__init__.py`, write `from .core import MainThing`. That line runs when `mypkg` is first imported, makes `MainThing` an attribute of the `mypkg` module object, and lets consumers write `from mypkg import MainThing` instead of the longer `from mypkg.core import MainThing`. The pattern is how you separate internal file structure from public import paths.

Step-by-step thought process: 1. "The default behaviour — if I put my class in `mypkg/core.py`, consumers must write `from mypkg.core import MainThing`. That couples the import path to the file layout." 2. "Re-exporting via `__init__.py` decouples them. `from .core import MainThing` in `__init__.py` puts `MainThing` on the package's module object. Now `from mypkg import MainThing` works." 3. "Why bother — refactoring freedom. Tomorrow I split `core.py` into `core/processor.py` and `core/handler.py`. I update the `__init__.py` re-exports; every consumer doing `from mypkg import MainThing` keeps working. The internal layout changed; the public API didn't." 4. "This is how every big package works. `pandas.DataFrame` lives somewhere deep in `pandas/core/frame.py`, but `from pandas import DataFrame` works because `pandas/__init__.py` re-exports it. Same with `requests.Session`, `flask.Flask`, `numpy.ndarray`." 5. "Combine with `__all__` to declare the public API explicitly. `mypkg/__init__.py` lists the re-exports and the `__all__` matches. Code reviewers can see at a glance what `mypkg` promises."

Edge cases to mention: - Heavy re-exports increase import time — every `from .submod import X` runs `submod`'s top-level code. If your package has 50 submodules and `__init__.py` imports from all of them, `import mypkg` takes a measurable hit. Lazy loading via PEP 562 `__getattr__` is the escape hatch. - Circular import risk — if a submodule imports from `mypkg` (because `mypkg.utils` is convenient) and `mypkg/__init__.py` imports from that submodule, you get a half-loaded module. Either restructure or use relative imports inside the package. - Tooling note — some linters flag the re-export pattern as "imported but unused." Annotate with `# noqa: F401` or use the `from .core import MainThing as MainThing` form which signals intent to type checkers.

What NOT to say: - "Just use `*` imports — `from .core import *` is fine." It works but loses control of the public surface and pollutes the package namespace with anything `core.py` happens to import. - "Put all the classes directly in `__init__.py`." Fine for a tiny package; unmaintainable past a few hundred lines.

Common follow-up: "How do you handle 50 submodules without making `import mypkg` slow?" — PEP 562 `__getattr__` inside `__init__.py` lets you defer the heavy imports until the attribute is actually accessed. `def __getattr__(name): if name == 'HeavyClass': from .heavy import HeavyClass; return HeavyClass` .

```
# mypkg/
# __init__.py
# core.py          ← MainThing defined here
# helpers.py
# exceptions.py

# mypkg/__init__.py
from .core import MainThing, AnotherThing
from .helpers import normalize, validate
from .exceptions import MyPkgError

__all__ = [
    "MainThing",
    "AnotherThing",
    "normalize",
    "validate",
    "MyPkgError",
]

# Consumer code — short, internal layout-independent
from mypkg import MainThing, normalize
# vs. the longer path that exposes internals:
# from mypkg.core import MainThing
# from mypkg.helpers import normalize
```

Q16 · `python -m pkg` — running a package as a script

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft GCC, Goldman · Topic: Packages

The question (interviewer's verbatim phrasing):

What does `python -m mypkg` do? When is it different from `python mypkg/__main__.py` ?

The 30-second answer: `python -m mypkg` tells Python to import the package `mypkg` and then run its `__main__.py` as the main module. It sets the working directory as the first entry of `sys.path` and sets the running module's `__package__` correctly, so relative imports work. `python mypkg/__main__.py` runs `__main__.py` as a top-level script — `__package__` is empty, so any `from .` import inside the package fails.

Step-by-step thought process: 1. "Convention — a package can ship a `__main__.py` so it's runnable. `python -m unittest`, `python -m pip`, `python -m http.server` are all using this. The `-m` flag means "module mode" — import this thing, then run it." 2. " `python -m mypkg` does three things — adds the cwd to `sys.path[0]` so other packages in the project are findable, imports `mypkg` (running `__init__.py`), then runs `mypkg/__main__.py` with `__name__ == '__main__'`." 3. " `python mypkg/__main__.py` skips the import machinery. It treats the file as a free-standing script. `sys.path[0]` becomes `mypkg/`'s directory, NOT the project root. `__package__` is empty. Relative imports inside that file fail." 4. "Practical impact — if your CLI uses `from .commands import deploy` inside `__main__.py`, `python -m mypkg` works and `python mypkg/__main__.py` does not. The error is the famous `ImportError: attempted relative import with no known parent package`." 5. "The packaging-side companion is `entry_points` in `pyproject.toml` — `console_scripts` create a shim that effectively does `python -m mypkg`. So `pip install mypkg` puts a `mypkg` executable on the path that runs the same thing."

Edge cases to mention: - `python -m mypkg.subcommand` also works for any submodule, not just `__main__`. Used by `python -m pip install`, `python -m venv myenv`, etc. - Inside `__main__.py`, `__name__` is `'__main__'` not `'mypkg.__main__'` — so the standard `if __name__ == '__main__':` guard fires. - Python 3.11+ improved error messages — if `__main__.py` uses relative imports and you accidentally run the file directly, the error suggests `python -m mypkg`.

What NOT to say: - " `python -m` is just a shortcut for `python file.py`." Misses the difference in `sys.path` setup and `__package__` resolution. - "Always use `python file.py`." Breaks relative imports inside packages; not a workable rule.

Common follow-up: "How do you make `mypkg` runnable both as `python -m mypkg` and as a `mypkg` console command after `pip install`?" — Define `__main__.py` for the `-m` form, and add `[project.scripts] mypkg = "mypkg.__main__:main"` (or similar) in `pyproject.toml` for the console script. Both end up calling the same `main()` function.


```
# mypkg/__main__.py – package entry point
from .cli import build_parser
from .commands import run

def main():
    args = build_parser().parse_args()
    return run(args)

if __name__ == "__main__":
    raise SystemExit(main())

# These all run the same code:
# python -m mypkg --help
# python -m mypkg deploy
# mypkg deploy                                (after pip install – entry_points)

# This does NOT work (relative imports break):
# python mypkg/__main__.py
```

Q17 · `__init__.py` re-export pattern for sub-packages

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred · Topic: Packages

The question (interviewer's verbatim phrasing):

If I have `mypkg/sub/inner.py` defining `Thing`, what's the cleanest way to expose `Thing` as `mypkg.Thing`?

The 30-second answer: Re-export it up the chain. `mypkg/sub/__init__.py` has `from .inner import Thing`. `mypkg/__init__.py` has `from .sub import Thing`. Now `from mypkg import Thing` works. Each level of `__init__.py` flattens the public API one step. The internal file layout (`sub/inner.py`) becomes an implementation detail that can change without breaking consumers.

Step-by-step thought process: 1. "Public API and file layout are different concerns. File layout reflects how I want my code organized; public API reflects how consumers should access it." 2. "Bottom-up re-export — `inner.py` defines `Thing`. `sub/__init__.py` does `from .inner import Thing`, which exposes `Thing` as `mypkg.sub.Thing`. `mypkg/__init__.py` does `from .sub import Thing`, exposing `Thing` as `mypkg.Thing`." 3. "Now consumers have three valid imports: `from mypkg.sub.inner import Thing` (deep, fragile), `from mypkg.sub import Thing` (medium), `from mypkg import Thing` (flat, stable). All three resolve to the same class object." 4. "Style consideration — declare your preferred surface explicitly. Put `__all__ = ['Thing']` in `mypkg/__init__.py`. Document that the deep form `mypkg.sub.inner` is internal and may move. When

you later refactor `sub/inner.py` into `sub/processor.py` and `sub/handler.py`, the surface form survives." 5. "For typing — use `from .sub import Thing as Thing` (PEP 484 explicit re-export). mypy treats `Thing as Thing` as "this is intentionally re-exported." Without the `as`, strict mypy emits a warning for re-export of an unused import."

Edge cases to mention: - Star re-export — `from .sub import *` is concise but propagates whatever `sub/__init__.py.__all__` decided. Less explicit but valid. - Heavy submodule load — re-exporting from `__init__.py` runs the submodule's top-level code. For a tree of 10 sub-packages all re-exporting, `import mypkg` runs all of them. Lazy `__getattr__` is the fix for big packages. - Don't re-export private names — `from .inner import _internal` in `__init__.py` is a code smell; if it's internal, leave it where it is.

What NOT to say: - "Re-exporting is unnecessary, consumers should use full paths." Forces fragile imports — internal file moves break consumers. - "Just put everything in one file." Fine for tiny packages; unmanageable past a couple thousand lines.

Common follow-up: "What if consumers have already started using `mypkg.sub.inner.Thing`?" — Leave the deep path importable forever (it's free — re-exports don't remove the original). Tag your docs to say the deep path is internal; gently migrate consumers via deprecation warnings if you ever need to actually remove `sub/inner.py`.

```
# mypkg/
#  __init__.py          ← from .sub import Thing
#  sub/
#    __init__.py        ← from .inner import Thing
#    inner.py           ← class Thing: ...

# mypkg/sub/inner.py
class Thing:
    def process(self): ...

# mypkg/sub/__init__.py
from .inner import Thing as Thing  # explicit re-export for mypy

# mypkg/__init__.py
from .sub import Thing as Thing

__all__ = ["Thing"]

# Consumer side — all three work, prefer the shortest:
from mypkg import Thing           # public, stable
from mypkg.sub import Thing       # also fine
from mypkg.sub.inner import Thing # internal — discouraged
```

Q18 · Empty `__init__.py` versus none at all

Tags: Difficulty: Easy · Asked at: TCS, Postman, Razorpay · Topic: Packages

The question (interviewer's verbatim phrasing):

Is there any difference between an empty `__init__.py` and no `__init__.py` file at all?

The 30-second answer: Yes, big difference. Empty `__init__.py` makes the directory a regular package — exactly one filesystem location, attributes you bind go on a real module object, `__file__` exists. No `__init__.py` makes it a namespace package — multiple directories can contribute portions, `__file__` doesn't exist, `__path__` is a special object. For a single-team codebase, prefer empty `__init__.py` to keep semantics predictable.

Step-by-step thought process: 1. "The presence of `__init__.py` (even empty) is the signal that says regular package. Python treats the directory as a single, owned-by-one-location module. The namespace is fixed; multiple `mypkg/` directories on different `sys.path` entries do NOT merge." 2. "No `__init__.py` means namespace package per PEP 420. Python walks `sys.path` and collects every directory matching the package name into a single virtual package. `__path__` is a `_NamespacePath` object listing all contributions." 3. "For a monolithic application package, regular wins. You get a real module object, can add metadata (`__version__`), can re-export from submodules, can put package-level constants. Empty `__init__.py` is one line of code; the benefits compound." 4. "For a multi-vendor plugin ecosystem — namespace wins. `pip install vendor-a` and `pip install vendor-b` both shipping under `myorg.*` work without coordination. Examples — `zope.*`, `azure.*`, modern company namespaces." 5. "Mixing is the gotcha. If half your distributions ship `myorg/__init__.py` and the other half don't, you get unpredictable behaviour — the regular-package directory wins and silently shadows the namespace contributions from elsewhere. Pick one mode and document it."

Edge cases to mention: - An `__init__.py` containing just a docstring is still a regular package — the file existing is what matters, not the content. - `__init__.py` with `__path__ = pkgutil.extend_path(__path__, __name__)` is the older PEP 382-style namespace pattern. Still works but PEP 420 has replaced it in new code. - Some test discoverers (older pytest) couldn't find tests inside namespace packages reliably. Modern pytest handles both.

What NOT to say: - "There's no difference." Wrong — file vs no file changes the import semantics meaningfully. - "Namespace packages are always better because they're more flexible." Trade-off — you lose package-level code organization, version constants, and the ability to put a docstring on the package.

Common follow-up: "How do you tell at runtime whether a package is a namespace package?" — `mypkg.__file__` exists on regular packages, missing or `None` on namespace packages. Or `isinstance(mypkg.__path__, list)` for regular vs `_NamespacePath` for namespace.

```
# Regular package
# mypkg/
#   __init__.py          ← empty file, just exists
#   submod.py

import mypkg
print(mypkg.__file__)      # 'path/to/mypkg/__init__.py'
print(mypkg.__path__)      # ['path/to/mypkg']

# Namespace package
# mypkg/
#   submod.py            ← no __init__.py at this level

import mypkg
print(mypkg.__file__)      # AttributeError or None
print(mypkg.__path__)      # _NamespacePath(['path/to/mypkg'])
print(hasattr(mypkg, "__file__")) # False

# Check programmatically
import importlib.util
spec = importlib.util.find_spec("mypkg")
is_namespace = spec.submodule_search_locations is not None and spec.origin is None
```

Q19 · `__init__.py` with submodule loading via `__getattr__`

Tags: Difficulty: Hard · Asked at: Microsoft GCC, Razorpay, Postman · Topic: Packages

The question (interviewer's verbatim phrasing):

Your package has many heavy submodules and `import mypkg` is slow. How do you make submodule access lazy without breaking the API?

The 30-second answer: PEP 562 (Python 3.7+) added module-level `__getattr__`. Define `def __getattr__(name):` inside `__init__.py` — Python calls it whenever a missing attribute is accessed on the module. Use it to import the submodule on first access and cache the result. Consumers writing `mypkg.HeavyClass` see no API change; the actual loading is deferred to the first attribute lookup.

Step-by-step thought process: 1. "PEP 562 introduced two module-level hooks — `__getattr__(name)` for missing attribute access and `__dir__()` for `dir()` results. Both run at module level, NOT class level. They're functions defined directly inside the module." 2. "Inside `mypkg/__init__.py`, write `def __getattr__(name)`. Python calls it when `mypkg.something` doesn't find `something` in the package's namespace. Return the value to bind; raise `AttributeError` to signal "no such attribute"." 3. "For lazy submodule loading, the body does the

import: `if name == 'HeavyClass': from .heavy import HeavyClass; return HeavyClass`. After the first access, the name is bound to the module via the return — subsequent accesses don't call `__getattr__` again because the attribute now exists directly." 4. "Actually — that last sentence is wrong by default. `__getattr__` is called only when the lookup misses. To cache, you either let the attribute access also bind (some frameworks use `setattr(mod, name, value)` then return) or you accept that each access re-runs the import (which is cheap after the first time because of `sys.modules`)." 5. "This is how big libraries like SciPy keep import time bounded. `import scipy` doesn't trigger every subpackage. `scipy.optimize.minimize` triggers `scipy.optimize` on first access; before that it's not loaded. Combined with `__all__` and `__dir__`, the experience is identical to eager imports for consumers but much faster for cold start."

Edge cases to mention: - `__getattr__` is only called on missing attributes. If `__init__.py` already does `from .heavy import HeavyClass` at top level, the lazy path is bypassed. Choose: eager re-export OR lazy `__getattr__`, not both. - `__dir__()` should return the full public API list (including lazily-loaded names) so that `dir(mypkg)` and IDE tab completion still work. - Pickling can interact oddly — a lazily-loaded class needs to be reachable via the same fully-qualified name on unpickle. Test pickle round-trips for any lazy-loaded class.

What NOT to say: - "Just import everything at top level — performance doesn't matter." For libraries used by CLI tools, import time matters a lot. - "Use `importlib.util.LazyLoader`." That's the module-level lazy-load tool, more invasive — `__getattr__` is the right tool for attribute-level laziness.

Common follow-up: "Show me the pattern with caching so the import only happens once." — `setattr(sys.modules[__name__], name, value)` after the import, then return. Subsequent accesses find the cached attribute and skip `__getattr__`.

```
# mypkg/__init__.py – lazy heavy submodules
import sys
import importlib

_LAZY_SUBMODULES = {
    "HeavyProcessor": ".heavy",
    "MLModel":        ".ml",
    "Reporter":        ".reporting",
}

def __getattr__(name):
    if name in _LAZY_SUBMODULES:
        mod = importlib.import_module(_LAZY_SUBMODULES[name], package=__name__)
        value = getattr(mod, name)
        # Cache so future lookups skip __getattr__
        setattr(sys.modules[__name__], name, value)
        return value
    raise AttributeError(f"module 'mypkg' has no attribute {name!r}")

def __dir__():
    return sorted(list(globals().keys()) + list(_LAZY_SUBMODULES.keys()))

__all__ = list(_LAZY_SUBMODULES.keys())

# Consumer side – looks like eager import, behaves lazily:
import mypkg                # fast – only __init__.py runs
mypkg.HeavyProcessor()      # NOW the heavy module loads
mypkg.HeavyProcessor()      # cached on second call
```

Q20 · Sub-package discovery — what makes `pkg/sub/` a sub-package

Tags: Difficulty: Easy · Asked at: TCS, Postman, Razorpay · Topic: Packages

The question (interviewer's verbatim phrasing):

What makes `mypkg/sub/` a sub-package versus just a folder?

The 30-second answer: Two things, in order — the directory must be reachable as `mypkg.sub` via Python's import system (i.e., `mypkg` must be on `sys.path` either as a regular or namespace package), and `mypkg/sub/` must itself either contain an `__init__.py` (regular sub-package) or be discoverable as a namespace portion. A folder with neither `__init__.py` nor a way for the parent package to find it is just a folder — Python can't import from it.

Step-by-step thought process: 1. "Sub-package is the same concept as package, just nested. `import mypkg.sub` means Python imports `mypkg` first, then looks for `sub` inside it using the parent

package's `__path__`." 2. "For the lookup to succeed, the parent package must know about the sub-directory. For a regular package (parent has `__init__.py`), `__path__` is `[parent_dir]` — Python looks inside `parent_dir` for `sub` as either a `.py` file or a directory." 3. "If `parent_dir/sub/__init__.py` exists, `sub` is a regular sub-package. If `parent_dir/sub/` exists with no `__init__.py`, `sub` is a namespace sub-package — same PEP 420 rules apply within the parent. If `parent_dir/sub.py` exists, `sub` is a module, not a package." 4. "Order of resolution — Python prefers packages over modules. `mypkg/sub/__init__.py` and `mypkg/sub.py` both exist? The package wins, the module is shadowed. Don't do this; the linter will complain too." 5. "Common trap — adding a directory inside a package without `__init__.py` and expecting it to be importable. In a regular-package parent, the child directory is found as a namespace sub-package as long as no other `__init__.py` style child shadows it. Inconsistent enough to be confusing; add the `__init__.py` to be unambiguous."

Edge cases to mention: - Test directories often live inside packages and are not themselves importable. By convention, `tests/` has no `__init__.py` and is discovered by the test runner directly. That sidesteps the import system entirely. - Adding `__init__.py` to a folder that previously had none can change behaviour if the folder was a namespace portion that other distributions contributed to — flips it to regular mode, suppresses external portions. - Sub-packages can themselves have sub-sub-packages — recursion is unlimited in principle.

What NOT to say: - "Any folder under a package is a sub-package." False — depends on `__init__.py` or namespace package rules. - "You need `__init__.py` everywhere in the tree." Not since PEP 420; though it's still the safer choice for monolithic apps.

Common follow-up: "What if the same name exists as both a sub-package and a top-level package?" — The parent's `__path__` is checked first via the relative or `mypkg.x` lookup. Top-level lookup is a separate import; no conflict for explicit forms. For `import x` inside `mypkg/__init__.py`, that's absolute, so it goes to top-level — not the sub-package.

```
# mypkg/
# __init__.py          ← makes mypkg a regular package
# utils.py             ← module: mypkg.utils
# sub/
#   __init__.py        ← makes sub a regular sub-package
#   helpers.py         ← module: mypkg.sub.helpers
#   deep/
#     __init__.py      ← regular sub-sub-package
#     leaf.py          ← module: mypkg.sub.deep.leaf

# All these work:
import mypkg                # runs mypkg/__init__.py
import mypkg.utils
import mypkg.sub
import mypkg.sub.helpers
import mypkg.sub.deep.leaf

from mypkg.sub.deep.leaf import LeafThing

# This folder is NOT a sub-package — no __init__.py here:
# mypkg/sub/data/ (just contains CSV files for tests)
# But if you `import mypkg.sub.data`, it works as a namespace sub-package.
# Subtle. Prefer to keep non-import directories outside the package tree.
```

Section C — Circular Imports (Q21–Q25)

Five questions on circular imports — what causes them, what actually breaks, and the three real fixes. The interview separates juniors and seniors here — juniors say "circular imports mean your design is wrong"; seniors say "here's why it failed, here are three fixes, here's the trade-off for each."

Q21 · What causes a circular import to fail

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft GCC, Goldman · Topic: Circular Imports

The question (interviewer's verbatim phrasing):

Two modules import each other. What actually goes wrong and what's the exact error message?

The 30-second answer: When module A's top-level code starts importing B, Python creates a half-finished module object for A, registers it in `sys.modules`, then starts running A's body. If A's body imports B, Python starts B. If B's top-level body says `from A import X`, Python looks at

`sys.modules['A']` — finds the half-finished module — checks for attribute `X`, and finds it missing (because A hasn't gotten to defining X yet). Result:

`ImportError: cannot import name 'X' from partially initialized module 'A'.`

Step-by-step thought process: 1. "The import machinery's job is idempotency — every module loads exactly once. It achieves this by adding the module to `sys.modules` BEFORE the body finishes. So during a partial load, the module is visible but incomplete." 2. "The cycle — A starts loading. Python registers an empty A in `sys.modules` and runs A's top. A's first line is `from B import Y`. Python starts loading B. Python registers empty B in `sys.modules` and runs B's top. B's first line is `from A import X`. Python looks up `sys.modules['A']`, gets the half-loaded A, tries `getattr(A, 'X')`, gets `AttributeError` (because A hasn't run far enough to bind X yet)." 3. "The error message — Python 3.5+ wraps that `AttributeError` into `ImportError: cannot import name 'X' from partially initialized module 'A' (most likely due to a circular import)`. The wording is intentionally specific so you know it's not a typo, it's a cycle." 4. " `import A` (rather than `from A import X`) succeeds in the same scenario — because `import A` just binds the half-finished module object, and that's a legal module object. The failure only happens at the `from ... import name` step where Python has to look up the specific name." 5. "This is why "import module, use late" works as a fix — `import A` succeeds even mid-cycle, and `A.X` accessed later at call time works because by then A's body has finished and `X` is bound."

Edge cases to mention: - The error message is improved in Python 3.5+ to mention "partially initialized module." Older Pythons give a vague `ImportError: cannot import name 'X'` which is harder to diagnose. - If both modules use `import` (not `from import`) at top level, the cycle "succeeds" — both modules end up importing each other's module objects, and as long as no top-level code in either module tries to USE a name from the other, everything works. - A package's `__init__.py` importing a submodule that imports back from the package can trigger the same error, with the package being the half-loaded one.

What NOT to say: - "Python doesn't allow circular imports." It allows them; it just fails specifically when one half-loaded module accesses a name not yet defined in the other. - "Circular imports always mean bad design." Often true but not always — small bidirectional relationships are sometimes legitimate.

Common follow-up: "Show me an example where the cycle works versus where it breaks." — `import` style succeeds (no name lookup mid-load); `from X import name` breaks (name lookup mid-load). The fix is to either restructure, defer the `from import` inside a function, or switch to `import` style and access lazily.


```
# a.py — top-level uses from import — BREAKS
from b import b_thing

def a_thing():
    return b_thing() + 1

# b.py — same pattern — BREAKS
from a import a_thing

def b_thing():
    return a_thing() + 1

# Run: python a.py
# ImportError: cannot import name 'a_thing' from partially initialized module 'a'
# (most likely due to a circular import)

# Fix #1 — import the module, not the name
# a.py
import b

def a_thing():
    return b.b_thing() + 1    # late lookup; b is fully loaded by call time

# b.py
import a

def b_thing():
    return a.a_thing() + 1
```

Q22 · "Import at top, use late" — why it works

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred · Topic: Circular Imports

The question (interviewer's verbatim phrasing):

People say circular imports break when you do `from X import Y` but work when you do `import X` and reference `X.Y` inside a function. Why?

The 30-second answer: `import X` at top level just binds the module object — Python can give you the half-finished module object during a partial load, and that's a legitimate reference. `X.Y` inside a function does the attribute lookup at call time, by which point the module's body has finished and `Y` exists. `from X import Y` at top level forces the attribute lookup right now, while the module is half-loaded — and `Y` may not exist yet.

Step-by-step thought process: 1. "There are two attribute-lookup moments in any import-and-use sequence — when you write the import (early), and when you actually use the name (late). The cycle problem is about whether the early lookup catches the module in a half-loaded state." 2. " `from X import Y` does BOTH lookups at import time. First it loads X (giving you the half-loaded module if mid-cycle), then it binds the local name `Y` to `X.Y`. If `X.Y` hasn't been defined yet, the second lookup fails — `ImportError: cannot import name 'Y'.`" 3. " `import X` only does the module-load part. It binds `X` to the (possibly half-loaded) module object. No attribute lookup happens yet. The function body containing `X.Y` is compiled but not executed." 4. "At call time — by which point all module loads in the chain are complete — `X.Y` resolves correctly. Python looks at `sys.modules['X']`, finds the now-complete module, and finds `Y` defined on it." 5. "This is the basis of the "import-at-top, use-late" pattern. It doesn't really break the cycle — both modules still cross-reference each other — but it defers the attribute lookups to a moment when the partial-load state has cleared."

Edge cases to mention: - If you USE `X.Y` at the top level of your module (not inside a function), the late-lookup advantage is lost — the lookup happens at import time, same as `from X import Y`. - Late-bound attribute lookup is slightly slower per call (one extra dict lookup) than a directly-bound name. Negligible outside hot loops. - For type hints, `X.Y` inside annotations resolves at function-definition time by default — `from __future__ import annotations` defers annotation evaluation, which solves the same circular-typing problem.

What NOT to say: - "It works because Python is single-threaded." Irrelevant — the issue is purely about when attribute lookup happens. - "Both forms are equivalent." Not when there's a circular chain mid-load.

Common follow-up: "What if I have a top-level constant that I really do need to access at module load time?" — Then you've found a genuine cyclic dependency between top-level state in the two modules. Either move the shared constant to a third module both depend on, or defer your top-level usage.

```
# Pattern that breaks with from import
# orders.py
from payments import refund_handler          # ← fails if payments is half-loaded

def refund(order):
    refund_handler.process(order)

# Pattern that works with import + late use
# orders.py
import payments                             # ← just binds module object

def refund(order):
    payments.refund_handler.process(order)   # ← lookup at call time

# Late-binding for free function reference
# orders.py
def make_refund_handler():
    from payments import refund_handler     # function-scope; runs at call
    return refund_handler

# Both forms succeed even when payments.py also imports orders.py.
```

Q23 · Restructuring to break a circular dependency

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft GCC, Goldman · Topic: Circular Imports

The question (interviewer's verbatim phrasing):

You hit a circular import. Walk me through the restructuring options.

The 30-second answer: Three real fixes, in order of quality. One — extract the shared dependency into a third module both depend on (cleanest, no cycle remains). Two — switch from `from X import Y` to `import X` and access `X.Y` late inside functions (preserves the cycle but works because attribute lookup happens after full load). Three — defer the import inside the function that uses it (function-scope import; runs first call, cached thereafter). Pick option one if the cycle reveals real coupling; options two and three if the cycle is incidental.

Step-by-step thought process: 1. "Step one — characterise the cycle. Two-module cycle is the common case. Larger cycles ($A \rightarrow B \rightarrow C \rightarrow A$) need the same diagnosis. The question is whether the cycle reflects real bidirectional dependency or an accidental one." 2. "Fix one — extract. If A and B both need a `User` class, neither owns it. Move `User` to `models.py`; both A and B import from `models`; cycle gone. This is the cleanest fix because the result has cleaner architecture, not just a working build." 3. "Fix two — import the module, not the name. A's top has `import B` not `from B import thing`; A's functions use `B.thing`. Same in B. The cycle stays in the dependency graph

but doesn't break at load time because attribute access is deferred." 4. "Fix three — function-scope import. The line `from B import thing` moves inside the specific function that needs it. Each call does a `sys.modules` lookup (fast after first call). Works as a quick fix but smells; if I see ten function-scope imports in one module, I extract." 5. "For typing — `if TYPE_CHECKING:` is the dedicated tool. `from typing import TYPE_CHECKING` then `if TYPE_CHECKING: from B import BClass`. The block is skipped at runtime so no cycle; mypy still sees the import. Combined with `from __future__ import annotations` for the string-annotation form, type-only circulars are solved."

Edge cases to mention: - Sometimes the cycle isn't directly visible — A imports `framework` which imports B which imports A. The restructure may need to break something in `framework`, not in A or B directly. - For Django, circular model imports are common and the framework's `apps.get_model('app', 'Model')` lazy lookup is the official escape hatch. - The fix can hide a smell. If module A really shouldn't know about module B, breaking the cycle without removing the dependency just papers over a bigger problem.

What NOT to say: - "Just split the file." Useful sometimes but not a universal answer — splitting can move the cycle around without removing it. - "Use `globals()` to defer." Hacky and obscure; function-scope imports are the readable equivalent.

Common follow-up: "How do you decide between extracting versus deferring?" — Smell test. If A and B share a real concept (a model, a config), extract it. If A and B genuinely need to call each other and the concept is "they're peers," defer. Extract makes the architecture cleaner; defer accepts the cycle as a known shape.

```
# Cycle: orders.py ↔ payments.py
# orders.py uses Payment from payments
# payments.py uses Order from orders

# === Fix 1 – extract to models.py ===
# models.py
class Order: ...
class Payment: ...

# orders.py
from .models import Order, Payment

# payments.py
from .models import Order, Payment

# === Fix 2 – import module, late access ===
# orders.py
import payments
def get_payment(order): return payments.Payment.find(order.id)

# payments.py
import orders
def get_order(payment): return orders.Order.find(payment.order_id)

# === Fix 3 – function-scope import ===
# orders.py
def get_payment(order):
    from .payments import Payment          # deferred; cycle resolved by call time
    return Payment.find(order.id)
```

Q24 · **TYPE_CHECKING** for typing-only circulars

Tags: Difficulty: Medium · Asked at: Postman, Microsoft GCC, Razorpay · Topic: Circular Imports

The question (interviewer's verbatim phrasing):

I have a circular import that's only used for type hints. How do I keep the hints without the runtime cycle?

The 30-second answer: Use `from typing import TYPE_CHECKING` and put the circular import inside `if TYPE_CHECKING:`. The constant is `False` at runtime but `True` for type checkers (mypy, pyright). At runtime the block is skipped — no cycle. For the actual annotations to evaluate without the import being executed, either use string annotations (`"OtherClass"`) or `from __future__ import annotations` which makes ALL annotations strings.

Step-by-step thought process: 1. "PEP 484 introduced `typing.TYPE_CHECKING` as a constant — runtime value is `False`, type checkers treat it as `True`. So `if TYPE_CHECKING: import x` is a runtime no-op but visible to mypy." 2. "Without `from __future__ import annotations`, annotations are evaluated at function-definition time. So `def f(o: Order) -> Payment:` needs `Order` and `Payment` to be defined at that line. If they came from inside a `TYPE_CHECKING` block, the names don't exist at runtime — `NameError`." 3. "Two ways out — quote the annotations (PEP 484 style): `def f(o: 'Order') -> 'Payment':`. The strings are stored on `__annotations__` but not evaluated at definition time. mypy resolves them; runtime ignores them unless something calls `typing.get_type_hints()`." 4. "Or — PEP 563 + PEP 649 — `from __future__ import annotations` at the top of every file. This makes ALL annotations stringified automatically. Cleaner because you don't have to remember which annotations need quotes." 5. "Pattern combo — `from __future__ import annotations` at top, `if TYPE_CHECKING: from .other import Other` for the cycles. All annotations are strings; mypy resolves them via the type-checking import; runtime imports nothing extra. This is the modern idiom in 3.10+."

Edge cases to mention: - Code that calls `typing.get_type_hints(my_func)` at runtime evaluates the strings — if the imports aren't actually available at that moment, you get a `NameError`. Pydantic, FastAPI, dataclasses all do this; the `TYPE_CHECKING` trick fails for them. Workaround — use the real import behind a deferred mechanism. - Python 3.13 was supposed to make `from __future__ import annotations` the default (PEP 649) but deferred. Don't rely on it; keep the explicit future-import. - Forward references — even without circular imports, you sometimes need to reference a class defined later in the same file. Strings work for that case too.

What NOT to say: - "Just use Any for the type." Defeats the purpose of typing and loses checker information. - "Move all imports inside functions." Works but tedious — `TYPE_CHECKING` is the targeted tool.

Common follow-up: "What if my Pydantic model needs the annotation to resolve at runtime?" — Then `TYPE_CHECKING` doesn't help — Pydantic calls `get_type_hints` and needs the real class. Either restructure to break the runtime cycle, or use Pydantic's `from __future__ import annotations` support with explicit `model_rebuild()` calls.

```
# orders.py – type-only reference to Payment
from __future__ import annotations
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from .payments import Payment    # mypy sees it; runtime skips

class Order:
    id: int
    def latest_payment(self) -> Payment | None:
        # The string 'Payment' is what mypy resolves;
        # at runtime nothing tries to evaluate the annotation
        ...

# payments.py – type-only reference to Order
from __future__ import annotations
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from .orders import Order

class Payment:
    order_id: int
    def order(self) -> Order:
        from .orders import Order    # actual runtime use → function-scope
        return Order.find(self.order_id)
```

Q25 · `from __future__ import annotations`

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft GCC · Topic: Circular Imports

The question (interviewer's verbatim phrasing):

What does `from __future__ import annotations` do? Why would you put it at the top of every file?

The 30-second answer: PEP 563 makes all annotations into strings — they're not evaluated when the function or class is defined. Instead, they're stored as strings on `__annotations__`, and tools like mypy or `typing.get_type_hints()` resolve them later. The win — you can use names that haven't been imported yet (circular type references), forward references without quotes, and faster module load because annotation expressions never run.

Step-by-step thought process: 1. "By default (in 3.10 and 3.11), annotations are full Python expressions evaluated at function-definition time. So `def f(x: Order) -> Payment:` requires `Order` and `Payment` to be in scope at that line." 2. " `from __future__ import annotations` opts the

module into PEP 563 semantics — every annotation becomes a string stored on `__annotations__`. The expression is never evaluated by the interpreter." 3. "The win for cycles — type-only references to other modules can be inside a `if TYPE_CHECKING:` block without breaking annotations. The annotation never tries to resolve the name at runtime." 4. "The win for forward references — `def add_node(self, n: Node) -> Node:` inside the `Node` class body. Without future-annotations, you need quotes around `Node` because the name isn't bound yet. With future-annotations, it just works." 5. "The cost — frameworks that read annotations at runtime (Pydantic, FastAPI, attrs, dataclasses for some uses) need to call `typing.get_type_hints()` which evaluates the strings. If the names aren't importable at that moment, `get_type_hints` raises. Most modern frameworks handle this; some don't. Test."

Edge cases to mention: - PEP 563 (string annotations) and PEP 649 (lazy evaluation via descriptors) compete for what 3.13+ should default to. As of 2026 / Python 3.12 the future-import is still opt-in; behaviour from 3.13 onward may shift, watch the release notes. - `inspect.signature()` and similar runtime introspection see the strings, not the resolved types — code that programmatically inspects function signatures needs `get_type_hints()` to resolve them. - Class variables typed as `ClassVar[X]` and similar still work — `ClassVar` is resolved by the type checker, not at runtime.

What NOT to say: - "It's deprecated, don't use it." Not deprecated — it's still the recommended idiom for circular-typing avoidance in 3.10–3.12. - "It changes runtime behaviour for non-annotation code." It only affects annotation expressions, nothing else.

Common follow-up: "Show me a Pydantic model that breaks with future-annotations." — A model with a forward reference to another model defined later. `class A(BaseModel): b: B` where `B` is defined after `A`. Pydantic v2 calls `get_type_hints` and fails if `B` isn't importable at the call site. Fix — call `A.model_rebuild()` after `B` is defined.


```
# Without future annotations (Python 3.10/3.11 default)
class Order:
    def get_next(self) -> "Order":          # quotes required for self-reference
    ...

# With future annotations
from __future__ import annotations

class Order:
    def get_next(self) -> Order:            # no quotes needed
    ...

# Circular-type case
# orders.py
from __future__ import annotations
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from .payments import Payment

class Order:
    def latest_payment(self) -> Payment | None:  # OK – annotation is a string
        from .payments import Payment          # actual use deferred
        return Payment.find_by_order(self.id)
```

Section D — `pyproject.toml` & Modern Packaging (Q26–Q35)

Ten questions on PEP 517/518/660 packaging — `pyproject.toml`, build backends, editable installs, dependency groups, and the build-vs-runtime distinction. The 2024–2026 window has stabilized this area dramatically — `pyproject.toml` is now the canonical metadata location and `setup.py` is legacy. Interviewers at Razorpay, Postman, and especially infra-leaning teams test whether you've kept up.

Q26 · `pyproject.toml` — what it is and what's in it

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Cred, Microsoft GCC · Topic: Packaging

The question (interviewer's verbatim phrasing):

What is `pyproject.toml` and what does the `[project]` table contain?

The 30-second answer: `pyproject.toml` is the standardized project metadata file introduced by PEP 518 (for build-system declaration) and PEP 621 (for project metadata). The `[build-system]`

table tells pip how to build the project; the `[project]` table holds name, version, dependencies, Python version requirements, authors, license, and entry points. It replaces `setup.py` for static metadata; `setup.cfg` is also being phased out in favour of `pyproject.toml`.

Step-by-step thought process: 1. "Three relevant PEPs — PEP 518 (2016) introduced `pyproject.toml` and the `[build-system]` table, giving build tools a standard place to declare requirements. PEP 621 (2020) standardized the `[project]` table for metadata. PEP 660 (2021) standardized editable installs through the new system." 2. " `[build-system]` is the bootstrap section. `requires = ['setuptools>=61', 'wheel']` tells pip what to install before building. `build-backend = 'setuptools.build_meta'` names the build entry point. pip uses this to create an isolated build environment, install the requires, then call the backend." 3. " `[project]` is the static metadata. `name`, `version`, `description`, `requires-python = '>=3.11'`, `dependencies = ['requests>=2.31']`, `authors = [{name = '...', email = '...'}]`, `license = {text = 'MIT'}`, and optionally `[project.scripts]` for console entry points." 4. "Build backends — `setuptools.build_meta`, `hatchling`, `flit_core.buildapi`, `poetry.core.masonry.api`. They all consume `pyproject.toml` and produce wheels. Pick based on team preference; setuptools and hatchling are the most common in 2025/2026." 5. "For consumers, none of this matters. `pip install mypkg` works the same regardless of build backend. The choice matters to the package author — how easy is it to add plugins, how well does it cache builds, what's its dependency footprint."

Edge cases to mention: - `[project.urls]` is a free-form table for homepage / repo / docs links. Standardized keys help discoverability on PyPI. - `dynamic = ['version']` declares that the version comes from outside the static table — typically from a `__version__` constant or a VCS tag. Backend-specific config in `[tool.setuptools.dynamic]` or `[tool.hatch.version]` defines where. - `setup.py` is still allowed but increasingly limited — it's for custom build logic. Pure-Python projects rarely need it now.

What NOT to say: - " `pyproject.toml` replaces requirements.txt." Different files — `pyproject.toml` is the package's own dependency declarations; `requirements.txt` is for application-level pin lists. - "It's just a TOML version of setup.py." Misses the build-system isolation and the move from imperative to declarative metadata.

Common follow-up: "What's the minimum `pyproject.toml` you can ship?" — `[build-system]` with `requires` + `build-backend`, and `[project]` with at least `name`, `version`, and `description`. That's the smallest valid file for a publishable package.

```
# pyproject.toml – minimal modern Python package
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "myapp"
version = "0.1.0"
description = "Short one-line description."
readme = "README.md"
requires-python = ">=3.11"
license = { text = "MIT" }
authors = [
    { name = "Your Name", email = "you@example.com" },
]
dependencies = [
    "requests>=2.31",
    "click>=8.1",
]

[project.optional-dependencies]
dev = ["pytest>=8.0", "mypy>=1.10"]

[project.scripts]
myapp = "myapp.cli:main"

[project.urls]
Homepage = "https://github.com/you/myapp"
Repository = "https://github.com/you/myapp.git"
Documentation = "https://myapp.readthedocs.io"
```

Q27 · PEP 517 / 518 — build backends

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft GCC, Goldman · Topic: Packaging

The question (interviewer's verbatim phrasing):

What does PEP 517 do? What's the difference between setuptools, hatchling, and poetry as build backends?

The 30-second answer: PEP 517 standardized the interface between a project's build system and the installer (pip). The build-system independence lets you choose `setuptools.build_meta`, `hatchling.build`, `poetry.core.masonry.api`, `flit_core.buildapi`, or others — pip calls the backend through the standard hooks regardless of which one you picked. Setuptools is the

legacy default and has the deepest ecosystem; hatchling and flit are simpler newer backends; poetry-core is poetry's own backend (poetry is the broader workflow tool).

Step-by-step thought process: 1. "Before PEP 517, pip ran `python setup.py install` — meaning the build was tied to setuptools-isms and ran in your current environment (no isolation). PEP 518 introduced `[build-system].requires` ; PEP 517 standardized the hooks the backend must implement (`build_wheel` , `build_sdist`)." 2. "With PEP 517, pip creates a clean virtual environment, installs the `requires` , imports the named `build-backend` , calls its hooks. The result is reproducible builds independent of the host environment." 3. "Setuptools — original, ubiquitous. Supports compiled C extensions, deep plugin ecosystem, mature `setup.cfg` and `pyproject.toml` configs. Verbose for simple projects but battle-tested. Default for SciPy, Django, requests, basically everything older." 4. "Hatchling — modern, minimalist. From the Hatch project. Single-file `pyproject.toml` config, fast, good defaults. Increasingly the default for new projects. Adopted by attrs, FastAPI, many newer libraries." 5. "Poetry-core — used by Poetry-managed projects. If your team uses Poetry as the workflow tool, you also use poetry-core as the build backend. Standalone use is rare." 6. "Flit — extra minimalist, designed for pure-Python single-module packages. Tiny config. Common for stdlib-adjacent utilities."

Edge cases to mention: - Switching backends is mechanical — change `build-backend` in `[build-system]` and the relevant `[tool.X]` config section. The package metadata stays in `[project]` so consumers don't see the difference. - Some backends offer features the others don't — Hatchling has built-in version-from-VCS; setuptools requires plugins. - For pure-Python with no compiled extensions, all backends produce similar wheels. Differences matter for C extensions, build customization, and bundled assets.

What NOT to say: - "Setuptools is deprecated." Not deprecated — still the most widely used backend. Newer projects often pick hatchling for cleanliness, but setuptools is alive. - "You have to use poetry to use pyproject.toml." False — pyproject.toml is the universal metadata file; poetry is one of many tools that consume it.

Common follow-up: "How do you know which build backend a package is using?" — Read its `pyproject.toml` 's `[build-system].build-backend` field. Or look at the published wheel's metadata — the `Generated-By` header names the backend.

```
# Setuptools backend (legacy default, most ubiquitous)
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[tool.setuptools.packages.find]
where = ["src"]

# Hatchling backend (modern, minimal)
[build-system]
requires = ["hatchling>=1.18"]
build-backend = "hatchling.build"

[tool.hatch.version]
path = "src/myapp/__init__.py"

# Flit backend (simplest, pure-Python only)
[build-system]
requires = ["flit_core>=3.9"]
build-backend = "flit_core.buildapi"

# Poetry backend (only with poetry workflow)
[build-system]
requires = ["poetry-core>=1.7"]
build-backend = "poetry.core.masonry.api"
```

Q28 · `pip install -e .` — editable installs and PEP 660

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft GCC, Cred · Topic: Packaging

The question (interviewer's verbatim phrasing):

What does `pip install -e .` do, and how does it differ from `pip install .`?

The 30-second answer: `pip install .` builds a wheel and installs it — the installed package is a snapshot, isolated from your source directory. `pip install -e .` is an editable install — it links the package directly to your source so edits in the source tree take effect immediately without re-installing. PEP 660 (2021) standardized editable installs for non-setuptools backends; before that, editable installs only worked with setuptools-shipped `develop` mode.

Step-by-step thought process: 1. "Regular install — *pip builds the wheel via the backend, copies the wheel contents into site-packages, edits there are isolated. Re-install needed to pick up source changes.*" 2. "Editable install — *pip arranges for the installed package to point at your source directory. With setuptools, this used to mean dropping a `.pth` file in site-packages that added your source to `sys.path`. With PEP 660, the standard is that the backend ships an editable wheel that contains shims*

forwarding imports to the source tree." 3. "You can edit your code, run your tests, see the change. Even better — if your editor is the same window, you save the file and your next `python -c 'import mypkg; mypkg.foo()'` already reflects the change." 4. "Editable installs are the standard dev workflow. `git clone` → `python -m venv .venv` → `pip install -e .[dev]` and you're set up. The `[dev]` activates the optional dependency group named `dev` in `[project.optional-dependencies]`." 5. "Pre-PEP-660 mode (`python setup.py develop`) is deprecated. Modern is `pip install -e .` always, and as long as your build backend implements PEP 660 (which all major ones do as of 2024), it just works."

Edge cases to mention: - C extensions in editable installs need to be recompiled when you change them — `pip install -e .` doesn't auto-rebuild compiled code. Use `pip install -e . --no-build-isolation` plus a manual rebuild step, or `meson-python` for proper incremental compile. - Adding new top-level packages to a `src/` layout under editable install used to require re-running pip install (so the new package became visible to the shim). Modern backends handle this transparently. - Editable installs of namespace packages used to be fragile — improved drastically in 2023/2024 with PEP 660 implementations across backends.

What NOT to say: - "`-e` is the same as `--editable=True`." It's `--editable`, a flag, not a kv arg. - "Editable installs are slower than regular." Same speed at runtime; the install step is faster because no copy happens.

Common follow-up: "What goes wrong if you forget the `-e` flag?" — Source edits don't take effect. New developers waste hours adding `print()` statements that don't appear because the installed copy is stale. First-principles fix — make `pip install -e .[dev]` the README's only setup instruction.

```
# Regular install – snapshot, source edits do NOT take effect
pip install .

# Editable install – source edits take effect immediately
pip install -e .

# Editable with dev extras
pip install -e .[dev]

# Editable from a git URL
pip install -e git+https://github.com/you/mypkg.git#egg=mypkg

# Check what got installed
pip show mypkg
# Location: /path/to/source/src/mypkg      ← editable, points at source
# Location: /path/to/.venv/lib/python3.12/site-packages/mypkg ← regular install
```

Q29 · `pip install --user` versus `venv`**Tags:** Difficulty: Easy · Asked at: TCS, Postman, Razorpay · Topic: Packaging**The question (interviewer's verbatim phrasing):**

What's the difference between `pip install --user mypkg` and installing inside a `venv`?
Which is the right default?

The 30-second answer: `pip install --user` installs into the user's site-packages (`~/.local/lib/...` on Linux, similar on macOS). It's still shared across all your Python projects using that interpreter version. A `venv` installs into a per-project directory you created with `python -m venv`. Default — always use a `venv`. `--user` was the old escape hatch when system Python was the only option; modern PEP 668 environments now disallow system-wide `pip` installs entirely on managed distributions.

Step-by-step thought process: 1. "Three install locations exist — system-wide (`/usr/lib/python3.12/site-packages`), user (`~/.local/lib/python3.12/site-packages`), and `venv` (`/path/to/.venv/lib/python3.12/site-packages`). `pip` can target any of them with the right flags." 2. "System-wide is dangerous. Editing `/usr/lib` packages can break your OS — on Ubuntu, `apt-get install python3-requests` and `pip install requests` may fight. Avoid this; many distros now block it via PEP 668's `EXTERNALLY-MANAGED` marker." 3. "`--user` installs to the user-level site-packages. Still shared across projects — install `pandas 1.5` for project A, and project B also sees `pandas 1.5` unless project B has its own `venv`. Acceptable for hobby scripts, painful at the moment two projects need different versions." 4. "`venv` — `python -m venv .venv && source .venv/bin/activate && pip install mypkg`. Self-contained, project-local, throwable. The `pandas` in `projectA/.venv` and `projectB/.venv` are independent. This is the universal default." 5. "`pipx` is the right tool for installing applications (not libraries) that need their own isolated environment but should be usable as a global command — `pipx install black` puts `black` on your `PATH`, in its own private `venv`. Solves the "I want a tool, not a library" use case that `--user` used to half-solve."

Edge cases to mention: - PEP 668 — distributions like Debian, Ubuntu 23.04+, Fedora 38+ block `pip` from installing into the system Python by default. The error directs you to use a `venv` or `--break-system-packages` (almost never the right answer). - `--user` interacts with `venvs` — inside an active `venv`, `pip install --user` still installs to user-level, ignoring the `venv`. Almost always a mistake. - macOS Homebrew Python similarly externally-manages now, so `pip install x` without a `venv` tends to be refused.

What NOT to say: - "Just use `sudo pip install`." NEVER — corrupts the system Python, breaks the package manager's view of what's installed, may break OS tools. - "`--user` is good enough." Fine for one-off scripts; falls apart the moment you have two projects with conflicting dependencies.

Common follow-up: "How do you isolate dependencies for a single CLI tool you want available system-wide?" — `pipx install toolname` . `pipx` creates a dedicated venv, links the tool's entry-point script onto your PATH, and never lets it pollute other Python environments.

```
# Wrong – system-wide install (now blocked on most distros)
pip install requests
# error: externally-managed-environment (PEP 668)

# Old escape hatch – user install
pip install --user requests
# Installs to ~/.local/lib/...
# Shared across all your projects – fragile at scale

# Correct default – venv per project
python -m venv .venv
source .venv/bin/activate      # Linux/macOS
.venv\Scripts\activate        # Windows
pip install requests
# Installs to .venv/lib/... – project-local

# For CLI tools you want global access to
pipx install black             # creates ~/.local/pipx/venvs/black/
pipx install ruff
pipx list
```

Q30 · `[project.dependencies]` versus `[project.optional-dependencies]`

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Microsoft GCC · Topic: Packaging

The question (interviewer's verbatim phrasing):

What's the difference between `dependencies` and `optional-dependencies` in `pyproject.toml` ? When is each one used?

The 30-second answer: `dependencies` is the mandatory list — `pip` installs every package in this list when anyone runs `pip install mypkg` . `optional-dependencies` is a dict of named groups (extras) like `dev` , `test` , `excel` — they're installed only when the consumer asks: `pip install mypkg[dev]` . Use `optional-dependencies` for capabilities that not every consumer needs, like dev tooling, alternative backends, or large add-ons.

Step-by-step thought process: 1. "`dependencies = ['requests>=2.31', 'click>=8.1']` — `pip` installs both every time someone installs `mypkg`. These are the minimum needed for the package's basic functionality." 2. "`[project.optional-dependencies]` is a table. Each key names a group;

the value is a list. `dev = ['pytest', 'mypy', 'ruff']` defines the `dev` group. Consumers activate it with `pip install mypkg[dev]`." 3. "Two main use cases. One — dev tooling. `pip install mypkg[dev]` for contributors who'll run tests; `pip install mypkg` for end users. End users don't get 200 MB of pytest plugins." 4. "Two — feature flags. `pip install mypkg[excel]` pulls in `openpyxl` for the Excel export feature; `pip install mypkg[pdf]` pulls in `reportlab`. Users who don't need Excel skip the dependency. The package code handles the missing dependency gracefully — `try/except ImportError` pointing at the install command." 5. "Multiple groups can be activated — `pip install mypkg[dev,docs,test]` installs all three. A common convention is to have an `all` group that combines them all for completeness: `all = ['mypkg[dev]', 'mypkg[docs]', 'mypkg[test]']`."

Edge cases to mention: - PEP 735 (Python 3.13 / pip 24.1+ era) introduced "dependency groups" — a NEW table `[dependency-groups]` for groups not tied to install extras. Useful for dev/test/build groups that aren't really package features but workflow conveniences. Tooling support is rolling out. - An extra group can depend on a specific minimum version of the same package — `extra-pdf = ['mypkg==1.2', 'reportlab>=4']` would pin the package version, but the convention is to leave that to the consumer's pin file. - Some package indexes (private repos) handle extras differently — most respect them, but exotic mirrors may not.

What NOT to say: - "Just put everything in dependencies, simpler." Forces 200 MB of dev tooling on end users. - "Optional dependencies are deprecated, use dependency groups." Both coexist as of 2026 — extras are still standard for "install-time feature flags."

Common follow-up: "What's the right place for `pytest` and `mypy` — dependencies, optional-dependencies, or somewhere else?" — Optional dependencies (the `[dev]` extra) is the legacy answer and still works fine. PEP 735's `[dependency-groups]` is the modern answer if your tooling supports it. Either way — NOT in `dependencies`; you don't want users installing `pytest` just to use your library.

```
# pyproject.toml – dependency layout
[project]
name = "myapp"
version = "1.0.0"
dependencies = [
    "requests>=2.31",      # always installed
    "click>=8.1",          # always installed
    "pydantic>=2.5",        # always installed
]

[project.optional-dependencies]
# Dev tooling – activate with: pip install -e .[dev]
dev = [
    "pytest>=8.0",
    "mypy>=1.10",
    "ruff>=0.5",
]
# Feature flag – activate with: pip install myapp[excel]
excel = [
    "openpyxl>=3.1",
    "xlsxwriter>=3.2",
]
# Combined group – activate with: pip install myapp[all]
all = ["myapp[dev]", "myapp[excel]"]

# PEP 735 – modern alternative for non-feature groups
[dependency-groups]
test = ["pytest>=8.0", "pytest-cov"]
docs = ["sphinx>=7", "myst-parser"]
```

```
# Inside myapp/excel_export.py – handle missing optional dep
def to_excel(df, path):
    try:
        import openpyxl
    except ImportError as e:
        raise RuntimeError(
            "Excel export requires the [excel] extra. Install with:\n"
            "  pip install myapp[excel]"
        ) from e
    df.to_excel(path, engine="openpyxl")
```

Q31 · Dependency groups (PEP 735)

Tags: Difficulty: Hard · Asked at: Razorpay, Microsoft GCC, Postman · Topic: Packaging

The question (interviewer's verbatim phrasing):

What are dependency groups in `pyproject.toml` ? How do they differ from optional dependencies?

The 30-second answer: PEP 735 (accepted 2024, supported in pip 24.1+) added a new table `[dependency-groups]` for declaring named lists of packages that are NOT tied to the installable package's features. They're for development workflow — tests, docs, lint, type-check. Activation is `pip install --group test` (or via a tool like uv that consumes them). Optional dependencies (`[project.optional-dependencies]`) become "feature extras" again — meant for installable package features users may opt into, like `[excel]`.

Step-by-step thought process: 1. "Before PEP 735, both feature flags (`pip install mypkg[excel]`) and dev tooling (`pip install mypkg[dev]`) lived in `optional-dependencies`. Worked but mixed two concepts — features advertised to package consumers, and workflow groups meant for contributors only." 2. "`[dependency-groups]` solved that split. The table holds named lists. `test = ['pytest', 'pytest-cov']` declares a group. Tools install it via `pip install --group test` or `uv pip install --group test` — separate flag from `--extra`, signalling intent." 3. "The cleaner model — `optional-dependencies` is for the package itself (features users opt into), `dependency-groups` is for the project (workflow tooling). End users see the former; contributors use the latter." 4. "Composability — a group can reference another group:
`dev = [{include-group = 'test'}, {include-group = 'lint'}]`. Lets you build hierarchical workflow groups without copy-pasting." 5. "Tooling adoption — pip 24.1+, uv from the start, hatch from 1.10+. Some older CI/CD images don't yet support `--group`. If your tooling doesn't, fall back to `optional-dependencies` and label the dev groups by convention."

Edge cases to mention: - A group can declare a Python version requirement on individual entries — `pytest>=8; python_version >= '3.10'`. Same syntax as `dependencies`. - Unlike optional-dependencies, dependency groups CANNOT be installed via the `pkg[group]` syntax — it's `--group` only, by design. - The proposal had an explicit goal of replacing patterns like Poetry's `[tool.poetry.group.dev]` with a standard, tool-agnostic spelling.

What NOT to say: - "Dependency groups are just renamed optional dependencies." Misses the semantic split between features (extras) and workflow (groups). - "They're a Poetry thing." PEP 735 is a Python standard now, not Poetry-specific. Poetry's older syntax inspired it but the spec is independent.

Common follow-up: "My team uses Poetry — should we migrate?" — Poetry 1.8+ understands `[dependency-groups]` reading and writing, but the migration is mechanical. You'd flatten `[tool.poetry.group.dev.dependencies]` into `[dependency-groups].dev`. Worth doing once your CI runners support pip 24.1+.

```
# pyproject.toml – both tables present
[project]
name = "myapp"
version = "1.0.0"
dependencies = ["requests>=2.31"]

# Feature extras for end users
[project.optional-dependencies]
excel = ["openpyxl>=3.1"]
pdf = ["reportlab>=4"]

# Workflow groups for contributors (PEP 735)
[dependency-groups]
test = [
    "pytest>=8.0",
    "pytest-cov>=4.1",
    "hypothesis>=6.100",
]
lint = [
    "ruff>=0.5",
    "mypy>=1.10",
]
docs = [
    "sphinx>=7.3",
    "myst-parser>=3.0",
]
# Composed group
dev = [
    {include-group = "test"},
    {include-group = "lint"},
    {include-group = "docs"},
]
```

```
# Install workflow groups
pip install --group test
pip install --group dev          # the composed group

# Install feature extras (consumer side)
pip install myapp[excel]
pip install myapp[excel,pdf]
```

Q32 · `setup.py` versus `setup.cfg` versus `pyproject.toml`

Tags: Difficulty: Medium · Asked at: Razorpay, Goldman, Postman · Topic: Packaging

The question (interviewer's verbatim phrasing):

A project I'm joining has `setup.py`, `setup.cfg`, and `pyproject.toml`. Which one is authoritative?

The 30-second answer: Modern is `pyproject.toml` for everything possible. `setup.cfg` was an intermediate step (static config in INI format); its options have moved into `pyproject.toml`'s `[project]` and `[tool.setuptools]` tables. `setup.py` is now for custom imperative build logic only — most projects don't need one. If all three files exist, `pyproject.toml` wins for metadata; `setup.py` runs for custom build steps the backend can't express declaratively.

Step-by-step thought process: 1. "Historical layers — `setup.py` was the original. `setuptools` took it over from `distutils`. `setup.cfg` (around 2017) let people move metadata out of imperative Python code into static INI. `pyproject.toml` (PEPs 518/621, 2016-2020) made TOML the modern standard." 2. "Today, `pyproject.toml` should hold ALL static metadata — name, version, dependencies, scripts, classifiers. `setup.cfg` is redundant once `pyproject.toml` is complete; many projects deleted it in 2023-2024." 3. " `setup.py` is now opt-in — needed only for custom imperative build logic (compiling C, generating files at build time, conditional dependencies based on platform inspection). If you have neither, your build is fully declarative; if you need it, keep it minimal." 4. "Reading order — `pip` consults `pyproject.toml` for the build system. The build backend (`setuptools`, `hatchling`, etc.) reads `pyproject.toml`, then possibly `setup.cfg`, then possibly executes `setup.py`. `Setuptools` merges them, with later sources overriding earlier ones." 5. "When you join a project — the modern move is consolidate. Move `setup.cfg`'s content into `pyproject.toml`, delete `setup.cfg`, leave `setup.py` only if there's custom build logic. PEP 621 metadata in `pyproject.toml` is the single source of truth from there on."

Edge cases to mention: - Some projects keep a one-line `setup.py` (`from setuptools import setup; setup()`) for compatibility with very old `pip` versions or tools that shell out to `python setup.py`. Mostly cargo-cult; modern `pip` doesn't need it. - `setup.cfg` is still officially supported by `setuptools`; the deprecation is informal — there's no plan to remove it. Just no reason to use it for new projects. - If three files declare conflicting names or versions, behaviour is undefined and frequently broken. Pick one source of truth.

What NOT to say: - "Delete `setup.py` immediately." Sometimes there's a real custom build step — check first. - " `setup.cfg` is deprecated." It's not — just no longer recommended for new projects.

Common follow-up: "What custom logic actually needs `setup.py` today?" — C extension build steps (`Extension(...)` objects), platform-specific file inclusion, conditional dependencies (sometimes — PEP 508 markers cover most cases), build-time code generation. Most pure-Python projects need none of this.

```
# Modern minimal – only pyproject.toml
# pyproject.toml exists, no setup.py, no setup.cfg

# When you need setup.py (rare, custom build logic)
# setup.py
from setuptools import setup, Extension

setup(
    ext_modules=[
        Extension(
            "myapp._native",
            sources=["src/myapp/_native.c"],
            extra_compile_args=["-O3"],
        ),
    ],
)
# pyproject.toml still holds the metadata; setup.py adds the C build

# Legacy combo (often seen in older projects)
# pyproject.toml – minimal, just [build-system]
# setup.cfg – metadata, package finding, optional deps
# setup.py – shim `setup()` call

# Modern consolidation
# Just pyproject.toml – everything in [project] and [tool.setuptools]
```

Q33 · Source distributions versus wheels

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft GCC, Goldman · Topic: Packaging

The question (interviewer's verbatim phrasing):

What's the difference between a sdist and a wheel? What does `pip install mypkg` actually download?

The 30-second answer: A wheel (`.whl`) is a pre-built binary distribution — already compiled, ready to copy into site-packages, install is fast. A source distribution (`.tar.gz` named `sdist`) is the project source; pip downloads it, runs the build backend to produce a wheel, then installs the wheel. `pip install mypkg` prefers wheels — it falls back to sdists only when no matching wheel exists for your interpreter and platform. Building from sdist requires a working build environment, which is why wheels are strongly preferred for binary deps like NumPy.

Step-by-step thought process: 1. "Two distribution formats live on PyPI. sdist is a tarball of the source. Wheel is a zip with a `.whl` extension — pre-built, contains the importable package files plus metadata." 2. "Pure-Python wheels are universal — same wheel works on Linux, macOS, Windows, any Python

version they declare. The filename encodes the support — `mypkg-1.0-py3-none-any.whl`." 3. "Compiled wheels are platform-specific. NumPy 1.27 ships dozens of wheels — `numpy-1.27-cp312-cp312-manylinux_2_17_x86_64.whl`, `numpy-1.27-cp312-cp312-win_amd64.whl`, etc. `pip` picks the right one based on your platform tags." 4. "pip install behaviour — first preference is a compatible wheel from the index. Fallback is the sdist plus local build. For pure-Python, both produce the same result. For C extensions, fallback can fail if you lack the C toolchain." 5. "manylinux — a set of compatibility tags (`manylinux2014`, `manylinux_2_28`, etc.) for Linux wheels that work across most distros. Authors build wheels against an older glibc to maximize the set of systems the wheel works on. This is why PyPI numpy wheels run on Ubuntu, RHEL, Alpine (mostly), and Amazon Linux."

Edge cases to mention: - Some projects ship only sdists (no wheel) — usually pure-Python with no build, but installation still requires the build backend to be installable. `setuptools`-based projects always need `setuptools` available. - `pip wheel mypkg` downloads the package and builds a wheel locally without installing — useful for caching or air-gapped environments. - `pip download --only-binary=:all: mypkg` refuses to fall back to sdist — fails if no wheel exists. Useful in production where you don't want surprise builds at install time.

What NOT to say: - "Wheels are just zip files." True technically, but misses the metadata, the install hooks, and the platform-tag machinery. - "sdist is the source code." Close but slightly off — it's a release-tagged snapshot of source, not a git checkout. Conventional `MANIFEST.in` controls what's included.

Common follow-up: "How do you build a wheel for your project locally?" — `python -m build` runs the build backend in an isolated environment and produces both an sdist and a wheel in `dist/`. Or `pip wheel . -w dist/` builds just the wheel.

```
# Inspect what pip would install
pip install --dry-run --report - mypkg

# Force install from sdist (skip prebuilt wheel)
pip install --no-binary=:all: mypkg

# Force install only from a wheel (no fallback build)
pip install --only-binary=:all: mypkg

# Build sdist + wheel locally
pip install build
python -m build
# Produces dist/mypkg-1.0.0.tar.gz (sdist)
# Produces dist/mypkg-1.0.0-py3-none-any.whl (wheel)

# Inspect a wheel
unzip -l dist/mypkg-1.0.0-py3-none-any.whl
```

Q34 · Version constraints — `~=` , `>=` , `==` , exclusion

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Postman · Topic: Packaging

The question (interviewer's verbatim phrasing):

What's the difference between `pandas>=2.0` , `pandas~2.0` , `pandas==2.0` , and `pandas>2.0,!=2.0.5` ? When do you use each?

The 30-second answer: `>=2.0` is "2.0 or anything newer" — open-ended, accepts 3.x. `~2.0` is "compatible release" — accepts 2.x but not 3.x; expands to `>=2.0,<3` . `==2.0` is exact pin — `2.0.0` only. `>2.0,!=2.0.5` is a compound — strictly greater than 2.0, but not exactly 2.0.5. Use `>=` for libraries (loose), `~` for medium-trust deps, `==` only in lock files / `requirements.txt` , and `!=` for specific known-bad versions.

Step-by-step thought process: 1. "PEP 440 defines all the version specifiers. The big ones — `==` , `!=` , `>=` , `<=` , `>` , `<` , `~` , plus the wildcard `==2.0.*` ." 2. "`>=2.0` is open. Says nothing about upper bound. Risk — if pandas 3 ships and breaks our usage, we pull in the breakage. Use only when you trust the package to follow semver and have actively tested forward-compatibility." 3. "`~2.0` is compatible release. Expands to `>=2.0,<3` . Picks up 2.x patches and minor updates but stops at the next major. Good middle ground when you trust semver but want a safety stop." 4. "`==2.0` is exact, `==2.0.*` accepts the patch-level (`2.0.3` matches `2.0.*`). Use exact pinning in `requirements.txt` for reproducible deployments; don't use exact in library `dependencies` because it forces incompatible pins on every consumer." 5. "Compound — `>2.0,!=2.0.5,<3` is "between 2.0 and 3, but not exactly 2.0.5." Useful when a specific patch version had a bug; pin around it. Don't overuse — long compound specs are unreadable."

Edge cases to mention: - Pre-release versions (alphas, betas, rcs) are excluded by default from version resolution. Use `--pre` to opt in. - Python version markers — `requests>=2.31; python_version>= '3.10'` only applies the dep on Python 3.10+. Useful for backward compatibility. - Tilde-equal acts differently for two-segment vs three-segment versions — `~2.0` is `>=2.0,<3` , `~2.0.0` is `>=2.0.0,<2.1` . Subtle.

What NOT to say: - "Always pin exact versions." Fine for applications, terrible for libraries — forces incompatible pins on every consumer of the library. - "`>=` is dangerous." Only if you don't trust the upstream — most well-maintained packages have stable APIs across minor versions.

Common follow-up: "What's the right specifier for a library that consumers will install alongside other libs?" — `>=` with a known-working minimum and no upper bound, OR `>=min,<next-major` if you've validated up to a specific major version. Avoid `==` in library deps; defer pinning to the application layer.


```
# Library pyproject.toml — loose, lower bounds only
[project]
dependencies = [
    "requests>=2.31",      # any 2.31+ works
    "pandas~=2.0",         # any 2.x — semver-trusting cap
    "numpy>=1.24,<2",      # bounded — known not to work with 2.x yet
    "rare-pkg>0,!0.5.2",   # avoid known-bad version
]

# Application requirements.txt — exact pin (after pip-compile or similar)
requests==2.32.3
pandas==2.2.2
numpy==1.26.4

# Python-version-conditional
[project]
dependencies = [
    "tomli>=2.0; python_version < '3.11'", # tomllib is stdlib from 3.11
]
```

Q35 · requirements.txt versus pyproject.toml

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred · Topic: Packaging

The question (interviewer's verbatim phrasing):

If `pyproject.toml` has dependencies, why does anyone still use `requirements.txt` ?

The 30-second answer: `pyproject.toml` 's `dependencies` declares what the package needs in loose terms — minimum versions, compatibility ranges. `requirements.txt` is for applications and pins exact versions of every transitive dependency for reproducible installs. The two coexist — libraries declare in `pyproject.toml` ; applications use a `requirements.txt` (often generated by `pip-compile` or `uv pip compile`) for deterministic deploys. Lock files are the modern alternative — `uv.lock` , `poetry.lock` , `pdm.lock` .

Step-by-step thought process: 1. "Two different use cases. `pyproject.toml` is for packages — the declaration of what you depend on, loose enough that consumers can resolve compatible versions for themselves." 2. " `requirements.txt` is for applications — a flat list of exact pins including transitive deps, so production installs are identical to what was tested in CI." 3. "Generation flow — write loose specs in `pyproject.toml` . Run `pip-compile pyproject.toml -o requirements.txt` (or `uv pip compile` for the faster modern tool). Output is `requirements.txt` with one line per package and the resolved exact version. Commit both to git." 4. "Lock files do the same job in a more structured way. `uv.lock` , `poetry.lock` , `pdm.lock` — they track exact versions of all transitive deps plus the resolution graph plus hashes for verification. Restore is deterministic — same lock file,

same install, every time." 5. "Modern workflow — `pyproject.toml` for the loose declaration, `uv.lock` (or equivalent) for the resolved state, no separate `requirements.txt`. The CI/CD uses `uv sync` (or `poetry install`) to install from the lock. `requirements.txt` remains common for legacy reasons and for situations where the deploy target only speaks pip."

Edge cases to mention: - `pip install -r requirements.txt` does NOT use `pyproject.toml`. They're independent install paths. - `pip-tools`'s `pip-compile` and `uv pip compile` produce the same kind of output but the latter is dramatically faster (sub-second versus tens of seconds). - `requirements-dev.txt` is a common pattern — separate locked file for dev tooling. Modern tools (uv, poetry) handle dev groups inside one lock file.

What NOT to say: - "`requirements.txt` is obsolete." Still in widespread use; many CI pipelines and Docker images target it. - "You only need one of them." For an application, you need both — `pyproject.toml` for declaration, `requirements.txt` (or lock file) for pinning.

Common follow-up: "How do you ensure `requirements.txt` matches `pyproject.toml`?" — Regenerate. Don't hand-edit `requirements.txt`; always run `pip-compile` or `uv pip compile` against `pyproject.toml`. Commit the regenerated file. CI checks it's up-to-date.

```
# pyproject.toml – loose, written by hand
[project]
dependencies = [
    "requests>=2.31",
    "click>=8.1",
    "pydantic>=2.5",
]
```

```
# Generate requirements.txt from pyproject.toml
uv pip compile pyproject.toml -o requirements.txt
# Or: pip-compile pyproject.toml --output-file=requirements.txt
```

```
# requirements.txt – generated, exact pins for reproducible installs
# Do not edit by hand. Generated by uv 0.4.0:
# uv pip compile pyproject.toml -o requirements.txt
annotated-types==0.7.0
certifi==2024.7.4
charset-normalizer==3.3.2
click==8.1.7
idna==3.7
pydantic==2.8.2
pydantic-core==2.20.1
requests==2.32.3
typing-extensions==4.12.2
urllib3==2.2.2
```

Section E — Virtual Environments & Tools (Q36–Q40)

Five questions on venvs, pip, pipx, and the 2024-era arrival of uv. The landscape has shifted sharply in the last 18 months — uv replaced pip-compile for most teams, dependency groups are getting first-class support, and PEP 668 changed how system Pythons behave. Interviewers test currency here.

Q36 · `venv` versus `virtualenv` versus `conda`

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Postman, Cred · Topic: Virtual Environments

The question (interviewer's verbatim phrasing):

What's the difference between `venv`, `virtualenv`, and `conda` environments? When would you pick each?

The 30-second answer: `venv` is stdlib (Python 3.3+) — creates a lightweight env that uses your system Python interpreter. `virtualenv` is a third-party predecessor that still gets used because it supports older Pythons, custom interpreter discovery, and slightly faster creation. `conda` is from the Anaconda ecosystem — it manages both Python packages AND non-Python binaries (CUDA, MKL, etc.) and supports multiple Python versions side-by-side. Default — `venv` for everyday Python projects; `conda` only when you need its binary ecosystem (data science with CUDA, R alongside Python).

Step-by-step thought process: 1. "`venv` ships with Python — `python -m venv .venv` creates a new env in `.venv/`. Uses your current Python interpreter version. Sufficient for 99% of Python projects." 2. "`virtualenv` is third-party — predates `venv` and survives because it has features stdlib `venv` lacks. Slightly faster creation, lets you specify a different Python interpreter to bootstrap from, supports Python 2 (rarely needed today)." 3. "`conda` is a fundamentally different beast. It's a binary package manager, not just for Python — it also handles C libraries, R, Julia. The environment includes the Python interpreter itself, so one machine can have envs with Python 3.10, 3.11, and 3.12 simultaneously without any of them being "system Python." " 4. "Picking — `venv` for ordinary web/backend work. `conda` for ML and scientific computing where you need CUDA-aware NumPy or PyTorch builds, or when you need a Python version you can't `apt install`. For pure-Python projects, conda is overkill and adds complexity." 5. "`uv venv` is the modern speed king — same conceptual env as `python -m venv`, but the uv binary creates it in milliseconds versus seconds. Adopt uv for the workflow speedup; it's a drop-in replacement for the venv-creation step."*

Edge cases to mention: - `venv --system-site-packages` makes the venv see system-level packages. Useful for using OS-installed binary packages without re-installing in the venv; usually a workaround for binary deps that don't pip-install easily. - conda envs don't activate with `source activate .venv/bin/activate` — they use `conda activate myenv`. Different mental model. -

Pyenv is yet another tool — manages multiple Python VERSIONS (not envs) on one machine. Common combo — pyenv + venv (pyenv to install Python 3.12, venv to make project envs).

What NOT to say: - "Conda is always slower." Faster for binary-heavy scientific stacks; slower for pure-Python. - "Just use the system Python." PEP 668 mostly blocks this now and it was a bad practice anyway.

Common follow-up: "What's the activation difference between venv and conda?" — venv writes shell scripts (`activate` , `activate.bat` , `activate.fish`) you source. Conda registers shell hooks and uses `conda activate name` from any cwd. Both work; the conda flow is more aware of which env is active.

```
# venv (stdlib)
python -m venv .venv
source .venv/bin/activate           # Linux/macOS
.venv\Scripts\activate             # Windows cmd
.venv\Scripts\Activate.ps1         # Windows PowerShell
which python                       # → /path/to/.venv/bin/python
deactivate

# virtualenv (third-party)
pip install virtualenv
virtualenv -p python3.12 .venv      # explicit interpreter version

# conda
conda create -n myenv python=3.12 numpy pandas
conda activate myenv
conda install -c conda-forge pytorch
conda deactivate

# uv (modern, fast)
uv venv .venv                       # creates in ms
source .venv/bin/activate
uv pip install requests             # 10-100x faster than pip
```

Q37 · How activation actually works

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft GCC · Topic: Virtual Environments

The question (interviewer's verbatim phrasing):

When you `source .venv/bin/activate` , what actually changes? Why does `which python` now point inside the venv?

The 30-second answer: `activate` is a shell script that modifies `PATH`, saves the old `PATH`, sets `VIRTUAL_ENV` to the venv directory, and changes the shell prompt. The key step is prepending `.venv/bin` to `PATH` so the shell finds `.venv/bin/python` before the system one. `deactivate` is a shell function that restores the original `PATH`. The Python interpreter inside the venv also has a slightly modified `site.py` that sets `sys.prefix` to the venv root, redirecting site-packages lookup.

Step-by-step thought process: 1. "The shell side — `activate` is a bash/zsh/fish/pwsh script (different file per shell). It does roughly: save the current `PATH` into `_OLD_VIRTUAL_PATH`, prepend `.venv/bin` to `PATH`, save the current prompt, set the new prompt prefix `(.venv)`, define a `deactivate` shell function." 2. "`deactivate` is the function that undoes all of the above — restores `PATH`, restores the prompt, unsets `VIRTUAL_ENV`, then unsets itself so it can't be called twice." 3. "The Python side — the venv directory contains a special `pyvenv.cfg` file. When `.venv/bin/python` starts, Python's startup code reads this file and adjusts `sys.prefix` and `sys.exec_prefix` to point at the venv. This causes `site-packages` lookup to resolve to `.venv/lib/python3.12/site-packages` rather than the system one." 4. "The interpreter binary inside `.venv/bin/python` is either a copy of the system Python (Windows default) or a symlink to it (Linux/macOS default). The "venv-ness" is determined by the `pyvenv.cfg` it sees on startup, not by the binary itself." 5. "You can actually skip activation entirely. `.venv/bin/python script.py` runs Python inside the venv. `.venv/bin/pip install x` installs into the venv. Activation is just a convenience — adds `.venv/bin` to `PATH` so you don't have to type the full path each time."

Edge cases to mention: - On Windows, activation in PowerShell requires the right execution policy. `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` is the common one-time fix. - Some IDEs and editors automatically activate the venv for terminal sessions they spawn. VS Code's Python extension does this; PyCharm has venv awareness too. - A venv with a deleted/moved interpreter breaks silently — `.venv/bin/python` becomes a dangling symlink. Re-create the venv.

What NOT to say: - "Activation changes the Python interpreter." It doesn't — the interpreter is selected by `PATH` lookup. The interpreter itself is just running. - "You must activate to use the venv." False — `.venv/bin/python` works fine without activation.

Common follow-up: "Does activation persist across shells or sessions?" — No. It's per-shell. New terminals start with the system Python. Tools like `direnv` or `autoenv` can auto-activate when you `cd` into a project directory.

```
# Look at what activate does
cat .venv/bin/activate | head -30
# Key lines:
# _OLD_VIRTUAL_PATH="$PATH"
# PATH="$VIRTUAL_ENV/bin:$PATH"
# export PATH
# VIRTUAL_ENV="$(dirname $(dirname $(readlink -f $0)))"
# export VIRTUAL_ENV

# Look at pyvenv.cfg
cat .venv/pyvenv.cfg
# home = /usr/bin
# include-system-site-packages = false
# version = 3.12.4

# Use the venv without activating
.venv/bin/python -c "import sys; print(sys.prefix)"
# /path/to/.venv

# Compare with system Python
/usr/bin/python -c "import sys; print(sys.prefix)"
# /usr
```

Q38 · `pipx` for tools versus `pip` for libraries

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred · Topic: Tools

The question (interviewer's verbatim phrasing):

What is `pipx` and why use it instead of `pip install --user black` ?

The 30-second answer: `pipx` installs Python applications into isolated environments and links their entry-point scripts onto your PATH. Each tool gets its own venv, so installing `black` doesn't pull in dependencies that conflict with `ruff` or `mypy` you'll install separately. `pip install --user black` puts black AND all its transitive deps into your user-level site-packages, which can collide with other tools you install the same way. Use `pipx` for any standalone CLI tool; use `pip-in-venv` for libraries.

Step-by-step thought process: 1. "`pipx install black` creates `~/.local/pipx/venvs/black/`, installs `black` into that private venv, then creates a symlink (or shim on Windows) at `~/.local/bin/black` pointing at the venv's executable. `black` is now on your PATH but isolated." 2. "Installing `ruff` next does the same in a separate venv — no shared dependencies, no conflict. `pipx list` shows what's installed; `pipx upgrade-all` updates everything; `pipx uninstall black` removes it cleanly." 3. "Comparison with `pip install --user` — all `--user`-installed packages share the same site-packages. If `black` needs `click<9` and another tool needs

`click>=9`, one of them breaks. `pipx` avoids this by giving each tool its own world." 4. "Use cases — formatters (`black`, `ruff`, `isort`), type checkers (`mypy`, `pyright`), CLIs (`httpie`, `pre-commit`, `aws-cli`, `pre-commit`). Anything that's run from the terminal and doesn't need to be imported by your project." 5. "Anti-use-case — don't `pipx` install your own libraries; use a `venv`. `pipx` is specifically for tools that get invoked via their command, not via `import`. If you need to `import mypkg` in your shell, that's a library, not a `pipx` candidate."

Edge cases to mention: - `pipx run blackd` runs a tool without installing it permanently — equivalent to `npx`. The tool is installed in a temporary `venv` and discarded after. - `pipx inject black mypkg-plugin` adds a package into an existing tool's `venv` — for tools with plugins (black plugins, `mypy` plugins). - `pipx` itself can be installed via `apt/brew/scoop` or via `python -m pip install --user pipx; python -m pipx ensurepath`.

What NOT to say: - "pipx is just renamed pip." No — different install model (per-tool `venv`), different use case (apps not libs). - "Use pipx for everything Python." Only for tools. Libraries belong in your project's `venv`.

Common follow-up: "What if I need a tool to be available across multiple projects with different Python versions?" — `pipx` installs each tool with whatever Python `pipx` itself used. That Python becomes the tool's runtime. Most tools are version-tolerant. For Python-version-pinned tools, install per-project inside the `venv` instead.

```
# Install pipx itself
python -m pip install --user pipx
python -m pipx ensurepath          # adds ~/.local/bin to PATH
# Restart shell

# Install tools
pipx install black
pipx install ruff
pipx install mypy
pipx install pre-commit
pipx install httpie

# List, upgrade, uninstall
pipx list
pipx upgrade black
pipx upgrade-all
pipx uninstall black

# Run a tool without installing (npx-style)
pipx run cookiecutter https://github.com/example/template

# Add a plugin to a pipx-installed tool
pipx inject mypy types-requests
pipx inject black black-macchiato

# Check where pipx installed something
pipx list
# venvs are in ~/.local/pipx/venvs/<toolname>/
# entry-points symlinked at ~/.local/bin/<toolname>
```

Q39 · uv — the 2024 Rust-based installer

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Microsoft GCC · Topic: Tools

The question (interviewer's verbatim phrasing):

What is `uv` and why is everyone switching to it?

The 30-second answer: `uv` is a Rust-written package and project manager from Astral (the team behind ruff). It replaces parts of the pip / virtualenv / pip-tools / pipx stack with a single fast binary — 10-100x faster than pip for most operations. `uv pip install` is a drop-in for `pip install`; `uv venv` creates a venv in milliseconds; `uv pip compile` replaces pip-compile; `uv sync` and `uv add` provide a poetry-like project workflow. The adoption since late 2024 has been rapid because the speedup is real and the migration cost is low.

Step-by-step thought process: 1. "Astral shipped uv in early 2024. The pitch — same UX as pip, dramatically faster, written in Rust. Speed comes from parallel downloads, an efficient resolver, and aggressive caching of resolved metadata." 2. "Day-one use case — drop-in pip replacement. `uv pip install requests` is the same as `pip install requests` but typically 10-100x faster. `uv venv` creates venvs in milliseconds. For most teams, just aliasing `pip` to `uv pip` gives an instant speedup with zero workflow change." 3. "Compile replacement — `uv pip compile pyproject.toml -o requirements.txt` does what `pip-compile` does, but sub-second instead of tens of seconds for a typical project. Resolution and download caching are the big wins." 4. "Project mode — uv has a poetry-like workflow. `uv init` scaffolds a project. `uv add requests` installs and updates `pyproject.toml`. `uv sync` installs from `uv.lock`. `uv run pytest` runs a command inside the project's env without explicit activation. This is the "modern Python workflow" replacement for the `pip/venv/pip-tools` combo." 5. "Python-version management — uv 0.4+ also installs Python interpreters via `uv python install 3.12`. So one binary handles Python install, venv creation, package install, lock file, command run. The full lifecycle in a single tool."

Edge cases to mention: - uv supports both PEP 621 `[project]` and PEP 735 `[dependency-groups]` natively. The lock file `uv.lock` is structured, hash-verified, and reproducible across machines. - Strict mode (`uv pip install --strict`) catches dependency conflicts that pip silently allows. - Some CI images don't have uv pre-installed yet. The `setup-uv` GitHub Action handles this; for other CI systems, `pip install uv` then use uv from there.

What NOT to say: - "uv is just hyped." The speedup is real and measurable. Skepticism is fine; dismissal misses the actual ergonomic win. - "uv replaces pip completely." It can, but most teams keep pip available as a fallback. uv is a superset for most workflows.

Common follow-up: "How do you migrate a poetry project to uv?" — uv reads `pyproject.toml`'s `[project]` table directly. If the project uses `[tool.poetry]`, convert that to `[project]` first (mostly a mechanical move). Then `uv init` (or just `uv lock` + `uv sync`) produces a `uv.lock` and you're off poetry.

```
# Install uv
curl -LsSf https://astral.sh/uv/install.sh | sh
# Or: pip install uv

# Drop-in pip replacement
uv pip install requests pandas
uv pip install -r requirements.txt
uv pip install -e .[dev]

# Create venv (much faster than python -m venv)
uv venv .venv
source .venv/bin/activate

# Compile pinned requirements (replaces pip-compile)
uv pip compile pyproject.toml -o requirements.txt

# Project mode (poetry-like workflow)
uv init myproj
cd myproj
uv add requests pydantic           # writes pyproject.toml AND uv.lock
uv add --group dev pytest mypy     # PEP 735 dependency group
uv sync                            # install from lock file
uv run pytest                       # run inside project env

# Install Python itself (uv 0.4+)
uv python install 3.12
uv python install 3.11 3.10
uv python list
```

Q40 · Why never `sudo pip install`

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Postman · Topic: Tools

The question (interviewer's verbatim phrasing):

Why does every Python guide tell you never to run `sudo pip install` ? What actually goes wrong?

The 30-second answer: `sudo pip install` writes packages into the system Python's site-packages — the same directory the OS package manager owns. You can overwrite OS-managed files, install incompatible versions of packages the system depends on, and create a state the package manager can't reason about. On modern distros (Ubuntu 23.04+, Fedora 38+) PEP 668 blocks it by default with the `EXTERNALLY-MANAGED` marker. Always use a venv (or pipx for tools) instead.

Step-by-step thought process: 1. "System Python is owned by the OS package manager — apt on Debian/Ubuntu, dnf on Fedora, brew on macOS Homebrew. Those packages are tested as a unit; replacing one of them with a pip-installed version can break OS scripts that depend on the exact version." 2. "Concrete breakage — Ubuntu's `apt` tool itself is written in Python and imports `apt`. `sudo pip install --upgrade apt` (or any incompatible dependency upgrade) bricks apt. You can no longer install or remove anything. The fix is painful — manual download of debs and dpkg-install." 3. "Even when nothing breaks, you've created an unmanaged state. apt thinks `python3-requests` 2.25 is installed; pip wrote 2.32 over it. Future apt operations may roll the file back or get confused. The system Python becomes a write-only liability." 4. "PEP 668 (Python 3.11+ era) lets distros mark the system Python as externally-managed. apt-installed Pythons drop a `EXTERNALLY-MANAGED` file in the site-packages directory; pip refuses to install there unless you pass `--break-system-packages`. The flag's name is intentional — it tells you what you're doing is dangerous." 5. "Right answer — venv for project deps, pipx for CLI tools you want global. The system Python stays untouched. If you really need a Python version the OS doesn't have, install it via pyenv, conda, or `uv python install` rather than touching system Python."

Edge cases to mention: - In Docker containers, `sudo pip install` (without sudo, since you're root in the container) into the system Python is common and acceptable — the container is single-purpose, throwaway, and not running as a multi-user OS. Still better practice to use a venv even there, but the consequence is bounded. - `pip install --user` doesn't need sudo and installs into `~/.local/lib/...` — safer than system-wide but still shared across all projects for that user. Modern guidance is venv even over `--user`. - Some older guides recommend `sudo pip install virtualenv` to bootstrap. Modern equivalent — `python3 -m venv .venv` uses the stdlib and needs no global install.

What NOT to say: - "It works on my machine." Until it doesn't, and then the failure mode is "OS package manager broken." - "Just `--break-system-packages` when needed." The flag exists for emergency repair, not for daily use.

Common follow-up: "What's the right way to install a tool I need globally available?" — `pipx install toolname`. Each tool gets its own venv, the entry-point script lands on your PATH, no system Python pollution. For a Python version itself, use pyenv or `uv python install`.

```
# WRONG — risks breaking system Python and the OS package manager
sudo pip install requests
sudo pip install --upgrade pip
sudo pip install some-tool

# Modern Ubuntu/Debian response
# error: externally-managed-environment
# × This environment is externally managed
# ↳ To install Python packages system-wide, try apt install python3-xyz
#   If you wish to install a non-Debian-packaged Python package,
#   create a virtual environment using python3 -m venv path/to/venv.

# RIGHT — venv per project
python3 -m venv .venv
source .venv/bin/activate
pip install requests
deactivate

# RIGHT — pipx for global CLI tools
pipx install black
pipx install ruff

# RIGHT — uv for fast modern workflow
uv venv
source .venv/bin/activate
uv pip install requests

# If you really really really need system pip (you don't)
# Read the warning, accept the consequence, then:
pip install --break-system-packages some-tool
# Don't say I didn't warn you.
```

End of Chapter 10

Forty questions across the five sub-areas of Python's module and packaging system — the import machinery, packages and `__init__.py`, circular-import surgery, `pyproject.toml` and modern build backends, and the `venv` / `pipx` / `uv` tooling layer. The combined ground covered here is what separates a Python developer who can `pip install` and ship a Flask app from one who can debug a circular import in someone else's package, set up a clean `pyproject.toml` from scratch, pick the right build backend for a new library, and explain to a new teammate why `sudo pip install` is the wrong instinct. Pair this chapter with Chapter 9 (Typing + Protocols) for the modern-Python pair, and with Chapter 1 (OOP + Dunder) for how the import system intersects with class hierarchies and metaclasses.

Chapter 11 — Strings, Bytes & Regex (50 Qs)

Python Interview Prep — 2026 Edition · Learn & Excel · File 04 Core Library

Strings are the surface most candidates think they know — and the surface most panels use to filter the candidates who actually know it. Indian service-MNC rounds open with `str.split` / `str.join` and "what's the difference between `str` and `bytes`?" Product cos (Razorpay, Flipkart, Swiggy, Cred, Postman, Zerodha) go three layers deeper — f-string format specs, Unicode normalization, `latin-1` round-tripping, named groups, lookbehind. GCC panels (JP Morgan, Goldman, Walmart Labs, Microsoft India) lean on regex performance and ReDoS, because that's the bug that takes a payments service down at 3 AM.

This chapter is fifty questions split as follows: **string fundamentals (Q1–Q10)**, **f-strings deep (Q11–Q15)**, **Unicode + encoding (Q16–Q25)**, `bytes` / `bytearray` / `memoryview` (Q26–Q30), **regex basics (Q31–Q40)**, **regex advanced (Q41–Q50)**. Every question follows the same shape: tag line · interviewer phrasing · 30-second answer · step-by-step thought process · edge cases · what NOT to say · common follow-up · runnable code on Python 3.12+.

Section A — String Fundamentals (Q1–Q10)

Q1 · Why `str` Is Immutable and What That Costs You

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Cognizant · Topic: String Fundamentals

The question (interviewer's verbatim phrasing):

Is `str` mutable or immutable in Python? Walk me through what happens when I do `s = "hello"; s += " world"`.

The 30-second answer: `str` is immutable. Every "modification" — concatenation, replace, slicing — allocates a brand-new string object and rebinds the name. `s += " world"` doesn't extend `s` in place; it builds `"hello world"` as a new string and points the name `s` at it. The old `"hello"` becomes garbage once nothing references it.

Step-by-step thought process: 1. Lead with the contract: `str` belongs in the immutable family alongside `int`, `float`, `tuple`, `frozenset`, `bytes`. Immutability means the object's bytes never change after construction. 2. Walk through the `+=` mechanics: Python evaluates `s + "`

"world", that expression produces a new `str` object, then the augmented assignment rebinds `s`. The old string is unchanged — it just becomes unreachable. 3. State the consequence: immutability makes `str` hashable (usable as a dict key), thread-safe (no torn reads), and interning-friendly. The cost is allocation pressure — every "edit" is an allocation. 4. Anticipate the follow-up about CPython's optimisation: CPython has a special case where `s += t` reuses the buffer if the refcount is 1, but you can't rely on it across interpreters (PyPy, GraalPy don't). Treat `+=` as allocation.

Edge cases to mention: - A `str` containing only ASCII still uses the compact Unicode representation in CPython (1 byte per char); a `str` with any code point above U+FFFF jumps to 4 bytes per char for the whole string. Immutability lets the interpreter pick the storage at creation time and stick with it. - `id(s)` can be the same before and after `s += "x"` in CPython if the buffer was reusable, but `id` equality is not a language guarantee. - `bytes` is immutable for the same reasons; `bytearray` and `list` are the mutable counterparts.

What NOT to say: - "Python optimises `+=` so it's fine" — sometimes true, never guaranteed, and PyPy / a refcount-2 situation will reallocate every time. - "Strings are passed by reference so they can be modified" — no. Names point at objects; the object itself doesn't change.

Common follow-up: "Then how do I build a long string from a million pieces without going quadratic?" — `"".join(parts)` over a list; do not use `+=` in a loop.

```
s = "hello"
print(id(s))           # some id
s += " world"
print(id(s))           # usually different – new object
# s now points at "hello world"; "hello" is garbage if no one else holds it
```

Q2 · The `+=` in a Loop Trap — Why It's $O(n^2)$

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, JP Morgan, Walmart Labs · Topic: String Performance

The question (interviewer's verbatim phrasing):

I'm building a CSV row by appending strings inside a loop with `s += piece`. Why does it get slow on big inputs, and what should I use instead?

The 30-second answer: Because `str` is immutable, every `s += piece` allocates a new string and copies all bytes from the old `s` plus the new `piece`. After `n` iterations you've copied `1 + 2 + ... + n  $\approx$   $n^2/2$` bytes. Use `"".join(parts)` where `parts` is a list — `join` measures the total length once, allocates one final buffer, and copies each piece in.

Step-by-step thought process: 1. State the asymptotic cost first — $O(n^2)$ for repeated `+=` versus $O(n)$ for `join`. That's the headline. 2. Walk through why: at iteration `k`, the running `s` has length proportional to `k`. The `+=` operation has to copy all `k` bytes plus the new piece. Sum over `n` iterations and you get the quadratic. 3. Mention CPython's in-place optimisation — when the refcount of `s` is exactly 1, CPython will try to resize the existing buffer instead of copying. It works in trivial scripts; it fails the moment another reference exists, you switch interpreters, or you store `s` in a structure. 4. The fix is one of three: `"".join(parts)`, `io.StringIO` and its `write` method, or a list-comprehension into `join`. All are linear and idiom-correct.

Edge cases to mention: - The optimisation also fails when `s` is a slice, a class attribute, or anything where Python can't prove single ownership. - For huge byte streams, `bytearray` plus `extend` is the mutable analogue — it's linear because the buffer grows geometrically. - `f"{a}{b}{c}"` is fine for a small fixed number of pieces; it's not a substitute for `join` in a loop.

What NOT to say: - "CPython optimises it, so the quadratic is theoretical" — interviewers know this answer and the next question will be "now show me on PyPy or with a stored reference." - "Use `+` instead of `+=`" — same problem; `+` is just `s = s + piece` and allocates exactly the same way.

Common follow-up: "Show me the linear version that you'd actually write."

```
# Bad — O(n^2) in the general case
def slow_join(pieces: list[str]) -> str:
    s = ""
    for p in pieces:
        s += p
    return s

# Good — O(n)
def fast_join(pieces: list[str]) -> str:
    return "".join(pieces)
```

Q3 · `str.split` — What the Empty String and `maxsplit` Do

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Postman, Freshworks · Topic: String Methods

The question (interviewer's verbatim phrasing):

What does `"a,,b".split(",")` return, and how is it different from `"a b".split()` with no argument?

The 30-second answer: `"a,,b".split(",")` returns `['a', '', 'b']` — the empty string between the two commas is preserved as an empty field. `"a b".split()` with no argument calls the

whitespace-collapsing version: it splits on any run of whitespace and drops empty fields. They look similar; they aren't.

Step-by-step thought process: 1. Anchor the two modes: `split(sep)` is strict — every separator delimits a field, empty fields included. `split()` (no arg) is forgiving — runs of whitespace are one delimiter, leading and trailing whitespace are stripped. 2. Mention `maxsplit` : `"a,b,c,d".split(",", maxsplit=2)` returns `['a', 'b', 'c,d']` — at most two splits, the rest is one tail field. Useful for parsing "first word, rest of line." 3. The mirror is `rsplit` — splits from the right, useful when you want "last token, everything before." 4. `splitlines` is the right tool for line-by-line parsing — handles `\n` , `\r\n` , `\r` , and the rare Unicode line separators. Don't roll your own.

Edge cases to mention: - `"".split(",")` returns `['']` — one empty field, not an empty list. `"".split()` returns `[]` . Trips parsers when input might be blank. - `"abc".split("")` raises `ValueError: empty separator` — use list/iteration if you want characters. - `splitlines(keepends=True)` keeps the line terminators with each line — useful when you need to preserve `\r\n` vs `\n` .

What NOT to say: - "`split` and `split()` are the same" — they're not; the no-arg form is whitespace-collapsing. - "I'd use regex for this" — overkill for a fixed-character separator and three to ten times slower.

Common follow-up: "How would you parse a CSV line that has commas inside quoted fields?" — Don't use `str.split` ; use the `csv` module, which handles the quoting state machine correctly.

```
print("a,,b".split(","))      # ['a', '', 'b']
print("a b".split())         # ['a', 'b']
print(" a b ".split())       # ['a', 'b'] — leading/trailing stripped
print("a,b,c,d".split(",", 2)) # ['a', 'b', 'c,d']
print("".split(","))         # [''] — gotcha
print("".split())            # []
```

Q4 · `str.join` — Why It's a Method on the Separator

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Freshworks · Topic: String Methods

The question (interviewer's verbatim phrasing):

Why does `",".join(["a", "b"])` look backwards? Shouldn't it be the list calling `join` ?

The 30-second answer: The separator is the constant; the iterable is the variable. `str.join(self, iterable)` is a method on `str` because the separator type controls what comes out —

`",".join` produces a `str`, `b",".join` produces a `bytes`. The pieces being joined just need to be iterable. The "list calls join" design would require every iterable type to implement the method.

Step-by-step thought process: 1. State the design rule: methods belong to the type that controls the return value. `join` returns a `str` (or `bytes`); the separator decides which, so the separator owns the method. 2. Walk through the type behaviour: `",".join(["a", "b"])` produces a `str`; `b",".join([b"a", b"b"])` produces a `bytes`. Mixing them raises `TypeError`. 3. Performance call-out: `join` measures the total length of all pieces first, allocates once, copies in. That's why it's linear and beats `+=`. 4. The iterable can be a generator: `"".join(str(i) for i in range(10))` is fine and doesn't materialise the list first — handy when the input is large or lazy.

Edge cases to mention: - `join` raises `TypeError` if any piece isn't a `str` — `",".join([1, 2])` fails. Cast first: `",".join(str(x) for x in items)`. - A generator gets consumed once; if `join` raises midway, the generator is in an unpredictable state. - `"".join([])` is the empty string — useful base case in conditional builders.

What NOT to say: - "Because lists don't have a join method" — they don't, but the *reason* is the type-of-return-value rule. The interviewer wants the design rationale. - "Use `+` for two strings, `join` for many" — fine for tiny cases, but the moment "two" becomes "two-in-a-loop" you're back to the quadratic.

Common follow-up: "Can I `join` a dict?" — Yes; iterating a dict yields its keys, so `",".join({"a": 1, "b": 2})` returns `"a,b"`. Surprising the first time you see it.

```
print(",".join(["a", "b", "c"]))          # 'a,b,c'
print("".join(str(i) for i in range(4))) # '0123' - generator
print(",".join({"a": 1, "b": 2}))        # 'a,b' - iterates keys
# ",".join([1, 2])                      # TypeError: must be str
```

Q5 · `strip` / `lstrip` / `rstrip` — What the Argument Actually Means

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Postman · Topic: String Methods

The question (interviewer's verbatim phrasing):

What does `"xxhelloxxx".strip("x")` return, and what's the difference between that and `"hello".strip()`?

The 30-second answer: `strip("x")` removes every leading and trailing `x` character — the argument is a *set of characters to strip*, not a substring. So `"xxhelloxxx".strip("x")` returns `"hello"`. `strip()` with no argument removes leading/trailing whitespace (spaces, tabs, newlines, and the rest of `str.isspace()`'s definition).

Step-by-step thought process: 1. Drive in the misconception first: the argument is a *character set*, not a substring. `"abxxxba".strip("ab")` removes any leading/trailing `a` or `b` — result is `"xxx"`. 2. Whitespace mode: `strip()` with no argument trims everything that `str.isspace()` returns true for — `' '`, `'\t'`, `'\n'`, `'\r'`, `'\v'`, `'\f'`, plus Unicode whitespace. 3. Mention the directional variants: `rstrip` only at the start, `rstrip` only at the end. The most common real use is `line.rstrip("\n")` when iterating a file. 4. For substring stripping (Python 3.9+), use `removeprefix` / `removesuffix` — they do what beginners expect from `strip`.

Edge cases to mention: - `"https://example.com".strip("https://")` returns `"example.com"` — and beginners think that's a URL parser. It's not; `://example.co` would be mangled the same way. Use `removeprefix("https://")`. - `strip` is idempotent — calling it twice has the same effect as calling it once. - Empty input: `"".strip()` is `""`; `" ".strip()` is `""`.

What NOT to say: - `strip('abc')` removes the substring `abc` — wrong. That's `removeprefix` / `removesuffix`. - `strip` removes spaces — partially true; it removes all whitespace, and the broader set of Unicode whitespace too.

Common follow-up: "How do I remove a known prefix like `'http://'` from a URL?" — `url.removeprefix("http://")` since Python 3.9; before that, `url[len("http://"):]` if you confirmed `startswith` first.

```
print("xxhelloxx".strip("x"))           # 'hello'
print("abxxxba".strip("ab"))           # 'xxx' — set, not substring
print(" hello\n".rstrip())             # ' hello'
print("https://api.razorpay.com".removeprefix("https://")) # 'api.razorpay.com'
```

Q6 · `replace`, `find` vs `index`, `startswith` / `endswith`

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant, Razorpay · Topic: String Methods

The question (interviewer's verbatim phrasing):

What's the difference between `str.find` and `str.index`? And how would you check that a filename ends in `.csv` or `.tsv`?

The 30-second answer: `find` returns the index of the first match, or `-1` if not found. `index` returns the same index, but raises `ValueError` if not found. Pick `find` when "not present" is a normal case; pick `index` when absence is an error. For "ends in `.csv` or `.tsv`", `name.endswith((".csv", ".tsv"))` — `endswith` and `startswith` take a tuple of options.

Step-by-step thought process: 1. Distinguish the two by failure mode: sentinel return vs exception. The interviewer is checking that you know absence-is-data vs absence-is-error. 2. Show the EAFP idiom: if a missing substring is an error, use `index` inside a try; if it's a routine "did this URL contain `?`", use `find` and check against `-1`. 3. The tuple-argument trick on `startswith` / `endswith` is the single most useful undocumented-feeling feature here. It saves the awkward `name.endswith(".csv") or name.endswith(".tsv")` pattern. 4. `replace(old, new, count=-1)` replaces all by default; `count=1` replaces only the first occurrence. Drop the `count` argument unless you specifically want a one-shot.

Edge cases to mention: - `find` and `index` both take optional `start` and `end` arguments — `s.find("x", 5)` searches from index 5 onwards. Useful for "find the next one after the one I just found." - `replace` does not use regex. `"a.b".replace(".", "")` removes the literal dot, not "any character." - `startswith` and `endswith` are case-sensitive — for filenames where case varies (`.CSV`), lowercase first or use a regex with `re.IGNORECASE`.

What NOT to say: - "Use regex" for any of these — three to ten times slower than the built-in method, and reads worse to anyone reviewing the code. - "`replace` modifies the string in place" — `str` is immutable; `replace` returns a new string.

Common follow-up: "How would you replace only the first occurrence of `'old'` with `'new'`?" — `s.replace("old", "new", 1)`.

```
url = "https://x.com/path?q=1"
print(url.find("?"))           # 17 - sentinel
print(url.find("#"))          # -1 - not found, no exception
# url.index("#")              # ValueError

print("report.csv".endswith(("csv", "tsv", "xlsx"))) # True
print("ABC".lower().startswith("a"))                # True

s = "one two two three"
print(s.replace("two", "TWO"))                       # 'one TWO TWO three'
print(s.replace("two", "TWO", 1))                    # 'one TWO two three'
```

Q7 · `str.format` vs f-string vs `%`-formatting — The History

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Postman, Walmart Labs · Topic: String Formatting

The question (interviewer's verbatim phrasing):

Walk me through `%`-formatting, `str.format`, and f-strings. Why do all three exist, and which one do you reach for first?

The 30-second answer: Three eras: `%`-formatting (1991, inherited from C), `str.format` (3.0, PEP 3101 — name- and position-based), f-strings (3.6, PEP 498 — inline expressions). f-strings are the default since 3.6 because they're faster, read top-to-bottom, and support arbitrary expressions. `str.format` survives for templates loaded from config or i18n files. `%`-formatting survives in logging and in legacy code.

Step-by-step thought process: 1. Walk the timeline. `%`-formatting (`"hello %s" % name`) was Python's first option, copied from C's `printf`. `str.format` (`"hello {}".format(name)`) arrived in Python 3.0 to give names and richer specs. f-strings (`f"hello {name}"`) arrived in 3.6 and read more naturally because the value is right where it's interpolated. 2. State the default: f-strings, always, for in-source string assembly. They're parsed at compile time and run faster than the other two. 3. The two real exceptions. **Templates from config:** if the format string is loaded from a file or DB, you can't use f-strings (they need to be source literals) — use `str.format`. **Logging:** `logger.info("Got %s", value)` defers formatting until the log level is enabled, which f-strings can't do because they format eagerly. 4. Anticipate the safety follow-up: f-strings on attacker-controlled format text are a code-execution vector because they evaluate expressions. `str.format` has its own attribute-access risk (`"{0.__class__}"`). For untrusted templates, `string.Template` is the safe option.

Edge cases to mention: - `%`-formatting collides with `%` characters in the input — `"100%% done" % {}` is the escape. f-strings need `{{ / }}` to produce literal braces. - `str.format` supports indexed (`{0}`), named (`{name}`), and attribute access (`{user.email}`) inside the braces. - f-strings can nest format specs (`f"{value:{width}.{precision}f}"`) — the inner braces are evaluated first.

What NOT to say: - "f-strings are always best" — not for deferred logging, not for templates loaded at runtime, not for untrusted input. - "`%`-formatting is deprecated" — it isn't; `logging` still uses it idiomatically and removing it would break decades of code.

Common follow-up: "Why does `logging` still use `%`-formatting?" — Lazy evaluation. `logger.debug("expensive %s", expensive())` doesn't call `expensive()` if debug is off; an f-string would format eagerly.

```
name = "Asha"
age = 28

print("Hello %s, age %d" % (name, age))      # %-formatting
print("Hello {}, age {}".format(name, age))  # str.format
print(f"Hello {name}, age {age}")           # f-string — default

# Logging — keep %-style for lazy eval
import logging
logging.info("user %s logged in", name)      # not f"...{name}..."
```

Q8 · `str.translate` and `str.maketrans` — The Fast Character Mapper**Tags:** Difficulty: Medium · Asked at: Razorpay, Postman, Walmart Labs, Goldman · Topic: String Methods**The question (interviewer's verbatim phrasing):**

I have a string and I want to remove all digits, swap `'a'` with `'@'`, and `'e'` with `'3'`.
What's the fastest way?

The 30-second answer: `str.translate` with a table built by `str.maketrans`. `maketrans` accepts three forms — a dict mapping code points to replacements, a (from, to) pair of equal-length strings, or a (from, to, delete) triple. `translate` applies the table in a single C-level pass. It's the fastest way to do simultaneous character-level substitutions.

Step-by-step thought process: 1. Start with the call shape: `s.translate(table)`, where `table` is a dict-like that maps `ord(char)` to either an `int`, a `str`, or `None` (meaning delete). 2. `maketrans` builds the dict for you. The three-argument form is the most flexible: `str.maketrans("ae", "@3", "0123456789")` maps `a → @`, `e → 3`, and removes all digits. 3. Why it's faster than chained `replace`: `translate` walks the input once and looks up each code point in a C-level dispatch. Chained `replace` walks the input once per call and allocates an intermediate string each time. 4. The same machinery exists for `bytes` — `bytes.maketrans` and `bytes.translate` with a 256-byte table. Useful for binary clean-up.

Edge cases to mention: - `maketrans` only accepts equal-length `from` / `to` strings in the two-argument form — otherwise raises `ValueError`. The dict form lets unequal lengths. - Characters outside the table are passed through unchanged. - Multi-character replacements work via the dict form — `{ord("&"): "and"}` substitutes `&` with `and` in a single pass.

What NOT to say: - "Use multiple `replace` calls" — works for two or three substitutions; reads worse and runs slower for ten or more. - "Use regex" — `re.sub` is slower for fixed character-by-character mappings and reads worse.

Common follow-up: "How would you do the same for a `bytes` object?" — `bytes.maketrans` and `bytes.translate`, with `from` and `to` as `bytes` of equal length and an optional `delete` `bytes`.

```
s = "leet code 2026"

# 3-arg maketrans: from, to, delete
table = str.maketrans("ae", "@3", "0123456789")
print(s.translate(table))           # 'l33t cod3 '

# Dict form — multi-char replacements
table2 = str.maketrans({"&": "and", "<": "&lt;", ">": "&gt;"})
print("a & b".translate(table2))    # 'a and b'
```

Q9 · Case Methods, `casefold` vs `lower`, and `title`'s Quirks

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Walmart Labs · Topic: String Methods

The question (interviewer's verbatim phrasing):

What's the difference between `str.lower` and `str.casefold`? And what does `"a's".title()` return?

The 30-second answer: `lower` does the standard lowercase mapping; `casefold` does an *aggressive* lowercase intended for case-insensitive comparison — it handles German ß by mapping to `ss` and other locale-specific cases that `lower` leaves alone. `"a's".title()` returns `"A'S"` because `title` treats the apostrophe as a word boundary, which usually surprises people.

Step-by-step thought process: 1. Use `casefold` for case-insensitive comparison; use `lower` for display. The German ß is the canonical example — `"groß".lower()` stays `"groß"`, but `"groß".casefold()` becomes `"gross"`, so `"groß".casefold() == "GROSS".casefold()` works while `lower` doesn't. 2. `title` capitalises after every non-letter boundary — that's why apostrophes mangle it. `"can't stop".title()` returns `"Can'T Stop"`. For real title-case, you usually want a library or a `re`-based approach. 3. `capitalize` only capitalises the first character and lowercases the rest. `"hello WORLD".capitalize()` returns `"Hello world"` — not `"Hello WORLD"` — which catches people. 4. `swapcase` flips case per character. Cute for puzzles, rarely useful in real code.

Edge cases to mention: - `casefold` is defined by Unicode CaseFold tables (see UAX #44) — every Python release follows the latest Unicode version. If you need stable behaviour across Python versions, document the version dependency. - `upper` can change length: `"ß".upper()` returns `"SS"` (two characters). Same for other Unicode characters with case-folded expansions. - `isupper` / `islower` / `istitle` are predicate checks — `isupper` requires at least one cased character; the empty string returns `False`.

What NOT to say: - "`lower` and `casefold` are the same" — for ASCII they are; for non-ASCII they aren't, and the question is about the non-ASCII case. - "`title` capitalises each word correctly" — it doesn't; it capitalises after every non-letter, which mangles apostrophes and hyphens.

Common follow-up: *"How would you compare two user-entered email addresses case-insensitively?"* — Both `.casefold()` and `compare`. Don't trust `.lower()` for international input.

```
print("groß".lower())           # 'groß'
print("groß".casefold())        # 'gross'
print("groß".casefold() == "GROSS".casefold()) # True

print("a's".title())            # "A'S" – apostrophe gotcha
print("hello WORLD".capitalize()) # 'Hello world'
print("ß".upper())              # 'SS' – length changes
```

Q10 · `in` on Strings — Substring Check Complexity

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Postman · Topic: String Performance

The question (interviewer's verbatim phrasing):

What's the time complexity of `"abc" in some_string` in CPython? Is it $O(n)$ or $O(n \times m)$?

The 30-second answer: CPython uses a tuned variant of the two-way / Boyer-Moore-ish algorithm for substring search, so the average case for `pattern in text` is closer to $O(n)$ on natural input. The pathological worst case is $O(n \times m)$, but CPython picks the algorithm based on pattern length and skips that pathological branch for typical strings. For interview answers, say "linear on average, with $O(n \times m)$ worst case" and move on.

Step-by-step thought process: 1. State the API: `"abc" in s` returns a bool; `s.find("abc")` returns the index or `-1`. They share the same underlying search. 2. The algorithm has evolved — current CPython uses a two-way string matching variant tuned for short patterns. For interview purposes, "amortised linear" is the right summary. 3. Where it matters in practice: a single substring check in a hot loop on long strings can dominate. Profile before assuming. 4. Avoid the common over-optimisation: rolling your own KMP in Python will be *slower* than the C-level `in` operator. Python-level optimisation of substring matching is almost always a loss.

Edge cases to mention: - The empty pattern is always `in` every string — `"" in "anything"` is `True`. Bites parsers that check `if pattern in s` without validating the pattern. - For matching many patterns against many strings, `re.compile("|".join(map(re.escape, patterns)))` outperforms one-at-a-time substring checks; even faster, a `set` lookup if you can enumerate the candidates. - For repeated checks against the same pattern, there's no built-in "compile once" — substring search is C-level fast enough that it doesn't need compilation.

What NOT to say: - "`in` is $O(n)$ " — not literally; the worst case is $O(n \times m)$. The defensible answer is "linear on average." - "I'd implement KMP for speed" — you'd lose. The C-level built-in beats any Python implementation.

Common follow-up: "What about checking if a string contains any of a set of substrings?" — `re.compile("|".join(re.escape(p) for p in patterns))` and check `pattern.search(s)`. Aho-Corasick (in `pyahocorasick`) for very large pattern sets.

```
big = "a" * 10_000 + "needle"
print("needle" in big)                # True  - linear-on-average
print("" in "anything")              # True  - empty pattern trap

# Many patterns - compile a union
import re
patterns = ["error", "fail", "exception"]
rx = re.compile("|".join(map(re.escape, patterns)))
log = "process failed at 03:14"
print(bool(rx.search(log)))           # True
```

Section B — f-strings Deep (Q11–Q15)

Q11 · f-string Syntax — Expressions and Format Specs

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Postman, Freshworks · Topic: f-strings

The question (interviewer's verbatim phrasing):

What can you put inside the braces of an f-string? Just variables, or actual expressions?

The 30-second answer: Any Python expression. f-string braces evaluate their contents like normal Python — `f"{x + 1}"`, `f"{func(arg)}"`, `f"{a if b else c}"`, even `f"{[i*2 for i in range(3)]}"` all work. Plus an optional format spec after a colon (`f"{x:.2f}"`) and an optional conversion (`!r`, `!s`, `!a`).

Step-by-step thought process: 1. State the rule: the braces are eval'd. There's no "f-strings only support variables" restriction — that's a tutorial-only myth. 2. Conversions: `!r` calls `repr`, `!s` calls `str`, `!a` calls `ascii`. They run before the format spec. `f"{x!r:>20}"` repr-s `x` then right-aligns in 20 chars. 3. Format specs after `:` follow PEP 3101 — `f"{x:.2f}"` for two decimals, `f"{x:>10}"` for right-aligned width 10, `f"{x:,}"` for thousands-separator (`1,000,000`), `f"{x:%}"` for percentage. 4. Nesting: width and precision can themselves be expressions — `f"{value:{width}.{precision}f}"`. The inner braces are evaluated first.

Edge cases to mention: - Comments aren't allowed in f-string expressions before 3.12 — they are in 3.12+ (see PEP 701). - Quote reuse: in 3.12+, you can use the same quote character inside and outside (`f"{d['key']}"`). Before 3.12, you had to switch (`f"{d[\"key\"]}"`) or use different quote

types). - Walrus operator inside an f-string needs parens: `f"{{n := 5}}"` works; the bare walrus doesn't because of expression ambiguity.

What NOT to say: - "f-strings only let you put variable names in braces" — wrong; full expressions are legal. - "Format specs are the same as `%`-formatting" — they overlap but differ; PEP 3101 introduced the `{name:spec}` syntax and a richer set of options.

Common follow-up: "What's `f"{{x!r}}"` doing differently from `f"{{x}}"`?" — `f"{{x}}"` calls `str(x)`; `f"{{x!r}}"` calls `repr(x)`. For debugging, prefer `!r` because it shows quotes around strings.

```
x = 3.14159
print(f"{{x:.2f}}")           # '3.14'
print(f"{{x:>10.2f}}")        # '      3.14'
print(f"{{x:>{{12}}.{{3}}f}}") # nested width/precision
print(f"{{1_000_000:,}}")     # '1,000,000'
print(f"{{[i*2 for i in range(3)]}}") # '[0, 2, 4]'
print(f"{{'hi'!r}}")          # "'hi'" — repr conversion
```

Q12 · The `=` Self-Documenting f-string (3.8+)

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Flipkart, Cred · Topic: f-strings

The question (interviewer's verbatim phrasing):

What does `f"{{x=}}"` do? When did it land in Python?

The 30-second answer: `f"{{x=}}"` (added in Python 3.8) expands to `"x=<value>"` — it prints both the expression and its value. The default conversion is `repr`, so strings appear quoted. It's the single best debug-print upgrade in modern Python.

Step-by-step thought process: 1. State the syntax: any expression followed by `=` inside the braces gets the expression text plus the value. `f"{{a+b=}}"` produces `"a+b=7"` when `a=3, b=4`. 2. Default conversion: `!r` is implied. To get `str` conversion, use `f"{{x=!s}}"`; for repr explicitly, `f"{{x=!r}}"`. 3. Whitespace: any whitespace around the `=` is preserved verbatim in the output. `f"{{x = }}"` outputs `"x = <value>"` with spaces, which matches typical Python style. 4. When to use it: throwaway debug prints and assertion messages. For production logging, prefer `logger.debug("x=%s", x)` so it skips formatting when the level is off.

Edge cases to mention: - Format specs still work alongside `=`: `f"{{x:~.2f}}"` produces `"x=3.14"`. The format spec applies to the value, not the expression text. - Before 3.8, the workaround was `f"x={{x!r}}"` — duplicated identifier, easy to drift. - `=` works on any expression — `f"{{[*range(3)]=}}"` is valid.

What NOT to say: - "It's only for variables" — works on any expression. - "It's syntactic sugar for `f'x={x}'`" — close, but the default is `repr`, not `str`.

Common follow-up: "Why did this land in 3.8 specifically?" — PEP 572 (walrus) work made tweaking the parser around f-strings tractable; PEP that added `=` was bpo-36817. Practical motivation: thousands of throwaway `print(f"x={x}")` lines across every Python codebase.

```
a = 3
b = 4
print(f"{a+b=}")           # 'a+b=7'
print(f"{a:=>5}")          # 'a=   3' - spec applied to value
print(f"{'hello'}=")       # "'hello'='hello'" - repr default
print(f"{a = }")           # 'a = 3' - whitespace preserved
```

Q13 · PEP 701 — What Changed in 3.12 f-strings

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Walmart Labs, Flipkart · Topic: f-strings · 3.12+

The question (interviewer's verbatim phrasing):

What did PEP 701 change about f-strings in Python 3.12?

The 30-second answer: PEP 701 made f-strings part of the regular Python grammar — before, they were a hand-rolled mini-parser with several arbitrary limits. Three practical wins in 3.12: you can reuse the same quote character inside and outside (`f"{d['key']}"`), you can break expressions across multiple lines, and you can put `#` comments inside the expression. Backslashes in expressions also work now.

Step-by-step thought process: 1. Frame the why: before 3.12, f-strings were parsed by a separate tokeniser that couldn't recurse into full Python. Three nuisance limits fell out of that. 2. Quote reuse: `f"{d['key']}"` was illegal before 3.12 because the outer `"` and inner `'` were tokenised together — the workaround was switching quote types or escaping. 3.12 lets you nest the same quote. 3. Multi-line expressions: `f"{some_func( a, b, c )}"` across lines works in 3.12; before, the expression had to fit on a single line. 4. Backslashes and comments: `f"{ord('\n')}"` is legal in 3.12 (backslash inside expression); `f"{x # comment\n}"` is legal too. Before, both were syntax errors.

Edge cases to mention: - Nesting depth — pre-3.12 f-strings were limited to a few levels of recursive f-string nesting; 3.12 removes the limit. - This is a strictly additive change — every pre-3.12 f-string still works. If you care about wider compatibility (still supporting 3.10 / 3.11), don't lean on the new flexibility unless you've gated by version. - Error messages also improved — pre-3.12 syntax errors inside f-strings often pointed at the wrong column.

What NOT to say: - "f-strings are new in 3.12" — they landed in 3.6. PEP 701 is the *grammar* unification.
 - "It changed how f-strings format" — formatting semantics are unchanged; only the syntax rules expanded.

Common follow-up: "Should I drop `f"{d['key']}"` workarounds from old code?" — Only if your minimum supported Python is 3.12. Otherwise keep the workaround.

```
# These are all 3.12+ only
d = {"name": "Asha"}
print(f"{d['name']}")                # Same quote inside – legal in 3.12+

# Multi-line expression (3.12+)
items = [1, 2, 3]
print(f"{
    sum(
        i*2 for i in items
    )
}")                                # 12

# Comments inside (3.12+)
x = 5
# print(f"{x  # the value\n}")      # legal in 3.12+
```

Q14 · When NOT to Use f-strings — Security and Deferred Formatting

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Goldman, Walmart Labs · Topic: f-strings · Security

The question (interviewer's verbatim phrasing):

When would you avoid f-strings? Are there cases where another formatter is better?

The 30-second answer: Three cases. **User-controlled templates** — f-strings need source literals and evaluate any expression, so they can't safely render attacker-supplied templates; use `string.Template` (no eval) or `str.format` with a sanitised input. **Logging** — `logger.info("user %s", name)` defers formatting until the log level is enabled, saving work when debug is off; f-strings format eagerly. **i18n / templates loaded at runtime** — f-strings are source-literal only; use `str.format` or a template engine.

Step-by-step thought process: 1. State the security model first. f-string braces evaluate Python expressions. If you `eval(f"... {user_input}...")` or even render an f-string from a runtime string (which you can't directly), you're handing over execution. Even `str.format` is risky on untrusted templates because `{x.__class__.__init__.__globals__}` is an expression you can sneak in. 2. The right safe choice for untrusted templates is `string.Template` — `${name}`

substitution only, no attribute access, no expressions. 3. Logging: every `logger.debug(f"big {payload}")` formats `payload` even when debug is disabled. Use `%`-style: `logger.debug("big %s", payload)` — `payload` is only formatted if the log handler will emit it. 4. Performance-sensitive hot loops: f-strings are usually fastest in-source, but if the format string is reused across millions of calls with the same shape, a pre-compiled `str.format_map` over a `format` template can win.

Edge cases to mention: - f-strings have no equivalent of `str.format(**dict)`. If you need dict-driven formatting, `str.format_map` or `Template.substitute(mapping)` are the options. - `string.Template` rejects unknown placeholders by default (`KeyError`) — switch to `safe_substitute` for partial substitution. - For Markdown / HTML rendering of user input inside an f-string, the issue isn't the f-string but the lack of escaping. Use the right escaper (`html.escape`, `markupsafe.escape`).

What NOT to say: - "f-strings are always safe because Python is safe" — eval'ing expressions on attacker input is the textbook RCE. - "Logging with f-strings is fine because it's faster" — wrong; eager formatting is the cost.

Common follow-up: "Show me the safe template render for user input." — `string.Template("Hello $name").substitute(name=user_name)`.

```
import logging
from string import Template

# Good — deferred formatting
logger = logging.getLogger(__name__)
logger.debug("payload size %d bytes", 1_000_000)  # only formatted if debug on

# Untrusted template — use string.Template
user_template = "Hello $name, your balance is $amount"  # from DB
out = Template(user_template).substitute(name="Asha", amount=499)
print(out)

# NOT this — str.format gives expression access on attacker templates
# evil = "{x.__class__.__init__.__globals__}"
# evil.format(x=anything)  # potential leak path
```

Q15 · f-string Performance — Why It's Fast (and When It Isn't)

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Walmart Labs, Goldman · Topic: f-strings · Performance

The question (interviewer's verbatim phrasing):

Is an f-string faster than `str.format` or `%`-formatting? Why?

The 30-second answer: Yes, in CPython. f-strings are compiled to a `BUILD_STRING` bytecode that joins pre-evaluated pieces in one allocation. `str.format` calls a method that has to parse the template string at runtime every call. `%`-formatting is a binary operator that also parses the format string each invocation. For a tight loop, f-strings are typically 2-3× faster than `str.format` and 1.5-2× faster than `%`.

Step-by-step thought process: 1. Walk through the compilation. The CPython compiler turns `f"{a}{b}"` into something like `BUILD_STRING 3` (with the literal pieces and the evaluated expressions already on the stack). No method call. No template parsing. 2. `str.format` parses `"hello {} {}"` every call to figure out which positional argument goes where. The parse cost is small per call but accumulates over millions of iterations. 3. `%`-formatting parses similarly and has special-case branches per format code. Faster than `format` but slower than f-strings. 4. The honest performance answer at the end: micro-benchmarks are misleading for one-shot uses. The difference matters only inside hot loops, and at that point you should be using `join` or `bytes` assembly anyway.

Edge cases to mention: - For *deferred* formatting (logging), f-strings lose the speed argument because eager evaluation costs more than the parse savings. - `str.format_map(mapping)` is slightly faster than `str.format(**mapping)` because it avoids unpacking — useful when the keys are already a dict. - Compile-time concatenation: adjacent string literals (including f-string fragments) merge — `f"a" f"b"` becomes `"ab"` at compile time, but mixing literal and f-string fragments doesn't always merge predictably.

What NOT to say: - "f-strings are always faster" — true on CPython, but the gap closes in PyPy and the gap doesn't matter outside hot loops. - "Performance is the reason to use f-strings" — no, readability is. Performance is a nice-to-have.

Common follow-up: *"Profile this hot loop — show me the difference."*

```
import timeit

# Run-time format parsing — slowest
t1 = timeit.timeit('f"x={x} y={y}"', setup='x=1; y=2', number=1_000_000)
t2 = timeit.timeit('"x={x} y={y}".format(x, y)', setup='x=1; y=2', number=1_000_000)
t3 = timeit.timeit('"x=%s y=%s" % (x, y)', setup='x=1; y=2', number=1_000_000)
print(f"f-string: {t1:.3f}s")      # fastest
print(f"format:   {t2:.3f}s")      # ~2-3x slower
print(f"%-fmt:    {t3:.3f}s")      # in between
```

Section C — Unicode + Encoding (Q16–Q25)

Q16 · `str` vs `bytes` in Python 3 — Why They're Different Types

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman, Swiggy · Topic: Unicode

The question (interviewer's verbatim phrasing):

What's the difference between `str` and `bytes` in Python 3? Why aren't they the same type?

The 30-second answer: `str` is a sequence of Unicode code points — abstract characters. `bytes` is a sequence of raw 8-bit values — concrete bytes on the wire or on disk. Python 3 made them distinct types to force you to decode at the boundary: bytes from a file or socket become `str` via `.decode("utf-8")`; `str` becomes bytes for output via `.encode("utf-8")`. The split prevents the silent `UnicodeDecodeError` surprises that defined Python 2.

Step-by-step thought process: 1. State the conceptual divide. `str` is text; `bytes` is binary. They share many method names (`split`, `upper`, `find`) but they don't interoperate — `"abc" + b"def"` raises `TypeError`. 2. The boundary rule: decode on input, encode on output. A file opened in binary mode (`open(..., "rb")`) yields `bytes`; text mode (`open(..., "r", encoding="utf-8")`) yields `str` after decoding. 3. Default encoding: UTF-8 is the right default — supports every Unicode character, ASCII-compatible, ubiquitous on the wire. Specify it explicitly anyway; relying on the platform default (`locale.getencoding()`) bites cross-platform code. 4. Python 2 vs 3 history: Python 2's `str` was bytes and there was a separate `unicode` type. Mixing them silently decoded as ASCII and exploded on non-ASCII input. Python 3 ended that.

Edge cases to mention: - `bytes` literals look like strings with a `b` prefix: `b"hello"`. Each element is an `int` 0-255 — `b"abc"[0]` is `97`, not `b"a"`. - `str` indexing yields a one-character `str`; `bytes` indexing yields an `int`. Catches people coming from Python 2. - ASCII-only `str` and `bytes` look similar but compare unequal: `"abc" == b"abc"` is `False` in Python 3.

What NOT to say: - "They're basically the same" — they aren't, and the interview is testing that you know it. - "Just use `str` everywhere" — you can't; file I/O, sockets, hashing libraries all return `bytes`.

Common follow-up: "Convert this `bytes` to a `str` and back."

```

s = "ज्ञान"                                # 4-char string, Hindi for "knowledge"
b = s.encode("utf-8")                      # encode → bytes
print(b)                                  # b'\xe0\xa4\x9c\xe0...'
print(len(s), len(b))                     # 4, 12 – bytes count > chars

# Round-trip
print(b.decode("utf-8") == s)              # True

# Mixing fails
# print("hi" + b"hi")                     # TypeError

```

Q17 · `encode` / `decode` — What the Argument Actually Does

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Walmart Labs · Topic: Encoding

The question (interviewer's verbatim phrasing):

What does `.encode("utf-8")` do, and what's the second argument that I sometimes see (`errors="..."`)?

The 30-second answer: `.encode("utf-8")` takes a `str` (Unicode code points) and produces `bytes` according to the UTF-8 encoding rules — ASCII characters stay 1 byte, others expand to 2-4 bytes. The `errors` argument tells Python what to do when the codec can't represent a character (or can't decode a byte): `"strict"` raises (default), `"replace"` inserts `?` or `␣`, `"ignore"` drops it, `"surrogateescape"` round-trips arbitrary bytes via the surrogate range.

Step-by-step thought process: 1. Anchor: encode is `str → bytes`; decode is `bytes → str`. Memorise that direction or every Unicode bug becomes a guessing game. 2. The codec name controls the rules. UTF-8 is the default for almost everything. `latin-1` (a.k.a. ISO-8859-1) is the only encoding that round-trips every byte 0-255 unchanged, which makes it the escape hatch for "I just need to hold arbitrary bytes in a `str`". 3. The `errors` argument — `"strict"` is the default and the right choice for input validation. `"replace"` is fine for display (you'll see `?` or `␣`). `"ignore"` is for data exploration only — silent data loss in production. 4. `"surrogateescape"` is the elegant round-trip trick. It maps undecodable bytes into Unicode's private surrogate range so you can decode → modify → encode and get back exactly what you started with. The standard library uses this for filesystem paths on POSIX.

Edge cases to mention: - `bytes.decode("utf-8")` raises `UnicodeDecodeError` on invalid bytes by default — that's a feature, not a bug. Silently coercing would hide upstream corruption. - Common encodings worth knowing: UTF-8 (default), UTF-16 (Windows internals), Latin-1 (round-trip-everything), CP1252 (Windows extended Latin), ASCII (7-bit subset). - `chardet` and `charset-`

`normalizer` libraries guess the encoding of unknown text — useful for legacy data but never authoritative.

What NOT to say: - "Use `errors='ignore'` to handle bad input" — silently drops data; you'll regret it in audit. - "`encode` works on bytes too" — no, it works on `str`. `bytes` has `decode`.

Common follow-up: "What's `errors='surrogateescape'` good for?" — Round-tripping arbitrary bytes through `str` operations: read raw bytes, decode with `surrogateescape`, manipulate as if it were text, encode back to identical bytes.

```
s = "café"
b = s.encode("utf-8")           # b'caf\xc3\xa9'
print(b.decode("utf-8"))        # 'café'

# Strict (default) raises on bad input
# b'\xc3'.decode("utf-8")        # UnicodeDecodeError

# Replace inserts the replacement character
print(b'\xc3'.decode("utf-8", errors="replace")) # '?'

# surrogateescape round-trips arbitrary bytes
data = b"good\xffbad"
s = data.decode("utf-8", errors="surrogateescape")
print(s.encode("utf-8", errors="surrogateescape") == data) # True
```

Q18 · `UnicodeEncodeError` vs `UnicodeDecodeError` — Diagnosing the Difference

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Postman, JP Morgan · Topic: Encoding

The question (interviewer's verbatim phrasing):

A production job fails with `UnicodeEncodeError`. Walk me through the diagnosis.

The 30-second answer: `UnicodeEncodeError` means you tried to `encode` a `str` that contains a character the target codec can't represent — classic case: `"ज़ान".encode("ascii")`.

`UnicodeDecodeError` means you tried to `decode` bytes that aren't valid in the source codec — classic case: reading UTF-8 bytes as Latin-1 and getting mojibake, or reading binary as text. Diagnosis: print `error.object`, `error.start`, `error.end`, `error.reason` from the exception — they tell you exactly which character and why.

Step-by-step thought process: 1. The error class names the direction. `Encode` = `str → bytes`. `Decode` = `bytes → str`. Reading the error class tells you which side of the boundary is wrong. 2.

The exception object exposes the failure: `e.object` is the source, `e.start` / `e.end` are the offending offsets, `e.encoding` is the codec, `e.reason` is the human-readable cause. Log these, not just `str(e)`. 3. For encode errors, the fix is one of: switch to a wider codec (UTF-8 supports everything), use `errors="replace"` if data loss is acceptable, or sanitize the string before output. 4. For decode errors, the fix is: confirm the actual source encoding (check the file's BOM, the HTTP `charset` header, or the database collation), or use `errors="replace"` for display.

Edge cases to mention: - A `UnicodeEncodeError` on writing to stdout in a Docker container usually means the container's locale isn't set — `PYTHONIOENCODING=utf-8` fixes it. - Reading a UTF-8 file with Latin-1 codec succeeds but produces mojibake (`café` becomes `cafÃ©`) — no exception, silent corruption. Always specify the encoding. - The error attributes are the same on both classes — same code can log either.

What NOT to say: - "Just catch the exception and continue" — masks data corruption upstream. - "Switch the codec until it works" — works once, breaks the next time the data has a new character.

Common follow-up: "How would you log the offending character in a structured way?" — `logger.error("encode failed at offset %d-%d: %r in %s", e.start, e.end, e.object[e.start:e.end], e.encoding)`.

```
try:
    "ज्ञान".encode("ascii")
except UnicodeEncodeError as e:
    print(e.encoding, e.start, e.end, e.reason)
    print(repr(e.object[e.start:e.end]))

try:
    b"\xff\xfe".decode("utf-8")
except UnicodeDecodeError as e:
    print(e.encoding, e.start, e.end, e.reason)
    print(e.object[e.start:e.end])
```

Q19 · BOM Handling — UTF-8, UTF-16 and `utf-8-sig`

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Goldman, Postman · Topic: Encoding

The question (interviewer's verbatim phrasing):

Excel saves CSVs with a BOM at the start of the file. How do you read those in Python without the BOM showing up in your first column header?

The 30-second answer: Open with `encoding="utf-8-sig"` — that codec strips a leading UTF-8 BOM (`\xef\xbb\xbf`) on read and adds one on write. Plain `"utf-8"` reads the BOM as a literal

character that shows up as ` ` at the start of your first field. For UTF-16, the BOM is mandatory and signals byte order; `encoding="utf-16"` consumes it automatically.

Step-by-step thought process: 1. State what a BOM is: a leading marker (`U+FEFF`) that signals encoding and byte order. UTF-8 doesn't technically need one — UTF-8 has no byte-order ambiguity — but Windows tools (Excel, Notepad) write one anyway as an "I am UTF-8" hint. 2. The cost of ignoring it: read with plain `utf-8` and ` ` appears as a literal character in your first string. Your CSV header becomes `"name "` and matching against `"name"` fails. 3. The fix: `encoding="utf-8-sig"` on read strips a leading BOM if present, or no-ops if not. Safe default for CSVs of unknown origin. 4. On write, use `utf-8-sig` if you specifically want Windows tools to recognise the file as UTF-8 (Excel needs it); use `utf-8` otherwise — the BOM is noise.

Edge cases to mention: - UTF-16 files need a BOM to disambiguate `\xff\xfe` (LE) vs `\xfe\xff` (BE). Reading as `utf-16` reads and consumes the BOM; reading as `utf-16-le` or `utf-16-be` does not — you'll see ` ` as a literal character. - A BOM mid-stream is not a BOM; it's a zero-width no-break space. Some sloppy concatenations introduce this — strip with `s.lstrip("\u200b")`. - `csv.reader` on a `utf-8-sig` file works correctly; on plain `utf-8` the BOM leaks into the first cell.

What NOT to say: - "Strip the first character of the first line" — brittle; misses some BOM variants and doubles up if the file already has none. - "UTF-8 always has a BOM" — it doesn't; many Unix tools write UTF-8 without a BOM.

Common follow-up: "How do you write a CSV that Excel will open as UTF-8?" — `open(..., "w", encoding="utf-8-sig", newline="")` — Excel needs the BOM and CRLF.

```
# Write a CSV the Windows way and read it back two ways
with open("/tmp/x.csv", "w", encoding="utf-8-sig", newline="") as f:
    f.write("name,price\u094d\u0940\u094d,499\u094d\u0940\u094d")

with open("/tmp/x.csv", "r", encoding="utf-8") as f:
    first = f.readline()
    print(repr(first))                # 'name,price\u094d\u0940\u094d' - BOM leaks

with open("/tmp/x.csv", "r", encoding="utf-8-sig") as f:
    first = f.readline()
    print(repr(first))                # 'name,price\u094d\u0940\u094d' - clean
```

Q20 · `latin-1` as the Byte Round-Trip Bypass

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Walmart Labs · Topic: Encoding

The question (interviewer's verbatim phrasing):

Why do some libraries use `encoding="latin-1"` even for non-Latin data? What does that get you?

The 30-second answer: Latin-1 (ISO-8859-1) is the only common encoding where every byte 0-255 maps to a single Unicode code point with no errors and no ambiguity — bytes round-trip through `str` losslessly. That makes it the "container" encoding when you need to carry arbitrary bytes through a string-typed API (HTTP headers in some servers, legacy databases, hash inputs in older crypto). It is *not* a text encoding for unknown data.

Step-by-step thought process: 1. The property that matters:

`bytes(range(256)).decode("latin-1")` produces a 256-character string with no errors; `.encode("latin-1")` of that string gives back the exact same bytes. UTF-8 can't do this — UTF-8 has invalid byte sequences. 2. Where you see it used: WSGI headers used to require Latin-1 byte-string round-trips; some MySQL collation defaults are Latin-1; some hash libraries decode arbitrary byte input as Latin-1 to operate on `str` internally. 3. Where you shouldn't use it: as the encoding for real human text. Decoding UTF-8 as Latin-1 produces mojibake (`café` becomes `cafÃ©`) — silent corruption that survives encode/decode cycles. 4. The right modern pattern: keep `bytes` as `bytes` end-to-end. Only fall back to Latin-1 as a container when the API insists on `str`.

Edge cases to mention: - `bytes.decode("latin-1")` always succeeds. That's the property; it's also what makes it dangerous as a default. - Latin-1 vs CP1252: CP1252 (Windows-1252) is a superset that fills the C1 control range with extra printable characters (smart quotes, Euro sign). Most "Latin-1" data in the wild is actually CP1252 — try CP1252 if Latin-1 produces control characters. - The pattern `s.encode("latin-1", errors="surrogateescape")` is the standard idiom for converting a surrogateescape-decoded string back to original bytes.

What NOT to say: - "Latin-1 is for European text" — it covers Western European Latin characters, but the round-trip property is why people use it programmatically. - "Use Latin-1 for unknown encoding" — you'll get mojibake. Use `chardet/charset-normalizer` to detect, then decode properly.

Common follow-up: "How does WSGI handle headers in Python 3?" — PEP 3333 requires headers to be `str` containing only Latin-1 round-trippable code points (bytes 0-255). The server decodes the raw header bytes as Latin-1 for the application.

```
raw = bytes(range(256))           # all 256 byte values
s = raw.decode("latin-1")         # never raises
print(len(s) == 256)             # True
print(s.encode("latin-1") == raw) # True — perfect round-trip

# UTF-8 can't do this
# bytes([0xff, 0xfe]).decode("utf-8") # UnicodeDecodeError
```

Q21 · Unicode Normalization — NFC vs NFD vs NFKC vs NFKD

Tags: Difficulty: Hard · Asked at: Razorpay, Walmart Labs, Goldman, JP Morgan · Topic: Unicode

The question (interviewer's verbatim phrasing):

Two strings look identical on screen but `==` returns False. What's going on, and how do you fix it?

The 30-second answer: Unicode has multiple ways to represent the same visible character. `é` can be a single code point (`U+00E9`) or `e` plus a combining acute accent (`U+0065 U+0301`). They render identically but compare unequal. Fix: normalise both sides before comparison via `unicodedata.normalize("NFC", s)` — NFC composes them into the precomposed single code point.

Step-by-step thought process: 1. State the cause: Unicode allows precomposed and decomposed forms of the same character. macOS filesystems hand you decomposed (NFD); most web input is composed (NFC). Same name typed on a Mac and on Windows can compare unequal in your database. 2. The four normalisation forms: NFC (composed, default for almost everything), NFD (decomposed), NFKC (compatibility composed), NFKD (compatibility decomposed). The K variants also collapse "compatibility" characters — `①` becomes `1`, `fi` ligature becomes `fi`. 3. The rule for storage: pick one form (NFC for human text) and normalise at the boundary. Don't try to normalise on every comparison — pre-normalise on input. 4. The K forms (NFKC, NFKD) are aggressive — they drop semantic distinctions. Use them for search and dedup; never for storage of source-of-truth text (you'd lose the original ligature).

Edge cases to mention: - The number of code points changes — NFD often produces longer strings than NFC. Hash and length checks differ across forms. - `casefold` and normalization should be combined for case-insensitive search: `unicodedata.normalize("NFKC", s).casefold()` is the strongest equivalence. - Database collation matters too — Postgres `citext` and MySQL `utf8mb4_unicode_ci` normalise comparisons; relying on the DB instead of Python normalisation is sometimes simpler.

What NOT to say: - "Just use `==`" — fails on Unicode equivalent strings. - "Normalise to NFKD always" — strips legitimate distinctions you might need later.

Common follow-up: "What's the difference between NFC and NFKC?" — NFC composes canonical equivalents; NFKC additionally maps compatibility characters (ligatures, superscripts, full-width forms) to their canonical equivalents.

```
import unicodedata

a = "café"                # composed: c, a, f, é (4 code points)
b = "café"                # decomposed: c, a, f, e + combining acute (5)
print(a == b)             # False – different code-point sequences
print(len(a), len(b))    # 4, 5

c = unicodedata.normalize("NFC", b)
print(c == a)            # True

# NFKC strips compatibility distinctions
print(unicodedata.normalize("NFKC", "①fi")) # '1fi'
```

Q22 · `len(str)` and Surrogate Pairs — Why It's Not Character Count

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, JP Morgan · Topic: Unicode

The question (interviewer's verbatim phrasing):

What does `len("\U0001F600")` return? And what's a "grapheme cluster"?

The 30-second answer: `len("\U0001F600")` returns `1` in Python 3 — Python's `str` indexes by Unicode code points and an emoji is one code point. But what users perceive as "one character" — a grapheme cluster — can span multiple code points: family emoji like 🧑👤 is a sequence of code points joined by zero-width joiners and `len` returns 5, not 1. For user-visible character count, use the `regex` (not `re`) module's `\X` or the `grapheme` library.

Step-by-step thought process: 1. State Python's design: `len(s)` counts code points, not graphemes. For most strings these match. For emoji with modifiers and combining marks, they don't. 2. Surrogate pairs are a UTF-16 implementation detail — Python 3 `str` uses a wider internal representation, so surrogate pairs aren't a Python concern. They become an issue when interoperating with JavaScript or Java, where `length` counts UTF-16 code units. 3. Grapheme clusters: defined in Unicode UAX #29. The family emoji 🧑👤 is `Man + ZWJ + Woman + ZWJ + Girl` — three faces, two joiners, perceived as one symbol. `len` returns 5; a grapheme-aware count returns 1. 4. For "how many characters did the user type", reach for the `regex` module: `len(regex.findall(r"\X", s))` counts grapheme clusters.

Edge cases to mention: - `s[0]` on a string starting with a family emoji returns just the first man — a partial grapheme. Indexing user-typed text by integer offset is unsafe. - Slicing similarly: `s[:1]` can split a grapheme. Use grapheme-aware slicing for any UI that displays prefixes. - UTF-8 byte length is different again: `"😊".encode("utf-8")` is 4 bytes. Three distinct "lengths" — bytes, code points, graphemes.

What NOT to say: - "`len(str)` returns the number of characters" — only true for plain ASCII. Code points ≠ characters for users. - "Emoji are surrogate pairs in Python" — they aren't in Python 3; that's a Java/JS legacy concern.

Common follow-up: "How would you truncate a string to *N* user-visible characters?" — Use `regex.findall(r"\X", s)[:n]` and `"".join(...)`. Never integer-slice.

```
import unicodedata

print(len("😊")) # 1 - one code point
print(len("\U0001F600".encode("utf-8"))) # 4 - bytes
print(len("👨👩👦")) # 5 - code points (3 faces + 2 ZWJs)

# Grapheme count needs the third-party `regex` module
# import regex
# print(len(regex.findall(r"\X", "👨👩👦"))) # 1
```

Q23 · `chr` / `ord` / `\u` / `\U` / `\N` — Producing Code Points

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred · Topic: Unicode

The question (interviewer's verbatim phrasing):

How would you write the Indian Rupee sign (₹) in a string literal? Show me three ways.

The 30-second answer: Three options. Literally typed: `"₹"` if your editor supports UTF-8 (which is almost everything in 2026). Escape: `"\₹"` for code points up to `U+FFFF`, `"\U0001F600"` for higher ones (note the capital U and 8 hex digits). By Unicode name: `"\N{INDIAN RUPEE SIGN}"`. At runtime: `chr(0x20B9)`.

Step-by-step thought process: 1. State the four mechanisms. Literal (paste the character), `\u` (4-digit), `\U` (8-digit), `\N{NAME}` (by Unicode name). `chr(n)` produces a one-character string at runtime. 2. The matching reverse: `ord("₹")` returns `8377`, the decimal of `U+20B9`. `hex(ord(c))` formats it as `0x20b9`. 3. When to use which. Literal for source clarity. `\u` / `\U` for control characters that can't be typed (zero-width space). `\N{NAME}` for documentation value — `"\N{COPYRIGHT SIGN}"` reads better than `"©"`. `chr` for dynamic generation. 4. Raw strings disable backslash escapes — including `\u`. `r"₹"` is six literal characters, not a rupee sign.

Edge cases to mention: - `\u` requires exactly 4 hex digits; `\U` requires exactly 8. Anything else is a `SyntaxError` or unintended literal. - `\N{...}` `SyntaxError` on unknown names — caught at compile time, which is what you want. - The Unicode name database is built into Python; no separate library needed.

What NOT to say: - "I'd paste from a website" — fine for the editor; the question is asking for the syntax.
 - " ₹ is the only way" — there are four. Knowing them signals breadth.

Common follow-up: "What does `ord` return for a multi-character string?" — `TypeError`. `ord` is for single-character strings only.

```
print("₹")                # literal – works because source is UTF-8
print("\u0947")          # 4-digit hex escape
print("\U000020B9")      # 8-digit form
print("\N{INDIAN RUPEE SIGN}") # by Unicode name
print(chr(0x20B9))       # dynamic
print(ord("₹"), hex(ord("₹"))) # 8377 0x20b9
```

Q24 · Reading and Writing Files — `encoding=` Is Not Optional

Tags: Difficulty: Medium · Asked at: TCS, Razorpay, Walmart Labs · Topic: Encoding · I/O

The question (interviewer's verbatim phrasing):

What encoding does `open("file.txt")` use by default? Why is that a problem?

The 30-second answer: The default is `locale.getencoding()` — UTF-8 on modern Linux/macOS, CP1252 on Windows by default. Code that works on your laptop can fail on a Windows CI runner. Always pass `encoding="utf-8"` explicitly. Python 3.10+ also added the `PYTHONUTF8=1` environment variable and PEP 686 will make UTF-8 the default in 3.15.

Step-by-step thought process: 1. State the failure mode. Open a file with non-ASCII content, get a `UnicodeDecodeError` on Windows but not on Linux. Same code, same file, different platform. Specifying `encoding=` makes the platform irrelevant. 2. The platform default exists for historical reasons — Python predates the universal-UTF-8 era. The PEP 686 (3.15) move makes UTF-8 the default but flag it for compatibility. 3. The `EncodingWarning` (3.10+) — set `PYTHONWARNINGS=default::EncodingWarning` or run with `-X warn_default_encoding` to get explicit warnings whenever code opens a file without an encoding. Excellent for finding latent bugs in legacy code. 4. For binary files (images, PDFs, pickles), open with `"rb"` / `"wb"` — no encoding, no decoding, raw bytes only.

Edge cases to mention: - `newline=""` is needed for `csv` readers and writers — otherwise Python's universal newline mode mangles embedded `\r\n` inside CSV cells. - For network sources, the encoding comes from the HTTP `Content-Type` header's `charset` parameter. `requests`'s `.encoding` and `.text` handle this; raw `urllib` doesn't. - `errors=` is also available on `open` — `open(..., encoding="utf-8", errors="replace")` for "best effort" reads.

What NOT to say: - "It uses UTF-8 by default" — false on Windows pre-3.15. - "Encoding doesn't matter for ASCII files" — until someone writes a € and the file becomes non-ASCII.

Common follow-up: "Show me how you'd open a CSV that Excel produced." — `open(path, "r", encoding="utf-8-sig", newline="")`.

```
import locale
print(locale.getencoding())           # UTF-8 on Linux/macOS, often CP1252 on Windows

# Always specify
with open("/tmp/data.txt", "w", encoding="utf-8") as f:
    f.write("₹499 🍕")

# Binary: no encoding
with open("/tmp/blob.bin", "wb") as f:
    f.write(b"\x00\x01\x02")
```

Q25 · `unicodedata` Module — `category`, `name`, `numeric`

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, JP Morgan · Topic: Unicode

The question (interviewer's verbatim phrasing):

How would you strip every punctuation character from a string, including Unicode punctuation like Hindi danda (।)?

The 30-second answer: Use `unicodedata.category(c)` — it returns a two-letter Unicode general category code. Punctuation categories start with `"P"`. Build the result with `"".join(c for c in s if not unicodedata.category(c).startswith("P"))`. `str.punctuation` (from `string`) only covers ASCII punctuation, so Hindi danda, Chinese full-width punctuation, and curly quotes slip through.

Step-by-step thought process: 1. Map the problem to Unicode general categories. Every code point has a 2-letter category — `Lu` (Letter, uppercase), `Ll` (Letter, lowercase), `Nd` (Number, decimal), `Po` (Punctuation, other), `Zs` (Separator, space), and so on. See UAX #44. 2. For "strip punctuation", filter on category prefix `"P"`. Covers all 7 punctuation subcategories (Pc, Pd, Pe, Pf, Pi, Po, Ps). 3. `unicodedata.name(c)` returns the Unicode name — useful for diagnostic logging ("removed character U+0964 DEVANAGARI DANDA"). 4. `unicodedata.numeric(c)` returns the numeric value of digit-like characters (Roman numerals, fractions, superscripts). Avoid for parsing — use `int(s)` which is stricter.

Edge cases to mention: - `unicodedata.name` raises `ValueError` for unnamed code points (control chars, private use area). Use the two-arg form with a default: `unicodedata.name(c, "")`.

For "is this a digit?", `c.isdigit()` accepts Unicode digits too (`"१"` returns `True` for Devanagari one). `c.isdecimal()` is stricter. - The Unicode version that ships with Python is fixed per release — `unicodedata.unidata_version` tells you which. Categories of recently-added characters depend on this.

What NOT to say: - "Use `str.translate` to remove punctuation" — only works if you enumerate every punctuation code point, which you can't reliably do. - "Use `string.punctuation`" — ASCII-only; misses every Unicode punctuation character.

Common follow-up: "How would you check if a string is all digits including Devanagari digits?" — `all(c.isdecimal() for c in s)` works (it's Unicode-aware).

```
import unicodedata

text = "Hello, ज्ञान। नमस्ते!"
clean = "".join(c for c in text if not unicodedata.category(c).startswith("P"))
print(clean)                                # 'Hello ज्ञान नमस्ते'

print(unicodedata.name("।"))                # 'DEVANAGARI DANDA'
print(unicodedata.category("।"))            # 'Po'
print("१".isdecimal())                      # True — Devanagari digit
```

Section D — bytes, bytearray, memoryview (Q26–Q30)

Q26 · bytes vs bytearray — Immutable vs Mutable

Tags: Difficulty: Easy · Asked at: Razorpay, Walmart Labs, Postman, Goldman · Topic: Bytes

The question (interviewer's verbatim phrasing):

When would you use `bytearray` instead of `bytes` ? Show me a case where it matters.

The 30-second answer: `bytes` is immutable — every modification allocates a new buffer.

`bytearray` is the mutable twin — you can append, extend, slice-assign in place. Use `bytearray` when you're assembling a binary buffer in a loop (the same way you'd use `list` instead of repeated `tuple` concatenation). Use `bytes` for keys, hashes, network output, and anything you want to be safely shared.

Step-by-step thought process: 1. State the symmetry: `bytes` is to `str` as `bytearray` is to `list` — immutable vs mutable sequence of small integers (0-255). 2. Mutability buys you: `b.append(0x41)`, `b.extend(data)`, `b[5] = 0x42`, `del b[3]`. None of those work on `bytes`. 3. Concatenation cost: same $O(n^2)$ problem as strings. Building a 10 MB binary by

repeated `bytes` concatenation is hours; `bytearray.extend` in a loop is milliseconds. 4. Conversion is cheap: `bytes(ba)` makes an immutable copy; `bytearray(b)` makes a mutable copy. Pass `bytearray` internally, return `bytes` from public APIs.

Edge cases to mention: - Hashing: `bytes` is hashable (usable as dict key, set member); `bytearray` is not. - `bytearray` has the same methods as `bytes` plus mutation methods (`append`, `extend`, `pop`, `clear`). - Indexing both returns `int`: `b"abc"[0]` is `97`. To get a one-byte `bytes`, slice: `b"abc"[0:1]`.

What NOT to say: - "Use `bytes` always; mutation isn't needed" — true for high-level code, false for binary protocol assembly. - "`bytearray` is faster" — only for in-place build; for lookup and pass-around, `bytes` is the right type.

Common follow-up: "How would you build a 1 MB random buffer efficiently?" — `bytearray(os.urandom(1024))` for chunk in ... then `bytes(ba)` at the end, or `os.urandom(1_000_000)` directly if size is known.

```
# Mutable build
buf = bytearray()
for chunk in [b"\x00\x01", b"\x02\x03", b"\x04\x05"]:
    buf.extend(chunk)
print(bytes(buf))                # b'\x00\x01\x02\x03\x04\x05'

# bytes is hashable, bytearray is not
d = {b"key": 1}                  # OK
# {bytearray(b"key"): 1}          # TypeError: unhashable type

# Indexing returns int; slice returns bytes
print(b"abc"[0])                 # 97
print(b"abc"[0:1])               # b'a'
```

Q27 · `memoryview` — Zero-Copy Slicing

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, JP Morgan, Walmart Labs · Topic: Memory

The question (interviewer's verbatim phrasing):

I'm parsing a 100 MB binary file and I keep slicing chunks out of it. Why is `memoryview` faster than indexing into the `bytes` directly?

The 30-second answer: Slicing `bytes` copies. `b[10:20]` allocates a new 10-byte object. With a `memoryview`, `mv[10:20]` returns a view — no copy, no allocation, just a pointer-offset record. For

a 100 MB buffer parsed via many small slices, the allocation pressure of slicing `bytes` dominates; `memoryview` removes it.

Step-by-step thought process: 1. Define the type: `memoryview` is a Python wrapper around the C-level buffer protocol. It exposes the underlying buffer of `bytes`, `bytearray`, `array.array`, NumPy arrays, etc., without copying. 2. The win: slicing a view returns another view, sharing the same underlying memory. Reading values (`mv[5]`) goes straight to the source bytes. No allocations until you actually materialise (`bytes(mv[10:20])`). 3. Mutability: a `memoryview` over `bytes` is read-only; over `bytearray` it's writable. Writes to the view modify the underlying buffer. 4. When it matters: parsers, binary protocol decoders, numerical pipelines that pass large arrays between layers. For 1 KB strings it's negligible; for 100 MB it's order-of-magnitude.

Edge cases to mention: - The view holds a reference to the underlying buffer — you can't free it while a view exists. - Multi-dimensional views (over NumPy arrays) are real; `mv.shape` and `mv.strides` give the layout. - `mv.release()` releases the view explicitly; useful when you want to free the underlying buffer for an in-place operation.

What NOT to say: - "Use `memoryview` for everything" — overhead for small buffers; only wins on large ones. - "`memoryview` is a `bytes` view" — it's a buffer-protocol view; the source can be `bytes`, `bytearray`, `array.array`, NumPy, mmap.

Common follow-up: "What's the difference between a `memoryview` and a NumPy array slice?" — Both are zero-copy views via the buffer protocol; NumPy adds dtype, broadcasting, and arithmetic. Same memory under the hood when interoperating.

```
big = bytes(range(256)) * 1000          # 256 KB
mv = memoryview(big)

# Zero-copy slicing
chunk = mv[100:200]                    # view, no copy
print(len(chunk), bytes(chunk[:5]))    # 100 b'dghij' (wait, idx 100 in pattern)

# Writable view over bytearray
ba = bytearray(b"hello world")
mv2 = memoryview(ba)
mv2[0:5] = b"HELLO"
print(ba)                              # bytearray(b'HELLO world')
```

Q28 · The Buffer Protocol — Why It Matters

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Goldman · Topic: Memory

The question (interviewer's verbatim phrasing):

What does it mean when a library says "any object that supports the buffer protocol"? Give me three examples.

The 30-second answer: The buffer protocol is the C-level interface that lets one object expose its raw bytes to another without copying. `bytes`, `bytearray`, `array.array`, `memoryview`, NumPy `ndarray`, and `mmap` all support it. That's why `socket.send(numpy_array)` or `hashlib.md5(memoryview)` work without round-tripping through `bytes`.

Step-by-step thought process: 1. State the goal: zero-copy data sharing between Python objects. Pickling a NumPy array, hashing a `bytearray`, sending an `array.array` over a socket — all need direct access to underlying bytes. 2. The protocol has two flavors: simple (the object is a flat byte buffer) and structured (with shape, stride, dtype info — NumPy uses this). 3. Common producers: `bytes`, `bytearray`, `memoryview`, `array.array("i", [...])`, `mmap.mmap(...)`, `numpy.ndarray`, anything wrapping a C-level buffer. 4. Common consumers: `struct.unpack_from`, `hashlib.md5(...).update(...)`, `socket.sendall(...)`, `os.write(...)`, `pickle` for large arrays. All accept any buffer-protocol object.

Edge cases to mention: - The C-level `Py_buffer` struct carries a release callback — that's why `with memoryview(...) as mv:` is the idiomatic pattern (releases the source). - Read-only vs writable is part of the protocol. `socket.recv_into(b)` needs a writable buffer; a `bytes` won't work, a `bytearray` will. - `struct.pack_into(buf, ...)` writes into any writable buffer — useful for binary protocol assembly.

What NOT to say: - "It's how `bytes` works internally" — partial; it's how `bytes` *interoperates*, not how it's stored. - "Only C extensions use it" — Python code uses it transparently every time you pass `bytes` to a hashing or socket call.

Common follow-up: "Why is `hashlib.md5(numpy_array)` faster than `hashlib.md5(numpy_array.tobytes())`?" — `.tobytes()` allocates a copy; passing the array directly uses the buffer protocol with zero copy.

```
import hashlib, array

# All four are buffer-protocol providers
data = b"hello"
print(hashlib.md5(data).hexdigest())

ba = bytearray(b"hello")
print(hashlib.md5(ba).hexdigest())

arr = array.array("b", b"hello")
print(hashlib.md5(arr).hexdigest())

mv = memoryview(b"hello")
print(hashlib.md5(mv).hexdigest())
# All four produce the same digest – no copying
```

Q29 · Converting `bytes` ↔ `int` , `hex` , `base64`

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Walmart Labs, Postman · Topic: Bytes

The question (interviewer's verbatim phrasing):

How do you convert `bytes` to a hex string, an integer, and base64? And back?

The 30-second answer: Hex: `b.hex()` and `bytes.fromhex(s)` . Integer (3.2+): `int.from_bytes(b, "big")` and `i.to_bytes(length, "big")` . Base64: `base64.b64encode(b)` and `base64.b64decode(s)` . All are stdlib; no third-party library needed.

Step-by-step thought process: 1. Hex is the fastest way to print bytes for debugging. `b"\xff\x00".hex()` → `"ff00"` . The reverse, `bytes.fromhex("ff00")` , accepts spaces (`"ff 00"`) but not arbitrary separators. 2. Integers from bytes — needs a byte order. Network/serialization typically uses `"big"` (network byte order, most-significant byte first). x86 native is `"little"` . Be explicit; defaults differ by API. 3. Base64 is for text-safe byte transport — emails, JSON values, URL parameters. `urlsafe_b64encode` replaces `+` and `/` with `-` and `_` for URL contexts. 4. Avoid `b.decode("ascii")` on binary data — it raises on bytes ≥ 128 . The encodings above are the right tools.

Edge cases to mention: - `int.to_bytes(length, "big")` raises `OverflowError` if the int doesn't fit in `length` bytes. For dynamic-length encoding, compute `length = (i.bit_length() + 7) // 8` . - Base64 with padding — `b64encode` always pads with `=` to a multiple of 4 chars. `b64decode` accepts both padded and (with `validate=True`) strictly valid. - The `bytes.hex(sep)` form (3.8+) takes a separator: `b.hex(":")` → `"ff:00"` . Useful for MAC-address style output.

What NOT to say: - "Use `str(b)`" — that gives you the `b'...'` repr literal, not a clean hex string. - "Cast to int directly" — `int(b)` raises; you need `int.from_bytes`.

Common follow-up: "What's the difference between `base64.b64encode` and `base64.urlsafe_b64encode`?" — Same encoding except `+` → `-` and `/` → `_` for URL/path safety.

```
import base64

b = b"\xff\x01\x02"

# Hex
print(b.hex())           # 'ff0102'
print(bytes.fromhex("ff0102")) # b'\xff\x01\x02'
print(b.hex(":"))        # 'ff:01:02' (3.8+)

# Integer
i = int.from_bytes(b, "big")
print(i)                 # 16711938
print(i.to_bytes(3, "big") == b) # True

# Base64
print(base64.b64encode(b)) # b'/wEC'
print(base64.urlsafe_b64encode(b)) # b'_wEC'
```

Q30 · `bytes` Methods — `split`, `replace`, `decode` Mirror `str`

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Postman · Topic: Bytes

The question (interviewer's verbatim phrasing):

Can I call `.split()` and `.replace()` on `bytes` the same way I do on `str`?

The 30-second answer: Yes — `bytes` mirrors most of `str`'s methods.

`b"a,b,c".split(b",")` works; so do `replace`, `find`, `index`, `startswith`, `endswith`, `strip`, `upper`, `lower`. The arguments must be `bytes`, not `str` — mixing raises `TypeError`. Same applies to `bytearray`.

Step-by-step thought process: 1. State the mirror principle: `bytes` and `str` share API for sequence operations because both are sequences of small units. Different element types, same operations. 2. The arguments must match the receiver type — `b"abc".split(",")` raises (str arg on bytes); `b"abc".split(b",")` works. 3. The case methods (`upper`, `lower`, `title`, `swapcase`) operate on ASCII letters only — they don't know what to do with bytes that aren't ASCII letters. 4. Some methods are bytes-specific: `bytes.fromhex`, `bytes.hex`, `bytes.translate`

with `delete` keyword. And some `str` methods are absent on `bytes` — there's no `bytes.encode` (you can only encode `str`).

Edge cases to mention: - `bytes.translate` takes a 256-byte table (one entry per byte value) or a single `str → bytes` map from `bytes.maketrans`. Useful for byte-level character mapping. - `bytes.splitlines()` handles all the same line terminators as `str.splitlines()`, returning `bytes` per line. - `bytes.startswith` accepts a tuple of options, same as `str`: `b.startswith(("GET ", "POST "))`.

What NOT to say: - "`bytes` is read-only so you can't call methods on it" — methods return new `bytes`; that's exactly the same as `str`. - "Convert to `str` first" — wastes a decode and an encode and risks `UnicodeDecodeError` on binary.

Common follow-up: "How would you *replace* bytes case-insensitively?" — `re.sub` with `re.IGNORECASE` works on `bytes` too: `re.sub(rb"foo", b"BAR", data, flags=re.IGNORECASE)`.

```
b = b"GET /api/users HTTP/1.1"

print(b.split(b" "))           # [b'GET', b'/api/users', b'HTTP/1.1']
print(b.startswith(("GET ", "POST "))) # True
print(b.replace(b"/users", b"/orders")) # b'GET /api/orders HTTP/1.1'
print(b.upper())               # b'GET /API/USERS HTTP/1.1' - ASCII only

# Mixing fails
# b"x".split(",")           # TypeError
```

Section E — Regex Basics (Q31–Q40)

Q31 · `re.match` vs `re.search` vs `re.fullmatch`

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman, Cognizant · Topic: Regex

The question (interviewer's verbatim phrasing):

What's the difference between `re.match`, `re.search`, and `re.fullmatch`?

The 30-second answer: `re.match` anchors to the start of the string only — succeeds if the pattern matches at position 0. `re.search` scans the whole string for a match anywhere. `re.fullmatch` requires the pattern to match the *entire* string from start to end. Most code wants `search`; `fullmatch` is the safe choice for validation; `match` is rarely the right tool.

Step-by-step thought process: 1. Anchor each to its anchoring behaviour. `match` is start-anchored, no end constraint. `search` is unanchored. `fullmatch` is both-anchored. 2. The common confusion: `re.match(r"\d+", "abc123")` returns `None` because the pattern doesn't match at position 0. `re.search` returns a match for "123". Beginners reach for `match` thinking it means "look for a match" — it means "match at the start." 3. For validation ("is this string a valid postcode?"), `fullmatch` is the right tool — it guarantees nothing trails after the pattern. `match` would accept "110001abc" because it doesn't anchor the end. 4. All three return either a `Match` object (truthy) or `None`. Use `if (m := re.search(...)):` (walrus) to capture and test in one expression.

Edge cases to mention: - `re.match` is *not* `re.search` with `^` — `^` matches start of line (or string), but `match` doesn't anchor the end either way. They differ on patterns that succeed mid-string. - All three are wrappers around `Pattern.match` / `.search` / `.fullmatch` on a compiled pattern. The wrappers use an internal LRU cache for the compiled pattern. - With `re.MULTILINE`, `^` matches at the start of each line, but `re.match` still only checks position 0 (not the start of lines).

What NOT to say: - "They're aliases" — they aren't. - "Use `match` for validation" — `fullmatch` is what you want.

Common follow-up: "How would you check if a string is exactly a valid 10-digit phone number?" — `re.fullmatch(r"\d{10}", s)`.

```
import re

s = "abc123def"
print(re.match(r"\d+", s))           # None — no digits at position 0
print(re.search(r"\d+", s))          # <Match span=(3, 6) match='123'>
print(re.fullmatch(r"\d+", s))       # None — there's non-digits
print(re.fullmatch(r"\d+", "123"))   # <Match ...> — valid
```

Q32 · `re.findall` vs `re.finditer` — When to Use Which

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Postman, Flipkart · Topic: Regex

The question (interviewer's verbatim phrasing):

What's the difference between `re.findall` and `re.finditer`? Why would I pick one over the other?

The 30-second answer: `findall` returns a list of all matches (or, if the pattern has groups, a list of group tuples). `finditer` returns an iterator of `Match` objects. Use `finditer` when you want access to the `Match` API (`m.start()`, `m.group()`, `m.groupdict()`) or when the input is

large enough that materialising every match into a list is wasteful. Use `findall` for "give me the strings" — it's terser.

Step-by-step thought process: 1. Output shape first. `findall` returns strings (no groups), single strings (one group), or tuples (multiple groups). `finditer` always returns Match objects regardless of group count. 2. Memory: `findall` materialises everything; `finditer` is lazy. For multi-GB log files, `finditer` wins on memory. 3. API: `Match` objects expose positions (`m.start` , `m.end`), the full match (`m.group(0)`), each group (`m.group(1)` , `m.group("name")`), and the dict of named groups (`m.groupdict()`). `findall` discards all that. 4. Performance: per-match they're comparable; `finditer` skips the list allocation for huge match counts. For small inputs, the difference is noise.

Edge cases to mention: - `findall` returns a list of tuples *only* when the pattern has 2+ groups. With 1 group, it returns a list of strings. With 0 groups, list of full matches. Inconsistent, but documented. - `Match.expand(template)` lets you substitute `\1` , `\g<name>` etc. into a template — handy for transformations during iteration. - Zero-width matches: a pattern like `r"(?=\w)"` matches at every word-character start; `findall` returns a list of empty strings, `finditer` returns Match objects with `start() == end()` .

What NOT to say: - "They're the same with different output" — different memory model and different API surface matters. - "Always use `findall`" — wastes memory on large inputs and loses positional info.

Common follow-up: "Show me the difference in output for a pattern with one group vs two."

```
import re

text = "Order 1001 - 499. Order 1002 - 999."

# findall - strings (1 group) or tuples (2+ groups)
print(re.findall(r"Order (\d+)", text))          # ['1001', '1002']
print(re.findall(r"Order (\d+) - (\d+)", text))  # [('1001', '499'), ('1002', '999')]

# finditer - Match objects with positions and groups
for m in re.finditer(r"Order (\d+) - (\d+)", text):
    print(m.start(), m.group(1), m.group(2))
```

Q33 · `re.compile` — When the LRU Cache Isn't Enough

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, JP Morgan, Goldman · Topic: Regex · Performance

The question (interviewer's verbatim phrasing):

Does `re.compile` actually save time? The module-level functions also cache patterns.

The 30-second answer: The module-level functions (`re.match` , `re.search` , etc.) maintain an LRU cache of about 512 compiled patterns, so the same pattern string reuses its compilation. But the cache lookup itself has overhead — for a tight loop, `pattern = re.compile(...)` once and `pattern.search(...)` repeatedly skips the lookup. The win is small per call but adds up. For variable patterns or in hot loops, compile explicitly.

Step-by-step thought process: 1. State the cache behaviour: `re._MAXCACHE = 512` by default; the cache stores `(pattern_string, flags) -> compiled` and evicts LRU. So the second call with the same pattern hits the cache. 2. Why compile explicitly anyway: cache lookup involves a dict lookup, a tuple hash, and the function-call overhead of the wrapper. `pattern.search(s)` is a direct method call. 3. The other reason: a `Pattern` object has methods (`pattern.findall` , `pattern.sub` , `pattern.split`) and attributes (`pattern.pattern` , `pattern.groups` , `pattern.groupindex`) that are useful for introspection. 4. When the cache fails: if you have more than 512 distinct patterns (rare), the LRU evicts and recompilation costs add up. Compile and store explicitly to avoid eviction.

Edge cases to mention: - `re.purge()` clears the cache — useful in tests that want to measure compilation cost. - The cache is per process; subprocesses re-compile from scratch. - For very simple patterns (constants), `str.find` is faster than `re.search` — measure before reaching for regex.

What NOT to say: - "Always compile for performance" — true for hot loops, irrelevant for one-shot use. - "The cache is unlimited" — it's bounded; 512 entries.

Common follow-up: "How big is the regex cache and can I change it?" — `re._MAXCACHE = 1024` is supported but undocumented. Compile-and-store is the cleaner pattern.

```
import re

# Module-level – cached automatically
for _ in range(1000):
    re.search(r"\d+", "abc123")           # cache hit after first call

# Explicit compile – slightly faster in tight loops
pattern = re.compile(r"\d+")
for _ in range(1000):
    pattern.search("abc123")

# Pattern object has introspection
print(pattern.pattern)                   # '\\d+'
print(pattern.groups)                    # 0
```

Q34 · Raw Strings — Why `r"..."` for Patterns

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman · Topic: Regex

The question (interviewer's verbatim phrasing):

Why do regex examples always use `r"..."` ? What happens if I forget?

The 30-second answer: Regex uses backslash heavily (`\d` , `\w` , `\b` , `\s`). Without a raw string, Python's string parser sees the backslash first and interprets escape sequences — `"\b"` becomes a backspace character before the regex engine sees it. Raw strings disable Python-level backslash interpretation, so `r"\b"` reaches the regex engine intact. Always use `r"..."` for patterns; it removes one whole class of bug.

Step-by-step thought process: 1. State the two-level escape problem. Python sees the string literal first and applies its own escape rules. Then the regex engine sees the resulting string and applies *its* escape rules. Without raw strings, you'd need to double-escape every backslash. 2. Concrete example: `"\d+"` and `r"\d+"` are the same pattern. Easy to forget the doubling; raw strings remove the choice. 3. `\n` , `\t` — those *are* meaningful in regex (newline, tab). Even there, `r"\n"` works because the regex engine handles `\n` itself. 4. The only case where raw strings bite: a raw string can't end with a single backslash (`r"foo\"` is a syntax error). Rare in regex patterns; relevant for Windows paths.

Edge cases to mention: - `re.escape` returns a regular (not raw) string — but the contents are pre-escaped, so it's safe to concatenate. - Raw f-strings: `rf"\d+{name}"` works since 3.6 — combines both. - `b"..."` and `rb"..."` for byte-pattern regex on `bytes` input.

What NOT to say: - "Use raw strings for performance" — they're parsed the same way at runtime; the win is correctness. - "It's just convention" — it's a correctness guarantee against escape-sequence confusion.

Common follow-up: "Why can't a raw string end with `\` ?" — Lexer rule: raw strings still have to find a closing quote, and a backslash before the quote means "literal backslash followed by literal quote" which is ambiguous with "escaped quote."

```
import re

# Without raw: backslash chaos
print("\b" == "\x08")           # True — \b is backspace
# print(re.search("\b\w+", "  hello")) # No match — \b became backspace

# With raw: regex sees it intact
print(re.search(r"\b\w+", "  hello")) # <Match ...>

# Raw f-string
name = "user"
print(fr"\b{name}\b")           # '\\buser\\b'
```

Q35 · Groups — `(...)`, `(?:...)`, and `(?P<name>...)`

Tags: Difficulty: Medium · Asked at: TCS, Razorpay, Walmart Labs, Postman, Flipkart · Topic: Regex

The question (interviewer's verbatim phrasing):

What's the difference between `(\d+)` and `(?:\d+)`? And what does `(?P<year>\d{4})` give me?

The 30-second answer: `(...)` is a capturing group — the matched substring goes into `m.group(1)`, `m.group(2)`, etc. `(?:...)` is a non-capturing group — same grouping for alternation and quantifier purposes, but no slot in the groups list. `(?P<name>...)` is a named capture — accessible via `m.group("name")` or `m.groupdict()["name"]`, plus the positional number. Use named groups for readability when you have more than two captures.

Step-by-step thought process: 1. Capturing groups cost a tiny bit at match time and consume a numbered slot. If you're grouping for `(a|b)+` purposes only and not extracting, use `(?:...)`. 2. Named groups make the regex self-documenting and the code self-explanatory. `m.group("year")` reads better than `m.group(1)` when you have more than two groups. 3. Numbering: groups are numbered left-to-right by the position of their opening parenthesis. `(a)(b(c))` numbers `a` as 1, `bc` as 2, `c` as 3. 4. `m.group(0)` is the whole match. `m.group()` (no arg) is the same. `m.group(1)` is the first capturing group.

Edge cases to mention: - Named groups also work in backreferences and `sub` templates: `(?P=year)` references the named group; `\g<year>` in the replacement. - A group inside a quantifier captures only the *last* iteration: `(\d)+` on `"123"` captures `"3"`, not `"123"`. - `re.findall` with one capturing group returns the *group*, not the whole match. With non-capturing, you get the full matches.

What NOT to say: - "Use `(?:...)` for everything" — defeats the point of having captures. - "Named groups are slower" — no measurable difference.

Common follow-up: "How would you reference a named group in the replacement of `re.sub`?" — `\g<name>` or backreference `\1` for positional.

```
import re

m = re.search(r"(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})", "2026-05-30")
print(m.group("year"), m.group("month"), m.group("day"))    # '2026' '05' '30'
print(m.groupdict())                                       # {'year': '2026', ...}

# Non-capturing groups don't take slots
m = re.search(r"(?:Mr|Ms)\. (\w+)", "Mr. Patel")
print(m.group(1))                                          # 'Patel'

# Last-iteration trap
m = re.match(r"(\d)+", "123")
print(m.group(1))                                         # '3' — last only
```

Q36 · Backreferences — `\1`, `\g<name>`, `(?P=name)`

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Postman, JP Morgan · Topic: Regex

The question (interviewer's verbatim phrasing):

How would you find doubled words in a sentence — like "the the" in "find the the bug"?

The 30-second answer: A backreference. `(\w+)\s\1` — capture a word, then `\1` matches the same captured text. Add `\b` word boundaries to avoid matching inside larger words. For named groups, use `(?P=name)` in the pattern or `\g<name>` in `re.sub` replacements.

Step-by-step thought process: 1. State what a backreference does: matches the *exact same text* the captured group matched. Not "another word of the same shape" — the same characters. 2. The two syntaxes: numbered (`\1` through `\9`, then `\g<10>` and beyond) and named (`(?P=name)` in the pattern, `\g<name>` in replacements). 3. Common uses: detect duplicates (doubled words), match balanced quotes (`("'|')(.*?)\1`), find HTML/XML tag pairs. 4. Backreferences are why regex is more powerful than a regular language in the formal sense — formal regular languages don't allow them. They also enable catastrophic backtracking, so use carefully.

Edge cases to mention: - `\1` inside a character class `[ \1 ]` is a backreference *or* an octal escape depending on context — almost always not what you want. Don't put backrefs in classes. -

`re.sub(r"(\w+) \1", r"\1", text)` collapses doubled words to a single occurrence — the

replacement also uses `\1` . - Backreferences interact badly with empty matches and lookarounds; test edge cases.

What NOT to say: - "Backreferences are slow" — not categorically; they enable nice patterns and only hurt when combined with greedy quantifiers on ambiguous input. - "Just use two separate captures" — that finds two words of the same *shape*, not the same word.

Common follow-up: *"Now collapse those doubled words to a single occurrence in the output."* — `re.sub(r"\b(\w+)\s+\1\b", r"\1", text)` .

```
import re

text = "find the the bug bug now"

# Detect
for m in re.finditer(r"\b(\w+)\s+\1\b", text):
    print(m.group(0), "at", m.start())    # 'the the' at 5; 'bug bug' at 13

# Collapse with replacement
print(re.sub(r"\b(\w+)\s+\1\b", r"\1", text))    # 'find the bug now'

# Named backref
m = re.search(r"(?P<quote>['\"])(hello(?P=quote))", '"hello"')
print(bool(m))                                # True
```

Q37 · `re.sub` — The Function-Callback Power Move

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Goldman, Postman · Topic: Regex

The question (interviewer's verbatim phrasing):

Replace every number in a string with its doubled value — `"a 3 b 7"` becomes `"a 6 b 14"` . Show me how.

The 30-second answer: `re.sub` accepts a callable as the replacement. The callable receives each `Match` object and returns the replacement string. `re.sub(r"\d+", lambda m: str(int(m.group()) * 2), "a 3 b 7")` returns `"a 6 b 14"` . This is the single most-used regex pattern in real data wrangling.

Step-by-step thought process: 1. State the signature: `re.sub(pattern, repl, string, count=0, flags=0)` . `repl` can be a string (with backreferences) or a callable. 2. The callable shape: `repl(m: Match) -> str` . You get the match, return the replacement. No template syntax needed. 3. Why use it: anything beyond simple text-to-text substitution. Counting, lookup tables, decisions based on context — all natural with a callback. 4. Performance: a Python callback is slower

than a string replacement, but it unlocks complex logic. For one-off transforms, fine. For massive throughput, consider a compiled pattern and `re.sub` with a precomputed dict.

Edge cases to mention: - Return type must be `str` (or `bytes` if the input is bytes). Returning `None` raises `TypeError`. - The callback can use `m.group()`, `m.groupdict()`, `m.start()`, `m.end()` — full Match API. - `re.subn` returns a `(new_string, count)` tuple; useful when you need to know how many substitutions occurred.

What NOT to say: - "Just chain `replace` calls" — works for fixed text-to-text; useless for "double the number" kind of transformations. - "Use `findall` and reassemble" — that's what `sub` with a callback does, but better and faster.

Common follow-up: "What if I want to skip some replacements based on context?" — Return `m.group()` from the callback to leave the match unchanged.

```
import re

# Function callback
out = re.sub(r"\d+", lambda m: str(int(m.group()) * 2), "a 3 b 7 c 100")
print(out)                                # 'a 6 b 14 c 200'

# Lookup table via callback
roman = {"1": "I", "2": "II", "3": "III"}
out = re.sub(r"\d", lambda m: roman.get(m.group(), m.group()), "step 1, step 2, step 9")
print(out)                                # 'step I, step II, step 9'

# Count substitutions
new, n = re.subn(r"\d+", "N", "1 a 2 b 3 c")
print(new, n)                             # 'N a N b N c' 3
```

Q38 · Character Classes — `\d` / `\w` / `\s` and Their Inverses

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman · Topic: Regex

The question (interviewer's verbatim phrasing):

What does `\w` match exactly? Is it just `[a-zA-Z0-9_]` ?

The 30-second answer: By default on `str` input, `\w` matches any Unicode "word" character — letters, digits, underscore, plus accented letters, Devanagari, CJK, etc. It's *not* just `[a-zA-Z0-9_]`. To restrict to ASCII, pass `re.ASCII` flag or use the explicit class `[a-zA-Z0-9_]`. On `bytes` input, `\w` is ASCII-only by default.

Step-by-step thought process: 1. State the four common classes: `\d` (digits), `\w` (word chars), `\s` (whitespace), and their uppercase complements `\D`, `\W`, `\S` (anything-but). 2. Unicode vs ASCII: by default on `str`, all four are Unicode-aware. Devanagari `१२३` matches `\d+`; that may or may not be what you want. 3. The `re.ASCII` flag (or `(?a)` inline) restricts these classes to ASCII. The `re.UNICODE` flag is the default on `str` (and exists for explicitness). 4. Character classes inside `[...]` are independent of mode flags for some classes — `[\d]` follows the same rules as `\d` alone.

Edge cases to mention: - `\b` (word boundary) depends on `\w` — Unicode mode by default. A Hindi word followed by ASCII punctuation has a `\b` between them. - For "digits only, no Devanagari" use `[0-9]` explicitly or `\d` with `re.ASCII`. - `\s` matches `\t \n \r \f \v` and the broader Unicode whitespace set; depends on mode.

What NOT to say: - "`\w` is `[A-Za-z0-9_]`" — wrong in Python 3 by default. - "Use `\d` to validate phone numbers" — accepts Unicode digits like `१२३४५६७८९०`; use `[0-9]` if you mean ASCII.

Common follow-up: "Show me how to validate an Indian PAN number." — `re.fullmatch(r"[A-Z]{5}[0-9]{4}[A-Z]", pan)` — explicit classes, no `\d`.

```
import re

print(re.findall(r"\d+", "abc १२३ 456 ⑦⑧⑨"))      # ['१२३', '456'] Unicode digits match
print(re.findall(r"\d+", "abc १२३ 456", flags=re.ASCII)) # ['456'] ASCII only
print(re.findall(r"[0-9]+", "abc १२३ 456"))        # ['456'] explicit ASCII

# Bytes is always ASCII for these classes
print(re.findall(rb"\d+", b"abc 456"))              # [b'456']
```

Q39 · Anchors — `^`, `$`, `\b`, `\Z`

Tags: Difficulty: Medium · Asked at: TCS, Razorpay, Walmart Labs, Postman · Topic: Regex

The question (interviewer's verbatim phrasing):

What does `\b` match? And what's the difference between `$` and `\Z`?

The 30-second answer: `\b` matches a *position* (zero-width) between a `\w` and a `\W` character (or start/end of string). `\bword\b` finds "word" as a whole word, not as a substring of "password". `$` matches end of string (or end of line in MULTILINE mode); `\Z` matches end of string only — never end of line. `^` mirrors with start of string vs start of line.

Step-by-step thought process: 1. Walk through each anchor. `^` and `$` are the usual; their behaviour changes with `re.MULTILINE`. `\A` and `\Z` are always whole-string anchors regardless of flags. 2.

`\b` is the workhorse for word-aware matching. `\bcat\b` matches "cat" but not "catalog" or "scatter". The dual `\B` matches non-boundaries. 3. Anchors are zero-width — they don't consume characters. `\b\w+\b` is the standard "match a whole word" pattern. 4. The MULTILINE flag: without it, `^ / $` are whole-string; with it, they match at each line break. `re.fullmatch` with MULTILINE is still whole-string — the flag affects `^ / $` but not `fullmatch`'s anchoring.

Edge cases to mention: - `\b` definition depends on `\w` mode (Unicode vs ASCII). Hindi word adjacent to ASCII punctuation: there's a boundary because Hindi chars are `\w` and punctuation is `\W`. - `$` matches the position just before a final `\n` even without MULTILINE — that's a historical edge case. `\Z` doesn't. - In Python 3.12+, the regex engine has minor improvements in anchor handling speed.

What NOT to say: - "`\b` matches a space" — it matches a zero-width position, not a character. - "`$` and `\Z` are the same" — they differ on trailing newlines and MULTILINE.

Common follow-up: "How would you find lines that start with `ERROR` in a log file?" — `re.findall(r"^ERROR.*", log, flags=re.MULTILINE)`.

```
import re

text = "find the cat in catalog"
print(re.findall(r"cat", text))          # ['cat', 'cat'] - substring inside catalog
print(re.findall(r"\bcat\b", text))      # ['cat']      - whole word only

log = "INFO ok\nERROR bad\nINFO done"
print(re.findall(r"^ERROR.*", log, flags=re.MULTILINE)) # ['ERROR bad']

# $ vs \Z on trailing newline
print(bool(re.search(r"abc$", "abc\n"))) # True - $ tolerates trailing \n
print(bool(re.search(r"abc\Z", "abc\n"))) # False - \Z is strict
```

Q40 · `re.split` — Powerful When You Need Multi-Delimiter

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Postman · Topic: Regex

The question (interviewer's verbatim phrasing):

Split a string on commas, semicolons, and any whitespace — all at once.

The 30-second answer: `re.split(r"[,;\s]+", text)`. The character class matches any of the listed delimiters; `+` means "one or more in a row" so consecutive delimiters don't produce empty fields. For `str.split`, you'd need three nested splits. For more complex delimiters, `re.split` keeps the logic single-pass.

Step-by-step thought process: 1. State the API: `re.split(pattern, string, maxsplit=0, flags=0)`. Returns a list. With `maxsplit > 0`, stops splitting after that many splits. 2. The `+` quantifier prevents empty fields when delimiters are adjacent. `re.split(r"[,;\s]", "a,,b")` returns `['a', '', 'b']`; with `+`, `['a', 'b']`. 3. Capturing groups in the pattern: those go into the result. `re.split(r"(\s+)", "a b c")` returns `['a', ' ', 'b', ' ', 'c']`. Useful when you need the delimiters back. 4. `re.split` is slower than `str.split` for fixed delimiters. Don't use it when a single-character `str.split` will do.

Edge cases to mention: - An empty pattern raises `ValueError` (3.7+); previously it returned everything as one element. - Leading/trailing matches: `re.split(r",", ",a,b,")` returns `['', 'a', 'b', '']`. Empty strings at the ends. - Zero-width patterns work in 3.7+: `re.split(r"\b", "hello world")` returns `['', 'hello', ' ', 'world', '']`.

What NOT to say: - "Always use `re.split`" — overkill for single-character splits. - "Use `str.split(",").split(";")` style chaining" — fails on whitespace and is awkward.

Common follow-up: "How would you split on `,` or `;` (with the space) and discard empty results?" — `[p for p in re.split(r"[,;\s]*", text) if p]`.

```
import re

print(re.split(r"[,;\s]+", "a, b;c   d,, e"))      # ['a', 'b', 'c', 'd', 'e']

# Capturing group keeps delimiters
print(re.split(r"(\s+)", "a b c"))                # ['a', ' ', 'b', ' ', 'c']

# maxsplit
print(re.split(r",", "a,b,c,d", maxsplit=2))      # ['a', 'b', 'c,d']
```

Section F — Regex Advanced (Q41–Q50)

Q41 · Greedy vs Non-Greedy Quantifiers — `*?`, `+?`, `??`

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Postman, JP Morgan · Topic: Regex

The question (interviewer's verbatim phrasing):

What's the difference between `<.*>` and `<.*?>` when matching HTML tags?

The 30-second answer: `*` and `+` are *greedy* by default — they grab as much as they can, then back off if the rest of the pattern fails. `<.*>` on `"<b>x</b>"` matches the whole string because `.*` greedily consumes everything until the last `>`. The non-greedy form `<.*?>` matches the shortest

possible — just `<b>`, then `</b>` on the next iteration. Append `?` to any quantifier (`*?`, `+?`, `??`, `{n,m}?`) to make it non-greedy.

Step-by-step thought process: 1. State greedy behaviour: the engine tries the longest match first, then backtracks character by character until the rest of the pattern succeeds. On `<.*>` over `"<b>x</b>"`, the engine grabs everything, then backs off until the final `>` matches. 2. Non-greedy: tries the shortest first, expanding only if needed. `<.*?>` immediately tries the smallest match, succeeds with `<b>`, moves on. 3. The classic gotcha: greedy is correct surprisingly often because the constraint after `.*` does the work. Non-greedy is needed when you want "the next occurrence" rather than "the last occurrence." 4. Possessive quantifiers (`*+`, `++`) — these *don't backtrack at all*. Available in Python 3.11+ regex. Useful for performance and to avoid ReDoS.

Edge cases to mention: - Non-greedy doesn't mean "minimum" — it still requires the overall pattern to match. If the only valid match requires going further, non-greedy expands. - For HTML/XML, neither form is correct. Use an HTML parser (`lxml`, `BeautifulSoup`). Regex on nested markup is a recipe for bugs. - Greedy and non-greedy have the same worst-case time complexity — non-greedy doesn't avoid catastrophic backtracking by itself.

What NOT to say: - "Non-greedy is faster" — same complexity; just different match. - "Use non-greedy for HTML" — use a real parser.

Common follow-up: "What's a possessive quantifier?" — Like non-greedy in syntax (`*+`, `++`), but unlike non-greedy, never gives back the match. Prevents backtracking entirely.

```
import re

s = "<b>x</b><i>y</i>"
print(re.findall(r"<.*>", s))          # ['<b>x</b><i>y</i>'] - greedy
print(re.findall(r"<.*?>", s))        # ['<b>', '</b>', '<i>', '</i>'] - non-greedy

# Greedy then a constraint - still correct
print(re.search(r"<(.*?)>([<]*)", s).groups()) # ('b>x</b><i>y</i>', '')
```

Q42 · Lookahead — `(?=...)` and `(?!...)`

Tags: Difficulty: Hard · Asked at: Razorpay, Walmart Labs, JP Morgan, Goldman · Topic: Regex

The question (interviewer's verbatim phrasing):

Find every number in a string that's followed by `kg` — but I don't want `kg` in the match. Show me how.

The 30-second answer: A positive lookahead. `\d+(?=kg)` — match digits, assert that `kg` follows, but don't consume `kg`. `re.findall(r"\d+(?=kg)", "5kg of rice, 12kg of dal, 7lb of sugar")` returns `["5", "12"]`. The negative variant `(?!kg)` asserts "NOT followed by kg." Both are zero-width — they check without consuming.

Step-by-step thought process: 1. Define the four lookarounds. Positive lookahead `(?=...)`, negative lookahead `(?!...)`, positive lookbehind `(?<=...)`, negative lookbehind `(?<!...)`. Zero-width means they don't consume characters. 2. The win: assertions about context without including the context in the match. Useful for filtering, splitting, and "find X where Y is nearby" patterns. 3. Common uses: password validators (`(?=.*\d)(?=.*[A-Z])` — must contain a digit AND an uppercase), CSV parsing (commas not inside quotes), word-context matching. 4. Lookahead can include any pattern, including other groups. The pattern inside the lookahead can match arbitrarily long text in modern Python; the assertion succeeds or fails as a whole.

Edge cases to mention: - Lookahead doesn't capture by default; if you need to capture, the captured group still has to be inside the lookahead, but you'd usually restructure. - Multiple lookaheads stack: `(?=.*\d)(?=.*[A-Z])(?=.*[a-z])` is the standard password-strength pattern. - Performance: each lookahead is a re-anchored sub-match. Fine for most uses; matters when stacked deeply.

What NOT to say: - "Lookahead consumes characters" — it doesn't; it's an assertion. - "Use `(?=...)` to capture" — captures need their own parens, not lookaheads.

Common follow-up: "Write a regex for a password with at least one digit, one lowercase, one uppercase, and 8+ chars." — `re.fullmatch(r"(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}", pw)`.

```
import re

text = "5kg rice, 12kg dal, 7lb sugar, 3kg flour"
print(re.findall(r"\d+(?=kg)", text))          # ['5', '12', '3']

# Password validation — all lookaheads at start
def strong(pw: str) -> bool:
    return bool(re.fullmatch(r"(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}", pw))

print(strong("Abc12345"))                      # True
print(strong("abcdefgh"))                      # False — no digit, no upper

# Negative lookahead
print(re.findall(r"\d+(?!kg)", text))          # all numbers NOT followed by kg
```

Q43 · Lookbehind — `(?<=...)` and `(?<!...)`

Tags: Difficulty: Hard · Asked at: Razorpay, Walmart Labs, JP Morgan, Goldman · Topic: Regex

The question (interviewer's verbatim phrasing):

Find every dollar amount that follows `USD` (and only the amount, not `USD`). Use lookbehind.

The 30-second answer: Positive lookbehind: `(?<=USD)\d+(?:\.\d+)?`. The lookbehind `(?<=USD)` asserts that the position is preceded by `USD` without including it in the match. Negative lookbehind `(?<!\...)` asserts the position is NOT preceded by the pattern. Until Python 3.7, lookbehind required a fixed-width pattern; since 3.7 the built-in `re` module still requires fixed width, but the third-party `regex` library supports variable-width.

Step-by-step thought process: 1. Mirror of lookahead, but checks the position before the match. Zero-width, doesn't consume. 2. The fixed-width constraint in built-in `re`: `(?<=abc)` works (length 3); `(?<=a.*)` fails because `.*` is variable. The error is `error: look-behind requires fixed-width pattern`. 3. If you need variable-width lookbehind, use the `regex` library (third-party) — drop-in replacement with extra features. 4. Common uses: split on delimiters without consuming, find substrings in specific context, extract data after fixed prefixes.

Edge cases to mention: - The fixed-width restriction applies to the *entire* lookbehind alternative — `(?<=abc|defg)` fails because alternatives have different lengths. Restructure into two passes or use `regex`. - Lookbehind can include character classes `(?<=[A-Z])` — those count as fixed width 1. - For "match X anywhere except after Y", negative lookbehind is the right tool: `(?<!\$)\d+` matches numbers not preceded by `$`.

What NOT to say: - "Lookbehind is variable-width in Python" — only in the third-party `regex` library; built-in `re` is fixed-width. - "Use `(?<=...)` to skip backward in the input" — it asserts, it doesn't consume.

Common follow-up: "What if the prefix has variable length?" — `import regex` (third-party). The built-in `re` won't accept variable-width lookbehind.

```
import re

text = "Pay USD 499 today, INR 50 service, USD 1200 total"
print(re.findall(r"(?<=USD)\d+", text))          # ['499', '1200']

# Negative lookbehind
prices = "$5 €10 ₹50 $20"
print(re.findall(r"(?<!\$)\d+", prices))         # ['10', '50'] — not after $

# Fixed-width restriction
try:
    re.compile(r"(?<=a.*)x")
except re.error as e:
    print(e)                                     # look-behind requires fixed-width pattern
```

Q44 · Flags — `re.IGNORECASE` , `re.MULTILINE` , `re.DOTALL` , `re.VERBOSE`**Tags:** Difficulty: Medium · Asked at: TCS, Razorpay, Walmart Labs, Postman · Topic: Regex**The question (interviewer's verbatim phrasing):**

Walk me through the most common regex flags. When would I reach for each one?

The 30-second answer: `re.IGNORECASE` (`re.I`) — case-insensitive matching. `re.MULTILINE` (`re.M`) — `^` and `$` match per-line. `re.DOTALL` (`re.S`) — `.` matches newline too (by default it doesn't). `re.VERBOSE` (`re.X`) — allow whitespace and `#` comments inside the pattern for readability. Combine with `|` : `re.compile(pat, re.I | re.M)` .

Step-by-step thought process: 1. `re.IGNORECASE` : most common. Matters for any user input. Note this doesn't normalize Unicode — `"groß"` won't match `"gross"` even with `IGNORECASE`; you'd need `casefold` plus normalisation. 2. `re.MULTILINE` : `^` and `$` start meaning "start/end of line" instead of "start/end of string." Required for log file regex. 3. `re.DOTALL` : `.` normally matches anything except `\n` ; `DOTALL` removes that exception. Useful for multi-line capture patterns where the section can contain newlines. 4. `re.VERBOSE` : write multi-line, commented regex. Whitespace is ignored (except inside character classes or escaped). The single biggest readability win for complex patterns.

Edge cases to mention: - Inline flags: `(?i)pattern` is the same as `re.IGNORECASE` . Useful inside `re.findall` calls without specifying a flag arg. - Scoped inline flags (3.7+): `(?i:pattern)` applies the flag only to the parenthesised section. The unscoped `(?i)` at start applies to the whole pattern. - `VERBOSE` doesn't ignore whitespace *inside* character classes: `[a b]` matches `a` , space, or `b` — three chars. Use `\` or `[\s]` to be explicit.

What NOT to say: - "MULTILINE makes `.` match newlines" — that's `DOTALL`. `MULTILINE` is about `^` / `$` . - "VERBOSE slows down matching" — it's a compile-time flag; no runtime cost.

Common follow-up: "Show me a `VERBOSE` regex that validates an email."

```
import re

# Combine flags
pattern = re.compile(r"^error:.*", re.IGNORECASE | re.MULTILINE)
log = "INFO ok\nError: timeout\nINFO done"
print(pattern.findall(log)) # ['Error: timeout']

# DOTALL - . matches newline
print(re.findall(r"<note>.*?</note>", "<note>hi\nthere</note>", flags=re.DOTALL))
# ['<note>hi\nthere</note>']

# VERBOSE - readable email regex
email_re = re.compile(r"""
    ^                # start
    [a-zA-Z0-9._%+-]+ # local part
    @
    [a-zA-Z0-9.-]+    # domain
    \.[a-zA-Z]{2,}    # TLD
    $                # end
""", re.VERBOSE)
print(bool(email_re.match("abhi@learn-and-excel.in"))) # True
```

Q45 · Catastrophic Backtracking — ReDoS and How to Avoid It

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, JP Morgan, Walmart Labs, Microsoft · Topic: Regex · Security

The question (interviewer's verbatim phrasing):

A regex on user input is hanging your service. What's catastrophic backtracking, and how do you fix the pattern?

The 30-second answer: Catastrophic backtracking happens when a regex with nested quantifiers tries every permutation of which sub-pattern consumes which character before giving up. Pattern `(a+)+$` on `"aaaaaaaaaaaaab"` tries 2^n combinations before failing. Fixes: rewrite to eliminate ambiguity (`a+$`), use atomic groups or possessive quantifiers in Python 3.11+ (`(?>a+)`), or set a timeout via the `regex` library's `timeout` parameter.

Step-by-step thought process: 1. Explain the mechanism. When a greedy quantifier fails to match the rest of the pattern, the engine backtracks one character at a time, retrying. Nested quantifiers (a quantifier inside another quantifier) compound the backtracking exponentially. 2. The canonical example: `(a+)+$` on a string of `a` s followed by anything else. Internally, the engine tries every partition of `aaa...` between the inner and outer quantifiers. With `n` a's, that's exponential. 3. ReDoS = Regex Denial of Service. An attacker submits crafted input that triggers seconds-or-minutes of CPU time for

what should be sub-millisecond matching. Real-world incident: Cloudflare's 2019 outage was a ReDoS bug. 4. Fixes in order of preference: (a) rewrite to remove ambiguity — most patterns can be expressed unambiguously; (b) use possessive quantifiers `a++` or atomic groups `(?>a+)` in 3.11+; (c) set a regex timeout via the third-party `regex` package; (d) precompile to a deterministic engine (`regex.compile` with `regex.POSIX`).

Edge cases to mention: - Python 3.11+ supports atomic groups `(?>...)` and possessive quantifiers `(**)`, `++`. These don't backtrack. Pre-3.11 needs a workaround. - Some innocent-looking patterns hide nested quantifiers — `(.+) +$` and `(.*) *$` are the famous ones, also `(\w+)*` after a word. - The `regex` third-party module has a `regex.compile(..., timeout=1.0)` option that aborts long-running matches.

What NOT to say: - "Use a non-greedy quantifier" — doesn't help; same complexity. - "All regex is dangerous" — most regex is fine; nested-quantifier patterns on attacker input are the danger zone.

Common follow-up: "How would you defend against ReDoS without rewriting the pattern?" — Run the regex in a thread with a timeout, use the `regex` module's timeout, or run untrusted patterns in a subprocess. None are great; rewriting is best.

```
import re

# Vulnerable pattern (don't run on attacker input)
# pattern = re.compile(r"^(a+)+$")
# pattern.match("a" * 30 + "b")           # would hang seconds

# Fixed: unambiguous, single quantifier
pattern = re.compile(r"^a+$")
print(pattern.match("a" * 30 + "b"))      # None — fast

# Possessive (3.11+) — no backtracking
pattern2 = re.compile(r"^(a++)+$")
print(pattern2.match("a" * 30 + "b"))     # None — fast
```

Q46 · `re.escape` — Safe Substring as Pattern

Tags: Difficulty: Medium · Asked at: TCS, Razorpay, Walmart Labs, Postman · Topic: Regex · Security

The question (interviewer's verbatim phrasing):

A user enters a search term and you want to find it in a document. Why is `re.search(user_input, text)` a bug, and how do you fix it?

The 30-second answer: Because user input might contain regex metacharacters — `.`, `*`, `+`, `?`, `[`, `(` — which change the meaning. `re.search("a.b", "axb")` matches (`.` is "any char"); the user expected literal `"a.b"`. `re.escape(user_input)` escapes every metacharacter, so the search becomes literal. It's also a partial defence against ReDoS — escaped input can't introduce ambiguous quantifiers.

Step-by-step thought process: 1. State the threat. Regex metacharacters in untrusted input change matching semantics. At best it's a bug; at worst it's ReDoS or info disclosure. 2. `re.escape(s)` returns `s` with every metacharacter prefixed by `\`. Safe to concatenate into a pattern. 3. The common pattern: `re.compile(r"\b" + re.escape(term) + r"\b")` to find `term` as a whole word. 4. Don't escape what you control. If the pattern is a literal in source, no escaping needed. Escape only the parts derived from untrusted input.

Edge cases to mention: - `re.escape` over-escapes some characters — `re.escape("abc")` might return `"abc"` or `"\\a\\b\\c"` depending on Python version. The output is always safe; sometimes more escaping than strictly necessary. - For `bytes` patterns, `re.escape` accepts `bytes` and returns `bytes`. Same logic. - `re.escape` doesn't defend against algorithmic complexity attacks if the *pattern* (not the input) is attacker-controlled — only against metacharacter abuse in the search term.

What NOT to say: - "Just sanitize the input manually" — easy to miss a character; `re.escape` is exhaustive. - "User input is safe because it's just a search term" — until someone types `.*` and wonders why it matches everything.

Common follow-up: "What if the user wants to do regex searches intentionally?" — That's a feature. Don't escape, but do timeout the regex with the `regex` library to defend against ReDoS.

```
import re

# Buggy – metachars in user input
user_input = "a.b"
print(re.search(user_input, "axb"))      # matches! 'a.b' was a pattern

# Fixed
print(re.search(re.escape(user_input), "axb"))  # None – literal search
print(re.search(re.escape(user_input), "a.b"))  # matches the literal

# Common pattern: word-boundary search for user term
term = "raz."                               # the . is literal here
rx = re.compile(r"\b" + re.escape(term) + r"\b")
```

Q47 · `re.Pattern` Object — Compiled Pattern API

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Goldman · Topic: Regex

The question (interviewer's verbatim phrasing):

What methods does a compiled `re.Pattern` object have that the module-level functions don't?

The 30-second answer: The compiled pattern exposes everything — `match`, `search`, `fullmatch`, `findall`, `finditer`, `sub`, `subn`, `split`. Plus useful attributes: `.pattern` (original pattern string), `.flags` (effective flags), `.groups` (number of capture groups), `.groupindex` (named-group name → number map). And a `scanner()` method for low-level iteration.

Step-by-step thought process: 1. Compiled patterns aren't just a perf optimisation — they're an object with state and introspection. `pattern.pattern` lets you recover the source pattern for logging. 2. `pattern.groupindex` is a dict mapping group names to indices. Useful for generic code that processes named-group results: `for name in pattern.groupindex: ...`. 3. The methods on compiled patterns take an optional `pos` and `endpos` argument (the module-level functions don't). `pattern.search(s, pos=10)` starts searching at index 10. 4. `pattern.scanner(s)` returns a low-level scanner object — rarely used, but the basis for `finditer`.

Edge cases to mention: - `Pattern.flags` includes the implicit UNICODE flag for `str` patterns. Use `pattern.flags & re.IGNORECASE` to check a specific flag. - Patterns are pickleable in 3.7+ — useful for multiprocessing. - Two patterns with the same source and flags compare unequal — they're distinct objects.

What NOT to say: - "Compiled patterns are just for performance" — they have a richer API. - "Use `re.findall(p, s)` always" — `pattern.findall(s)` is the cleaner form once compiled.

Common follow-up: "How would you list every named group in a pattern?" — `list(pattern.groupindex)`.

```
import re

pat = re.compile(r"(?P<year>\d{4})-(?P<month>\d{2})", re.IGNORECASE)
print(pat.pattern)           # '(?P<year>\d{4})-(?P<month>\d{2})'
print(pat.groups)            # 2
print(pat.groupindex)        # {'year': 1, 'month': 2}
print(bool(pat.flags & re.IGNORECASE)) # True

# pos / endpos
print(pat.search("ignore 2026-05 also 2024-12", pos=15).group()) # '2024-12'
```

Q48 · Inline Flags and Scoped Flags — `(?i)` and `(?i:...)`

Tags: Difficulty: Hard · Asked at: Razorpay, Walmart Labs, JP Morgan · Topic: Regex

The question (interviewer's verbatim phrasing):

Make this pattern case-insensitive only for one part — `Foo` should match `foo` but the next group `Bar` should be case-sensitive.

The 30-second answer: Scoped inline flags. `(?i:Foo)Bar` — `(?i:...)` applies `IGNORECASE` only inside the parens. `Foo` part matches case-insensitively; `Bar` is case-sensitive. Without scoping, `(?i)` at any position applies to the whole pattern; in modern Python (3.7+) it must be at the very start. Scoped flags solve the "only part of the pattern needs the flag" case.

Step-by-step thought process: 1. Distinguish unscoped from scoped. `(?i)` (no colon) applies to the entire pattern — and as of 3.7, must be at the very start. `(?i:...)` applies only to the parenthesised section. 2. The set of single-letter flags: `i` (`IGNORECASE`), `m` (`MULTILINE`), `s` (`DOTALL`), `x` (`VERBOSE`), `a` (`ASCII`), `u` (`UNICODE`). Combine: `(?ims:...)`. 3. You can also *unset* flags inside a scope: `(?-i:...)` turns off `IGNORECASE` within. Useful when the outer pattern uses `re.IGNORECASE` but one sub-part needs to be strict. 4. Practical use: validating mixed-case identifiers, parsing log formats where part is case-insensitive (level) and part is case-sensitive (module name).

Edge cases to mention: - Pre-3.7 allowed unscoped `(?i)` anywhere in the pattern — Python 3.7+ deprecated mid-pattern unscoped flags and 3.12 raises a `re.error` for them. - Scoped flags don't affect anchors outside the scope — `(?m:...)` doesn't make `^` outside the scope match per-line. - Combining: `(?i-s:...)` enables `IGNORECASE` and disables `DOTALL` within the scope.

What NOT to say: - "Flags apply globally" — that's the unscoped behaviour. Scoped flags exist precisely for the local case. - "Use two passes" — the scoped form is one pattern, one scan.

Common follow-up: "What flags can you toggle inline?" — `i`, `m`, `s`, `x`, `a`, `u`. `ASCII` and `UNICODE` are mutually exclusive.

```
import re

# Scoped IGNORECASE – only Foo is case-insensitive
print(re.findall(r"(?i:Foo)Bar", "fooBar FooBar F00Bar foobar"))
# ['fooBar', 'FooBar', 'F00Bar'] – fourth fails because 'bar' is case-sensitive

# Scoped DOTALL – only this group spans newlines
text = "head: stuff\nmore\nend\nnext line"
print(re.findall(r"head: (?s:.*)end", text))    # ['head: stuff\nmore\nend']

# Unset a flag inside a scope
pat = re.compile(r"(?i:abc)(?-i:DEF)")
print(pat.match("AbcDEF"))                    # matches
print(pat.match("AbcDef"))                    # None – Def fails strict
```

Q49 · `regex` (Third-Party) — When Built-In `re` Isn't Enough

Tags: Difficulty: Hard · Asked at: Razorpay, Walmart Labs, Goldman · Topic: Regex

The question (interviewer's verbatim phrasing):

When would you reach for the third-party `regex` library instead of the built-in `re` ?

The 30-second answer: Three real reasons. **Variable-width lookbehind** — `re` requires fixed width; `regex` doesn't. **Unicode-aware properties** — `regex` supports `\p{Lu}` , `\p{Devanagari}` , full Unicode script and category classes. **Grapheme clusters** — `\X` matches a user-visible character, including emoji sequences. Plus extras: timeout per match, named character set syntax, fuzzy matching. API is mostly drop-in compatible with `re` .

Step-by-step thought process: 1. Frame the choice: `re` is built in, fine for most use. `regex` is third-party (`pip install regex`), drop-in compatible at the call-level, and adds the features above. 2. Unicode properties: `\p{Letter}` , `\p{Devanagari}` , `\p{Number}` . The built-in `re` supports the few `\w` / `\d` / `\s` aliases; `regex` exposes the full Unicode category tree. 3. Variable-width lookbehind: a real win when matching multiple prefixes of different lengths. 4. Fuzzy matching `(?:pattern){e<=1}` allows up to 1 edit. Useful for typo-tolerant search.

Edge cases to mention: - `regex` is generally a few percent slower than `re` for simple patterns but faster for some complex ones (better algorithmic choices for Unicode-heavy patterns). - Timeout: `regex.compile(pattern, timeout=1.0).search(text)` raises `TimeoutError` if matching exceeds the budget. Important defence against ReDoS. - API differences are minor — `regex.match` , `regex.search` , etc. all exist. Drop-in for most code.

What NOT to say: - " `regex` is required for Python regex" — built-in `re` covers the majority of use cases. - " `regex` is always faster" — close, sometimes faster, sometimes slower.

Common follow-up: "Show me grapheme-aware matching." — `regex.findall(r"\X", "👤")` returns `['👤']` (1 grapheme).

```
# pip install regex
# import regex

# Variable-width lookbehind
# regex.search(r"(?<=Mr|Mrs|Dr) \w+", "Hello Mr Patel")      # works in regex
# re.search(r"(?<=Mr|Mrs|Dr) \w+", "Hello Mr Patel")          # fixed-width fails

# Unicode property
# regex.findall(r"\p{Devanagari}+", "Hello नमस्ते world")    # ['नमस्ते']

# Grapheme cluster
# regex.findall(r"\X", "👤 abc")                              # ['👤', ' ', 'a', 'b', 'c']

# Timeout— ReDoS defence
# regex.compile(r"^(a+)+$", timeout=0.1).match("a"*30 + "b") # TimeoutError
```

Q50 · Regex Performance — Choosing Pattern Over Code

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, JP Morgan, Walmart Labs · Topic: Regex · Performance

The question (interviewer's verbatim phrasing):

Profile this log-parsing code. It uses `re.search` in a tight loop and is slow. What's the standard set of optimisations you'd try?

The 30-second answer: Four passes. **First:** compile once outside the loop, not inside (or rely on the LRU cache). **Second:** anchor what you can — `^` at the start of patterns lets the engine fail-fast on non-matches. **Third:** use the most specific character class — `[0-9]` is faster than `\d` when ASCII is enough; specific classes are faster than `.`. **Fourth:** replace `.*` with `[^\n]*` where you don't need to span lines — the engine can't optimise `.*` as aggressively.

Step-by-step thought process: 1. Profile first. Use `cProfile` or `timeit` to confirm regex is actually the bottleneck. Often the bottleneck is I/O or string slicing, not the regex itself. 2. Compile-and-store pattern objects outside hot loops. Even with the LRU cache, the lookup costs measurable time at 10M iterations. 3. Anchors and specific classes help the engine prune branches early. A regex starting with `^ERROR` will fail in $O(1)$ per line that doesn't start with `E`. 4. Avoid greedy `.*` in patterns

scanned over long input. Replace with `[^\n]*` for line-bounded, or non-greedy `.*` with a strict end condition. 5. For very high throughput, consider whether you need regex at all. `str.startswith` and `str.split` are faster for fixed prefixes/delimiters. For multi-pattern dispatch, Aho-Corasick (`pyahocorasick`) beats alternation of many patterns.

Edge cases to mention: - Compile once and call repeatedly is the single biggest win — typically 1.5-3× for tight loops. - `re.sub` with a string replacement is faster than with a callback because callback invocation has Python-level overhead. - For "match against a fixed set of strings", a `set` lookup is faster than any regex. Translate the regex to a set if possible.

What NOT to say: - "Regex is always slow" — at C-level it's plenty fast; usually Python-level overhead around the regex is the cost. - "Rewrite in C" — almost never necessary; pattern-level changes win first.

Common follow-up: "What's the difference in speed between `re.search` and `pattern.search` on a hot path?" — Typically 1.5-3× on tight loops; the module-level call has cache lookup, attribute access, and Python-level wrapper overhead.

```
import re
import timeit

# Bad: compile inside loop
def bad(lines):
    return [re.search(r"ERROR: (\d+)", line) for line in lines]

# Good: compile once
PATTERN = re.compile(r"ERROR: (\d+)")
def good(lines):
    return [PATTERN.search(line) for line in lines]

# Better: anchor early-rejection
ANCHORED = re.compile(r"^ERROR: (\d+)")
def better(lines):
    return [ANCHORED.match(line) for line in lines]

lines = ["INFO ok"] * 1000 + ["ERROR: 42"]
t1 = timeit.timeit(lambda: bad(lines), number=100)
t2 = timeit.timeit(lambda: good(lines), number=100)
t3 = timeit.timeit(lambda: better(lines), number=100)
print(f"bad: {t1:.3f}s · good: {t2:.3f}s · better: {t3:.3f}s")
```

End of Chapter 11 — Strings, Bytes & Regex (50 Qs)

This chapter has covered the surface every Python interview uses for filtering — `str` immutability and the `+=` quadratic, formatter history and f-string deep features, Unicode and encoding boundaries, the buffer protocol and `memoryview`, regex from `re.match` through to ReDoS defence. The candidates who clear the first round on these questions are the ones who treat strings as Unicode sequences,

encode and decode deliberately at every boundary, and reach for `re.compile` plus `re.VERBOSE` instead of one-line cryptic patterns. Next chapter — data structures and Big-O — continues the same posture: the API is friendly; the internals are where the questions live.

Chapter 12 — Data Structures + Big-O (70 Questions)

Python Interview Prep — 2026 Edition · Learn & Excel · File 04 Core Python Library

This is the largest chapter of the Core Library and the one Indian backend, SDE, and data-platform interviewers lean on hardest in round one. Seventy questions across the eight surfaces that get probed: `list` internals, `dict` internals, `set` and `frozenset`, `tuple` and `namedtuple`, the `collections` module, `heapq`, `bisect`, the Big-O table that every CPython runtime obeys, and the DSA patterns that look idiomatic in Python rather than Java-translated-into-Python. Each question follows the same shape — tags, the interviewer's verbatim phrasing, a 30-second answer, the step-by-step "what to say out loud" thought process, edge cases to surface, what NOT to say, and the common follow-up. Answers assume Python 3.12 unless a version is called out explicitly.

Table of Sections

- Section A — `list` Internals and Big-O (Q1–Q10)
 - Section B — `dict` Internals and Big-O (Q11–Q20)
 - Section C — `set` and `frozenset` (Q21–Q25)
 - Section D — `tuple` and `namedtuple` (Q26–Q30)
 - Section E — `collections` Module (Q31–Q40)
 - Section F — `heapq` and Priority Queues (Q41–Q45)
 - Section G — `bisect` and Sorted Lists (Q46–Q50)
 - Section H — Big-O in Practice (Q51–Q60)
 - Section I — DSA Patterns in Idiomatic Python (Q61–Q70)
-

Section A — `list` Internals and Big-O

The first ten questions are about the data structure every Python program touches in its first line of code. `list` is a dynamic array of `PyObject*` pointers — not a linked list, despite the name — and almost every interview surprise about list comes from that one fact. The pointer-array implementation is why indexing is $O(1)$, why appending is amortized $O(1)$, why inserting in the middle is $O(n)$, and why

prepending is the bug that ships to production and gets quietly replaced with a `deque`. Get these ten right and you've cleared the first round at any product company in India.

Q1 · How is Python's `list` actually implemented under the hood?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Swiggy, Postman, Microsoft India · Topic: list internals

The question (interviewer's verbatim phrasing):

When you write `x = [1, 2, 3]` in Python, what data structure does CPython actually allocate? Is it a linked list?

The 30-second answer: A Python list is a dynamic array — specifically a contiguous block of `PyObject*` pointers, plus a length and an allocated capacity. It is never a linked list. The name `list` is historical and misleading; if you've come from C++ or Java, the closest analogue is `std::vector` or `ArrayList`, not `std::list` or `LinkedList`.

Step-by-step thought process: 1. Lead with the structural fact: contiguous array of pointers to Python objects. Each slot is the size of one pointer — 8 bytes on a 64-bit build — and holds the address of the actual `PyObject` somewhere else on the heap. 2. State the three header fields the C struct `PyListObject` carries: a length (`ob_size`), an allocated capacity (`allocated`), and the pointer-array itself (`ob_item`). The capacity is at least the length and is usually larger because of over-allocation. 3. Explain why the pointer-array layout matters: indexing is one pointer dereference — $O(1)$ — and you can grow at the end cheaply because the spare capacity at the tail is already there in many cases. 4. Push back on the linked-list misconception directly: a linked list would make `lst[1000]` an $O(n)$ walk; in Python `lst[1000]` is $O(1)$ because it's an array offset.

Edge cases to mention: - The pointers in `ob_item` always point to Python objects on the heap, never to raw integers or floats; this is why a Python list of one million integers uses far more memory than a NumPy array of the same numbers. - The "allocated" field can be larger than the length — that spare capacity is what makes `append` amortized $O(1)$. - `list.__sizeof__()` returns the struct size plus the pointer-array capacity in bytes, not the size of the contained objects.

What NOT to say: - "Python list is a linked list" — wrong; this is the answer that gets you cut. - "Each slot stores the integer or string directly" — also wrong; slots store pointers to objects.

Common follow-up: "If list slots are pointers to objects, why is iterating a `list[int]` of a million items so much slower than iterating a NumPy array of a million ints?" — Each list slot fetches a separate `PyObject` (cache miss, type dispatch, refcount bump), while NumPy stores the ints contiguously and the loop runs in C with no per-element object overhead.


```
import sys
x = [1, 2, 3]
sys.getsizeof(x)
# 88 bytes on CPython 3.12 — that's struct overhead + capacity for ~5 pointers,
# NOT the size of the integers themselves
```

Q2 · What is the time complexity of `list.append` and why is it called "amortized" $O(1)$?

Tags: Difficulty: Easy · Asked at: Flipkart, Razorpay, JP Morgan, Goldman, Postman · Topic: list Big-O

The question (interviewer's verbatim phrasing):

What's the Big-O of `list.append` ? And what does "amortized constant" mean in that context?

The 30-second answer: `list.append` is $O(1)$ amortized. Most appends just write to the next free slot and bump the length — true constant time. Occasionally the capacity is exhausted and CPython has to allocate a larger backing array and copy every element across, which is $O(n)$. But because the new capacity is roughly $1.125\times$ the old size, the resizes happen geometrically less often as the list grows, so the average cost per append over a long run flattens to $O(1)$. That's what amortized constant means — worst-case-per-operation is $O(n)$, but average-per-operation over many ops is $O(1)$.

Step-by-step thought process: 1. Open with the headline number — amortized $O(1)$ — then immediately unpack what amortized means, because that's the term the interviewer is checking you know. 2. Walk through the two cases. Common case: there's free capacity, the new element goes into the next slot, the length is incremented, done. Resize case: the array is full, CPython allocates a bigger one (using the over-allocation formula), copies the existing pointers across, frees the old buffer, then writes the new element. 3. Show the math intuitively. If you grow the array by a constant factor ($1.125\times$ in CPython, $2\times$ in classical implementations), the total copy work across n appends is bounded by a constant times n . Total work / n appends = $O(1)$ per append on average. 4. Note the practical consequence: appending one million items doesn't trigger one million reallocations; it triggers roughly $\log_{1.125}(1_000_000) \approx 110$ reallocations. The vast majority of appends are pure pointer writes.

Edge cases to mention: - The CPython over-allocation formula in `listobject.c` is roughly `new_allocated = (newsize >> 3) + (newsize < 9 ? 3 : 6) + newsize`, giving growth around $1.125\times$ — gentler than $2\times$ to save memory. - If you `del lst[-1]` repeatedly down to roughly half the allocation, CPython will eventually shrink the buffer; `pop()` does this too in some versions, so capacity is not monotone. - "Amortized" hides per-call variance. A single append can be slow if it triggers a multi-MB realloc — relevant if you have a real-time SLA on tail latency.

What NOT to say: - "append is always $O(1)$ " — technically wrong; the worst case is $O(n)$ on the resize step. - "append is $O(n)$ because it sometimes copies" — wrong framing; you must say *amortized* $O(1)$ or you sound like you don't understand the distinction.

Common follow-up: "How would you pre-size a list to avoid the resize cost during appending one million items?" — Either preallocate with `lst = [None] * 1_000_000` and assign by index, or use `lst.extend(iterable_of_known_length)` which sizes once if the iterable supports `__len__`.

```
import sys
lst = []
prev = 0
for i in range(50):
    lst.append(i)
    curr = sys.getsizeof(lst)
    if curr != prev:
        print(f"len={len(lst)} sizeof={curr}") # shows the resize jumps
        prev = curr
```

Q3 · What is the time complexity of indexing a list, like `lst[k]` ?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant, Razorpay · Topic: list Big-O

The question (interviewer's verbatim phrasing):

If `lst` has ten million elements, how long does `lst[5_000_000]` take? Does it depend on `k` ?

The 30-second answer: $O(1)$, regardless of `k`. A list slot is a fixed-size pointer (8 bytes on a 64-bit build), so the address of slot `k` is `base + k * 8` — one pointer arithmetic, one dereference. The list length is irrelevant; index 0 and index 5_000_000 take the same number of nanoseconds.

Step-by-step thought process: 1. State the $O(1)$ headline. Don't hedge — this is the cleanest constant-time operation Python has. 2. Walk through the arithmetic. CPython holds a pointer to the start of `ob_item`. Indexing is `*(ob_item + k)`. There's a bounds check first — `if (0 <= k < ob_size)` — but that's also $O(1)$. 3. Contrast with a linked list. In a singly-linked list, getting to slot `k` requires walking `k` next-pointers, which is $O(k)$. That is why the linked-list-vs-array distinction in Q1 actually matters for performance. 4. Note negative indexing. `lst[-1]` is also $O(1)$ — CPython rewrites it to `lst[len + -1]` and indexes once.

Edge cases to mention: - Slicing — `lst[a:b]` — is $O(b - a)$, not $O(1)$, because it allocates a new list and copies pointer references across. - Indexing past the end raises `IndexError`; the bounds check is part of the $O(1)$ work, not a separate slow check. - For numpy arrays the same is true — $O(1)$ random

access — but for `deque` random access is $O(n)$, and that surprises people switching from list to deque.

What NOT to say: - "Indexing is $O(\log n)$ because of some binary search" — there is no binary search; the array is not sorted by index, it is *indexed* by index. - "Indexing depends on where the element is" — no, every slot is the same distance from the base pointer.

Common follow-up: "If list indexing is $O(1)$, why would you ever use a `dict` keyed by integer instead of a list?" — When the integer keys are sparse (e.g., `{1: 'a', 1_000_000: 'b'}`) a list would waste a million empty slots; a dict stores only the pairs you write. Also when keys are not contiguous or not non-negative integers.

```
lst = list(range(10_000_000))
import time
t = time.perf_counter()
for _ in range(1_000_000):
    _ = lst[5_000_000]    # index 5M
print(time.perf_counter() - t)  # same speed as lst[0] in a 10-element list
```

Q4 · What is the cost of `list.insert(0, x)` and `list.pop(0)` ?

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Flipkart, PhonePe, Postman · Topic: list Big-O

The question (interviewer's verbatim phrasing):

I have a list of ten million elements and I want to remove from the front in a loop. Why is that slow, and what should I use instead?

The 30-second answer: Both `list.insert(0, x)` and `list.pop(0)` are $O(n)$ because every element after the inserted or removed slot must be shifted by one position. For a ten-million-element list, that's roughly ten million pointer copies per call, and a loop of pops becomes $O(n^2)$ overall. The idiomatic fix is `collections.deque`, which is a doubly-linked block list that supports $O(1)$ prepend and $O(1)$ popleft.

Step-by-step thought process: 1. Anchor on the memory layout. A list is a contiguous array; the only way to keep it contiguous after inserting at position 0 is to slide every existing pointer right by one slot. CPython does this with a single `memmove`, but it's still $O(n)$ work. 2. Pop from the front is the mirror image — every pointer after slot 0 slides left by one. Again $O(n)$. 3. Put the loop math on the table. A queue that uses `list.pop(0)` in a producer-consumer pattern processes n items in $O(n^2)$ time. For $n = 100_000$ that's 10 billion pointer copies — minutes of wall time. 4. State the fix unambiguously: switch to `collections.deque` for queue or stack-at-both-ends patterns. Deque's append-left / popleft are $O(1)$ because the underlying block list lets it move the head pointer instead of sliding data.

Edge cases to mention: - `list.pop()` with no argument pops the last element and is $O(1)$ — only `pop(0)` and `pop(k)` for k far from the end are $O(n)$. - `list.insert(-1, x)` inserts before the last element, which still requires shifting one element — $O(1)$ in this corner case but pop-the-end-and-append-the-pair is the cleaner idiom. - `list.remove(value)` is $O(n)$ twice — once to find the value, once to shift the tail.

What NOT to say: - "list.insert(0, x) is $O(1)$ because it just changes a pointer" — wrong; it's $O(n)$ because of the shift. - "Use a different list" — vague; name `deque` specifically.

Common follow-up: "Why does deque give you $O(1)$ popleft when list can't?" — Deque is a doubly-linked list of fixed-size blocks (typically 64 pointers each). Popping from the left just advances a head index inside the first block; the block itself doesn't move, and other blocks are untouched.

```
from collections import deque
import time

# slow path
data = list(range(100_000))
t = time.perf_counter()
while data:
    data.pop(0)
print("list.pop(0):", time.perf_counter() - t, "seconds")

# fast path
data = deque(range(100_000))
t = time.perf_counter()
while data:
    data.popleft()
print("deque.popleft:", time.perf_counter() - t, "seconds")
```

Q5 · How does CPython over-allocate when growing a list?

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, JP Morgan, Postman, Razorpay · Topic: list internals

The question (interviewer's verbatim phrasing):

When `list.append` triggers a resize, how much extra space does CPython allocate? What's the growth factor?

The 30-second answer: CPython does not double the buffer. It uses a gentler formula — roughly `new_capacity = newsize + (newsize >> 3) + (3 or 6)` — which works out to about 1.125× growth, slightly more for small lists. The growth is still geometric, so appends remain amortized

$O(1)$, but the over-allocation is smaller than the $2\times$ factor used in classical textbooks. CPython picked the smaller factor to save memory on the millions of short lists that exist in a typical Python process.

Step-by-step thought process: 1. State the factor — $\sim 1.125\times$ — and contrast it with the $2\times$ factor most amortized-analysis textbooks assume. Both give amortized $O(1)$ appends; CPython's choice trades a few extra reallocations for a tighter memory footprint. 2. Walk through the actual formula from

`listobject.c` : `new_allocated = (newsize >> 3) + (newsize < 9 ? 3 : 6) + newsize`. For a small list (length 4), the bump is 3; for a list of length 1000, the bump is roughly 125. 3. Explain the design choice. Most Python lists are short and many never grow past their initial size. Allocating $2\times$ would waste memory on those; allocating $1.125\times$ costs a few more reallocations on large lists but saves a lot on small ones. 4. Note that the formula is implementation-defined and has changed across Python versions. Pinning down the exact number isn't the point; knowing it's smaller than $2\times$ and why, is.

Edge cases to mention: - The formula gives `new_allocated >= newsize`, so capacity always covers the requested size; it never under-allocates. - For lists created from a generator of unknown length, CPython has to grow as it goes — that's the path that hits the over-allocation logic most often. For lists created from a sized iterable (`list(range(1_000_000))`), CPython allocates the exact capacity up front. - The shrink behavior is also implementation-defined; the buffer is not shrunk after every pop, only when significantly over-allocated.

What NOT to say: - "CPython doubles the list" — that's the textbook answer, not the actual implementation. - "The growth factor is fixed at X" — it's not fixed; it's a function of current size and was tuned by hand for the common case.

Common follow-up: "Does PyPy use the same growth factor?" — No, PyPy and other implementations are free to pick their own; PyPy historically used a larger factor closer to the textbook $2\times$ for speed at the cost of memory.

```
import sys
lst = []
last = 0
for i in range(200):
    lst.append(i)
    capacity = (sys.getsizeof(lst) - 56) // 8 # approx, excludes struct header
    if capacity != last:
        print(f"len={len(lst):3d}  capacity≈{capacity}")
        last = capacity
```

Q6 · What is the time complexity of searching for a value in a list with `in` ?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Cred · Topic: list Big-O

The question (interviewer's verbatim phrasing):

If I write `if user_id in user_list`, what's the cost? When should I rewrite that?

The 30-second answer: $O(n)$. Python walks the list from index 0 and compares each element to the target with `==` until it finds a match or runs out of slots. There is no index, no sort assumption, no caching. The rewrite is a `set`: `if user_id in user_set` is $O(1)$ average for the membership test. If you're going to ask the question more than a handful of times against a list of more than a few hundred items, build the set once.

Step-by-step thought process: 1. Headline: $O(n)$ — linear scan. Same Big-O whether the value is the first element or absent. 2. Note the comparison is `==`, not `is`. That means each compare invokes the element's `__eq__`. For strings of length k , that's an $O(k)$ operation per compare, so list-of-strings membership is technically $O(n \cdot k)$. 3. State the fix: build a set. The trade-off is one $O(n)$ pass to construct the set, after which every lookup is $O(1)$. For one lookup, list is fine; for many lookups, set wins immediately. 4. Mention `list.index(value)` and `list.count(value)` have the same $O(n)$ cost for the same reason.

Edge cases to mention: - The early-exit makes "found near the start" cheap and "found at the end or absent" worst-case — but the Big-O is still $O(n)$. - `in` on a list of unhashable items (e.g., list of lists) cannot be sped up by a set conversion because lists aren't hashable; use a sorted list with `bisect` or restructure the data. - For sorted lists, `bisect_left` gives $O(\log n)$ membership — but Python doesn't auto-sort, so you'd have to maintain the order yourself.

What NOT to say: - "in is $O(1)$ " — true for set, dict, and frozenset; not for list, tuple, or range. - "list is fine because the early exit makes it fast" — that's a constant-factor optimization, not a Big-O improvement.

Common follow-up: "Is `in` on a `tuple` faster than on a `list`?" — Same Big-O — both are $O(n)$. Constant factors are similar; tuple has slightly less overhead because it's immutable, but the difference is rarely measurable.

```
import time
ids = list(range(1_000_000))
target = 999_999
t = time.perf_counter()
for _ in range(100):
    _ = target in ids          #  $O(n)$  each  $\rightarrow O(n \cdot 100)$  total
print("list:", time.perf_counter() - t)

s = set(ids)
t = time.perf_counter()
for _ in range(100):
    _ = target in s           #  $O(1)$  each
print("set:", time.perf_counter() - t)
```

Q7 · Why is `list.sort()` $O(n \log n)$ and what algorithm does CPython use?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Goldman, Flipkart · Topic: list
Big-O

The question (interviewer's verbatim phrasing):

What's the complexity of `list.sort()` ? And which sorting algorithm is behind it?

The 30-second answer: $O(n \log n)$ worst case, with a famously good constant factor on real-world data because CPython uses Timsort. Timsort is a hybrid of merge sort and insertion sort that exploits "runs" of already-sorted elements — if your data has any pre-existing order, Timsort runs closer to $O(n)$ on the sorted portions. It is stable, which means equal keys keep their original relative order, and that property matters for multi-key sorts done in passes.

Step-by-step thought process: 1. State the Big-O — $O(n \log n)$ worst case, $\Omega(n)$ best case — and then name the algorithm: Timsort, designed by Tim Peters for Python in 2002 and later adopted by Java's `Arrays.sort` for objects and by V8. 2. Walk through what makes Timsort fast on real data: it scans for ascending or descending runs, reverses descending runs in place, then merges the runs with merge sort. Random data behaves like a normal merge sort; partially-sorted data is sub- $O(n \log n)$ by a constant factor. 3. Call out stability. A sort is stable if it preserves the order of equal keys. Timsort is stable. The practical use: to sort by `(last_name, first_name)`, you sort by `first_name` then by `last_name`; the stability guarantees ties in `last_name` keep `first_name` order. 4. Mention the in-place vs returns distinction. `list.sort()` is in-place and returns `None`; `sorted(iterable)` returns a new list and works on any iterable. Same algorithm, different signature.

Edge cases to mention: - Timsort needs $O(n)$ auxiliary memory in the merge phase — not strictly $O(1)$ like a textbook heapsort. - The `key=` parameter is computed once per element and cached, not re-computed at each comparison; that means `sorted(items, key=expensive_func)` calls the key function exactly n times, not $n \log n$. - Comparing items that raise `TypeError` (e.g., sorting `[1, "x"]`) fails partway through and leaves the list partially mutated for `list.sort()`.

What NOT to say: - "Python uses quicksort" — wrong since Python 2.3; it's Timsort. - "It's stable because Python is interpreted" — stability is a property of the algorithm, not the runtime.

Common follow-up: "Why would I use `key=` instead of `cmp=`?" — Python 3 removed the `cmp` parameter entirely. `key=` is faster because it pre-computes one value per element; a comparator function would be called $O(n \log n)$ times. If you genuinely need a comparator, wrap it with `functools.cmp_to_key`.

```
# stability demo – sort by last name, ties preserve first-name order
people = [("Aarav", "Sharma"), ("Vivaan", "Sharma"), ("Aarav", "Patel")]
people.sort(key=lambda p: p[0]) # first by first name
people.sort(key=lambda p: p[1]) # then by last name – stable
# [('Aarav', 'Patel'), ('Aarav', 'Sharma'), ('Vivaan', 'Sharma')]
```

Q8 · What is the time complexity of slicing a list and does it copy?

Tags: Difficulty: Medium · Asked at: Swiggy, PhonePe, Postman, Microsoft India, JP Morgan · Topic: list Big-O

The question (interviewer's verbatim phrasing):

What does `lst[1000:2000]` actually do under the hood? Is it a view or a copy, and what's the cost?

The 30-second answer: It is a copy, not a view. CPython allocates a new list of length 1000 and copies pointers from the source slice into it — $O(k)$ where k is the slice length. The contained objects are not deep-copied; only the pointers are. NumPy slicing returns a view, but Python's built-in list does not.

Step-by-step thought process: 1. State copy vs view up front. List slice always copies. This is the source of a thousand surprised bug reports where a developer expected `b = a[:]` to alias and got a fresh list, or expected fresh and got aliasing. 2. Walk through the cost. The copy is shallow — the new list is a new pointer array; the pointed-to objects are shared. Cost is $O(k)$ for the slice length, regardless of the original list length. 3. Contrast with NumPy. `arr[1000:2000]` on a NumPy array returns a view (same buffer, new shape/stride), which is $O(1)$ but mutations leak back to the original. List's choice of copy semantics is intentional — Python prioritizes "you said `b = a[:]`, you get an independent `b`." 4. Mention the assignment-target form. `lst[a:b] = iterable` is the in-place mutation form — it can shrink or grow the list and is the only place where slice assignment doesn't allocate a new list (it mutates the existing one).

Edge cases to mention: - `lst[:]` is the canonical idiom for "make a shallow copy" — same as `list(lst)` and `lst.copy()`. - `lst[::-1]` reverses the list with a copy — $O(n)$. In-place reversal is `lst.reverse()`, also $O(n)$ but without the allocation. - Negative or out-of-range slice indices are silently clamped; `lst[1000:2000]` on a 10-element list returns `[]`, not an error.

What NOT to say: - "Slicing returns a view" — wrong for list; that's NumPy. - "Slicing is $O(1)$ " — wrong; it allocates a new pointer array of size k .

Common follow-up: "How do I get a view-like slice for a list without copying?" — You can't with built-in list; use `collections.deque` if you only need ends, `array.array` for typed sequences, or NumPy if you can require the dependency. Or hand-roll an index window with two integers.


```
a = [1, 2, 3, 4, 5]
b = a[1:4]          # b = [2, 3, 4] – new list, shallow copy
b[0] = 99
print(a)            # [1, 2, 3, 4, 5] – original untouched
print(b is a[1:4]) # False – even repeated slices return different lists
```

Q9 · When would you reach for `array.array` instead of `list` ?

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Microsoft India, Postman, Razorpay · Topic: list alternatives

The question (interviewer's verbatim phrasing):

When would you use `array.array` instead of a regular `list` ?

The 30-second answer: Almost never in application code — but when you specifically need a memory-compact, typed sequence of primitive numbers (ints, floats, bytes), `array.array` stores them as packed C values rather than as Python objects. A list of one million `int` takes ~28 MB (each int is a full `PyObject`); the equivalent `array.array("q", ...)` takes ~8 MB. Use it when you need that memory win without pulling in NumPy as a dependency.

Step-by-step thought process: 1. Frame the trade-off. List stores pointers to full Python objects; `array.array` stores raw machine values of a single declared type code (`"b"` , `"i"` , `"q"` , `"d"` , etc.). That gives you compactness and locality but loses heterogeneity. 2. Quantify the memory win. A `PyLongObject` for a small int is ~28 bytes; a slot in a `list` is 8 bytes for the pointer; total per int in a list is ~36 bytes (after deduping via the small-int cache the cost is lower). In `array.array("q", ...)` an int is 8 bytes flat. 3. State the limitations. No heterogeneous types — you can't mix int and str. Indexing still returns Python objects (the C int gets boxed on read), so per-element access is not faster than list. The win is memory, not CPU. 4. Pick the right tool. For pure-Python large numeric workloads where NumPy is overkill or unavailable, `array.array` is the right answer. For any real numeric workload — broadcasting, vectorized math, linear algebra — use NumPy.

Edge cases to mention: - `array.array("u", ...)` for unicode characters was deprecated in 3.3+; the standard advice is to use `str` for text. - `array.array` supports `tobytes()` / `frombytes()` for fast serialization to binary, which is sometimes the real reason to reach for it (file formats, network protocols). - Memory savings compound: list of one million pairs each containing a list of two ints is *much* larger than the equivalent array-of-pairs because every nested list has its own header overhead.

What NOT to say: - "array.array is faster than list" — generally not for compute; it's smaller, not faster. - "Use array.array instead of NumPy" — only if NumPy is genuinely unavailable; NumPy is the right tool for numerical work.

Common follow-up: "How does `array.array` compare to `bytes` and `bytearray`?" — `bytes` is an immutable sequence of single bytes (type code `"B"`); `bytearray` is the mutable version. `array.array("B", ...)` is functionally close to `bytearray` for unsigned-byte data. For wider types (4-byte int, 8-byte int, double) use `array.array`.

```
import array, sys
lst = [i for i in range(1_000_000)]
arr = array.array("q", lst)
print(sys.getsizeof(lst))    # ~8 MB just for the pointer array
print(sys.getsizeof(arr))    # ~8 MB for the packed ints, no per-int PyObject
```

Q10 · When should you NOT use a list at all?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Flipkart, Swiggy · Topic: list anti-patterns

The question (interviewer's verbatim phrasing):

Give me three scenarios where reaching for `list` would be the wrong call.

The 30-second answer: Three classic anti-patterns: (1) you need $O(1)$ prepend or popleft — use `collections.deque`; (2) you need $O(1)$ membership tests — use `set` or `frozenset`; (3) you need to maintain a fixed-size most-recently-used buffer — use `deque(maxlen=N)` or `OrderedDict`. The deeper rule: `list` is for sequences whose access pattern is iterate-or-append. The moment your access pattern is anything else, there's a better container.

Step-by-step thought process: 1. Pre-frame: the question is asking you to demonstrate knowledge of alternatives. Answer with three named scenarios and the right replacement for each. 2. Scenario one — front operations. `list.insert(0, x)` and `list.pop(0)` are $O(n)$. Replace with `collections.deque`, which gives you $O(1)$ at both ends. Use cases: queue, sliding window, BFS frontier. 3. Scenario two — membership tests. `value in lst` is $O(n)$. Replace with `set` for $O(1)$ average lookup. Use case: deduplication, membership testing, set algebra. 4. Scenario three — bounded buffers. A list does not auto-evict; you'd have to write `lst.append(x); if len(lst) > N: lst.pop(0)` which is $O(n)$ per push. Replace with `deque(maxlen=N)` which evicts the oldest automatically in $O(1)$. Use cases: rolling averages, last-N-events ring, recent-items cache. 5. Bonus scenario worth mentioning if time permits: a priority queue. `list` would require sorting on every push ($O(n \log n)$) or scanning for the min on every pop ($O(n)$). Replace with `heapq` which gives $O(\log n)$ push and pop.

Edge cases to mention: - "I need a sorted list" — `bisect.insort` works on a list and is $O(n)$ for the insert (shift cost). For heavy churn, a balanced BST library like `sortedcontainers.SortedList`

gives $O(\log n)$. - "I need a list of fixed length where order matters" — that's still a list; this isn't an anti-pattern. The anti-patterns are about access *pattern*, not length. - "I need a list of one type only" — type system, not container choice; you'd still use list with a type hint like `list[int]`.

What NOT to say: - "list is bad" — no; list is the right answer for the iterate-or-append case, which is the common case. - "Use a generator instead of a list" — generators are about laziness, not about the access-pattern anti-patterns above.

Common follow-up: *"What's the trade-off when you switch from list to deque?"* — Deque loses $O(1)$ random indexing (it's $O(n)$ in the worst case because you'd walk block links) and slicing isn't supported the same way. If you need both random indexing AND $O(1)$ front ops, you genuinely have to pick — usually a `dict` keyed by index or a doubly-indexed structure is the answer.

```
from collections import deque

# anti-pattern: list-as-queue
q = []
q.append("event1"); q.append("event2")
q.pop(0) # O(n)

# idiomatic: deque-as-queue
q = deque()
q.append("event1"); q.append("event2")
q.popleft() # O(1)

# bounded buffer in one line
recent_events = deque(maxlen=100)
for e in stream:
    recent_events.append(e) # auto-evicts when full
```

Section B — `dict` Internals and Big-O

Ten questions on Python's most-used data structure after `list`. `dict` is a hash table — specifically, since Python 3.6, a compact-and-ordered hash table — and almost every interview question about it is really about hashing, collision resolution, or the insertion-order guarantee. The CPython implementation underwent two famous redesigns: PEP 412 (key-sharing dicts for instance attributes, 3.3) and the compact dict layout (3.6, made an official guarantee in 3.7). Knowing the timeline and the why is what separates a candidate who has used dicts from one who understands them.

Q11 · How is Python's `dict` implemented under the hood?

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft India, Goldman, JP Morgan, Flipkart · Topic: dict internals

The question (interviewer's verbatim phrasing):

Describe how CPython's dict is laid out in memory. What collision-resolution strategy does it use?

The 30-second answer: CPython's dict is a hash table with open addressing and perturbation-based probing. Since 3.6 it uses a two-array compact layout: a small "indices" array (typically int8 / int16 / int32 depending on size) that maps hash slots to indices, and a packed "entries" array that holds (hash, key, value) triples in insertion order. The compact layout cut dict memory roughly 20–25% and gave us the insertion-order guarantee for free. Collisions are resolved by probing — not by chaining buckets like Java's `HashMap`.

Step-by-step thought process: 1. Open with "hash table" and immediately specify *open addressing* — that's the discriminator between Python's design and Java's `HashMap` (which uses separate chaining). 2. Describe the two-array compact layout: indices array (sparse, sized to the table capacity, holds indices into the entries array or a sentinel for empty/deleted), entries array (dense, sized to the number of entries actually present, holds the records in insertion order). 3. Walk through the lookup. Compute `hash(key)`, take it modulo the table size to get a starting index slot. Read `indices[slot]`; if it's empty, the key is absent. If it's an index, read `entries[index]`. If the stored key equals the lookup key, return the value. If not, perturb the slot and try again (probing). 4. Mention the load factor. CPython resizes the table when 2/3 of slots are occupied, keeping average probe length low. This is why "O(1) average lookup" is defensible: you never let the table get full.

Edge cases to mention: - The indices array is sized as a power of two so that `hash % size` reduces to a bitmask. Sizes start at 8. - The perturbation formula is `j = (5*j + 1 + perturb) % size; perturb >= 5` — it cycles all slots, guaranteed to find an empty one if one exists. - Deleted entries leave a "dummy" sentinel in the indices array (so probing doesn't break), but the entry in the entries array is cleared. A resize compacts away the dummies.

What NOT to say: - "dict uses chaining like Java's HashMap" — wrong; it uses open addressing. - "dict slots store linked lists" — wrong; one entry per slot, collisions handled by probing.

Common follow-up: "Why did Python switch to the compact layout in 3.6?" — Raymond Hettinger's prototype showed it saved 20–25% memory and incidentally preserved insertion order at zero CPU cost. The order preservation was promoted from implementation detail to language guarantee in 3.7 because too much real code already depended on it.

```
d = {"a": 1, "b": 2, "c": 3}
# Conceptually: indices = [-, -, 0, -, -, 1, 2, -] (sparse, length 8)
# entries = [(hash("a"), "a", 1), (hash("b"), "b", 2), (hash("c"), "c", 3)] (dense)
# Iteration walks `entries` in order — that's the insertion-order guarantee
```

Q12 · What is the average and worst-case time complexity of `dict` lookup?**Tags:** Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Postman · Topic: dict Big-O**The question (interviewer's verbatim phrasing):**

What's the complexity of `d[key]` ? Is it always constant time?

The 30-second answer: Average case $O(1)$, worst case $O(n)$. The average case assumes a well-distributed hash function and a load factor capped at $2/3$, both of which CPython enforces. The worst case happens when every key collides into the same probe chain — typically only seen with adversarial inputs or a broken `__hash__` method that returns a constant. For normal application code, treat dict lookup as $O(1)$.

Step-by-step thought process: 1. State the two cases together. Average $O(1)$, worst $O(n)$. Don't omit the worst case; senior interviewers ask specifically about it. 2. Explain what guarantees the average. CPython resizes when $2/3$ full, so the expected number of probes for a lookup is bounded by a small constant. Python's built-in types have well-behaved `__hash__` (small ints hash to themselves, strings use SipHash with a random seed since 3.3 specifically to make collision attacks hard). 3. Explain what causes the worst case. Two routes: (a) you wrote a custom `__hash__` that returns a constant — every insert collides, every lookup walks the entire probe chain; (b) an adversarial attacker crafts inputs that hash-collide with your specific seed — mitigated by SipHash randomization. 4. Note that " $O(1)$ average" is the right answer for an interview question and you should not lead with "well actually $O(n)$ worst case" — that's the follow-up.

Edge cases to mention: - A constant `__hash__` doesn't crash; it just turns your dict into an $O(n)$ linear-probe list. - The string hash uses a per-process random seed (`PYTHONHASHSEED`) so two runs produce different hash orders unless you pin the seed; this is the security mitigation against hash flooding. - Resize is amortized $O(1)$ per insert, similar to list's append — you periodically double the table.

What NOT to say: - "dict lookup is always $O(1)$ " — wrong; worst case is $O(n)$. - "dict lookup is $O(\log n)$ " — wrong; that's a balanced tree, not a hash table.

Common follow-up: "What's `PYTHONHASHSEED` and why does it exist?" — It's the random seed for the string hash function. It exists to prevent hash-flooding DoS attacks where an attacker submits keys crafted to all hash to the same slot. You can pin it (`PYTHONHASHSEED=0`) for reproducible test runs; production should leave it random.

```
class BadHash:
    def __init__(self, v): self.v = v
    def __hash__(self): return 0          # everything collides
    def __eq__(self, other): return self.v == other.v

import time
d = {}
for i in range(5_000):
    d[BadHash(i)] = i          # progressively slower – probe chain grows
# vs. a normal int-keyed dict which stays O(1) per insert
```

Q13 · Why does `dict` preserve insertion order? Is this guaranteed?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Microsoft India, Goldman, Postman · Topic: dict behavior

The question (interviewer's verbatim phrasing):

Since when does `dict` preserve insertion order, and is it a language guarantee or just an implementation detail?

The 30-second answer: Insertion order has been preserved by CPython since 3.6 as an implementation detail, and was promoted to a language guarantee in 3.7. So any code targeting Python 3.7+ can rely on `for k in d` iterating in insertion order. The reason CPython gets this property "for free" is the compact dict layout: the entries array is dense and written in insertion order, and iteration walks the entries array directly.

Step-by-step thought process: 1. Lead with the two versions. 3.6 = CPython implementation detail, no language guarantee. 3.7+ = official language guarantee, all conforming implementations must preserve insertion order. 2. Explain *why* it's automatic in the compact layout. The entries array is append-only on insert; iteration walks that array index by index. There is no extra bookkeeping needed. 3. Address the "is OrderedDict still useful?" question proactively. Yes — `OrderedDict` has `move_to_end(key)` and equality compares order, while plain dict equality ignores order. Don't reach for `OrderedDict` for ordering alone any more, only for those extra capabilities. 4. Mention the deletion behavior. When you delete a key, it leaves a hole in the entries array; subsequent inserts may fill the hole or extend the array. The exact visible behavior across versions is "iteration order is insertion order minus deletions"; tools that depend on tight semantics should be aware.

Edge cases to mention: - `dict` comparison with `==` ignores order: `{"a": 1, "b": 2} == {"b": 2, "a": 1}` is True. Only `OrderedDict.__eq__` against another `OrderedDict` considers order. - Update with the same key does not change position. `d["a"] = 99` for an existing key updates the value in place, keeping `"a"` at its original position. - For 3.6 specifically, CPython

preserved order but other implementations (Jython, IronPython at the time) didn't have to — so portable 3.6 code could not rely on it. The 3.7 guarantee made it universal.

What NOT to say: - "dict was always ordered" — wrong; pre-3.6 CPython dicts had no order guarantee.
 - "OrderedDict is now useless" — wrong; the `move_to_end` and order-aware `__eq__` capabilities remain.

Common follow-up: "What's the difference between `dict` and `OrderedDict` in 3.12?" — Both preserve insertion order. `OrderedDict` has `move_to_end(key, last=True/False)` and `popitem(last=True/False)` with a `last` argument; equality compares order; the LRU cache pattern via `move_to_end` is its niche use case.

```
d = {}
d["c"] = 3
d["a"] = 1
d["b"] = 2
for k in d: print(k)           # c, a, b — insertion order, guaranteed in 3.7+
d["c"] = 99                   # update in place, position unchanged
for k in d: print(k)           # c, a, b
```

Q14 · What's the difference between `d[k]`, `d.get(k)`, and `d.setdefault(k, default)` ?

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Flipkart, Postman, Cred · Topic: dict idioms

The question (interviewer's verbatim phrasing):

What are the three idiomatic ways to access a dict value when the key might be missing, and when do you pick each?

The 30-second answer: `d[k]` raises `KeyError` if the key is absent — use when absence is a bug. `d.get(k, default)` returns the default (None by default) and never raises — use when absence is a normal case and you want a fallback. `d.setdefault(k, default)` returns the existing value if present, or inserts and returns the default if absent — use when you're about to write something to the key and want to handle the "first time we see this key" case in one call.

Step-by-step thought process: 1. Walk through `d[k]`. Simple, fast, raises `KeyError` on miss. The right tool when the key *must* exist; absence means the upstream invariant was violated. 2. Walk through `d.get(k, default)`. Same lookup, but returns `default` (or None) instead of raising. Used heavily in HTTP request parsing, config reading, and "maybe-present field" patterns. Does not mutate the dict — important distinction from `setdefault`. 3. Walk through `d.setdefault(k, default)`. The "lookup, and if missing, insert this default and return it" idiom. The classic use is building a dict-of-lists:

`d.setdefault(key, []).append(item)` — one call instead of an if-else branch.

`defaultdict` does the same job more cleanly if you know the factory up front; `setdefault` is for the case where you don't. 4. Performance note: `get` and `setdefault` both hash the key once; the conditional branch is in C and is faster than the Python-level if-else equivalent.

Edge cases to mention: - `d.get(k)` with no second argument returns `None`, not raises — easy bug if `None` is a legal value distinct from "missing". - `d.setdefault(k, expensive_factory())` always evaluates the factory, even when the key exists. Use `defaultdict(expensive_factory)` if you want lazy evaluation. - The "default" in `get` and `setdefault` is a positional argument; passing a tuple as default does what you'd expect, not what you'd guess if you confused it with `dict.fromkeys`.

What NOT to say: - "Just use try/except `KeyError`" — works, but less idiomatic and slower for the miss case. - "setdefault and get are the same" — wrong; `setdefault` inserts the default; `get` does not.

Common follow-up: "When would you reach for `defaultdict` over `setdefault`?" — When the factory is known in advance and you'll be doing many missing-key inserts. `defaultdict(list)` for dict-of-lists is cleaner than calling `setdefault` everywhere; `setdefault` is right when only a few specific calls need a default or the factory varies.

```
d = {"name": "Aarav", "age": 23}

# strict: must be present
city = d["city"]           # raises KeyError

# tolerant: maybe present
city = d.get("city")       # None
city = d.get("city", "Mumbai") # "Mumbai"

# tolerant + initialize-on-first-touch
counts = {}
for word in ["a", "b", "a", "c", "a"]:
    counts.setdefault(word, 0) # inserts 0 if missing
    counts[word] += 1
# {'a': 3, 'b': 1, 'c': 1}
```

Q15 · What does it mean that `dict.keys()`, `.values()`, and `.items()` return views?

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft India, Goldman, Postman, Flipkart · Topic: dict views

The question (interviewer's verbatim phrasing):

What's special about the objects returned by `d.keys()` and `d.items()`? What does "view" mean?

The 30-second answer: They are dynamic views — lightweight wrapper objects that read directly from the underlying dict without making a copy. Iterating a view walks the dict's current entries; if you mutate the dict while iterating, the view sees the change. `keys()` and `items()` also support set-like operations — `d.keys() & other.keys()` returns the intersection — because keys are unique and hashable. This is why views exist instead of returning a list: it's $O(1)$ to create, supports set algebra, and stays in sync with the dict.

Step-by-step thought process: 1. State the headline: views are not lists, not copies, not generators — they are live references to the dict's keys/values/items. $O(1)$ to construct, $O(n)$ to iterate. 2. Show the set-like behavior. `d.keys() | other.keys()` is the union, `d.keys() & other.keys()` is the intersection. Same for items (because (key, value) tuples are hashable when the value is). Not for values, because values can be duplicate and aren't required to be hashable. 3. Explain the staleness contract. If you mutate the dict — add or remove a key — while iterating a view, you get `RuntimeError: dictionary changed size during iteration`. Updating an existing key's value is allowed; adding or removing is not. 4. Contrast with Python 2's behavior. In Python 2, `d.keys()` returned an actual list (a copy). The 3.x move to views was a memory and performance optimization — and code that does `for k in list(d.keys())` is the explicit "give me a snapshot" idiom when you do need to mutate while iterating.

Edge cases to mention: - `len(d.keys())` is $O(1)$ — views know their size. - `key in d.keys()` is equivalent to `key in d` and is $O(1)$ average — both go through the hash table. `key in list(d.keys())` is $O(n)$ because it does a linear scan over the snapshot. - View objects are not subscriptable: `d.keys()[0]` raises `TypeError`. If you need indexed access, materialize: `list(d.keys())[0]`.

What NOT to say: - "keys() returns a list" — true for Python 2, wrong for Python 3. - "Views are generators" — wrong; views are reusable (you can iterate them more than once), generators are single-pass.

Common follow-up: "What happens if I do `for k in d: d[k+'_x'] = 1`?" — `RuntimeError: dictionary changed size during iteration`. To do this safely, iterate `list(d.keys())` — that snapshots into a list and you can mutate the dict freely during the loop.

```
d = {"a": 1, "b": 2}
keys = d.keys()
d["c"] = 3
print(list(keys))           # ['a', 'b', 'c'] – view sees the new key

# set algebra on views
a = {"x": 1, "y": 2}
b = {"y": 3, "z": 4}
print(a.keys() & b.keys())  # {'y'}
print(a.items() | b.items()) # {('x', 1), ('y', 2), ('y', 3), ('z', 4)}
```

Q16 · What's the memory cost of a dict per entry, and how does PEP 412 help?

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman, JP Morgan, Razorpay, Postman · Topic: dict internals

The question (interviewer's verbatim phrasing):

Roughly how much memory does a dict use per entry, and what does PEP 412 (key-sharing dicts) do?

The 30-second answer: A dict entry costs around 50–100 bytes after the compact-layout improvements in 3.6 — the indices array contributes a few bytes per slot, and the entries array contributes (hash, key pointer, value pointer) = 24 bytes per actual entry, plus rehashing slack. PEP 412 ("Key-Sharing Dictionary") is the optimization for instance `__dict__` objects: when many instances of a class share the same set of attribute names, the keys array is stored once on the class and instance dicts just hold the values. This roughly halves the per-instance dict cost in typical OO code.

Step-by-step thought process: 1. Set scale. A single empty dict is ~64 bytes; a one-entry dict is ~232 bytes (sizes from `sys.getsizeof`). Per-entry cost emerges as the dict grows: roughly 30 bytes of indices slot + 24 bytes of entries record, with load-factor slack pushing this to ~50–80 bytes amortized. 2. Explain the PEP 412 design. Every Python instance carries a dict (`__dict__`) for its attributes. Across millions of instances of one class, the *keys* are nearly always the same (the set of attribute names declared in `__init__`). PEP 412 stores those keys once on the class and points all instance dicts at the shared keys table; the instance dict just holds a packed array of values. 3. Note the trade-off. Key-sharing dicts work only as long as all instances stick to the same set of keys. If one instance adds a new key, that instance's dict is converted back to a normal "combined" dict (keys + values together). So the optimization is silent and automatic but breaks if your code is dynamic about attribute names. 4. The headline saving. For instance dicts that obey the contract, the saving is ~30–50%. Combined with `__slots__` (which eliminates the dict entirely) you have two tiers: `__slots__` for the tightest classes, key-sharing dict for the common case.

Edge cases to mention: - Watch out for "dict is small" intuitions when scaling to millions of entries; even with the compact layout, a dict of one million int→int pairs is ~75 MB, not 16 MB. -

`sys.getsizeof(d)` returns the dict's own size but not the size of its keys or values. A dict of one million `str` keys is dominated by the strings, not the dict overhead. - Key-sharing breaks if you assign attributes outside `__init__`; lint tools sometimes flag this, but it's not an error — the dict just transparently de-optimizes.

What NOT to say: - "dict is small" — vague; for big workloads it isn't. - "PEP 412 is about ordering" — wrong; ordering came from the 3.6 compact layout, not PEP 412.

Common follow-up: "How does `__slots__` compare to PEP 412 key-sharing dicts?" —

`__slots__` removes the dict entirely and stores attributes in a fixed array, saving the dict overhead. It's stricter (no dynamic attributes) but tighter. Key-sharing dicts are the silent middle path that works without any code change.

```
import sys
empty = {}
print(sys.getsizeof(empty))    # 64 bytes
d = {"a": 1}
print(sys.getsizeof(d))        # ~232 bytes (allocates table of size 8)

class WithSlots:
    __slots__ = ("name", "age")
    def __init__(self): self.name = "x"; self.age = 1

class WithDict:
    def __init__(self): self.name = "x"; self.age = 1

print(sys.getsizeof(WithDict().__dict__))    # ~104 bytes (key-sharing)
# WithSlots has no __dict__ at all
```

Q17 · Can you use a list as a dict key? Why or why not?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Flipkart, Cred · Topic: hashability

The question (interviewer's verbatim phrasing):

Can I use a list as a dict key? If not, why? And what's the workaround?

The 30-second answer: No — lists are unhashable because they are mutable. A dict key's hash is used to find the right slot, and if the hash could change after insertion (because the list was mutated), the dict would lose the key forever. The workaround is `tuple` for an immutable equivalent, or `frozenset` if you need set-like membership without order, or `str.join` for a stringified composite key.

Step-by-step thought process: 1. Lead with the requirement. dict keys must be hashable, which means `__hash__` is defined and the hash is stable across the object's lifetime. Built-in mutable types — list, dict, set, bytearray — deliberately do not implement `__hash__` and raise `TypeError` on `hash(x)`. 2. Explain why mutability and hashability conflict. Once a key is placed in slot `i`, the dict relies on `hash(key)` continuing to return the same value. If you mutate the key, its hash changes, you can no longer find the entry, and the dict is silently broken. 3. State the workaround. Convert the list to a tuple — `tuple(my_list)` — which is hashable as long as its contents are hashable. For unordered collections, use `frozenset`. For long composite keys you may stringify with a separator, though tuple is usually cleaner. 4. Mention the custom-class case. If you write your own class and want instances usable as dict keys, you need `__hash__` and `__eq__` to be consistent. If you make `__eq__` care about a mutable field, you must either freeze the field or accept that mutating it breaks dict semantics.

Edge cases to mention: - `frozenset` is hashable; `set` is not. `frozenset([1, 2, 3])` works as a key. - Tuples containing a list are still unhashable: `hash((1, [2, 3]))` raises `TypeError` because the contained list is mutable. - `bytes` is hashable; `bytearray` is not.

What NOT to say: - "You can use a list as a key if you don't mutate it" — wrong; the language refuses to even compute `hash([])` so you can't. - "Convert to string with `str(list)`" — works but is fragile (different string forms can hash differently if list contents have weird `__repr__`); prefer tuple.

Common follow-up: "What's the hash of a tuple?" — A combination of its elements' hashes, computed via a specific formula in `tuplehash` in `tupleobject.c`. Two equal tuples (same elements in the same order) have the same hash, which is the requirement for dict and set to work.

```
d = {}
key = [1, 2, 3]
# d[key] = "x"           # TypeError: unhashable type: 'list'

# fix: tuple
d[tuple(key)] = "x"
print(d[(1, 2, 3)])      # "x"

# unordered key: frozenset
seen = {frozenset({"role:admin", "team:eng"})}
print(frozenset({"team:eng", "role:admin"}) in seen)  # True – order-independent
```

Q18 · What happens when two keys hash to the same slot in a dict?

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, JP Morgan, Razorpay, Postman · Topic: collision resolution

The question (interviewer's verbatim phrasing):

Walk me through what happens when two different keys hash to the same slot.

The 30-second answer: CPython uses open addressing with perturbation probing. The first key takes the slot; when the second key hashes to the same slot, the dict probes a sequence of alternative slots — `j = (5*j + 1 + perturb) % size; perturb >= 5` — until it finds an empty one, where it places the new entry. Lookups for either key follow the same probe sequence, comparing the stored hash and key at each slot until a match (or an empty slot signaling absence) is found. This avoids the linked-list-per-bucket overhead of separate chaining.

Step-by-step thought process: 1. Set the framing: open addressing, not separate chaining. Java's `HashMap` chains; Python's `dict` probes. This is the difference the interviewer is checking. 2. Walk the insert path. Compute `hash(key) % size` → starting slot. If empty, place the entry. If occupied, run the perturbation formula to get the next candidate slot, test again, and repeat until an empty slot is found. 3. Walk the lookup path. Same starting slot, same probe sequence. At each slot, compare the stored hash to the lookup hash (fast int compare); if equal, fall through to comparing keys with `__eq__` (slower); if neither matches, continue probing. An empty slot signals "key not present" and ends the probe. 4. Mention the perturbation. The `perturb` variable starts as the full hash and gets shifted right by 5 each step. This means the probe sequence is not a simple linear scan — it spreads collisions out across the table, reducing clustering.

Edge cases to mention: - Stored hash (not recomputed) is checked first. If the stored hash differs from the lookup hash, `__eq__` is skipped. This is the fast path for "different keys that happen to collide" — they're rejected on hash compare alone. - Deleted entries leave a "dummy" sentinel so probes can continue past them; without the dummy, a probe would stop at the deleted slot and miss subsequent entries. - A resize rebuilds the table from scratch, eliminating dummies and re-probing every entry into the new (larger or smaller) table.

What NOT to say: - "Same-hash keys go into a linked list at that slot" — that's separate chaining (Java `HashMap`), not Python. - "Same-hash keys overwrite each other" — wrong; the dict probes to find an empty slot.

Common follow-up: "What's the worst-case probe length?" — In theory $O(n)$ if all keys probe through every slot, but in practice CPython resizes at 2/3 load factor, so the expected probe length stays under 2. Pathological cases require either a constant `__hash__` or an adversarial key set that defeats SipHash randomization.

```
class CollideAlways:
    def __init__(self, v): self.v = v
    def __hash__(self): return 1 # everything collides into the same starting slot
    def __eq__(self, other): return isinstance(other, CollideAlways) and self.v == other.v

d = {}
d[CollideAlways(1)] = "a"
d[CollideAlways(2)] = "b" # probes past slot for hash=1, lands somewhere else
d[CollideAlways(3)] = "c"
print(d[CollideAlways(2)]) # "b" — probe sequence finds it
```

Q19 · What's the cost of deleting a key from a dict?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, JP Morgan, Flipkart · Topic: dict Big-O

The question (interviewer's verbatim phrasing):

What's the time complexity of `del d[k]` ? Does it leave any tombstones?

The 30-second answer: $O(1)$ average. The dict probes to the key's slot, marks the indices entry with a "dummy" sentinel so future probes skip past it, and clears the corresponding entries record. The dummy is what allows lookups for *other* keys to continue probing past the deleted slot without false negatives. Tombstones accumulate over many deletes; CPython will compact them away on the next resize, which itself is amortized $O(1)$ per operation.

Step-by-step thought process: 1. Headline: $O(1)$ average. Same probe cost as a lookup. 2. Walk the dummy mechanism. If you simply zeroed the slot, a probe for some *other* key that originally probed through this slot would think the chain ended early and report a false miss. The dummy says "this slot was once occupied; keep probing". 3. Explain the cleanup. A dict that has accumulated many dummies — many delete-then-insert cycles — will eventually be resized either up (because of insertion-driven growth) or down (CPython does this for shrinking workloads in some versions). The resize rehashes live entries into a fresh table with no dummies. 4. Memory note. The entries array is dense (insertion-order). When you delete an entry, its slot in entries is cleared but the array is not compacted until a resize. Iteration skips cleared slots — there's no visible gap.

Edge cases to mention: - `dict.pop(k, default)` is the "delete and return" form; same $O(1)$ average cost. - `dict.clear()` is $O(n)$ — it resets the dict to empty by allocating a fresh small table. The freed entries' values may keep their references alive elsewhere; clear doesn't deep-free anything. - A long-lived dict with a heavy delete-and-reinsert workload may benefit from `dict(d)` to force a clean rebuild, though this is rarely necessary in practice.

What NOT to say: - "del d[k] is O(n) because of shifting" — wrong; that's list. Dict doesn't shift anything.
 - "del leaves a hole that breaks future lookups" — wrong; the dummy sentinel keeps probing correct.

Common follow-up: "What's the cost of `d.popitem()`?" — O(1). Since 3.7 it pops the most-recently-inserted entry (LIFO), so it's just "delete the last entry in the dense entries array" — no probe needed. Pre-3.7 it popped an arbitrary entry.

```
d = {"a": 1, "b": 2, "c": 3, "d": 4}
del d["b"]           # 0(1) — marks slot as dummy
print(list(d))       # ['a', 'c', 'd'] — insertion order preserved minus 'b'
print(d.pop("c"))     # 3, same cost
print(d.popitem())    # ('d', 4) — LIFO from insertion order
```

Q20 · What does `dict | other_dict` do, and when was it added?

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Cred, Microsoft India, PhonePe · Topic: dict syntax

The question (interviewer's verbatim phrasing):

What does the `|` operator do on dicts? When did it get added?

The 30-second answer: PEP 584 added `|` and `|=` to `dict` in Python 3.9. `d1 | d2` returns a new dict with the merged contents — values from `d2` win where keys overlap. `d1 |= d2` mutates `d1` in place. Before 3.9, the idioms were `{**d1, **d2}` (new dict) or `d1.update(d2)` (in-place). The operator is just nicer syntax for the common merge case.

Step-by-step thought process: 1. State the operator and version. `|` since Python 3.9, both binary form (new dict) and augmented form (`|=`, in-place). 2. Clarify the merge precedence. `d1 | d2` — keys present in both take the value from `d2`, the right operand. This matches dict-update semantics ("the second one wins") and matches what people expect. 3. Walk the alternatives. `{**d1, **d2}` is the pre-3.9 idiom and still works; same semantics, slightly more verbose. `d1.update(d2)` is the in-place alternative to `|=`. Both `{**d1, **d2}` and `d1 | d2` produce a new dict, so they don't mutate. 4. Mention the type-stability rule. `d1 | d2` returns a regular `dict`, not a subclass — so if `d1` is an `OrderedDict`, you get a plain dict back. For subclass-aware merging, use `type(d1)({**d1, **d2})` or roll your own.

Edge cases to mention: - `|` requires both operands to be dicts (or dict subclasses). `d | [{"a", 1}]` raises `TypeError`; use `d | dict([("a", 1)])` or `d.update([...])`. - Order is preserved across both operands. `{"a": 1} | {"b": 2, "a": 99}` is `{"a": 99, "b": 2}` — keys from `d1` first in their original positions, new keys from `d2` appended, overlapping keys take `d2`'s value but stay at `d1`'s position. - For dataclass-style merging where you want "fill in defaults", the idiom is `defaults | overrides` — overrides on the right so they win.

What NOT to say: - "`|` is symmetric like set union" — false; values from the right operand win, so the operation is not symmetric. - "`|` existed since Python 3" — wrong; added in 3.9.

Common follow-up: "What does the `|=` form do compared to `update`?" — Same effect on `d1`. The only difference: `|=` requires another dict on the right (or another iterable of pairs in some implementations), and returns the updated `d1`. `update` accepts an iterable of pairs as well as kwargs. For most use cases they're interchangeable; pick the one that reads better.

```
defaults = {"timeout": 30, "retries": 3}
overrides = {"retries": 5, "verbose": True}
config = defaults | overrides
# {'timeout': 30, 'retries': 5, 'verbose': True}

# pre-3.9 idiom
config = {**defaults, **overrides}      # same result

# in-place
defaults |= overrides                  # mutates defaults
```

Section C — set and frozenset

Five questions on Python's hash-based unordered collection. `set` is built on the same hash-table machinery as `dict` — minus the value array — so most of the dict Big-O carries across directly. The interview hook is membership testing: "your `value in lst` is $O(n)$; rewrite it with a set" is one of the most-asked optimisation prompts at Indian product companies. The other hook is hashability, which is the same constraint as dict keys.

Q21 · What is the average time complexity of set membership, and why is it faster than list?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Swiggy, Postman, TCS · Topic: set Big-O

The question (interviewer's verbatim phrasing):

If I have a million-element collection and I'll do a million membership checks, should I use a list or a set, and what's the cost of each?

The 30-second answer: Use a set. Set membership (`x in s`) is $O(1)$ average because it hashes `x` and probes one slot of the underlying hash table; list membership is $O(n)$ because it walks every element.

For a million checks against a million-element collection, list is $\sim 10^{12}$ compares and set is $\sim 10^6$ hash lookups — a million-fold gap.

Step-by-step thought process: 1. Headline the two numbers. Set `in` = $O(1)$ average, list `in` = $O(n)$. The factor is the entire point of switching. 2. Explain the mechanism. Set shares dict's hash table; an `in` check computes `hash(x)`, indexes to a slot, compares the stored key with `==`. Done in constant time on average. 3. Walk the build-once trade-off. Constructing the set is $O(n)$ — you have to hash and insert each element once. After that every lookup is $O(1)$. The break-even is around two or three lookups; beyond that, set wins on every additional check. 4. Mention the hashability constraint. Set members must be hashable. A list of lists cannot be turned into a set without converting the inner lists to tuples first.

Edge cases to mention: - Worst case is $O(n)$ — same adversarial-hash scenario as dict. Application code rarely sees it. - `frozenset` has the same membership cost as `set` and is hashable itself, so it can be a member of another set. - Set construction from an iterable of length n is $O(n)$ average, not $O(n \log n)$ — no sorting required.

What NOT to say: - "Set is always faster than list" — wrong for one-off membership; building the set isn't free. - "Set is $O(\log n)$ " — wrong; hash-based, not tree-based.

Common follow-up: "What's the memory cost of a set vs a list of the same size?" — Set is larger per element — roughly 200 bytes per entry vs 8 bytes per list pointer — because of the hash table overhead. You're trading memory for lookup speed.

```
import time
n = 1_000_000
data = list(range(n))
targets = [n - 1, n // 2, -1]

t = time.perf_counter()
for tg in targets * 10_000:
    _ = tg in data          # O(n) each
print("list:", time.perf_counter() - t)

s = set(data)
t = time.perf_counter()
for tg in targets * 10_000:
    _ = tg in s             # O(1) each
print("set:", time.perf_counter() - t)
```

Q22 · What set operations does Python support and what are their Big-O costs?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Goldman, Flipkart · Topic: set algebra

The question (interviewer's verbatim phrasing):

Walk me through union, intersection, difference, and symmetric difference on sets. What's the cost of each?

The 30-second answer: Four set-algebra operations, all backed by operators and methods: union (`|` / `.union()`), intersection (`&` / `.intersection()`), difference (`-` / `.difference()`), symmetric difference (`^` / `.symmetric_difference()`). Each is roughly $O(\text{len of smaller set})$ for intersection and $O(\text{len of both sets})$ for union and symmetric difference. The augmented forms `|=`, `&=`, `-=`, `^=` — mutate the left-hand set in place.

Step-by-step thought process: 1. Walk the four operations with their operators side by side. Union returns members in either set; intersection returns members in both; difference returns members in the first but not the second; symmetric difference returns members in exactly one of the two. 2. Quantify the costs. Union is $O(\text{len}(a) + \text{len}(b))$ — walk both. Intersection is $O(\min(\text{len}(a), \text{len}(b)))$ — Python is smart enough to iterate the smaller set and check membership in the larger. Difference is $O(\text{len}(a))$. Symmetric difference is $O(\text{len}(a) + \text{len}(b))$. 3. Note the method vs operator distinction. `a | b` requires both to be sets; `a.union(b)` accepts any iterable. Same pattern for the rest. This matters when you want to union a set with a list without materialising the list as a set. 4. Mention the in-place forms. `a |= b` adds members of `b` to `a` and is faster than `a = a | b` because it doesn't allocate a new set.

Edge cases to mention: - `set` operations require both sides to be sets when using operators; methods accept any iterable. `{1, 2} | [3]` is `TypeError`; `{1, 2}.union([3])` is `{1, 2, 3}`. - `a.isdisjoint(b)` is $O(\min(\text{len}(a), \text{len}(b)))$ and short-circuits on first overlap — faster than `bool(a & b)`. - Subset and superset tests — `a <= b`, `a >= b` — are $O(\text{len}(a))$ and $O(\text{len}(b))$ respectively.

What NOT to say: - "Set operations are all $O(n*m)$ " — wrong; intersection is $O(\min)$. - "Use `+` for union" — wrong; that's list. Sets use `|`.

Common follow-up: "How would you find elements unique to one of two large lists efficiently?" — Convert both to sets, take symmetric difference: `set(a) ^ set(b)`. $O(n + m)$ total.

```
admins = {"raj", "priya", "amit"}
on_call = {"priya", "amit", "neha"}

print(admins | on_call)    # {'raj', 'priya', 'amit', 'neha'}
print(admins & on_call)    # {'priya', 'amit'}
print(admins - on_call)    # {'raj'}
print(admins ^ on_call)    # {'raj', 'neha'}
print(admins.isdisjoint({"karan", "vikram"}))  # True — no overlap
```

Q23 · What's the difference between `set` and `frozenset` ?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Postman, Microsoft India, Cred · Topic: hashability

The question (interviewer's verbatim phrasing):

When would you pick `frozenset` over `set` ?

The 30-second answer: `set` is mutable; `frozenset` is immutable. The practical difference: `frozenset` is hashable, so it can be a dict key or an element of another set; `set` cannot. Pick `frozenset` when you need immutability, hashability, or when the collection genuinely won't change after construction — the same reasoning as tuple-over-list for hashable collections.

Step-by-step thought process: 1. State the mutability split. `set` supports `.add()`, `.remove()`, `.update()`, etc.; `frozenset` has none of those — any attempt raises `AttributeError`. 2. State the hashability consequence. `frozenset` implements `__hash__` based on its members (which must themselves be hashable). `set` deliberately raises `TypeError` on `hash(s)` because mutating a set after using it as a key would silently break dicts. 3. Walk the use case. Composite keys for caches and lookup tables — like `{frozenset(permissions): role_name}` — work because `frozenset` is hashable. Same for sets-of-sets: each inner set must be a `frozenset`. 4. Performance note. Both have the same $O(1)$ average membership cost. Construction cost is the same. The only meaningful run-time difference is the absence of mutation methods.

Edge cases to mention: - `frozenset()` is the empty `frozenset`; there's no `{}` literal for it because `{}` is an empty dict. - A `frozenset` of `frozensets` is fine: `frozenset({frozenset({1, 2})})` works; you can't do the same with regular sets. - Set operations between `set` and `frozenset` are allowed; the return type follows the left operand: `set() | frozenset()` returns a `set`; `frozenset() | set()` returns a `frozenset`.

What NOT to say: - "frozenset is just a faster set" — wrong; same operations, same Big-O. The difference is immutability and hashability. - "frozenset is rarely useful" — wrong; whenever you need a hashable set, it's the only option.

Common follow-up: "How do you hash a list of strings?" — Convert to tuple (order-preserving) or `frozenset` (order-independent), depending on whether the original order matters for equality.

`hash(tuple(my_list))` or `hash(frozenset(my_list))`.

```
permissions = frozenset({"read", "write"})
role_for = {
    frozenset({"read"}): "viewer",
    frozenset({"read", "write"}): "editor",
    frozenset({"read", "write", "delete"}): "admin",
}
print(role_for[permissions])  # 'editor'

s = {1, 2}
fs = frozenset({1, 2})
# {s}                # TypeError: unhashable type: 'set'
print({fs})          # {frozenset({1, 2})}
```

Q24 · Why must set elements be hashable?

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft India, Postman, Goldman, Flipkart · Topic: hashability

The question (interviewer's verbatim phrasing):

A set stores values, not key-value pairs, so why does it still need its elements to be hashable?

The 30-second answer: A set is built on the same hash-table machinery as dict. To check whether `x in s` in $O(1)$, the set hashes `x` to find the right slot. Without a hash, it would have to walk every element — at which point you've reinvented `list`. The hashability requirement is the price of constant-time membership.

Step-by-step thought process: 1. Anchor on the implementation. A set is a hash table where every "value" is also the "key" — there's no separate value array, just an indices array and an entries array of unique elements. 2. Walk through the consequence. `x in s` computes `hash(x)`, finds the slot, compares with `__eq__`. No hash, no slot lookup, no $O(1)$. 3. Mention the stability requirement. The hash must be stable across the object's lifetime; if a stored element's hash changed, the set would silently lose it. This is why mutable built-ins (list, dict, set) don't implement `__hash__`. 4. The workaround. Wrap mutable collections in tuple or frozenset to get a hashable equivalent. For custom classes, implement `__hash__` to return a hash of the immutable fields only.

Edge cases to mention: - A class can inherit `__hash__` and `__eq__` from `object` (identity-based) and still go in a set — the set ends up keyed on object identity, which is sometimes what you want. - Implementing `__eq__` without `__hash__` automatically sets `__hash__` to `None` — that's a common surprise that makes instances unhashable. - A `NaN` value is hashable (it's a float), but it's not equal to itself, so it has weird set behaviour: you can add multiple NaNs and `nan in s` may be `False` even after adding it.

What NOT to say: - "Set elements don't need to be hashable" — false; the language enforces it. - "Hashability is about uniqueness" — uniqueness is enforced by `__eq__`; hashability is about finding the slot.

Common follow-up: "What's the rule about `__hash__` and `__eq__` consistency?" — If two objects compare equal with `__eq__`, they must have equal hashes. The reverse is not required — hash collisions are fine. Violating the equal-hash rule breaks set and dict silently.

```
class User:
    def __init__(self, id, name):
        self.id, self.name = id, name
    def __eq__(self, other):
        return isinstance(other, User) and self.id == other.id
    def __hash__(self):
        return hash(self.id) # hash only the immutable identity field

s = {User(1, "Aarav"), User(1, "Aarav (renamed)")}
print(len(s)) # 1 - same id collapses
```

Q25 · When would a list of unique items be better than a set?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Flipkart, Microsoft India, Swiggy · Topic: set vs list

The question (interviewer's verbatim phrasing):

If I need a collection of unique items, is `set` always the right answer?

The 30-second answer: No. Pick a list when you need order-by-index access, when the items aren't hashable, when memory matters and the count is small, or when the collection's purpose is iteration rather than membership testing. The set advantage is $O(1)$ membership; if you're never going to ask "is x in here?", the set isn't paying for itself.

Step-by-step thought process: 1. State the rule of thumb. Set wins when membership testing dominates; list wins when iteration or indexed access dominates. 2. Walk through the indexing case. Sets don't support `s[i]`. If you need "the third item I added", that's a list (or a dict keyed by index, or `tuple`). 3. Walk through the hashability case. If the items can't be hashed (e.g., dicts, lists of lists with mutable inner items, NumPy arrays), the set isn't an option — list is the only built-in choice. Or convert items to a hashable form. 4. Memory note. For very small collections (≤ 5 items), the constant overhead of a set's hash table exceeds the linear scan cost of list `in`. The crossover is empirical — measure if it matters.

Edge cases to mention: - For ordered-unique semantics, `dict.fromkeys(iterable)` gives you a dict that's an ordered set — keys preserve insertion order in 3.7+. - `tuple` is the immutable-ordered counterpart; sometimes the right answer is "tuple, with dedup if needed". - If you need sorted unique, `sorted(set(items))` is $O(n \log n)$ — efficient enough for most workloads.

What NOT to say: - "Set is always faster" — wrong for iteration; the speedup is membership-specific. - "Set preserves order" — wrong; only `dict` and `OrderedDict` do.

Common follow-up: "How do you dedupe a list while preserving order?" —

`list(dict.fromkeys(items))`. The dict drops duplicates and keeps insertion order; the list() materialises it.

```
# dedup preserving order
items = ["apple", "banana", "apple", "cherry", "banana"]
unique = list(dict.fromkeys(items))  # ['apple', 'banana', 'cherry']

# indexed access — set won't help
events = ["login", "view_cart", "checkout"]
first = events[0]  # 'login' — set has no [0]
```

Section D — tuple and namedtuple

Five questions on Python's immutable sequence and its three named-field cousins

(`collections.namedtuple`, `typing.NamedTuple`, and `dataclasses.dataclass`). The interview hook is immutability — why it matters for hashability, for safe sharing across threads, for fixed-shape records — and the three flavours of named tuple, each with a different trade-off.

Q26 · What's the difference between a tuple and a list, beyond mutability?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Flipkart, Postman · Topic: tuple basics

The question (interviewer's verbatim phrasing):

Apart from "tuple is immutable", what are the real differences between tuple and list?

The 30-second answer: Three real differences. One, hashability — tuples (of hashable items) can be dict keys and set elements; lists cannot. Two, semantic convention — tuples are for fixed-shape heterogeneous records (a row of data), lists are for variable-length homogeneous sequences. Three, memory and speed — tuples are slightly smaller and slightly faster to construct because CPython knows the size up front and doesn't carry capacity slack.

Step-by-step thought process: 1. Open with hashability — the only behavioural difference that affects what you can do, not just how it performs. Tuples can be dict keys; lists can't. 2. Walk the semantic convention. The Python idiom is tuple-for-record, list-for-sequence. `(x, y)` is a 2D point; `[x, y, z, w]` is "all my points so far". Style guides and type hints both lean into this convention. 3. Mention the memory and speed gap. A tuple of n items is ~40 + 8n bytes flat; a list with n items is ~56 + 8 * capacity bytes (capacity > n). Tuple construction is faster because there's no over-allocation logic to

run. 4. Note the named-tuple variants. If you find yourself indexing tuples by position like `point[0]`, that's a code-smell that says "use `namedtuple` or `dataclass` for named field access".

Edge cases to mention: - A tuple containing a mutable element (like a list) is still a tuple — you can't change which list it points to, but you can mutate the list itself. So "immutable tuple" is shallow. - A single-element tuple needs the trailing comma: `(1,)` is a tuple, `(1)` is just `1` in parentheses. - `tuple` is iterable, indexable, and sliceable — but slicing produces a new tuple, same as list slicing.

What NOT to say: - "Tuple is just an immutable list" — undersells the convention and the hashability win. - "Tuple is always faster" — small differences only; pick by semantics, not performance.

Common follow-up: "When would you use a list of tuples vs a dict?" — List of tuples is iteration-friendly and order-preserving for sequential records (like CSV rows). Dict is for keyed lookup. Convert one to the other based on whether you'll be iterating or looking up.

```
# tuple as record – heterogeneous
user = ("Aarav", 25, "Mumbai")
name, age, city = user           # unpacking

# tuple as dict key – composite identity
prices = {"INR", "small": 99, ("INR", "large"): 199}
print(prices[("INR", "small")])

# single-element gotcha
print(type((1)))                 # <class 'int'> – not a tuple
print(type((1,)))                # <class 'tuple'>
```

Q27 · What's the difference between `collections.namedtuple`, `typing.NamedTuple`, and `dataclasses.dataclass`?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Flipkart, Cred · Topic: record types

The question (interviewer's verbatim phrasing):

Python has three ways to make a record-like class — `namedtuple`, `NamedTuple`, and `dataclass`. When do you use which?

The 30-second answer: `collections.namedtuple` is the old factory-function form — immutable, tuple-based, iterable, indexable, fast. `typing.NamedTuple` is the class-syntax version with type annotations — same runtime behaviour, nicer syntax for typed code. `dataclass` is the mutable record class with no tuple backing — supports defaults, methods, inheritance, `__eq__` / `__repr__` for free; use it when you want a struct that may grow methods. Rule of thumb: `dataclass` for typical app

code, `NamedTuple` when you want immutability and unpacking, `namedtuple` only for legacy compatibility.

Step-by-step thought process: 1. Tabulate the three. `namedtuple` → tuple subclass, immutable, iterable. `NamedTuple` → same runtime, class syntax with annotations. `dataclass` → plain class, mutable by default, no tuple parent. 2. Walk the immutability axis. `namedtuple` and `NamedTuple` are immutable; trying to assign `obj.field = x` raises `AttributeError`. `dataclass` is mutable by default; pass `frozen=True` to lock it. 3. Walk the unpacking axis. `namedtuple` and `NamedTuple` support `a, b = obj` because they're tuples. `dataclass` does not — you'd have to define `__iter__` manually or unpack via attribute access. 4. Walk the feature axis. `dataclass` supports default values, default factories, post-init, inheritance, comparison generation — the fullest API. `NamedTuple` is in the middle. `namedtuple` is the minimum.

Edge cases to mention: - `dataclass(frozen=True, slots=True)` in 3.10+ gives you a small, immutable, hashable record with no `__dict__` — competitive with `NamedTuple` on memory. - All three have the same iteration over fields via `dataclasses.fields()` or the field-name attribute. - Performance: `namedtuple` instantiation is slightly faster than `dataclass` instantiation; field access is identical.

What NOT to say: - "namedtuple is deprecated" — wrong; still fully supported. - "Use dataclass for everything" — wrong; if you want unpacking and immutability, `NamedTuple` is cleaner.

Common follow-up: "Can a dataclass be a dict key?" — Only if it's frozen (`@dataclass(frozen=True)`) — that's when `__hash__` is generated. Mutable dataclasses are unhashable by default for the same reason mutable lists are.


```

from collections import namedtuple
from typing import NamedTuple
from dataclasses import dataclass

# factory form
Point1 = namedtuple("Point1", ["x", "y"])

# class form with annotations
class Point2(NamedTuple):
    x: int
    y: int

# dataclass – most flexible
@dataclass(frozen=True, slots=True)
class Point3:
    x: int
    y: int

p1, p2, p3 = Point1(1, 2), Point2(1, 2), Point3(1, 2)
x, y = p1           # tuple unpacking
x, y = p2           # tuple unpacking
# x, y = p3         # TypeError – dataclass is not iterable
print(hash(p3))     # works because frozen=True

```

Q28 · What are the common tuple-unpacking idioms?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Postman, TCS, Swiggy · Topic: tuple idioms

The question (interviewer's verbatim phrasing):

What are the unpacking patterns you reach for with tuples?

The 30-second answer: Four idioms cover 90% of cases. Basic: `a, b = pair`. Star: `head, *tail = lst` captures the rest in `tail`. Swap: `a, b = b, a` swaps without a temp variable. Multiple return: `def f(): return x, y; result = f(); a, b = result` — the standard Python way to return multiple values. All four work on any iterable, not just tuples.

Step-by-step thought process: 1. Open with the basic form. `a, b = (1, 2)` binds `a = 1` and `b = 2`. The right side can be any iterable of the right length. 2. Show the swap idiom. `a, b = b, a` builds a tuple `(b, a)` on the right and unpacks it on the left — no temporary variable needed, no XOR trickery. This is the canonical swap in Python. 3. Walk the star form (PEP 3132). `first, *rest = [1, 2, 3, 4]` gives `first = 1` and `rest = [2, 3, 4]`. Useful for "head and the rest" or "init and last" (`*init, last = lst`). Star is always a list, even if the original was a tuple. 4. Mention nested

unpacking. `(a, b), c = ((1, 2), 3)` works — Python recurses into nested structure as long as shapes match.

Edge cases to mention: - Length mismatch raises `ValueError`: "too many values to unpack" or "not enough values to unpack". Use star to be flexible. - `_` is conventional for "I don't care about this slot": `name, _, age = ("Aarav", "M", 25)`. Just a regular variable named `_`, no language magic. - Star can appear only once on the left side: `a, *b, *c = lst` is a `SyntaxError`.

What NOT to say: - "Tuple unpacking is slow" — it's a single bytecode operation, faster than indexed access. - "Star only works with tuples" — wrong; works with any iterable.

Common follow-up: "How do you swap three variables in one line?" — `a, b, c = c, a, b` — same idiom. Python builds the right-side tuple once and unpacks it.

```
# basic
x, y = (10, 20)

# swap
a, b = 1, 2
a, b = b, a           # a=2, b=1

# star
first, *middle, last = [1, 2, 3, 4, 5]
# first=1, middle=[2, 3, 4], last=5

# multiple return
def divmod_named(a, b):
    return a // b, a % b
q, r = divmod_named(17, 5)
```

Q29 · Why does `(1)` not give you a tuple?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Flipkart, Cred · Topic: tuple syntax

The question (interviewer's verbatim phrasing):

What's the difference between `(1)`, `(1,)`, and `1`, ? Which ones are tuples?

The 30-second answer: The tuple is created by the comma, not by the parentheses. `(1)` is just the integer `1` inside parentheses — grouping, not a tuple. `(1,)` is a one-element tuple. `1`, (no parentheses at all) is also a one-element tuple. The empty tuple is the one exception: `()` is a tuple because there's no value to be ambiguous about.

Step-by-step thought process: 1. State the core rule. "The comma makes the tuple." Parentheses are grouping syntax; they don't construct anything. 2. Walk the three cases. `(1)` parses as `1`, no tuple. `(1,)` parses as `1` followed by a comma, making it a tuple of one element. `1,` is the same with implicit grouping — Python sees a comma in expression position and builds a tuple. 3. Note the multi-element case. `1, 2, 3` is a tuple. `(1, 2, 3)` is the same tuple, just with explicit parentheses for clarity. Either is fine; the parentheses are stylistic except in some grammar contexts (function args, comprehensions). 4. The empty-tuple exception. `()` is a tuple because there's no integer to be confused with. `tuple()` also works and is equivalent.

Edge cases to mention: - Function call vs tuple in args: `f(1)` calls `f` with arg 1; `f((1,))` calls `f` with a one-tuple. The comma resolves the ambiguity. - `return 1, 2` returns a tuple `(1, 2)`. `return (1, 2)` is the same thing; parens are stylistic. - A trailing comma in `(1, 2, 3,)` is allowed and stylistic — does not add a phantom element.

What NOT to say: - "Parentheses make a tuple" — false; the comma does. - "Empty tuple is `(,)`" — wrong; `(,)` is a `SyntaxError`. Empty tuple is `()`.

Common follow-up: "What does `x, = some_iterable` mean?" — That's the single-element-unpack form. It binds `x` to the only element and raises `ValueError` if the iterable has zero or two-plus elements. Useful for "I expect exactly one result, fail loudly if not".

```
print(type((1)))      # <class 'int'>
print(type((1,)))     # <class 'tuple'>
print(type(1,))       # <class 'tuple'>
print(type(()))       # <class 'tuple'>

# single-element unpack
[only] = ["a"]         # only = "a"
only, = ("a",)         # also only = "a"
# only, = ("a", "b")   # ValueError – too many values
```

Q30 · Are tuples really immutable?

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft India, Postman, Flipkart, Goldman · Topic: tuple internals

The question (interviewer's verbatim phrasing):

A tuple is immutable, right? Then why does `t = ([1], [2]); t[0].append(99)` work?

The 30-second answer: The tuple itself is immutable — you can't change which objects its slots point to. But if a slot points to a mutable object (like a list), you can still mutate that object through the pointer. So tuple immutability is shallow, not deep. The contained objects' mutability is unchanged.

Step-by-step thought process: 1. Define what "tuple is immutable" means precisely. The tuple's `ob_item` array — the pointers to contained objects — is fixed at construction. You cannot reassign `t[0] = x` or call `t.append(x)`. Those raise `TypeError`. 2. Walk the example. `t = ([1], [2])` constructs a tuple whose slots point to two list objects. `t[0].append(99)` accesses the first slot's list and calls `.append` on it. The tuple's pointer is unchanged; the list it points to grew. 3. Generalise. Containers that are immutable at the top level (tuple, frozenset, str — well, str is fully immutable) carry no opinion about whether their elements are mutable. The seal is on the container's shape, not on the contents. 4. Implication for hashability. A tuple of (hashable) elements is hashable; a tuple containing a list is NOT hashable, even though the tuple itself is technically immutable, because the contained list's value-equality is not stable.

Edge cases to mention: - `tuple.__hash__` raises `TypeError` if any element is unhashable — you find out at `hash(t)` time, not at construction. - `(1, 2) + (3,)` creates a new tuple — the original is untouched. Concatenation is allocation, not mutation. - For "really immutable" semantics including contents, you'd recursively freeze: convert lists to tuples, sets to frozensets. There's no built-in `freeze()` function.

What NOT to say: - "Tuples are mutable if they contain a list" — wrong framing; the tuple is immutable, the list is mutable. - "Lists inside tuples are illegal" — they're legal but the result isn't hashable.

Common follow-up: "What's the hash of `(1, [2, 3])`?" — `TypeError: unhashable type: 'list'`. Tuples compute hash from their elements; one unhashable element poisons the whole tuple.

```
t = ([1, 2], [3, 4])
t[0].append(99)           # OK — mutates the contained list
print(t)                  # ([1, 2, 99], [3, 4])
# t[0] = [9]              # TypeError — tuple immutable
# hash(t)                 # TypeError — list inside is unhashable

# fully immutable equivalent
ft = (tuple([1, 2]), tuple([3, 4]))
print(hash(ft))           # works
```

Section E — collections Module

Ten questions on the standard-library container toolkit: `Counter`, `defaultdict`, `deque`, `OrderedDict`, `ChainMap`, `UserDict` and `UserList`. These are the "you'd write it in five lines but the stdlib already has it" types. Knowing them by name and knowing exactly when to reach for each is one of the higher-signal differentiators between a candidate who has read the docs and one who has only written Python.

Q31 · What is `collections.Counter` and what makes it different from a plain dict?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Swiggy, Postman, TCS · Topic: Counter

The question (interviewer's verbatim phrasing):

What does `collections.Counter` give you that a regular dict doesn't?

The 30-second answer: `Counter` is a dict subclass specialised for counting hashable items. Five extras you don't get from a plain dict: (1) missing keys return 0 instead of raising `KeyError`; (2) `.most_common(n)` returns the top-n pairs sorted by count; (3) `.update(iterable)` adds counts instead of overwriting; (4) arithmetic operators (`+`, `-`, `&`, `|`) work between two Counters; (5) constructor accepts an iterable directly: `Counter("abracadabra")`.

Step-by-step thought process: 1. Anchor on "dict subclass". `Counter` is a dict — it has all dict methods — plus the extras above. Performance characteristics are the same as dict. 2. Walk the missing-key behaviour. `c["never_seen"]` returns 0, not `KeyError`. This is what makes `c[word] += 1` work without an initial check. 3. Walk `most_common`. Implemented on top of `heapq.nlargest` — $O(n \log k)$ where k is the requested count, not $O(n \log n)$. So `c.most_common(10)` on a million-key Counter is fast. 4. Walk the arithmetic. `Counter(a) + Counter(b)` sums per-key counts; `-` subtracts and drops non-positive results; `&` is min per key (multiset intersection); `|` is max per key (multiset union). Behaviour matches the multiset algebra you'd expect.

Edge cases to mention: - `c["x"] += 1` works because of the 0 default; but `c["x"]` alone doesn't insert the key — only the assignment does. `c.get("x", 0)` is equivalent for read-only access. - Counter subtraction drops zero and negative counts: `Counter({"a": 3}) - Counter({"a": 5})` is `Counter()`, not `Counter({"a": -2})`. For raw signed arithmetic, use `Counter.subtract` which keeps negatives. - `Counter.elements()` returns an iterator that repeats each key by its count — useful for "expand back to a list".

What NOT to say: - "Counter is the same as dict" — wrong; the missing-key behaviour and the algebra are language-level differences. - "Counter is slow" — same Big-O as dict; the convenience is free.

Common follow-up: "How would you find the most common word in a million-line file?" — `Counter(line.split() for line in file).most_common(1)`. Or more idiomatically with `chain.from_iterable` to flatten. $O(n)$ construction, $O(n \log 1) = O(n)$ for the top-1.

```

from collections import Counter
votes = ["yes", "no", "yes", "abstain", "yes", "no"]
c = Counter(votes)
print(c)                                # Counter({'yes': 3, 'no': 2, 'abstain': 1})
print(c.most_common(2))                 # [('yes', 3), ('no', 2)]
print(c["never_voted"])                 # 0 — no KeyError
print(Counter("aab") + Counter("abc"))  # Counter({'a': 3, 'b': 2, 'c': 1})

```

Q32 · What is `collections.defaultdict` and how does it differ from `dict.setdefault` ?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Postman, Microsoft India, TCS · Topic: defaultdict

The question (interviewer's verbatim phrasing):

What does `defaultdict` do, and when do you pick it over `dict.setdefault` ?

The 30-second answer: `defaultdict(factory)` is a dict subclass that automatically calls `factory()` to create a value when you access a missing key. Pick it when you'll be doing many missing-key initialisations with the same factory — say, building a dict-of-lists. `dict.setdefault(k, val)` does the same job per-call and works on regular dicts, but it always evaluates the default expression, even if the key exists. Use defaultdict for bulk patterns; use setdefault for one-off cases.

Step-by-step thought process: 1. State the constructor. `defaultdict(callable)` — the callable is invoked with zero arguments to produce the default for any missing key. 2. Walk the dict-of-lists pattern. With plain dict: `d.setdefault(key, []).append(item)` for every line. With defaultdict: `d = defaultdict(list); d[key].append(item)`. Same effect; defaultdict is cleaner when the pattern repeats. 3. Note the lazy-vs-eager distinction. `setdefault(k, expensive_call())` evaluates `expensive_call()` every time, even when the key exists, because Python evaluates positional arguments before the method runs. defaultdict's factory runs only when the key is actually missing. 4. Mention nesting. `defaultdict(lambda: defaultdict(int))` gives you a two-level dict where deep access just works: `d["india"]["mumbai"] += 1` with no setup.

Edge cases to mention: - Accessing a missing key (`d[k]`) inserts the default into the dict; this can surprise you if you're "just checking". Use `k in d` or `d.get(k)` for non-mutating reads. - The factory receives no arguments — it cannot see the key being looked up. If you need a key-aware default, subclass `dict` and override `__missing__`. - `defaultdict.__repr__` shows the factory and current contents, which makes debugging easier than reading a plain dict.

What NOT to say: - "defaultdict is faster than dict" — no; same Big-O, just less boilerplate. - "setdefault is deprecated" — wrong; it remains the right tool for varying-factory or single-call cases.

Common follow-up: "What's the difference between `defaultdict(list)` and `defaultdict([])`?" — Big one. `defaultdict(list)` passes the `list` type as the factory; calling `list()` returns a fresh empty list each time. `defaultdict([])` would pass an existing empty list — but the list isn't callable, so this raises `TypeError`. Always pass the type, not an instance.

```
from collections import defaultdict

# dict-of-lists pattern
events = [("login", 1), ("login", 2), ("logout", 3)]
by_type = defaultdict(list)
for kind, uid in events:
    by_type[kind].append(uid)
# {'login': [1, 2], 'logout': [3]}

# nested defaultdict
counts = defaultdict(lambda: defaultdict(int))
counts["india"]["mumbai"] += 1
counts["india"]["delhi"] += 1
# {'india': {'mumbai': 1, 'delhi': 1}}
```

Q33 · What is `collections.deque` and what are its operation costs?

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Microsoft India, Postman, Swiggy · Topic: deque

The question (interviewer's verbatim phrasing):

What's a deque, and what are the time complexities of its main operations?

The 30-second answer: `deque` is a doubly-linked list of fixed-size blocks — typically 64 pointers per block. Append and pop at either end are $O(1)$. Random indexing in the middle is $O(n)$ because you have to walk block links. Use deque for queues, stacks-at-both-ends, sliding windows, and BFS frontiers — anywhere you need $O(1)$ at both ends. Do NOT use it where you'd index into the middle frequently.

Step-by-step thought process: 1. State the structure. Block-list of contiguous arrays; head and tail pointers can move independently. Conceptually like a ring buffer that grows on demand. 2. Walk the four $O(1)$ operations: `append`, `appendleft`, `pop`, `popleft`. These are the deque's reason for existing — list can't do the "left" variants in $O(1)$. 3. Walk the $O(n)$ operations: `deque[k]` for middle indices, `deque.insert(k, x)`, `deque.remove(value)`, `in` for membership. These are the operations where you should pick a different container. 4. Mention `maxlen`. `deque(maxlen=N)` evicts from the opposite end when adding to a full deque — perfect for sliding windows or recent-items buffers. No manual size management.

Edge cases to mention: - `deque.rotate(n)` rotates by n positions in $O(|n|)$ — useful for round-robin scheduling. - Deque is thread-safe for append/pop on either end at the GIL level — no extra lock needed for the common queue pattern. (For full thread safety on `+`, `+=`, etc., use `queue.Queue`.) - A deque of one million ints is roughly the same memory as a list because both store pointers; the block overhead is amortised.

What NOT to say: - "Deque is faster than list for everything" — wrong; random indexing and slicing are slower. - "Deque is a stack" — partially right; it's stack + queue + double-ended.

Common follow-up: "When would you pick deque over queue.Queue?" — Deque is a data structure; `queue.Queue` is a thread-safe blocking queue with `put` / `get` semantics and is much slower per op. For single-thread or GIL-protected use, deque wins. For producer-consumer with blocking, use `queue.Queue` or `asyncio.Queue`.

```
from collections import deque

# queue
q = deque()
q.append(1); q.append(2); q.append(3)
print(q.popleft())      # 1 — FIFO

# stack
s = deque()
s.append(1); s.append(2); s.append(3)
print(s.pop())          # 3 — LIFO

# sliding window
window = deque(maxlen=3)
for x in [1, 2, 3, 4, 5]:
    window.append(x)
    print(list(window))  # [1], [1, 2], [1, 2, 3], [2, 3, 4], [3, 4, 5]
```

Q34 · Is `OrderedDict` still useful now that plain `dict` preserves insertion order?

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Postman, Microsoft India, Cred · Topic: OrderedDict

The question (interviewer's verbatim phrasing):

Plain dict has preserved insertion order since 3.7. So is `OrderedDict` obsolete?

The 30-second answer: No, but its niche is narrow now. Three things `OrderedDict` still does that plain dict doesn't: (1) `move_to_end(key, last=True/False)` to push a key to either end in $O(1)$

— used for LRU caches; (2) `popitem(last=True/False)` with an argument to pop from either end; (3) order-aware equality — `OrderedDict([("a",1),("b",2)]) != OrderedDict([("b",2),("a",1)])` while the plain-dict equivalent is equal. For ordering alone, plain dict is enough.

Step-by-step thought process: 1. Acknowledge the 3.7 change. Insertion order is a language guarantee for dict; `OrderedDict` is no longer needed just for ordering. 2. Walk the three remaining capabilities.

`move_to_end` is the showstopper for LRU cache implementations — you push the recently-used key to the end on every access, and the front of the dict is automatically the oldest.

`popitem(last=False)` then pops the oldest in $O(1)$. 3. Walk the equality difference. Plain dict equality compares contents only, not order. `OrderedDict` equality compares both. If you're modelling an ordered config or playlist where reordering matters semantically, `OrderedDict`'s equality reflects that. 4. Note the historical reason it stayed. Removing `OrderedDict` would have broken tons of code from the pre-3.7 era. Python keeps it for those use cases plus the three above.

Edge cases to mention: - `OrderedDict.move_to_end(key, last=False)` pushes the key to the *front* — useful for "most recent on top" patterns. - `OrderedDict` has slightly more memory overhead than plain dict (extra linked-list pointers for the order). For pure read-many-write-rarely dicts, prefer plain dict. - `functools.lru_cache` is the standard answer for LRU caching at the function-call level; `OrderedDict` is for when you need the LRU mechanics on application data, not function arguments.

What NOT to say: - "OrderedDict is deprecated" — wrong; still in the standard library, still has unique capabilities. - "OrderedDict is just plain dict now" — wrong; the three differences are real.

Common follow-up: "How would you build a manual LRU cache?" — `OrderedDict` is the textbook answer. On get, `move_to_end(key)`. On set, insert and then `popitem(last=False)` if size exceeds the limit. Both ops are $O(1)$.

```

from collections import OrderedDict

class LRU:
    def __init__(self, capacity):
        self.cap = capacity
        self.d = OrderedDict()

    def get(self, k):
        if k not in self.d: return None
        self.d.move_to_end(k)      # mark as recently used
        return self.d[k]

    def put(self, k, v):
        if k in self.d: self.d.move_to_end(k)
        self.d[k] = v
        if len(self.d) > self.cap:
            self.d.popitem(last=False) # evict oldest

cache = LRU(2)
cache.put("a", 1); cache.put("b", 2); cache.put("c", 3)
print("a" in cache.d)    # False — evicted

```

Q35 · What does `Counter.update()` do, and how is it different from `dict.update()` ?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Flipkart, Cred · Topic: Counter

The question (interviewer's verbatim phrasing):

If I have a Counter and I call `.update()` , what happens? Is it the same as dict's update?

The 30-second answer: No — Counter overrides update to *add* counts instead of *overwrite* values.

`c.update(["a", "b", "a"])` increments counts by counting the iterable. `c.update({"a": 5})` adds 5 to the existing count for "a". Plain dict's update replaces values: `d.update({"a": 5})` sets `d["a"] = 5` regardless of the prior value. The semantics match what each container is for — Counter sums, dict assigns.

Step-by-step thought process: 1. State the override behaviour. Counter inherits from dict but customises `update` to support the counting use case. 2. Walk the two call shapes. With an iterable, Counter counts the elements and adds. With a mapping, Counter takes each (key, count) pair and adds the count to the existing entry. 3. Contrast with dict. `dict.update` replaces; `Counter.update` adds. Both accept mappings or iterables of pairs, but the semantics differ at the per-key level. 4. Mention the inverse. `Counter.subtract(other)` does the corresponding subtraction — useful for "remove these counts" without going through arithmetic operators that drop negatives.

Edge cases to mention: - `Counter().update("abracadabra")` works because strings are iterables of characters — each char counted once per appearance. - `Counter.update` accepts kwargs too: `c.update(a=5, b=2)` adds 5 to "a" and 2 to "b". - For raw replacement on a Counter — overwriting like a plain dict — use direct assignment: `c["a"] = 5`. Or call `dict.update(c, ...)` to bypass the override.

What NOT to say: - "Counter.update is the same as dict.update" — wrong; the override matters. - "Counter is a thin wrapper" — undersells it; the algebra and update overrides are first-class.

Common follow-up: "What does `c.subtract(other)` do?" — Subtracts counts per key, allowing negative results. Different from the `-` operator which drops non-positives. Use `subtract` when you want signed deltas; use `-` when you want a clean multiset difference.

```
from collections import Counter
c = Counter("aabbcc")          # Counter({'a': 2, 'b': 2, 'c': 1})
c.update("abc")
print(c)                      # Counter({'a': 3, 'b': 3, 'c': 2})

c.update({'a': 10})
print(c["a"])                 # 13 — added, not replaced

c.subtract({'a': 20})
print(c["a"])                 # -7 — negatives allowed via subtract
```

Q36 · What is `collections.ChainMap` and when would you use it?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Flipkart, Goldman · Topic: ChainMap

The question (interviewer's verbatim phrasing):

What does ChainMap do, and when is it the right tool?

The 30-second answer: `ChainMap(d1, d2, d3, ...)` provides a single mapping view over multiple dicts, searched in order. Lookups walk left to right and return the first match; writes go to the first dict only. The classic use is layered configuration — runtime args > env vars > defaults — where you want "the effective value" without merging all three into a flat dict.

Step-by-step thought process: 1. State the structure. ChainMap holds a list of source mappings (the `.maps` attribute) and presents them as one. No copy; the sources are kept by reference. 2. Walk the lookup. `cm[key]` walks the maps in order, returning the first match. Misses raise `KeyError` after walking all. 3. Walk the write path. `cm[key] = value` writes to `cm.maps[0]` — the first (highest-priority) dict. Other maps are read-only via the ChainMap. 4. Show the config-layer pattern.

`ChainMap(cli_args, env, defaults)`. Lookups respect priority; updating a value via the `ChainMap` only affects the first layer, leaving `env` and `defaults` untouched.

Edge cases to mention: - `cm.new_child()` returns a new `ChainMap` with an empty dict prepended — useful for "scoped overrides" that don't pollute the original. - `cm.parents` returns a `ChainMap` excluding the first map — the "lookup as if my overrides weren't here" view. - Mutations to the underlying source dicts are visible immediately — `ChainMap` is a live view, not a snapshot.

What NOT to say: - "ChainMap is the same as `d1 | d2`" — wrong; merge creates a copy, `ChainMap` is a live view. - "ChainMap is rarely useful" — config layering is a real use case that shows up in CLIs and frameworks.

Common follow-up: "What's the difference between `ChainMap(d1, d2)` and `{**d2, **d1}`?" — `ChainMap` is a view (changes to `d1` or `d2` are visible; writes go to `d1` only). `{**d2, **d1}` is a snapshot copy at the moment of construction; subsequent changes to `d1` or `d2` don't affect it.

```
from collections import ChainMap

defaults = {"timeout": 30, "retries": 3, "verbose": False}
env       = {"timeout": 60}
cli       = {"verbose": True}

config = ChainMap(cli, env, defaults)
print(config["timeout"])    # 60 — from env
print(config["verbose"])    # True — from cli
print(config["retries"])    # 3 — from defaults

config["retries"] = 5        # writes to cli only
print(cli)                 # {'verbose': True, 'retries': 5}
print(defaults["retries"])  # 3 — untouched
```

Q37 · What does `defaultdict(list)` look like in memory after many missing-key accesses?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Flipkart, Cred · Topic: `defaultdict` gotchas

The question (interviewer's verbatim phrasing):

If I write `d = defaultdict(list); d[k]` for many keys `k` without appending anything, what's in the dict?

The 30-second answer: Every key you accessed is in the dict, mapped to an empty list. `defaultdict` inserts the default into the dict on a missing-key access — not just for the assignment shape. This is the

"I was just checking" surprise: reading from defaultdict mutates it. Use `k in d` or `d.get(k)` if you want a non-mutating read.

Step-by-step thought process: 1. State the behaviour bluntly. `__getitem__` on a missing key calls `__missing__` which calls the factory, inserts the result, and returns it. The "insert" is the part people forget. 2. Walk the implication. A logging loop that does `if d[key]: ...` against a defaultdict inserts every key it tests, even when the test was false. The defaultdict grows unbounded. 3. State the workarounds. `key in d` returns True/False without insertion. `d.get(key)` returns None (or your specified default) without insertion. `d.get(key, [])` is the "tolerant non-mutating read with default" pattern. 4. Mention the performance angle. The unwanted insertions cost a hash and a default-factory call each. In tight loops this can dominate.

Edge cases to mention: - `defaultdict.pop(k, default)` works like `dict.pop` — no insertion if the key is missing. - For "I just want to look it up safely", consider plain dict + `.get` rather than defaultdict. defaultdict is for *bulk insert-on-missing*, not for safer reads. - If you've polluted a defaultdict with empty reads, you can clean it: `{k: v for k, v in d.items() if v}`.

What NOT to say: - "defaultdict only inserts on write" — false; it inserts on any missing-key access. - "defaultdict is the same as dict.setdefault" — semantically similar for writes; defaultdict differs by inserting on read too.

Common follow-up: "How do you check membership without polluting a defaultdict?" — `key in d`. Membership tests do not go through `__getitem__` and do not call the factory.

```
from collections import defaultdict
d = defaultdict(list)

# the trap
if d["maybe_user"]:          # inserts "maybe_user": []
    print("has data")
print(d)                      # defaultdict(<class 'list'>, {'maybe_user': []})

# safe variants
d2 = defaultdict(list)
if "maybe_user" in d2:       # no insertion
    print("has data")
print(d2.get("maybe_user"))  # None, no insertion
print(d2)                    # still empty
```

Q38 · How is `deque(maxlen=N)` useful for sliding-window problems?

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Postman, Microsoft India, Swiggy · Topic: deque patterns

The question (interviewer's verbatim phrasing):

Show me how you'd compute a rolling average over the last 100 values from a stream.

The 30-second answer: Use `deque(maxlen=100)` — it keeps only the most recent 100 values, auto-evicting from the left as you append on the right. For the rolling average, maintain a running sum that you update when an item enters or leaves the deque; that gives you $O(1)$ per data point instead of $O(N)$ for re-summing the window. The deque handles the windowing; the running sum handles the math.

Step-by-step thought process: 1. Introduce the data structure. `deque(maxlen=N)` is a bounded ring buffer with $O(1)$ append and auto-eviction. 2. Walk the naive approach. Append, then `sum(window) / len(window)` each step — $O(N)$ per data point. For a stream of a million values that's a billion adds. 3. Walk the running-sum optimisation. Keep `total` alongside the deque. On append, add the new value to total; if the deque was already full, subtract the value about to be evicted before appending. Now per-step cost is $O(1)$. 4. Mention that this pattern generalises. Rolling min/max needs a more clever structure (monotonic deque); rolling sum, mean, variance work with simple accumulators.

Edge cases to mention: - `deque(maxlen=0)` is legal and discards every append — rarely useful, but doesn't error. - `deque(maxlen=N)` constructed from a longer iterable keeps only the last N items. - The bounded deque is thread-safe at append/popleft thanks to the GIL; for multi-threaded producers, `queue.Queue` with `maxsize` is the right call instead.

What NOT to say: - "Use a list with manual eviction" — works, but `pop(0)` is $O(n)$ per call so the whole thing is $O(n^2)$ per data point. - "Use NumPy" — overkill for a streaming case where you don't have the whole array.

Common follow-up: "What if I need the rolling max instead of rolling average?" — That's the monotonic-deque pattern: keep a deque of (value, index) pairs in decreasing order, evicting from the right when a larger value arrives and from the left when the front index falls out of the window. $O(1)$ amortised per step.

```

from collections import deque

class RollingMean:
    def __init__(self, n):
        self.window = deque(maxlen=n)
        self.total = 0
    def add(self, x):
        if len(self.window) == self.window.maxlen:
            self.total -= self.window[0] # about to be evicted
        self.total += x
        self.window.append(x)
        return self.total / len(self.window)

r = RollingMean(3)
for x in [10, 20, 30, 40, 50]:
    print(x, "→", r.add(x))    # 10, 15, 20, 30, 40

```

Q39 · What's the difference between `dict` subclassing and using `UserDict` ?

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman, Postman, Razorpay, Cred · Topic: UserDict / UserList

The question (interviewer's verbatim phrasing):

If I want a custom dict-like class, should I subclass `dict` directly or use `collections.UserDict` ?

The 30-second answer: Use `UserDict` when you want to override methods like `__setitem__` and have all the other dict methods route through your override. Subclass `dict` directly when you don't need that routing — for example, when you only want to add new methods. The reason: built-in `dict` methods are implemented in C and don't call back into Python-level overrides for performance, so `dict.update` won't go through your custom `__setitem__`. `UserDict` is a pure-Python wrapper around `self.data` (a real dict) where every method does go through `__setitem__`.

Step-by-step thought process: 1. Frame the problem. Subclassing built-in containers is the natural first move, but breaks the "all paths flow through my override" assumption because of the C-level shortcuts. 2. Give a concrete example. `class LoggingDict(dict): def __setitem__(self, k, v): print("set", k); super().__setitem__(k, v)`. Then `ld = LoggingDict(); ld["a"] = 1` prints; but `ld.update({"b": 2})` does not — `dict.update` bypasses `__setitem__`. 3. Walk the UserDict alternative. `class LoggingDict(UserDict): def __setitem__(self, k, v): print("set", k); self.data[k] = v`. Now `update` is implemented in Python on top of `__setitem__`, so every path logs. 4. Note the trade-off. UserDict adds a small overhead (extra Python

attribute access and dispatch) and isn't itself a `dict` — `isinstance(ud, dict)` is False. Pick subclassing if performance and dict-isinstance matter; pick UserDict if override completeness matters.

Edge cases to mention: - `UserList` and `UserString` are the analogous wrappers for list and str.

- A subclass of `dict` that defines `__missing__(self, key)` does intercept missing-key lookups via `d[k]` — that's how `defaultdict` works under the hood. -

`collections.abc.MutableMapping` is the most flexible base — provides default implementations of update/setdefault/pop on top of just `__getitem__`, `__setitem__`, `__delitem__`, `__iter__`, `__len__`.

What NOT to say: - "Always subclass dict directly" — wrong; the override routing trap is real and common. - "UserDict is a dict" — wrong; it has a `.data` dict but isn't one.

Common follow-up: "What about `collections.abc.MutableMapping` as the base?" — Often the cleanest answer. You implement the five abstract methods, and Mapping/MutableMapping provide the rest with correct routing. Use this when you're not just customising dict but implementing a backed-by-something container (Redis, DB, file).

```
from collections import UserDict

class LoggingDict(UserDict):
    def __setitem__(self, k, v):
        print(f"set {k!r} = {v!r}")
        self.data[k] = v

ld = LoggingDict()
ld["a"] = 1                # set 'a' = 1
ld.update({"b": 2, "c": 3}) # set 'b' = 2 / set 'c' = 3 (both logged)
```

Q40 · What's the cost of `Counter.most_common(k)` for a million-key Counter?

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Postman, Microsoft India, Goldman · Topic: Counter

The question (interviewer's verbatim phrasing):

If I have a Counter with a million keys and I want the top 10 by count, what's the cost?

The 30-second answer: $O(n \log k)$ where n is the total entries and k is the requested top.

`Counter.most_common(k)` uses `heapq.nlargest` under the hood — it maintains a heap of size k and processes each entry in $O(\log k)$. For a million-key Counter asking for top-10, that's $\sim 10^6 * \log(10) \approx 3 * 10^6$ ops, not $10^6 * \log(10^6) \approx 2 * 10^7$ ops you'd get from a full sort. Calling `most_common()` with no argument falls back to sorting and is $O(n \log n)$.

Step-by-step thought process: 1. Headline the formula. `most_common(k)` = $O(n \log k)$. `most_common()` = $O(n \log n)$. 2. Walk the implementation. With a `k`, it calls `heapq.nlargest(k, self.items(), key=itemgetter(1))`. The heap stays size-`k`; each entry is pushed and possibly popped in $O(\log k)$. With no `k`, it returns the entire sorted list — $O(n \log n)$ for sorting. 3. Practical guideline. If you only need the top few, *always* pass the `k`. The Big-O difference is real on big counters — the difference between subsecond and tens of seconds for $n = 10^7$. 4. Mention `heapq.nsmallest` as the mirror. Same Big-O, same heap technique. For "rare items" use `nsmallest` with a key, or `c.most_common() [-k:]` if you already have the sort cost.

Edge cases to mention: - `most_common(0)` returns an empty list, not an error. - Ties are broken by insertion order — Counter inherits dict's insertion-order property, so ties resolve in the order keys were first seen. - For repeated top-`k` queries on a stable Counter, materialising once and indexing is cheaper than calling `most_common` each time.

What NOT to say: - "most_common(k) sorts the whole thing" — wrong; it uses a size-`k` heap. - "It's $O(n)$ " — wrong; the $\log k$ factor is there even for small `k`.

Common follow-up: "What if I want the top 10 by count, breaking ties by alphabetical order?" — Use `sorted` with a composite key: `sorted(c.items(), key=lambda kv: (-kv[1], kv[0]))[:10]`. The negative count gives descending count; the key alphabetises within ties. That's $O(n \log n)$ — fine for a one-off, slower than `most_common(k)` for streaming.

```
from collections import Counter
import heapq
words = ["the"] * 500_000 + ["and"] * 200_000 + ["of"] * 100_000 # plus tail
# (in real code you'd build c from a large iterable)
c = Counter(words)
print(c.most_common(3))      # O(n log 3) — fast even for huge n
print(c.most_common())      # O(n log n) — sorts everything
```

Section F — `heapq` and Priority Queues

Five questions on Python's binary-heap module. `heapq` is a min-heap implemented on a regular list — that is the key implementation detail every interview question circles back to. The Big-O is textbook: push and pop are $O(\log n)$, heapify is $O(n)$ (not $O(n \log n)$), and top-`k` via `nlargest` / `nsmallest` is $O(n \log k)$. The interview hook is "implement a priority queue" or "find top-`k` from a stream" — both reduce to a few lines once you know the API.

Q41 · How is `heapq` implemented, and what are the costs of its operations?

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Microsoft India, Postman, Goldman · Topic: `heapq`

The question (interviewer's verbatim phrasing):

What is `heapq`, and what are the time complexities of `heappush`, `heappop`, and `heapify`?

The 30-second answer: `heapq` provides a binary min-heap as functions that operate on a plain Python list. The list is treated as the array form of a complete binary tree: children of index `i` are at `2i+1` and `2i+2`. `heappush` is $O(\log n)$ — append, then sift up. `heappop` is $O(\log n)$ — swap root with last, pop the tail, sift down. `heapify(list)` is $O(n)$ — applies sift-down from the middle of the array outward. Note `heapify` is $O(n)$, not $O(n \log n)$ — that's a common interview gotcha.

Step-by-step thought process: 1. State the structure. A binary heap is a complete binary tree with the heap property — every parent is $\leq$ its children for a min-heap. Stored as a flat array with children at `2i+1`, `2i+2`. 2. Walk the three core operations and their costs. 3. Explain why `heapify` is $O(n)$, not $O(n \log n)$. The naive analysis says "n calls to sift-down, each $O(\log n) = O(n \log n)$ ". The tighter analysis observes that most nodes are near the bottom and need very few sift steps; the bottom half of nodes need zero, the next quarter need one, etc. The geometric sum gives $O(n)$. 4. Mention that `heapq` is min-heap only. For max-heap behaviour, negate the keys or use tuples like `(-priority, item)`. Python doesn't ship a max-heap function set.

Edge cases to mention: - `heappushpop(h, x)` and `heapreplace(h, x)` are fused push+pop ops that are slightly faster than separate calls — single sift instead of two. - Element comparison uses `<` — so heap elements must be comparable. Tuples compare lexicographically, which is useful for priority + tiebreaker. - `heapify` runs in place — `heapify(lst)` modifies `lst` and returns `None`.

What NOT to say: - "heapq is a sorted list" — wrong; only the root is guaranteed minimum, the rest is partially ordered. - "heapify is $O(n \log n)$ " — wrong; it's $O(n)$ by the tighter analysis.

Common follow-up: "Why does Python not provide a max-heap directly?" — Design decision. Most heap use cases are top-k pickers (which can use `nlargest`), priority queues with explicit priority numbers (where negating works), or scheduling. Negating or wrapping in a tuple covers the cases without adding API surface.

```
import heapq
h = []
for x in [5, 1, 3, 8, 2]:
    heapq.heappush(h, x)      # O(log n) each
print(heapq.heappop(h))     # 1 – smallest
print(heapq.heappop(h))     # 2

# heapify is O(n)
data = [5, 1, 3, 8, 2, 9, 4]
heapq.heapify(data)         # in place, O(n)
print(data[0])              # 1 – root is min
```

Q42 · How do you implement a max-heap with `heapq` ?

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Microsoft India, Flipkart, Swiggy · Topic: `heapq` idioms

The question (interviewer's verbatim phrasing):

`heapq` is a min-heap. How do you get max-heap behaviour?

The 30-second answer: Negate the comparison key. For numeric items, push `-x` and remember to negate on pop. For complex items, wrap in a tuple `(-priority, tiebreaker, item)`. Alternatively, use `heapq.nlargest(k, iterable)` if you only need the top-k — that does the right thing without you maintaining a heap manually.

Step-by-step thought process: 1. State the negation trick. `heapq` always pops the smallest; if you negate, smallest-of-negated = largest-of-original. 2. Walk the int case. `heapq.heappush(h, -x)` then `-heapq.heappop(h)` gives you the largest. Easy and idiomatic. 3. Walk the priority-with-payload case. The payload isn't naturally comparable — you can't `<` two custom objects without a `__lt__`. Wrap as `(-priority, sequence_number, payload)`. The sequence number breaks priority ties without comparing the payload. 4. Offer the `nlargest` shortcut. For batch top-k from a known iterable, `heapq.nlargest(k, items, key=...)` is the right call — $O(n \log k)$, no manual heap maintenance, with a `key=` argument for arbitrary sort criteria.

Edge cases to mention: - For non-numeric keys (strings), you can't trivially negate. Either define a wrapper class with reversed `__lt__`, or use `nlargest` with the right key. - Negation only works on totally-ordered types where negation makes sense. For dates, use `-timestamp.timestamp()` or a wrapper. - Mixing positive and negative isn't allowed in one heap — pick a convention and stick to it.

What NOT to say: - "Use a max-heap from `heapq`" — there isn't one. - "Sort the list and pop" — defeats the $O(\log n)$ per op.

Common follow-up: "How would you implement a priority queue with a tiebreaker?" — Use `(priority, sequence_number, item)`. The sequence number is a monotonically increasing integer assigned on push. When two items have the same priority, the smaller sequence number wins (FIFO order); item itself never gets compared.

```
import heapq

# max-heap via negation
max_heap = []
for x in [5, 1, 3, 8, 2]:
    heapq.heappush(max_heap, -x)
print(-heapq.heappop(max_heap))    # 8 - largest

# priority queue with tiebreaker
import itertools
pq = []
counter = itertools.count()
def add(priority, item):
    heapq.heappush(pq, (priority, next(counter), item))

add(2, "send_email"); add(1, "process_payment"); add(2, "log_event")
while pq:
    print(heapq.heappop(pq)[-1])    # process_payment, send_email, log_event
```

Q43 · What's the "top-k from a stream" pattern with `heapq` ?

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Postman, Microsoft India, Swiggy · Topic: `heapq` patterns

The question (interviewer's verbatim phrasing):

I have a stream of a billion numbers and I want the top 100. How do you do it?

The 30-second answer: Maintain a min-heap of size 100. For each incoming value, if the heap has fewer than 100 items, push. Otherwise compare to the root (the current 100th-largest); if the new value is larger, `heappushpop` to swap it in. At the end, the heap holds the top 100 in arbitrary order. Total cost $O(n \log k)$ where $k = 100$, with $O(k)$ memory regardless of stream size.

Step-by-step thought process: 1. Anchor on the data structure choice. You need top-k. A min-heap of size k holds the k largest seen so far; the root is the smallest of those, which is the threshold for replacing. 2. Walk the algorithm. For each incoming x : if `len(heap) < k`, `heappush(heap, x)`. Else if `x > heap[0]`, `heappushpop(heap, x)`. The pushpop is a fused operation that's faster than separate push+pop. 3. Quantify the cost. $O(\log k)$ per item, n items in the stream, so $O(n \log k)$ total. Memory is $O(k)$ — fixed regardless of n . This is the win over sorting the whole stream. 4. Mention the

convenient form. `heapq.nlargest(k, iterable, key=...)` does this same thing in one call when the iterable is materialised. For a true stream that doesn't fit in memory, the manual heap loop is the right shape.

Edge cases to mention: - For "top-k by some priority other than natural order", use a tuple `(priority, payload)` and the heap compares on priority first. - If k is very close to n, sorting is competitive with the heap — the heap wins for $k \ll n$. - For top-k by frequency (e.g., top words in a stream), use a Counter and call `most_common(k)` at the end, which internally uses `nlargest` with a heap.

What NOT to say: - "Sort the whole stream and take the first k" — $O(n \log n)$ memory and time; fails for streams that don't fit. - "Use a max-heap" — you want min-heap so the root is the *threshold* for replacement.

Common follow-up: "What if I want top-k smallest?" — Use a max-heap of size k (negate keys), or call `heapq.nsmallest(k, iterable)`. Same Big-O.

```
import heapq, random

stream = (random.randint(1, 10_000_000) for _ in range(1_000_000))
k = 5
heap = []
for x in stream:
    if len(heap) < k:
        heapq.heappush(heap, x)
    elif x > heap[0]:
        heapq.heappushpop(heap, x)

print(sorted(heap, reverse=True))    # top 5, biggest first
```

Q44 · What's `heapq.merge` and when do you use it?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Goldman, Flipkart · Topic: `heapq.merge`

The question (interviewer's verbatim phrasing):

I have ten sorted files and I want to produce one sorted stream from them. How would you do that?

The 30-second answer: `heapq.merge(*iterables, key=None)` does exactly this — it lazily merges any number of pre-sorted iterables into a single sorted iterator. It maintains a heap of "current head" elements from each input, pops the minimum, and pulls the next item from that input. Total cost is

$O(N \log k)$ where N is the total number of elements and k is the number of inputs. Memory is $O(k)$ — only the heap of current heads, not the full data.

Step-by-step thought process: 1. State the API. `heapq.merge(it1, it2, ...)` returns an iterator yielding values in sorted order. Inputs must each be sorted individually. 2. Walk the mechanism. Each input contributes its current head to a min-heap. Pop the minimum, yield it, advance that input, push the next value onto the heap. Repeat until all inputs are exhausted. 3. Quantify. $O(\log k)$ per yielded value to maintain the heap; N total values, so $O(N \log k)$. Memory is $O(k)$ — independent of N . This is the right structure for "merge huge files" because nothing has to fit in memory at once. 4. Mention the optional `key` argument. Sorts by `key(item)` while still yielding the original items — useful for merging by timestamp where the records are dicts.

Edge cases to mention: - `heapq.merge` is lazy. It returns an iterator, not a list, so the merged stream is consumed on demand. - For descending-sorted inputs, pass `reverse=True` (added in 3.5). Inputs must be in descending order to match. - If inputs aren't individually sorted, the result is meaningless — merge doesn't validate; it trusts the precondition.

What NOT to say: - "Concatenate the lists and sort" — $O((N) \log(N))$ work and N memory; merge wins on both. - "Loop and zip them together" — doesn't produce a sorted result.

Common follow-up: "How would you merge two sorted lists in pure Python without `heapq`?" — Two-pointer merge: maintain index i into the first and j into the second; emit the smaller and advance. $O(n + m)$. `Heapq.merge` generalises this to k inputs.

```
import heapq

a = [1, 5, 9, 13]
b = [2, 4, 7, 10]
c = [3, 6, 11]
merged = list(heapq.merge(a, b, c))
print(merged)          # [1, 2, 3, 4, 5, 6, 7, 9, 10, 11, 13]

# with a key — merging dicts by timestamp
events_a = [{"ts": 1, "src": "a"}, {"ts": 4, "src": "a"}]
events_b = [{"ts": 2, "src": "b"}, {"ts": 5, "src": "b"}]
for ev in heapq.merge(events_a, events_b, key=lambda e: e["ts"]):
    print(ev)
```

Q45 · How is `heapq.nlargest(k, iterable)` faster than `sorted(iterable)[-k:]` ?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Flipkart, Goldman · Topic: `heapq` vs sort

The question (interviewer's verbatim phrasing):

Both produce the top-k items. Why prefer `heapq.nlargest` over `sorted(...)[-k:]` ?

The 30-second answer: `heapq.nlargest(k, iterable)` is $O(n \log k)$; `sorted(iterable)[-k:]` is $O(n \log n)$. For small k and large n , the gap is huge. `nlargest` maintains a heap of size k , processing each item in $O(\log k)$; `sorted` has to fully sort the entire input. For $k = 10$ and $n = 10$ million, that's $10M \log 10 \approx 33M$ ops vs $10M \log 10M \approx 230M$ ops — about 7× faster.

Step-by-step thought process: 1. State the two complexities. `nlargest` = $O(n \log k)$. Sort + slice = $O(n \log n)$. Same $O(n)$ scan, different log factor. 2. Walk why `nlargest` is faster for small k . It maintains a fixed-size min-heap; for each item it does $O(\log k)$ work. `Sorted` has to process all comparisons across the full list. 3. Note the break-even. When k approaches n (say $k = n/2$ or more), `sorted` becomes competitive because the constant factors favour Timsort. For k in the tens or hundreds against an n in the millions, `nlargest` wins decisively. 4. Mention the implementation. `heapq.nlargest` uses `heapify` ($O(n)$) for the initial seed of the top k from the iterable, then runs the streaming pattern from Q43. Smart implementation.

Edge cases to mention: - `nlargest(k, items, key=func)` accepts a key function — applied once per item, just like `sorted`'s key. - For $k = 1$, `max(iterable)` is even faster than `nlargest(1, ...)` because it skips the heap entirely. For $k = n$, `sorted(iterable, reverse=True)` is the right tool. - `nsmallest` is the mirror function with the same Big-O.

What NOT to say: - "Both are the same" — they are not; the log factor matters at scale. - "`nlargest` sorts internally" — wrong; it uses a heap.

Common follow-up: "*At what point does sort become competitive?*" — Roughly when $k > n / \log n$; the constant factor of sort's C implementation closes the gap. For everyday interview answers, "small k , large n " is the `nlargest` case.

```
import heapq, time

data = list(range(10_000_000))
k = 100

t = time.perf_counter()
top_a = heapq.nlargest(k, data)
print("nlargest:", time.perf_counter() - t)

t = time.perf_counter()
top_b = sorted(data)[-k:]
print("sorted+slice:", time.perf_counter() - t)
```

Section G — `bisect` and Sorted Lists

Five questions on the `bisect` module — binary search and binary-search-driven insertion into a sorted list. The classic interview win is "you can do membership in a sorted list in $O(\log n)$ without a dict"; the classic trap is "insert is $O(\log n)$ for finding the position but $O(n)$ for the actual insert because of list-shift cost". Knowing both is what separates the candidate who's read the module from the one who's used it.

Q46 · What does `bisect.bisect_left` do and what's its cost?

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Microsoft India, Flipkart, Swiggy · Topic: `bisect`

The question (interviewer's verbatim phrasing):

What does `bisect_left` return, and how is it different from `bisect_right` ?

The 30-second answer: Both return the index where you'd insert the target to keep the list sorted.

`bisect_left(a, x)` returns the leftmost such position — if `x` already exists, you get the index of the first occurrence. `bisect_right(a, x)` returns the rightmost — you get the index *after* the last occurrence. Both are $O(\log n)$ binary searches. The choice matters for "does this value exist?" — `bisect_left` gives the insertion point at the start of any run of equal values, which you'd compare against to check existence.

Step-by-step thought process: 1. State the headline. Both functions do binary search on a sorted list to find an insertion point; difference is left vs right tie-breaking. 2. Walk the existence check. `i = bisect_left(a, x); a[i] == x if i < len(a) else False`. This is $O(\log n)$ — no need to scan the list. The fastest "does this exist?" on a sorted list. 3. Walk the range query. To count how many times `x` appears: `bisect_right(a, x) - bisect_left(a, x)`. Both $O(\log n)$, so the difference is also $O(\log n)$. 4. Mention the `lo` and `hi` arguments. Both functions accept `lo` and `hi` to restrict the search range — useful for searching within a window of a larger list without slicing (which would allocate).

Edge cases to mention: - For an empty list, both return 0 — sensible behaviour, no error. - If `x` is greater than every element, both return `len(a)` — meaning "insert at the end". - `bisect.bisect` (no suffix) is an alias for `bisect_right`. Always use the explicit names for readability.

What NOT to say: - "bisect returns True/False for membership" — no, it returns the position. - "bisect_left and bisect_right are the same" — they differ when the target exists in the list.

Common follow-up: "How do you do a binary search that returns the index of `x` or -1 if not found?" — Wrap `bisect_left`: `i = bisect_left(a, x); return i if i < len(a) and a[i] == x else -1`. Concise and $O(\log n)$.


```

from bisect import bisect_left, bisect_right
a = [1, 2, 2, 2, 5, 7]

print(bisect_left(a, 2))      # 1 – first '2' position
print(bisect_right(a, 2))     # 4 – after last '2'
print(bisect_right(a, 2) - bisect_left(a, 2))  # 3 – count of '2's

# existence
def exists(arr, x):
    i = bisect_left(arr, x)
    return i < len(arr) and arr[i] == x

print(exists(a, 5))  # True
print(exists(a, 3))  # False

```

Q47 · What's the cost of `bisect.insort` and why is it not $O(\log n)$?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Flipkart, Goldman · Topic: `bisect.insort`

The question (interviewer's verbatim phrasing):

`bisect.insort` inserts into a sorted list. Is it $O(\log n)$?

The 30-second answer: No — it's $O(n)$. `insort` finds the position in $O(\log n)$ via `bisect`, but inserting into a Python list at a non-end position requires shifting every subsequent element by one slot — that's $O(n)$. For maintaining a sorted collection under heavy churn, this is the bottleneck. If you need $O(\log n)$ insert, reach for `sortedcontainers.SortedList` (third-party) or a balanced BST.

Step-by-step thought process: 1. State the headline. `insort` = $O(\log n)$ search + $O(n)$ insert = $O(n)$ overall. The list-shift cost dominates. 2. Walk through the mechanism. `bisect_left` returns position k in $O(\log n)$. `list.insert(k, x)` slides positions $k..n-1$ over by one slot via `memmove` — that's $O(n)$ for k near the start, $O(1)$ for k near the end. 3. State the trade-off. For a workload where you build the list once and query many times, insert during build is fine — $O(n^2)$ total to build but $O(\log n)$ per query afterwards. For high-churn workloads where you insert and query continuously, that $O(n)$ per insert kills you. 4. Mention the right tool. `sortedcontainers.SortedList` gives $O(\log n)$ insert and $O(\log n)$ lookup via a list-of-lists data structure; it's the go-to for sorted-collection problems in Python interviews.

Edge cases to mention: - `insort_left` and `insort_right` mirror `bisect_left` and `bisect_right` — picks which side of equal values to insert on. Default `insort` is `insort_right`. - For inserting many sorted items at once, `heapq.merge` is often a better fit than repeated inserts. - The shift cost is `memmove` — fast in absolute terms (C-level pointer copy) but still $O(n)$ in operation count.

What NOT to say: - "insert is $O(\log n)$ " — wrong; the shift makes it $O(n)$. - "Use insert for big sorted collections" — wrong; use `SortedList` instead for big churn.

Common follow-up: "What does `sortedcontainers.SortedList` use under the hood?" — A list of sorted sublists, each capped at a load size (default 1000). Insertion finds the right sublist in $O(\log)$ and inserts in $O(\text{load})$ — sqrt-n-ish behaviour that's effectively $O(\log n)$ for most workloads.

```
from bisect import insert
import time

# building a sorted list – slow at scale
sorted_ids = []
t = time.perf_counter()
for x in range(50_000):
    insert(sorted_ids, x)      #  $O(n)$  per insert –  $n^2$  total
print("insert:", time.perf_counter() - t)

# alternative if churn is high
# from sortedcontainers import SortedList  # pip install sortedcontainers
# s = SortedList()
# for x in range(50_000): s.add(x)         #  $O(\log n)$  per insert
```

Q48 · How do you find the closest value in a sorted list?

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Postman, Microsoft India, Cred · Topic: bisect patterns

The question (interviewer's verbatim phrasing):

Given a sorted list and a target, find the closest value to the target. What's the cost?

The 30-second answer: Use `bisect.bisect_left` to find the insertion position in $O(\log n)$, then compare the neighbours at positions `i-1` and `i` to pick the closer one. Total cost $O(\log n)$. The naive `min(a, key=lambda x: abs(x - target))` is $O(n)$ — fine for small lists, but the bisect approach is the right answer once the list is sorted.

Step-by-step thought process: 1. State the algorithm. Binary-search for the insertion point. The closest value is either at that position (if not at the end) or the one before it. 2. Walk the edge cases. If `i == 0`, the target is smaller than every element — closest is `a[0]`. If `i == len(a)`, the target is larger than every element — closest is `a[-1]`. Otherwise compare `abs(target - a[i-1])` against `abs(target - a[i])`. 3. Quantify. Single binary search plus constant-time comparison = $O(\log n)$. Beats the $O(n)$ min-by-key approach for any list big enough to matter. 4. Mention the variant. "Closest

below" or "closest above" come up in time-series ("the most recent measurement before timestamp t") — same bisect call, then pick the right neighbour.

Edge cases to mention: - Ties are broken by whichever neighbour you check first; if you want a deterministic rule, write the comparison explicitly. - For sorted lists of tuples, pass the bisect call a `key` argument (3.10+) or wrap the comparison manually. - If the list isn't pre-sorted, the whole approach doesn't apply — sort first or use the $O(n)$ min-by-key.

What NOT to say: - "Iterate and track the minimum distance" — $O(n)$, worse than necessary. - "Use a dict" — dict doesn't help for "closest" queries; you need ordering.

Common follow-up: "What if the list contains duplicates?" — Bisect returns the leftmost equal position with `bisect_left`. The "closest" answer is unambiguous; the only thing to watch is that you might prefer the leftmost match for stability.

```
from bisect import bisect_left

def closest(a, target):
    if not a: return None
    i = bisect_left(a, target)
    if i == 0: return a[0]
    if i == len(a): return a[-1]
    before, after = a[i-1], a[i]
    return before if abs(target - before) <= abs(target - after) else after

times = [10, 20, 35, 50, 80]
print(closest(times, 25))    # 20 — tied at 5 away, picks the lower (stable)
print(closest(times, 60))    # 50
print(closest(times, 100))   # 80 — past the end
```

Q49 · How do you use `bisect` to bucket values into ranges?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Flipkart, Microsoft India, Swiggy · Topic: bisect patterns

The question (interviewer's verbatim phrasing):

I have score boundaries [60, 70, 80, 90] for grades F, D, C, B, A. How do I map a score to a grade?

The 30-second answer: `bisect.bisect_right(boundaries, score)` returns the index of the grade. With boundaries `[60, 70, 80, 90]` and grades `["F", "D", "C", "B", "A"]`, a score of 75 gives `bisect_right([60,70,80,90], 75) == 2`, which indexes into "C". The bisect call is

$O(\log n)$; for fixed small n (like 4 grades), it's effectively constant. Faster, cleaner, and more maintainable than an if-elif chain.

Step-by-step thought process: 1. Map the problem to bisect. Boundaries are the sorted breakpoints; grades are the output buckets. For n boundaries you have $n+1$ buckets. 2. Pick left vs right based on the inclusive-exclusive convention. `bisect_right` puts equal values into the next bucket ($\geq$ boundary moves up); `bisect_left` puts them into the current bucket ($>$ boundary moves up). Match to your grading rule. 3. Quantify. $O(\log b)$ per lookup where b is the number of boundaries. For tens to thousands of buckets, indistinguishable from $O(1)$ in practice. 4. Mention scale. For millions of values bucketed into a fixed boundary list, this beats an if-chain (which is $O(b)$ per value) by a factor of $b / \log b$ — measurable on large workloads.

Edge cases to mention: - For "lookup-then-do-something" patterns, store the grades parallel to boundaries and index after bisect — no dict needed. - If boundaries change at runtime, just maintain a sorted list of breakpoints. If they're truly static, hardcode them as a tuple. - Negative scores or scores above the max are handled implicitly — `bisect_right([60,70,80,90], -5) == 0` → "F"; `bisect_right([60,70,80,90], 95) == 4` → "A".

What NOT to say: - "Use a dict" — dict doesn't do range lookups efficiently. - "Use a for loop over boundaries" — $O(b)$ per call vs $O(\log b)$.

Common follow-up: "What if I want histograms of millions of values?" — Use `numpy.digitize` or `pandas.cut` for the vectorised version. The bisect pattern is the idiomatic pure-Python answer when NumPy isn't on the table.

```
from bisect import bisect_right

boundaries = [60, 70, 80, 90]
grades = ["F", "D", "C", "B", "A"]

def grade(score):
    return grades[bisect_right(boundaries, score)]

for s in [45, 60, 65, 80, 95]:
    print(s, "→", grade(s))      # F, D, D, B, A
```

Q50 · When is binary search the right tool in Python?

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Postman, Microsoft India, Goldman · Topic: binary search

The question (interviewer's verbatim phrasing):

When do you reach for binary search via `bisect` ?

The 30-second answer: Three classic cases: (1) membership tests on a sorted list where a set isn't an option — e.g., the items aren't hashable; (2) range queries — "how many items fall between a and b" via `bisect_right(b) - bisect_left(a)` ; (3) bucketing values into ranges by sorted breakpoints. Plus algorithmic LeetCode-style problems where you binary-search on the answer space rather than on data. If you can hash, a set is usually simpler; bisect is for the cases set can't handle.

Step-by-step thought process: 1. Walk the three practical use cases above. For each, name the alternative if there is one and why bisect is the right choice. 2. Mention "binary search on the answer". A pattern where the problem is "find the smallest k such that P(k) is True for some monotone predicate P". You binary-search over possible k values, not over data. Comes up in LeetCode "minimum capacity" / "minimum days" problems. 3. State the precondition. Bisect requires the list to be already sorted. If it isn't, you're doing $O(n \log n)$ to sort and then $O(\log n)$ to bisect — for one query that's worse than $O(n)$ linear scan. Pre-sort only when you'll query many times. 4. Mention the alternatives. Set / dict for hashable items. SortedList for high-churn sorted collections. bisect for one-time sort + many queries.

Edge cases to mention: - The bisect functions don't handle "approximate equality" — that's a custom comparison that doesn't fit binary search neatly. - For binary search on a custom object, define `__lt__` or use the `key` parameter (3.10+). - bisect is $O(\log n)$ but Python-level comparisons cost more than a C-level scan — for very small lists, linear scan can win on absolute time.

What NOT to say: - "Use binary search for everything" — set is simpler for unsorted membership. - "Binary search needs a hash" — wrong; it needs ordering.

Common follow-up: "What's an example of binary search on the answer space?" — LeetCode 875 ("Koko Eating Bananas"): find the smallest eating speed k such that all bananas can be eaten in h hours. Binary search k in `[1, max(piles)]` ; the predicate "can finish in h hours at speed k" is monotone non-increasing in k.

```

from bisect import bisect_left, bisect_right

# range query: how many ids fall in [100, 200]?
sorted_ids = [10, 50, 100, 120, 150, 200, 300]
count = bisect_right(sorted_ids, 200) - bisect_left(sorted_ids, 100)
print(count)      # 4 - values 100, 120, 150, 200

# binary search on the answer
def min_eating_speed(piles, h):
    def can_finish(k):
        return sum((p + k - 1) // k for p in piles) <= h
    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        if can_finish(mid): hi = mid
        else: lo = mid + 1
    return lo

print(min_eating_speed([3, 6, 7, 11], 8))  # 4

```

Section H — Big-O in Practice

Ten questions on Big-O the way it shows up in interviews — not the textbook recital, but the practical "you've got the headline complexities; now do you understand the second-order effects?". Constant factors, amortised vs worst-case, space-time trade-offs, and when CPython's implementation choices change the answer in ways the textbook can't anticipate.

Q51 · Give me the Big-O table for the common built-in containers.

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft India, Goldman, Postman, Flipkart · Topic: Big-O table

The question (interviewer's verbatim phrasing):

Quick Big-O check: for list, dict, set, and deque, what are the costs of the main operations?

The 30-second answer: list: index $O(1)$, append amortised $O(1)$, insert/pop at middle $O(n)$, **in** $O(n)$, slice $O(k)$, sort $O(n \log n)$. dict: lookup / insert / delete $O(1)$ average / $O(n)$ worst. set: **in** / add / remove $O(1)$ average / $O(n)$ worst. deque: append / appendleft / pop / popleft $O(1)$, index in middle $O(n)$, **in** $O(n)$.

Step-by-step thought process: 1. Walk list first — it's the one candidates fumble most often. Index is $O(1)$ because it's an array; append is amortised $O(1)$ due to over-allocation; front operations are $O(n)$ because of shift. `in` and `index` and `count` are all $O(n)$ linear scans. 2. Walk dict. Average case is the "right answer"; mention the worst case for completeness. Iteration is $O(n)$. Compare with set, which is the same minus the value dimension. 3. Walk set. Same as dict for membership, add, remove. Set algebra (union, intersection) is $O(|a| + |b|)$ for union, $O(\min)$ for intersection. 4. Walk deque. The headline is $O(1)$ at both ends, but $O(n)$ for random access. Don't forget that `in` is $O(n)$ — same as list.

Edge cases to mention: - `list.pop()` with no argument is $O(1)$ — only `pop(k)` for $k < n-1$ is $O(n)$. - `dict.popitem()` is $O(1)$ and removes the last-inserted item in 3.7+. - set update (`s |= other`) is $O(|other|)$ — proportional to what's being added.

What NOT to say: - "list `in` is $O(\log n)$ " — wrong; it's $O(n)$. - "dict is $O(1)$ for everything" — average case yes; mention worst case too.

Common follow-up: "What about `frozenset` ?" — Same as set for read-only operations: $O(1)$ average membership, set algebra at the same costs. No mutation operations.

```
# quick reference
operations = """
    avg      worst      notes
list[k]      0(1)      0(1)      index
list.append  0(1)*    0(n)      *amortised; 0(n) on resize
list.insert(0) 0(n)    0(n)      shift
x in list     0(n)    0(n)      linear scan
list.sort     0(nlogn) 0(nlogn)  Timsort

dict[k]       0(1)    0(n)      hash collision worst
d.get / set   0(1)    0(n)
del d[k]      0(1)    0(n)

set add/in/rem 0(1)    0(n)      same as dict

deque.append  0(1)    0(1)      either end
deque[k]      0(n)    0(n)      middle index
"""
print(operations)
```

Q52 · What does "amortised $O(1)$ " actually mean, and how do you reason about it?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Goldman, Flipkart · Topic: amortised analysis

The question (interviewer's verbatim phrasing):

Explain "amortised constant time" — how is it different from "average case constant"?

The 30-second answer: Amortised means worst-case-averaged-over-many-ops. The single worst op might be expensive — like a list resize that's $O(n)$ — but the *total* work across n operations is $O(n)$, so per-operation it's $O(1)$. Average case is a probabilistic statement about a single op against random inputs; amortised is a deterministic statement about a sequence of ops. They sound similar; they're proven differently.

Step-by-step thought process: 1. Define the two terms carefully. Amortised = total cost over n ops / n . Average = expected cost per op over a distribution of inputs. 2. Show why `list.append` is amortised. Most appends are $O(1)$; occasional resizes are $O(n)$. Total work for n appends is $O(n)$ — bounded because resizes happen geometrically less often. Total / $n = O(1)$ per op. 3. Show why dict lookup is average — not amortised. Each lookup has a probabilistic cost; with a good hash function and load factor cap, the expected cost is $O(1)$. The worst case (adversarial inputs) is $O(n)$, which probability bounds away. 4. Pull on the practical implication. Amortised is solid for "I'll do many appends; total time is predictable". Average is solid for "each lookup is typically fast, but I should not bet a hard real-time SLA on a single one".

Edge cases to mention: - A single amortised- $O(1)$ op can be slow — the resize step. If you have a tail-latency SLA, amortised analysis doesn't protect you. Use a pre-sized container instead. - Adversarial inputs can break average-case guarantees; CPython's SipHash randomisation specifically defends against this for strings. - Both amortised and average can sit alongside worst-case bounds — they're not mutually exclusive.

What NOT to say: - "Amortised = average" — they're different; one is per-sequence, one is per-input. - "Amortised means the resize is free" — no, the resize is $O(n)$ on that one call; amortised distributes the cost over the calls between resizes.

Common follow-up: "*Is dict insertion amortised $O(1)$ too?*" — Yes, in the same sense as `list.append` — when the dict needs to resize, that single insertion is $O(n)$, but the next many insertions are $O(1)$ until the next resize. Total $O(n)$ over n inserts.


```
import sys, time
lst = []
slow_ops = 0
fast_ops = 0
prev_capacity = 0
for i in range(1_000_000):
    t = time.perf_counter_ns()
    lst.append(i)
    elapsed = time.perf_counter_ns() - t
    if elapsed > 1000:    # arbitrary threshold for "slow"
        slow_ops += 1
    else:
        fast_ops += 1
print(f"fast: {fast_ops}, slow: {slow_ops}")    # most are fast; resizes are rare
```

Q53 · Why do constant factors matter, and where in Python do they bite?

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman, Razorpay, Postman, JP Morgan · Topic: constant factors

The question (interviewer's verbatim phrasing):

Two algorithms are both $O(n)$. One runs in 1 second; the other runs in 10 seconds. What kinds of differences cause that, and which Python patterns trigger them?

The 30-second answer: Big-O hides the constant, and Python's interpreter overhead per operation is enormous compared to compiled C. The same $O(n)$ loop can be 10× slower depending on whether each step is a C-level builtin call or a Python-level bytecode sequence. Three classic 10× gaps: pure Python loop vs `sum(iterable)`, list comprehension vs explicit loop with append, and `for k in d` + dict lookup vs walking `d.items()`. Same Big-O, very different wall time.

Step-by-step thought process: 1. Frame the principle. Big-O says how the cost scales; constants say how big the cost is at a given size. For Python, where each bytecode op is hundreds of nanoseconds, the constant is what kills you. 2. Walk three examples. Sum a million ints: `sum(lst)` ~10 ms because it's a C function; `total = 0; for x in lst: total += x` ~50 ms because each step is interpreted. Both $O(n)$. 3. List comp vs explicit append. `[f(x) for x in lst]` vs `out = []; for x in lst: out.append(f(x))`. Comp is ~30% faster — same Big-O, fewer bytecode ops per iteration. 4. dict iteration. `for k in d: v = d[k]` does a hash lookup per iteration; `for k, v in d.items()` walks the entries array directly. Same Big-O, the `items()` form is roughly 2× faster.

Edge cases to mention: - Builtin C functions (`sum`, `max`, `min`, `any`, `all`) are dramatically faster than equivalent Python loops because they don't pay the per-op interpreter tax. -

`functools.reduce` is *not* faster than a Python loop most of the time — the lambda dispatch

overhead dominates. - `operator.itemgetter`, `operator.methodcaller`, etc., are C functions that beat lambdas for repeated calls; matters in sorts and map applications.

What NOT to say: - "Big-O is all that matters" — wrong for Python; constants are 10–100× in real workloads. - "Use NumPy for everything" — only when the data fits and the dependency is acceptable; the C-builtins argument applies to stdlib too.

Common follow-up: *"Where do I find which builtins are C-implemented?"* — Everything in `builtins` (sum, max, sorted, etc.), most of `itertools`, most of `operator`, and most of `collections`. Anything pure-Python is in `inspect.getsourcefile` and looks like a Python source file rather than `built-in`.

```
import time
lst = list(range(1_000_000))

t = time.perf_counter()
total = sum(lst)
print("sum:", time.perf_counter() - t)

t = time.perf_counter()
total = 0
for x in lst:
    total += x
print("loop:", time.perf_counter() - t)  # 4-5x slower for same Big-O
```

Q54 · What's the space complexity of a list comprehension vs a generator expression?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Postman, Microsoft India, Cred · Topic: space complexity

The question (interviewer's verbatim phrasing):

`[x*x for x in range(10**9)]` vs `(x*x for x in range(10**9))` — what's the difference?

The 30-second answer: The list comprehension allocates a list of a billion items — gigabytes of memory and likely a MemoryError or OOM kill. The generator expression allocates a generator object — a few hundred bytes — and produces values one at a time on demand. Same algorithm, $O(n)$ time for either, but space is $O(n)$ for the list and $O(1)$ for the generator. Use the generator whenever the downstream consumer doesn't need the materialised list.

Step-by-step thought process: 1. State the space difference. List comprehension stores all n results in memory at once; generator stores only the iteration state. 2. Walk the lazy semantics. The generator doesn't compute anything when constructed — it computes on each `next()` call. For `sum(x*x for x in range(10**9))`, `sum` pulls values one at a time and discards them; total memory stays bounded. 3. Quantify. A billion ints as a Python list is ~28 GB; a generator over the same is ~200 bytes. The space win is the entire point of the syntax. 4. Note the trade-off. Generators are single-pass. If you iterate twice, the second pass yields nothing. List comp can be iterated as many times as you like.

Edge cases to mention: - A generator expression inside a function call doesn't need its own parens: `sum(x*x for x in r)` is fine; `sum((x*x for x in r))` is the same. - `list(genexp)` materialises into a list — useful for "I wanted to be lazy at first but actually need the list". - Set and dict comprehensions exist too: `{x*x for x in r}` (set) and `{x: x*x for x in r}` (dict). Both eager.

What NOT to say: - "Generator is the same as list comp" — wrong; eager vs lazy is the entire point. - "Generator is slower" — usually about the same per element; the win is memory.

Common follow-up: "What if I want to iterate twice but lazily?" — `itertools.tee` can branch a generator into multiple independent iterators, but caches values internally — so for full-pass tee, you're back to the list memory. There's no free lunch; pick lazy or pick re-iterable.

```
import sys

list_comp = [x*x for x in range(10_000)]
gen_exp = (x*x for x in range(10_000))

print(sys.getsizeof(list_comp))    # ~85 KB
print(sys.getsizeof(gen_exp))      # 200 bytes regardless of range size

# sum works on both — generator never builds the list
print(sum(x*x for x in range(1_000_000))) # O(1) extra memory
```

Q55 · What's the Big-O of building a `Counter` from a million-element iterable?

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Flipkart, Microsoft India, Goldman · Topic: Counter cost

The question (interviewer's verbatim phrasing):

What's the cost of `Counter(stream_of_a_million_items)` ?

The 30-second answer: $O(n)$ time and $O(k)$ space where k is the number of unique items. Each item is hashed once and either inserts (new key) or increments (existing key) — both $O(1)$ on average. So one

pass over the stream gives you the full count. Compare against the naive "sort then count runs" approach which is $O(n \log n)$ — Counter wins.

Step-by-step thought process: 1. State the headline. $O(n)$ time, $O(k)$ space, single pass. 2. Walk the mechanism. For each item, Counter does a hash lookup in its underlying dict. New keys insert; existing keys read the current count, add 1, store back. Each step is amortised $O(1)$. 3. Compare alternatives. Sorting and counting runs: $O(n \log n)$ time + $O(n)$ memory. Building a dict manually with `d[x] = d.get(x, 0) + 1`: $O(n)$ too, but two lookups per item (get + set) instead of one fused increment. 4. Note the practical implication. For streaming workloads where the dataset doesn't fit in memory, you can pass an iterator to Counter and it'll consume lazily — same Big-O, no materialisation.

Edge cases to mention: - `Counter.update` does the same work in batches if you want to accumulate across multiple streams. - For very high-cardinality streams (k close to n), the space cost dominates; consider sketches like Count-Min if approximate is OK. - The constant factor of Counter is slightly higher than raw dict because of method dispatch, but the convenience usually wins.

What NOT to say: - "Counter is $O(n \log n)$ " — wrong, no sort involved. - "Counter loads the whole stream into memory" — wrong; it only stores the (key → count) entries.

Common follow-up: "What if I have a billion items and only a million unique keys?" — Memory is $O(k)$, so a million dict entries — manageable. The streaming cost is still $O(n)$ total time. Counter is the right fit.

```
from collections import Counter
import time

stream = (i % 1_000 for i in range(10_000_000)) # 1000 unique keys, 10M items
t = time.perf_counter()
c = Counter(stream)
print("time:", time.perf_counter() - t)
print("unique:", len(c), "top 3:", c.most_common(3))
```

Q56 · When does CPython implementation detail change the Big-O of your code?

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman, Razorpay, Postman, JP Morgan · Topic: CPython internals

The question (interviewer's verbatim phrasing):

Give me a case where CPython's implementation makes a textbook Big-O answer wrong in practice.

The 30-second answer: Two classic cases. First, string concatenation in a loop: `s = ""; for x in items: s += x` looks like $O(n)$ per iteration → $O(n^2)$ total, but CPython has an optimisation that

detects the in-place concat pattern and makes it amortised $O(n)$ when the string isn't shared. The textbook answer is $O(n^2)$; the actual answer is $O(n)$ in CPython, $O(n^2)$ in PyPy. Second, list `+=` vs `=` `+`: `lst += other` calls `extend` (in place); `lst = lst + other` allocates a new list — same Big-O but the second copies and is slower by a constant factor.

Step-by-step thought process: 1. Set the framing. The Python language defines semantics, not implementation. CPython has optimisations that change real-world costs without changing the language guarantee. 2. Walk the string-concat case. The classic anti-pattern is $O(n^2)$; CPython's small-string buffer optimisation makes it $O(n)$ when the string has exactly one reference and the next op is `+=`. So a tight loop is fast; the same op on a string captured in a list elsewhere is slow. 3. Walk the list-extend case. `lst.extend(other)` and `lst += other` both extend in place — $O(\text{len}(\text{other}))$. `lst = lst + other` allocates a new list of `len(lst) + len(other)` items — same Big-O, twice the memory traffic. 4. State the takeaway. Don't rely on these optimisations for portability — PyPy, Jython, etc., may not have them. Write the explicit `''.join(...)` or `extend` form; same speed in CPython, portable elsewhere.

Edge cases to mention: - The string-concat optimisation is fragile — it breaks if the string is captured by a debugger or by `sys.getrefcount`-affecting code. - `''.join(seq)` is always the right idiom for building a string from many parts; $O(n)$ and portable. - Small-int caching (-5 to 256) is another CPython detail — `1 + 1 is 2` is True, `1000 + 1 is 1001` is implementation-defined.

What NOT to say: - "Trust CPython optimisations" — they're real but not guaranteed. - "Use `+=` for strings; it's fast" — works in CPython, slow elsewhere, prefer `.join`.

Common follow-up: "Why is `''.join()` always the right answer for building strings?" — It allocates one final string of the right size by counting first, then copies all the parts in one pass. $O(\text{total length})$, no quadratic concat, no reliance on optimisations.

```
import time

# the trap that CPython hides
n = 100_000
parts = ["x"] * n

# CPython makes this O(n); PyPy and the textbook say O(n^2)
t = time.perf_counter()
s = ""
for p in parts:
    s += p
print("+=:", time.perf_counter() - t)

# portable, always O(n)
t = time.perf_counter()
s = "".join(parts)
print("join:", time.perf_counter() - t)
```

Q57 · What's the difference between space-complexity $O(n)$ and $O(1)$ for the same algorithm?

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft India, Goldman, Postman, Flipkart · Topic: space complexity

The question (interviewer's verbatim phrasing):

When an interviewer asks "can you do this in $O(1)$ extra space?", what do they mean?

The 30-second answer: "Extra space" means memory you allocate beyond the input. $O(1)$ extra means a constant amount regardless of n ; $O(n)$ extra means proportional to the input. For example, "reverse a list in $O(1)$ extra space" means in-place via two pointers swapping ends, not by allocating a new list. The input itself doesn't count against the budget; auxiliary storage does.

Step-by-step thought process: 1. Define the term. Auxiliary space is everything you allocate beyond the input — temporary lists, dicts, recursion frames, copies of slices. 2. Walk the reverse example. `lst.reverse()` is in-place, $O(1)$ auxiliary. `lst[::-1]` allocates a new list, $O(n)$ auxiliary. Same time complexity, different space. 3. Walk recursion. Each recursive call adds a frame to the call stack. A recursive in-place reverse is $O(n)$ auxiliary because of the n frames, even though no explicit data is allocated. 4. Mention common interview patterns. Two-pointer ($O(1)$), in-place partitioning ($O(1)$ auxiliary, Quicksort's promise), Floyd's tortoise-and-hare cycle detection ($O(1)$).

Edge cases to mention: - A generator is $O(1)$ auxiliary because it doesn't materialise — handy for converting an $O(n)$ -space pipeline into an $O(1)$ -space one. - Python's recursion limit (default 1000) effectively caps the auxiliary space recursion can use; deep recursion either gets converted to iteration or hits `RecursionError`. - "In-place" in Python sometimes still allocates (e.g., `lst.sort()` does $O(n)$ auxiliary for the Timsort merge buffer, despite "in-place" sounding $O(1)$).

What NOT to say: - "In-place means $O(1)$ space always" — not true; Timsort is in-place but uses $O(n)$ auxiliary. - "Recursion is free" — each call costs a frame.

Common follow-up: "How do you reverse a list in $O(1)$ extra space?" — Two-pointer swap: `i, j = 0, len(lst)-1; while i < j: lst[i], lst[j] = lst[j], lst[i]; i += 1; j -= 1`. $O(n)$ time, $O(1)$ auxiliary.

```
def reverse_in_place(lst):
    i, j = 0, len(lst) - 1
    while i < j:
        lst[i], lst[j] = lst[j], lst[i]
        i += 1; j -= 1

a = [1, 2, 3, 4, 5]
reverse_in_place(a)
print(a)    # [5, 4, 3, 2, 1] - no new list allocated
```

Q58 · How does Big-O of a comprehension compare to the equivalent loop?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Postman, Microsoft India, Cred · Topic: comprehension Big-O

The question (interviewer's verbatim phrasing):

Is `[f(x) for x in lst]` the same Big-O as the equivalent for loop with `append`?

The 30-second answer: Same Big-O — both $O(n)$ time and $O(n)$ space for a list comprehension. The comprehension is a constant-factor faster (roughly 20–40%) because it's a specialised bytecode with no name lookups for `.append`. Same applies to dict and set comprehensions. The comp is preferred for readability; the speed win is a nice secondary effect, not the reason to use it.

Step-by-step thought process: 1. State the equivalence. Both forms produce the same list with the same element-wise work — $O(n)$ for n elements. 2. Walk the bytecode difference. The comprehension has dedicated opcodes that skip the `LOAD_METHOD append` step every iteration; the explicit loop does that lookup every time. 3. Quantify. For a million-element transform, the comp is roughly 20–40% faster depending on what `f` does. Not a Big-O improvement, a constant-factor one. 4. Note when to prefer the explicit loop. Multiple statements per iteration, complex conditionals, mutation of multiple containers — at that point the comp's nested syntax stops being readable.

Edge cases to mention: - Nested comprehensions are $O(n*m)$ — they're not magically faster than nested loops; readability is the only benefit. - A generator expression inside `sum`, `max`, etc., is $O(1)$ space — same Big-O as the list comp but lower space. - `dict(...)` and `set(...)` constructors with a comprehension inside have a slight extra cost vs the dedicated dict/set comp syntax; prefer `{k: v for ...}` directly.

What NOT to say: - "Comprehensions are $O(\log n)$ " — wrong; they iterate. - "Comprehensions are slower because they look magical" — wrong; they're slightly faster.

Common follow-up: "When would you write the explicit for loop instead of a comp?" — When you have side effects, multiple containers being updated, complex multi-line bodies, or when the readability of the comp suffers from too many filter clauses or nested loops.

```
import time
lst = list(range(1_000_000))

t = time.perf_counter()
a = [x*x for x in lst]
print("comp:", time.perf_counter() - t)

t = time.perf_counter()
b = []
for x in lst:
    b.append(x*x)
print("loop:", time.perf_counter() - t)
```

Q59 · What's the Big-O of `str.join` and string formatting?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Postman, Microsoft India, TCS · Topic: string Big-O

The question (interviewer's verbatim phrasing):

What's the cost of `"".join(parts)` for n parts each of average length k ?

The 30-second answer: $O(n*k)$ — proportional to the total length of the output. `str.join` makes one pass to compute the total length, allocates the result string of that size in a single allocation, then copies each part into the right offset. Beats the `+= s` loop on portability (since join's Big-O is guaranteed; `+=` relies on CPython's optimisation). f-strings are $O(\text{length of the result})$ and are faster than `format()` for the same operation at a constant-factor level.

Step-by-step thought process: 1. Walk `str.join`'s algorithm. Two passes: first to sum lengths, second to copy. Both are $O(nk)$ in total. One allocation, no intermediate growth. 2. Compare with the loop.

`s = ""; for p in parts: s += p` in CPython is $O(nk)$ when the optimisation kicks in; $O(n^2k)$ when it doesn't. `join` is always $O(nk)$. 3. Mention f-strings. They produce a string of length proportional to the output. Internally an f-string is compiled to a specialised opcode that builds the result in one allocation — faster than `.format()` or `%` for the same result. 4. State the practical guidance. Always use `''.join(parts)` for collecting strings; always use f-strings for formatted output. They're both $O(\text{output length})$ and they're both the most idiomatic choice.

Edge cases to mention: - `join` accepts any iterable of strings — including a generator — but it has to materialise to count lengths first, so the generator gets consumed. - For non-string parts, you'd convert first: `''.join(str(x) for x in items)` or `''.join(map(str, items))`. - `bytes.join(parts)` is the bytes equivalent — same Big-O.

What NOT to say: - "join is $O(n^2)$ " — wrong; it's $O(\text{total output length})$. - "f-strings are slow" — wrong; they're the fastest formatting in 3.12+.

Common follow-up: "What's the difference between f-string and .format() at the Big-O level?" — Same Big-O. f-strings have a smaller constant because they compile directly to opcodes that build the string; .format() involves a method call and parsing the template string at runtime.

```
parts = ["alpha", "beta", "gamma", "delta"] * 1000
s = "".join(parts) # O(total length)

name, age = "Aarav", 25
greeting = f"Hello {name}, age {age}" # O(result length)
```

Q60 · How do you reason about Big-O when nesting data structures?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Flipkart, Goldman · Topic: nested Big-O

The question (interviewer's verbatim phrasing):

I have a dict of lists. What's the cost of `for k, v in d.items(): for x in v: process(x)` ?

The 30-second answer: $O(\text{total items across all values})$. If the dict has k keys and the total number of items across all values is n , the loop is $O(k + n)$ — k for iterating the keys, n for iterating across all inner lists. The dict's depth doesn't add a log factor; you're just walking everything once. Same applies to lists of dicts and other nested combinations — sum the work, don't multiply.

Step-by-step thought process: 1. State the rule. For nested iteration with no inner lookups, Big-O is the sum of all elements visited. Don't conflate "two for loops" with " $O(n^2)$ " — the inner loop's bound varies. 2. Walk the example. d has k keys, total entries across values is n . Outer loop runs k times, inner runs n times *across all outer iterations*. Total work is $k + n$, not $k * n$. 3. Contrast with the false case. If the inner loop ran n times per outer iteration — say a list of k items each scanned against another list of size n — that would be $O(k * n)$. The dict-of-lists case is not that. 4. Mention lookups inside loops. `for k in d: for x in d[k]: ...` — the `d[k]` is an $O(1)$ lookup that happens k times. Total $O(k + n)$ still, because the dict access is constant per outer iteration.

Edge cases to mention: - A dict of lists where one value list is enormous and the rest are empty still costs $O(n)$ — the work is in the big list, not in the dict structure. - Be careful with `len(d.keys())` vs `sum(len(v) for v in d.values())` — the former is the dict's count, the latter is the total entry count. They're different. - For very skewed distributions, the $O(k + n)$ bound dominates whichever is larger; for balanced distributions, they're often similar.

What NOT to say: - "Nested loops are $O(n^2)$ " — only if both bounds scale with n independently. - "Dict access inside the loop is free" — it's $O(1)$, not free; just so cheap it usually doesn't matter.

Common follow-up: "What if I wanted to find a value that appears in any inner list?" — Same $O(k + n)$ if you scan everything, or $O(1)$ if you keep a separate `set` of all-values that you update on insert/delete. Trade memory for query speed.

```
d = {"users": [1, 2, 3], "admins": [10], "guests": [50, 51, 52, 53]}
k = len(d)
n = sum(len(v) for v in d.values())
print(f"k={k}, n={n}")    # k=3, n=8

# total work  $O(k + n) = O(11)$ , not  $O(k * n) = O(24)$ 
for key, lst in d.items():
    for x in lst:
        pass
```

Section I — DSA Patterns in Idiomatic Python

Ten questions on the data-structure patterns that show up in round-one DSA interviews at Indian product companies — but written in idiomatic Python, not Java-translated-into-Python. The signature shift: dict-as-complement-store instead of nested loops, `collections.deque` for BFS instead of `LinkedList`, `functools.cache` instead of hand-rolled memo dicts, sorted-tuple keys for anagram grouping. These are the patterns where a Python-native answer beats a Java-port both on lines of code and on wall time, because each idiom rests on a C-level builtin doing the heavy lifting.

Q61 · How do you solve the two-sum problem in idiomatic Python?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Swiggy, Postman, Cred · Topic: hash-map pattern

The question (interviewer's verbatim phrasing):

Given an array of integers and a target, return the indices of the two numbers that add up to the target. What's the idiomatic Python approach?

The 30-second answer: One pass with a dict that maps "value already seen" to its index. For each new value `x`, check if `target - x` is in the dict; if yes, return the stored index and the current one. $O(n)$ time, $O(n)$ space. The naive $O(n^2)$ nested-loop approach is a textbook anti-pattern and exists only so the interviewer can ask you to improve it.

Step-by-step thought process: 1. Frame the trade-off out loud. Brute force is $O(n^2)$ with $O(1)$ extra space; the hash-map trick trades space for time and pulls it down to $O(n) + O(n)$ — almost always the right exchange in interview problems. 2. Walk the algorithm. Iterate with `enumerate`. For each `(i,`

`x`), compute the complement `c = target - x`. If `c in seen`, return `(seen[c], i)`. Otherwise insert `seen[x] = i` and continue. 3. Note the single-pass property. You don't need to populate the dict first and then scan; the lookup-before-insert order guarantees you don't match a value against itself. 4. Mention the duplicate-value edge. If the input has two equal values that sum to the target — like `[3, 3]` with target 6 — the single-pass form still works because the first 3 is in the dict when the second is scanned.

Edge cases to mention: - No solution: return None, raise, or return an empty tuple — match the contract the interviewer specified. - Multiple solutions: the single-pass form returns the *first* pair found; for all pairs, the algorithm shape changes to a different problem. - Negative numbers and zeros work without changes — dict keys can be any hashable.

What NOT to say: - "Sort and two-pointer" — $O(n \log n)$; the hash-map version is strictly better Big-O and loses the original indices. - "Use nested for loops" — that's the brute force the question is asking you to beat.

Common follow-up: "What if I needed all pairs that sum to the target?" — Collect into a list instead of returning on first hit. Watch the duplicate-handling — a dict that stored only the last index of each value will miss earlier pairs; use `defaultdict(list)` to store all indices.

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        c = target - x
        if c in seen:
            return (seen[c], i)
        seen[x] = i
    return None

print(two_sum([2, 7, 11, 15], 9))    # (0, 1)
print(two_sum([3, 3], 6))           # (0, 1) — duplicate handled
```

Q62 · What's the idiomatic Python pattern for BFS on a graph?

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Microsoft India, Postman, Swiggy · Topic: BFS / deque

The question (interviewer's verbatim phrasing):

Implement breadth-first search on a graph represented as an adjacency dict. What container do you reach for?

The 30-second answer: `collections.deque` for the frontier, `set` for visited nodes. The frontier is a FIFO queue — `append` to add, `popleft` to take — both $O(1)$ on a deque and $O(n)$ on a list. The visited set gives $O(1)$ membership tests so you don't revisit nodes. Total time $O(V + E)$; total space $O(V)$ for the queue and the visited set.

Step-by-step thought process: 1. Pick the right container. `list.pop(0)` is $O(n)$ per call — that turns the whole BFS into $O(V^2)$ instead of $O(V + E)$. `deque.popleft` is $O(1)$ and gives you the textbook complexity. The set is the dict-without-values version for $O(1)$ "have I seen this node?" checks. 2. Walk the standard shape. Seed the queue with the start node and add it to visited. Loop while queue is non-empty: `popleft`, process, then for each neighbour not in visited, mark and append. 3. State the "visited at enqueue, not at dequeue" rule. Marking on enqueue prevents the same node from being added multiple times by different parents; marking on dequeue is a common bug that explodes the queue size. 4. Mention the variants. Level-by-level BFS uses `len(queue)` at the top of each iteration to know the layer boundary. Shortest path tracks a `parent` dict for reconstruction. Multi-source BFS seeds the queue with all source nodes at once.

Edge cases to mention: - Disconnected graphs: a single BFS only reaches the start's component. Wrap in a `for node in graph` loop and skip already-visited nodes for full traversal. - Cycles are handled by the visited set; without it, BFS on a cyclic graph runs forever. - For weighted graphs, BFS gives shortest path only if all weights are 1; otherwise you want Dijkstra's via `heapq`.

What NOT to say: - "Use a list as the queue" — turns the algorithm $O(V^2)$ due to `pop(0)` cost. - "BFS is the same as DFS" — different traversal order, different use cases (shortest unweighted path vs reachability / topological order).

Common follow-up: "How would you find the shortest path, not just whether a node is reachable?" — Maintain a `parent: dict[node, node]` filled at the enqueue step. After BFS, walk parent chain backwards from the target to the source and reverse to get the path.

```
from collections import deque

def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nb in graph.get(node, ()):
            if nb not in visited:
                visited.add(nb)
                queue.append(nb)
    return order

graph = {1: [2, 3], 2: [4], 3: [4, 5], 4: [], 5: []}
print(bfs(graph, 1))  # [1, 2, 3, 4, 5]
```

Q63 · How do you write the next-greater-element problem with a monotonic stack?

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Postman, Microsoft India, PhonePe · Topic: monotonic stack

The question (interviewer's verbatim phrasing):

For each element in an array, find the next element to its right that's larger. Return -1 if none exists. How do you do this in $O(n)$?

The 30-second answer: Monotonic decreasing stack. Walk the array left to right. For each new value, pop everything on the stack that's smaller — for every popped index, the current value is its answer. Push the current index. At the end, anything still on the stack has no greater element to its right. Total cost $O(n)$ because every index is pushed and popped at most once.

Step-by-step thought process: 1. Establish the invariant. The stack holds indices whose answer hasn't been found yet, and the values at those indices are strictly decreasing from bottom to top. When a larger value arrives, it "resolves" the indices below it. 2. Walk the loop. For each (i, x) : while the stack is non-empty and the value at the top index is less than x , pop and set `result[popped_index] = x`. Then push i . 3. Quantify the cost. Each index enters the stack exactly once and leaves at most once. Two-pointer-style amortised analysis: $O(n)$ total, not the $O(n^2)$ you'd get from the nested-loop brute force. 4. State the post-loop step. Anything left on the stack has no answer — initialise the result array with -1 (or whatever sentinel the problem demands) up front so you don't need a cleanup pass.

Edge cases to mention: - Equal values: the problem statement decides whether ties count as "greater". Use `<` for strictly-greater, `<=` for greater-or-equal. - Circular arrays: iterate the array twice (effectively length $2n$) but only push indices on the first pass. - For "previous greater element", iterate right to left with the same pattern.

What NOT to say: - "Nested for loops" — $O(n^2)$; the question is testing whether you know the monotonic stack pattern. - "Use a heap" — heap gives $O(n \log n)$ and doesn't naturally produce "next" relationships.

Common follow-up: "What if I wanted the next smaller element?" — Flip the comparison: pop while the stack's top value is greater than the current. The pattern is symmetric.

```
def next_greater(nums):
    result = [-1] * len(nums)
    stack = [] # indices with decreasing values
    for i, x in enumerate(nums):
        while stack and nums[stack[-1]] < x:
            result[stack.pop()] = x
        stack.append(i)
    return result

print(next_greater([2, 1, 3, 4, 2])) # [3, 3, 4, -1, -1]
```

Q64 · How does `functools.cache` change how you write memoized recursion?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Postman, Microsoft India, Cred · Topic: memoization

The question (interviewer's verbatim phrasing):

Walk me through Fibonacci with memoization. How does the Python idiom differ from a hand-rolled cache dict?

The 30-second answer: Decorate the recursive function with `@functools.cache` (3.9+) and the memoization is done — the decorator wraps the call in a hash-map lookup on the argument tuple. Cache hits are $O(1)$; a memoized recursion of n calls is $O(n)$ total instead of $O(2^n)$. Hand-rolling with a dict works too but adds three lines of bookkeeping per function and an extra parameter that's easy to forget.

Step-by-step thought process: 1. State the canonical idiom. `from functools import cache; @cache; def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)`. Three lines, full memoization, no manual cache dict. 2. Walk what the decorator does. It wraps the function in a closure that holds a dict keyed by `(args, frozenset(kwargs.items()))`. On call, hash the key; if present, return the cached value; otherwise call the function, store, and return. 3. State the complexity transformation. Without the cache, Fibonacci is $O(2^n)$ — exponential, useless past $n=30$. With the cache, it's $O(n)$ — each n is computed exactly once. Same code, completely different asymptotic behaviour. 4. Mention `lru_cache(maxsize=...)` vs `cache`. `cache` is `lru_cache(maxsize=None)` — unbounded. `lru_cache(maxsize=128)` keeps only the last 128 entries, useful when the argument space is large and you don't want unbounded growth.

Edge cases to mention: - Arguments must be hashable. If the function takes a list, the decorator raises `TypeError`; pass a tuple or freeze the list. - The decorator stores references to argument objects — large arg objects can keep memory pinned for the lifetime of the cache. - Methods on classes need `@cache` carefully — the implicit `self` argument is part of the key, so cache grows per instance. For per-class caching, use `@cached_property` or a class-level cache.

What NOT to say: - "Memoization is slow because of the dict lookups" — the lookup is $O(1)$ and dominates only if the function body is trivially cheap. - "Write your own memo dict" — `@cache` is the modern idiom; hand-rolling is for cases where the decorator's semantics don't fit.

Common follow-up: *"How do I clear the cache between calls?"* — The decorated function has a `.cache_clear()` method that wipes all entries. `.cache_info()` returns hits, misses, cursize, maxsize — useful for tuning.

```
from functools import cache

@cache
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)

print(fib(100))          # 354224848179261915075 — instant
print(fib.cache_info())  # CacheInfo(hits=98, misses=101, maxsize=None, cursize=101)
fib.cache_clear()
```

Q65 · How do you group anagrams idiomatically in Python?

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Postman, Microsoft India, Swiggy · Topic: dict-keyed-by-signature

The question (interviewer's verbatim phrasing):

Given a list of strings, group anagrams together. What signature do you key on?

The 30-second answer: Use a `defaultdict(list)` keyed by the sorted-tuple of characters. Two strings are anagrams iff their sorted forms are equal — so `tuple(sorted("eat")) == tuple(sorted("tea"))`. Walk the input once, hash each string's sorted-tuple signature, append to the bucket. $O(n \cdot k \log k)$ where n is the number of strings and k is the average length — the sort dominates.

Step-by-step thought process: 1. Pick the signature. Sorted characters is the canonical form for "same multiset of letters". For ASCII-only inputs, a frequency tuple of length 26 is an alternative that's $O(k)$ per signature instead of $O(k \log k)$ — same asymptotic with a smaller constant for short strings. 2. Walk the algorithm. `groups = defaultdict(list); for s in strings: groups[tuple(sorted(s))].append(s); return list(groups.values())`. Four lines. 3. State the complexity. Per-string work is $O(k \log k)$ for the sort plus $O(k)$ for the hash. Total $O(nk \log k)$. Memory is $O(nk)$ for storing the strings in the groups. 4. Mention the frequency-tuple alternative. `tuple(Counter(s).items())` works but is fragile because the tuple order depends on insertion

order. Better: build a length-26 tuple of counts indexed by `ord(c) - ord('a')`. Hashable and deterministic.

Edge cases to mention: - Empty strings group together: `tuple(sorted(""))` is `()`, a valid hashable key. - Case sensitivity is in your hands — normalize with `.lower()` if needed before signing. - Unicode: `sorted` works on code points, so combining marks and decomposed forms may behave unexpectedly. Normalize with `unicodedata.normalize` if mixing forms.

What NOT to say: - "Compare every pair" — $O(n^2)$ and misses the point of hashing the signature. - "Use a string as the key" — works (`''.join(sorted(s))`) but the tuple form is slightly faster to build and avoids any string interning concerns.

Common follow-up: "What if the strings are very long and I want to avoid the sort?" — Use the length-26 frequency tuple: $O(k)$ to build, hashable, deterministic. For non-ASCII inputs, use a `frozenset(Counter(s).items())` or build the tuple over a fixed alphabet.

```
from collections import defaultdict

def group_anagrams(strings):
    groups = defaultdict(list)
    for s in strings:
        groups[tuple(sorted(s))].append(s)
    return list(groups.values())

print(group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
# [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
```

Q66 · How do you implement a trie in idiomatic Python?

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Postman, Microsoft India, PhonePe · Topic: trie

The question (interviewer's verbatim phrasing):

Implement a prefix trie that supports insert and starts-with queries. What data structure do you reach for at each node?

The 30-second answer: A nested dict where each key is a character and the value is the next node, plus a sentinel key (often `_end_`) on terminal nodes to mark complete words. Insert walks the chain, creating dicts as needed. Lookup walks the same chain, returning False if any step is missing. Insert and starts-with are both $O(k)$ where k is the length of the query, regardless of how many words are stored.

Step-by-step thought process: 1. Frame the structure. A trie is a tree where each edge is a character and a path from the root spells a stored prefix. The natural Python representation is a dict-of-dicts; no

class needed for the basic version. 2. Walk insert. Start at `root`. For each character, use `node = node.setdefault(c, {})` to step in or create. At the end, mark with `node['_end_'] = True` (or any sentinel that won't collide with a character key). 3. Walk starts-with. Same chain-walking, but if any step is absent, return False. For "contains word", check that the terminal sentinel is also present at the final node. 4. Quantify. Insert and query are $O(k)$. Total memory is $O(\text{total characters stored})$ in the worst case; shared prefixes reduce this — that's the trie's reason for existing.

Edge cases to mention: - Choose a sentinel that can't be a valid character. `_end_` or the empty string `""` work for ASCII alphabets; for arbitrary Unicode, use a separate flag in a class-based node. - Delete is harder than insert — walk to the terminal, clear the sentinel, then walk back removing empty dicts. Easier to mark "deleted" and skip in queries if you don't actually need to reclaim memory. - For autocomplete (return all words with a prefix), walk to the prefix node and DFS the subtree collecting words.

What NOT to say: - "Use a class with `__slots__` for each node" — overkill for the interview version. Dict-of-dicts is the readable answer; switch to classes when memory is a measured concern. - "A trie is the same as a hash set" — set membership is $O(k)$ too, but a trie also supports prefix queries in $O(k)$ which a set cannot.

Common follow-up: "How would you store the words for autocomplete?" — Same insert with a terminal sentinel. For autocomplete, walk to the prefix node, then DFS the subtree collecting all words. The DFS work is proportional to the size of the result, not the size of the whole trie — the prefix narrows the search.

```
class Trie:
    def __init__(self):
        self.root = {}

    def insert(self, word):
        node = self.root
        for c in word:
            node = node.setdefault(c, {})
        node["_end_"] = True

    def starts_with(self, prefix):
        node = self.root
        for c in prefix:
            if c not in node: return False
            node = node[c]
        return True

t = Trie()
for w in ["apple", "app", "application"]:
    t.insert(w)
print(t.starts_with("app"))    # True
print(t.starts_with("apz"))    # False
```

Q67 · How do you do topological sort with Kahn's algorithm in Python?

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Microsoft India, Postman, Goldman · Topic: topological sort

The question (interviewer's verbatim phrasing):

I have a DAG of task dependencies. Give me a valid execution order. How would you code Kahn's algorithm?

The 30-second answer: Compute in-degrees with a dict, seed a deque with all zero-in-degree nodes, then loop: pop a node, emit it, decrement neighbours' in-degrees, and enqueue any that drop to zero. $O(V + E)$ total. If the emitted list's length is less than the number of nodes, there's a cycle and no valid order exists.

Step-by-step thought process: 1. Build the structures. `in_degree = {node: 0 for node in graph}` ; then for every edge `u -> v` , increment `in_degree[v]` . This is $O(V + E)$. 2. Seed the queue. `queue = deque(n for n, d in in_degree.items() if d == 0)` . These are the nodes with no prerequisites. 3. Walk the main loop. Pop, append to result. For each successor, decrement its in-degree; if it hits zero, append it to the queue. Continue until the queue is empty. 4. Detect cycles. If `len(result) != len(graph)` , the unprocessed nodes form a cycle (or are reachable only from one). Either raise or return None — match the contract.

Edge cases to mention: - Disconnected DAGs work without changes — the seeding step picks up all zero-in-degree nodes including isolates. - Multiple valid orders exist for most DAGs; Kahn's returns one — the order depends on queue insertion order. For deterministic output, use a sorted structure or a heap-based variant. - Self-loops (`u -> u`) make the node never reach in-degree 0 — they're detected as part of the cycle check.

What NOT to say: - "Use DFS with post-order" — also valid and equally $O(V + E)$, but Kahn's is the idiomatic answer for "give me one valid order" because it naturally handles "earliest-first" semantics and detects cycles cleanly. - "Sort the nodes" — there's no total order to sort by; topological sort produces a valid order from a partial order.

Common follow-up: "What if I want the lexicographically smallest topological order?" — Replace the deque with a min-heap. `heappush` and `heappop` instead of `append` / `popleft` . The cost goes up to $O((V + E) \log V)$ but you get a deterministic, smallest-first order — useful when ties must break stably.

```

from collections import deque, defaultdict

def topological_sort(graph):
    in_degree = defaultdict(int)
    for u in graph:
        in_degree[u]      # ensure the key exists
        for v in graph[u]:
            in_degree[v] += 1

    queue = deque(n for n, d in in_degree.items() if d == 0)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        for v in graph.get(u, ()):
            in_degree[v] -= 1
            if in_degree[v] == 0:
                queue.append(v)

    if len(order) != len(in_degree):
        raise ValueError("graph has a cycle")
    return order

deps = {"build": ["test"], "test": ["deploy"], "deploy": [], "lint": ["test"]}
print(topological_sort(deps))  # ['build', 'lint', 'test', 'deploy']

```

Q68 · What's the idiomatic sliding-window pattern in Python?

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Postman, Microsoft India, Swiggy · Topic: sliding window

The question (interviewer's verbatim phrasing):

Find the length of the longest substring of a given string with no repeating characters. How does the sliding window pattern apply?

The 30-second answer: Two pointers `left` and `right` walking the same direction, with a set or dict tracking the contents of the current window. Expand `right` to include new characters; when a duplicate appears, advance `left` past the previous occurrence to restore the no-duplicate invariant. Track the max window length as you go. $O(n)$ total because each character is added and removed from the window at most once.

Step-by-step thought process: 1. State the invariant. The window `s[left:right+1]` always satisfies the constraint — in this case, no duplicate characters. Every step either extends the window (expand right) or shrinks it (advance left) to maintain the invariant. 2. Walk the algorithm with a dict-as-

last-seen. `last_seen: dict[char, int] = {}`. Iterate `right` from 0 to `n-1`. If `s[right]` is in `last_seen` and its position is $\geq$ left, advance left to `last_seen[s[right]] + 1`. Update `last_seen[s[right]] = right`. Update `best = max(best, right - left + 1)`. 3. Quantify. Each character is processed once on entry. The `left` pointer monotonically increases and visits each index at most once. Total $O(n)$ — not $O(n^2)$ even though the code has a pointer chase that looks quadratic. 4. Mention the variations. Fixed-window-size problems use a different shape: a deque of size `k` with running sum/max. Variable-window problems with "at most `k` distinct" use a Counter that tracks the count per character and shrinks when distinct count exceeds `k`.

Edge cases to mention: - Empty string returns 0 — the loop body just doesn't execute. - The "if its position is $\geq$ left" check handles the case where the previous occurrence is already outside the window — don't shrink past characters you've already discarded. - For unicode, iterating the string with `enumerate` works as expected — each step is a code point.

What NOT to say: - "Nested loops checking every substring" — $O(n^3)$ or $O(n^2)$ depending on the check; the sliding window is the linear-time fix. - "Sort the string" — destroys the ordering you're trying to find substrings in.

Common follow-up: "How would you adapt this to 'longest substring with at most `k` distinct characters'?" — Replace the no-duplicate invariant with "at most `k` distinct". Maintain a `Counter` of the window contents; when `len(counter) > k`, advance left and decrement the counter, deleting keys that hit zero, until distinct count is back to `k`.

```
def longest_unique_substring(s):
    last_seen = {}
    left = best = 0
    for right, c in enumerate(s):
        if c in last_seen and last_seen[c] >= left:
            left = last_seen[c] + 1
        last_seen[c] = right
        best = max(best, right - left + 1)
    return best

print(longest_unique_substring("abcabcbb")) # 3 - "abc"
print(longest_unique_substring("bbbbb"))   # 1 - "b"
print(longest_unique_substring("pwwkew"))  # 3 - "wke"
```

Q69 · How do you implement Union-Find (DSU) in Python?

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Microsoft India, Postman, Goldman · Topic: union-find

The question (interviewer's verbatim phrasing):

Implement disjoint-set union with path compression and union by rank. What's the per-operation cost?

The 30-second answer: Two dicts: `parent` mapping each node to its parent (itself initially) and `rank` mapping each root to a depth-ish hint. `find(x)` walks parent pointers to the root and compresses the path on the way back. `union(x, y)` finds both roots and attaches the smaller-rank one under the larger. Amortised near- $O(1)$ per operation — formally $O(\alpha(n))$ where α is the inverse Ackermann function, which is ≤ 4 for any n you'll ever see.

Step-by-step thought process: 1. Set up the data. `parent: dict[node, node]` initialised so each node is its own parent. `rank: dict[node, int]` initialised to 0. Both can use dicts so you don't need pre-allocated index arrays — useful when nodes are strings or arbitrary hashables. 2. Walk `find` with path compression. Recurse or iterate up `parent` chains until you reach a node whose parent is itself (the root). On the way back, point every visited node directly at the root. This flattens the tree, making future lookups $O(1)$. 3. Walk union by rank. Find both roots. If equal, do nothing — they're already in the same set. Otherwise attach the lower-rank root under the higher-rank one; if ranks are equal, pick either and increment the new root's rank. 4. Quantify. Two heuristics together (path compression + union by rank) give amortised $O(\alpha(n))$ per op, effectively constant. Without them, `find` is $O(n)$ in the worst case — degenerate chain.

Edge cases to mention: - Lazy initialisation: use `parent.setdefault(x, x)` inside `find` to auto-create nodes on first touch. Removes the upfront seed step. - Path compression is iterative or recursive — both work. Recursive is shorter; iterative is safer for deep chains relative to Python's recursion limit. - Iterative two-pass path compression: first pass walks to root; second pass repoints each visited node. Slightly cleaner than the "as we recurse back" variant.

What NOT to say: - "Without path compression" — accepted in some courses but considered the textbook wrong answer at modern interviews. - "Use a graph and BFS for connectivity" — works but is $O(V + E)$ per query; DSU is amortised $O(\alpha(n))$ for the same.

Common follow-up: "*What problems is DSU the right answer for?*" — Connectivity queries on a growing edge set (Kruskal's MST, online connectivity), cycle detection in undirected graphs during edge insertion, partition counting (number of components after each merge), equivalence-class grouping (account merge, friend circles).

```

class DSU:
    def __init__(self):
        self.parent = {}
        self.rank = {}

    def find(self, x):
        self.parent.setdefault(x, x)
        self.rank.setdefault(x, 0)
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # path compression
        return self.parent[x]

    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry: return False
        if self.rank[rx] < self.rank[ry]: rx, ry = ry, rx
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]: self.rank[rx] += 1
        return True

dsu = DSU()
for a, b in [("u1", "u2"), ("u2", "u3"), ("u4", "u5")]:
    dsu.union(a, b)
print(dsu.find("u1") == dsu.find("u3")) # True
print(dsu.find("u1") == dsu.find("u5")) # False

```

Q70 · How do you find subarrays summing to a target using a prefix-sum dict?

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Postman, Microsoft India, PhonePe · Topic: prefix-sum / hash-map

The question (interviewer's verbatim phrasing):

Count the contiguous subarrays whose elements sum to a target k. Array can contain negatives. What's the idiomatic Python approach?

The 30-second answer: Running prefix sum plus a dict mapping each prefix-sum value to how many times it has appeared. For each position, if `prefix - k` is in the dict, every prior occurrence of that prefix is the start of a subarray ending here with sum k. Increment the count by that many, then record the current prefix. O(n) time, O(n) space. Negatives are fine — the prefix-sum dict doesn't care about sign.

Step-by-step thought process: 1. Define the prefix sum. `P[i]` is the sum of the first i elements. A subarray `nums[a:b]` sums to `P[b] - P[a]`. We want pairs where that difference equals k, i.e., `P[a] = P[b] - k`. 2. Walk the algorithm. Maintain `prefix = 0` and `counts = {0: 1}` (the

empty-prefix base case). For each `x` in `nums`: `prefix += x`; if `prefix - k in counts`, add `counts[prefix - k]` to the answer; then increment `counts[prefix]`. 3. State the complexity. One pass, one $O(1)$ dict lookup per element. $O(n)$ total. 4. Mention the negatives angle. Sliding-window approaches assume monotone behaviour and break on negative numbers. Prefix-sum + dict handles negatives, zeros, and duplicates without any extra cases.

Edge cases to mention: - The `{0: 1}` initial state matters: it lets the algorithm count subarrays that start at index 0 (where the "prior prefix" is the empty prefix of sum 0). - For "find any one subarray" instead of counting, store the *index* of each prefix sum instead of the count. The first match gives you a valid range. - For "longest subarray with sum `k`", store the *earliest* index of each prefix sum and update best with the current span.

What NOT to say: - "Sliding window" — broken on negatives; use it only when all values are non-negative and the target is positive. - "Nested loop over all subarrays" — $O(n^2)$ or $O(n^3)$; prefix-sum + dict is the strict improvement.

Common follow-up: "How do you find the longest subarray with sum `k` instead of counting them?" — Replace the counter with `first_seen: dict[prefix, index]` storing only the first time you saw each prefix. For each `i`, if `prefix - k in first_seen`, candidate length is `i - first_seen[prefix - k]`. Don't update an existing key — earlier is better for "longest".

```
def subarrays_with_sum(nums, k):
    counts = {0: 1}
    prefix = total = 0
    for x in nums:
        prefix += x
        total += counts.get(prefix - k, 0)
        counts[prefix] = counts.get(prefix, 0) + 1
    return total

print(subarrays_with_sum([1, 1, 1], 2))      # 2
print(subarrays_with_sum([1, 2, 3], 3))     # 2 — [1,2] and [3]
print(subarrays_with_sum([1, -1, 1, -1], 0)) # 4 — handles negatives
```